AtCoder Beginner Contest 111 / AtCoder Regular Contest 103 解説

writer: nuip 並びに semiexp

2018 年 9 月 29 日

For International Readers: English editorial starts on page 200.

A: AtCoder Beginner Contest 999

与えられる 3 文字をそれぞれ問題文のとおりに置き換えればよいです。

```
#include<iostream>

using namespace std;

int main(){
	string n;
	cin >> n;
	for(char c:n){
		if(c == '1'){
			cout << 9;
		}else{
			cout << 1;
		}
	}
	return 0;
}
```

文字列を相互に置き換える処理は次のようにも書けます。

```
#include<iostream>

using namespace std;

int main(){
	string n;
	cin >> n;
	for(char c:n){
		cout << (char)('1' + '9' - c);
	}
	return 0;
}
```

これを応用して次のようなシンプルなコードでも解けます。

```
#include<iostream>

using namespace std;

int main(){
	int n;
	cin >> n;
	cout << 111 + 999 - n << endl;
	return 0;
}
```

B: AtCoder Beginner Contest 111

x 回目の ABC が黒橋君が初めて参加する ABC としてふさわしいかどうかは，各桁の数字を見ることで確かめられます．なので，$x = N$ から始めて，条件を満たすまで 1 ずつ x を大きくしていけば解くことができます．

ところで，この問題では，$100 \leq N \leq 999$ という条件から，答えとなる x は必ず 3 桁になります．3 桁で条件を満たす x はすべて 111 の倍数（逆もしかり）なので，N 以上の最小の 111 の倍数を求める方法でも解くことができます．

- C++ による解答例: `https://beta.atcoder.jp/contests/abc111/submissions/3279754`
- Java による解答例: `https://beta.atcoder.jp/contests/abc111/submissions/3279759`
- Python 3 による解答例: `https://beta.atcoder.jp/contests/abc111/submissions/3279761`
- Ruby による解答例: `https://beta.atcoder.jp/contests/abc111/submissions/3279762`
- Rust による解答例: `https://beta.atcoder.jp/contests/abc111/submissions/3279784`
- Bash による解答例: `https://beta.atcoder.jp/contests/abc111/submissions/3279843`

C: /\/\/\/

数列 $v_1, v_2, \ldots, v_n$ から偶数番目を取り出した数列 $v_2, v_4, \ldots, v_n$ を e とおき、奇数番目を取り出した数列 $v_1, v_3, \ldots, v_{n-1}$ を o とおきます。問題の条件を e と o を使って言い換えると、次のようになります。

1. e の要素はすべて同じ
2. o の要素はすべて同じ
3. $e \neq o$

みっつめの条件を無視して考えると、e と o それぞれについて、各要素を数列の最頻値に書き換えるのが最適になります。e と o それぞれの最頻値が異なっていれば、その書き換え方でみっつめの条件もクリアできます。そうでない場合はどちらかを最頻値でない値に書き換えなければなりません。

e で一番出現回数の多い要素を E_1 とおき、二番目に出現回数の多い要素を E_2 とおきます（E_1 と E_2 の出現回数が同じということもありえます)。同様に、o で一番出現回数の多い要素を O_1 とおき、二番目に出現回数の多い要素を O_2 とおきます。今、$E_1 = O_1$ であるため、次のふたつの書き換え方のうちどちらかが最適です。

- e を E_1 で書き換え、o を O_2 で書き換える
- e を E_2 で書き換え、o を O_1 で書き換える

この 2 通りを両方試して、書き換える項の数の小さい方が答えです。

D: Robot Arms

まず，ロボットアームを決めると，$(x_m + y_m) \bmod 2$ は，モードの設定によらず一定になります．なので，$(X_j + Y_j) \pmod 2$ が一定でなければ，条件を満たすことは不可能です．

$(X_j + Y_j) \pmod 2$ が一定であるとします．今，$(X_j + Y_j) \pmod 2 = 1$ と仮定します ($(X_j + Y_j) \pmod 2 = 0$ の場合は，長さ 1 の腕を追加することで，$(X_j + Y_j) \pmod 2 = 0$ の場合に帰着できます)．

座標 (x, y) に対して，$u = x + y, v = x - y$ とおいて (u, v) を考えます．すると，(u_i, v_i) と (u_{i+1}, v_{i+1}) の間に次の関係が成り立ちます：

- 腕 i のモードが `L` のとき，$(u_{i+1}, v_{i+1}) = (u_i - d_i, v_i - d_i)$
- 腕 i のモードが `R` のとき，$(u_{i+1}, v_{i+1}) = (u_i + d_i, v_i + d_i)$
- 腕 i のモードが `D` のとき，$(u_{i+1}, v_{i+1}) = (u_i - d_i, v_i + d_i)$
- 腕 i のモードが `U` のとき，$(u_{i+1}, v_{i+1}) = (u_i + d_i, v_i - d_i)$

よって，$\{d_i\}$ を決めたときに $\{u_i\}, \{v_i\}$ を決定する問題を u, v について独立に解けば，各関節のモードも決定できることになります．

この $\{u_i\}$ を決定する問題を考えます．$u_{i+1} = u_i + d_i$ なる $i \in \{1, \ldots, m\}$ の集合を S とすると，$u_m = -\sum_{i=1}^{m} d_i + \sum_{i \in S} 2d_i$ が成り立ちます．$(X_j + Y_j) \pmod 2 = 1$ の仮定より，U_j, V_j はすべて奇数です．また，$-2^{31} < -2 \cdot 10^9 \leq U_j, V_j \leq 2 \cdot 10^9 < 2^{31}$ が成り立っています．よって，$m = 31$, $d_i = 2^{i-1}$ とすると，$\sum_{i \in S} 2d_i = U_j + (2^{31} - 1)$ が成り立つように S を選ぶことができます（$\frac{U_j + (2^{31}-1)}{2}$ の二進展開を考えればよいです）．よって，この問題を解くことができました．

E: Tr/ee

s の i 文字目を s_i で表します。条件を満たす木が存在するためには、以下の条件が必要です。

- $s_1 = 1$
  葉につながっている辺を取るとサイズ 1 の連結成分ができるため
- $s_n = 0$
  どの辺をとっても非連結な頂点対ができるのだから、連結成分は n よりも真に小さくなるため
- $s_i = s_{n-i}$
  木から辺を 1 つとると連結成分はちょうど 2 こできるため。

逆に、以上の条件をすべてみたしている場合、条件を満たす木を構築することができます。直感的には、存在してはいけないサイズの部分木が存在しないように、うまく根付き木を構成していけばよいです。

s の n 文字目を 1 に書き換えた文字列を s' とします。頂点 n を根とした木で、次の条件を満たす木を構成します。

1. 辺 i は頂点 i とその親をつなぐ
2. $s'_i = 1$ であるとき、 頂点 i の部分木のサイズは i
3. $s'_i = 0$ であるとき、頂点 i は葉

このような木は、頂点 i の親を $i < j$ かつ $s'_j = 1$ となる最小の j とすると構成できます。これを確かめます。親となる頂点が $s'_i = 1$ であるような i に限られることから条件 3 が満たされることが分かります。また、$s'_j = 1$ かつ $i \leq j$ である場合、頂点 i から何度か親をたどることで頂点 j に到達することができることから、$1, 2, \ldots, j$ の j 個の頂点は頂点 j の部分木に含まれます。それ以外の頂点が部分木に含まれないことも明らかであるため、条件 2 も満たされます。

- C++ による解答例: `https://beta.atcoder.jp/contests/arc103/submissions/3288872`

F: Distance Sums

$S(v)$ で，頂点 v から他の点までの距離の和を表すことにします．木の中に辺 uv があるとき，辺 uv を除去したときの u, v を含む連結成分の頂点数をそれぞれ n_{uv}, n_{vu} と書くことにすると，$S(u) = S(v) + n_{vu} - n_{uv} = S(v) + N - 2n_{uv}$ が確かめられます．

一般性を失わず，$D_1 < D_2 < \cdots < D_N$ としてよいです．木を頂点 1 を根とする根付き木として考えます．ここで，頂点 1 から他の頂点までのパス $1, p_1, p_2, \ldots, p_k$ を考えます．まず，$S(1) < S(p_1)$ より，$n_{1p_1} > \frac{n}{2}$ です．また，$n_{1p_1} < n_{p_1p_2} < n_{p_2p_3} < \cdots < n_{p_{k-1}p_k}$ より，$n_{p_ip_{i+1}} > \frac{n}{2}$ であるから，$S(p_i) < S(p_{i+1})$ も成り立ちます．よって，頂点 1 以外の任意の頂点について，それより D の値が小さい頂点への辺がちょうど 1 本存在します．

頂点 N から順に，そのような辺を順次決定していくことを考えます．頂点 $i+1, \ldots, N$ までは確定したとして，頂点 i について考えます．頂点 i に隣接する，i より番号の小さい唯一の頂点を j とすると，$S(j) = S(i) + 2n_{ji} - N$ ですが，n_{ji} は頂点 i を根とする部分木の大きさに等しく，この値は計算できるので（辺を追加するときに順次更新すればよいです），$S(j)$ が決まります．D_i はすべて異なるという条件から，j を求めることができます．ここで，そのような j が存在しない（i より大きくなる場合を含む）ときは，条件を満たす木は存在しません．

頂点 2 まですべて決定できた場合は，一つの木ができます．この木について，実際に $S(i)$ の値を計算して（動的計画法などで可能です），D_i に一致していればその木が答えです．一致していない場合は，条件を満たす木は存在しません．

AtCoder Beginner Contest 111 / AtCoder Regular Contest 103 Editorial

writer: nuip and semiexp

September 29th, 2018

A: AtCoder Beginner Contest 999

Some example implementations:

```
#include<iostream>

using namespace std;

int main(){
	string n;
	cin >> n;
	for(char c:n){
		if(c == '1'){
			cout << 9;
		}else{
			cout << 1;
		}
	}
	return 0;
}
```

```
#include<iostream>

using namespace std;

int main(){
	string n;
	cin >> n;
	for(char c:n){
		cout << (char)('1' + '9' - c);
	}
	return 0;
}
```

```
#include<iostream>

using namespace std;

int main(){
	int n;
	cin >> n;
	cout << 111 + 999 - n << endl;
	return 0;
}
```

B: AtCoder Beginner Contest 111

Some example implementations:

- C++: `https://beta.atcoder.jp/contests/abc111/submissions/3279754`
- Java: `https://beta.atcoder.jp/contests/abc111/submissions/3279759`
- Python 3: `https://beta.atcoder.jp/contests/abc111/submissions/3279761`
- Ruby: `https://beta.atcoder.jp/contests/abc111/submissions/3279762`
- Rust: `https://beta.atcoder.jp/contests/abc111/submissions/3279784`
- Bash: `https://beta.atcoder.jp/contests/abc111/submissions/3279843`

C: /\/\/\/

Let E be the sequence $v_2, v_4, \ldots, v_n$, and let O be the sequence $v_1, v_3, \ldots, v_{n-1}$. We want to do the following (and minimize the number of changes):

1. Change all elements of E to x.
2. Change all elements of O to y.
3. $x \neq y$

If we ignore the third condition, the problem is easy: x should be the mode (the most frequent element) of E and y should be the mode of O. Thus, in case the mode of E and the mode of O are different, this is the optimal solution.

Otherwise, we should try two cases. Let e be the mode of E and e_2 be the second most frequent element of E. (Strictly speaking, e_2 is the most frequent element of E except for e. It's possible that e and e_2 appear the same number of times in E.) SImilarly, let o be the mode of O and o_2 be the second most frequent element of O. Then, the optimal solution is one of the following two:

- $x = e, y = o_2$
- $x = e_2, y = o$

We try both and choose the better one.

D: Robot Arms

For a fixed robot arm, the parity of $x_m + y_m$ is a constant (it must match the parity of the sum of d_i). Thus, in case $(X_j + Y_j) \pmod 2$ are not the same for all j, it is impossible to satisfy the conditions.

Assume that $X_j + Y_j$ is odd for all j. (In case they are all even, add a section of length 1 and we can reduce to the "all even" case.) It turns out that a robot arm with lengths $1, 2, 4, 8, \ldots$ can cover all such positions:

- If the robot arm has only one section and its length is 1, the set of positions where the last joint can go is $\{(1,0),(0,1),(-1,0),(0,-1)\}$.
- If the robot arm has two sections and their lengths are $\{1,2\}$, the set of positions where the last joint can go is $\{(x,y) \mid |x|+|y| \leq 3, x \equiv y \pmod 2\}$. In other words, it can cover all integer points (with correct parities) within a "rotated square" whose vertices are at $(3,0),(0,3),(-3,0),(0,-3)$.
- By induction, we can prove that, if the lengths of sections are $\{1,2,\ldots,2^k\}$, we can cover all vertices with correct parities within a "rotated square" whose vertices are at $(M,0),(0,M),(-M,0),(0,-M)$, where $M = 2^{k+1}-1$.

Thus, if we use $k = 30$, we can reach all given points.

To find a way to reach a given point, determine the mode of sections from the longest ones. For example, suppose that we know the position of (x_i, y_i). If we determine the mode of Section i, we can get the coordinates of (x_{i-1}, y_{i-1}). We should choose it such that (x_{i-1}, y_{i-1}) fits within the valid region (i.e., it should be reachable by lengths $\{1,2,\ldots,2^{i-2}\}$).

E: Tr/ee

Clearly, the following conditions are necessary:

- $s_1 = 1$ (If we cut a leaf we can get a component with size 1)
- $s_n = 0$ (If we cut an edge the sizes of components are less than n)
- $s_i = s_{n-i}$ (When we have a component of size i, the other component has size $n-i$)

These conditions are actually sufficient. Suppose that $0 < x_1 < x_2 < \cdots < x_k < n$ are the positions where s is one (i.e., $s_{x_i} = 1$). Then, the following caterpillar (https://en.wikipedia.org/wiki/Caterpillar_tree) satisfies the conditions:

1. The caterpillar has $k+1$ "body" vertices, labelled with $1,\ldots,k+1$. They form a path in this order.
2. For each i $(2 \leq i \leq k)$, the body vertex i has $x_i - x_{i-1} - 1$ "feets". (In other words, prepare $x_i - x_{i-1} - 1$ new vertices and attach them to vertex i.)

F: Distance Sums

Consider a vertex v with the largest value of D_v. Intuitively, v is a vertex that is far from majority of other vertices. We can prove that v is a leaf, and if w is the only vertex adjacent to v, $D_w = D_v - (n-2)$. We can determine one edge this way (vw), and by repeating similar process we can uniquely determine the entire tree. Let's formalize this.

Without loss of generality, we can assume that $D_1 < D_2 < \cdots < D_N$. Consider the tree as a rooted tree whose root is Vertex 1. In the decreasing order of i, we want to determine the parent of Vertex i.

Suppose that the tree contains an edge uv, and when the edge uv is removed from the tree, the sizes (number of vertices) of the two connected components are n_{uv} and n_{vu} (n_{uv} corresponds to the component with u). Then, $D_u = D_v + n_{vu} - n_{uv} = D_v + N - 2n_{uv}$ holds.

Consider a path $1, p_1, p_2, \ldots, p_k$. Since $D_1 < D_{p_1}$, $n_{1p_1} > \frac{n}{2}$. Also, since $n_{1p_1} < n_{p_1p_2} < n_{p_2p_3} < \cdots < n_{p_{k-1}p_k}$, $n_{p_ip_{i+1}} > \frac{n}{2}$ holds for all i and $D_{p_i} < D_{p_{i+1}}$ holds. Thus, if Vertex j is the parent of Vertex i, $j < i$ holds.

Suppose that we know the parents of Vertices $i+1, \ldots, N$. At this point, we know the subtree rooted at Vertex i. If Vertex j is the parent of Vertex i, as we mentioned above, $D_j = D_i + 2n_{ji} - N$ holds. Since n_{ji} is the size of the subtree rooted at i, we can uniquely determine j. (In case such j doens't exist, we get a contradiction.)

We repeat this process until we determine the parents of Vertices $2, \ldots, n$. Then, we have a tree, and check if this tree really satisfies the conditions. If yes, this is the desired tree (otherwise we get a contradiction).

ARC 102 解説

DEGwer

2018/09/01

For International Readers: English editorial starts on page 6.

A: Pair

K 以下の偶数は $\lfloor K/2 \rfloor$ 個、奇数は $\lceil K/2 \rceil$ 個あるので、これらを足し合わせればよいです。

```
#include<stdio.h>
int main()
{
        int k;
        scanf("%d", &k);
        printf("%d\n", (k / 2)*((k + 1) / 2));
}
```

B: Ruined Square

以下のコードのような式で、残りの点の座標を求めることができます。

```
#include<stdio.h>
using namespace std;
int main()
{
        int a, b, c, d;
        scanf("%d%d%d%d", &a, &b, &c, &d);
        int x = c - a, y = d - b;
        printf("%d %d %d %d\n", c - y, d + x, a - y, b + x);
}
```

C: Triangular Relationship

K が奇数の時、a, b, c を K で割ったあまりはすべて 0 である必要があります。K が偶数の時、a, b, c を K で割ったあまりはすべて 0 であるか、あるいはすべて $K/2$ である必要があります。このような組の個数は、N 以下で K で割って 0 あまるものの個数と $K/2$ あまるものの個数から求めることができるので、この問題を解くことができました。

D: All Your Paths are Different Lengths

$2^r \leq L$ なる最大の r を取ります ($r \leq 19$ に注意)。まず、以下のようにすれば、長さ 0 以上 $2^r - 1$ 以下のパスをすべて 1 本ずつ作ることができます。

- $r+1$ 個の頂点 $1, ..., r+1$ を用意し、$N = r+1$ とする
- 頂点 i から 頂点 $i+1$ へ、長さ 0 の辺と長さ 2^i の辺を 1 本ずつ張る

さて、こうして作ったグラフの頂点 t から頂点 N に長さ X の辺を張れば、長さ $X, X+1, ..., X+2^{t-1}-1$ のパスを 1 本ずつ作ることができます。よって、$t = N-1, ..., 1$ の順に見て、$L - 2^{t-1} \geq 2^r$ なら頂点 t から頂点 N に長さ $L - 2^{t-1}$ の辺を張り、L を 2^{t-1} だけ減らし、そうでなければ何もしないことを繰り返せばよいです。

E: Stop. Otherwise...

どの 2 つのサイコロの出目の和も t にならないような出目の組の個数を高速に数えることができれば、この問題を解くことができます。

t が奇数の時、条件は $a + b = t$ となるような各 $1 \leq a, b \leq K$ に対し、a, b のどちらか片方は現れないことです。このような (a, b) の組の個数を p とすれば、p 個の組のうちいくつか (q 個とする) については組の目の両方とも出目として現れず、残りについては片方のみ現れることになります。ここで、q を全通り試せばこの場合の数は簡単な二項係数 (と 2 べき) で表せる形になるので、この場合は $O(K)$ 時間で解くことができます。

t が偶数の時、条件は上記のものに $K/2$ が 1 個以下しか現れないという条件を加えたものです。よって、$K/2$ がいくつ (0 個または 1 個) 現れるかで場合分けをし、上と同様に解けばよいです。

時間計算量は $O(K^2 + N)$ です。

F: Revenge of BBuBBBlesort!

列 $\{a_i\} = (1, 2, ..., N)$ から $a_{i-1} < a_i < a_{i+1}$ なる連続 3 項を反転する (以下「i で操作する」) ことを繰り返して列 p を作れるか判定すればよいです。このようにして作れる列 p の性質を見ていくことにしましょう。

まず、隣接 2 要素の両方で操作することはないことがわかります。これは以下のように証明されます。

証明: i で操作する前に、$i-1$ や $i+1$ で操作したことがあるとする。最後に $i-1$ で操作したなら $p_{i-1} > p_i$ に、最後に $i+1$ で操作したなら $p_i > p_{i+1}$ になっているので、i では操作できない。

特に、i で操作したなら、i の位置にある数はそれ以降変化することはありません。

以下、$l+1, l+3, ..., r-1$ のすべてで操作しており、$l-1, r+1$ では操作していないような区間 $[l, r]$ に列を分解して考えます。これらの区間に対して操作は独立に行われるので、以下ひとつの区間についてのみ考えます。

$l+1, l+3, ..., r-1$ の位置にある数は ($l, l+2, ..., r$ では操作しないので)($l+1, l+3, ..., r-1$ のまま) 変化することはありません (これらをピボットと呼ぶことにしましょう)。ピボットでない数が操作によって移動したとします。もしこの数が左に移動したなら、この数のすぐ右にあるピボットの値はこの数より小さく、よってそれ以降この数は現在の位置より右に移動することはないことが分かります。右に移動した場合も同様に、それ以降この数が現在の位置より左に移動することはありません。この議論はすべての操作について成り立つので、ピボットでない数は左に移動し続けるか右に移動し続けるかのどちらかになります (区間の取り方より、移動しないことはありません)。

さて、左に移動し続ける数同士の順番が変化するなら、それらが入れ替わるときに片方は右に移動していることになって矛盾します。よって、初期状態より左に移動した数は、常にその相対的な順序を保つことになります。同様に、初期状態より右に移動した数も、常にその相対的な順序を保つことになります。

逆に、これらの条件が満たされるとき、すなわち各区間 (区間への分解は上記の議論より列 p から定まる) においてその偶数番目の数は移動せず、奇数番目の数はすべて移動し、左に移動する数たちと右に移動する数たちはそれぞれその相対的な順序を保つ場合、数が (まだ右に移動する数 ピボット まだ左に移動する数) という順に並んでいる場所を一個取って操作を行うことを繰り返すことで p を作ることができることが分かります。

よって、以上の条件を満たすかどうかを判定すればよく、これは線形時間で可能なので、この問題を解くことができます。

ARC 102 Editorial

DEGwer

2018/09/01

A: Pair

There are $\lfloor K/2 \rfloor$ even numbers and $\lceil K/2 \rceil$ odd numbers among positive integers up to K. The answer is the product of these values.

```
#include<stdio.h>
int main()
{
        int k;
        scanf("%d", &k);
        printf("%d\n", (k / 2)*((k + 1) / 2));
}
```

B: Ruined Square

You can compute the coordinates of the reaming points as in the code below:

```
#include<stdio.h>
using namespace std;
int main()
{
        int a, b, c, d;
        scanf("%d%d%d%d", &a, &b, &c, &d);
        int x = c - a, y = d - b;
        printf("%d %d %d %d\n", c - y, d + x, a - y, b + x);
}
```

C: Triangular Relationship

When K is odd, a triplet (a, b, c) satisfies the conditions when all of them are divisible by K. It's easy to count such triplets.

When K is even, a triplet (a, b, c) satisfies the conditions when all of them are divisible by K, or when all of them are equivalent to $K/2$ modulo K. We can count such triplets by counting the number of integers up to N that are equivalent to 0 or $K/2$, modulo K.

D: All Your Paths are Different Lengths

Solution 1.

Let's take the maximum integer r such that $2^r \leq L$.

First, consider the following graph:

- There are $N = r + 1$ vertices numbered $1, \ldots, N$.
- For each i, add two edges from vertex i to vertex $i + 1$. One of them has length 0, and the other has length 2^{i-1}.

In this graph, there are 2^r paths from vertex 1 to vertex N, and their lengths are $0, \ldots, 2^r - 1$.

What happens if we add an edge of length X from vertex t to vertex N? This way we can add $X, X + 1, \ldots, X + 2^{t-1} - 1$ to the set of lengths.

By adding edges of this type properly (by writing L as a sum of powers of two using binary representation), we can get a desired graph.

Solution 2.

Suppose that we have a graph G that satisfies the condition for $L = X$. We'll show how to construct solutions for $L = X + 1$ and $L = 2X$ using this graph. By repeating these operations properly, we can get a desired graph.

- $L = X + 1$ is easy: just add an edge of length X from source to sink to G.
- $L = 2X$ can be constructed as follows. First, double the lengths of all edges in G. Now the set of lengths is $0, 2, \ldots, 2X - 2$. Then, add a new vertex to G (it will be the new sink), and add two edges from the old sink to the new sink, with lengths 0 and 1. Now the set of lengths is $0, 1, 2, \ldots, 2X - 1$, and this is what we want.

E: Stop. Otherwise...

Let's handle each query independently. For a fixed value of t, we want to count the number of combinations of dice such that the sum of no two dice is t.

For simplicity, we say "x exists" when at least one die shows x. Suppose that t is odd. In this case, the condition can be restated as follows: for each pair (a, b) such that $a + b = t$, at least one of a or b should not exist. Let's divide integers $1, \ldots, K$ into pairs, such that the sum of two numbers in each pair is t (and some numbers will be left unpaired).

Let p be the number of pairs formed this way. Then, the combination of dice must be of the following form for some integer q:

- For q pairs, exactly one of the two numbers in the pair exists.
- For the remaining $p - q$ pairs, both numbers don't exist.
- $K - 2p$ numbers are left unpaired: we don't care if these numbers exist.

There are $\binom{p}{q}2^q$ ways to choose the set of existing numbers (among paired numbers). After that, the number of ways to distribute the number of occurrences of those numbers among N dice are equal to the number of solutions to the following equation:

- $x_1 + \cdots + x_q + y_1 + \ldots + y_{K-2p} = N$
- $x_1, \cdots, x_q$ are positive integers
- $y_1, \cdots, y_{K-2p}$ are non-negative integers

Here, $x_1, \cdots, x_q$ corresponds to the number of occurrences of each existing paired numbers, and $y_1, \cdots, y_{K-2p}$ corresponds to the number of occurrences of each unpaired numbers.

This is a well-known task and the answer can be represented by a binomial coefficient. By trying all possible values of q, we can count the desired number in $O(K)$ time (assuming that we do some pre-computation and we can get binomial coefficients and powers in $O(1)$).

When t is even, there is an additional constraint that the number of occrrences of $t/2$ must be either 0 or 1. Try both possibilities, and the remaining part is the same as above.

This solution works in $O(K^2 + N)$ time.

F: Revenge of BBuBBBlesort!

Let's see the operations in the reverse order: we start with a sequence $\{a_i\} = (1, 2, ..., N)$, and we repeat performing operations of the following type: Choose i such that $a_{i-1} < a_i < a_{i+1}$, and reverse their orders (we call it "an operation on i"). We want to check if we can get p this way.

First, we'll prove that we can never make operations in two adjacent positions. Suppose that before performing an operation on i, we have performed operations on $i-1$ or $i+1$. If the last operation on $i-1$ or $i+1$ was on $i-1$, after the opeartion, $a_{i-1} > a_i$ holds. We can never change this inequality: an operation on $i-2$ only makes a_{i-1} greater, by the assumption above we don't make operations on $i+1$, and operations on other positions obviously don't affest this inequality. Similarly, if the last operation was on $i+1$, $a_i > a_{i+1}$ holds, and we can never change this inequality. Thus, we can't make an opeation on i, and we get a contradiction.

Let's split the sequence into intervals as follows: $a_l, \ldots, a_r$ forms an interval if we perform operations on all of $l+1, l+3, ..., r-1$, but not on $l-1, r+1$. All elements move within these intervals, thus we can consider these intervals independently. From now on, we only considers the interval $a_l, \ldots, a_r$.

We start with $a_l, \ldots, a_r = l, \ldots, r$. How can we change these numbers by performing operations only at $l+1, l+3, ..., r-1$?

Elements at positions $l+1, l+3, ..., r-1$ will never move. Let's call these elements "pivots".

Consider a non-pivot element x in this interval. From the definition of intervals, this element must move (unless $l = r$). Suppose that it moves to the left in the first operation that involves this element. Then, after that, this element is always greater than the pivot to the right of it; and it can never move to the right in the future. We can do the same observaion when it moves to the right first. Thus, each non-pivot element keeps moving in the same direction.

Consider two non-pivot elements that moves to the left. The relative order of these elements will not change: to change their relative order, we must swap them, but this is impossible because they must keep moving left. We can say the same thing for elements moving right.

On the other hand, these conditions are sufficient; we can achieve the goal by repeating operations on consecutive three elements such that ((An element that wants to go right), (pivot), (An element that wants to go left)).

To summarize, the conditions are:

- $a_{l+1}, a_{l+3}, \ldots, a_{r-1}$ must not change, and other elements must change.
- If we only consider all elements that moves to the left, they are increasing from left to right.
- If we only consider all elements that moves to the right, they are increasing from left to right.

From the first condition, we can uniquly determine the way to split the sequence into intervals; after that, the remaining two conditions can be checked in linear time.

ABC #107 / ARC #101 Editorial

writer: sugim48

2018 年 8 月 25 日

A: Train

答えは $N+1-i$ です.

- C++ のコード例: https://beta.atcoder.jp/contests/abc107/submissions/3070852
- Java のコード例: https://beta.atcoder.jp/contests/abc107/submissions/3070854
- Python 3 のコード例: https://beta.atcoder.jp/contests/abc107/submissions/3070856

B: Grid Compression

まず，黒いマスが含まれる行および列をマークします．その後，マークされた行とマークされた列が交差する位置の $a_{i,j}$ のみを出力すればよいです.

- C++ のコード例: https://beta.atcoder.jp/contests/abc107/submissions/3070967
- Java のコード例: https://beta.atcoder.jp/contests/abc107/submissions/3070968
- Python 3 のコード例: https://beta.atcoder.jp/contests/abc107/submissions/3070969

C: Candles

連続する K 本のろうそくに火を付けるのが最適です．N 本のうち連続する K 本のろうそくを選ぶ方法は $N-K+1$ 通りしかないので，全探索することにします．連続する K 本のろうそくのうち，最も左のものを l とし，最も右のものを r とします．l と r を訪れれば，自然と残りのろうそくを訪れることになります．よって，「座標 0 を出発し，座標 x_l と座標 x_r をともに訪れるための最短時間」が求めるべき値です．これは次の 2 通りの値の最小値です:

- $l \to r$ の順に訪れる場合: $|x_l| + |x_r - x_l|$
- $r \to l$ の順に訪れる場合: $|x_r| + |x_r - x_l|$

連続する K 本のろうそくを選ぶ方法を全探索し，この値の最小値を求めれば，それが答えです．時間計算量は $\mathrm{O}(N)$ です.

D: Median of Medians

まず，中央値の性質を調べてみましょう．長さ M の整数列 b の中央値を x とします．このとき，x は次の性質を満たします:

- b 中に x 以上の要素が $\left\lceil \frac{M}{2} \right\rceil$ 個以上含まれる．
- x は，上の性質を満たす整数の中で最大である．

よって，中央値を求める際には二分探索が有効であると分かります．すなわち，前者の性質を満たすような最大の整数を二分探索で求めればよいです．

以上の方針をこの問題に適用すると，次の問題を解ければよいことになります:

> $m_{l,r}$ $(1 \leq l \leq r \leq N)$ のうち x 以上の要素は何個か？

上述の中央値の性質を用い，問題をさらに言い換えます:

> a の連続部分列 $a[l,r]$ $(1 \leq l \leq r \leq N)$ のうち，x 以上の要素を $\left\lceil \frac{r-l+1}{2} \right\rceil$ 個以上含むものは何通りか？

この言い換えにより，a の各要素は x 以上か否かという情報のみが必要であることが分かります．そこで，a の各要素を，x 未満ならば `A` へ，x 以上ならば `B` へ置き換えてしまいます．すると，問題は次のように言い換えられます:

> a の連続部分列 $a[l,r]$ $(1 \leq l \leq r \leq N)$ のうち，(`A` の個数) $\leq$ (`B` の個数) を満たすものは何通りか？

見通しをより良くするために，さらに `A` を -1 へ，`B` を $+1$ へ置き換えると，問題は次のように言い換えられます:

> a の連続部分列 $a[l,r]$ $(1 \leq l \leq r \leq N)$ のうち，要素の総和が 0 以上のものは何通りか？

連続部分列の要素の総和は，累積和を用いると簡単に扱えます．(± 1 へ変換後の) a の累積和の列を S とします．すなわち，各 $0 \leq i \leq N$ について $S_i = \sum_{j=1}^{i} a_i$ とします．すると，$a[l,r]$ の要素の総和は $S_r - S_{l-1}$ と計算できます．よって，最終的な問題は次のようになります:

> 組 (l,r) $(0 \leq l < r \leq N)$ のうち，$S_l \leq S_r$ を満たすものは何通りか？

これは，数列 S の転倒数を求める問題とほぼ同じです．非常に典型的なので，「転倒数 アルゴリズム」などで検索すると多くの解説が見つかります．この問題は $\mathrm{O}(N \log N)$ 時間で解くことができます．よって，二分

探索のステップ数を掛け合わせると，元の問題は $\mathrm{O}(N \log N \log A)$ 時間で解くことができます．ここで，A は a の要素の最大値です．

E: Ribbons on Tree

「辺に少なくとも 1 本のリボンが張られている」という条件よりも，「辺にリボンが張られていない」という条件の方が扱いやすいので，包除原理を用いることにします．包除原理を用いると，答えは次のように計算できます:

$$\sum_{F \subseteq E} (-1)^{|F|} (F \text{ のどの辺にもリボンが張られないようなペアの分け方の通り数})$$

ここで，E は与えられた木の辺集合とし，F は E の部分集合全体を動きます．

まず，F をひとつ固定したとき，「F のどの辺にもリボンが張られないようなペアの分け方の通り数」を求めてみましょう．木から F の各辺を取り除き，木を $|F|+1$ 個の連結成分へ分割します．すると，各ペアは同一の連結成分内の頂点からなる必要があります．よって，各連結成分の頂点数をそれぞれ $n_1, n_2, \ldots, n_{|F|+1}$ とすると，通り数は $g(n_1) \cdot g(n_2) \cdot \cdots \cdot g(n_{|F|+1})$ となります．ここで，$g(\cdot)$ は次のように定義されます:

$$g(n) := \begin{cases} (n-1) \cdot (n-3) \cdot \cdots \cdot 3 \cdot 1 & (n : \text{偶数}) \\ 0 & (n : \text{奇数}) \end{cases}$$

当然 F を全探索することはできないので，代わりに根付き木上で DP をします．DP の状態は，「今の部分木」「今の連結成分の頂点数」に加え，「これまでに取り除いた辺の本数の偶奇」が必要です．よって，$dp(\cdot,\cdot,\cdot)$ を次のように定義します:

$$dp(v,x,p) := \left(\begin{array}{l} \text{頂点 } v \text{ を根とした部分木において，} v \text{ を含む連結成分の頂点数が } x \text{ で，} \\ \text{部分木内で取り除いた辺の本数の遇奇が } p \text{ の場合の，ペアの分け方の通り数} \end{array} \right)$$

この dp を葉から根へ向かって順に更新し，最後に根 r における dp を参照することで，答えが求まります．

部分木どうしの dp をマージするとき，常に x を 0 から N まで回してしまうと，各マージごとに $\mathrm{O}(N^2)$ 時間が掛かります．マージは全体で $N-1$ 回行われるので，合計で $\mathrm{O}(N^3)$ 時間が掛かり，TLE してしまいます．ここで，頂点数 X の部分木の dp においては，x は X まで考慮すればよいことを利用します．すると，頂点数 X の部分木と頂点数 Y の部分木の dp をマージするのに掛かる時間は $\mathrm{O}(X \cdot Y)$ に抑えられます．これは定数倍改善でしかないように思えますが，実は計算量を本質的に改善し，全体で $\mathrm{O}(N^2)$ 時間となります．これは十分に高速です．計算量解析は省きますが，「木 DP 二乗」などで検索すると解説が見つかります．

F: Robots and Exits

まず，最も左の出口より左に置かれているロボットたちと，最も右の出口より右に置かれているロボットたちは無視してもよいです．

ある時点におけるロボットの変位の履歴を考えます．この履歴の最小値を x とし，最大値を y とします．最初，$(x,y) = (0,0)$ です．すぬけ君は，(x,y) から $(x+1,y)$ または $(x,y+1)$ のいずれかを自由に選び，変化させることができます．これは，すぬけ君が二次元平面上の原点から，右または上へ距離 1 ずつ移動していくことと等価です．

この言い換えのもとで，各ロボットがいずれの出口を通るかを考えます．ロボット i について，すぐ左の出口までの距離を a_i とし，すぐ右の出口までの距離を b_i とします．すると，x が a_i に達するタイミングと

y が b_i に達するタイミングを比較し，前者が早ければロボット i はすぐ左の出口を通り，後者が早ければロボット i はすぐ右の出口を通ります．すなわち，二次元平面上に点 (a_i, b_i) をプロットしたとき，すぬけ君がこの点の下側を通るか，左側を通るかによって，ロボット i の通る出口が変わります．

ABC #107 / ARC #101 Editorial

writer: sugim48

2018 年 8 月 25 日

A: Train

The answer is $N + 1 - i$.

- C++ Code Example: https://beta.atcoder.jp/contests/abc107/submissions/3070852
- Java Code Example: https://beta.atcoder.jp/contests/abc107/submissions/3070854
- Python 3 Code Example: https://beta.atcoder.jp/contests/abc107/submissions/3070856

B: Grid Compression

Mark all rows/columns that contain at least one black cell, and print intersections of marked rows and marked columns.

- C++ Code Example: https://beta.atcoder.jp/contests/abc107/submissions/3070967
- Java Code Example: https://beta.atcoder.jp/contests/abc107/submissions/3070968
- Python 3 Code Example: https://beta.atcoder.jp/contests/abc107/submissions/3070969

C: Candles

Clearly, we should light K consecutive candles. Let l, r be the leftmost/rightmost indices of candles we light. Since there are only $N - K + 1$ pairs of (l, r), let's try them all.

For a fixed (l, r), we should compute the minimum time required to visit both x_l and x_r (then all other candles between them will be visited too). There are two cases:

- Visit in the order $l \to r$: $|x_l| + |x_r - x_l|$
- Visit in the order $r \to l$: $|x_r| + |x_r - x_l|$

The answer is the minimum of these values. This solution works in $O(N)$ time.

D: Median of Medians

Let b be a sequence of length M. If the median of b is x,

- b contains at least $\lceil \frac{M}{2} \rceil$ elements that are greater than or equal to x.
- x is the largest possible value that satisfies the condition above.

Let's fix a parameter x. We want to solve the following problem:

> Count the number of pairs (l, r) $(1 \leq l \leq r \leq N)$ such that $m_{l,r} \geq x$.

If we can solve this, by doing a binary search on x (using the property of median mentioned above), we can solve the original problem.

For simplicity, we call an element "large" if the element is x or greater, and call it "small" otherwise. Notice that $m_{l,r} \geq x$ holds if and only if (the number of large elements in it) is greater than or equal to (the number of small elements in it). Thus, we replace each large element with 1 and each small element with -1. Now $m_{l,r} \geq x$ holds if and only if $a_l + \ldots + a_r \geq 0$.

To count such pairs, we use prefix sums. We want to count the number of pairs (l, r) such that $S_{l-1} \leq S_r$, where $S_i := \sum_{j=1}^{i} a_i$. Now the problem is reduced to a standard inversion-number problem.

This solution works in $\mathrm{O}(N \log N \log max_a)$ time in total.

E: Ribbons on Tree

By inclusion-exclusion principle, we want to compute the following value:

$$\sum_{F \subseteq E} (-1)^{|F|} f(F)$$

Here, E is the edge set of the given tree, and $f(F)$ is the number of ways to split vertices in pairs such that no edge in F is decorated with ribbons.

For a fixed F, we can compute $f(F)$ as follows. Suppose that the number of vertices of the $|F|+1$ connected components we get by removing edges in F from the tree are $n_1, n_2, \ldots, n_{|F|+1}$. Then, $f(F) = g(n_1) \cdot g(n_2) \cdot \cdots \cdot g(n_{|F|+1})$, where $g(n)$ is defined as follows (this is the number of ways to split n things into pairs):

$$g(n) := \begin{cases} (n-1) \cdot (n-3) \cdot \cdots \cdot 3 \cdot 1 & (n : even) \\ 0 & (n : odd) \end{cases}$$

To compute the sum of $f(F)$ quickly, let's use DP.

Suppose that the tree is rooted in some way. We decide pairings from leaves, and we keep "current subtree", "the number of vertices in current component", and "the parity of removed edges".

Define $dp(v, x, p)$ as follows:

> Consider the subtree rooted at v. We remove some edges from the subtree and get some connected components. Then, within each component, we split vertices into pairs. Here, the parity of the number of removed edges must be p, and the number of vertices in the component with v (the root of the subtree) must be x. How many ways are there to do this?

Strictly speaking, we need a slight technical modification to this. We split vertices into pairs within each component, but as an exception, we don't make pairs for the vertices in the component with v (in other words, we don't multiply $g(x)$). (Otherwise you will have trouble updating this DP table).

This solution works in $O(N^2)$ time.

F: Robots and Exits

We'll update it later!

ARC100 / ABC102 解説

writer : maroonrk

平成 30 年 7 月 1 日

For International Readers: English editorial starts from page 8.

A : Multiple of 2 and N

N が偶数のとき、答えは N です。また、N が奇数のとき、答えは $2N$ です。
C++による解答コード

B： Maximum Difference

数列の最大値を最小値を求め、その差を答えればよいです。最大値と最小値が一致する場合、添字が異なる、という条件を無視してしまうように思えますが、最大値と最小値が一致するということは、すべての要素が等しいので、問題ありません。

C++による解答コード

C : Linear Approximation

$B_i = A_i - i$ と定義すれば、問題は、自由に整数 b を選び、$abs(B_i - b)$ の総和を最小化する問題です。ここで、b を B_i の中央値にするのが最適であることがわかります。中央値でない場合は、中央値に近づけることで損をしないことからこれはわかります。よって、B_i をソートして中央値を求め、実際に答えを計算すればよいです。ソートがボトルネックとなり、この問題は $O(NlogN)$ で解けます。

D : Equal Cut

予め累積和を求めておき、ある区間の和を $O(1)$ で求められるようにしておきます。

切り込みを入れる 3 箇所のうち、真ん中の切れ込みを 1 つ固定したとします。真ん中の切れ込みで切ったあとの 2 つの数列を L,R とします。あとは、L と R をそれぞれ 1 回ずつ切ることを考えます。

L の切り方を考えると、切ったあとの 2 つの数列の要素の総和ができるだけ近いほうが良いことがわかります。よって、真ん中の切れ込みを固定すると、左の切れ込みの位置を決めることが出来ます。真ん中の切れ込みの位置が、i 番目の要素と $i+1$ 番目の要素の間の時、最適な左の切れ込みの位置を $F(i)$ とおきます。このとき、$F(i)$ は i に関して単調増加であり、尺取り方を用ることで $F(i)$ をすべての i に対して $O(N)$ で求めることが出来ます。

右の切れ込みについても、同様にして求めることが出来ます。これで、すべての真ん中の切れ込みの位置に対して、残りの 2 つの切れ込みの位置を求めることが出来ました。あとは、実際にそれぞれの場合に対して答えを計算すれば、答えが求まります。

よって、$O(N)$ でこの問題は解けました。

E : Or Plus Max

各 x について、$max\ A_i + A_j\ s.t.\ (i\ or\ j) = x$ が求められれば、あとは単に累積 max を取れば良いので、この問題は簡単です。しかし、これを求めるのは難しいです。そこで代わりに、各 x について、$max\ A_i + A_j\ s.t.\ (i\ or\ j) \subseteq x$ を求めます。ただしここで便宜的に、$a \subseteq b$ を、a を bit で表したときの 1 の位置が、b を bit で表したときの 1 の位置の部分集合となっている、の意味で使います。これを求めたあと累積 max を求めても、正しい答えは得られます。($(i\ or\ j) \subseteq x \rightarrow (i\ or\ j) \leq x$ なので)

これを求めるためには、各 x について、$i \subseteq x$ なる i について A_i を列挙したときの上位 2 つが求まれば良いです。逆から見ると、各 i について、$i \subseteq x$ なるすべての x について、A_i で上位 2 つを更新する、という操作ができれば良いです。これは、高速ゼータ変換の要領で、高速に行うことができます。よってこの問題は、$O(N2^N)$ で解くことが出来ます。

F : Colorful Sequences

カラフルとは限らない、長さ N で、各要素が 1 以上 K 以下の整数列すべてについて、A を含む個数の総和を求めるのは簡単です。A が登場する位置が $N-M+1$ 通り考えられ、それを一つ固定すると、残りの $N-M$ 要素を自由に決定できるので、その位置に A を含む整数列の総数は K^{N-M} 個になります。

あとは、カラフルでない長さ N の整数列すべてについて A を含む個数を数える問題を解きます。これが解ければ、引き算をして答えが求まります。

まず、そもそも A がカラフルであった場合、答えは 0 です。

残りのパターンを考える前に、次の問題を解いておきます。

> 長さ N のカラフルでない整数列の個数を求めよ。

DP をします。$dp[i][j]$ = 長さ i で、最後の j 個の要素はすべて異なるが、最後の $(j+1)$ 個の要素の中には重複があるような整数列の個数 と定めます。この DP の遷移を考えます。$dp[i][j]$ から $dp[i+1][j']$ への遷移にかかる係数は、以下のようになります。

- $j+1 < j'$ のとき: 0
- $j+1 = j'$ のとき: $K-j$
- $j+1 > j'$ のとき: 1

最初のパターンは簡単です。どんな要素を追加しても、最後の $(j+2)$ 個の中に必ず重複は存在するので、このような遷移はありません。次のパターンは、元の数列の最後 j 個以外の要素を追加することで起こる遷移です。追加する要素の候補は $K-j$ 個なので、この遷移になります。最後のパターンは、j' の値を決めると、追加する要素が一意に定まるので、この遷移になります。

この DP を愚直に書くと、状態数 $O(NK)$、遷移 (K) で合計 $O(NK^2)$ となり間に合いませんが、累積和を用いると遷移を高速化でき、$O(NK)$ で解くことが出来ます。

この DP の結果を用いて、A がカラフルでない場合を解きます。

まず、A 内に重複する要素がない場合を考えます。ここで、次の問題を考えてみます。

> 長さ N のカラフルでない整数列すべてについて、長さ M 個の連続する部分列であって、重複する値を含まないものの個数を数える。

この問題では、A の要素には一切関知せず、M だけを考えている点に注意してください。この問題は、カラフルでない数列を数える上記の DP を変形して（長さ M の重複を含まない連続部分列がすでに登場したか、という 0,1 の次元を足す）答えを求めることが出来ます。長さ M で、各要素が 1 以上 K 以下の、重複する要素を含まない整数列は、$N!/(N-M)!$ 通りありますが、この問題において、これら全ては同じ回数だけ数え上げられます。よって、この問題の答えを $N!/(N-M)!$ で割れば、A を含む個数の総和が求まることになります。

最後に、A 内に重複する要素がある場合を考えます。A 内に重複する要素があるので、次の条件を満たす整数 F,B を定義することが出来ます。

- A の先頭 F 個の要素はすべて異なるが、A の先頭 $F+1$ 個の要素の中には重複がある
- A の末尾 B 個の要素はすべて異なるが、A の末尾 $B+1$ 個の要素の中には重複がある

すべての $i(0 \leq i \leq N-M)$ について、A の先頭に i 個の要素を足し、A の末尾に $N-M-i$ 個の要素を足して、カラフルでない整数列を作る方法が何通りあるかを求めます。ここで、長さ $i+F$ で、最後の F 要素がすべて異なるカラフルでない数列の個数、を求めることが出来ます。これは単に、上記の DP の結果を利用すれば良いです。長さ F で、各要素が 1 以上 K 以下の、重複する要素を含まない整数列は、$N!/(N-F)!$ 通りありますが、これらはすべて、先程数えた数列の末尾 F 個として均等に登場します。よって、割り算をすることで、長さが $i+F$ で、最後の F 要素が A の先頭 F 個と一致するカラフルでない数列の個数が求まります。同様にして、長さが $B+N-M-i$ で、先頭の B 要素が A の末尾 B 個と一致するカラフルでない数列の個数が求まります。この 2 つの通り数を掛け合わせれば、ある i に対する答えを求められます。ここで数えた先頭、末尾に足される数列の組合せが必要十分であることは、簡単に確かめられます。

これで、すべてのパターンを数え上げることが出来ました。最初の DP がボトルネックとなり、$O(NK)$ でこの問題は解くことが出来ます。

Atcoder Regular Contest Editorial

writer : maroonrk

July 1, 2018

A : Multiple of 2 and N

If N is even, the answer is N; otherwise the answer is $2N$.

C++ solution

B : Maximum Difference

Print max minus min.

C++ solution

C : Linear Approximation

Define $B_i = A_i - i$. Then, we want to minimize the value of the sum of $abs(B_i - b)$ by choosing a variable b.

It turns out that the optimal value of b is the median of B_i. This is because, if b is not the median, if we continuously change b to the direction of the median, the sum of $abs(B_i - b)$ never increases.

We can get the median by sorting the array B. This solution works in $O(NlogN)$ time.

D : Equal Cut

Let's pre-compute prefix sums (and now we can get interval sums in $O(1)$.

We need to make three cuts. Let's fix the position of the second (the middle) cut. Now we have two parts, and we need to make one more cut for each of the two parts.

Let L, R be the sum of all elements in left and right parts, respectively. Let L_1, L_2 be the sum of all elements in the two smaller parts we get from the left part (define R_1, R_2 similarly). How should we choose L_1, L_2?

Notice that we should always make a cut that minimizes the value of $|L_1 - L_2|$, regardless of the value of R_1 and R_2. No matter what the values of R_1 and R_2, this choice always minimizes $max\{L_1, L_2, R_1, R_2\}$ and maximizes $mub\{L_1, L_2, R_1, R_2\}$. (This is because the average of L_1 and L_2 is always $L/2$. The value of $max\{L_1, L_2\}$ becomes the closest to $L/2$ (and the smallest) when $|L_1 - L_2|$ is minimized. The same observation holds for the min.)

Thus, once we decide the position of the second cut, we can uniquely determine the positions of the first cut and the third cut. Let $F(i)$ be the optimal position of the first cut when the second cut is made at position i. Since $F(i)$ is monotonously increasing, we can compute this function by using two-pointers. Similarly, we can compute the position of the third cut.

This solution works in $O(N)$ time.

E : Or Plus Max

In this editorial, for two integers a and b, "$a \subseteq b$" means that the set of positions of ones in the binary representation of a is a subset of that of b.

Ideally, for each x, we want to compute $max\ A_i + A_j\ s.t.\ (i\ or\ j) = x$. Then, we can compute the solutions as prefix maximums of this array. However, this is difficult.

Instead, for each x, compute $max\ A_i + A_j\ s.t.\ (i\ or\ j) \subseteq x$. Notice that the prefix maximums of this array give the same results because $(i\ or\ j) \subseteq x \rightarrow (i\ or\ j) \leq x$.

To compute this, for each x, we want to get the two largest values of A_i such that $i \subseteq x$.

Let's use an algorithm similar to Fast Zeta Transform. Imagine a set S_x, the (multi)set of all A_i such that $i \subseteq x$. We are only interested in the largest two elements of S_x.

- For each k from 0 to $N-1$,
  - For each x from 0 to 2^{N-1} such that the k-th bit of x is zero,
    - $S_{x|(1<<k)} := S_{x|(1<<k)} \cup S_x$
    - If $S_{x|(1<<k)}$ contains more than two elements, discard small elements while it has more than two elements.

This solution works in $O(N2^N)$ time.

F : Colorful Sequences

It's easy to compute the total number of occurrences of A in (not necessarily colorful) all sequences of length N: since there are $N - M + 1$ possibilities for the position of A and you can freely choose the remaining $N - M$ elements, this is $(N - M + 1)K^{N-M}$.

From now on, let's compute the total number of occurrences of A in non-colorful sequences of length N. The answer to the original problem is $(N - M + 1)K^{N-M}$ minus this number.

As a warm-up, consider the following problem:

> Count the number of non-coloful sequences of length N.

Define $dp[i][j]$ as the number of non-colorful sequences of length i, such that the last j elements are pairwise distinct but the last $j+1$ elements are not. The transition coefficients from $dp[i][j]$ to $dp[i+1][j']$ is as follows:

- In case $j + 1 < j'$, 0.
- In case $j + 1 = j'$, $K - j$.
- In case $j + 1 > j'$, 1.

This is $O(NK^2)$ with straightforward implementation, but with prefix sums we can do it in $O(NK)$.

Let's return to the problem. Consider three cases:

Case 1. A is colorful.

Obviously, the answer is 0.

Case 2. All elements in A are pairwise distinct. A is not colorful.

Consider the following problem:

> For each non-colorful sequence of length N, count the number of contiguous subsequences of length M whose elements are pairwise distinct. Find the sum of these counts.

Note that the elements in A are irrelevant to this problem: we only care the value of M. This problem can be solved in the almost same way as the DP solution in warm-up problem. Notice that each of $K!/(K - M)!$ sequences of length M occur in all non-colorful sequences the same number of times. Thus, the solution to the original problem is the solution to this problem, divided by $K!/(K - M)!$.

Case 3. There are duplicated elements in A. A is not colorful.

Define F, B as follows:

- The first F elements of A are pairwise distinct, but the first $F+1$ elements are not.
- The last B elements of A are pairwise distinct, but the last $B+1$ elements are not.

For each $i (0 \leq i \leq N-M)$, let's compute the number of ways to make a non-colorful sequence by attaching i elements to the beginning of A and $N-M-i$ to the end of A (the answer is the sum of these values). Notice that we can handle two directions independently. That is, this is the product of (the number of ways to make a non-colorful sequence (of length $M+i$) by attaching i elements to the beginning of A) and (the number of ways to make a non-colorful sequence by attaching $N-M-i$ elements to the end of A). These values can be computed in the same way as the warm-up problem.

For all three cases, this solution works in $O(NK)$ time.

AtCoder Beginner Contest 101 / AtCoder Regular Contest 099 解説

writer: semiexp

2018 年 6 月 23 日

For International Readers: English editorial starts on page 5.

A: Eating Symbols Easy

高橋君が思い浮かべている整数を，S の各文字ごとに更新していけばよいです．あるいは，S の中に現れる + の個数から，- の個数を引くことでも求められます．

- C++ による解答例: https://abc101.contest.atcoder.jp/submissions/2711190
- Ruby による解答例: https://abc101.contest.atcoder.jp/submissions/2711191
- Python による解答例: https://abc101.contest.atcoder.jp/submissions/2711198

B: Digit Sums Easy

$S(n)$ を計算できればよいです．これは,「現在の n の 1 の位を求め合計値に足し込み，1 の位を取り去る」ことを繰り返せばよいです．1 の位は 10 で割った余りとして求められます．また，1 の位を取り去った値は 10 で割った商として求められます．

- C++ による解答例: https://abc101.contest.atcoder.jp/submissions/2711205
- Ruby による解答例: https://abc101.contest.atcoder.jp/submissions/2711210
- Python による解答例: https://abc101.contest.atcoder.jp/submissions/2711212

C: Minimization

数列の最小値は 1 です．どのように操作を行っても，最小値が 1 以外に変化することはありません．なので，数列の要素をすべて 1 にすることを考えます．

数列中の 1 の個数は，はじめ 1 です．この個数は，1 回の操作で $K-1$ までしか増やすことができません．なぜならば，1 の個数を増やすためには，操作で選ぶ要素の中に 1 が含まれている必要があるためです．よって，少なくとも $\lceil \frac{N-1}{K-1} \rceil$ 回の操作が必要であることがわかります．

一方，うまく操作を行うことで，$\lceil \frac{N-1}{K-1} \rceil$ 回の操作ですべての要素を 1 にすることが可能です．はじめ $A_p = 1$ であるとします．ここで，$1 + g(K-1) \leq p \leq 1 + (g+1)(K-1)$ なる整数 g をとります．このとき，次のように操作を行えばよいです：

- $i = g, g-1, g-2, \ldots, 1$ に対して，この順で要素 $i(K-1)+1, i(K-1)+2, \ldots, i(K-1)+K$ を選んで操作を行う．

- $i = g+1, g+2, \ldots, \lceil \frac{N-1}{K-1} - 1 \rceil$ に対して，この順で要素 $i(K-1)+1, i(K-1)+2, \ldots, i(K-1)+K$ を選んで操作を行う．
- 要素 $N-K+1, N-K+2, \ldots, N$ を選んで操作を行う．

D: Snuke Numbers

「N 以上の整数 n であって，$\frac{n}{S(n)}$ を最小にするもの (複数ある場合は，そのうち n が最小になるもの)」を $f(N)$ で表します．すると，$N = 1$ から始めて，$N \leftarrow f(N+1)$ という更新を順次繰り返すことで，すぬけ数を小さいほうから順に列挙することができます．以降，この $f(N)$ について考えます．

まず，$\frac{x}{S(x)}$ の $x \geq N$ での最小値が常に存在することを示しておきます．$10^{d-1} \leq n < 10^d$ のとき，n の最小値は 10^{d-1}，$S(n)$ の最大値は $9d$ であることから，$\frac{x}{S(x)} \geq \frac{10^{d-1}}{9d}$ です．この下界は $d \geq 1$ では d を 1 増加させると少なくとも 5 倍になるので，十分 d を大きくすると $\frac{N}{S(N)}$ より大きくなります．よって，このような d に対して 10^d 未満の整数 x のみを考えればよいことから，最小値は存在します．

$x \geq N$ で $\frac{x}{S(x)}$ を最小にする x (複数ある場合は，そのうち最小のもの) をとります．$x > N$ と仮定して，x と N を上位の桁から比較していったとき，初めて異なる値になる桁を 10^d の位とします．このとき，$x = 10^{d+1}\lfloor \frac{N}{10^{d+1}} + 1 \rfloor - 1$ であること（つまり，x の $10^0, \ldots, 10^d$ の位の数字がすべて 9 であること）を示します．

まず，x の $10^0, 10^1, \ldots, 10^{d-1}$ の位の数字はすべて 9 です．さもなくば，$x' = 10^{d+1}\lfloor \frac{N}{10^{d+1}} \rfloor - 1$ を考えると，$N \leq x' < x$ かつ $S(x') \geq S(x)$ であることから，x が $\frac{x}{S(x)}$ を最小にすることに反します．

よって，$x_0 = 10^d \lfloor \frac{N}{10^d} + 1 \rfloor - 1$ とおくと，ある $1 \leq w \leq 9$ が存在して，$x = x_0 + w \cdot 10^d$ かつ $S(x) = S(x_0) + w$ となります．x の最適性より，$\frac{x_0}{S(x_0)} > \frac{x}{S(x)} = \frac{x_0 + w \cdot 10^d}{S(x_0)+w}$ が成立します．よって，$x_0 > S(x_0 \cdot 10^d$ が成り立ちます．一方，同様にして，w を大きくすればするほど $\frac{x}{S(x)}$ が小さくなることもわかります．実際には x の 10^d の位が 9 を超えることはできないので，その範囲で w を大きくするのが最適です．よって，x の 10^d の位も 9 であることが示されました．

よって，x の候補として考えられるものが N 並びに $d = 0, 1, 2, ...,$ に対して $x = 10^{d+1}\lfloor \frac{N}{10^{d+1}} + 1 \rfloor - 1$ の形で書けるものに限られることがわかりました．ここで，d としては $\log_{10} N + 1$ 以下で考えれば十分です．よって，候補を十分少なく絞ることができたので，これらすべてに対して $\frac{x}{S(x)}$ を計算して比較すれば答えが得られます．

E: Independence

問題を言い換えると，次のようになります：

> グラフから，できるだけ多くの辺を取り除いて，グラフ全体がちょうど 2 つのクリークの合併になるようにする．

これを，グラフを補グラフ（グラフの辺の有無を反転して得られるグラフ）にして考えると，次のように言い換えることができます：

> グラフに，できるだけ多くの辺を加えて，グラフ全体が完全二部グラフになるようにする．

以降，補グラフについて考えることにします．

元のグラフが二部グラフでなければ，如何に辺を加えてもグラフ全体を完全二部グラフにすることは不可能です．一方，元のグラフが二部グラフの場合は，頂点集合を 2 つの disjoint な集合 S, T に分割して，S, T にまたがる辺のみが存在するようにできるので，S, T にまたがる辺すべてを追加することで，グラフ全体を完全二部グラフにすることができます．

グラフを連結成分分解すると，各成分においては頂点集合の（いずれの辺も異なる集合間にまたがるような）二分割は，集合の入れ替えを除いて一意に定まります．i 番目の連結成分の頂点集合をこのように二分割したものを S_i, T_i とし

ます．すると，S は各 i に対して S_i, T_i のちょうど片方を選んで，それらの和集合をとることで構成することができます．(T は頂点全体から S に含まれる頂点を除くことで得られます)

さて，この問題中で問われている「両端の都市が同じ州内に属しているような道の本数」は，S の大きさのみによって定まることがわかります．よって，S の大きさとしてありうるものをすべて列挙すればこの問題を解くことができます．これは,「$1, 2, ..., i$ 番目の連結成分に対して S_i, T_i のうち選ぶほうを決めて，和集合をとることでできる集合の大きさ」を動的計画法で順次求めることで計算することができます．

F: Eating Symbols Hard

$L = 10^9$ とおきます．A, P を定めたとき，X の多項式 $f_{A,P}$ を次で定めます：

$$f_{A,P} = \sum_{i=-L}^{L} A_i X^{i-P}$$

明らかに，A と A' の各要素が一致することと，$f_{A,0} \equiv f_{A',0}$ は同値です．

文字列 S に対して，高橋君が S を 1 文字目から順に食べた後に高橋君が思い浮かべている数列 A を $T(S)$ で表し，$t(S) = f_{T(S),0}$ とします．このとき，$t(S)$ に対して，次の関係が成り立つことがわかります：

- $t(\epsilon) = 0$ (ϵ は空文字列)
- $t(\texttt{+}S) = t(S) + 1$ ($\texttt{+}S$ は，文字 + と文字列 S をこの順でつなげたもの．以下同様)
- $t(\texttt{-}S) = t(S) - 1$
- $t(\texttt{>}S) = t(S)X^{-1}$
- $t(\texttt{<}S) = t(S)X$

よって，X の多項式から X の多項式への関数 $\tau_{\texttt{+}}, \tau_{\texttt{-}}, \tau_{\texttt{>}}, \tau_{\texttt{<}}$ を次で定めると，$t(S) = \tau_{S_1} \circ \tau_{S_2} \circ \cdots \circ \tau_{S_{|S|}}(0)$ となることがわかります（$\circ$ は関数合成)：

- $\tau_{\texttt{+}}(p) = p + 1$
- $\tau_{\texttt{-}}(p) = p - 1$
- $\tau_{\texttt{>}}(p) = pX^{-1}$
- $\tau_{\texttt{<}}(p) = pX$

よって，求める値は，組 (i, j) $(1 \leq i \leq j \leq |S|)$ であって，$\tau_{S_i} \circ \cdots \circ \tau_{S_j}(0) = t(S)$ なるものの個数です．i を固定したときに，これを満たす j が何個あるかを効率的に数えることを考えます．

ここで，τ_c を合成して得られる関数は，$p \mapsto Ap + B$ (A, B は X の多項式，$A \not\equiv 0$) の形をしています．$A_{[i,j)}, B_{[i,j)}$ を，それぞれ $\tau_{S_i} \circ \cdots \circ \tau_{S_{j-1}}$ を $p \mapsto Ap + B$ の形で書いたときの A, B とします．$A_{[i,j)}, B_{[i,j)}, A_{[j,k)}, B_{[j,k)}$ がわかっているとき，$A_{[i,k)}, B_{[i,k)}$ は次のようにして求められます：

$$A_{[i,k)} = A_{[i,j)} \cdot A_{[j,k)},\ B_{[i,k)} = A_{[i,j)} \cdot B_{[j,k)} + B_{[i,j)}$$

このことから，

$$B_{[i,j)} = B_{[i,k)} - A[i,j) \cdot B_{[i,k)} = A_{[i,k)} \left(\frac{B_{[i,k)}}{A_{[i,k)}} - \frac{A_{[i,j)} \cdot B_{[j,k)}}{A_{[i,k)}} \right) = A_{[i,k)} \left(\frac{B_{[i,k)}}{A_{[i,k)}} - \frac{B_{[j,k)}}{A_{[j,k)}} \right)$$

がわかります．よって，$\tau_{S_i} \circ \cdots \circ \tau_{S_{j-1}}(0) = t(S)$ のとき，

$$\frac{B_{[j,|S|+1)}}{A_{[j,|S|+1)}} = \frac{B_{[i,|S|+1)} - t(S)}{A_{[i,|S|+1)}}$$

が成り立ちます．よって，各 j に対して $\frac{B_{[j,|S|+1)}}{A_{[j,|S|+1)}}$ を計算しておけば，i を固定したとき，$j > i$ であって $\frac{B_{[j,|S|+1)}}{A_{[j,|S|+1)}} = \frac{B_{[i,|S|+1)} - t(S)}{A_{[i,|S|+1)}}$ なる j の個数を求めることで，問題の答えが得られます．

ところで，$\frac{B_{[j,|S|+1)}}{A_{[j,|S|+1)}}$ は，一般には (N 次多項式) / (N 次多項式) の有理式になるため，これをすべての j に対して求めて記録することは難しいです．一方，この有理式を直接記録する代わりに，十分多くの X および $\bmod M$ で有理式の値を計算しておくことは可能です．

十分多くの X, M に対して有理式の値が一致すれば，十分高い確率で正しい答えが得られることを示します．この問題で考える有理式は，高々 (N 次多項式) / (N 次多項式) であるため，$P/Q = P'/Q'$ すなわち $PQ' = QP'$ かの判定は，高々 $2N$ 次の多項式同士の比較に相当します．$PQ' \neq QP'$ であるにもかかわらず $(PQ')(X) = (QP')(X) \pmod M$ となるような X は，高々 $2N$ 個しか存在しません．よって，X を一様ランダムに選んでくると，高々確率 $\frac{2N}{M}$ でこの 2 つは一致していると誤って判定されます．

比較すべき有理式の組は $\frac{(N+1)N}{2}$ 組です．よって，ランダムに X を k 個とって比較する場合，いずれかの組で誤った判定が起きる確率は，高々 $\frac{(N+1)N}{2} \cdot \left(\frac{2N}{M}\right)^k$ です．$M \sim 10^9$ 程度の素数をとれば，例えば $k = 6$ でこの確率は 10^{-12} 未満となり，十分高確率で正解することができます．

このアルゴリズムの計算量は，i ごとに一致する j の発見を `map` などを用いて行うと，$O(kN \log N)$ となります．

AtCoder Beginner Contest 101 / AtCoder Regular Contest 099 Editorial

writer: semiexp

June 23rd, 2018

A: Eating Symbols Easy

- C++ example: `https://abc101.contest.atcoder.jp/submissions/2711190`
- Ruby example: `https://abc101.contest.atcoder.jp/submissions/2711191`
- Python example: `https://abc101.contest.atcoder.jp/submissions/2711198`

B: Digit Sums Easy

- C++ example: `https://abc101.contest.atcoder.jp/submissions/2711205`
- Ruby example: `https://abc101.contest.atcoder.jp/submissions/2711210`
- Python example: `https://abc101.contest.atcoder.jp/submissions/2711212`

C: Minimization

The minimum of the entire sequence is 1. Since this value never changes, we want to make all elements equal to 1.

Initially, the sequence contains only one 1. In each step, the number of 1s increases by at most $K-1$ (because the minimum of the chosen interval doesnt't change). Thus, we need at least $\lceil \frac{N-1}{K-1} \rceil$ steps.

On the other hand, we can always achieve the goal within this number of steps, as follows. Let I_c be the interval $c(K-1)+1, c(K-1)+2, \ldots, c(K-1)+K$.

Note that for each c, I_c and I_{c+1} shares an element, and $I_1, \ldots, I_{\lceil \frac{N-1}{K-1} - 1 \rceil}$ covers the entire sequence. (Strictly speaking, to avoid the intervals going outside the sequence, we shuold shift the last interval to fit within the sequence.)

Suppose that initially, I_g contains the only 1. Then, perform the following:

- For each $i = g, g-1, g-2, \ldots, 1$ in this order, perform an operation on the interval I_i.
- For each $i = g+1, g+2, \ldots, \lceil \frac{N-1}{K-1} - 1 \rceil$ in this order, perform an operation on the interval I_i.

D: Snuke Numbers

Let $f(N)$ be the integer $n \geq N$ that minimizes the value of $\frac{n}{S(n)}$ (in case of tie, choose the smallest one). We start with $N = 1$, and by repeating $N \leftarrow f(N+1)$, we can list all Snuke numbers in the increasing order. Now we want to compute $f(N)$ for a given N.

We'll prove that $f(N)$ can be always obtained by changing the last (least important) k digits of N to 9 for some k.

Let $x = f(N)$. For simplicity, let x_i denote the i-th digit of x (i.e., the digit with coefficient 10^i). Assume that $x > N$ and the d-th digit is the first (most significant) digit where x and N differ. (Note that N and x always have the same number of digits - it's easy to check that 999...999 is always a Snuke Number.) We want to prove that $x_0 = \cdots = x_d = 9$.

Suppose that for some $i < d$, $x_i < 9$. Let y be the integer obtained by changing x_i to 9 and x_d to $x_d - 1$. Clearly, $N \leq y < x$ and $S(y) \geq S(x)$ holds. This contradicts the minimality of $\frac{x}{S(x)}$. Thus, $x_0 = \cdots = x_{d-1} = 9$.

Now, we know that $x_0 = \cdots = x_{d-1} = 9$, x_d is unknown, and $x_{d+1}, \ldots$ are the same as corresponding digits of N. Let z be the value of x in case $x_d = 0$, and let $w = x_d$. Then, we get $x = z + w \cdot 10^d$, $S(x) = S(z) + w$, and $\frac{x}{S(x)} = \frac{z+w\cdot 10^d}{S(z)+w}$. Since this is a monotonous function of w, the optimal value of w is either the smallest valid value (the d-th digit of N) or the largest valid value (9). We know that d-th digit is the digit where x and N differ, thus, $x_d = 9$. (This observation is not necessary - even without this observation, the number of candidates is limited enough.)

Now we have limited candidates for x. For each candidate compute the value of $\frac{x}{S(x)}$, and $f(N)$ is the one that minimizes this value.

E: Independence

By taking the complement of the graph, we can restate the condition of the problem as follows:

> Color the vertices of the graph with two colors. Between two vertices of the same color, there must not be an edge (of the complement graph).

From now on, simply the "graph" means "complement graph".

The condition above is equivalent to the standard rule of coloring: for each edge, the color of its two endpoints must be different. Clearly, if the given graph is not bipartite, the answer is -1.

Otherwise, for each connected component, we can uniquely determine a valid coloring (except for swapping two colors). Let S_i, T_i be the set of vertices of each color in the i-th component. Then, the set of vertices in State Taka must be formed in the following way: for each i, choose one of S_i or T_i, and take their union.

"The number of roads whose endpoint cities belong to the same state" only depends on the size of S. Since we can list all possible sizes of S by a simple DP, we can solve the problem.

F: Eating Symbols Hard

Let's use rolling hashes. Fix a large prime M, and also fix a random integer X. For a string (of symbols) S, we define $t(S)$ as follows. Let A be the sequence we get when we eat S. Then,

$$t(S) = (\sum_i A_i X^i) \pmod M$$

We can use the following properties to compute $t(S)$ (note that all equations will be evaluated modulo M):

- $t(\epsilon) = 0$ (ϵ is an empty string)
- $t(\texttt{+}S) = t(S) + 1$ ($\texttt{+}S$ is a concatenation of a character + and a string S in this order)
- $t(\texttt{-}S) = t(S) - 1$
- $t(\texttt{>}S) = t(S)X^{-1}$
- $t(\texttt{<}S) = t(S)X$

Thus, $t(S) = \tau_{S_1} \circ \tau_{S_2} \circ \cdots \circ \tau_{S_{|S|}}(0)$ （$\circ$ are compositions of functions）, where functions $\tau_{\texttt{+}}, \tau_{\texttt{-}}, \tau_{\texttt{>}}, \tau_{\texttt{<}}$ are defined as follows:

- $\tau_{\texttt{+}}(p) = p + 1$
- $\tau_{\texttt{-}}(p) = p - 1$
- $\tau_{\texttt{>}}(p) = pX^{-1}$
- $\tau_{\texttt{<}}(p) = pX$

Let c be the hash value we get when we execute the entire sequence. We want to count the number of pairs (i, j) such that $\tau_{S_i} \circ \cdots \circ \tau_{S_j}(0) = c$.

Notice that the functions $\tau_{\texttt{+}}, \tau_{\texttt{-}}, \tau_{\texttt{>}}, \tau_{\texttt{<}}$ are bijective, and we can define their inverses. We have the following:

$$\begin{aligned} & \tau_{S_i} \circ \cdots \circ \tau_{S_j}(0) = c \\ \Leftrightarrow \quad & \tau_{S_N}^{-1} \circ \cdots \circ \tau_{S_i}^{-1} \circ \tau_{S_i} \circ \cdots \circ \tau_{S_j}(0) = \tau_{S_N}^{-1} \circ \cdots \circ \tau_{S_i}^{-1}(c) \\ \Leftrightarrow \quad & \tau_{S_N}^{-1} \circ \cdots \circ \tau_{S_{j+1}}^{-1}(0) = \tau_{S_N}^{-1} \circ \cdots \circ \tau_{S_i}^{-1}(c) \end{aligned}$$

For an integer i, define a_i, b_i as follows:

- $a_i = \tau_{S_N}^{-1} \circ \cdots \circ \tau_{S_i}^{-1}(0)$
- $b_i = \tau_{S_N}^{-1} \circ \cdots \circ \tau_{S_i}^{-1}(c)$

Notice that the inverses of $\tau_{\texttt{+}}, \tau_{\texttt{-}}, \tau_{\texttt{>}}, \tau_{\texttt{<}}$ are linear functions, and their compositions are also linear functions. Thus, by keeping $\tau_{S_N}^{-1} \circ \cdots \circ \tau_{S_i}$ as a linear function, we can compute a_i, b_i for all i in linear time.

Now our task is to count the number of pairs (i, j) such that $a_{j+1} = b_i$. It can be easily done with maps, and works in $O(N \log N)$ time.

Let's estimate the collision probability of hashes. If we regard $t(S)$ as a function of X, $S = S'$ if and only if $t(S) = t(S')$. Otherwise, $t(S)$ and $t(S')$ are different as functions, and since they are polynomials of degree $2N$ (after some scaling), the number of X that makes $t(S) = t(S')$ is at most $2N$. By choosing X uniformly at random from the field F_M, the probability of collision is $\frac{2N}{M}$. By using k different values for X, since we compare $\frac{(N+1)N}{2}$ pairs of polynomials, the total failure probability is at most $\frac{(N+1)N}{2} \cdot \left(\frac{2N}{M}\right)^k$. This is less than 10^{-12} for $M \sim 10^9$ and $k = 6$.

Atcoder Regular Contest 解説

writer : maroonrk

平成 30 年 5 月 26 日

For International Readers: English editorial starts from page 7.

A : Add Sub Mul

整数の足し算、引き算、掛け算はほとんどのプログラミング言語で標準サポートされているでしょう。あとは、3 つの整数の最大値が求められれば良いです。これは if 文による条件分岐で実装出来ますが、プログラミング言語によっては、与えられたいくつかの整数の最大値を求める関数が用意されている場合もあります。

C++による解答コード

B : Cut and Count

文字列を分割する位置を全探索します。英小文字 26 種類すべてについて、分割されたあとの 2 つの文字列の両方に含まれるかの判定を行えば、「X と Y の両方に含まれる文字の種類数」が求まります。あとはその中で最大値を求めれば良いです。

26 種類の文字全てに対して、$if(s[i] == `a`)$, $if(s[i] == `b`)$ $\cdots$ と言ったふうに判定関数を書くのは骨の折れる作業ですが、ASCII コードで、$`a`$ $`b`$ $\cdots$ $`z`$ が並んでいるという性質を利用するとコードが簡単になります。

C++による解答コード

C : Attention

西から i 番目の人がリーダーになった際、向く方向を変える必要のある人は、西から $j(j < i)$ 番目に並んでいて西を向いている人と、西から $j(j > i)$ 番目に並んでいて東を向いている人です。

ここで、「ある範囲に存在する西 (東) を向いている人の人数」は、累積和を使うと、前処理 $O(N)$ 時間、1 クエリあたり $O(N)$ 時間で求められます。

よって、まず最初に $O(N)$ 時間で累積和を求めておくと、各人について、その人がリーダーになった際に向く方向を変える必要のある人の人数を求めることが $O(1)$ 時間ででき、合計 $O(N)$ 時間で全員分求まります。あとはその中で最小値を求めればよいです。

D : Xor Sum 2

整数列 $c_1,\ c_2,\ \cdots,\ c_m$ が、$c_1\ xor\ c_2\ \cdots\ xor\ c_m\ = c_1 + c_2 + \ \cdots\ + c_m$ を満たす条件を考えると、これは、すべての $k(0 \leq k)$ に対して、2 進法表記したときの 2^k の位が 1 であるよう c_i の個数が 1 以下であることと同値です。これは、$a + b - (a\ xor\ b) = 2 \times (a\ and\ b)$（ただしここで and はビットごとの論理積を表す）であることからわかります。

l を固定したときに、(l, r) が条件を満たすような最大の r を $f(l)$ とおきます。すると、f は l に対して広義単調増加です。よって、l を増加させながら尺取り法の要領で $f(l)$ を求めることができ、$O(N)$ で答えを求めることが出来ます。

この他にも、各 bit についてそのビットが立っている位置を保存することで、直接的に $f(l)$ を求める解法も存在します。

E : Range Minimum Queries

取り除いく要素の最小値 X を決めたとします。この条件のもので、乗り除く要素の最大値 Y を最小化することを考えます。

最小値が X という条件を守るためには、X より小さい要素を含むような区間に対して操作をすることは出来ません。そこで、与えられた数列を、X より小さい要素の位置で切ると、いくつかの列に分割されます。分割されたあとの列の中で完結する操作は自由に行なえます。

分割したあとの列の一つを $c_1,\ c_2,\ \cdots,\ c_m$ とします。この列からは、高々$m-K+1$ 個の要素を取り除くことが出来ます。小さい方から取り除くほうが好ましいので、この列から取り除かれる可能性があるのは、$c_1,\ c_2,\ \cdots,\ c_m$ の中で小さい方から $m-K+1$ 個の要素です。

分割したあとの各列について、取り除かれる可能性のある値を列挙すると、それらすべての中で Q 番目に小さいものが、Y の最小値になります。

列の要素の小さい方から何要素かを得るのは、列をソートすれば出来ます。よって、X を決めておくと、Y の最小値は $O(NlogN)$ で求まることになります。

X は N 通りあるので、合計 $O(N^2logN)$ でこの問題は解けます。

余談ですが、実はこの問題は $N \leq 10^5$ で解くことが出来ます。

F : Donation

各頂点 v について、$C_v = max(A_v - B_v, 0)$ と定義します。ここで、問題で行う操作を逆から見ると、次のようなゲームをしていると思うことが出来ます。

> 適当な頂点 s から、スタートする。このときの所持金は好きな値でよい。これから、以下の 2 つの操作を好きな順序で好きなだけ繰り返す。
>
> - 今いる頂点を v としたとき、B_v 円を得る。ただし、今いる頂点をでお金をもらえるのは 1 度だけである。
> - 今いる頂点を v としたとき、所持金が A_v 円以上なら、隣接する頂点から好きなものを選んでそこに移動する。
>
> すべての頂点でお金をもらい、かつ、最後に立っている頂点を t としたときに、所持金が A_t 円以上ならゲームクリアとなる。

このゲームの性質を考えると、今まで訪れたことのない頂点に移動してきた際には、すぐにお金を得るのが最善です。また、一度訪れた頂点は、その後自由に移動できます。以上の考察より、ゲームを更に次のように簡略化出来ます。

> 適当な頂点 s から、スタートする。このときの所持金は C_s 円以上でなくてはならない。これから、以下の操作を、すべての頂点を訪れるまで繰り返す。
>
> - 今まで訪れた頂点に隣接する、まだ訪れていない頂点を一つ選び、移動する。移動先の頂点を v とすると、このときの所持金は C_v 円以上でなくてはならない。そして、移動したあと B_v 円を得る。
>
> すべての頂点を訪れることができればゲームクリアとなる。

このゲームでは、次に移動可能な頂点の中で C_v が最小のものを選び、移動するという戦略が最適だとわかります。

C_v が最大の頂点を z とします。ここで、z をグラフから取り除いたあとの連結成分を P_1, P_2, $\cdots P_L$ とおきます。（ここで L は 0 かもしれません）最適解においてスタートする頂点が z だとします。その場合、ゲームクリアのための所持金の最小値は、C_z に、B_v の総和を足したものになります。最適解においてスタートする頂点が P_i 内にあったとします。その場合、ゲームクリアのための所持金の最小値は、P_i だけでゲームをクリアするための所持金の最小値を Q_i とすると、$max(Q_i, C_z) + P_i$ 以外の頂点の B_vの総和 となります。

これで多項式解法が得られましたが、愚直にグラフを分割していては間に合いません。グラフが分割される様子を逆から見ると、C_v の小さい頂点だけでできるだけ大きな連結成分を作っていると思うことが出来ます。よって、各辺のコストを、端点の C_v の max だと思って、小さいコストの辺から優先的に使っていくと考えれば、Union-Find を使って、グラフが分割される様子を表す根付き木を作ることが出来ます。この木の各頂点は、C_v がある一定の値以下の頂点のみからなるある連結成分に対応することになります。この木の各頂点に対して、対応するグラフ内だけの問題を解いたときの答え、及び B_v の総和を求めると、木 DP によって全体の答えが求まります。

C_v の昇順に頂点を並べたり、Union-Find の操作をするのがボトルネックになり、この問題は $O(NlogN)$ で解けます。

Atcoder Regular Contest Editorial

writer : maroonrk

平成 30 年 5 月 26 日

A : Add Sub Mul

C++ Example

B : Cut and Count

C++ Example

C : Attention

If the i-th (from the west) person is chosen as the leader, the j-th person have to change the direction in the following two cases:

- $j < i$ and the j-th person is facing west.
- $j > i$ and the j-th person is facing east.

First, let's pre-compute prefix sums: for each i, count the number of people among the west-most i people who are facing east. This can be done in $O(N)$ time. After that, we can answer a query of the form "How many people in a given range are facing east (west)?" in $O(1)$ time, thus the two values above can be computed in $O(1)$ after we fix the leader.

This solution works in $O(N)$ time in total.

D : Xor Sum 2

Notice that $a + b - (a\ xor\ b) = 2 \times (a\ and\ b)$. The inequality $a + b \geq a\ xor\ b$ always hold, and the equality holds if and only if $a\ and\ b = 0$. (Intuitively, the equality holds if we get no "carries" when we add a and b as binary numbers.)

Therefore, for a sequence of non-negative integers $c_1,\ c_2,\ \cdots,\ c_m$, the equation

$c_1\ xor\ c_2\ \cdots\ xor\ c_m\ = c_1 + c_2 + \ \cdots\ + c_m$

holds if and only if for all k, the number of elements among $c_1, \ldots, c_m$ that contains the k-th bit (in binary representation) is at most one.

For a fixed l, the interval $[l, r]$ satisfies the condition in the statement if no bit is repeated among $a_l, \ldots, a_r$. Thus, there exists an integer $f(l)$, and the condition is satisfied if and only if $r \leq f(l)$. Since $f(l)$ is monotonously non-decreasing, we can compute the value of $f(l)$ for all l in $O(N)$ time using two-pointers.

It is also possible to directly compute $f(l)$: for each k, list the indices of all elements that contains the k-th bit.

E : Range Minimum Queries

Let's fix the value X: the lower bound of removed integers. Under the condition that we are never allowed to remove integers less than X, let's minimize the value of Y (the upper bound of removed integers). Note that we slightly modified the definition of X: now it is not necessary to remove X.

In order to satisfy the condition, we can never choose an interval that contains an integer less than X. Thus, let's split the entire sequence by integers less than X. Now we get (possibly multiple) sequences that don't contain integers less than X, and we can freely perform operations within each of these sequences.

Let $c_1,\ c_2,\ \cdots,\ c_m$ be one of the sequences we get after the split. We can remove at most $m - K + 1$ elements from this sequence (because before the last operation there must be at least K elements). It is clearly optimal to remove integers from the smallest ones. Thus, the $m-K+1$ smallest elements in this sequence have possibility to be removed.

We do the same thing for each part of splitted sequences, and list all elements that may be removed. If there are Q or more such elements, the minimum possible value for Y is the Q-th smallest element among them.

We can do this in $O(N \log N)$ for a fixed X. If we try all N possibilities for X, this solution works in $O(N^2 \log N)$ time in total.

Note that this problem is solvable even for $N \leq 10^5$. This part is left as an exercise for readers.

F : Donation

Let's modify the problem a bit. Instead of the condition about A_v, we require the following constraint: for each vertex v, whenever you are at vertex v, your money must be at least $C_v := max(A_v - B_v, 0)$ (even after you donate money to the vertex).

It turns out that this problem is equivalent to the original problem. It's clear that a valid sequence of moves in the original problem is also a valid sequence of moves in the modified problem. On the other hand, since it never makes sense to visit a vertex again after you donate for this vertex (in this case, we can "postpone" the donation), a valid sequence of moves in the modified problem can be converted to a valid sequence of moves in the original problem.

Let v be the vertex that maximizes the value of C_v. From the observation above, we can assume that, after we donate for this vertex we never visit this vertex. Let $G_1, \ldots, G_k$ be connected components we get when we remove the vertex v from the graph. Suppose that we end the game inside G_i. Then, after we donate for v, we must move inside G_i.

We claim that (one of) optimal solutions is of the following form:

- We donate for all vertices in $G_1, \ldots, G_{i-1}, G_{i+1}, \ldots, G_k$.
- Move to the vertex v, and donate for it.
- We donate for all vertices in G_i. We never go out of this subgraph during this phase.

Suppose that there is an optimal solution that is not of this form. Then, there exists a vertex w in G_i such that we donate for w before v. In this case, we can cancel the donation for w, and instead we add a donation for w right after the donation for v, and improve the solution. Therefore, we proved the claim above.

Let's construct a rooted tree as follows:

- v (the vertex that maximizes C_v) is the root of the tree.
- Recursively construct rooted trees for subgraphs $G_1, \ldots, G_k$.
- Add an edge between v and the root of the tree for G_i for each i.

We can construct this tree in $O(N \log N)$ time in the order from leaves to root by handling vertices in the increasing order of C.

Now, we can do a DP on this tree. For each vertex u, define $dp[u]$ as the minimum amount of money required to donate for all vertices in the subtree rooted at u. Then we can fill the DP table in the order from leaves to the root.

This solution works in $O(N \log N)$ time.

ABC097 / ARC097 解説

sigma425

For International Readers: English editorial starts on page ???.

A: Colorful Transceivers

問題文の通り実装します。二点 a,b 間の距離は標準的な言語なら絶対値関数で計算できるでしょう。自分で定義しても良いと思います。たとえば C++ なら abs($a-b$) と書けます。あとは、(A と C が直接会話可能) または ((A と B が直接会話可能) かつ (B と C が直接会話可能)) という論理構造を、論理演算子を使ってコードに落とし込めばよいです。以下は C++ のコード例です。

```
#include <bits/stdc++.h>
using namespace std;
int main(){
        int a,b,c,d;
        cin>>a>>b>>c>>d;
        if(abs(a-c)<=d || (abs(a-b)<=d && abs(b-c)<=d)) puts("Yes");
        else puts("No");
}
```

B: Exponential

各 1 以上 X 以下の整数 x に対して、$expo_x := x$ がべき乗数か という真偽値を考えます。これが正しく計算できれば答えは求まります。

1 以上の整数 b と、 2 以上の整数 p 全てのペアに対して $x := b^p$ を計算し、x が 1 以上 X 以下なら $expo_x$ を true に書き換える ということをすると定義より正しく求まりますが、当然すべての整数のペアを試すのは不可能です。

まず、b として試すべき値は 1 以上 X 以下の整数 のみでよいです (なぜなら、$b > X$ なら任意の $p >= 2$ に対して $b^p > X$ だから答えに影響を及ぼさない)。

同様に、 p として試すべき値は (とても雑に見積もっても) 2 以上 X 以下の整数 のみで十分です。よって二重ループで b, p の全てのペアを試すことで $expo_x$ が正しく求まります。 時間計算量は $O(X^2)$ です。

しかし、オーバーフローを起こさないためにも以下の方法が良いと思います。

各 b に対して、$p = 2, 3, ..$ と順番に, $expo_{b^p}$ を true に書き換えていきます。ここで、もし今調べている値 b^k が X より大きかったら、k 以降の値は p として考える必要がありません。従ってそこで p のループを打ち切って良いです。ただし $b = 1$ の時にはこの条件での打ち切りは発生しないので注意してください。

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){
        int X;
        cin>>X;
        vector<bool> expo(X+1);

        expo[1] = 1;
        for(int b=2;b<=X;b++){
                int v = b*b;
                while(v<=X){
                        expo[v] = 1;
                        v *= b;
                }
        }
        for(int i=X;i>=1;i--) if(expo[i]){
                cout<<i<<endl;
                return 0;
        }
}
```

C: K-th Substring

解説では $N := |s|$ とおきます。

まず部分点解法 ($N < 50$) を説明します。全ての s の substring を列挙し、重複を除き、ソートします。これにはいろいろな方法がありますが、例えば C++ だと sort したあと unique 関数を使ったり、set を使うなどが考えられます。その後、小さい方から K 番目の文字列を出力すればよいです。

計算量は, $O(N^2)$ 個の 長さ $O(N)$ の文字列をソートするので、 $O(N^3 logN)$ になります。(比較回数が $O(N^2 logN)$, 1 回の比較にかかる時間が $O(N)$)

満点解法では、答えの文字列の長さが高々 K であるということに着目します。(これは、文字列 t の prefix(であって t でないもの) が 全て異なり、かつ strict に t より小さいことから示せます。) この事実からはじめに列挙する必要があるのは 長さ K 以下の substring $O(NK)$ 個だけです。これをソートすると、 時間計算量は $O(NK^2(logNK))$ となり、十分高速です。

D: Equals

以下のような無向グラフ G を考えます。

頂点集合: 1 以上 N 以下の整数

辺集合: 各 $1 \leq j \leq M$ に対して、(x_j, y_j) という辺を張る

すると次のことが示せます:

$S := \{i | G$ において i と p_iが同じ連結成分にある $\}$ とおくと、 最終状態で全ての $i \in S$ に対して同時に $p_i = i$ とすることが出来る

証明は後に回します。逆に G において i と p_i が異なる連結成分に属してしまっている場合, 操作を行っても $p_i = i$ と出来ないことは明らかです。従って、$|S|$ が答えになります。各 i が S に 属するかは UnionFind 等を使うと求めることが出来ます。

証明: 気持ち的には同じ連結成分内では好きに値を入れ替えられることからわかります。ちゃんとやると、各連結成分 C に対して、C から $\{p_i | i \in C\}$ への全単射 f であって $i \in S$ なら $f(i) = i$ となるようなものが

とれて、連結成分の全域木をとって、ひとつの葉 i に正しい値 $f(i)$ を置くということを繰り返せば示せます。

E: Sorted and Sorted

簡単のため、 i が書かれた 黒い/白い ボールを 黒/白 の i と呼びます。

最終状態を一つ固定した時に、その状態になるまでに必要な最小の操作回数 f は転倒数で表せます。すなわち、$f = \#\{(x, y) : \text{ボールの組 } | x \text{ は初期状態で } y \text{ より左にある} \wedge x \text{ は最終状態で } y \text{ より右にある}\}$

ボール x を固定した時に上の集合に入る (x, y) の個数を f_x と置きます。 $f = \sum f_x$ です。

次のような DP を考えます。$dp_{i,j}$ = 最終状態においてボールを左から順に置いていって、黒いボールを i 個, 白いボールを j 個 置いた時の, これまで置いたボール x に関する f_x の和 の最小値

これは次のように更新できます。$dp_{i,j} = \min(dp_{i-1,j} + costb_{i-1,j}, dp_{i,j-1} + costw_{i,j-1})$ ここで、$costb_{i,j}$ とは、 黒いボールを i 個, 白いボールを j 個既に置いている時に 黒の $i+1$ を置いた時の $f_{\text{黒の } i+1}$ の値です。これは well-defined です。(つまり、これからどう置くかや、これまでどう置いたかには依存していません) このことは x の左にあるボールの集合が i, j だけから一意に定まることからわかります。$costw$ も同様です。

$costb, costw$ は BIT を使って $O(N^2 logN)$ で求めたり, “左 k 個にある i 以下の黒いボールの個数” などを前計算したりすることで $O(N^2)$ で求めることが可能です。すると全ての $dp_{i,j}$ も $O(N^2)$ で求められます。答えは $dp_{N,N}$ です。

F: Monochrome Cat

まず黒い葉をわざわざ訪れる必要はありません。なので、黒い葉があればそれを消す、ということをできるだけ繰り返します (これは queue 等を使うと簡単にできます)。するといつかは全ての葉が白くなります。以下この木について考えます。

“現在いる頂点と隣接した頂点をひとつ選びその頂点に移動する。その後、移動先の頂点の色を反転する。” という操作で 頂点 a から 頂点 b に移動した場合、この操作を $a \Rightarrow b$ で表します。

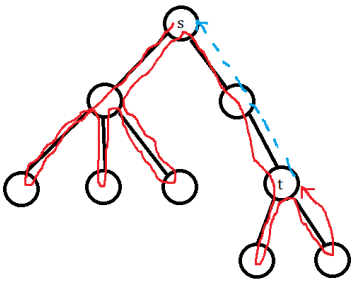
また、” 今いる頂点の色を反転する” という操作を 頂点 a で行った場合、この操作を flip a で表します。

まず、ひとつの辺を 3 回以上渡ることはありません。なぜなら、(操作列 1) $\rightarrow$ ($a \Rightarrow b$) $\rightarrow$ (操作列 2) $\rightarrow$ ($b \Rightarrow a$) $\rightarrow$ (操作列 3) $\rightarrow$ ($a \Rightarrow b$) $\rightarrow$ (操作列 4) という操作列は、(操作列 1) $\rightarrow$ (操作列 3) $\rightarrow$ (flip a) $\rightarrow$ ($a \Rightarrow b$) $\rightarrow$ (flip b) $\rightarrow$ (操作列 2) $\rightarrow$ (操作列 4) と変更すると、操作回数は保ったまま、他の辺の使用回数を変えずに、辺 (a, b) の使用回数を減らせるからです。

よって各辺は高々 2 回しか使わないとしてよいです。

また、全ての葉が白なので一度は訪れる必要があることから、すべての頂点を訪れる必要があることがわかります。特に各辺を少なくとも 1 回は使います。

この 2 つの考察から、猫の移動の様子は次の図の赤線のように限定されます:



すなわち、始頂点 s から Euler tour をする途中、すべての頂点を訪れたあとなら 終頂点 t で移動をやめて良い という形です。

移動操作の列が決まっている時、flip 操作 を何回行えばいいかは一意に定まります。なぜなら、各頂点を少なくとも一度は訪れているので、もし” すべての移動操作が終わったあとに頂点 v の色が白になっている” ならば、 頂点 v を訪れたタイミングで flip v をすることにすればよいからです。

ここで、Euler tour が最後まで完了するように、操作列に青の破線のように追加することを考えます。この時の操作回数は、$2(N-1)+\#\{v \in G |$ (v : white) xor ($\deg_v$ is odd) $\}$ となり、s,t や動き方には依存しません。

ここから青の追加分を消すことを考えます。操作回数がどのくらい減るかは、青の破線の path を $t = b_0, b_1, .., b_{K-1}, b_K = s$ とおくと、次の f で表せます。

$f := K + \#\{v \in \{b_1, b_2, ., b_K\} |$ (v : white) xor ($\deg_v$ is odd) $\}$ - $\#\{v \in \{b_1, b_2, ., b_K\} |$ (v : white) xor ($\deg_v$ is even) $\}$

変形して、

$f = 2 \times \#\{v \in \{b_1, b_2, ., b_K\} |$ (v : white) xor ($\deg_v$ is odd) $\}$

この f を最大化するのが目的です。これは t, s のみに依存することに注意してください。

この問題は次のように言い換えられます:

無向グラフが与えられる。各頂点には値 0 か 1 が書いてある。最大で何個 1 が書いてある頂点を含める path がとれるか?

($\because b_0, .., b_K$ は任意の path をとれます. path をわざわざ短くする必要はないので, b_0, b_K は葉としてよいです。**頂点数が 2 以上なら** 定義から葉, 特に b_0 に書いてある値は 0 なので、結局 (上の問題の答え) $\times 2$ が f になります。)

これは木の直径を求めるのと同様のアルゴリズムで dfs 2 回で可能です。 これで $O(N)$ で元の問題の答えが求まりました。

初期状態で白い頂点が 0 個,1 個のコーナーケースに注意してください。(1 個の場合は上の $\because$ 内の太字部分が成り立たないので)

ABC097 / ARC097 Editorial

sigma425

A: Colorful Transceivers

C++ example:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
        int a,b,c,d;
        cin>>a>>b>>c>>d;
        if(abs(a-c)<=d || (abs(a-b)<=d && abs(b-c)<=d)) puts("Yes");
        else puts("No");
}
```

B: Exponential

C++ example:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
        int X;
        cin>>X;
        vector<bool> expo(X+1);

        expo[1] = 1;
        for(int b=2;b<=X;b++){
                int v = b*b;
                while(v<=X){
                        expo[v] = 1;
                        v *= b;
                }
        }
        for(int i=X;i>=1;i--) if(expo[i]){
                cout<<i<<endl;
                return 0;
        }
}
```

C: K-th Substring

For an arbitrary string t, each of its proper suffix is lexicographically smaller than t, and the lexicographic rank of t is at least $|t|$. Thus, the length of the answer is at most K.

Generate all substrings of s whose lengths are at most K. Sort them, unique them, and print the K-th one. This solution works in $O(NK^2(logNK))$ time, where $N = |S|$.

D: Equals

Consider a graph G: there are N vertices numbered 1 through N, and for each j there is an edge (x_j, y_j).

Then, by performing some operations, p_i can be i if they are in the same connected component of the graph. We can also prove that we can satisfy $p_i = i$ for all such i simultaneously. (Formal proof: take a spanning tree of a connected component. Choose its leaf v, perform opeartions such that $p_v = v$ is satisfied, and never make operations involving v after that.)

We can compute the number of such i by DFS or DSU.

E: Sorted and Sorted

Suppose that we know the final order of balls. Then, we can compute the number of operations as an inversion number: the number of ordered pairs of two balls (x, y) such that x is to the left of y at the beginning, but x is to the right of y in the final order.

Let's decide the positions of balls and place them one by one, from left to right (in the final order). From the constraints in the statement, at any moment, the set of placed balls must be of the form "i black balls numbered 1 through i, and j white balls numbered 1 through j". Define $dp_{i,j}$ as the minimum possible inversion number among them when we place those balls. The answer is $dp_{N,N}$.

Then, the recurrence formula will be as follows:

$dp_{i,j} = \min(dp_{i-1,j} + costb_{i-1,j}, dp_{i,j-1} + costw_{i,j-1})$

Here, $costb_{i,j}$ is the cost required to append black $i+1$ when the first i black balls and the first j white balls are already placed. That is, the number of balls that are already placed and initially placed to the right of black $i+1$. Similarly, $costw_{i,j}$ are defined.

We can pre-compute the arrays $costb_{i,j}$, $costw_{i,j}$ like prefix sums in $O(N^2)$. Thus, this solution works in $O(N^2)$ time.

F: Monochrome Cat

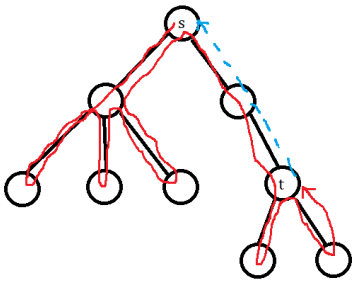
If the tree contains a black leaf, it never makes sense to visit it. Thus, we can repeatedly remove black leaves until all leaves become white. (This can be implemented by, for example, using a queue.) From now on, we assume that all leaves in the input tree are white. We also assume that the tree has at least two white vertices to avoid some special cases.

Suppose that we are given two vertices s and t, and the cat must start at s and end at t. How many operations do we need in this case?

First, for each edge e, let's compute the number of times the cat passes through e.

- It never makes sense to pass through it three or more times. For example, suppose that we move between two vertices a and b three times. The sequence of operations will look like "W $\to$ ($a \Rightarrow b$) $\to$ X $\to$ ($b \Rightarrow a$) $\to$ Y $\to$ ($a \Rightarrow b$) $\to$ Z" However, we can replace it with "W $\to$ Y $\to$ (flip a) $\to$ ($a \Rightarrow b$) $\to$ (flip b) $\to$ X $\to$ Z" and the number of operations will be smaller. Here, W, X, Y, Z represent some sequences of operations, $a \Rightarrow b$ denotes an operation of the first type, and flip a denotes an operation of the second type.
- We must pass through each edge at least once (otherwise some white leaves will be left unvisited.)

Thus, if e is on the path between s and t, we pass through it exactly once, otherwise we pass through it exactly twice. The trajectory of the cat will look as follows:



Now, we know the number of operations of the first type. How can we compute the number of operations of the second type?

Suppose that if we never make operations of the second type, k white vertices remains. Then, we need k extra steps to change the color of those vertices. This is always possible because we visit all vertices.

Let's call those k vertices "bad vertices", and other vertices "good vertices". Depending on the initial color and the parity of degree of the vertex, there are two types of vertices:

- It becomes a bad vertex if it is on the $s - t$ path, otherwise it becomes a good vertex.
- It becomes a good vertex if it is on the $s - t$ path, otherwise it becomes a bad vertex.

In summary, the total number of opeartions can be computed as follows (for a given pair of s, t):

- Add twice the total number of edges.

- Subtract the distance between s and t.
- Subtract the number of bad vertices.

This can be simplified as follows. For each vertex, the cost of zero or two is assigned depending on the type of the vertex mentioned above. Then,

- Add twice the total number of edges, plus one.
- Subtract the total cost of all vertices on the path between s and t.

Now our objective is to find a path that maximize the total cost of vertices on it. This is similar to the computation of diamater of a weighted tree, and can be done in $O(N)$.

ABC 095 / ARC 096 Editorial (Japanese)

問題・解説: evima

2018 年 4 月 21 日

For International Readers: English editorial starts on page 7.

A: Something on It

この問題を解くには、以下の手順を踏む必要があります。

0. （言語によっては不要）文字列変数 S を宣言する。
1. 標準入力から S を文字列として受け取る。
2. S から何らかの方法で答えとなる整数を得る。
3. 求めた値を標準出力に出力する。

手順 0, 1, 3 については、practice contest の問題 A の問題別サンプルコードが参考になる（少し書き換えれば使える）でしょう。手順 2 については、S の 3 つの文字それぞれを if 文でチェックするか、もしくは言語の機能を用いて S に含まれる `o` の個数を数えることになります。前者の C++ による実装例と後者の Python(3) による実装例を示します。

```cpp
#include <iostream>
using namespace std;
int main(){
    string S;
    cin >> S;
    int ans = 700;
    if(S[0] == 'o') ans += 100;
    if(S[1] == 'o') ans += 100;
    if(S[2] == 'o') ans += 100;
    cout << ans << endl;
}
```

```python
print(700 + 100 * input().count('o'))
```

B: Bitter Alchemy

入力例の説明でほのめかされているように、「まずすべての種類のドーナツを 1 個ずつ作り、残ったお菓子の素で最も『安い』ドーナツを作れるだけ作る」という方針で問題ありません。つまり、$m_1, m_2, ..., m_N$ の和を S、$m_1, m_2, ..., m_N$ のうち最小の値を M として、答えは $N + \lfloor (X - S)/M \rfloor$ となります。[*1]主な課題は、$m_1, m_2, ..., m_N$ の値を読み込むことと、S や M の値を実際に求めることです。なお、割り算と小数部分切り捨ての部分に関しては、多くのプログラミング言語では単に `A / B` と書くことで $\lfloor A/B \rfloor$ が求められます。[*2]

言語によってやや事情が異なるかもしれません。C++, Java などの言語では、ループ構造を用いるのが素直でしょう。C++ での実装例を挙げます。

```cpp
#include <iostream>
using namespace std;

int main(){
    int N, X, m[101];
    cin >> N >> X;
    int S = 0, M = 1001001001;
    for(int i = 0; i < N; ++i){
        cin >> m[i];
        S += m[i];
        if(m[i] < M){
            M = m[i];
        }
    }
    int ans = N + (X - S) / M;
    cout << ans << endl;
}
```

Python, Ruby といった言語でももちろん上のようなアプローチが可能ですが、こういった言語では `sum`, `max` といった関数がはじめから用意されていることがあり、便利です。[*3]これを用いた Python(3) での実装例を次ページに挙げます。

[*1] $\lfloor x \rfloor$ は x の整数部分を表します。
[*2] A, B が正でない場合はこの限りでなく、またここに実装例を掲載した Python3 はこれに該当しません。
[*3] このような機能のため、AtCoder Beginner Contest で出題されうる程度の問題の解法を手っ取り早く実装する上で、スクリプト言語はとても役立ちます。ただし、この範囲を超えると、どうしても実行速度が足りない場面が急速に増えます。競技プログラミングに本格的に取り組む場合、C++ とは言わずとも Java に匹敵する実行速度を有する言語を一つ習得する必要がある、ということをお伝えしておきます。

```python
N, X = map(int, input().split())
m = []
for i in range(N):
    m.append(int(input()))
print(N + (X - sum(m)) // min(m))
```

C: Half and Half

まず、AB ピザを奇数枚買って 1 枚余らせるのは無意味なので、AB ピザは 2 枚 1 組で考えます。つまり、$2C$ 円で A ピザ 1 枚と B ピザ 1 枚を買えると考えることにし、この 2 枚の組み合わせをこれ以降 AB セットと呼ぶことにします。

この問題の重要な制約は、$X, Y \leq 10^5$ です。これは、A ピザ、B ピザ、AB セットのどれについても、買う個数は 10 万個以下でよいことを意味します。したがって、この三種類のうちどれか一種類を選んで、それを買う個数を 0 から 10 万まですべて試す、という方針が考えられます。[*4]

さて、三つのうちどれについて購入個数を全列挙するべきでしょうか？結論を述べると、AB セットです。AB セットを i 個 $(0 \leq i \leq 10^5)$ 購入した場合、$i \geq X$ であれば A ピザを買い増す必要はなく、$i < X$ であれば A ピザを $X - i$ 枚買い増す必要があります。これらをまとめて、買い増すべき A ピザの数は $\max(0, X - i)$ 枚であるということもできます。同様に、B ピザを $\max(0, Y - i)$ 枚買い増す必要があります。以上から、AB セットを i 枚購入した場合の所要金額は $i \times 2C + \max(0, X - i) \times A + \max(0, Y - i) \times B$ 円であり、この値を 0 以上 10 万以下のすべての整数 i について計算して最小値を取ることで答えが求まります。

なお、定数時間で答えを求めることもできます。練習問題として解説は省きます。

[*4] なお、二種類を選んでしまうと考慮する可能性の数が約 100 億通りになり、2018 年現在の一般的な計算機が 2 秒で探索するには厳しいです。今日の一般的な計算機が 1 秒あたりに処理できる式の数はおよそ 1 億個といったところで、短い単純な式であれば 10 億個程度まで伸び、逆に実数の割り算など複雑な演算を伴う場合は数千万個程度に落ちます。

D: Static Sushi

中橋君の靴にペンキが塗られていて、通ったところの床にペンキが塗られると想像してください。最終的に、ペンキが塗られた区間に含まれる寿司をすべて食べることになります。

初期位置を O とし、退店するときにペンキが塗られている区間を円弧 AB とします（O から時計回りに進んだときに先に到着する方の円弧の端を A とします。なお、円周全体が塗られる場合は最適でないため考えません)。点 A と B の片方もしくは両方（即座に退店したとき）が O と一致する場合もあります。すると、最適な歩き方は、「A まで時計回りに歩き、方向転換して反時計回りに B まで歩いて退店」もしくは「B まで反時計回りに歩き、時計回りに A まで歩いて退店」となります。

前者での移動距離は $OA + AB = 2OA + OB$、後者での移動距離は同様に $OA + 2OB$ となります。A と B を定めたとき、$OA \leq OB$ のとき前者、$OA \geq OB$ のとき後者の歩き方をするのが最適となりますが、実はそれを意識する必要はあまりなく、前者の歩き方をすると決めたときの最適解と後者の歩き方での最適解を別々に求め、よりよい方を答えとするのが単純です。以下では、後者の歩き方での最適解を求めます。前者の場合も同様に求められます。

解を求めるには、もう一つの観察が必要です: A, B はどちらも、寿司のある位置か初期位置のいずれかであるべきです。そうでなければ、含まれる寿司を減らすことなく円弧を縮められるためです。よって、$N \leq 100$ の部分点のためのテストセットは、A, B の位置をそれぞれ全探索することで解けます。愚直に実装すれば $\mathrm{O}(N^3)$ 時間、$v_1, v_1 + v_2, ..., v_1 + v_2 + ... + v_N$ や $v_N, v_N + v_{N-1}, ..., v_N + v_{N-1} + ... + v_1$ を事前に計算しておけば $\mathrm{O}(N^2)$ 時間の解法が得られます。

$N \leq 10^5$ の満点用テストセットも、これを少し改善することで解けます。A, B のうちどちらかの位置を全て試す時間はあるため、ここでは B の位置をすべて試すことにします。B の位置を x_b に固定したとき、A の位置は $x_1, x_2, ..., x_{b-1}$ のいずれかであり、このうち $f(a) := v_1 + v_2 + ... + v_a - x_a$ が最大となるような a に対応する x_a を選択するべきです（差し引きの摂取カロリーは、$f(a)$ に $v_N + v_{N-1} + ... + v_b - (C - x_b)$ を加えたものですが、この部分は a に依存せず一定値をとるためです)。よって、事前に $f(0), f(1), ..., f(N)$ の値を求めておき、さらに $g(a) := \max(f(0), f(1), ..., f(a))$ を $a = 0, 1, ..., N$ に対して計算しておけば（$g(0)$ から順に求めます)、B の位置を固定したときに直ちに最適な A の位置を選ぶことができ、$\mathrm{O}(N)$ 時間の解法が得られます。

E: Everything on It

まず、包除原理 (Wikipedia) を適用しないことには進展はないでしょう。トッピングの対称性から、答えを A として次が得られます。[*5]

$$A = \sum_{i=0}^{N} (-1)^i C(N,i) ways(i)$$

ここで $C(N,i)$ は二項係数であり、$ways(i)$ は何杯かのラーメンの組み合わせであって、トッピング $1,2,...,i$ がいずれも 1 杯以下にしか乗っていないようなものの個数です（トッピング $i+1, i+2, ..., N$ については不問)。各 $ways(i)$ を $O(N)$ 時間で求めることができれば、満点が得られます。

$j = 0, 1, .., i$ のそれぞれに対して、上の条件を満たすようなラーメンの組み合わせであって、トッピング $1,2,...,i$ のうち一つ以上が乗っているラーメンがちょうど j 杯あるようなものの個数（$ways2(i,j)$ とします）を $O(1)$ 時間で求めることができれば、それらを足し合わせることで $O(N)$ 時間で $ways(i)$ が求まります。そして、これは可能です。

このようなものを数える上で最も重要な「部品」は、トッピング $1,2,...,i$ がどのように分割されるか、すなわち、「i 個の区別可能な物体から一部を選んで j 組の空でないグループを作る方法の数（同じ物体は複数組に属せないが、ゼロ組に属することは可能)」です（下記の図を参照してください)。これは、第二種スターリング数 (Wikipedia) に非常に近い概念で、それと同様に動的計画法による $O(N^2)$ 時間の事前計算により必要なすべての値を求めることができます。ここで、i 個のトッピングのうち何個が実際にラーメンに乗っているか、はさほど重要でないことに注意してください（この数で場合分けをしてしまうと、解法の所要時間が $O(N^3)$ になってしまいます)。

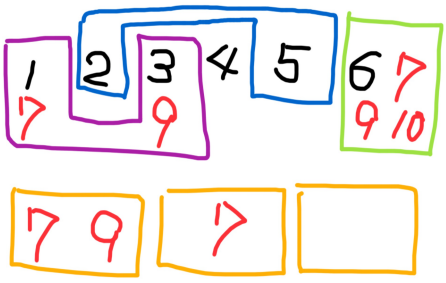


図: $N = 10$ のとき $ways2(6,3)$ に数えられるラーメンの組み合わせの例（各枠がラーメン一杯に対応）

これに、残りのトッピング $i+1, i+2, ..., N$ を考慮して $2^{(N-i)j}$ や $2^{2^{N-i}}$ を掛け合わせることで $ways2(i,j)$ を定数時間で求めることができ（上記の他にも事前計算が必要です)、目標が達成されます。

[*5] この文と次の式の意味がよく分からない方は、$N = 3$ や 4 のケースで包除原理をそのまま適用した結果を観察してみてください。ここでの「対称性」という単語の意味は「交換可能」といったところです。

F: Sweet Alchemy

$d_i = c_i - c_{p_i}$ とします。例外として $i = 1$ (根) のとき $d_i = c_i$ とします。
これを用いると、次のように言い換えられます：

- 各 $i > 1$ について $0 \leq d_i \leq D$ が成り立つ。d_1 は任意の非負整数である。
- 各 i について以下を d_i 回行う: i を根とする部分木内の各頂点 j に対し、ドーナツ j を一個作る。
- 上の条件の下でドーナツの個数を最大化せよ。

これはさらにナップザック問題に言い換えられます。頂点 v を根とする部分木内の頂点を $w_1, \cdots, w_k$ とすると、この部分木に対する操作を行うことは $Y_v := k$ 個のドーナツを $X_v := m_{w_1} + \cdots + m_{w_k}$ グラムの素を使って作ることと同値です。これを重さ X_v 価値 Y_v のアイテムとみなします。
以下を解きたいです：

- N 種類のアイテムがある。
- i 種類めの重さは X_i 価値は Y_i である。
- 重さの合計が X 以下、種類 i を選ぶ個数は Z_i 以下となるようにするとき、価値の合計の最大値を求めよ。
- Constraints: $N \leq 50, Y_i \leq 50$. 他の値は大きい。

直感的には、効率の良いものから greedy にとるべきです。アイテムが効率の良い順にソートされているとします ($Y_1/X_1 \geq Y_2/X_2 \geq \cdots$)
これを厳密に述べると、二種類のアイテム p, q $(p < q)$ を考えます。アイテム p が 50 個以上選ばれずに残り、またアイテム q が 50 個以上選ばれずに残っているとします。このとき、アイテム q を Y_p 個捨てて、アイテム p を Y_q 個新たに選ぶことで、価値の総和を増やすことができます。したがって、このような組 p, q は存在しないと仮定することができます。
これにより、次が示されます：二つの袋があり、各 i に対して、アイテム i のうち $min(50, Z_i)$ 個を一つ目の袋に入れ、残りのアイテム i を二つ目の袋に入れます。このとき、最適解においては、一つ目の袋に入っているアイテムから一部を選び、二つ目の袋からはアイテムを単に greedy に（効率の良い順に）選ぶべきです。
あとは簡単です。一つ目の袋に入っているすべてのアイテムの価値の総和は N^3 以下です。各 $0 \leq Y \leq N^3$ に対し、一つ目の袋から選んだアイテムの価値の総和が Y であると仮定して、それらのアイテムの重さの総和を最小化します（これは $O(N^4 \log N)$ 時間の単純な DP で可能です）。そして、二つ目の袋からさらにアイテムを貪欲に（重さの上限に達するまで）選んでいきます。
この解法は $O(N^4 \log N)$ 時間で動作します。

ABC 095 / ARC 096 Editorial (English)

evima

April 21, 2018

A: Something on It

To solve this problem, you need to go through the procedure below:

0. (Unnecessary for some languages) Declare a string variable S.
1. Receive S as a string from Standard Input.
2. Obtain the answer from S in some way.
3. Print the found value to Standard Output.

For steps 0, 1 and 3, the sample code for each language for Problem A in practice contest would be helpful (can be actually used after modifying a bit). For step 2, you would need to check each of the three characters in S using if statements, or count the number of `o` in S using some feature in your language. The former approach in C++ and the latter approach in Python(3) are shown below:

```cpp
#include <iostream>
using namespace std;
int main(){
    string S;
    cin >> S;
    int ans = 700;
    if(S[0] == 'o') ans += 100;
    if(S[1] == 'o') ans += 100;
    if(S[2] == 'o') ans += 100;
    cout << ans << endl;
}
```

```python
print(700 + 100 * input().count('o'))
```

B: Bitter Alchemy

As implied in the notes for the sample inputs, the following strategy works fine: "first make one doughnut for each kind, then make as many 'cheapest' doughnuts as possible". That is, the answer is $N + \lfloor (X - S)/M \rfloor$ [*1], where S is the sum of $m_1, m_2, ..., m_N$, and M is the minimum value among $m_1, m_2, ..., m_N$. The main challenge is to read the values of $m_1, m_2, ..., m_N$, and actually find the values of S and M. For the "division and rounding down" part, $\lfloor A/B \rfloor$ can be found by just writing `A / B` in many languages [*2].

The situation may be different for different languages. In languages such as C++ and Java, it would be straightforward to use loop structures. A sample implementation in C++ is shown below:

```cpp
#include <iostream>
using namespace std;

int main(){
    int N, X, m[101];
    cin >> N >> X;
    int S = 0, M = 1001001001;
    for(int i = 0; i < N; ++i){
        cin >> m[i];
        S += m[i];
        if(m[i] < M){
            M = m[i];
        }
    }
    int ans = N + (X - S) / M;
    cout << ans << endl;
}
```

The above approach is also possible in languages such as Python and Ruby, but in these languages there are occasionally built-in functions such as `sum` and `max`, which are convenient. [*3] A sample implementation in Python(3) using these functions is shown in the next page:

[*1] $\lfloor x \rfloor$ represents the integer part of x.

[*2] Not necessarily true when A or B is not positive. Also, Python3, which is used here for the sample implementation, is an exception of this statement.

[*3] Because of these functionalities, script languages are very useful in quickly implementing the solution of problems of difficulties suited for AtCoder Beginner Contest. However, for harder problems, there are suddenly many situations where these languages are simply too slow. I'm letting you know that, if you intend to do competitive programming seriously, you would need to learn one language that can rival Java for speed, if not C++.

```python
N, X = map(int, input().split())
m = []
for i in range(N):
    m.append(int(input()))
print(N + (X - sum(m)) // min(m))
```

C: Half and Half

First, since buying an odd number of AB-pizzas and having one leftover is nonsense, we will always buy these in pairs. That is, we will assume that we can buy one A-pizza and one B-pizza for $2C$ yen, and we will call this set an AB-set.

The important constraint in this problem is $X, Y \leq 10^5$. This means that we need to buy at most 100000 A-pizzas, at most 100000 B-pizzas and at most 100000 AB-sets. Thus, we can select one of these three menus and try every possible number of that menu purchased from 0 to 100000. [*4]

Which among the three should we focus on? Our conclusion is AB-sets. When we buy i AB-sets ($0 \leq i \leq 10^5$), we don't need to buy extra A-pizzas if $i \geq X$, and we need to buy $X - i$ extra A-pizzas if $i < X$. In other words, we should buy $\max(0, X - i)$ extra A-pizzas. Similarly, we should buy $\max(0, Y - i)$ extra B-pizzas. Therefore, we need $i \times 2C + \max(0, X - i) \times A + \max(0, Y - i) \times B$ yen if we buy i AB-sets, and the answer can be found by computing this value for every integer i from 0 to 100000, and taking the minimum obtained value.

A constant time solution is also possible, which is left as an exercise.

[*4] If we select two, there are about 10 billion possibilities to consider, which is too much for an ordinary computer today to enumerate in 2 seconds. A rough guess of the number of formula that an ordinary computer today can process in 1 second is about 100 million. This number can grow up to about 1 billion if the formula is a short, simple one. On the other hand, it can decline down to about several ten millions if the formula involves complex operation such as division of real numbers.

D: Static Sushi

Imagine that his shoes are wet with paint, and he paints the floor where he walks. In the end, he eats all sushi in the painted segment.

Let the initial place be O, and the painted segment in the end be arc AB (If we walk clockwise from O, we reach A first. We ignore the case where the whole counter is painted, which is not optimal). Note that A and/or B may coincide with O. The optimal way to walk is to "walk clockwise until A, then turn around and walk counterclockwise until B", or "walk counterclockwise until B, then turn around and walk clockwise until A".

The distance covered in the first way of walking is $OA + AB = 2OA + OB$, and $OA + 2OB$ in the second way. When A and B is fixed, it is optimal to perform the first way if $OA \geq OB$, and the second way if $OA \geq OB$, but we actually don't have to care this too much. We just need to find the optimal solution when we stick to the first way, and the optimal solution when we stick to the second way, and take the better one. We will deal with the second way from now on (the first way can be dealt similarly).

One more observation is required to find the solution, both A and B must be a position where a sushi is placed or O. This is because the arc could be shortened without losing sushi otherwise. Thus, the partial test set can be solved with trying every possible choice of A and B. This results in an $\mathrm{O}(N^3)$ time solution if implemented naively, and $\mathrm{O}(N^2)$ if $v_1, v_1+v_2, ..., v_1+v_2+...+v_N$ and $v_N, v_N+v_{N-1}, ..., v_N+v_{N-1}+...+v_1$ are precomputed.

The full test set can be solved with just a little improvement from here. There is enough time to try every choice of A or B, and we will go with B here. When B is fixed as x_b, A is one of $x_1, x_2, ..., x_{b-1}$, and we should select x_a corresponding to a that maximizes $f(a) := v_1 + v_2 + ... + v_a - x_a$. (This is because, the calories taken in is equal to f(a) plus $v_N + v_{N-1} + ... + v_b - (C - x_b)$, which has a constant value independent of a when b is fixed). Thus, if we find $f(0), f(1), ..., f(N)$ beforehand, and find $g(a) := \max(f(0), f(1), ..., f(a))$ for each $a = 0, 1, ..., N$ (starting from $g(0)$), we can immediately make the optimal choice of A when B is fixed, thus we have an $\mathrm{O}(N)$ time solution.

E: Everything on It

We have no choice but to apply inclusion-exclusion principle (Wikipedia). Let A be the answer. From the symmetry of the toppings, the following holds:[*5]

$$A = \sum_{i=0}^{N} (-1)^i C(N, i) ways(i)$$

Here, $C(n, i)$ is binomial coefficients, and $ways(i)$ is the number of the sets of some ramen such that each of the toppings $1, 2, ..., i$ is on at most one of those ramen (we don't care toppings $i+1, i+2, ..., N$). If we can find each $ways(i)$ in $O(N)$ time, we can obtain the full score.

For each $j = 0, 1, .., i$, if we can find the number of those sets of ramen that satisfy the condition above such that there are exactly j ramen topped with one or more toppings among toppings $1, 2, ..., i$ (let this number be $ways2(i, j)$) in $O(1)$ time, we can find $ways(i)$ in $O(N)$ by summing them up. This is indeed possible.

The most important "part" for counting these entities is, how the toppings $1, 2, ..., i$ are divided, that is, "the number of ways to make j non-empty groups out of i distinguishable objects (an object can't belong to multiple groups, but can belong to zero group)". This is a very similar concept to Stirling numbers of the second kind (Wikipedia), and all the required values can be computed in $O(N^2)$ time beforehand by dynamic programming. Note that it does not really matter how many of the i toppings are actually used in some ramen (if we classify the sets according to this number, the solution would take $O(N^3)$ time).

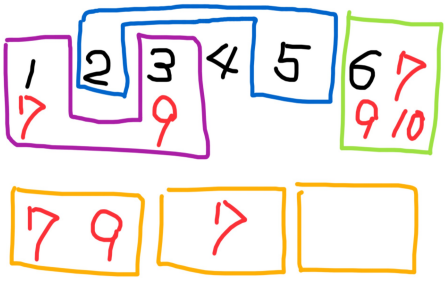


Figure: One example of a set of ramen counted in $ways2(6, 3)$ where $N = 10$ (Each frame corresponds to a ramen)

We then multiply this by $2^{(N-i)j}$ and $2^{2^{N-i}}$, consider the remaining toppings $i+1, i+2, ..., N$, and we can find $ways2(i, j)$ in a constant time (other pre-computations are also required), which was our goal.

[*5] If you are not sure of this sentence and the following formula, please try observing the direct application of inclusion-exclusion principle to the case $N = 3$ or 4. The meaning of the word "symmetry" here is "interchangable".

F: Sweet Alchemy

Let $d_i = c_i - c_{p_i}$. As an exception, when $i = 1$ (the root of the tree), we define $d_i = c_i$.

Using this, the problem can be restated as follows:

- For each $i > 1$, $0 \leq d_i \leq D$ must hold. d_1 can be an arbitrary non-negative integer.
- For each i, we do the following d_i times: for each node j in the subtree rooted at j, make one Doughnut j.
- Under the conditions above, compute the maximum number of Doughnuts you can make.

This can be further restated as a knapsack problem. Consider a node v, and let $w_1, \cdots, w_k$ be the nodes in the subtree rooted at v. Then, performing an operation for this subtree is equivalent to making $Y_v := k$ doughnuts using $X_v := m_{w_1} + \cdots + m_{w_k}$ grams of Moto. Regard it as an item with weight X_v and value Y_v.

Now we want to solve the following problem:

- There are N types of items.
- The weight of an i-th type of item is X_i. Its value is Y_i.
- You want to choose some items such that the total weight is at most X. You can choose at most Z_i items of type i. What is the maximum total value you can achieve?
- Constraints: $N \leq 50, Y_i \leq 50$. Other parameters can be very large.

Intuitively, we should choose the most "efficient" ones greedily. For simplicity, assume that the items are sorted in the decreasing order of efficiency: that is, $Y_1/X_1 \geq Y_2/X_2 \geq \cdots$.

Now, we formally state the intuition. Consider two types of items p, q ($p < q$). Assume that there are at least 50 unchosen items of type p and at least 50 chosen items of type q. Then, we can improve the value by taking Y_q itmes of type p and discarding Y_p items of type q. Thus, we can assume that there are no such pairs of p, q.

This proves the following: We have two bags of items. For each i, we put $min(50, Z_i)$ items of type i to the first bag, and the remaining items of type i to the second bag. In the optimal solution, we should choose some subset of items from the first bag, and from the second bag we just choose items greedily (by efficiency).

The remaining part is easy. The total value of all items in the first bag is at most N^3. For each $0 \leq Y \leq N^3$, assume that the total value of all items you choose from the first bag is Y, and minimize their total weight (this can be done by a simple DP in $O(N^4 \log N)$ time). Then, we choose more items from the second bag greedily (until we reach the weight capacity).

This solution works in $O(N^4 \log N)$ time.

AtCoder Beginner Contest 094 / AtCoder Regular Contest 095 解説

writer: semiexp, nuip

2018 年 4 月 14 日

For International Readers: English editorial starts on page ?.

A: Cats and Dogs

猫の数は，B 匹すべてが犬のときに最小 (A 匹) になります．一方，B 匹すべてが猫のときに最大 ($A+B$ 匹) になります．

よって，X が A 以上 $A+B$ 以下かどうか判定すればよいです．

- C++ による解答例: `https://abc094.contest.atcoder.jp/submissions/2343785`
- Python による解答例: `https://abc094.contest.atcoder.jp/submissions/2344330`

B: Toll gates

ゴールするまでに，元いたマスへ引き返すことをする必要はありません．すなわち，マス X からマス $X-1, X-2, \ldots, 0$ と進むか，$X+1, X+2, \ldots, N$ と進むかのいずれかです．この 2 パターンそれぞれについて，通る料金所の個数を数え，最小値を求めればよいです．

- C++ による解答例: `https://abc094.contest.atcoder.jp/submissions/2343740`
- Python による解答例: `https://abc094.contest.atcoder.jp/submissions/2344333`

C: Many Medians

入力をソートしたものを $Y_1 \leq Y_2 \leq \cdots \leq Y_N$ とします．このとき B_i を考えます．ソートした後に A_i が l_i 番目に現れるとします（すなわち $Y_{l_i} = A_i$ が成り立ちます）．B_i は $Y_1, \ldots, Y_{l_i-1}, Y_{l_i+1}, \ldots, Y_N$ の中央値であることに注意します．すると，$Y_1 \leq \cdots \leq Y_{l_i-1} \leq Y_{l_i+1} \leq \cdots \leq Y_N$ より，この $N-1$ 個の値のうち $\frac{N}{2}$ 番目の値を求めればよいことがわかります．これは，$l_i \leq \frac{N}{2}$ であれば $Y_{N/2+1}$，$l_i \geq \frac{N}{2}+1$ であれば $Y_{N/2}$ になることが確かめられます．

D : Binomial Coefficients

$\mathrm{comb}(n, r)$ を並べた表はパスカルの三角形 (https://ja.wikipedia.org/wiki/と呼ばれています。これを観察すると下に行くほど（n が大きくなると）値が大きくなることが分かります。また、同じ段なら真ん中よりのほうが値が大きくなることが分かります。このことから、$a_1, a_2, \ldots, a_n$ のうち最大のものを n として使い、n 以外で $n/2$ に最も近いものを r として使うと最大になることが予想できます。

この予想が正しいことを示します。次の事実を使います。

$$\mathrm{comb}(n, r) = \frac{n(n-1)(n-2)\ldots(n-r+1)}{r(r-1)(r-2)\ldots 1}$$

まず、最初の予想が正しいことを確かめるために次を示します。

主張. $r > 0$ のとき、$\mathrm{comb}(n+1, r) > \mathrm{comb}(n, r)$

証明. $r = 1$ のときは明らか。$r > 1$ とする。

$$\begin{aligned}
&\mathrm{comb}(n+1, r) - \mathrm{comb}(n, r) \\
=&\frac{(n+1)n(n-1)\ldots(n-r+2)}{r(r-1)(r-2)\ldots 1} - \frac{n(n-1)(n-2)\ldots(n-r+1)}{r(r-1)(r-2)\ldots 1} \\
=&\frac{n(n-1)\ldots(n-r+2)}{r(r-1)(r-2)\ldots 1}\left((n+1) - (n-r+1)\right) \\
=&\frac{n(n-1)\ldots(n-r+2)}{r(r-1)(r-2)\ldots 1} r > 0
\end{aligned}$$

□

次に、二番目の予想が正しいことを示すのには、n 個から r 個選ぶのと n 個から選ばない $n-r$ 個を決めるのが同じであることを踏まえると、次を確かめれば十分です。

$$r + 1 \leq n/2 \text{ のとき、} \mathrm{comb}(n, r+1) > \mathrm{comb}(n, r)$$

これは $\frac{\mathrm{comb}(n,r+1)}{\mathrm{comb}(n,r)}$ を計算すれば分かります。

E: Symmetric Grid

行の入れ替えの操作と，列の入れ替えの操作は，交換可能であることに注意します．よって，最初に何回か行の入れ替えの操作のみを行った後，何回か列の入れ替えの操作のみを行うとしてもかまいません．

まず，行の入れ替えの方法を固定して，その後列の入れ替え操作のみでマス目を点対称的にできるかどうか判定することを考えます．$j = 1, 2, \ldots, W$ に対して，点対称的なマス目の j 列目は，$W + 1 - j$ 列目を上下反転したものになっていることに注意します．特に，W が奇数のとき，$(W + 1)/2$ 列目は上下対称になります．逆に，この条件が成り立てば，マス目は点対称的になります．

よって，これは次の方法で判定することができます．(実際の実装では，具体的に点対称的にする方法を構成する必要はありません）

- はじめ，すべての列を未使用としておく．盤面も，すべての列を取り出して，空にしておく．
- 未使用の列がある場合，それを上下反転したような未使用の他の列が存在するか判定する．
  - もし存在すれば，その 2 つの列を使用済みにし，まだ埋まっていない最も左，最も右の列にそれぞれ配置する．
  - 存在しない場合，もし W が偶数ならば，点対称的にすることは不可能である．W が奇数ならば，その列が上下対称か判定し，対称ならばその列を $(W + 1)/2$ 列目に置く．すでに $(W + 1)/2$ 列目が埋まっているか，対称でないならば，点対称的にすることは不可能である．

このアルゴリズムは $O(HW^2)$ で動作します.（列をソートするなどの方法で，$O(HW \log W)$ にすることもできます）

次に，行の入れ替えの方法を考えます．単純に考えると，行の入れ替えの方法は $H!$ 通りあり，大きすぎます．ここで，$1, H$ 行目，$2, H-1$ 行目，... のように行同士をペアにして考えると (H が奇数の場合，ペアにならない行が 1 行だけ存在します)，ペア関係が同一な入れ替え方法では，点対称的にできるかどうかは全く同じになることがわかります．なので，行同士をペアにする異なる方法をすべて試し，それぞれの方法に対しては適当に行の入れ替えをしてから，先の列の入れ替えによる判定を行うことで，この問題を解くことができます．ペアにする方法は，最大で $11!! = 11 \times 9 \times 7 \times 5 \times 3 \times 1 = 10395$ 通りしかないので，十分この問題を解くことができます．

F : Permutation Tree

高橋くんの能力は言い換えると次のようになります。
max を -1 として、p_i の小さい順に次の処理をする。

- $p_i \neq 1$ のとき、max と i を辺でつなぐ
- $i >$ max であれば、 max を i で書き換える

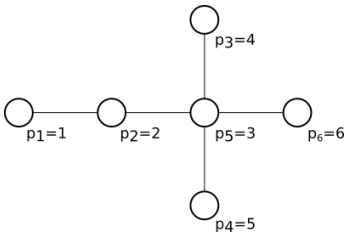
つまり、今までに出てきた添字のうちの最大のところに辺を貼っていく操作になります。添字の最大が更新された頂点だけに注目すると、これらの頂点は p_i の小さい順に頂点を並べたパスをなしていることが分かります。一方で、最大が更新されていない頂点は、このパスに含まれる頂点と直接つながることが分かります。このようなグラフは caterpillar(`https://en.wikipedia.org/wiki/Caterpillar_tree`) と呼ばれています。先程言及したパスのことを今後胴体と呼び、胴体に含まれていない頂点のことを脚と呼ぶことにします。
以上で、高橋くんの能力で作れるグラフは必ず caterpillar であることが分かりました。実は caterpillar であれば必ずその木を作れるような順列が存在します。

caterpillar 判定

与えられた木が caterpillar であるかどうかを判定する方法は次のとおりです。まず、木の直径を求めます (`http://www.prefield.com/algorithm/graph/tree_diameter.html`)。与えられた木が caterpillar である場合、この木の直径を胴体とみなすことができます。これは脚の長さが 1 であることから明らかです。胴体が求まったので、あとは残りの頂点が直接胴体に繋がれているか判定するだけです。

順列の生成

次に、与えられた caterpillar を作る順列の作り方を考えます。胴体が左右に並んでいて、左から右に添字も値も増加しているとします。胴体の頂点の添字を左から $b_1, \ldots, b_l$ とします。



胴体のうちのある頂点 b_i に注目します。この頂点から脚が k_i 本生えているとき、その k_i 個の頂点は頂点 b_i よりも値が大きくて、添字が小さい必要があります。この条件の元で辞書順で小さくなるよう気をつけると、次のようにしてできる順列が答えの候補になります。

$(1, 2, \ldots, n)$ という順列から始める。
各 $i = 1, 2, \ldots, l$ について、左から $\sum_{j=0}^{i-1}(k_i + 1)$ 番目の数を k_i こ分右にずらす。

caterpillar のとり方と答え

与えられた木を caterpillar とみなす時、胴体にあたるパスの取り方は複数通りありえます。上記の caterpillar の構築方法をふまえると、胴体はできるだけ長くなるように取った方がいいことが分かります。胴体の向きが 2 通りありますが、これは両方試して辞書順で小さい順列が得られたほうを答えとすれば良いです。

AtCoder Beginner Contest 094 / AtCoder Regular Contest 095 Editorial

writer: semiexp, nuip

April 14th, 2018

A: Cats and Dogs

- C++ example: `https://abc094.contest.atcoder.jp/submissions/2343785`
- Python example: `https://abc094.contest.atcoder.jp/submissions/2344330`

B: Toll gates

- C++ example: `https://abc094.contest.atcoder.jp/submissions/2343740`
- Python example: `https://abc094.contest.atcoder.jp/submissions/2344333`

C: Many Medians

Let $Y_1 \leq Y_2 \leq \cdots \leq Y_N$ be the sorted list of X. Suppose that $Y_{l_i} = A_i$. B_i is the median of $Y_1, \ldots, Y_{l_i-1}, Y_{l_i+1}, \ldots, Y_N$. Thus, B_i is the $\frac{N}{2}$-th value of $Y_1, \ldots, Y_{l_i-1}, Y_{l_i+1}, \ldots, Y_N$. This is $Y_{N/2+1}$ if $l_i \leq \frac{N}{2}$, and $Y_{N/2}$ if $l_i \geq \frac{N}{2} + 1$.

D : Binomial Coefficients

We shuold choose the maximum of $a_1, a_2, \ldots, a_n$ as n because for a fixed r, the function $comb(n, r)$ is monotonously increasing. This follows from the following fact:

$$\text{If } r > 0,\ \mathrm{comb}(n+1, r) > \mathrm{comb}(n, r)$$

(because $\mathrm{comb}(n+1, r) = \mathrm{comb}(n, r) + \mathrm{comb}(n, r-1)$)
Once we choose n, we should choose r that is closest to $n/2$. This follows from the following fact:

$$\text{If } r + 1 \leq n/2,\ \mathrm{comb}(n, r+1) > \mathrm{comb}(n, r)$$

Compute $\frac{\mathrm{comb}(n,r+1)}{\mathrm{comb}(n,r)}$ to prove this.

E: Symmetric Grid

Notice that an operation about rows and an operation about columns are commutative. Thus, assume that we first perform operations about rows, and then operations about columns.

Suppose that we finished operations about rows. How can we check if we can achieve a goal by operations about columns? Notice that, for each $j = 1, 2, \ldots, W$, the j-th column must be the reverse of the $W + 1 - j$-th column. In particular, when W is odd, $(W + 1)/2$ is a palindrome.

Therefore, we can use the following method do check if we can achieve a goal by operations about columns:

- Initially, all rows are "unused".
- Choose a particular unused row. Reverse it, and check if it matches with another unused row.
  - If it matches with another row, mark the two rows as "used".
  - Otherwise, if W is even, the answer is "NO". If W is odd and the current row is not a palindrome, the answer is "NO". If W is odd and the current row is a palindrome, we should put it in the $(W + 1)/2$-th row. If this row is already filled, the answer is "NO".

It works in $O(HW^2)$ time (or by binary search you can do it in $O(HW \log W)$)

If we consider all $H!$ possible orders of rows, it will be too slow. Instead, notice that only the set of pairs ($(1, H)$-th rows, $(2, H - 1)$-th rows, ...) matters. (In case H is odd, one row is left unpaired.)

Thus, we only need to try all pairings of rows. Since there are at most $11!! = 11 \times 9 \times 7 \times 5 \times 3 \times 1 = 10395$ ways of pairings, it's fast enough.

F : Permutation Tree

We can restate the generation of a tree as follows:

Let max $= -1$. Initially, all vertices are painted black. Perform the following operation in the increasing order of p_i (that is, first we choose an i such that $p_i = 1$ and perform the operation, and so on):

- If $p_i \neq 1$, add an edge between vertex max and vertex i.
- If $i >$ max, replace max with i, and color vertex i red.

It's clear that when you add an edge, it is always between vertex i (the current vertex) and the most recently painted red vertex. Also, when we paint a vertex, it is always the current vertex.

Thus,

- The red vertices will form a path.
- A black vertex is a leaf, and it's incident to a red vertex.

Such graphs are called caterpillar(`https://en.wikipedia.org/wiki/Caterpillar_tree`). From now on, we call red vertices "body", and black vertices "feet". Also, we call one end of the body "head", and the other end "tail". The head corresponds to the vertex with $p_i = 1$.

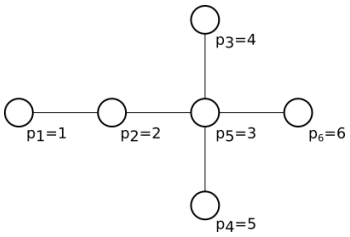
Therefore, if the given tree is not a caterpillar, we should print "-1". Otherwise, it turns out that the solution always exists, as we describe later.

Check if the given tree is a caterpillar

Find the diameter of the tree. The tree is a caterpillar if and only if it forms the body (i.e., all vertices are either in the diameter or adjacent to one vertex in the diameter).

Find the lexicographically smallest permutation that generates a given caterpillar

Let's fix a caterpillar. How can we find a permutation that generates this caterpillar? Let $b_1, \ldots, b_l$ be the vertices in the body in the order from head to tail. Let k_i be the number of feet that grow from b_i.



For example, in the picture above, $\{k_i\} = \{0, 0, 2, 0\}$.

For each foot, the index of its parent must be greater than its index. The value of its parent must be less than its value. The lexicographically smallest permutation that satisfies these conditions is the following permutation:

> Consider a permutation $(1, 2, \ldots, n)$. For each $i = 1, 2, \ldots, l$ in this order, do the following: move the $\sum_{j=0}^{i-1}(k_i + 1)$-th (from left, 1-indexed) element to the right by k_i positions.

For example, we get $1, 2, 4, 5, 3, 6$ for the example above.

It's easy to see that this permutation generates a desired caterpillar.

How to choose a head and a tail of a caterpillar

There are multiple ways to choose a head and a tail of a caterpillar. From the construction above, it's clear that we shuold maximize the length of the body, so we should choose a diameter and the head and the tail should be two ends of the diameter. There are two ways to orient the body: we should try both ways.

ARC 094/ABC 093 解説

DEGwer

2018/04/07

A: abc of ABC

3 つの文字がどの 2 つも相異なるかどうかを調べればよいです。

```
#include<stdio.h>
int main()
{
        char s[10];
        scanf("%s", s);
        if (s[0] != s[1] && s[1] != s[2] && s[2] != s[0]) printf("Yes\n");
        else printf("No\n");
}
```

B: Small and Large Integers

出力すべき範囲を適切に記述し、その範囲の整数をすべて出力すればよいです。A 以上 B 以下の整数が $2K$ 個未満の場合に注意しましょう。

```
#include<stdio.h>
#include<vector>
#include<algorithm>
using namespace std;
int main()
{
        int a, b, k;
        scanf("%d%d%d", &a, &b, &k);
        for (int i = a; i <= min(b, a + k - 1); i++)printf("%d\n", i);
        for (int i = max(b - k + 1, a + k); i <= b; i++)printf("%d\n", i);
}
```

C: Same Integers

3 つの整数の和の偶奇は操作によって不変です。また、整数を減少させることはできないので、3 つの整数が等しくなったならば、その時の値は 3 つの整数の最大値 (以下これを M とする) 以上です。よって、もし $3M$ と最初の 3 つの整数の和の偶奇が等しいなら、3 つの整数が等しくなった時の値は M 以上、そうでなければ $M+1$ 以上です。

逆に、この下限が達成できることは簡単にわかるので、この問題を解くことができました。

D: Wide Flip

1 回目のコンテストで A 位、2 回目のコンテストで B 位を取ることを (A, B) を取ると呼ぶことにします。

高橋君が (A, B) を取ったとします。$A \leq B$ として一般性を失いません。

$A = B$ の場合、高橋君よりスコアが小さい人は 1 回目のコンテストまたは 2 回目のコンテストで $A-1$ 位以内の順位を取っています。よって、そのような人の人数は $2A-2$ 人以下です。逆に、$(1, 2A-1), \cdots, (A-1, A+1), (A+1, A-1), \cdots, (2A-1, 1)$ を取った人が存在する場合、その上限値を達成できます。

$A+1 = B$ の場合、高橋君よりスコアが小さい人は 1 回目のコンテストで $A-1$ 位以内を取るか、2 回目のコンテストで A 位以内を取っています。また、2 回目のコンテスト A 位を取った人のスコアが高橋君より小さくなるためには、その人は 1 回目のコンテストで $A-1$ 位以内を取る必要があります。よって、高橋君よりスコアが小さい人の人数は $2A-2$ 人以下です。逆に、$(1, 2A), \cdots, (A-1, A+2), (A+2, A-1), \cdots, (2A, 1)$ を取った人が存在する場合、その上限値を達成できます。

そうでない場合、$C^2 < AB$ を満たす最大の整数 C を取ります。

$C(C+1) \geq AB$ のとき、高橋君よりスコアが小さい人は 1 回目のコンテストまたは 2 回目のコンテスト

で C 位以内の順位を取っています。1 回目のコンテストで C 位を取った人のスコアが高橋君のスコアより小さくなるためには、その人は 2 回目のコンテストで C 位以内を取らなければなりません。よって、高橋君よりスコアが小さい人の人数は $2C-2$ 人以下です。逆に、$(1, A+B-1), \cdots, (A-1, B+1), (A+1, 2C-A-1), \cdots, (C, C), \cdots, (2C-1, 1)$ を取った人が存在する場合、その上限値を達成できます。

$C^2 < AB$ のとき、高橋君よりスコアが小さい人は 1 回目のコンテストまたは 2 回目のコンテストで C 位以内の順位を取っています。よって、高橋君よりスコアが小さい人の人数は $2C-1$ 人以下です。逆に、$(1, A+B-1), \cdots, (A-1, B+1), (A+1, 2C-A), \cdots, (C, C+1), (C+1, C), \cdots, (2C, 1)$ を取った人が存在する場合、その上限値を達成できます。

以上の場合で尽くされるので、この問題を解くことができました。

E: Tozan and Gezan

とざん君を先手、げざん君を後手と呼ぶことにしましょう。もし数列 A, B が最初から等しいなら、答えは 0 です。そうでない場合、答えは以下の値になることを証明します。

- 数列 A の総和から、$A_i > B_i$ を満たす B_i の中で最小のものを引いた値

一般性を失わずに、$A_i > B_i$ を満たす i の中で B_i を最小にするものが 1 であるとして良いです。まず、先手が操作回数をこの値以上にできることを示します。

先手は以下の戦略にのっとって操作を進めればよいです: A の 1 番目以外の整数をすべて 0 になるまで減らし、そのあと A_i を減らしていく

この操作で上記の値以上の回数の操作が必要になるのは明らかです。

さて、後手が操作回数をこの値以下にできることを示しましょう。後手は以下の戦略にのっとって操作を進めればよいです: $B_i > A_i$ なる i をひとつ選び、B_i を減らす

後手がこの戦略にのっとって行動したとき、ある $A_i > B_i$ なる i が存在し、操作の終了時にも B_i の値が変わっていないことを証明できればよいです。$A_i > B_i$ である間、後手が B_i を減少させることはないので、$A_i > B_i$ であるような i がなくなる直前に先手が操作を行った i について、B_i の値が減らされていることはありません。これが操作の終了まで変化しない B_i の例となっているので、操作回数の上界と下界が一致し、この問題を解くことができました。

F: Normalization

もし S のすべての文字が等しいなら、答えは 1 です。以下、そうでない場合を考えます。

文字 A,B,C を順に整数 $0, 1, 2$ に対応させることにすると、操作によって文字列全体の対応する値の和を 3 で割ったあまりは変化しません。すなわち、文字列全体の対応する値の和を 3 で割ったあまりが S と等しい文字列のみ作ることができます。また、S に 1 回以上の操作を行って作ることのできる文字列には、必ず同じ文字が連続する箇所があります。

さて、作ることのできる可能性のある文字列は上記の条件を満たすものと S 自身に限られました。逆に、これらの文字列をすべて作ることはできるでしょうか？

実は、$|S| \geq 4$ ならこれらの文字列をすべて作ることができることが証明できます。$|S| = 4$ の場合は全通り試せば示すことができ、$|S|$ が 5 以上の場合は先頭の文字を適切に変化させることによって $|S|$ が 1 小さい

場合に帰着できます。

よって、この問題は以下のようにして解くことができます。

- S のすべての文字が等しいなら、1 を出力
- $|S| \leq 3$ なら、全探索を行う
- そうでないなら、 上記の条件を満たす文字列の個数を求め、S がその条件を満たさないならさらに 1 を足した値を答えとする

上記の条件を満たす文字列の個数は簡単な線形時間の DP で求めることができるので、この問題を線形時間で解くことができました。

ARC 094/ABC 093 Editorial

DEGwer

2018/04/07

A: abc of ABC

Check if the characters in s are pairwise distinct.

```
#include<stdio.h>
int main()
{
        char s[10];
        scanf("%s", s);
        if (s[0] != s[1] && s[1] != s[2] && s[2] != s[0]) printf("Yes\n");
        else printf("No\n");
}
```

B: Small and Large Integers

The integers you should print will be two intervals. Make sure to handle cases such that the number of integers between A and B is less than $2K$.

```
#include<stdio.h>
#include<vector>
#include<algorithm>
using namespace std;
int main()
{
        int a, b, k;
        scanf("%d%d%d", &a, &b, &k);
        for (int i = a; i <= min(b, a + k - 1); i++)printf("%d\n", i);
        for (int i = max(b - k + 1, a + k); i <= b; i++)printf("%d\n", i);
}
```

C: Same Integers

Suppose that when you finish the operations, all integers are X. Since the sum of three integers always increases by two in each operation, the total number of operations is $(3X - (A + B + C))/2$. Thus, we want to minimize X.

Let M be the maximum of A, B, C. Since we can never decrease integers, $X \geq M$ must hold. Also, since we can never change the parity of sum of three integers, $3X \equiv A + B + C \pmod 2$ must hold. (It's easy to see that these are sufficient conditions).

Therefore,

- If $3M \equiv A + B + C \pmod 2$, $X = M$.
- Otherwise, $X = M + 1$.

and we should print $(3X - (A + B + C))/2$.

D: Worst Case

Let us denote a participant who was ranked x-th in the first contest and y-th in the second contest as (x, y). Takahashi is (A, B). Without loss of generality, assume that $A \leq B$.

One possible way to solve the problem is to consider the following cases:

- If $A = B$, the answer is at most $2A - 2$ because you must get top $A - 1$ places in at least one contest to beat Takahashi. We can achieve this value by $(1, 2A-1), \cdots, (A-1, A+1), (A+1, A-1), \cdots, (2A-1, 1)$. Thus, the answer is $2A - 2$.
- If $A + 1 = B$, the answer is at most $2A - 2$ because you must get top $A - 1$ places in at least one contest to beat Takahashi. (Note that the A-th place in the first contest is filled by Takahashi, and getting the $A + 1$-th in the first and A-th in the second is not enough.) We can achieve this value by $(1, 2A), \cdots, (A - 1, A + 2), (A + 2, A - 1), \cdots, (2A, 1)$. Thus, the answer is $2A - 2$.
- Otherwise, let C be the maximum integer such that $C^2 < AB$.
  - If $C(C+1) \geq AB$, the answer is at most $2C-2$ because you must get top $C-1$ places in the first contest or top C places in the second contest to beat Takahashi. (Note that one of "the top $C - 1$ places in the first contest" is Takahashi, we shouldn't count him.) We can achieve this value by $(1, A+B-1), \cdots, (A-1, B+1), (A+1, 2C-A-1), \cdots, (C, C), \cdots, (2C-1, 1)$. Thus, the answer is $2C - 2$.
  - If $C(C+1) < AB$, the answer is at most $2C-1$ because you must get top C places in the first contest or top C places in the second contest to beat Takahashi. (Note that one of "the top C places in the first contest" is Takahashi, we shouldn't count him.) We can achieve this value by $(1, A + B - 1), \cdots, (A - 1, B + 1), (A + 1, 2C - A), \cdots, (C, C + 1), (C + 1, C), \cdots, (2C, 1)$. Thus, the answer is $2C - 1$.

Another possible solution is binary search on the answer: for a fixed X, we want to know the following:

- Consider the smallest X positive integers except for A.
- Consider the smallest X positive integers except for B, and reverse the order.
- Check if for all i, the product of the i-th integer in the first list and the i-th integer in the second list is smaller than AB.

To do this, use the fact that the lists are "piecewise" arithmetic sequences (and the number of pieces is constant).

E: Tozan and Gezan

If A, B are the same, the answer is 0. We assume that $A \neq B$.

What is a good strategy for the second player? If $A_i < B_i$ for some i, the second player needs to decrease B_i, regardless of the moves of the first player. He can do this without giving any valuable information for the first player. Thus, the second player should always choose one of such i (it always exists because the sums are the same), and decrease B_i by one.

What is a good strategy for the first player? Intuitively, when $A_i = B_i + 1$, it's not a good idea to decrease A_i by one. Let's make sure not to do this unless the first player is forced to do so.

If both players follow the rules mentioned above, when the game finishes, for all i but one (let's say $i \neq k$), $A_i = B_i = 0$ holds. B_k is unchanged from the beginning.

The first player wants to minimize B_k because the number of steps is (the sum of all elements) minus B_k. Here, $A_k > B_k$ must hold, but among such k's the first player can choose anything (by properly choosing what to do when he is forced to perform an operation for $A_i = B_i + 1$).

It turns out that the answer is the following:

- $X :=$ (The sum of all elements in A) minus (The minimum B_i such that $A_i > B_i$)

Here is a formal proof:

Without loss of generality, assume that the minimum B_i such that $A_i > B_i$ is B_1.

First, we prove that the first player can make sure that the number of steps become at least X. He follows the following strategy: first, decrease $A_2, \ldots, A_N$ to 0, and then decrease A_1. It's clear that by this strategy, the steps will be at least X.

Then, we prove that the second player can make sure that the number of steps become at most X. The second player follows the strategy mentioned above. Consider the last moment when $A_i > B_i$ holds for some i. Then, the first player decreases this A_i, and in the next step the game finishes. For this i, B_i is unchanged during the entire game. Thus, the number of steps will be at most X.

F: Normalization

When can we convert a string S into a string T? We can make the following observations:

- Let's assume the letters A, B, C as integers $0, 1, 2$. By performing the operations, the sum of all integers in the string never changes in modulo 3. Thus, if the sum of all integers in S and the sum of all integers in T are different (in modulo 3), the answer is "no".
- If all letters in S are the same, we can perform any operations. The answer is "no" unless $S = T$.
- If no two adjacent letters in T are the same, T can't be a result of an operation. Thus, the answer is "no" unless $S = T$.

Do we find all "no" cases? It's very difficult to answer this question on paper, so let's write a brute force solution. What we get is the following:

- If $|S| \leq 3$, the behavior is mysterious. From now on, assume that $|S| \geq 4$.
- If $S = T$, "yes". From now on, assume that $S \neq T$.
- If at least one of the three conditions above (the sum of S and T are different in modulo 3, all letters in S are the same, or no two adjacent letters in T are the same), "no". Otherwise, "yes".

(We can prove this by induction on $|S|$: $|S| = 4$ can be verified by brute force, otherwise we can change the first letter properly and decrease $|S|$ by one.)

Thus, we can solve the original problem as follows:

- If all letters in S are the same, print 1.
- If $|S| \leq 3$, compute the answer by brute force.
- Otherwise, count the number of strings T such that
  - $|T| = |S|$
  - $sum(S) = sum(T) \pmod 3$
  - There is at least one pair of adjacent same letters in T.

  (This can be done by a simple DP). Make sure not to forget to add one to the answer in case no two adjacent letters in T are the same.

ARC093 / ABC092 解説

wo01

2018 年 3 月 25 日

A 問題

答えは $\min(A, B) + \min(C, D)$ となります。

以下に C++ による実装例を示します。

```
#include<cstdio>
#include<algorithm>

using namespace std;

int main(){
    int A, B, C, D;
    scanf("%d%d%d%d", &A, &B, &C, &D);
    printf("%d\n", min(A, B) + min(C, D));
    return 0;
}
```

B 問題

i 人目の参加者は合宿中にチョコレートを $1 + \lfloor (D-1)/A_i \rfloor$ 個食べます。ここで、$\lfloor x \rfloor$ は x を超えない最大の整数です。

よって、答えは

$$X + \sum_{1 \leq i \leq N} (1 + \left\lfloor \frac{D-1}{A_i} \right\rfloor)$$

となります。

以下に C++ による実装例を示します。C++ を含む多くの言語で、正の整数同士の除算の際はあまりが切り捨てられます。

```
#include<cstdio>

using namespace std;

const int MAX_N = 100;

int N;
int A[MAX_N];
int D;
int X;

int main(){
    scanf("%d", &N);
    scanf("%d%d", &D, &X);
    for(int i = 0; i < N; ++i){
        scanf("%d", A + i);
    }
    int ans = X;
    for(int i = 0; i < N; ++i){
        int tmp = (D - 1) / A[i] + 1;
        ans += tmp;
    }
    printf("%d\n", ans);
    return 0;
}
```

C 問題

入力で与えられる $A_1, A_2, \ldots, A_N$ に加えて、$A_0 = A_{N+1} = 0$ としておきます。求めたいものは、各 i $(1 \leq i \leq N)$ に対する

$$S_i = |A_0 - A_1| + |A_1 - A_2| + \cdots + |A_{i-2} - A_{i-1}| + |A_{i-1} - A_{i+1}| + \\ |A_{i+1} - A_{i+2}| + \cdots + |A_N - A_{N+1}|$$

の値です。

ここで、

$$S = \sum_{0 \leq j \leq N} |A_j - A_{j+1}|$$

とします。この値は $O(N)$ 時間で求められます。

このとき、

$$S_i = S + |A_{i-1} - A_{i+1}| - (|A_{i-1} - A_i| + |A_i - A_{i+1}|)$$

となります。よって、S_i の値は各 i に対して $O(1)$ 時間で求められます。

以上から、全体で $O(N)$ 時間で解くことができます。

D 問題

条件を満たすグリッドは以下のようにして構成することができます。

- $K = 50$ として、$2K \times 2K$ のグリッドを用意する。
- 上から K 行以内にあるマスを全て黒く塗り、残りのマスをすべて白く塗る。
- 上から $K-1$ 行以内のマスのうち $A-1$ 個のマスを上下左右斜めに隣り合わないように選び、それらの色を白に変える。
- 下から $K-1$ 行以内のマスのうち $B-1$ 個のマスを上下左右斜めに隣り合わないように選び、それらの色を黒に変える。

E 問題

与えられたグラフを G とし、 G の最小全域木 T を一つ選びます。また、その重みを S とします。

T の 2 頂点 u, v に対して、$pathMax(u, v)$ を T の uv パス上の辺の重みの最大値と定義します。さらに、G に含まれるが T に含まれない辺 $e = \{u, v\}$ が重み c を持つとき、$diff(e) = c - pathMax(u, v)$ と定義します。T が最小全域木であることから、すべての e に対して $diff(e) \geq 0$ となります。

さて、G の辺を白色または黒色に塗ったとき、白く塗られた辺と黒く塗られた辺をともに含む全域木のうち最小の重みのものの重み S' は以下のようになります。

- T に白色の辺と黒色の辺がともに含まれる場合、$S' = S$
- そうでない場合、T の辺と異なる色で塗られた辺のうち最小の $diff$ の値を持つものを e として、$S' = S + diff(e)$

このことから、求めたい塗り方の個数は以下のようにして求められます。

まず、$D = X - S$ とします。さらに、$diff(e) < D$, $diff(e) = D$, $diff(e) > D$ を満たす e の個数をそれぞれ $lower$, $equal$, $upper$ とします。このとき、

- $D < 0$ のとき、塗り方の個数は 0
- $D = 0$ のとき、
  - T の辺に 2 種類の色を両方用いる場合、塗り方の個数は $(2^{N-1} - 2)2^{M-N+1}$
  - そうでない場合、塗り方の個数は $2 \times (2^{equal} - 1) \times 2^{upper}$
- $D > 0$ のとき、塗り方の個数は $2 \times (2^{equal} - 1) \times 2^{upper}$

以上で答えが求まりました。

F 問題

条件を満たす p のうち $p_1 = 1$ であるものの個数がわかれば十分です。以下、$p_1 = 1$ を仮定します。

選手 1 が優勝または敗退するまでに対戦する可能性がある選手の番号は

$$p_2, \min\{p_3, p_4\}, \ldots, \min\{p_{2^{N-1}+1}, \ldots, p_{2^N}\}$$

です。上の N 個の値がどれも集合 $\{A_1, A_2, \ldots, A_M\}$ に含まれないとき、選手 1 が優勝します。

選手 $2, 3, \ldots, 2^N$ を番号順に**場所** $p_2, p_3, \ldots, p_{2^N}$ に割り当てていくことを考えます。また、これらの場所は N 個のグループ $G_1 = \{p_2\}, G_2 = \{p_3, p_4\}, \ldots, G_N = \{p_{2^{N-1}+1}, \ldots, p_{2^N}\}$ に分かれているとします。グループ G_i が**開始済みグループ**であるとは、グループ G_i に属する選手が 1 人以上決まっていることを言うことにします。また、$\{1, 2, \ldots, N\}$ の部分集合 U に対して、

$$\mathrm{sz}(U) = \sum_{i \in U} 2^{i-1}$$

と定めます。

次のような DP を考えます。

$dp_{1,i,U}$ = 選手 $2, 3, \ldots, A_i$ の場所を決める方法であって、
開始済みグループの集合 T が $T = U$ を満たすものの個数

$dp_{2,i,U}$ = 選手 $2, 3, \ldots, A_i$ の場所を決める方法であって、
開始済みグループの集合 T が $T \subseteq U$ を満たすものの個数

$dp_{3,i,U}$ = 選手 $2, 3, \ldots, A_i - 1$ の場所を決める方法であって、
開始済みグループの集合 T が $T \subseteq U$ を満たすものの個数

$dp_{4,i,U}$ = 選手 $2, 3, \ldots, A_i - 1$ の場所を決める方法であって、
開始済みグループの集合 T が $T = U$ を満たすものの個数

この DP テーブルは以下のように計算することができます。

- 各 U に対して $dp_{1,i,U}$ が求まっているとき、各 U に対する $dp_{2,i,U}$ は高速ゼータ変換 (の変形) により求められます。
- 各 U に対して $dp_{2,i,U}$ が求まっているとき、各 U に対する $dp_{3,i+1,U}$ は、$t = \max(0, \mathrm{sz}(U) - A_i)$, $d = A_{i+1} - 1 - A_i$ として、

  $$dp_{3,i+1,U} = t \times (t-1) \times \cdots \times (t-d+1) \times dp_{1,i,U}$$

  として求められます。
- 各 U に対して $dp_{3,i,U}$ が求まっているとき、各 U に対する $dp_{4,i,U}$ は高速ゼータ変換 (の変形) により求められます。
- 各 U に対して $dp_{4,i,U}$ が求まっているとき、各 U に対する $dp_{1,i,U}$ は

  $$dp_{1,i+1,U} = (\mathrm{sz}(U) - (A_i - 1)) \times dp_{4,i,U}$$

  として求められます。

この DP テーブルが求められれば、あとは適切に足し合わせて適切な係数をかけるだけで答えが求められます。

以上で答えが求まりました。

ARC093 / ABC092 Editorial

wo01

March 25th, 2018

Problem A

The answer is $\min(A, B) + \min(C, D)$.

C++ example:

```
#include<cstdio>
#include<algorithm>

using namespace std;

int main(){
    int A, B, C, D;
    scanf("%d%d%d%d", &A, &B, &C, &D);
    printf("%d\n", min(A, B) + min(C, D));
    return 0;
}
```

Problem B

The i-th participant eats $1 + \lfloor (D-1)/A_i \rfloor$ chocolates. Here $\lfloor x \rfloor$ denotes the largest integer that doesn't exceed x.

Therefore, the answer is

$$X + \sum_{1 \leq i \leq N} (1 + \left\lfloor \frac{D-1}{A_i} \right\rfloor)$$

C++ example:

```
#include<cstdio>

using namespace std;

const int MAX_N = 100;

int N;
int A[MAX_N];
int D;
int X;

int main(){
    scanf("%d", &N);
    scanf("%d%d", &D, &X);
    for(int i = 0; i < N; ++i){
        scanf("%d", A + i);
    }
    int ans = X;
    for(int i = 0; i < N; ++i){
        int tmp = (D - 1) / A[i] + 1;
        ans += tmp;
    }
    printf("%d\n", ans);
    return 0;
}
```

Problem C

Let $A_0 = A_{N+1} = 0$. For each i $(1 \leq i \leq N)$, we want to compute the following value:

$$S_i = |A_0 - A_1| + |A_1 - A_2| + \cdots + |A_{i-2} - A_{i-1}| + |A_{i-1} - A_{i+1}| + \\ |A_{i+1} - A_{i+2}| + \cdots + |A_N - A_{N+1}|$$

Let

$$S = \sum_{0 \leq j \leq N} |A_j - A_{j+1}|$$

and we pre-compute this value in $O(N)$ time.

Then, since

$$S_i = S + |A_{i-1} - A_{i+1}| - (|A_{i-1} - A_i| + |A_i - A_{i+1}|)$$

we can compute the value of S_i in $O(1)$ for each i.

This solution works in $O(N)$ time in total.

Problem D

We can construct the grid as follows:

- Let $K = 50$ and prepare a grid of dimensions $2K \times 2K$.
- Paint all cells in the topmost K rows black. Paint the other cells white.
- From the topmost $K - 1$ rows, choose $A - 1$ cells such that no two chosen cells are 8-adjacent, and paint them white.
- From the bottommost $K - 1$ rows, choose $B - 1$ cells such that no two chosen cells are 8-adjacent, and paint them black.

Problem E

Let G be the given graph, and let T be (one of) its MSTs. Let S be its weight.

For two vertices u, v in T, define $\mathit{pathMax}(u, v)$ as the weight of the heaviest edge on the path uv along T. Also, if an edge $e = \{u, v\}$ is not contained in T, define $\mathit{diff}(e) = c - \mathit{pathMax}(u, v)$, where c is the weight of edge e. Note that for edges e in T, the value of $\mathit{diff}(e)$ is undefined (make sure not to regard them as zeroes). Since T is an MST, for all edge e outside T, $\mathit{diff}(e) \geq 0$ holds.

Suppose that we paint the edges of G black and white. The weight of the MST that contains both black and white edges can be computed as follows (let's denote it S'):

- If T contains both black and white edges, $S' = S$.
- If T contains only black edges, $S' = S + \mathit{diff}(e)$. Here, e is a white edge that minimizes the value of $\mathit{diff}(e)$.
- (Similarly, we can handle the case where all edges in T are white).

Therefore, the answer can be computed as follows.

First, let $D = X - S$. Let *lower*, *equal*, *upper* be the number of edges e such that $\mathit{diff}(e) < D$, $\mathit{diff}(e) = D$, and $\mathit{diff}(e) > D$, respectively.

Then,

- If $D < 0$, the answer is 0.
- If $D = 0$,
  - In case we use both colors at least once for the edges in T, there are $(2^{N-1} - 2)2^{M-N+1}$ ways to paint edges: $(2^{N-1} - 2)$ ways to color T by using both colors, and 2^{M-N+1} ways to arbitrarily color other edges.
  - Otherwise, there are $2 \times (2^{\mathit{equal}} - 1) \times 2^{\mathit{upper}}$ ways to paint edges: two ways to color edges in T, $(2^{\mathit{equal}} - 1)$ ways to color edges in "equal" (the color of at least one edge must be different from T's color), and 2^{upper} ways to arbitrarily color "upper" edges.
- If $D > 0$, the answer is $2 \times (2^{\mathit{equal}} - 1) \times 2^{\mathit{upper}}$: two ways to color edges in T and "lower", $(2^{\mathit{equal}} - 1)$ ways to color edges in "equal" (the color of at least one edge must be different from T's color), and 2^{upper} ways to arbitrarily color "upper" edges.

Problem F

Assume that $p_1 = 1$ (and later multiply the answer by 2^N).

Player 1 can win the tournament if he can beat all of the following players:

$$p_2, \min\{p_3, p_4\}, \ldots, \min\{p_{2^{N-1}+1}, \ldots, p_{2^N}\}$$

Thus, we want to compute the number of permutations such that none of the N numbers above are contained in the set $\{A_1, A_2, \ldots, A_M\}$.

Consider N groups of positions: $G_1 = \{p_2\}$, $G_2 = \{p_3, p_4\}$, $\ldots$, $G_N = \{p_{2^{N-1}+1}, \ldots, p_{2^N}\}$. We want to count the number of permutations such that for each group g in $\{G_1, \ldots, G_N\}$, the minimum element in g is *not* in $\{A_1, A_2, \ldots, A_M\}$. Let's use inclusion-exclusion principle. For each subset S of $\{G_1, \ldots, G_N\}$, we want to compute $f(S)$, the number of permutations such that

- For each group g in S, the minimum element of g is in $\{A_1, A_2, \ldots, A_M\}$.

To compute these values, do the following DP. Assume that $A_1 > A_2 > \cdots > A_M$. For each $i = 1, \cdots, M$ in this order, do one of the following:

- For some integer k, choose $2^k - 1$ players greater than A_i, and assign them to the group G_k, together with the player A_i. Now the minimum element in G_k is in $\{A_1, A_2, \ldots, A_M\}$.
- Do nothing.

For each integer $0 \leq i \leq M$ and a set $S \subseteq \{G_1, \ldots, G_N\}$, define $dp_{i,S}$ as the number of ways such that when we process players $A_1, \cdots, A_i$ as described above, the groups in S are filled, and other groups are empty. Then, we can compute $f(S)$ as $dp_{M,S}$ times some coefficients, and by inclusion-exclusion principle, the answer is $\sum f(S)(-1)^{|S|}$.

ARC 091/ABC 090 解説

Kohei Morita(yosupo)

平成 30 年 3 月 19 日

For International Readers: English editorial starts on page 7.

A: Two Coins

A, B, C を入力し，$A + B \geq C$ を判定すれば良いです。
[https://beta.atcoder.jp/contests/abc091/submissions/2218039]

B: Two Colors Card Game

どのカードにも書かれていない，めちゃくちゃな文字列を言えば 0 点を取ることが出来ます。そして，1 点以上を取るためには，少なくとも青いカードのうちどれかに書かれた文字列を言う必要があります。

よって，青いカードに書かれた文字列それぞれについて，その文字列を言ったら何点貰えるかを計算し，その最大値を計算すれば良いです。

[https://beta.atcoder.jp/contests/abc091/submissions/2218376]

C: 2D Plane 2N Points

青い点のうち，最も x 座標が小さいもの (bA とします) に注目します。この点と仲良しペアになれる点が存在した場合，そのような点のうち最も y 座標が大きいもの (rA とします) と仲良しペアにしてよいです。

証明ですが，bA がどの点ともペアにならなかった場合と，rA 以外とペアになった場合それぞれについて，bA と rA をペアにしても同じ個数，もしくはそれ以上の個数の仲良しペアが作れることを示します。

bA がどの点ともペアにならなかった場合　rA がどの点ともペアになっていなかったら，bA と rA をペアにすれば良いです。rA が他の青い点 (bB とします) とペアになっていたら，そのペアを解消し，bA と rA をペアにすれば良いです。

bA が rA 以外とペアになった場合 (bA とペアになっている点を rB とします)　rA がどの点ともペアになっていなかったら，bA の仲良しペアを解消し，bA と rA をペアにすれば良いです。

rA が他の青い点 (bB とします) とペアになっていた場合，つまり bA-rB, rA-bB というペアが出来ていた場合を考えます。この場合は，ペアの相手を交換し，必ず bA-rA, rB-bB というペアに出来ます。これは，以下の 2 つから示せます。

- bA が青い点のなかで一番 x 座標が小さいので，bB の x 座標は必ず bA より大きい，つまり rB よりも大きいです。
- rB は (bA とマッチング出来ていたので，) 必ず rA より y 座標が小さいです。また，bB は (rA とマッチング出来ていたので，) 必ず rA より y 座標が大きいです。よって，bB の y 座標は，必ず rB より大きいです。

以上より，bA はかならず rA とペアにしていいことが示せます。
この性質より，青い点を x 座標の小さい順に見ていき，

- 青い点より x, y 座標が小さく，まだ仲良しペアになっていない赤い点を探す
- なかったらなにもしない
- あったら，その中で最も y 座標が大きいものを探し，仲良しペアにする

というアルゴリズムで最適解が求まります。

余談　この問題は，二部グラフの最大マッチング問題としても考えることが可能で，これは最大流アルゴリズムを使用すれば解くことが出来ます。

D: Two Sequences

まず，この問題では bit は 0-indexed とします。つまり，2 進数で一番下の桁から，0-bit 目，1-bit 目, ..., と呼びます。

答えを各 bit ごとに求めます。k-bit 目を求めたいとします。

xor の定義より，N^2 個の $a_i + b_j$ のうち，k-bit 目が 1 のものが偶数個あるか奇数個あるか判定できれば良いです。

ここで重要な考察として，$a_i + b_j$ の k-bit 目を考えているので，a_i, b_j の $k+1$-bit 目以降は無視してよい，つまり a_i, b_j は 2^{k+1} で mod を取ってしまって良いというのがあります。

すると，$a_i + b_j$ はたかだか $4T (T = 2^k)$ 未満です。そして，k-bit 目が 1 の範囲は，$[T, 2T), [3T, 4T), [5T, 6T), ...$ なので，$a_i + b_j$ のうち，$[T, 2T)$, $[3T, 4T)$ の範囲にあるものの個数を求めればよいことがわかります。

これは，a_i を固定してしまえば，b_j のうち $[T - a_i, 2T - a_i), [3T - a_i, 4T - a_i)$ の範囲にあるものの個数を求めれば良くなり，これは b_j を sort しておけば二分探索で $O(logN)$ で求められます。

余談　この問題の $O(N^2)$ 解は非常に単純なうえ，SIMD で高速化が効き，とてつもなく高速に動作します。

writer が素直な $O(N^2)$ を書き，ブロック化を行ったら 3770ms となったので，SIMD が得意な人ならばもしかすると愚直解で通せてしまうのではとも考えています。

ぜひ SIMD が得意な人は挑戦してみてください。

E: Both Sides Merger

最終的な数列の要素は，当然 a_i をいくつか足したものになります。

ここで，この i の集合に注目します。例えばサンプル 1 なら最終的な要素は $a_2 + a_4$ なので，$\{2, 4\}$ という集合に注目します。

この集合にはどのような性質があるでしょうか？実は，必ずこの集合の要素は，偶奇は全て等しくなります。

つまり，偶奇の違うペア，例えば a_1, a_2 や a_4, a_7 は，絶対に一つの要素に合わせられないです。

これは，どのような操作をしてもこのペアの要素は偶奇が異なったまま (なので，一つの要素にまとめることは不可能) であることから示せます。

実は，これが必要十分です。つまり，すべての要素が偶奇が等しい集合が与えれば，最終的な要素がそれに対応した要素の和になるような手順が必ず存在します。

構築の方法ですが，両端のいらない要素をまず全てカットします。その後

- 両端がどちらも残したい要素，かつ自分自身は残したくない要素 (A)
- 両端がどちらも残したくない，かつ自分自身も残したくない要素 (B)

この A, B を満たす要素のうちどちらかは必ず取れるので，これを取っていけばかならず要素数が少なくなっていき，最終的に 1 つになります。

なぜこのどちらかは必ず存在するのかというと，集合の要素の偶奇が等しいということは，oxxxoxo，のように残したい要素同士の間には，残したくない要素が奇数個挟まっています。

もし 1 個挟まっていれば，その要素は A の条件を満たし，3 個以上挟まっていれば，(そのうち両端以外は)B の条件を満たします。よって，必ずどちらかを満たす要素が存在します。

以上より，この問題は，「数列から添字の偶奇が同じように要素を取って，総和を最大化してください」という問題になります。これは，偶奇を固定し，0 以上の要素をすべて取るだけでよいです。

コーナーケースは全部の要素が負の場合です。十分注意してください。

F: Two Faced Edges

各辺 i に対し，以下の条件を満たすかを調べたいです。

- 1. もし辺 (a_i, b_i) を消したとしても，a_i から b_i に到達可能か？
- 2. b_i から a_i に到達可能か？

この 2 つの条件を両方達成するか，両方達成しない場合，辺の反転で SCC の成分数が変わらないことが容易に示せます。

条件 2 は $O(N+M)$ や $O(N(N+M))$ で容易に調べられます。ですので，この問題の本質は条件 1 を高速に求めることです。

方針としては，頂点を固定し，そこから出ていく辺すべてについての条件 1 を $O(N+M)$ で求めます。つまり，a_i を固定します。

まず，a_i から出てくる辺に，適当に連番を振っておきます。

当然 (a_i, b_i) を使わずに a_i から b_i に到達可能ならば，a_i から出ていく他の辺を使う必要があります。そしてその辺の番号は (a_i, b_i) より小さいかまたは大きいです。

よって，まずは小さい辺を使った場合のみ，つまり以下の条件を考えます。当然小さい辺を使った場合のみに限定して解ければ，辺に降った番号を逆順にしてもう一度同じ問題を解けば良いので，この問題が解けます。

- 3. もし辺 (a_i, b_i) を消したとしても，a_i から (a_i, b_i) より小さい番号の辺を使い，b_i に到達可能か？

条件 3 は，辺の番号が小さい順に辺を追加していき，追加する直前に今まで追加した辺を利用して b_i に到達できるか，を調べられれば良いです。

これは，辺を追加するたびに dfs を走らせ，二度同じ頂点は辿らないようにするだけで，まとめて $O(N+M)$ となります。

よってこの問題はまとめて $O(N(N+M))$ で解けました。

A: Two Coins

[https://beta.atcoder.jp/contests/abc091/submissions/2218039]

B: Two Colors Card Game

[https://beta.atcoder.jp/contests/abc091/submissions/2218376]

C: 2D Plane 2N Points

Let A be a blue point with the minimum x-coordinate. If A can be paired with red points, we should choose a red point B such that B can be paired with A, and among all such points the y-coordinate is the maximum possible.

(If B is paired with other point, for example if there are pairs $B-C$ and $A-D$, we can rearrange them because $A-C$ and $B-D$ are also valid pairs).

By doing this, we can reduce the number of points by 1 (if A can't be paired at all) or 2 (if we find a pair $A-B$).

In summary, we first sort all $2N$ points together by x-coordinates, and process points in this order. We also keep a set of points (initially this is empty). Then, for each point, do the following:

- If a red point appears, add its y-coordinate to the set.
- If a blue point appears, let p be its y-coordinate. If all elements in the set are greater than p, do nothing (we can't do anything with this blue point). Otherwise, choose the maximum element q in the set such that $q < p$, remove it, and increase the answer by one (the current blue point and the point that corresponds to q are paired).

D: Two Sequences

Since we can handle each bit independently, let's consider only the k-th bit. We want to count the number of pairs (i, j) such that the k-th bit of $a_i + b_j$ is one. (And if this number is add, we should add 2^k to the answer.)

Let $T = 2^k$. An important observation is that, we are only interested in the values of a_i, b_i in modulo $2T$. Thus, let's replace a_i with $a_i\%(2T)$ and b_i with $b_i\%(2T)$, and assume that $a_i, b_i < 2T$.

Then, there are two cases when the k-th bit of $a_i + b_j$ is one:

- $T \leq a_i + b_j < 2T$
- $3T \leq a_i + b_j < 4T$

Let's sort b in an increasing order. For a fixed i, the set of j that satisfies $T \leq a_i + b_j < 2T$ forms an interval. Thus, we can count the number of such js by binary search (or two-pointers). Similarly, we can handle the second case.

This solution works in $O(NlogNlogMAX)$ time, where MAX is the maximum number that can appear in the input.

E: Both Sides Merger

The answer of this problem will be the sum of a subset of $\{a_i\}$.

For example, consider example 1. In this sample, the answer is $a_2 + a_4$, so the set of indices is $\{2, 4\}$

Suppose that we are given a set of indices, I. Let's check if we can perform operations such that the final integer corresponds to the sum of all elements with indices in I.

It turns out that the following are neccesary and sufficient conditions:

- The parity of all indices in I are the same.
- I is non-empty.

It's easy to prove that these conditions are necessary. Suppose that initially, x is in an odd-indexed position and y is in an even-indexed position. Then, no matter what operations we perform, the parities of the indices of x, y never become the same.

These are also sufficient conditions because we can construct a sequence of operations, as we describe below.

To compute the answer, fix a parity, and take all non-negative elements in the positions with chosen parity. When all elements are negative, you need to be careful not to make it empty.

To construct a sequence of operations, we first remove all unnecessary elements from both ends. Then, we repeat choosing elements while we have more than one elements. We can choose an element x if one of the following holds:

- x is adjacent to two elements (call it y, z), and we want to keep both y and z in the final element.
- x is adjacent to two elements (call it y, z), and we want to remove both y and z from the array.

It's easy to see that such elements always exist.

F: Two Faced Edges

For each edge i, we want to determine the following two things:

- 1. If we remove an edge from a_i to b_i, is b_i still reachable from a_i (by using other edges)?
- 2. Is a_i reachable from b_i?

It's easy to see that the number of SCCs won't change after the reversion of edge i if and only if the answers to the two questions above are the same (i.e., both are yes or both are no). Since the second question can be solved easily in $O(NM)$ by running dfs from all vertices, we will focus on the first question.

Consider a particular vertex x. We will solve the first question for all edges such that $a_i = x$, in $O(M)$ time. Then, we can solve the entire problem in $O(NM)$.

Suppose that there are k edges that goes out of the vertex x, and let's name the destinations $y_1, \ldots, y_k$. We will do the following:

- By running a dfs from y_1, we mark all vertices that are reachable from y_1 without visiting x.
- By running a dfs from y_2, we mark all unmarked vertices that are reachable from y_2 without visiting x.
- By running a dfs from y_3, we mark all unmarked vertices that are reachable from y_3 without visiting x.
- And so on.

Then, for each vertex z, we know the minimum integer $p(z)$ such that z is reachable from x by using an edge $x- > y_{p(z)}$ (and without visiting the vertex x again). Similarly, by running dfs in the reverse order, for each vertex z, we know the maximum integer $q(z)$ such that z is reachable from x by using an edge $x- > y_{q(z)}$ (and without visiting the vertex x again). The question 1 for an edge $x- > y_i$ is "yes" if and only if at least one of $p(y_i) \neq i$ or $q(y_i) \neq i$ holds.

This solution works in $O(NM)$ time.

ABC 090/ARC 091 解説

DEGwer

2018/03/11

For International Readers: English editorial starts on page 6.

A: Diagonal String

c_{11}, c_{22}, c_{33} を順に出力すればよいです。

```
#include<stdio.h>
int main()
{
	char a[10][10];
	scanf("%s%s%s", a[0], a[1], a[2]);
	printf("%c%c%c\n", a[0][0], a[1][1], a[2][2]);
}
```

B: Palindromic Numbers

A 以上 B 以下の全ての整数について、(5 桁であることを適宜利用して) 回文数かどうか判定していけばよいです。

```
#include<stdio.h>
int main()
{
	int a, b;
	scanf("%d%d", &a, &b);
	int r = 0;
	for (int i = a; i <= b; i++)
	{
		int s = i % 10, t = i / 10000 % 10;
		int u = i / 10 % 10, v = i / 1000 % 10;
		if (s == t&&u == v)r++;
```

```
        }
        printf("%d\n", r);
}
```

C: Flip,Flip, and Flip.....

$N \leq M$ として一般性を失いません。偶数回裏返されるカードは全操作の終了後には表を、奇数回裏返されるカードは裏を向くことになるので、各マスのカードが何回裏返されるかを考えることにしましょう。

$N = 1$ かつ $M = 1$ のとき、1 枚だけあるカードは 1 回裏返されるので、答えは 1 です。

$N = 1$ かつ $M \neq 1$ のとき、M 枚のカードのうち両端のカードは 2 回、それ以外のカードは 3 回裏返されるので、答えは $M - 2$ です。

$N \geq 2$ のとき、$N \leq M$ より $M \geq 2$ で、このとき、

- 四隅のカードは 4 回
- それ以外の周上のカードは 6 回
- それ以外のカードは 9 回

裏返されるので、答えは $(N-2)(M-2)$ です。

以上で答えがすべて求まりました。

D: Remainder Reminder

b を固定して考えましょう。$b = 1, 2, \cdots N$ に対し、a を b で割った余りが K 以上であるような $1 \leq a \leq N$ の個数が高速に求められれば良いです。簡単のため、$a = 0$ も許すことにして、あとで $a = 0$ の場合 (これは簡単に求められます) を引くことにしましょう。

さて、整数 p, q を用いて $N = pb + r (0 \leq r < b)$ という形で N を一意的に表したとき、a を 0 から N まで順に動かせば、a を b で割った余りを順に並べたものは、$0, 1, 2, \cdots b-1$ という列が p 回繰り返され、最後に $0, 1, 2, \cdots r$ という列が付け加わったものになります。

$0, 1, 2, \cdots b-1$ という列が p 回繰り返される部分には条件を満たす a の個数は $p \times \max(0, b-K)$ 個、最後の部分には $\max(0, r-K+1)$ 個あるので、条件を満たす a の個数が $O(1)$ 時間で求められ、$O(N)$ 時間でこの問題を解くことができました。

E: LISDL

まず、条件を満たす列が存在する条件を考えてみましょう。増加部分列と減少部分列は 2 つ以上の要素を共有できないため、$A + B - 1 \leq N$ が必要です。また、長さ N の列 P と $1 \leq i \leq N$ について、$f(i)$ で P の i 番目の要素を最後の要素とするような P の増加部分列の最大長を表すことにすることにすれば、$f(i)$ の値が等しいような i について i 番目の要素を取ってきて全て順に並べた列は減少列となるので、$f(i)$ が等しいような P の要素の個数は最大で B 個以下であり、よって ($f(i) \leq A$ より) $AB \geq N$ が必要です。

逆に、これらの条件が満たされれば、条件を満たす列を構成することができます。以下、これを示します。

$AB = N$ の場合に、ある最長増加部分列とある最長減少部分列が 1 つの要素を共有するようなものが構成

できれば、あとはそこから (その最長増加部分列または最長減少部分列に含まれる $A+B-1$ 個の要素を残すように) 要素を必要なだけ取り除き、座標圧縮を行うことで条件を満たす列を構成することができます (もちろん、計算量が大きくならないような適切な実装をする必要がありますが、それについては簡単なので割愛します)。よって、以下ではそのような構成のみ考えます。

N 個の要素を A 個ずつの要素を含む B 個のブロックに分け、i 個目のブロックの j 個目の要素を P_{ij} と呼ぶことにしましょう。すなわち、作る列では、要素が $P_{11}, \cdots, P_{1A}, P_{21}, \cdots, P_{2A}, \cdots, P_{B1}, \cdots, P_{BA}$ の順に並ぶようにするとしましょう。このとき、要素を小さい方から順に $P_{B1}, \cdots, P_{11}, P_{B2}, \cdots, P_{12}, \cdots, P_{BA}, \cdots, P_{1A}$ と並ぶようにすれば、条件をすべて満たすことを証明できます。実際、長さ A の増加部分列と長さ B の減少部分列が存在することは容易にわかり、また、

- 増加部分列について、ブロック内でその要素が何番目かを表す値 (P の 2 番目の添え字) は狭義単調増加
- 減少部分列について、その要素が属するブロックの番号 (P の 1 番目の添え字) は狭義単調増加

することから、その長さを超える増加部分列や減少部分列が存在しないことも分かります。よって構成ができ、この問題が解けました。

F: Strange Nim

各山については独立したゲームとなっているため、各山についてその Grundy 数が求められれば良いです。石 N 個からなり、整数 K の定まった山を山 (N, K) と呼ぶことにします。

このとき、以下が主張できます。

- N が K の倍数のとき、山 (N, K) の Grundy 数は N/K
- そうでないとき、山 (N, K) の Grundy 数は山 $(N - \lfloor N/K \rfloor - 1, K)$ の Grundy 数に等しい

これを帰納法で証明しましょう。帰納法の仮定に「山 $(N, K), (N-1, K), \cdots, (N - \lfloor N/K \rfloor, K)$ の Grundy 数はすべて異なる 0 以上 $\lfloor N/K \rfloor$ 以下の整数である」という条件を追加し、帰納法を回します。

$N = 0$ のときは明らかです。$N = t - 1$ で正しいとし、$N = t$ で主張を証明しましょう。

山 (N, K) の Grundy 数は $(N-1, K), (N-2, K), \cdots, (N - \lfloor N/K \rfloor, K)$ の Grundy 数のどれとも異なる最小の値です。N が K の倍数のとき、帰納法の仮定よりこれらはすべて異なる 0 以上 $\lfloor N/K \rfloor - 1$ 以下の整数であり、よって 0 以上 $\lfloor N/K \rfloor - 1$ 以下の全ての整数値をとり、よって山 (N, K) の Grundy 数は N/K です。そうでない場合、これらはすべて異なる 0 以上 $\lfloor N/K \rfloor$ 以下の整数であり、よって 0 以上 $\lfloor N/K \rfloor$ 以下の整数のうちちょうど 1 つだけここに現れないものがあります。山 $(N-1, K)$ の Grundy 数を求めたときにはその Grundy 数への遷移が考慮されていたことから、結局山 (N, K) の Grundy 数は山 $(N - \lfloor N/K \rfloor - 1, K)$ の Grundy 数に等しいことが分かります。また、追加した条件が満たされることは Grundy 数の定義から容易に導かれます。

よって帰納法が回り、上記が示されました。しかし、この通りに計算してもまだ計算量は大きいままです。以下、計算量を落とすことを考えましょう。

この計算量は、遷移をある程度「一気に行う」ことで削減可能です。$N < K^2$ のとき、上記を愚直に実装すれば N の値は N/K ほどしか減少しません。しかし、もし遷移先でも $\lfloor N/K \rfloor$ の値が変化しないのならば、N の減少分も変化しません。よって、$\lfloor N/K \rfloor$ の値が変化しないギリギリまで N の値を減らす操作をまとめ

て行うことで、遷移の回数を減らすことができます。

計算量を見積もりましょう。$N < K^2$ の領域では、遷移をまとめて行えば $\lfloor N/K \rfloor$ の値は定数回の操作で 1 減少するため、$O(\min(N/K, K))$ 時間ですべての遷移を行うことができます。そうでない領域では、N の値は 1 回の操作で必ず $\frac{K-1}{K}$ 倍以下になり、よって K 回の操作でおよそ $\frac{1}{e}$ 倍以下になります。よって必要な遷移の回数は $O(K \log(N/K^2))$ 回であり、この値は K を動かせば $K = \sqrt{N}e^{-1}$ で最大値 $O(K)$ を取ります。$N \geq K^2$ よりこれは $O(\min(N/K, K))$ で、よって各山について $O(\min(N/K, K)) \leq O(\sqrt{N})$ 時間で Grundy 数を求めることができるので、この問題を解くことができました。

ABC 090/ARC 091 Editorial

DEGwer

2018/03/11

A: Diagonal String

Print c_{11}, c_{22}, c_{33} in this order.

```
#include<stdio.h>
int main()
{
        char a[10][10];
        scanf("%s%s%s", a[0], a[1], a[2]);
        printf("%c%c%c\n", a[0][0], a[1][1], a[2][2]);
}
```

B: Palindromic Numbers

For each integer between A and B, check if it is a palindrome (possibly by using the fact that it's a 5-digit number).

```
#include<stdio.h>
int main()
{
        int a, b;
        scanf("%d%d", &a, &b);
        int r = 0;
        for (int i = a; i <= b; i++)
        {
                int s = i % 10, t = i / 10000 % 10;
                int u = i / 10 % 10, v = i / 1000 % 10;
                if (s == t&&u == v)r++;
        }
        printf("%d\n", r);
```

}

C: Flip,Flip, and Flip.....

Without loss of generality, we can assume that $N \leq M$. For each card, let's count the number of times it is flipped: it will be faced down if this number is odd.

In case $N = 1$ and $M = 1$, the only card will be flipped once. The answer is 1.

In case $N = 1$ and $M \neq 1$, the two cards at the end will be flipped twice, and other cards will be flipped three times. The answer is $M - 2$.

In case $N \geq 2$, since $N \leq M$, we have $M \geq 2$, and in this case

- Four cards at the corners will be flipped 4 times.
- Other cards at the edge will be flipped 6 times.
- The cards that are completely surrounded by other cards will be flipped 9 times.

Thus, the answer is $(N - 2)(M - 2)$.

D: Remainder Reminder

Let's fix b. For each $b = 1, 2, \cdots N$, we want to count the number of a that satisfies both $1 \leq a \leq N$ and $a\%b \geq K$. For simplicity, we allow $a = 0$ and assume that $0 \leq a \leq N$ (and subtract the cases with $a = 0$ later).

For a fixed b, we want to count the number of terms that are greater than or equal to K, in the sequence $0\%b, 1\%b, \cdots, N\%b$. If $N = pb + r (0 \leq r < b)$ (we can represent N in this form uniquely using two integers p, r), the sequence starts with p repetitions of $0, 1, 2, \cdots b - 1$, followed by $0, 1, 2, \cdots r$.

In the former part, there are $p \times \max(0, b - K)$ terms that are greater than or equal to K. In the latter part, there are $\max(0, r - K + 1)$ terms that are greater than or equal to K. Thus, for a fixed b, we can compute the number of such as in $O(1)$ time.

This solution works in $O(N)$ time in total.

E: LISDL

When does a solution exist? First, since a LIS and a LDS can share at most one element in common, $A + B - 1 \leq N$ is necessary. Also, we can prove that $AB \geq N$ as follows. Let $f(i)$ be the length of the longest increasing sequence that ends with the i-th element. For a fixed c ($1 \leq c \leq A$), consider a subsequence that contains the i-th element if and only if $f(i) = c$. This sequence must be decreasing, and the length must be at most B. Thus, we get $AB \geq N$.

On the other hand, when $A + B - 1 \leq N \leq AB$ holds, we can construct a sequence as follows.

First, suppose that $N = AB$. In this case, the sequence $(B-1)A+1, (B-1)A+2, \cdots, BA, (B-2)A+1, (B-2)A+2, \cdots, (B-1)A, \cdots, 1, 2, \cdots, A$ satisfies the conditions. (It consists of B blocks of length A, each block is an increasing consecutive sequence).

It's clear that this sequence contains an increasing sequence of length A and a decreasing sequence of length B. Also, these are the longest because

- All elements in an increasing sequence must be in the same block.
- No two elements in a decreasing sequence must be in the same block.

When $N < AB$, we first construct a sequence above, keep $A + B - 1$ elements that contain both LIS and LDS, and remove arbitrary $AB - N$ elements from others. Then, the length of LIS and LDS won't change. We compress the sequence while keeping the relative order of elements, and we get a desired sequence. (Of course, make sure not to spend $O(AB)$ to implement this.)

F: Strange Nim

Since the piles are independent, it is sufficient to compute grundy numbers of all piles. Let $g(N, K)$ be the grundy number of a pile with N stones and a parameter K.

If we perform an experiment for a fixed K, we notice that $g(N, K) = N/K$ when N is divisible by K. Also, we notice that even if we remove the K-th, $2K$-th, $3K$-th, $\cdots$ elements from the sequence $g(1, K), g(2, K), \cdots$, the sequence doesn't change. Thus, we get

- If N is a multiple of K, $g(N, K) = N/K$
- Otherwise, $g(N, K) = g(N - \lfloor N/K \rfloor - 1, K)$

Once we get the pattern it's straightforward to prove this: just make sure that from each state with grundy number g, it's possible to reach states with grundy numbers $0, 1, \ldots, g - 1$, but not g.

How can we compute grundy numbers using the fact above? We start with an integer N, and while N is not divisible by K, we keep replacing N with $N - \lfloor N/K \rfloor - 1$. We are interested in the final value of N. However, a straightforward implementation of this will get TL.

To make it faster, notice that if the value of $\lfloor N/K \rfloor$ doesn't change after an operation, we can perform multiple steps at once. More explicitly, if currently $\lfloor N/K \rfloor = d$, we keep decreasing N by $d + 1$ while $N \geq dK$. Thus, instead of performing steps one by one, we can make multiple steps at once until we first get $N < dK$.

What's the time complexity after this improvement?

- Since the value of $\lfloor N/K \rfloor$ decreases in each step, it's $O(N/K)$.
- Since the value of N is multiplied by a factor of at most $\frac{K-1}{K}$ in each step, in every K steps this is multiplied by a factor of approximately $\frac{1}{e}$. Thus, this is $O(K \log N)$.

We should take the better of the two analysis above: it's $O(\min(N/K, K \log N) = O(\sqrt{N \log N})$.

ARC090 / ABC087 解説

wo01

2018 年 1 月 28 日

A 問題

答えは $X - A$ を B で割った余りとなります。
整数を整数で割った余りを求める演算子は多くの言語で用意されています。
以下に C++ による実装例を示します。

```
#include<cstdio>

using namespace std;

int main(){
    int X, A, B;
    scanf("%d%d%d", &X, &A, &B);
    printf("%d\n", (X - A) % B);
    return 0;
}
```

B 問題

ループを用いて 500 円玉、100 円玉、50 円玉を選ぶ枚数の組 (i, j, k) をすべて試すことで答えが求められます。
以下に C++ による実装例を示します。

```
#include<cstdio>

using namespace std;

int A, B, C;
int X;
```

```
int solve(){
    int ans = 0;
    for(int i = 0; i <= A; ++i){
        for(int j = 0; j <= B; ++j){
            for(int k = 0; k <= C; ++k){
                int tmp = i * 500 + j * 100 + k * 50;
                if(tmp == X) ans++;
            }
        }
    }
    return ans;
}

int main(){
    scanf("%d%d%d%d", &A, &B, &C, &X);
    int ans = solve();
    printf("%d\n", ans);
    return 0;
}
```

C 問題

移動方法は「ある i $(0 \leq i \leq N)$ を選び、以下のように移動する」というものになります。

1. 右に i 回移動する
2. 下に 1 回移動する
3. 右に $N - i$ 回移動する

最初に右に移動する回数 i の候補は $O(N)$ とおりしかなく、その値を決めたとき回収できるアメの個数は $O(N)$ 時間で計算できるので、求める最大値は $O(N^2)$ 時間で計算できます。

D 問題

各 i について $0 \leq x_i \leq 10^9$ であるという条件は考慮しなくて良いことがすぐにわかります。よって、x_i として M 個の情報と矛盾しない限り任意の整数をとることができるものとして考えることにします。

以下のような重み付き有向グラフ G を考えます。

- 頂点は N 個あり、$1, 2, \ldots, N$ の番号が付けられている
- 各 i $(1 \leq i \leq M)$ に対して、頂点 L_i から頂点 R_i に向かう重み D_i の辺と、頂点 R_i から頂点 L_i に向かう重み $-D_i$ の辺が存在する
- それ以外に辺は存在しない

問題の条件を満たす $(x_1, x_2, \ldots, x_N)$ が存在することは、グラフ G の各頂点 v に整数 x_v を割り当てて、G に頂点 u から頂点 v へ向かう重み d の辺があるならば $x_v - x_u = d$ が成り立つようにできることと同値です。

このような割り当てを行うことができるかどうかの判定は、グラフ G に深さ優先探索や幅優先探索を行い、実際に各頂点に値 x を割り当てて矛盾が生じないか確かめることで行えます。

実装の際は、グラフ G が非連結な場合があることに注意してください。

E 問題

各頂点 v について、頂点 S から頂点 v への最短距離を d_v と表すことにします。頂点の組 (u, v) について、頂点 u から頂点 v への最短路を逆向きにすると頂点 v から頂点 u への最短路が得られることに注意します。

頂点 u と頂点 v を結ぶ長さ c の辺 (このような辺を辺 $(\{u, v\}, c)$ と書くことにします) が、頂点 S から頂点 v への最短路に使われうる条件は $d_u + c = d_v$ が成り立つことです。このことを用いると、各頂点 v について、頂点 S から頂点 v への最短路の個数 $dp_1[v]$ を d_v が小さい順に動的計画法を行うことで求めることができます。同様に、頂点 v から頂点 T への最短路の個数 $dp_2[v]$ を d_v が大きい順に動的計画法を行うことで求められます。

dp_1 および dp_2 の値から、以下のことがわかります。

- 2 人が途中で出会うようなものも含めた、2 人の最短路の選び方の組は $dp_1[T]^2$ 通りある
- 2 人が頂点上で出会うような最短路の選び方の組は、$2d_v = d_T$ となる各頂点 v について、$dp_1[v]^2 dp_2[v]^2$ 通りある
- 2 人が辺上で出会うような最短路の選び方の組は、$2d_u < d_T$ かつ $2d_v > d_T$ かつ $d_u + c = d_v$ を満た各辺 $(\{u, v\}, c)$ について、$dp_1[u]^2 dp_2[v]^2$ 通りある

求めるべき最短路の組の個数は、2 人が途中で出会うようなものも含めた最短路の選び方の組の個数から、2 人が途中で出会うような最短路の選び方の組を引くことで求められます。

F 問題

d 桁の正の整数の個数 $10^d - 10^{d-1}$ を N_d と書くことにします。

$f(r) - f(l) \leq 2$ のとき、$\sum_{l \leq i \leq r} f(i) > (f(l) + 1) N_{f(l)+1}$ となります。$f(r) - 1 \geq 8$ のとき、$(f(l) + 1) N_{f(l)+1} \geq 8 N_8 > 10^8$ となるので、問題の条件 $S \leq 10^9$ のもと、$f(r) - f(l) \geq 2$ ならば $f(r) - 1 < 8$ であることがわかります。

このことに注意して、以下のように場合分けを行います。

1. $f(l) = f(r)$ の場合
2. $f(r) < 9$ かつ $f(r) \geq f(l) + 1$ の場合
3. $f(r) \geq 10$ かつ $f(r) = f(l) + 1$ の場合

上記のそれぞれの場合について (l, r) の個数を求め、足し合わせれば良いことになります。

それぞれの場合における (l, r) の個数の求め方について見ていきます。

1. $f(l) = f(r)$ の場合
   値 $d = f(l)$ として考えられるものは S の約数だけです。また、値 $d = f(l)$ を決めると、組 (l, r) の個数は $\max(0, N_d - S/d + 1)$ と求められます。よって、d の値をすべて試すことで組 (l, r) の個数が求められます。
2. $f(r) < 9$ かつ $f(r) \geq f(l) + 1$ の場合
   2 つの組 (l_1, r_1) と (l_2, r_2) がともに条件を満たし、$l_1 < l_2$ が成り立つとき $r_1 < r_2$ も成り立つことに注意して、l としてあり得る値をすべて試すことにより組 (l, r) の個数が求められます。
3. $f(r) \geq 10$ かつ $f(r) = f(l) + 1$ の場合
   整数 t を $10t < S$ かつ t は S の約数でないようなものとします。このとき、$r - l + 1 = t$ であるような (l, r) はちょうど 1 つ存在することがわかります。すなわち、l は $\lfloor S/t \rfloor$ 桁の整数のうち大きい方から $t - (S \mod t)$ 番目、r は $\lceil S/t \rceil$ 桁の整数のうち小さい方から $S \mod t$ 番目となります。
   このことから (l, r) の個数は $S/10$ 未満の整数のうち S の約数でないものの個数と一致し、この値は簡単に求められます。

すべての場合について (l, r) の個数を求めることができたので、これらを足し合わせることで答えが求まります。

ARC090 / ABC087 Editorial

wo01

January 28th, 2018

Problem A

C++ Example:

```
#include<cstdio>

using namespace std;

int main(){
    int X, A, B;
    scanf("%d%d%d", &X, &A, &B);
    printf("%d\n", (X - A) % B);
    return 0;
}
```

Problem B

C++ Example:

```
#include<cstdio>

using namespace std;

int A, B, C;
int X;

int solve(){
    int ans = 0;
    for(int i = 0; i <= A; ++i){
        for(int j = 0; j <= B; ++j){
            for(int k = 0; k <= C; ++k){
                int tmp = i * 500 + j * 100 + k * 50;
                if(tmp == X) ans++;
            }
        }
    }
    return ans;
}

int main(){
    scanf("%d%d%d%d", &A, &B, &C, &X);
    int ans = solve();
    printf("%d\n", ans);
    return 0;
}
```

Problem C

The sequence of movements must be in the following form for some integer i ($0 \leq i \leq N$):

1. Move to the right i times.
2. Move downward.
3. Move to the right $N - i$ times.

There are $O(N)$ possibilites for i. For a fixed i, we can compute the total number of candies in $O(N)$. Thus, brute force works in $O(N^2)$ time.

Problem D

Consider the following directed graph G:

- There are N vertices numbered $1, 2, \ldots, N$.
- For each i, there are an edge from vertex L_i to vertex R_i with weight D_i, and an edge from vertex R_i to vertex L_i with weight $-D_i$.

The problem asks whether we can assign an integer x_v to each vertex v in G, such that for each edge from u to v with cost d, $x_v - x_u = d$ holds. (Clearly, we can ignore the condition $0 \leq x_i \leq 10^9$.)

We handle each connected component in G independently. For each connected component, we choose an arbitary vertex v and assume that $x_v = 0$. By running a dfs from v, we can uniquly determine the values of x_i in this component. After that, we should check if the conditions are actually satisfied.

Problem E

For each vertex v, let $d1_v$ be the length of the shortest path from S to v. Let $dp_1[v]$ be the number of shortest paths from S to v.

An edge between u and v with cost c can be a part of a shortest path from S to v (in the direction from u to v) if and only if $d1_u + c = d1_v$. By using this, we can compute the values of $dp_1[v]$ from the verteices with smaller values of d_1.

Similarly, define $d2_v$ as the length of the shortest path from T to v, and $dp_2[v]$ as the number of shortest paths from T to v.

From dp_1 and dp_2, we can get the following values:

- The total number of pairs of shortest paths is $dp_1[T]^2$.
- Let $D = d1_T$. For each vertex v such that $d1_v = d2_v = D/2$, there are $dp_1[v]^2 dp_2[v]^2$ pairs of paths such that the two people meet on v.
- Consider an edge between vertices u and v with cost c. If $d1_u < D/2$, $d2_v < D/2$, and $d1_u + d2_v + c = D$, there are $dp_1[u]^2 dp_2[v]^2$ pairs of paths such that the two people meet on this edge (except for endpoints).

The answer is (the first value) - (the second value) - (the third value) mentioned above.

Problem F

Let $N_d = 10^d - 10^{d-1}$ be the number of d-digit integers.

If $f(r) - f(l) \geq 2$, it means that all integers with $l+1$ digits are in the chosen range. Since the total number of digits of 8-digit integers is $8N_8 = 720000000 > 10^8$, $l+1$ must be less than 8 in this case. Now, consider the following two cases:

The case with $f(l) \leq 7$

In this case, $l < 10000000$, and under the constraints of the problem, it turns out that $r < 23000000$. Thus, we can generate the first 23000000 terms of the sequence $f(1), f(2), \cdots$, and we can get the answer by two-pointers method.

The case with $f(l) \geq 8$

Let t be an integer between 1 and $floor(S/8)$. It turns out that, except for special cases, there is exactly one pair (l, r) such that $f(l) \geq 8$, $f(l) + \cdots + f(r) = S$, and $r - l + 1 = t$ (i.e., there are exactly t terms in the chosen range). This is because there is exactly one interval that starts with $t - (S \mod t)$ occurrences of $\lfloor S/t \rfloor$-digit integers followed by $S \mod t$ occurrences of $\lceil S/t \rceil$-digit integers. Thus, we should add $floor(S/8)$ to the answer.

The only exception happens when t is a divisor of S. In this case, there are $N_t - S/t + 1$ such intervals. To handle this, we should add $N_t - S/t$ to the answer for each divisor t between 1 and $floor(S/8)$.

Exercise: can you compute the answer if $S \leq 10^{12}$?

ABC086 / ARC089 解説

sigma425

For International Readers: English editorial starts on page 5.

A: Product

整数 a, b を読み込み、$a \times b$ を計算し、2 で割ったあまりを計算すればいいです。

```
int main(){
        int a,b;
        cin>>a>>b;
        int c = a * b;
        if(c%2 == 0) puts("Even");
        else puts("Odd");
        return 0;
}
```

B: 1 21

整数として読み込むよりも、文字列として読み込んで連結したものを整数に直すのが簡単かもしれません。C++ には標準で文字列を整数に直す stoi 関数があります。この整数は高々 100100 なので、平方数かどうかは、1×1, 2×2, .. , 1000×1000 くらいまでのどれかと一致するかループを回して確かめれば良いです。

```
int main(){
        string s,t;
        cin>>s>>t;
        int x = stoi(s+t);
        for(int i=1;i<=1000;i++){
                if(i*i == x){
                        puts("Yes");
                        return 0;
                }
        }
        puts("No");
}
```

C: Traveling

$t_0 = 0, (x_0, y_0) = (0, 0)$ と置きます。各 0 以上 $N-1$ 以下の i について、時刻 t_i に場所 (x_i, y_i) にいた後、時刻 t_{i+1} に場所 (x_{i+1}, y_{i+1}) にいることができるか、を判定して、どれか一つでも不可能なら No を、全て可能なら Yes を出力すれば良いです。

この判定は、$t := t_{i+1} - t_i$, $d := abs(x_i - x_{i+1}) + abs(y_i - y_{i+1})$ とおくと、$d \leq t \wedge t\%2 = d\%2$ で判定できます。($d \leq t$ でないと不可能なのは明らか、毎秒 $x + y$ の偶奇は変わるので 2 で割ったあまりは一致する必要がある。 逆に d, t がこれを満たすなら実際に条件のように動くことが可能)

D: Checker

マス (x, y) が白 と マス $(x, y + K)$ が黒 は同値なので、$(x, y,' W')$ という入力を $(x, y + K,' B')$ に置き換えても条件は同じです。これで入力の c_i を全て'B' に変換することが出来ます。

マス (x, y) の色とマス $(x, y + 2K), (x + 2K, y)$ の色は一致します。なので、入力の x, y を $x\%2K, y\%2K$ で置き換えても答えは変わりません。すると $0 \leq x, y < 2K$ と変換することが出来ます。

市松模様の黒い部分の左下のマスのうちひとつを定めるとマスの塗られ方全体が定まります。ここで、定めるマスは $[0, 2K) \times [0, 2K)$ の中にあると仮定して問題ありません (なぜなら、どんな市松の塗られ方でも黒い部分の左下のマスのうちひとつはこの中にあるため)。なので、左下マスを $(2K)^2$ すべて試すと、左下を決めれば各条件が満たされるかどうかは $O(1)$ で判定できるので、全体で $O(NK^2)$ になります。

ここから計算量を落とすには、条件 i を満たせるような左下マスの集合が $K * K$ の正方形区間のようになっていることに着目して、その範囲に $+1$ を足し、全体での max を取ります。ただし毎回範囲に $+1$ をしていては遅いので、差分を取って左下と右上に $+1$, 左上と右下に -1 を足しておき、あとで累積和を取ることで $O(N + K^2)$ で同じことが可能です。

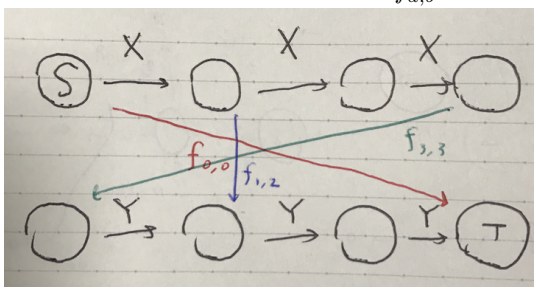
E: GraphXY

まずグラフ,S, T が与えられた時に,$X = x, Y = y$ の時の最短距離 $D_{x,y}$ がどういう振る舞いをするかを考えます。0 以上の整数の組 (a, b) に対して、「X の辺をちょうど a 回,Y の辺をちょうど b 回使った時に S から T への (整数が書いてある辺の重みのみを考慮した) 最短距離」 を $f_{a,b}$ とおきます。

すると、

$$D_{x,y} = \min_{a,b \geq 0} xa + yb + f_{a,b}$$

逆に次のようなグラフを考えれば、$f_{a,b}$ を 0 以上の整数値に自由に決めて上のように $D_{x,y}$ を設定できます。



なので、$f_{a,b}$ を決めることでこの D を入力の d と一致させることが目標です。

入力 d が与えられた時,d の値の範囲が高々 100 であることから、a や b が 100 より大きいと $xa + yb + f_{a,b}$ が 100 より大きくなってしまい D を d と一致させる役には立ちません。

よって $0 \leq a, b \leq 100$ をみたす整数の組 (a, b) に対して $f_{a,b}$ を決めることにします。$f_{a,b}$ は、任意の $1 \leq x \leq A, , 1 \leq y \leq B$ に対し $d_{x,y} \leq xa + yb + f_{a,b}$ を満たす、という条件のもとで最も小さくする (ただし 0 以上である必要はある) のがベストです。

なので、

$$f_{a,b} = \max(0, \max_{0 \leq a,b \leq 100}(d_{x,y} - xa + yb))$$

として、これで計算した D と入力の d が一致していればこれを出力、していなければ Impossible です。

F: ColoringBalls

以下では赤、青、白色のボールをそれぞれ R,B,W で表します。まずボールの列が与えられた時に、それが実現できるか判定することを考えます。例えば”WRRBRBBWWRRRRWRBBWWRRBBRR” というボール列を考えます。まず’W’ で区切って R,B からなる列の (多重) 集合に変換します。

{“RRBRBB”, “RRRR”, “RBB”, “RRBBRR”}

次に、RR を R に、BB を B にする操作をできるだけ繰り返します (つまり連続している R(B) をひとつの R(B) に置き換える操作をします)。

{“RBRB”, “R”, “RB”, “RBR”}

さらに、これらが次のうちどのグループに属しているかをカウントします：

グループ 1：“R”
グループ 2：“B”, “RB”, “BR”, “RBR”
グループ 3：“BRB”, “RBRB”, “BRBR”, “RBRBR”
グループ 4：“BRBRB”, “RBRBRB”, “BRBRBR”, “RBRBRBR”
:
(以下同様)

グループ id を降順にならべた配列を f とおきます。

上の例だと $f = [3, 2, 2, 1]$ です。

重要なのは、この配列 f が一緒になるようなどんなボール列も実現できるかどうかは変わらないということです。以下説明します。

まず列の集合に変換した結果が同じなら (つまり列の順番を並び替えたり無駄な W を挿入したりするだけ) 結果も明らかに同じなのでここは OK です。

次に連続する R の個数を増やしたりしてもそういう塗り方が出来るかは変わらないのでこれも OK です。

最後にグループ分けですが、赤で塗る操作を r, 青で塗る操作を b, どちらでもいいので塗る操作を?とかくと、

グループ 1 は r で実現できる
グループ 2 は rb で実現できる
グループ 3 は rb? で実現できる
グループ 4 は rb?? で実現できる
グループ 5 は rb??? で実現できる
:

ということがわかります。つまり同グループに属していれば操作列によって実現可能かどうかは一緒です。

最終的に、配列 f が同じならば、実現可能性は同じであることがわかりました。

配列 f となるようなボール列が実現可能かどうかは次のように判定できます。操作列を s とおきます。

f の各要素 x に、操作列の部分列であってグループ x を実現できるもの を割り当てられるかの判定をすれば良いです。(操作列の各文字はひとつにしか割り当てられません)

実は最適な割当は以下の貪欲法で構成することが出来ます (以下の貪欲で構成できなければ不可能です)。

はじめに、s の要素を左から見ていって、k 個目にでてきた r を f の k 個目の要素に割り当てます。これが出てきた場所を r_k とおきます。

次に、f の要素を左から見ていって、もし値が 2 以上なら、r_k より右にあってまだ割り当てていないうち最も左の b を割り当てます。これが出てきた場所を b_k とおきます。

最後に、f の要素を左から見ていって、値 x が 3 以上なら、b_k より右にあってまだ割り当てられていないもののうち左にある $x-2$ 個 (r,b どちらでもよい) を割り当てます。

これらはそれぞれグループ分け部分で説明した必要な文字列のうち、r,b,? の部分に対応するものをとる操作です。

この貪欲の証明は、

i) r を左から貪欲にとってよい

ii) b を (対応する r より右で) 左から貪欲にとってよい

iii) グループ番号が大きいものをできるだけ左に対応させるのがよい

iv) 残りの?に対応する部分も、対応する b より右で左から貪欲にとってよい

の 4 つを示せば完了します。省略しますがこれは証明できます。

この貪欲は $O(NK)$ で走ります。

ところで $N=70$ でありうるボールの色の列から生成される f は 418662 通りしか無いことがわかります。なので、実際に f を生成し、上の貪欲アルゴリズムで可能かどうか判定し、可能なら,f を生成するボール列が何通りあるかを求めて (これは適切な combination をかけ合わせるとできます) 十分高速に答えが求まります。

N, K の多項式時間で走る DP で計算することも出来ます。f にある 1 の個数、1 以外の個数 を全探索して、そのあと f の要素を小さい方から追加していく (1 を何個追加するか、2 を何個追加するか・・・ を順番に決めていく) のですが、「今追加しようとしている値」「これまで追加した値の総和」「これまで何個値を追加したか」を DP のキーに持てば可能です。

ABC086 / ARC089 Editorial

sigma425

A: Product

```
int main(){
        int a,b;
        cin>>a>>b;
        int c = a * b;
        if(c%2 == 0) puts("Even");
        else puts("Odd");
        return 0;
}
```

B: 1 21

```
int main(){
        string s,t;
        cin>>s>>t;
        int x = stoi(s+t);
        for(int i=1;i<=1000;i++){
                if(i*i == x){
                        puts("Yes");
                        return 0;
                }
        }
        puts("No");
}
```

C: Traveling

Let $t_0 = 0, (x_0, y_0) = (0, 0)$. For each i between 0 and $N-1$, we check the following: can we reach (x_{i+1}, y_{i+1}) at time t_{i+1} if we start (x_i, y_i) at time t_i? If the answer is no for at least one i, we print "No", otherwise we print "Yes".

We can check this as follows. Let $t := t_{i+1} - t_i$, $d := abs(x_i - x_{i+1}) + abs(y_i - y_{i+1})$. If $d \leq t$ and $t\%2 = d\%2$, the answer is yes, otherwise the answer is no.

D: Checker

The cell (x, y) is white iff the cell $(x, y + K)$ is black. Thus, we can replace a query $(x, y,' W')$ with $(x, y + K,' B')$. Assume that all c_i are 'B's.

The colors of (x, y) and $(x, y+2K), (x+2K, y)$ are same. Thus, we can replace a query $(x, y,' B')$ with $(x\%2K, y\%2K,' B')$. Assume that $0 \leq x, y < 2K$ for all queries.

If we fix a black cell at the lower-left corner of a $K \times K$ block, we can uniquely determine the colors of all cells. Here, we can assume that the black cell is inside $[0, 2K) \times [0, 2K)$. Therefore, by trying all $(2K)^2$ possibilities, and by checking the colors of all N cells that appear in the queries, we get an $O(NK^2)$ solution.

To make it faster, after we fix the black cell at the corner, we should compute the number of queried cells inside certain rectangles in $O(1)$. This can be done by pre-computing rectangle sums. Now the solution becomes $O(N + K^2)$.

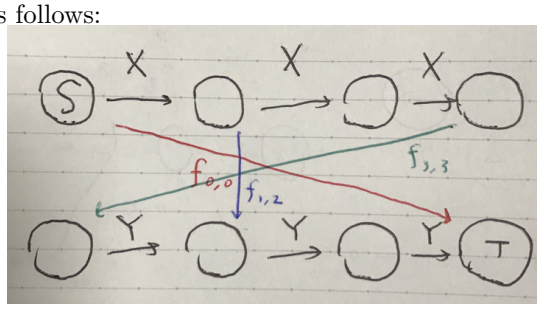
E: GraphXY

Let's fix a graph (and S, T). How does $D_{x,y}$ (the shortest path when $X = x, Y = y$) behave?

For a pair of non-negative integers (a, b), define $f(a, b)$ as the smallest possible sum of integers on an $S - T$ path that contains exactly a edges with 'X' and exactly b edges with 'Y'. Then,

$$D_{x,y} = \min_{a,b \geq 0} xa + yb + f_{a,b}$$

On the other hand, if D satisfies the equation above (for some $f_{a,b}$), we can construct a desired graph as follows:



Thus, our objective is to find $f_{a,b}$ that matches with given d.

Since each integer in d is at most d, we don't need to care the values of $f(a, b)$ for $a > 100$ or $b > 100$ because $xa + yb + f_{a,b} > 100$ in this case. Thus, we want to decide $f_{a,b}$ for each pair (a, b) such that $0 \leq a, b \leq 100$. We should minimize the value of $f_{a,b}$ under the constraint that $d_{x,y} \leq xa + yb + f_{a,b}$ for all $1 \leq x \leq A, , 1 \leq y \leq B$.

Therefore, define

$$f_{a,b} = \max(0, \max_{0 \leq a,b \leq 100} (d_{x,y} - xa + yb))$$

and compute D. If it matches with the input, we should print the graph described above. Otherwise, the answer is "Impossible".

F: ColoringBalls

We start with a slow solution: generate all 3^N sequences of balls of length N, and for each sequence, check if the sequence can be constructed by given operations.

However, we don't need to check all 3^N sequences. Some sequences turn out to be equivalent regardless of the operations.

Let R,B,W denote red, blue, and white, respectively. As an example, we use a sequence of balls "WRRBRBBWWRRRRWRBBWWRRBBRR". We want to transform it into a "canonlcal form" (while keeping the string equivalent to the original string).

First, obviously, we can split the string by 'W':

{"RRBRBB", "RRRR", "RBB", "RRBBRR"}

Next, we can replace "RR" with "R" and "BB" with "B":

{"RBRB", "R", "RB", "RBR"}

Next, replace each string with its group number:

Group 1 : "R"

Group 2 : "B", "RB", "BR", "RBR"

Group 3 : "BRB", "RBRB", "BRBR", "RBRBR"

Group 4 : "BRBRB", "RBRBRB", "BRBRBR", "RBRBRBR"

:

Here, the groups represent strings that can be obtained by the following sequences of operations:

Group 1 : r

Group 2 : rb

Group 3 : rb?

Group 4 : rb??

Group 5 : rb???

:

Finally, we can arbitrary shuffle the elements of the array of integers we get. In this case, we get $f = [3, 2, 2, 1]$.

It turns out that the number of distinct final arrays we get is $O(partition_number(N) * N)$. In particular, when $N = 70$, there are only 418662 possible fs.

Now, our solution is as follows. We generate all 418662 possible fs. For each f, check if it can be obtained by the given sequence of operations. If yes, count the number of sequences of balls that lead to f (this is simply the product of some binomial coefficients), and add it to the answer.

How to check if we can get f with the given sequence of operations?

Let s be the sequence of operations. For each integer x in f, we want to assign a subsequence of s that matches with x (for example, if $x = 4$, the subsequence must be "rb??"). Here, the subsequences must be disjoint.

We can assign subsequences greedily as follows:

- Suppose that f is sorted in non-decreasing order.
- For each k, assign the k-th 'r' in s (suppose that this is the r_k-th character in s) to $f[k]$.
- For each $f[k] = x$, (we check elements of f from left to right) if $x \geq 2$, assign the leftmost 'b' in s to the right of the r_k-th character of s to $f[k]$. Suppose that this is the b_k-th character.
- Finally, for each $f[k] = x$, if $x \geq 3$, we assign the leftmost $x - 2$ unused letters of s to the right of b_k-th to $f[k]$.

We omit the proof here, but it is relatively straightforward.

The solution above is fast enough to get accepted.

If you want a polynomial solution, we can do DP. We first fix the total number of 1s in f and the number of elements in f. Then, we add integers to f in the increasing order. Here, the keys of DP is (the ineger we will add next), (the sum of all integers we've added so far), and (the number of integers we've added so far). Since it's complicated and not significantly faster than the brute force above, we omit details.

ARC 088/ABC 083 解説

DEGwer

2017/12/23

A: Libra

整数 A, B, C, D を読み込み、$A + B$ と $C + D$ の大小比較をすればよいです。

```
#include<stdio.h>
int main()
{
        int a, b, c, d;
        scanf("%d%d%d%d", &a, &b, &c, &d);
        if (a + b > c + d)printf("Left\n");
        else if (a + b == c + d)printf("Balanced\n");
        else printf("Right\n");
}
```

B: Some Sums

N 以下の整数の各桁の和を求め、適切に場合分けをして足し合わせてやればよいです。

```
#include<stdio.h>
int main()
{
        int n, a, b;
        scanf("%d%d%d", &n, &a, &b);
        int r = 0;
        for (int i = 1; i <= n; i++)
        {
                int c = 0, t = i;
                for (int j = 0; j < 5; j++)c += t % 10, t /= 10;
                if (a <= c&&c <= b)r += i;
        }
        printf("%d\n", r);
}
```

C: Multiple Gift

作る数列を $A_1, ..., A_{|A|}$ とします。条件より $X \leq A_1$ であり、またすべての i に対し $2A_i \leq A_{i+1}$ となります。よって、$A_i \geq X \times 2^{i-1}$ がわかります。

逆に、$A_i = X \times 2^{i-1}$ とすれば、これは条件を満たします。すべての i に対し A_i の最小値を達成できているので、条件を満たす数列のうちこのようにして作られるもののみを考慮すればよく、時間計算量 $O(\log \frac{B}{A})$ でこの問題を解くことができます。

D: Wide Flip

与えられる文字列の長さを n とします。

k 文字目と $k+1$ 文字目が異なるとします。このとき、すべての文字を 0 にするためには、k 文字目と $k+1$ 文字目の片方のみを書き換えるような操作を 1 回以上行わなければなりません。すなわち、$max(k, n-k)$ 文字以下の連続する部分を書き換えるような操作が必要となります。

S の、k 文字目と $k+1$ 文字目が異なるような箇所のうちの、$max(k, n-k)$ の最小値を $T(S)$ とします。$K \geq T(S)$ のとき、T の値の最小値をとる k をとり、$k = max(k, n-k)$ なら最初 k 文字を、そうでないなら最後 $n-k$ 文字を書きかえれば、連続する文字が異なるような箇所の個数を、$T(S)$ を減らさずに 1 減らすことができます。また、$T(S) = 0$ のとき、もし全文字が 0 ならこれが目的の文字列であり、そうでないなら全体に対して操作を行えば目的の文字列を作ることができます。

よって答えは最初の文字列 S における $T(S)$ の値で、これは $O(n)$ 時間で求められます。

E: Papple Sort

まず、奇数回しか出てこない文字が 2 つ以上ある場合、回文を作ることはできません。以下、それ以外の場合を考えます。

最終的に (最小の交換回数で) 作る回文を T とし、S で添え字 i の文字が T では添え字 p_i となるとします。

同じ文字同士を入れ替えるのは無意味なので、各文字に対してその文字の出現回数を求めれば、文字列中のすべての添え字の ($|S|$ 個の) 文字に対し、その文字が最終的に T の左半分に属するか、ちょうど真ん中の文字となるか、右半分に属するかは一意に定めることができます。

上述の 3 つの添え字集合をそれぞれ A, B, C とします。A の添え字の文字を順序を保ったまま左半分に、B の添え字の文字を真ん中に、C の添え字の文字を順序を保ったまま右半分に移動して作った文字列を X とします。このとき、これら 3 種類の添え字の文字同士の swap が起こりますが、T に関する上述の条件より、$i < j$ かつ $p_i > p_j$ となるような文字 S_i, S_j の swap のみが起こることがわかります。よって (バブルソートの交換回数とはそのような添え字 (i, j) の組の個数に他ならないので) S から T への変換での交換回数は、S から X への変換での交換回数と X から T への変換での交換回数の合計と等しくなることが分かります。

さて、X から T に変換するとき、A 内の要素同士、または C 内の要素同士の swap のみが起こります。もし A 内の要素同士の swap が起きる場合、その代わりに C 内での対応する要素同士を swap することにしても、最終的な文字列が回文になるという条件は依然満たされます。よって、A 内の要素同士は swap しないとして問題ありません。

A 内の要素の順番が定まれば C 内の要素の順番も定まるので、結局 p_i の値がすべての i に対して一意に定められたことになります。あとは、順列 p のバブルソートの交換回数を、BIT などを用いて数えてやればよいです。時間計算量は $O(|S| \log |S|)$ です。

F: Christmas Tree

この問題は、以下の条件を満たす辞書順最小の (A, B) を求める問題だと言い換えることができます。

- 木の辺たちを A 色で塗り分ける。このとき、同じ色が塗られた辺の集合は、長さ B 以下のパスをなしている必要がある。

まず、A の最小値を求めましょう。次数奇数の頂点に対し、その点を端点とするようなパスが 1 つ以上必要です。逆に、次数奇数の頂点間のパスをとって取り除いていくことを考えれば、次数奇数の頂点の個数の半分の個数のパスがあれば十分であることもわかります。よって、A の最小値は、次数奇数の頂点の個数の半分です。

B の最小値を求めるために、二分探索を考えましょう。B の値を固定し、そのような塗りわけが存在するかどうかを求める DP を考えましょう。

まず、次数 1 の頂点を根として入力の木を根付き木にします。便宜上、根からも上向きに辺が出ているものとして扱います。各頂点 v に対し、DP$[v]$ を、v を根とする部分木に属するパスの長さがすべて B 以下となるように塗り分けるときの、v から親方向にのびるパスの長さの最小値 とします。

この DP 値はどのように求めればよいでしょうか。v の子を $u_1, ..., u_k$ とし、DP$[u_i]$ の小さい順に並べ替え、改めて添え字を ($u_1, ..., u_k$ と) つけなおしておきます。条件より、

- k が奇数のとき、u_i たちから親方向に伸びるパスからペアを $\frac{k-1}{2}$ 個作ってつなぎ合わせ、残ったパスを v から親方向に伸ばすことにする
- k が偶数の時、u_i たちから親方向に伸びるパスからからペアを $\frac{k}{2}$ 個作ってつなぎ合わせ、v から新しいパスを親方向に伸ばすことにする、またはペアを $\frac{k}{2}-1$ 個作ってつなぎ合わせ、残った 2 本のパスのうち 1 本の端点を v にし、もう 1 本のパスを v から親方向に伸ばすことにする

する必要があります。

パスたちをペアにするときは、各ペアについてその合計長さが B を超えないようにする必要があります。すなわち、合計の長さの最大値を最小にするようなペアの組み方を考え、そのときの最大長が B 以下かどうか判定すればよいです。このとき、greedy な考察により、$2m$ 本のパスを m 個のペアに分けるとき、すべての x に対し、x 番目に長いパスと x 番目に短いパスを組み合わせるのが最適であることがわかるので、この判定は $O(m)$ 時間で行うことができます。

上の場合分けでの前者の場合、1 本のパスを選んで取り除いた後に、パスをペアにしていく必要があります。取り除くパスは、DP 値の定義より、なるべく短いものを選びたいです。これは、「どのパスを取り除くべきか」で二分探索を行うことで最適化することができます。

後者の場合、ペアを $\frac{k}{2}$ 個作る場合は上述のアルゴリズムが動作します。そうでない場合、端点を v にするパスは最も長いパスを選ぶのが良いです。このように選べば、あとは k が奇数の場合と同じ問題に帰着されます。

以上より、子を k 個もつ頂点での DP 値の更新が $O(k \log k)$ 時間でできることが分かりました。最後に、(次数 1 の頂点を根に選んだことに注意すれば) 根から親方向に伸びるパスの長さを評価することで外側の二分探索の評価関数が正しく動作し、よって全体で $O(N \log^2 N)$ 時間でこの問題を解くことができました。

ARC 088/ABC 083 Editorial

DEGwer

2017/12/23

A: Libra

```
#include<stdio.h>
int main()
{
        int a, b, c, d;
        scanf("%d%d%d%d", &a, &b, &c, &d);
        if (a + b > c + d)printf("Left\n");
        else if (a + b == c + d)printf("Balanced\n");
        else printf("Right\n");
}
```

B: Some Sums

```
#include<stdio.h>
int main()
{
        int n, a, b;
        scanf("%d%d%d", &n, &a, &b);
        int r = 0;
        for (int i = 1; i <= n; i++)
        {
                int c = 0, t = i;
                for (int j = 0; j < 5; j++)c += t % 10, t /= 10;
                if (a <= c&&c <= b)r += i;
        }
        printf("%d\n", r);
}
```

C: Multiple Gift

Let $A_1, ..., A_{|A|}$ be the sequence. $X \leq A_1$ must be satisfied, and for each i, $2A_i \leq A_{i+1}$ must be satisfied. Thus, we get $A_i \geq X \times 2^{i-1}$.

Therefore, the maximum length can be achieved when $A_i = X \times 2^{i-1}$. (For all i, the lower bound of A_i is achieved by this sequence.)

This solution works in $O(\log \frac{B}{A})$ time.

D: Wide Flip

Let n be the length of the string.

If the k-th and the $k+1$-th (1-based) characters of the string are different, in order to achieve the goal, we must perform at least one operation that flips exactly one of the two characters. In this operation, we flip at most $max(k, n-k)$ consecutive characters.

Let $T(S)$ be the minimum vaule of $max(k, n-k)$ such that the k-th and the $k+1$-th characters of S are different. As we see above, K must be at most $T(S)$.

On the other hand, we can prove that when $K = T(S)$, we can achieve the goal.

- If $i <= N - K$, we can flip a single character S_i by flipping $[i, N]$ and $[i+1, N]$.
- Similarly, if $i > K$, we can flip a single character S_i.
- By the definition of $T(S)$, all characters between the $N-K+1$-th and the K-th are the same.

Thus, by repeating the first two types of operations, we can make all characters equal, and (possibly by performing the third operation once) we can achieve the goal.

Therefore, the answer is $T(S)$, and it can be computed in $O(n)$ time.

E: Papple Sort

If two or more letters appear odd number of times, we can't make a palindrome. For simplicity, suppose that each letter appears even number of times (when a letter appears odd number of times, we know which letter will go to the center of the final string, and the solution doesn't change a lot.)

Let n be the length of the string. Let T be the final palindrome we make (by performing the minimum possible number of operations), and let T_{p_i} be the destination of S_i.

Consider a particular letter c, and suppose that it appears as i_1-th, i_2-th, ..., i_{2k}-th letters of S. Since it makes no sense to swap the same letters, the relative positions of these letters won't change. Thus, we know that i_1 and i_{2k} will be paired in T (i.e., $p_{i_1} + p_{i_{2k}} = N + 1$), i_2 and i_{2k-1} will be paired, and so on. Therefore, we know which characters will be paired in T, and we can assume that S contains $N/2$ different types of characters, and each character appears exactly twice.

Now, consider two particular types of letters in S (call it 'A', 'B').

There are six possibilities for their relative positions. For example, they can appear as

...A...A...B...B...

If the pair of A becomes "outer" pair and the pair of B becomes "inner" pair in T, we need two swaps among these characters. If the pair of B becomes "outer" pair and the pair of A becomes "inner" pair in T, we need two swaps among these characters. Thus, it doesn't matter.

By similar observations, it turns out that when they appear as

...A...B...B...A...

A should be "outer" and B should be "inner", and if they appear as

...B...A...A...B...

B should be "outer" and A should be "inner". In other cases it doesn't matter.

Thus, when we sort the pairs, we should make sure that these properties hold. For example, when we compare two pairs at positions (l_1, r_1) and (l_2, r_2), we make (l_1, r_1) "outer" if $l_1 < l_2$.

Now, we know which characters in A will go to which position in T (i.e. we know the values of p_i), and we can compute the inversion number in $O(|S| \log |S|)$.

F: Christmas Tree

We want to paint the edges of the tree using A colors. The set of edges of a specific color must form a path of length at most B.

What is the minimum possible value of A? Suppose that the tree has O vertices with odd degree. Since an odd vertex must be an endpoint of at least one path, we need at least $O/2$ paths. On the other hand, by repeatedly removing a path that connects two odd vertices, we can achieve $A = O/2$.

Now, we want to minimize B under the condition that $A = O/2$. Let's do a binary search on B. Now we want to check if a valid coloring exists for a fixed B.

Choose an arbitrary leaf of the tree, and make it the root. For convenience, we attach an edge to the root (an edge between the root and "the root's parent").

Let's define $DP[v]$ as follows. Consider a subtree rooted at v, plus an edge between v and v's parent (we call it e_v). We want to color each edge inside this subtree, such that

- The length of each color must be at most B (of course, each color must form a path).
- We should minimize the number of paths: each odd vertex is an endpoint of exactly one path, and each even vertex is an endpoint of no path.
- Under these conditions, we want to minimize the length of the color that contains e_v. Let $DP[v]$ be this length.

How do we compute $DP[v]$? Suppose that a vertex v has k children $u_1, ..., u_k$, and assume that $DP[u_1] \leq DP[u_2] \leq ....$

If k is odd, we need to make $\frac{k-1}{2}$ pairs among the children, and pair the only remaining child (call it u_r) with e_v. Obviously, we want to minimize r. Let's do a binary search again on r. For a fixed r, we want to check if the values $DP[u_1], \ldots, DP[u_{r-1}], DP[u_{r+1}], \ldots, DP[u_k]$ can be divided into pairs whose sums are at most B. This can be done greedily by pairing the x-th largest value and the x-th smallest value. Then, $DP[v] = DP[u_r] + 1$.

If k is even, add an imaginary child u_0 such that $DP[u_0] = 0$, and it reduces to the case with odd k.

If we don't get contradictions and $dp[root] \leq B + 1$, the chosen value of B is valid.

Since we need $O(k \log k)$ time to handle a vertex with k children, we need $O(N \log N)$ per step, and in total this solution works in $O(N \log^2 N)$ time.

ABC #082 / ARC #087 Editorial

writer : sugim48

2017 年 12 月 16 日

For International Readers: English editorial starts on page 5.

A: Round Up the Mean

- C++ のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872729
- Java のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872732
- Python 3 のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872734

B: Two Anagrams

s の文字を昇順にソートし，t の文字を降順にソートした後，$s < t$ を判定すればよいです．

- C++ のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872811
- Java のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872812
- Python 3 のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872814

C: Good Sequence

それぞれの正の整数 x について，a 中の値 x の個数を n_x とします．$n_x < x$ の場合，a 中の値 x はすべて取り除かなければならないので，n_x 回の操作が必要です．$n_x \geq x$ の場合，a 中の値 x の個数を x にすればよいので，$n_x - x$ 回の操作が必要です．すべての x についてこれらの操作回数を合計したものが答えです．

実際には，すべての x ではなく，a 中の値 x のみ調べればよいです．x の値は最大で 10^9 と大きいので，普通の配列の代わりに連想配列を用いて各 n_x を数えればよいです．

- C++ のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872852

D: FT Robot

s の長さを N とし，目標の座標を (x_t, y_t) とします．

まず思いつく解法は，$\mathrm{dp}_{i,x,y,k} :=$ (i 文字目の命令後に座標 (x, y) に向き $k \in \{$ 左, 上, 右, 下 $\}$ でいられるか)

と定義し，DP をする解法です．しかし，この DP の状態数は $\mathrm{O}(N^3)$ で，遷移は $\mathrm{O}(1)$ なので，全体の時間計算量は $\mathrm{O}(N^3)$ となり，TLE してしまいます．

ロボットの動き方をよく観察することで，DP の状態数を減らしましょう．命令列を `T` を区切り文字として分割したとき，i 番目の区間に含まれる `F` の個数を d_i とします．このとき，ロボットの動き方は

- 右に距離 d_1 だけ動く
- 上または下の好きな向きに距離 d_2 だけ動く
- 左または右の好きな向きに距離 d_3 だけ動く
- 上または下の好きな向きに距離 d_4 だけ動く
- 左または右の好きな向きに距離 d_5 だけ動く
- 以下同様

となっています．よって，x 軸方向の移動と y 軸方向の移動を独立に考えることができます．例えば，x 軸方向の移動については，

- 右に距離 d_1 だけ動く
- 左または右の好きな向きに距離 d_3 だけ動く
- 左または右の好きな向きに距離 d_5 だけ動く
- 以下同様

という移動を終えた後，x 座標が x_t になっているようにできるか判定すればよいです．

これは次のような DP で計算できます．

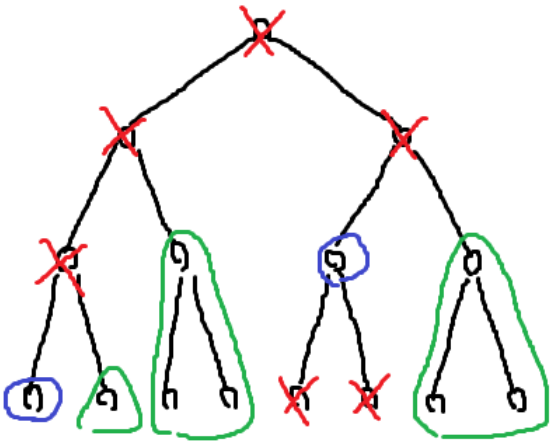
- 定義: $\mathrm{dp}_{i,x} :=$ (i 回目の x 軸方向の移動の後に x 座標が x になっているようにできるか).
- 初期化: 1 回目の x 軸方向の移動距離を d_1 とすると，$\mathrm{dp}_{0,d_1} = \mathrm{true}$.
- 漸化式: i ($i \geq 2$) 回目の x 軸方向の移動距離を d_i とすると，$\mathrm{dp}_{i,x} = \mathrm{dp}_{i-1,x-d_i} \vee \mathrm{dp}_{i-1,x+d_i}$.

この DP の状態数は $\mathrm{O}(N^2)$ で，遷移は $\mathrm{O}(1)$ なので，全体の時間計算量は $\mathrm{O}(N^2)$ となり，十分に高速です．y 軸方向の移動についても，同様の DP で計算できます．

E: Prefix-free Game

長さ L 以下の `0`, `1` のみからなる文字列全体からトライ木を作ると，これは高さ $L+1$ の完全二分木となります．ただし，完全二分木の高さは，根から葉までのパスに含まれる頂点の個数と定義します．この完全二分木を用いると，ゲームは次のように言い換えられます．

> 高さ $L+1$ の完全二分木がある．最初，いくつかの頂点には駒が置かれている．ただし，どの駒どうしも先祖-子孫の関係ではない．Alice と Bob は交互に，駒のない頂点をひとつ選んで駒を置く．ただし，どの駒どうしも先祖-子孫の関係ではないという条件を保つようにする．先に駒を置けなくなった方が負けである．



上図は初期状態の例 ($L = 3$) です．青いマル印は駒を表しており，赤いバツ印は駒を置けない頂点を表しています．このとき，駒を置ける頂点の集合は，いくつかの完全二分木 (緑の三角形) の和になっています．これらの完全二分木は先祖-子孫の関係ではないので，独立に考えることができます．また，完全二分木のある頂点に駒を置くと，その完全二分木はいくつかの完全二分木に分裂します．具体的には，高さ i の完全二分木の深さ j $(0 \leq j \leq i-1)$ の頂点に駒を置くと，高さ $i-1, i-2, ..., i-j$ の完全二分木 1 個ずつに分裂します．

以上より，ゲームはさらに次のように言い換えられます．

> 最初，いくつかのアイテムがある．各アイテムには正整数のレベルが設定されている．Alice と Bob は交互に次の操作を行う．
>
> - アイテムを 1 個選ぶ．選んだアイテムのレベルを i とする．選んだアイテムを取り除き，整数 k $(1 \leq k \leq i)$ を選んでレベル $i-1, i-2, ..., k$ のアイテムを 1 個ずつ追加する．
>
> 先に操作を行えなくなった方が負けである．

このようなゲームの勝敗判定には Grundy 数が有用です．レベル i $(i \geq 1)$ のアイテムの Grundy 数を g_i とします．すると，g_i は

- 0
- g_{i-1}
- $g_{i-1} \oplus g_{i-2}$
- $\vdots$
- $g_{i-1} \oplus g_{i-2} \oplus \cdots \oplus g_1$

に含まれない最小の非負整数です．実験すると，$(g_1, g_2, g_3, g_4, g_5, g_6, g_7, g_8, \ldots) = (1, 2, 1, 4, 1, 2, 1, 8, \ldots)$ となっています．実験結果から分かるように，$g_i =$ (i を割り切る最大の 2 の冪) であることが示せます．各アイテムの Grundy 数の総 XOR が 0 でないならば Alice が勝ち，0 ならば Bob が勝ちます．

元の問題に戻ると，レベル i のアイテムは高さ i の平衡二分木に対応します．よって，初期状態におけるそれぞれの完全二分木の高さを求められればよいことになります．これは，入力文字列から作られるトライ木を辿り，子の個数がちょうど 1 個の頂点を見ていくことで可能です．

F: Squirrel Migration

まず，移動距離の総和を最大化するような移動方法がどのようなものか考えます．そのために，移動距離の総和の最大値がどのような値か考えます．まずは，移動距離の総和の上限を見積もってみましょう．ある移動方法に対して，各辺 e について e を通過したリスの個数を s_e とします．すると，移動距離の総和は $\sum_e s_e$ となります．ここで，辺 $e=(v,w)$ の v 側の頂点数を n_v とし，w 側の頂点数を n_w とすると，$s_e \le 2\min\{n_v, n_w\}$ です．よって，移動距離の総和の上限はこれらの総和となります．実は，この上限を達成するような移動方法が常に存在します．それは次のような移動方法です．

木の重心に注目します．重心は 1 個または 2 個です．重心が 1 個の場合，それを c とします．すると，どのリスの移動経路にも c が含まれるとき，かつそのときに限り，移動距離の総和は上限を達成することが分かります．また，重心が 2 個の場合，それらを c_1, c_2 とします．すると，どのリスの移動経路にも 辺 (c_1, c_2) が含まれるとき，かつそのときに限り，移動距離の総和は上限を達成することが分かります．

以上の移動方法を数え上げます．重心が 2 個の場合の方が簡単なので，先に考えます．重心を c_1, c_2 とします．さらに，c_1 側の部分木を T_1 とし，c_2 側の部分木を T_2 とします．このとき，木の頂点数 N は偶数であり，T_1, T_2 の頂点数はともに $N/2$ です．T_1 の各リスは，T_2 のどの頂点へも移動できます．同様に，T_2 の各リスは，T_1 のどの頂点へも移動できます．よって，移動方法は $(N/2)! \times (N/2)!$ 通りです．

次に，重心が 1 個の場合を考えます．重心を c とします．さらに，c の周りの部分木を $T_1, T_2, ..., T_K$ とします．このとき，c のリスはどの頂点へも移動できます．また，各 i について，T_i のリスは T_i 以外のどの頂点へも移動できます．この移動方法を $\mathrm{O}(N^2)$ 時間で直接数え上げるのは難しいです．

そこで，包除原理を用います．$T := T_1 \cup T_2 \cup \cdots \cup T_K$ とします (すなわち，T は木の頂点集合から c を除いたもの)．すると，答えは $\sum_{S \subseteq T} (-1)^{|S|} f_S$ となります．ただし，$f_S :=$ (「S 中の各リスは自分の部分木の頂点へ移動する」という条件を満たす移動方法の通り数) です．各 $S_1 \subseteq T_1$, ..., $S_K \subseteq T_K$ について，次が成り立ちます．

$$f_{S_1 \cup \cdots \cup S_K} = \left(N - \sum_{k=1}^{K} |S_k|\right)! \prod_{k=1}^{K} \binom{|T_k|}{|S_k|}^2 |S_k|!$$

よって，答えは

$$\sum_{S_1 \subseteq T_1, \ldots, S_K \subseteq T_K} (-1)^{\sum_{k=1}^{K} |S_k|} \left(N - \sum_{k=1}^{K} |S_k|\right)! \prod_{k=1}^{K} \binom{|T_k|}{|S_k|}^2 |S_k|!$$

となります．$\binom{|T_k|}{|S_k|}^2 |S_k|!$ の部分は，各 k ごとに独立に計算できることに注目します．また，$(-1)^{\sum_{k=1}^{K} |S_k|} \left(N - \sum_{k=1}^{K} |S_k|\right)!$ の部分は，$\sum_{k=1}^{K} |S_k|$ さえ分かっていれば計算できることに注目します．

以上の考察をもとに DP を設計します．

- 定義: $\mathrm{dp}_{i,x} := \left(\sum_{k=1}^{i} |S_k| = x \text{ のとき } \prod_{k=1}^{i} \binom{|T_k|}{|S_k|}^2 |S_k|! \text{ の総和}\right)$.
- 初期化: $\mathrm{dp}_{0,0} = 1$.
- 遷移: 各 $0 \le y \le |T_i|$ について，$\mathrm{dp}_{i,x+y}$ に $\mathrm{dp}_{i-1,x} \times \binom{|T_i|}{y}^2 y!$ を足し込む．

x の状態数は $\mathrm{O}(N)$ で，遷移は $1 \le i \le K$ の合計で $\mathrm{O}(N)$ なので，全体の時間計算量は $\mathrm{O}(N^2)$ となり，十分に高速です．最終的に，$\sum_{x=0}^{N-1} (-1)^x (N-x)! \times \mathrm{dp}_{K,x}$ が答えです．

ABC #082 / ARC #087 Editorial

writer : sugim48

December 16, 2017

A: Round Up the Mean

- C++ Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872729
- Java Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872732
- Python 3 Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872734

B: Two Anagrams

Sort s, sort t, reverse t, and check if $s < t$.

- C++ Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872811
- Java Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872812
- Python 3 Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872814

C: Good Sequence

Let n_x be the number of occurrences of x in a. If $n_x < x$, add n_x to the answer.
If $n_x \geq x$, add $n_x - x$ to the answer.

- C++ Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872852

D: FT Robot

Let N be the length of s and (x_t, y_t) be the coordinates of the goal.

Let's separate the sequence of operations by `T`, and let d_i be the number of `F` in the i-th part. Then, the movement of the robot can be described as follows:

- Move d_1 to right.
- Move d_2 upward or downward.
- Move d_3 to left or right.
- Move d_4 upward or downward.
- Move d_5 to left or right.
- And so on.

Thus, the x-coordinate and the y-coordinate can be handled independently. For example, for the x-coordinate, the robot will perform the following operations:

- Move d_1 to right.
- Move d_3 to left or right.
- Move d_5 to left or right.
- And so on.

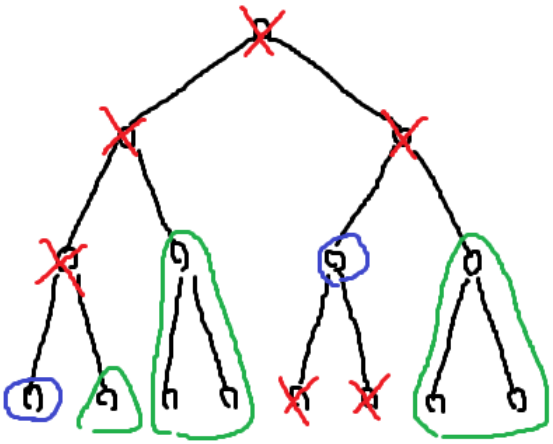
We want to check if x can be x_t after all operations. This can be done by a straightforward DP in $O(N^2)$. The y-coordinate can be handled similarly.

E: Prefix-free Game

Let's construct a trie of all strings of length L. It will be a complete binary tree. On this tree, the game can be described as follows:

> Initially, some vertices contain tokens. Alice and Bob alternately put tokens. Here, no two tokens can be put on two vertices with "ancestor - descendant" relation. The player who fails to put a token loses.

For example, the picture below shows the case with $L = 3$ and two strings 000 and 10.



Blue vertices contain tokens, red vertices are forbidden, and you can put tokens on green vertices. The green vertices will be a disjoint union of complete binary trees, and each part (complete binary tree) can be handled independently. Thus, we can use Grundy Numbers.

Let g_i be the Grundy Number of a complete binary tree of height i (i.e., a complete binary tree with i layers). If you perform an operation on a complete binary tree of height i, it will be splitted into complete binary trees with heights $i-1$, $i-2$, ..., $i-j$ for some j $(0 \leq j \leq i-1)$. Thus, g_i is the smallest integer that doesn't appear in the following:

- 0
- g_{i-1}
- $g_{i-1} \oplus g_{i-2}$
- $\vdots$
- $g_{i-1} \oplus g_{i-2} \oplus \cdots \oplus g_1$

If you perform an experiment, you get $(g_1, g_2, g_3, g_4, g_5, g_6, g_7, g_8, \ldots) = (1, 2, 1, 4, 1, 2, 1, 8, \ldots)$. In general, g_i is the maximum power of two that divides i. (We omit the proof, it is straightforward.) If the XOR of all Grundy Numbers is 0, Bob wins, otherwise Alice wins.

To get the set of green binary trees, you should construct a trie for all input strings, and check all nodes with exactly one child.

F: Squirrel Migration

Consider an edge $e = (v, w)$ in the tree. Let n_v, n_w be the sizes of two subtrees we get when we cut the tree by e. Then, the number of squirrels that pass through this edge (call it s_e) is st most $2\min\{n_v, n_w\}$.

The total distance defined in the statement is the sum of s_e, thus it is at most $\sum 2\min\{n_v, n_w\}$. It turns out that this value is always achievable. There are two different cases:

The tree has two centroids

Call them c_1, c_2. The maximum can be achieved when all squirrels pass through an edge between c_1 and c_2. There are $(N/2)! \times (N/2)!$ such ways.

The tree has one centroid

Call it c. The maximum can be achieved when all squirrels pass through the vertex c.

Let T_1, T_2, ..., T_K be the subtrees we get when we remove c from the tree. We want to count the number of permutations such that for each i, no squirrel in T_i moves to a vertex in T_i.

We use inclusion-exclusion principle. Let $T := T_1 \cup T_2 \cup \cdots \cup T_K$ (i.e., the entire vertex set minus c). The answer is $\sum_{S \subseteq T} (-1)^{|S|} f_S$, where f_S is the number of permutations of squirrels such that each squirrel in S moves to a vertex in the same group.

Now, for each k, we want to compute the sum of f_S for all S such that $|S| = k$.

Let g_k be the number of ways to choose k squirrels and there destinations, such that all chosen squirrels will travel to a vertex in the same group. Formally, g_k is the number of pairs of two k-tuples of vertices $(s_1, \ldots, s_k), (t_1, \ldots, t_k)$ such that:

- $s_1 < s_2 < \cdots$ (the order of chosen squirrels doesn't matter)
- For each i, s_i and t_i are in the same subtree (and they are not c).
- t_i are pairwise distinct.

Since we can freely choose destinations for unchosen squirrels, the sum of f_S for all S such that $|S| = k$ is $g_k(N-k)!$. Therefore, the answer is $\sum_{k=0}^{N-1} g_k(N-k)!(-1)^k$. Since g_k can be easily computed in $O(N^2)$ by a simple DP, we can also get the answer in $O(N^2)$.

AtCoder Beginner Contest 081 / AtCoder Regular Contest 086 解説

writer: semiexp 並びに anonymous

2017 年 12 月 9 日

For International Readers: English editorial starts on page 7.

A: Placing Marbles

それぞれの文字が 1 か 0 かを判定して 1 の数を数えればよいです．マスの数は高々 3 つなので，for 文を使わなくても簡単に実装できます．

```
#include <iostream>
#include <string>
using namespace std;
int ans;
string s;
int main(){
    cin >> s;
    if(s[0]=='1') ans++;
    if(s[1]=='1') ans++;
    if(s[2]=='1') ans++;
    cout << ans << endl;
}
```

B: Shift only

黒板に書いてある整数を管理しておいて,「書かれている整数がすべて偶数である限り」「書かれている整数すべてを 2 で割る」をシミュレートすればよいです.

あるいは，ほとんど同じことですが，最初書かれている整数それぞれに対して「最大で何回 2 で割れるか」を求め，その最小値をとってもよいです.

- C++ による解答例: `https://abc081.contest.atcoder.jp/submissions/1842477`
- Python による解答例: `https://abc081.contest.atcoder.jp/submissions/1842030`

C: Not so diverse

「できるだけ少ない数のボールの整数を書き換える」というのは,「できるだけ多くのボールの整数を，変更しないままにしておく」というのと同じです．ここで，変更を行わないボールたちに書かれている整数は K 種類以下でなければなりません．逆に，これが K 種類以下なら，他のボールに書かれている整数も，変更しないでおくボールに書かれている整数から選んで書き換えることで，全体として書かれている整数を K 種類以下にすることができます．

よって，問題は「整数を K 個選んで，それらのうちのどれかが書かれているボールの数を最大化する」と言い換えられます．選ぶ整数としては，できるだけそれが書かれているボールの数が多いようなものから順に選んでいくのが最適です．

最初書かれている整数として考えられるものは 1 以上 N 以下なので，各整数に対して「その整数が書かれているボールは何個あるか」を容易に求めることができます．この個数分布をソートして，大きいほうから K 個の和をとることで，書き換えないボールの個数の最大値がわかります．あとは N からこの値を引くと，求める値を計算することができます．

D: Non-decreasing

簡単な場合から考えていきます．a が非負整数のみからなる場合を考えてみます．このとき，a の先頭からの累積和を取った数列は明らかに条件を満たします．よって，$a_2 += a_1, a_3 += a_2, \ldots, a_N += a_{N-1}$ という順番で操作をすればそのような数列を作ることができ，$N-1$ 回の操作で条件を満たすことができます．a が 0 以下の整数のみからなる場合もほぼ同様の方法で $N-1$ 回の操作で条件を満たすことができます．

a に負の数も正の数も含まれるとき，N 回以下の操作で a が非負整数のみからなるように，あるいは 0 以下の整数からなるようにすることができるかどうかを考えてみます．$\mathrm{MAX} := \max(a), \mathrm{MIN} := \min(a)$ とします．このとき，$|\mathrm{MAX}| \geq |\mathrm{MIN}|$ ならば，すべての数に MAX を足せば a を非負整数のみからなる数列にすることが可能です．同様に，$|\mathrm{MAX}| < |\mathrm{MIN}|$ ならば，すべての数に MIN を足せば a を 0 以下の整数のみからなる数列にすることが可能です．

以上より $2N$ 回以下の操作で条件を満たすことができることが分かりました．これは $O(N)$ で実行可能です．

E: Smuggling Marbles

同じ頂点の上に複数のビー玉が乗ったとき，木の上から取り除かれるという手順について考えます．ある 2 つの頂点に置かれたビー玉が同じ頂点の上に乗るためには，これらの頂点の根からの距離が等しい必要があります．よって，根からの深さ d の頂点にのみビー玉を置くとき，最終的に箱にビー玉が移動するようなビー玉の初期配置がいくつあるかを考えていくことにします．

$dp[i][j] =$ 頂点 i を根とする部分木に着目したとき，頂点 i に $j(0 \leq j \leq 1)$ 個のビー玉が置かれるような頂点 0 からの距離が d の頂点へのビー玉の配置の数，として DP をすることで $O(N)$ で数え上げることができます．なお，DP の計算途中では頂点にビー玉が置かれる個数が 2 個以上になりえますが，最終的に取り除かれるため 2 個として扱っても問題ありません．こうして得られた結果は，深さ d 以外の頂点の配置を考慮していないので，$dp[0][1] \times 2^{N-\text{深さ } d \text{ にある頂点の個数}}$ が求めるべき答えです．これを各深さごとに行えば $O(N^2)$ で数え上げることができ部分点を獲得できます．

ここで，この DP には無駄な遷移が多く含まれていることに着目します．例えば，ある頂点に複数個のビー玉が置かれるためにはその頂点の直接の子が 2 つ以上ある必要がありますが，先程の DP では子が 1 つしかないような場合でも毎回計算を行っていました．これらをうまく無視して計算する方法を考えます．

$dp[i][d][j] =$ 頂点 i に j 個のビー玉が置かれるような，頂点 i から深さ d の頂点へのビー玉の初期配置の数，とします．この DP の d についてデックを用いて動的に管理をします．$d = 0$ については，頂点 i のみ考えればよいため，最後にデックの先頭に付け加えることにします．それ以外の深さの更新については以下の図 1 のように子頂点どうしの 2 つのデックのうち長さの短い方を長い方へとマージするような要領で更新をしていきます．このようにすることで全体として $O(N)$ で数え上げることができ，満点を獲得できます．

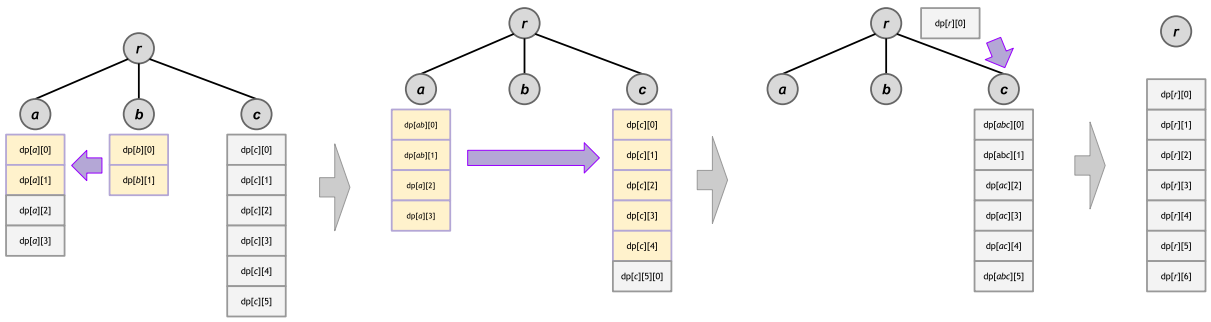


図 1: 頂点 r を根とする DP の更新の模式図

最後にこれが $O(N \log N)$ ではなく $O(N)$ になることの概略を述べておきます．全ての頂点には 0 が書かれているとします．根から距離 d に a_d 個の頂点があるとします．まずこれら a_d 個の頂点たちを行きがけ順に並べます．その後，隣り合う頂点のペア (u, v) について，それぞれ u と v の LCA(最小共通先祖) であるような頂点に 1 を加算します．このとき，各頂点に書かれた整数の数が，その頂点において，根から深さ d の頂点たちに対する DP の更新を行う必要がある (あるいはデックのマージを行う) 回数になります．これらの和は明らかに $a_d - 1$ 回となります．よって，更新の回数は $O(\sum_{d=0}^{N} \max(0, a_d - 1)) = O(N)$ となります．

F: Shift and Decrement

操作 B を行う回数を k とします．入力で与えられる整数は $10^{18} < 2^{60}$ 以下なので，$0 \leq k \leq 60$ として問題ありません (60 回 B を行うとすべての整数が 0 になるため，61 回目を行う意味はありません)．$i = 0, 1, \ldots, k$ に対して，i 回目の操作 B と $i+1$ 回目の操作 B の間で行う操作 A の回数を p_i で表すことにします (ただし，p_0 は「最初の操作 B を行う前に行う操作 A の回数」，p_k は「最後の操作 B を行った後に行う操作 A の回数」とします)．

p_i を $2d$ 減らして p_{i+1} を d 増やしても，黒板上の整数の書かれ方は変わらないことに注意します．このことから，$i = 0, 1, \ldots, k-1$ に対して，$p_i = 0$ または $p_i = 1$ としてよいということがわかります．(なぜならば，$p_i \geq 2$ であれば，p_i を 2 減らして p_{i+1} を 1 増やすことで，最終的な黒板上の整数の書かれ方を保ちつつ，操作回数を減らすことができるからです．) また，p_i を 1 減らして p_0 を 2^i 増やしても，黒板上の整数の書かれ方は変わらないことも従います．よって，$\{p_i\}$ に対応する一連の操作は，「操作 A を $P = \sum_{i=0}^{k-1} 2^i p_i$ 回行い，操作 B を k 回行い，最後に操作 A を p_k 回行う」一連の操作と等価です．$p_i = 0, 1$ としたので，$0 \leq P \leq 2^k - 1$ が成り立ちます．

A_i を 2^k で割った商を B_i，余りを C_i $(0 \leq C_i \leq 2^k - 1)$ とします．また，C_i $(i = 1, \ldots, N)$ から重複を除去し，ソートしたものを $C'_1 \leq C'_2 \leq \cdots \leq C'_M$ とします．$0 \leq P \leq 2^k - 1$ であることから，一連の操作によって A_i は次の値になることがわかります．

- $P \leq C_i$ ならば，$B_i - p_k$
- $C_i \leq P$ ならば，$B_i - p_k - 1$

よって，すべての i に対して「$P \leq C_i$ かどうか？」が同じであれば，異なる P に対しても同じ整数の書かれ方が得られることがわかります．ゆえに，考えるべき P は次のように区分されます．

- 0 以上 C'_1 以下
- $C'_1 + 1$ 以上 C'_2 以下
- $\ldots$
- $C'_{M-1} + 1$ 以上 C'_M 以下

ここで，P ごとに実際に行われる操作 A の回数は，2 進表記したときに現れる 1 の個数と等しいです．この操作回数をできるだけ少なくすることを考えると，P としては「区分の範囲内で，最も 2 進表記における 1 の個数が少なくなるもの」を用いるべきであることがわかります (具体的方法は後述します)．なお，$C'_M + 1$ 以上については，$P = 0$ として p_k に 1 を加えるのと同じであることから，考える必要がありません．

よって，各 k に対して考えるべき P の候補が $M \leq N$ 通りにまで絞られました．それぞれの候補に対して，最後の操作 B を行うまでの操作回数を計算することができるので，p_k の範囲を求めることができます．(ここで，負の整数が現れてはいけないことに注意します．) 黒板に書かれているすべての整数に同じ整数を加減することで移り変わるような整数の書かれ方同士を同じグループとみなすことにすると，p_k を変更しても所属するグループは変わりません．なので，考えるべきグループは全体で (k, P のとり方に対応して) $61 \times N$ 個以下です．それぞれのグループごとに，「A_1 は最終的に何に書き換えられるか」としてありうる範囲の集合を求めることができます．このような範囲も合計で $61 \times N$ 個以下です．あとは，各グループごとに，範囲の重複を除いて可能なパターン数を求めればよいです．

このアルゴリズムの計算量は，グループおよび範囲が全部で $O(\log A_{\max} \cdot N)$ 個あり，グループのインデックスは $N-1$ 要素の配列で管理できることから，$O(\log A_{\max} \cdot N^2 \log(\log A_{\max} \cdot N))$ になります．

最後に，X より大きく Y 以下の整数で $(X < Y)$，最も 2 進表記における 1 の個数が少なくなるものを求める方法を述べておきます．2 進表記において X, Y で異なる値になる桁が存在するので，そのうち最も上位のものを l ビット目とします．すると，l ビット目より上位のビットについては，X, Y と同じにするほかありません．一方，l ビット目およびそれより下位のビットをすべて 0 にすることはできない (X 以下になるため) ので，少なくともこの範囲で一つは 1 が必要です．ところで，l ビット目のみを 1 にし，より下位のビットはすべて 0 にすると，X より大きく Y 以下となります．よって，このようにして得られた整数が，最も 2 進表記における 1 の個数が少なくなるものになります．

AtCoder Beginner Contest 081 / AtCoder Regular Contest 086 Editorial

writer: semiexp and anonymous

December 10th, 2017

A: Placing Marbles

```
#include <iostream>
#include <string>
using namespace std;
int ans;
string s;
int main(){
    cin >> s;
    if(s[0]=='1') ans++;
    if(s[1]=='1') ans++;
    if(s[2]=='1') ans++;
    cout << ans << endl;
}
```

B: Shift only

- C++ implementation: `https://abc081.contest.atcoder.jp/submissions/1842477`
- Python implementation: `https://abc081.contest.atcoder.jp/submissions/1842030`

C: Not so diverse

For each integer x between 1 and N, count the number of occurrences of x in the input, and sort it. The answer is N minus the sum of K greatest integers in the frequency list.

D: Non-decreasing

In case all numbers in a are non-negative, the sequence of prefix sums (i.e., $a_1, a_1 + a_2, a_1 + a_2 + a_3, \cdots$) is non-decreasing. Thus, the sequence of operations "$a_2 \mathrel{+}= a_1, a_3 \mathrel{+}= a_2, \ldots, a_N \mathrel{+}= a_{N-1}$" satisfies the condition. Similarly, in case all numbers are non-positive, "$a_{N-1} \mathrel{+}= a_N, a_{N-2} \mathrel{+}= a_{N-1}, \ldots, a_1 \mathrel{+}= a_2$" satisfies the condition.

In general, there are two cases. Let $\mathrm{MAX} := \max(a), \mathrm{MIN} := \min(a)$. If $|\mathrm{MAX}| \geq |\mathrm{MIN}|$, we first add MAX to all elements of a, and a will be a sequence of non-negative numbers. Then, we can make it non-decreasing as we described above. Similarly, if $|\mathrm{MAX}| < |\mathrm{MIN}|$, we first add MIN to all elements of a, and a will be a sequence of non-positive numbers.

In both cases, we need $2N - 1$ operations. This solution works in $O(N)$.

E: Smuggling Marbles

When multiple marbles are on the same vertex, they will be removed. This happens only when the marbles come from vertices of the same depth. Thus, we can handle each depth independently. For each d, consider all subsets of vertices of depth d, and count the number of subsets that will end up with a single marble at the root.

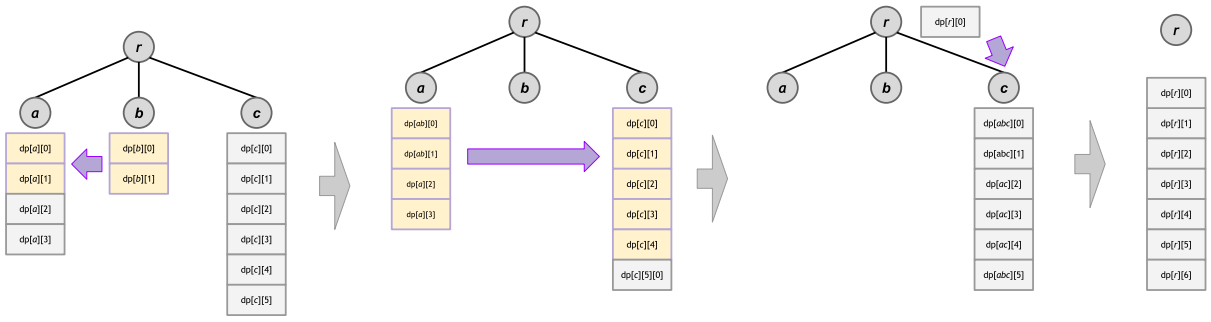
How can we solve this problem for a fixed d? For simplicity, from now on, we use probabilities - each vertex initially contains a marble with probability $1/2$. Let $dp[i][j]$ be the probability that vertex i contains $j (0 \leq j \leq 1)$ marble at the end, when we only consider the subtree rooted at i (and initially we put marbles only on vertices of depth d). The answer is sum of $dp[0][1] \times 2^N$ over all depths. This solution works in $O(N)$ per depth, and $O(N^2)$ in total. You can get partial score.

To make it faster, we compute these values for all depths at once. Let $dp[i][d][j]$ be the following value: Consider a subtree rooted at vertex i. We put marbles to some vertices of depth d (i.e., the distance to i is d), with probability $1/2$ for each such vertex. What is the probability that vertex i contains j marbles at the end?

Let's call a tuple of three doubles $dp[i][d] = (dp[i][d][0], dp[i][d][1], dp[i][d][2])$ "state". Here, $j = 2$ means that a vertex contains two or more marbles (it happens during the computation, as described below). Then, $dp[i]$ is a sequence of states $(dp[i][0], dp[i][1], ...)$ and its length is (the depth of subtree rooted at i) plus one. We represent it as a deque of states.

The figure below shows how to compute $dp[r]$ when it has three children a, b, c. While r has two or more children, we "merge" the shortest deque into the second shortest deque. For example, if $dp[a][i]$ contains x marbles and $dp[b][i]$ contains y marbles, the new deque $dp[ab][i]$ will contain $min(x + y, 2)$ marbles. It can be done in $O(min(len(dp[a]), len(dp[b])))$. Make sure that you don't create a new deque - if $len(dp[a]) > len(dp[b])$, reuse $dp[a]$, and rewrite its first $len(dp[b])$ elements.

Then, if r has only one children, you just need to push a state $(1/2, 1/2, 0)$ to the front. Don't forget to convert $j = 2$ to $j = 0$ at the end.



Why is it $O(N)$ (and not $O(NlogN)$)? Suppose that initially all vertices contain 0. Let a_d be the number of vertices of depth d. First, we sort these vertices in dfs pre-order. For each consecutive pair of two vertices (u, v) (of depth d), add 1 to the integer written on LCA of u and v. Then, the integers written on each vertex represents the number of merges that comes from vertices of depth d. The sum of those integers is $a_d - 1$. Thus, the total number of merges is $O(\sum_{d=0}^{N} \max(0, a_d - 1)) = O(N)$.

F: Shift and Decrement

Suppose that we perform operation A k times. Since $10^{18} < 2^{60}$, we can assume that $0 \leq k \leq 60$ (after 60 operations all numbers will be zero). For each $i = 0, 1, \ldots, k$, let p_i be the number of operation Bs we perform between i-th A and $i+1$-th A. (p_0 and p_k are defined naturally).

An operation A maps an integer x to $floor(x/2)$, and an operation B maps an integer x to $x-1$. Notice that we don't need to take floor after each opertaion A - instead, we compute numbers as doubles, and only at the end of all operations we apply the floor function. Thus, the sequence of operations is equivalent to the following:

- Perform operation B $P = \sum_{i=0}^{k-1} 2^i p_i$ times.
- Perform operation A k times.
- Perform opertaion B p_k times.

Also, we can assume that $0 \leq P \leq 2^k - 1$ (otherwise, we can decrease P by 2^k and increment p_k by one, and decrease the total number of operations).

Let $A_i = 2^k \times B_i + C_i$ ($0 \leq C_i \leq 2^k - 1$). Let $C'_1 \leq C'_2 \leq \cdots \leq C'_M$ be a sorted list of integers that appear in C (after removing duplicates).

After all operations, the value of A_i will be following:

- If $P \leq C_i$, $B_i - p_k$
- If $C_i \leq P$, $B_i - p_k - 1$

Thus, the only important thing for P is "Is $P \leq C_i$?" for each C_i. P will be in one of the following intervals:

- $[0, C_1]$
- $[C_1 + 1, C_2]$
- $\ldots$
- $[C_{M-1} + 1, C_M]$

The number of opertaion Bs in the first phase is $popcount(P)$ (the number of 1s in the binary representation of P). Thus, in each interval, we should choose P that minimizes this value (see below for the way to compute P).

Now, for each k, we have only $M \leq N$ candidates for P. For each candidate, we can compute the valid range for p_k (to make the total number of opertaions at most K, make sure that it won't be negative).

We consider two sequences of integers to be in the same group if we can convert one of them to the other by performing operation Bs. Since the value of p_k doesn't affect its group, we need to consider at most $61 \times N$ groups (at most one group per each (k, P) pair). For each group, compute the possible range for A_1 (the total number of those ranges is also at most $61 \times N$). Then, for each group, we compute the number of possible values for A_1 (after remoging duplicates).

There are $O(\log A_{\max} \cdot N)$ groups and ranges, and since each group can be represented as a sequence of length $N-1$, this solution works in $O(\log A_{\max} \cdot N^2 \log(\log A_{\max} \cdot N))$ time.

Now, we describe how to compute an integer between $X+1$ and Y ($X < Y$) that minimizes its popcount. Let l be the largest integer such that the l-th digits in X and Y differ. Since the integer we choose must be between $X+1$ and Y, all digits above l-th must be the same as X, Y. If we make the l-th digit one and the remaining digits zero, the number will be between $X+1$ and Y. This is optimal because it only adds one to popcount (and we can't make all digits up to l-th zeroes because the number must be at most $X+1$).

ARC 085/ABC 078 解説

Kohei Morita(yosupo)

平成 29 年 11 月 11 日

For International Readers: English editorial starts on page 8.

A: HEX

16 進数に直さずとも，実は入力した文字の ASCII コードを比較するだけで十分です (C, C++ならば char 型のまま比較すれば ASCII コードで比較できます)。

想定解

B: ISU

人が n 人座るためには椅子の幅は何センチメートルあればよいでしょうか?

これは $Y \times n + Z \times (n+1)$ センチメートルです。$Y \times n$ が人の占有する幅で，$Z \times (n+1)$ が人と人，人と椅子の間に開ける幅です。

よって $n = 1, 2, ...$ と順に増やしていき，座れる人数のうち最も多い人数を調べればよいです。なお，人数の最大は入力例 3 の 49999 人です。

想定解

C: HSI

1 回の実行にかかる時間を x ms，全てのケースに正解する確率を p，この問題の答えを y ms とします。$x = 1900M + 100(N - M), p = 1/2^M$ です。

まず，一回目の提出で必ず x ms はかかります。その後は，確率 p で終了します，そして確率 $1 - p$ でその提出には失敗し，更に提出を繰り返します。確率 $1 - p$ で失敗した場合の，その時点からかかる時間の期待値は y ms です。

よって，$y = x + (1 - p) \times y$ が成立し，これを解くと $y = x/p$ が得られます。

よって答えは $x/p = (1900M + 100(N - M)) \times 2^M$ です。

D: ABS

まず，N=1 の場合の答えは $|a_1 - W|$ です。

$2 \leq N$ の場合を考えます。X は初手で全部のカードを引くことにより，$|a_N - W|$ が達成できます。また，初手で $N - 1$ 枚のカードを引くことで $|a_{N-1} - a_N|$ が達成できます。

実は，初手で行うべき操作はこの 2 通りのみ，つまり 2 枚以上カードを残す理由はありません。なぜならば，その直後の手番で Y さんは，残り 1 枚になるようにカードを引くと $|a_{N-1} - a_N|$ を達成できます。よって 2 枚以上カードを残してもスコアが $|a_{N-1} - a_N|$ より高くなることはないからです。

以上より，この問題の ($2 \leq N$ の時の) 答えは $max(|a_N - W|, |a_{N-1} - a_N|)$ です。

E: MUL

結論から言うと，この問題は最大流問題の双対問題である最小カットを使い解くことができます。

具体的には，頂点 $S, V, 1, 2, ..., N$ を用意し，各 i について，

- $a_i \leq 0$ ならば，$S \to i$ に容量 $-a_i$ の辺を張る
- $a_i > 0$ ならば，$i \to T$ に容量 a_i の辺を張る
- 各 $j = 2i, 3i, 4i, ...$ について，$i \to j$ に容量 ∞ の辺を張る

と辺を張り，$a_i > 0$ なる a_i の総和から，構築したグラフの最小カットを引いたものが答えです。

最小カット問題の答えは最大流問題を解くことで得られます。頂点数が O(N) で，変数は $O(N \log N)$，そして $N \leq 100$ と小さいため，計算量が流量に依存しない最大流アルゴリズムならだいたい間に合うと思います。

理由

最小カット問題とは，S, T 以外の頂点を S 側と T 側に振り分け，S 側から T 側へ張られる辺の容量の総和を最小化する問題です。

ここで，S 側を 0，T 側を 1 として見ると，S には 0, T には 1，それ以外の頂点には 0, 1 を割り振り，(0 が割り振られた頂点) から (1 が割り振られた頂点) へ張られる辺の容量の総和を最小化する問題，と考えられます。

これを更に言い換えると，以下のような問題が解けることがわかります。

- 変数 $q_s, q_t, q_1, q_2, ..., q_v$ を考える
- 罰金条件 (x, y, z) がたくさん与えられる。これは，q_x が 0 で q_y が 1 なら罰金 z 円，という意味である。
- あなたは変数にそれぞれ $0, 1$ のどちらかを割り当て，罰金を最小化したい。ただし $q_s = 0, q_t = 1$ は固定である。

当然，変数が頂点，罰金条件が辺に対応します。なお，罰金条件の罰金 z は必ず非負でないといけないことに注意してください。非負を許した場合，この問題は最大カット問題も含み，NP-hard となります。

では，元の問題をこのような問題に変形することを考えます。

まず，ある i, j(j は i の倍数) について，i が破壊されていて，j が破壊されていない場合，明らかにおかしいです。

よって，この条件を表すために，i が破壊されていて j が破壊されていないと罰金 ∞ 円とすることを考えます。

これは，(0 を割り振る)=破壊されている，(1 を割り振る)=破壊されていない，とし，罰金条件 (i, j, ∞) を足せばよいです。

実は，全ての i, j(j は i の倍数) について,「i が破壊されているならば，j が破壊されている」が成立する場合，このような破壊の方法が存在します。

よって，あとは破壊されていないと a_i 円，つまり $q_i = 1$ ならば a_i 貰える，という処理です。

$a_i \leq 0$ ならばこれは罰金と考えられるので問題ないです。罰金条件 $(S, i, -a_i)$ を足します。

$a_i > 0$ ならば，言い換えて,「i が破壊されていると，a_i 円が貰えない=a_i 円罰金」と考えます。すると事前に答えに a_i を貰っておくことにし，罰金条件 (i, T, a_i) を足します。

以上より，この問題は最小カット問題に帰着できました。

今回の問題は N 個の宝石に破壊された，破壊されていない，の 2 つの状態を割り振る問題でした。このように，幾つかの要素に 2 つの状態を割り振り，何かを最大化/最小化する問題で，更に DP ではとても解けそうにない場合，最小カットを疑うとよいです。

F: MUL

この問題は，$a_i \neq b_i$，つまり $(a_i, b_i) = (0,1), (1,0)$ の個数を最小化する問題です。

$((a_i, b_i) = (0,1), (1,0)$ の個数)
$=$ ($(a_i, b_i) = (0,1)$ の個数) + ($(a_i, b_i) = (1,0)$ の個数)
$=$ ($(a_i, b_i) = (0,1)$ の個数) + ($b_i = 0$ の個数) − ($(a_i, b_i) = (0,0)$ の個数)

であり，$b_i = 0$ の個数は定数であるため，($(a_i, b_i) = (0,1)$ の個数) − ($(a_i, b_i) = (0,0)$ の個数) を最小化することを考えても良いです。

言い換えると，$a_i = 1$，つまり塗った場合のコストは 0 で，塗らなかった場合のコストは 1 だったり -1 するので，コストの総和を最小化してくださいという問題です。

操作それぞれについて行うかどうかを全探索していてはとても間に合いません。うまく探索する必要があります。

操作を行うかどうかとスコアの計算を並び換え，

$l_i = 1$ の操作たちを行うかどうか決め，a_1 のスコアを計算する
→ $l_i = 2$ の操作たちを行うかどうか決め，a_2 のスコアを計算する
→ $l_i = 3$ の操作たちを行うかどうか決め，a_3 のスコアを計算する
→ $l_i = 4$ の操作たちを行うかどうか決め，a_4 のスコアを計算する
→ …

という風に，操作を行うかどうかとスコアの計算を交互に行うことを考えます。このような順番にしても答えは変わりません。

なぜこのような順番にするかというと，a_x のスコアを計算した後は $a_1, a_2, ..., a_x$ の色は忘れてよく，更に $a_{x+1}, a_{x+2}, ..., a_N$ については，あるところまでは 1，その後全て 0 という形になっているからです。

よって，この並び替えを行うだけで (まだ TLE ですが)DP を行い，計算量を多項式にすることができます。DP のキーは (x, y) の 2 つで，「a_x のスコアまで計算し，a_y まで 1 になっている」という状態を現します。

$(x, x), (x, x+1), (x, x+2), ..., (x, N)$ から $(x+1, x+1), (x+1, x+2), (x+1, x+3), ..., (x, N)$ まで同時に遷移することを考え，この遷移を SegTree で高速化します。

区間 $l(= x), r$ の操作については，$(x, x), (x, x+1), ..., (x, r)$ の最小値を $(x+1, r)$ に遷移させれば良いです。a_x のスコアを計算する，のパートは簡単です。最初の考察により，$a_x = 0$，つまり $y = x$ の場合以外はスコアが 0 だからです。

以上より，区間の min を取るのが本質であり，この操作は $SegTree$ で簡単に実装できます。

ARC 085/ABC 078 Editorial

Kohei Morita(yosupo)

November 11, 2017

A: HEX

https://abc078.contest.atcoder.jp/submissions/1751369

B: ISU

https://abc078.contest.atcoder.jp/submissions/1751697

C: HSI

Let $x = 1900M + 100(N - M)$ be the time required to submit (and judge) one solution. Let $p = \frac{1}{2^M}$ be the probability that one submission passes. Let y be the answer of this problem.

It takes x ms to submit once. With probability p, it passes and the process ends, and with probability $1 - p$, you need more submissions and you need y ms more on average.

Thus, you get $y = x + (1 - p) \times y$, and $y = x/p$.

Therefore, the answer is $y = x/p = (1900M + 100(N - M)) \times 2^M$.

D: ABS

If $N = 1$, the answer is $|a_1 - W|$.

Suppose that $2 \leq N$. X can achieve the final result of $|a_N - W|$ all cards in his first turn. Also, X can achieve the final result of $|a_{N-1} - a_N|$ by drawing $N - 1$ cards in his final turn.

It turns out that X can't do better than the two strategies above. Suppose that X leaves at least two cards (k cards) in his first turn. Then, Y can take $k - 1$ cards in the next turn and the result will be $|a_{N-1} - a_N|$.

Therefore, the answer is $max(|a_N - W|, |a_{N-1} - a_N|)$.

E: MUL

Let's solve this problem as a minimum S-T cut problem.

We divide all integers $1, 2, \ldots, N$ into two groups. Group S is a set of integers that are smashed, and Group T is a set of integers that are not smashed.

Ideally, we should smash all integers with negative costs, and we shouldn't smash any integers with non-negatove costs. Let X be the total money you can get in this ideal situation.

However, this situation is not always possible. If for some i, j such that j is a multiple of i, i is smashed and j is not smashed, we get a contradiction. (Note that this is a sufficient condition: if a set of integers satisfies this condition, we can perform operations on all smashed integers and still we don't smash anything else.)

Now, you get the following penalties:

- If for some i, j such that j is a multiple of i, i is in group S and j is in group T, we get an infinite penalty.
- If $a_i \leq 0$ and i is in group T, we get a penalty of $-a_i$.
- If $a_i > 0$ and i is in group S, we get a penalty of a_i.

To represent these penalties, construct the following graph:

Construct a graph with $N + 2$ vertices. The vertices are labelled with $S, T, 1, 2, ..., N$, and for each i, we add the following edges:

- If $a_i \leq 0$, add an edge $S \to i$ with capacity $-a_i$.
- If $a_i > 0$, add an edge $i \to T$ with capacity a_i.
- For each $j = 2i, 3i, 4i, ...$, add an edge $i \to j$ with capacity ∞.

The answer is X minus the minimum S-T cut of the graph above.

Since there are $O(N)$ vertices and $O(N \log N)$ edges, it works in time.

F: NRE

In this problem, we want to minimize the number of i such that $a_i \neq b_i$, that is, $(a_i, b_i) = (0, 1) or (1, 0)$.

The number of i such that $(a_i, b_i) = (0, 1), (1, 0)$
= (The number of i such that$(a_i, b_i) = (0, 1)$) + (The number of i such that $b_i = 0$) - (The number of i such that $(a_i, b_i) = (0, 0)$)

Here, since the number of i such that $b_i = 0$ is a constant, we want to minimize (The number of i such that$(a_i, b_i) = (0, 1)$) - (The number of i such that $(a_i, b_i) = (0, 0)$)

In other words, when you paint the i-th cell ($a_i = 1$), there's no cost, and when you don't paint it ($a_i = 0$), you get a cost of 1 or -1.

Let's perform the operations and compute the cost in the following order:

Perform some operations with $l_i = 1$ and compute the cost of a_1
→ Perform some operations with $l_i = 2$ and compute the cost of a_2
→ Perform some operations with $l_i = 3$ and compute the cost of a_3
→ Perform some operations with $l_i = 4$ and compute the cost of a_4
→ ⋯

Here, after you compute the cost of a_x, you can forget about $a_1, a_2, ..., a_x$, and at this point, $a_{x+1}, a_{x+2}, ..., a_N$ is of the form "111...111000...000".

Therefore, we get the following DP, and it works in polynomial time. $dp(x, y)$ means the minimum possible cost you get from $a_1, \ldots, a_x$, given that at this point $a_{x+1}, \ldots, a_y = 1$ and the remaining elements are zeroes.

Now, how can we get the DP values of $(x+1, x+1), (x+1, x+2), (x+1, x+3), ..., (x, N)$ from $(x, x), (x, x+1), (x, x+2), ..., (x, N)$?

To handle an operation in the interval $[l(= x), r]$, we replace $(x + 1, r)$ with the minimum of $(x, x), (x, x + 1), ..., (x, r)$. It's easy to compute the cost of a_x because when $a_x = 0$, you get no cost.

Therefore, the only data structure you need is to get the minimum of an interval, and this can be done in segment tree (or RMQ).

ARC084/ABC077 解説

DEGwer

2017/11/04

For International Readers: English editorial starts on page 5.

A: Accepted...?

改行を無視して 6 文字を読み込み、1 文字目と 6 文字目、2 文字目と 5 文字目、3 文字目と 4 文字目がそれぞれ等しいかどうかを調べればよいです。

以下の実装では、1 文字目から 3 文字目までを配列 a に、4 文字目から 6 文字目までを配列 b に、それぞれ読み込んでいます。

```
#include<stdio.h>
int main()
{
        char a[10], b[10];
        scanf("%s%s", a, b);
        if (a[0] == b[2] && a[1] == b[1] && a[2] == b[0]) printf("YES\n");
        else printf("NO\n");
}
```

B: Different Distribution

答えの候補を小さい順に試していきます。i^2 が n を初めて超えたときの、$(i-1)^2$ が答えです。

```
#include<stdio.h>
int main()
{
        int num;
        scanf("%d", &num);
        for (int i = 1;; i++)
        {
                if (i*i > num)
                {
                        printf("%d\n", (i - 1)*(i - 1));
                        break;
                }
        }
}
```

C: Snuke Festival

$A_i < B_j < C_k$ なる組 (i, j, k) の個数を数える問題です。j を固定すれば、求める個数は $A_i < B_j$ なる i の個数と、$B_j < C_k$ なる k の個数の積となります。

よって、全ての j に対し、上述の i, k の個数を数えて掛け合わせ、足し合わせればよいです。

では、i の個数や k の個数はどのように求めればいいでしょうか？　愚直に配列をすべて見ると、$O(N^2)$ となり間に合いません。

配列 A, C をあらかじめソートしておきます。すると、i や k の個数は、二分探索を用いて求めることができます。C++ の場合、標準ライブラリに用意されている lower_bound 関数や upper_bound 関数を用いるのが良いでしょう。

時間計算量は $O(NlogN)$ となり、間に合います。

D: Small Multiple

全ての正整数は、1 から始めて、以下の 2 つの操作を繰り返すことで作ることができます。

- 今の数に 1 を足す。このとき、各桁の和は 1 増える。(1 の位が 9 のときはこの限りではありませんが、この問題を解くにあたっては考慮しなくていいことをあとで示します)
- 今の数を 10 倍する。このとき、各桁の和は変わらない。

さて、全ての正整数を頂点とし、以上の操作の「今の数」から「新しい数」に辺をはったグラフを考えます。求めるべきは、1 から K の倍数のうちのいずれかへの、このグラフ上での最短経路長に 1 を足したものです。

このままでは頂点数は無限ですが、辺のはられ方を考えれば、各正整数に対応する頂点たちは $modK$ で同一視できることが分かります。よって、各頂点を $modK$ で同一視した K 頂点のグラフ上で、1 から 0 への最短路を求めればよく、これは 01BFS を用いることで $O(K)$ 時間で求めることができます。1 の位が 9 の整数に 1 を足す遷移は、最短路のアルゴリズムを思い出せば行き先の頂点がすでに訪れられていることがわかるので、特別に扱う必要はありません。

なお、01BFS とは、deque (両端キュー) を用意し、コスト 0 の辺の遷移は deque の先頭に、1 の辺の遷移は deque の末尾に要素を追加することによる、幅優先探索のアルゴリズムです。

E: Finite Encyclopedia of Integer Sequences

数列に使える整数の種類数 K が偶数の場合、最初の整数が $K/2$ 以下である数列と $K/2+1$ 以上である数列の個数は等しいので、答えは $K/2$ の後に K が $N-1$ 個続いた数列になります。以下 K が奇数の場合を考えます。

$(K+1)/2$ が N 個続いた数列 (これを B とおく) の前にある数列の個数と、後ろにある数列の個数の差はいくつでしょうか？　ある数列 X が B の前にあるとき、X の各要素 X_i を $K+1-X_i$ で置き換えれば、X が B の接頭辞でない限りは、B の後ろにきます。逆に、ある数列 X が B の後ろにあるとき、同様に置き換えれば、X が B の接頭辞でない限りは、B の前にきます。

よって、B の接頭辞以外の数列については B の前にある数列と後ろにある数列の間で一対一対応が成り立つため、B の前後の数列の個数の差は、B の接頭辞の個数、すなわち $N-1$ になります。すなわち、求める数列は、B の $\lfloor \frac{N-1}{2} \rfloor$ 個前にある数列になります。

さて、これはどのように求めればいいでしょうか？　1 個前の数列を得る操作は、

- 末尾が 1 の場合、取り除く
- 末尾が 1 でない場合、その整数を 1 減らし、さらに N 文字になるまで末尾に K を付け加える

という操作です。これはならし $O(1)$ 時間で実装できるので、この問題を $O(N)$ 時間で解くことができました。

F: XorShift

与えられた 2 進数たちを、$\mathbb{F}_2$ 上の多項式として見ます。すなわち、2 進数 $a_k a_{k-1} \ldots a_1 a_0$ を、多項式 $a_k x^k + a_{k-1} x^{k-1} + \ldots + a_1 x + a_0$ と同一視し、さらにその係数は $mod2$ で見ることにします。

操作を多項式の言葉に翻訳します。数を 2 倍することは多項式を x 倍することに、数同士の xor を取ることは多項式同士の和を取ることに対応します。

多項式を 2 つとってきます。次数の大きいほうを P 、小さいほうを Q として、その次数の差を d とします。Q を d 回 x 倍し、P と足し合わせることで、P より次数の低い多項式を作ることができます。こうしてできた多項式を新しい P として (適宜 P, Q を入れ替えて) 同様の操作を続けるアルゴリズムは、ユークリッドの互除法にほかなりません。すなわち、2 つの多項式の GCD を黒板に書くことができます。

これを繰り返せば、最初に黒板に書かれた多項式たちすべての GCD である多項式 P を黒板に書くことができます。また、全ての多項式は P を因子に持ち、操作によってその性質が崩される

ことはないため、黒板に書ける多項式は P の倍数のみです。さらに、x 倍をする操作と和を取る操作を適切に繰り返せば、P の任意の多項式倍を黒板に書くことができます。よって、黒板に書ける多項式は P の倍多項式全てです。

この多項式たちの中で、2 進数として見て X 以下のものの個数を数えましょう。P の次数を d とします。P の倍多項式の d 次以上の係数がすべて定まれば、残りの係数は一意に定まります。d 次以上の部分が X の d 次以上の部分より (2 進数としてみて) 小さいようなものに対しては、ちょうど 1 個の P の倍多項式が定まります。等しいものに対しては、その $d-1$ 次以下の係数を実際に求め (これは高次の係数から順に決めて行くことで求められます)、X より小さいかどうかを調べればよいです。

時間計算量は、GCD は 1 回あたり $O(d^2)$ 時間で求められ (ただし、d は与えられる整数の桁数)、$O(Nd^2)$ となります。また、bitset などを用いて xor を並列的に計算することで、($m = 32$ または 64 として) $O(Nd^2/m)$ 時間で計算することも可能です。

ARC084/ABC077 Editorial

DEGwer

2017/11/04

A: Accepted...?

```
#include<stdio.h>
int main()
{
        char a[10], b[10];
        scanf("%s%s", a, b);
        if (a[0] == b[2] && a[1] == b[1] && a[2] == b[0]) printf("YES\n");
        else printf("NO\n");
}
```

B: Different Distribution

```
#include<stdio.h>
int main()
{
        int num;
        scanf("%d", &num);
        for (int i = 1;; i++)
        {
                if (i*i > num)
                {
                        printf("%d\n", (i - 1)*(i - 1));
                        break;
                }
        }
}
```

C: Snuke Festival

We want to compute the number of triplets (i, j, k) such that $A_i < B_j < C_k$. If we fix j, the number of such triplets is the product of the number of i such that $A_i < B_j$ and the number of k such that $B_j < C_k$. Therefore, for each j, we compute the number of i, k mentioned above, and we can get the answer.

How can we compute the number of such i (or k) for all j? Straightforward solution works in $O(N^2)$ and too slow.

Let's sort A, C in advance. Then, the number of such i (or k) can be computed using a binary search.

This solution works in $O(NlogN)$ time.

D: Small Multiple

All positive integers can be obtained by performing the following two types of operations to 1 zero or more times:

- Add 1. By this operation, the sum of digits increases by one (unless the last digit is 9, but this is not important).
- Multiply the current number by 10. The sum of digits remains the same.

Construct a graph with infinite number of vertices. Each vertex corresponds to a positive integer. In this graph,

- For each x, we add an edge from x to $x+1$ with cost 1.
- For each x, we add an edge from x to $10x$ with cost 0.

The answer is (the shortest distance from 1 to one of multiples of K in this graph) plus one.

We can compree this graph to K vertices. When $x \equiv y \pmod{K}$, assume that x and y are the same vertex. Then, in this graph, the answer is the shortest distance from 1 to 0, plus one.

This can be done in $O(K)$ by 01-bfs (a variant of Dijkstra's algorithm, when all costs are 0 or 1, you can use a deque instead of priority_queue).

Note: Isn't the trouble with '9' important? Suppose that the algorithm above finds an invalid solution that contains a digit "10" at position i. However, due to the periodicity of powers of tens, (when K is comprime to 10) we can find another j such that 10^i and 10^j are the same in modulo K, so we can move one digit from i to j. Even when K is not coprime to 10, we can make it periodic by attaching sufficient number of zeroes at the end.

E: Finite Encyclopedia of Integer Sequences

If K is even, exactly half of sequences starts with an integer $K/2$ or less. Thus, the answer is $K/2$ followed by $N-1$ occurrences of K.

Suppose that K is odd. Let B be a sequence that contains N occurrences of $(K+1)/2$ (this is very close to the middle). What is the difference between the number of sequences before B and after B?

For a sequence X, let $f(X)$ be a sequence such that X_i in X is replaced with $K+1-X_i$. This is almost an bijection from a sequence after B to a sequence before B. The only exception happens for prefixes of B. Therefore, the difference is $N-1$ (the number of non-trivial prefixes of B), and B is the $ceil(X/2) + floor(N/2)$-th sequence.

We start with B, and repeat the following operation $\lfloor \frac{N-1}{2} \rfloor$ times:

- If the current sequence ends with 1, remove it.
- Otherwise, decrement the last element by 1, and while the sequence contains less than N elements, attach K at the end.

This operation takes $O(1)$ on average, so the amortized complexity of this solution is $O(N)$.

F: XorShift

Let's interpret the given binary numbers as polynomials on $\mathbb{F}_2$. That is, a binary number $a_k a_{k-1} \ldots a_1 a_0$ corresponds to a polynomial $a_k x^k + a_{k-1} x^{k-1} + \ldots + a_1 x + a_0$, here the coefficients are computed modulo 2.

You can perform the following operations for the polynomials:

- Multiply a polynomial by x. (corresponds to "double")
- Compute the sum of two polynomials. (corresponds to "xor")

Let P, Q be two given polynomials, and suppose that $degP > degQ$. By computing $P' = P + (Q * x^{degP - degQ})$, you can get a polynomial whose degree is smaller than P. Then, you can repeat the same operation for two polynomials P' and Q (when necessary, you swap P and Q). This is similar to Euclid's algorithm, and you will eventually get $gcd(P, Q)$.

Let G be the GCD of all given polynomials. As we see above, we can write G on the blackboard. Then, by using the two types of operations properly, you can write all multiples of G (H is a multiple of G if H can be written as $H = H'G$ using some polynomial H'). On the other hand, all polynomials on the blackboard are always divisible by G.

Therefore, we want to compute the number of polynomials P such that:

- P is a multiple of G.
- P is less than or equal to X (as a binary number).

Let $d = degG$ and $D = degX$. If we know all coefficients of a polynomial except for the last (least significant) d coefficients, we can uniquely determine the remaining d coefficients to make it a multiple of G. Therefore, if the first $D - d + 1$ coefficients of P are smaller than that of G (as a binary number), we can uniquely determine P. If the first $D - d + 1$ coefficients are the same, we compute the remaining d coefficients and compare it with X.

It takes $O(d^2)$ time per one gcd computation (where d is the number of digits), thus the complexity of this solution is $O(Nd^2)$. This is fast enough. You can also use bitsets to achieve $O(Nd^2/m)$ ($m = 32 or 64$).

ARC083 / ABC074 解説

三谷庸 (wo01)

2017 年 9 月 16 日

For International Readers: English editorial starts on page 6.

A 問題

$N \times N$ のマス目にはマスが N^2 個あります。そのうち A マスを白く塗るので、黒く塗るマスは $N^2 - A$ 個あります。

したがって、入力から N, A を読み込んで $N^2 - A$ を出力すればよいことになります。

以下に C++ によるコード例を示します。

```
#include<cstdio>

using namespace std;

int main(){
    int N, A;
    scanf("%d%d", &N, &A);
    printf("%d\n", N * N - A);
    return 0;
}
```

B 問題

i 番目のボールは i 番目のタイプ A のロボット、または i 番目のタイプ B のロボットで回収できます。タイプ A のロボットで回収するときはロボットの移動距離は $2x_i$ となり、タイプ B のロボットで回収するときはロボットの移動距離は $2(K - x_i)$ となります。

$i = 1, 2, \ldots, N$ について順にボールの位置を見ていき、$2x_i$ と $2(K - x_i)$ のうち小さい方 (同じであるときはその値) を足し合わせれば答えが求まります。

以下に C++ によるコード例を示します (このコード例ではループの各段階ではなく最後に答えを 2 倍しています)。

```
#include<cstdio>
#include<algorithm>

using namespace std;

const int MAX_N = 100;

int N, K;
int X[MAX_N];

int main(){
    scanf("%d", &N);
    scanf("%d", &K);
    for(int i = 0; i < N; ++i){
        scanf("%d", X + i);
    }
    int ans = 0;
    for(int i = 0; i < N; ++i){
        int tmp = min(X[i], K - X[i]);
        ans += tmp;
    }
    printf("%d\n", ans * 2);
    return 0;
}
```

C 問題

ビーカーにちょうど x [g] の水を入れられる必要十分条件は、非負整数 i, j が存在して $x = 100Ai + 100Bj$ となることです。同様に、ビーカーに y [g] の砂糖を入れられる必要十分条件は、非負整数 i, j が存在して $y = Ci + Dj$ となることです。

x, y としては F 以下の値だけを考えればよいので、x, y としてありうる値は $O(F^2)$ 時間かけてすべての i, j を試すことで列挙できます。

最後に、すべての (x, y) の組 ($O(F^2)$ 個しかありません) について、ビーカーに水を x [g] と砂糖を y [g] 入れたときに砂糖が水にすべて溶けているかどうか調べ、溶けているものについて濃度が最大のものをとって出力すればよいです。

D 問題

都市を頂点、道路を辺としてグラフ理論の言葉で説明します。

条件を満たすグラフが存在するとき、任意の頂点 u, v について、これらの間には辺がないか、長さ $A_{u,v}$ の辺があるかのどちらかとなります。なぜなら、

- u, v の間に $A_{u,v}$ よりも長い辺があった場合、この辺は取り除いても最短距離に影響を与えず、
- u, v の間に $A_{u,v}$ よりも短い辺があった場合、u, v 間の最短距離は明らかに $A_{u,v}$ よりも短くなる

からです。

グラフ G を任意の u, v の間に長さ $A_{u,v}$ の辺を張ったグラフとします。上の考察より、G の部分グラフに関してのみ考えればよいことになります。

どの頂点間に辺を張る必要があるか考えます。

A (の対角線上以外の要素) のうち $A_{u,v}$ が最小値であるとき、A が最短距離を表す表であるようなグラフには u と v を長さ $A_{u,v}$ で結ぶ辺が必要です。なぜなら、そうでないとすると他の頂点 w を経由して頂点 u から頂点 v まで距離 $A_{u,v}$ で移動できる必要がありますが、そのとき頂点 u と頂点 w の距離は $A_{u,v}$ よりも小さくなるからです。

同様に G のすべての辺について短い方から見ていくことで、求めるべきグラフは以下のとおりであることがわかります (G から辺 $\{u,b\}$ を取り除いたグラフを $G\backslash\{u,v\}$ と書いています)。

- ある u, v について、$G\backslash\{u,v\}$ における uv 最短距離が $A_{u,v}$ 未満であるとき、条件を満たすグラフは存在しない。
- そうでないとき、$G\backslash\{u,v\}$ における uv 最短距離が $A_{u,v}$ より大きいような u, v に限り辺を張ったグラフが求めるべきグラフである。

$G\backslash\{u,v\}$ における uv 最短路の長さを $A'_{u,v}$ と書くことにします。$A'_{u,v}$ と $A_{u,v}$ の大小を比較するために、まず G における全点対最短距離を Warshall-Floyd 法を用いて求め、その値を B とします。

$B_{u,v} < A_{u,v}$ となる u,v が存在するとき、この u,v について、$A'_{u,v} = B_{u,v} < A_{u,v}$ となります。よって条件を満たすグラフは存在しません。

あとは、任意の u,v について $A_{u,v} = B_{u,v}$ であったとして、$A'_{u,v}$ が $A_{u,v}$ と等しいか、それより大きいかを判定できればよいことになります。$A'_{u,v} = A_{u,v}$ であることは、以下と同値です。

$$\exists w, u \neq w, v \neq w, B_{u,v} = B_{u,w} + B_{w,v} \tag{1}$$

実際、$A'_{u,v} = A_{u,v}$ のとき、w として $G\backslash\{u,v\}$ における uv 最短路上の頂点をとれます。逆に、上のような w が取れるとき、G における w を通った uv 最短路は $G\backslash\{u,v\}$ における uv 最短路にもなっています。

よって、条件を満たすグラフが存在するとき、答えは条件 (1) を満たす u,v について $A_{u,v}(= A'_{u,v} = B_{u,v})$ の和をとったものになります。

N をグラフの頂点の数とするとき、B を求めるための Warshall-Floyd 法の計算量は $O(N^3)$ です。また、条件 (1) は各 u,v について $O(N)$ で確かめられるので、全体の計算量は $O(N^3)$ です。以上から、アルゴリズム全体の計算量は $O(N^3)$ となるので、十分高速です。

E 問題

頂点 v の子孫について色と重みの割り当てができているとして、頂点 v の色と重みを決めることを考えます。頂点 v の直接の子を $u_1, u_2, \ldots, u_k$ とします。

各 u_i について、u_i およびそのすべての子孫に割り当てた色を反転させても、頂点 u_i およびその子孫への色と重みの割り当ては条件を満たしたままです。また、これらの頂点のうち頂点 v と異なる色の重みの和 y はできるだけ小さくしたほうが v の先祖に色と重みを割り当てやすくなります。

よって、頂点 v を根とする部分木に含まれる頂点の色と重みを決めるには、次の問題を解けばよいことになります。

- $u_1, u_2, \ldots, u_k$ のうちいくつかを v と同じ色に、それら以外を v と異なる色に塗り、v の子孫 (v を除く) に含まれる v と同じ色の頂点の重みの和を X_v 以下にできるか。できる場合、それらの頂点のうち v と異なる色であるものの重みの和の最小値はいくらか。

この問題は、$x_1, x_2, \ldots, x_k$ (ただし $x_i = X_{u_i}$) の情報の他に、u_i を根とする部分木に含まれる頂点であって u_i と異なる色のものの重みの和 y_i の情報がわかれば、動的計画法により解くことができます。

与えられた木の頂点を根から遠い順に並べ、それぞれについて上の動的計画法を行うことで、与えられた木全体への色と重みの割り当てが可能かどうか判定できます。

F 問題

まず、与えられた点集合から、以下のようなグラフ G を構築します。

- 頂点集合は $(X,1), (X,2), \ldots, (X,N), (Y,1), (Y,2), \ldots, (Y,N)$ の $2N$ 個。これらにそれぞれ番号 $0, 1, \ldots, 2N-1$ を付ける。

- 頂点 (X, x) と頂点 (Y, y) は、ある i が存在して $x_i = x$ かつ $y_i = y$ であるときに限り辺で結ばれている。
- それら以外に辺は存在しない。

グラフ G の頂点はボールを回収するロボットに、辺は回収されるボールに対応します。また、辺 $\{u, v\}$ に対応するボールは、頂点 u または頂点 v に対応するロボットによってのみ回収できます。以下、このグラフに関する問題として考えることにします。

まず、G の連結成分に頂点の数と辺の数が異なるものが存在する場合、答えは 0 です。そうでない場合、G の連結成分ごとに順序を数えられれば十分であるので、以下では G が連結であるとして説明します。

ロボットの起動順序を考える前に、ロボットとそれが回収するボールの対応付けの方法を考えます。

$2N$ 頂点 $2N$ 辺のグラフにはサイクルがちょうど 1 つあります。このサイクルに含まれない頂点に対応するロボットについては、それが回収すべきボールは一意に決まります。また、サイクルに含まれる頂点に対応するロボットについては、回収すべきボールとの対応付けが (サイクル全体で) 2 とおりに決まります。これらを両方試すことにすると、ロボットとボールの対応付けが定まったことになります。

次に、ロボットの起動順序について考えます。

グラフ G において頂点 v は頂点 $u_1 < u_2 < \cdots, u_k$ に隣接しているとします。また、頂点 v に対応するロボットは辺 $\{v, u_i\}$ に対応するボールを回収することになっているとします。このとき、頂点 $u_1, u_2, \ldots, u_{i-1}$ に対応するロボットは頂点 v に対応するロボットよりも先に起動する必要があります。また、この条件がすべての v に対して満たされていれば、決めた対応のとおりにロボットがボールを回収できることもわかります。

グラフ G と同じ頂点集合について、各頂点 v について v から上の記法における $u_1, u_2, \ldots, u_{i-1}$ に向けた辺を張ったグラフを G' とします。このグラフ G' は森になるので、G' 上で動的計画法を行うことで求める順序の個数が得られます。

以上で答えが求まりました。

ARC083 / ABC074 Editorial

wo01

September 16, 2017

A

```
#include<cstdio>

using namespace std;

int main(){
    int N, A;
    scanf("%d%d", &N, &A);
    printf("%d\n", N * N - A);
    return 0;
}
```

B

```
#include<cstdio>
#include<algorithm>

using namespace std;

const int MAX_N = 100;

int N, K;
int X[MAX_N];

int main(){
    scanf("%d", &N);
    scanf("%d", &K);
    for(int i = 0; i < N; ++i){
        scanf("%d", X + i);
    }
    int ans = 0;
    for(int i = 0; i < N; ++i){
        int tmp = min(X[i], K - X[i]);
        ans += tmp;
    }
    printf("%d\n", ans * 2);
    return 0;
}
```

C

We can pour exactly x grams of water if there exist non-negative integers i, j such that $x = 100Ai + 100Bj$. Similarly, we can pour exactly y grams of sugar if there exist non-negative integers i, j such that $y = Ci + Dj$.

Since $x, y \leq F$, we can generate all possible values of x, y in $O(F^2)$ time.

Finally, for each possible pair (x, y) (There are $O(F^2)$ such pairs), consider the case where we put x grams of water and y grams of sugar, check whether it satisfies the conditions, and compute the maximum density.

D

If there exist three vertices u, v, w such that $A_{u,v} + A_{v,w} < A_{u,w}$, the distances are contradictory, thus you should output -1. From now on, we assume that there are no such triplets.

We call a (unordered) pair of two distinct vertices (u, v) *important* if there exists no w such that $w \neq u, w \neq v, A_{u,w} + A_{w,v} = A_{u,v}$.

- If (u, v) is important, the shortest path between u and v must not pass through other vertices. Thus, we must add an edge between u and v with cost $A_{u,v}$.
- If (u, v) is not important, we don't need to add any edges between these vertices. For some w, it satisfies $A_{u,w} + A_{w,v} = A_{u,v}$, and you can just follow the shortest path between u and w and the shortest path between w and v.

Therefore, the answer is the sum of $A_{u,v}$ over all important pairs (u, v).

E

Suppose that we've already decided the colors and weights of all descendants of v, and among those vertices, the conditions are satisfied. The only things that matter for the decision of colors and weights of other vertices (ancestors of v) are:

- The sum of weights of all black vertices in the subtree rooted at v.
- The sum of weights of all white vertices in the subtree rooted at v.

One of these two values must be exactly X_v. To make things better in the future, we want to minimize the other value. Let $dp[v]$ be the minimum of the other value.

How can we compute $dp[v]$? Let $u_1, u_2, \ldots, u_k$ be the children of v. Without loss of generality, we can assume that v is black.

We keep two values: B (the sum of weights of black vertices) and W (the sum of weights of white vertices). They are initialized to zeroes. Now, for each u_i, we do one of the following:

- Make u_i black: add X_{u_i} to B and $dp[u_i]$ to W.
- Make u_i white: add $dp[u_i]$ to B and X_{u_i} to W.

After that, we want to minimize the value of W, under the constraint that B must be at most X_v. This can be done in $O(kX_v)$ by a simple DP.

F

First, construct a graph G as follows:

- There are $2N$ vertices: $X_1, \ldots, X_N, Y_1, \ldots, Y_N$. (Also, let's number the vertices $0, 1, \ldots, 2N-1$ in this order.)
- For each i, add an edge between two vertices X_{x_i} and Y_{y_i}.

Each vertex in G corresponds to a robot, and each edge in G corresponds to a ball. An edge (ball) between two vertices u and v can be collected by either vertex (robot) u or vertex (robot) v.

For each connected component in this graph, the number of edges and vertices must be the same. Otherwise, the answer is 0. We will assume that the graph is connected (otherwise, solve the problem independently for each component and combine the results). Thus, we have a connected graph with $2N$ vertices and $2N$ edges.

How can we assign edges to vertices? The graph contains exactly one cycle, and for edges in the cycle, there are two ways to assign them to vertices. For other edges, the assignment can be uniquely determined.

Let's fix an assignment of edges to vertices. In how many ways can we activate the vertices to satisfy this assignment?

Suppose that in the graph G, a vertex v is adjacent to vertices $u_1 < u_2 < \cdots, u_k$. Also, suppose that v is assigned to the edge $\{v, u_i\}$. Then, the vertices $u_1, u_2, \ldots, u_{i-1}$ must be activated before v. These conditions are sufficient: if this is satisfied for all v, the robots can collect all balls.

Now, construct another graph G': for each v, add an edge from v to $u_1, u_2, \ldots, u_{i-1}$ in G'. The answer is the number of topological orderings in this graph. Since G' is a (directed) forest, it's east to compute the answer.

ABC072 / ARC082 解説

sigma425

For International Readers: English editorial starts on page 3.

A: Sandglass2

$X > t$ なら $X - t$[g] 残っていて、$X \leq t$ なら 0[g] になっています。これは $max(X - t, 0)$ と書くことも出来ます。コードにすると次のようになります。

```
int main(){
        int X,t;
        cin>>X>>t;
        if(X>t) cout<<X-t<<endl;
        else cout<<0<<endl;
        return 0;
}
```

B: OddString

for 文をまわすなどして答えの文字列を求めると良いです。

```
int main(){
        string s;
        cin>>s;
        int N = s.size();
        string ans;
        for(int i=0;i<N;i+=2) ans += s[i];
        cout<<ans<<endl;
}
```

C: Together

X を先に選んでから数に対する操作をすることにします。X を決めると、この個数を最大にするには、

- すべての $X - 1$ に $+1$ する
- すべての X をそのままにする
- すべての $X + 1$ に -1 する

をすると他は関係ないので、($X - 1$ の個数)+(X の個数)+($X + 1$ の個数) となります。従って先に各数の個数を数えておき、この X を考えうる範囲で全て試すことで答えが求まります。

D: Derangement

$p_i \neq i$ となっている部分に o を, そうでない部分に x をつけることにします。N 箇所全て o になれば ok です。ox とならんでいる部分は、swap することで oo となります (xo も同様)。($\because$ もともと $p_i = x(x \neq i)$,$p_{i+1} = i+1$ だったとすると、$x \neq i+1$ でもあるから、$p_i = i+1$ も $p_{i+1} = x$ も o になる)

xx とならんでいる部分も、swap することで oo となります。($\because$ もともと $p_i = i(x \neq i)$,$p_{i+1} = i+1$ だったとすると、$p_i = i+1$ も $p_{i+1} = i$ も o になる)

従って、先頭から順番に見ていき、箇所 i に x があったら i と $i+1(i = N$ なら $i-1$ と $i)$ を swap する、とするのが最善になります。(i にある x を消すのに $i-1, i$ の swap と $i, i+1$ の swap どちらかは必要で、前から見ると $i-1$ はもう o なので $i+1$ を巻き込んだほうが得)

E: ConvexScore

$n - |S|$ というのは,「S の凸包 (境界含む) に含まれる与えられた点のうち S の点を除いた集合」(T_S とおく) の要素数と等しいです。なので、$2^{n-|S|}$ は T_S の部分集合の個数と等しくなります。

ここで、凸包の面積が正の部分集合 X を任意に取ってきます。そして X の凸包の頂点集合を S_X とします。すると、S を決めた時、$\{X|S = S_X\}$ と $\{S \cup t|t \in T_S\}$ は集合として等しいです。

このことから、凸包の面積が正の部分集合 X はちょうど一回 $S = S_X$ の時に一回分カウントされる、とみなすことが出来ます。

従って、答えは凸包の面積が正の部分集合の個数になります。これは $2^N -$ (共線な (一直線上に載せることの出来る) 頂点集合の数) となるので共線な頂点集合の数を計算すれば良いです。0 点集合は 1 個,1 点集合は N 個で固定なので、2 点以上で共線なものの個数を数えます。

これは、2 点以上乗りうる直線（高々 ${}_NC_2$ 本) をすべて試し、この上に k 点乗っていれば $2^k - k - 1$ を足すことで求まります。計算量は $O(N^3)$ です。

直線を正規化し set で管理するなどによって $O(N^2 logN)$ で解くことも出来ます。

F: Sandglass

t 秒後にパーツ A に入っている砂の量を、はじめパーツ A に入っている砂の量 x の関数とみて $f_t(x)$ とおきます。

すると、$f_t(x)$ は常に次のような形の関数になります。

$$f_t(x) = \begin{cases} a + c & (0 \leq x \leq a) \\ x + c & (a < x < b) \\ b + c & (b \leq x \leq X) \end{cases}$$

(ただし $a = b$ となり定数関数に潰れている場合もあります) これは、d を定数として、$f(x) + t$,$max(f(x), 0)$,$min(f(x), X)$ が上述の形で閉じていることからわかります。(時間の経過やパーツに入るのが 0 以上 X 以下であることから起こる $f_t(x)$ の変化は全て上の 3 つのどれかの合成として書けるので、この 3 つだけ考えれば良い)

よって、上の関数に含まれる a, b, c を保持して更新しながらその都度クエリに答えていけば良いです。

ABC072 / ARC082 Editorial

sigma425

A: Sandglass2

```
int main(){
        int X,t;
        cin>>X>>t;
        if(X>t) cout<<X-t<<endl;
        else cout<<0<<endl;
        return 0;
}
```

B: OddString

```
int main(){
        string s;
        cin>>s;
        int N = s.size();
        string ans;
        for(int i=0;i<N;i+=2) ans += s[i];
        cout<<ans<<endl;
}
```

C: Together

Let's choose X first. For a fixed X, the optimal choices are:

- Add 1 to all $X-1$.
- Do nothing for all X.
- Subtract 1 from all $X+1$.

The number of X after these operations is (the number of $X-1$ in the initial sequence) + (the number of X in the initial sequence) + (the number of $X+1$ in the initial sequence). We can compute the optimal value by trying all possible values for X.

D: Derangement

Define a sequence of 'o' and 'x' of length N as follows: if $p_i \neq i$, the i-th symbol is 'o', otherwise the i-th symbol is 'x'. Our objective is to change this sequence to 'ooo...ooo'.

- If there is a part "ox" (or "xo") in the sequence, we can change it to "oo" by swapping these two elements. ($\because$ If $p_i = x (x \neq i)$ and $p_{i+1} = i + 1$ in the initial sequence, after the swap, both $p_i = i + 1$ and $p_{i+1} = x$ will be 'o'.)
- If there is a part "xx" in the sequence, we can change it to "oo" by swapping these two elements. ($\because$ If $p_i = i (x \neq i)$ and $p_{i+1} = i + 1$ in the initial sequence, after the swap, both $p_i = i + 1$ and $p_{i+1} = i$ will be 'o'.)

Thus, we should check the sequence from left to right, and if we find an 'x' at the i-th position, we should swap i and $i + 1$ (unless $i = N$, in this case we should swap i and $i - 1$).

E: ConvexScore

For a set S that forms a convex polygon (from now on, we call it "convex set"), let T_S be the set of points in the convex hull of S, except for the vertices (points in S). Since $|T_S| = n - |S|$, $2^{n-|S|}$ corresponds to the number of subsets of T_S.

In the problem, for each convex set S, we are asked to count it $2^(n - S)$ times. Instead, for each pair of sets (S, U) such that S is a convex set and U is a subset of T_S, let's count $S \cup U$ once.

- When we know $S \cup U$, we can recover S: it must be the convex hull of $S \cup U$. Thus, this way each set is counted at most once.
- A set of points X is counted if and only if the convex hull of X has positive area: let S be the convex hull of X and U be $X \setminus S$.

Therefore, the answer is equal to the number of subsets of all N points whose convex hull has a positive area.

This can be computed by subtracting the number of colinear sets (sets whose points are on the same line) from 2^N. This can be done in $O(N^3)$ or $O(N^2 logN)$.

F: Sandglass

Let the amount of sand in bulb A at time t be a function $f_t(x)$, where x is the initial amount of sand in bulb A. We can prove that this function is always of the following form (for some constants a, b, c):

$$f_t(x) = \begin{cases} a + c & (0 \leq x \leq a) \\ x + c & (a < x < b) \\ b + c & (b \leq x \leq X) \end{cases}$$

When bulb A contains y grams of sand at some time, one second later, the amount of sand in bulb A

is either $g_1(y) = max(y-1, 0)$ (in case A is above B) or $g_2(y) = max(y+1, X)$ (in case B is above A). Since the set of functions of the form above is closed under functions g_1 and g_2, f_t is always of this form.

Therefore, we can simulate the process while keeping the function f_t (i.e., parameters a, b, c) and answer queries.

AtCoder Beginner Contest 071 / AtCoder Regular Contest 081 解説

writer: semiexp

2017 年 8 月 20 日

For International Readers: English editorial starts on page 5.

A: Meal Delivery

定義どおりに，すぬけ君の住んでいる位置と，店 A, B の間の距離を求めます．この距離の大小に応じて，より近いほうを出力すればよいです．

- C++ による解答例: `https://abc071.contest.atcoder.jp/submissions/1518940`
- Ruby による解答例: `https://abc071.contest.atcoder.jp/submissions/1518942`
- Rust による解答例: `https://abc071.contest.atcoder.jp/submissions/1516215`

B: Not Found

英小文字 26 種類のそれぞれについて，S の中に現れるかを記録する配列を持っておきます．この配列は最初すべて「現れない」(`false`) で初期化しておきます．S を 1 文字目から順に見ていって，出てきた文字それぞれについて，配列上でその文字に対応する位置を「現れる」(`true`) に更新します．S の最後の文字まで見終わった後には，この配列を見るだけである文字が S の中に現れるかわかるようになります．

最後に，この配列を，`a`, `b`, ..., `z` の順に見ていって，`false` となった最初の文字を出力すればよいです．ただし，`z` まで見終わっても `false` が現れなかった場合は，すべての英小文字が S 中に現れているということなので，`None` を出力します．

- C++ による解答例: `https://abc071.contest.atcoder.jp/submissions/1518956`
- Ruby による解答例: `https://abc071.contest.atcoder.jp/submissions/1518966`
- Rust による解答例: `https://abc071.contest.atcoder.jp/submissions/1516216`

C: Make a Rectangle

$H \times W$ の長方形を作るためには，長さ H, W の棒が 2 本ずつ必要です．ただし，$H = W$ の場合は，長さ H の棒が 4 本必要です．

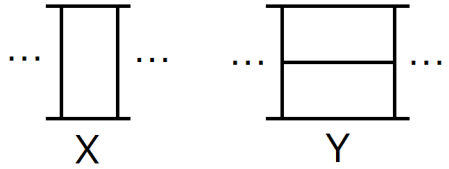
各辺に使う棒は，上の条件のもとで，できるだけ長い棒を貪欲に使えばよいです．$\{A_i\}$ をソートすると，同じ値は連続した区間に集まるため，各長さの棒が何本あるかを知ることができます．なので，長さの長い順に見ていって，次のように辺の長さを決定すればよいです．

- 同じ長さの棒が全部で 4 本以上ある場合: その長さの棒を縦，横両方に使う (すでに片方の辺が決定している場合はそちらを優先)
- 同じ長さの棒が全部で 2 本または 3 本ある場合: その長さの棒を縦，横のうち片方に使う
- 同じ長さの棒が全部で 1 本しかない場合: その長さの棒は使えない

途中で両方の辺が決定した場合は，長方形が作れているので，その長方形の面積 (縦 $\times$ 横) を出力します．最後の辺まで見ても長方形の辺が足りない場合は，長方形を作ることはできないので，0 を出力します．

D: Coloring Dominoes

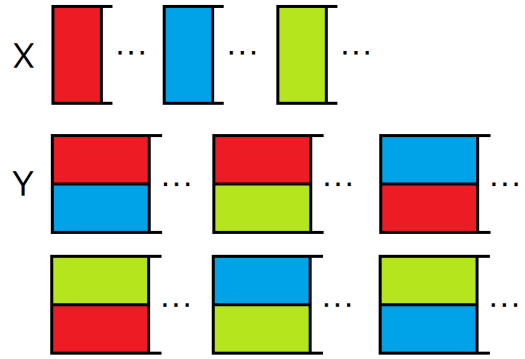
ドミノの並べ方は，縦 2 $\times$ 横 1 のドミノを 1 個並べるか (X) ，縦 1 $\times$ 横 2 のドミノを 2 個縦に並べるか (Y) のどちらかを，横につなげたような並べ方になります．



左から順にドミノを塗ることを考えます．縦に 2 個ドミノが並んでいる箇所については，この 2 個を同時に塗ります．一番左の箇所については，

- X: 3 通り
- Y: 6 通り

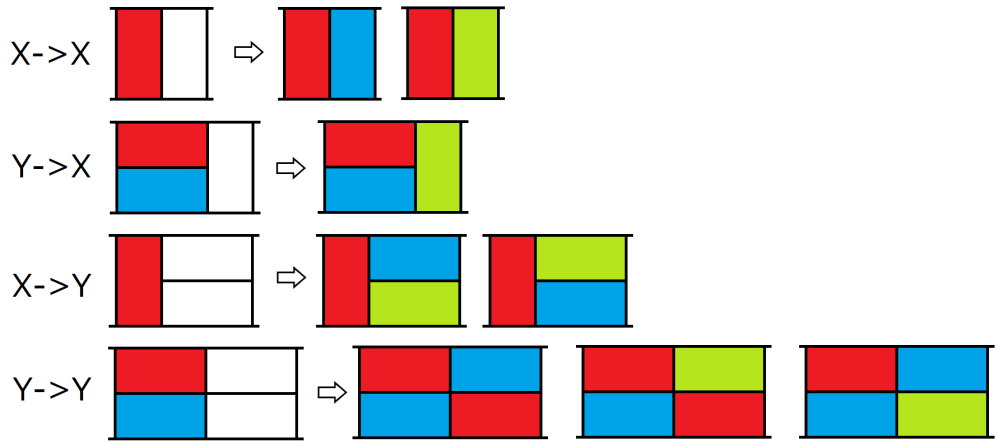
の塗り方があります．



また，それ以外の箇所については，それより左の箇所がすべて塗られているとしたとき，

- X (すぐ左が X): 2 通り
- X (すぐ左が Y): 1 通り
- Y (すぐ左が X): 2 通り
- Y (すぐ左が Y): 3 通り

の塗り方があります．



よって，ドミノの並べ方を判定した上で，左から順に上の規則で並べ方の個数を求めていけばよいです．

E: Don't Be a Subsequence

以下，文字列は英小文字のみからなるもののみを考えます．

文字列 A に対して,「その文字列の suffix であって，すべての英小文字を含むもののうち，最も短いもの」を順次取り除くことを考えます．ただし，そのようなものが存在しなくなったら終了します．この時，suffix を取り除くことができた回数を K とします．また，i 番目に取り除いた suffix を S_{K-1-i}，最後に残った文字列を A' とします．このとき，A は $A', S_1, S_2, \ldots, S_K$ をこの順に並べた文字列になります．

A の部分列でない最短の文字列の長さは $K+1$ になることを示します．まず，長さ K の任意の文字列 B が A の部分列であることを示します．(長さ K 未満の場合は，末尾に適当に文字を加えて長さ K にした場合について示されるので十分です) $S_1, \ldots, S_K$ はいずれもすべての英小文字を含むため，S_i を構成する文字を B の i 文字目に一致する文字 1 個を除いて取り除くことができます．また，A' を構成する文字はすべて取り除きます．すると，S から文字を取り除いて B が得られるため，B は S の部分列です．

次に，A の部分列でない長さ $K+1$ の文字列が存在することを示します．文字列 $C = c_0 c_1 \ldots c_K$ を次で定めます．

- c_0 は，A' に現れない文字のうち 1 つとする (A' からは suffix を取り除けなかったことから，このような文字は必ず存在します)．
- c_i $(i = 1, 2, \ldots, K)$ は，S_i の最初の文字とする．

すると，C は A の部分列ではありません．実際，A を構成する文字をいくつか取り除いて C を得ようとすると，

- c_0 が A の中で初めて現れるのは S_1 の中であるため，A' はすべて取り除かなければなりません．
- c_1 は S_1 の中では 1 文字目にしか現れないため (取り除く suffix の最小性より)，c_0 の次に c_1 を得るためには，S_1 は c_0 を得るために残した文字を除いてすべて取り除かなければなりません．
- ...
- c_K は S_K の中では 1 文字目にしか現れないため，c_{K-1} の次に c_K を得るためには，S_K は c_{K-1} を得るために残した文字を除いてすべて取り除かなければなりません．すると，c_K のために使える文字が A にまったく残らなくなってしまします．

よって，A の部分列でない最短の文字列の長さは $K+1$ になることがわかりました．

上の議論から，suffix を取り除く作業を行うことで，A の任意の suffix についても「部分列でない最短の文字列の長さ」を求めることができます．

すると，1 文字目から貪欲に文字を決定していくことで，辞書順最小の答えを求めることができます．文字 c を使うことができるかどうかは，現在の位置以降の最初の c の現れの位置 p_c を求め，S の p_c 文字目以降についての「部分列でない最短の文字列の長さ」が，現在の位置以降の「部分列でない最短の文字列の長さ」よりちょうど 1 小さくなっているかで判定することができます．p_c は，各文字の現れる位置をソートして持っておくと，二分探索で $O(\log |A|)$ で求めることができます．

各位置に対しては文字種を m として $O(m \log |A|)$ で次の文字を決定することができ，答えの長さは $O(|A|/m)$ なので，以上のアルゴリズムにより，答えを $O(|A| \log |A|)$ で求めることができます．

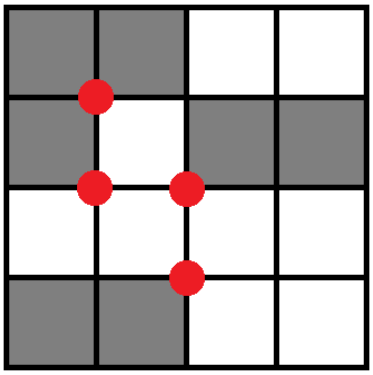
F: Flip and Rectangles

マス目に含まれる 2×2 の部分であって，黒マスを奇数個 (1 個または 3 個) 含むものを「悪い部分」とよぶことにします．うまく操作を行えば長方形 S を選ぶことができることと，S が悪い部分を含まないことは同値です．

実際，S が悪い部分を含むならば，どう操作を行ってもその部分をすべて黒で塗られた状態にすることはできません．

次に S が悪い部分を含まない場合を考えます．このとき，S の一番上の行，一番左の列をすべて黒く塗られた状態にすることができます．ここで，どのような操作を行っても，悪い部分が悪くない部分に変わったり，その逆が起きたりすることはないことに注意します．よって，この状態においても，S は悪い部分を含まないため，S の上から 2 行目，左から 2 列目のマスは黒で塗られています．同様に，上から 2 行目，左から 3 列目，... のマスはすべて黒で塗られていることがわかり，上から 2 行目はすべて黒で塗られていることがわかります．さらに同様にして，上から 3 行目，... のマスもすべて黒で塗られていることがわかり，結果として S 全体が黒で塗られていることがわかります．よって，S 全体を黒で塗られた状態にすることができ，S を選ぶことができることがわかります．

ここで，マス目において．すべての悪い部分の中心の格子点に，印を付けることを考えます．すると，選ぶことができる長方形は，内部に (頂点や辺上にはあってもよい) 印を含まない長方形になります．



この「印を含まない長方形」は，最大長方形のアルゴリズムを用いると $O(HW)$ で求めることができます．

AtCoder Beginner Contest 071 / AtCoder Regular Contest 081 Editorial

writer: semiexp

August 20th, 2017

A: Meal Delivery

- C++ solution: `https://abc071.contest.atcoder.jp/submissions/1518940`
- Ruby solution: `https://abc071.contest.atcoder.jp/submissions/1518942`
- Rust solution: `https://abc071.contest.atcoder.jp/submissions/1516215`

B: Not Found

- C++ solution: `https://abc071.contest.atcoder.jp/submissions/1518956`
- Ruby solution: `https://abc071.contest.atcoder.jp/submissions/1518966`
- Rust solution: `https://abc071.contest.atcoder.jp/submissions/1516216`

C: Make a Rectangle

In order to make an $H \times W$ rectangle, you need two sticks of length H and two sticks of length W. In case $H = W$, you need four sticks of length H.

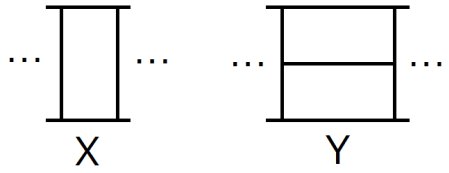
Under the constraints above, you should choose the longest possible sticks greedily. First, you sort $\{A_i\}$ in the decreasing order. Since sticks of the same lengths are grouped together after the sorting, we can count the number of sticks of each length. We check the sticks in the decreasing order of their lengths, and do the following:

- If we haven't chosen any sticks and we find four (or more) sticks of the same length, we choose them and finish the process.
- If we haven't chosen any sticks and we find two or three sticks of the same length, we choose them (and continue the process).
- If we have chosen two sticks and we find two (or more) sticks of the same length, choose two of them and finish the process.
- Otherwise, skip sticks of current length.

If we successfully choose four sticks after this process, print the area of the rectangle. Otherwise, print 0 instead.

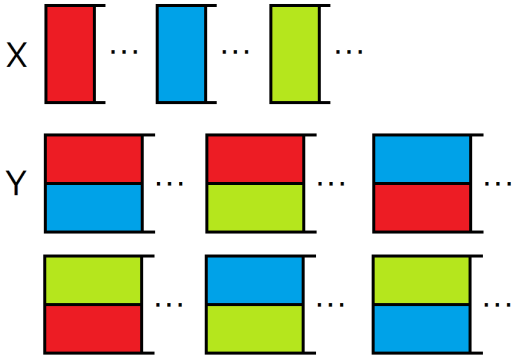
D: Coloring Dominoes

The arrangement of dominoes is a concatenation of the following two patterns:



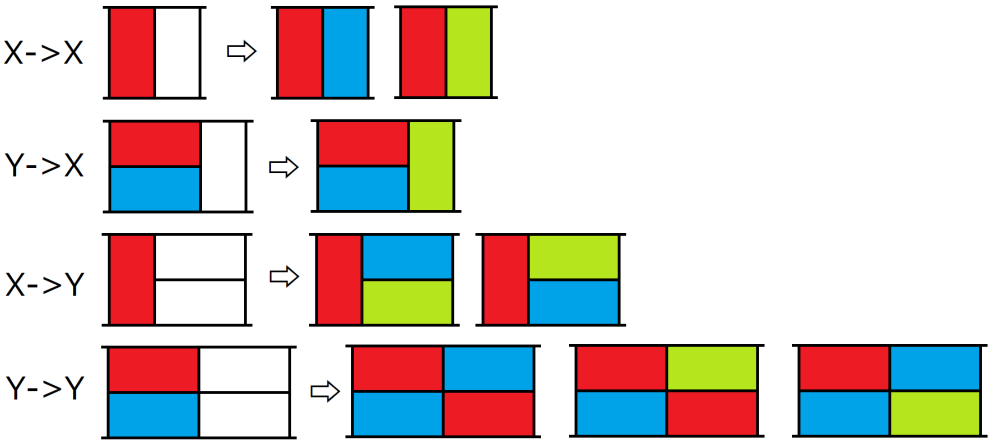
We paint dominoes from left to right.
The number of ways to paint the leftmost part is as follows:

- X: 3
- Y: 6



For other parts, the number of ways to paint a part is as follows (assuming that the left part is already painted):

- X (to the right of X): 2
- X (to the right of Y): 1
- Y (to the right of X): 2
- Y (to the right of Y): 3



The answer is the product of these numbers.

E: Don't Be a Subsequence

First, let's compute the length of the desired string. Later, we'll describe how to extend the solution to compute the lexicographically smallest string.

Let $f(s)$ be the length of the shortest string that is not a subsequence of s. Suppose that t is not a subsequence of s. What t should we choose, assuming that t starts with a c (arbitrary character)?

- If s doesn't contain c, the length of t can be 1: a single c.
- Otherwise, let i be the leftmost occurrence of c in s ($s[i] = c$). t is not a subsequence of s if and only if $t.substr(1)$ is not a subsequence of $s.substr(i + 1)$.

Here, $s.substr(i)$ means the suffix of s from the i-th character.

Therefore, we can compute the length of $f(s)$ using the following DP:

Define $dp[i]$ as the length of $f(s.substr(i))$. Also, precompute $next(i, c)$: the minimum index j such that $j \geq i$ and $s[j] = c$. Then,

$$dp[i] = \min_c \{dp[next(i, c) + 1]\} + 1 \tag{1}$$

holds.

How can we compute the lexicographically smallest string? The first character of the answer should be the lexicographically smallest character c that satisfies:

$$dp[i] = dp[next(i, c) + 1] + 1 \tag{2}$$

The remaining part of the solution should be the lexicographically smallest shortest string that is not a subsequence of $s.substr(next(0, c))$, so we can repeat the same process.

This solution works in $O(|A|m)$, where $m = 26$ is the number of characters.

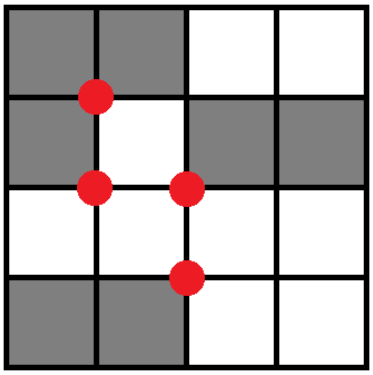
F: Flip and Rectangles

Consider a 2×2 subsquare inside the grid. If it contains odd (1 or 3) number of black squares, we call it "bad". We can prove that, we can choose a rectangle S (after some "invert" operations) if and only if S doesn't contain any bad parts.

If S contains a bad part, the bad part always contain odd number of black squares even after operations, so we can never choose S.

Suppose that S doesn't contain bad parts. By performing operations properly, we can make all squares in the topmost row and the leftmost column of S black. Notice that a good part (not bad part) always remain good after operations. Thus, when all squares in the topmost row and the leftmost column of S are black, the square on the second row of the second column of S must be black. Similarly, we can prove that all squares in S are black.

Now, let's put a token on the center of each bad part. We can choose a rectangle S if and only if S doesn't contain any tokens.



This problem can be solved in the same way as the well-known problem "find the largest rectangle that consists only of black squares".

ABC #069 / ARC #080 Editorial

writer : sugim48

2017 年 8 月 6 日

For International Readers: English editorial starts on page 5.

A : K-City

南北方向の通りの本数は n なので，それらの間の個数は $n-1$ です．同様に，東西方向の通りの本数は m なので，それらの間の個数は $m-1$ です．区画の個数はこれらの積なので，$(n-1)(m-1)$ です．

- C++ のコード例 : https://abc069.contest.atcoder.jp/submissions/1480689
- Java のコード例 : https://abc069.contest.atcoder.jp/submissions/1480691
- Python 3 のコード例 : https://abc069.contest.atcoder.jp/submissions/1480692

B : i18n

s の先頭文字，$|s|-2$，s の末尾文字の順に出力すればよいです．ただし，$|s|$ は s の長さを表します．

- C++ のコード例 : https://abc069.contest.atcoder.jp/submissions/1480693
- Java のコード例 : https://abc069.contest.atcoder.jp/submissions/1480695
- Python 3 のコード例 : https://abc069.contest.atcoder.jp/submissions/1480696

C : 4-adjacent

2 で割り切れる回数によって a の要素を分類します．0 回ならば ① と書き，1 回ならば ② と書き，2 回以上ならば ④ と書くことにします．また，①, ②, ④ の個数をそれぞれ b_1, b_2, b_4 とします．すると，「隣り合う要素の積が 4 の倍数である」という条件は，「① と隣り合う要素は ④ でなければならない」という条件へ言い換えられます．

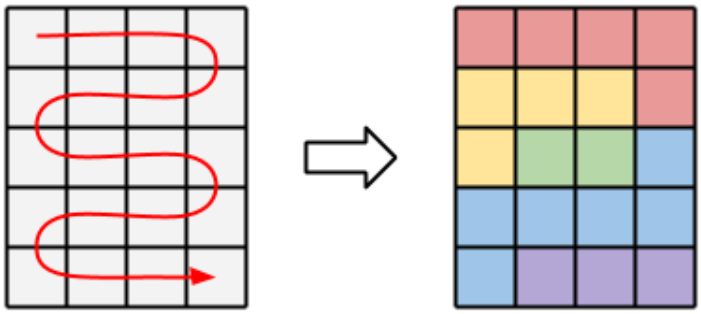
まず，② が存在しない場合を考えます．この場合，①④①④① のように並べるのが最適です．よって，$b_1 \leq b_4 + 1$ ならば `Yes`，そうでないならば `No` となります．

次に，② が存在する場合を考えます．例として ②④①④②②④②② と並べてみます．この例を見ると，ひと繋がりの ② は単体の ① と等価であることが分かります．① はできるだけ少ない方が嬉しいので，② はすべてまとめてひと繋がりにするのが最適です．すると，① を 1 個だけ増やした上で，② が存在し

ない場合へ帰着できます．よって，$b_1 \leq b_4$ ならば `Yes`，そうでないならば `No` となります．具体的には，①④①④①④②②② のように並べるのが最適です．

D : Grid Coloring

次図のようにマスを順序付けした上で，最初の a_1 マスを色 1 で塗り，次の a_2 マスを色 2 で塗り，…，最後の a_N マスを色 N で塗ればよいです．



E : Old Maid

p, q 上の位置を左から順に 0, 1, ..., $N-1$ とします．辞書順で最小の列を求める問題では，「列の先頭から順に，その時点であり得る最小の値を確定していく」という方法が有効です．

まず，q_0 としてあり得る最小の値を確定しましょう．q_0 になり得るのは，p 上で偶数番目の要素です．これらのうち最小の要素を q_0 として確定します．また，この要素の p 上の位置を i とします．次に，q_1 としてあり得る最小の値を確定しましょう．q_1 になり得るのは，p 上で奇数番目の要素であって，位置 i より右にあるものです．これらのうち最小の要素を q_1 として確定します．また，この要素の p 上の位置を j とします．すると，p の残った要素は，3 つの区間 $[0, i), [i+1, j), [j+1, N)$ へ分割されます．続いて，q_2 としてあり得る最小の値を確定しましょう．q_2 になり得るのは，各区間上で偶数番目の要素です．これらのうち最小の要素を q_2 として確定します．また，この要素が区間 S 上にあるとし，この要素の S 上の位置を i' とします．次に，q_3 としてあり得る最小の値を確定しましょう．q_3 になり得るのは，S 上で奇数番目の要素であって，位置 i' より右にあるものです．これらのうち最小の要素を q_3 として確定します．また，この要素の S 上の位置を j' とします．すると，S の残った要素は，位置 i', j' を境にして 3 つの区間へ分割されます．以上のような手続きを繰り返せば，辞書順で最小の q が求まります．

以上の方法を愚直に実装すると，全体で $\mathrm{O}(N^2)$ 時間となり TLE してしまいます．TLE を回避するためには，1) ある区間上の偶数 (奇数) 番目の要素の最小値を求めるステップ，2) 次にどの区間から要素を選ぶかを決めるステップ，の両方を高速化する必要があります．1) については，前処理として，p の偶数 (奇数) 番目の要素だけを取り出したものをセグメント木に乗せておけばよいです．このとき，最小値と同時に位置も得られるようにしておきます．2) については，今あるすべての区間を優先度付きキューに入れておき，各区間の優先度には，その区間の偶数番目の要素の最小値を設定すればよいです．以上の方法は，全体で $\mathrm{O}(N \log N)$ 時間で，十分に高速です．

F : Prime Flip

各カードの向きによって，数列 $a = (a_1, a_2, \ldots)$ を定義します．具体的には，カード i が表ならば $a_i = 1$ で，カード i が裏ならば $a_i = 0$ とします．さらに，a の差分をとった数列 $b = (b_0, b_1, b_2, \ldots)$ を定義します．具体的には，$b_i = a_{i-1} \oplus a_i$ とします．ただし，$a_0 = 0$ とします．すると，カード $[l, r)$ をそれぞれひっくり返すという操作は，b_l, b_r の 0/1 をそれぞれ反転するという操作と等価です．また，すべてのカードが裏向きであるという状態は，すべての b_i が 0 であるという状態と等価です．以上より，問題は次のように言い換えられます．

> 0/1 の列 $b = (b_0, b_1, b_2, \ldots)$ がある．「0 以上の整数 l と 3 以上の素数 p を選び，b_l, b_{l+p} の 0/1 をそれぞれ反転する」という操作を繰り返し行うことができる．すべての b_i を 0 にするために必要な操作回数の最小値を求めよ．

操作の順序は結果に影響しません．また，ある時点で $b_i = 1$ であるならば，後に必ず b_i を反転する操作を行う必要があります．よって，各操作において b_l または b_{l+p} の少なくとも一方は 1 である，と仮定できることが分かります．以上より，問題は次のように言い換えられます．

> マス 0, 1, 2, ... がある．最初，$b_i = 1$ ならばマス i には駒が 1 つ置かれており，$b_i = 0$ ならばマス i は空である．次の操作を繰り返し行うことができる．
>
> - 3 以上の素数 p を選ぶ．ある駒をちょうど p だけ離れたマスへ移動する．移動先のマスに既に駒があるならば，2 つの駒は共に消える．
>
> すべての駒を消すために必要な操作回数の最小値を求めよ．

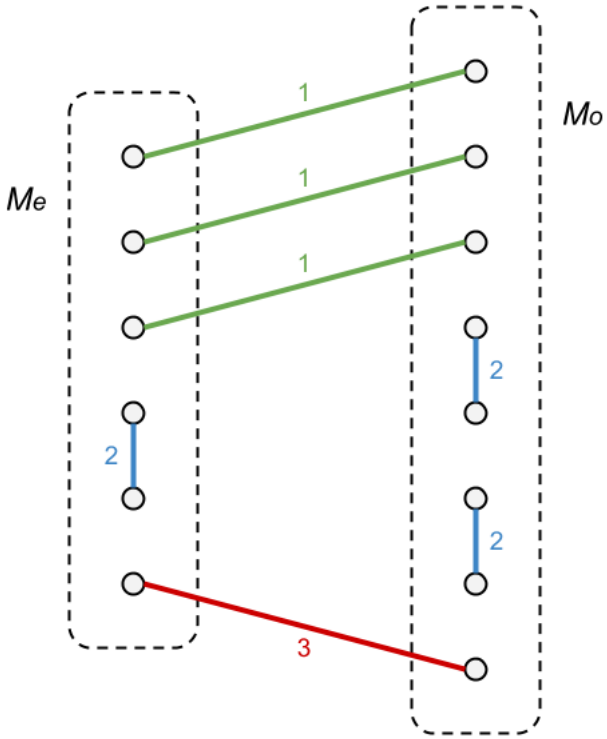
b の定義より，初期状態において駒の個数は偶数です．これを M とします．M 個の駒を $M/2$ 組のペアに分け，各ペアの 2 つの駒を共に消すことにします．このときの操作回数の最小値は，各ペアを消すための最小手数の総和になります．マス i, j $(i \neq j)$ 上の 2 つの駒を共に消すための最小手数を $d(i, j)$ とすると，

$$d(i,j) = \begin{cases} 1 & (|i-j| \text{ が奇数であって，3 以上の素数である}) \\ 2 & (|i-j| \text{ が偶数である}) \\ 3 & (|i-j| \text{ が奇数であって，3 以上の素数でない}) \end{cases}$$

となります (ゴールドバッハ予想を主に用いて示せます)．初期状態において，偶数 / 奇数マス上の駒の個数をそれぞれ M_e, M_o とします．ここで，最小手数が 1 のペアを k $(k \leq M_e, M_o)$ 組作ることができたとします．すると，残った駒たちの最適なペア分けは，偶数マス上の駒たちから $\lfloor (M_e - k)/2 \rfloor$ 組ペアを作り，奇数マス上の駒たちから $\lfloor (M_o - k)/2 \rfloor$ 組ペアを作り，もし $M_e - k, M_o - k$ が奇数ならば，余った偶数マス上の駒と奇数マス上の駒で 1 組ペアを作る，というようになります (次図)．このときの最小手数の総和は

$$k \times 1 + (\lfloor (M_e - k)/2 \rfloor + \lfloor (M_o - k)/2 \rfloor) \times 2 + ((M_e - k)\%2) \times 3$$

となりますが，これは k の増加に対して広義単調減少です．よって，k の最大値を求められればよいことが分かります．これは二部グラフの最大マッチングなので，フローを用いて簡単に解くことができます．


Me
Mo
1
1
1
2
2
2
3

ABC #069 / ARC #080 Editorial

writer : sugim48

2017 年 8 月 6 日

A : K-City

- C++ Code Example : https://abc069.contest.atcoder.jp/submissions/1480689
- Java Code Example : https://abc069.contest.atcoder.jp/submissions/1480691
- Python 3 Code Example : https://abc069.contest.atcoder.jp/submissions/1480692

B : i18n

- C++ Code Example : https://abc069.contest.atcoder.jp/submissions/1480693
- Java Code Example : https://abc069.contest.atcoder.jp/submissions/1480695
- Python 3 Code Example : https://abc069.contest.atcoder.jp/submissions/1480696

C : 4-adjacent

- When an element in a is an odd number, we represent it as ①.
- When an element in a is an even number but not divisible by 4, we represent it as ②.
- When an element in a is divisible by 4, we represent it as ④.

Let b_1, b_2, b_4, be the number of ①, ②, ④. We want to arrange them such that all elements that are adjacent to ① are ④.

In case $b_2 = 0$

The optimal arrangement will look like ①④①④①. If $b_1 \leq b_4 + 1$, the answer is `Yes`, otherwise the answer is `No`.

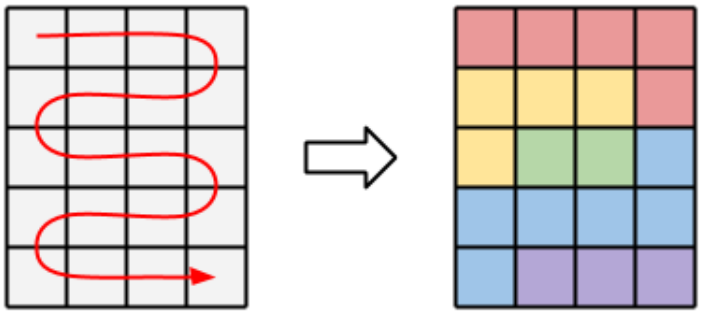
In case $b_2 > 0$

For example an arrangement will look like ②④①④②②④②②. Here, a maximal consecutive sequence of ② is equivalent to ①. Thus, we should concatenate all ② and handle them like a single ①. If

$b_1 \leq b_4$, the answer is `Yes`, otherwise the answer is `No`. In particular, we should arrange them like ①④①④①④②②②.

D : Grid Coloring

We number the cells as follows. Then, we color the first a_1 cells with color 1, the next a_2 cells with color 2, and so on.



E : Old Maid

Here we use 0-based indices, i.e., $p = (p_0, \ldots, p_{N-1}$ and $q = (q_0, \ldots, q_{N-1})$.

Which element can be q_0? It must be at an even position in p. Since we want to minimize q lexicographically, we want q_0 to be minimized. We can determine q_0 as the minimum number among elements in p at even positions. Let $q_0 = p_i$.

Which element can be q_1? If $q_1 = p_j$, j must be odd, and $j > i$. We should choose j that minimizes p_j under the constraints above.

Now, p is divided into (at most) three intervals: $[0, i)$, $[i + 1, j)$, $[j + 1, N)$.

How should we choose q_2? In p, it must be in one of the three intervals above, and the parity of its position must be the same as the parity of the first element of its intervals. We should choose q_3 from the same intervals (in the same way as q_1). By repeating this, we get an $O(N^2)$ solution.

To make it $\mathrm{O}(N \log N)$,

- We should use Range Minimum Query to get the minimum in an interval.
- We should keep all intervals in a priority_queue. The priority of an interval $[l, r)$ is the minimum of p_i such that $l \leq i < r, i \equiv l \pmod 2$.

F : Prime Flip

Define b_i as follows:

- If the card i and the card $i - 1$ are faced in the same way, $b_i = 0$.

- Otherwise, $b_i = 1$.

(Assume that card 0 is face down.)

When you perform an operation on cards $[l, r)$, you flip b_l and b_r. Thus, we can restate the problem as follows:

> You are given a sequence of 0/1, $b = (b_0, b_1, b_2, \ldots)$. In each operation you can choose a (non-negative) integer l and an odd prime p, and flip both b_l and b_{l+p}. Compute the number of operations required to transform the sequence to $0, 0, 0, \ldots$.

Consider each 1 in the sequence as a token. In each operation, you can move a token by a distance of odd prime. When two tokens meets, they disappear altogether.

When there are two tokens at i and j, $d(i, j)$, the cost required to erase them, can be computed as follows:

- If $|i - j|$ is an odd prime, $d(i, j) = 1$.
- If $|i - j|$ is an odd composite, $d(i, j) = 3$.
- If $|i - j|$ is even, $d(i, j) = 2$.

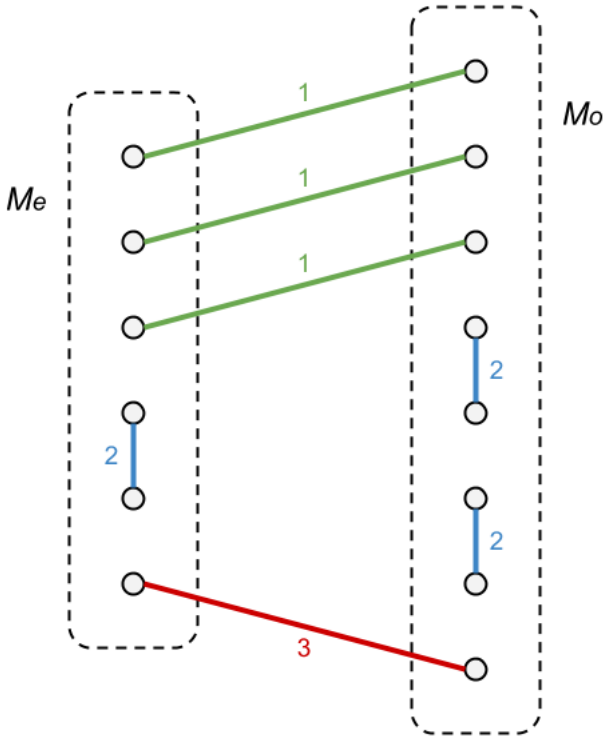
(Strictly speaking, this is related to distribution of primes, for example we have to prove that arbitrary even number can be a difference of two primes. We confirmed this up to the constraints of the problem experimentally.)

Let $y_1, \ldots, y_M$ be the initial positions of the tokens. We want to divide them into $M/2$ pairs, $(a_1, b_1), \ldots, (a_{M/2}, b_{M/2})$, and minimize the sum of $d(a_i, b_i)$.

Let M_e, M_o be the number of tokens initially at even, odd positions, respectively. If we make k ($k \leq M_e, M_o$) pairs of cost 1, the total cost will be:

$$k \times 1 + (\lfloor (M_e - k)/2 \rfloor + \lfloor (M_o - k)/2 \rfloor) \times 2 + ((M_e - k)\%2) \times 3$$

Since this is a monotonous function of k, we want to maximize k. This maximum k can be computed by a bipartite matching.


Me
Mo
1
1
1
2
2
2
3

AtCoder Regular Contest 079 Editorial

Kohei Morita(yosupo)

平成 29 年 7 月 29 日

A: ABCxxx

まず、標準入力から n を入力します。
次に、ABC という文字列を出力します。
最後に、最初に入力した n を出力し、改行文字を出力します。
C++のコード例は以下です。

```cpp
#include <iostream>

using namespace std;

int main() {
    int n;
    cin >> n; // n を入力
    cout << "ABC"; // ABC を出力
    cout << n; // n を出力
    cout << endl; // 改行文字
    return 0;
}
```

B: Break Number

たとえば、2 で 3 回割れる数のうち最も小さいものはなんでしょうか？

これは、2 を 3 回かけた数、つまり 8 です。

同様に、2 で k 回割れる数のうち最も小さいものは 2 を k 回かけた数です。

2 を k 回かけた数、という形になっているのは、100 以下だと $1, 2, 4, 8, 16, 32, 64$ の 7 種類しかありません。

よって、この 7 種類のうち、N 以下で最も大きいものを出力すれば良いです。

C: Cat snuke and a voyage

島 $i(i = 2, 3, ..., N-1)$ について、島 1 と島 i、島 i と島 N のどちらの間にも定期便が通っているような i が存在するかどうかを調べれば良いです。

最初に辺をハッシュマップに入れると、島 1 と島 i の間に辺があるかを $O(1)$ で調べることができます。

他にも、長さ N の boolean 配列を 2 つ用意して、島 1 と島 i の間に辺があるか、島 i と島 N の間に辺があるか、を格納しても良いです。こちらのほうが定数倍が速いです。

よって、$O(N)$ で判定を行うことができます。

辺の前処理に $O(M)$ かかるため、まとめて $O(N + M)$ です。

なお、ダイクストラ法というアルゴリズムを使用すると、最小でいくつの定期便を使えば島 N にたどり着けるかも調べることができます。

D: Decrease (Contestant ver)

操作を逆から考えます。

すると、1 つ要素を選び、値を N 増やす。そしてそれ以外を 1 減らす。となります。更に条件として、

1. 逆操作後に、選んだ要素の値が最大になっている
2. 逆操作後に、選ばれなかった要素の値が -1 以下になっていない

を満たしている必要があります。

すべての要素が 0 以上 $N-1$ の数列から、以上の条件を満たしながら K 回逆操作を行うことが出来れば、その数列が答えとなることが示せます。

結論から言うと、$0, 1, 2, ..., N-1$ という数列を考え、$1, 2, ..., N, 1, 2, ..., N, 1, 2, ...$ 番目の順で逆操作を行えばよいです。

これを発想するのは難しいですが、これが以上の条件を満たすことを示すのはあまり難しくありません。

$1, 2, ..., N$ 番目に 1 回ずつ逆操作を行うと、全体が $+1$ されることに注目すると、最後のたかだか $N-1$ 回の操作のみ真面目にシミュレーションすればよいので、十分高速です。

E: Decrease (Judge ver)

> 数列のうち最も大きい要素を求める、複数ある場合はどれか 1 つ選ぶ。この要素の値を N 減らす。これ以外の要素の値を 1 増やす。

という操作を

> 数列のうち N 以上の要素を好きに 1 つ選ぶ。この要素の値を N 減らす。これ以外の要素の値を 1 増やす。

としても、操作回数に変わりはないという性質が重要です。

これは、直感的にはあまりおかしくは感じないと思うのですが、証明するのはやや大変です。

ある手順で操作をすると A 回で全部の要素が $N-1$ 以下になったとします。このとき、どのような手順で操作しても A 回操作できたならば、その時点での数列は一意になります。

なぜならば、この操作というのは $mod(N+1)$ で考えるとすべての要素に $+1$ という操作になります。よってどのような手順で操作しても $mod(N+1)$ は不変です。また、どのような手順で操作をしても $\sum a_i$ は不変で、操作回数のみに依存します。

更に、すべての要素が 0 以上 $N-1$ 以下になる手順が存在することより、操作後の数列が一意になることがわかります。

以上より、どのような手順で操作をしても操作回数が一意になることが示せました。

この性質により、

> $S = \sum_{i=1}^{N} \lfloor a_i/N \rfloor$ とする。各 i について a_i を $\lfloor a_i/N \rfloor N$ 減らし、$S - \lfloor a_i/N \rfloor$ 増やす。

という行動を考えることができます。

これは、S 回の操作をまとめて考えているだけです。内訳は、i 番目の要素には $\lfloor a_i/N \rfloor$ 回操作をしています。

この行動を、すべての要素が $N-1$ 以下になるまで繰り返せば答えが求まります。

そして、実はこれは十分高速であることが示せます。

行動前の数列を a_i、行動後の数列を b_i とします。

$S = \sum \lfloor a_i/N \rfloor \geq \sum (a_i - (N-1))/N$ ゆえ、

$\sum b_i \leq \sum a_i - \sum (a_i - (N-1))/N$

よって

$\sum (b_i - (N-1)) \leq \sum (a_i - (N-1))(1 - 1/N) = (1 - 1/N) \sum (a_i - (N-1))$

以上より、$\sum(a_i-(N-1))$ は指数的に減っていくことがわかり、この指数の底は $(1-1/N)\leq 49/50$ であることがわかります。

x を行動回数として、

${}^{50\times(10^{16}+1000)}/_{(49/50)^x}<1$、つまり

$(49/50)^x>50\times(10^{16}+1000)$ となるくらい行動すると、

$\sum(a_i-(N-1))$ が 0 以下となります。なお、この x を計算すると 2000 弱となります。

そしてこのとき、

$\sum a_i\leq N(N-1)\leq 2450$ となり、

1 回の行動で $\sum a_i$ が 1 は減ることを考えると、

ここからの行動回数はかならず 2450 回以下です。

よって、多くとも 4450 回ほど行動を行えば操作は終了し、1 回の行動は $O(N)$ でシミュレーションできるため、十分間に合います。

F: Namori Grundy

まず、定義より、グラフはどのような形をしているかを考えます。

これは、サイクルが 1 つあり、サイクルの各頂点から木が生えているグラフになっています。

また、a_i は、頂点 i から辺が伸びている頂点に書かれていない値のうち最小です。(これは Grundy Numbers と呼ばれる数の定義に酷似しており、今回の問題タイトルの由来となっています)

まず、サイクルに含まれない頂点たちについては、葉からどんどん削っていく幅優先探索などを使い、簡単に a_i を定めることができます。

サイクルに含まれる頂点の値を考えます。まず、サイクルの頂点のうち 1 つを適当に選びます。そして、この頂点から辺が伸びている頂点で、サイクルに含まれないものに書かれていない値のうち、最小と 2 番目に小さいものを求めます。

すると、この頂点の値はこの 2 つのうちどちらかになることがわかります。これは、この頂点から辺が伸びている、サイクルに含まれる頂点の値で場合分けをするとわかります。

そしてこの頂点の値を定めると、他のサイクルの頂点の値もすべて決まるため、すべて決めた後に整合性を保っているか判定すれば良いです。

計算量は O(N) となります。

AtCoder Regular Contest 079 Editorial

Kohei Morita(yosupo)

July 29, 2017

A: ABCxxx

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cin >> n;
    cout << "ABC";
    cout << n;
    cout << endl;
    return 0;
}
```

B: Break Number

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 0; i < 20; i++) {
        if (n < (1<<(i+1))) {
            cout << (1<<i) << endl;
            break;
        }
    }
    return 0;
}
```

C: Cat snuke and a voyage

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<bool> vis(n+1, false);
    bool ans = false;
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        assert(1 <= a && a < b && b <= n);
        if (a == 1) {
            if (vis[b]) {
                ans = true;
            }
            vis[b] = true;
        } else if (b == n) {
            if (vis[a]) {
                ans = true;
            }
            vis[a] = true;
        }
    }

    if (!ans) cout << "IM";
    cout << "POSSIBLE" << endl;
    return 0;
}
```

D: Decrease (Contestant ver)

Consider the inverse of an operation.

In each inverse-operation, you choose an element and increase it by N, and decrease each of the other elements by 1. Furthermore, the following conditions must be satisfied:

1. After the inverse-operation, the chosen element must become the largest.
2. After the inverse-operation, all elements must be non-negative.

Let's start from a sequence whose elements are betwen 0 and $N-1$, inclusive. If you can perform inverse-operations K times on this sequence, that is a valid answer for this problem.

For example, if you start from the sequence $\{0, 1, 2, ..., N-1\}$, you can perform the inverse-operations infinite number of times, by choosing elements $1, 2, ..., N, 1, 2, ..., N, 1, 2, ...$ in this order. (You should use $N = 50$ to fit within the constraints of output.)

Notice that if you perform the inverse-operations once for each of the elements $1, 2, ..., N$, each element will be incremented by one. Using this, you can easily simulate $floor(K/N) * K$ steps. Also, you can simulate the remaining $K\%N$ steps by brute force.

E: Decrease (Judge ver)

In each operation, the sum of all elements in the sequence decreases by exactly one. Let S be the sum of all elements in the initial sequence. Until the maximum becomes at most $N-1$, we must perform at least $S-N^2$ operations.

Let's perform X operations, for a given large X. Instead of performing the operations as they are described in the statement,

- We first increase each element by X.
- Then, repeat the following X times: choose the maximum element, and decrease it by $D = N+1$.

How can we perform the second step quickly? By binary search, we can find the smallest integer Y such that

$$T = floor(\max\{(a_1 - Y)/D, 0\}) + \ldots + floor(\max\{(a_N - Y)/D, 0\}) \ldots X \quad (1)$$

Once we find this Y, we can perform T steps at once, by replacing each a_i by $a_i - floor(\max\{(a_i - Y)/D, 0\})$. Since $X - T \leq N$, we can simulate the remaining $X - T$ steps by brute force.

After the simulation of $S - N^2$ steps, the sum of elements will be at most N^2, so there will be at most N^2 remaining steps/ Therefore, we can handle the remaining part by straightforward simulations.

F: Namori Grundy

It is well-known that in this type of graph, there is exactly one cycle in the graph, and from each vertex on the cycle a tree may "grow".

We want to assign integers a_i to each vertex, such that a_i is the smallest non-negative integer that doesn't appear in $a_{v_1}, \ldots, a_{v_k}$, where $v_1, \ldots, v_k$ are the destinations of edges from the vertex i. (This is known as Grundy Numbers).

Let's call a vertex in the cycle "cycle vertex", and call the others "forest vertex".

First, for each forest vertex, we can uniquely determine their Grundy Numbers in the order from leaves to roots.

How can we determine the Grundy Number of a cycle vertex v? Let $c_1, \ldots, c_k$ be forest vertices that are direct children of v, and let w be the cycle vertex that follows v. Then, a_v must be the smallest non-negative integer that doesn't appear in the set $a_{c_1}, \ldots, a_{c_k}, a_w$. We know the values of $a_{c_1}, \ldots, a_{c_k}$, but we don't know the value of a_w. However, regardless of the value of a_w, there are only two candidates for a_v: one of the two smallest integers that don't appear in $a_{c_1}, \ldots, a_{c_k}$.

Now, choose an arbitrary cycle vertex x. There are two candidates for a_x, so we try both. Once we fix a_x, we can determine all Grundy Numbers along the cycle, and we can verify if the Grundy Numbers we get are valid.

This solution works in $O(N)$.

ABC 067 / ARC 078 解説

writer : camypaper

2017 年 7 月 15 日

For international readers: English editorial starts from page 7.

A : Sharing Cookies

$A, B, A+B$ のいずれかが 3 で割り切れるかどうかを調べればよい．なお，答えが `Impossible` となるのは $A \equiv B \equiv 1 \bmod 3$ あるいは $A \equiv B \equiv 2 \bmod 3$ の場合のみである．

```
#include <iostream>
using namespace std;
int main(){
    int a,b;
    cin>>a>>b;
    if(a%3 == 0 || b%3 == 0 || (a+b)%3==0)
        cout<< "Possible" <<endl;
    else cout<< "Impossible" <<endl;
}
```

B : Snake Toy

N 個の棒が長さの昇順に並んでいる場合には，末尾から K 個の棒の長さの和が答えとなる．よって，初めに棒たちを長さの昇順に並び替えたのち，上述したような貪欲法を適用すればよい．

C++ などの多くのプログラミング言語では，ソート用の関数が予め用意されていることが多いため，ソート関数を実装する必要はないだろう．

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
int main(){
    int n,k;
    cin>>n>>k;
    vector<int> l(n);
    for(int i=0;i<n;i++){
        cin>>l[i];
    }
    sort(l.begin(),l.end());
    int ans=0;
    for(int i=0;i<k;i++){
        ans+=l[n-1-i];
    }
    cout<<ans<<endl;
}
```

C : Splitting Pile

N 枚のカードに書かれた数の総和を X とする．カードの山の先頭 i 枚のカードに書かれた数の総和が x_i であったとすると，残ったカードたちに書かれた数の和 y_i は $X - x_i$ であり，$|y_i - x_i|$ は $|X - 2x_i|$ となる．

1 から $N - 1$ までの全ての i について $|X - 2x_i|$ を試せばよい．これは $O(N)$ で実行可能であり，十分高速である．

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;
int main(){
    int n;
    long long X=0,x=0,ans=1000000000000000000LL;
    cin>>n;
    vector<long long> a(n);
    for(int i=0;i<n;i++){
        cin>>a[i];
        X+=a[i];
    }
    for(int i=0;i<n;i++){
        x+=a[i];
        if(i+1<n)ans=min(ans,abs(X-2*x));
    }
    cout<<ans<<endl;
}
```

D : Fennec VS. Snuke

「マスに色を塗った回数の多いプレイヤーの勝ち，同数の場合は後手の勝ち」というルールのゲームを考えることにする．元のゲームでどちらかのプレイヤーが敗北したあとも色を塗ることが可能である，と考えると新しいゲームの勝敗と元のゲームの勝敗が一致することが分かる．このとき，2 人の目的は，自分が色を塗る回数を最大化し，相手が色を塗る回数を最小化することである．

与えられるグラフが木であることから，2 人の最善戦略は「マス 1 からマス N へのパス上に色が塗られていないマスが存在するならば色を塗る」であることが示される．マス 1 からマス N へのパス上に色が塗られていないマスが存在するにも関わらず，それ以外のマスに色を塗った場合，自分が不利になり相手が有利になることは直感的にも明らかであろう．例えば図 **??** に示されるような配置からゲームを開始したとき，上記の戦略に従ったときのみ先手は勝利可能である．従わなかった場合，後手が最適に行動すると 7 個以上のマスを白く塗ることが可能となり，先手が勝つことは不可能である．

上記の戦略に従うと，マス i と j の距離を $d(i,j)$ として，マス i の色は $d(1,i) \leq d(N,i)$ ならば黒，そうでなければ白となる．結論としてマス 1 とマス N の 2 点から幅優先探索や深さ優先探索などを行うことで $O(N)$ でこの問題を解くことが可能である．

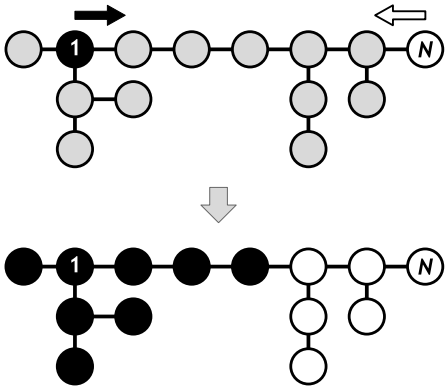


図 1　ゲームの初期状態と最終状態の例

E: Awkward Response

解の一例を説明する．

$q(n)$ を n についての質問の答えが `Yes` ならば 1 を，そうでなければ 0 を返す関数とする．N が L 桁の整数であったと仮定する．このとき，L 桁の整数 n と N の大小は，$\mathrm{str}(n)$ と $\mathrm{str}(N)$ の大小と一致する．ここで，$L+1$ 桁の整数 $10n$ と N の大小を考えたとき，$10n > N$ が必ず成立し，$\mathrm{str}(10n)$ と $\mathrm{str}(N)$ の大小は以下のように表される．

- $n < N$ のとき： $\mathrm{str}(10n) < \mathrm{str}(N)$
- $n \geq N$ のとき： $\mathrm{str}(10n) > \mathrm{str}(N)$

以上より，$q(10n)$ が 1 を返す最小の整数 n が N と一致することが分かる．さらに，$q(10n)$ には単調性が存在するので n について二分法を行うことで $\lg N$ 回程度の質問で N を求めることが可能である．

次に，N の桁数 L を求める方法を 1 つ説明する．

$q(10^{k-1})$ が式 (**??**) で表されることから，$N \neq 10^{L-1}$ の場合は，$q(10^{k-1}) = 1$ を満たす最大の k を求めることで L を求められる．

$$q(10^{k-1}) = \begin{cases} 1 & (k < L) \\ 1 & (k = L) \\ 0 & (k > L \text{ かつ } N \neq 10^{L-1}) \\ 1 & (k > L \text{ かつ } N = 10^{L-1}) \end{cases} \tag{1}$$

$N = 10^{L-1}$ の場合は，$q(2 \times 10^{k-1})$ が式 (**??**) のように表されることから，$q(2 \times 10^{k-1}) = 1$ を満たす最小の k を求めることで，L を求められる．

$$q(2 \times 10^{k-1}) = \begin{cases} 0 & (k < L) \\ 1 & (k = L) \\ 1 & (k > L) \end{cases} \tag{2}$$

解法をまとめると，以下のように表される．

1. $N = 10^{L-1}$ の場合に注意して，N の桁数 L を求める．
2. 二分法を用いて $q(10n) = 1$ なる最小の n を求める．

どのような場合でも質問回数は $\max(2L, L + \lg N)$ 回以下となり，64 回という制限に対しては十分余裕がある．

F : Mole and Abandoned Mine

はじめに，問題設定を以下のように言い換えておく．言い換えた問題設定での答えを x として，$\sum_{i=1}^{M} c_i - x$ が元の問題の答えである．

> N 頂点の辺のないグラフが与えられる．いくつか辺を追加して，頂点 1 から頂点 N への点素パスがただ 1 つ存在するようにしたい．
> 辺を追加する方法は M 個あり，頂点 a_i, b_i 間に辺を追加すると c_i 利得を得る．得られる利得の総和を最大化せよ．

頂点 1 から頂点 N への点素パスがただ 1 つ存在するための条件は，「頂点 1 から頂点 N へのパス上にある辺がいずれも橋である」ことである．頂点 1 から頂点 N へのパスがただ 1 つ存在し，そのパス上に，橋でない辺が存在したとする．このとき，そのような辺を取り除いてもグラフは連結であるから，頂点 1 から頂点 N へのパスは他にも存在するはずである．これは点素パスがただ 1 つ存在することに矛盾する．

ここで，頂点 1 から頂点 N への点素パスがただ 1 つ存在するグラフにおいて，頂点 1 から N へのパス上 (両端を含む) にある頂点をターミナルと呼ぶことにする．最適解においては，ターミナル同士をつなぐ辺を取り除いたとき，各連結成分内においては可能な限り辺が追加されているはずである．そのため，$O(2^N \times M)$ かけて事前に計算をしておけば，各連結成分内での利得の総和は容易に計算可能である．

さて，パスを 1 つ固定して，その他の頂点たちがどのターミナルに属するかを全探索することを考えよう．このままでは，実行時間制限には間に合わない．そこで，頂点 1 から N までのターミナルを順番に決めていきながら，現在着目しているターミナルに属する頂点たちを全て試していくことを考える．S を $\{1, 2, 3, \ldots, N\}$ の部分集合として，$dp(S, t)$ を「S に含まれる頂点たちを使用しており，t が現在着目しているターミナルであるときの，得られる利得の総和の最大値」とする．遷移としては，次のターミナルに接続するか，現在のターミナルに属する頂点集合を試すか，の 2 つの遷移を試せばよい．適切に実装を行うことで，全体として $O(3^N \times N)$ で実行可能である．

ABC 067 / ARC 078 Editorial

writer : camypaper

July 15th, 2017

A, B, C

See the sample codes of Japanese editorial above.

D : Fennec VS. Snuke

Let $v_1 = 1, v_2, \ldots, v_{k-1}, v_k = N$ be the only path from cell 1 to cell N.

Let $d(i, j)$ be the distance between cell i and cell j.

In the optimal strategy, Black should color the vertices in the order $v_2, v_3, \ldots$ (while this is possible), and after that Black can choose arbitrary valid moves. This way, Black can make sure that she can get all vertices v such that $d(1, v) \leq d(N, v)$, no matter how White plays.

On the other hand, White should color the vertices in the order $v_{k_1}, v_{k-2}, \ldots$ (while this is possible), and after that White can choose arbitrary valid moves. This way, White can make sure that he can get all vertices v such that $d(1, v) > d(N, v)$, no matter how Black plays.

To summarize, we can solve this task in $O(N)$ by computing distances from two cells 1 and N.

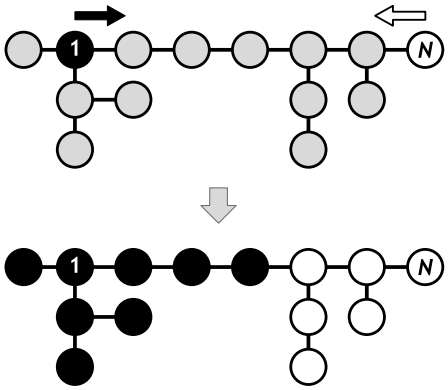


図 1　An example of a game

E: Awkward Response

First, we should query with 10^9. We get 'Y' iff $N = 1, 10, \ldots, 10^9$.
Next, we want to compute the number of digits in N. Let's call it L.
Let $q(n)$ denote the result of the query: 1 means `Yes`, 0 means `No`.
In case $N = 10^{L-1}$, we use the following fact:

$$q(2 \times 10^{k-1}) = \begin{cases} 0 & (k < L) \\ 1 & (k = L) \\ 1 & (k > L) \end{cases} \tag{1}$$

Thus, L is the minimum k that satisfies $q(2 \times 10^{k-1}) = 1$.
In case $N \neq 10^{L-1}$, we use the following fact:

$$q(10^{k-1}) = \begin{cases} 1 & (k < L) \\ 1 & (k = L) \\ 0 & (k > L \mathrm{and} N \neq 10^{L-1}) \\ 1 & (k > L \mathrm{and} N = 10^{L-1}) \end{cases} \tag{2}$$

Thus, L is the maximum k that satisfies $q(10^{k-1}) = 1$.
Next, for a given L-digit number n, we can check if $N < n$ by querying $10n$.

- If $n < N$, $q(10n) = 0$.
- If $n \geq N$, $q(10n) = 1$.

Thus, we can compute N in $\lg N$ queries by binary search.
The number of queries is about $\max(2L, L + \lg N)$.

F : Mole and Abandoned Mine

Suppose that we have a graph that has exactly one path from vertex 1 to vertex N. In this graph, all edges on the unique path from 1 to N must be bridges. (Otherwise, we can find a path from 1 to N even if we remove one of those edges, and this is a contradiction).

Let S be a subset of vertices, and t be a vertex in S. Define $dp(S, t)$ as the minimum cost required to achieve the condition "There is a unique path from 1 to t that only uses vertices in S". We have two transitions:

(Here, $cost(S, T)$ denotes the sum of costs of all edges between two sets S and T.)

- Let $u \notin S$ be a vertex. If there is an edge between t and u, we can reach $dp(S \cup \{u\}, u)$ with cost $dp(S, t) + cost(S \setminus \{t\}, \{u\})$.
- Let T be a set of vertices that is disjoint with S. We can reach $dp(S \cup T, t)$ with cost $(dp(S, t) + cost(S \setminus \{t\}, T)$.

With proper pre-computation, we can compute $cost(S, T)$ in $O(1)$ and this solution works in $O(3^N \times N)$.

ABC066 / ARC077 解説

writer: nuip

2017 年 7 月 1 日

For International Readers: English editorial starts from page 8.

A : ringring

$a+b$ と $b+c$ と $a+c$ の中で一番小さいものを出力する問題です。以下の実装例では、かわりに a, b, c の中で最も大きいものを求めて、$a+b+c$ から引いています。

```
#include<stdio.h>

int main(){
  int a, b, c;
  scanf("%d␣%d␣%d", &a, &b, &c);
  int max=a;
  if(b>max) max=b;
  if(c>max) max=c;
  printf("%d\n", a+b+c-max);
  return 0;
}
```

B : ss

偶文字列かどうかを調べるためには、文字列の前半と後半が全く同じかどうかを調べれば良いです。

偶文字列の長さは必ず偶数長なので、偶数の文字列についてのみ調べれば十分です。最長のものを探しているので、長さ $|S|-2$ の文字列、長さ $|S|-4$

の文字列、…という順に調べて、条件に合うものが見つかった時にその長さを出力すると答えが求まります。

```c
#include<stdio.h>
#include<string.h>

int main(){
  char str[222];
  scanf("%s", str);
  int n=strlen(str);
  for(int i=n-2; i; i-=2){
    if(strncmp(str, str+i/2, i/2)==0){
      printf("%d\n" ,i);
      return 0;
    }
  }
  return 0;
}
```

C : pushpush

毎回逆向きの並び替えをしていると時間がかかりすぎてしまうので ($O(n^2)$)、これを避ける方法を考えます。入力例を参考に考えると、数列の最初と最後に交互に a_i を追加していくと答えが求まることが分かります。また、最後に追加した項は最初に来るはずなので、

1. i と n の偶奇が一致していれば、数列の前に a_i を追加する。
2. i と n の偶奇が一致していなければ、数列の後ろに a_i を追加する。

という操作をすれば良いことが分かります。

この操作を効率よくするためにはどうすれば良いでしょうか?C++を使っている場合は、stl::deque という標準ライブラリの機能を使うと簡単に出来ます。そうでない場合は、大きめの配列を確保して、真ん中あたりから順番に使っていくと実装できます。この方針で実装した C 言語の実装例は以下のとおりです。

```c
#include<stdio.h>

int array[512345];
```

```
4  int a[212345];
5
6  int main(){
7    int n;
8    scanf("%d", &n);
9    for(int i=0; i<n; ++i) scanf("%d", &a[i]);
10   int left=212345, right=left+1;
11   for(int i=0; i<n; ++i){
12     if(i%2 == (n-1)%2){
13       array[left--]=a[i];
14     }else{
15       array[right++]=a[i];
16     }
17   }
18   for(int i=left+1; i<right; ++i){
19     printf("%d␣",array[i]);
20   }
21   return 0;
22 }
```

D : 11

数の種類が n 種で、項が $n+1$ 項あるので、必ず同じ数の項が複数存在します[1]。また、すべての数が 1 度以上現れるという制約から、同じ数である項はちょうど 1 組であることが分かります。

同じ数があるので、単純に $n+1$ 個から k 個を選ぶ組合せ ${}_{n+1}C_k$ を計算するだけでは答えが求められません。そこで、重複を除く方法を考えてみます。

同じ数が出てくるのはちょうど 1 組だけなので、${}_{n+1}C_k$ から重複してるものの数を引けば答えが求まります。重複している数列はどのような数列でしょうか？

取り出した部分列を見てどこから取り出した部分列であるかを考えたとき、1 通りに定まらない場合、その数列は 2 回数えられていることになります。次のような例を考えてみます。

$$2\mathbf{3}1\mathbf{4}5\mathbf{1}67$$

[1] 参考: 鳩ノ巣原理

重複している数である 1 は太字で書いてあります。もし部分列で 1 を一度も選んでいなければ、簡単に復元できます。

$$256 \to \underline{2}3\mathbf{1}45\underline{\mathbf{1}}\underline{6}7$$

部分列で 1 を 2 つとも選んでいる場合も、やはり簡単に復元できます。

$$211 \to \underline{2}3\underline{\mathbf{1}}45\underline{\mathbf{1}}67$$

また、1 を選んでいても、2 つの 1 と 1 の間の数 4, 5 を選んでいれば、これらの数と 1 との位置関係から、使われている 1 がどちらの 1 であるか特定できます。

$$51 \to 23\mathbf{1}45\underline{\mathbf{1}}67$$

よって、以上のような場合は重複して数えられない数列です。1 を 1 つだけ選んでいて、2 つの 1 と 1 の間の数 4, 5 を選んでいない場合はどちらの 1 を選んでいるかわかりません。これが重複する場合です。

$$\begin{aligned} 31 &\to 2\underline{3}\mathbf{1}45\underline{\mathbf{1}}67 \ ? \\ &\to 2\underline{3\mathbf{1}}45\mathbf{1}67 \ ? \end{aligned}$$

よって、重複している項を $a_l, a_r (l < r)$ とすると、このような部分列の数は、「a_l か a_r の片方を選んで a_l と a_r で囲われていない項から残りの $k-1$ 個を選ぶ場合の数」になります。a_l と a_r で囲われていない項は a_l の左に $l-1$ 項と、a_r の右に $n-r$ 項あるので、求める場合の数は ${}_{l-1+n-r}C_{k-1}$ であることが分かります。

よって、k 行目には、${}_{n+1}C_k - {}_{l-1+n-r}C_{k-1}$ を出力すれば良いです。

${}_nC_k$ の求め方は、ABC042/ARC058 の D 問題の解説を参照して下さい。

`http://arc058.contest.atcoder.jp/data/arc/058/editorial.pdf`

E : guruguru

まず、お気に入りボタンは必ず明るさを切り替える一番最初に押した方が効率的です。なぜなら、それ以前に他のボタンを押していたとしても、お気に入りボタンを押した後の明るさは変わらないためです。

順送りボタンのみを使って明るさ A から B に切り替えるためボタンを押す回数は、$A \leq B$ なら $B-A$ 回で、$A > B$ なら $B+N-A$ 回です。これは、まとめて $(B+N-A)\%N$ と書けます。ただし、$x\%y$ で x を y で割ったあまりを表します。

よって、お気に入りボタンを最初に押した場合のボタンを押す回数は $1+(B+N-X)\%N$ 回で、そうでない場合は $(B+N-A)\%N$ 回です。

また、お気に入りボタンを使うのは、a_i と a_{i+1} 間（a_{i+1} を含む）の状態にお気に入りの明るさ x が存在しているときのみです。

以上を元に、お気に入りが x であるときと $x+1$ である時でボタンを押す回数がどのように変わるか考えてみます。

- $a_{i+1}=x$ である i
  お気に入りボタンを 1 回押すだけで済んでいたのが、順送りボタンを $(a_{i+1}+N-a_i)\%N$ 回押さないといけなくなります。
- a_i と a_{i+1} 間（両端を含まない）に x があるような i
  順送りボタンを押す回数が 1 回減ります。
- それ以外
  変化無しです。

まとめると、ボタンを押す回数は、$a_{i+1}=x$ であるすべての i について $(a_{i+1}+N-a_i)\%N-1$ 回分増えて、a_i と a_{i+1} 間に x があるような i の数だけ減ります。

a_i と a_{i+1} 間に x があるような i の数は、累積和を取る操作を使うと $O(n+m)$ で求めることができます。imos 法という名前で呼ばれることもあります。

$a_{i+1}=x$ である i については、当然全部で n 個しか無いので、いちいち計算して足しても問題ありません。

以上から、$x=0$ である場合の答えを求めた後、$x=i$ の場合を使って $x=i+1$ の場合を順番に求めていくことで、すべての x についてボタンを押す回数が求まるため、最小値も求まります。

F : SS

まず f がどのような関数か考えてみましょう。

A,B,C をそれぞれアルファベット 1 文字として、

ABCABC

という偶文字列を考えます。これに文字を付け加える偶文字列を作るためには、付け加える文字の数は偶数でないといけません（偶文字列の長さは偶数文字なので）。まずは 2 文字付け加えることを考えてみます。

ABCABC??

これが偶文字列であるなら、赤い文字列と青い文字列は同じである必要があります。よって、“AB” と “BC” は同じ文字である必要があります。

同じように、 $2k$ 文字付け加えることを考えてみます。

$$S_1S_2\dots S_nS_1S_2\dots S_k\ S_{k+1}\dots S_nX_1X_2\dots X_{2k}$$

この場合、S の最初の $n-k$ 文字と、S の $k+1$ 文字目以降とが一致している必要があることが分かります。これは、S が周期 k の文字列であるということを意味します。$X_1X_2\dots X_{2k}$ は自由に決められるので、S が周期 k の文字列である場合、必ず $2k$ 文字付け加えて偶文字列が作れます[2]。

以上から、$f(S)$ を求めるには、S の最小周期 k を求めればいいことがわかりました。

ここで、同じ文字列 S を n 回繰り返してできる文字列を $S\times n$ と書くことにします。例えば、 “abc”$\times 3$ は”abcabcabc” となります。

ここからは、問題を偶文字列 $S\times 2$ のままでなく、「半分」にした文字列 S の状態で考えることにします（そうした方が扱いやすいからです）。半分にした状態の文字列を扱うために、関数 g を導入します。関数 g を

$$g(S)\times 2=f(S\times 2)$$

で定めます。右辺は必ず偶文字列であるため、このような $g(S)$ は必ず存在します。$g(S)$ は $f(S)$ の接頭辞になるため、$f^{10^{100}}(S)$ のかわりに $g^{10^{100}}(S)$ を考えても同じ結果が得られます。

g がどのような関数であるかを考えます。S の最小周期を x として、S の長さ x の接頭辞 を T とします。このとき、正の整数 n と S は T の接頭辞である文字列 T' を使って

$$S=T\times n+T'$$

と書けます。

もし x が $|S|$ の約数である場合は単純で、$S=T\times n$ と書けるので、

$$f(S\times 2)=f(T\times 2n)=T\times(2n+2)$$

となります。よって、$g(T\times n)=T\times(n+1)$ です。

そうでない場合、先程の結果を使うと、$g(S)$ の長さは $|S|+x$ であるはずなので、

$$g(S)=T\times n+T'+T$$

[2] $(X_1X_2\dots X_n)=(S_{n-k+1}S_{n-k+2}\dots S_nS_1S_2\dots S_k)$ とすれば良いです。

と分かります。x の最小性から、この文字列$T \times n + T' + T$ の周期は $|T \times n + T'|$ です。よって、もう一度 g でこの文字列を写すと、

$$\begin{aligned} g^2(S) = g(T \times n + T' + T) &= (T \times n + T' + T) + (T \times n + T') \\ &= g(S) + S \end{aligned}$$

となります。これはどの偶文字列 S に対しても成り立つため、$g^{n+2}(S) = g^{n+1}(S) + g^n(S)$ であることが分かります。

あとは、この式をもとに、各文字 c について、$g^n(S)$ に現れる回数を計算し、i 番目までに c が現れる回数を再帰的に数えれば良いです。

ABC066 / ARC077 Editorial

writer: nuip

July 1st, 2017

A : ringring

```c
#include<stdio.h>

int main(){
  int a, b, c;
  scanf("%d␣%d␣%d", &a, &b, &c);
  int max=a;
  if(b>max) max=b;
  if(c>max) max=c;
  printf("%d\n", a+b+c-max);
  return 0;
}
```

B : ss

```c
#include<stdio.h>
#include<string.h>

int main(){
  char str[222];
  scanf("%s", str);
  int n=strlen(str);
  for(int i=n-2; i; i-=2){
    if(strncmp(str, str+i/2, i/2)==0){
      printf("%d\n" ,i);
      return 0;
    }
  }
  return 0;
}
```

C : pushpush

You can observe that a_i are appended to the beginning or the end, alternately. The last element (a_n) will be appended to the beginning.

Therefore, we start from an empty sequence, and in the order $a_1, \ldots, a_n$,

1. If i and n have the same parity, append a_i to the beginning of the sequence.
2. If i and n have different parities, append a_i to the end of the sequence.

This can be implemented in $O(n)$ with a deque.

D : 11

There are $\binom{n+1}{k}$ ways to choose k elements from the sequence, but this way we may count the same subsequence multiple times.

For example, consider the following case:

$$23\underline{1}45\underline{1}67$$

Fix a subsequence s of this sequence. How many times will s be counted? We can observe that:

- If s contains two 1s, the positions can be uniquely determined and it will be counted only once.
- If s contains no 1, the positions can be uniquely determined and it will be counted only once.
- If s contains 4 or 5, the positions can be uniquely determined and it will be counted only once.
- Otherwise, we can't determine which 1 we chose; it will be counted twice.

So, we must subtract the number of ways to choose $k-1$ elements from $2, 3, 6, 7$.

In general, if the distance between two elements of the same value is d, the answer is $\binom{n+1}{k} - \binom{n-d}{k-1}$.

E : guruguru

Let $f(s, t, x)$ be the number of buttons we must push to change the brightness from s to t, when the favorite brightness is set to x.

First consider the following slow solution. We have an array c of length m. Initially all values are zeroes. Then, for each i and x, we add $f(a_i, a_{i+1}, x)$ to $c[x]$. The answer is the minimum value in the sequence c.

How can we improve this solution? We want to do this efficiently for a fixed i. Let's fix i, and let $s = a_i, t = a_{i+1}$. Assume that $s < t$ (the other case can be handled similarly).

- If $x \leq s$, $f(s, t, x) = t - s$.
- If $s < x \leq t$, $f(s, t, x) = t - x + 1$.
- If $t < x$, $f(s, t, x) = t - s$.

Notice that in all three cases, the value of f is a linear function of x.

Thus, we can solve this problem by adding $O(N)$ linear functions to given ranges of c. This can be done by two prefix sums: one for linear part and one for constant part.

F : SS

Suppose that initially we have a string of length $2n$.

$$S_1 S_2 \ldots S_n S_1 S_2 \ldots S_n$$

Can we make it an even string by appending $2k$ characters?

$$S_1 S_2 \ldots S_n S_1 S_2 \ldots S_k \; S_{k+1} \ldots S_n X_1 X_2 \ldots X_{2k}$$

From this, you see that the first $n-k$ characters of $S_1 S_2 \ldots S_n$ and the last $n-k$ characters of $S_1 S_2 \ldots S_n$ must be the same. It means that $S_1 S_2 \ldots S_n$ has a period of k.

In general, $f(SS) = STST$, where T is the first k characters of S, where k is the shortest period of S.

Let's define $g(S) = ST$, where T is defined in the same way. We can prove the following:

- If $g(S) = ST$ and $|T|$ is a divisor of $|S|$, $g(ST) = STT$.
- If $g(S) = ST$ and $|T|$ is not a divisor of $|S|$, $g(ST) = STS$.

By repeating this, we get:

- If $g(S) = ST$ and $|T|$ is a divisor of $|S|$, $g^{\infty}(S) = STTTTT....$
- If $g(S) = ST$ and $|T|$ is not a divisor of $|S|$, $g^{i+2}(S) = g^{i+1}(S) + g^{i}(S)$ for each i.

The remaining part is not very hard. Note that in the actual implementation you don't need to consider the first case: you get the same value for $g^{\infty}(ST)$ from the second case anyway.

===

Sketch of the proof for the second case (the first case is easy).

Suppose that

$$S = T \times n + T'$$

where n is a positive integer, $|T|$ is the shortest period of S, and T' is a non-trivial prefix of T.

We want to prove that the shortest period of

$$g(S) = T \times n + T' + T$$

is $|T \times n + T'|$.

Assume that there exists a shorter period $x < |t \times n + T'|$.

- If $x \neq |T'| \pmod{|T|}$ and $x < |t \times n + T'|$, we can prove that T has a period of $gcd(|T|, x - |T'|)$. Thus, S also has a period of $gcd(|T|, x - |T'|)$, and it contradicts the fact that the shortest period of S is $|T|$.
- Otherwise, $x \neq 0 \pmod{|T|}$ and $x \leq |t \times (n-1) + T'|$. In this case we can prove that S has a period of $gcd(|T|, x)$, and again we get a contradiction.

ARC 076 解説

DEGwer

2017/06/24

For International Readers: English editorial starts on page 4.

A: Expired?

$A \geq B, A < B \leq A + X, B > A + X$ の 3 通りに場合分けをする問題です。C++ での実装例を以下に示します。

```
#include<stdio.h>
int main()
{
        int x, a, b;
        scanf("%d%d%d", &x, &a, &b);
        if (-a + b <= 0)printf("delicious\n");
        else if (-a + b <= x)printf("safe\n");
        else printf("dangerous\n");
}
```

B: Trained?

どのボタンが光るかを順にシミュレートしていき、もしボタン 2 が光ったなら、それまでにボタンを押した回数を出力すればよいです。

では、ボタン 2 が永遠に光らない場合はどうでしょうか。コンピュータは有限回の操作しかできないため、「いつまでたってもボタン 2 は光らない」ということを直接シミュレーションで確かめることはできません。

N 回ボタンを押してもボタン 2 が光らなかったとします。このとき、その間に $N+1$ 回いずれかのボタンが光るので、そのうちのある 2 つは同じボタンです。同じボタンが光っている状態から光っているボタンを押すことを繰り返しても、同じ順にボタンが光るだけなので、この場合ボタン 2 は永遠に光ることはありません。

よって、N 回までボタンを押してみて、もし途中でボタン 2 が光ればそれまでにボタンを押した回数を出力し、そうでなければ -1 を出力すれば、この問題を解くことができます。

以下に C++ による実装例を示します。

```
#include<stdio.h>
#include<vector>
#include<algorithm>
using namespace std;
int main()
{
        int n;
        scanf("%d", &n);
        vector<int>v;
        for (int i = 0; i < n; i++)
        {
                int z;
                scanf("%d", &z);
                z--;
                v.push_back(z);
        }
        int now = 0, c = 0;
        for (;;)
        {
                if (now == 1)
                {
                        printf("%d\n", c);
                        break;
                }
                if (c >= n)
                {
                        printf("-1\n");
                        break;
                }
                c++;
                now = v[now];
        }
}
```

C: Reconciled?

N と M の差 (の絶対値) が 2 以上のとき、条件を満たすように並べることはできません。

N と M の差が 1 のとき、$N+1=M$ とすれば、条件を満たすためには猿犬猿... 犬猿 の順に並べることになります。このような並べ方は、犬 N 匹を並べる場合の数と、猿 M 匹を並べる場合の数の積なので、$N!M!$ 通りです。$M+1=N$ の場合も同様です。

$N=M$ のとき、猿犬猿... 猿犬と犬猿犬... 犬猿の 2 種類の並べ方があります。上記の場合と同じように考えれば、それぞれ $N!M!$ 通りの並べ方があることがわかるので、全体の場合の数は $2N!M!$ 通りです。

10^9+7 で割ったあまりを求めるのを忘れないようにしてください。

D: Built?

最小全域木を求める問題です。素直にグラフを構築し、最小全域木を求めた場合、辺数が $O(N^2)$ 本となり、間に合いません。

さて、座標 (a,b) の点と座標 (c,d) の点を結ぶコストは $min(|a-c|,|b-d|)$ ですが、最小全域木を求める上では、(多重辺はコストが最小のもののみ考えればいいので) コスト $|a-c|$ の辺とコスト $|b-d|$ の辺がそれぞれ存在すると考えても構いません。

さて、x 座標と点の番号の組がそれぞれ $(s,i)<(t,j)<(u,k)$ (ただし、不等号は辞書式順序で入れる) である 3 点を考えたときに、もし 1 番目の点と 3 番目の点をコスト $u-s$ の辺で結ぶ辺を全域木に使うならば、その代わりに $1,2$ 番目の点を結ぶコスト $u-t$ の辺と $2,3$ 番目の点を結ぶコスト $t-s$ の辺を使うことにしても、全域木のコストは大きくなりません。よって、$1,3$ 番目の点を直接結ぶコスト $u-t$ の辺は、考えなくていいことが分かります。(点の番号を一緒にして並べているのは、同じ x 座標の点が出現した場合に順序をつけるための便宜的な措置です)

以上より、点たちを x 座標でソートした列において隣接する 2 点と、y 座標でソートした列において隣接する 2 点の間にそれぞれ辺を張ったグラフの最小全域木を求めればよいです。辺の本数は $O(N)$ 本に減ったため、全体で $O(NlogN)$ 時間でこの問題を解くことができます。

E: Connected?

まず、少なくとも片方が長方形の周上以外に書かれている整数については、考慮しなくてよいことを示します。

両方が長方形の周上に書かれている整数たちをうまく結べたとします。片方が周上に書かれている整数についてはその周上の位置に、そうでない整数については適当な長方形内部に、その整数を 2 つ書き、すでに書かれている曲線たちを「押し出す」ように連続的に変形させることで、新しく書いた整数たちを所望の位置に移動することができるため、示されました。

さて、両方が長方形の周上に書かれている整数たちだけを考えたとき、条件を満たすように整数たちを結べるかどうかは、どのように判定すればいいでしょうか？もし整数 i,j が、i,j,i,j の順に周上に現れたなら、このようなことは不可能です。逆に、そのような i,j が存在しなければ、可能であることもわかります。

これは、周上の適当な位置から順に、時計回りに周上の点たちを stack などを用いて管理していけば、$O(N)$ 時間 (ただし、最初のソートに $O(NlogN)$ 時間) で判定することができます。

F: Exhausted?

高橋君の集合 X に対し、その集合内のいずれかの高橋君が座ることのできる椅子の集合を $\Gamma(X)$ と書くことにします。

Hall の定理より、追加すべき椅子の個数の最小値は、全ての高橋君の部分集合 X に対する、$|X| - \Gamma(X)$ の最大値と一致します。

さて、$\Gamma(X)$ としてありうるものは、高橋君のこだわりの条件より、「ある $s, t(s+1<t)$ に対し、座標 s 以下または座標 t 以上にある椅子すべて」もしくは「椅子すべて」という形をしています。$\Gamma(X)$ を固定したとき、先の X としてはサイズの一番大きいもののみを考えればよいです。

$\Gamma(X)$ が全体でない場合、全ての $s+1<t$ に対する、座標 s 以下または座標 t 以上にある椅子の個数から、$L_i \leq s$ かつ $t \leq R_i$ なる i の個数を引いたものの最大値を求めればいいです。$\Gamma(X)$ が全体の場合、X として高橋君全員からなる集合を取れます。以上より、求める答えは、$s+1<t$ に対して求めた最大値と、$N-M$ の大きい方となります。

さて、$s+1<t$ に対しては、どのように最大値を求めればいいでしょうか？これは、高橋君たちを L_i の昇順にソートして順に見ていきながら、区間加算と全体 min のクエリの処理できる segment tree を用いることで、$O(NlogN)$ で求めることができます。

ARC 076 Editorial

DEGwer

2017/06/24

A: Expired?

There are three cases: $A \geq B, A < B \leq A + X, B > A + X$.

```
#include<stdio.h>
int main()
{
        int x, a, b;
        scanf("%d%d%d", &x, &a, &b);
        if (-a + b <= 0)printf("delicious\n");
        else if (-a + b <= x)printf("safe\n");
        else printf("dangerous\n");
}
```

B: Trained?

We simulate the process. If 2 is lighten up during the simulation, we get the answer. Otherwise, if 2 is not lighten up in the first N steps, we can prove that this button will never be lighten up, thus the answer is -1.

```
#include<stdio.h>
#include<vector>
#include<algorithm>
using namespace std;
int main()
{
        int n;
        scanf("%d", &n);
        vector<int>v;
        for (int i = 0; i < n; i++)
        {
                int z;
                scanf("%d", &z);
                z--;
```

```
            v.push_back(z);
        }
        int now = 0, c = 0;
        for (;;)
        {
            if (now == 1)
            {
                printf("%d\n", c);
                break;
            }
            if (c >= n)
            {
                printf("-1\n");
                break;
            }
            c++;
            now = v[now];
        }
}
```

C: Reconciled?

If the absolute difference of N and M is at least 2, it is impossible to satisfy the conditions.

If $N + 1 = M$, the animals must be arranged "MDMDM...DM" (here M stands for monkeys and D stands for dogs). There are $N!M!$ ways to do this. The case where $M + 1 = N$ is similar.

If $N = M$, both "MDMD...MD" and "DMDM...DM" are possible, so the answer is $2N!M!$. Don't forget to compute the answer modulo $10^9 + 7$.

D: Built?

We are asked to compute the minimum spanning tree.

Between two points (a, b) and (c, d), instead of adding an edge of cost $min(|a - c|, |b - d|)$, we add two edges: one edge with cost $|a - c|$ and one edge with cost $|b - d|$.

Suppose that there are three points p, q, r, and their x-coordinates satisfy $x_p < x_q < x_r$. Then, the edge between p and r with cost $x_r - x_p$ never appear in the MST (it is better to use an edge between p and q with cost $x_q - x_p$ and an edge between q and r with cost $x_r - x_q$).

Thus, we only need to consider the following $2(N - 1)$ edges:

- We sort the point by their x-coordinates, and for each adjacent pair of points add an edge between them (the cost is the difference of their x-coordinates).
- We sort the point by their y-coordinates, and for each adjacent pair of points add an edge between them (the cost is the difference of their y-coordinates).

We compute the MST of these edges. This can be done in $O(NlogN)$.

E: Connected?

We call an integer "important", if both occurrences of this integer is on the boundary of the rectangle.

Suppose that there are two important integers i and j, and they appear in the boundary in the order i, j, i, j. In this case, it is clear that we can't connect both integers without crossing, thus the answer is "No".

Otherwise, we can connect all integers in the following order:

- First, we connect non-important integers one by one.
- Then, we connect important integers one by one.

This way, we never get stuck.

How can we check if there exists such a pair (i, j) of important integers? Start from an arbitrary point on the boundary and check all important points on the boundary one by one in clockwise order (ignore non-important integers on the boundary). Initially we have an empty stack, and whenever we find an important integer x on the boundary, do the following:

- If the top element of the stack is also x, pop it.
- Otherwise, push x into the stack.

If the stack becomes empty after we check all points, the answer is "Yes". Otherwise the answer is "No". This algorithm works in $O(NlogN)$.

F: Exausted?

Let X be a set of people. Define $\Gamma(X)$ as the set of all chairs that can be used by at least one person in X. By Hall's Theorem, the answer of this task is the maximum of $|X| - \Gamma(X)$ (here X moves among all subsets of Takahashis).

$\Gamma(X)$ is one of the following forms:

- There exists some $s, t (s + 1 < t)$ and $\Gamma(X)$ is the set of all chairs whose coordinates are at most s or at least t.
- $\Gamma(X)$ is the set of all chairs.

For a fixed $\Gamma(X)$, we only need to consider X with the maximum cardinality.

If $\Gamma(X)$ is the set of all chairs, we can assume that X is the entire set. In this case, $|X| - \Gamma(X) = N - M$.

If $\Gamma(X)$ is the set of all chairs whose coordinates are at most s or at least t, the cardinality of X is the number of i such that $L_i \leq s$ and $t \leq R_i$. How can we compute the maximum of this value? For each i, we plot a point (L_i, R_i) on a plane, and we do a sweepline algorithm with a segment tree. The segment tree supports two queries: "add a given constant to a given range" and "find the maximum value among a given range". This works in $O(NlogN)$.

ABC 063 / ARC 075 Editorial (Japanese)

問題・解説: evima

2017 年 6 月 3 日

For international readers: English Editorial starts at page 6.

A: Restricted

問題の要求に素直に従うことが求められています。以下に、C++ での実装の例を示します。

```cpp
#include <iostream>
using namespace std;

int main(){
    int A, B;
    cin >> A >> B;
    if(A + B >= 10){
        cout << "error" << endl;
    }else{
        cout << A + B << endl;
    }
}
```

B: Varied

いくつかの方針が考えられます。

- ループを二重に組み合わせ、$1 \leq i < j \leq |S|$ であるような添字のペア (i, j) すべてについて S_i と S_j が異なるか確認する（$|S|$ が大きいと遅くなりますが、今回は $|S| \leq 26$ であり問題ありません[*1]）
- S に含まれる文字をアルファベット順にソートし、同じ文字が二つ隣り合わないか確認する
- 26 種類の文字それぞれについて S での出現回数を数え、どの文字の出現回数も 1 以下であるか確認する

以下に、最初の方針の C++ での実装の例を示します。

```cpp
#include <algorithm>
#include <iostream>
using namespace std;

int main(){
    string S;
    cin >> S;
    int N = S.size();
    string ans = "yes";
    for(int i = 0; i < N; ++i){
        for(int j = i + 1; j < N; ++j){
            if(S[i] == S[j]){
                ans = "no";
            }
        }
    }
    cout << ans << endl;
}
```

[*1] この問題の制約に反しますが、仮に $|S| = 10^5$ であったとすると、実行制限時間の 2 秒に間に合わないでしょう。余談ですが、鳩の巣原理より $|S| > 26$ のときに直ちに no と出力すれば、そのような長さの入力に対しても実行が間に合います。

C: Bugged

方針 1: 「正攻法」

すべての問題の配点の合計を S とします。

- S が 10 の倍数でない場合、S が画面に表示される最大の成績です。
- S が 10 の倍数である場合、画面に 0 でない成績を表示させるためには一問以上の問題で不正解する必要があります。もし、配点が 10 の倍数でないような問題が一問以上存在するなら、それらのうち配点が最も低い一問のみで不正解することで、画面に表示される最大の成績を得られます。配点が 10 の倍数でないような問題が一問も存在しない場合は、画面に 0 が表示されることを防げません。

方針 2: 「オーバーキル」

動的計画法を用いて、「解いた問題の配点の合計」として可能性のあるすべての値を時間計算量 $\mathrm{O}(N\sum s_i)$ で列挙することができます（詳細は省きます)。これは、答えを求めるのに十分過ぎる情報です。

D: Widespread

「残りの体力が最も多い魔物を中心に爆発を起こすことを繰り返す」のは最適な戦略ではありますが、この戦略を素直にシミュレートしてしまうと、例えば体力 10^9 の魔物が 10^5 体存在して $A = 2, B = 1$ の場合に長大な時間を要します。

整数 T に対し、 $\mathit{enough}(T)$ を「T 回以内の爆発ですべての魔物を消すことは可能か？」という問いの答え (Yes または No) とします。このとき、求めたい答えは $\mathit{enough}(T) = \mathrm{Yes}$ であるような T の最小値です。enough は単調性を持つ ($\mathit{enough}(X) = \mathrm{Yes}$ であれば $X \leq Y$ のとき $\mathit{enough}(Y) = \mathrm{Yes}$) ため、この最小値を二分探索[*2]で求めることができます。

定められた T の値に対して $\mathit{enough}(T)$ を判定するには、爆発を起こすことを以下のように捉え直すとよいでしょう:「すべての魔物の体力を B ずつ減らし、魔物を一体選んでその体力をさらに $A - B$ 減らす」。
すると、T 回の爆発を起こすことは、すべての魔物の体力を $B \times T$ ずつ減らし、その上で T 回にわたって一体の魔物の体力を $A - B$ 減らすこと（以下、この行為を「追加ダメージを与える」と表現します）と同等です。すべての魔物を消すには、体力が $B \times T$ より高い魔物 i それぞれに対して追加ダメージを $\lceil (h_i - B \times T)/(A - B) \rceil$ 回与える必要があり（$\lceil x \rceil$ は x 以上の最小の整数を表します)、この回数の合計が T 以下であれば $\mathit{enough}(T) = \mathrm{Yes}$、合計が T を超える場合は $\mathit{enough}(T) = \mathrm{No}$ と判定できます。

以上により、一つの T の値に対する $\mathit{enough}(T)$ の判定を時間計算量 $\mathrm{O}(N)$ で行うことができ、この解法の全体の時間計算量は $\mathrm{O}(N \log(h_{\max}/B))$ となります。

[*2] 二分探索自体の説明は省きます。例えば、日本語版 Wikipedia の当該記事に詳細な記述があります。

E: Meaningful Mean

問題文中の表記から a の添え字の範囲をずらし、この解説では $a = \{a_0, a_1, \ldots, a_{N-1}\}$ とします。

a の空でない連続する部分列 $\{a_l, a_{l+1}, \ldots, a_{r-1}\}$ $(0 \leq l < r \leq N)$（これも問題文中の表記と異なり、a_r が含まれないことに注意してください）の平均が K 以上であることは、次のように言い換えられます。

$$\sum_{i=l}^{r-1} a_i \geq (r-l)K \quad \Leftrightarrow \quad \sum_{i=0}^{r-1} a_i - \sum_{i=0}^{l-1} a_i \geq rK - lK \quad \Leftrightarrow \quad \sum_{i=0}^{r-1} a_i - rK \geq \sum_{i=0}^{l-1} a_i - lK$$

ここで $b_j = \displaystyle\sum_{i=0}^{j-1} a_i - jK$ $(0 \leq j \leq N)$ とおくと、この条件は単に $b_r \geq b_l$ と書けます。b_j $(0 \leq j \leq N)$ すべての値は容易に $\mathrm{O}(N)$ 時間で求めることができ、残るは $b_r \geq b_l$ であるようなペア (l, r) $(0 \leq l < r \leq N)$ の個数を数えることのみです。これには、何らかのデータ構造を用いることが必要となるでしょう。以下に、最も簡単と思われる方法を述べます。

まず、$N+1$ 個の値 $b_0, b_1, \ldots, b_N$ を、大小関係を保ったまま 0 から N の範囲の値に圧縮し、その値を $c_0, c_1, \ldots, c_N$ とします（ソートと二分探索を用います）。そして、Binary Indexed Tree[*3]を用いて $0, 1, \ldots, N$ のそれぞれの値の出現回数を保持します。これにより、$i = 0, 1, \ldots, N$ のそれぞれに対し、$c_0, c_1, \ldots, c_{i-1}$ に c_i 以下の値が何回出現したかをそれぞれ $\mathrm{O}(\log(N))$ 時間で計算することができ、合計 $\mathrm{O}(N \log(N))$ 時間で求めるべきペアの個数を計算することができます。

[*3] Binary Indexed Tree 自体の詳細な説明は省きますが、このデータ構造は長さ n の数列を特殊な形式で保持し、二種類の操作「数列の指定された位置の要素に指定された値を加算する」「数列の先頭から指定された位置までの要素の和を求める」をそれぞれ時間計算量 $\mathrm{O}(\log(n))$ で行うことができます。より詳細な情報は、例えば英語版 Wikipedia の記事 “Fenwick Tree” にあります。

F: Mirrored

与えられた式を $rev(N) - N = D$ とみなします。N の十進表記における桁数を L とし、$N = \sum_{i=0}^{L-1} 10^i n_i$ $(0 \leq n_i \leq 9, n_{L-1} \neq 0)$ と表記すると、$rev(N) - N$ は次のように変形できます。

$$rev(N) - N = \sum_{i=0}^{L-1} 10^{L-1-i} n_i - \sum_{i=0}^{L-1} 10^i n_i = \sum_{i=0}^{L-1} (10^{L-1-i} - 10^i) n_i = \sum_{i=0}^{\lfloor L/2 \rfloor - 1} (10^{L-1-i} - 10^i)(n_i - n_{L-1-i})$$

ここで $d_i = n_i - n_{L-1-i}$ $(0 \leq i \leq \lfloor L/2 \rfloor - 1)$ とおくと、

$$rev(N) - N = \sum_{i=0}^{\lfloor L/2 \rfloor - 1} (10^{L-1-i} - 10^i) d_i$$

この式の右辺を $f(L, d)$ とおきます。与えられた式を満たす N の個数を求めることは、$f(L, d) = D$ であるような L と -9 以上 9 以下の整数の列 $d_0, d_1, \ldots, d_{\lfloor L/2 \rfloor - 1}$ の組み合わせを列挙することとほぼ同等です(一つの d_i の列に対し、それに対応する N が複数個存在することを考慮する必要がありますが)。ここで、$0 \leq i < \lfloor L/2 \rfloor - 1$ であるような任意の i に対して次の式が成立します。

$$10^{L-1-i} - 10^i > \sum_{j=i+1}^{\lfloor L/2 \rfloor - 1} ((10^{L-1-j} - 10^j) \cdot 9) + 10^{L - \lfloor L/2 \rfloor}$$

(この式の「意味」は次の通りです:「ある i $(\neq \lfloor L/2 \rfloor - 1)$ に対して $d_i = 1$ とすると、$j > i$ であるようなすべての j に対して $d_j = -9$ としても、$d_i = 1$ による $f(L, d)$ への正の方向への影響が残り、その残りは $10^{L - \lfloor L/2 \rfloor}$ より大きい」) この式から、次の二点がいえます。

- D の十進表記における桁数を L_D とすると、$L > 2L_D$ で $f(L, d)$ が正のとき、$f(L, d)$ が D より大きくなることがわかります。したがって、L の値として L_D 以上 $2L_D$ 以下のすべての値を検討すれば十分です。
- L の値を定めたとき、$f(L, d) = D$ であるような整数列 $d_0, d_1, \ldots, d_{\lfloor L/2 \rfloor - 1}$ を列挙するために(そして与えられた式を満たす N の個数を数えるために)、d_0 から順に値を定めていく深さ優先探索を行うことを考えます。d_{i-1} までの値をすでに定めているとき、d_i の値として検討する必要がある候補は、$dif = N - \sum_{j=0}^{i-1} ((10^{L-1-j} - 10^j) \cdot d_j)$ として、$(10^i - 10^{L-1-i}) d_i \leq dif$ であるような最大の値と、$(10^i - 10^{L-1-i}) d_i > dif$ であるような最小の値の高々二つのみであることがわかります。(直感的に述べると、探索の途中で $f(L, d)$ の「途中まで」の値が D からあまり離れてしまうとそれ以降「戻ってくる」ことはできないため、D の「近くから離れる」べきではありません。) したがって、L の値を定めたとき、与えられた式を満たす N であって桁数が L であるものの個数を時間計算量 $\mathrm{O}(2^{\lfloor L/2 \rfloor})$ で求めることができます。

以上の二点より、求めるべき N の個数を時間計算量 $\mathrm{O}(\sum_{i=L_D}^{2L_D} 2^{\lfloor i/2 \rfloor}) = \mathrm{O}(2^{L_D})$ で求めることができます。

ABC 063 / ARC 075 Editorial (English)

Problems and editorial by evima

June 3, 2017

A: Restricted

It is required to do exactly what is told. C++ implementation follows:

```cpp
#include <iostream>
using namespace std;

int main(){
    int A, B;
    cin >> A >> B;
    if(A + B >= 10){
        cout << "error" << endl;
    }else{
        cout << A + B << endl;
    }
}
```

B: Varied

There are several solutions:

- Nest two loops and check if S_i and S_j differs for all pairs of indices (i, j) such that $1 \leq i < j \leq |S|$ (slower if $|S|$ is larger, but fine in this problem since $|S| \leq 26$[*1])
- Sort the characters in S in alphabetical order, and check if no two adjacent letters are the same
- For each of the 26 letters, count the occurrences in S, and check if no letter appears more than once

C++ implementation of the first solution follows:

```cpp
#include <algorithm>
#include <iostream>
using namespace std;

int main(){
    string S;
    cin >> S;
    int N = S.size();
    string ans = "yes";
    for(int i = 0; i < N; ++i){
        for(int j = i + 1; j < N; ++j){
            if(S[i] == S[j]){
                ans = "no";
            }
        }
    }
    cout << ans << endl;
}
```

[*1] The constraints in this problem does not allow this, but if $|S| = 10^5$, it would not fit within the execution time limit of 2 seconds. A little off-topic, but if we immediately print "no" when $|S| > 26$ by pigeonhole principle, it will run in time.

C: Bugged

Solution 1: "Straight-forward"

Let S be the total points allocated to all the problems.

- If S is not a multiple of 10, S is the maximum value we seek.
- If S is a multiple of 10, in order to display a value other than 0, we need to incorrectly answer at least one question. If there is at least one question with a score which is not a multiple of 10, incorrectly answer one of the problems with the lowest score allocated among them, and we can obtain the maximum grade that can be correctly displayed. If there is no such question, there is no way to prevent the system from displaying 0.

Solution 2: "Overkill"

By Dynamic Programming, we can enumerate all the possible candidates of "the total score allocated to correctly answered problems" in $O(N \sum s_i)$ time (we will omit the detail). This is more than enough to find the answer.

D: Widespread

It is indeed optimal to repeatedly cause an explosion centered at the monster with the most health remaining, but straight-forward simulation of this strategy will take a long time when, for example, there are 10^5 monsters with 10^9 health each and $A = 2, B = 1$.

For an integer T, let *enough*(T) be the answer to the question "is it possible to vanish all the monster in at most T explosions?": either "yes" or "no". Then, the answer we seek is the minimum value of T such that *enough*(T) = yes. Since *enough* is monotonic (that is, if *enough*(X) = yes, *enough*(Y) = yes when $X \leq Y$), we can perform binary search[*2] to find this minimum.

In order to determine *enough*(T) for a fixed value of T, it would be useful to understand causing an explosion in the following way: "decrease the health of all the monsters by B, then further decrease the health of one selected monster by $A - B$." Causing T explosions is now equivalent to decreasing the healths of all the monsters by $B \times T$, then do the following T times: further decrease the health of one selected monster by $A - B$ (we will call this an "extra attack"). In order to vanish all the monsters, we need to deliver $\lceil (h_i - B \times T)/(A - B) \rceil$ extra attacks to each monster i with a health greater than $B \times T$. If the total required number of extra attacks is at most T, *enough*(T) = yes, and otherwise no.

We can now determine *enough*(T) for a fixed value of T in $\mathrm{O}(N)$ time, which leads to the total time complexity of $\mathrm{O}(N \log(h_{\max}/B))$.

[*2] We will not explain the notion of binary search itself here. Resources can be found in, for example, Wikipedia.

E: Meaningful Mean

We will shift the range of the index of a and let $a = \{a_0, a_1, \ldots, a_{N-1}\}$ in this editorial.

We can transform the condition "the non-empty contiguous subsequence of a, $\{a_l, a_{l+1}, \ldots, a_{r-1}\}$ $(0 \leq l < r \leq N)$ (note that a_r is excluded), has a mean greater than or equal to K" into:

$$\sum_{i=l}^{r-1} a_i \geq (r-l)K \quad \Leftrightarrow \quad \sum_{i=0}^{r-1} a_i - \sum_{i=0}^{l-1} a_i \geq rK - lK \quad \Leftrightarrow \quad \sum_{i=0}^{r-1} a_i - rK \geq \sum_{i=0}^{l-1} a_i - lK$$

Let $b_j = \sum_{i=0}^{j-1} a_i - jK$ $(0 \leq j \leq N)$, and this condition can be simply written as $b_r \geq b_l$. We can easily find all the values of b_j $(0 \leq j \leq N)$ in $\mathrm{O}(N)$ time, and what remains is to count the number of the pairs (l, r) $(0 \leq l < r \leq N)$ such that $b_r \geq b_l$. It would be necessary to utilize some kind of data structure. We will explain a method that we think is easiest.

First, compress the $N+1$ values $b_0, b_1, \ldots, b_N$ into a range between 0 and N, keeping the magnitude relationship (by sorting and binary search), and let the compressed values be $c_0, c_1, \ldots, c_N$. Then, maintain the numbers of occurrences of the values between 0 and N using Binary Indexed Tree[*3]. In this manner, for each of $i = 0, 1, \ldots, N$, we can found the number of occurrences of values less than or equal to c_i among $c_0, c_1, \ldots, c_{i-1}$ in $\mathrm{O}(\log(N))$ time, which enables us to compute the desired number of pairs in a total of $\mathrm{O}(N \log(N))$ time.

[*3] We will not explain Binary Indexed Tree in detail here, but here is the summary: this data structure maintains a sequence of length n in a special format, and it can perform each of the following kinds of operations in $\mathrm{O}(\log(n))$ time: "incrementing the element at the specified position by a specified amount" and "finding the sum of the elements from the beginning of the sequence up to the specified position". More information can be found in, for example, Wikipedia (in the name of "Fenwick Tree" in English version).

F: Mirrored

We will see the given formula as $rev(N) - N = D$. Let the number of digits of N in decimal notation as L, and $N = \sum_{i=0}^{L-1} 10^i n_i$ $(0 \leq n_i \leq 9, n_{L-1} \neq 0)$. Then, $rev(N) - N$ can be transformed into:

$$rev(N) - N = \sum_{i=0}^{L-1} 10^{L-1-i} n_i - \sum_{i=0}^{L-1} 10^i n_i = \sum_{i=0}^{L-1} (10^{L-1-i} - 10^i) n_i = \sum_{i=0}^{\lfloor L/2 \rfloor - 1} (10^{L-1-i} - 10^i)(n_i - n_{L-1-i})$$

Here, let $d_i = n_i - n_{L-1-i}$ $(0 \leq i \leq \lfloor L/2 \rfloor - 1)$, which results in:

$$rev(N) - N = \sum_{i=0}^{\lfloor L/2 \rfloor - 1} (10^{L-1-i} - 10^i) d_i$$

Let the right side of this formula be $f(L, d)$. Finding the count of N that satisfy the given formula is almost equivalent to enumerating the pairs of L and a sequence of integers between -9 and 9, $d_0, d_1, \ldots, d_{\lfloor L/2 \rfloor - 1}$ (we also need to take into account that there is more than one N that corresponds to a sequence of d_i, though). Here, for any i such that $0 \leq i < \lfloor L/2 \rfloor - 1$, the following holds:

$$10^{L-1-i} - 10^i > \sum_{j=i+1}^{\lfloor L/2 \rfloor - 1} ((10^{L-1-j} - 10^j) \cdot 9) + 10^{L - \lfloor L/2 \rfloor}$$

(The "meaning" of this is as follows: "Suppose that $d_i = 1$ for some i $(\neq \lfloor L/2 \rfloor - 1)$. Then, even if we set $d_j = -9$ for all j such that $j > i$, the 'positive effect' on $f(L, d)$ by $d_i = 1$ is not completely negated, and more than $10^{L - \lfloor L/2 \rfloor}$ still remains.") Based on this, the following can be observed:

- Let L_D be the number of digits of D in decimal notation. Then, when $L > 2L_D$ and $f(L, d) > 0$, it can be seen that $f(L, d) > D$. Therefore, it is enough to consider the values between L_D and $2L_D$ as the value of L.
- For a fixed value of L, consider enumerating the sequences $d_0, d_1, \ldots, d_{\lfloor L/2 \rfloor - 1}$ such that $f(L, d) = D$ (and finding the count of N that satisfy the given formula), by performing Depth First Search starting from d_0. When the values up to d_{i-1} are already decided, it can be seen that there are at most two candidates of the value of d_i that have to be considered: the maximum value such that $(10^i - 10^{L-1-i}) d_i \leq dif$, and the minimum value such that $(10^i - 10^{L-1-i}) d_i > dif$. (Intuitively, if the "halfway" value of $f(L, d)$ during the search gets too far from D, it is not possible to "get back", and thus it should "stay close" to D.) Therefore, for a fixed value of L, we can find the count of N that satisfy the given formula in $O(2^{\lfloor L/2 \rfloor})$ time.

The above enables us to find the desired count of N in $O(\sum_{i=L_D}^{2L_D} 2^{\lfloor i/2 \rfloor}) = O(2^{L_D})$ time.

ABC #062 / ARC #074 Editorial

writer : sugim48

2017/05/20

For International Readers: English editorial starts on page 5.

A: Grouping

すべての (x, y) のペアについて `Yes` / `No` を記録しておく方法では，$\binom{12}{2} = 66$ 通りの場合分けが必要になり，大変です．代わりに，グループに $1, 2, 3$ と番号を振っておき，各 $1 \leq x \leq 12$ ごとに x が属するグループの番号を記録しておけば，x, y が属するグループの番号を比較することで `Yes` / `No` は簡単に判定できます．

C++ のコード例

```
int main() {
        int a[] = {0, 1, 3, 1, 2, 1, 2, 1, 1, 2, 1, 2, 1};
        // 配列の添字は 0 から始まるので，a[0] は 0 にしている．
        int x, y; cin >> x >> y;
        cout << (a[x] == a[y] ? "Yes" : "No") << endl;
}
```

B: Picture Frame

最初に，縦 $H + 2$ 行，横 $W + 2$ 列の文字配列 b を用意し，`#` で埋めておきます．次に，各 $1 \leq i \leq H$, $1 \leq j \leq W$ ごとに $a_{i,j}$ を $b_{i+1,j+1}$ へコピーします．最後に，配列 b の内容を出力すればよいです．

ちなみに，迷路などが入力として与えられたとき，この問題のようにあらかじめ外壁で囲んでおくと，その後の実装が簡単になることがあります．

C++ のコード例

```
char a[100][101], b[102][102];

int main() {
        int H, W; cin >> H >> W;
        for (int i = 0; i < H; i++)
                scanf("%s", a[i]);
```

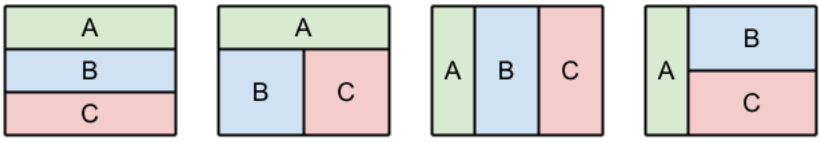
```
        for (int i = 0; i < H + 2; i++)
                for (int j = 0; j < W + 2; j++)
                        b[i][j] = '#';

        for (int i = 0; i < H; i++)
                for (int j = 0; j < W; j++)
                        b[i + 1][j + 1] = a[i][j];

        for (int i = 0; i < H + 2; i++) {
                for (int j = 0; j < W + 2; j++)
                        cout << b[i][j];
                cout << endl;
        }
}
```

C : Chocolate Bar

分割の方法は次図の 4 ケースを考えれば十分です．



左の 2 ケースについて答えを求める方法があれば，H, W を入れ替えて同じ方法を用いることで右の 2 ケースについても答えを求められます．これら 2 通りの答えの最小値が全体の答えとなります．

では，左の 2 ケースについて答えを求める方法を考えましょう．まず，長方形 A の縦幅 h $(1 \leq h \leq H-1)$ を全探索します．10^5 オーダーの計算回数ならば，TLE の心配はありません．これで，長方形 A の面積 S_A は $S_A = hW$ と決まります．あとは，下半分の長方形を長方形 B, C へ分割する方法を決めればよいです．

下半分の長方形を分割する方法は，縦 2 つに分割するケースと，横 2 つに分割するケースがあります．これらはどちらも試すことにしましょう．ここでは，横 2 つに分割するケースを説明します．先程と同様に，長方形 B の横幅を全探索したいところですが，全体で 10^{10} オーダーの計算回数となり，TLE してしまいます．実は，$S_{max} - S_{min}$ をできるだけ小さくするためには，下半分の長方形もできるだけ均等に分割するのがよいことが示せます．できるだけ均等に分割しようとすると，長方形 B の横幅 w は $w = \lfloor W/2 \rfloor$ となります．すると，長方形 B の面積 S_B は $S_B = (H-h)w$，長方形 C の面積 S_C は $S_C = (H-h)(W-w)$ と決まります．S_A, S_B, S_C がすべて決まったので，この分割における $S_{max} - S_{min}$ が求まります．以上すべての分割における $S_{max} - S_{min}$ の最小値が，左の 2 ケースについての答えです．

D : 3N Numbers

数列 a のうち，取り除く N 要素を黒色で，a' の前半 N 要素を赤色で，a' の後半 N 要素を青色で表示することにします．例えば，数列 $(8,2,2,7,4,6,5,3,8)$ の最適解は $(8,2,2,7,4,6,5,3,8)$ となります．

次の条件が成り立つような整数 k を $N \leq k \leq 2N$ の範囲で全探索することにします．

- a の前半 k 要素は黒色または赤色のみで，a の後半 $3N-k$ 要素は黒色または青色のみである．

各 k について答えを求められれば，それらの最大値が全体の答えとなります．では，各 k について答えを求める方法を考えていきましょう．

k をひとつ固定します．このとき，a の前半 k 要素のうち，ちょうど N 要素が赤色で，残りの要素が黒色です．同様に，a の後半 $3N-k$ 要素のうち，ちょうど N 要素が青色で，残りの要素が黒色です．ここで，赤い要素の選び方と青い要素の選び方は独立であることに注意してください．答えは (赤い要素の総和) − (青い要素の総和) の最大値なので，(赤い要素の総和) をできるだけ大きくし，(青い要素の総和) をできるだけ小さくすればよいことが分かります．以降は，(赤い要素の総和) の最大値の求め方のみを考えることにします．

(赤い要素の総和) をできるだけ大きくするためには，a の前半 k 要素のうち大きい方から N 要素を赤くすればよいです．これは，a の前半 k 要素を大きい順にソートし，前半 N 要素の総和を求めれば，可能ではあります．しかし，この方法をすべての k $(N \leq k \leq 2N)$ について行うと，全体の計算量は $\mathrm{O}(N^2 \log N)$ となって TLE してしまいます．すべての k について，a の前半 k 要素のうち大きい方から N 要素の総和を求める効率的な方法はないでしょうか？

これは，優先度付きキューと呼ばれるデータ構造を用いて簡単に行うことができます．優先度付きキューは，「要素の push」「最小要素の pop」がともに $\mathrm{O}(\log N)$ 時間で行えるキューです．あらかじめ，キューに a の前半 N 要素を push しておきます．また，現在キューに入っている要素の総和を保持しておきます．以降，k を N から $2N$ までインクリメントしていきますが，a の前半 k 要素のうち大きい方から N 要素がキューに入っている状態を常に保つことにします．これは，要素 a_k をキューに push した後，キューの最小要素を pop すれば可能です．また，現在キューに入っている要素の総和も，push / pop された要素の分だけ足し引きすれば，保持できます．以上の方法で，すべての k について (赤い要素の総和) が $\mathrm{O}(N \log N)$ 時間で求まります．

同様にして，すべての k について (青い要素の総和) を求めておけば，あとは各 k について (赤い要素の総和) − (青い要素の総和) を計算し，それらの最大値を取れば全体の答えが求まります．

E : RGB Sequence

この問題は $\mathrm{O}(N^3 + N^2 M)$ 時間の DP で解けます．

左から順番にマスの色を塗っていくことにします．既に色を塗ったマスのうち，最も右にある赤いマスの index (1-origin) を r とします．ただし，赤いマスが存在しなければ $r = 0$ とします．同様に，緑のマス，青いマスについてもそれぞれ g, b を定義します．この状態に対応する色の履歴の通り数を $dp[r][g][b]$ とします．

まず，DP の遷移を考えます．$k = \max\{r, g, b\}$ とすると，次に色を塗るマスの index は $k+1$ であることが分かります．このマスに 赤 / 緑 / 青 のどれを塗るかによって，それぞれ r / g / b が $k+1$ へ変わります．例えば，このマスに赤を塗ることに対応する遷移は，`dp[k+1][g][b] += dp[r][g][b]` となります．

次に，条件が成り立たない配色を数え上げない方法について考えます．区間 $[l_i, r_i]$ に対する条件のチェックは，$\max\{r, g, b\} = r_i$ のタイミングで行うことにします．説明のため，$r < g < b (= r_i)$ とすると，「区間 $[l_i, r_i]$ にちょうど x_i 色含まれる」という条件は，l_i と r, g の大小関係の条件で表せます．例えば，$x_i = 2$ の場合，$r < l_i$ かつ $l_i \leq g$ でなければなりません．このような条件が成り立たないような $dp[r][g][b]$ は 0 にしてしまえばよいです．

F : Lotus Leaves

この問題は，グラフの最小カット問題に帰着できます．以降は，グラフの作り方を説明します．

グラフの頂点は，始点 s，終点 t，赤い頂点 1, 2, ..., H，青い頂点 1, 2, ..., W です．赤い頂点 y $(1 \leq y \leq H)$ は，カエルが行 y のどこかのマスにいる，という状態を表します．同様に，青い頂点 x $(1 \leq x \leq W)$ は，カエルが列 x のどこかのマスにいる，という状態を表します．よって，マス S が (y_s, x_s) のとき，始点 s から赤い頂点 y_s と青い頂点 x_s へ，それぞれ容量 ∞ の有向辺を張ります．同様に，マス T が (y_t, x_t) のとき，赤い頂点 y_t と青い頂点 x_t から終点 t へ，それぞれ容量 ∞ の有向辺を張ります．さらに，それぞれの蓮の葉 (y, x) について，赤い頂点 y と青い頂点 x の間に，容量 1 の無向辺を張ります．この無向辺は，カエルが葉 (y, x) で行 y 内の移動と列 x 内の移動を切り替えられる，ということを表します．

以上のグラフにおいて，$s \to t$ 間の最小カットが問題の答えとなります．これは，$s \to t$ 間の最大フローと一致するので，Ford-Fulkerson のアルゴリズムなどで求まります．

ABC #062 / ARC #074 Editorial

writer : sugim48

2017/05/20

A: Grouping

C++ Code Example

```
int main() {
        int a[] = {0, 1, 3, 1, 2, 1, 2, 1, 1, 2, 1, 2, 1};
        int x, y; cin >> x >> y;
        cout << (a[x] == a[y] ? "Yes" : "No") << endl;
}
```

B: Picture Frame

C++ Code Example

```
char a[100][101], b[102][102];

int main() {
        int H, W; cin >> H >> W;
        for (int i = 0; i < H; i++)
                scanf("%s", a[i]);

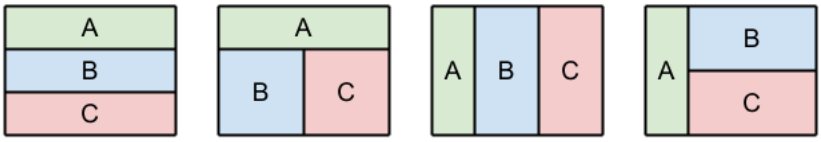
        for (int i = 0; i < H + 2; i++)
                for (int j = 0; j < W + 2; j++)
                        b[i][j] = '#';

        for (int i = 0; i < H; i++)
                for (int j = 0; j < W; j++)
                        b[i + 1][j + 1] = a[i][j];
```

```
        for (int i = 0; i < H + 2; i++) {
                for (int j = 0; j < W + 2; j++)
                        cout << b[i][j];
                cout << endl;
        }
}
```

C : Chocolate Bar

There are four cases:



In the first two cases, we fix the height h of the rectangle A (and try all possible values of h).

The difference between B and C should be as small as possible. In the first case, the heights of B and C should be $floor((H-h)/2)$ and $ceil((H-h)/2)$, and in the second case, the widths of B and C should be $floor(W/2)$ and $ceil(W/2)$.

The last two cases are similar to the first two cases.

D : 3N Numbers

Let's color the removed elements black, the first N elements of a' red, and the last N elements of a' blue. For example, one optimal solution for the sequence $(8,2,2,7,4,6,5,3,8)$ can be represented as $(8,2,2,7,4,6,5,3,8)$.

There exists an integer $k(N \leq k \leq 2N)$ such that:

- The first k elements of a are either red or black.
- The last $3N-k$ elements of a are either blue or black.

Fix such an integer k. Obviously, we should choose the largest N integers from the first k elements as red elements, and the smallest N integers from the last $3N-k$ elements as blue elements. For each $k(N \leq k \leq 2N)$, we compute the sum of the largest N integers among $a_1, \ldots, a_k$. This is a well-known task, and it can be done in $O(NlogN)$ using priority queue. Similarly, we can compute the sum of blue elements for each k, and the answer is the maximum of red minus blue.

E : RGB Sequence

We color the squares from left to right. Define $dp[r][g][b]$ as the number of ways to color the first $k = \max\{r, g, b\}$ squares, such that:

- The last square we painted red is the r-th square (1-based). $r = 0$ if no square was painted red.
- The last square we painted green is the g-th square.
- The last square we painted blue is the b-th square.
- There's no contradiction in the first k squares: if $r_i \leq k$, the number of different colors in the range $[l_i, r_i]$ must be exactly x_i.

The transitions will be like "`dp[k+1][g][b] += dp[r][g][b]`, `dp[r][k+1][b] += dp[r][g][b]`, `dp[r][g][k+1] += dp[r][g][b]`". However, when $dp[r][g][b]$ is contradictory, we should do "$dp[r][g][b] = 0$;" instead.

This solution works in $O(N^3 + N^2M)$.

F : Lotus Leaves

Convert the grid into a bipartite graph. We should see the cell (i, j) as an edge that connects the i-th red vertex and the j-th blue vertex. Then, a path of the frog is a path in this bipartite graph.

We want to cut S and T by removing leaves (i.e., edges in the bipartite graph). The answer is the minimum s-t cut of the following graph:

- The vertices are s, t, red vertices 1, 2, ..., H, and blue vertices 1, 2, ..., W.
- If $S = (x_s, y_s)$, add an edge between s and the x_s-th red vertex, and an edge between s and the y_s-th blue vertex, each with infinite capacity.
- If $T = (x_t, y_t)$, add an edge between t and the x_t-th red vertex, and an edge between t and the y_t-th blue vertex, each with infinite capacity.
- For each leaf (x, y), add an edge between the x-th red vertex and the y-th blue vertex with capacity 1.

AtCoder Regular Contest 073 Editorial

Kohei Morita(yosupo)

平成 29 年 4 月 29 日

A: Shiritori

文字列の入力と if 文を書くことが必要になります。
以下に python3 でのコード例を示します。

```
a, b, c = input().split()
if a[len(a)-1] == b[0] and b[len(b)-1] == c[0]:
    print('YES')
else:
    print('NO')
```

B: Choice Integers

A の倍数はいくつ足しても A の倍数です。よって、実は選ぶ数は 1 個だけで良いです (いくつか選んで足さなくても、最終的な総和を直接選べます)。

次に、$A\%B, 2A\%B, 3A\%B, ...$ という数列を考えます。なお $A\%B$ は A を B で割ったあまりを表します。

ここで、$(k+B)A\%B = (kA+BA)\%B = kA\%B$ に注目すると、この数列は周期的で、最初の B 個の要素を繰り返す数列になっていることがわかります。

よって、この問題は A から BA まで、愚直に B で割った余りを求めて調べれば良いです。

C: Sentou

i 人目がスイッチを押した後、$i+1$ 人目が

- T 秒以内に来るならば、ずっとお湯は出続ける
- T 秒よりも経ってからくるならば、お湯は T 秒間出て止まる

ということがわかります。

よってこの問題の答えは、それぞれの人について、次の人が何秒後に来るかを求め、$min(T, $次の人が来るまでの時間$)$ の総和を求めれば良いです。

D: Simple Knapsack

この問題はナップザック問題として知られていて、効率良く解くアルゴリズムは存在しないと信じられています。ただし、特殊な制約がある場合はその限りではありません。例えば N や W や v_i のうちどれかが小さい場合は ABC032 D 問題で出題されています。

今回は、すべての $i = 2, 3, \cdots, N$ について、$w_1 \leq w_i \leq w_1 + 3$ という特殊な制約を利用するのだろうと考えられます。

この性質に注目すると、物の重さは高々 4 種類であることがわかります。

よって、各重さについてその重さの物を何個選ぶか決めてしまいます。すると、各重さごとに価値の高い順に使うことにすれば良くなります。

そして選び方は、最大でも $(N/4)^4 = 25^4 = 390625$ 通りありますが、この程度ならば全探索が可能です。

使う個数を決めた後は、各重さごとに価値の高いものから使っていくことになります。この、価値の総和は O(選んだ個数) で計算できます。

C++や Java や D 言語など、高速な言語ならばこれで間に合うと思いますが、Python や Ruby などの場合、累積和などを使い $O(1)$ で計算できるようにしないと厳しいかもしれません。

E: Ball Coloring

ボールは 400,000 個もありますが、答えに影響するパラメーターは $R_{max}, R_{min}, B_{max}, B_{min}$ の高々 4 個だけであることに注目します。

更に、

$$MIN = min(x_1, x_2, ..., x_n, y_1, y_2, ..., y_n)$$

$$MAX = max(x_1, x_2, ..., x_n, y_1, y_2, ..., y_n)$$

とすると、$R_{min} = MIN, B_{min} = MIN$ のどちらかは満たし、$R_{max} = MAX, B_{max} = MAX$ のどちらかは満たすことがわかります。

もっというと、$R_{min} = MIN \& B_{max} = MAX$ と $R_{min} = MIN \& B_{max} = MAX$ のどちらかを仮定して良いです。

よってこれで場合分けをします。

$R_{min} = MIN \& B_{max} = MAX$ の場合

この場合、R_{min} や B_{max} がこれ以上小さくなったり大きくなったりすることはありません。よって、各袋について、小さい方を赤色、大きい方を青色にすれば良いです。

$R_{min} = MIN \& R_{max} = MAX$ の場合

最小のボールと最大のボールが同じ色の場合です。

この場合、赤色の方には何も考えず好きなボールを押し付けられます。

よって青色に塗るボールのみ着目すればよいです。

つまり、各袋からボールを 1 個選び最大-最小を最小化する、という問題を考えれば良いです。

これは、最初全ての袋について小さい方のボールを選んだと仮定し、

それをそのなかで小さい順に並べ、順番に、そのボールを袋のもう一つのボールと交換する、ということを行えば良いです。

F: Many Move

この問題は愚直な DP を考え、それを高速化していく方針を取ります。

まず、$O(N^3)$ の DP として以下の様なものが考えられます。

$dp[i][a][b] := i$ 個のクエリを処理しており、コマは a と b にある。今までかかった時間の最小

ここで、どちらかのコマは必ず直前のクエリの位置にいることを考えると、$O(N^2)$ に高速化が出来ます。

$dp[i][a] := i$ 個のクエリを処理しており、コマは a と x_i にある。今までかかった時間の最小

$x_0 = B$ とすると、

$dp[0][A] = 0, dp[0][x] = \infty (x \neq A)$

を初期値として、

$\min(dp[Q][1], dp[Q][2], ..., dp[Q][N])$

を答えとすれば良いことがわかります。

この DP の漸化式は、

$dp[i][a] = dp[i-1][a] + |x_{i-1} + x_i| (a \neq x_i)$

$dp[i][x_{i-1}] = \min(dp[i-1][x_{i-1}] + |x_{i-1} + x_i|, \min_{j=1,2,...,n}(dp[i-1][j] + |j - x_i|))$

となります。

ここで、DP テーブル $dp[i-1][1], dp[i-1][2], ..., dp[i-1][n]$ を配列として持っているとします。そしてここから一気に $dp[i][1], dp[i][2], ..., dp[i][n]$ を計算することを考えます。

上の漸化式から考えると、

1. 配列全体に $|x_{i-1} + xi|$ を足し
2. $\min_{j=1,2,...,n}(dp[i-1][j] + |j - x_i|)$ を求め、$dp[i][x_{i-1}]$ より小さければ代入

という操作ができれば良いことがわかります。

難しい操作は $\min_{j=1,2,...,n}(dp[i-1][j] + |j - x_i|)$ ですが、これは j と x_i の大小で場合分けをすると

$\min_{j=1,2,...,x_i}(dp[i-1][j] - j + x_i)$

$\min_{j=x_i+1,2,...,n}(dp[i-1][j] + j - x_i)$

が求められれば良くなります。

これは、$dp[i-1][j] - j$ と $dp[i-1][j] + j$ を持った配列に対する区間の min を求める操作になります

整理すると、実は持つべきは $dp[i-1][j]$ の配列ではなく、$dp[i-1][j] - j$ と $dp[i-1][j] + j$ の配列です。

$dp[i-1][j]-j$ と $dp[i-1][j]+j$ の配列を持っておくと、全体に add と区間 min ができれば良くなります。これは、Segment Tree で解くことが出来ます。

AtCoder Regular Contest 073 Editorial

Kohei Morita(yosupo)

平成 29 年 4 月 29 日

A: Shiritori

python3 example:

```
a, b, c = input().split()
if a[len(a)-1] == b[0] and b[len(b)-1] == c[0]:
    print('YES')
else:
    print('NO')
```

B: Choice Integers

The sum of multiples of A is always a multiple of A. Thus, we can assume that we choose only one integer.

Consider the sequence $A\%B, 2A\%B, 3A\%B, ...$ Since $(k+B)A\%B = (kA+BA)\%B = kA\%B$, this sequence has a period of B.

Therefore, we can check the first B elements of this sequence by brute force.

C: Sentou

After the i-th person pushes the switch, the $i+1$-th person

- If the i+1-th person comes within T seconds, the water keeps emitting.
- If the $i+1$-th person comes after at least T seconds, the water emits for T seconds and stops.

Therefore, the answer is the sum of $min(T, \mathrm{t_{i+1}} - \mathrm{t_i})$.

D: Simple Knapsack

This problem is known as the knapsack problem, and it's hard to solve in general case.

This time we use the constraint $w_1 \leq w_i \leq w_1 + 3$. Because of this, there are at most 4 possible weights for a single item.

For each weight w, let's fix k_w, the number of items of weight w we choose. Obviously, we should choose k_w items greedily (in the descending order of values).

Let A, B, C, D be the number of items of weights w_1, w_1+1, w_1+2, w_1+3, respectively. This algorithm works in $O(ABCD)$. In the worst case, this is $(N/4)^4 = 25^4 = 390625$.

E: Ball Coloring

Let

$$MIN = min(x_1, x_2, ..., x_n, y_1, y_2, ..., y_n)$$

$$MAX = max(x_1, x_2, ..., x_n, y_1, y_2, ..., y_n)$$

.

One of $R_{min} = MIN, B_{min} = MIN$ will be satisfied, and one of $R_{max} = MAX, B_{max} = MAX$ will be satisfied. Without loss of generality, we can consider the following two cases:

$R_{min} = MIN \& B_{max} = MAX$

In this case we want to minimize R_{max} and maximize B_{min}. For each bag, we should color the ball with the larger integer blue, and color the other ball blue.

$R_{min} = MIN \& R_{max} = MAX$

In this case we are only interested in blue balls (the value of $B_{max} - B_{min}$. First, for each bag, we color the ball with the smaller integer blue, and sort the blue balls in ascending order. Call them $z_1, \ldots, z_n$ (in ascending order). Then, for each $i = 1, \ldots, n$, color z_i red and color the opponent of z_i blue.

F: Many Move

Let $dp[i][a][b]$ be the minimum cost required to process the first i queries such that the tokens are at a and b after the queries. This DP works in $O(N^3)$. We'll improve this.

Let $dp[i][a]$ be the minimum cost required to process the first i queries such that the tokens are at a and x_i after the queries. This DP works in $O(N^2)$.

Suppose that we have an array $dp[i-1][1], dp[i-1][2], ..., dp[i-1][n]$, and we want to convert it to the array $dp[i][1], dp[i][2], ..., dp[i][n]$.

The following operations are required:

1. Add $|x_{i-1} + xi|$ to the whole array.
2. Compute $\min_{j=1,2,...,n}(dp[i-1][j] + |j - x_i|)$.

This can be done by two RMQs: one of them holds the values of $dp[i-1][j] - j$ and the other one holds the values of $dp[i-1][j] + j$.

ABC 059 / ARC 072 解説

writer : hogloid

2017 年 4 月 22 日

A : Three-letter acronym

「小文字を大文字にする」という操作は、ライブラリ関数を用いるか、ASCII 文字コードで大文字や小文字は連続して配置されていることを利用して文字コードの差を足し算して求めることもできます。

C++ のコード例

```
int main(){
  string a, b, c;
  cin >> a >> b >> c;
  char dif='A' - 'a';
  printf("%c%c%c\n", a[0] + dif, b[0] + dif, c[0] + dif);
        return 0;
}
```

B : Comparison

まず、2 つの数の桁数が異なる場合、桁の多い数の方が大きいです。桁数が等しい場合、数の大小は大きい桁から順に並べた文字列の辞書式比較と等しくなることをプログラムすればよいです。

C : Sequence

偶数番目と奇数番目どちらを正にするかで 2 通り考え、小さい方を答えることにします。番号の小さい順に数を確定させていきます。

i 桁目までの和の符号が欲しい符号と同じ場合、i 番目の数を変更する必要はありません。すでに i 番目までの和の符号が合致しているのに i 番目の数を変更して最適解が得られるなら、i 番目の数に対する変更を $i+1$ 番目の数に持ってくることで同じ最適解が得られるからです。

i 桁目までの和の符号が欲しい符号と異なる場合、まず欲しい符号のうち絶対値最小までは変更しなくてはいけません (1 か -1)。これ以後は上と同様の議論が成り立つので、1 か -1 まで変更すれば十分です。

この規則で前から順に決めコストを求めていくことで答えが求まります。

D : Alice&Brown

結論としては、$|X-Y| \leq 1$ のとき Brown、そうでなければ Alice が勝ちます。以下の帰納法を $X+Y$ の昇順に回すことで、この性質が成り立つことがわかります。

- $X+Y \leq 1$ のとき有効な操作がないため、Brown が勝ちます。
- $X+Y>1, |X-Y| \leq 1$ のときどんな操作をしても、$|X-Y|>1$ で相手に手番を渡してしまうため、帰納法の仮定により相手が勝ち、自分は負けます。
- $X+Y>1, |X-Y|>1$ のとき山のうち石が多くある方の山から適切な数取ることで、$|X-Y| \leq 1$ で相手に手番を渡すことができます。帰納法の仮定により相手は負け、自分は勝ちます。

E : Alice in linear land

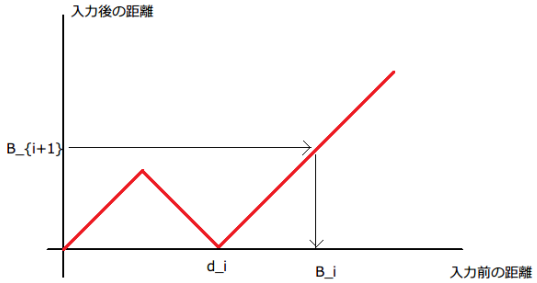
i 番目までの数を入力したときに目的地までの距離 A_i を前もって計算しておきます。

i 番目に入力する数を自由に変更できるとき、その数の入力が終わった時点で $\forall x \in [0, B_i]$ について Alice が目的地 x の距離にいるよう数を設定することができます。逆に、P_i より目的地に遠い場所に移動させることはできません。

よって、i 番目以降の数を入力しても Alice が目的地に辿りつけない最小の距離 B_i が求まれば、i 番目を変更する計画が可能かは $A_i \geq B_{i+1}$ かどうかと等しくなります。

B_i は実は後ろから順に求めていくことができます。まず、$B_{N+1}=1$ です。

- $B_{i+1} \leq \lfloor d_i/2 \rfloor$ のとき $B_i = B_{i+1}$
  $\because$ B_{i+1} の距離から d_i を入力しても $B_{i+1} \leq \lfloor d_i/2 \rfloor$ より動きません。よって、B_{i+1} から i 番目以降の数を入力しても目的地にたどり着きません。これより近い場合、同様に i 回目の数の入力では動きませんが、$i+1$ 番目の数を入力する時点で B_{i+1} より近い場所にいるので、目的地にたどり着きます。
- それ以外のとき $B_i = B_{i+1} + d_i$
  $\because$ 同様に、距離 B_i から i 番目以降の入力を行っても目的地にたどり着きません。距離 B_i より近い位置から i 番目の入力を行ったとき、B_{i+1} より目的地に近い位置に動くので、仮定から目的地にたどり着きます。(以下の図参照)



F : Dam

i 日目までに流入した水を、流入した日が後になるにつれ水温が昇順するよう管理することを考えます。こうすることで、水を入れるためにダムから排水するときは、今ある水のうち最も水温の低い (=最も早く流入した) ものを過去に排出したことにすることで対応できます。

水が流入したとき、その前で流入した水より温度が低いときは、上記の性質が壊れてしまいますが、代わりにその 2 種類の水が混ざって同時に流入してきたものとして扱うことができます。なぜなら、その前で流入した水を捨てるより、冷たい水が混ざってから捨てたほうが残った水の温度が高くなるからです。

以上の操作は、両端キューを使うことで水の排出は先頭からの削除、水の流入は末尾の削除と追加により毎回の操作を O(1) で行うことができます。i 日目には、水の排出と流入を終えた後、全体の熱量 (体積×温度) の総和を L で割った値が答えです。1 日の間に要素を複数削除することはありますが、追加は 1 日につき 1 回なのでならしで O(N) となります。

ABC 059 / ARC 072 Editorial

writer : hogloid

2017 年 4 月 22 日

A : Three-letter acronym

C++ Code

```
int main(){
  string a, b, c;
  cin >> a >> b >> c;
  char dif='A' - 'a';
  printf("%c%c%c\n", a[0] + dif, b[0] + dif, c[0] + dif);
        return 0;
}
```

B : Comparison

If the two numbers have different lengths, the longer number is greater. Otherwise, if the two numbers have the same lengths, you can compare them lexicographically.

C : Sequence

Try two possibilities independently: even-length prefixes are positive or odd-length prefixes are positive.

You determine the numbers greedily from left to right. When you check the i-th term, if the sum of the first i terms already has the desired sign, you don't need to change the i-th term. (Instead of changing it, you can do the same thing by performing the same operation for the $i + 1$-th term).

Otherwise, you should change it such that the sum of the first i terms becomes 1 (in case the desired sign is positive) or -1 (in case the desired sign is negative).

This way, you can compute the optimal cost.

D : Alice&Brown

If $|X-Y| \leq 1$, Brown wins. Otherwise Alice wins.

- $Incase |X-Y| \leq 1$, no matter how you move, after your turn $|X-Y| > 1$ will be held, and your opponent wins.
- In case $X+Y > 1, |X-Y| > 1$, if you take appropriate number of stones, you can satisfy $|X-Y| \leq 1$ after your turn and you win.

E : Alice in linear land

Suppose that the witch changes the q-th number.

Let A_i be the distance to the goal after you finish the first i operations (when the which doesn't do anything). Then, before the q-th operation, you will be at A_{q-1}. After the q-th operation, you can be at arbitrary integer place between 0 and A_{q-1}, inclusive (depending on the change made by the witch).

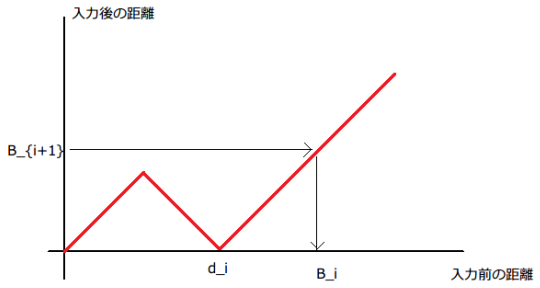
Let B_i be the smallest integer such that, if you are at B_i after the first i operations, you won't reach the goal. Clearly, the answer to the query is 'YES' if and only if $B_q \leq A_{q-1}$.

You can compute B_i in the decreasing order of i as follows:

- Obviously, $B_N = 1$.
- If $B_{i+1} \leq \lfloor d_i/2 \rfloor$, $B_i = B_{i+1}$

  $\because$ If you start from B_{i+1} and perform the operation d_i, you won't be moved by this operation and you won't reach the goal. If you start from closer positions, you will reach the goal by the definition of B_{i+1}.
- Otherwise, $B_i = B_{i+1} + d_i$

  $\because$ Check the following picture. Here, the horizontal axis shows the distance before the operation d_i, and the vertical axis shows the distance after the operation d_i.



F : Dam

The state of the dam can be represented by two parameters: [the amount of water] and [the product of the amount of the water and the temperature]. This way, when we mix two cups of water with parameters (x_1, y_1) and (x_2, y_2), we get water with the parameter $(x_1 + x_2, y_1 + y_2)$.

At each time, we should keep the set of possible states of the water in the dam. If you plot this set on an x-y plane, it will always be a concave polyline. For example, after the first day, this is a line segment that connects $(0, 0)$ and $(v_1, t_1 v_1)$.

How can we store/update this polyline? The polyline can be seen as a sequence of vectors. We keep it using a deque.

- When we add water with the state (x, y), we simply push it to the front of the deque.
- When we discharge water, replace the polyline with its convex hull.

And for each query, we answer the y-coordinate of the polyline at $x = V$. The amortized complexity is $O(N)$.

ABC058 / ARC071 解説

writer: nuip

2017 年 4 月 8 日

For International Readers: English editorial starts from page 7.

A : $\iota \perp l$

問題文にかかれているとおり、$b-a=c-b$ であるかどうか判定すればよいです。例えばこの処理は C++では次のように書けます。

```
#include <iostream>
using namespace std;
int main(){
  int a, b, c;
  cin >> a >> b >> c;
  cout << ( b-a == c-b ? "YES" : "NO" ) << endl;
}
```

B : ∴∵∴∵

E の文字と O の文字を交互に並べるとパスワードが復元できます。$|E|-|O|=1$ のケースに気をつけてください。

```
#include <iostream>
using namespace std;
int main(){
  string E, O;
  cin >> E >> O;
  for(int i=0; i < E.size(); i++){
    cout << E[i];
```

```
    if(i < O.size()) cout << O[i];
  }
  return 0;
}
```

C：怪文書 / Dubious Document

最長の文字列を作るにはできるだけ多く文字を使ったほうがいいです。答えとなる文字列に使える a の数を c_a とします。また、$c_b, \ldots, c_z$ についても同様に定義します。

すぬけ君が文字列を作る時に a を k 個使うためには、どの文字列にも a が k 個出てきている必要があります。よって、c_a は、「a が最も少ない文字列」に出てくる a の個数となります。同じことが他の b から z までの文字についても言えます。

辞書順で最小の文字列にするためには、a から順番に並べればよいので、答えは

$$\underbrace{\mathrm{a}\mathrm{a}\ldots\mathrm{a}}_{c_a\text{個}}\underbrace{\mathrm{b}\mathrm{b}\ldots\mathrm{b}}_{c_b\text{個}}\ldots\underbrace{\mathrm{z}\mathrm{z}\ldots\mathrm{z}}_{c_z\text{個}}$$

となります。

D：井井井 /

答えを数式で表すと以下のようになります。

$$\sum_{1\leq i<j\leq n}\sum_{1\leq k<l\leq m}(x_j-x_i)(y_l-y_k)$$

これをそのまま計算すると計算量が $O(n^2m^2)$ で間に合いません。

この式は次のように因数分解することができます。

$$\left(\sum_{1\leq i<j\leq n}(x_j-x_i)\right)\left(\sum_{1\leq k<l\leq m}(y_l-y_k)\right)$$

これで x 座標と y 座標を別々に扱うことができるようになりました。しかもどちらも同じ形をしているので、x 座標の式 $\sum_{1\leq i<j\leq n}(x_j-x_i)$ をどう計算するかだけ考えれば良いです。

この式もこのままでは計算に $O(n^2)$ かかってしまいます。式をよく見ると、n 個の変数 $x_1, \ldots, x_n$ を足したり引いたりしてるだけなので、$x_1, \ldots, x_n$ がそれぞれ何回足されて何回引かれているかさえ分かればこの値は求まります。x_k が足される回数は、ループの中で $j = k$ である回数と同じです。これは k より小さい添字の個数と同じなので $k-1$ 回だとわかります。同様に x_k が引かれる回数は、k より大きい添字の個数と同じなので $n-k$ 回だとわかります。以上のことから、次の等式が成り立ちます。

$$\sum_{1 \le i < j \le n} (x_j - x_i) = \sum_{1 \le k \le n} ((k-1)x_k - (n-k)x_k)$$

この式の右辺の形式だと計算量が $O(n)$ となるため、制限時間内に答えを出すことができます。よって、この式を使って x, y 両方について計算し、掛け算すれば答えが $O(n+m)$ で求まります。

E : TrBBnsformBBtion

操作 1 や 2 を $\mathrm{A} \xrightarrow{1} \mathrm{BB}$ や $\mathrm{AAAA} \xrightarrow{2} \mathrm{A}$ のように書くことにします。

まず、どちらの操作も（空文字列を作らない限りは）逆の操作が可能です。

$\mathrm{A} \xrightarrow{1} \mathrm{BB}$ の逆の操作は次のように作れます。

$$\mathrm{BB} \xrightarrow{1} \mathrm{AAB} \xrightarrow{1} \mathrm{AAAA} \xrightarrow{2} \mathrm{A}$$

A と B が逆の場合でも、この操作の A と B を入れ替えればよいです。

$$\mathrm{AA} \xrightarrow{1} \mathrm{BBA} \xrightarrow{1} \mathrm{BBBB} \xrightarrow{2} \mathrm{B}$$

操作 2 の逆の操作は、消した AAA や BBB の隣の文字を利用して行えます。たとえば隣の文字が A であれば、次のようにします。

$$\mathrm{A} \xrightarrow{1} \mathrm{BB} \xrightarrow{1} \mathrm{AAB} \xrightarrow{1} \mathrm{AAAA}$$
$$\mathrm{A} \xrightarrow{1} \mathrm{BB} \xrightarrow{1} \mathrm{AAB} \xrightarrow{1} \mathrm{ABBB}$$
$$\mathrm{A} \xrightarrow{1} \mathrm{BB} \xrightarrow{1} \mathrm{BAA} \xrightarrow{1} \mathrm{BBBA}$$

隣の文字が B の場合でも、この操作の A と B を入れ替えればよいです。

ここで、文字列を「文字列 T に変換できるもの」というグループに分類することを考えましょう。すると、操作が双方向に行えるので、「文字列 X を Y に変換できるか」と「文字列 X と文字列 Y は同じグループか」が同じ意味になります。

ではグループにはどのようなものが考えられるでしょうか？簡単にするために、A だけが使われるように変換することにします。また、できるだけ短くすることを目指します。すると、どの文字列も次の 3 つのうちどれかになることがわかります。

- A に変換できるもの
- AA に変換できるもの
- AAA に変換できるもの

逆に、これら 3 つは互いに変換可能ではありません。これを確かめるために、文字列について「各文字について A を 1、B を 2 として合計して 3 で割ったあまり」を考えます。例えば、文字列 A は 1、AA は 2、AAA は 0 です。この値は操作 1 でも操作 2 でも変わらないので、これら 3 つの文字列は互いに変換不可能です。

以上のことから、文字列 X と Y のこの値が等しいこと（同じグループであること）と、X を Y に変換できることは同値です。そのため、各クエリに対してその部分文字列の値を計算して 3 で割ったあまりが等しいかを判定すれば良いことになります。これは、事前に累積和を計算しておくことで、各クエリに対して $O(1)$ で判定できます。

F : Infinite Sequence

2 つめの条件がややこしいので、まずこれについて考えましょう。1 である項については 2 つめの条件は何も関係ありません。では 2 以上の項についてはどうなるでしょうか？数列のどこかに x, y というふうに 2 以上の数 x, y が連続して出てきた場合、x についての条件から x, y, y のように y がもう 1 つあとに続いているはずです。さらに、1 つめの y についての条件から x, y, y, y のようにさらにもう 1 つ y がついていることが分かります。同様の議論が 2 つめ以降の y についても可能なので、ずっと y が続くということが分かります。つまり、2 以上の数が連続すると、それ以降の項は 1 通りに定まってしまいます。

次に 1 つめの条件について考えてみましょう。第 n 項からはずっと同じ数が続いているということですが、この「同じ数の繰り返し」が始まったのはどこからでしょうか？その数が a_1 からずっと続いているかもしれない (A) ですし、その前に別の数である項がある (B) かもしれません。

今後、第 n 項の数を x で表して、第 n 項に下線をひいて表すことにします。

(A) $x, x, \ldots, x, \underline{x}, x, \ldots$

(B) $?, ?, \ldots, ?, t, x, x, \ldots, x, \underline{x}, x, \ldots$

(A) の場合は単純なので、(B) の場合を考えてみましょう。x の繰り返しの直前にある数を t と呼ぶことにします ($t \neq x$)。ではその t のさらに前にはどんな数が出てきているでしょうか？t が最初の項から続いている場合 (B1) とそうで無い場合 (B2) があります。

(B1) $t, t, \ldots, t, x, x, \ldots, x, \underline{x}, x, \ldots$

(B2) $?, ?, \ldots, ?, s, t, t, \ldots, t, x, x, \ldots, x, \underline{x}, x, \ldots$

t の前の数を s と呼ぶことにします。いま t がいくつか続いているように書きましたが、前に見たとおり、$t > 1$ だと t が連続しているはずはありません。$t > 1$ ということにして場合分けしておきましょう。

(B1-1) $1, 1, \ldots, 1, x, x, \ldots, x, \underline{x}, x, \ldots$

(B1-2) $t, x, x, \ldots, x, \underline{x}, x, \ldots$

(B2-1) $?, ?, \ldots, ?, s, 1, 1, \ldots, 1, x, x, \ldots, x, \underline{x}, x, \ldots$

(B2-2) $?, ?, \ldots, ?, s, t, x, x, \ldots, x, \underline{x}, x, \ldots$

$t \neq x$ であったので、(B1-1) と (B2-1) では $x > 1$ です。まぎらわしいのでこれらについては x' と書いておきます。また、(B2-2) の場合 s としては 1 しかありえません。

(B1-1) $1, 1, \ldots, 1, x', x', \ldots, x', \underline{x'}, x', \ldots$

(B1-2) $t, x, x, \ldots, x, \underline{x}, x, \ldots$

(B2-1) $?, ?, \ldots, ?, s, 1, 1, \ldots, 1, x', x', \ldots, x', \underline{x'}, x', \ldots$

(B2-2) $?, ?, \ldots, ?, 1, t, x, x, \ldots, x, \underline{x}, x, \ldots$

以上で、第 n 項から左に辿っていくと、(B1-1) と (B2-1) と (B2-2) では 1 が出てくることが分かりました。そのようなケースは 1 で終わる有限列に “$x', x', \ldots, x', \underline{x'}, x', \ldots$” か “$t, x, x, \ldots, x, \underline{x}, x, \ldots$” を付けた数列になっています。

以上でどのような数列が条件をみたすのかがわかったので、答えを求めていきましょう。

(A) のケースは n 通りです。(B1-2) のケースは $(n-1)^2$ 通りです[1]。それ以外のケースの答えを求めるには、「1 で終わる長さ i の列で最後の項以外が条件を満たすものの数」が必要になります。これを $dp[i]$ とします。この値は動的計画法で求められます（後述）。(B2-2) のように後ろに "$t, x, x, \ldots, x, \underline{x}, x, \ldots$" が付くケースは $(dp[1] + dp[2] + \ldots + dp[n-2])(n-1)^2$ 通りです。(B1-1) と (B2-1) のように後ろに "$x', x', \ldots, x', \underline{x'}, x', \ldots$" が付くケースについては、$x' > 1$ なので、$(dp[1] + dp[2] + \ldots + dp[n-1])(n-1)$ 通りです。よって、答えは

$$n+(n-1)^2+(dp[1]+dp[2]+\ldots+dp[n-2])(n-1)^2+(dp[1]+dp[2]+\ldots+dp[n-1])(n-1)$$

となります。

さて、あとは $dp[i]$ を求めるだけです。「最後の項以外が条件を満たす、1 で終わる長さ i の列」は、次のどれかです。

- (最後の項以外が条件を満たす、1 で終わる長さ $i-1$ の列) + "1"
- (最後の項以外が条件を満たす、1 で終わる長さ $i-3$ の列) + "$2, 1, 1$"
- (最後の項以外が条件を満たす、1 で終わる長さ $i-4$ の列) + "$3, 1, 1$"

  $\vdots$
- (最後の項以外が条件を満たす 1 で終わる長さ 1 の列) + "$i-2, 1, 1, \ldots, 1$"
- "$i-1, 1, 1, \ldots, 1$"

よって、

$$dp[i] = \begin{cases} 1 & (i = 1) \\ 1 + dp[1] + dp[2] + \ldots + dp[i-1] & (i > 1) \end{cases}$$

と計算できます。$dp[i]$ の累積和を計算しておくことで $dp[1], dp[2], \ldots, dp[n]$ を $O(n)$ で求めることができます。

この $dp[i]$ の計算を行列とベクトルの掛け算で実装すると $O(\log n)$ で解くこともできます。

[1] $t > 1$ に注意

ABC058 / ARC071 Editorial

writer: nuip

April 8th, 2017

A : $\iota \perp l$

Check if $b - a = c - b$.

```
#include <iostream>
using namespace std;
int main(){
  int a, b, c;
  cin >> a >> b >> c;
  cout << ( b-a == c-b ? "YES" : "NO" ) << endl;
}
```

B : ∴∴∴

Arrange characters in E and O alternately.

```
#include <iostream>
using namespace std;
int main(){
  string E, O;
  cin >> E >> O;
  for(int i=0; i < E.size(); i++){
    cout << E[i];
    if(i < O.size()) cout << O[i];
  }
  return 0;
}
```

C : Dubious Document

Let c_a be the maximum integer such that each string contains at least c_a occurrences of the character 'a'. Define $c_b, \ldots, c_z$ similarly.

The answer is

$$\underbrace{\mathrm{aa}\ldots\mathrm{a}}_{c_a \text{occurrences}}\ \underbrace{\mathrm{bb}\ldots\mathrm{b}}_{c_b \text{occurrences}}\ \ldots\ \underbrace{\mathrm{zz}\ldots\mathrm{z}}_{c_z \text{occurrences}}$$

D :

The answer is

$$\sum_{1\le i<j\le n}\sum_{1\le k<l\le m}(x_j-x_i)(y_l-y_k)$$

which is equal to

$$\left(\sum_{1\le i<j\le n}(x_j-x_i)\right)\left(\sum_{1\le k<l\le m}(y_l-y_k)\right)$$

Thus, we can solve the problem for each coordinate independently.

In order to solve the problem for x-coordinates quickly, use the following equation:

$$\sum_{1\le i<j\le n}(x_j-x_i)=\sum_{1\le k\le n}((k-1)x_k-(n-k)x_k)$$

It works in $O(n+m)$.

E : TrBBnsformBBtion

Replace 'A' with '1' and 'B' with '2'. We claim that we can convert a string s to another string t if and only if the sum of digits of s and the sum of digits of t are equivalent in modulo 3.

It is trivial that this is a necessary condition. We want to prove that this is sufficient.

First, the operations are revertable (except for the case where the initial string is empty):

$$\mathrm{BB} \xrightarrow{1} \mathrm{AAB} \xrightarrow{1} \mathrm{AAAA} \xrightarrow{2} \mathrm{A}$$

$$\mathrm{A} \xrightarrow{1} \mathrm{BB} \xrightarrow{1} \mathrm{AAB} \xrightarrow{1} \mathrm{AAAA}$$
$$\mathrm{A} \xrightarrow{1} \mathrm{BB} \xrightarrow{1} \mathrm{AAB} \xrightarrow{1} \mathrm{ABBB}$$
$$\mathrm{A} \xrightarrow{1} \mathrm{BB} \xrightarrow{1} \mathrm{BAA} \xrightarrow{1} \mathrm{BBBA}$$

Using the operations $B \to AA$, we can convert any (non-empty)string to a string that consists only of 'A', so any string can be converted into one of the strings 'A', 'AA', or 'AAA'. These three strings have different values in modulo 3. Thus, this condition is also sufficient.

F : Infinite Sequence

Let N be a fixed integer. Let $dp[k]$ be the number of infinite sequences $a_1, a_2, \ldots$ such that

- For each i, $1 \leq a_i \leq N$ (Note that this N is constant regardless f the value k)
- $a_k = a_{k+1} = \cdots$
- For each (i, j, k), if $i < j < k \leq i + a_i$, $a_j = a_k$.

The problem asks to compute $dp[N]$.
Let's compute $dp[k]$ by case-analysis:

- If the first element is 1, there are $dp[k-1]$ ways to decide the remaining elements.
- If the first element is not 1 (call it c), and the second element is also not 1 (call it d), the sequence must be $c, d, d, d, \ldots$. In total there are $(N-1)^2$ sequences of this form.
- If the first element is $c > 1$ and the second element is 1,
  - If $c + 1 \leq k$, there are $dp[k - c - 1]$ ways to decide the remaining elements.
  - Otherwise, the sequence must be $c, 1, 1, 1, \ldots$.

Thus, $dp[k] = dp[k-1] + (N-1)^2 + dp[k-3] + dp[k-4] + \cdots + dp[1] + (N - k + 2)$.

ARC070 / ABC056 解説

sigma425

For International Readers: English editorial starts on page 5.

A: HonestOrDishonest

$a =' H', b =' H'$ の場合は $'H'$ を、$a =' H', b =' D'$ の場合は $'D'$ を、$a =' D', b =' H'$ の場合は $'D'$ を、$a =' D', b =' D'$ の場合は $'H'$ を出力すればよいです。コードにすると次のようになります。

```
int main(){
        char a,b;
        cin >> a >> b;
        if(a=='H' && b=='H') cout << 'H';
        if(a=='H' && b=='D') cout << 'D';
        if(a=='D' && b=='H') cout << 'D';
        if(a=='D' && b=='D') cout << 'H';
        return 0;
}
```

B: NarrowRectanglesEasy

もし既に連結なら答えは 0 です。これは $|a-b| \leq W$ かどうかで判定できます。そうでない場合は、答えは $|a-b| - W$ になります。

C: Go Home

もし $1+2+...+t < X$ なら時刻 t には座標 X にはたどり着けません。逆に、$1+2+...+t \geq X$ なら時刻 t に座標 X にたどり着けることが示せます。実際、$\{1,2,..,t\}$ の部分集合で和が X になるものが取れて、そうすると部分集合に選んだ長さのときだけ右にジャンプし、それ以外のときはジャンプしないという戦略によって座標 X にたどり着けます。よってあとはこのような最小の t を求めれば良いです。これはおおよそ $\frac{-1+\sqrt{8X+1}}{2}$ になるので、この関数によって計算すると $O(1)$ で求まります。そこまでしなくても、答えが $O(\sqrt{X})$ になることから、t を前から順番に試して $\frac{t(t+1)}{2} \geq X$ か毎回判定するだけでも, 時間計算量 $O(\sqrt{X})$ で答えを求めることが出来ます。

D: No Need

まず、「カード i が必要」という条件は次のように言い換えられます: 「カード i を使わずに、総和が $K-a_i$ 以上 K 未満な集合が取れる」.

左 ⇒ 右 ・・・ 必要ならその証拠となる i を含むよい集合があって、これから i を除いたものが言い換え後の条件を満たす証拠になっています。

右 ⇒ 左 ・・・ 上とは逆に、この条件を満たす集合に i を加えたものが元の条件を満たす証拠となっています。

よって、各カード i ごとに、そのカードを除いた後つぎのような DP をします。

「$dp[x][y] = x$ 枚目まで使ったときに総和を y にできるか?」

y としては K 未満しか考えなくて良いため、この DP の計算量は $O(NK)$ になります。これを N 枚のカードに対してやるので、全体の計算量は $O(N^2K)$ となり、部分点を取ることが出来ます。

満点解法を取るにはいくつか方法があります。ひとつは、カード i が不必要な時、$a_i \geq a_j$ を満たすカード j も不必要である (つまり、単調性が成り立つ) ことを利用する方法で、どのカードまで不必要かを二分探索することで、上述の dp の回数を $O(logN)$ 回に減らせるので、全体で $O(NKlogN)$ になります。

他の方法として、dp を, カード 1,2,..,N の順に使う方と N,$N-1$,...,1 の順に使う方の両方を途中経過を残して計算しておいて、その結果を利用することで各 i について $O(K)$ で判定する方法があります。実際、カード i について判定するには、1,2,...,$i-1$ を使って出来る値の集合と、N,$N-1$,..,$i+1$ を使ってできる値の集合がわかっていれば良いです。これがわかっているので、あとは累積和を使ったりすると $O(K)$ で判定できて、全体の計算量は $O(NK)$ になります。

また、bitset 等を使うことで DP を高速化すると計算量は変わらなくても少し遅い解法でも通るかもしれません。

E: NarrowRectangles

まず部分点を解くには、次のような dp をすればよいです。

$dp[i][x] =$ 上から i 個までの長方形を動かしていて、i 個めの長方形の横座標を x に動かした時点でのそこまでのコストの最小値

上から順番に動かし方を決めていくと、そこまでが連結なら、そこから下を連結に出来るかどうかは直前の長方形の一にしか関係がないためこのような dp が出来ます。

計算すると.

$$dp[i][x] = |x - l_i| + \min_{x-(R_{i-1}-L_{i-1}) \leq x' \leq x+(R_i-L_i)} dp[i-1][x'] \tag{1}$$

となる (min についている条件は $i-1$ と i が連結になる条件です) ので、これを愚直に計算すると、座標幅を X とおくと、$O(NX^2)$ になり、部分点を取ることが出来ます。

満点を取るには、$dp[i][]$ がどのような形になっているか考える必要があります。$dp[i-1]$ から $dp[i]$ を計算するには、

1. x 軸方向に平行移動する
2. ある一定長さの区間の min をとる
3. $|x - l_i|$ を足す

が出来る必要があります。この操作をしていくとどうなるか考えると、$dp[i]$ では、「十分に左では傾きが $-i$ で、そこから単調に傾き (常に整数) が大きくなっていき、十分に右では傾きが i になる」という形になっていることがわかります。従って、$dp[i][x]$ をすべて持つ代わりに、「傾きが変わる点」の x 座標をすべて持つことにします。

重要な事実として、このような形に対して「ある一定長さの区間の min をとる」をすると、「傾き 0 の部分 (つまり底の部分) がその一定長さ延びる」という結果になることがわかります。なのでこれは底の部分の左右にわけて両者を平行移動するとみなせます。従って、傾き 0 の区間より左で傾きが変わる点, 右で傾きが変わる点をそれぞれ set で持って、平行移動は set 全体に対して足された値として管理すると操作 1 と 2 に関しては処理できます.

これを持てば操作 3 も簡単に処理できて、「ある点で傾きが 2 回変わることになる」という捉え方をすると、その点が傾き 0 の区間の中にあるか、左にあるか、右にあるかで場合分けして、set に点を追加して、必要に応じて片方の set の中身をひとつもう片方の set に移し替えることで実現できます。

最終的な答えは、傾き 0 の部分の高さを常に保持することで求めることが出来ます。計算量は $O(NlogN)$ です。

F: HonestOrUnkind

まず、$A \leq B$ のときは $Impossible$ です。これは、不親切な人が次の戦略を取ると何度質問をしても絶対に特定できないことからわかります:「不親切な人 B 人のうち A 人をあらかじめ選んでおいて、不親切な人はその A 人が正直者であるかのように質問に答える」

それ以外の場合は常に可能なことを構成的に示します。

まず頂点が 0 から $N-1$ の N 個あるグラフを持ち、質問の結果に応じて次のように辺を張っていきます。a が b のことを正直者だと言ったときには a から b に'Y' のラベルの付いた辺を張ります。($a \xrightarrow{Y}$b とかく) 同様に不親切な人だと言ったときには'N' のラベルの付いた辺を張ります。($a \xrightarrow{N}$b とかく)

まず N 回使って正直者を一人特定します。$a_B \xrightarrow{Y} a_{B-1} \xrightarrow{Y} ... \xrightarrow{Y} a_0$ という部分グラフがあるとします (a_i はそれぞれ異なるとする)。この時、a_0 は正直者です。($\because$ ある a_i が正直者だったとすると、推移的に a_0 も正直者になります。a_0 から a_B まで全員不親切な人だとすると、不親切な人が $B+1$ 人いることになり矛盾します。)

よって基本的には $\xrightarrow{Y}$ でつながった path を伸ばしていくことを目指します。

$a \xrightarrow{N} b$ となった場合は、この二人を一旦 N 人から取り除きます。a と b の少なくとも一方は不親切な人であることがわかるので、こうしたとしても $A > B$ という条件は成り立ったままです。

このことから次のようなアルゴリズムが考えられます。

```
bool ask(int a,int b){
        cout<<"? "<<a<<" "<<b<<endl;
        char c;
        cin>>c;
        return c=='Y';
}

int need = B+1;
vector<int> path;
for(int i=0; i<N; i++){
        if(path.empty()){
                path.push_back(i);
                if(path.size()>=need) break;
                continue;
        }
        bool b = ask(i,path.back());
        if(b){
```

```
                path.push_back(i);
                if(path.size()>=need) break;
        }
        else{
                path.pop_back();
                need--;
        }
}
int honest = path.front();
```

このアルゴリズムは N 回の質問で正直者を一人特定できます。あとはその人に他の人が正直者かを聞くことで全体で $2N$ 回で全ての正直者を特定できます。

ARC070 / ABC056 Editorial

sigma425

A: HonestOrDishonest

```
int main(){
        char a,b;
        cin >> a >> b;
        if(a=='H' && b=='H') cout << 'H';
        if(a=='H' && b=='D') cout << 'D';
        if(a=='D' && b=='H') cout << 'D';
        if(a=='D' && b=='D') cout << 'H';
        return 0;
}
```

B: NarrowRectanglesEasy

If $|a-b| \leq W$, the answer is 0. Otherwise the answer is $|a-b|-W$.

C: Go Home

The answer is the minimum t such that $1+2+...+t \geq X$.

D: No need

The card a_i is unnecessary if there is a subset of the other $N-1$ cards whose sum is in the interval $[K-a_i, K)$. Thus, the straightforward DP is $O(N^3)$.

There are two ways to improve it:

- $O(N^2 logN)$: It turns out that if $a_p < a_q$ and a_q is unnecessary, a_p is also unnecessary. Do binary search using this fact.
- $O(N^2)$: Do DP twice: for prefixes and for suffixes.

Also, it can be 64 times faster if you use bitset.

E: NarrowRectangles

Define $dp[i][x]$: the minimum cost to move the first i rectangles such that the last (the i-th) rectangle's leftmost coordinate is x. This will lead to a solution for partial score.

Now, see $dp[i]$ as a function: it returns $dp[i][x]$ for given x. It turns out that this function is a polyline consisting of $2i+3$ sections, and the slopes of the sections are $-i-1, -i, \ldots, i, i+1$ from left to right. Thus, this polyline can be represented using $2i+3$ integers $l_0, l_1, \ldots, l_i$, $r_0, r_1, \ldots, r_i$, and c.

- In the interval $(-\infty, l_i]$, the slope of the polyline is $-i-1$.
- In the interval $[l_i, l_{i-1}]$, the slope of the polyline is $-i$.
- $\cdots$
- In the interval $[l_1, l_0]$, the slope of the polyline is -1.
- In the interval $[l_0, r_0]$, the polyline is constant, and the value is c.
- In the interval $[r_0, r_1]$, the slope of the polyline is 1.
- $\cdots$
- In the interval $[r_{i-1}, r_i]$, the slope of the polyline is i.
- In the interval $[r_i, \infty)$, the slope of the polyline is $i+1$.

Now we compute the polylines in the order $dp[0], dp[1], \ldots, dp[N]$. We should keep two sets (or priority queues) representing $\{l_0, \ldots, l_i\}$ and $\{r_0, \ldots, r_i\}$ meanwhile, and the solution works in $O(NlogN)$.

F: NarrowRectangles

When $A \leq B$, the solution is "impossible" because A of unkind people can behave like honest people (call each other "honest" and call everyone else "unkind"). Otherwise, we can determine honest people in the following way:

First, notice that when a person p says "q is unkind", p and q can't be honest at the same time. Thus, even if we ignore both p and q, more than half of the remaining people is honest. Whenever we get the response "unkind", we ignore the two people.

Create an empty stack and try to push N people to this stack one by one. When we push a person p in to the stack, we do the following. Let q be the person that is currently at the top of the stack. Ask q about person p, and if we get the answer "unkind", we pop q from the stack and ignore p and q. Otherwise, push p into the stack.

After we try to push N people, what happens in the stack? Let $a_0, \ldots, a_{k-1}$ be the people in the stack from bottom to top. From the construction of the stack, for each i, a_i says "a_{i+1} is honest". Also, at least one of $a_0, \ldots, a_{k-1}$ is honest. Thus, we are sure that a_{k-1} is honest.

Now we can ask additional N queries to this honest person and get the answer.

ABC055 / ARC069 解説

writer : camypaper

2017 年 2 月 18 日

A : Restaurant

「何回 200 円もらえるか？」というのが本質的な問題です．$n/15$ とすることで C++ などのプログラミング言語では n を 15 で割った値の小数点以下切り捨てを求めることが可能です．$200 * n/15$ としてしまうと、正しい値を求めることができないことにも注意が必要です．

```
#include <iostream>
using namespace std;
int main(){
    int n;
    cin >> n;
    cout<<n*800-(n/15)*200<<endl;
}
```

B : Training Camp

「$N!$ を $10^9 + 7$ で割ったあまりを求めよ」という問題です．$N!$ そのものを求めようとすると、この値は非常に値が大きくなるため 64 ビット符号付き整数ではオーバーフローによって正しく値を求められなかったり，多倍長整数の乗算計算は時間がかかるため実行時間制限内に計算を終えることが難しくなります．

求める必要があるのは「$10^9 + 7$ で割ったあまり」であることに着目すると，現在のパワーを x として $x = i \times x \bmod 10^9 + 7$ として更新していけば、64 ビット符号付整数で収まる範囲で計算をすることが可能です．32 ビット符号付整数で扱っている場合 $x \times i$ が 10^9 より大きくなってしまいオーバーフローしてしまうことに注意してください．

C : Scc Puzzle

「S 字型のピースと c 字型のピースを組み合わせて Scc という組を可能な限り多く作りなさい」という問題です．ただし，c 字型のピースを 2 つ組み合わせて，S 字型のピースを 1 つ作ることが可能です．このままでは複雑なので，可能な操作を以下の 2 種類だと考えます．

1. S 字型のピース 1 つと c 字型のピース 2 つを組み合わせて Scc の組を 1 つ作る
2. c 字型のピース 4 つを組み合わせて Scc の組を 1 つ作る

なぜ，このように言い換えていいかを考えてみましょう．S 字型のピースが残っているにも関わらず，c 字型のピースを 2 つを組み合わせて S 字型のピースを作る必要がないのは直感的にも明らかでしょう．すると，S 字のピースがないときのみ c 字型のピース 2 つを使って S 字型のピースを作る，という操作が行われます．その直後の操作はやはり，S 字型のピース 1 つと c 字型のピース 2 つを組み合わせて Scc の組を作る，という操作になります．この 2 つの操作を 1 つにまとめると，「c 字型のピース 4 つを組み合わせて Scc の組を 1 つ作る」操作だとみなすことができます．

上記のような 2 種類の操作であることがわかると，可能な限り 1 番の操作を行ったのち，可能な限り 2 番の操作を行うというのが最適なことが分かります．このようにして作ることができる Scc の組の数は簡単な四則演算により $O(1)$ で答えを求めることが可能です．

D : Menagerie

「円環状に並んでいる N 匹の動物たちに両隣の動物が同じ種類かどうか訪ねた結果と矛盾しないような動物の割当てが存在するか？」という問題です．動物は羊と狼の 2 種類だけですが，羊は本当のことを言うのに対して狼は嘘をつくのが問題を複雑にしています．

この問題の重要な性質は「ある連続した 2 匹の種類が分かれば，その隣にいる動物の種類も分かる」ということです．例えば $i-1$ 番と i 番の動物の種類がそれぞれ分かっているとすると，i 番の言っていることが本当か嘘かがまず分かります．次に $i-1$ 番の種類が分かっているので，$i+1$ 番の種類がどちらか分かる，ということです．さらに，i 番と $i+1$ 番の種類が分かっているので $i+2$ 番の種類が分かり，$i+1$ 番と $i+2$ 番の種類が分かっているので $i+3$ 番の種類が分かり，というように連鎖的に動物の種類が分かります．

上の性質から 1 番と 2 番の動物の種類を仮定すると $3, 4, 5, \ldots, N$ 番の動物の種類が連鎖的に定まります．最後にこの仮定が正しかったかどうかを s と矛盾しないかどうかで調べればよいです．試す候補は高々 4 通りなので $O(N)$ でこの問題を解くことが可能です．

E : Frequency

「N 個の石の山からあるルールで石の山を取り除いて長さ $\sum a_i$ の数列 s を辞書順最小になるように構成するとき，s に $1, 2, 3, \ldots, N$ がそれぞれ何回含まれるか？」という問題です．石を 1 個取り除く山を選ぶときのルールは「(a_i, i) が最大であるような i を選ぶ」としてよいです (以下，これを単にルールと呼びます).

ルールに従ったとき s が辞書順最小の数列となることを示します．x を石の数が最大である山のうち最も左側にある石の山の番号とします.

$i < x$ かつ $0 < a_i < a_x$ であるような i が存在しない場合を考えます．x 番以外の山の石が取り除かれたとき，次の操作において s の末尾に追加される数は明らかに x です．x 番の石の山から石が取り除かれたとき，その他の石の山は全て $a_x - 1$ 個以下の石からなります．さらに x 番より左側には石の数が 0 個の山しか存在しないので，石を 1 個取り除いた後も石の数最大であって最も左側にある石の山は x 番です．よって，このときも次の操作において s の末尾に追加される数は x です ($a_x = 0$ となった場合は処理が終了するため，考慮する必要はありません)．結果として，x のみからなる数列が得られます．x より小さな数が s に追加されることはないため，明らかに辞書順最小です.

$i < x$ かつ $0 < a_i < a_x$ であるような i が存在している場合を考えます．y を $i < x$ において (a_i, i) が最大であるような i とします．このとき，辞書順最小の数列を得るためには x より大きな値が s に追加されてはならないようにいくつかの石を取り除いて y が可能な限り早く s に現れるようにする必要があります．これを，石の数を $\max(0, a_i - a_y)$ としたような石の山たちに対して辞書順最小の数列を作っていくと考えると，x 番より左側には 0 個の山しかないという状況に帰着させることが可能です．こうして石を取り除いたあと，y 番の山が石の数最大であって最も左側にある石の山となります．再帰的にこのような処理を行っていけばいつか全ての石がなくなります.

以上より，(a_i, i) が最大であるような山から石を取り除く，という操作を繰り返していけば s を辞書順最小の数列にすることが可能なことが示されました.

ルールを愚直にシミュレーションすると実行時間制限に間に合わないので高速化する必要があります．石の数が同じ石の山をまとめる，石をまとめて取り除く，という 2 つの高速化を行った以下のアルゴリズムにより $O(N \log N)$ で解くことが可能です.

- 石の山と石の番号 (a_i, i) で降順にソートしたものを (x_i, y_i) とする
- (ソートしたあとの並びで) 1 番目から順に処理を行う
- 今 i 番目の石の山に着目しているとして，$i(x_i - x_{i+1})$ 個の石を取り除き，s に $\min\{y_1, y_2, \ldots, y_i\}$ を $i(x_i - x_{i+1})$ 個追加する (便宜上 $x_{N+1} = 0$ とする)

これは i 番目の山を見ている時点で $1, 2, 3 \ldots, i$ 番の山に含まれる石の数が等しく，i 個の石を取り除く，という操作を $x_i - x_{i+1}$ 回行っていると考えると分かりやすいかもしれません.

F: Flags

「i 番の旗を x_i か y_i のどちらかに設置する，という操作を N 回行ったときの旗同士の距離の最小値を最大化せよ」という問題です．求める答えが d だとして，旗同士の距離が $d-1, d-2, \ldots, 0$ となるように設置することが可能であったとみなしても問題ありません．そこで二分法を用いて d を求めていくことを考えます g．d を固定したとき，この問題は以下のように表せます．

> N 本の旗があり，i 番の旗は x_i か y_i のどちらかに設置しなくてはならない．どの 2 つの旗も距離が d 以上になるように設置することは可能か判定せよ．

2 つの旗の距離が d 未満にならないかどうかにのみ着目すればよくなり，見通しがよくなりました．旗同士の距離は 2 つの旗の位置関係のみに依存しており，その他の旗は関係ないことは明らかです．すると「i 番の旗を x_i に置いたとき，j 番の旗は y_j に置いてはならない」というような制約が最大 $O(N^2)$ 個与えられるので，条件を満たすように旗を設置することが可能か？という問題になります．これを 2-SAT に帰着させて解いていきます．

まず「i 番の旗を x_i と y_i のどちらに置くか」という N 個の変数に関する 2-SAT ではなく，z を x と y を連結して昇順に並び替えた数列として「i 番の旗を座標 z_i に設置するかどうか」という $2N$ 個の変数に関する 2-SAT として考えます．このようにすると z は昇順の数列となっているため「距離 d 未満に別の旗があってはならない」という制約が扱いやすくなります．2-SAT として考えやすいよう，問題を以下のように言い換えます．

> 真偽値をとる論理変数 $v_1, v_2, \ldots, v_{2N}$ と以下に示されるようないくつかの論理式が与えられたとき，全ての論理式が真となるような真偽値の割り当てが存在するか判定せよ．
>
> - $\neg(v_i \wedge v_j), \neg(\neg v_i \wedge \neg v_j)$ (ただし i, j は旗としての対応関係がある)
> - $\neg(v_i \wedge v_l), \neg(v_i \wedge v_{l+1}), \ldots, \neg(v_i \wedge v_{i-1})$ (l は $|z_i - z_l| < d$ となる最小の l)
> - $\neg(v_i \wedge v_{i+1}), \neg(v_i \wedge v_{i+2}), \ldots, \neg(v_i \wedge v_r)$ (r は $|z_i - z_r| < d$ となる最大の r)

このままでは最大で $O(N^2)$ 個程度の節からなる 2-SAT となってしまいます．ここで，v_i が真のとき，偽となる必要がある論理変数たちが区間をなすことに着目します．論理変数をさらに 2 倍程度の個数になるよう増やし，旗であった論理変数と葉が対応するように論理変数をセグメント木状に配置し，以下のルールで節を作ることで変数の数を $O(N)$，節の数を $O(N \log N)$ に抑えることが可能です．

1. セグメント木の葉であるような i と旗の対応関係がある j について $\neg(v_i \wedge v_j), \neg(\neg v_i \wedge \neg v_j)$
2. セグメント木上で親の頂点の番号を p として $(v_i \to v_p)$
3. $[l, i), [i+1, r+1)$ をセグメント木上で $O(\log N)$ 個の頂点 $u_1, u_2, \ldots, u_k$ に分割したとして $\neg(v_i \wedge u_1), \neg(v_i \wedge u_2), \ldots, \neg(v_i \wedge u_k)$

図 1 にルール 2, 3 の具体例として v_{26} に着目した場面を示します．v_{26} に真を割り当てるとき，ルール 2 から先祖である赤色の頂点たちもまた真が割り当てられる必要があります．この「ある頂点に真が割り当てられたならば，その先祖たちは全て真」という性質を「ある頂点に偽が割り当てられたならば，その子孫たち全てもまた偽が割り当てられている」という性質だと考えると，ルール 3 のように図中の濃い青色の頂点 $v_{19}, v_5, v_{12}, v_{27}, v_{28}$ に対して節を追加することで「v_{26} に真が割り当てられたとき，$v_{19}, v_{20}, \ldots, v_{25}, v_{27}, v_{28}$ は偽が割り当てられる必要がある」という条件を $O(\log N)$ 個の節で表すことが可能となることが分かります．

変数の数 $O(N)$，節の数 $O(N \log N)$ の 2-SAT は 1 回あたり $O(N \log N)$ で調べることが可能なので，二分法と合わせて $O(N \log N \log \max z_i)$ で解くことが可能となります．

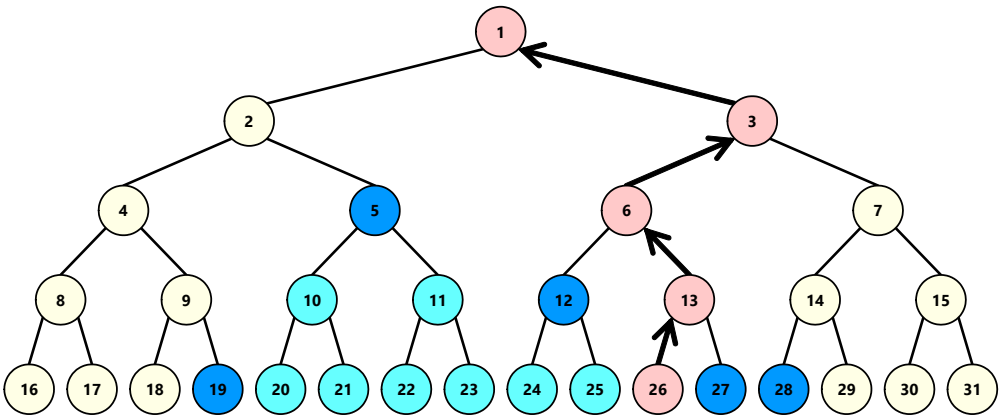


図 1　v_{26} に着目したときの節の追加ルール

ABC053 /ARC068 解説

writer : camypaper

2017 年 1 月 28 日

A : ABC /ARC

整数 x が標準入力から与えられるので x が 1200 未満かどうか判定せよ，という問題です．C++ などのプログラミング言語では以下のように if 文と呼ばれる構文を用いることで条件分岐を行うことが可能です．

```
#include <iostream>
using namespace std;
int main(){
    int x;
    cin >> x;
    if(x < 1200) cout << "ABC" << endl;
    else cout << "ARC" << endl;
}
```

B : A to Z String

文字列 s の部分文字列のうち先頭が A ，末尾が Z であるようなものの最大の長さを求めよ，という問題です．

簡単な考察により， A は可能な限り先頭に近いものを用いるのが最適であり，Z は可能な限り末尾に近いものを選ぶのが最適なことが分かります．それぞれの位置は，$O(|s|)$ で調べることが可能です．

C : X: Yet Another Dice Game

6 面サイコロに対して 90° 回転をしたのち，上を向いている面に書かれた整数を y として y 点得る，という操作を何度も繰り返して合計 x 点以上得るために必要な最小の操作回数を求めよ，という問題です．

はじめ，$1, 6$ 以外の面が上を向くように置き，6 の面と 5 の面が交互に上を向くように転がすのが最適な操作手順です．このように操作を行ったときに必要な操作回数を求めればよく，これは簡単な四則演算で $O(1)$ で求められます．

D : Card Eater

N 枚のカードの山に対して何回か操作を行ってカードに書かれた値が全て異なるようにするとき，最大で何枚のカードを残せるか，という問題です．基本的には 1 枚しかないカードを取り除かれないカードに，余っているカードを取り除かれるカードに選ぶことができるので操作は「2 枚のカードを選んで取り除く」とみなしてほぼ構いません．

カードの山に k 種類のカードがあったとして，k が奇数なら余っているカードは偶数枚あるので答えは k であり，偶数ならばどこかで必ず 1 枚しかないカードを 1 回取り除く必要があるので答えは $k-1$ となります．

E : Snuke-Train

数直線上を原点から一定間隔 d で移動したとき何種類の区間を訪れることが可能か $d = 1, 2, 3, \ldots, M$ について調べよ，という問題です．

間隔 d を固定して考えてみましょう．このとき $r_i - l_i + 1 > d$ を満たす区間は必ず 1 回以上訪れなくてはなりません．また， $r_i - l_i + 1 \leq d$ を満たす区間は最大でも 1 回しか訪れることはできません．

必ず 1 回以上訪れる区間は単に区間の長さによってのみ定まることが分かったので簡単に計算することができます．残っているのは最大でも 1 回しか訪れない区間だけなので，ある位置を含む区間の種類数を累積和などにより求めたのち， d の倍数を全て調べることで $O(N+M)$ で調べることができます．

さて d を固定した場合については解けましたが，$1, 2, 3, \ldots, M$ の M 種類について調べる必要があります．M 以下の数について，1 の倍数,2 の倍数,...,M の倍数と全て調べることは $O(M \log M)$ で行うことが可能ですが，毎回全ての区間について $O(N)$ で調べると $O(NM)$ となり実行時間制限に間に合いません．d を 1 から増加させていくと，必ず 1 回以上訪れる区間の数は減少し，最大でも 1 回しか訪れない区間の数は増加していきます．そのため，N 個の区間を予めその長さごとに分類しておけば各区間について区間に加算する，という操作は 1 回ずつで済み

ます．区間への加算も Fenwick Tree 等を用いることで 1 回あたり $O(\log M)$ で行うことが可能です．解法をまとめると，

- 区間を長さごとに分類する
- d を 1 から M まで順に増加させながら調べる
- i 番目の区間が初めて $r_i - l_i + 1 \leq d$ となったとき，$[l_i, r_i]$ に対して Fenwick Tree 等を用いて 1 加算する
- d の倍数を全て調べる

となり，これらを合わせると $O((M \log M + N) \log M)$ で解くことが可能です．

F: Solitaire

結論から言えば，この問題は最終的に以下のように言い換えられます．

> $(1, 2, \ldots, N)$ の並び替えであるような数列 A のうち，以下の条件を満たすものは何通りあるか？
>
> - $A_k = 1$
> - $k \leq i < j$ なる i, j について $A_i > \max(A_j)$ あるいは $A_i < \min(A_j)$ を満たす．
> - $i < k$ において $A_i > \max(A_j)(i \leq j)$ あるいは $A_i < \min(B_j)(j < i)$ を満たす．

$N - 1$ 個のうち $k - 1$ 個を取って先頭側に，残りを末尾側に条件を満たすように並べることにします．また，末尾側に置いた要素の最大値を m とします．

末尾側の並べ方は 2^{N-k-1} 通りです．先頭側の並べ方について考えます．まず，$A_i > \max(A_j)(i \leq j)$ を満たす要素は赤く塗られている，そうでない要素は青く塗られていると呼ぶことにします．先頭側に置いた要素のうち m 未満のものは全て青く塗る必要があります．m より大きいものは赤く塗ることも青く塗ることも可能ですが，ひとまず赤く塗ったことにしておきます．こうすると，赤く塗られた要素の最小値は青く塗られた要素の最大値よりも大きい，という性質を持つため状態としてまとめやすくなります．

$dp(r, b)$ を「赤く塗られた要素が r 個，青く塗られた要素が b 個あるときの要素の並べ方」とします．この求め方はあとで考えることにすると，このときの並べ方の総数は $\binom{m-2}{N-k-1} dp(N - m, k - 1 - (N - m)) 2^{N-k-1}$ です．なお，$k = N$ の場合は単に $dp(N - 1, 0)$ となることに注意してください．

さて， $dp(r, b)$ は以下の漸化式で定められます．

$$dp(r,b) = \begin{cases} 1 & (r = 0, b = 0) \\ 0 & (r < 0 \text{ または } b < 0) \\ dp(r, b-1) + \sum_{i=0}^{r-1} dp(i, b + r - (i+1)) & (otherwise) \end{cases}$$

この DP では $O(N^3)$ となってしまい間に合いません．しかし，この漸化式の $\sum_{i=0}^{r-1} dp(i, b+r-(i+1))$ という部分の遷移は $r-(i+1)$ 個赤い要素を青く塗り直し，赤い要素を 1 つ取り除く，という形になっています．これを「赤い要素を 1 つ青く塗った次の操作は，赤い要素を 1 つ青く塗るか，赤い要素を 1 つ取り除くかのどちらかでなくてはならない」と考えると，遷移を $O(1)$ にまとめることが可能です．

このようにすると $dp(i,j)$ の前計算に $O(N^2)$ かかり，取得は $O(1)$ となります．まとめて $O(N^2)$ となり正解することができました．

最後に上で示した条件を満たす数列であることと，$2N$ 回の操作で構成可能であることが同値であることを示します．

操作を逆から見ていくことにします．$k \leq i$ を満たす範囲においては A_i は現在デックに含まれるどの数よりも大きい，あるいは小さい，という条件を満たすためデックの中身は 1 が最小値であるような単調増加列となるように並べることが可能です．$i < k$ を満たす範囲においては A_i は現在デックに含まれるどの数よりも大きい，あるいは今後デックに含まれるどの値よりも小さい，という条件を満たします．青く塗られた要素は先頭に，赤く塗られた要素は末尾に挿入することにします．するとデックの中身は途中まで単調減少したのち，1 のある位置を境界としてその後単調増加するような数列になります．こうしてできた数列について 1 を仕切りとして 2 つの単純減少列をマージして新しい単調減少列を作る，と考えると $1, 2, 3, \ldots, N$ という数列を構成することが可能なことが分かります．

次にこの条件を満たさない数列は構成不可能なことを示します．$k \leq i$ を満たす範囲において，A_i が現在デックに含まれるどの数よりも大きい，あるいは小さい，という条件を満たさない箇所があるとすると 1 のある位置を仕切りとした 2 つの単調減少列となるように出来ないため構成することが不可能です．同様に $i < k$ を満たす範囲において，A_i が現在デックに含まれるどの数よりも大きい，あるいは今後デックに含まれるどの値よりも小さい，という条件を満たさない箇所があるとしても，やはり先程と同様に構成することが不可能です．

以上より上で述べた条件を満たす数列であることと，$2N$ 回の操作により構成可能であることが同値であることが示されました．

ABC 052 / ARC 067 解説

writer : maroonrk

2017 年 1 月 15 日

A : Two Rectangles

二つの長方形の面積の max を出力すればよいです。

C++ のコード例

```
int a,b,c,d;
cin>> a >> b >> c >> d;
cout<< max(a*b, c*d) <<endl;
```

B : Increment Decrement

整数 $ans = 0$ を用意して、x の値を変更する度に ans と x の max を ans に代入すると答えが求まります。

C : Factors of Factorial

ある整数 x が、素因数分解によって $x = p^n \times q^m \times ...$ $(p, q, ...$は素数$)$ と表せる時、x の約数の個数は $(n+1) \times (m+1) \times ...$ となります。よって、$N!$ の素因数全てについて、その素因数で $N!$ が何回割り切れるかを求めればよいです。そして、$N!$ のがどの素因数で何回割れるかは、1 から N までの数がそれぞれどの素因数で何回割れるかを求めて足し合わせると求まります。今回の問題は N が小さいので、これを愚直に実装すれば解くことができます。

D : Walk and Teleport

実はこの問題では、「一つ東の町まで歩く」と「一つ東の町までテレポートする」の二つの行動だけで最適解を達成できます。よって、全ての $i(i \leq N-1)$ について、町 i から町 $i+1$ に移動する際の疲労度の合計の最小値を求め、これを全て足し合わせると答えが得られます。町 i から町 $i+1$ に移動する際の疲労度の合計の最小値は、$min((X_{i+1} - X_i) \times A, B)$ で求めることができるので、これでこの問題は解けました。

E : Grouping

$DP[i][j] = i$ 人以下のグループのみで、j 人使っている場合の数

という DP を考えます。この DP の遷移は、以下のようになります。

$DP[i][j] = \sum_{k \in (0,C,C+1,...D)} DP[i-1][j-i \times k] \times_{(n-j+i \times k)} P_{(i \times k)} / (i!)^k / k!$

これでは一見 $O(N^3)$ に見えますが、k の動く範囲は j/i 以下しか考えなくて良いので、そこだけ計算するようにすると $O(N^2 logN)$ になってこの問題が解けます。

F : Yakiniku Restaurants

最適な行動においては、焼肉店間の移動は、番号の小さい方から大きい方への移動のみと考えて良いです。そこで、

$F(i,j) =$ 焼肉店 i から出発し、焼肉店 j で終了するように行動した時の、食べる焼き肉の美味しさの合計の最大値

と置きます。この $F(i,j)$ の値が、全ての i,j の組について求められれば、幸福度を減らす要因である移動距離は簡単に計算できるため、最終的な答えもすぐにわかります。では、$F(i,j)$ はどのように求めればよいでしょうか。とりあえず、M 種類のチケットごとに考えてみます。

$G(x,i,j) =$ 焼肉店 i から焼肉店 j までの焼肉店で、チケット x を引き換えに食べられる焼き肉の美味しさの最大値

と置いてみます。全てのチケットについて、$G(x,i,j)$ を愚直に求めていては間に合いません。そこで、x を固定した時の、$G(x,i,j)$ の値について考えてみましょう。まず、$G(x,1,N) = G(x,opt,opt)$ となる焼肉店 opt が存在します。（この opt は、RMQ を使うと高速に求めることができます）そして、$i \leq opt \leq j$ なる全ての i,j について、$G(x,i,j) = G(x,opt,opt)$ となります。これが何を表しているかというと、格子点 (i,j) の値が $G(x,i,j)$ になる格子を考えた時に、$i \leq opt \leq j$ に対応する長方形領域内の値が全て同じであるということです。残った、$i,j < opt$ のパターンと、$opt < i,j$ のパターンも、再帰的に同じことを繰り返すと、格子点 (i,j) の値が $G(x,i,j)$ になる格子は、N 個の長方形領域に数字を割り振るという操作で作られるということになります。ここで、$F(i,j)$ について考えると、これは、全ての x について、$G(x,i,j)$ を足して作られます。このことから、格子点 (i,j) の値が $F(i,j)$ になる格子は、M 種類全てのチケットごとに、N 個の長方形領域に値を加算する、という操作で作られることになります。これは、imos 法を使うことで十分高速に処理できます。よって、$O(NlogNM + N^2)$ でこの問題は解けました。

ABC 050 / ARC 066 解説

writer : maroonrk

2016 年 12 月 17 日

A : Addition and Subtraction Easy

整数、文字列、整数を空白区切りで読み込み、読んだ文字列が “+” か “−” で条件分岐をします。

C++ のコード例

```
int a,b;
string op;
cin>> a >> op >> b;
if(op == "+")
        cout << a + b << endl;
else
        cout << a - b << endl;
```

B : Contest with Drinks Easy

それぞれの問題にかかる時間を配列に保存しておきます。ドリンクごとに、配列を一時的に変更し、その総和を求めて出力すればよいです。なお、予めかかる時間の総和を求めておいて、ドリンクが与えられた時に差分だけ計算する方法だと、より高速に答えが求まります。

C : Lining Up

N が偶数である場合を考えます。N 人が並んだ場合、「自分の左に並んでいた人数と自分の右に並んでいた人数の差の絶対値」は、$N-1, N-3, N-5, ...3, 1, 1, 3, ...N-5, N-3, N-1$ となるはずです。この配列を B_i とします。

するとこの問題は、A_i を並び替えて B_i にする方法は何通りか、という問題になります。まず、A_i と B_i で、ある値の登場する回数が等しくなかった場合、明らかに答えは 0 です。

では、等しい場合はどうなるでしょうか。この配列には、N/2 種類の値が、それぞれ 2 回ずつ登場しているはずです。それぞれの値について、並び替える方法は、「A_i で前にあったものが B_i で前にある」と、「A_i で前にあったものが B_i で後ろにある」の 2 通り考えられます。

$N/2$ 種類の値で独立に並び替えがあるので、求める答えは $2^{(N/2)}$ になります。N が奇数である場合も同様

に解けますが、その場合 Ai や Bi の中に 0 という値が 1 度だけ登場することに注意してください。

D : Xor Sum

説明のため、and をビットごとの論理積、$<<$ を左シフト演算、$>>$ を右シフト演算とします。

方針としては、a と b のビットを上から決めていく桁 DP を考えます。$u = a\ xor\ b$ としている限り、$0 \leq u$ は常に成り立ちます。また、$v = a + b = ((a\ and\ b) << 1) + (a\ xor\ b) = ((a\ and\ b) << 1) + u \geq u$ なので、$v \leq N$ という条件のみに注意すればよいです。そこで、次のような DP を考えます。

$dp[i][j] = a$ と b の i ビット目以上が決定していて、その段階で $v = j$ となる通り数。

この DP では、以下の 3 パターンの遷移が考えられます。

・a の i ビット目と b の i ビット目が共に 1 の場合

・a の i ビット目と b の i ビット目のどちらか片方のみが 1 の場合

・a の i ビット目と b の i ビット目が共に 0 の場合

しかし、このままではこの DP は $O(NlogN)$ となってとても間に合いません。そこで、この DP を少し修正して、以下のようにしてみます。

$dp[i][j] = a$ と b の i ビット目以上が決定していて、その段階で $(v >> i) = (N >> i) - j$ となる通り数。

これは一見、先程より複雑になっただけで、計算量が改善していないように見えます。しかし、$j \geq 2$ のパターンは、その後どのように a, b のビットを選んでも、v が N を超えることが起きません。なので、それらのパターンを全て $j = 2$ として扱ってよくなります。よって j の範囲は 0〜2 だけを考えればよくなり、この DP が $O(logN)$ で行えるようになりました。

E : Addition and Subtraction Hard

簡単のため、最初の数の前に “+” という記号があるものとして扱います。まず、“+” のあとで開き括弧を挿入しても意味がないことがわかります。また、“−” のあとに開き括弧を必ず挿入しても問題ありません。なぜなら、そこで開いた括弧を、次の数の直後で閉じれば、括弧をつけていないのと変わらないからです。また、同じ箇所に開き括弧を 2 個以上挿入するのも無意味です。ここで、次のような DP を考えます。

$dp[i][j] = i$ 項目の直後で、j 個括弧が開いている時の、そこまでの式の値の最大値。

これをどのように更新すればいいのでしょうか。上の議論より、開き括弧の直前には必ず “−” があるはずです。つまり、x 個括弧が開いている中にある数は、最終的に $(-1)^x$ 倍された値として計算されるということです。これがわかれば更新するのは簡単です。この DP で答えが求まりましたが、これでは $O(N^2)$ となり、時間制限に間に合いません。

しかし実は、括弧は 3 個以上開く必要がありません。なぜなら、括弧を 2 個開いたあとは、“−” の前で必ず一つ括弧を閉じるようにすることで、残りの数を全て “+” として扱うことができるからです。よって j の範囲は 0〜2 だけを考えればよくなり、この DP が $O(N)$ で行えるようになりました。

F : Contest with Drinks Hard

最初に、ドリンクがない状態で、最大スコアを求めましょう。この問題は、

「いくつか区間を選んで、

$\sum_{\text{選んだ区間}}$(区間の長さ) $*$ (区間の長さ $+1$)$/2-$区間に含まれる問題にかかる時間の合計
を最大化せよ」
と言い換えることができます。まず、次のような DP を考えます。
$dp[i]=$問題 i まで考えた時の最大スコア。
この DP の遷移は、以下のようになります。
$dp[i]=\max(\max_{\{0\leq j<i\}}\{dp[j]+(i-j)*(i-j+1)/2-(sum(i)-sum(j))\},dp[i-1])$
ここで、$sum(x)$ は、問題 1〜x までを解くのにかかる時間の総和です。さらに、式を整理すると、以下のようになります。
$dp[i]=\max(\max_{\{0\leq j<i\}}\{-j*i+j*(j-1)/2+dp[j]+sum(j)\}+i*(i+1)/2-sum(i),dp[i-1])$
これは、内側の max の中が、i に関する 1 次式になっているので、ConvexHullTrick を使って高速化することができます。よって、この DP は $O(N)$ で計算することができます。
では、クエリに答えるにはどうすればいいのでしょうか。そのためには、各問題について、それを解く時の最大スコア、解かない時の最大スコアを前計算しておけばよいです。
では、前計算はどうすればいいのでしょうか。上記の DP を、左から進めていったものと、右から進めていったものを用意すると、ある問題を解かない時の最大スコアはすぐに求まります。
次に、ある問題を解くときの最大スコアについて考えます。まず、$M=N/2$ として、問題 $i(i\geq M)$ と M を含む区間を解くときの最大スコアを考えます。これは、さっきの DP を少し変えると求まり、先ほどと同様に ConvexHullTrick で $O(N)$ で求めることができます。同様に、問題 $i(i\leq M-1)$ と M を含む区間を解くときの最大スコアも求められます。すると、ほかに考えるべきものは、問題 $i(1<M)$ を含んで、M を含まない区間と、問題 $i(M<i)$ を含んで、M を含まない区間だけです。問題を、M の左の部分と右の部分に分けることができたので、分割統治をすると良いとわかります。分割統治の 1 ステップが $O(N)$ なので、合計 $O(NlogN)$ で求めることが出来ました。

ABC 049 / ARC 065 解説

writer : hogloid

2016 年 12 月 10 日

English Editorial starts on page 5.

A : 居合を終え、青い絵を覆う

文字を入力し、それが a, e, i, o, u のいずれかであるか判定すればよいです。
if の中に 5 つの条件を連ねる以外にも、以下のように母音全てを含む文字列が c を含むか判定することもできます。

C++ のコード例

```
char c;
cin >> c;
const string vowel = "aeiou";
cout << vowel.find(c) == string::npos ? "vowel" : "consonant" << endl;
```

B : たてなが

この問題では、出力する画像をピクセルごとに求める方針でも解くことができますが、出力する画像の 1, 2 行目は元の画像の 1 行目、3, 4 行目は元の画像の 2 行目…のように、上の行から順に 2 回出力を繰り返せばよいです。

C++ のコード例

```
int h, w;
cin >> h >> w;
char buf[105];
for (int i = 0; i < h; ++i) {
  scanf("%s", buf);
  printf("%s\n%s\n", buf, buf);
}
```

C：白昼夢

文字列 S を dream, dreamer, erase, eraser に分解していくことを考えます。先頭から分解していこうとすると、例えば dreamer まで読んだとき、dream で切るべきなのか、dreamer で切るべきなのか判定することができません。(dreameraser は dream eraser と切らなければならないので、dreamer まで読んだときに dream で切らなければいけない場合が存在することが分かります)

逆に、後ろから読んでみましょう。4 つの単語を後ろから読むと、それぞれ maerd, remaerd, esare, resare となります。この 4 つの文字列は、ある文字列が他の文字列の接頭辞 (prefix) になっていないため、後ろから読んで当てはまるものが見つかれば即座に分解するしかありません。(参考: 語頭符号) S を最終的に分解することができなかった場合 `NO` を、そうでない場合 `YES` を出力します。

D：連結

配列 ar で、都市 i と都市 j が道路で連結しているとき $ar[i] = ar[j]$、そうでないとき $ar[i] \neq ar[j]$ となるようなものを取ります。これは、道路の辺を使い深さ優先探索をしたり、道路で直接結ばれた都市をつないだ Union-Find での根の都市の番号を取ることにより実現できます。

同様に、配列 br で、都市 i と都市 j が鉄道で連結しているとき $br[i] = br[j]$、そうでないとき $br[i] \neq br[j]$ となるようなものを取ります。

このとき、i と j が鉄道・道路の両方で連結している $\Leftrightarrow ar[i] = ar[j] \wedge br[i] = br[j]$ です。よって、各 i について、$(ar[i], br[i])$ のようにペアとし、同じペアがいくつ出てくるかを配列のソートや連想配列などを用い求めることで解くことができます。$\mathrm{O}(K + L + NlogN)$ です。

E：へんなコンパス

まず、$(x_i, y_i) \mapsto (x_i + y_i, x_i - y_i)$ と穴の座標を置き換えます。これは、平面を 45 度回転させる操作に対応します。

これにより、$d(i, j) = max(|x_i - x_j|, |y_i - y_j|)$ となります。

必須ではありませんが、この方が何かと実装が平易で済みます。

コンパスにより指される点の組を求めますが、コンパスの指す 2 穴の間の距離 D は常に不変なので、コンパスが指すことのできる穴の集合を求めれば、そこから距離 D の頂点の数を求め総和を取り 2 で割ることで答えが求まります。

コンパスが指すことのできる穴の集合 S は、以下のように再帰的に求めることができます。

擬似コード

```
S := empty
dfs(int v){ // 穴 v をコンパスが指すことができると分かった
  S に v を追加

  S に追加されておらず、v から距離の穴D u それぞれについて{
    dfs(u)
  }
}
dfs(a)
```

ここで、

- S に追加されておらず
- v から距離 D の穴

の 2 の条件をともに満たす穴のみを列挙することで O(N) 回の操作になります。
一つ目の条件が欠けていると、例えば変換後に $x_i = i/2, y_i = (i \bmod 2) * 1000000000$ (割り算は切り捨て) といった場合に、距離 D の穴の組は O(N^2) 個あるため間に合いません。
また、まだ追加されていない全ての頂点について調べる、などを行うと、これもやはり O(N^2) となってしまいます。
「まだ S に追加されておらず、v から距離 D の穴」を 4 つの部分に分けてみましょう。

- $(x_v + D, y_v - D)...(x_v + D, y_v + D)$ の間の点 (v の右側)
- $(x_v - D, y_v - D)...(x_v - D, y_v + D)$ の間の点 (v の左側)
- $(x_v - D, y_v + D)...(x_v + D, y_v + D)$ の間の点 (v の上側)
- $(x_v - D, y_v - D)...(x_v + D, y_v - D)$ の間の点 (v の下側)

さらに、順序集合 (C++ なら std::set) を使ってみましょう。順序を x での比較、同じなら y での比較とします。v の右側の穴を列挙するためには、$(x_v + D, y_v - D)$ 以上 $(x_v + D, y_v + D)$ 以下の穴のみが条件を満たします。これらを 1 つずつ走査していけばよいです。S に v が追加されたときに、この順序集合からも穴の座標を取り除いておけば、一つ目の条件も満たすことができます。左側も同様です。上側・下側の穴は、x, y の比較の順番を逆にした順序集合を持てば同様に実現できます。

コンパスが指すことのできる点から距離 D の穴の数が求める処理が残っています。これは、穴を順序集合で持つ代わりに配列で持ち、ソートし、条件を満たす穴の範囲を二分探索で求め、範囲の長さについて加算すれば実現できます。先ほどの操作とは違い、条件を満たすものの数を重複なく数えなければいけないため、四隅の条件に注意してください。

順序集合からの削除や、二分探索があるため、計算量は O($NlogN$) となります。

おまけ: ソートと O(N) 回の Union-Find の操作の他は線形で解くこともできます。

F：シャッフル

まず、$r_i \geq r_j, i < j$ なる j については、j 番目の操作を行っても行わなくても答えは変わりません。
条件を満たす i のうち最大のものを取ると、$k(i < k \leq j)$ 番目の操作によりシャッフルされる範囲は全て i 番目のシャッフルにより自由に並べ替えることができるためです。
また、$l_i = l_j, r_i > r_j$ なる j も、同様に j 番目の操作を行っても行わなくても答えは変わりません。

これにより、答えの変わらない操作を取り除くことで、$i < j$ のとき $l_i < l_j, r_i < r_j$ を満たすよう操作の列を取ることができます。

次に、シャッフルは部分文字列の配置を全て決めてしまう操作で、これを行うと状態が爆発してしまうため、逐次的な操作に置き換えることを考えましょう。文字列を左から順に決めていくことを考えます。k 番目より左の文字列を決めたとき、

- どこまでがシャッフルされることになっているか (すなわち、自由に並べ替えることができるか)
- シャッフルされることになる部分文字列のうち、1 はいくつあるか

を覚えておけば、1 か 0 を置いていくことができます。なぜなら、シャッフルにより並べ方は自由になってしまうため、1 の数さえ覚えておけば大丈夫だからです。どこまでがシャッフルされることになっているか、は k により一意に決めることができます。$l_i < k$ なる最大の i を取ったときの r_i です。これを $rightend_k$ と表します。

ここで、DP を考えてみましょう。
$dp[k][j] :=$ k 番目より左の文字列は決め、$S[k...rightend_k]$ に 1 が j 個あるときの、k 番目までの文字列の決め方の数
とします。シャッフルされることになる文字列の中の 1 の数を one とします。

- $l_i = k$ となる i があるとき、$S[rightend_k...r_i]$ が新たにシャッフルすることになる部分に加わるので、$one = j +$ ($S[rightend_k...r_i]$ の中の 1 の数) とします。
- そうでないとき、$one = j$ とします。

k 番目の文字を決めるとき、

- $one > 0$ なら 1 を置くことに対応する状態遷移を書きます ($dp[k+1][one-1] += dp[k][j]$)。
- 同様に、全てが 1 でないとき、0 を置く状態遷移を書きます ($dp[k+1][one] += dp[k][j]$)

最終的な答えは、$dp[N+1][0]$ となります。

この DP では、ある文字がどの操作によってもシャッフルされないときを考慮していないため、そのような場合は「1 文字の部分をシャッフルする」(=実質的に何も行わない) ダミーの操作を加えたり、初めから文字列を切り分けてしまうことで対応できます。

計算量は $O(N^2)$ です。

ABC 049 / ARC 065 Rough Editorial

writer : hogloid

2016 年 12 月 10 日

A : UOIAUAI

C++ Code Example

```
char c;
cin >> c;
const string vowel = "aeiou";
cout << vowel.find(c) == string::npos ? "vowel" : "consonant" << endl;
```

B : Thin

Repeat each row twice.

C++ Code Example

```
int h, w;
cin >> h >> w;
char buf[105];
for (int i = 0; i < h; ++i) {
  scanf("%s", buf);
  printf("%s\n%s\n", buf, buf);
}
```

C : Daydream

Reverse the strings. Then, the four words will be: maerd, remaerd, esare, resare. This way you can determine the composition greedily.

D : Connectivity

Construct a graph with roads and find the connected components in it. Then, you can assign values a_i to each city i such that the cities p and q are reachable using roads iff $a_p = a_q$. Similarly, define values b_i for railways.

Then, a pair of two cities satisfies the conditions iff their (a, b) values are the same.

E : Manhattan Compass

Let D be the distance between the two points initially pointed by the compass.

We have two things to do:

- Construct a graph with N vertices. Two vertices a and b are connected if the distance between them is D. Find connected components in this graph.
- For each vertex a, count the number of vertices b such that the distance between a and b is D.

The latter is relatively easy, so we describe the former.

First rotate the entire plane by 45 degrees. Let dx, dy be the difference of x, y coordinates. Now we are intersted in a pair of points such that $max(dx, dy) = D$. To do this, we want to find all pairs of points (a, b) such that

- $x_b - x_a = D$
- $-D \leq y_b - y_a \leq D$

Let's sort the points by the pair (x, y) and renumber them. Then, for a fixed a, the set of b that satisfies the conditions above will form an interval. Let $[L, R]$ be this interval (the interval can be found using binary search). Then, we want to add edges a - L, a - $L + 1$, ..., a - R.

Instead, it is equivalent to add edges a - L and L - $L+1$, $L+1$ - $L+2$, ..., $R-1$ - R. This way, except for $O(N)$ edges, each edge connect adjacent points, so if we compute all edges that connect adjacent points in a clever way, there will be $O(N)$ edges in total.

We should do similar things after swapping x and y. After that, we get the desired graph.

F : Shuffle

If $l_i = l_j, r_i > r_j$, we can ignore the pair j. Thus, we can assume that $M = N$ and for each i, $l_i = i$.

Let $dp[L][R][c]$ be the number of strings we can obtain under the following state:

- We've fixed all characters to the left of L.
- The characters in the range $[L, R)$ are shuffled. c of them are 1.
- In the future, we will perform operations with $l_i \geq L$.

We compute the values as follows:

- When $L = N$, we've finished; the value is 1.
- If $a_L > R$, we can extend the "shuffled range"; change R to a_L and increase c by the number of ones in the interval $[R, a_L]$.
- Now consider two cases for the characters at L. If this is '0' (or '1'), there are $dp[L+1][R][c]$ (or $dp[L+1][R][c-1]$) ways to determine the rest. Thus $dp[L][R][c] = dp[L+1][R][c] + dp[L+1][R][c-1]$. (Make sure not to access invalid values)

This looks like an $O(N^3)$ solution, but we can notice that for a fixed L, only one R appears during the computation. Thus, we can ignore the value of R and it turns out to be an $O(N^2)$ solution.

ABC 048 / ARC 064 解説

writer : sugim48

2016 年 12 月 4 日

A : AtCoder *** Contest

3 個の文字列を入力し，それらの 1 文字目を続けて出力すればよいです．

C++ のコード例

```
string a, b, c;
cin >> a >> b >> c;
cout << a[0] << b[0] << c[0] << endl;
```

B : Between a and b ...

a 以上 b 以下の整数のうち条件を満たすものの個数を求める問題です．このような問題では，

$f(n) := 0$ 以上 n 以下の整数のうち条件を満たすものの個数

と定義しておくと，答えは $f(b) - f(a-1)$ で求まるので楽です．ただし，$a = 0$ のときに $f(-1)$ が呼ばれることに注意してください．このことに注意すると，f は次のように書けます．

$$f(n) = \begin{cases} n/x + 1 & (n \geq 0) \\ 0 & (n = -1) \end{cases}$$

ただし，/ は切り捨ての除算です．

C : Boxes and Candies

各 $1 \leq i \leq N-1$ について $a_i + a_{i+1} \leq x$ となるように，a の各要素を (非負整数の範囲で) 減らしていきます．最後に，元の a と比べることで操作回数が分かります．この操作回数を最小化することを目指します．

まず，$a_1 > x$ ならば $a_1 = x$ としておきます．次に，$i = 1, 2, ..., N-1$ の順に $a_i + a_{i+1} \leq x$ を成立させていきます．とりあえず，最初から $a_i + a_{i+1} \leq x$ ならば何もする必要はありません．$a_i + a_{i+1} > x$ ならば，$a_i + a_{i+1} = x$ となるまで a_i と a_{i+1} を減らさなければなりませんが，このとき a_{i+1} だけを減らすのが最適であることが分かります．なぜならば，これから各 $j > i$ について $a_j + a_{j+1} \leq x$ を成立させていくときに，a_{i+1} が小さいほど操作回数が少なくなることが期待できるからです．

以上の貪欲法により正しく答えを求めることができます．なお，オーバーフローに注意してください．

D : An Ordinary Game

結論から述べると，「s の長さが偶数である」と「s の先頭文字と末尾文字が同一である」の排他的論理和が真ならば後手が勝ち，偽ならば先手が勝ちます．以降はこれを示します．

どちらかのプレイヤーが操作を行えなくなったときの，最終的な s について考えます．最終的な s は “abababab...” のように，異なる 2 文字が交互に並んでいるはずです．なぜならば，ルールより同一の文字が隣り合う箇所はなく，また “abc” のように異なる 3 文字が連続する箇所があれば，まだ真ん中の文字を取り除けるからです．

最終的な s の長さは一意には定まりませんが，その偶奇は一意に定まります．具体的には，s の先頭文字と末尾文字が同一ならば “ababa” のように奇数長となり，s の先頭文字と末尾文字が異なるならば “ababab” のように偶数長となります．以上より，最初と最後で s の長さの偶奇が分かっているので，ターン数の偶奇も分かります．よって，どちらのプレイヤーが先に操作を行えなくなるかが求まります．

E : Cosmic Rays

簡単のため，始点 (x_s, y_s) および終点 (x_t, y_t) にも半径 0 のバリアが張られていると仮定します．すると，始点および終点はそれぞれあるバリアの中心に位置することになります．

始点から終点までの最適経路を求めるとき，すぬけ君はバリアの中心どうしを結ぶ線分上のみを移動すると仮定してもよいことが分かります．なぜならば，あるバリアの境界から別のバリアの境界まで移動するときは，それらのバリアの中心を結ぶ線分上を移動するのが最適であり，また，あるバリアの内部を移動するときは，一度そのバリアの中心を経由しても損はしないからです．

よって，各バリアの中心を頂点として全点間に辺を張ったグラフを考え，このグラフの最短経路問題を解けばよいことになります．バリア i, j の中心を結ぶ辺のコストは w_{ij} は

$$w_{ij} = \max\left\{0, \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} - (r_i + r_j)\right\}$$

と計算できます．辺のコストがすべて非負の最短経路問題なので，Dijkstra 法で解くことができます．時間計算量は $\mathrm{O}(N^2)$ です．なお，実装によっては，始点から終点まで直接移動する経路を見落とすことがあるので，注意してください．

F : Rotated Palindromes

長さ N の数列 a を考えます．d が N の約数であり，長さ d の数列の繰り返しとして a が表されるとします．このとき，d は a の周期であるとします．a の周期のうち最小のものを最小周期と呼ぶことにします．たとえば，$(1,2,1,2,1,2,1,2)$ の最小周期は 2 で，$(1,2,1)$ の最小周期は 3 です．

高橋君が用意する数列 a の最小周期 d を全探索することを考えます．最小周期 d を固定したとき，高橋君が用意できる数列 a の個数を $num[d]$ とします．$num[d]$ は次のように計算できます．まず，a は回文なので，a の先頭 d 文字分も回文であることが分かります．長さ d の回文の個数は $K^{\lceil d/2 \rceil}$ です．では $num[d] = K^{\lceil d/2 \rceil}$ なのかというと，そうではありません．d' $(\neq d)$ を d の約数としたとき，$num[d']$ も余分

に数えてしまっているからです．これを踏まえると，$num[d]$ は

$$num[d] = K^{\lceil d/2 \rceil} - \sum_{d'|d,\ d' \neq d} num[d']$$

と計算できます．以上のようにして，小さい d から順に $num[d]$ を埋めていくことができます．

次に，高橋君が最小周期 d の数列 a を用意したとき，青木君の操作によって何通りの数列が得られるかを考えます．最小周期が d なので，$0, 1, ..., d-1$ 回操作を行って得られる数列はすべて異なることが分かります．また，d 回操作を行うと元の a に戻ることが分かります．よって，最小周期 d の数列 a からは d 通りの数列が得られることになります．では問題の答えは $\sum_{d|N} num[d] \cdot d$ なのかというと，そうではありません．高橋君が用意した別々の数列 a から，青木君の操作によって同一の数列が得られる可能性があるからです．具体的には，最小周期 d が奇数の数列 a から得られる各数列は，全体でちょうど 1 回ずつ生成されており，最小周期 d が偶数の数列 a から得られる各数列は，全体でちょうど 2 回ずつ生成されています (証明は後述)．よって，正しい答えは

$$\sum_{d|N,\ d:\text{odd}} num[d] \cdot d + \sum_{d|N,\ d:\text{even}} num[d] \cdot \frac{d}{2}$$

と計算できます．

時間計算量は $\mathrm{O}(d(N)^2)$ です．ここで，$d(N)$ は N の約数の個数です．$N \leq 10^9$ における $d(N)$ の最大値は，$d(735{,}134{,}400) = 1{,}344$ とそこまで大きくありません．よって，$\mathrm{O}(d(N)^2)$ でも十分間に合います．

(証明) 高橋君が用意した別々の数列 a, b から，青木君の操作によって同一の数列が得られたとします．これはすなわち，a に何回か操作を行って得られる数列が b に一致するということです．よって，a と b の最小周期は等しいことが分かります．これを d とおきます．すると，a に $1, 2, ..., d-1$ 回操作を行って得られる数列のどれか 1 つのみが b に一致することになります．ここでは，a に t 回操作を行って得られる数列が b に一致するとします．このとき，a に t 回逆操作を行って得られる数列も b に一致します．なぜならば，数列 a を逆順にした数列を $\overline{a}$ と書くことにすると，

$$\begin{aligned}
(a \text{ に } t \text{ 回逆操作をした数列}) &= (\overline{a} \text{ に } t \text{ 回逆操作をした数列}) \\
&= \overline{(a \text{ に } t \text{ 回操作をした数列})} \\
&= \overline{b} \\
&= b
\end{aligned}$$

となるからです．よって，a に $d-t$ 回逆操作を行って得られる数列も b に一致します．a に $1, 2, ..., d-1$ 回操作を行って得られる数列のどれか 1 つのみが b に一致するので，$t = d - t$ でなければならず，$d = 2t$ が得られます．

以上より，最小周期 d が奇数の数列 a については，同一の数列を生成するような別の数列 b は存在しません．逆に，最小周期 d が偶数の数列 a については，同一の数列を生成するような別の数列 b がちょうど 1 つ存在します．そのような b は，a に $\frac{d}{2}$ 回操作を行って得られる数列です．このとき，仮に $a = b$ とすると $\frac{d}{2}$ も周期となり矛盾するので，a と b は別々であることが分かります．

ARC062 / ABC046 解説

sigma425

A: AtCoDeer くんとペンキ

$a = b = c$ の場合は 1 を、$a = b \neq c$ または $a = c \neq b$ または $b = c \neq a$ の場合は 2 を、それ以外の場合は 3 を出力すれば良いです。

B: AtCoDeer くんとボール色塗り

まず左端のボールに塗る色を決めます。これは K 通りあります。さらに左から順番に塗る色を決めていきます。この時、これまでどのように塗っていようが、”ひとつ左のボールに塗られている色”以外の $K-1$ 通りの色が塗れるので、全体では $K \cdot (K-1)^{N-1}$ 通りになります。答えが大きいので、塗り方をすべて試す方法では間に合いません。

C: AtCoDeer くんと選挙速報

各途中状態で、できるだけ投票数が少ない方がよいです (投票数を増やすのはいつでも出来るので)。今高橋君に A 票、青木君に B 票入っていて、次に満たすべき比率が $x : y$ だとすると、$A \leq nx \ \wedge \ B \leq ny$ なるような最小の自然数 n を取れば、次にあり得る最小の得票数は nx, ny であることがわかります。このような n は $max(\lceil A/x \rceil, \lceil B/y \rceil)$ で計算できます。はじめに $A = 1, B = 1$ として、これを N 回繰り返すことで $O(N)$ で答えが得られます。

D: AtCoDeer くんと変なじゃんけん

相手が P 回パーを出したとします。まず自分が全てグーを出したとすると、この時の得点は $-P$ 点です。自分のグーのどれかひとつをパーに変えることを考えると、

- そのターンの相手の手がグーのとき　・・・ あいこ (0 点) から勝ち (1 点) に変わるので、 得点は 1 増えます。
- そのターンの相手の手がパーのとき　・・・ 負け (−1 点) からあいこ (0 点) に変わるので、 得点は 1 増えます。

従って、自分や相手がどのターンでパーを出したかによらず、得点は (自分がパーを出した回数)-(相手がパーを出した回数)　になります。なのでできるだけ多くパーを出せばよいです。グー, パー, グー, パー,... と出すことでこれが実現できて、$\lfloor N/2 \rfloor$ 回出せます。よって答えは $\lfloor N/2 \rfloor - P$ です。

E: AtCoDeer くんと立方体づくり

立方体の上面と底面のパネルとその向きを、どれを使うか全探索します。すると 8 つの頂点の色がすべて決まるので、あとは各側面に対し置けるパネルの数を数えれば良いです。これはパネルを”正規化”した形で表して、回転による差異を無視できるようにしてから map に入れて数を数えるなどの方法でできます。(例えば、入力のように 4 つの数の列でパネルを表すとして、回転によって 4 通りの列が考えられるがそのうち辞書順最小のものでパネルを表す など)

他の部分に使ったパネルは使えないのでちゃんと使った分は引くことと、パネルの向きの置き方が何通りあるか (例えば 4 点同じ色のパネルだと,4 通り置ける) に注意すれば、側面の置き方を数えることが可能です。あとは、同じ立方体を何回重複して数えているか (これは数え方によって異なります) で割れば答えが求まります。計算量は $O(N^2 logN)$ ですが定数倍はかなり重いです。

F: AtCoDeer くんとグラフ色塗り

まず二重 (頂点) 連結成分分解します。異なる辺 x,y に対し, x と y が同じ成分に属する $\Leftrightarrow$ x と y を両方含む単純なサイクルがある なので、各操作はある連結成分内でしか行なえません。なので、各連結成分ごとに独立に考えられます。

1. 成分が辺一本のとき

   塗り方は K 通りです。

2. 成分がサイクルをなすとき

   サイクルを塗る方法なので、これはポリアの定理から、サイクルを n 頂点とすると、

   $$\frac{1}{n}\sum_{i=0}^{n-1} K^{gcd(n,i)}$$

   で計算できます。

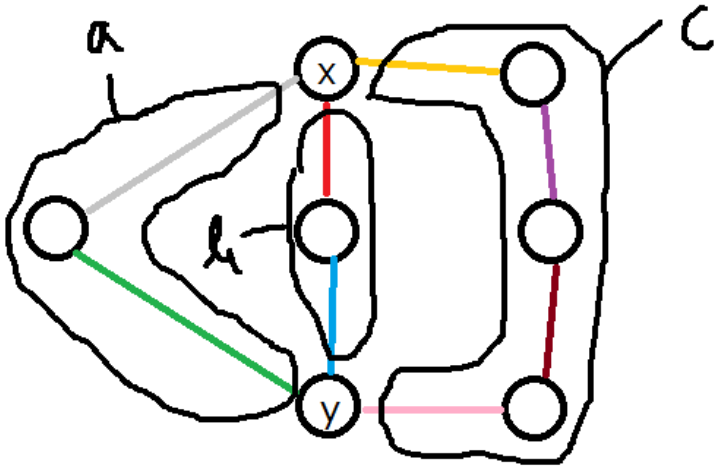
3. それ以外

   この時成分内で任意の置換が作れることがわかります。従って、連結成分の辺の個数を m 本とすると、答えは、${}_{m+K-1}C_{K-1}$ になります。

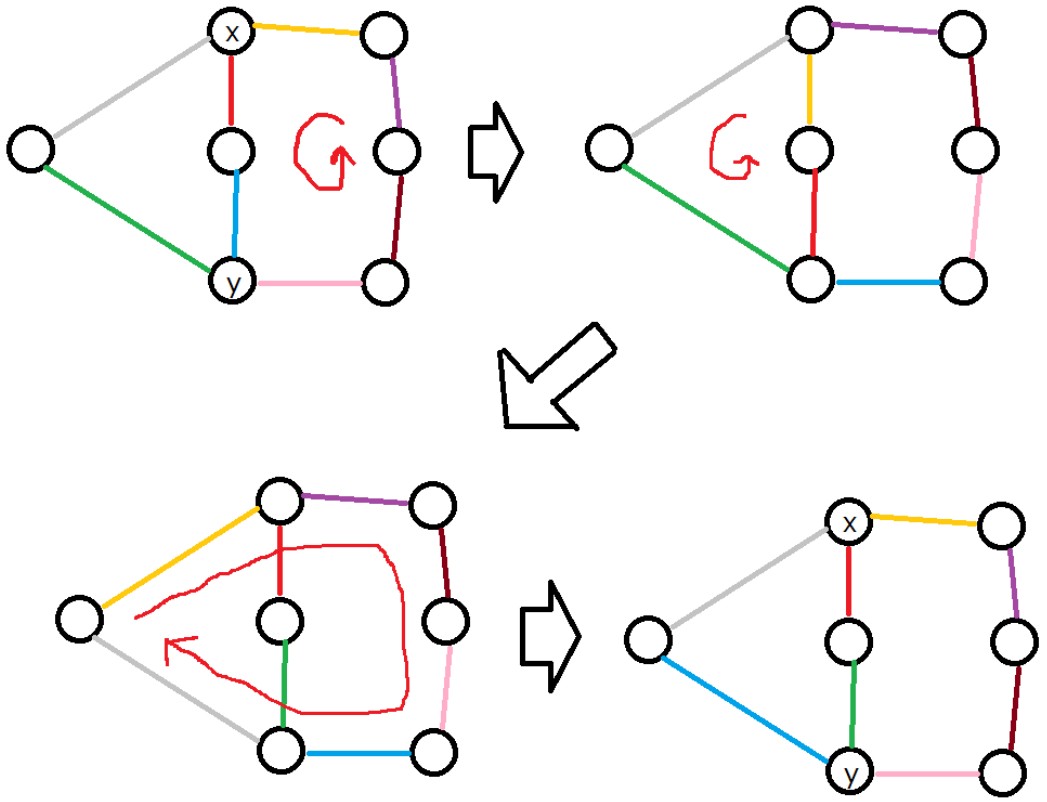
従って各連結成分ごとにこれらの値をかけ合わせると答えが求まります。

3 の場合に任意の置換が作れることの証明をします。

まず次の図のように、異なる頂点 x,y を,3 つの edge disjoint な (互いに辺を共有しない)path a,b,c が結んでいる状況を考えます。このグラフを便宜的に G と呼びます。



このとき、次のように操作をすることで、頂点 y に接続する path a の辺と path b の辺を swap することが出来ます。

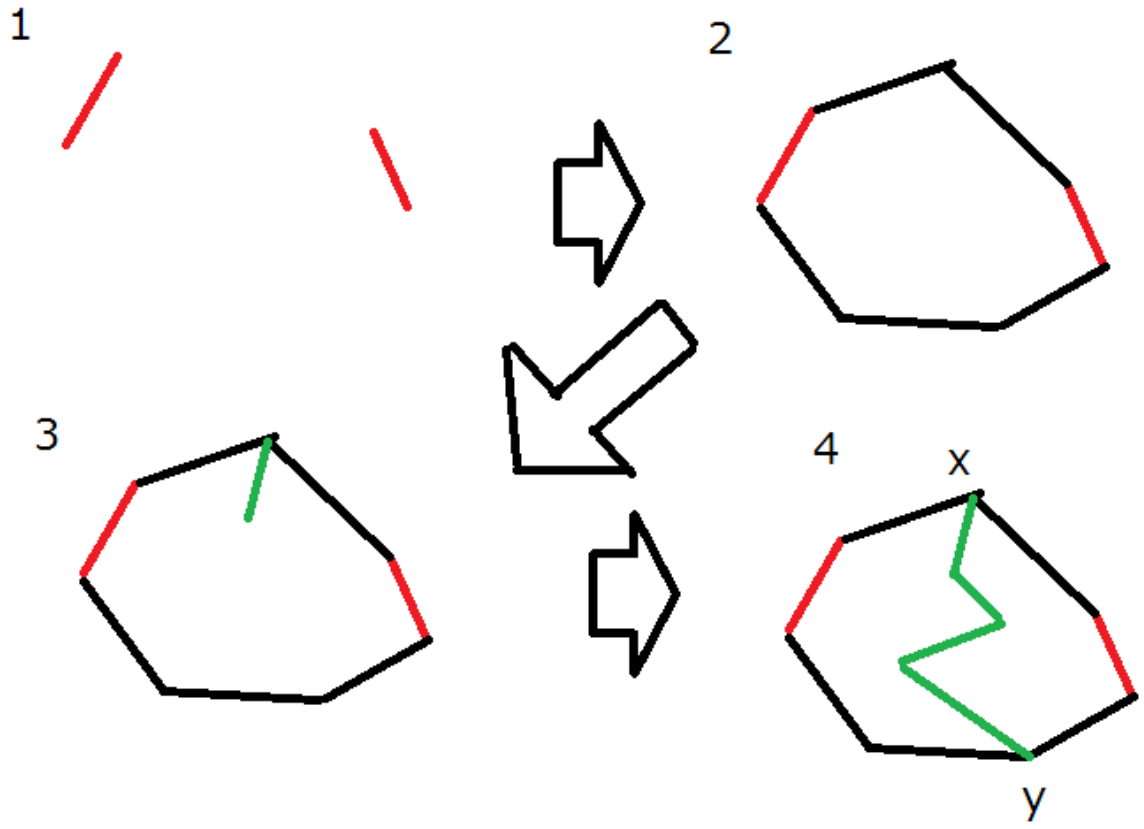


また、この操作を使って、グラフ G の任意の隣接する二辺を swap することが可能です。具体的には、まずその二辺が y に隣接するようにサイクルを回して、上の操作を行い、はじめにサイクルを回した部分の逆操作を行えばよいです。

これを使うと、グラフ G 上の任意の二辺が swap 出来ます。具体的には、その二辺を両端とする path をとり、その path 上の隣接する辺を swap していくことを二回すると出来ます。

あとは、元のグラフの任意の異なる二辺に対して、その二辺を含む、グラフ G のような構造が取れることが示せれば、その二辺は swap できるため結局全ての置換が作れることがわかり、証明は完了します。よってこれを示します。

まず二辺は同じ二重連結であり成分は辺一本ではないので、その二辺を含むサイクルがとれます (図の 2)。また、成分はただのサイクルではないため、そのサイクルとは別に辺が含まれます。当然二重連結成分は連結なので、そのような辺としてサイクルに接続しているものがとれます (図の 3)。この辺とサイクルが同じ二重連結成分に属するためには、また別のサイクルであって、サイクルの辺とこの辺をともに含むサイクルの存在が必要です。そのサイクルの辺を取ってくると、目的の構造が取れていることがわかります (図の 4)。



よって示されました。

二重 (頂点) 連結成分分解は、関節点を求めるのと同様なアルゴリズムで O(N+M) のものがありますが、今回の制約だと、dfs してサイクル基底を得て、各サイクルの辺を unionfind でつなぐ O(MNlogM) なども許されます。

ABC 045 / ARC 061 解説

tozangezan

2016 年 9 月 11 日

1　A: 台形

台形の面積は、上底を a、下底を b、高さを h とすると、$(a+b)h/2$ で求められます。この問題は h が偶数なので、答えは整数に収まるため、int 型で計算できます。

2　B: 3 人でカードゲームイージー

ルールの通りにシミュレーションします。実装の方針としては、3 人それぞれのカードの列を string 等で文字列にして持っておき、現在その文字列のうち何番目の文字がその人の持ってるカードのうち先頭にあるかを変数に持ちます。また、現在誰のターンであるかを表す変数も持っておきます。

それぞれのターンにおいては、まずそのターンの人がカードを全部使い切ったかを確認し、そうでないならば自分のカードの位置を表すカウントを 1 増やしてから、文字に対応する人のターンに移動します。

計算量は $O(|S_A| + |S_B| + |S_C|)$ なので、特に問題なく動きます。

3　C: たくさんの数式

与えられる文字列の長さの最大が 10 と小さいので、それぞれの文字と文字の間に ‘+’ を入れるかどうかを全探索で試すことが出来ます。

何番目の数字の後に ‘+’ を入れるかを全て決めたら、split 等の関数がある言語ならば、実際に新しく ‘+’ を入れた文字列を作り、区切って計算することができます。そうでなくても、前から順に見ていって、「数字があるたびに現在の値を 10 倍して今書いてある数字だけ足す」「‘+(’ が出てきたら現在の値が ‘+(’ の直前の整数に等しいので、それを答えに足す」「最後は ‘+(’ が登場せずに数式が終わるので、そこは別に足す」とすると、新しく出来た式の値を求めることができます。

4 D: すぬけ君の塗り絵

盤面が大きいので、実際に 2 次元配列で塗ることをシミュレーションすることはできません。また、 N が 10^5 と大きいので、座標圧縮したとしても、メモリ使用量が $O(N^2)$ となり厳しいです。

ここで気づくべきポイントは、「一つの黒く塗られたマスが影響を与える範囲は、その周辺にあるいくつかの 3×3 領域しかない」ことです。すなわち、それぞれの塗られたマスに対し、これらの周辺の少しのマスにのみ何かの処理をするだけで計算できれば、高速に答えが求められることになります。

(i,j) を、上から i 行目で左から j 行目のマスを左上とする 3×3 の正方形を表すことにします。例えば、上から p 行目で左から q 行目のマスを黒く塗ると、 $(p-k,q-l)(k=0,1,2)(l=0,1,2)$ に含まれる黒いマスが 1 つ増えます。(ただし、これらの正方形が盤面からはみ出すことがあるので、そういうケースは除外します)

ということで、入力で与えられる点に対してこれらの正方形の左上のマスを全部列挙しておき (最大で $9N$ 個列挙される)、その列挙されたものの中で、それぞれの正方形が何回出現するかがカウントできればよいです。

これは、pair 型などで (i,j) をソートして前から順に同じ値が何回出てくるかを数えても良いですし、map 等のデータ構造を使ってもよいです。

最後に、黒いマス 0 個の個数は、全体 $((H-2)(W-2)$ 個) から 1 回以上黒いマスが出現した 3×3 領域の個数を引けば求められます。

計算量は、$O(NlogN)$ となります。

5 E: すぬけ君の地下鉄旅行

(現在いる駅, 最後に使った会社) を頂点に最短路問題を解くことで答えが求められます。この解法では、頂点数は高々 $O(M)$ です。なぜならば、それぞれの駅に到達するのに最後に使った会社の種類数は、最大でもその駅から出ている路線の個数です。この個数を全ての駅で合計しても、路線の個数の定数倍にしかなりません。

しかし、効率的な辺の張り方をしないと、辺の数が $O(M^2)$ 等になってしまい、制限時間内に実行が終わらなくなります。

例えば、次のような辺の張り方をすると辺の数は高々 $O(N+M)$ で済みます。

- 同じ会社の路線に乗り換えるケースに関しては、そのまま長さ 0 の辺を全ての路線の目的地に対して張る
- 違う会社の路線に乗り換えるケースの代わりとして、$(\text{現在いる駅}, -1)$ という頂点に移動できるように長さ 0 の辺を張る。(改札から外に出るイメージ)
- $(\text{現在いる駅}, -1)$ の頂点から全ての会社の路線に対して、目的地に長さ 1 の辺を張る (改札の中に入るイメージ、この時点で新たに料金がかかる)。
- スタートは、 $(1,-1)$ の頂点。ゴールは、 $(N,-1)$ の頂点。

あとは、Dijkstra 法等で十分早い時間で答えを求めることができます。

6　F: 3 人でカードゲーム

6.1　部分点解法

この問題では、3 人がそれぞれカードを N, M, K 枚ずつ持っている設定ですが、誰かがカードを使い果たしてゲームに勝利するまでの動作に関しては、「誰かがカードを出す」操作の繰り返しだと考えることができます。そこで 3 つのカード列を考える代わりに、1 つの列にまとめて考えることができます。

この状況下で A さんが勝つ必要十分条件は、一列にしたカード列において、b が M 回以下、c が K 回以下しか出ていないときに a が N 回出ることです。N 回目の a が出るのが、全体で何ターン目にあたるかを全て試します。

i ターン目で N 回目の a が出るとき、条件を満たす配置の個数は、「i ターンで使われなかった余りのカードの選び方 $3^{N+M+K-i}$ 通り」「$i-1$ ターン目以前のうち a が出る $N-1$ 回を選ぶ方法の数 ${}_{i-1}C_{N-1}$ 通り」「a 以外が出る $i-N$ ターンを b と c で埋める方法の数」の積です。

最初の 2 つは簡単に計算することができます。最後の「a 以外が出る $i-N$ ターンを b と c で埋める方法の数」は、${}_{i-N}C_k$ $(max(0, i-N-c) \leq k \leq min(i-N, b))$ の総和です。これらを全部一つずつ計算すると、ここだけで $O((M+K)^2)$ となり、部分点は取れますが、満点は取れません。

6.2　満点解法

さっきの二項係数を足す連続する範囲を全ての i についてパスカルの三角形上に書いてみると、このような形になります。

```
*
**
***
***.
***..
.**...
..*....
........
```

これらを、上の段から一段ずつまとめて計算していきます。最初、一番上の段を見ていて、この範囲における求めたい合計は 1 です。

ここから降りていくとき、左下、右下ともに新たに追加されるうちは、その段の合計は、一つ上の段の合計の 2 倍となります。

左下、右下の片方しか追加されなくなったとき、その段の合計は、一つ上の段の求める範囲の合計の 2 倍から、追加されないほうの端に影響を与える、一つ上の段で求める範囲の中にあった点の値を引きます。

左下、右下のどちらも追加されなくなったとき、その段の合計は、一つ上の段の求める範囲の合計の 2 倍から、追加されない両端に影響を与える、一つ上の段で求める範囲の中にあった点の値を両方引きます。

このようにまとめて段ごとに計算することで、計算量は $O(M+K)$ となり、満点を取ることができます。

ABC 044/ARC 060 解説

1　A: 高橋君とホテルイージー

$N \leq K$ の場合，一泊あたり X 円で N 泊宿泊することになるため，答えは NX となる．$N > K$ の場合，最初の K 泊については一泊あたり X 円，残りの $N-K$ 泊については一泊あたり Y 円となるため，答えは $KX + (N-K)Y$ となる．

この問題においては，整数の入出力，四則演算等の基本的な演算，条件分岐が必要となる．

2　B: 美しい文字列

各英小文字 (‘a’–‘z’) ごとに，以下を行えばよい．

1. w 中における出現回数を数える．
2. 出現回数が奇数か偶数かを判定する．

出現回数が奇数の文字が 1 種類でもあった場合は “No” を，それ以外の場合は “Yes” を出力する．

この問題においては，ループ，文字列の入力および長さの取得などが必要となる．なお，言語によっては，入力した文字列の末尾に改行文字が含まれる可能性がある．特に，長さを取得する際に問題となりうるため，場合によっては末尾の改行を除去した方が良いこともある．普段使用している言語で改行がどのような取り扱いとなっているか，確認しておくことが望ましい．

また，多くの言語では，文字は整数として表現されているため，文字と整数は相互に変換できる (https://ja.wikipedia.org/wiki/ASCII 等を参照)．これを用いると実装が楽になるかもしれない．

3　C: 高橋君とカード

以下，$X = \max\{x_1, \ldots, x_N, A\}$ とおく．

3.1　部分点解法

部分点制約では N が小さいので，すべてのカードの選び方 ($2^N - 1$ 通り) を試すことができる．全通り試すにはビット演算を用いるのが楽である．具体的には ABC 014 B 問題 (http://abc014.contest.atcoder.jp/tasks/abc014_2) などを参照するとよい．計算量は全体で $O(N \cdot 2^N)$ となる．

3.2　満点解法 1

以下の 3 次元配列 $\mathrm{dp}[j][k][s]$ (ただし $0 \leq j \leq N$, $0 \leq k \leq N$, $0 \leq s \leq NX$) を考える．

$$\mathrm{dp}[j][k][s] = (x_1, \ldots, x_j \text{ から } k \text{ 枚選んで } x_i \text{ の合計を } s \text{ にするような選び方の総数})$$

この配列は次のように計算することができる．

$$\mathrm{dp}[j][k][s] = \begin{cases} 1 & (j = 0,\ k = 0,\ s = 0 \text{ のとき}) \\ \mathrm{dp}[j-1][k][s] & (j \geq 1,\ s < x_j \text{ のとき}) \\ \mathrm{dp}[j-1][k][s] + \mathrm{dp}[j-1][k-1][s-x_j] & (j \geq 1,\ k \geq 1,\ s \geq x_j \text{ のとき}) \\ 0 & (\text{上記いずれでもないとき}) \end{cases}$$

さて，選んだカードの x_i の平均を A にするには，各 k $(1 \leq k \leq N)$ について，k 枚選んだときの合計が kA であればよい．すなわち $\sum_{k=1}^{N} \mathrm{dp}[N][k][kA]$ が答えとなる．

動的計画法を用いることで，計算量は全体で $O(N^3 X)$ 時間となる．

3.3　満点解法 2

「選んだカードの x_i の平均が A」であることは，「選んだカードの $(x_i - A)$ の合計が 0」と言い換えることができる．すなわち，各カード i について $y_i = x_i - A$ と定義すると，選んだカードの y_i の合計が 0 となるようなカードの選び方の総数を求めればよいことになり，選んだ枚数を考慮する必要がなくなる．この考え方に基づく，$O(N^2 X)$ 時間の解法も存在する．

実装においては，y_i が負となりうることを考慮し，以下の 2 次元配列 $\mathrm{dp}[j][t]$ (ただし $0 \leq j \leq N$, $0 \leq t \leq 2NX$) を考えるとよい．

$$\mathrm{dp}[j][t] = (y_1, \ldots, y_j \text{ から } 0 \text{ 枚以上選んで } y_i \text{ の合計を } t - NX \text{ にするような選び方の総数})$$

この配列は次のように計算することができる．

$$\mathrm{dp}[j][t] = \begin{cases} 1 & (j = 0,\ t = NX \text{ のとき}) \\ \mathrm{dp}[j-1][t] & (j \geq 1,\ \text{かつ},\ t - y_j < 0 \text{ または } t - y_j > 2NX \text{ のとき}) \\ \mathrm{dp}[j-1][t] + \mathrm{dp}[j-1][t-y_j] & (j \geq 1,\ 0 \leq t - y_j \leq 2NX \text{ のとき}) \\ 0 & (\text{上記いずれでもないとき}) \end{cases}$$

1 枚も選ばないパターンを除外する必要があるため，答えは $\mathrm{dp}[N][NX] - 1$ となる．

4　D: 桁和

明らかに，$s = n$ の場合は $n + 1$ が答えである．

それ以外の場合，はじめに $2 \leq b \leq \sqrt{n}$ および $f(b, n) = s$ を満たす整数 b が存在するかどうか全探索する．もしもそのような b が存在するならば，その最小値が答えである．

そうでない場合には，$\sqrt{n} < b \leq n$ および $f(b, n) = s$ を満たす整数 b が存在するかどうか検証する必要がある．ここで，$b > \sqrt{n}$ である場合には，n は b 進表記で 2 桁となることに着目する．すなわち，上位桁を p，下位桁を q $(1 \leq q < b, 0 \leq q < b)$ とすると，

$$n = pb + q \tag{1}$$

と書くことができる．また，題意より

$$p + q = s \tag{2}$$

が成立する．ところで $b > \sqrt{n}$ であるから，式 1 より $n = pb + q \geq pb > p^2$ となり，$p < \sqrt{n}$ が導かれる．そこで，上位桁 p を全探索すればよい．式 1,2 より $b = (n - s)/p + 1$ となるから，p から b は一意に定まる．最後に，そのような b について，実際に $f(b, n) = s$ を満たすか確認すればよい．

計算量は全体で $O(\sqrt{n})$ 時間となり，満点が得られる．

5　E: 高橋君とホテル

以下の 2 次元配列 r を考える．

$$r[k][i] = (i \text{ 番目のホテルから } 2^k \text{ 日以内に到達可能な最右のホテル番号})$$

$r[0][i]$ は，$x_j \leq x_i + L$ を満たす最大の j であり，これは $x_1, \ldots, x_N$ に対し二分探索などを用いることで，効率よく求めることができる．さらに，$r[k+1][i] = r[k][r[k][i]]$ となるから，ダブリングにより $r[1][\cdot], r[2][\cdot], \ldots$ を順次求めることができる．各クエリでは，この配列 r を用いて二分探索を行えばよい．計算量は全体で $O((N + Q) \log N)$ となる．

6　F: 最良表現

x を文字列，p を正整数とする．$0 \leq i < |x| - p$ を満たす任意の整数 i に対し $x[i] = x[i+p]$ を満たすならば，p を x の周期と呼ぶ．また，x の周期の最小値を $\mathrm{per}(x)$ と表記する．例えば $\mathrm{per}(\texttt{abcabcabcab}) = 3$ である．

入力文字列 w について，その長さを N とする．以下のとおりの場合分けを行う．

(a) w が良い文字列である場合 (例: $w = \texttt{ababa}$)
(b) $\mathrm{per}(w) = 1$ の場合 (例: $w = \texttt{aaaaa}$)
(c) それ以外の場合 (例: $w = \texttt{abcabcabc}$)

(a) の場合，明らかに最良表現の項数は 1 であり，また最良表現の総数も 1 である．良い文字列かどうかの判定法については後述する．

(b) の場合，最良表現の項数は N であり，また最良表現の総数も 1 である．

(c) の場合，最良表現の項数が 2 となることが証明できる (後述の定理 5 を参照)．したがって，最良表現の総数を求めるためには，次の条件をすべて満たす整数 i の総数を求めれば良いことになる．

- $1 \leq i < N$
- $w[0..i-1]$ は良い文字列である
- $w[i..N-1]$ は良い文字列である

すなわち，w の各接頭辞および各接尾辞について，良い文字列かどうか判定できれば良いことになる．以下，w の各接頭辞が良い文字列かどうか判定する方法を述べる．接尾辞については，w の逆文字列に対し同じ方法を適用すればよい．

以下の配列 $Z[0..N-1]$ を考える．

$$Z[i] = (w \text{ と } w[i..N-1] \text{ の最長共通部分列の長さ})$$

この配列は，Z algorithm [2] を用いることで $O(N)$ 時間で求められることが知られている．また，次の観察が得られる．

観察 1. p を，$1 \leq p < N$ を満たす整数とする．$k \geq 2$ および $(k-1)p \leq Z[p]$ を満たす任意の整数 k について，$w[0..kp-1]$ は $w[0..p-1]$ を k 回繰り返した文字列である．すなわち，$w[0..kp-1]$ は良い文字列ではない．

いま，$G[1..N]$ を，次の条件を満たす bool 型配列とする．

$$G[i] = \text{true} \iff w[0..i-1] \text{ は良い文字列}$$

観察 1 を用いることで，次の処理により配列 G を求めることができる．すなわち，w の各接頭辞が良い文字列であるかどうかを効率的に判定できる．

```
1 for (i = 1; i <= N; i++) { G[i] = true; }
2 for (p = 1; p < N; p++) {
3     for (k = 2; (k - 1) * p <= Z[p]; k++) {
4         G[k * p] = false;
5     }
6 }
```

上記の計算量は $O(N \log N)$ 時間となり，満点が得られる．

なお，証明は割愛するが，上記のコードを次のように変更すると，$O(N)$ 時間解法となることが保証される．

```
for (i = 1; i <= N; i++) { G[i] = true; }
for (p = 1; p < N; p++) {
    if (G[p] == false) { continue; }        // この行を追加
    for (k = 2; (k - 1) * p <= Z[p]; k++) {
        G[k * p] = false;
    }
}
```

さらに，Z algorithm を用いない，$O(N)$ 時間の別解も存在する．ある文字列 x が良い文字列でないことと，$|x|/\mathrm{per}(x)$ が 2 以上の整数であることは同値である (後述の補題 3 を参照)．すなわち，w の各接頭辞が良い文字列かどうか判定するためには，w の各接頭辞の最小周期を求めれば良いことが分かる．これは Knuth-Morris-Pratt 法 [3] を用いることで，$O(N)$ 時間で求められることが知られている．

6.1 正当性の証明

前述の解法は，(c) の場合において，最良表現の項数が 2 となることを基にしていた．ここではその証明を述べる．

定理 2 ([1,3] 等を参照)**.** 正整数 p, q が文字列 x の周期であり，かつ $p+q-\gcd(p,q) \leq |x|$ を満たすならば，$\gcd(p,q)$ もまた x の周期である．

補題 3. x を空でない文字列とする．以下の 2 つは同値である．

(i) x は良い文字列ではない．
(ii) $|x|/\mathrm{per}(x)$ は 2 以上の整数である．

証明. 良い文字列の定義より，(ii) ならば (i) であることは明らかである．以下，(i) ならば (ii) であることを示す．

x が良い文字列でない場合，$|x|/\mathrm{per}(x) \geq 2$ は定義より明らかである．次に，$|x|/\mathrm{per}(x)$ が整数となることを示す．x が良い文字列でないということは，y を k 回繰り返した文字列が x となるような，文字列 y および整数 $k \geq 2$ が存在する．ここで $p = \mathrm{per}(x)$, $q = |y|$ とおくと，$p \leq q = |x|/k \leq |x|/2$ が成立する．p および q はともに x の周期であり，かつ $p+q-\gcd(p,q) \leq |x|$ を満たすから，定理 2 により $\gcd(p,q)$ は x の周期となる．ここで $|x|/\mathrm{per}(x)$ が整数でないと仮定すると，q は p の倍数でないことになる．その場合，$\gcd(p,q) < p$ となってしまうので，$p = \mathrm{per}(x)$ が x の最小周期であることに矛盾する．よって，$|x|/\mathrm{per}(x)$ は整数である． □

補題 4. x を長さ 2 以上の文字列とする．また $m = |x|$ とする．さらに $y = x[1..m-1]$ とおく．x が良い文字列でなく，かつ $\mathrm{per}(x) \neq 1$ であるならば，y は良い文字列である．

証明. y が良い文字列でないと仮定する．$p = \mathrm{per}(x)$, $q = \mathrm{per}(y)$ とおく．補題 3 と仮定より，p は m の約数であり，q は $|y| = m-1$ の約数ということになる．一般に m と $m-1$ は互いに素なので，p と q も互いに素，すなわち $\gcd(p,q) = 1$ となる．さらに $p \leq m/2$ および $q \leq (m-1)/2$ である．ここで，p は y の周期でもある．したがって，定理 2 より，$\gcd(p,q) = 1$ は y の周期となる．よって，x の末尾 $m-1$ 文字はすべて同じ文字となってしまい，さらに $x[0] = x[p]$ より，$x[0]$ もまた同じ文字となってしまう．結局 $\mathrm{per}(x) = 1$ となり，前提に矛盾する．したがって，y が良い文字列でないという仮定は誤りであることがわかる．ゆえに y は良い文字列である． □

定理 5. 文字列 w について，w が良い文字列ではなく，かつ $\mathrm{per}(w) \neq 1$ を満たすと仮定する．このとき，w の最良表現の項数は 2 である．

証明. 長さ 1 の文字列は明らかに良い文字列である．さらに，補題 4 より，$w[1..|w|-1]$ は良い文字列であるから，結局，列 $(w[0], w[1..|w|-1])$ は w の良い表現の一つである．また，項数 1 以下の w の良い表現が存在しないことは明らかである．よって，w の最良表現の項数は 2 である． □

参考文献

[1] N. J. Fine and H. S. Wilf. Uniqueness theorems for periodic functions. *Proc. Amer. Math. Soc.*, 16:109–114, 1965.

[2] D. Gusfield. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.

[3] D. E. Knuth, J. H. Morris, and V. R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.

ARC 060 / ABC 044 Editorial

climpet

August 28th, 2016

A: Tak and Hotels (ABC Edit)

If $N \leq K$, the answer is NX. Otherwise the answer is $KX + (N-K)Y$.

B: Beautiful Strings

For each lowercase letter c from 'a' to 'z', count the number of occurrences of c in s, and check if the number is even.

C: Tak and Cards

This problem can be solved by dp.

Let $dp[i][j][k]$ be the number of ways to choose j numbers among the first i numbers such that the sum becomes k. You can update this array from smaller i, and when $i > 0$, $dp[i][j][k] = dp[i-1][j][k] + dp[i-1][j-1][k-x_i]$. (Make sure that you don't access to negative indices).

Then, the answer is the sum of $dp[N][t][At]$ for $0 \leq t \leq N$.

Exercise: the algorithm above is $O(N^4)$. Can you improve it to $O(N^3)$?

D: Digit Sum

Consider two cases: $b \leq sqrt(n)$ or $b > sqrt(n)$.

The former case is easy. Just try all possible values of b.

In the latter case, when you write n as a base-b number, the number of digits will be at most two.

If you write n as a base-b number, you get $n = pb + q$ for some $p, q < b$. From the statement, you get $p + q = s$. By comparing these two equations, you get $n - s = (pb + q) - (p + q)$, which is equivalent to $(p-1)b = n - s$.

Thus, except for the special case $n - s$, b must be a divisor of $n - s$, and again you can try all possible divisors of $n - s$ as candidates for b.

The total complexity is $O(sqrt(n))$.

E: Tak and Hotels

Assume that $a_i < b_i$ (the other case is similar).

First, for each i, we compute $right(i)$: the maximum integer that satisfies $distance(i, right(i)) \leq L$. It means that if you start the hotel i, you can reach up to the hotel $right(i)$ in a single day. This can be computed using binary search.

In this task, you want to compute the minimum k such that $right^k(a_i) \geq b_i$.

For each i and t, you need to precompute the value of $right^{2^t}(i)$. Then, you can compute the answers for queries using binary search.

The total complexity is $O((N + Q) \log N)$.

F: Best Representation

The key observation in this task is that, unless all characters in w are the same, the minimum number of elements is always 1 or 2.

Let n be the length of w. If neither of the first $n - 1$ characters of w and the last $n - 1$ characters of w are good, in other words, if both the prefix of length $n - 1$ and the suffix of length $n - 1$ are periodic, we can prove that all characters in w are identical.

The key lemma for the proof is the following:

- Let p, q are coprime integers, and let s be a string of legnth at least $p + q - 1$. If both p and q are periods of s, all characters in s are identical.

The remaining part of the proof is an exercise for readers.

Thus, if we can get all good prefixes and good suffixes of w, we can solve the task.

In order to check if a given string s is good, for each divisor d of $len(s)$, you need to check if $s[0, len(s) - d)$ and $s[d, len(s))$ are the same. You can compare two strings in $O(1)$ time after you pre-compute rolling hashes, so in total this problem can be solved in $O(nlogn)$.

Another way to compute all good prefixes/suffixes is to use Z-algorithm (again, an exercise for readers).

ABC043 / ARC059 解説

evima & sigma425

A - キャンディーとN人の子供イージー / Children and Candies (ABC Edit)

ループを回して1+2+..+Nを計算すればよい。また、これはN*(N+1)/2に等しいのでこれを出力してもよい。

B - バイナリハックイージー / Unhappy Hacking (ABC Edit)

素直に、キーが押されるたびに文字列の変化をシミュレートすれば解くことができます。

プログラミング言語に用意された文字列関連のライブラリを用いると簡潔に実装できるでしょう。多くの主要なプログラミング言語では文字列を演算子 "+" で連結でき、文字列の末尾に文字 '0' や '1' を挿入する処理はこれで済みます。文字列の末尾の 1 文字を削除する処理では、言語によって大きく違いが出るかもしれません。文字列から文字を直接消すのが難しければ、文字列から最後の 1 文字を除いた部分文字列をとって、それで元の文字列を上書きするという方法も考えられます。(この方法は文字列が長いと時間がかかるおそれがありますが、今回は問題ありません)

もしこのようなライブラリの使い方が分からなければ、文字の配列を文字列に見立てるなどして文字列の処理を「自前で」実装することもできます。その場合、現在の文字列の長さを保持する変数を用意すると見通しがよくなります。

C - いっしょ / Be Together

全て同じ整数に変えるのだが、変える先の整数 X を全探索してそのうちコストが最小のものを出力する。この時-100 から 100 までの整数しか考える必要はない。(-100 未満や 100 より大きくしても、各$(a_i-X)^2$ は X=-100 や 100 の時と比べ大きくなるだけなので、無駄。) 計算量は O(N*座標幅)。

D - アンバランス / Unbalanced

以下、文字列 s の長さを n とします。s には、長さ 2 以上の部分文字列が n(n-1)/2 個存在します。

部分点のデータセットでは n が最大で 100 と小さいため、s の部分文字列をすべて列挙することができます。それぞれの部分文字列について、各アルファベットの出現回数を数えることでアンバランスか判定すれば、時間計算量 $O(n^3)$ で問題を解くことができます。

満点を得るためには、n が 10^5 に達するケースにも正解する必要があります。この n の値では、すべての部分文字列を列挙する時間はありません。どうすればよいでしょうか？

実は、長さ 2 の "XX"（X は何らかの同じアルファベット）というパターンと、長さ 3 の "XYX"（Y は何らかのアルファベット）というパターンのいずれも s に存在しなければ、アンバランスな部分文字列は存在しません。なぜなら、もしこれらのパターンが存在しなければ、同じアルファベットの 2 つの文字の間に必ず他の文字が 2 つ以上挟まっていることになり、一種類のアルファベットが部分文字列の文字の過半数を占めることはないからです。よって、時間計算量 O(n) で問題を解くことができます。

E - キャンディーとN人の子供 / Children and Candies

まず $f(x_1,x_2,...,x_N)$が具体的にどのような意味のある値になるかを考える。幼稚園の活発度の式を考えると、N人にC個のキャンディーを分配する というのは、変数 x_iたちの多項式でどの変数にいくら次数を割り当てるか(総次数はC) というものだと思える。するとfは、N変数C次の単項式を集めたもの になる。
(例: N=3,C=3として、見やすさのため $x=x_1,y=x_2,z=x_3$と置くと、
$f(x,y,z)=x^3+y^3+z^3+x^2y+x^2z+y^2x+y^2z+z^2x+z^2y+xyz$)
まずは部分点を考える。$x_1,x_2,...,x_N$が($A_i=B_i$として)与えられるので、$f(x_1,x_2,...,x_N)$を求めればよい。
NやCが大きいので全ての単項式を列挙することは出来ない。次のようなDPを考える。dp[i][j]=「x_1からx_iまでの変数を使って次数がjになる単項式の値の和」
先程の例を使うと、

i \ j	0	1	2	3
0	1	0	0	0
1	1	x	x^2	x^3
2	1	x+y	x^2+xy+y^2	$x^3+x^2y+xy^2+y^3$
3	1	x+y+z	$x^2+y^2+z^2+xy+xz+yz$	$x^3+y^3+z^3+x^2y+x^2z+y^2x+y^2z+z^2x+z^2y+xyz$

実際は式ではなくこの値を保持する。
dp[N][C]が答えとなる。
dpの計算は、まずdp[0][0]=1とし、遷移はdp[i+1][j]= sum {dp[i][j-k]* x_i^k} (k=0~j)となる。(つまり、変数x_iで次数をいくつ使うかで場合分け)
すると計算量は$O(NC^2)$となり、N=C=400なので間に合う。

$$\sum_{x_1=A_1}^{B_1}\sum_{x_2=A_2}^{B_2}\cdots\sum_{x_N=A_N}^{B_N} f(x_1,x_2,\ldots,x_N)$$

満点解法も同様のDPによる。
ΣΣ・・・Σf (上式を簡単のためこう書く) の値を考えるために、まず、x_iの次数がc_iである単項式Πx_i^{ci}に対しΣをとった値がどうなるかを考えると、$\Sigma\Sigma\cdots\Sigma\ (\Pi x_i^{ci}) = \Pi\ (\Sigma x_i^{ci})$になる(Πはi=1～N)。(例: $\Sigma\Sigma(x^2y) = (\Sigma x^2) * (\Sigma y)$) つまり、各変数ごとにわけて足しあわせてから積をとったものと同じになる。これは実際に積をとった式を展開すると元の単項式のΣの各項が出てくることからわかる。
すると結局、部分点のDPの式でx_iでk次消費した時にかけるべき係数を、x_i^kから、$A_i^k + (A_i+1)^k + \cdots + B_i^k$ に変えれば満点の場合でも答えが求まる。事前に、自然数の0~C乗の、$A_i.B_i$の上限(Xとおく)である400までの累積和を計算しておけばこ

の係数は O(1)で得られるので、この前計算が O(CX)で、DP が $O(NC^2)$で、結局 $O(CX + NC^2)$で答えが得られる。

F – バイナリハック / Unhappy Hacking

純粋な全探索を行うとしたら、探索中に保持する状態は「現在までにキーを押した回数」と「現在エディタに表示されている文字列」の組になるでしょう。しかし、「現在の文字列」を完全に保持する必要はなく、「文字列の長さ」と「そのうち何文字目まで s と一致するか」という 2 つの整数値の組を持てば十分です。この全探索アルゴリズムを動的計画法に変換することで時間計算量 $O(N^3)$ で問題が解け、部分点を得られます。

満点を得るには、状態の数をさらに減らす必要があります。s について考えると、s の中のそれぞれの文字が '0' であるか '1' であるかは問題の答えに関係しないことがわかります（N 回のキータイピングと最終的な文字列の関係を考えると、文字列の中のそれぞれの文字はどれか 1 回のキータイピングと結びついているからです）。すなわち、s の長さを M とおくと、'0', '1' のみからなる長さ M の文字列は 2^M 通り存在しますが、N 回キーを押してそれらの文字列のうちどれを得る方法の数も等しいです。

したがって、N 回キーを押して「何らかの」長さ M の文字列を得る方法の数を求めれば、それを 2^M で割ったものが s を得る方法の数、すなわち元の問題の答えとなります。部分点解法で保持する必要があった「現在の文字列の何文字目まで s と一致するか」という値が不要になり、時間計算量 $O(N^2)$ で問題を解くことができます。

なお、「mod 1000000007 の世界で数を 2 で割る」(以下、P = 1000000007 とします) には、$2a \equiv 1 \pmod P$ を満たす整数 a = 500000004（これを P を法とする 2 の逆元と呼びます）を掛ければよいです。一般に整数 n の逆元を求めるには、例えば n^{P-2} を繰り返し二乗法で求める方法があります（フェルマーの小定理より、$n \times n^{P-2} = n^{P-1} \equiv 1 \pmod P$ となります）。

ABC042/ARC058 解説

三上和馬 (kyuridenamida)/森田晃平 (yosupo)

A 問題 和風いろはちゃんイージー

文字列 A, B, C の中に、長さ 5 の文字列が 2 つ、長さ 7 の文字列が 1 つあるときに限り、五七五を作ることができる。

B 問題 文字列大好きいろはちゃんイージー

全ての文字列の長さが全て L であるという条件から、与えられる文字列を辞書順で昇順に並び替え、その順で結合すれば辞書順最小を達成することができる。

一般に、文字列の長さが異なる場合でも、2 つの文字列 a, b に対し a が b より小さいことを $a + b < b + a$ と定義し、それらに基づいて文字列をソートしたものを昇順に結合すれば辞書順最小を達成することができる。証明は省略します。

C 問題 こだわり者いろはちゃん

$\{1, 2, 3, 4, 5, 6, 7, 8, 9\} \notin \{D_1, D_2, ..., D_N\}$ であることが保証されているので、0 しか使えないようなケースは無く、必ず解は存在する。

仮に使える数字の種類が 1 通りだったとしても、それらを N の桁数より 1 つ多い数だけ並べた数を考えればそれが解の上限となるため、解の上限は $10N$ である。N,$N+1$,$N+2$,... について順に解の条件を満たすか確かめ、条件を満たすと分かった時点でその数を出力するという解法で間に合う。

全体の時間計算量は $O(N \log N)$ である。

D 問題 いろはちゃんとマス目

※逆元の計算方法について誤記があったので修正しました。申し訳ありません。(7/24)

左上の座標が $(0, 0)$、右下の座標が $(H - 1, W - 1)$ であるとする。

$B \leqq i \leqq W$ を満たす全ての i について、 $(0, 0)$,$(H - A - 1, i)$,$(H - A, i)$,$(H - 1, W - 1)$ を順に通る経路の個数を数え上げ、それらを足し合わせることで、過不足なく求める経路が数えられる。図 1 はこのことを説明している。

上記の考え方に基づいて考えると、下に y 回、右に x 回移動するような経路の個数を効率的に求める必要があるが、これは $C(x + y, x)$ に等しい ($C(n, r)$ は n 個のもの中から r 個選ぶ組み合わせの数を表す)。

$C(n, r) = \frac{n!}{r!(n-r)!}$ であるから、 $0 \leqq x \leqq H + W - 2$ に対して $x!(\mathrm{mod}\ 10^9 + 7)$ と $(x!)^{-1}(\mathrm{mod}\ 10^9 + 7)$ を予め $O(H + W)$ かけて計算してテーブルにしておくことで、各

$C(n,r)$ は $O(1)$ で求めることができる。10^9+7 は素数なので、以下が言える。

- $x \equiv 0 (\mathrm{mod}\ 10^9+7)$ でない全ての x に対し $x^{-1} (\mathrm{mod}\ 10^9+7)$ が存在する。
- フェルマーの小定理より $x^{-1} \equiv x^{10^9+5} (\mathrm{mod}\ 10^9+7)$ である。

したがって、階乗数の逆元 $(x!)^{-1} \equiv (x!)^{10^9+5} (\mathrm{mod}\ 10^9+7)$ を二分累乗法等の適切な方法で計算して予め求めておけば良い。二分累乗法のオーダーは $O(1)$ ではあるが定数倍が 30 倍程あることに注意せよ。ちなみに、求めたい階乗数の最大値を $M!$ として、最初にその逆元 $(M!)^{-1}$ を二分累乗法で求めた後、その数に M をかけると $((M-1)!)^{-1}$ が求まり、その数に $M-1$ をかけると $((M-2)!)^{-1}$ が求まり、...、というふうに計算していけばより高速に逆元テーブルを構築することができるが、今回その必要はない。

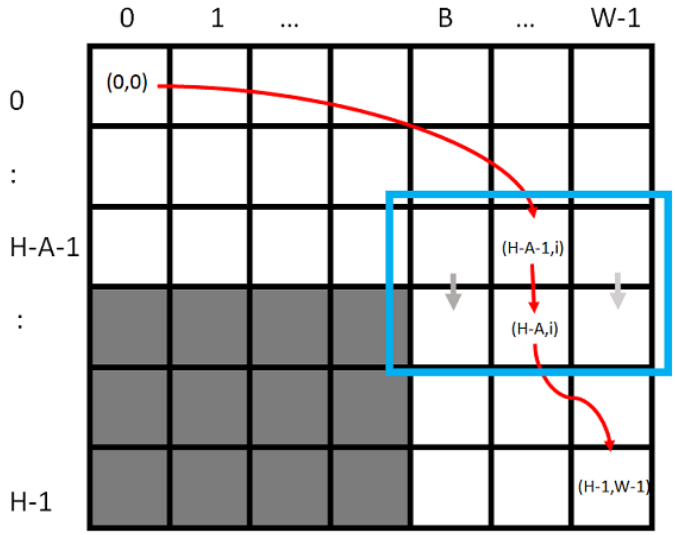


Fig1. (H,W,A,B)=(6,7,3,4)

E 問題 和風いろはちゃん

どこにも XYZ を含まない数列の個数を数え上げれば良い。もちろん XYZ を含む数列の数を数えても問題ないが、以下 XYZ を含まない数列の個数を数える方法を説明する。

左から 1 つずつ数列の値を決定していく全探索をメモ化することを考える。つまり、数列の今見ているところの値を XYZ が生じないように定め、今見ているところを 1 個右に移し、また値を決めて…というのを繰り返す。

これはどのようにメモ化出来るだろうか？

まず、XYZ=575 の時の、$1,1,...,1$(1 が 17 個) の場合が、数列の値を最も使うパターンである。つまり、直前 16 個の値を保存すれば良い。

これで状態数は $N * 10^{16}$ になる。全探索に比べたらかなりの改善だが、もちろん間に合わない。

もう少し考える。例えば直前に現れた値が 5,5,5,5,7 などであればこれを全部保存する必要はない。つまり、直前に現れた値を合計が 16 以下でとなる範囲でだけ保存すれば良いことがわかる。

これにより状態数は減るが、どのぐらい減るのだろうか？

ここで、1=”1”, 2=”10”, 3=”100”, 4=”1000”... と 0/1 文字列に変換して考えてみる。つまり、直前に現れた値が 1,2,3 だとすると”110100”と変換する。この文字列の長さは値の sum と等しい。つまり、前の値を合計が 16 以下まで保存するというのは、長さ 16 の 0/1 文字列に対応することがわかる。よって状態数は $N * 2^{16}$ 。もちろん実際には文字列で保存するのではなく、16 bit の整数として保存し bit DP を行う。

これにより計算量は $O(A * N * 2^{X+Y+Z})$ となる (A は数列の値の種類、つまり 10)。これで間に合う。

F 問題 文字列大好きいろはちゃん

dp[i][j] := i 番目以内の文字列を使い、ちょうど長さを j 文字にするときの辞書順最小の文字列

と定義する。ただし $i+1$ 文字目以降の文字列を使って埋めても $K-j$ 文字にできないときは定義されないこととする。$K-j$ 文字にできるかどうかの判定をするための DP テーブルを構築するのは容易である。

文字列をそのまま持つと空間計算量は $O(NK^2)$ だが、最適な文字列のみをうまく保持することを考えると、dp[i][0], dp[i][1], ..., dp[i][K] はそれぞれ「定義されない」or「ある共通の文字列の prefix」のどちらかになる。つまり、各 i に対してある共通文字列 $\mathrm{CS_i}$ (長さは高々 K) と、dp[i][j] が定義されているかどうかの判定フラグを持つことで、空間計算量が $O(NK)$ に抑えられる。

各 i に対する共通文字列について、それ自体が辞書順最小となるような dp[i][j] と判定フラグを保持することだけを考えれば良い。

dp テーブルの更新時に、dp テーブルの文字列や s_i、またはそれらを結合したもの同士の文字列の辞書順比較を高速に行う必要が出てくるが、LCP (Longest Common Prefix) を求めればその処理が高速に行えることが分かる。扱う文字列を全てローリングハッシュテーブルで管理し部分文字列のハッシュ値を $O(1)$ で求められるようにしておき、それら対して二分探索を行うことにより、 時間計算量 $O(logk)$ で LCP を求めることができる。

全体の時間計算量は $O(NKlogK)$、空間計算量は $O(NK)$ となる。128 以下の 2 べきの数を法としたローリングハッシュでは通らないようにテストケースを工夫した。したがって除算を行う必要があるが、除算は重くその定数倍のせいで、ローリングハッシュを用いた解法を通すのは難しい。

そこで、比較部分のアルゴリズムをさらに改善することを考える。

どのような文字列の比較を行うか考える。すると

- $\mathrm{CS_i}$ のうち先頭何文字か
- $\mathrm{CS_i}$ のうち先頭何文字か + s_i

という文字列同士のみを比較することがわかる。ここで、$s_i + \mathrm{CS_i}$ について $Z-Algorithm$ を適用したテーブルを用意しておくと、これらの文字列同士の比較は O(1) で行うことが出来る。

よって時間計算量は $O(NK)$ となる。これが想定解である。

以下、上記の説明に基づいた dp テーブル更新の一例を示す。定義されている値のみ列挙している。比較パートについては触れていない。

$N = 5, K = 7$ のケースで、各 s_i は以下の通りとなっている。

- s_1=”aaaaaa”
- s_2=”abc”
- s_3=”xxxx”
- s_4=”ddd”
- s_5=”abcd”

$i = 0, 1, 2, 3, 4, 5$ における DP テーブルの更新は次のようになっている。

1. $i = 0$: dp[0][0]=””
2. $i = 1$: dp[1][0]=””
    - s_1 を使うと以降どう使っても長さ $K = 7$ の文字列を達成できないことから、dp[1][6] は定義されない。
3. i=2: dp[2][0]=””, dp[2][3]=”abc”
    - 共通文字列 $\mathrm{CS_2}$ =”abc”
4. $i = 3$: dp[3][0]=””, dp[3][3]=”abc”, dp[3][7]=”abcxxxx”
    - 共通文字列 $\mathrm{CS_3}$ =”abcxxxx”
    - dp[1][4]=”xxxx”としても、辞書順最小の共通文字列にはなりえないので dp[1][4] は定義されない。
5. $i = 4$: dp[4][0]=””, dp[4][3]=”abc”, dp[4][7]=”abcxxxx”
6. $i = 5$: dp[5][0]=””, dp[5][3]=”abc”, dp[5][7]=”abcabcd”
    - 共通文字列 $\mathrm{CS_5}$ =”abcabcd”

ARC057 解説

DEGwer

2016 年 7 月 9 日

A 問題

問題概要: 高橋君は最初 A 円を持っている。一日経つごとに所持金が $K *$ (現在の所持金) $+1$ 円だけ増える。所持金が最初に 2 兆円以上になるのは何日後か。

解説:

まず、$K = 0$ のとき、所持金は毎日 1 円ずつ増えるので、求める日数は $2 \times 10^{12} - A$ 日です。

そうでない場合、日数を一日ずつすすめて問題文に書かれている通りにシミュレーションすることを考えます。このとき、1 日後の高橋君の所持金は 1 円以上であり、かつそれ以降高橋君の所持金は毎日 $K + 1$ 倍以上、特に 2 倍以上になるので、求める日数は高々 $\log_2 10^{12} + 1$ 日となり、愚直にループを回しても間に合います。($K = 0$ の場合は、所持金が指数的に増えないので愚直なループでは間に合わず、この場合のみ場合分けをしてやる必要があります。)

B 問題

問題概要: N 日間にわたって、i 日目には a_i 回のゲームをする。合計の勝利回数 K が与えられるので、前日までより勝率が上がった回数の最大値を求めよ。ただし、0 日目の勝率は 0 とする。

解説:

まず、$a_1 + a_2 + ... + a_N = K$ のときは、すべての日のすべてのゲームに勝つことになります。この場合、1 日目以降の勝率はすべて 1 であり、勝率が上がった回数は 1 です。この場合は特別に処理することにして、以下そうでない場合のみ考えます。

そうでない場合、勝利回数を K 回ちょうどでなく、K 回以下と言い換えても答えが変わらないことを証明します。

$L(< K < a_1 + ... + a_N)$ 回の勝利回数で t 回、前日までより勝率が上がったとします。このとき、勝利回数の列を後ろから順に見て、勝利回数が K 回に達するまで、その日のゲームの回数より勝利回数が真に少ない日の勝利回数を増やす操作を考えます。

この操作で勝利回数が増えた (前から見て) 最初の日を x 日目とします。$x+1$ 日目以降はすべてのゲームに勝っており、$x-1$ 日目までに (場合分けの条件より) 少なくとも 1 敗はしていることから、$x+1$ 日目以降のすべての日で勝率が上がっています。また、x 日目にはこの操作のせいで勝率が下がるようになることはなく、$x-1$ 日目以前の勝率の上下は、この操作では変わりません。よって、L 回の勝利で t 回勝率が上昇する状況が存在するなら、K 回の勝利で t 回以上勝率が上昇する状況も存在することが示されました。

ここで、以下のような DP を考えます。$DP[i][j]$ を i 日目までに j 回勝率が上昇した場合の勝利数の最小値とします。この DP の遷移は、i 日目の勝利回数として 0 から a_i までの a_i+1 通りが考えられますが、勝利数を最小化したいので、この中の 0 勝の場合と、勝率が上昇するぎりぎりの勝ち数の場合だけ遷移をすればいいことが分かります。

この DP を計算していき、$DP[N][i]$ が K 以下となる最大の i が求める答えとなります。

これで時間計算量は $O(N^2)$ となり、正答を得ることができます。実装によっては $a_1=1$ の場合がコーナーケースとなるので、気を付けましょう。

C 問題

問題概要: $\sqrt{N}$ の上 k 桁が $a_1a_2...a_k$ となるような最小の N を求めよ。

解説:

10 進法での整数 $a_1a_2...a_k$ を A と書くことにします。

$\sqrt{N}$ の上 k 桁が $a_1a_2...a_k \Leftrightarrow$ ある正とは限らない整数 t が存在し、$A\times10^t \leq \sqrt{N} < (A+1)\times10^t \Leftrightarrow$ ある正とは限らない整数 t が存在し、$A^2\times10^{2t} \leq N < (A+1)^2\times10^{2t}$

です。この条件を満たす N は、t が大きいほど桁数が大きくなるので、t を小さいほうから順に試し、$A^2\times10^{2t} \leq N < (A+1)^2\times10^{2t}$ を満たす N が見つかったらそれを出力して終了するアルゴリズムで、正解を得ることができます。

実装では、多倍長整数 A と $A+1$ の 2 乗をそれぞれ求め、適切に先頭に 0 を付け加えて偶数桁にしておき、さらに末尾に偶数個の 0 を付け加えたものを用意し、N の桁数 d を 2 桁ずつ増やしていって、A の 10 進表記の先頭 d 個の数字を並べてできる整数と、それに 1 を足したものを答えの候補とし、これらが条件を満たすかどうか調べます。はじめて見つかった条件を満たすものが、求める N となります。

増やす桁数が 2 桁ずつであることに気を付けてください。例えば、$\sqrt{1}$ の上のほうの位は 100... ですが、$\sqrt{10}$ の上のほうの位は 316... になります。

D 問題

問題概要: N 頂点で、各辺の重みが $1,2,...,N(N-1)/2$ の並び替えである完全無向グラフであって、最小全域木に使われる辺のコストが小さいほうから

順に $A_1, A_2, ..., A_{N-1}$ であるようなものの個数を求めよ。頂点同士は互いに区別する。

解説:

最小全域木は、クラスカル法で求めることができます。条件を満たすグラフにクラスカル法を適用して最小全域木を求めることを考えると、コストが小さいほうから順に辺を追加していったとき、連結成分同士がつながる $N-1$ 回のタイミングが、順に $A_1, A_2, ..., A_{N-1}$ であるということになります。

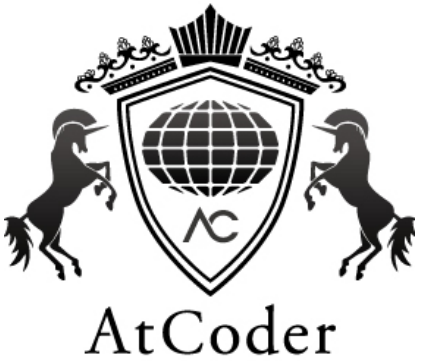
いま、条件を満たすグラフを、辺をコストが小さい順に追加していくことでつくることを考えます。$A_1, A_2, ..., A_{N-1}$ 回目の操作では、異なる連結成分の頂点同士をつなぎ、そうでない場合は同じ連結成分の頂点同士をつなぐように辺をはると、このとき、そしてこのときのみ、できるグラフは条件を満たします。

いま、連結成分同士をつなぐ操作を k 回やったとします。このとき、$N-k+1$ 個の連結成分がありますが、この連結成分のサイズたちの集合が同じなら、次以降に辺を張れる頂点対の個数は同じになることが分かります。よって、連結成分のサイズの集合をキーにした DP を考えることができます。

この DP の計算量を評価します。連結成分サイズたちの集合は、N をいくつかの整数の和で書く方法の場合の数と同じです。これは分割数と呼ばれる値で、$N = 30$ では 5604 となります。遷移は、異なる連結成分同士を結ぶときのみ $O(N^3 logN)$ の計算量でできるように実装できます。(結ぶ連結成分の組をすべてためします。)

以上で、$O((N \text{ の分割数}) \times N^3 \log N)$ でこの問題を解くことができます。

AtCoder Regular Contest 056
解説



AtCoder株式会社 代表取締役
高橋 直大

競技プログラミングを始める前に

AtCoder

- 競技プログラミングをやったことがない人へ
  - まずはこっちのスライドを見よう！
  - http://www.slideshare.net/chokudai/abc004

- 作題・準備:hogloid

A問題　みんなでワイワイみかん

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

- みかんを少なくともK個買いたい
- みかんは1個A円、またはL個B円で買うことができる
- 払うお金の最小値は？
- 制約
  - $1 \leqq A, B, K \leqq 10^9$
  - $2 \leqq L \leqq 10^9$
  - $B \leqq A*L$

A問題　アルゴリズム

AtCoder

- B ≦A*L の条件より、1個ずつA円でL個買うならセットで買ったほうがお得
- よって、1個ずつ買うのはL個未満
- このことから、考えられるパターンは
  - K/L(切り捨て)回L個のセットを買う+K mod L回1個ずつ買う
  - K/L+1回L個のセットを買い L - K mod L 個余らせる
- のどちらか

B問題　駐車場

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

B問題　問題概要

AtCoder

- N頂点M辺の無向グラフが与えられ、ある始点Sが定まっている
- 頂点1からNまで順番に、
  - 始点から頂点iまで到達できるとき、頂点iを削除
  - 到達できないとき、何もしない
- 最終的に削除される頂点の番号を昇順に出力
- 制約
  - $1 \leqq N, M \leqq 200000$

B問題　アルゴリズム

AtCoder

- 部分点:
  - 毎回問題文通りの操作を実装すれば大丈夫
  - 必要な道具は、
    - 2頂点が辺でつながっているか判定
      - UnionFind/dfsなどを用いましょう
    - 頂点を削除
      - 有効かどうかの状態を頂点ごとに持っておきましょう
- O(MN)

B問題　アルゴリズム

AtCoder

- 始点からある頂点まで到達できない場合、何もしない、とあるが、始点から到達できない以上、削除してもそれ以後の答えに影響ない
- すなわち、以下のような問題と考えることができる
- 頂点1からNまで順番に、
  - 始点から頂点iまで到達できるとき、答えにiを追加
  - 頂点iを削除
- 頂点iまで到達できるか判定しているとき、番号i以上の頂点のみ残っている
- すなわち、i以上の頂点のみを通り始点から頂点iまで到達できる ことと、iが答えに含まれることは等価

B問題　アルゴリズム

AtCoder

- Sからある頂点iまでのパスのうち、通る頂点番号最小のものの最大値を$cost_i$とおく
  - すなわち、$cost_i$より大きい番号の頂点のみ使ってiに到達することはできないような数のうち最小のもの
- これは、ダイクストラのように$cost_i$が大きい頂点から決めていけばOK
  - $cost_i$は$cost_j \leqq cost_i$なる頂点jを使ってさらに小さくなることはないため、大きい頂点から決めていくことができる
- O(MlogN)

C問題　部門分け

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

C問題　問題概要

AtCoder

- N*Nの重み付き無向グラフが与えられる
- 頂点を彩色する
- 使われた色の数*K-異なる色の間の辺の重みの総和 をスコアとするとき、スコアの最大値はいくらか
- 制約
  - $1 \leqq w_{i,j} \leqq 100000$
  - $1 \leqq N \leqq 17$
  - $1 \leqq K \leqq 100000$

C問題　アルゴリズム

AtCoder

- 部分点:
  - 色は1以上N以下としていいので、頂点の彩色を全て試していく
  - 愚直に試すとO(N^N)となってしまうので、少し工夫
  - 頂点を番号が増える順番で塗っていくことにする
  - 頂点iを今までに使われていない色で塗るとき、1種類だけ試せば十分
    - それ以後の塗り分けの色を交換することで、同じスコアを達成できるから
  - これで、i番目の頂点での分岐が高々i個となり、塗り方は高々O(N!)通り

C問題　アルゴリズム

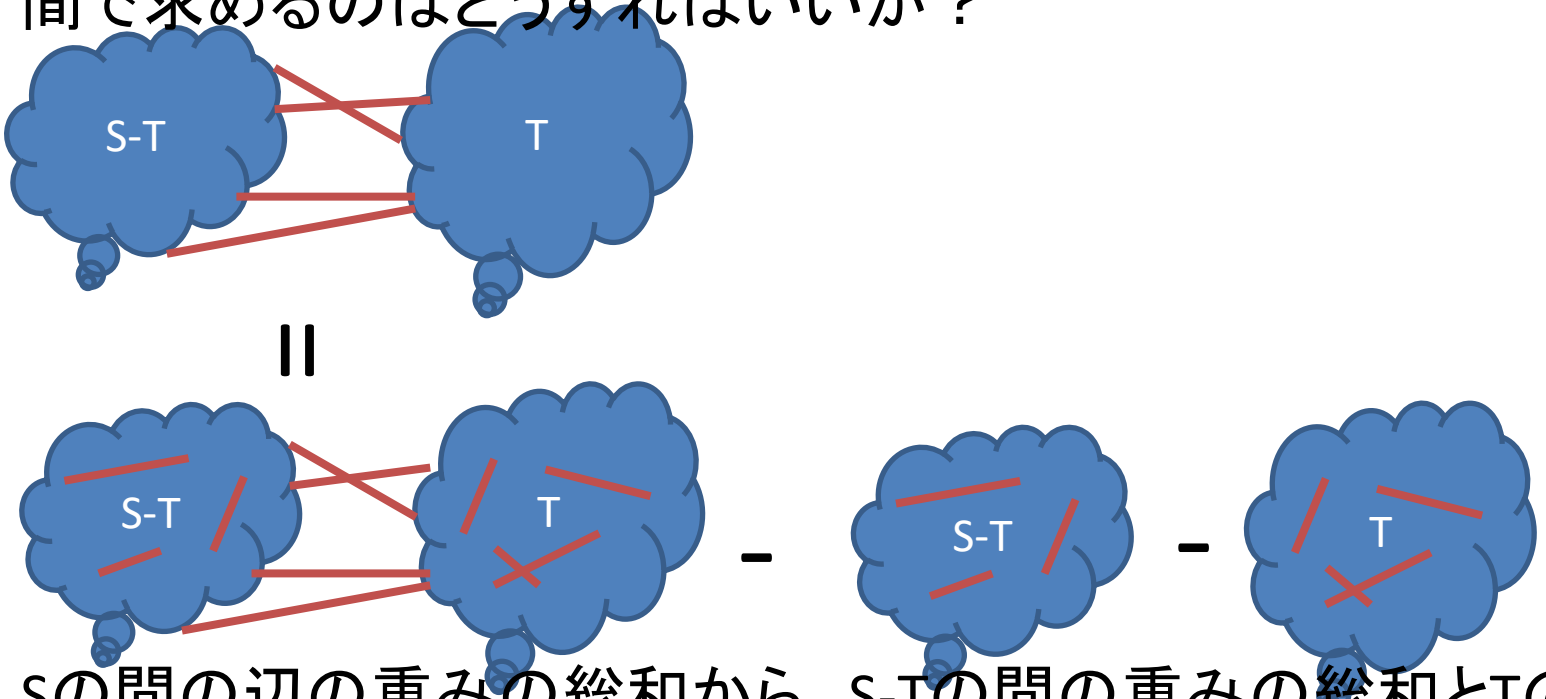
AtCoder

- dp[S]:=頂点集合Sのみに対する問題の答え、というDPを考えてみよう
- Sのうち、ある1つの色で塗られる集合Tを固定すると、そのスコアは
  dp[S-T]+K-(S-TとTの間の辺の重みの総和) となる
  - S-TはSのうちTに含まれない頂点
  - Tの頂点たちは他の色と異なる色で塗られることが決まったので、残りのS-Tのスコアは独立に決めることができる
- この計算量はO(3^N)
  - 全ての頂点集合(S)に対し、その全ての部分集合(T)を試す、ということは、N個の頂点をT、S-T、残りに分ける方法を試していることと等価
    - 後者はN個の頂点を3つに分けているので、3^N

C問題　アルゴリズム

AtCoder

- DPの遷移のうち、(S-TとTの間の辺の重みの総和) を定数時間で求めるのはどうすればいいか？



- Sの間の辺の重みの総和から、S-Tの間の重みの総和とTの間の重みの総和を引けばOK
  - ある部分集合の中の重みの総和は予め計算しておく(O(2^N))

D問題　サケノミ

1. 問題概要
2. 考察
3. アルゴリズム

©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

- N種類のドリンクとN種類のグラスがある
- それぞれのドリンクiは、対応するグラスiにM_i回決まった時間に補充される
- ドリンクには美味しさが決まっている
- 行える行動は、好きな時刻に注がれているドリンクを全て飲むことのみ
- 飲んだ美味しさの総和の最大値を求めよ
- 制約
  - $1 \leqq N \leqq 500{,}000$
  - $0 \leqq \sum M_i \leqq 500{,}000$

D問題　考察

AtCoder

- 部分点解法:
  - dp[i]:=時刻iで最後にドリンクを飲み干したときの美味しさの最大値、というDPを考えてみよう
  - 次にドリンクを飲み干す時刻jを全て試し、次に飲み干すときの美味しさの総和$d_{i,j}$を求め、dp[j]:=max(dp[j],dp[i]+$d_{i,j}$)と更新していく
    - i以後でそれぞれのドリンクが最初に補充される時刻を最初に計算し、jがその時刻を超えるごとに$d_{i,j}$に加えていくと上手くいく
  - O(N*max(t))

D問題　考察

AtCoder

- 満点解法
  - 上記のDPが高速にならないか考えてみる
  - 配るDPだったのを、もらうDPに書き直す
  - すなわち、iを昇順に、その中でjをj<iの範囲で昇順に走査し、dp[i]:=max(dp[i],dp[j]+$d_{j,i}$) で更新していく
  - dp[j]+$d_{j,i}$を1つの項として見てみる(dp2[j]と呼ぼう)
  - 時刻iで美味しさw_kのドリンクkが注がれ、ドリンクkがその前に注がれた時刻をlと置くと、l<j<iなるjに対し、dp2[j]:=dp2[j]+w_k となる
    - 最後に飲んだ時刻がjで、次に飲む時刻がi以後のとき、ドリンクkを新たに飲むことになるから
  - これは単純な区間add

D問題　考察

AtCoder

- dp2を使うと、dp[i]=max{dp2[j]|j<i}
  - 単純な区間max
    - 実際は、iを昇順に見ていくので、初期値を負の大きな数にしておけば、全体のmaxを取ってもOK
- よって、区間add区間maxの機能をもつセグメント木などを実装すればいいことが分かる
- O((N+∑M)log(max(t))

ARC055 解説

sigma425

A: 数え上げ

10^N + 7 を出力する問題。N が 1 以上なので、1(0 を N-1 個並べる)7 を出力すれば良い。

B: せんべい

最大のせんべいを食べる確率を最大化する問題。i 枚目を見るとき、そのせんべいの 1~N の id がどれかを考えなくても、「今何枚食べたか」と、そのせんべいが「現時点での max かどうか」しか食べるかどうかの判断には関係がなく、その確率は常に 1/i である。従って次のような dp が考えられる。dp[i][j][b] := 既に i 枚みて、j 枚食べていて、b=「現時点での max を食べているか」(bool 値) の時の最終的に条件を満たせる確率の最大値
遷移は、状態(i,j,b)の時,「確率(i-1)/i で(max を更新しないので)食べない」と「確率 1/i で max が出てくるので,この時食べる(dp[i+1][j+1][1]) or 食べない(dp[i+1][j][0]) のうち良い方を選ぶ 」のようにすればできる。O(NK)

(オマケ) N,K<=100000 でも解けます。(ただし,許容誤差に依存する解法です)

C: ABCAC

文字列 S を,非空文字列 A,B,C を使って S=ABCAC と表す方法は何通りか という問題。N=|S|とする。まず、A,B,C のうちどれか二つを全探索するなどの解法では、ちゃんと分割できているかの check に O(N)かけると O(N^3)になって、部分点も通らない(check を素早くやると部分点は取れるかもしれない)。そこで、ABC/AC の切れ目を全探索する。切れ目の左,右の文字列をそれぞれ x,y として、次を考える
a := x と y の prefix が何文字まで一致するか
c := x と y の suffix が何文字まで一致するか
A,C は非空なので、a=0 か c=0 だと ABCAC とは書けない。A,C が合計で|y|文字でその分割を試すのだが、a と c がわかっていると、そのような分割のうち何個が正当(つまり,x が A の prefix で y が A の suffix になっている)かがわかる。
実際、a,c>0 で a+c >= |y|とすると、a+c- |y| + 1 通りの分け方がある。
(a+c < |y| なら正当な分割はない)

例:
x=”abdaaabc”, y=”ababc” とする
a=2 (“ab”まで一致)
c=3 (“abc”まで一致)
この時各 y の分割は
a/babc ダメ(a>=1 と c>=4 を要求している)
ab/abc OK (a>=2 と c>=3 を要求している)
aba/bc ダメ(a>=3 と c>=2 を要求している)
abab/c ダメ(a>=4 と c>=1 を要求している)
となる.()で書いている条件を考えると OK の個数は a+c-|y|+1 になることがわかる.

a,c を愚直に求めることで、全体で O(N^2)なので部分点が得られる。
a は、S と S の suffix たちの先頭何文字が一致しているかがわかっていればすぐ求められる。c も同様で S を reverse したものを S'とおくと,S'と S'の suffix たちの先頭何文字が一致しているかがわかればすぐ求まる。
これにはいろいろな求め方があるが、
・Suffix Array と lcp 配列と segtree を使う
・Suffix Array と ローリングハッシュで二分探索をする
・Z-algorithm というものを使う
などがある。

これらを使うことで O(Nlog^2N)から O(N)で解くことが出来る。

D: 隠された等差数列

与えられた数字列を、あるどこかの桁としてもつような等差数列の内で初項の最小を求める問題。

i)d_i たちが x 桁目に対応しているとする。

まず、{d_(i+1) – d_i}は i によらず高々二種類の値しか取れないことに注意する(∵x-1 桁目から繰り上がりが来るか、来ないかで二通り以外ありえない。しかも、その二つの値は mod 10 で差が 1 である。)

なので、この条件を満たしていなければ-1 を返せば良い。

また、一通りしか無い場合は A=d_i,B=0 が明らかにベストなので、d_i を返せば良い。

以下、上の条件を満たすとする。

この時、A,B は高々 x 桁として良い(x+1 桁以上は d_i に影響を及ぼさないので、無駄に大きくする必要はない)ことに注意すると、

a_i = A + B*i (0<=i<N) に対し、L_i :=「a_i の x 桁目より左の部分(a_i を 10^x で割った商)」が一意に求められる(d_(i+1)-d_(i) >0 の時は繰り上がっていなくて、<0 のときは繰り上がっている。=0 の時だけ判断が難しいが、x 桁目に足されている値が二通りのうちのどちらかがわかるので、x 桁目で繰り上がってるかどうか分かる)

(例:

d={1,1,2}なら 1->1 で x 桁目での繰り上がりは起こっていない(15,18,21 みたいに)

d={1,1,0}なら、1->1 で x 桁目での繰り上がりは起こっている(11,110,209 のように)

)

以上より、次のような問題になる。

A,B:x 桁の非負整数で次のような条件をみたすうち A の最小を求めよ

L_i * 10^x <= A+B*i < (L_i + 1) * 10^x

よく考えると、L_i を求める(あるいは求めるのに失敗する)のに x の値は関係がない。

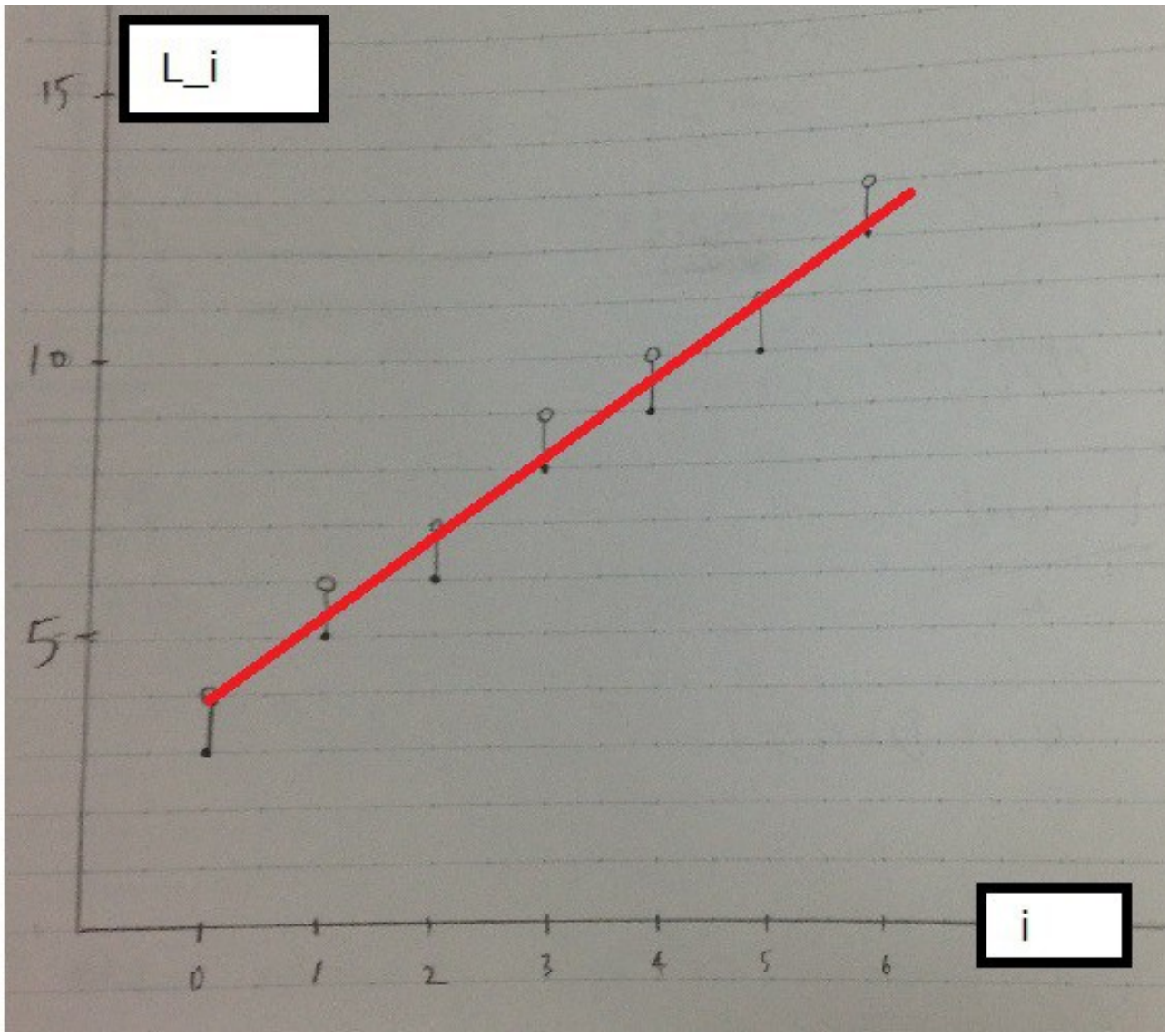
さらに、この不等式全体を 10^x で割ると、同値な次の問題に変形できる。

A',B':高々小数点 x 桁(つまり,(整数)/10^x とかける)の非負実数で次の条件を満たす内 A'の最小を求めよ

L_i <= A' + B' * I < L_i + 1

すると、次のように、「線分たちが与えられて、L_i = A' + B' * i という直線がその線分たちの間を通る」という条件に言い換えられる

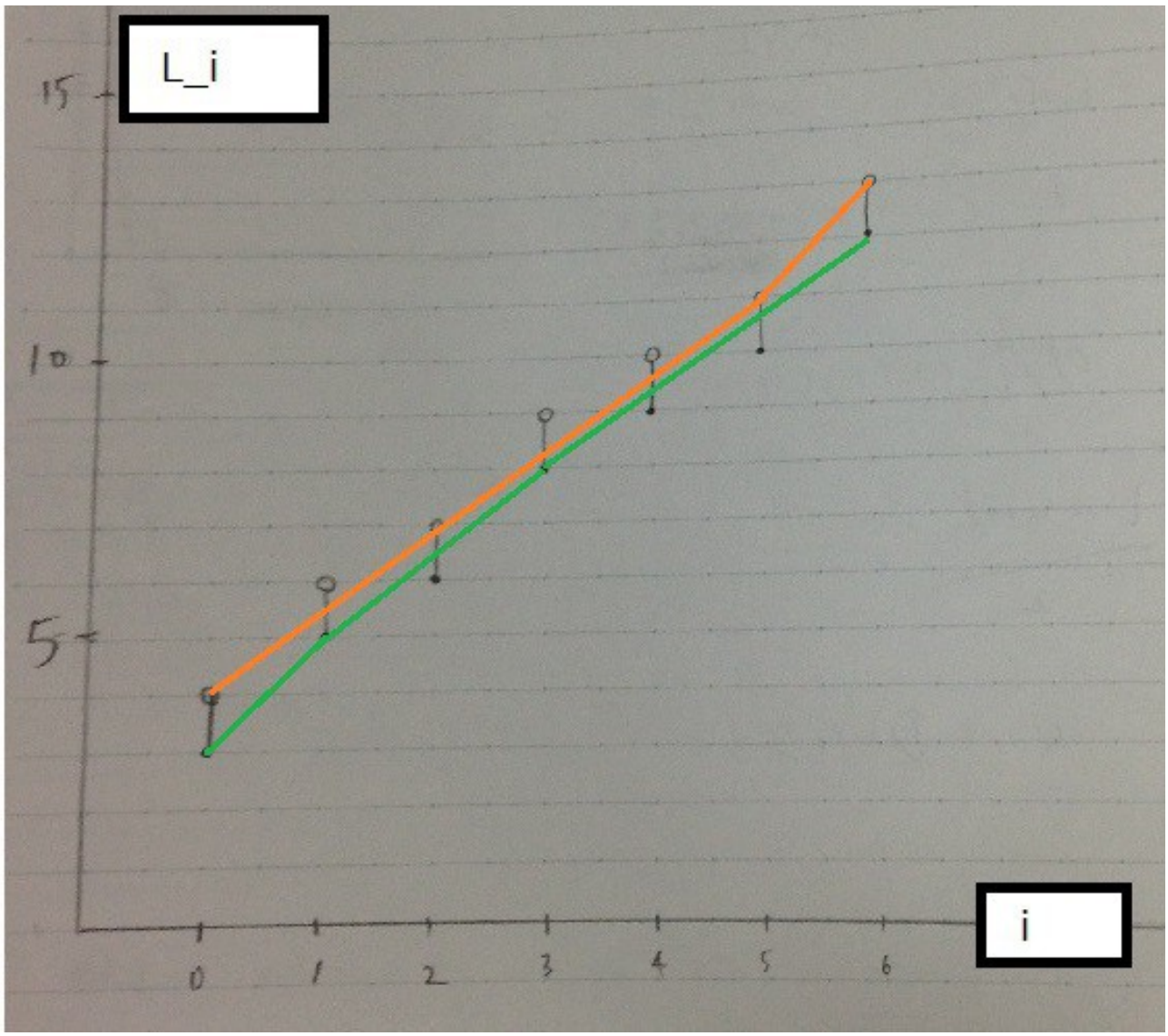
例えば,d={3,5,6,8,9,0,2} とすると,



このように赤い直線が線分たちの間を通るのと条件をみたすのが同値。
この図は x によらずに書ける。x に関する条件は、「A'も B'も 整数/10^x」すなわち,「L_0 も傾きも整数/10^x」という条件になる。

B'を整数/10^x で固定すると、A'を最小化するには各線分の黒い点のどれかにぶつかるまで直線を下に動かせば良い。この時自動的に A'は整数/10^x となるので、A'に関する条件はないものとしてよい。なので、「B'としてありうる値の範囲」を求めて、x を 1 から順番に調べていき、その範囲に B'が整数/10^x で表せるものがあれば、そのうちで A'が最小になるものを答えとする(d_0 ≠0 より、10^x <=A <10^(x+1) となるから,これで A の最小が求まっている)。A'が最小⇔B'が最大 が示せるので、あとは B'の範囲さえわかれば解ける。

B'の範囲を(有理数範囲で)求めるには、次のことをする。
まず上側の点の下向きの凸包を取り,同様に下側の点の上向きの凸包を取る(この凸包に含まれない点は無駄な条件である)



この前処理をしておくと、(以下凸包のサイズを K とする)
直線を下の凸包(緑線)にはわせながらしゃくとりのようなことをすると O(K)
上の凸包上の点と,下の凸包上の点を結んだ直線が valid か試す(これは結んだ点の高々左右 2 つとの傾きを比べれば O(1)でできる) をすると O(K^2)

K は座標幅が N の凸包のサイズなので高々 O(N ^ (2/3))であることが知られており、全体で O(N^(4/3))もしくは O(N)で解くことが出来る (実際には前者も十分高速)

ちなみに答えの最大サイズは N^2 ほどで、d= {9,0,0,.....0,1} などが答えが大きな値になります。

ARC054 解説

DEGwer

2016 年 5 月 21 日

A 問題

問題概要: 長さ L の円形の動く歩道があり、速度 X で回っている。高橋君が速度 Y で歩けるとき、高橋君が位置 S から位置 D まで行くのにかかる最小の時間を求めよ。

解説:

位置 S から位置 D まで動く歩道の進む方向に測った距離を K とします。これは、$S \leq D$ のときは $D - S$ に等しく、そうでないときは $D + L - S$ に等しいです。

X と Y の大小で場合分けをします。

$X < Y$ のとき、高橋君は動く歩道を順方向と逆方向のどちらに歩いても、前に進むことができます。順方向に進んだときにかかる時間は $\frac{K}{X+Y}$、逆方向に進んだときにかかる時間は $\frac{K}{Y-X}$ です。これらのうち小さいほうが答えです。

そうでないとき、高橋君は順方向に進んだときのみ前に進むことができます。このとき $\frac{K}{X+Y}$ の時間がかかり、これが答えです。

場合分けをしないと、$\frac{K}{Y-X}$ の計算でゼロ除算が発生したり、負の値が出てきたりするので注意してください。

B 問題

問題概要: コンピュータは 2 年に 1.5 倍のペースで高速になる。現在 P 年かかる計算を、適切なタイミングで始めて計算が終わるまでの時間を最小化せよ。

解説:

x 年後に計算を始めるとき、計算が終わるまでの時間は $f(x) = x + 2^{-x/1.5}P$ 年です。これは、凸関数と凸関数の和なので、凸関数です。よって、三分探索によって最小値を求めることができます。

また、$f'(x)$ の零点を求め、そのときの値を出力しても大丈夫です。その場合、零点が 0 未満のときは $f(0) = P$ を出力する必要があります。

C 問題

問題概要: 頂点を同じ集合の間に辺がないように 2 つに分けたとき、両側ともに頂点数が $N \leq 200$ となる二部グラフが与えられる。(以下この分割によってできた頂点集合を A, B と呼ぶ。) 完全マッチングの個数の偶奇を答えよ。
解説:

完全二部マッチングの個数の数え上げは、NP 困難な問題として知られています。しかし、その偶奇だけなら高速に求めることができます。

完全マッチングにおいて、A の i 番目の要素とマッチングされている要素を B の $\sigma(i)$ 番目の要素とすると、$\sigma(i)$ たちは 1 から N までの順列となります。つまり、求めたいのは、すべての順列 σ について、A の i 番目の要素が B の $\sigma(i)$ 番目の要素にマッチングされているものの個数を合計したものの偶奇です。

A の i 番目の頂点と B の j 番目の頂点の間の辺の数を (i, j) 成分とする行列 M(入力で与えられる行列) を考えます。上記の値は、

$$\sum_{\sigma \in S_N} M_{1\sigma(1)} \dots M_{N\sigma(N)}$$

の偶奇に等しいです。ただし、S_N は 1 から N までの順列すべての集合を表します。

さて、M の行列式は、

$$\sum_{\sigma \in S_N} sgn(\sigma) M_{1\sigma(1)} \dots M_{N\sigma(N)}$$

とあらわされます。$sgn(\sigma)$ は 1 または -1 の値をとりますが、これらは $mod 2$ ではどちらも 1 です。すなわち、これは $mod 2$ では先ほどの式と一致します。行列式を求めるのは $O(N^3)$ でできるので、$O(N^3)$ でこの問題を解くことができます。

D問題

問題概要: つぎの 3 種類の操作を繰り返してでできる数列のバブルソートの交換回数を求めよ。

- k のみからなる単項数列を作る
- 数列を k 回繰り返す
- 2 つの数列をつなげる

はじめの二つの操作の回数の和 (以下 N とおく) は 10^5 以下。
解説:

数列 X のバブルソートの交換回数は、$i < j$ かつ $X_i > X_j$ なる組 (i, j) の個数に等しいので、これを数えることにする。

現在存在する数列 S に対し、以下のデータを持っておきます。

- S のバブルソートの交換回数 X_S
- S にどの数が何回現れたかをすべて列挙したもの D_S
- S を 2 回繰り返したとき、バブルソートの交換回数が $2X_S$ に比べ何回多いかを表す Y_S

これを持っておけば、単項数列を作る操作は $O(1)$、数列を繰り返す操作も $O(1)$、2 つの数列 S_1 と S_2 をつなげる操作は $O(D_{S_1}$の要素数$+D_{S_2}$の要素数$)$ ででき、全体で $O(N^2)$ となり、部分点が得られます。

各 S に対し、D_S を適切なデータ構造で管理すれば、以下のようなアルゴリズムで満点を得ることができます。

- 項 k のみからなる単項数列 S を作る操作において、空の $Tree_S$ を作り、$X_S = 0, Y_S = 0$ としたうえで、D_S に値 k を 1 個追加する。
- 数列 S を k 回繰り返す操作において、新しい (X_S, Y_S) をそれぞれ $(kX_S + \frac{k(k-1)}{2}Y_S, k^2Y_S)$ とおき、さらに D_S に格納されている各要素の個数をすべて k 倍する。
- 数列 S と数列 T をつなげる操作 (この操作でできる数列を L とする) において、D_S と D_T のサイズを比べ、
  - $|D_S| < |D_T|$ ならば、D_S の要素をすべて見る。値 k が t 個あるとき、T の値 k 未満の要素の数を数えてそれを t 倍したものたちすべての和と、X_S と X_T との和が X_L となる。さらに、値 k が t 個あるとき、T の値 k 未満の要素と k より大きい要素の数を数えてそれを t 倍したものたちすべての和と、Y_S と Y_T との和が Y_L となる。その後、D_T に D_S の要素をすべて加え、それを D_L とする。
  - $|D_S| \geq |D_T|$ ならば、D_T の要素をすべて見る。値 k が t 個あるとき、S の値 k より大きい要素の数を数えてそれを t 倍したものたちすべての和と、X_S と X_T との和が X_L となる。さらに、値 k が t 個あるとき、S の値 k 未満の要素と k より大きい要素の数を数えてそれを t 倍したものたちすべての和と、Y_S と Y_T との和が Y_L となる。その後、D_S に D_T の要素をすべて加え、それを D_L とする。
- 最後にできた数列 S の X_S の値が答えである。

このアルゴリズムの正当性は以下の通りです。X_S が S のバブルソートの交換回数、Y_S が S を 2 回繰り返した数列において、1 個目の S から 1 個、2 個目の S から 1 個の要素をとってきたときにそれらの大小関係がひっくり返っているものの個数を表すことに注意します。

単項数列を作る操作において、$X_S = Y_S = 0$ は明らかです。数列 S を k 回繰り返した数列 (ここにおけるひとつの S をブロックと呼ぶことにする) を S^k とおくと、S^k での $S_i^k > S_j^k$ なる $i < j$ の個数は、

- i, j が同じブロックに属する場合、kX_S 個
- そうでない場合、$\frac{k(k-1)}{2}Y_S$ 個

となるので、この操作も正当です。

数列 S,T をつなげる操作において、S と T をつなげた数列 L での $L_i > L_j$ なる $i < j$ の個数は、

- i, j 番目の要素がともに S の要素であった場合 X_S 個
- i, j 番目の要素がともに T の要素であった場合 X_T 個
- どちらでもない場合、S の要素 S_i について、$S_i > T_j$ なる j の個数すべての和、もしくは T の要素 T_j について、$S_i > T_j$ なる i の個数すべての和 (これらは同じ値を表す)

であるので、上記のアルゴリズムは正当です。

さらに、数列をつなげる操作において、D_S のサイズが小さいほうの数列を大きいほうの数列にマージしているので、単項数列として作った各数列において、その値が D_S に新しく入れられる回数は高々$O(\log N)$ 回です。(次の列に入れられるまでに、自分が属する数列 S の D_S のサイズが 2 倍以上になります。)

よって、あとは「数列 S の k 以下の要素の個数を数える」「D_S に値を追加する、すなわちある箇所の値をある値だけ増やす」「D_S のすべての要素の個数を k 倍する」という操作ができればよいことになります。

最初の 2 つの操作は、各数列について segment tree を動的に構築することで可能になります。最後の操作は、実際に segment tree 上の値を更新はせず、各 segment tree についてかわりに「segment tree に書かれている値を何倍すると本当の値になるか」を表す値 P_S を持っておくことにします。すると、

- 数列 S の値 k 以下の要素の個数を求めるときは、segment tree 上での位置 k より前のものの合計に P_S をかけたものを求める
- 位置 k の値を t だけ増やすときは、segment tree 上での位置 k に tP_S^{-1} を加算する
- D_S のすべての要素を k 倍するときは、P_S を k 倍する

という操作を行うことで、全体で $O(N \log^2 N)$ となり、満点を得ることができます。

ARC 053 解説

A：ドミノ色塗り

解法

答えは $H(W-1)+(H-1)W$ 通り．

まず，左右に隣り合う 2 マスを塗る方法が何通りか考えます．左右に隣り合う 2 マスを塗るとき，左のマスとしてあり得るものは，図 1 の ☆ のマスです．左右に隣り合う 2 マスを塗る方法は ☆ の個数に等しいので，$H(W-1)$ 通りです．

同様にして，上下に隣り合う 2 マスを塗る方法は $(H-1)W$ 通りです．よって，答えは $H(W-1)+(H-1)W$ 通りとなります．

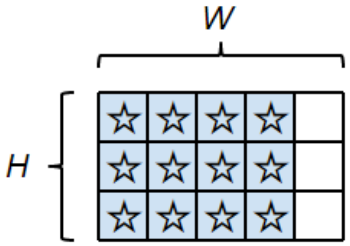


図 1

B：回文分割

解法

文字列 S の長さを N とする．また，26 種類のアルファベットのうち，文字列 S に奇数回現れるものが K 種類であるとする．すると，答えは次式．計算量は $\mathrm{O}(N)$．

$$\begin{cases} N & (K=0) \\ 2\lfloor \frac{N-K}{2K} \rfloor + 1 & (1 \leq K \leq 26) \end{cases}$$

$K = 0$ の場合，すべてのアルファベットが偶数個ずつあります．そのため，同じアルファベット同士のペアを $N/2$ 組作り，それらを左右対称に並べることで，1 個の回文を作ることができます．よって，答えは N です．

$K = 1$ の場合，1 種類のアルファベットのみが奇数個です．このアルファベットを中央に置くことに決めると，残り $N - 1$ 文字はすべてのアルファベットが偶数個ずつあります．そのため，同じアルファベット同士のペアを $(N - 1)/2$ 組作り，それらを左右対称に並べることで，1 個の回文を作ることができます．

$K = 2$ の場合，2 種類のアルファベットが奇数個です．この場合，少なくとも 2 個の回文に分割する必要があります．ここでは 2 個の回文に分割することにして，奇数個である 2 種類のアルファベットをそれぞれの中央に置くことに決めます．すると，残り $N - 2$ 文字はすべてのアルファベットが偶数個ずつあり，同じアルファベット同士のペアを $(N - 2)/2$ 組作ることができます．より短い方の回文の長さ X をできるだけ大きくしようとすると，ペアをできるだけ均等に割り振るのがよいことがわかります．よって，より短い方の回文には $\lfloor (N - 1)/4 \rfloor$ 組のペアが割り振られ，答えは $X = 2\lfloor (N - 1)/4 \rfloor + 1$ となります．

一般の $1 \leq K \leq 26$ の場合も，$K = 2$ と同様に考えると，答えは $2\lfloor (N - K)/2K \rfloor + 1$ となることがわかります．

C：魔法使い高橋君

解法

N 個の魔法を唱える順番を **貪欲法** で決める．$a_i < b_i$ の魔法をグループ 1 とし，$a_i \geq b_i$ の魔法をグループ 2 とする．まず，グループ 1 の魔法を a_i の昇順にソートし，その順番に唱える．続いて，グループ 2 の魔法を b_i の降順にソートし，その順番に唱える．このときの X が答えである．計算量は $\mathrm{O}(N \log N)$.

とりあえず，グループ 1 の要素だけしかない場合を考えてみます．このとき，a_i の昇順にソートした順番が最適解であることを示します．ある順番において，ある隣接要素 i，$i + 1$ が $a_i \geq a_{i+1}$ であったとします．すると，図 2 からわかるように，X の値を増加させずに i と $i + 1$ をスワップできます．よって，任意の順番は X の値を増加させずに a_i の昇順にソートできることがわかります．以上より，a_i の昇順にソートした順番が最適解であることが示せました．

同様にして，グループ 2 の要素だけしかない場合も，b_i の降順にソートした順番が最適解であることが示せます．

では，グループ 1 の要素とグループ 2 の要素が混ざっている場合はどうでしょうか？ ある順番において，ある隣接要素 i，$i + 1$ がそれぞれグループ 2，グループ 1 であったとします．すると，図 3 からわかるように，X の値を増加させずに i と $i + 1$ をスワップできます．よって，任意の

順番は X の値を増加させずに，(グループ 1 の要素たち) → (グループ 2 の要素たち) という順番に並べ替えられることがわかります．グループ 1 の要素とグループ 2 の要素が完全に分離されていれば，X の値を増加させずに，グループ 1 を a_i の昇順にソートし，グループ 2 を b_i の降順にソートできます．以上より，(グループ 1 を a_i の昇順にソートしたもの) → (グループ 2 を b_i の降順にソートしたもの) という順番が最適解であることが示せました．

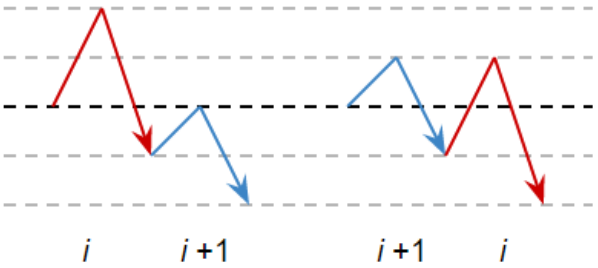


図 2

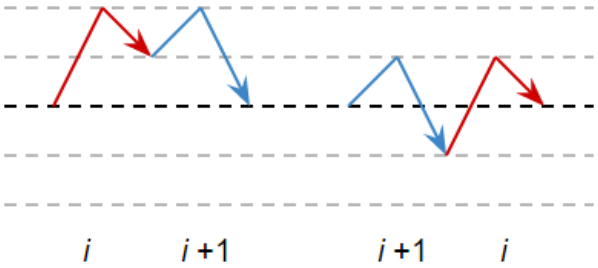


図 3

D : 2 つの山札

まずは，問題を視覚的に捉えましょう．例えば，サンプル 3 を視覚的に表すと，「図 4 のグリッドグラフにおいて，左上の点から右下の点まで移動するとき，パス上の数列は何通りか？」という問題になります．

グリッドグラフのパスの数え上げは，**動的計画法** (DP) の典型問題として有名です．この DP では，$dp[i][j] :=$ (点 (i, j) までのパスの個数) と定義して，$dp[i][j] = dp[i-1][j] + dp[i][j-1]$ という漸化式を計算します．もちろん，この DP ではパス上の数列を正しく数え上げられません．例えば図 4 では，数列 $(3, 3, 3, 3, 5, 5, 5, 5)$ を 2 重に数えてしまいます．これは，ある数列に対応する終点が複数個あり得ることが原因です．例えば図 5 では，数列 $(3, 3, 3, 3, 5, 5)$ に対応する終点が 2 個あります $((2, 4), (4, 2))$.

パス上の数列を正しく数え上げるためには，次のような工夫が必要です．ある数列に対応する終点が複数個あり得ることに対応するため，$dp[S] :=$ (あり得る終点の集合が S であるような数列の

個数) と定義します．例えば図 5 では，あり得る終点の集合が $\{(2,4),(4,2)\}$ であるような数列は $(3,3,3,3,5,5)$，$(3,3,4,4,4,4)$ の 2 通りなので，$dp[\{(2,4),(4,2)\}]=2$ となります．このように定義した $dp[S]$ をすべて計算した後，$dp[\{$ 右下の点 $\}]$ を参照すると正しい答えが求まります．

しかし，S をそのまま状態として持つと DP の状態数が大きくなりすぎるので，より効率的な状態の持ち方を考えなければなりません．実は，「数列の長さ」と「数列の末項」がペアで指定されると，対応する S は高々 4 通りしかないことがわかります．よって，$dp[l][p][S]:=$ (長さが l かつ末項が p の数列であって，あり得る終点の集合が S であるものの個数) と定義すると，DP の状態数が $\mathrm{O}(N^2)$ に抑えられます．さらに，各状態からの遷移も高々 4 通りしかないことがわかります．以上より，全体の計算量が $\mathrm{O}(N^2)$ に抑えられ，DP で問題を解くことができます．

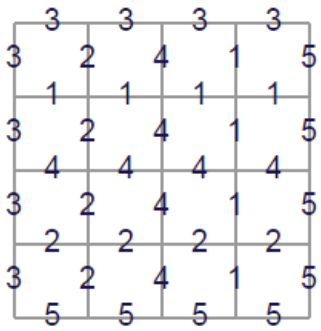


図 4

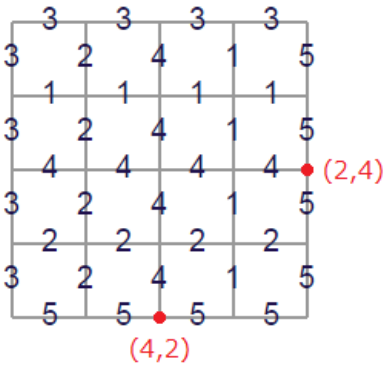


図 5

ARC052 A – 何期生？

@namonakiaccount

問題概要

- 3文字以上10文字以下の文字列が与えられる
- 1桁か2桁の数字が含まれているのでそれを出力せよ

解法

- 文字列の先頭から一文字ずつ見ていき、一番最初に数字が出てくるところに着目する
- この次の文字も数字なとき、答えは2桁

  このとき、この2文字をこの順で出力すればよい
- 違うとき、答えは1桁

  このとき、この1文字を出力すればよい

解法

- また、文字数≦10、桁数≦2より、数字の含まれる位置の組み合わせは高々19通りしかない
- これらを総当たりして求めてもよい

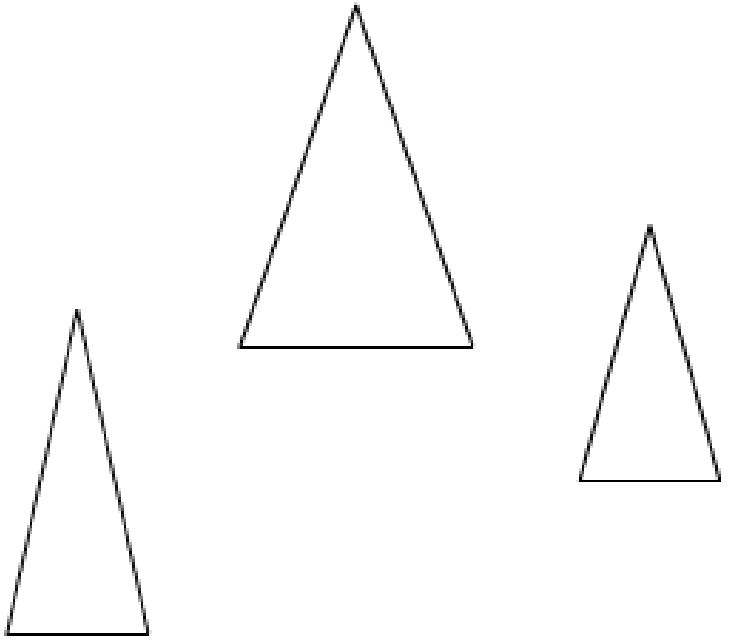
ARC052解説
B:円錐

B-問題概要

- 3次元空間上に様々な大きさの円錐が底面をyz平面に平行にして同じ向きに浮かんでいる
- 円錐は互いに重なり合わない
- A≦ x ≦ Bの範囲に含まれる部分の体積を求めよ

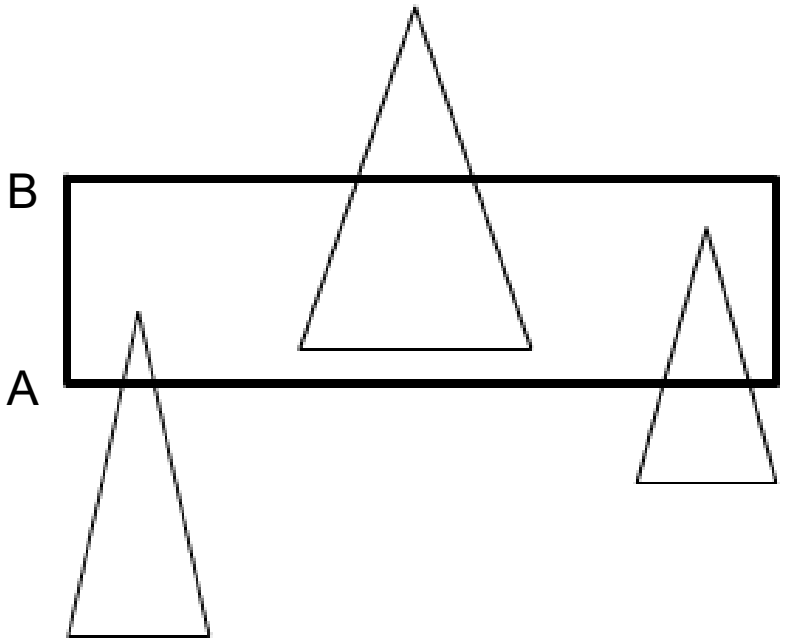
B-考察

- 簡単のために円錐ではなくて二等辺三角形で考えてみる



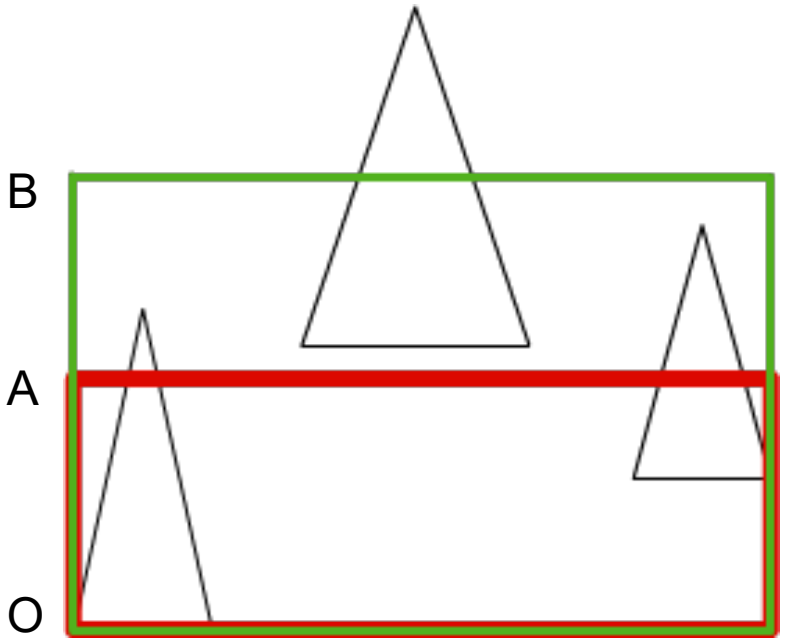
B-考察

- ある区間[A,B]のなにかしらの総和を求める時に使えるアイデア



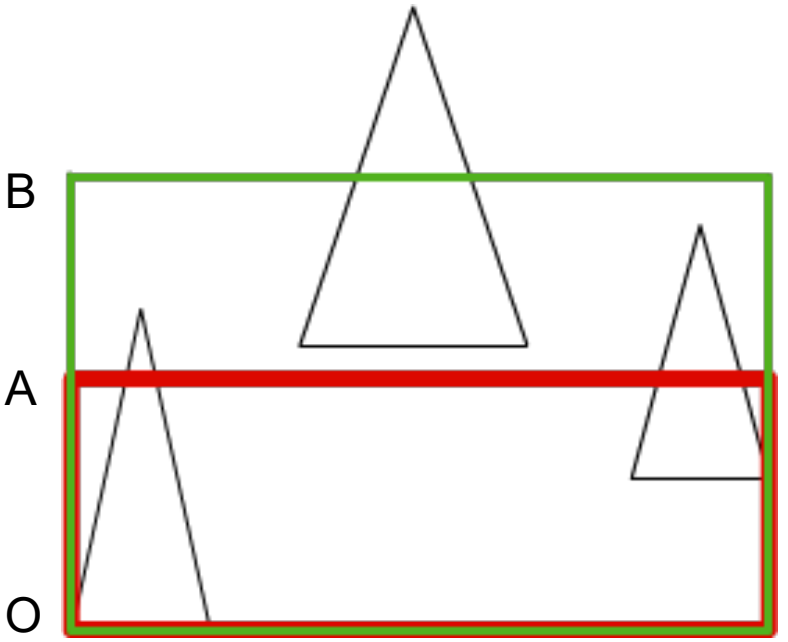
B-考察

- ある区間[A,B]のなにかしらの総和を求める時に使えるアイデア
  - ある基準点Oをつくって
    [O,B] – [O,A]をもとめる



B-考察

- ある区間[A,B]のなにかしらの総和を求める時に使えるアイデア
  - ある基準点Oをつくって[O,B] – [O,A]をもとめる
  - 今回なら[0,X]の範囲の面積(体積)を求めて置くと便利



B-解法

- 各X(0 ≦ X ≦ 10^5)について[X,X+1]に含まれる部分の体積をv[X]とする
  - これは円錐の情報が与えられるたびに、各v[X]の増加量を計算して加算していけば求まる
  - O(NH)
- 最後にvの累積和を取れば[0,X]に含まれる部分の体積を表すsum[X]を計算することが出来る
- クエリに対してはsum[B]-sum[A]を出力すれば良い。
- 満点獲得

B-別解

- クエリが来るたびに各円錐のうちその区間に含まれる部分の体積を求めて足し合わせれば良い
- O(NQ)
- こちらのほうがシンプルにコードを組めるだろう。

ARC052 C – 高橋くんと不思議な道

@namonakiaccount

問題概要

- N個の町があり、M本の双方向に移動可能な道で結ばれている。
- 道にはコストが1かかるタイプAと、(今まで通ったタイプBの道の本数+1)かかるタイプBがある。
- 町0からすべての町へのコストをそれぞれ求めよ
- $1 \leqq N \leqq 10000$
- $1 \leqq M \leqq 100000$

考察

- タイプBの道を何本通ったかに着目してみる
- (今いる町,今まで使ったタイプBの道)を頂点にしたグラフでのダイクストラ法を考えてみる
- 頂点数O(N^2) → 通らない

考察

- 町0から町vまで、タイプBの道をa本使って行けるルート、b本使っていけるルートがあるとする

  (ただしa<b)
- 町どうしの移動に通る道の本数は高々N-1本なので、

  b(b+1)/2-a(a+1)/2>N-1なら、b本使っていけるルートは無視してもよい

簡単な証明

- タイプBの道をa本使うルートも、b本使うルートも、タイプAの道は高々N-1本しか使わない
- よって(タイプBの道をb本使うことによるコスト)-(タイプAの道をa本使うことによるコスト)>N-1なら、タイプBの道をb本使うルートが最短路となることはない。

簡単な証明

- 他の頂点への伝播についても考えてみる
- 町vから町uまで、タイプBの道をk本使って行けるとすると、

  $b(b+1)/2-a(a+1)/2>N-1$のとき

  $(b+k)(b+k+1)/2-(a+k)(a+k+1)/2>N-1$

  が必ず成り立つ。
- タイプAについては先ほどと同様に考えるとよい

解法

- 先に、各頂点への、タイプBの道の最小使用本数を求めておく
- 先ほどの理由から、考慮に入れる必要があるタイプBの道の使用本数は各頂点ごとに高々O(√N)通り(最大で√‾(2N)通り程度)
- よって、頂点数O(N√N)
- ダイクストラ法などで解くことができる

ARC052解説
D:9

D-問題概要

- １ ≦ Ｎ ≦ Ｍ なる整数Nのうち以下の性質を満たすものの個数を求めよ
- NをKで割った余り ＝ Nの各桁の和をKで割った余り
  - ただし右辺は10進法表記での和を取るとする

D-部分点1(M ≦ 10^5)

- 1 ≦ N ≦ M なる全てのNを試しても間に合う
- あらかじめ整数xの各桁の和sum[x]を計算しておき N%K == sum[N]%K となる物を数え上げれば良い
- O(M*log(M))
- 10点獲得

D-考察

- N%K == sum[N]%K を変形してみる
- (N-sum[N]) % K == 0

- ここでf(N) = N-sum[N]%Kとする
- f(N) == 0となるNを数えれば良くなった

D-考察

- f(N)の性質を考える
- むやみやたらにAとBを選んでも
  f(A) + f(B) = f(A+B)とはならない
- sum[A] + sum[B] = sum[A+B]がなりたつならばf(A) + f(B) = f(A+B)となる
- sum[A] + sum[B] = sum[A+B]となるのはどのような時か
  - 例：Aの下5桁が0　かつ　Bが5桁のとき

D-考察

- f(N)をもとめるとき
  - N = A*10^5 + Bとする
  - つまりAはNの上5桁、Bは下5桁
  - sum[A*10^5] + sum[B] = sum[N]なので
    f(N) = f(A*10^5) + f(B)とすることができる
  - AもBも10^5通りしか無いので予め全通り求めておくことが出来る

D-満点解法

- 0≦ A,B ≦ 10^5に対してf(A*10^5)とf(B)をあらかじめ求めておき保存しておく
- A*10^5 ≦ Mとなる Aに対して(f(A*10^5) + f(B)) % K == 0となるBを数え上げる
  - setやmapを利用すればlog(10^5)で処理できる
- 上記のステップで処理していない整数Nは10^5個以下しか無いので部分点1と同様に愚直に計算する
- あわせてO(√M log √M)
- 満点獲得

ARC051解説

A問題 塗り絵

この問題は,2次元平面で円と長方形が与えられるので

- 円が完全に長方形に内包されているか
- 長方形が完全に円に内包されているか

の2つを判別すればよい.

円が完全に長方形に内包されているか

- x2 <= x1-r
- x1+r <= x3
- y2 <= y1-r
- y1+r <= y3

上記4つが必要十分条件となる.

長方形が完全に円に内包されているか

これは,長方形の4つの頂点が全て円に内包されているかを調べれば良い.

B問題 互除法

部分点は乱数を使い,いろんな入力をたくさん試せば作れる.

満点はフィボナッチ数列を使うと良い.

- F_1 = F_2 = 1
- F_k = F_{k-1} + F_{k-2} (k >= 3)

フィボナッチ数列とは上記の規則によって生成される数列であり,gcd(F_i+1, F_i)を呼ぶと,そこからgcd(F_i, F_i-1)が呼ばれることがわかる. これを繰り返すと, gcd(F_2, F_1), つまりgcd(1, 1)が呼ばれ, そこからgcd(1, 0)を呼び, 関数が終了する.

よってgcd(F_{K+1}, F_K)を呼ぶとこのプログラムの出力はKになることがわかる.更に、F_41は267914296であるので, 10^9という制約を満たせる.

C問題 掛け算

10^9倍するという動作を10^9回ぐらい行うので, 普通に計算することはできないが, 整数たちの値が十分近ければ, 周期的に掛け算をするようになる.

整数たちをa_1, a_2, ..., a_N(a_1 <= a_2 <= ... <= a_N)とする. 実は a_1 * A >= a_N とすると, a_1, a_2, ..., a_N, a_1, a_2, ..., a_N, a_1, a_2, ..., a_N, ... と掛け算をしていくようになる.

なので, a_1 * A >= a_N となるまでは愚直にシミュレーションをして, そのあとは何回A倍されるかというのを求めれば良い.ただしK=1の場合はコーナーケースとなるので注意すること.

計算量は, 最初のシミュレーションで行う掛け算が高々O(N * log(max a_i))回ぐらいなので, 最小値を探すのにO(N)かけてもO(N^2 * log(max a_i)). そして, 何回A倍されるかを求めた後の高速累乗でO(N * logA).

よって十分間に合う.

D問題 長方形

まず, 長方形を考えたときに値の総和はいくつになるか考える.列の幅をw, 数列の和をXとして, 行の幅をh, 数列の和をYとすると, wY+hXとなることがわかる.

よって列の幅を固定したら, 列の数列の和が最大となるような選び方のみ考えれば良い. 行についても同様.

つまり

- max_w_sum[i][j] := 左からi番目以内までで, 幅jとなるように選んだ時の数列の和のmax
- max_h_sum[i][j] := 上からi番目以内までで, 幅jとなるように選んだ時の数列の和のmax

という配列を作っておけば, クエリは以下のように言い換えられる.

- A, Bが与えられるので, w×max_h_sum[B][h] + h×max_w_sum[A][w]の最大値を求める.(1 <= w <= A, 1 <= h <= B)

更に, wを固定して考えてみると,

- w×(max_h_sum[B][h] + h×max_w_sum[A][w]/w)の最大値を求める.(1 <= h <= B)

という問題になるが, これは以下のように言い換えられる.

- y = h×x + max_h_sum[B][h]という直線が1 <= h <= Bそれぞれについて B本ある. xにmax_w_sum[A][w]/wを代入した時の, y座標の最大値を求める.

これは, convex hull trickを使うと初期化O(B)クエリO(1)で求めることができる.ただし, 事前にmax_w_sum[A][w]/wをsortしておく必要があり, それにO(AlogA)かかる.

よってクエリあたりの計算量はO(B+AlogA) ⊂ O(H+WlogW).

よって計算量はO(Q(H+WlogW))で, 間に合う.

AtCoder Regular Contest 049
解説

AtCoder株式会社

A: ”強調”

森田　晃平

問題概要

- 文字列と、4つの挿入箇所が与えられる

- 4つの箇所に”を挿入して出力する

解法1

- 文字列に対して以下の動作を行えば良い
- A文字目の後ろに”を挿入
- B+1文字目の後ろに”を挿入
- C+2文字目の後ろに”を挿入
- D+3文字目の後ろに”を挿入
- 文字列の途中に文字を入れる機能は、大体の言語にあるはずです
- 例えばc++なら、stringにはinsertという関数があります

解法2

- 逆から入れていくともう少し簡単になります
- D文字目の後ろに”を挿入
- C文字目の後ろに”を挿入
- B文字目の後ろに”を挿入
- A文字目の後ろに”を挿入

B - 高橋ノルム君

三上和馬(@kyuridenamida)

問題概要

- 二次元平面上の点がN個与えられます。各点には係数c_iが定まっています
- 適切に点(X,Y)を選んで
  $c_i*\max(|x_i-X|,|y_i-Y|)$の最大値を最小化

- 1<=N<=1000
  誤差はあまり気にしないでよさそうな制約

部分点

- すべてのiに対して$c_i=1$という制約の下では
  (X,Y)=(xの最大値と最小値の平均,yの最大値と最小値の平均)
  とすれば最適になる

- 30点得られます

考察

- **最大値を最小化**
  ↑二分探索を考えるのが定石

- コスト(=求める最大値,tとする)を固定してみよう!

考察

- コストtを固定すると以下のような判定問題ができる

  「点(x_i,y_i)を中心とした長さ$2t/c_i$の正方形がたくさんあります．全ての正方形の共通部分はありますか?」

- これはx,y軸独立に区間集合の共通部分を求め，どちらにも共通部分が存在していれば長方形としての共通部分があると言える

区間集合の共通部分

- 区間$[L_1,R_1],[L_2,R_2],...,[L_N,R_N]$があったときに
- その共通部分は，存在するならば

$[\max(L_1,L_2,\cdots,L_N),\min(R_1,R_2,\cdots,R_N)]$

で表される.

- 左の値<=右の値なら共通部分が存在する
- 左の値>右の値なら共通部分は存在しない

解法

- 考察で作った判定関数に単調性があり，共通部分が存在するギリギリのコストtを二分探索で見つければそれが答え
- 判定関数の時間計算量はO(N)
- 二分探索は64回くらい反復しとけば安心

C - ぬりまーす

三上和馬(@kyuridenamida)

問題概要

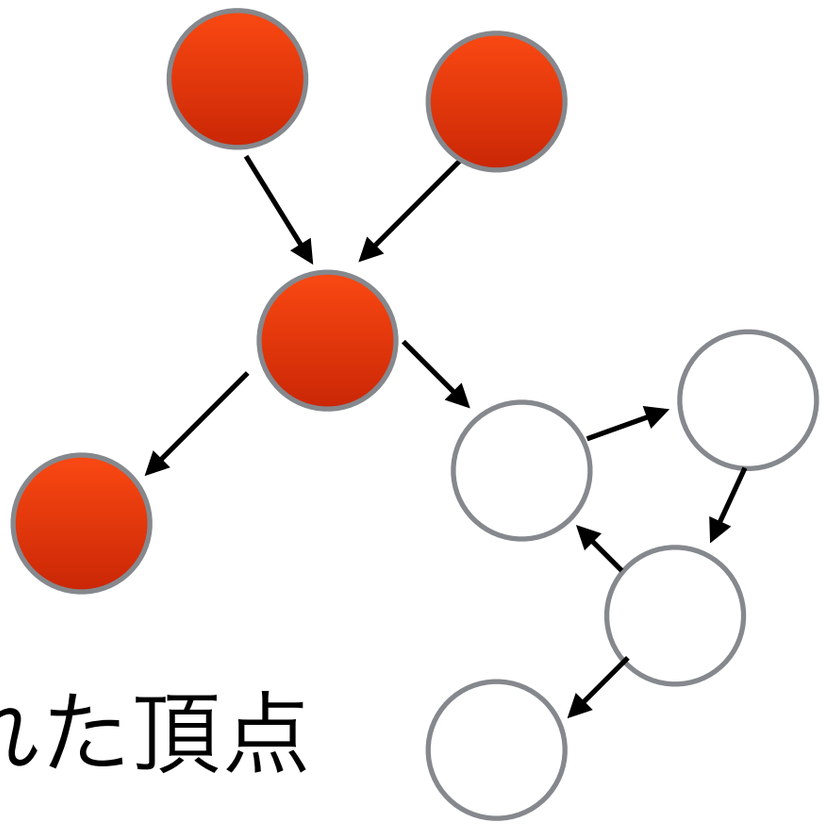
- N頂点のグラフに色を塗る。ただし以下のタイプの制約がそれぞれA個,B個ずつある。塗れる頂点の最大数を求めよ。
  タイプ1.ある頂点 x に色を塗るとき、既に頂点 y に色が塗られてなければならない。
  タイプ2.ある頂点 u に色を塗るとき、既に頂点 v に色が塗られていてはいけない。

- 1≦N,A≦100,1≦B≦10

考察

- タイプ1の制約のみしかない場合を考え、制約を辺(y→x)としてグラフにしてみよう。実はこれが解ければほぼ解けている。

- 有向閉路がない場合なら、適切な順番で塗れば全て塗ることができる。ある場合はそこから伸びている辺は塗れない。



赤色の部分は塗れた頂点

考察

- タイプ1の制約のみしかない場合でも、どうやって塗るか考えなければならない

- 今回は次のうちどちらの方法でも通るので好きな方で良い

- **頂点の色変化がなくなる(=収束する)までループを回し続ける方法**
  時間計算量はO(N(N+辺の数))

- **キューを使って、入次数=(入ってくる頂点で色を塗られている頂点数)になった瞬間のみ頂点をpushするような幅優先探索**
  時間計算量はO(N+辺の数)

元の制約に対する解法

- タイプ2の制約「ある頂点 u に色を塗るとき、既に頂点 v に色が塗られていてはいけない。」を考える

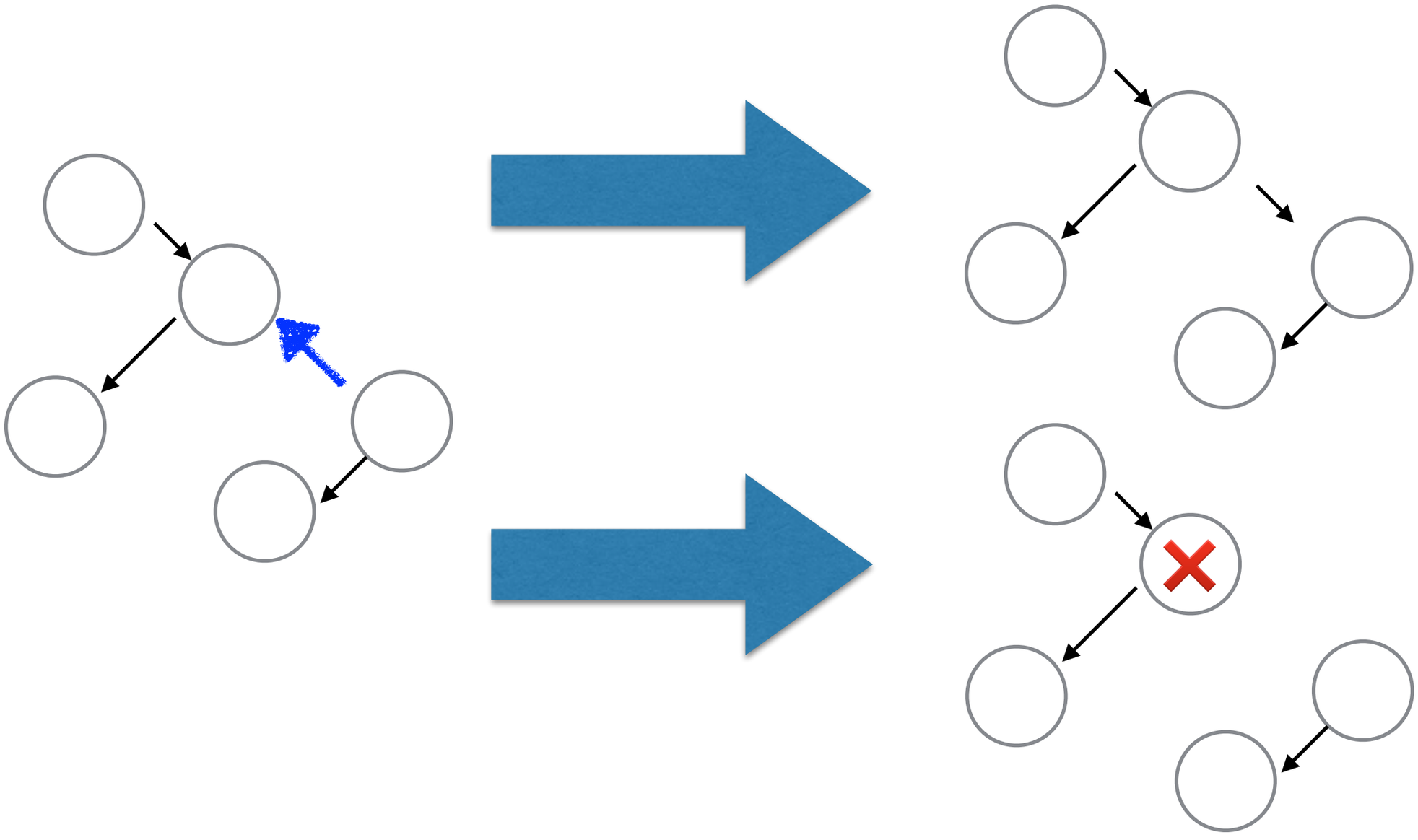
- タイプ2の制約は各々以下のように言い換えることができる
  「頂点uを決して塗らないか、頂点vを塗るためには頂点uを塗らないといけない」

- つまり、各タイプ2の制約について、2通りの可能性を網羅すれば十分。後者はタイプ1の制約とみなすことができる

- 全ての可能性を全列挙しても2^B通りで、Bは小さいので間に合う

- 全体の時間計算量はO(2^B * (先ほど述べた塗るための計算量))

言い換え例



・ 青辺はタイプ2の制約の辺(v→u)

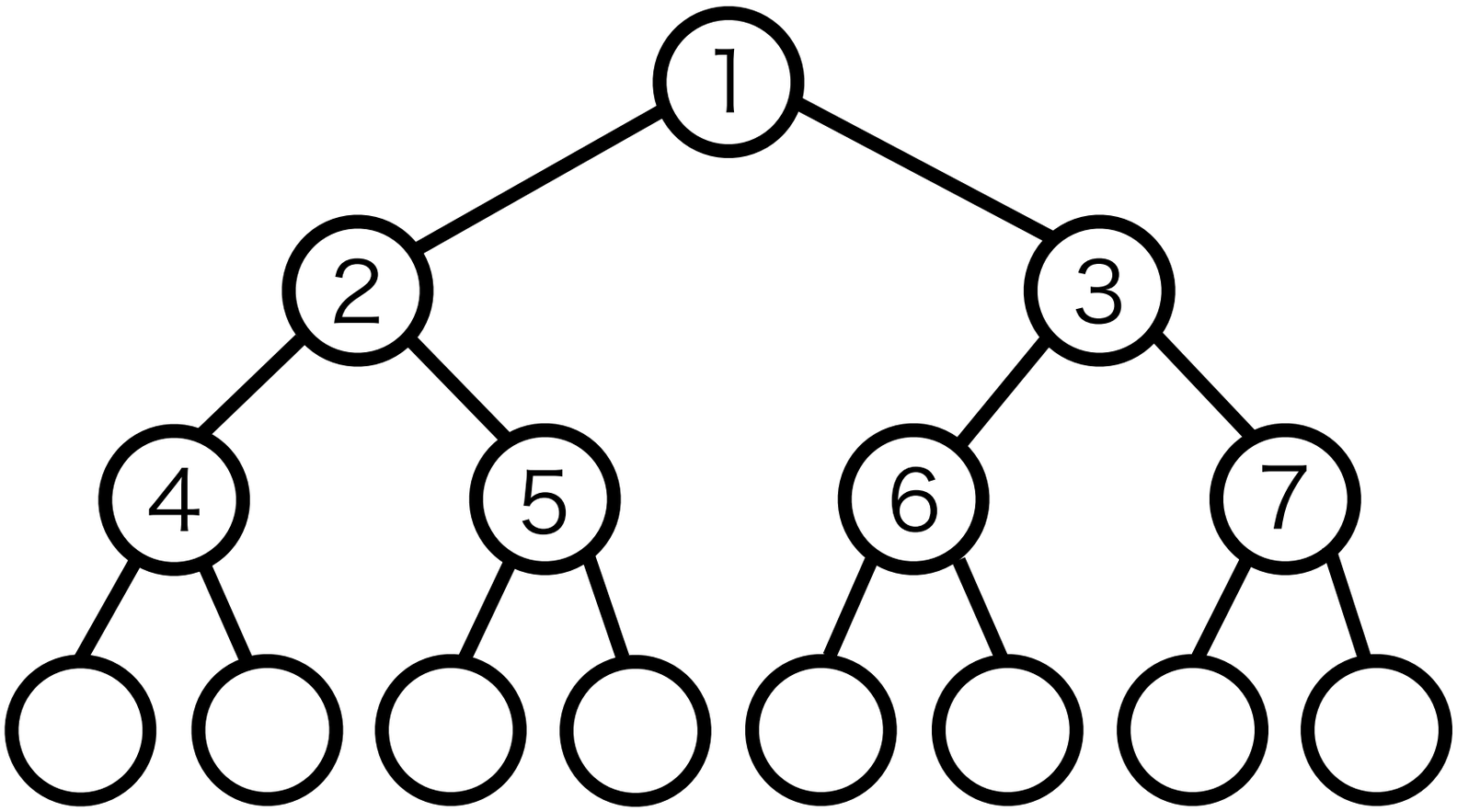
D: すわっぷしまーす

森田　晃平

問題概要

- 完全二分木が与えられる、葉には1,2,...と数が書かれている
- ので、以下のクエリを処理
  - 左からk番目の葉の値を求める
  - swap(a), swap(a+1), …, swap(b)を行う
- ただし、swap(x)とは位置xの頂点の左右の子をswap

位置

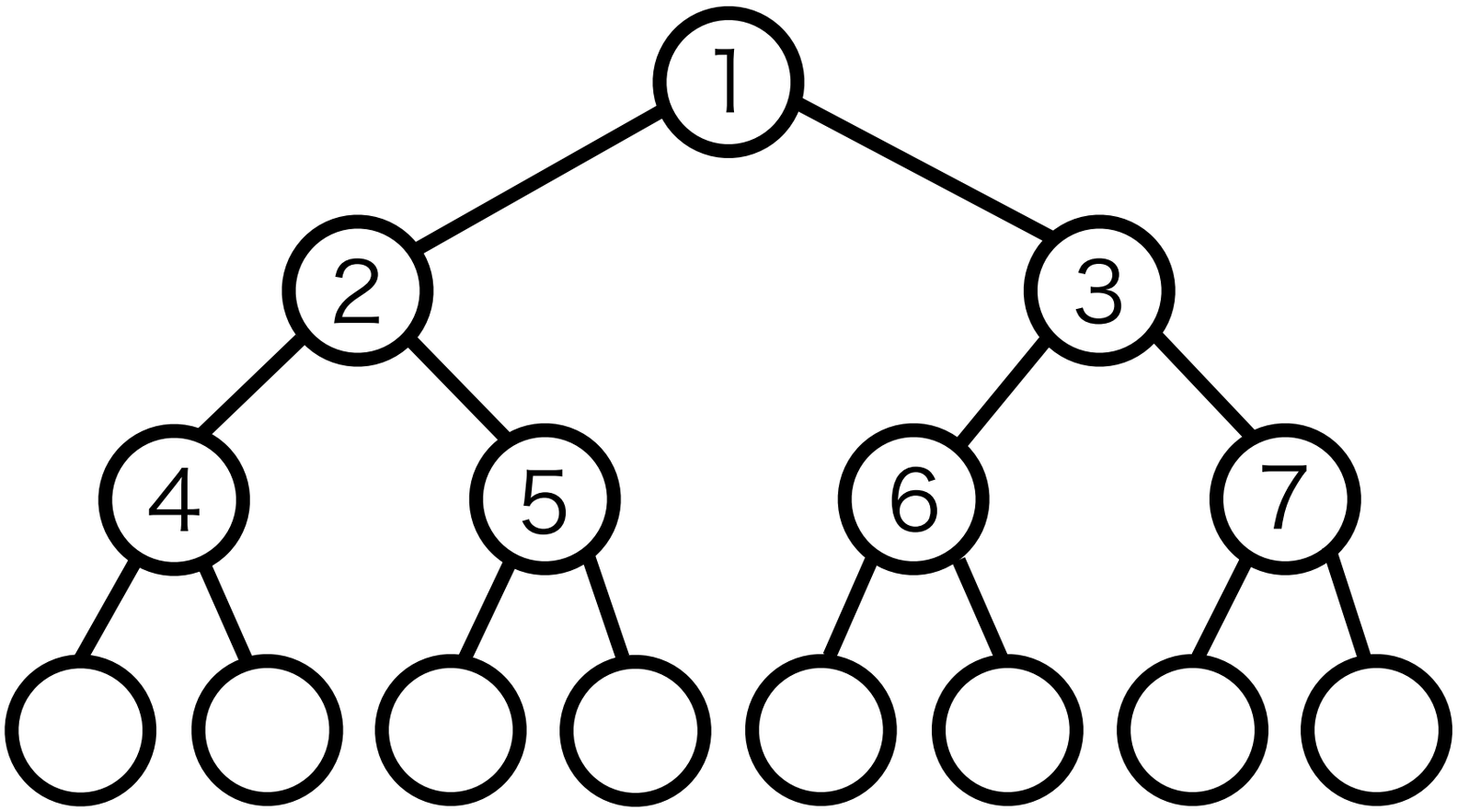


swapクエリ

- クエリ2は、範囲swapのようなものだが、そのままではどうしようも無いので、木の高さごとにバラす

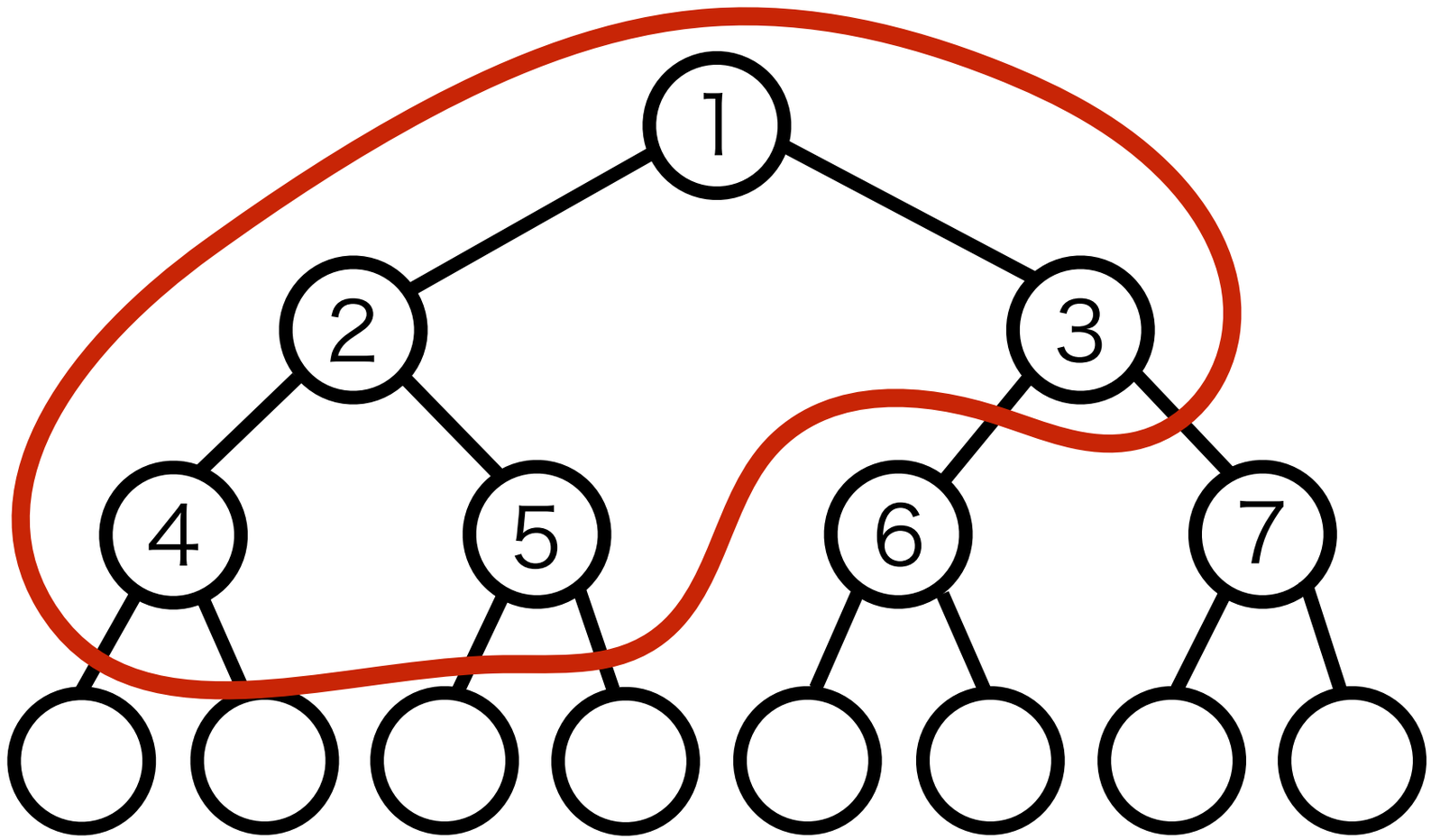
- これで、木の特定の高さの範囲swapを考えればよくなった

例

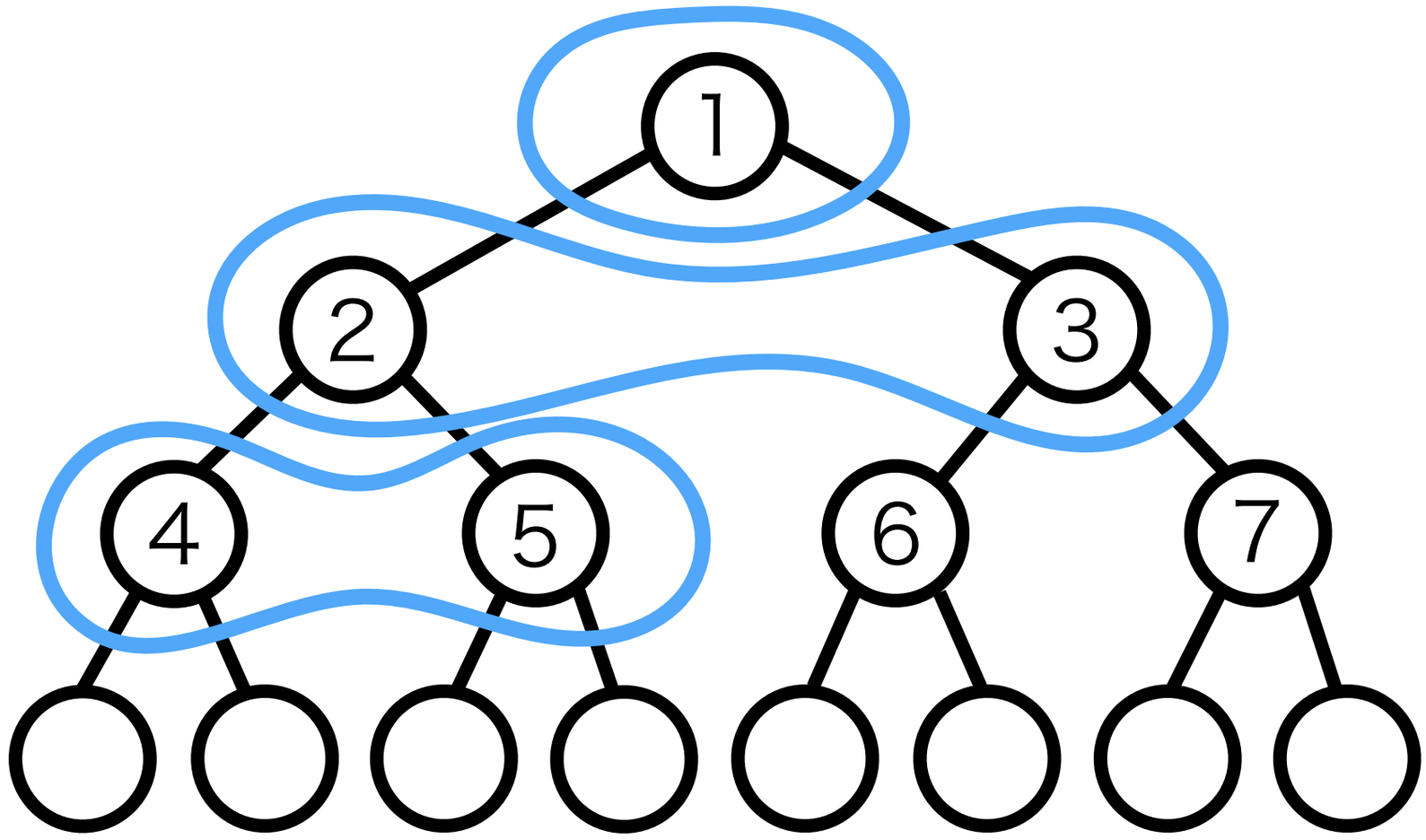


これにswap(1), swap(2), …, swap(5)を行うとする

例



これにswap(1), swap(2), …, swap(5)を行うとする



- これを、
  - swap(1)(高さ1)
  - swap(2), swap(3)(高さ2)
  - swap(4), swap(5)(高さ3)
- の3種類に分割する

swapクエリ

- これで分割したクエリを、さらに分割する

- SegmentTreeに対する区間クエリ

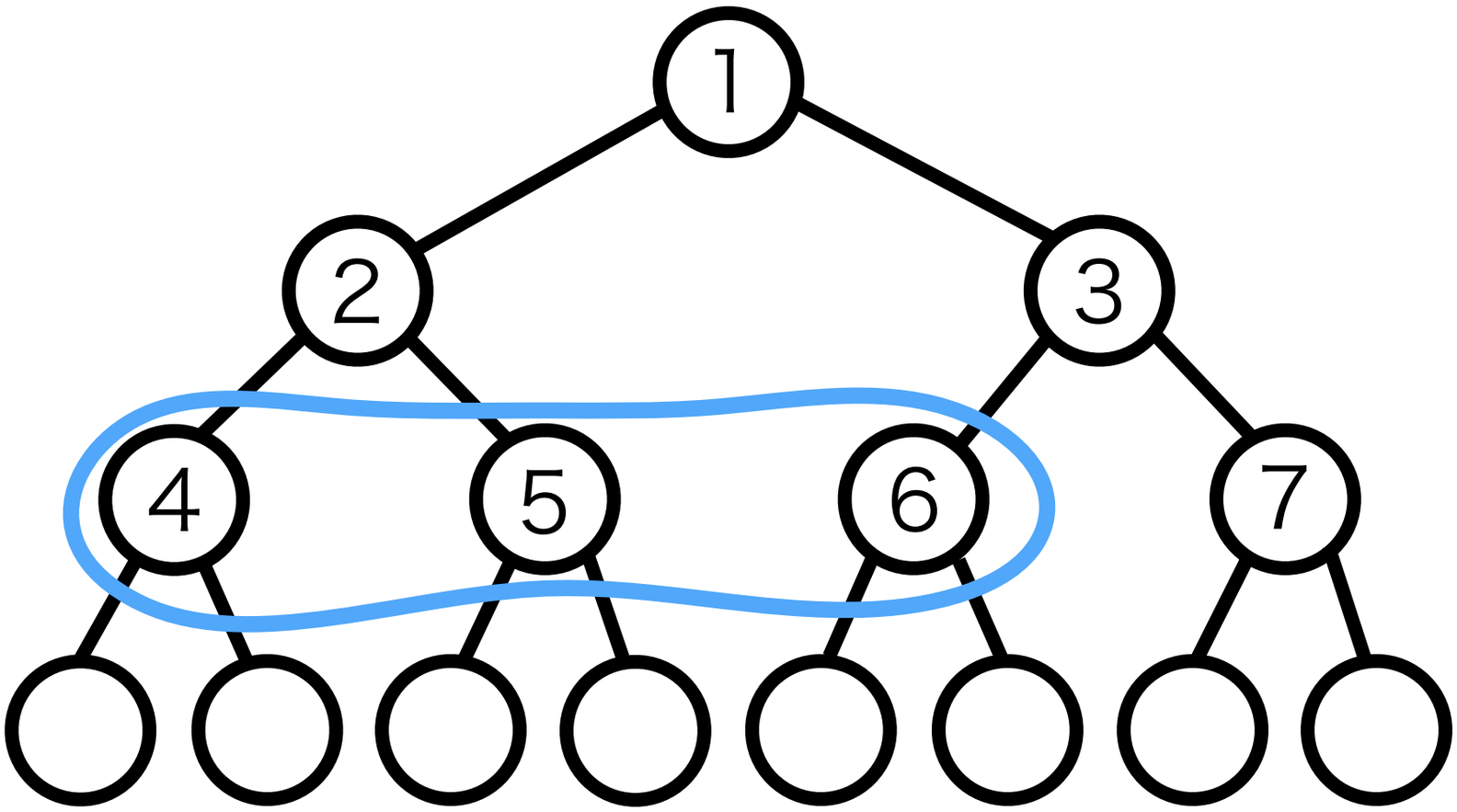
- 木について、”高さiの頂点を全てswapする”というクエリになるように分割したい

- たとえば、RMQ(Range minimum Query)をSegTreeでやるとどうなるか？

- [a, b]の最小値というクエリを、O(logn)個の”木の全ての葉の値の最小値”というものに分割している

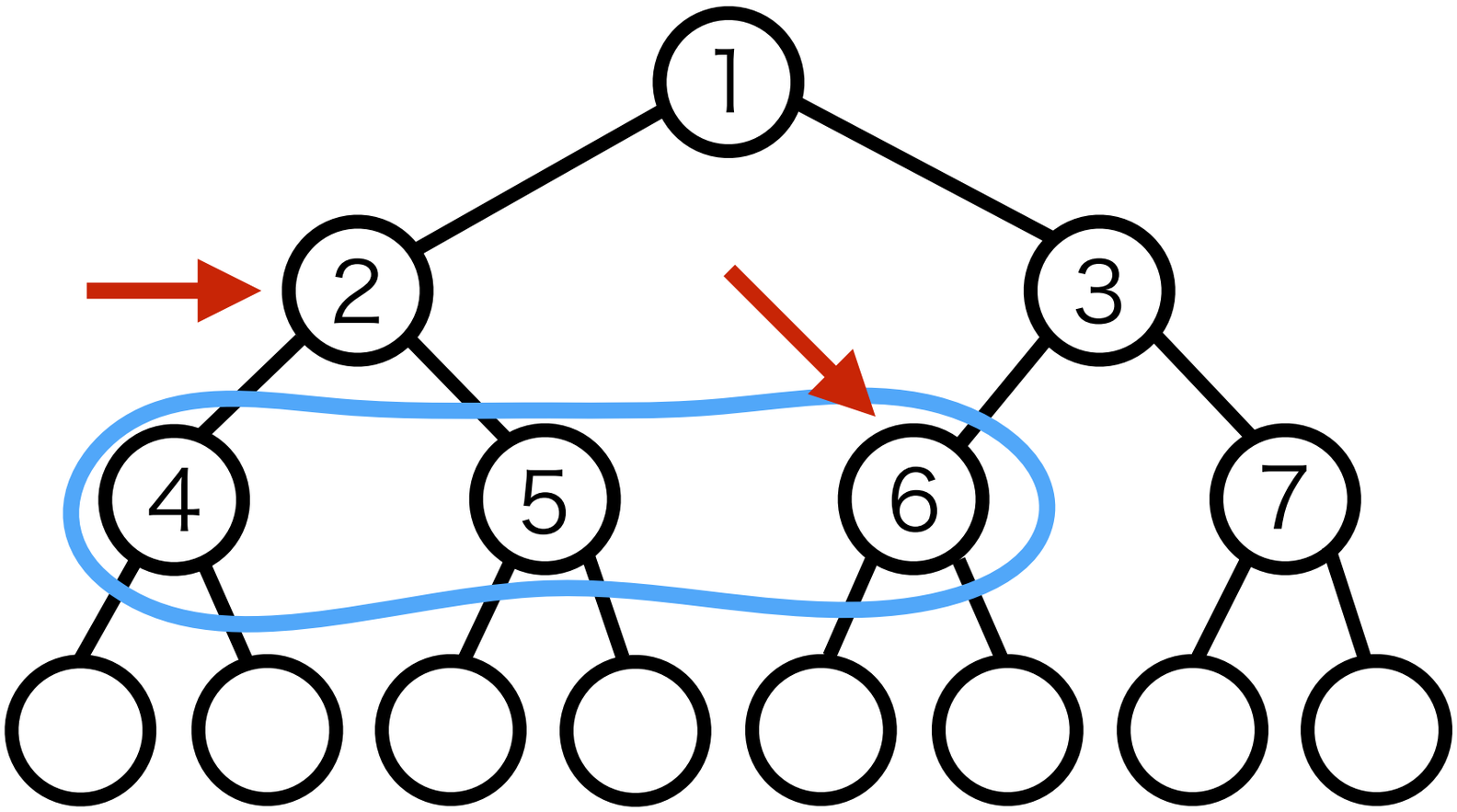
- 今回は、i段目の[a,b]のswapというクエリなので、O(logn)個の”木のi段目の全ての頂点をswap”というクエリに分割できる

例



これにswap(4), swap(5), swap(6)を行うとする

例



これは、位置2, 位置6の頂点に対して、

”高さ3の頂点を全てswap”

というクエリになる

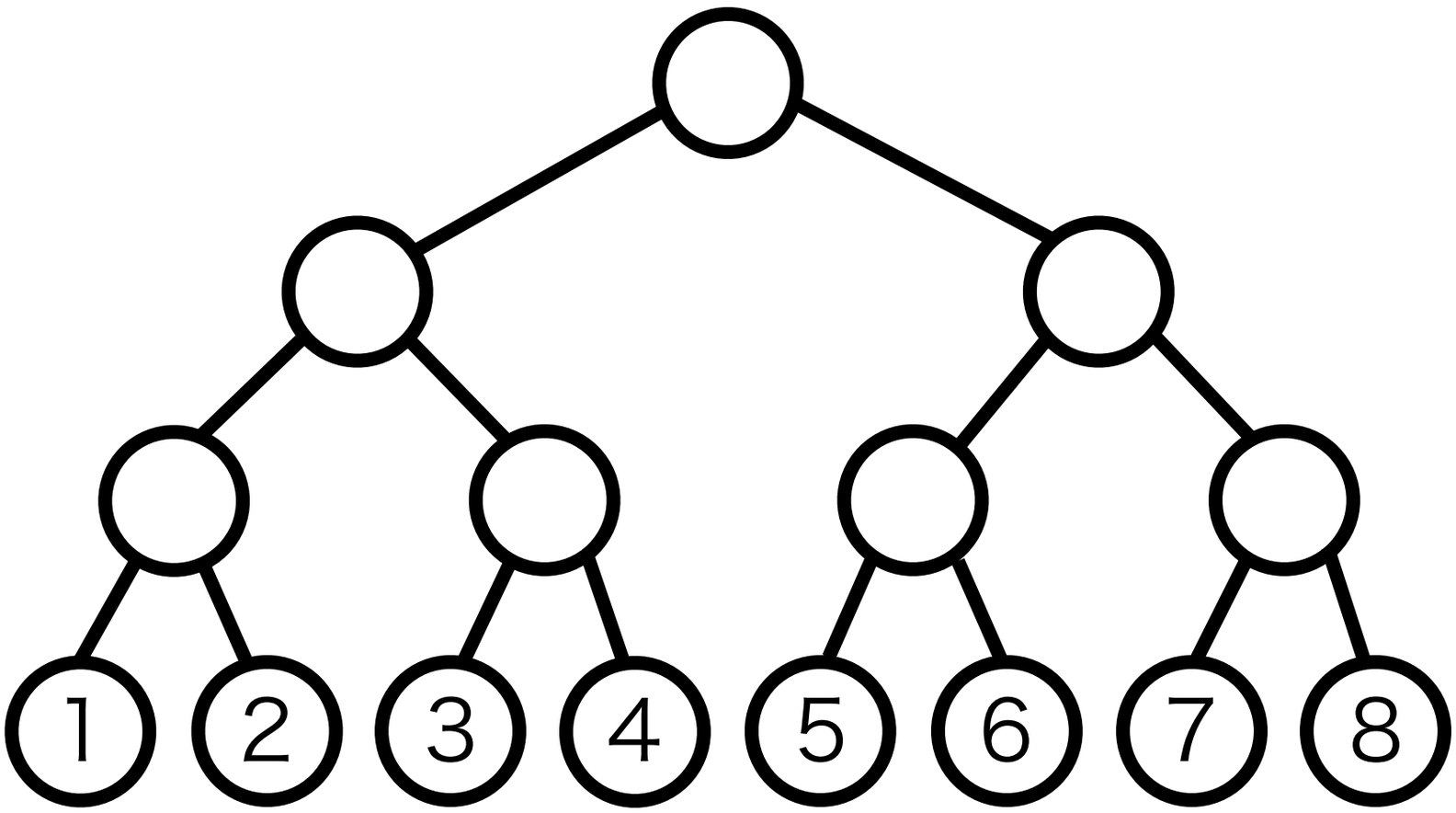
i段目を全てswap

- “i段目を全てswap”というクエリになると何が嬉しい？

- とりあえず、位置1(木の根)に対して”i段目を全てswap”というクエリだけがくると考える

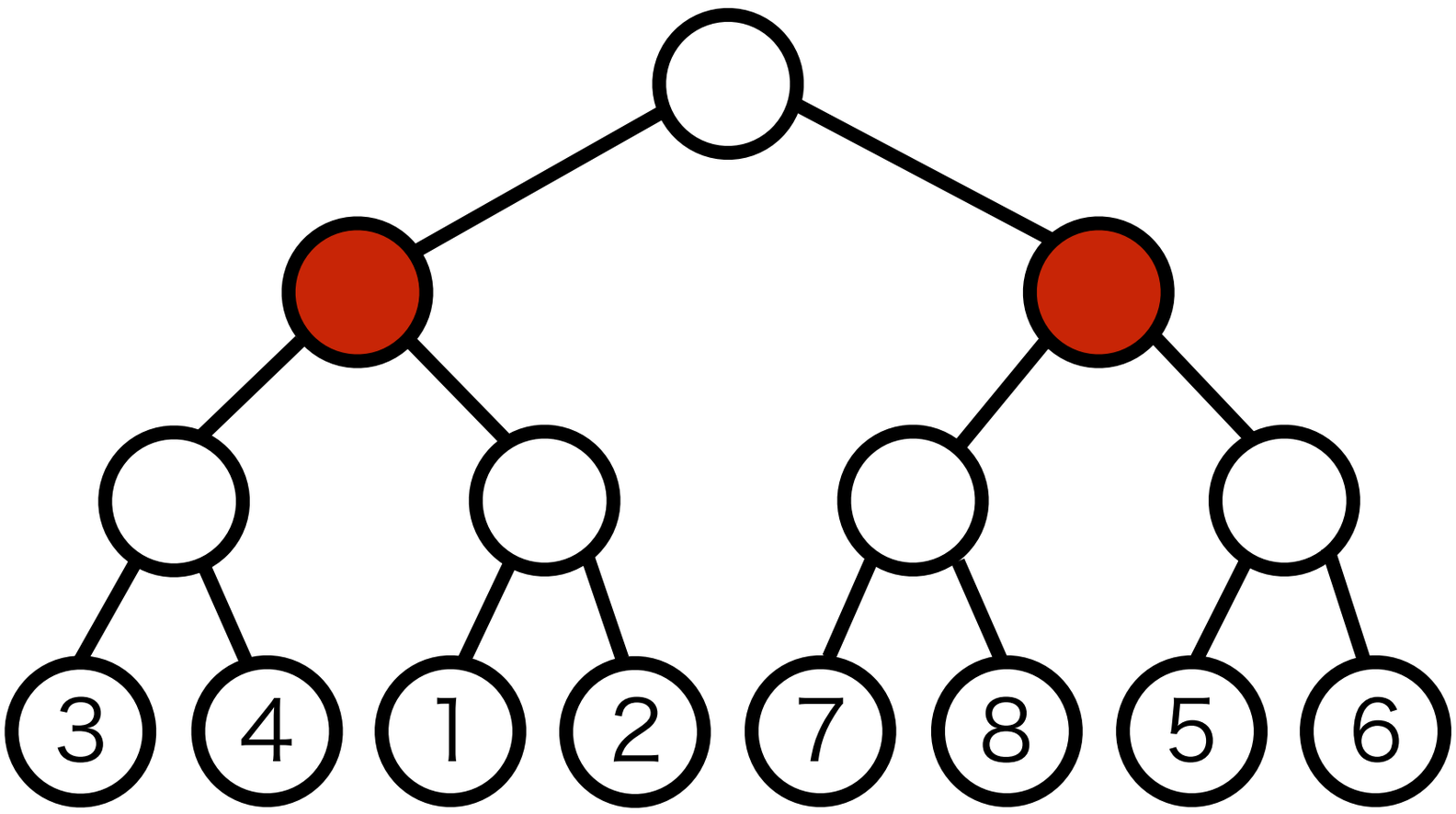
- こうすると、”i段目を全てswap”→”j段目を全てswap”をしても、”j段目を全てswap”→”i段目を全てswap”しても、できる木は変わらないことがわかる

例



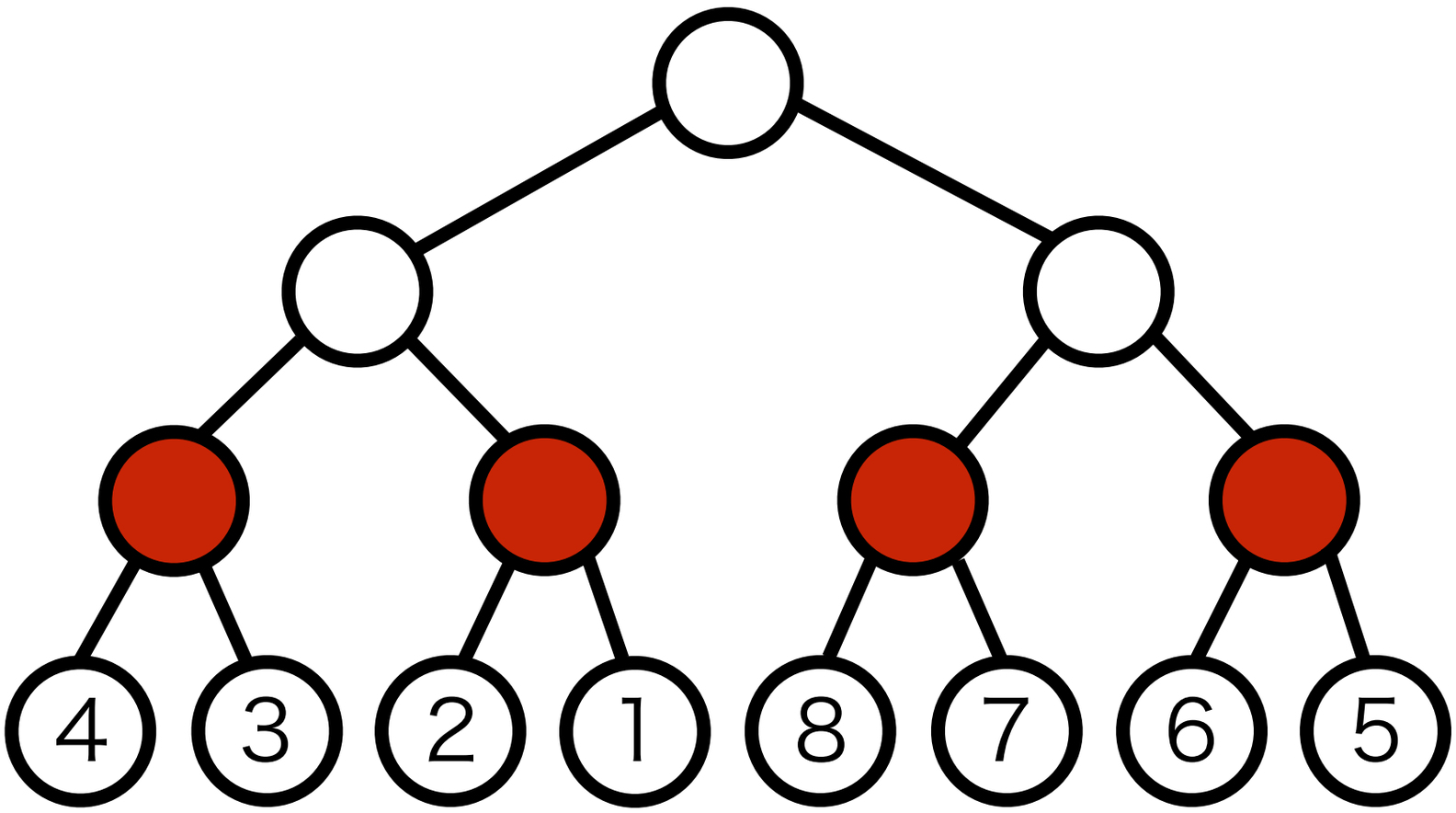
高さ2をswap → 高さ3をswap

例



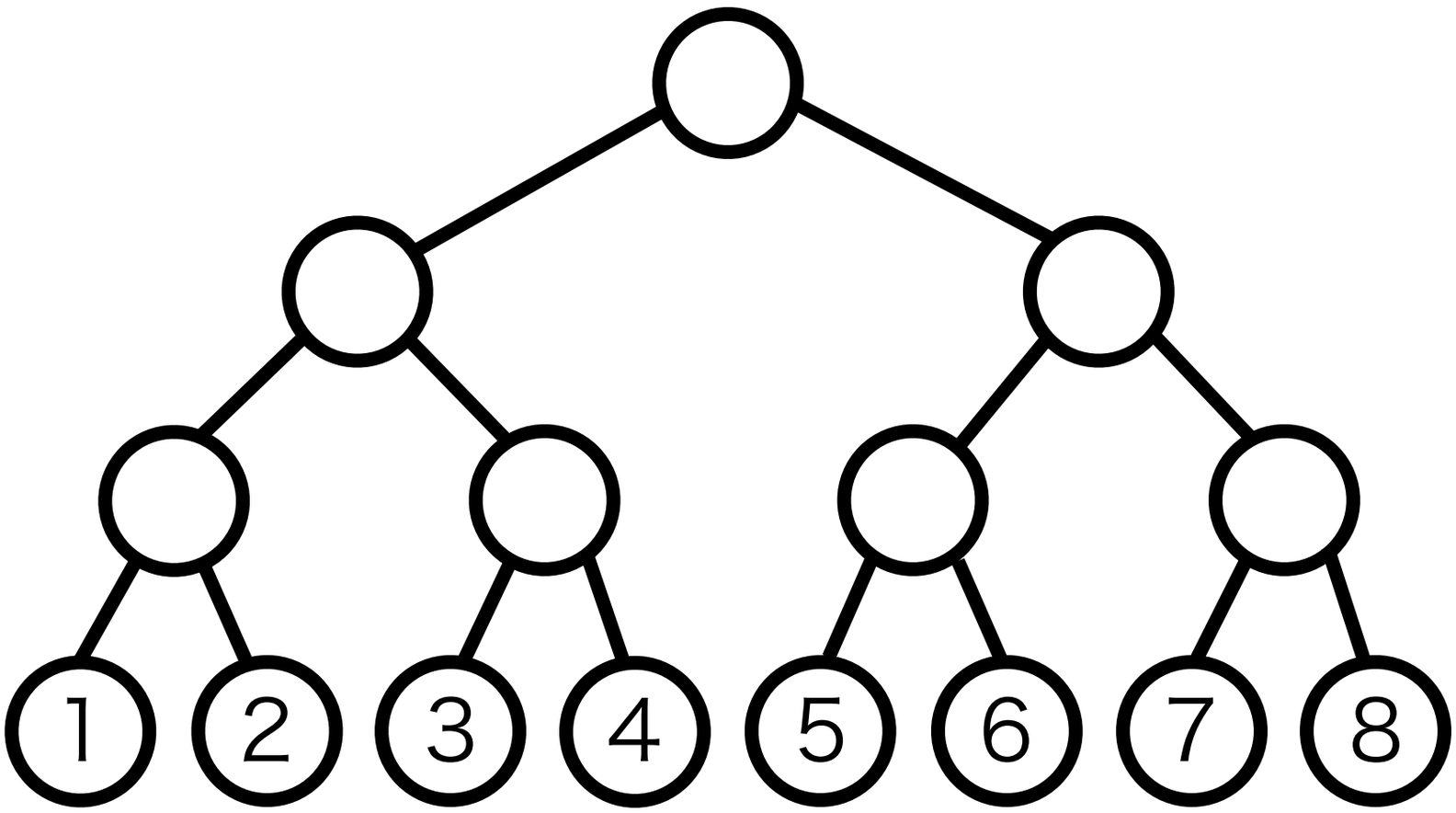
高さ2をswap → 高さ3をswap

例



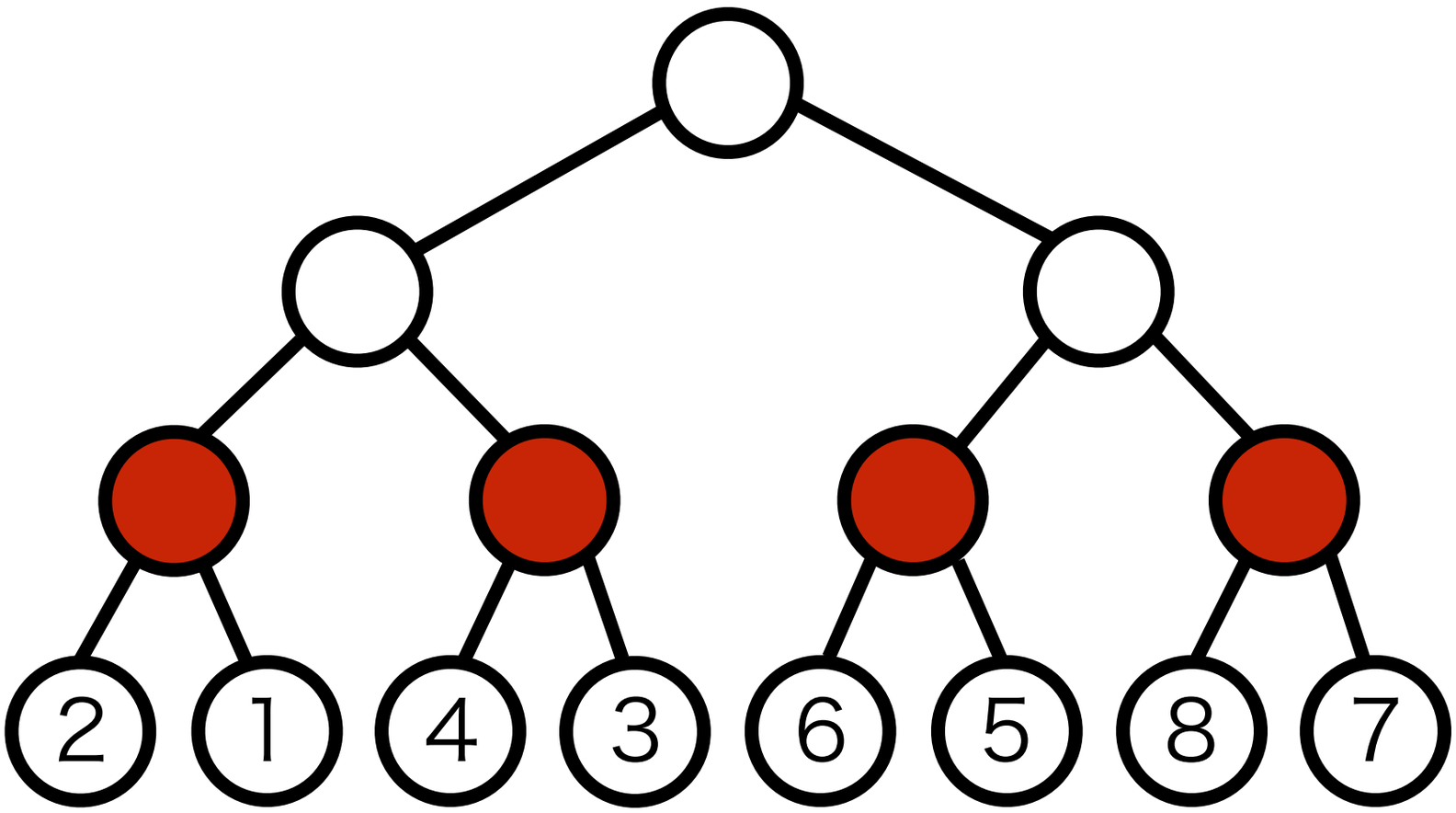
高さ2をswap → 高さ3をswap

例



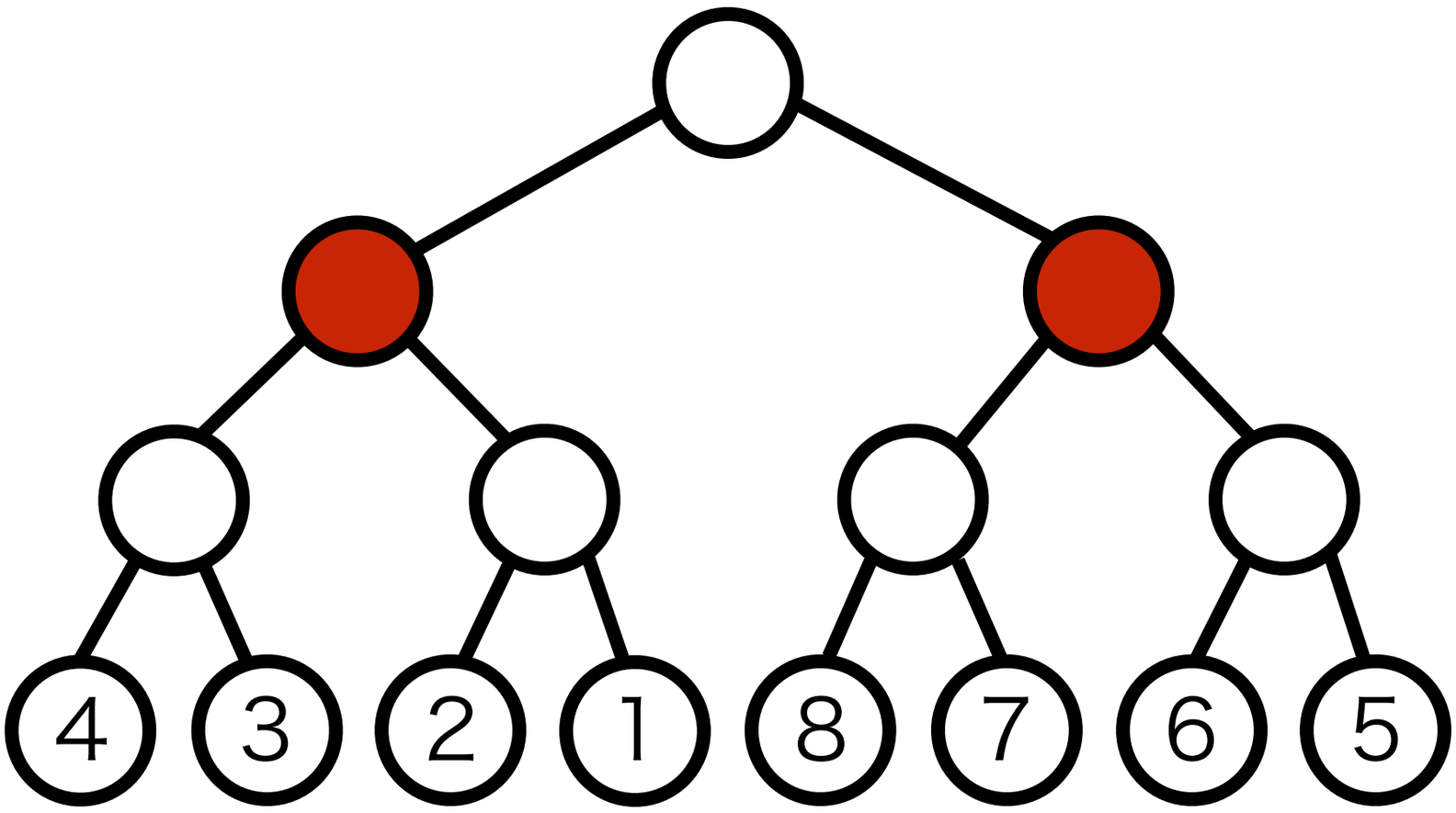
高さ3をswap → 高さ2をswap

例



高さ3をswap → 高さ2をswap

例



高さ3をswap → 高さ2をswap

i段目を全てswap

- 結局、それぞれの高さごとに”反転しているか”というフラグを持てば良い

- “高さiを反転”というクエリが来たら、それに対応するフラグを反転させる

i段目を全てswap

- 位置1(木の根)以外にもクエリが来るなら?

- 先述のflagを各頂点にもたせて、伝搬させていく

- 高さiのノードにアクセスするときは、i番目のフラグが立っていたら左右の子をswapして、i+1番目以降のフラグを全て子のフラグに伝搬させる

i段目を全てswap

- これをsegtreeを潜る時に行うと、アクセスするノードの上の頂点は常に何もフラグが立っていないようにできるので、うまく動く

- 計算量はクエリを分割する個数がO(logn)なので、クエリあたりO(logn)

AtCoder Regular Contest 049
解説

AtCoder株式会社

A: ”強調”

森田　晃平

問題概要

- 文字列と、4つの挿入箇所が与えられる

- 4つの箇所に”を挿入して出力する

解法1

- 文字列に対して以下の動作を行えば良い
- A文字目の後ろに”を挿入
- B+1文字目の後ろに”を挿入
- C+2文字目の後ろに”を挿入
- D+3文字目の後ろに”を挿入
- 文字列の途中に文字を入れる機能は、大体の言語にあるはずです
- 例えばc++なら、stringにはinsertという関数があります

解法2

- 逆から入れていくともう少し簡単になります
- D文字目の後ろに”を挿入
- C文字目の後ろに”を挿入
- B文字目の後ろに”を挿入
- A文字目の後ろに”を挿入

B - 高橋ノルム君

三上和馬(@kyuridenamida)

問題概要

- 二次元平面上の点がN個与えられます。各点には係数ciが定まっています。

- 適切に点(X,Y)を選んで
  $c_i * \max(|x_i - X|, |y_i - Y|)$の最大値を最小化

- 1<=N<=1000
  誤差はあまり気にしないでよさそうな制約

部分点

- すべてのiに対して$c_i=1$という制約の下では
  (X,Y)=(xの最大値と最小値の平均,yの最大値と最小値の平均)
  とすれば最適になる.

- 30点得られます.

考察

- **最大値を最小化**
  ↑二分探索を考えるのが定石

- コストtの最大値を固定してみよう!

考察

- コストtを固定すると以下のような判定問題ができる

  「点(x_i,y_i)を中心とした長さ$2t/c_i$の正方形がたくさんあります．全ての正方形の共通部分はありますか?」

- これはx,y軸独立に区間集合の共通部分を求め，どちらにも共通部分が存在していれば長方形としての共通部分があると言える．

区間集合の共通部分

- 区間$[L_1,R_1],[L_2,R_2],...,[L_N,R_N]$があったときに
- その共通部分は，存在するならば

$[\max(L_1,L_2,\cdots,L_N),\min(R_1,R_2,\cdots,R_N)]$

で表される.

- 左の値<=右の値なら共通部分が存在する
- 右の値>左の値なら共通部分は存在しない.

解法

- 考察で作った判定関数に単調性があり，共通部分が存在するギリギリのコストtを二分探索で見つければそれが答え
- 判定関数の時間計算量はO(N)
- 二分探索は64回くらい反復しとけば安心.

C - ぬりまーす

三上和馬(@kyuridenamida)

問題概要

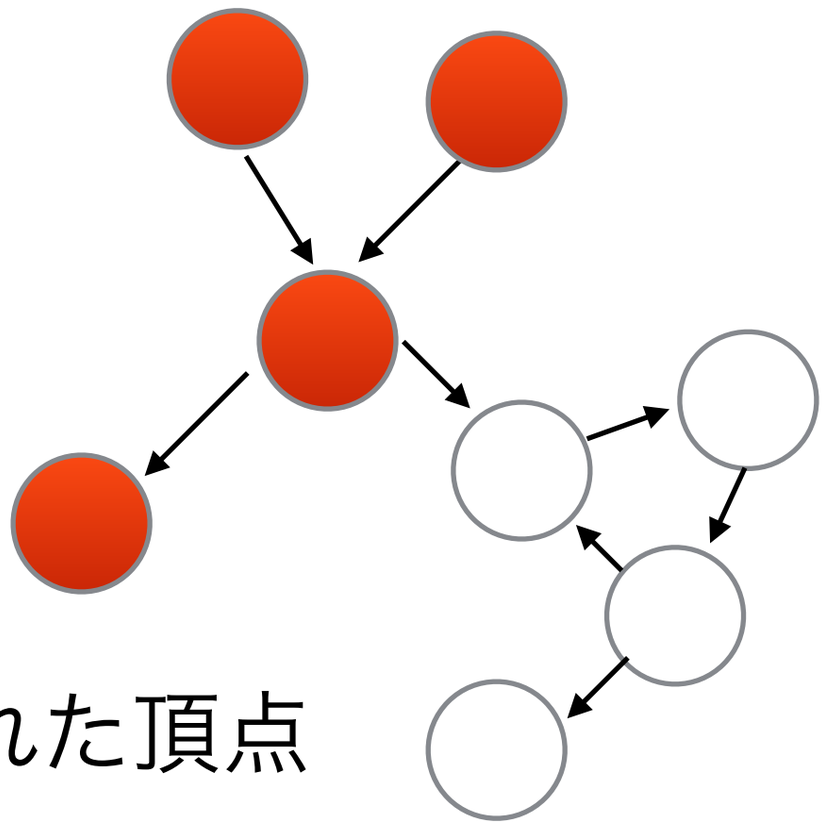
- N頂点のグラフに色を塗る。ただし以下のタイプの制約がそれぞれA個,B個ずつある。塗れる頂点の最大数を求めよ。
  タイプ1.ある頂点 x に色を塗るとき、既に頂点 y に色が塗られてなければならない。
  タイプ2.ある頂点 u に色を塗るとき、既に頂点 v に色が塗られていてはいけない。

- 1≦N,A≦100,1≦B≦10

考察

- タイプ1の制約のみしかない場合を考え、制約を辺(y→x)としてグラフにしてみよう。実はこれが解ければほぼ解けている。

- 有向閉路がない場合なら、適切な順番で塗れば全て塗ることができる。ある場合はそこから伸びている辺は塗れない。



赤色の部分は塗れた頂点

考察

- タイプ1の制約のみしかない場合でも、どうやって塗るか考えなければならない

- 今回は次のうちどちらの方法でも通るので好きな方で良い

- **頂点の色変化がなくなる(=収束する)までループを回し続ける方法**
  時間計算量はO(N(N+辺の数))

- **キューを使って、入次数=(入ってくる頂点で色を塗られている頂点数)になった瞬間のみ頂点をpushするような幅優先探索**
  時間計算量はO(N+辺の数)

元の制約に対する解法

- タイプ2の制約「ある頂点 u に色を塗るとき、既に頂点 v に色が塗られていてはいけない。」を考える

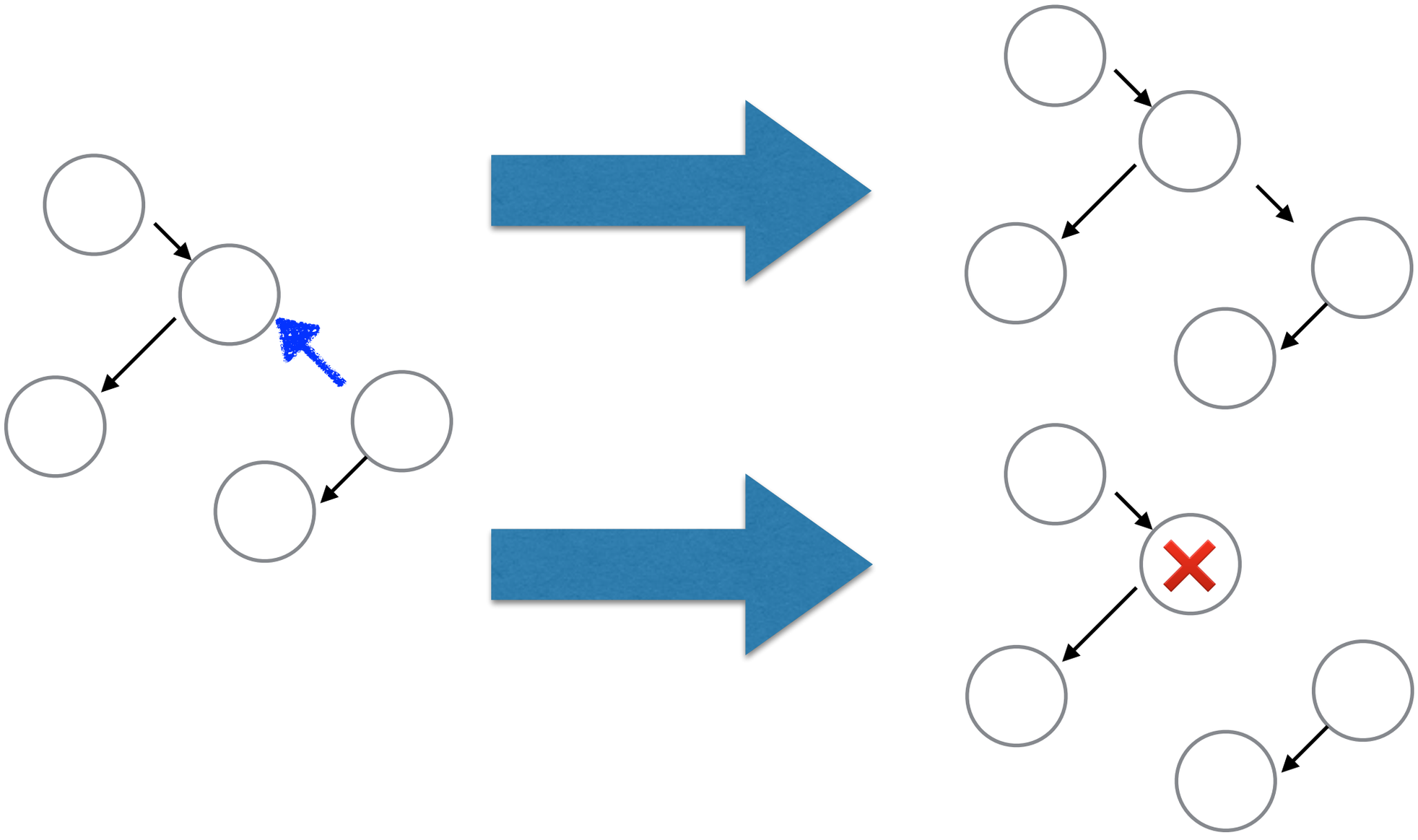
- タイプ2の制約は各々以下のように言い換えることができる
  「頂点uを決して塗らないか、頂点vを塗るためには頂点uを塗らないといけない」

- つまり、各タイプ2の制約について、2通りの可能性を網羅すれば十分。後者はタイプ1の制約とみなすことができる

- 全ての可能性を全列挙しても2^B通りで、Bは小さいので間に合う

- 全体の時間計算量はO(2^B * (先ほど述べた塗るための計算量))

言い換え例



・ 青辺はタイプ2の制約の辺(v→u)

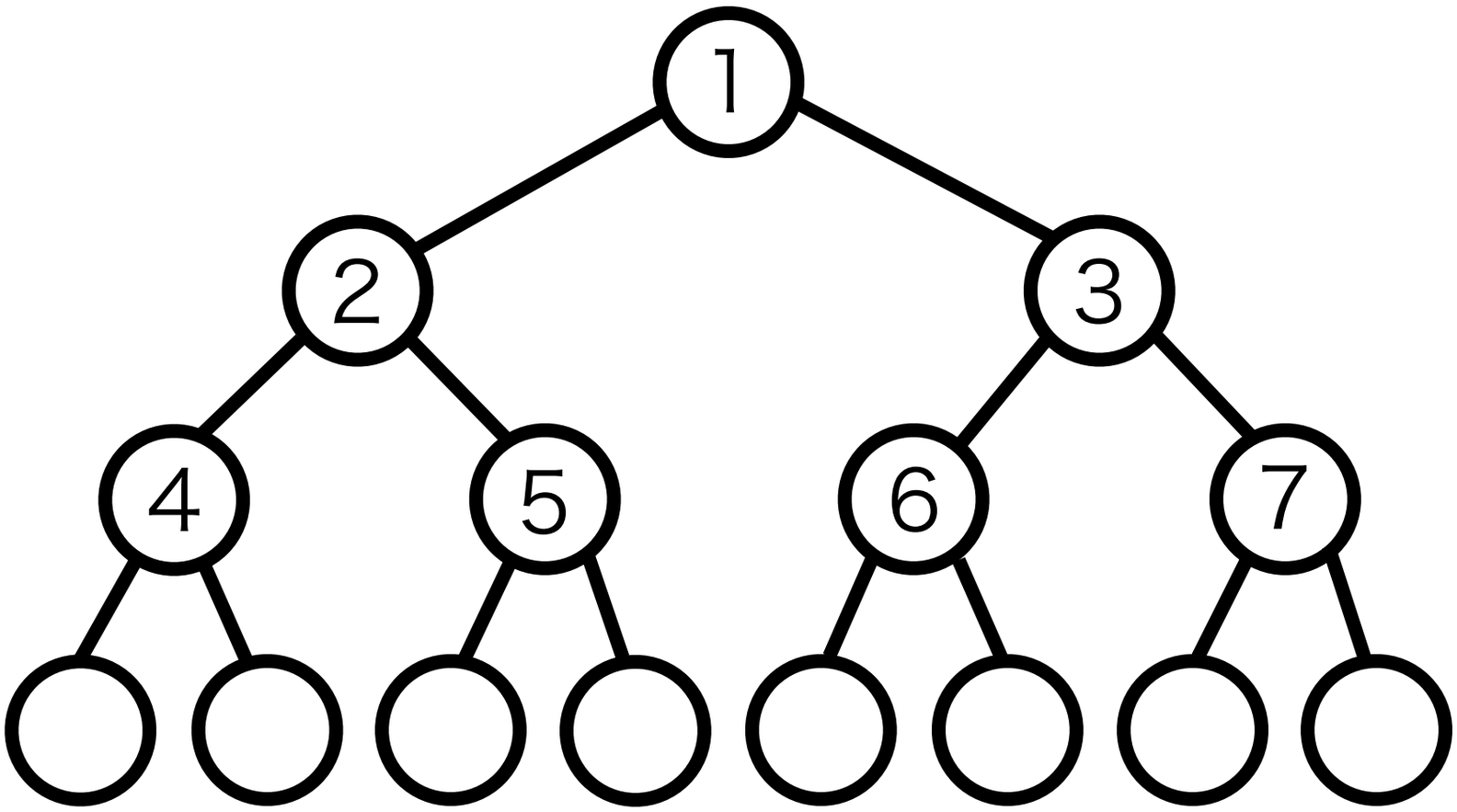
D: すわっぷしまーす

森田　晃平

問題概要

- 完全二分木が与えられる、葉には1,2,...と数が書かれている
- ので、以下のクエリを処理
  - 左からk番目の葉の値を求める
  - swap(a), swap(a+1), …, swap(b)を行う
- ただし、swap(x)とは位置xの頂点の左右の子をswap

位置

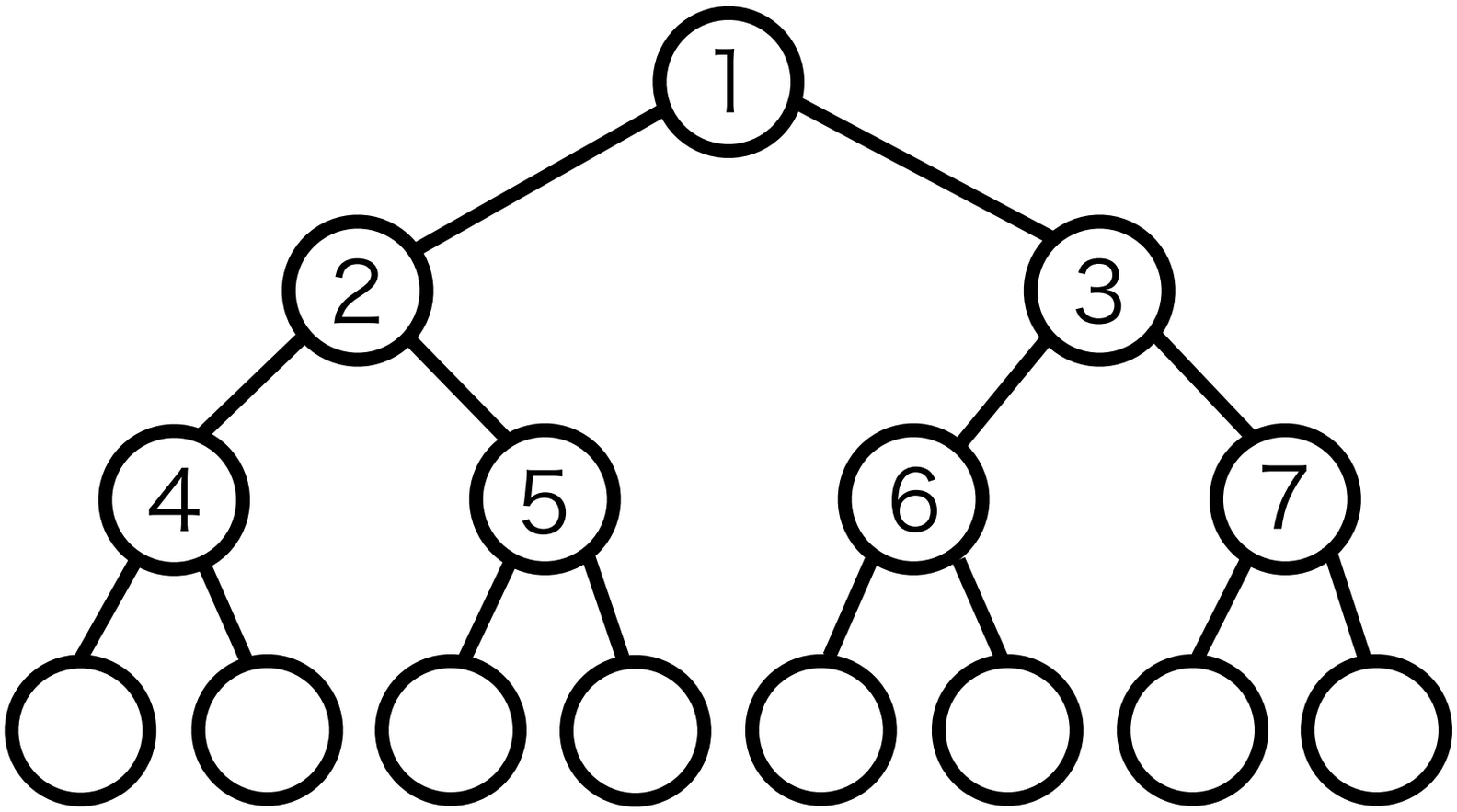


swapクエリ

- クエリ2は、範囲swapのようなものだが、そのままではどうしようも無いので、木の高さごとにバラす

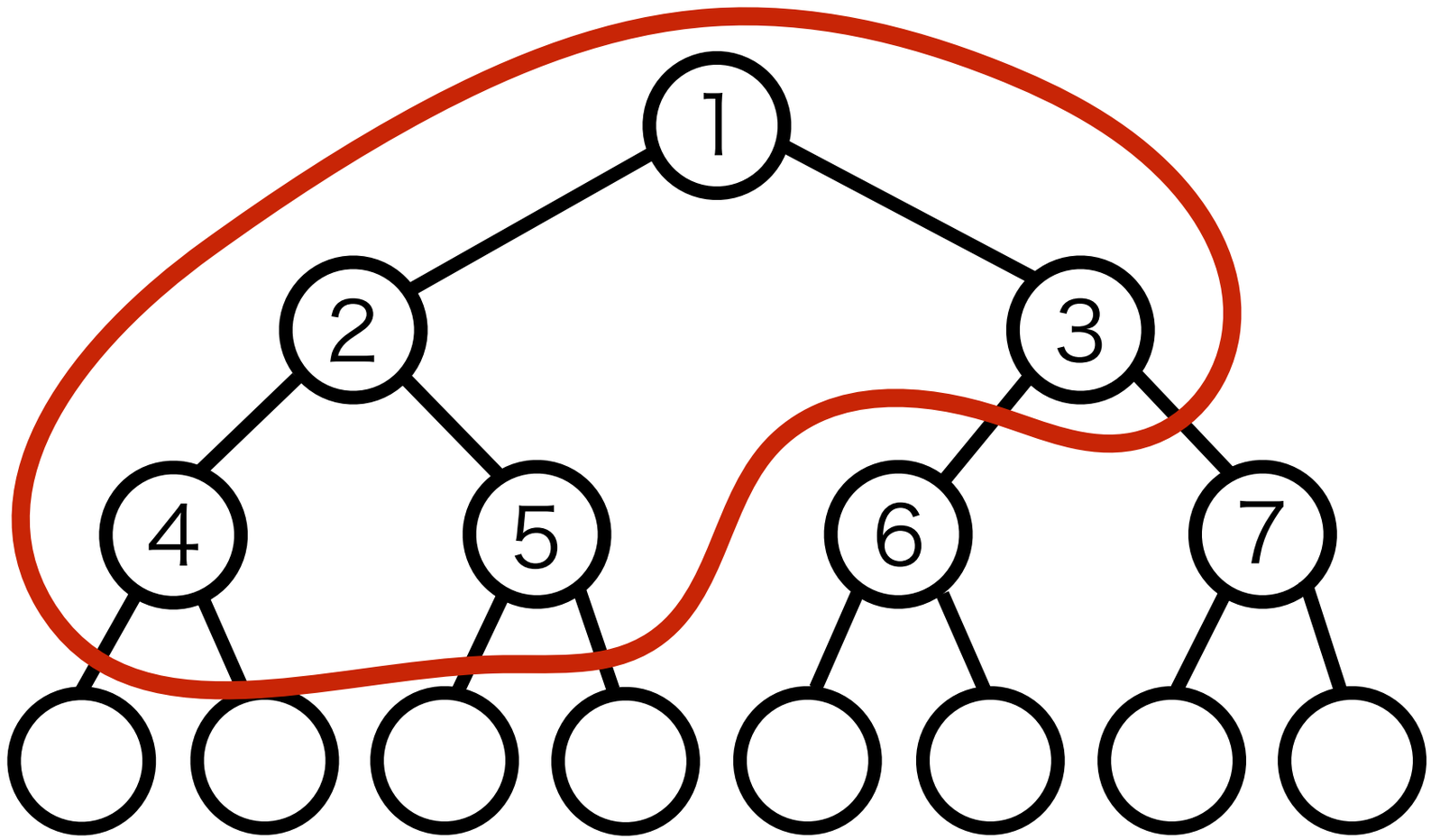
- これで、木の特定の高さの範囲swapを考えればよくなった

例

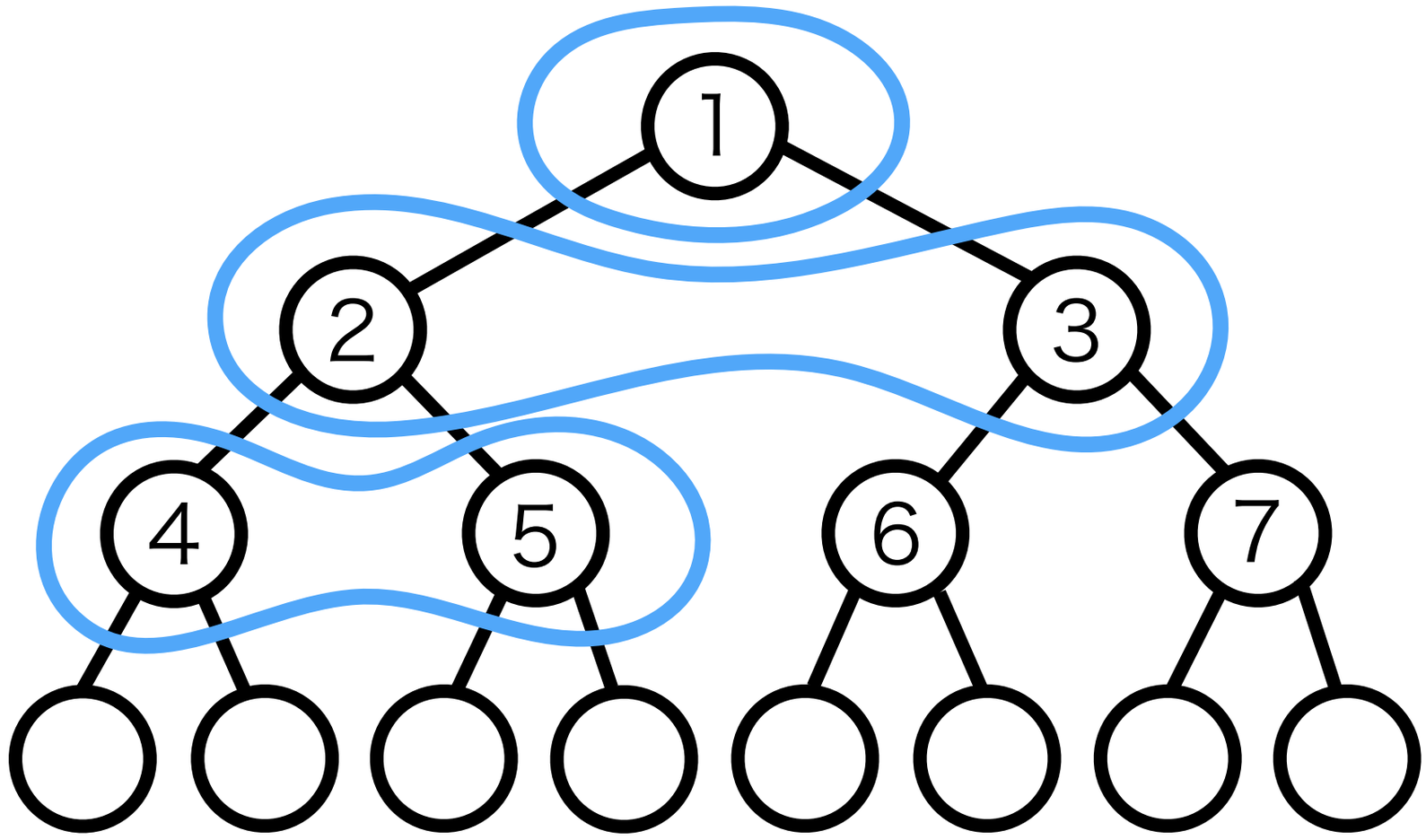


これにswap(1), swap(2), …, swap(5)を行うとする

例



これにswap(1), swap(2), …, swap(5)を行うとする



- これを、
  - swap(1)(高さ1)
  - swap(2), swap(3)(高さ2)
  - swap(4), swap(5)(高さ3)
- の3種類に分割する

swapクエリ

- これで分割したクエリを、さらに分割する

- SegmentTreeに対する区間クエリ

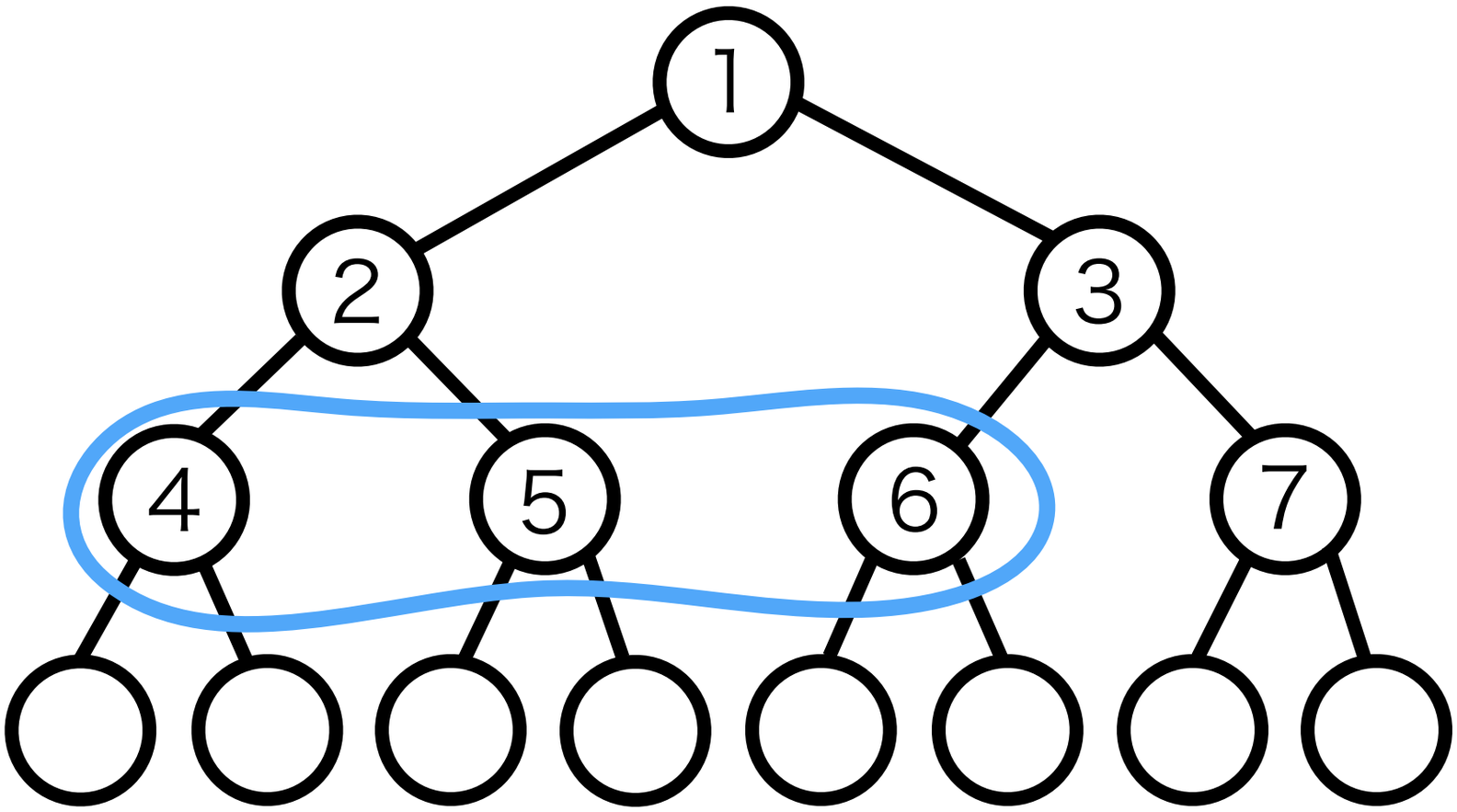
- 木について、”高さiの頂点を全てswapする”というクエリになるように分割したい

- たとえば、RMQ(Range minimum Query)をSegTreeでやるとどうなるか？

- [a, b]の最小値というクエリを、O(logn)個の”木の全ての葉の値の最小値”というものに分割している

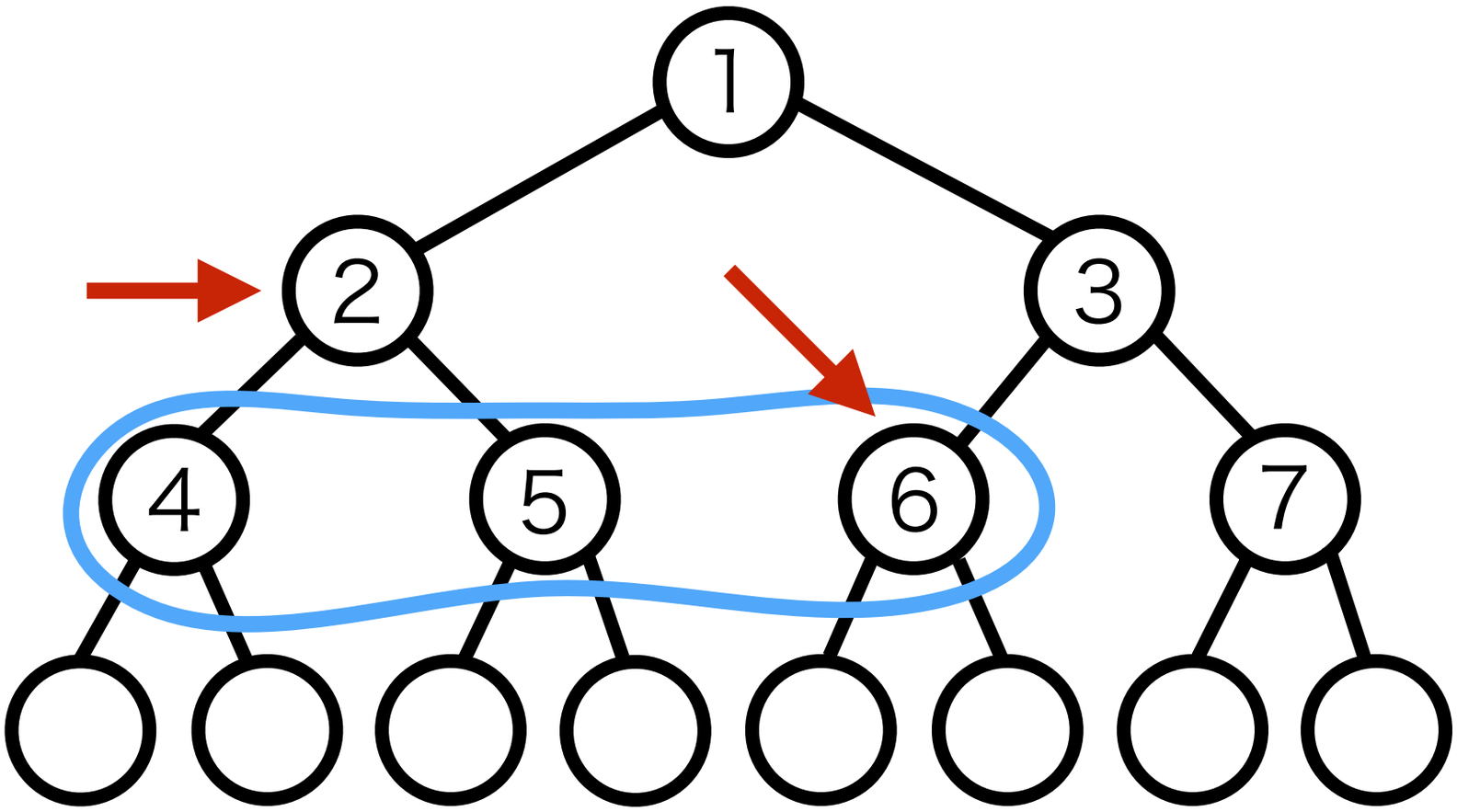
- 今回は、i段目の[a,b]のswapというクエリなので、O(logn)個の”木のi段目の全ての頂点をswap”というクエリに分割できる

例



これにswap(4), swap(5), swap(6)を行うとする

例



これは、位置2, 位置6の頂点に対して、

”高さ3の頂点を全てswap”

というクエリになる

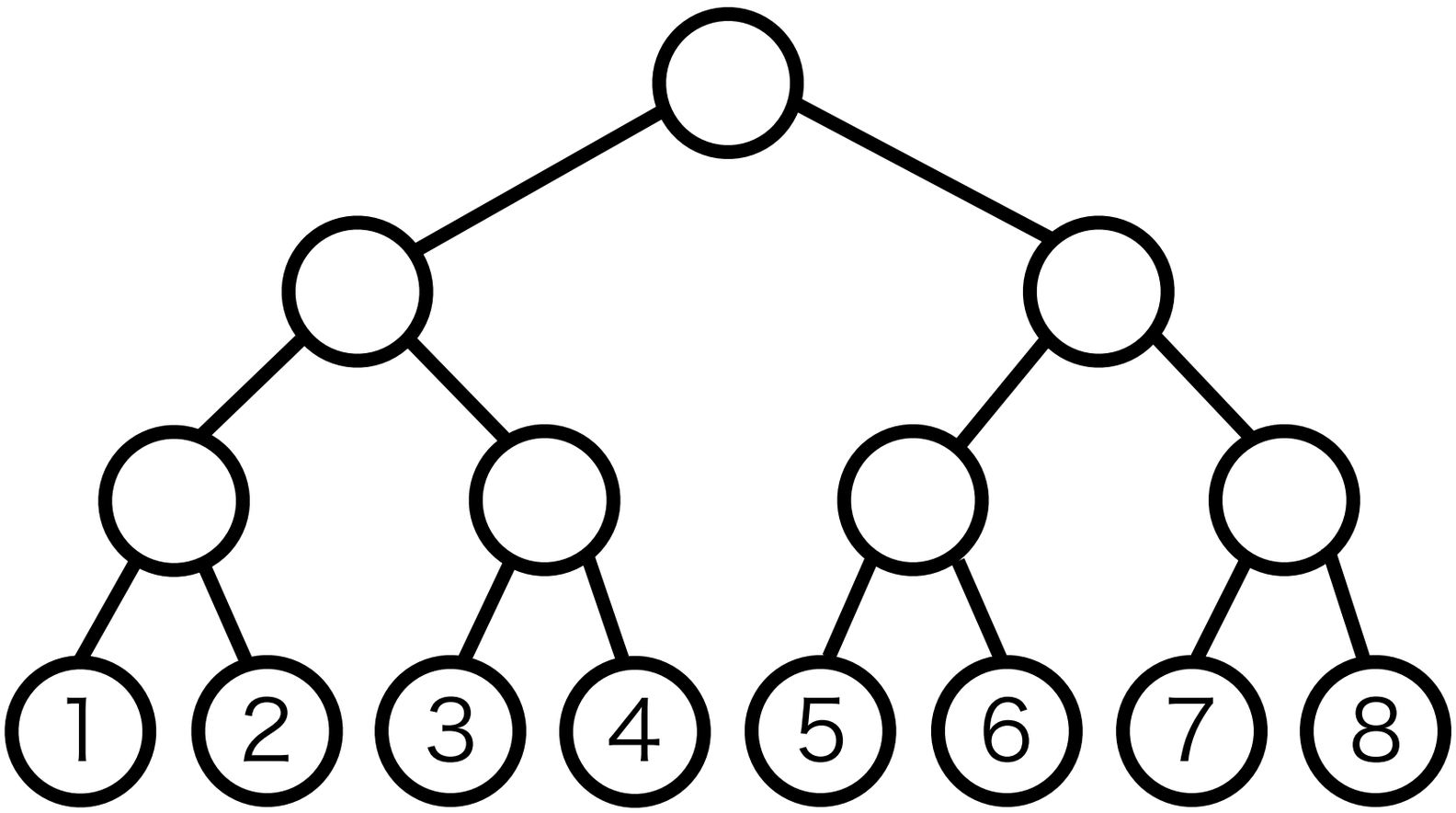
i段目を全てswap

- “i段目を全てswap”というクエリになると何が嬉しい？

- とりあえず、位置1(木の根)に対して”i段目を全てswap”というクエリだけがくると考える

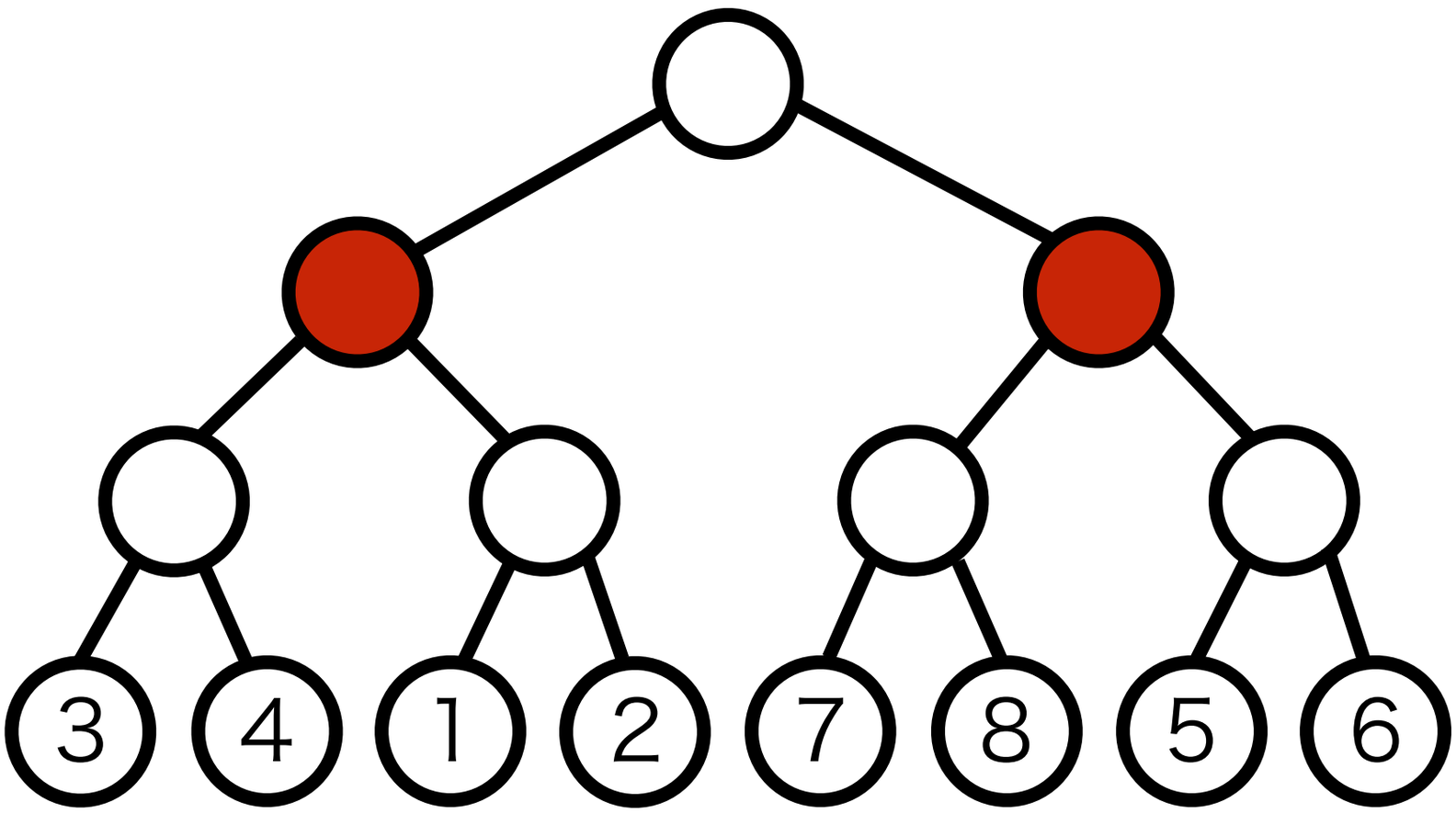
- こうすると、”i段目を全てswap”→”j段目を全てswap”をしても、”j段目を全てswap”→”i段目を全てswap”しても、できる木は変わらないことがわかる

例



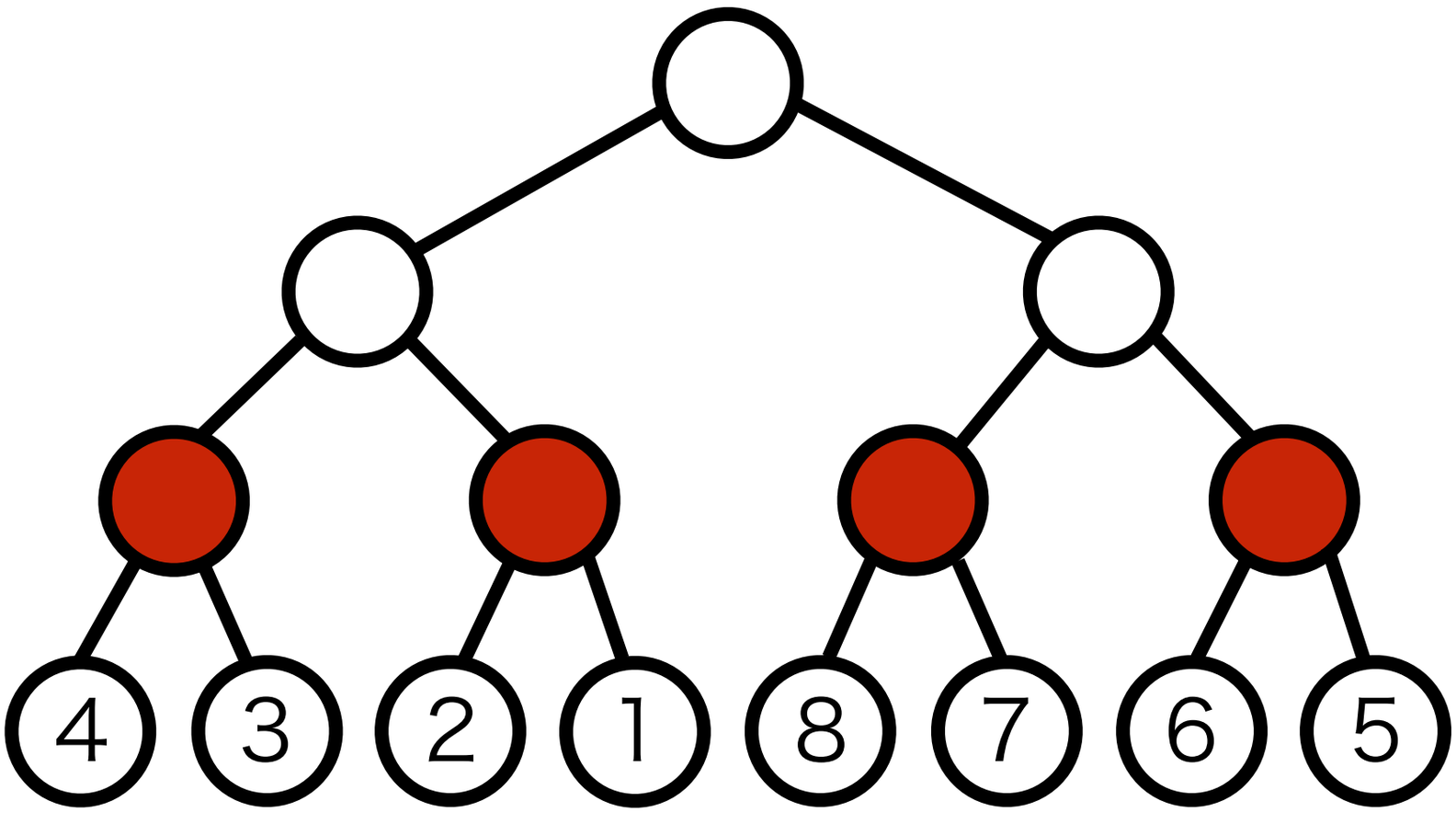
高さ2をswap → 高さ3をswap

例



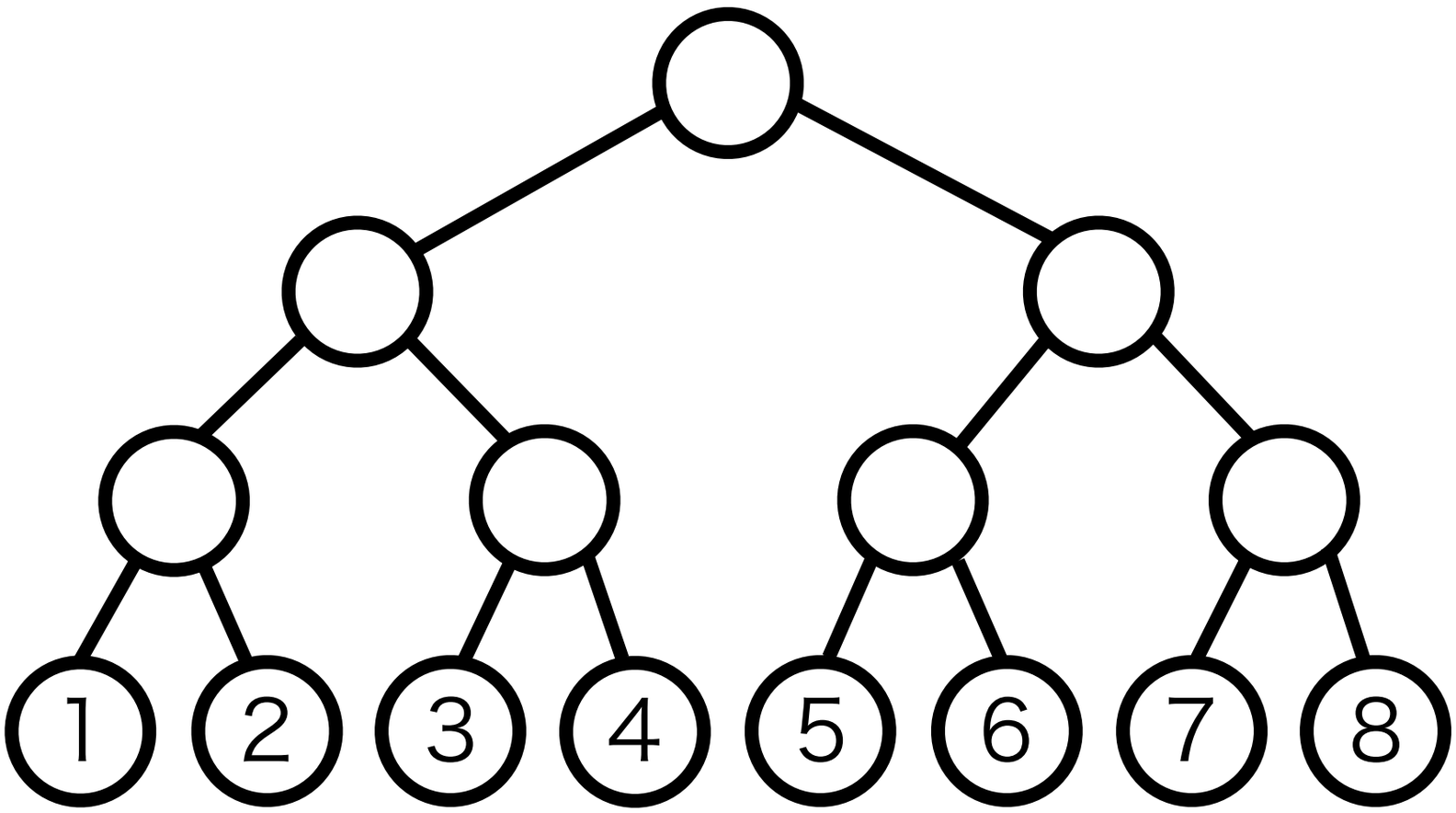
高さ2をswap → 高さ3をswap

例



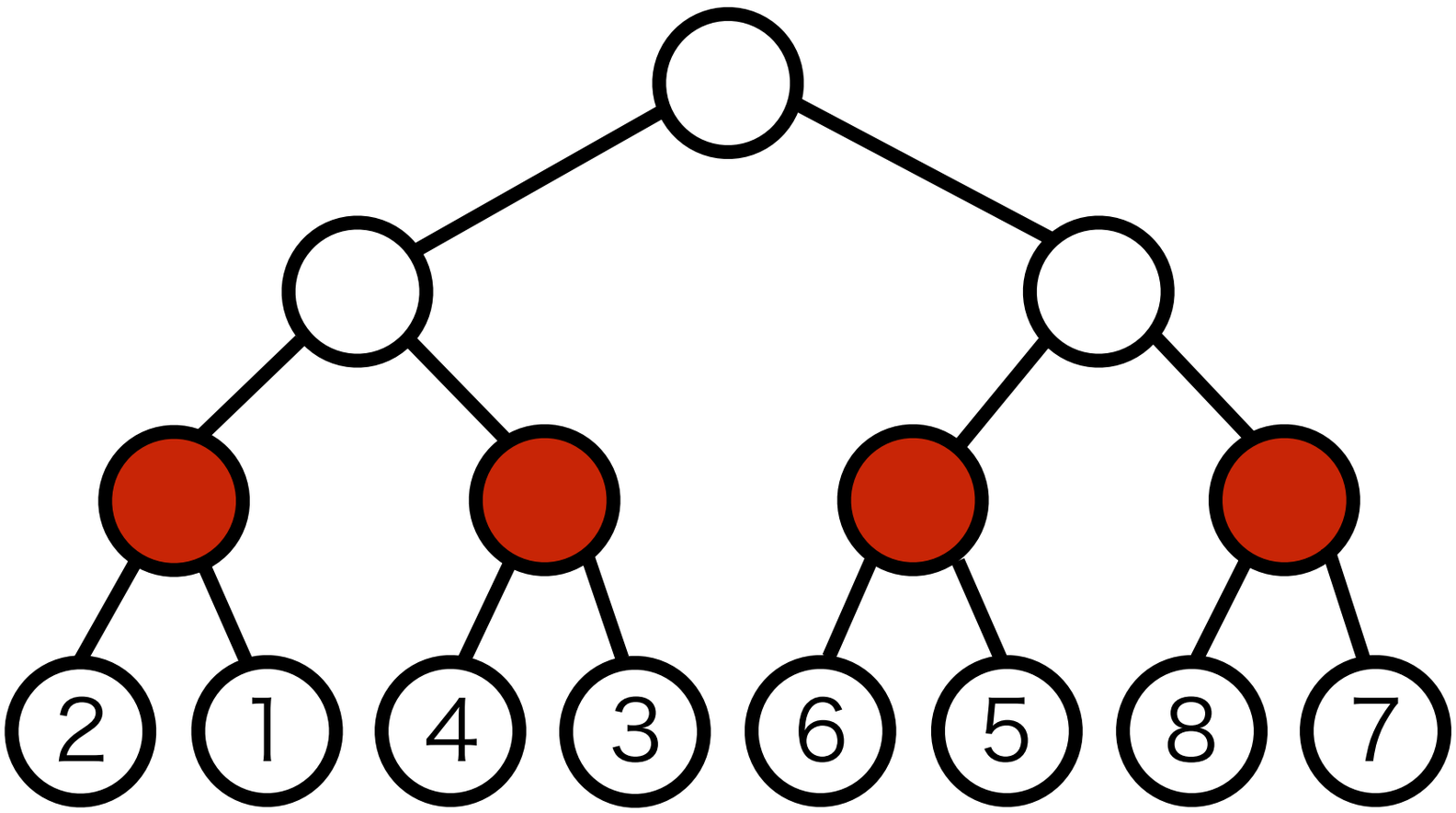
高さ2をswap → 高さ3をswap

例



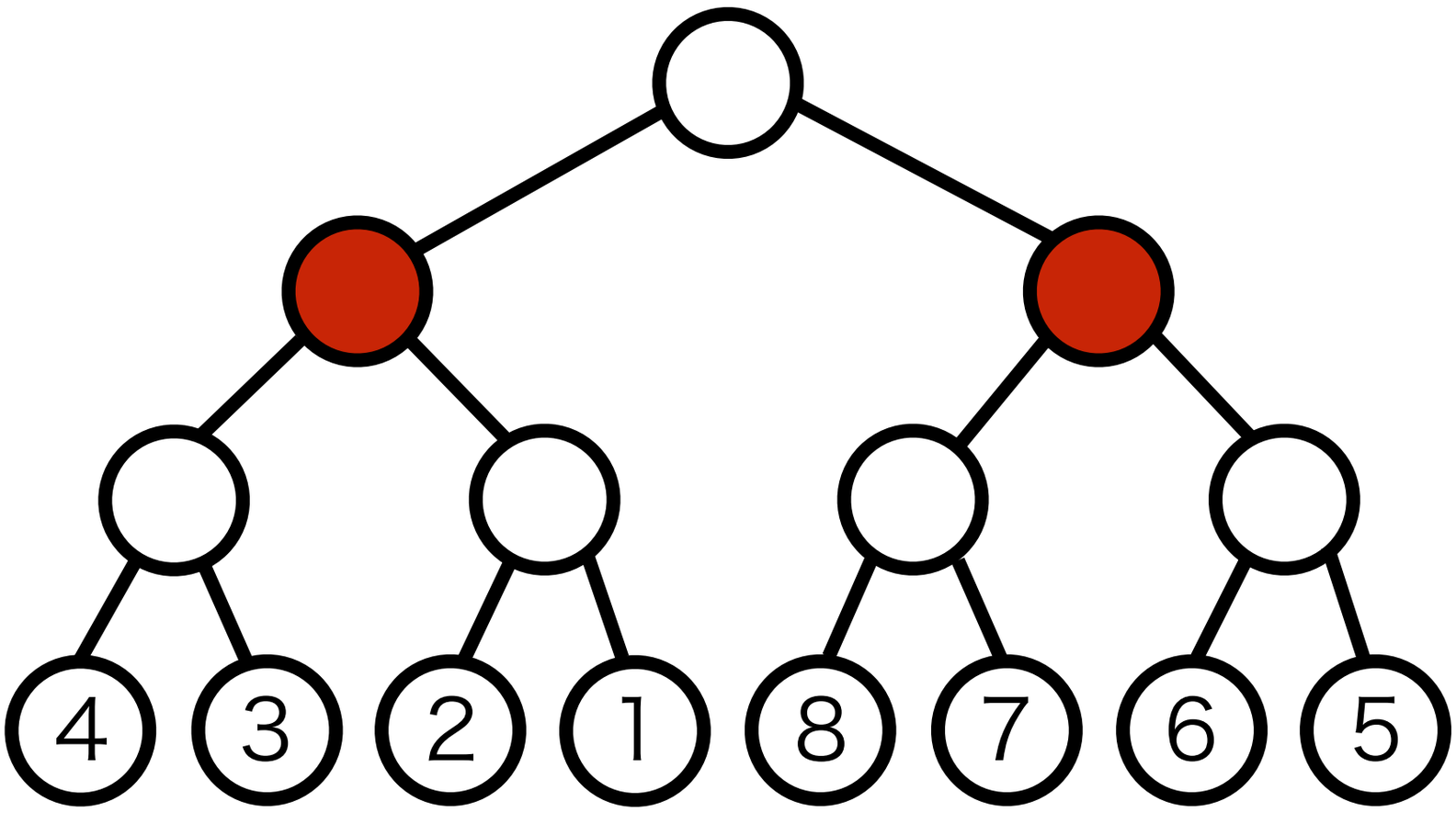
高さ3をswap → 高さ2をswap

例



高さ3をswap → 高さ2をswap

例



高さ3をswap → 高さ2をswap

i段目を全てswap

- 結局、それぞれの高さごとに”反転しているか”というフラグを持てば良い

- “高さiを反転”というクエリが来たら、それに対応するフラグを反転させる

i段目を全てswap

- 位置1(木の根)以外にもクエリが来るなら?

- 先述のflagを各頂点にもたせて、伝搬させていく

- 高さiのノードにアクセスするときは、i番目のフラグが立っていたら左右の子をswapして、i+1番目以降のフラグを全て子のフラグに伝搬させる

i段目を全てswap

- これをsegtreeを潜る時に行うと、アクセスするノードの上の頂点は常に何もフラグが立っていないようにできるので、うまく動く

- 計算量はクエリを分割する個数がO(logn)なので、クエリあたりO(logn)

ARC048 解説

DEGwer

A問題 階段の下

問題概要

- A階からB階まで、何回階段を上る必要があるか?
- ただし、x>0に対し、-x階は地下x階を表す
- $-10^9 \leq A < B \leq 10^9$

解法

- 地下1階の上は地上1階
- それ以外に関しては、k階の上はk+1階

場合分け①

- AとBの符号が等しい場合

- 「地下1階と1階の間」の階段を使わないので、B-Aが答え

場合分け②

- AとBの符号が異なる場合

- 「地下1階と1階の間」の階段を使うので、①に比べて使う階段の数が一つ減る
- B-A-1が答え

- 以上でこの問題が解けた

B問題 AtCoderでじゃんけんを

問題概要

- N人の人がいて、レーティングとじゃんけんで出す手が決まっている
- AtCoderじゃんけんでは、二人のレーティングが異なる場合高いほうが勝ち、そうでない場合はじゃんけんの結果で勝敗を決める
- 各人について、ほかのN-1人とAtCoderじゃんけんをした時の対戦成績が何勝何敗何引き分けか求めよ
- N<=100000, レーティング<=100000

解法

- 各レーティングと手ごとに、人のIDを要素とした配列を持っておく
- 変数Kを「今見ているレーティングより低いレーティングの人の人数」として使うことにする
- レーティングが低いほうから見ていく
- 最初はK=0

解法

- 各レーティングについて、そのレーティングでグー、チョキ、パーを出した人の人数をそれぞれA,B,Cとすると、グーを出した人の対戦成績は、K+B勝N-K-A-B敗A-1引き分けとなる
- チョキ、パーについても同様に計算できるので、合計O(N+(レーティングの最大値))でこの問題が解けた

C問題 足の長い高橋君

問題概要

- N個の0,1からなる文字列の組であって、以下の条件を満たすものの個数をmod 10億7で答えよ(N<=100000)
- 条件:
  - i番目の文字列の長さは$L_i (L_i \leq 10^9)$
  - 任意の$i \neq j$に対し、i番目の文字列の後にj番目の文字列をひっくり返したものをつなげたものが回文となる

考察

- 一番短い文字列の長さをDとおくと、すべての文字列の先頭D文字は等しい
  - 一番短い文字列の後ろにほかの文字列をひっくり返したものをつなげたものが回文となるため
  - 逆に、各文字列の先頭D文字に関してはこれ以上の制約がつくことはない

考察

- 各文字列に対し、その先頭D文字を取り除いた文字列は回文
  - その文字列に、一番短い文字列をひっくり返したものをつなげたものが回文となるため

考察

- 2つの異なる文字列に対し、1つめの文字列の先頭D文字を取り除いたものの後ろに2つめの文字列の先頭D文字を取り除いたものをひっくり返したものをつなげたものは回文

考察

- 2つの異なる文字列に対し、1つめの文字列の先頭D文字を取り除いたものの後ろに2つめの文字列の先頭D文字を取り除いたものをひっくり返したものをつなげたものは回文

- 赤字の2つは前の考察により回文だとわかっている
- 「回文と回文をつなげて回文をつくっている」という主張

考察

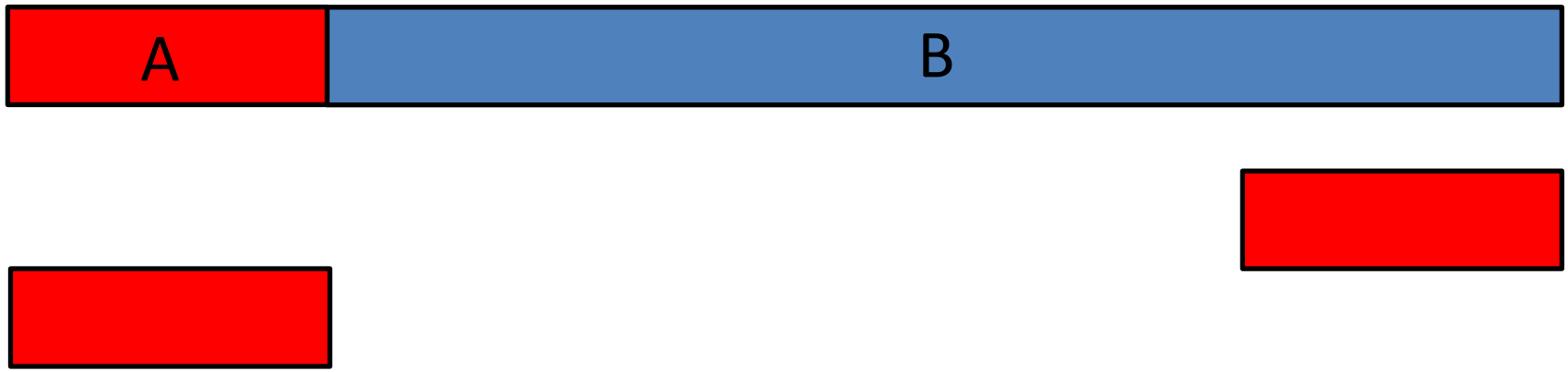
- 次の補題を考える
- 「長さXの回文Aと長さYの回文Bをつなげると回文になるとき、AもBも周期gcd(X,Y)をもち、さらにその1周期は回文」
- 以下これを示す

証明

- AとBは回文で、以下のようにAとBをつなげたものも回文

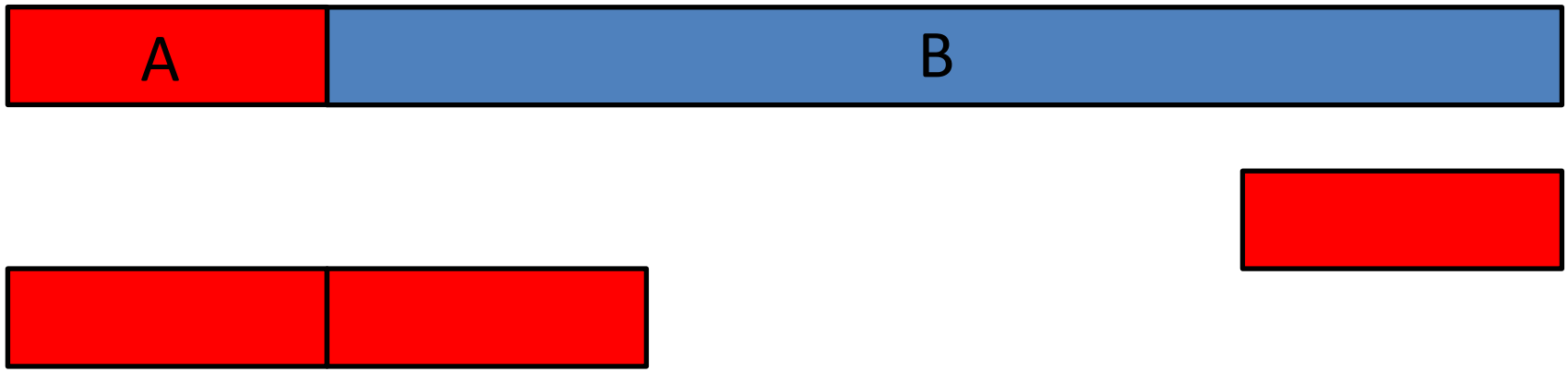
証明

- Aが回文で全体が回文なので、下の赤い領域は同じ文字列



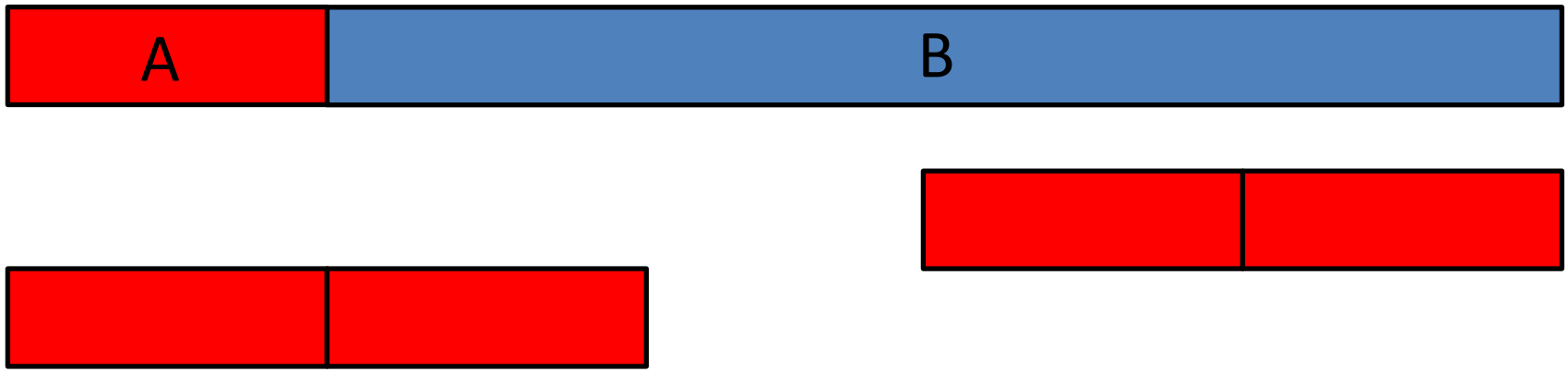
証明

- Bが回文なので、ここも等しい



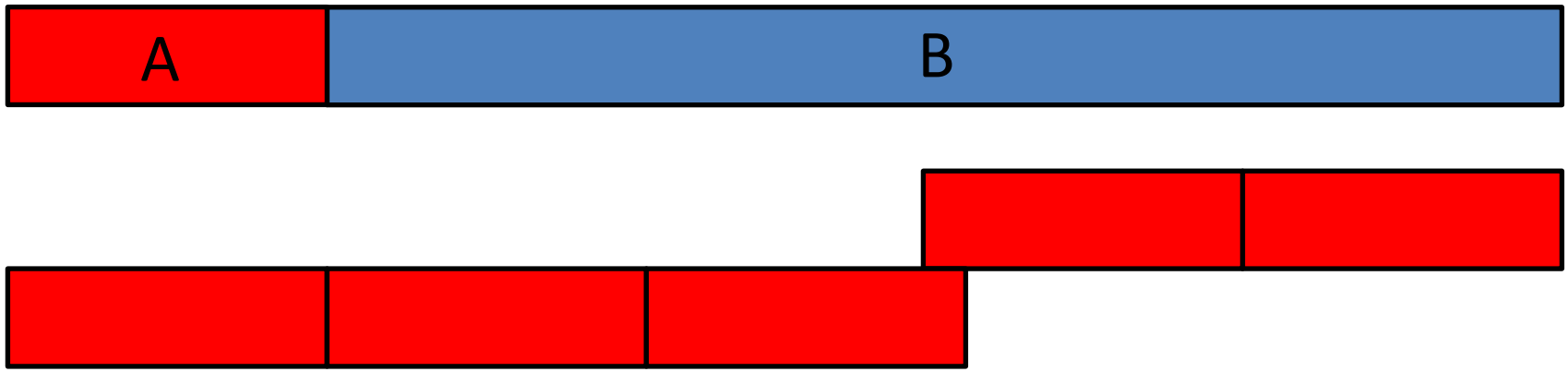
証明

- 全体が回文なので、ここも等しい



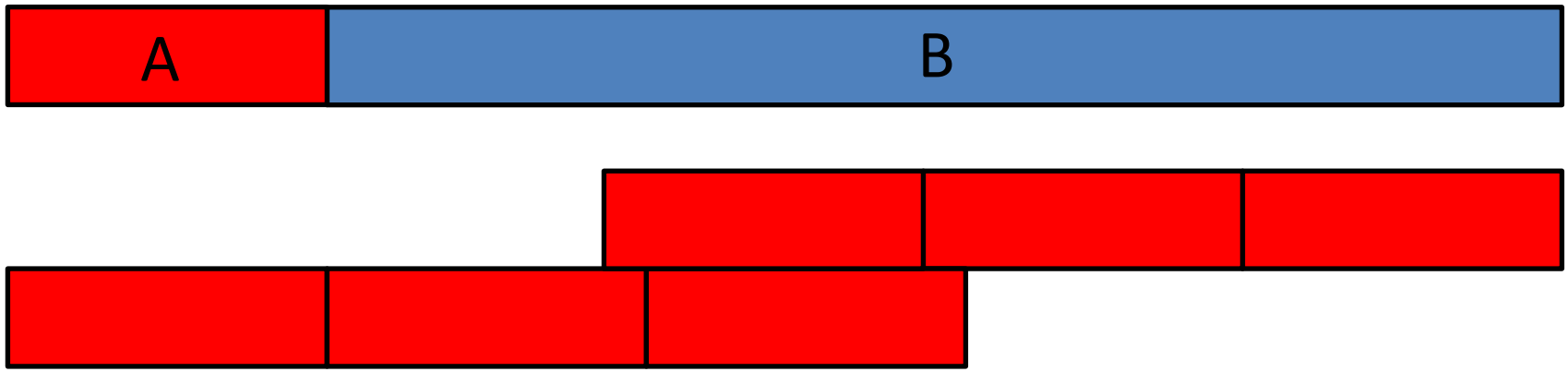
証明

- Bが回文なので、ここも等しい



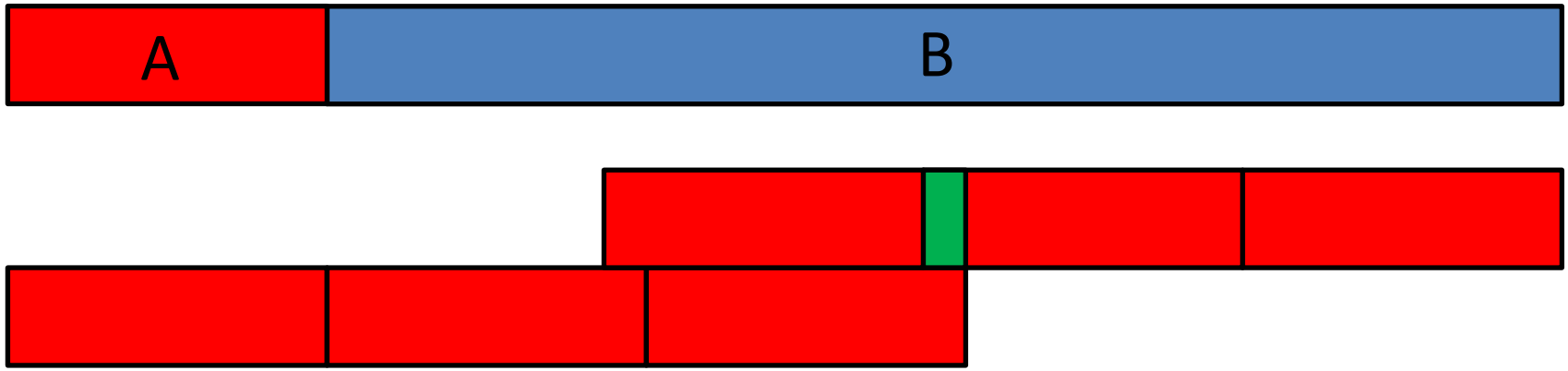
証明

- 全体が回文なので、ここも等しい



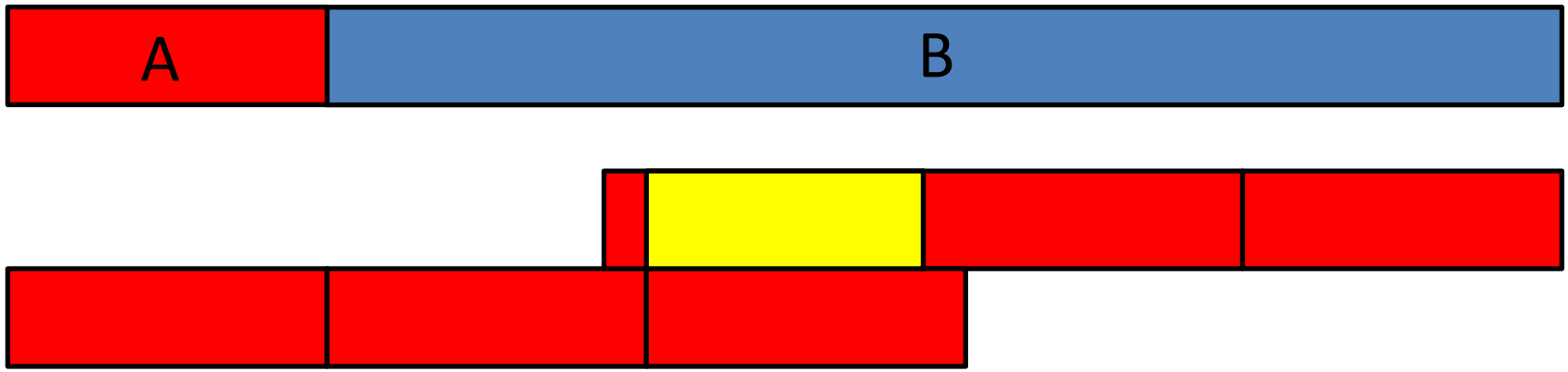
証明

- この緑色の部分は、同じ回文の先頭と末尾の共通部分なので回文



証明

- この黄色の部分も、同じ回文の先頭と末尾の共通部分なので回文



証明

- 赤、黄、緑すべての部分が回文
- これは最初と同じ状況
- 黄色い部分の長さは、AとBをつなげた文字列の長さをAの長さで割った余り
- (黄色の長さ, 赤の長さ)の組は(A+B,B)からEuclidの互除法の1ステップで得られるものと同じ

証明

- 黄色の長さが0でない限りこの操作は続けられる
- この終了条件もEuclidの互除法と同じ
- 結局、この操作でEuclidの互除法と同じことができ、Euclidの互除法はGCDを正しく求めるので、結局AもBも周期gcd(X,Y)を持ち、その1周期は回文であることが分かった

解法

- この補題を元の問題に適用する
- 2つの異なる文字列に対し、1つめの文字列の先頭D文字を取り除いたものの後ろに2つめの文字列の先頭D文字を取り除いたものをひっくり返したものをつなげたものは回文

解法

- つまり、1つめの文字列(長さX)の先頭D文字を取り除いたものと2つめの文字列(長さY)の先頭D文字を取り除いたものは、どちらも周期gcd(X,Y)を持ち、その1周期は回文
  - 逆にこの条件が満たされていれば大丈夫なことは明らか

解法

- つまり、$L_i - D$たちのGCDをgとおくと、各文字列は、先頭D文字が等しく、かつそのあとは周期gの同じ回文が並ぶ文字列となる
- このような文字列の総数は$2^{D+\left\lceil \frac{g}{2} \right\rceil}$通りであるので、O(N+log ($L_i$たちのmax))でこの問題が解けた

D問題 たこ焼き屋とQ人の高橋君

問題概要

- N頂点の重みなし無向木があり、いくつかの頂点にはたこ焼き屋がある
- 次のクエリをQ個処理せよ
- クエリ:「始点と終点が与えられるので、たこ焼き屋による前は速度1/2で進みたこ焼き屋に寄ったあとは速度1で進む人が始点から終点まで行く最小の時間を求めよ」
- N,Q<=100000

考察

- 最適な移動経路は次の2つのうちのいずれか
- ①: たこ焼き屋を経由せずに始点から終点まで最短経路で向かう
- ②: 始点からあるたこ焼き屋まで最短経路で向かった後、そのたこ焼き屋から終点まで最短経路で向かう

考察

- 最適な移動経路は次の2つのうちのいずれか
- ①: たこ焼き屋を経由せずに始点から終点まで最短経路で向かう
- ②: 始点からあるたこ焼き屋まで最短経路で向かった後、そのたこ焼き屋から終点まで最短経路で向かう

考察

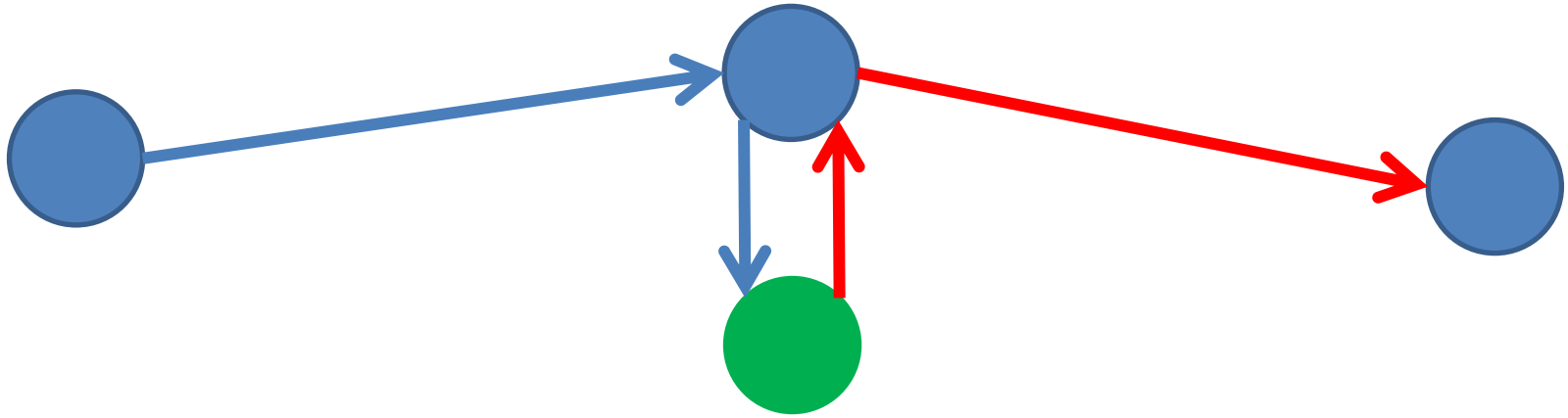
- ①: たこ焼き屋を経由せずに始点から終点まで最短経路で向かう
- このケースは、単に始点と終点のLCAを求めて深さを見ればいい

考察

- 最適な移動経路は次の2つのうちのいずれか
- ①: たこ焼き屋を経由せずに始点から終点まで最短経路で向かう
- ②: 始点からあるたこ焼き屋まで最短経路で向かった後、そのたこ焼き屋から終点まで最短経路で向かう

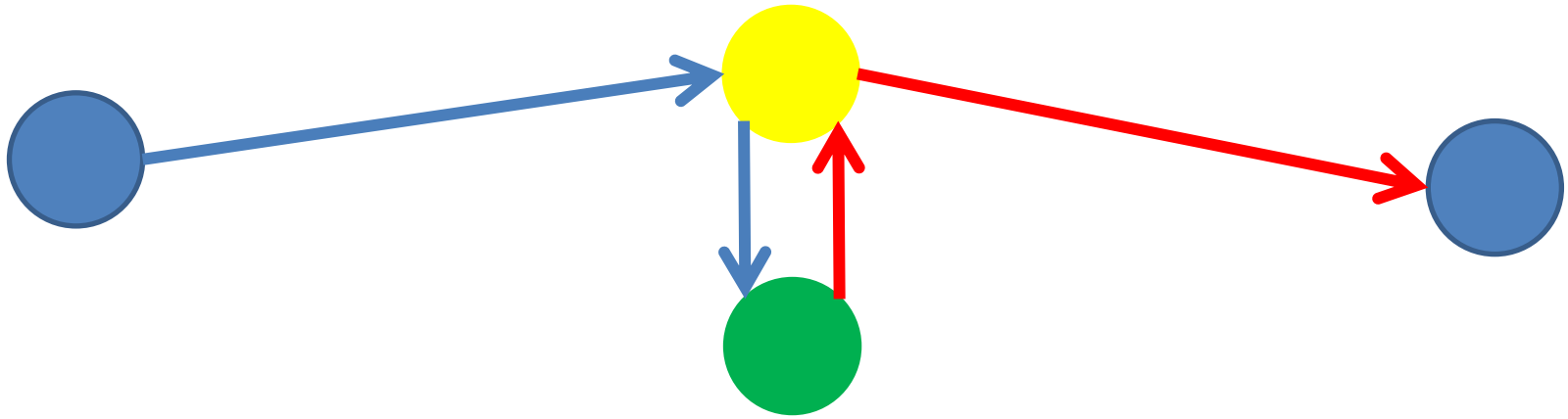
考察

- ②: 始点からあるたこ焼き屋まで最短経路で向かった後、そのたこ焼き屋から終点まで最短経路で向かう
- それは以下のような経路(緑がたこ焼き屋、青い辺は速度1/2で進むパス、赤い辺は速度1で進むパスを表す)



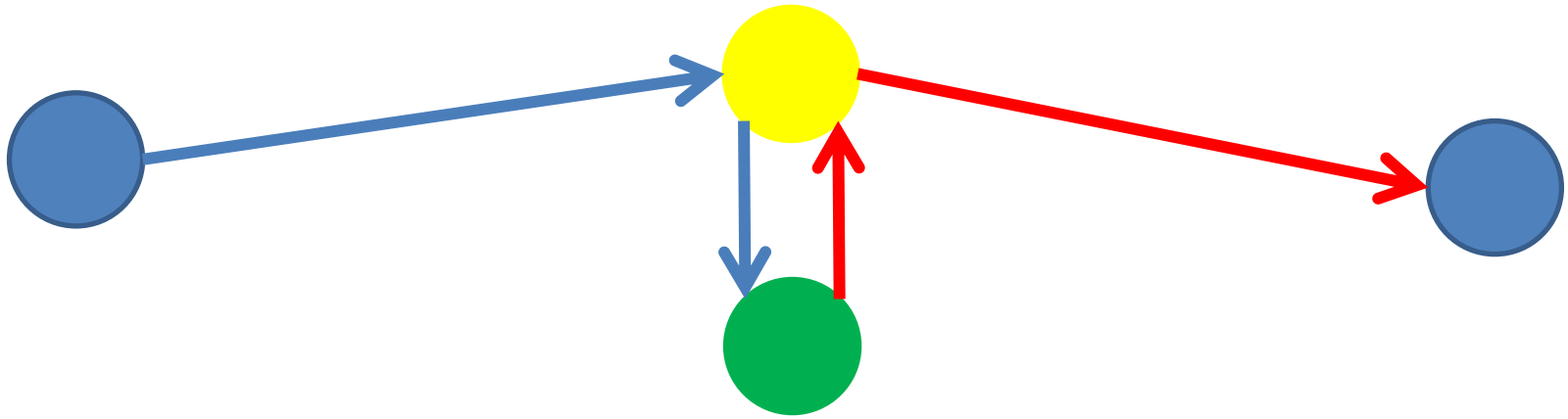
考察

- 与えられるグラフは木なので、黄色の点は始点と終点を結ぶ最短経路上にあり、この経路を通る時間は
- d(始点,終点)+d(始点,黄色)+d(黄色,緑)*3
  - ただしd(A,B)でA-B間の最短路長を表す



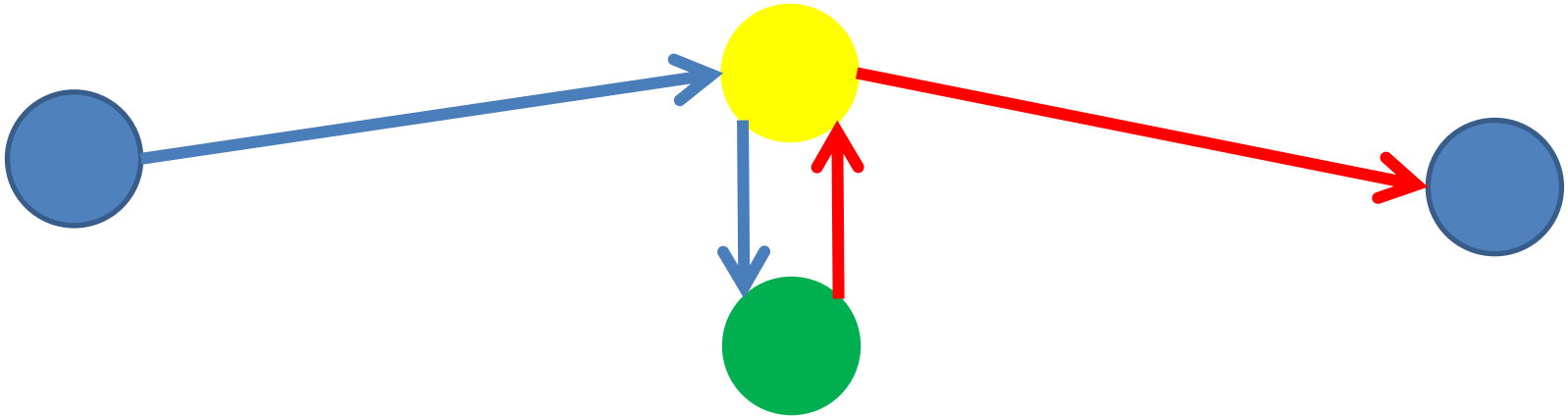
考察

- つまり、この問題は、始点と終点間のパス上の頂点を黄色の頂点、たこ焼き屋のある頂点を緑の頂点としてうまく選び、
- d(始点,終点)+d(始点,黄色)+d(黄色,緑)*3
- を最小化するクエリにこたえる問題



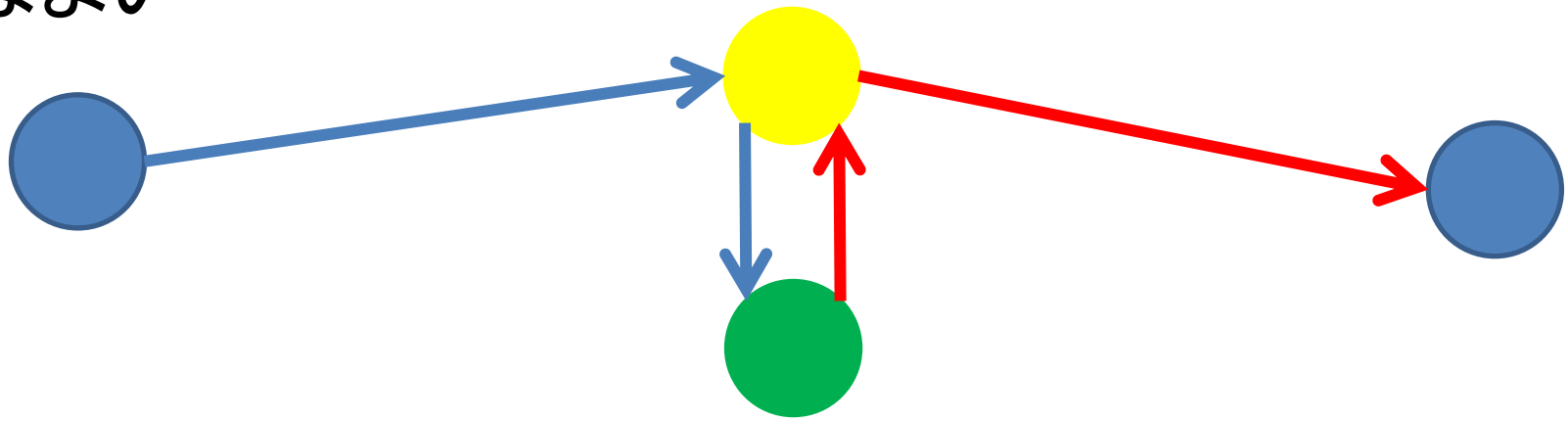
考察

- 黄色の頂点を固定するとd(黄色,緑)を最小化すればよく、黄色の頂点から一番近いたこ焼き屋のある頂点を緑の頂点として選べばよい
  - 各頂点から一番近いたこ焼き屋のある頂点への距離は、たこ焼き屋のある頂点を最初に全部queueに入れて幅優先探索をすれば求まる



考察

- 各頂点vから一番近いたこ焼き屋のある頂点までの距離の3倍をX(v)とおくと、
- d(始点,終点)+d(始点,黄色)+X(黄色)
- の最小値を求めればよい
- つまり、d(始点,黄色)+X(黄色)を最小化すればよい

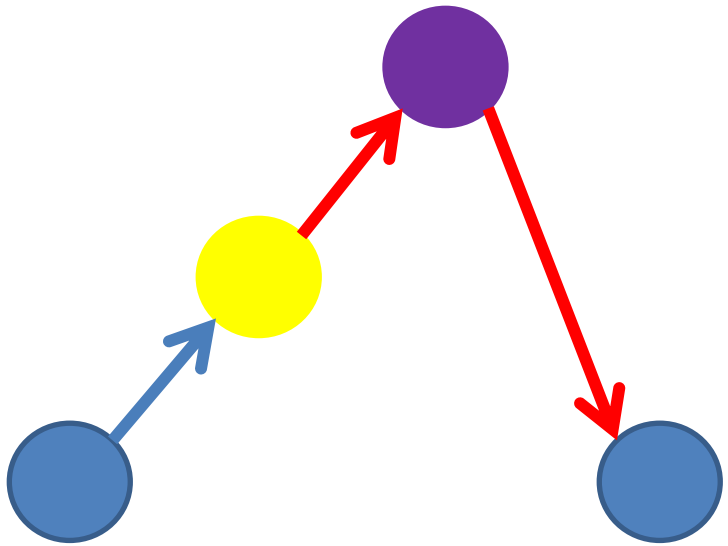


解法

- まず、適当な頂点を根とする根つき木にしておく
- depth(v)で頂点vの根からの深さを表すことにする
- 始点と終点のLCAをLとおく

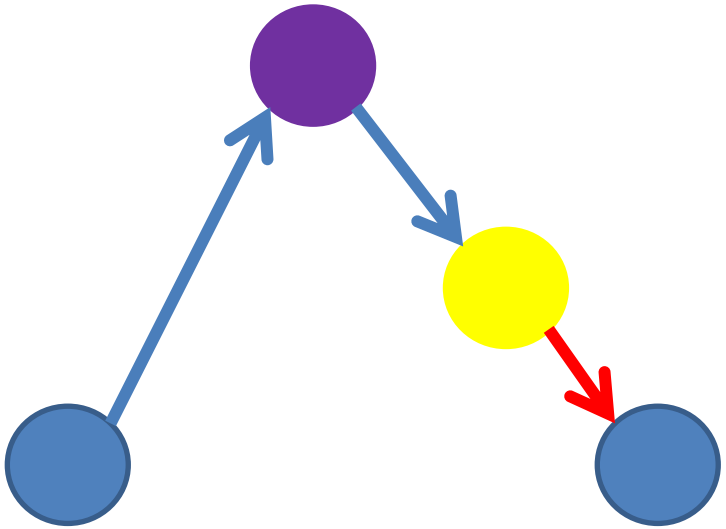
場合分け

- 黄色い頂点をLより始点側にとる場合
- d(始点,黄色)+X(黄色)=depth(始点)-depth(黄色)+X(黄色)
- よって、始点からLまでのパス上の
- X(黄色) -depth(黄色)
- の最小値を求めればよい



場合分け

- 黄色い頂点をLより終点側にとる場合
- d(始点,黄色)+X(黄色)=depth(始点)+depth(黄色)-2*depth(L)+X(黄色)
- よって、Lから終点までのパス上の
- X(黄色) +depth(黄色)
- の最小値を求めればよい



解法

- どちらの場合も、X(黄色) ±depth(黄色)をあらかじめ計算しておけば、この最小値はダブリングを用いて求めることができる
- 前処理はLCAの初期化でO(Nlog N)
- 各クエリに対してはダブリングのO(log N)
- で計算できるので、O((N+Q)log N)でこの問題が解けた

ARC047解説

A:タブの開きすぎ

A-問題概要

- L個以上タブを開くとクラッシュして、タブが1個に減ってしまうブラウザがある
- 初めタブは1個だとして、そのあとの「タブを開く」「タブを閉じる」の履歴が与えられるので、クラッシュの回数をもとめよ。

A-考察

- タブの個数以外気にしなくて良い。

A-解法

- タブの個数を変数で管理しておく。

A-解法

- タブの個数を変数で管理しておく。
- タブを閉じるとき
  - 変数の値を1減らす

A-解法

- タブの個数を変数で管理しておく。

- タブを閉じるとき
  - 変数の値を1減らす
- タブを開くとき
  - タブの個数がL個未満ならば、変数の値を1増やす

A-解法

- タブの個数を変数で管理しておく。

- タブを閉じるとき
  - 変数の値を1減らす
- タブを開くとき
  - タブの個数がL個未満ならば、変数の値を1増やす
  - タブの個数がL個ならば変数の値を1にして、クラッシュの回数を1増やす

ARC047解説

B:同一円周上

B-問題概要

- 格子点上の点が N 個与えられる。
- 全ての点からのマンハッタン距離が等しい格子点上の点Pを1つもとめよ

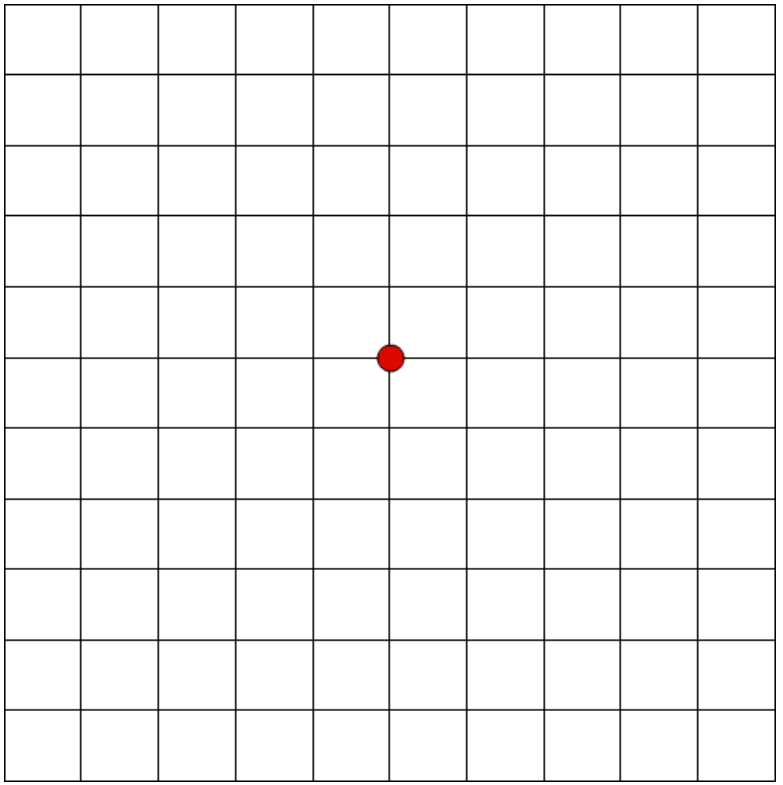
- N ≦ 10^5
- -10^9 ≦ 座標の値 ≦ 10^9

B-考察

- ある点からマンハッタン距離が等しい点の集合はどのようなものか？

B-考察

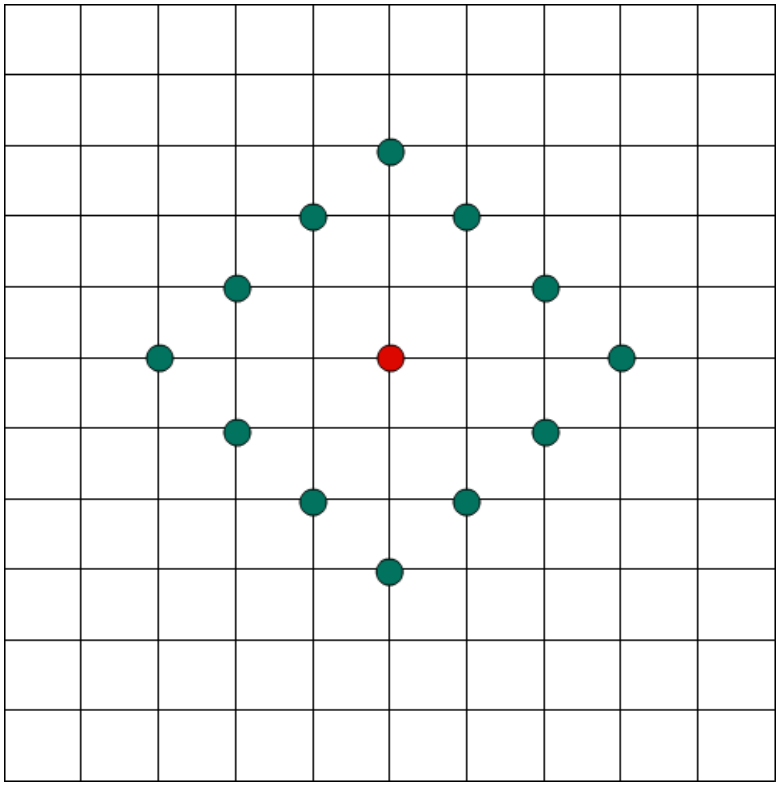
- ある点からマンハッタン距離が等しい点の集合はどのようなものか？



点P

B-考察

- ある点からマンハッタン距離が等しい点の集合はどのようなものか？



点Pからマンハッタン距離3の点

B-考察

- ある点からマンハッタン距離が等しい点の集合はどのようなものか？

  →45度傾いた正方形の外周になる

B-考察

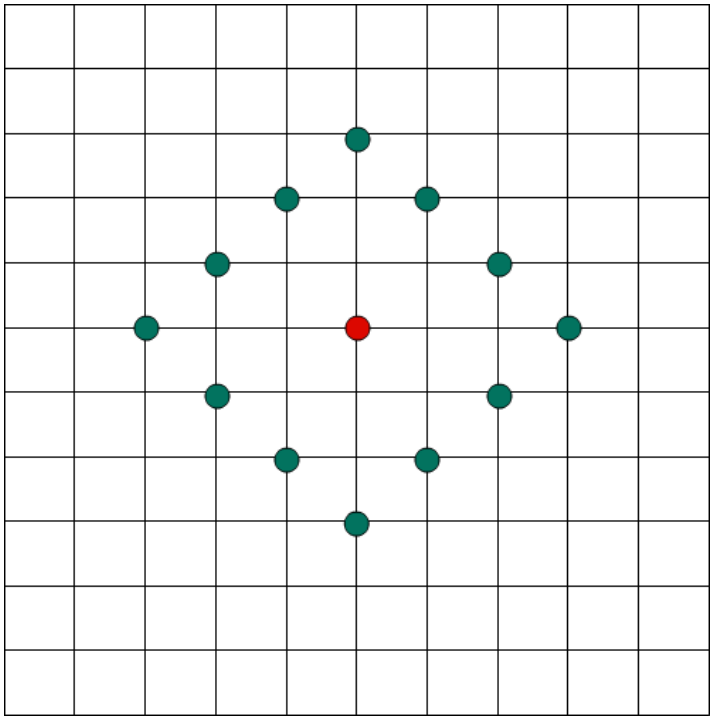
- ある点からマンハッタン距離が等しい点の集合はどのようなものか？

  →45度傾いた正方形の外周になる

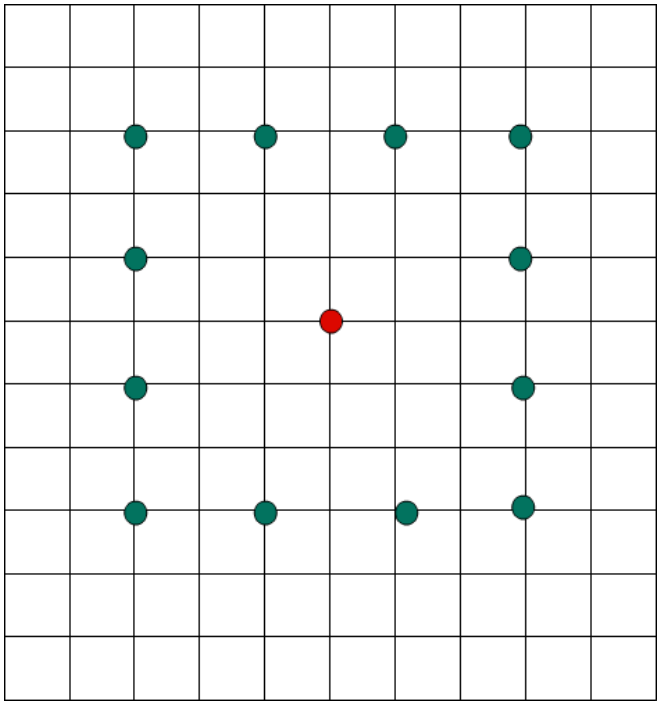
- この正方形の大きさがわかれば、点Pのおおよその見当がつく。

B-45度傾ける

- 45度傾いた状態で扱うのはわかりづらいので、もとに戻すために、X,YではなくX+Y,X-Yで各点を表してみる。
- すると軸に平行な正方形がでてくる。



→

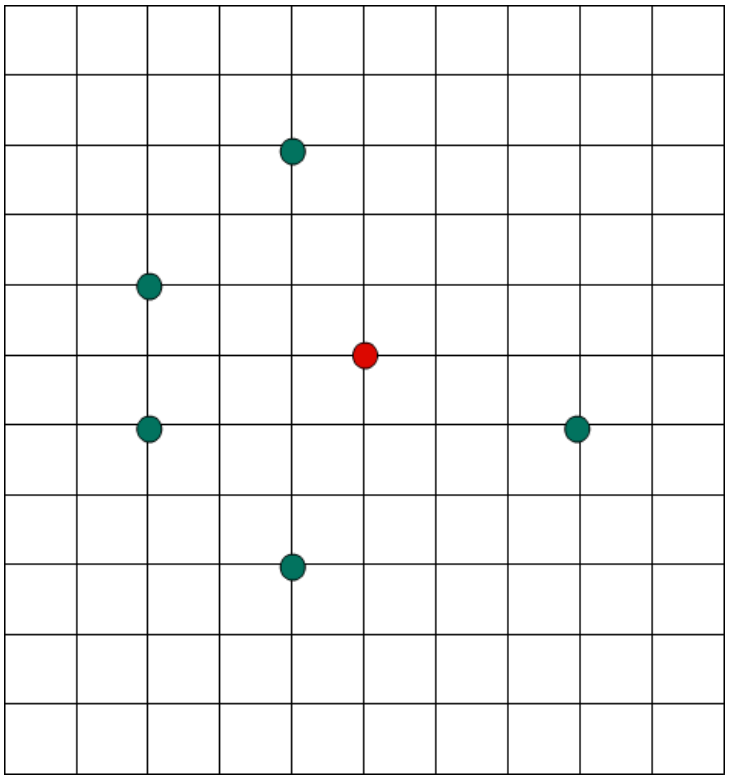


B-考察

- (回転後の)X座標の最大値と最小値の差、と（回転後の）Y座標の最大値と最小値の差のうち大きい方が、正方形の１辺の長さと一致する。

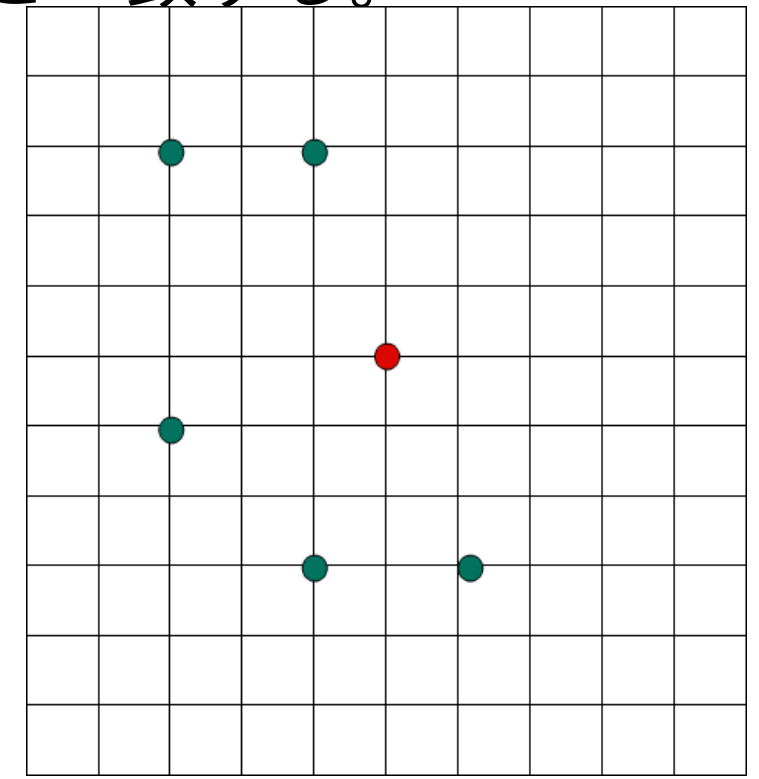
B-確認

- 4辺の内全ての辺から1つ以上残っている場合
  - Xの最大値と最小値の差も、Yの最大値と最小値の差も一辺の長さと一致する。



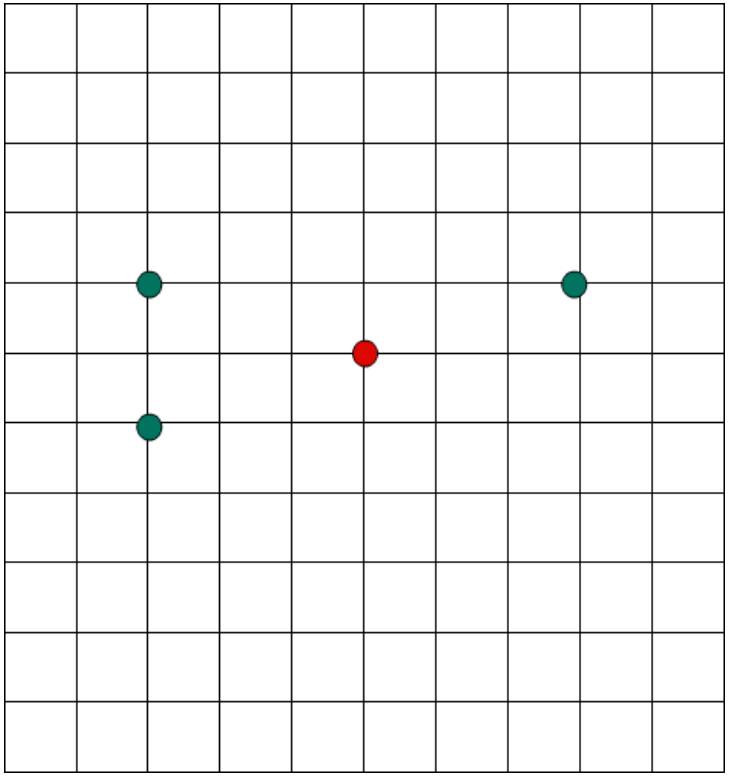
B-確認

- 4辺の内3つの辺から1つ以上残っている場合
  - Xの最大値と最小値の差か、Yの最大値と最小値の差の大きい方が一辺の長さと一致する。



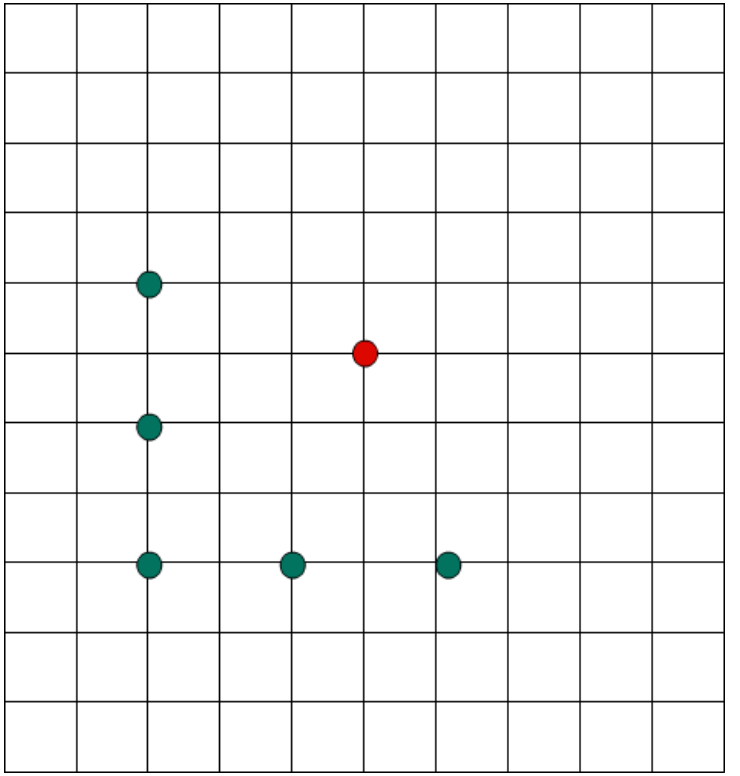
B-確認

- 4辺のうち向かい合う2辺について1つ以上残っている場合
  - Xの最大値と最小値の差か、Yの最大値と最小値の差の大きい方が一辺の長さと一致する。



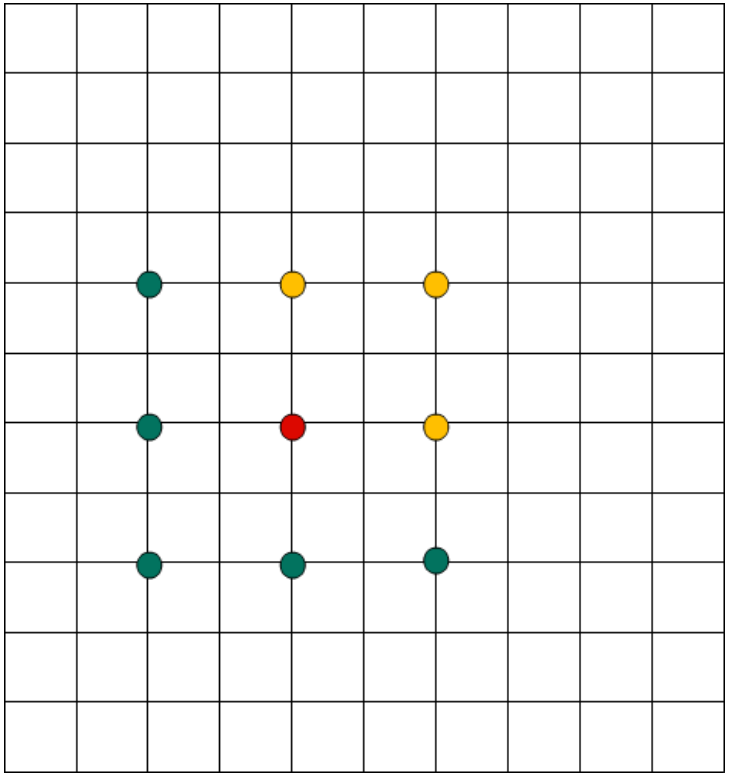
B-確認

- 4辺のうちとなりあう2辺について1つ以上残っている場合
  - Xの最大値と最小値の差か、Yの最大値と最小値の差の大きい方が一辺の長さと一致しない？？？



B-確認

- 4辺のうちとなりあう2辺について1つ以上残っている場合
  - Xの最大値と最小値の差か、Yの最大値と最小値の差の大きい方が一辺の長さと一致する点Pがある。



B-考察

- 1辺の長さについては、X座標の最大値と最小値の差とY座標の最大値と最小値の差をみて、大きい方をとればよい。
- １辺の長さをDとする。与えられたX座標、最大最小をXmax,Xminで表し、点PのX座標、Y座標をPx,Pyとすると

  Xmax-Px, Px -XminのいずれかがD/2と一致する。

  - Y座標も同様
- よってDの値と全体のX座標、Y座標の最大最小から点Pの候補が有限個に決まる

B-考察

- さきほどの考察は45度回転した後の世界での計算である。
- 回転後のXやYの値は、回転前のX+YやX-Yの値と対応するので、実装するときはその値でプログラムを組まなければならない。

B-解法

- 与えられた点について(X+Y)の最大値と最小値の差、（X-Y）座標の最大値と最小値の差を求め、大きい方をDとする
- 先ほどの考察を使ってPx,Pyの値を有限個にしぼりこむ
- そのうちすべての点とのマンハッタン距離が等しい点を挙げれば良い

ARC047解説

C:N!÷K番目の単語

C-問題概要

- [1,2,..,N]を並び替えたものを「単語」という
- 単語のうち辞書順で小さい方から N! / K番目(1-index)のものを求めよ

- K ≦ N ≦ 10^5

C-考察1

- 単語の中で辞書順X番目(0-index)のものを求めるにはどうすればいいか？

C-考察1

- 単語の中で辞書順X番目(0-index)のものを求めるにはどうすればいいか？

  → 1文字目から順番に決めていけば良い

C-X番目の求め方

- 1文字目が1であるような単語はいくつあるか？
  - 2文字目以降の並びを考えると(N-1)!通り
- 1文字目が2であるような単語はいくつあるか？
  - 同様に(N-1)!通り
- X番目の単語の1文字目は何か？
  - 上の考察より、[X / (N-1)!] + 1
  - []はガウス記号

C-X番目の求め方

- 1文字目が求まった！→Aとする
- つぎにもとめるべきものは？
  [1,2,..., A-1, A+1,...N]を並び替えたものの中で
  X % (N-1)!番目の物
- さきほどと同様の計算で求めることが出来る。
- 順番に繰り返せばすべての桁の値が決まる。
- 以降は[X / (N-1)!] + 1ではなくて[X / (N-1)!] + 1番目に小さい値であることに注意

C-部分点

- いまのような計算方法でN!/K番目の値をもとめる。

- N ≦ 20なのでN!/Kは64bit整数に収まるので、様々な言語で扱える。

C-考察2

- Nが大きいと N!/Kを陽に持つことは出来ない。

C-考察2

- Nが大きいと N!/Kを陽に持つことは出来ない。
- やりたい操作は
  - (N-1)!で割る
  - その余りを求める

C-考察2

- Nが大きいと N!/Kを陽に持つことは出来ない。
- やりたい操作は
  - (N-1)!で割る
  - その余りを求める

実際にやってみる

C-考察2

- N!/Kを(N-1)!で割った商は
  - (N/K)×(N-1)! ÷ (N-1)!　の商
  - → N÷K　の商
- N!/Kを(N-1)!で割ったあまりは
  - (N/K)×(N-1)! ÷ (N-1)!　のあまり
  - →「N÷Kのあまり」÷ K × (N-1)!
  - これは 整数÷K×(N-1)! の形をしている

C-考察2

- このあと2桁目を求めるときに
  - 整数÷K×(N-1)! を (N-2)!で割った商と余り

  を求めることになる。商は先ほど同様に簡単に求めることができ、あまりは
  - 整数÷K×(N-2)!

  になる。

C-考察2まとめ

- 1桁目を求めるときに扱う値が 1÷K×N!
- 2桁目を求めるときに扱う値が 整数÷K×(N-1)!
- 3桁目を求めるときに扱う値が 整数÷K×(N-2)!
- 4桁目を求めるときに扱う値が 整数÷K×(N-3)!

  :

C-考察2まとめ

- 1桁目を求めるときに扱う値が 1÷K×N!
- 2桁目を求めるときに扱う値が 整数÷K×(N-1)!
- 3桁目を求めるときに扱う値が 整数÷K×(N-2)!
- 4桁目を求めるときに扱う値が 整数÷K×(N-3)!

  :

C-考察2まとめ

- 1桁目を求めるときに扱う値が 1÷K×N!
- 2桁目を求めるときに扱う値が 整数÷K×(N-1)!
- 3桁目を求めるときに扱う値が 整数÷K×(N-2)!
- 4桁目を求めるときに扱う値が 整数÷K×(N-3)!

  :

赤字の整数だけ保存しておくだけで済む

C-解法

- 部分点1と同じ解法を、N!/Kを陽に持たずに計算する。
- 保存しておく整数はあまり大きくならないことが保証される。
  - 考察すると 整数÷K ≦ N となることがわかる
- 残っている数字の中でX番目に小さいものをもとめる作業はBIT上の二分探索で実装できる

  →計算量O(Nlog^2N)　満点獲得

ARC047解説
D:ナナメクエリ

D-問題概要

- 全てのマスが0に初期化されたN×Nの方眼紙上で以下のクエリをQ個処理せよ。
- 1:A≦ X+Y ≦ Bとなるマス(X,Y)にCを足す
- 2:A≦ X-Y ≦ Bとなるマス(X,Y)にCを足す
- 3:A≦X≦ B,C≦Y≦Dとなるマス(X,Y)の範囲内の最大値とその個数を求める

- N,Q≦5,000

D-部分点解法

- N ≦ 50
- 指示通りシミュレートする

- 一度のクエリで参照する必要のマスは最大で2500個程度

- 充分少ないので間に合う
- O(N^2Q）10点獲得

D-考察

- sum[i] := 「X+Y=iとなるマス(X,Y)にクエリ1で足された値の総和」
- dif[i] := 「X-Y=iとなるマス(X,Y)にクエリ2で足された値の総和」

  という2つの変数を用意する。
- sum[X+Y]とdif[X-Y]がわかればマス(X,Y)の値もすぐわかる。

D-考察

- sum[X+Y]とdif[X-Y]がわかればマス(X,Y)の値もすぐわかる
- クエリ1とクエリ2はO(N)で処理できる
- 問題はクエリ3

D-考察

- クエリ3の範囲のなかでとりうるX-Yの値は連続している。

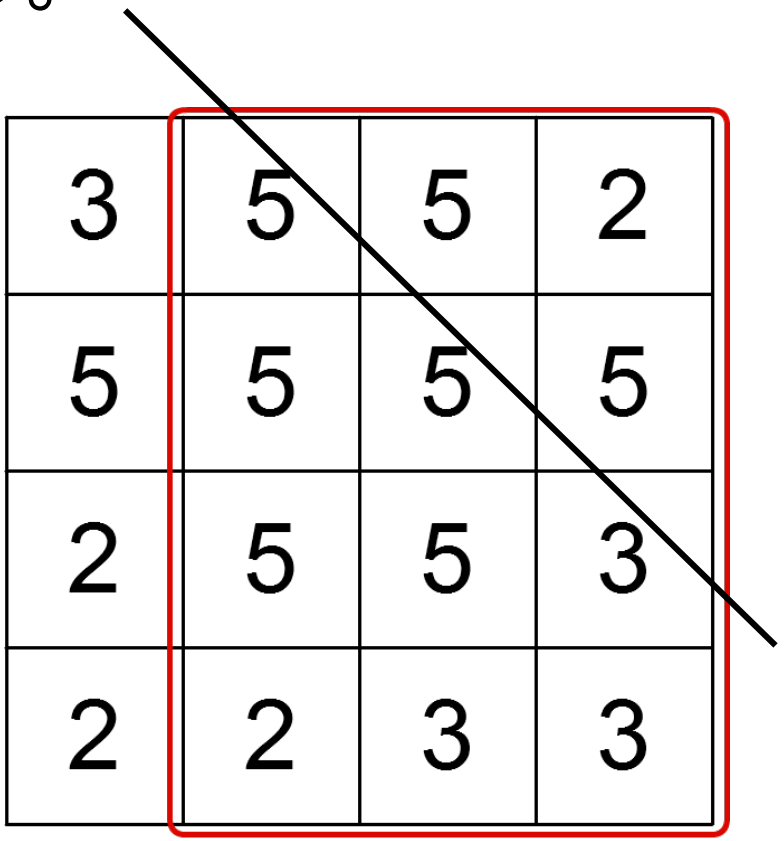
- 右図のような範囲の場合
X-Yは-3～2の値を取る

3	5	5	2
5	5	5	5
2	5	5	3
2	2	3	3

D-考察

- X-Yの値を決めてやると、X+Yの値の範囲も決まり、1個飛ばしの値をとる。

- X-Y = -1とする
  X+Yは1～5の奇数をとる



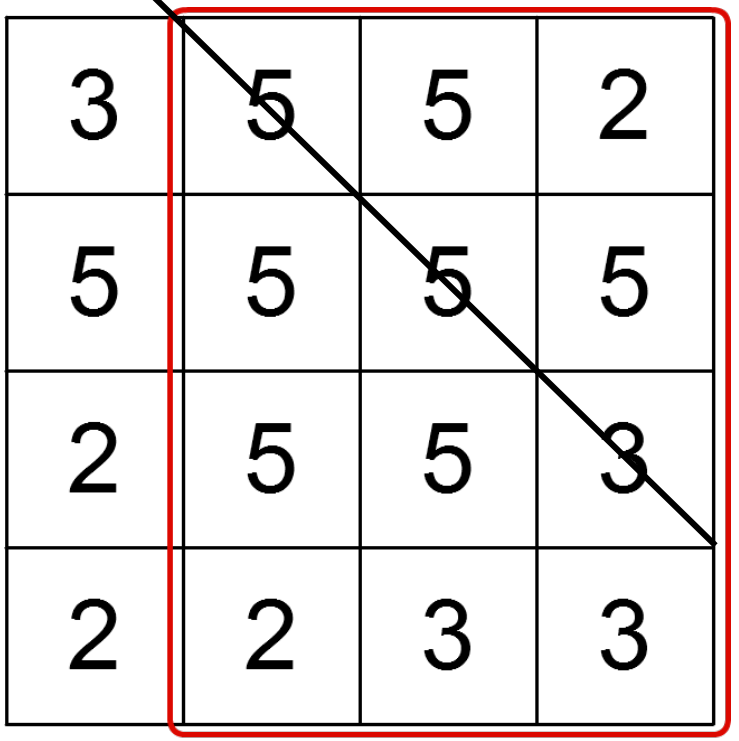
D-考察

- X-Yを決めた時に、その中で最大の値を持つマスは、適切な範囲のsumの最大値 + dif[X-Y]となる。

- X-Y=-1ならば

  max{sum[1],sum[3], sum[5]}
  +dif[-1]

  が右線上の最大値である



D-偶奇でわける

- さきほどの考察で、X-Yを固定した時のX+Yの取りうる値の範囲が連続だったら、bitやセグメントツリーをつかってO(logN)で最大値を求めることが出来る。
- X+Yの範囲は連続ではないものの2飛ばしであるため、2つセグメントツリーを作ってX+Yが奇数のものと偶数のものを分けて管理すれば、きっちり連続した区間になってくれる。

D-部分点2まとめ

- sum[i]やdif[i]を使ってマスの値を保存する
- クエリ1,2は愚直にO(N)でsumやdifを更新
- クエリ3の時はdif[X-Y]を全通り試して、各X-Yに対して対応するX+Yに値の範囲をもとめ、セグメントツリーで最大値とその個数をもとめる。
- O(NlogNQ) 30点獲得

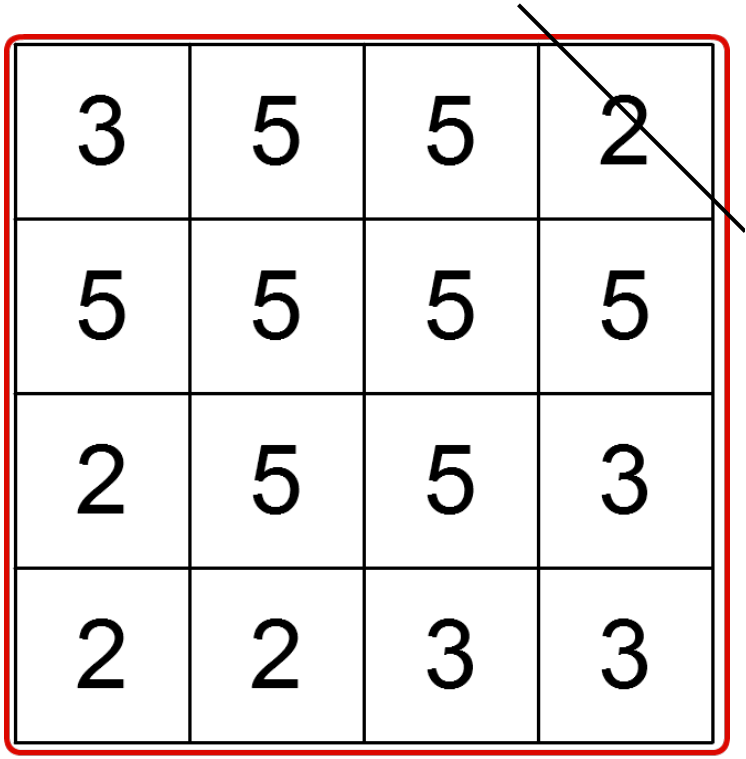
D-考察

- クエリの範囲が正方形である時を考える
- 偶奇は別々に考えていいので今回は奇数だけ考える
- 部分点2で注目したX+Yの範囲をよく見てみる

3	5	5	2
5	5	5	5
2	5	5	3
2	2	3	3

D-考察

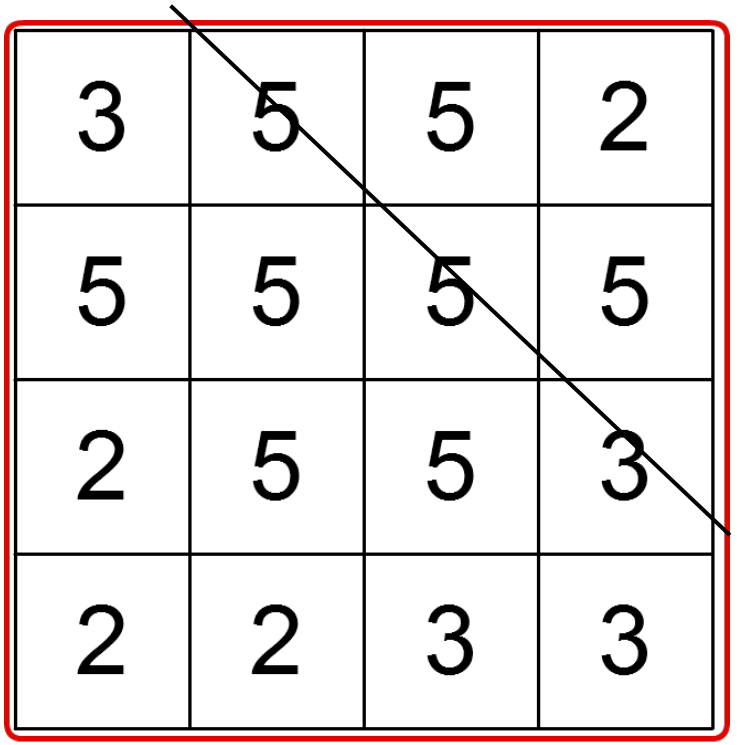
- X-Y = -3のときX+Yの範囲は[3]



3	5	5	2
5	5	5	5
2	5	5	3
2	2	3	3

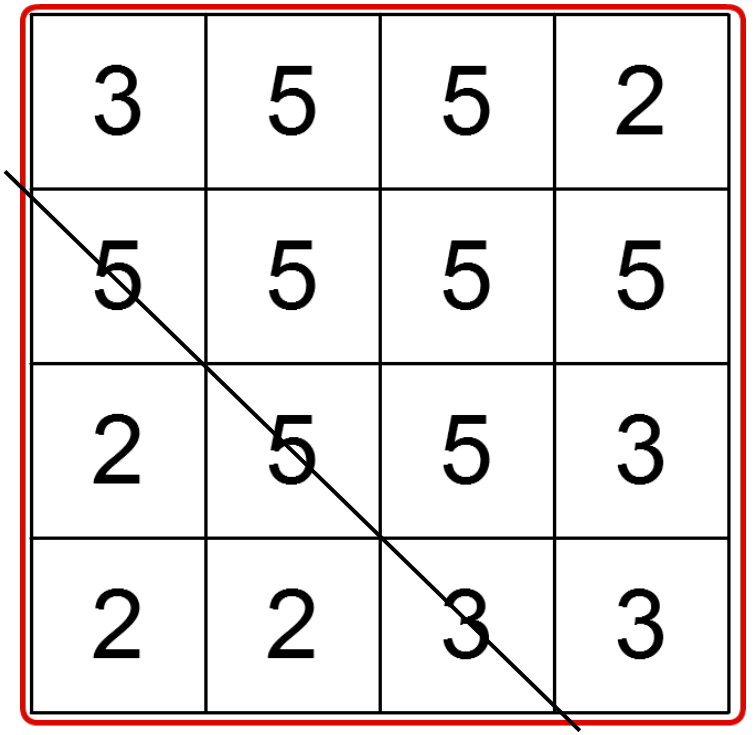
D-考察

- X-Y = -3のときX+Yの範囲は[3]
- X-Y = -1のときX+Yの範囲は[1,3,5]



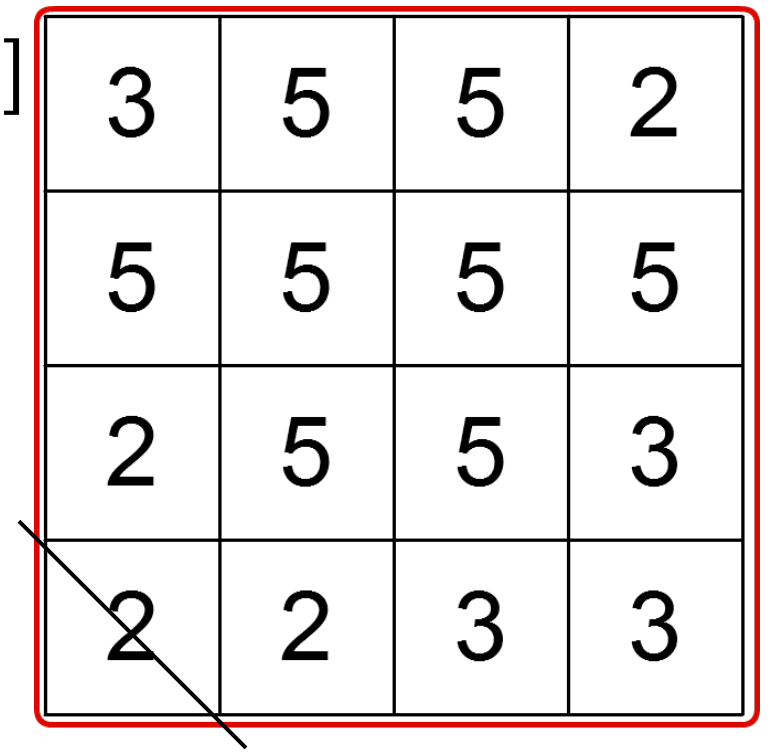
D-考察

- X-Y = -3のときX+Yの範囲は[3]
- X-Y = -1のときX+Yの範囲は[1,3,5]
- X-Y = 1のときX+Yの範囲は[1,3,5]



D-考察

- X-Y = -3のときX+Yの範囲は[3]
- X-Y = -1のときX+Yの範囲は[1,3,5]
- X-Y = 1のときX+Yの範囲は[1,3,5]
- X-Y = 3のときX+Yの範囲は[3]



D-観察

- 出てきた範囲の種類は
  - [3]
  - [1,3,5]
- これを8×8で実験した場合だと
  - [7]
  - [5,7,9]
  - [3,5,7,9,11]
  - [1,3,5,7,9,11,13]

  となる

D-考察

- 良い順番で処理するとX+Yの範囲は増えるだけになる。
  - 値が増えるだけならセグメントツリーをつかわなくても随時maxをとるだけで最大値がわかる
- 正方形のクエリならば一辺の長さに対する線形時間で処理することが出来る。

D-考察

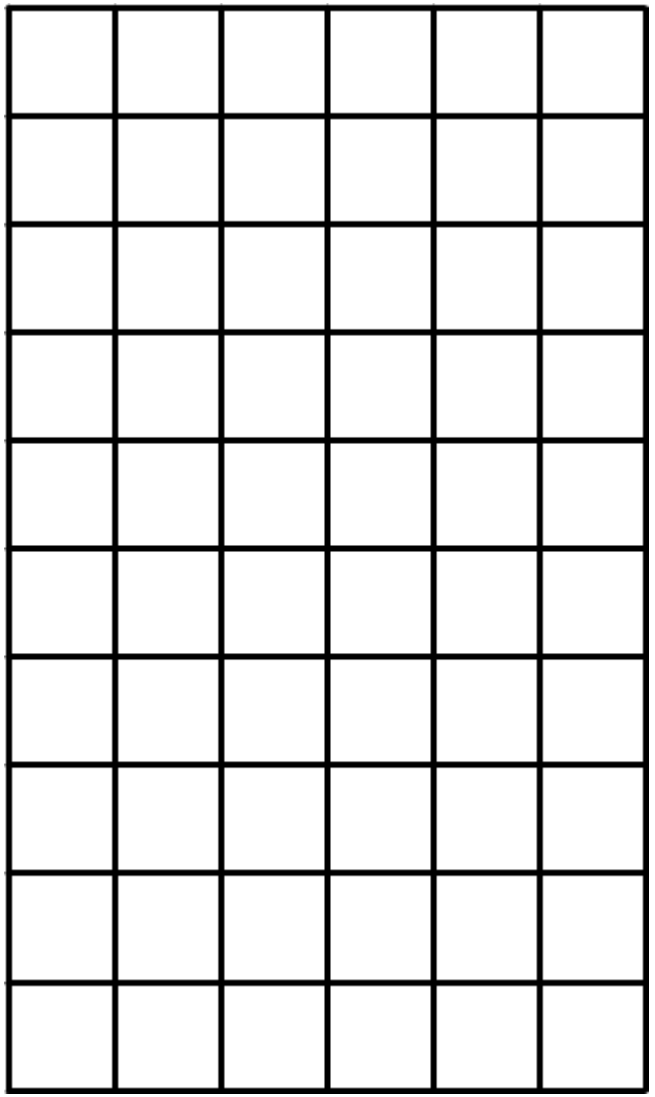
- クエリの範囲が長方形の時は？
  - 正方形に分割してやれば良い。

D-考察

- クエリの範囲が長方形の時は？
  - 正方形に分割してやれば良い。
- どうやって分割する？
  - ユークリッドの互助法の要領でできるだけ大きい正方形を削り取るように分割していく

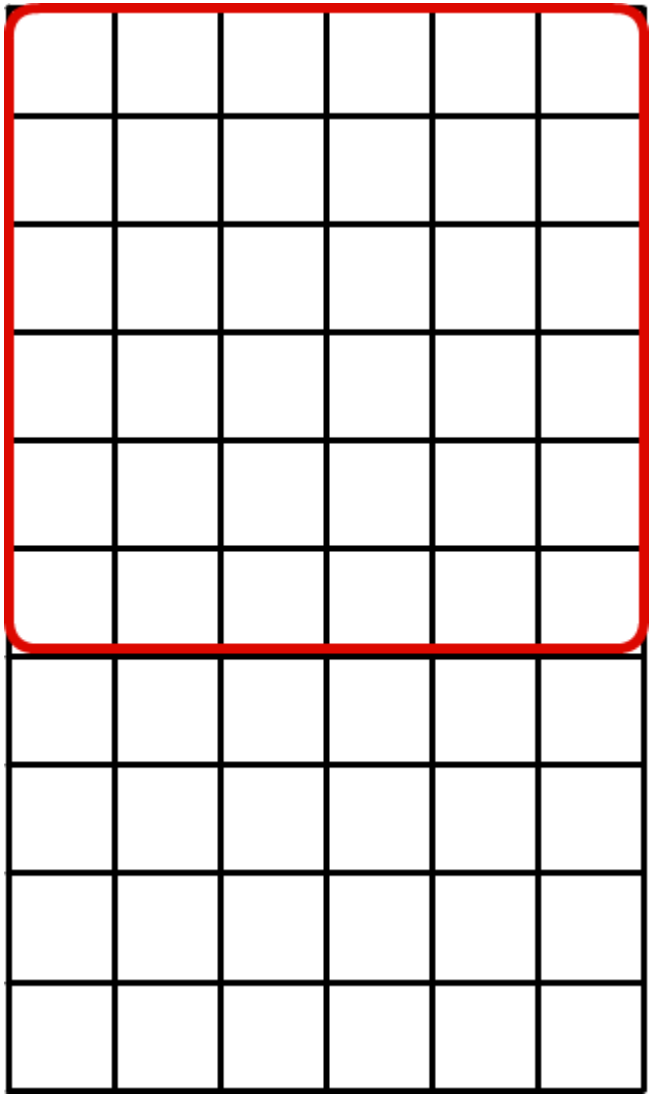
D-考察

- 例



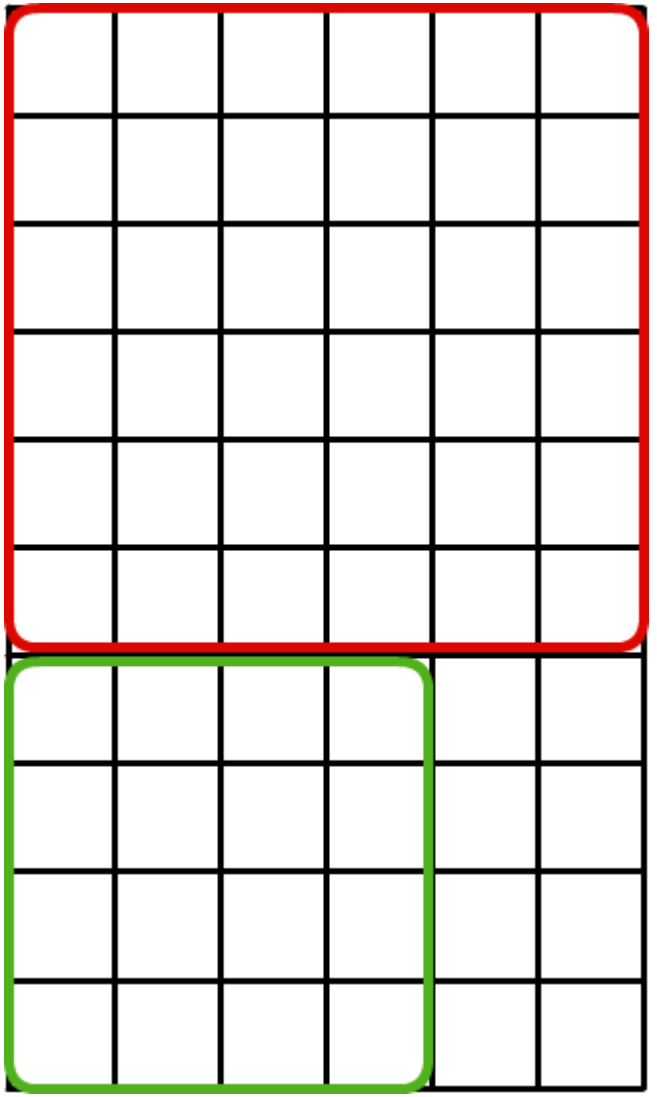
D-考察

- 例



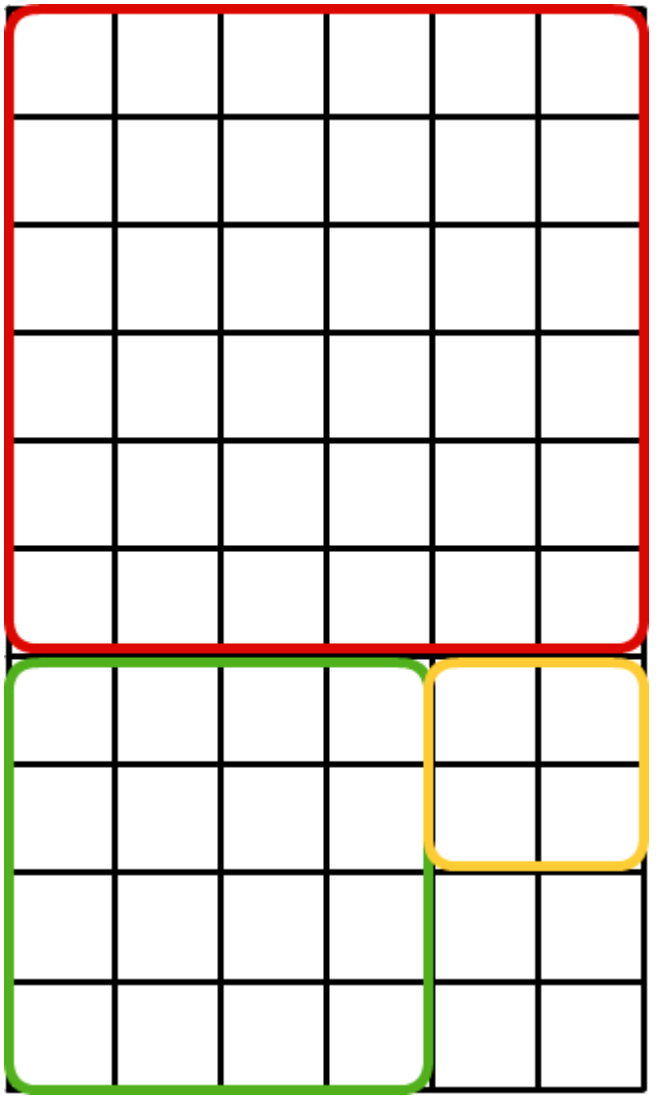
D-考察

- 例



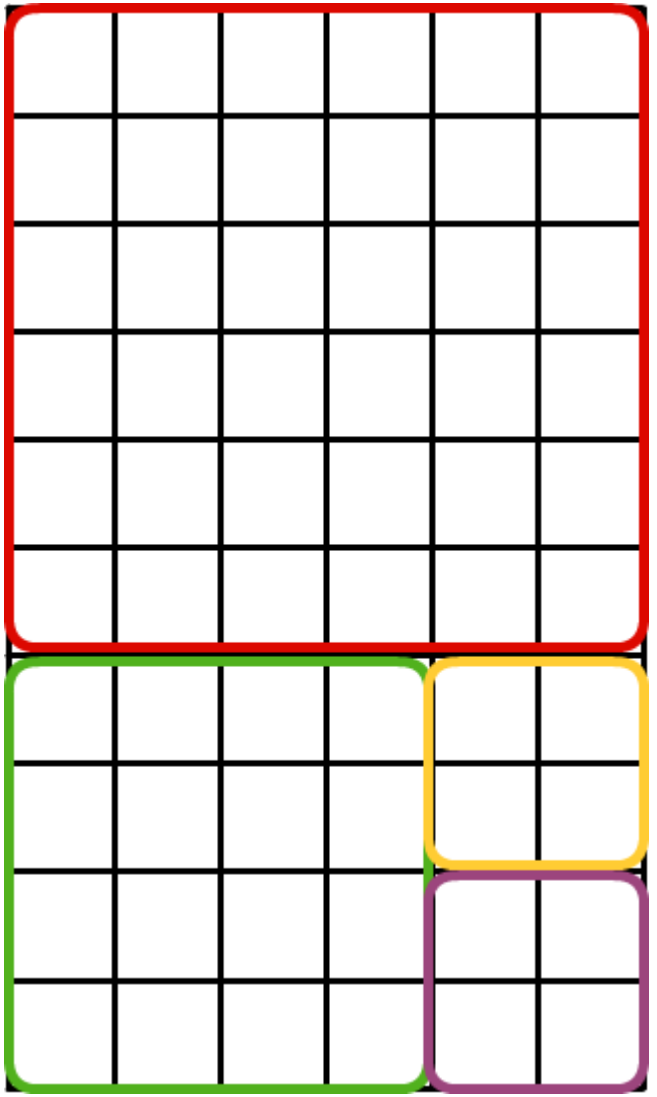
D-考察

- 例



D-考察

- 例



D-計算量

- １辺がXの正方形のクエリを処理するのにO(X)の時間がかかる
- タテとヨコの和がSの長方形から先ほどの方法で1辺がXの正方形を剥ぎとった時、のこる長方形のタテとヨコの和はS-X以下になる
- よって、全て正方形に分割し終えたときの総計算量はO(S)＝O(N)となり無事に線形時間で処理が終わる。

D-満点まとめ

- 部分点2同様にsumとdifを作る。
- 正方形のクエリならば線形時間で解ける。
- 長方形のクエリでも正方形に分割すると線形時間でとける。

- O(NQ)　満点

D-満点別解(紹介のみ)

- 部分点2でセグメントツリーを使うかわりにスライド最小和を使うことで、logを一つ取ることが出来る。
- hashmap(sortされていない代わりにアクセスが定数時間であるmap)を使えば最大値の個数を数えるのが線形時間でできるようになる

AtCoder Regular Contest 046
解説

AtCoder株式会社

問題 A – ゾロ目数

問題概要

- すべての桁の数字が同じであるような正の整数をゾロ目数と呼ぶ。
- 小さい方から N 番目のゾロ目数を求めよ。

- $1 \leq N \leq 50$

解法 (1)

- ゾロ目数を列挙してみる。

$$1, 2, 3, 4, 5, 6, 7, 8, 9,$$
$$11, 22, 33, 44, 55, 66, 77, 88, 99,$$
$$111, 222, 333, 444, 555, 666, 777, 888, 999, \ldots$$

- i 行目の j 番目のゾロ目数は、数字 j を i 回繰り返した数である。

- 数字 (N-1)%9+1 を (N-1)/9+1 回出力すればよい。

解法 (2)

- サンプル 3 を見ると、$N = 50$ のゾロ目数は 555555 と小さい。
- 1 から 555555 まで順にゾロ目数か判定しても、間に合う。

解法 (3)

- $N \leq 50$ と小さい。
- 小さい方から 50 番目までのゾロ目数を、コードに埋め込む。

B: 石取り大作戦

問題概要

- N 個の石を使って石取りゲームを行う
  - プレイヤーは交互に 1 個以上の石を取る
  - 最後の石を取ったほうの勝利である
- 先手は A 個、後手は B 個まで一度に石を取れる
- 2 人が最適に行動したとき勝利するのはどちらか？

- 制約
  - $1 \leqq N \leqq 10^9$
  - $1 \leqq A, B \leqq 10^9$

考察

- $N \leqq A$ のとき
  - 先手が全ての石を取れば勝利
  - 先手必勝

- $N > A$ のとき
  - $A = B$ のとき
    - ?
  - $A \neq B$ のとき
    - ?

(1) $N > A$ かつ $A = B$ のとき

- $N = A + 1$ のときを考える (サンプル 2 参照)
  - 先手がどのように石を取っても必ず後手が勝つ
  - 後手必勝

- N を $A + 1$ で割ったあまりが 0 のとき、後手必勝
  - 先手がどのように石を取っても、後手は石の数を $A + 1$ で割ったあまりが 0 となるように石を取り続けることが可能
  - 後手はいずれ最後の石を取り勝利する

- N を $A + 1$ で割ったあまりが 0 以外のときは？

(1) $N > A$ かつ $A = B$ のとき

- N を $A + 1$で割ったあまりが 0 以外のときは？
  - 石の数を $A + 1$ で割ったあまりが 0 になるように取ると、後手必勝の状況を相手に押しつけられる
  - よって先手必勝

(2) $N > A$ かつ $A \neq B$ のとき

- $A > B$ のときを考える
  - 直感的には先手が有利そう
  - この直感は正しく先手必勝

- 証明
  - $N > A$ のときを考える
  - 先手が 1 個の石を取ったとき、後手がどのように石を取ったとしても再び先手の手番が訪れる
    - $N - 1 > A - 1 \geqq B$ より明らか
  - いずれ $N \leqq A$ の盤面で先手の手番が訪れるので先手必勝

(2) $N > A$ かつ $A \neq B$ のとき

- $A < B$ のときを考える
  - こちらの場合には後手必勝

- 証明
  - 先手がどのように石を取っても後手の手番が訪れる
  - このとき残りの石の数 N について $N \leqq B$ ならば後手の勝利
  - $N > B$ のとき、後手が 1 個の石を取ったとき、先手がどのように石を取ったとしても再び後手の手番が訪れる
  - いずれ $N \leqq B$ の盤面で後手の手番が訪れるので後手必勝

まとめ

- $N \leqq A$ のとき
  - 先手必勝

- $N > A$ のとき
  - $A = B$ のとき
    - N を $A + 1$ で割ったあまりが 0 以外: 先手必勝
    - それ以外: 後手必勝
  - $A \neq B$ のとき
    - $A > B$: 先手必勝
    - $A < B$: 後手必勝

いずれの場合も $O(1)$ で判定可能

C: 合コン大作戦

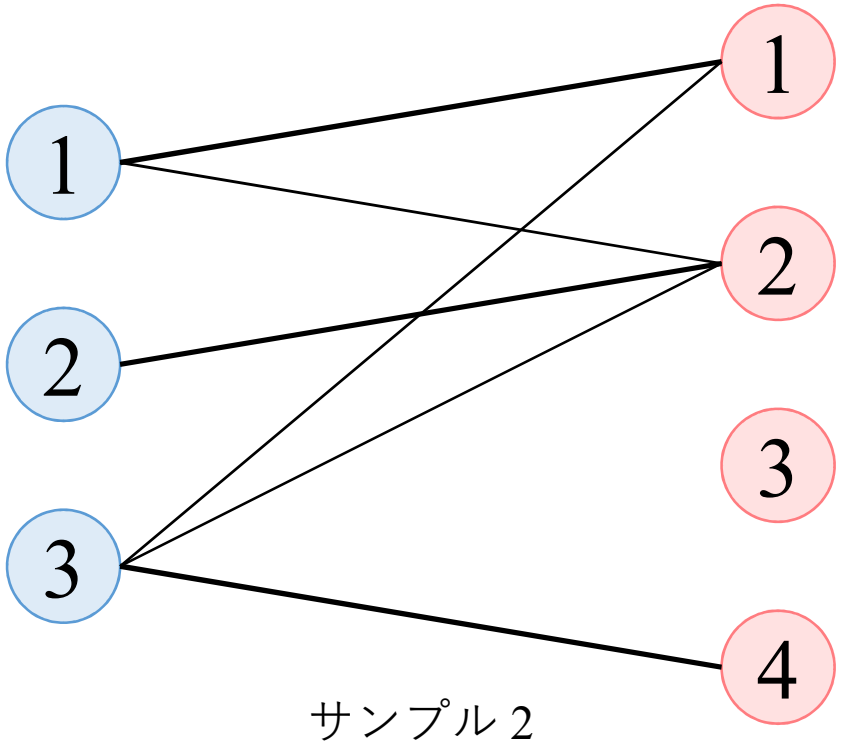
問題概要

- N 人の男性と M 人の女性がいる
- i 番目の男性は整数 A_i , B_i を持つ
- j 番目の女性は整数 C_j , D_j を持つ

- $D_j \leqq A_i$ かつ $B_i \leqq C_j$ となるペアの数を最大化せよ
  - ただし，それぞれの人は複数のペアに所属してはならない
- 制約
  - $1 \leqq N, M \leqq 150{,}000$
  - $1 \leqq A_i, B_i, C_j, D_j \leqq 10^9$

考察

- この問題は二部グラフの最大マッチング問題
  - 最大流を求めるアルゴリズムで解くことができるが…
    - 今回は頂点数，辺の数ともに大きく間に合わない
    - 問題で与えられた制約や性質をうまく利用することを考える



サンプル 2

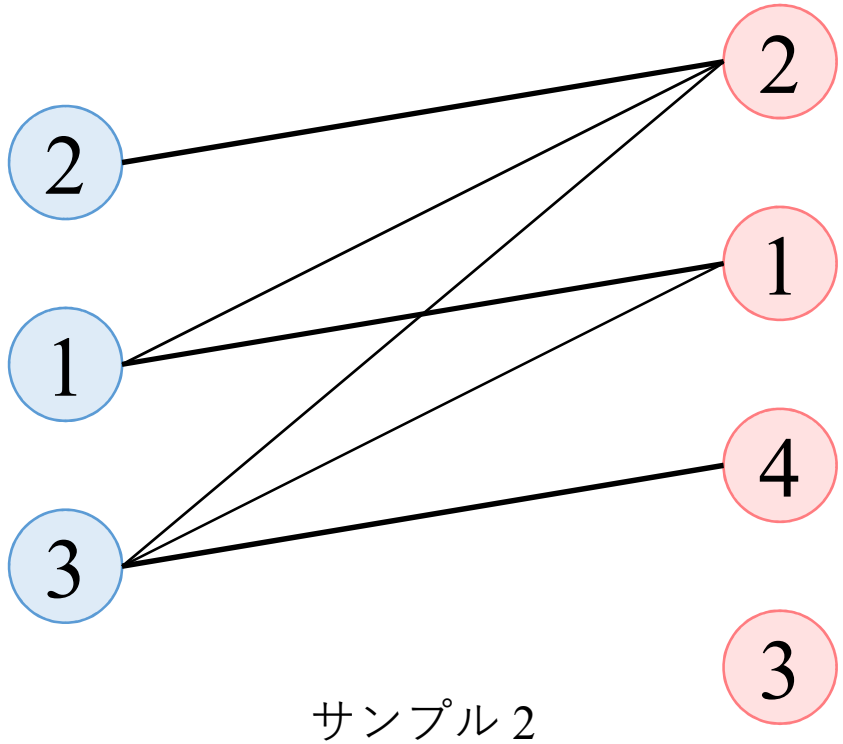
部分点 (30 点)

- 任意の i, j ($1 \leqq i \leqq N, 1 \leqq j \leqq M$) で $B_i \leqq C_j$ が成立
  - 任意の男性の要求を全ての女性が満たす

- 各女性について要求を満たす男性がいるか知りたい
  - A_i, D_j にのみ着目すればよい
  - A_i, D_j で男女それぞれをソートする

部分点 (30 点)

- サンプル 2 についてA_i, D_jで昇順にソートすると…
- D_j が小さい女性から順に、まだペアを組んでいない $D_j \leqq A_i$ を満たす A_i が最小の男性とのペアを貪欲に成立させていくのが最適だと分かる



サンプル 2

部分点解法 (30 点)

- D_j が小さい女性から順に、先述の条件を満たす男性と貪欲にマッチングを行えばよい

- 愚直に毎回調べると $O(NM)$ となって間に合わないがしゃくとり法を用いると $O(N+M)$ で実現できる

- 計算量はソート部分が最も重く $O(N\log N + M\log M)$

満点 (100 点)

- 今度は $B_i > C_j$ であるようなケースが存在する
  - A_i, D_j に関する単純な貪欲法ではうまくいかない
- 突然だが、部分点解法を以下のように言い換える
  - 集合 S がある。 S ははじめ空集合である
  - 以下の $N + M$ 個の 2 種類のクエリに答えよ
    - S に要素を 1 つ追加する
    - S から要素を 1 つ取り除く。 S が空集合ならば何もしない
  - 要素の追加はマッチングを待っている女性の追加を、要素の削除は男性とのマッチングを成立させるのに対応
  - S から要素を取り除くのに成功した回数が求める答え

満点 (100 点)

- 満点の状況に対応するようさらに言い換える
  - 多重集合 S がある。 S ははじめ空集合である
  - 以下の $N + M$ 個の 2 種類のクエリに答えよ
    - S に C_j を 1 つ追加する
    - S から B_i 以上の要素を 1 つ取り除く。 S に条件を満たすものが存在しないならば、何もしない
  - 要素の追加はマッチングを待っている女性の追加を、要素の削除は男性とのマッチングを成立させるのに対応
  - S から要素を取り除くのに成功した回数が求める答え
  - 削除する要素はどのように選ぶのが最適か？
    - 条件を満たすような**最小**の要素を選ぶのが最適

満点解法 (100 点)

- この問題は最終的に以下の問題に帰着できる
  - 多重集合 S がある。 S ははじめ空集合である
  - 以下の $N + M$ 個の 2 種類のクエリに答えよ
    - S に C_j を 1 つ追加する
    - S から B_i 以上の最小の要素を 1 つ取り除く。 S に条件を満たすものが存在しないならば、何もしない
  - S から要素を取り除くのに成功した回数が求める答え
- 上記のクエリをmultisetを用いて処理してやればよい
  - multisetが標準ライブラリに存在しない場合でも、セグメント木、Fenwick Tree、平方分割などのデータ構造を用いて解くことが可能
- 全体の計算量は $O((N + M)\log(N + M))$

問題 D – うさぎとマス目

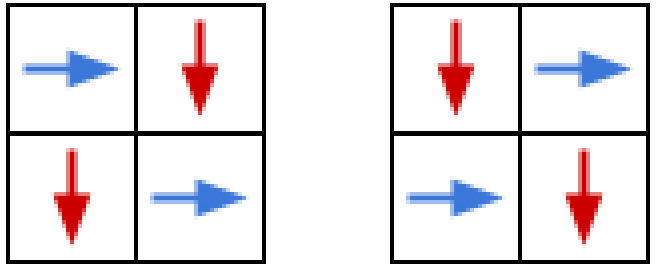
問題概要

- H 行、W 列のマス目がある。
- 最初、$(0, 0)$ にうさぎがいる。
- うさぎは (i, j) から $((i + 1)\%H, j)$ または $(i, (j + 1)\%W)$ へ移動できる。
- うさぎがすべてのマスをちょうど 1 回ずつ訪れ、$(0, 0)$ へ戻ってくる方法は何通りか？

例（サンプル 1）

- $H = 2,\ W = 2$

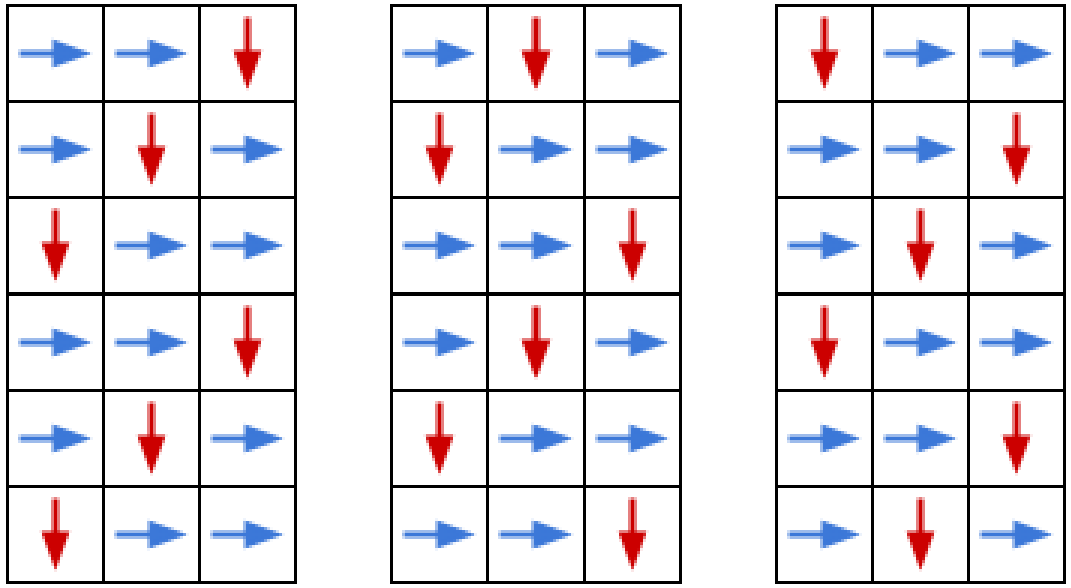
→ 2 通り



例（サンプル 2）

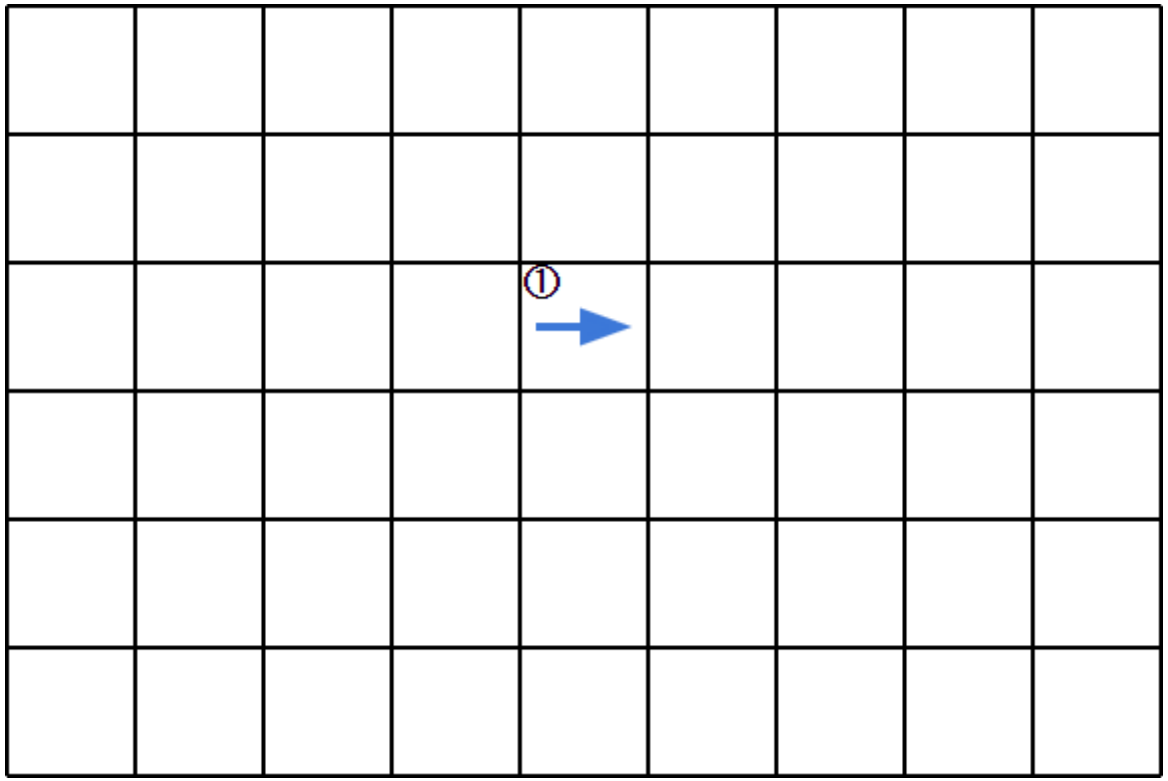
- $H = 6,\ W = 3$

→ 3 通り



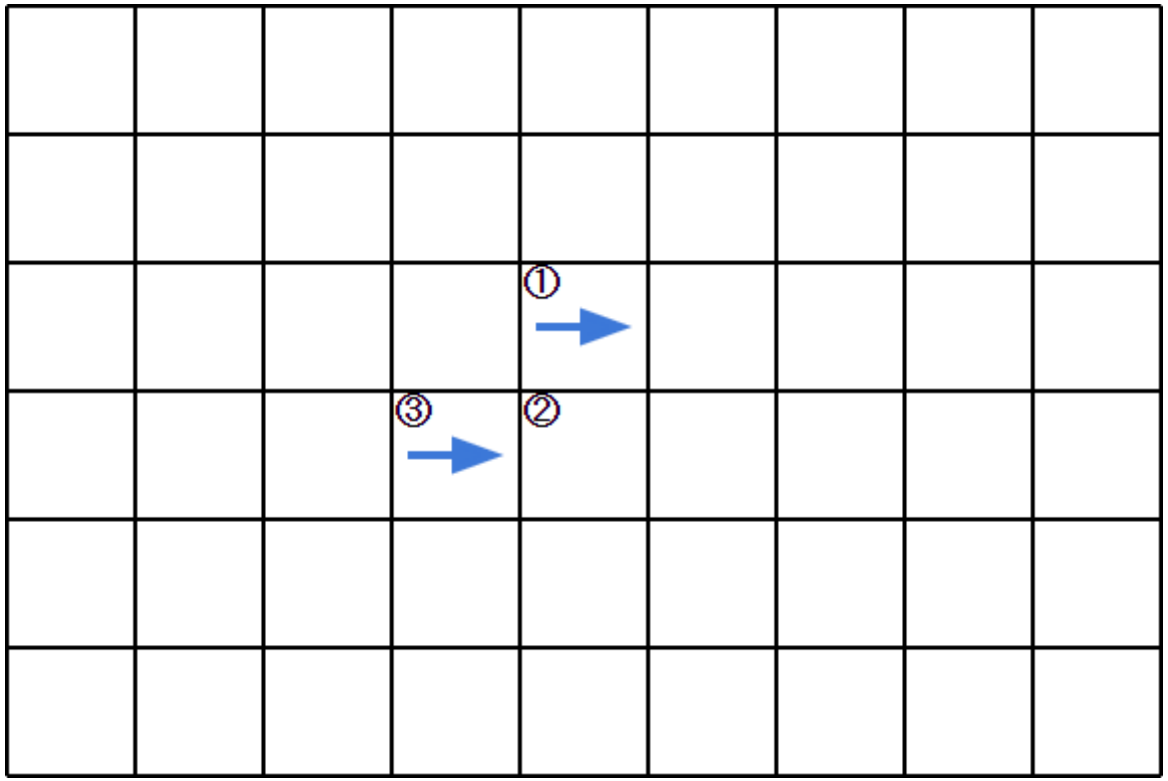
考察 (1)

- 仮に、マス ① から → へ移動するとする。



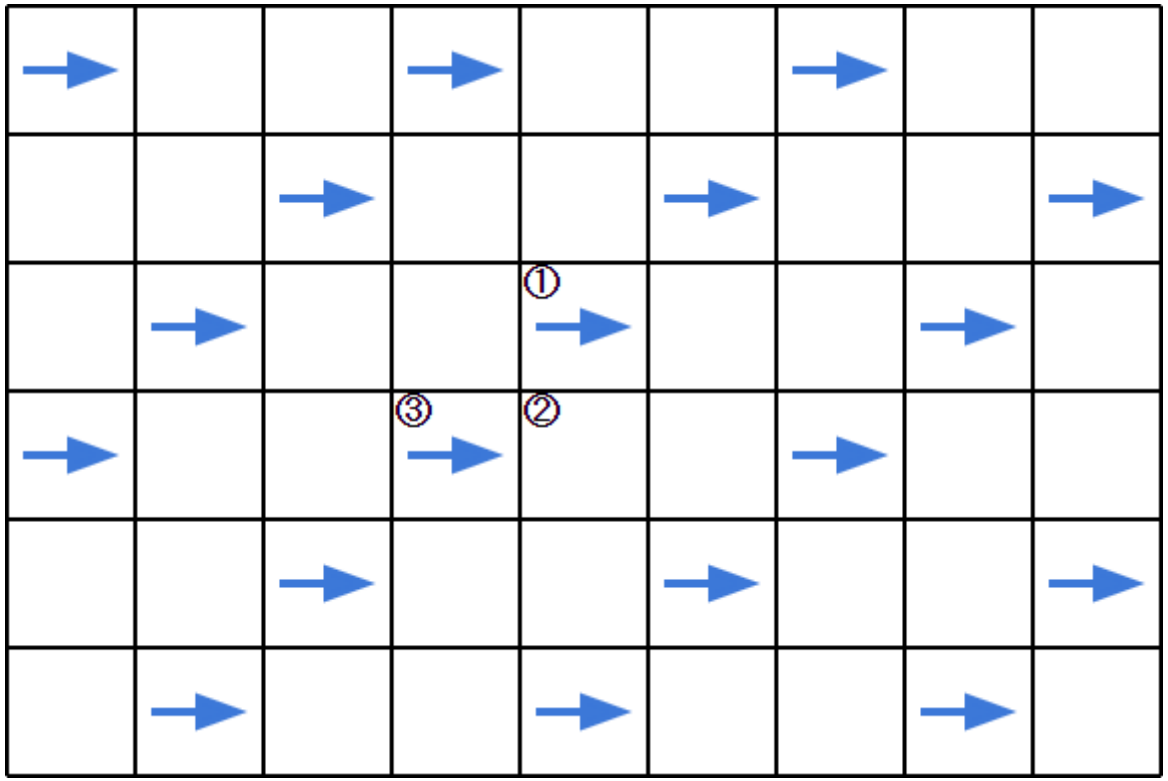
考察 (1)

- マス ② を訪れるためには、マス ③ から → へ移動しなければならない。



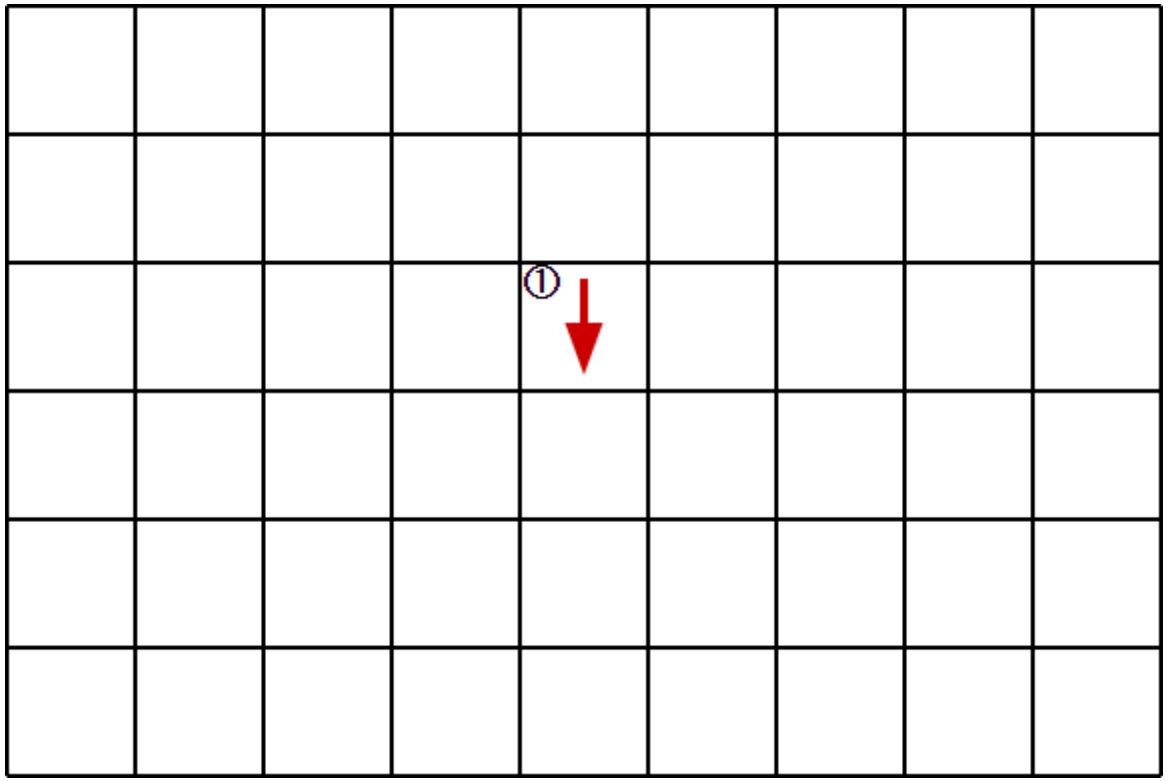
考察 (1)

- 連鎖的に、→ へ移動しなければならないマスが決まっていく。



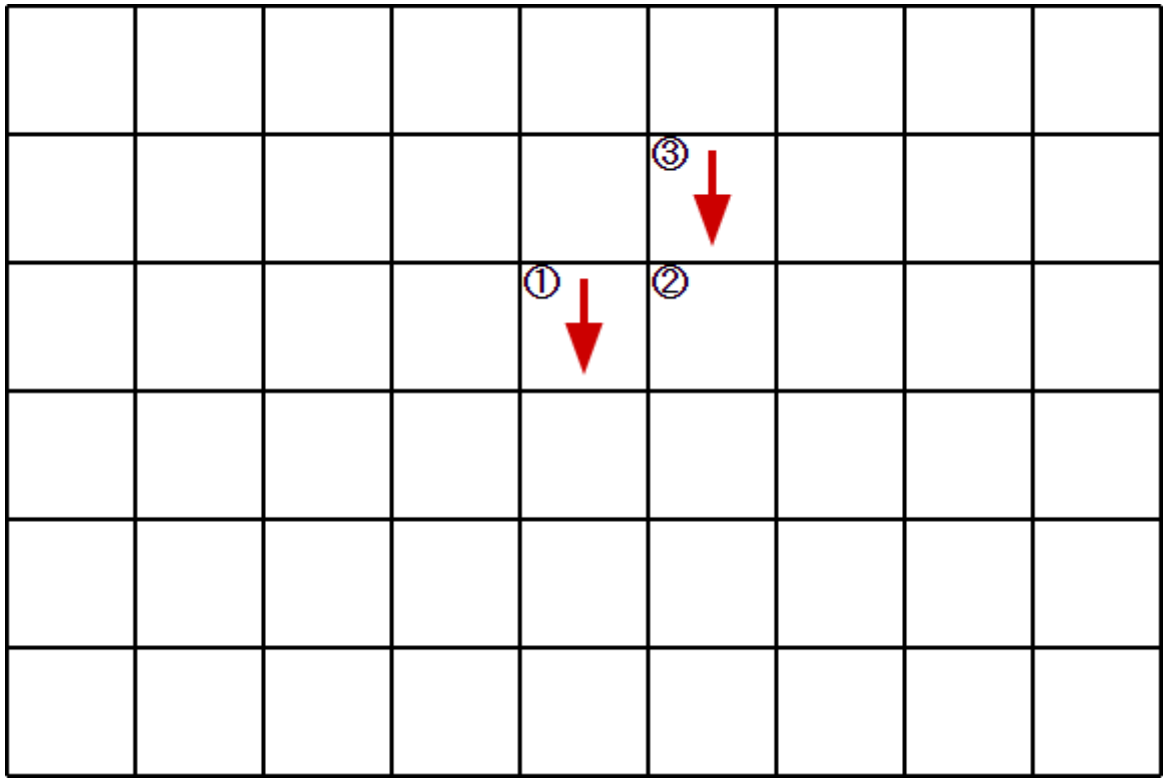
考察 (1)

- 仮に、マス ① から ↓ へ移動するとする。



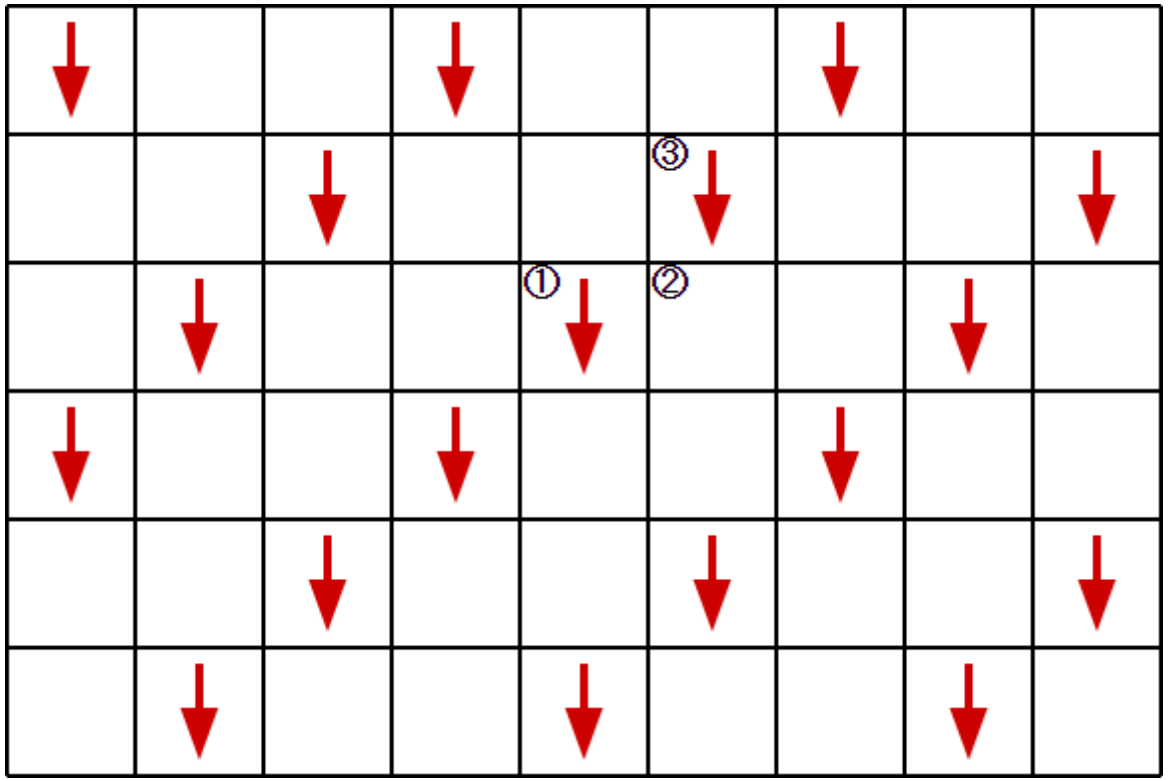
考察 (1)

- マス ② を訪れるためには、マス ③ から ↓ へ移動しなければならない。



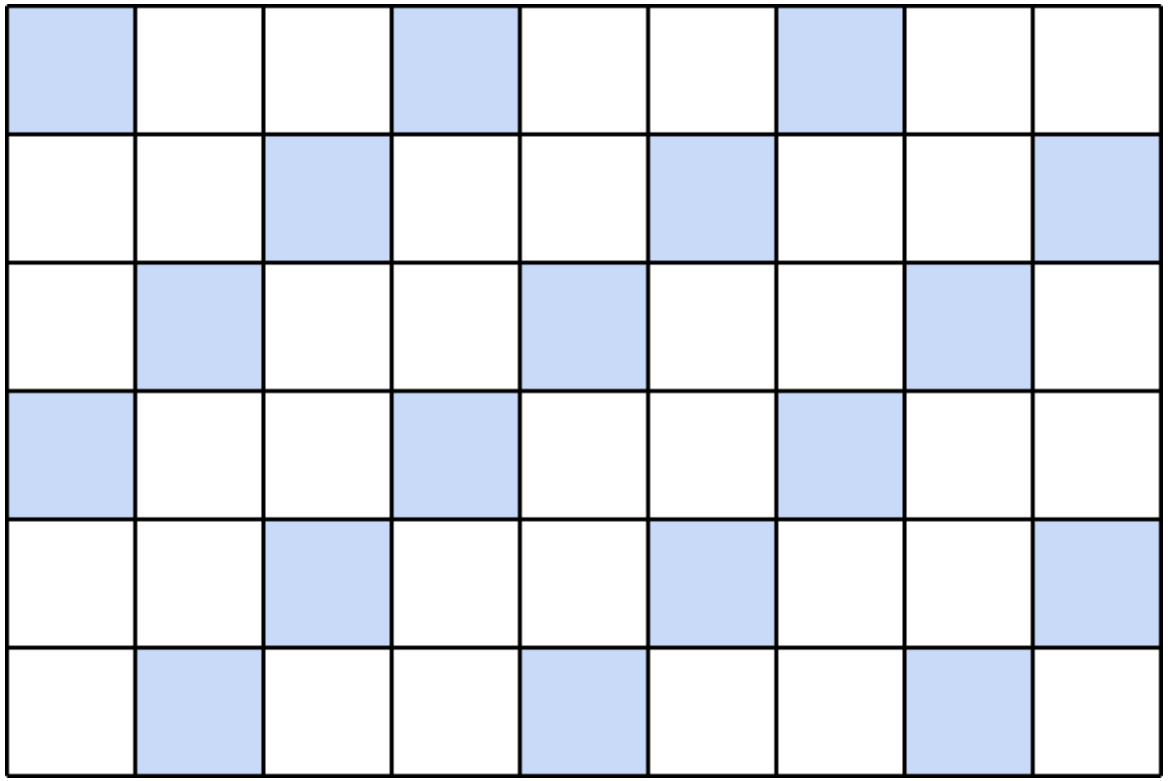
考察 (1)

- 連鎖的に、↓ へ移動しなければならないマスが決まっていく。



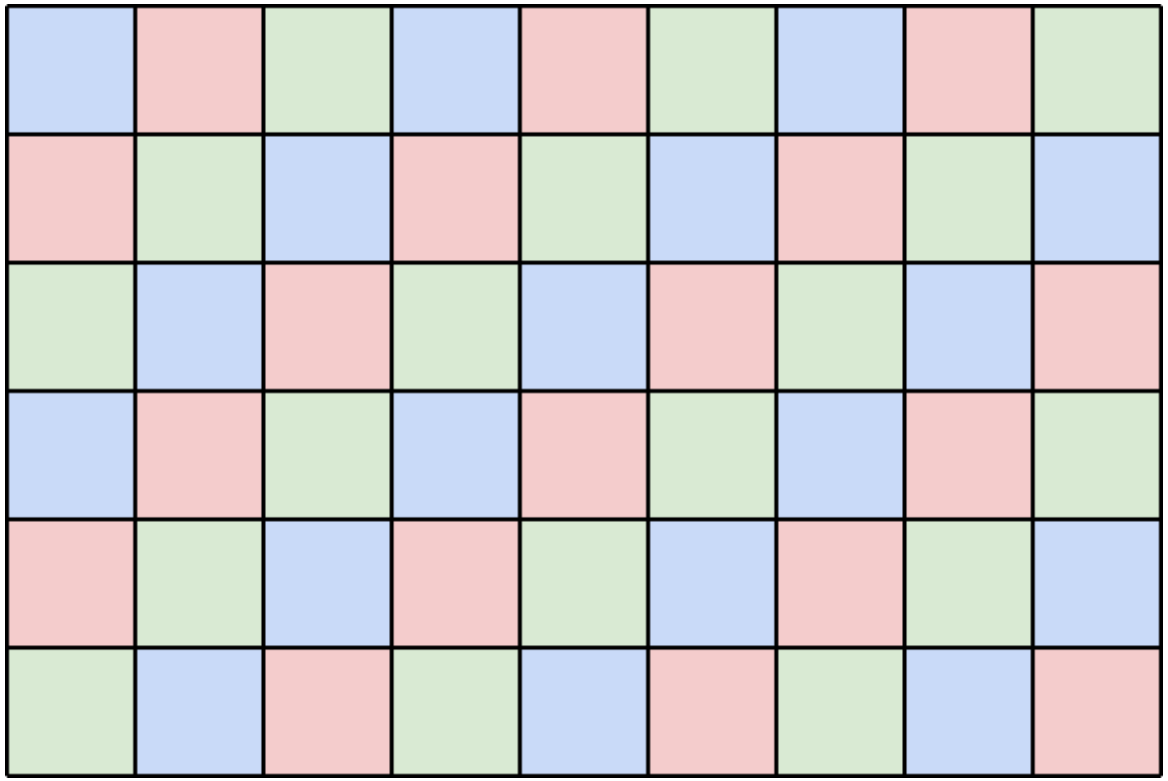
考察 (1)

- 結局、青のレーンはすべて同じ向きへ移動しなければならない。



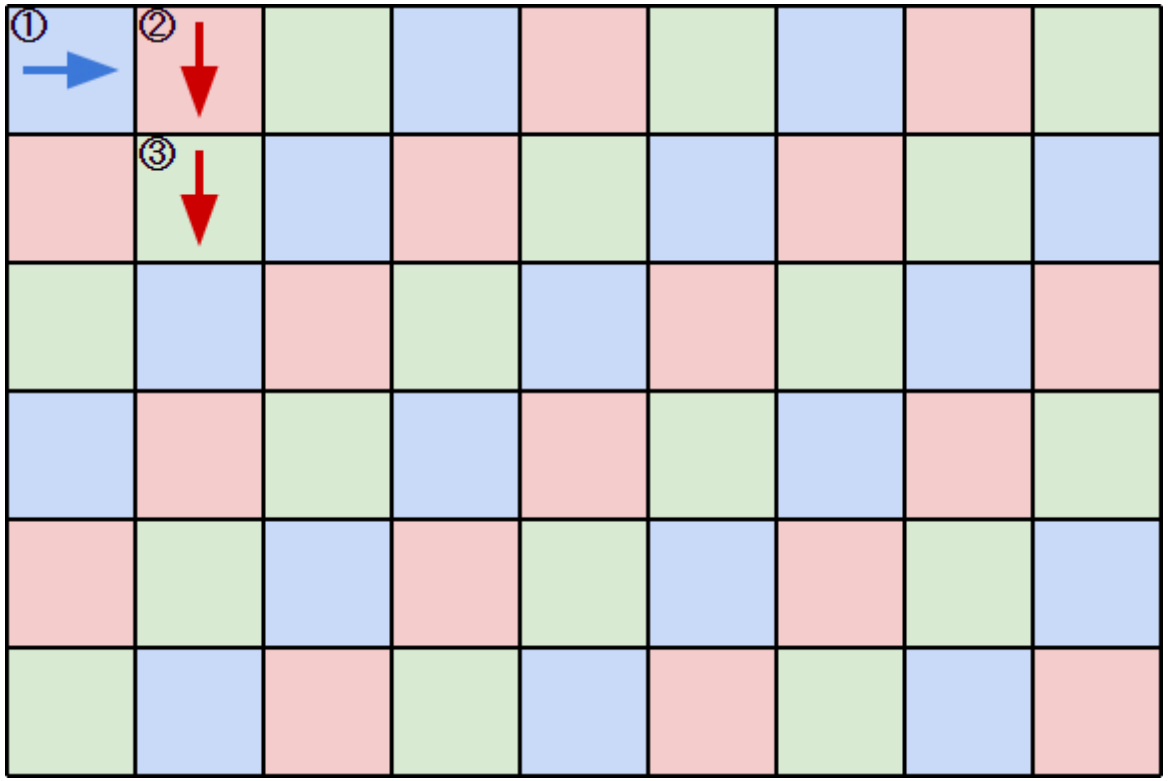
考察 (1)

- 同様に、赤や緑のレーンはすべて同じ向きへ移動しなければならない。
- ここで、レーンの本数 d は $d = \mathrm{GCD}(H, W)$ と計算できる。



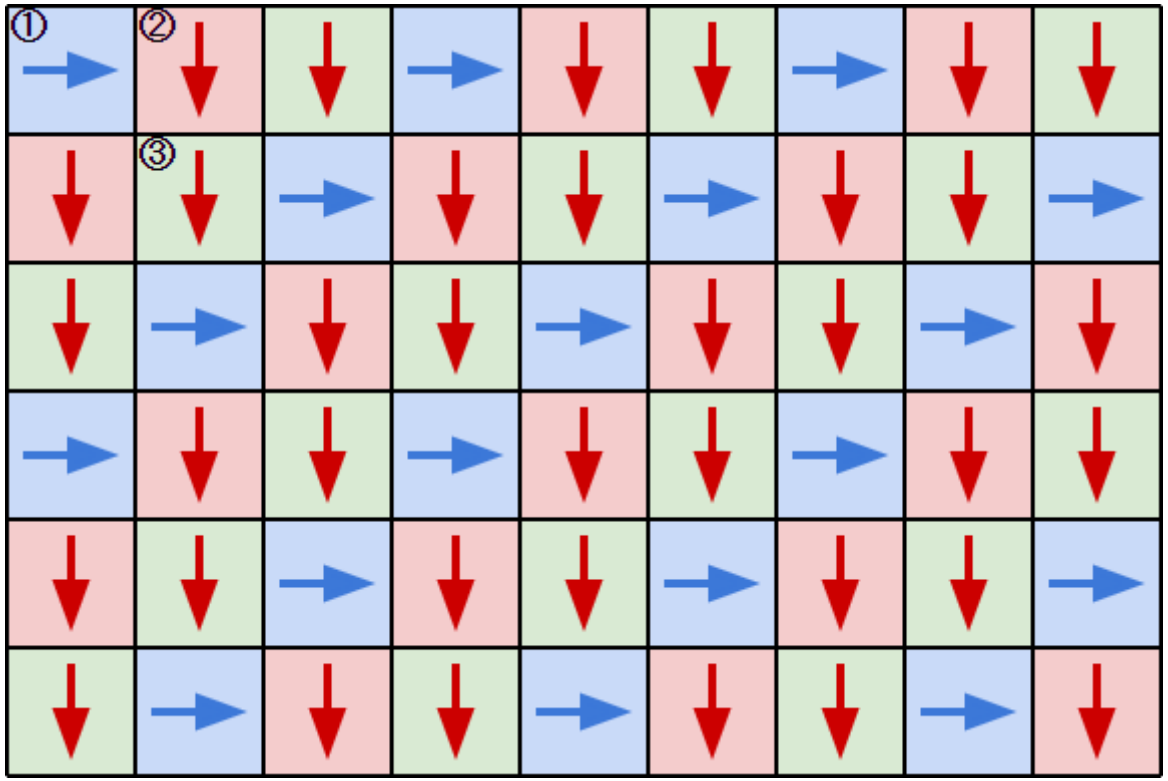
考察 (1)

- 最初の d 歩の向きを決めると、



考察 (1)

- 最初の d 歩の向きを決めると、全体の向きが一意に決まる。
- しかも、最初の d 歩の繰り返しになっている！

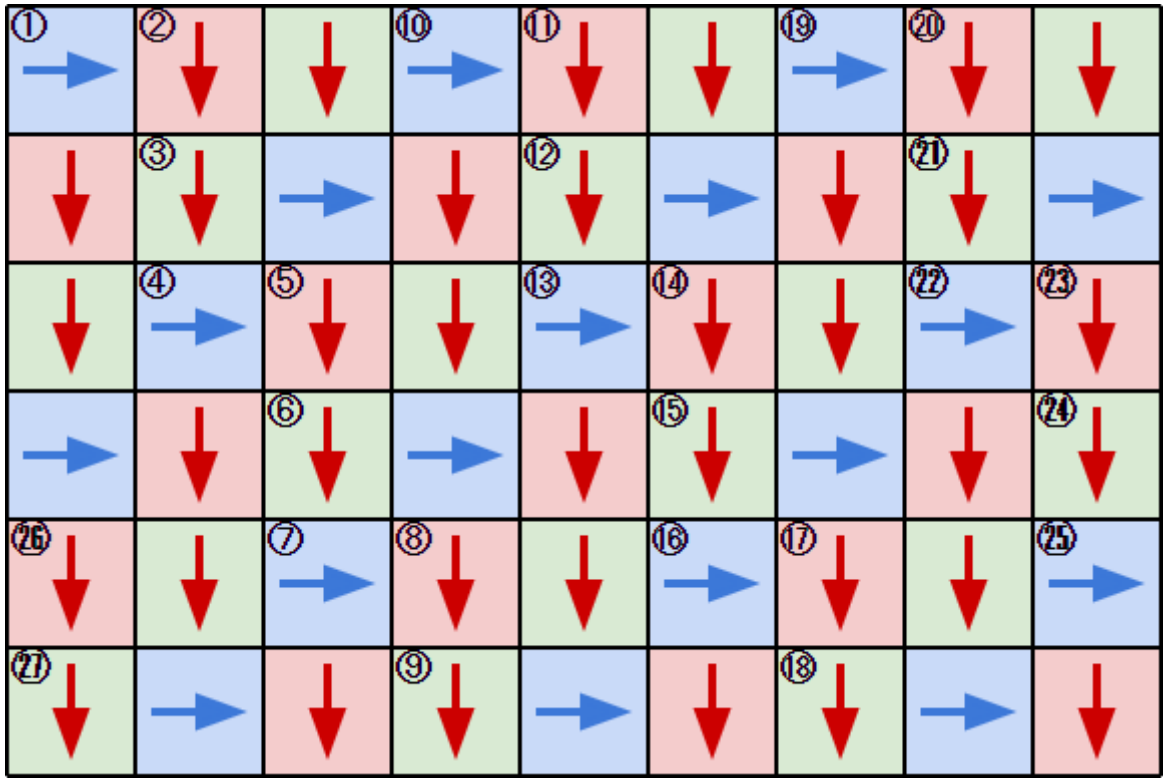


考察 (1)

- 「すべてのマスをちょうど 1 回ずつ訪れ、$(0, 0)$ へ戻ってくる」ためには、「最初の d 歩を繰り返す」ことが必要条件。

考察 (2)

- しかし、「最初の d 歩を繰り返す」ことは必要十分条件ではない。
- 先の例は、すべてのマスを訪れる前に $(0, 0)$ へ戻ってしまっている。



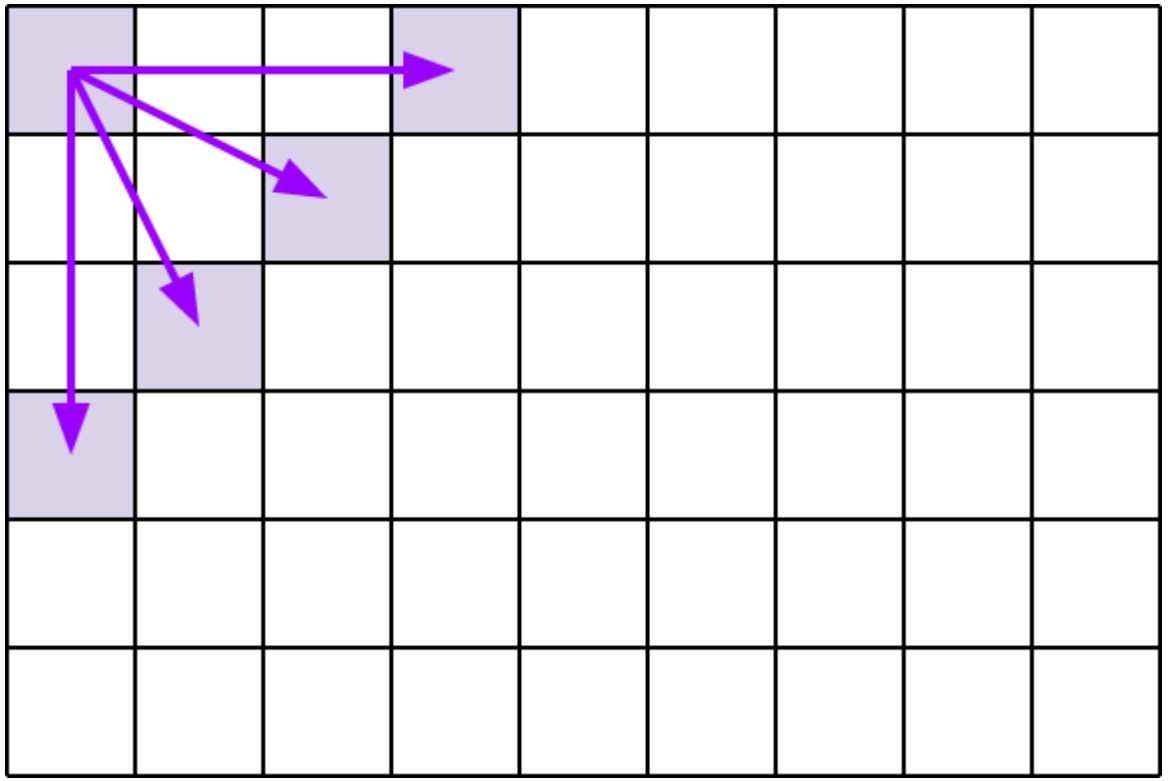
考察 (2)

- 最初の d 歩を全通り試し、すべてのマスを訪れているか個別にチェックする？

→ 最初の d 歩は 2^d 通りもあるので、TLE してしまう。

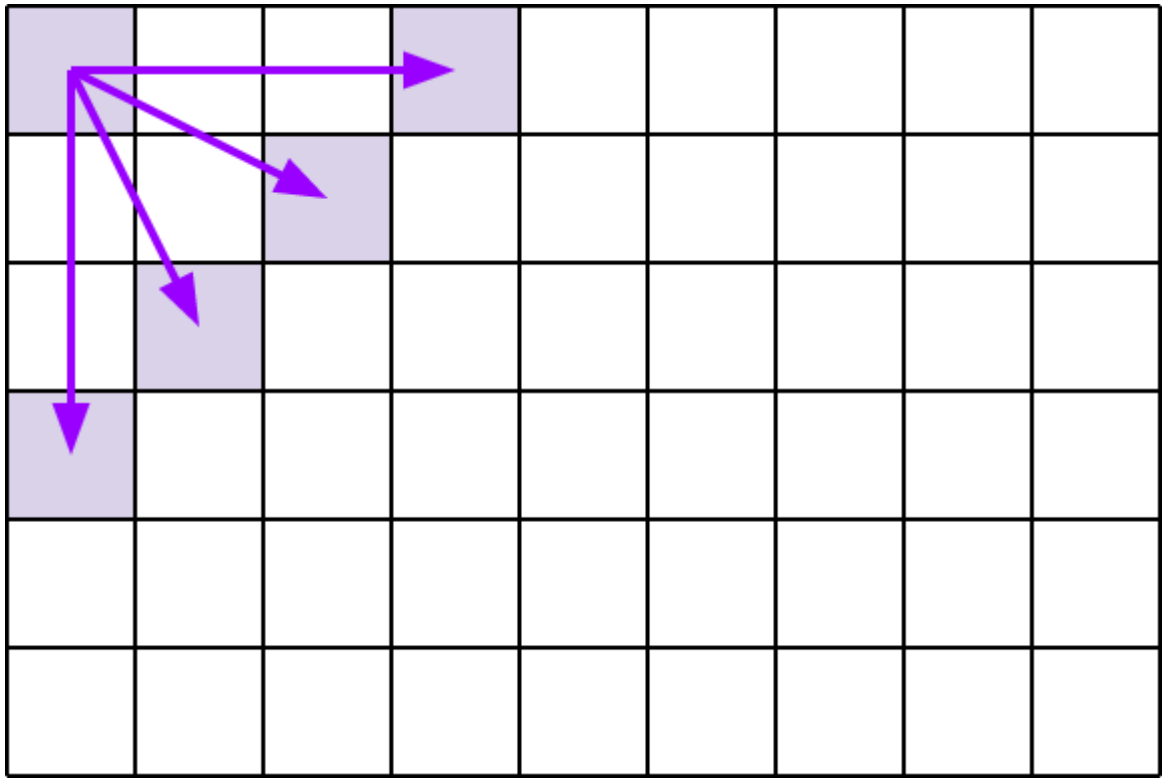
考察 (2)

- 「最初の d 歩の行き先」は $d+1$ 通りしかないので、全通り試せる。
- 最初の d 歩をまとめて、紫の矢印で表す。



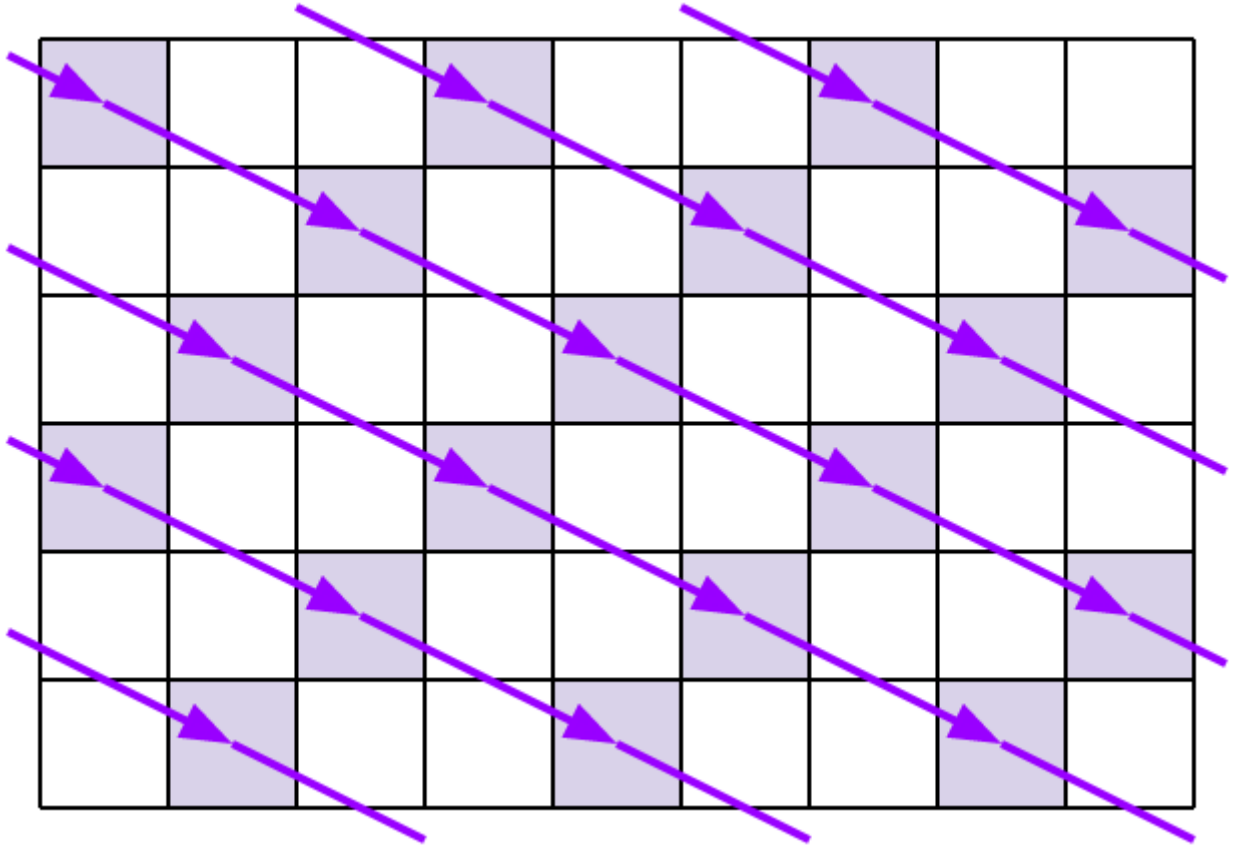
考察 (2)

- 紫の矢印は d 歩分なので、紫の矢印を $\frac{HW}{d}$ 回繰り返して $(0, 0)$ へ戻ってくれば、すべてのマスを訪れたことになる。



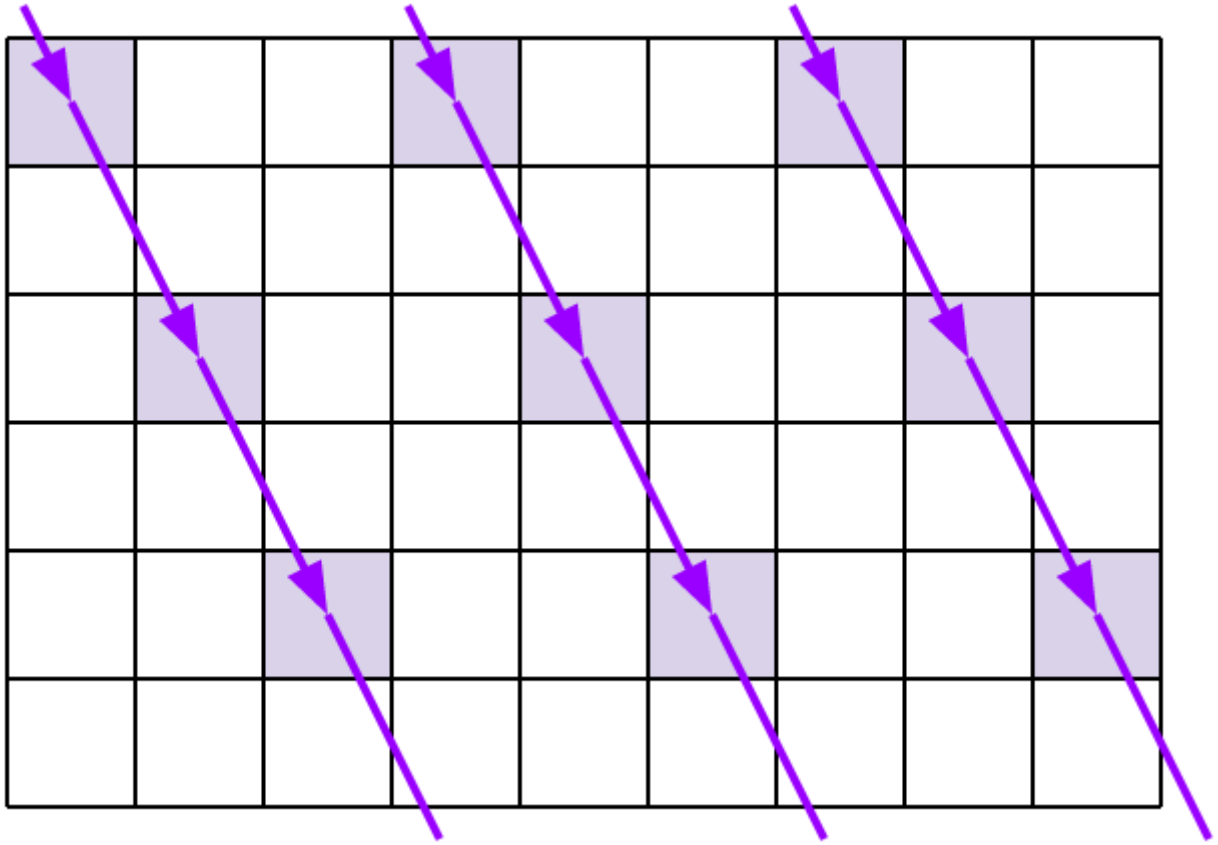
考察 (2)

- OK の例



考察 (2)

- NG の例



考察 (2)

- 紫の矢印を何回繰り返すと $(0, 0)$ へ戻ってくるか？
- 紫の矢印が右へ x マス、下へ y マス移動するとする。
  （ただし、$x + y = d$）
- 紫の矢印の横方向の周期は $\frac{\mathrm{W}}{\mathrm{GCD}(W,x)}$
- 紫の矢印の縦方向の周期は $\frac{\mathrm{H}}{\mathrm{GCD}(H,y)}$
- よって、紫の矢印が $(0, 0)$ へ戻ってくるまでの周期は

$$\mathrm{LCM}\left(\frac{\mathrm{W}}{\mathrm{GCD}(W,x)}, \frac{\mathrm{H}}{\mathrm{GCD}(H,y)}\right)$$

解法

- $d = \mathrm{GCD}(H, W)$ を計算する。
- $x \geq 0,\ y \geq 0,\ x + y = d$ を満たす (x, y) の組を全探索する。
- 各 (x, y) の組に対して、

$$\mathrm{LCM}\left(\frac{\mathrm{W}}{\mathrm{GCD}(W, x)}, \frac{\mathrm{H}}{\mathrm{GCD}(H, y)}\right) = \frac{HW}{d}$$

  ならば、$\binom{x+y}{x}$ を答えに足す。

- 計算量は $\mathrm{O}(d \log d)$

AtCoder Regular Contest 045
解説

AtCoder株式会社

スペース高橋君

解説スライド担当: 森田 晃平(@yosupot)

問題概要

- 文字列が与えられる
- Left → <
- Right → >
- AtCoder → A
- と置換して出力

解法(C++)

- getline等の関数を使うと1行まるまる読み込めます
- cin, scanfで読み込む場合は空白文字に注意してください
- 空白後の1文字を見て、Lなら<、Rなら>、AならAを出力し、そのあと次の空白まで進む。などの解法が考えられます

解法

- 文字列を扱う高級な関数を使うと楽です
- たとえばpythonならreplace('AtCoder', 'A')でAtCoderを全部Aにできます
- プロコンにおいてC++しか使わない人は多いと思いますが、他にpython,ruby等の文字列処理のライブラリが充実している言語を1個は使えるとよいです

AtCoder

B問題　ドキドキデート大作戦高橋君(解説スライド:三上)

1. 問題概要
2. 考察
3. 解法
4. アルゴリズム動作例

©AtCoder Inc. All rights reserved.

AtCoder

- Nコの教室があり，全ての教室を掃除したい
- 担当すると掃除しなければならない区間(掃除区間)がMコ与えられる(これは重なることがある)
- ある区間iを掃除しなかったとしても，全ての教室が問題なく掃除されているような区間iを全て列挙せよ.
- Mコの区間を全て使った場合に全ての教室が掃除できることは保証されている.

- 1≦N≦10^6
- 1≦M≦10^5

AtCoder

- まず，各教室がいくつの区間に被覆されているかを求めておくと見通しがよくなる.
- これはimos法(※)を用いて時間計算量 O(N+M) で達成可能

※ http://www.slideshare.net/chokudai/abc014 (P3参照)

- 各教室がいくつの区間に被覆されてるか分かれば,

ある教室を除いても全ての教室が掃除できる
⇔区間内の全ての教室が2つ以上の区間に被覆されている

- 追加制約
  - 任意の教室 $i(1 \leqq i \leqq N)$ について，その教室を含む掃除区間の数は高々 2 つである
- この制約上では，全ての区間について，区間内の全ての教室の被覆数を繰り返し文で愚直にチェックして，それらの最小値が2以上ならその区間を解の候補に入れるというアルゴリズムで正解する.

- 理由は，for文で愚直にチェックしたところで，2回より多く同じ教室を見ることはないので，高々 2N 回しかループしないから

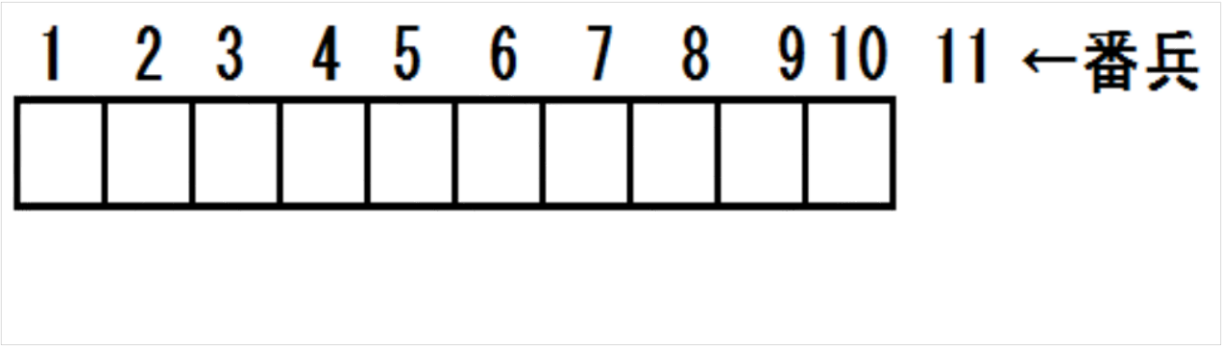
- 別解もたくさんある
  - 各教室について，その教室を被覆している区間の番号の列を持つ配列を用意する等

- 各教室がいくつの区間に被覆されてるか分かれば，
  ある教室を除いても全ての教室が掃除できる
  ⇔区間内の全ての教室が2つ以上の区間に被覆されている
  ⇔区間内に，被覆数が1の教室が含まれない

- 教室iの被覆数が1ならX[i]=1，そうでないならX[i]=0を記録した配列Xに対し累積和配列Sを生成すると，ある区間に含まれる被覆数1の教室の個数がO(1)で計算できるようになる

- 各区間に対してO(1)でそれが解に含まれるかどうかが判定できるようになり，判定にかかる計算量はO(M)となった.
- 全体の時間計算量,空間計算量は O(N+M) となり満点

- いもす法を行う



入力例

N=5,M=10

[1,4]

[5,5]

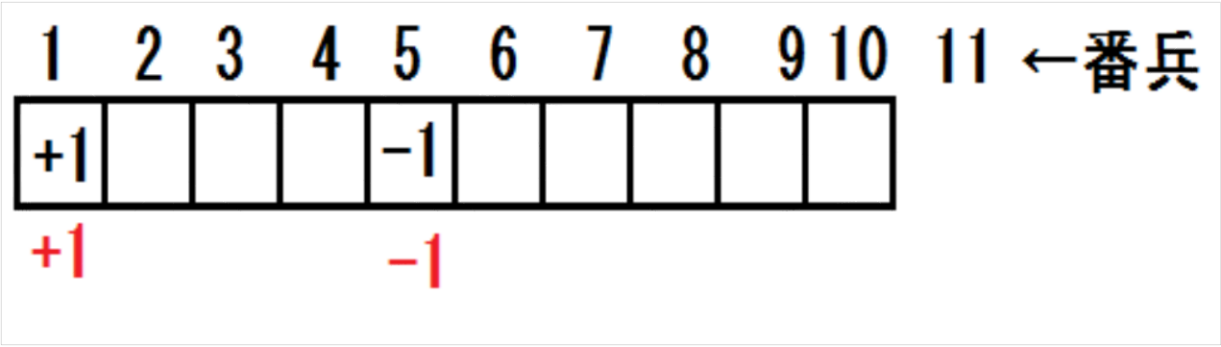
[5,6]

[6,8]

[9,10]

AtCoder

- いもす法を行う



入力例

N=5,M=10

[1,4]

[5,5]

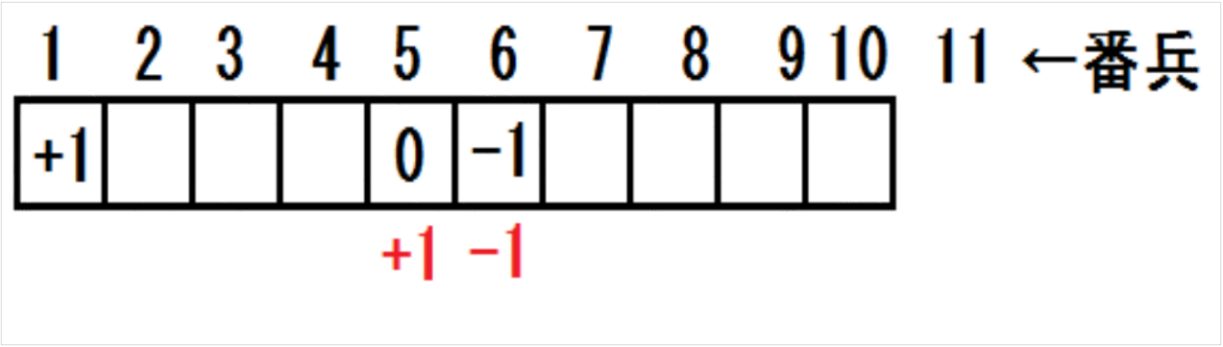
[5,6]

[6,8]

[9,10]

AtCoder

- いもす法を行う



入力例

N=5,M=10

[1,4]

[5,5]

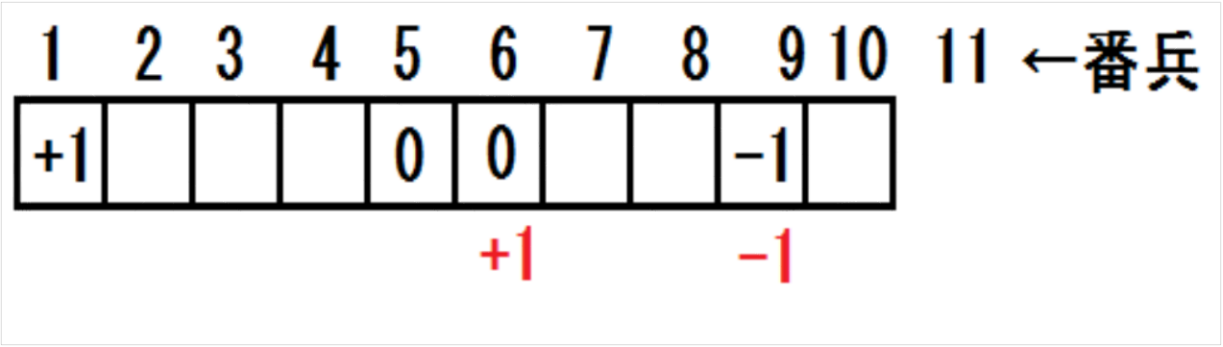
[5,6]

[6,8]

[9,10]

AtCoder

- いもす法を行う



入力例

N=5,M=10

[1,4]

[5,5]

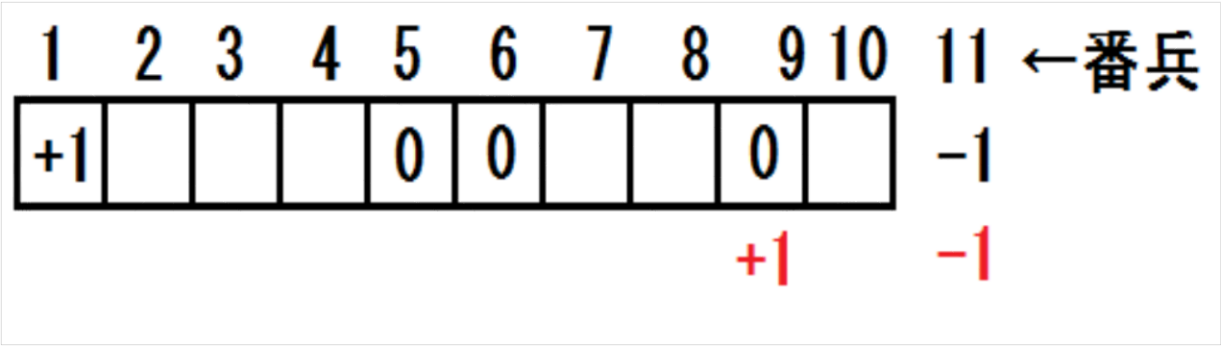
[5,6]

[6,8]

[9,10]

AtCoder

- いもす法を行う



入力例

N=5,M=10

[1,4]

[5,5]

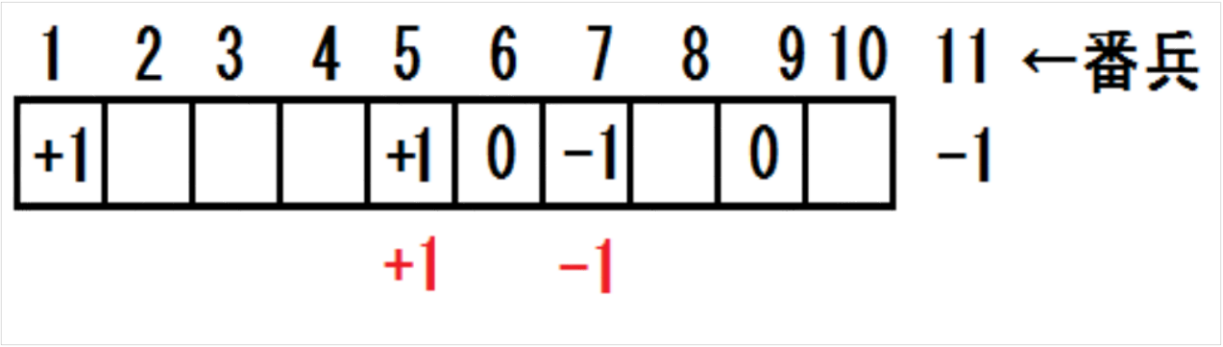
[5,6]

[6,8]

[9,10]

AtCoder

- いもす法を行う



入力例

N=5,M=10

[1,4]

[5,5]

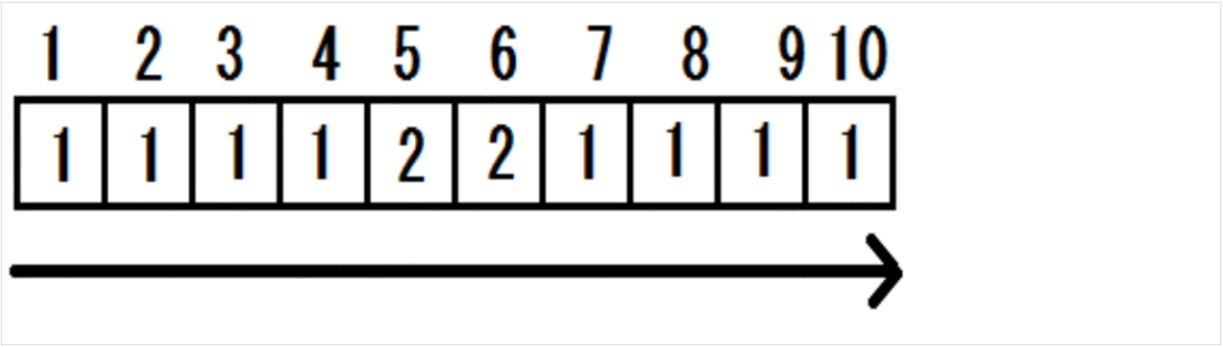
[5,6]

[6,8]

[9,10]

AtCoder

- いもす法を行う



→の方向に累積和を取るとこうなる

入力例

N=5,M=10

[1,4]

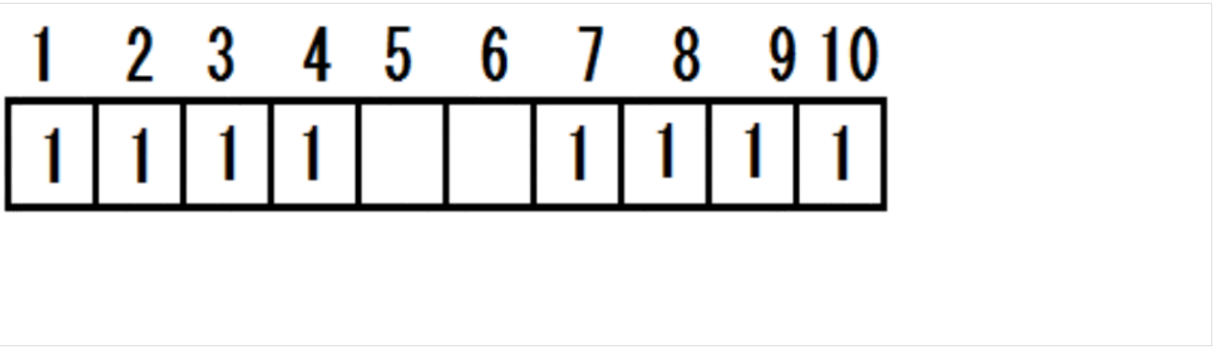
[5,5]

[5,6]

[6,8]

[9,10]

- 被覆数が1の部分だけに着目



- 満点解法で述べた配列Xを生成

入力例

N=5,M=10

[1,4]

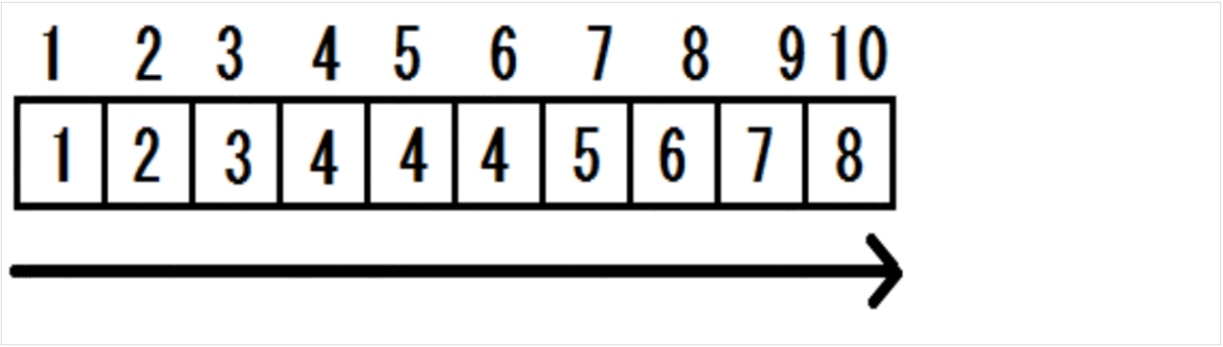
[5,5]

[5,6]

[6,8]

[9,10]

- 被覆数が1の部分だけに着目



再度→方向に累積和を取り
満点解法で述べた配列Sを生成

入力例

N=5,M=10

[1,4]

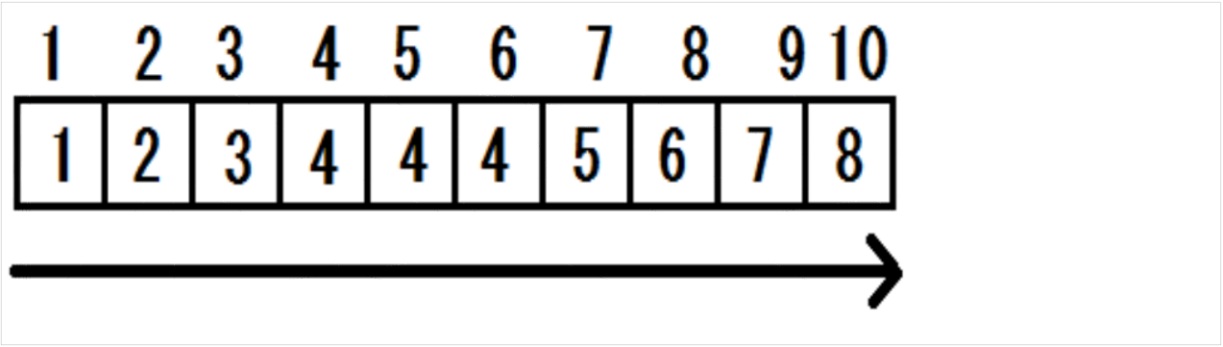
[5,5]

[5,6]

[6,8]

[9,10]

- 後は各区間について調べる



- 被覆数1の教室を含むかどうか判定
- [1,4] → S[4] – S[0](=0とする) = 4なので×
- [5,5] → S[5] – S[4] = 0なので○
- [6,8] → S[8] – S[5] = 2なので×
- [9,10] → S[10] – S[8] = 2なので×
- [5,6] → S[6] – S[4] = 0なので○

入力例

N=5,M=10

[1,4]

[5,5]

[5,6]

[6,8]

[9,10]

AtCoder

C問題　エックスオア多橋君(解説スライド:三上)

1. 問題概要
2. 考察
3. 解法
4. アルゴリズム動作例

©AtCoder Inc. All rights reserved.

AtCoder

- 辺に非負整数の重みがついたN頂点の木が与えられる.
- 整数Xも与えられる.
- aからbの単純パス上の辺の重みのxor和がXになるような(a,b)の組(a<b)の個数を求めよ.

- 1≦N≦10^5
- 0≦X,辺のコスト≦10^9

AtCoder

- 適当な頂点xを根に行ったdfsで，根からの経路の総和 xor を持っておくと嬉しい．なぜなら，

  パスa→bのxor値 = (a→xのxor 値) xor (x→bのxor 値)

- が成り立つから．

- 理由は a→x→bという経路の中で2回通る辺のコストは打ち消し合うから

- 適当な頂点からdfsを行い，根からの頂点iまでの単純パスのxor値x[i]が出現した回数をハッシュや平衡二分木(C++のmap等)を用いて記録していく.
- 各x[i]が出現した回数を加算する前に，X xor x[i] という値が今までに出現した回数を答えに加算していけば良い.
- 時間・空間計算量はハッシュを用いた場合 O(N)

[コーナーケース]

- X=0のとき，(x,x)を組として数えないように気をつけること(求めるのはa<bとなる組の個数です).
- 32bit整数だとオーバーフローします.

- X=0のとき，(x,x)を組として数えないように気をつけること(a<bとなる組の個数です)
- 答えは32bit整数だとオーバーフローします.

- 別解はいくつかあります
  - データ構造をマージする一般的なテク（平衡二分木をマージしていくとO(nlog^2n)の時間計算量等で達成可能）
  - オイラーツアーを構築して列の問題に帰着させる（本質的にはこのスライドで述べた方法と同じ）

みんな仲良し高橋君

解説スライド担当: 森田 晃平(@yosupot)

問題概要

- 点が2N+1個与えられる
- x座標かy座標が同じ点はマッチング出来る
- “この点以外の点で完全マッチングが作れる”かどうかを全ての点について判定
- N ≦ 100,000

考察

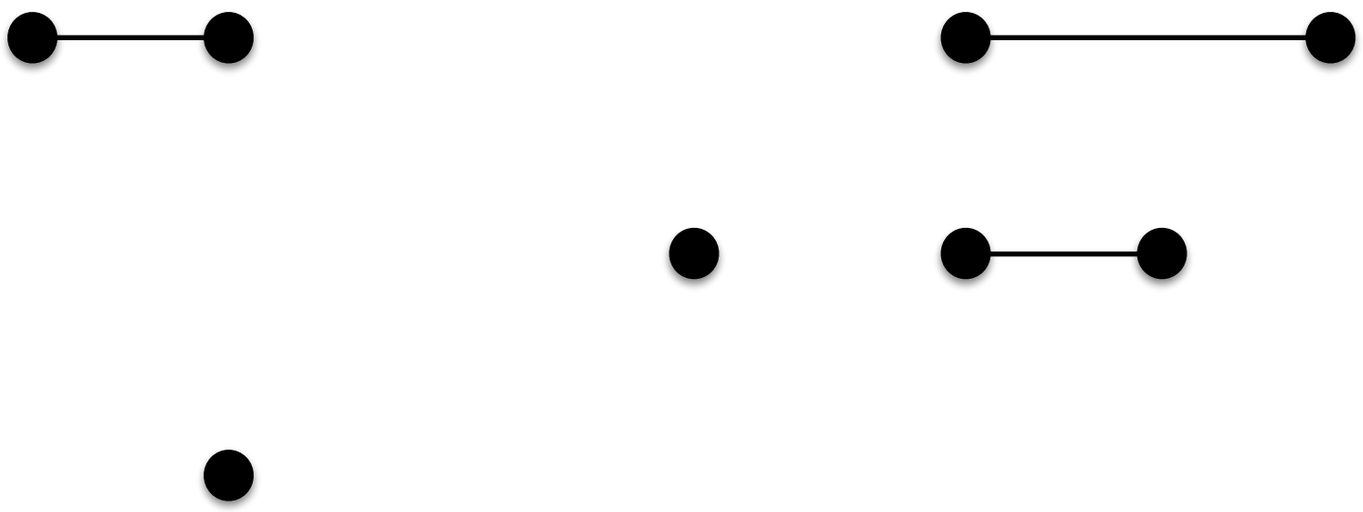
- 実は、連結成分のサイズが偶数ならば必ず完全マッチングが作れる

考察

- マッチングを作っていく
- もしどのマッチングにも含まれない点が2個以上存在するならば、必ずマッチングが増やせることを証明する

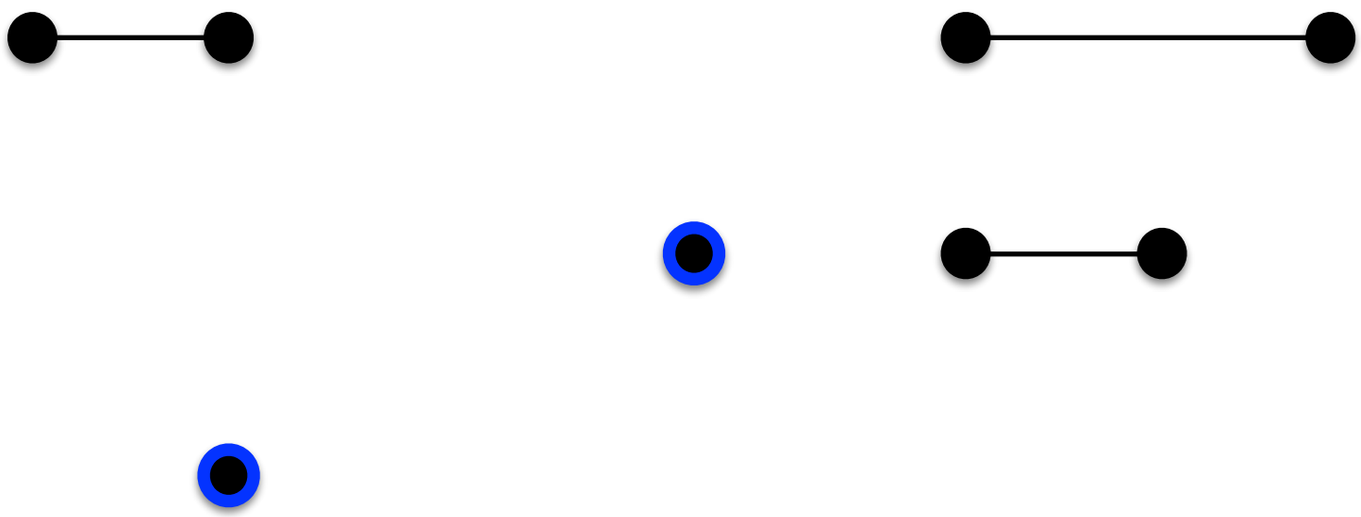
例

- 以下のグラフで考える



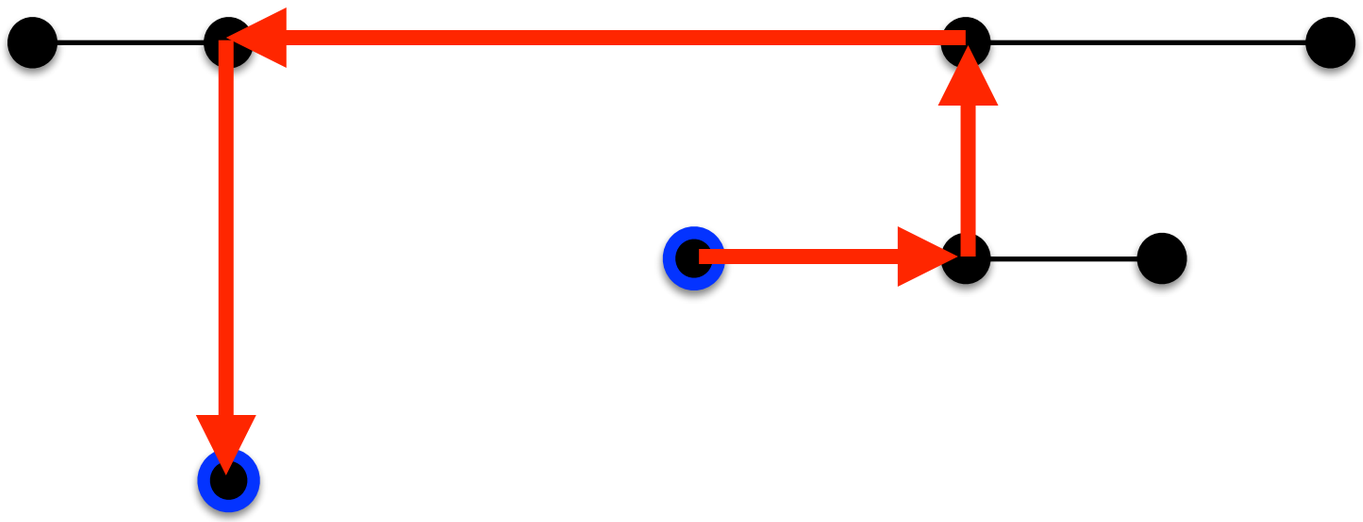
例

・まず使ってない点を2点選ぶ



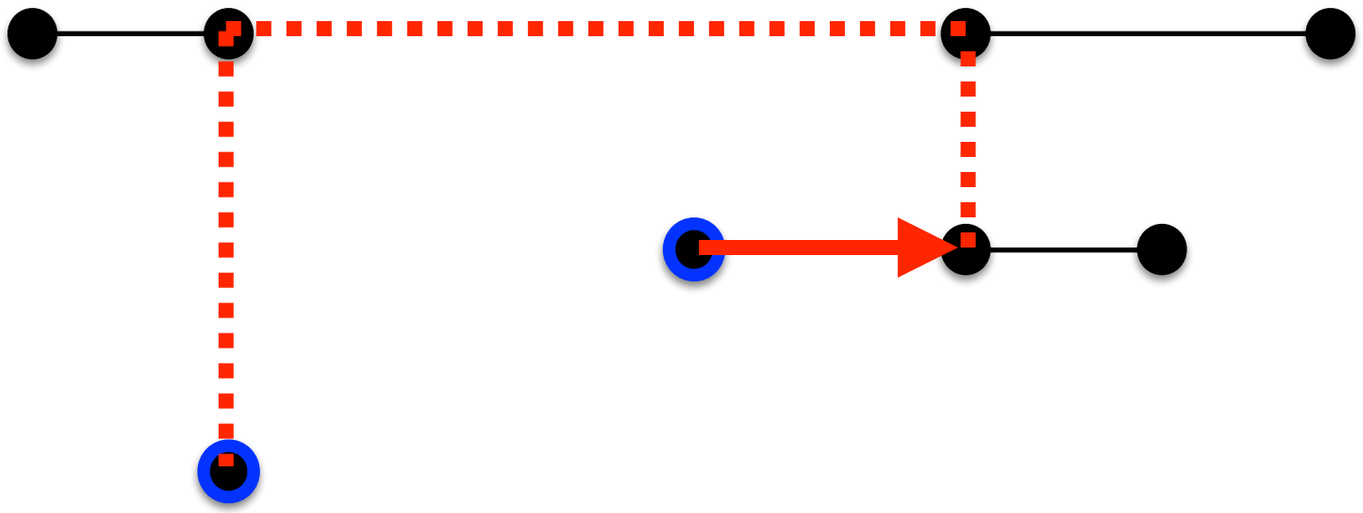
例

- この2点間をつなぐパスを用意する
- パスにそってマッチングしていない点を動かしていく



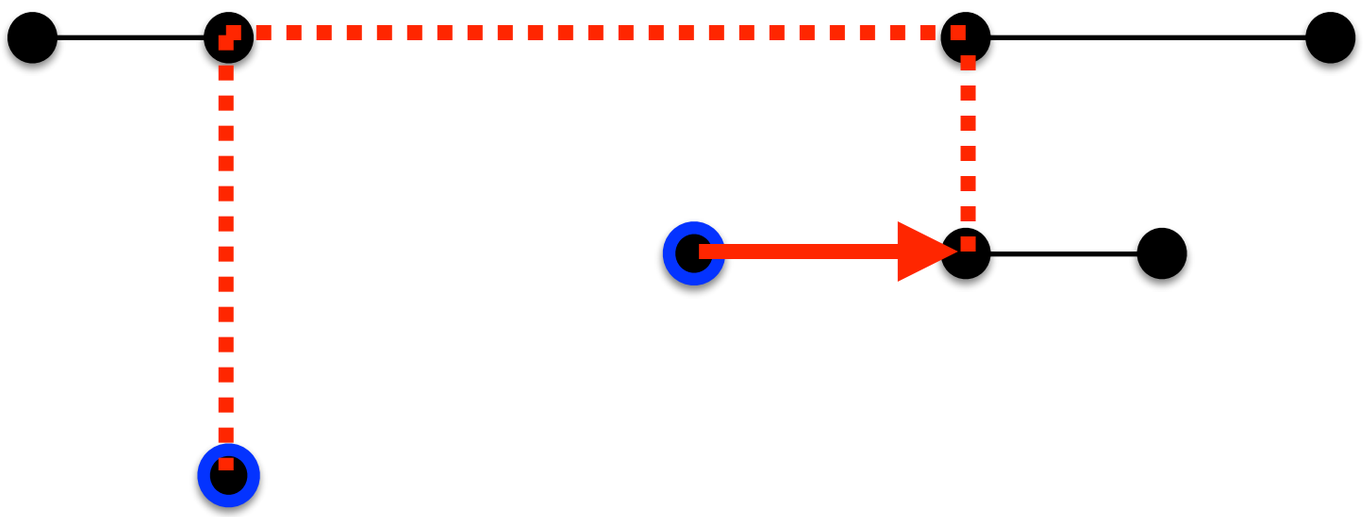
例

- パスにそってマッチングしていない点を動かしていく



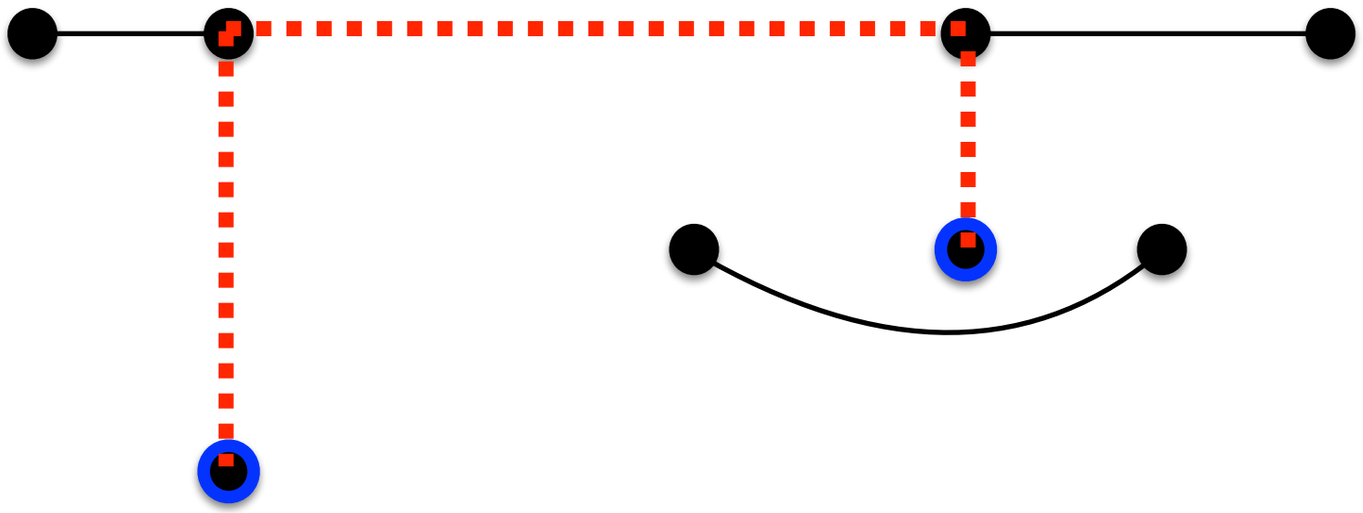
例

- 動かす先の点のマッチングが、矢印と並行な場合



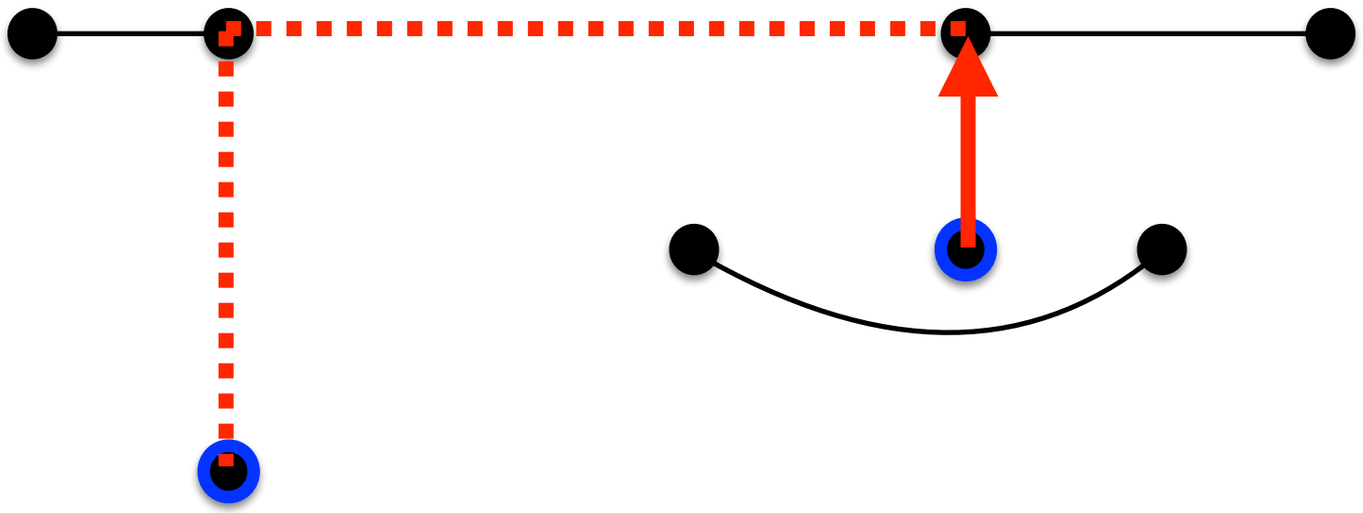
例

- 動かす先の点のマッチングが、矢印と並行な場合



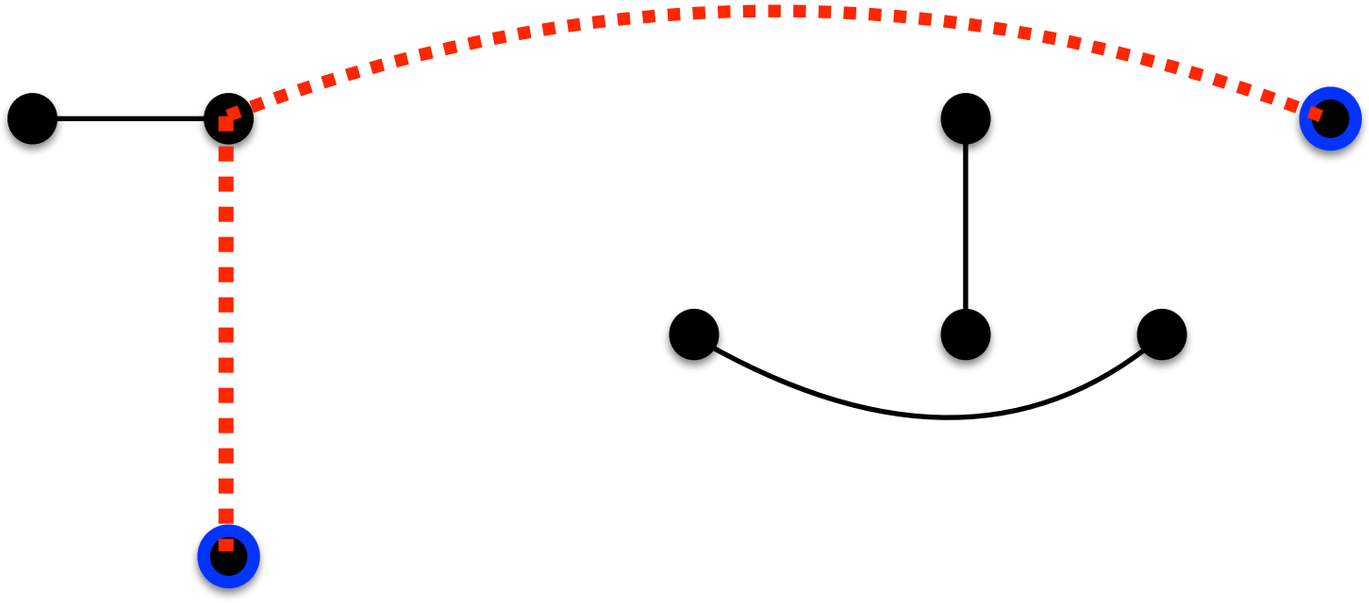
例

- 動かす先の点のマッチングが、矢印と垂直な場合



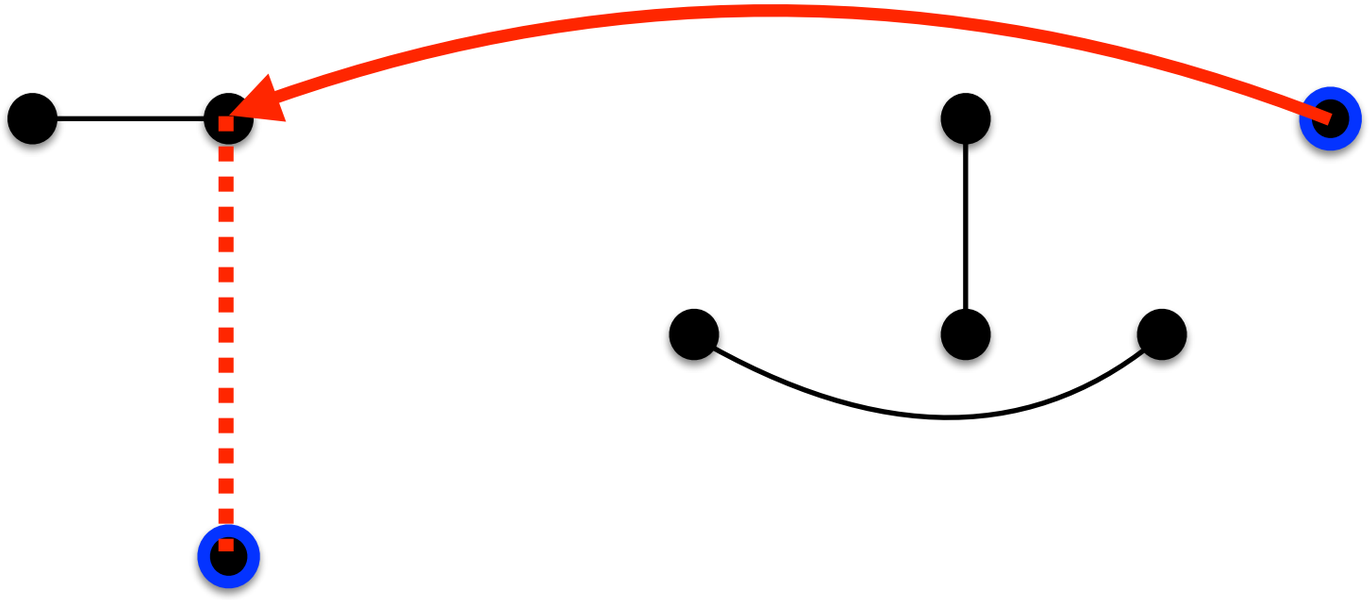
例

- 動かす先の点のマッチングが、矢印と垂直な場合



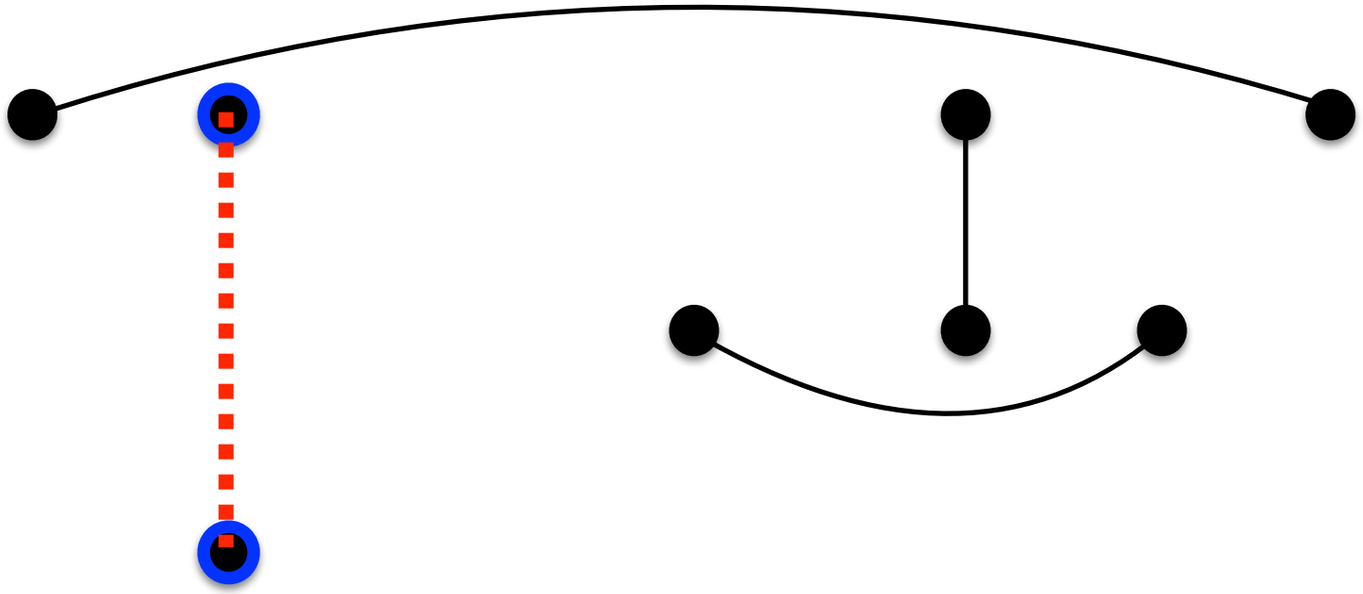
例

- 動かす先の点のマッチングが、矢印と並行な場合



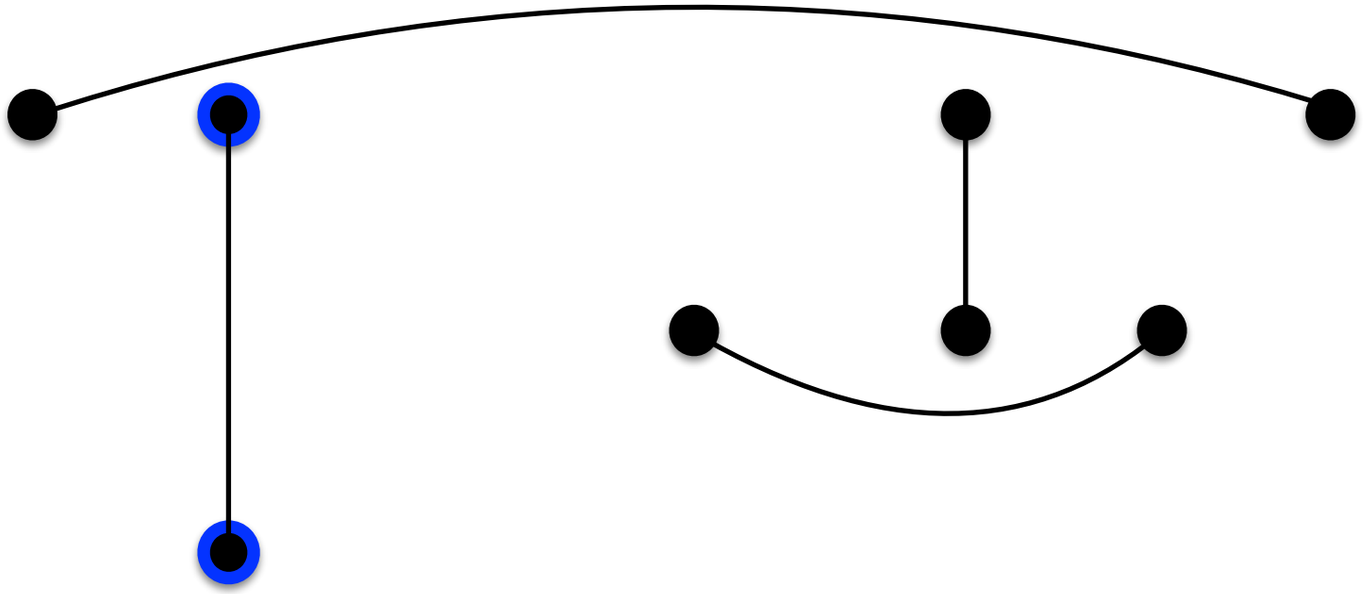
例

- 動かす先の点のマッチングが、矢印と並行な場合



例

・これでマッチングしてない点同士を繋げられた



考察

- 連結成分のサイズが偶数ならば必ず完全マッチング
  が作れることがわかった

- この問題は、点を消した時に偶数の連結成分だけが
  残るような点を列挙する問題

考察

- 最初の状態で連結成分が1個だけの場合を考えればよい
  - 奇数の連結成分が複数あったら全部NG
  - 偶数の連結成分はなかったものと考えて良い
- 点を削除して連結成分が増える、かつ増えた連結成分のうち点が奇数個のものがある場合のみNG

考察

- 点を削除した結果連結成分が増えるような点を関節点と呼ぶ。これはO(V + E)で列挙可能(後述)

- 消した結果出来た連結成分のサイズが奇数かどうかの判定も実はまとめてできる(後述)

- ただ普通にグラフを作るとEは最悪θ(V^2)になるので、減らさなくてはいけない

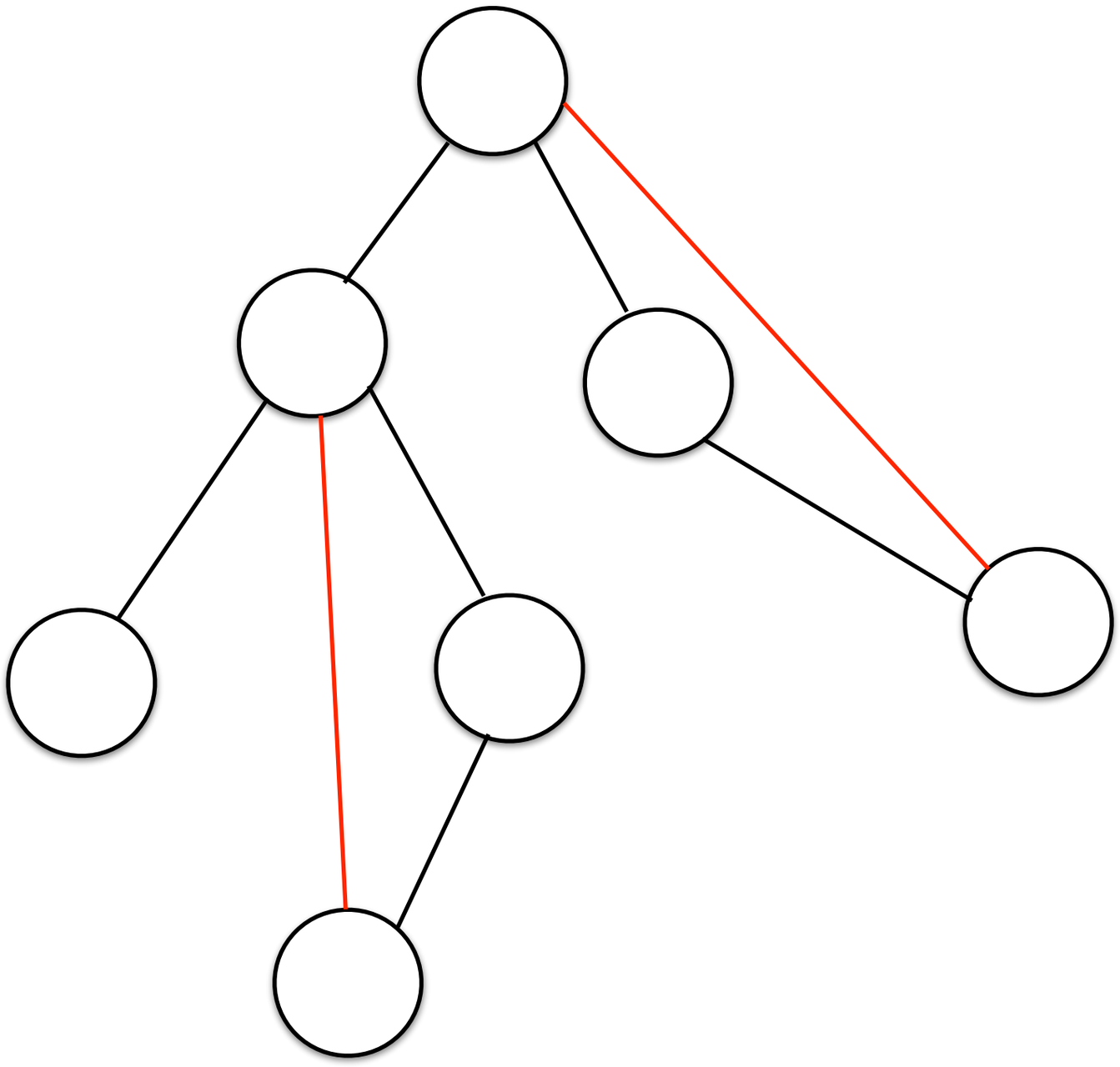
考察

- O(V^2)本の辺を全て張る必要はない
- もし一つの点からたくさん辺が伸びている場合は、4方向毎に2番目に近い点までのみに辺を貼ればよい
- これで辺の本数はO(V)、これであとはO(V+E)で関節点列挙すれば良い

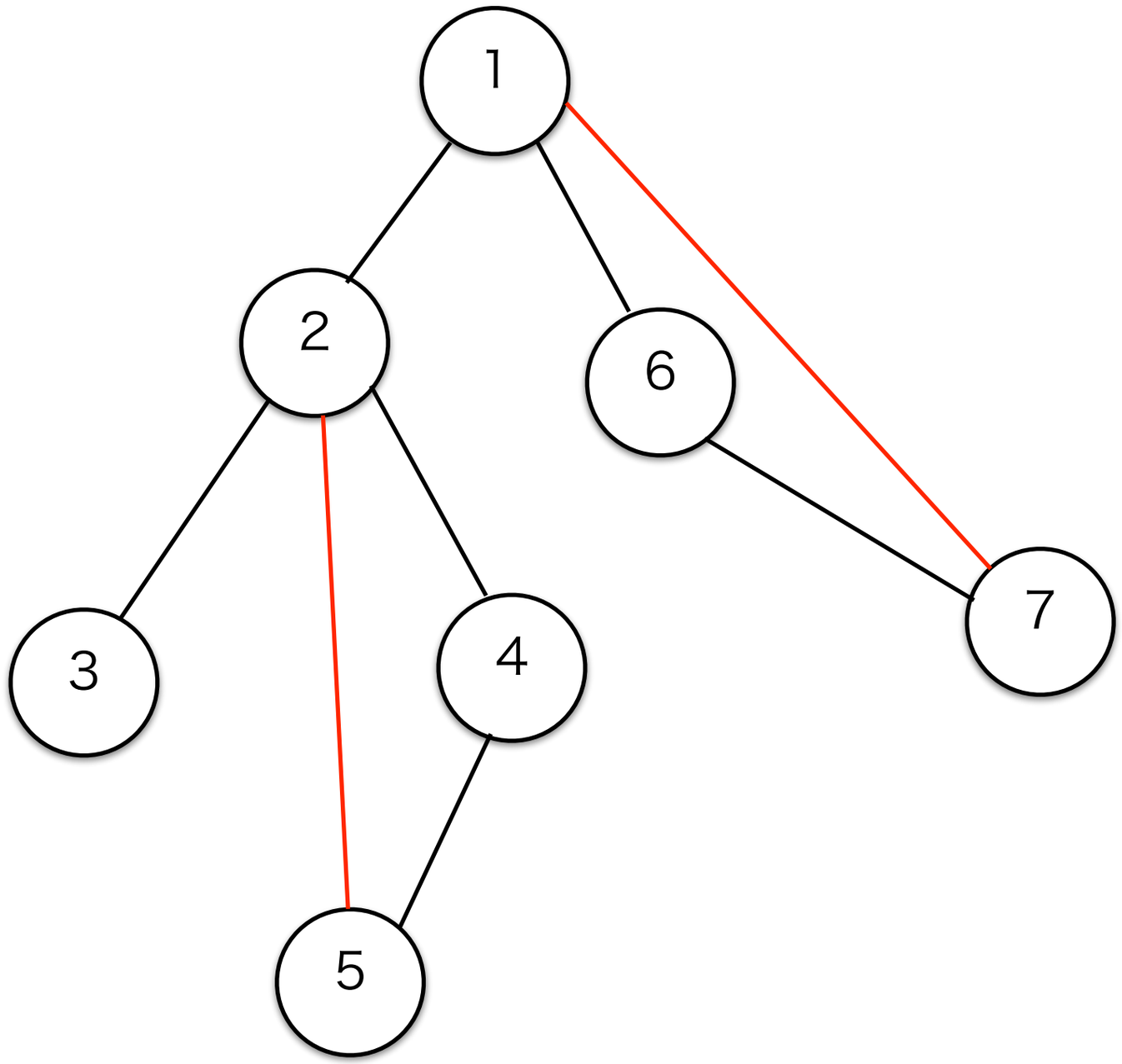
関節点列挙

- lowlinkという方法で列挙できます
- dfs木を作り、頂点毎にordとlowというパラメーターを計算する
- ordはdfs木上での行きがけ順序
- lowは木の辺は降りることのみ可能、ただし後退辺を最後に1度だけ登っても良い　という条件下での到達可能な頂点の中でのordの最小値

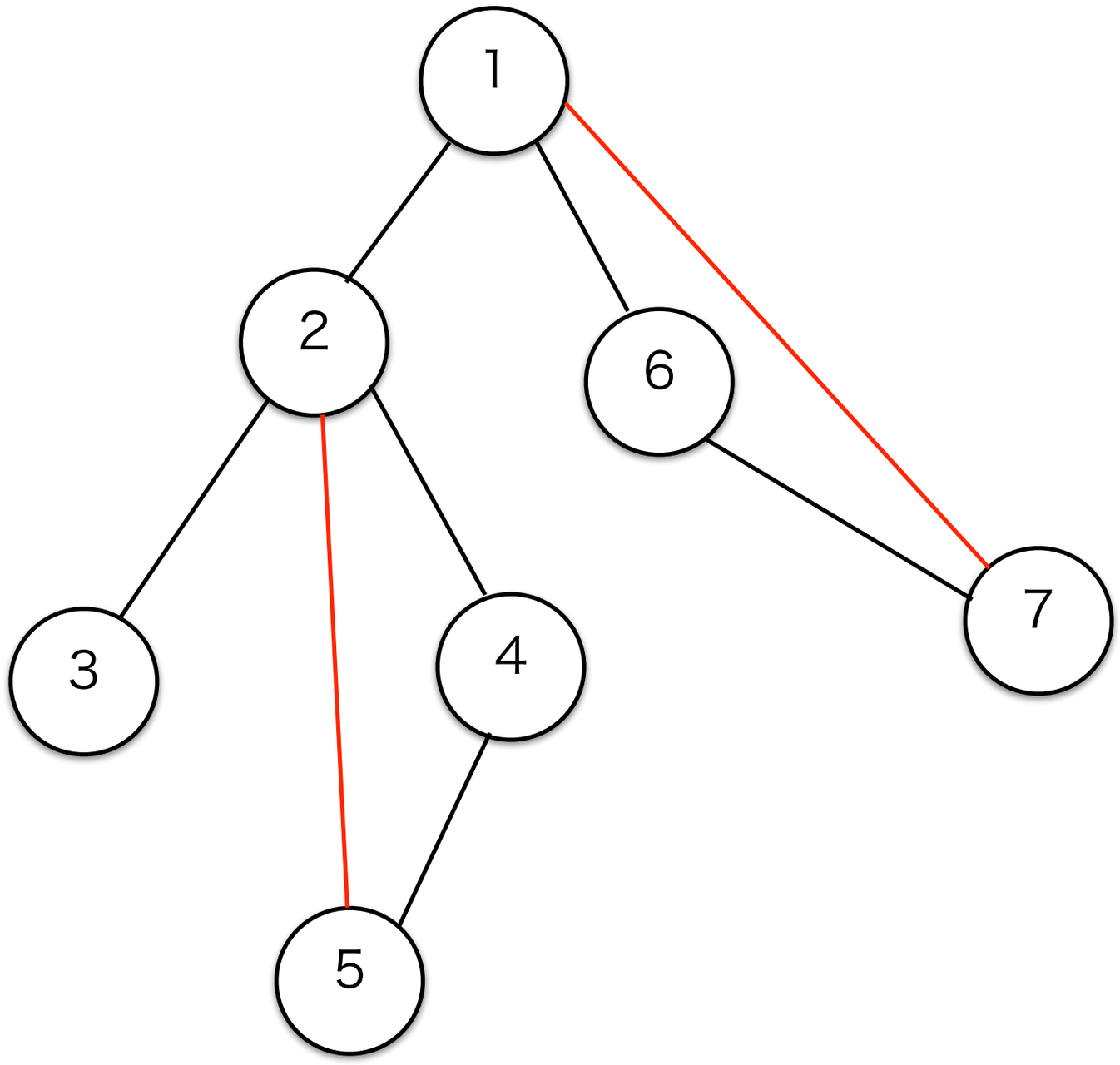
関節点列挙



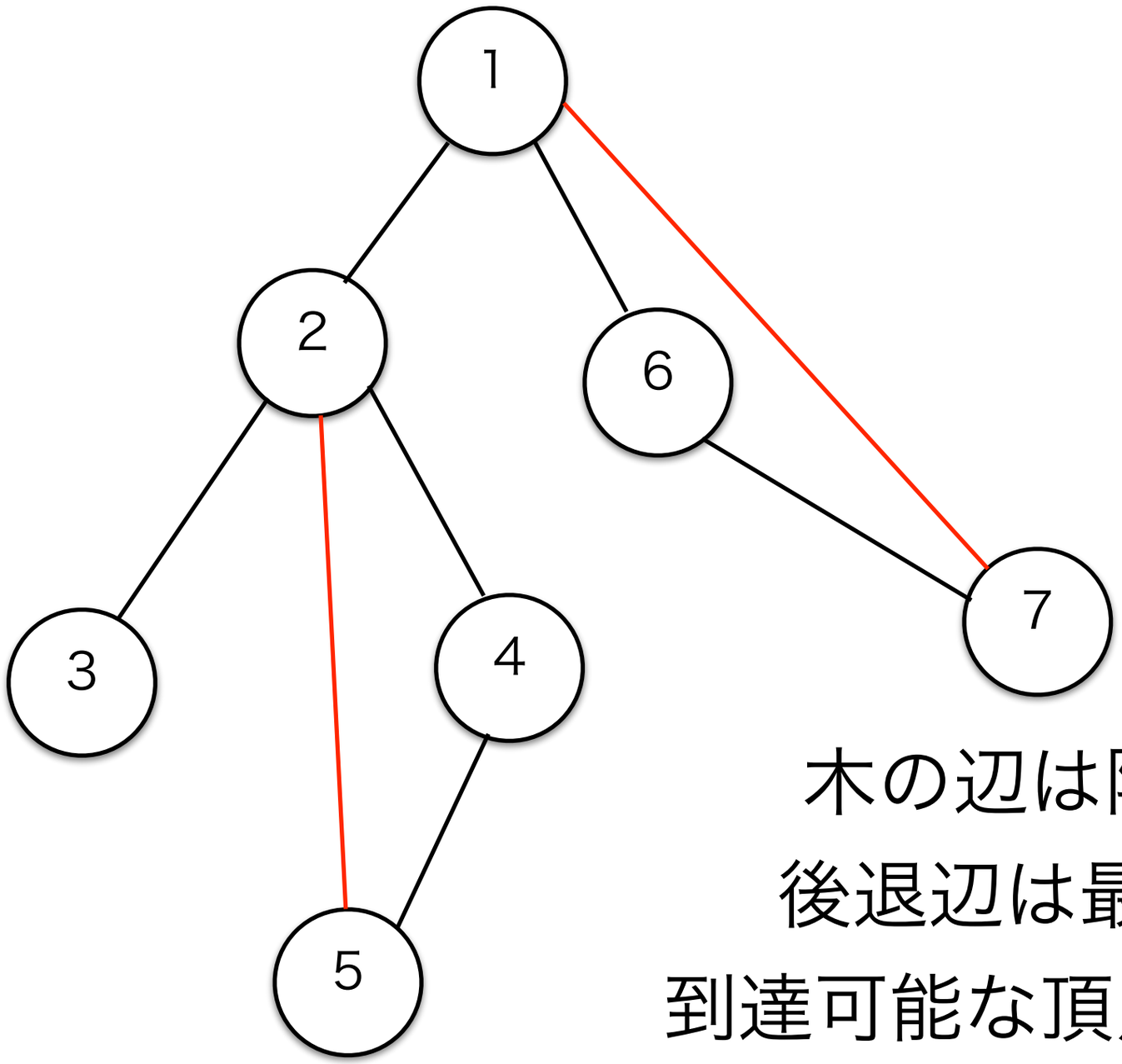
ord(行きがけ順序)



lowとは？？？



lowとは？？？

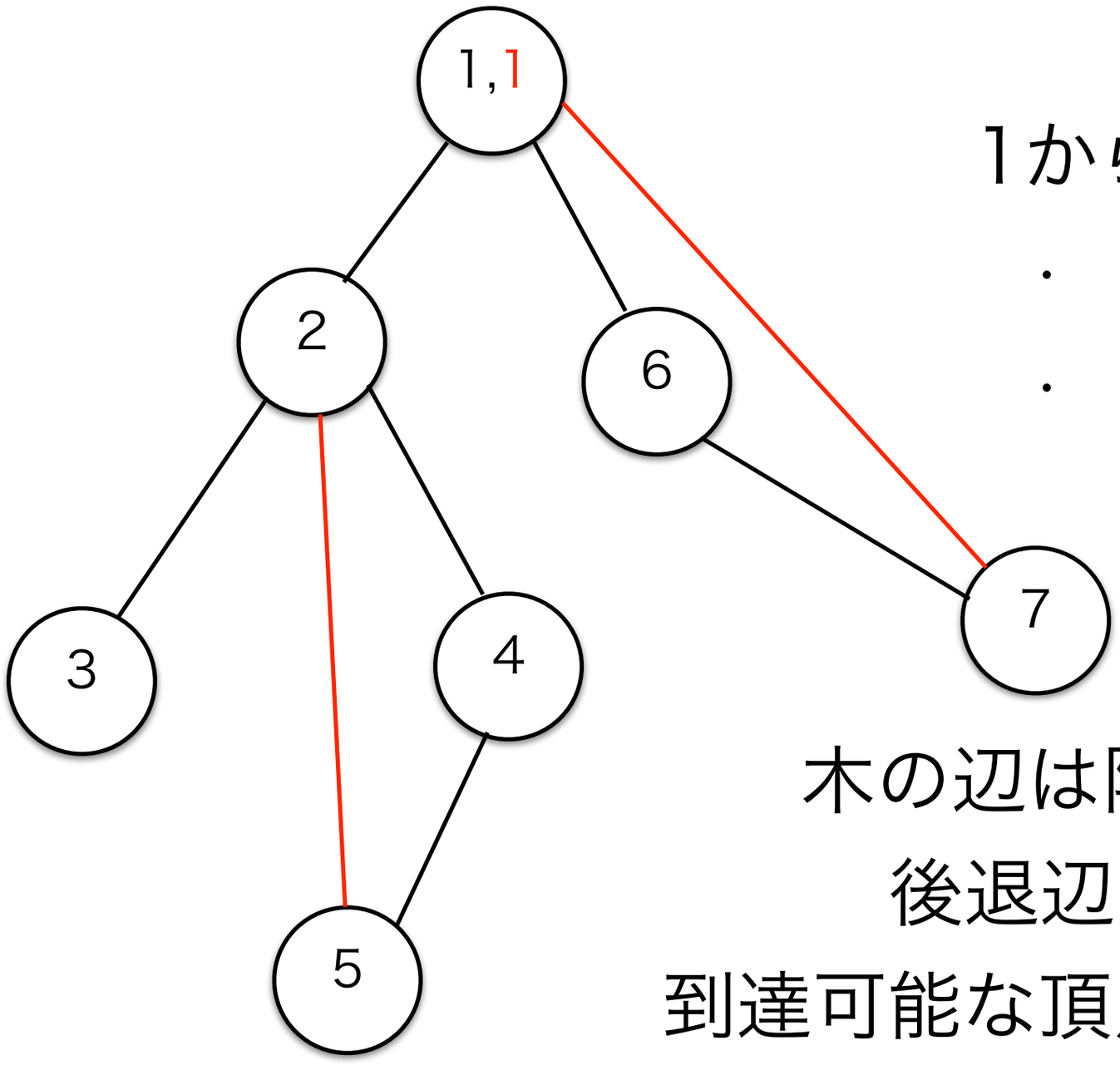


木の辺は降りることのみ可能

後退辺は最後に1度だけ登れる

到達可能な頂点の中でのordの最小値

lowとは？？？



1から到達可能な頂点は？

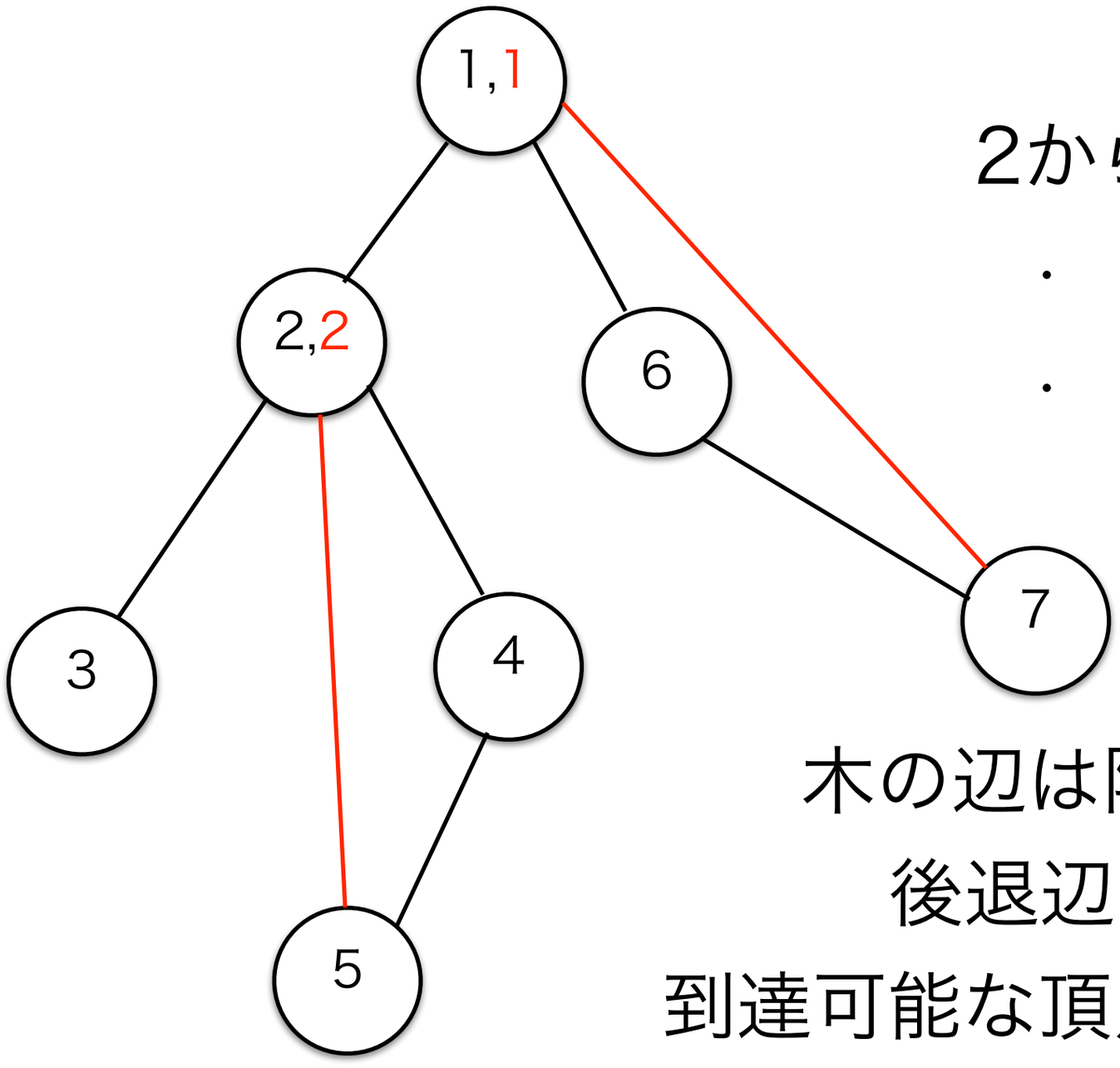
- 当然全ての頂点
- 頂点1のlowは1

木の辺は降りることのみ可能

後退辺は1度だけ登れる

到達可能な頂点の中でのordの最小値

lowとは？？？



2から到達可能な頂点は？

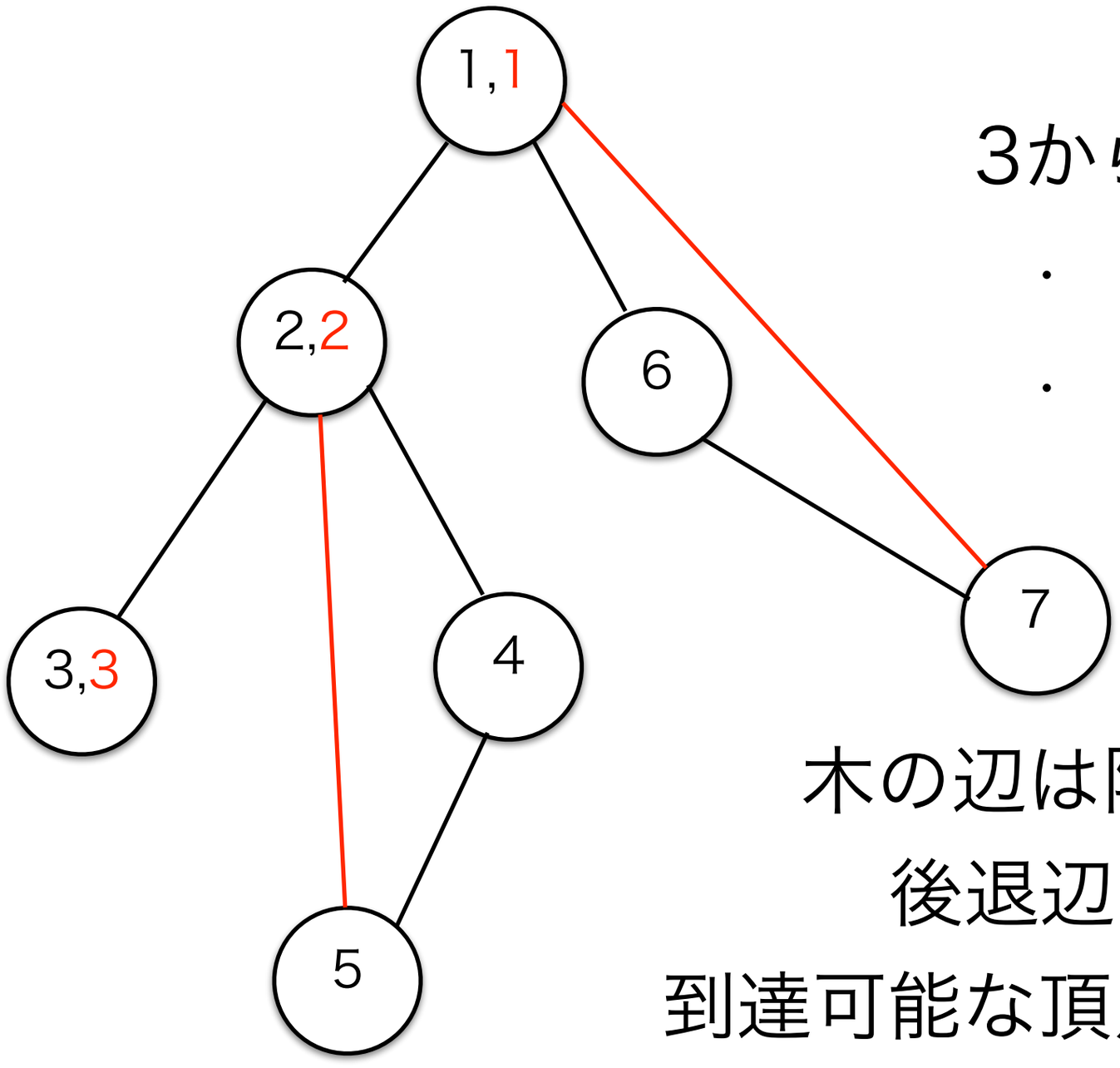
- 2,3,4,5
- 頂点2のlowは2

木の辺は降りることのみ可能

後退辺は1度だけ登れる

到達可能な頂点の中でのordの最小値

lowとは？？？



3から到達可能な頂点は？

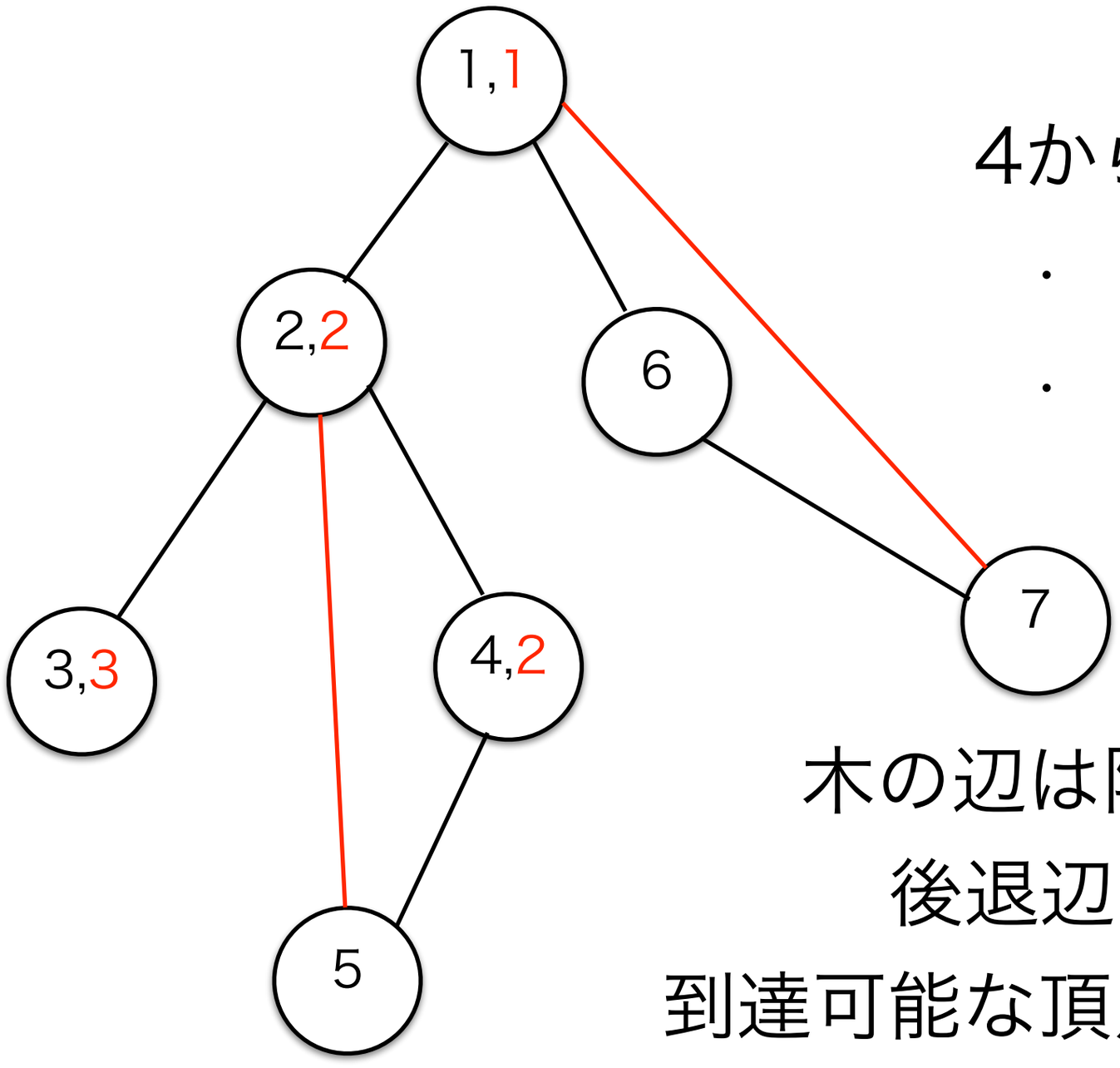
- 3
- 頂点3のlowは3

木の辺は降りることのみ可能

後退辺は1度だけ登れる

到達可能な頂点の中でのordの最小値

lowとは？？？



4から到達可能な頂点は？

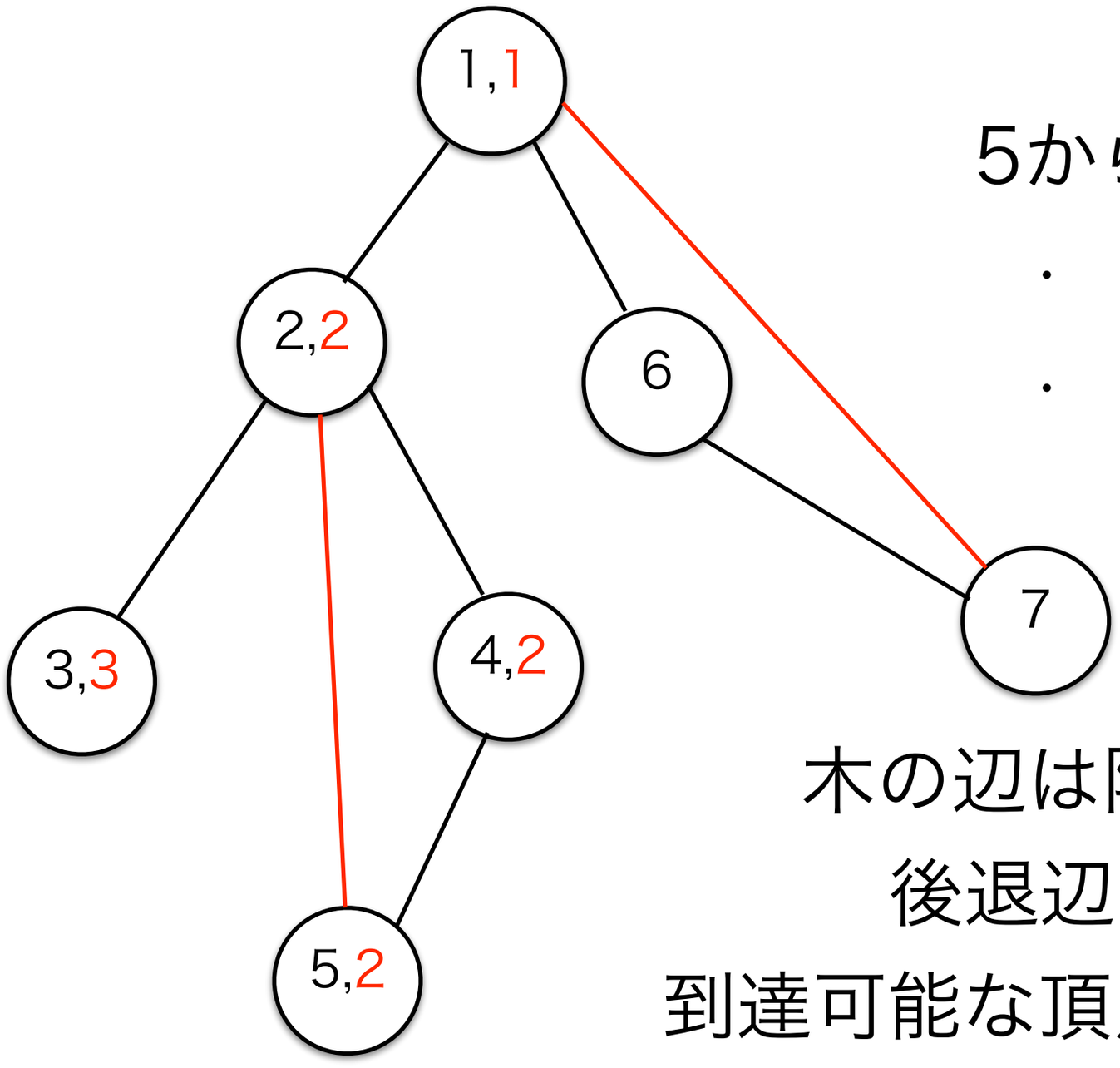
- 4,5,2
- 頂点4のlowは2

木の辺は降りることのみ可能

後退辺は1度だけ登れる

到達可能な頂点の中でのordの最小値

lowとは？？？



5から到達可能な頂点は？

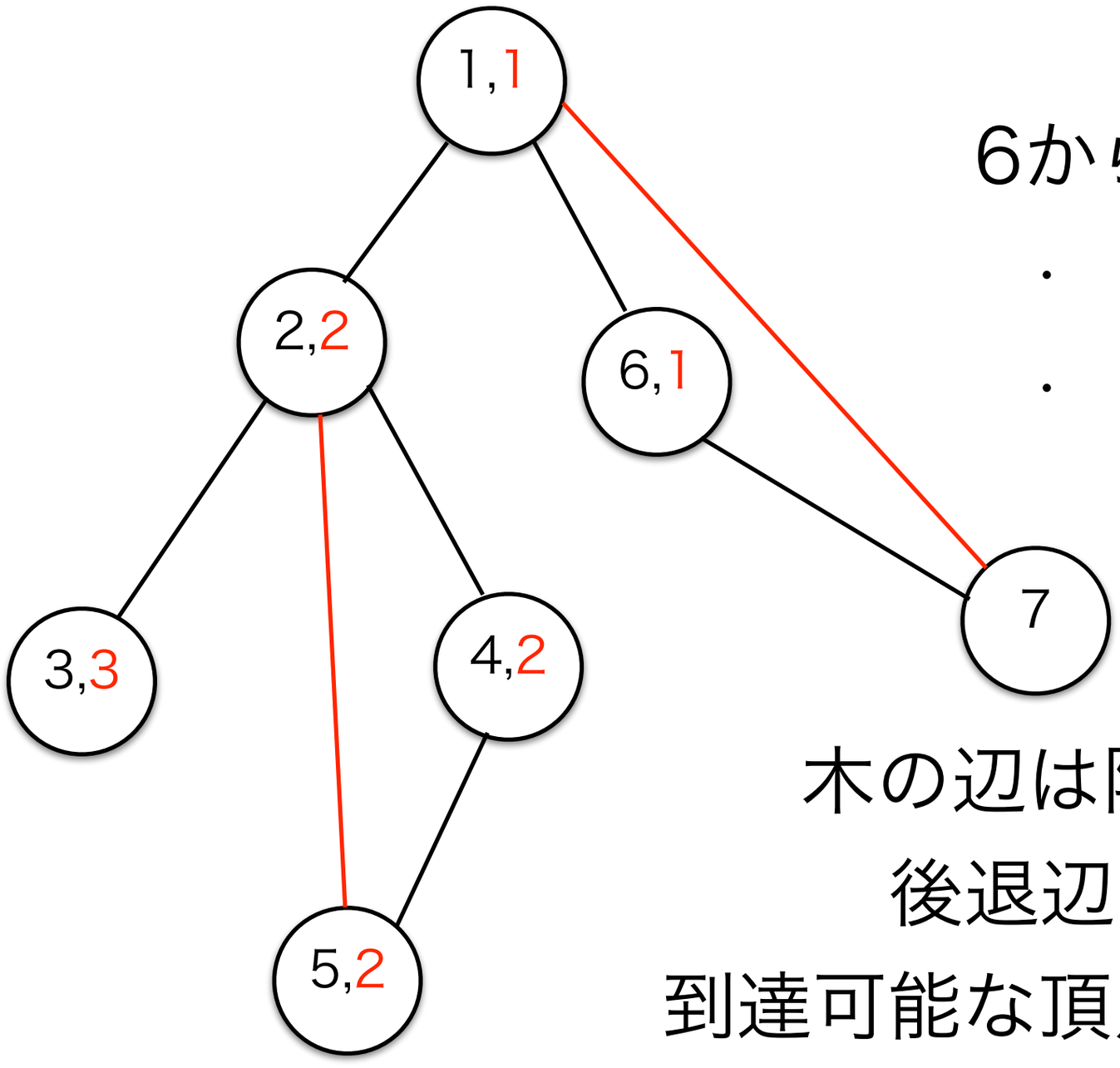
- 5,2
- 頂点5のlowは2

木の辺は降りることのみ可能

後退辺は1度だけ登れる

到達可能な頂点の中でのordの最小値

lowとは？？？



6から到達可能な頂点は？

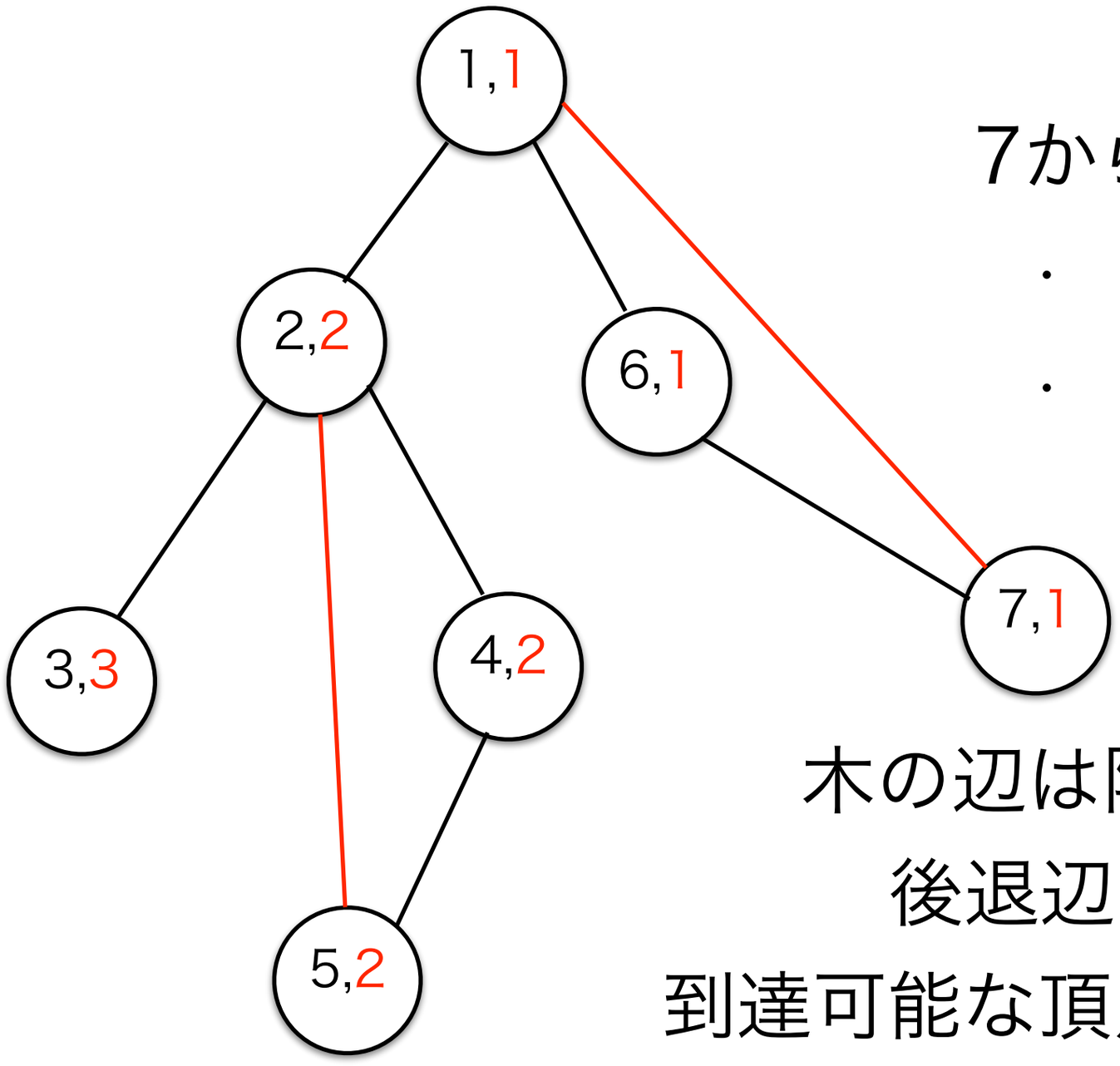
- 6,7,1
- 頂点6のlowは1

木の辺は降りることのみ可能

後退辺は1度だけ登れる

到達可能な頂点の中でのordの最小値

lowとは？？？



7から到達可能な頂点は？

- 7,1
- 頂点7のlowは1

木の辺は降りることのみ可能

後退辺は1度だけ登れる

到達可能な頂点の中でのordの最小値

関節点列挙

- これがわかると何が嬉しいの？
- 実は、関節点であるかは以下の条件を満たすかで判定できる
  - (根)次数が2以上
  - (根以外, vとする)子uの中にlow[u] ≧ ord[v]なる頂点が存在する

関節点列挙

- これがわかると何が嬉しいの？
- 実は、関節点であるかは以下の条件を満たすかで判定できる
  - (根)次数が2以上　これはdfs木の性質から明らか
  - (根以外, vとする)子uの中にlow[u] ≧ ord[v]なる頂点が存在する

関節点列挙

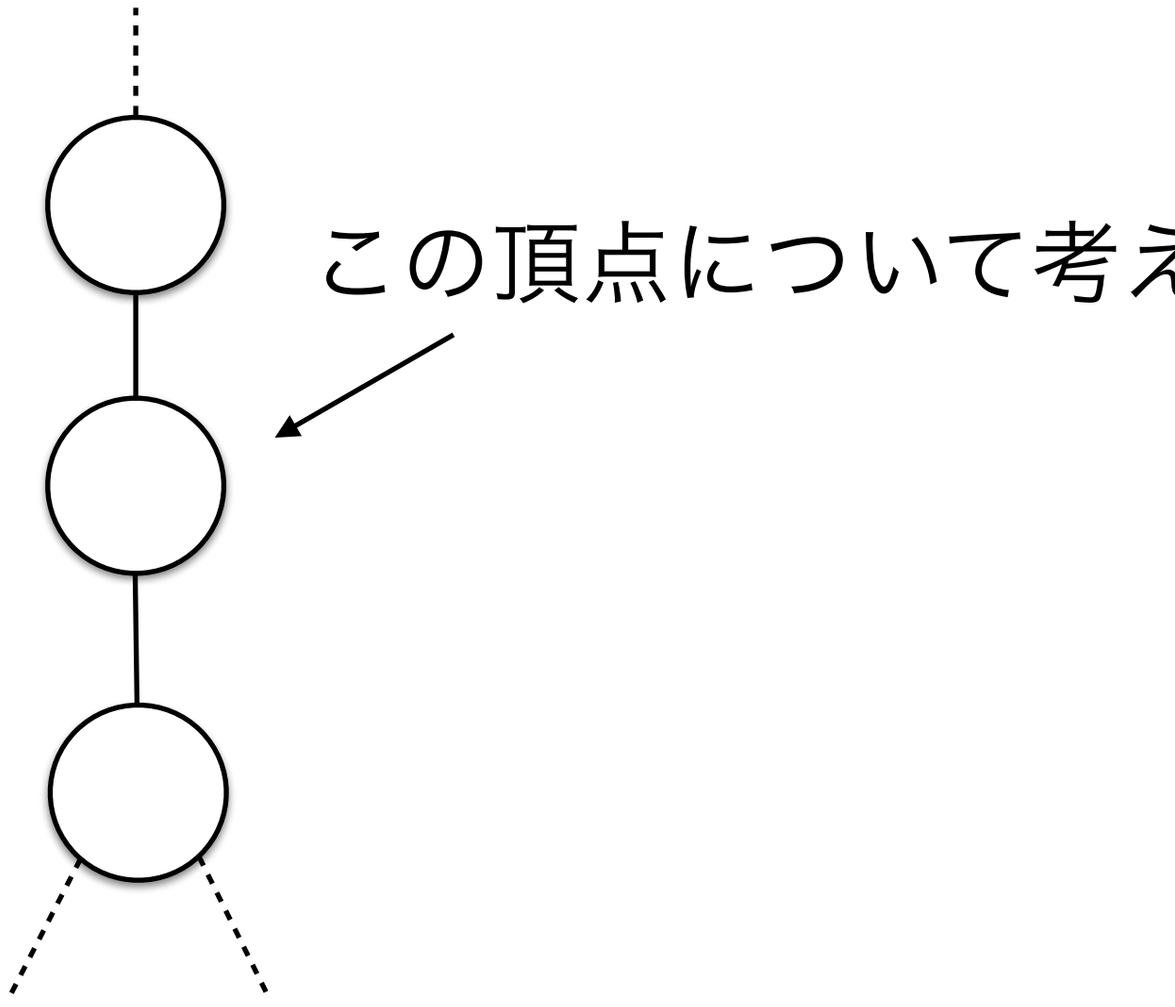
- これがわかると何が嬉しいの？
- 実は、関節点であるかは以下の条件を満たすかで判定できる
  - (根)次数が2以上
  - (根以外, vとする)子uの中にlow[u] ≧ ord[v]なる頂点が存在する

こっちは？

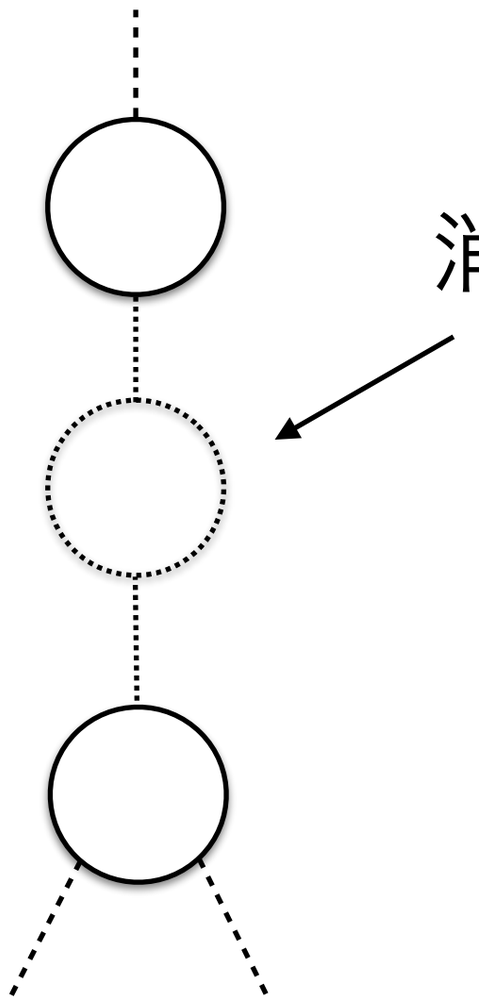
関節点列挙

- 関節点はdfs木上でどのような点になるか？
- 頂点が消された時にその頂点の子のなかに、分離してしまうものがあるならば関節点

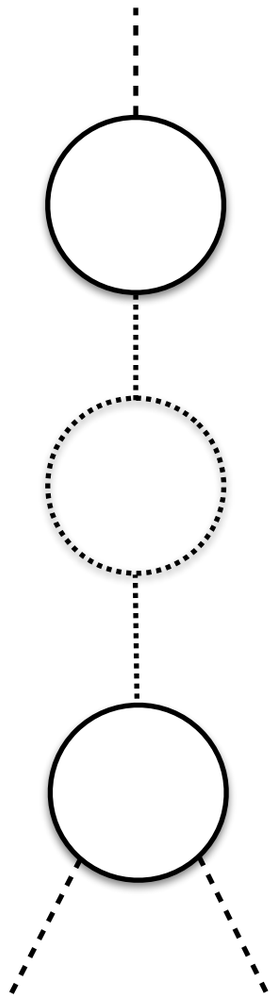
関節点列挙



関節点列挙

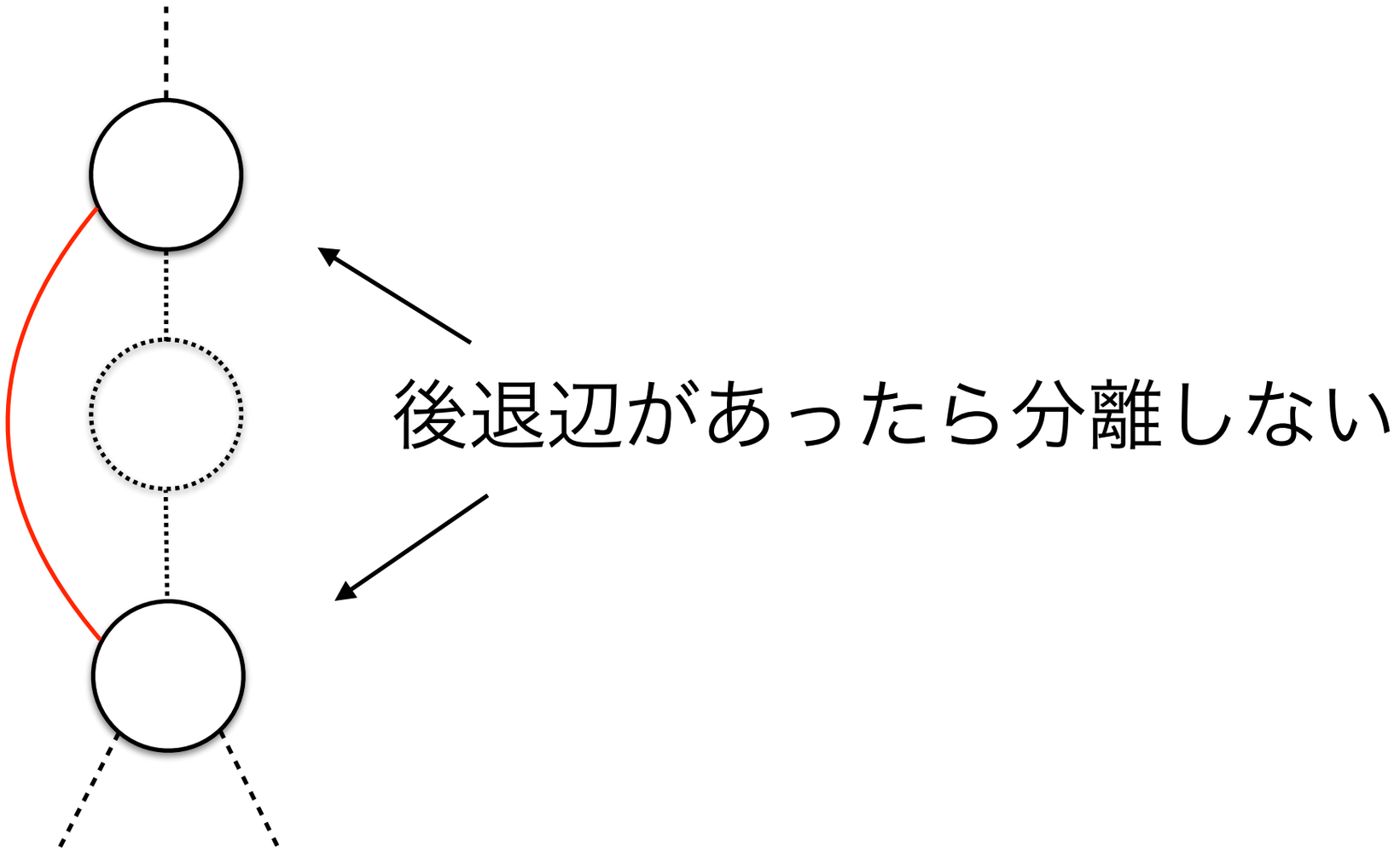


関節点列挙

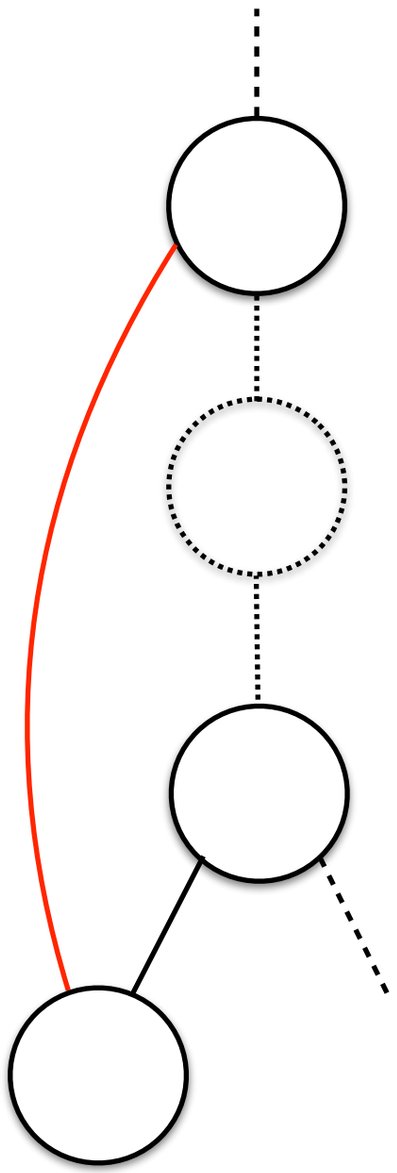


もしこの2つが分離したら関節点

関節点列挙

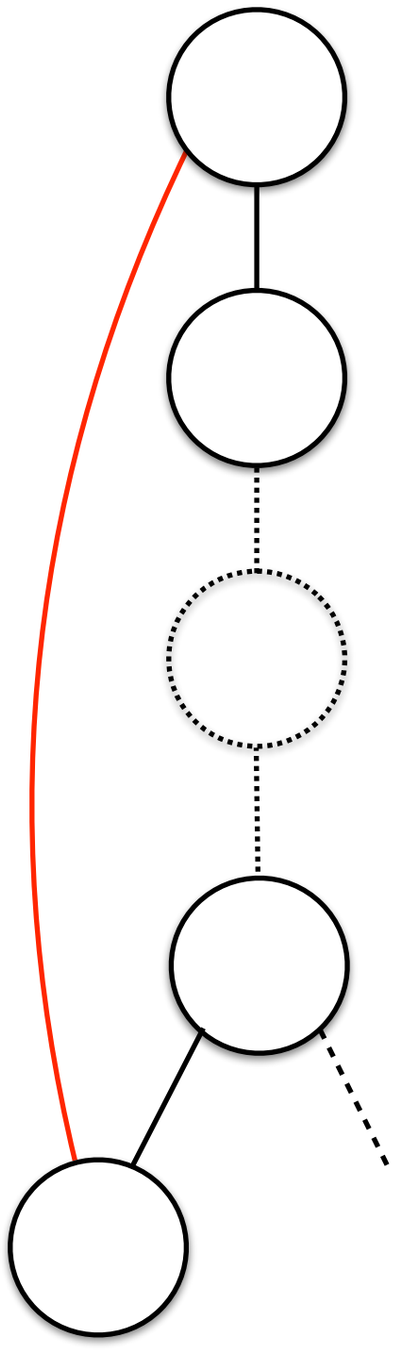


関節点列挙



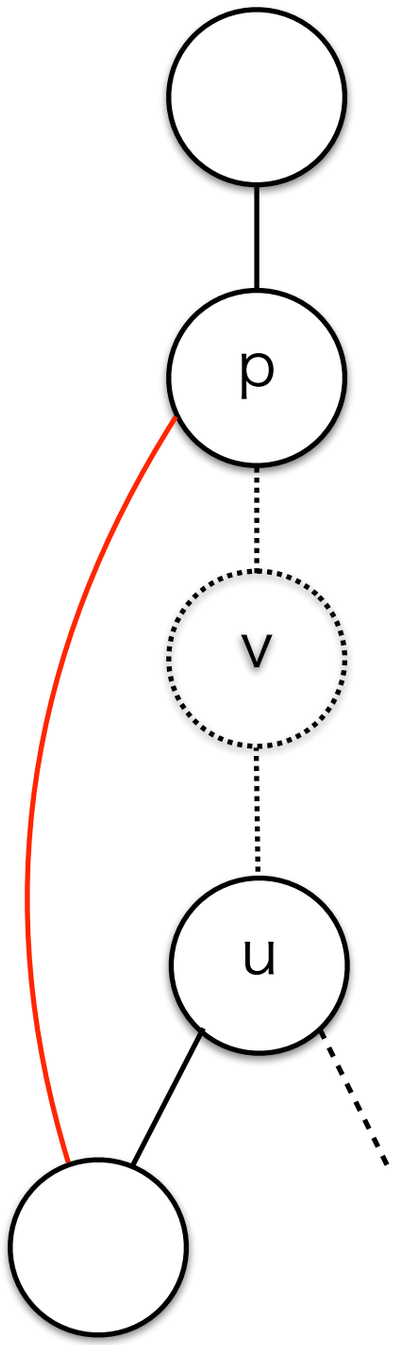
こういう後退辺があっても分離しない

関節点列挙



こういう後退辺があっても分離しない

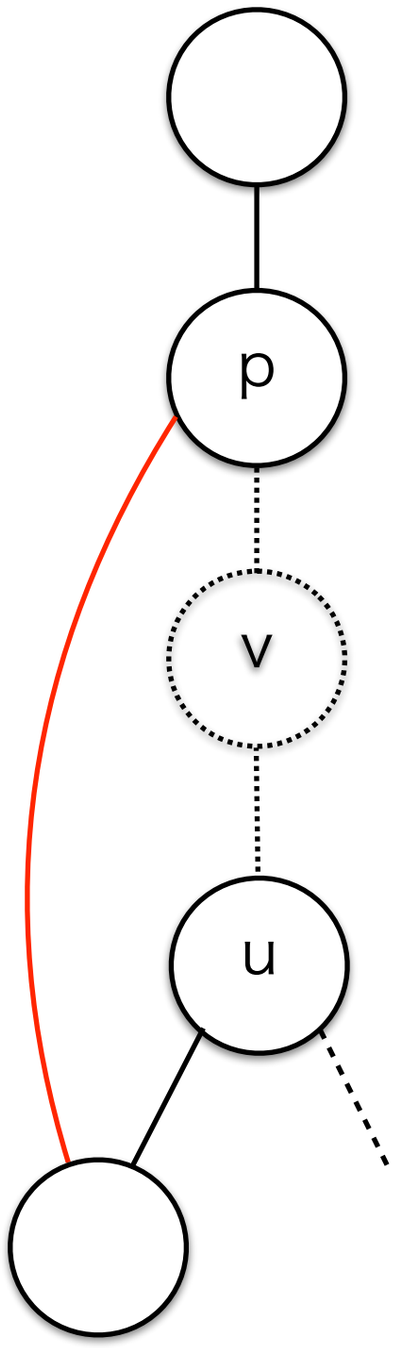
関節点列挙



どういう後退辺があったら分離しないか？

uの部分木のどこかと、
pとpの親のうちどこかをつなぐような
後退辺

関節点列挙

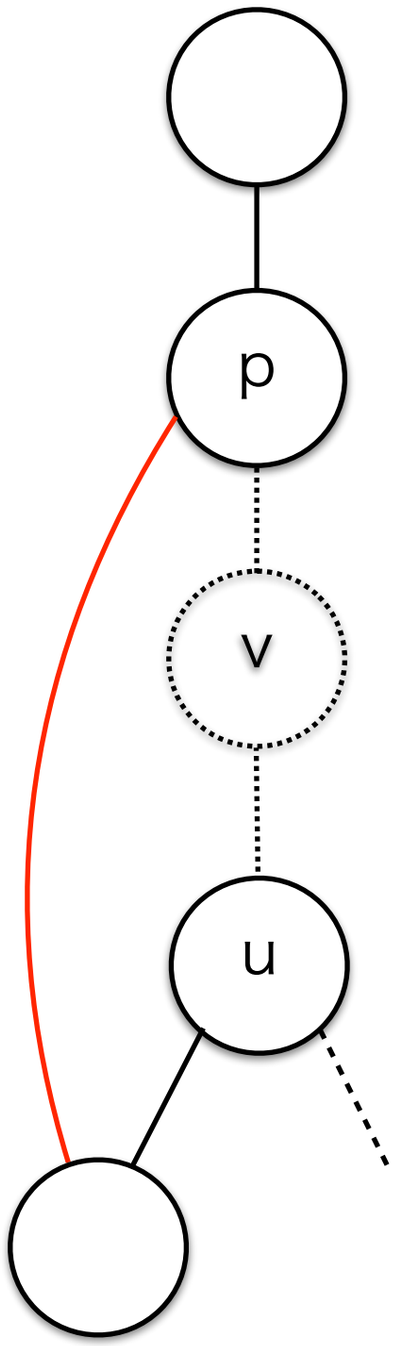


どういう後退辺があったら分離しないか？

uの部分木のどこかと、
pとpの親のうちどこかをつなぐような
後退辺

uから木を降りて、
後退辺を1回登ってpかpの親に行ければ
よい

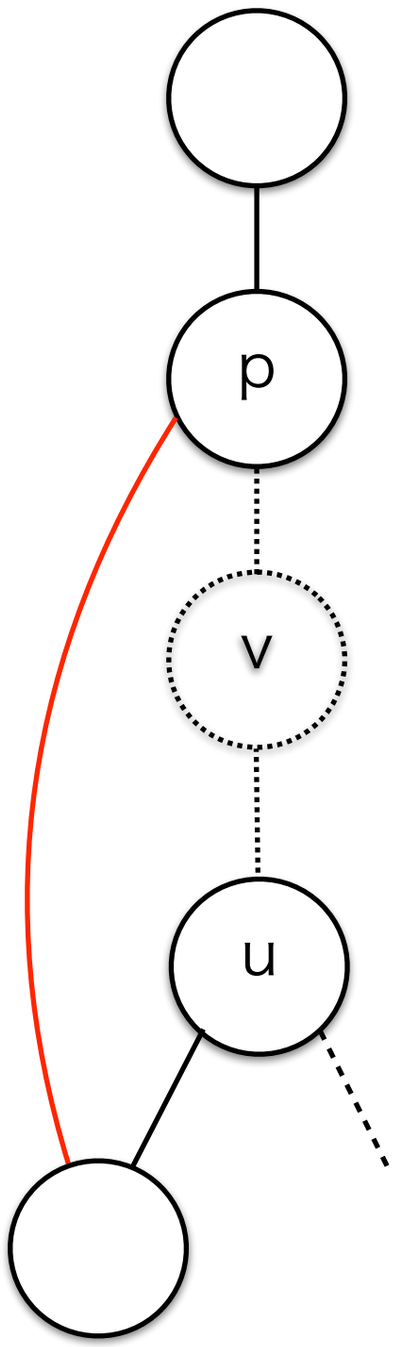
関節点列挙



uから木を降りて、
後退辺を1回登ってpかpの親に行ければよい

uから木を降り後退辺を1回登っていける頂点
このなかでもっとも上のものは？

関節点列挙

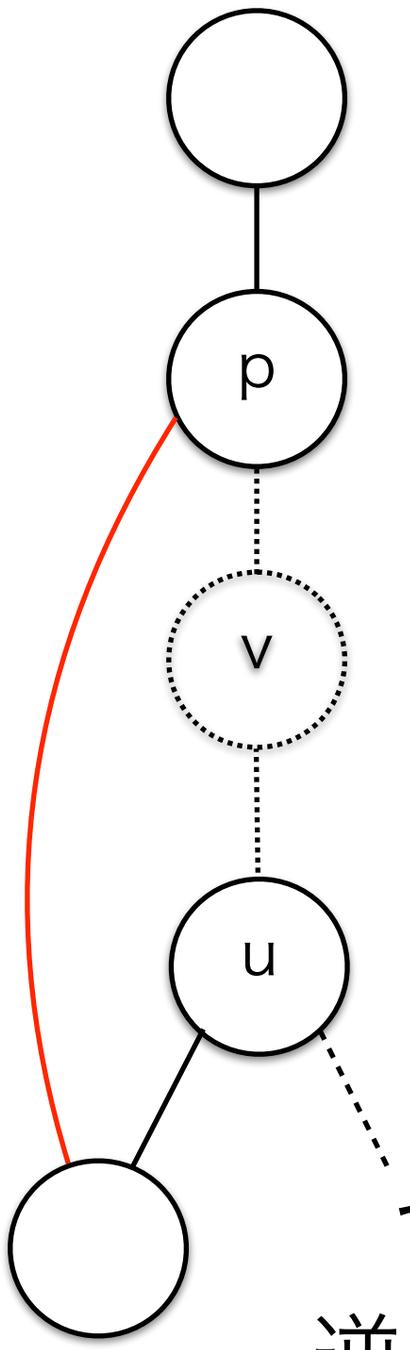


uから木を降りて、
後退辺を1回登ってpかpの親に行ければよい

uから木を降り後退辺を1回登っていける頂点
このなかでもっとも上のものは？

→ low(u)!

関節点列挙



uから木を降りて、
後退辺を1回登ってpかpの親に行ければよい

uから木を降り後退辺を1回登っていける頂点
このなかでもっとも上のものは？

→ low(u)!

つまり、low(u) < ord(v)か判定すれば良い
逆にlow(u) ≧ ord(v)なる子があったら関節点

関節点列挙

- ordとlowは全頂点まとめてO(V+E)で計算可能
- この条件下で関節点とは以下の条件を満たす頂点
  - (根)次数が2以上
  - (根以外, vとする)子uの中にlow[u] ≧ ord[v]なる頂点が存在する
- これらもO(V+E)で計算可能
- さらに、これらを満たす子uのなかで(uの部分木のサイズ)が奇数になるものがあったらNG
- まとめてO(V+E)

lowを求める方法

- 具体的にlowはどう求めればいいのか？
- 普通にdfsをする
  - 今頂点uにいて、辺(u, v)に注目してるとする
    - vをもう通ったならlow(u)=min(low(u), ord(v))
    - 通って無いならlow(u)=min(low(u), low(v))
  - これだけで求められる

ARC044 解説

DEGwer

A問題「素数判定」

問題概要

- 整数N(1≦N≦10^9)が素数っぽいかどうか判定せよ
- Nは以下のいずれかの条件を満たす時素数っぽい
  - 素数である
  - 合成数であり、かつ1の位が2でも5でも割り切れず、各桁の和が3で割り切れない

解法1

- 愚直に計算
- 素数判定にO($\sqrt{N}$)
- 各桁の和を求めるのにO(log N)
- あわせてO($\sqrt{N}$)

解法2

- 各桁の和とかは求めなくていい
  - Nの1の位が2でも5でも割り切れず、各桁の和が3の倍数であることは、Nが2,3,5で割り切れないことと同値
- 実は素数判定も要らない
  - 2,3,5以外のNについて、Nが素数ならNは2,3,5で割り切れない

解法2

- 以上をまとめると
  - N=1ならNot Prime(1は素数でも合成数でもない)
  - それ以外でN=2,3,5ならPrime
  - それ以外でNが2,3,5で割り切れるならNot Prime
  - それ以外ならPrime
- と出力すればよい
- O(1)で解けた

B問題「最短路問題」

問題概要

- 整数Nと整数列$A_1, A_2, \ldots, A_N$が与えられる
- 頂点数Nですべての辺のコストが1の単純無向グラフであって、任意のiに対して頂点1から頂点iまでの距離がA_iであるようなものの個数をmod 10^9+7で求めよ
- N ≦ 10^5

解法

- まず、自明に答えが0になる場合を取り除く
  - $A_1 \neq 0$だとだめ(頂点1から頂点1への距離は0)
  - 0が複数あってもだめ(頂点1から頂点1以外への距離は正)

解法

- 2点間に辺を張れる条件は?
  - その2点の、A_1からの距離の差が1以下
  - 距離の差が2以上あると、その辺を使うことで最短距離の性質に矛盾する

- 2点間の距離が0の場合と1の場合に分けて考える

解法

- 2点間の距離が0のとき辺はどう張ってもいい
  - 辺があろうがなかろうが最短距離は変わらない
- $A_i = C$なるiがK_i個あるとき、距離Cの頂点の間に辺を張る場合の数は$2^{\frac{K_i(K_i-1)}{2}}$通り
- これは、N乗までの2冪をループで順にもとめていき、さらにそれらの掛け算で2の三角数乗を順に求めていけばあわせてO(N)で求まる
  - 二分累乗ならO(Nlog N)

解法

- 2点間の距離が1のとき、そのA_1からの距離をそれぞれi, i+1とし、A_1からの距離がiであるような頂点の数をt, i+1であるような頂点の数をsとする
- 距離i+1の頂点たちからは、1本以上の辺が距離iの頂点のいずれかに張られていなければならない
  - 逆に、それ以外はどう張ってもよい

解法

- このような場合の数は$(2^t - 1)^s$通り
- これも2冪のテーブルをO(N)で前計算しておけば、合計O(N)で求まる
  - 頂点数の和はNよりこの操作にはO(s)かけられる
  - おなじく二分累乗ならO(Nlog N)
- ということでこの問題が解けた

C問題「ビーム」

問題概要

- W*Hのグリッドに高橋君がいる
- 1列全体、もしくは1行全体にQ回ビームが飛んでくる
- 高橋君は任意時刻にすきなだけ縦横に移動できる
- 全部よけるとき、高橋君の縦横への移動回数の最小値は?
- W,H,Q,ビームが飛んでくる時刻 ≦ 10^5

部分点解法(30点)

- W,H,Q≦ 100
- 1 ≦ ビームが飛んでくる時刻(この最大値をTとする) ≦ 100

- DP[i][j][k]: 時刻iに、マス(j,k)にいるときの最小の移動回数 として、DP配列を更新していく
- 各iごとにdijkstra法などを用いてDP配列を順次更新していくと、O(WHT log WH)

満点解法

- x座標、y座標は独立に考えられる(重要アイデア)
  - ビームは一列、もしくは一行すべてに飛んできて、高橋君の移動距離の定義もマンハッタン距離
- x座標、y座標独立に問題を解いて、あとで足し合わせればいい
- 以下x座標についてのみ考える

満点解法

- DP[i][j]: 時刻iに座標jにいるときの移動距離の最小値 とする
- これを愚直に計算するとO((W+H)T)
- よく考えると、DP[i][j]たちとDP[i+1][j]たちは「あまりかわらない」
  - 具体的には、異なる箇所は時刻i+1に飛んでくるビームの本数分のみ

満点解法

- 結局、DP配列を使いまわしてこれらが切り替わる点のみ更新すれば、O(W+H+T)でこの問題が解ける

D問題「suffix array」

問題概要

- 10^6項以下のpermutationが与えられる
- このpermutationをsuffix arrayに持つ辞書順最小の文字列を求めよ

解法

- 順列$A_1, A_2, \ldots, A_N$が文字列$S_1 S_2 \ldots S_N$のsuffix arrayである
- ⇔すべての1≦i≦N-1に対し、文字列 $S_{A_i} S_{A_i+1} \ldots S_N < S_{A_{i+1}} S_{A_{i+1}+1} \ldots S_N$
- ⇔すべての1≦i≦N-1に対し、$S_{A_i} < S_{A_{i+1}}$または、$S_{A_i} = S_{A_{i+1}}$かつ$S_{A_i+1} \ldots S_N < S_{A_{i+1}+1} \ldots S_N$

解法

- すべての1≦i≦N-1に対し、$S_{A_i} < S_{A_{i+1}}$または、$S_{A_i} = S_{A_{i+1}}$かつ
  $S_{A_i+1} \cdots S_N < S_{A_{i+1}+1} \cdots S_N$
- ⇔すべての1≦i≦N-1に対し、
  - $S_{A_i+1} \cdots S_N < S_{A_{i+1}+1} \cdots S_N$ならば$S_{A_i} \leqq S_{A_{i+1}}$
  - そうでないなら、$S_{A_i} < S_{A_{i+1}}$

解法

- $S_{A_i+1} \ldots S_N < S_{A_{i+1}+1} \ldots S_N$かどうかは、suffix arrayが持っている情報
- ということは、各iについて、S_{A_i}と$S_{A_{i+1}}$の満たすべき関係がわかる
- 前から順に見ていけば、文字列の各文字が最も小さくて何であるかがすべてわかる

解法

- 逆に、suffix arrayを前から順に見ていき、このような方法で各文字を決めていけば、そうしてできた文字列のsuffix arrayは元の順列に一致することは簡単に示せる
- よって、前から貪欲に文字を決めていけば、O(N)でこの問題が解けた

ARC043解説

A:平均点

A-問題概要

- N個の得点 S[0], S[1], … が与えられる
- 平均が A, 最大と最小の差がBとなるように線形変換せよ
  - 線形変換とはある実数P,QにたいしてXをPX+Qにする変換

A-考察

- すべてをP倍すると
  - 平均　P倍される
  - 最大と最小の差　P倍される

- すべてにQ足すと
  - 平均　Q足される
  - 最大と最小の差　変わらない

A-解法

- 最大と最小の差がBになるように適切なPを探す
  - その後すべてをP倍する
- 平均がAになるように適切なQを探す
  - その後すべてにQを足す
- すべての得点が適切に変換できている
  - O(N)

A-具体的数式

- 変換前の得点の平均をE,最大と最小の差をDとすると
- $P = B / D$
- $Q = A - (EP) = A - (EB/D)$

A-注意点

- 変換前の得点の平均をE,最大と最小の差をDとすると
- $P = B / D$
- $Q = A - (EP) = A - (EB/D)$

A-割り算には要注意

- 変換前の得点の平均をE,最大と最小の差をDとすると
- P = B / D
- Q = A – (EP) = A – (EB/D)

- D = 0のとき、つまり全得点が同じ時、適切なPが存在しない
  - コーナーケース

ARC043解説
B:難易度

B-問題概要

- 要素数 N の正数列が与えられる
- 次の条件をみたすように 4 つ要素を選ぶ方法の通り数を求めよ
  - 2番目に小さい値は1番目に小さい値の 2 倍以上
  - 3番目に小さい値は2番目に小さい値の 2 倍以上
  - 4番目に小さい値は3番目に小さい値の 2 倍以上
- MOD 1,000,000,0007
- N ≦ 10^5

B-考察

- 4つではなくて2つの場合はどうか？
  - 2つのうち小さい方が X である場合の数 A[x] を求めたい
  - 大きい方となれるのは 2X 以上の値
  - よって A[x] は2X以上の値の数の個数
  - 予めソートしていれば容易に求めることが出来る
    - 二分探索 (最悪O(logN))
    - しゃくとり法　(平均O(1))

B-考察

- ４つではなくて2つの場合はどうか？
  - 2つのうち大きい方が X である場合の数 B[x] を求めたい
  - A[x]と全く同様の方法で求めることが出来る
    - 二分探索 (最悪O(logN))
    - しゃくとり法　(平均O(1))

B-部分点1

- 2番目に小さい値 x と3番目に小さい値 y を全通り試す
  - この2つが固定されている時、選び方の通り数は

    2x ≦ y ならば　　B[x] × A[y]

    そうでないならば　　0
  - xとyの組は全部でO(N^2)ある
    - 計算量は O(N^2)
    - 部分点 (50点) 獲得

B-考察

- 部分点解法は何か無駄がありそう
  - 以下のように言い換えることが出来る
- 2番目に小さい値 xを固定する
  - $2x \leqq y$となるすべてのyに対して

    $B[x] \times A[y]$

    の総和をとる
  - $2x \leqq y$となるすべてのyに対するA[y]の総和をAsum[2x]とすると

    $B[x] \times Asum[2x]$

B-考察

- Asum[2x]が高速で求まればO(N)で答えを求めることが出来る
- Asumはsumの累積和なので、O(N)で全要素を前計算することが出来る
  - Asum[max(y)] = sum[max(y)]
  - Asum[x] = Asum[x + 1] + Asum[x]

B-満点解法

- x ≦ y を満たすyに対する A[y]の総和 Asum[y]を累積和で前計算する
- 2番目に小さい値 x を固定した時の場合の数は

  B[x] × Asum[y]

- すべての x に対して計算して総和を取る

ARC043解説

C:転倒距離

C-問題概要

- [1,2,..,N]を並び替えたものを「順列」という
- 2つの順列の「転倒距離」とは2つで順序が入れ替わっている数字の組の個数である
- 2つの順列A,Bが与えられるので、どちらとも転倒距離が等しい順列を1つ挙げよ

- N ≦ 10^5

C-考察1

- A,Bの転倒距離をT(A,B)と書くとする
- T(A,C) = T(B,C)となるCがあるならばT(A,B)は偶数
- 数字の組について考えると
  - A,CでもB,Cでも逆
  - A,Cで逆だがB,Cでは同じ
  - A,Cで同じだがB,Cでは逆
  - A,CでもB,Cでも同じ
- それぞれx1, x2, x3, x4組ずつあったとする

C-考察1

- 逆逆:x1組　逆同x2組　同逆 x3組 同同 x4組
- このとき以下が成り立つ
  - T(A, C) = x1 + x2
  - T(B, C) = x1 + x3
  - T(A, B) = x2 + x3
- よって T(A, B) = T(A, C) + T(B, C) − 2*x1
- T(A,C) = T(B,C)なのでT(A, B)は偶数
  - (Q.E.D)

C-考察1

- T(A,C)=T(B,C)なるCがあるならばT(A,B)は偶数
  対偶を取ると
- T(A,B)が奇数ならばT(A,C)＝T(B,C)なるCはない
  - 今回の問題で言う-1を出力するパターン

C-考察2

- 数字の大小は転倒距離とは関係ないので、Aが[1,2,…,N]となるように数字を置換しても構わない
- 例
  - A:[3, 1, 4, 2, 5], B:[2, 5, 1, 3, 4]
  - 3→1, 1→2, 4→2, 2→3, 5→5というふうに変換
  - A:[1, 2, 3, 4, 5], B[3, 5, 2, 1, 2]

C-考察2

- Aが[1, 2, … ,N]になったのでBとの転倒距離が計算しやすくなった
  - 各数字について、それ以左にあるそれ以上の数字の個数を数えれば良い
- これはBの転倒数
  - バブルソートにおける交換回数と一致することで有名
  - つまりBをバブルソートしていくと少しずつAとの転倒距離が縮み、元のBとの転倒距離が伸びる

C-部分点

- あらかじめA,Bの転倒距離を求める
  - Tとする
  - Tが奇数なら-1を返す (∵ 考察1)
- A,Bとの転倒距離がおなじになるまでBをバブルソートしていく
  - T/2回交換したら停止して出力
- バブルソートの計算量はO(N^2)
  - N≦3000なので間に合う　部分点獲得

C-満点解法

- バブルソートにおける「隣り合う2つの要素の交換」をいちいちやらずに、いくつかまとめて処理する
- バブルソートをダイジェストで見ると以下のようになる
  - [6, 1, 4, 2, 3, 5]
  - [1, 6, 4, 2, 3, 5]
  - [1, 2, 6, 4, 3, 5]
  - [1, 2, 3, 6, 4, 5]
  - [1, 2, 3, 4, 6, 5]
  - [1, 2, 3, 4, 5, 6]

↓これらの値がわかっていれば交換の操作をスキップできる

1回交換

2回交換

2回交換

1回交換

1回交換

実はxを正しい位置まで動かすときの交換回数は、初めの数列におけるx以左のxより大きい値の個数と一致する

C-考察

- ある数列における x 以左のxより大きい値の個数を高速に求める有名なデータ構造があった
  - BITやセグメントツリー
- 上記データ構造で以左に現れたかどうかのフラグを管理しながら左から順番に走査する
- [3, 5, 4, 2, 6, 1]を例に実際に走査してみる

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	0	0	0	0

x以左に現れたxより大きい値の個数

1	2	3	4	5	6

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	0	0	0	0

総和 = 0

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
		0			

確定

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	1	0	0	0

更新

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
		0			

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	1	0	0	0

総和 = 0

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
		0		0	

確定

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	1	0	1	0

更新

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
		0		0	

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	1	0	1	0

総和 = 1

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
		0	1	0	

確定

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	1	1	1	0

更新

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
		0	1	0	

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	0	1	1	1	0

総和 = 3

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
	3	0	1	0	

確定

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	1	1	1	1	0

更新

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
	3	0	1	0	

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	1	1	1	1	0

総和 = 0

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
	3	0	1	0	0

確定

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	1	1	1	1	1

更新

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
	3	0	1	0	0

C-考察

- [3, 5, 4, 2, 6, 1]
  ↑

以左に現れたかどうかのフラグ

1	2	3	4	5	6
0	1	1	1	1	1

総和 = 5

x以左に現れたxより大きい値の個数

1	2	3	4	5	6
5	3	0	1	0	0

確定

C-満点解法

- バブルソートにおける「隣り合う2つの要素の交換」をいちいちやらずに、いくつかまとめて処理する
- T/2回を超えないようにスキップしながらバブルソートをシミュレート
  - スキップするための値は先述のデータ構造が実装
- スキップできなくなったら、交換回数がT/2に達するまで普通にバブルソートをする
  - スキップがO(N)回の交換のまとめなので、たかだかO(N)回で終了する
- 合計でO(NlogN)　満点獲得

ARC043解説

D:引っ越し

D-問題概要

- 数直線上で原点からの距離が1以上N以下の整数である場所に空き家がある
- M世帯の家族を1つの家に複数の世帯が割り当てられないように振り分ける
  - i番目の家族がA[i]に割り振られたとする
- i世帯目の家族の構成人数をP[i]とする
- (1 ≦ i, j ≦ M)を満たすすべての対(i, j)に対する|A[i] − A[j]| * P[i] * P[j]の総和が最大になるように割り振った時のその値を求めよ

D-部分点解法

- 全ての振り分けを試せば良い
- N≦10なのでN!通りの全探索が可能
- 各振り分けの住民の距離の値はO(N^2)で求めることが出来る
- O(N!×N^2)　10点獲得

D-考察

- 式が全家族対についての総和なので計算がしづらい
  - 式変形をして扱いやすくする

D-式変形

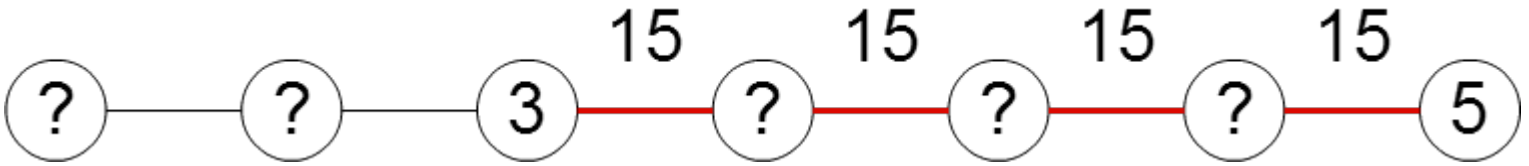
- |A[i] - A[j]| * P[i] * P[j]
- 空き家達を以下のように捉える

D-式変形

- |A[i] - A[j]| * P[i] * P[j]
- 空き家達を以下のように捉える
  - A[i]とA[j]の間には|A[i] – A[j]|個の「辺」がある

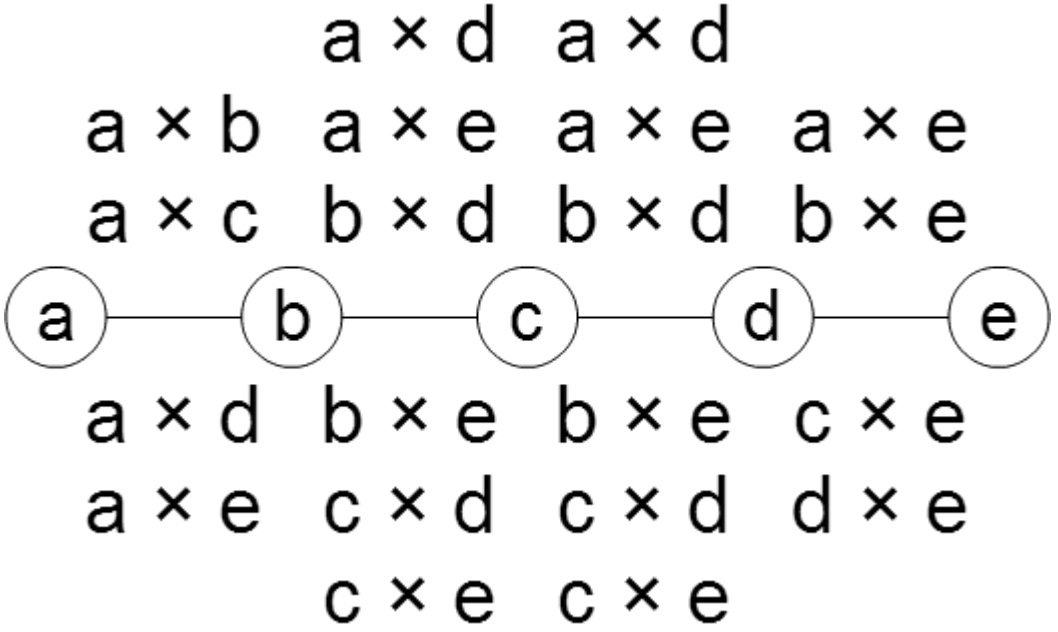
D-式変形

- |A[i] - A[j]| * P[i] * P[j]
- 空き家達を以下のように捉える
  - A[i]とA[j]の間には|A[i] – A[j]|個の「辺」がある
  - それぞれにP[i] * P[j] を足すと考えてみる



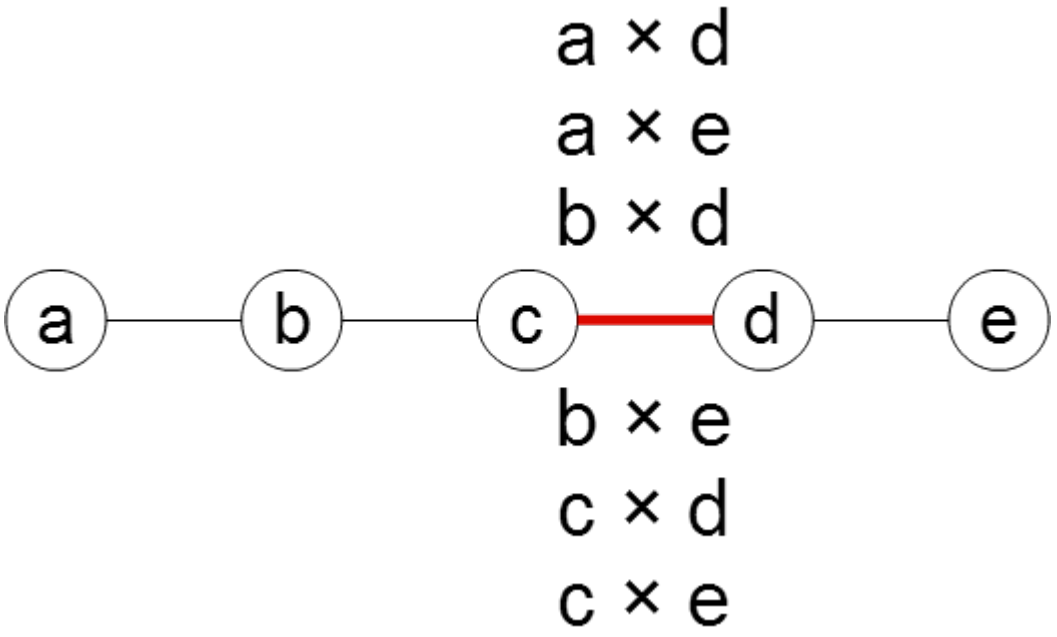
D-式変形

- 全家族対について、その間のすべての辺に家族の人数の積を足す
  - 最後に全部の辺の総和をとれば求めたい値になる



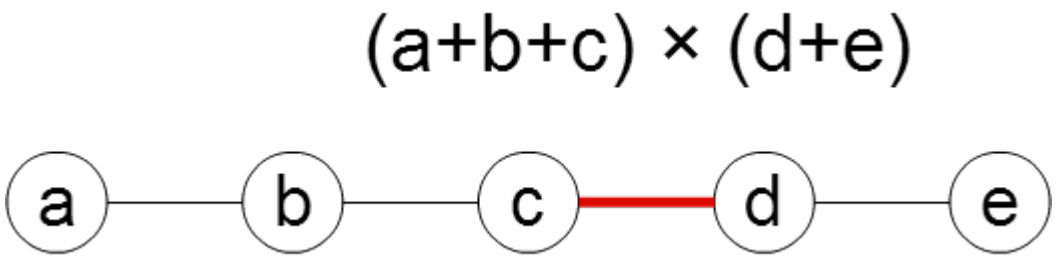
D-式変形

- 一つの辺に注目してみる
  - それ以左にある全ての家族と、それ以右にあるすべての家族について1家族ずつ取ってきて対を作って積を取ったものの総和である



D-式変形

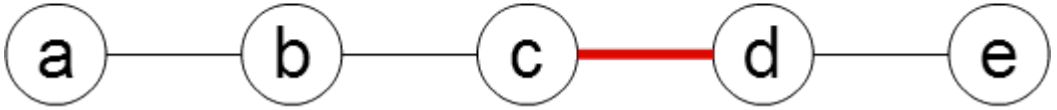
- 一つの辺に注目してみる
  - これは「以左の総和」と「以右の総和」の積と一致する



D-式変形

- 各「辺」について以左の人数の総和と以右の人数の総和がわかっていれば、「辺」に足される値が簡単に計算できる
- 「以左の総和」＋「以右の総和」＝ 全人数
  ということにも注意

$$(a+b+c) \times (d+e)$$



D-式変形　整理

- 各辺について、以左の総和と以右の総和の積をとる
- さらにその総和をとれば求めるもの

- i番目の家以左にいる人の総和を sum[i]、すべての家族を合わせた人数をAllとすると求める値は
  - Σ sum[i] * (All - sum[i])

D-考察

- 空き家の扱いがすこし面倒
  - 家族に振り分けられる空き家と、そうでない空き家がある
- 0人家族が沢山存在するとみなせば、すべての家に1家族ずつ振り分けることが出来る
  - 0人家族はいくら追加しても結果にかかわらない
  - 並び替えの問題として扱うことが出来る

D-帰着される問題

- 正整数列A(1-index)が与えられる。総和はSumである。
- S[i] = A[1] + A[2] + … + A[i] とする
- Aを並び替えたときの以下の値の最大値を求めよ

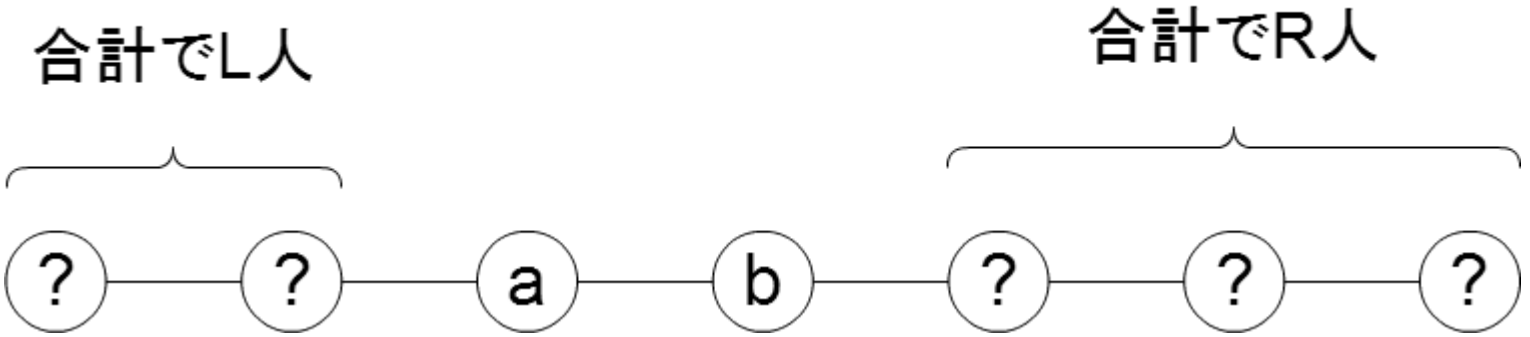
$$\Sigma \ S[i] * (Sum - S[i])$$

D-定石

- 最適な並び替えを求める問題の定石
  - 隣り合う二つを交換した時どうなるか考える

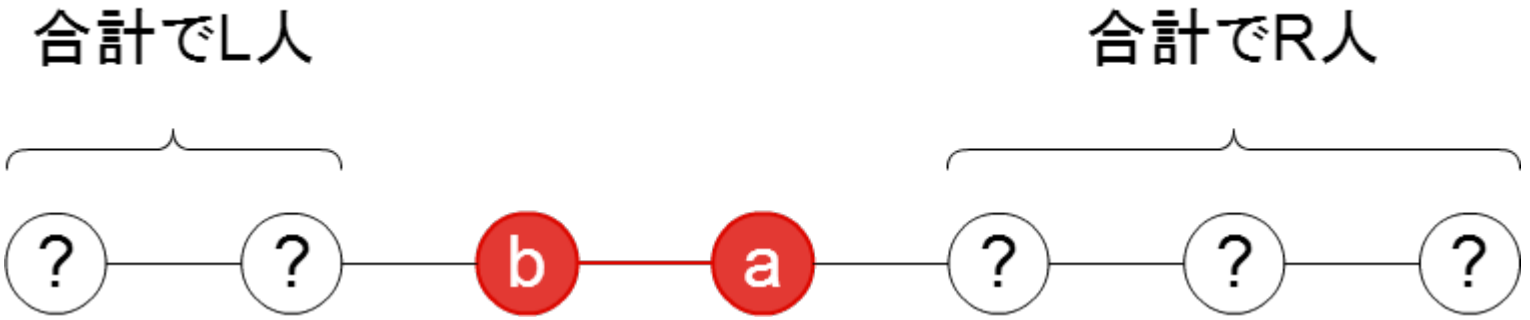
D-考察

- 隣り合う二つの家族を交換したときどうなるか？
- 以下のようにa人家族,b人家族が並んでいる時aとbを入れ替えるとどうなるか？



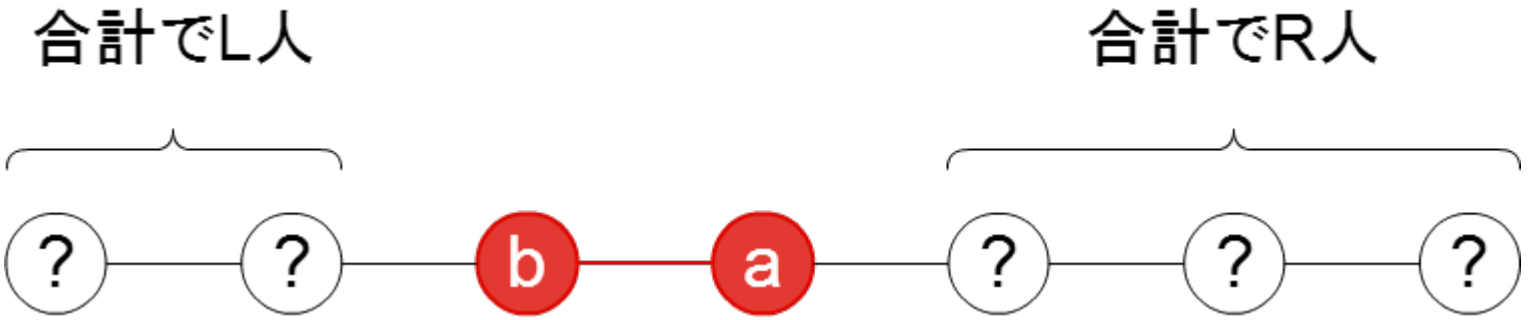
D-考察

- 隣り合う二つの家族を交換したときどうなるか？
- 以下のようにa人家族,b人家族が並んでいる時aとbを入れ替えるとどうなるか？



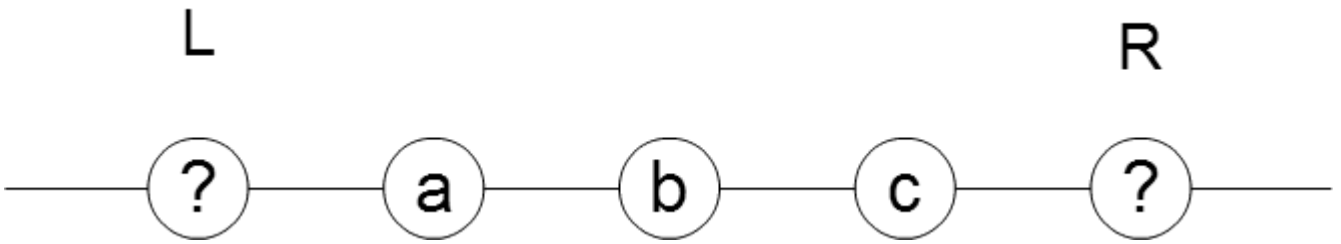
D-考察

- aとbの間の辺のみ考えれば良い
- (L+a) × (b+R) が (L+b) × (a+R)
- 整理すると(R - L) × (b – a)だけ増える事がわかる



D-考察

- 以下のように a, b, cが並んでいたとする
  - さらに a < b, b > c を仮定する
- a,bを交換すると((R-L) +c)× (b - a)増える
- b,cを交換すると((L-R) +a) ×(b - c)増える



D-考察

- 以下のように a, b, cが並んでいたとする
  - さらに a < b, b > c を仮定する
- a,bを交換すると((R-L) +c)× (b - a)増える
- b,cを交換すると((L-R) +a) ×(b - c)増える
- 以下に注意
  - a, c, b-a, b-cはすべて正である
  - (R-L)もしくは(L-R)の少なくとも一方が負ではない
- どちらかの交換は結果が悪化しない

D-考察

- a < b > c が成立している時、bはどちらか一方と交換することが出来る
  - a < b > c となる部分を含まないような最適解が存在する
- A[1] ≧ A[2] ≧ … ≧ A[i] ≦ A[i+1] ≦ … A[n] となるような最適解が存在する

D-考察

- A[1] ≧ A[2] ≧ … ≧ A[i] ≦ A[i+1] ≦ … A[n]
- このパターンの並び替えを探索する一般的なテクニックがある
- DP[i][x] = Aの中で大きい方からi番目までを左右に振り分けて、左側の総和がxのときの何らかの値の最大(小)値
- Aの中で大きい物から順番に左もしくは右に振り分けていくイメージ

D-考察

- A[1] ≧ A[2] ≧ … ≧ A[i] ≦ A[i+1] ≦ … A[n]
- このパターンの並び替えを探索する一般的なテクニックがある
- DP[i][x] = Aの中で大きい方からi番目までを左右に振り分けて、左側の総和がxのときの何らかの値の最大(小)値
- Aの中で大きい物から順番に左もしくは右に振り分けていくイメージ

D-考察

- A[1] ≧ A[2] ≧ … ≧ A[i] ≦ A[i+1] ≦ … A[n]
- このパターンの並び替えを探索する一般的なテクニックがある
- DP[i][x] = Aの中で大きい方からi番目までを左右に振り分けて、左側の総和がxのときの何らかの値の最大(小)値
- Aの中で大きい物から順番に左もしくは右に振り分けていくイメージ

どんな値？

D-帰着される問題（再掲）

- 正整数列A(1-index)が与えられる。総和はSumである。
- S[i] = A[1] + A[2] + … + A[i] とする
- Aを並び替えたときの以下の値の最大値を求めよ

$$\Sigma \ S[i] * (Sum - S[i])$$

D-帰着される問題（再掲）

- 正整数列A(1-index)が与えられる。総和はSumである。
- S[i] = A[1] + A[2] + … + A[i] とする
- Aを並び替えたときの以下の値の最大値を求めよ

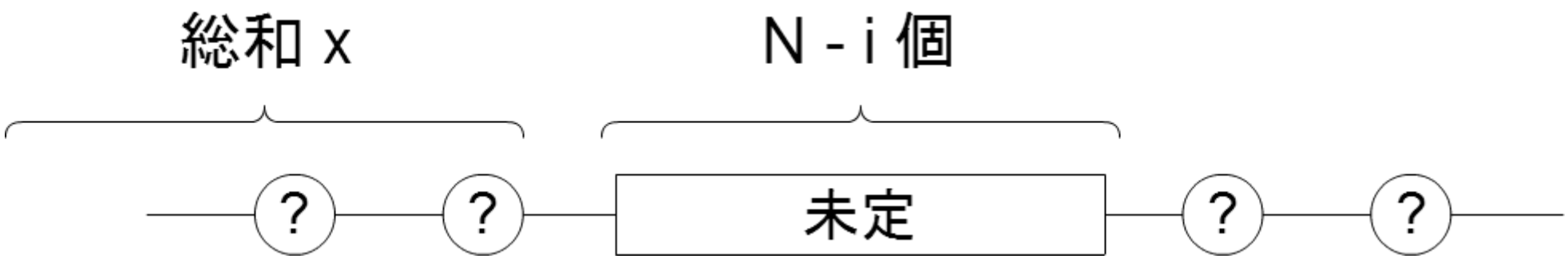
  Σ S[i] * (Sum - S[i])

D-満点解法

- DP[i][x] = Aの大きい方からi番目までを左右に振り分けて左側の総和がxで在るときのもう定まってる S[i] * (Sum - S[i])の総和の最大値

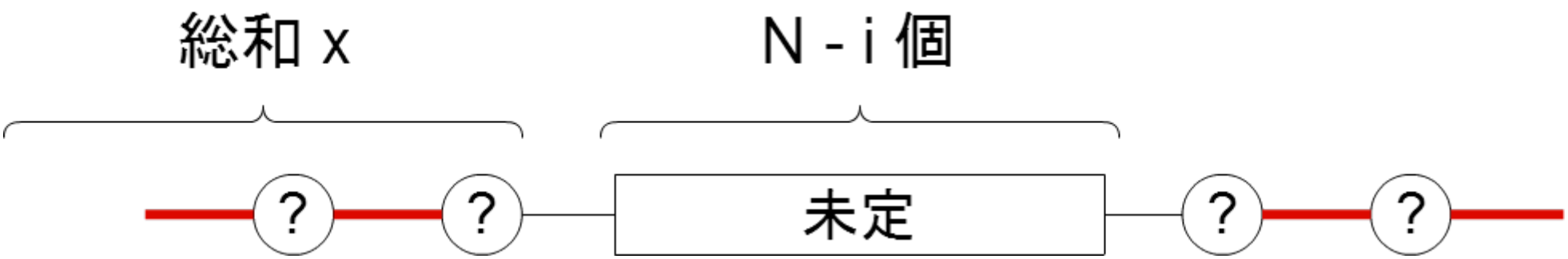
D-満点解法

- DP[i][x] = Aの大きい方からi番目までを左右に振り分けて左側の総和がxで在るときのもう定まってる S[i] * (Sum - S[i])の総和の最大値
  - DP[i][x]は以下の様な全ての状態の中で「決まっているS[i] * (Sum – S[i])の総和」の最大値



D-満点解法

- DP[i][x] = Aの大きい方からi番目までを左右に振り分けて左側の総和がxで在るときのもう定まってる S[i] * (Sum - S[i])の総和の最大値
  - DP[i][x]は以下の様な全ての状態の中で「決まっているS[i] * (Sum - S[i])の総和」の最大値



D-満点解法

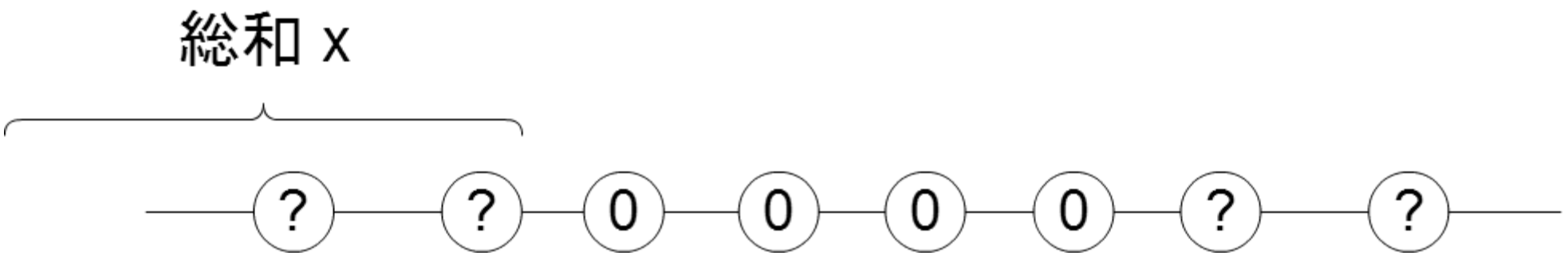
- DP[N][0], DP[N][1], … ,DP[N][Sum] の中で最大の物を探せば良い？
  - 残念ながらNのオーダーは10^6なので間に合わない

D-満点解法

- DP[N][0], DP[N][1], … ,DP[N][Sum] の中で最大の物を探せば良い？
  - 残念ながらNのオーダーは10^6なので間に合わない
- 実はほとんどが0人家族である
  - A[1] ≧ A[2] ≧ … ≧ A[i] ≦ A[i+1] ≦ … A[n]
    この制約のために0人家族は真ん中のあたりに固まっている
  - これらだけ別処理すれば良い
  - NがMまで減る（Mのオーダーは10^3)

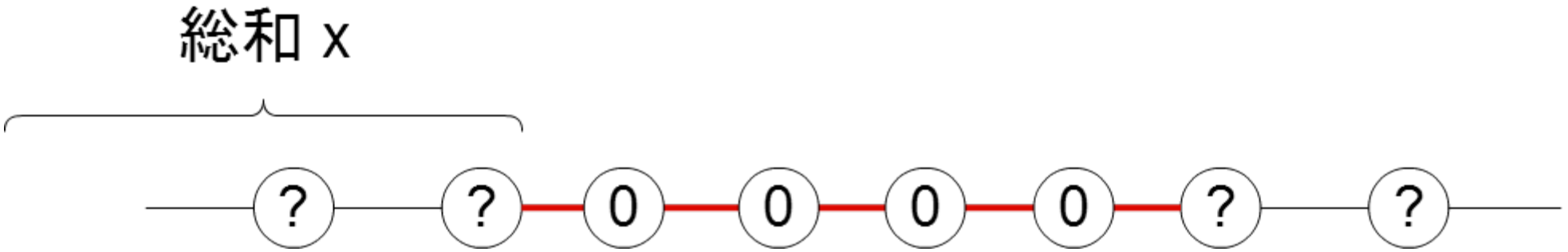
D-満点解法

- 0人家族の別処理



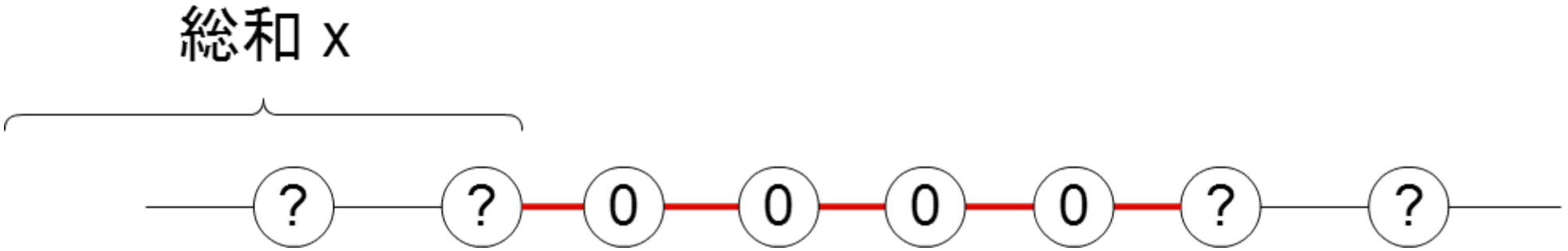
D-満点解法

- 0人家族の別処理
  - 以下のように0人家族のあたりの辺には全く同じ値が足される



D-満点解法

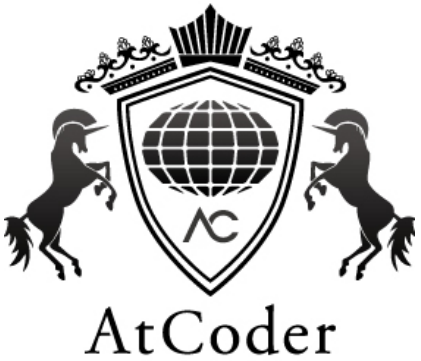
- 0人家族の別処理
  - 以下のように0人家族のあたりの辺には全く同じ値が足される
  - この値はxから求まるのでDP[m][x]ごとに別々に計算してやればDP[n][x]に相当するものが求まる



D-満点解法まとめ

- DP[i][x] = 大きいほうからi番目までで左の総和がxになるように置いた時の値が決まっている辺の総和の最大値
- DP[m][x]まで求めて、0人家族の辺は別処理
  - mのオーダーは10^3, xのオーダーは10^5なので配列を普通に確保するとMLEしてしまう
  - I に関しては昇順に操作し、更新時 i – 1しか参照しないのでメモリをリサイクルできる
  - 空間計算量はO(10^5)
  - 時間計算量はO(10^8)

AtCoder Regular Contest 042
解説



競技プログラミングを始める前に

AtCoder

- 競技プログラミングをやったことがない人へ
  - まずはこっちのスライドを見よう！
  - http://www.slideshare.net/chokudai/abc004

A問題　掲示板

1. 問題概要
2. 解法

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

- N個のスレッドがある掲示板がある
- 書き込みがあるとそのスレッドが一番上になる
- 書き込みがM個与えられるので、書き込み後のスレッドの順番を求めよ
- 制約

  $1 \leqq N \leqq 10^5, 1 \leqq M \leqq 10^5$
- 部分点

  $1 \leqq N \leqq 100, 1 \leqq M \leqq 100$

A問題　解法

AtCoder

- 部分点解法
  - 毎回の書き込みでスレッドの順番がどうなるかシミュレーションする
  - O(NM)
- 満点解法
  - それぞれのスレッドの最後の書き込みの順でソートする
  - 書き込みがなかったスレッドについてはスレッドの番号が小さい順にソートする
  - O(N log N + M)

B問題　アリの高橋くん

1. 問題概要
2. 解法

©AtCoder Inc. All rights reserved.

B問題　問題概要

AtCoder

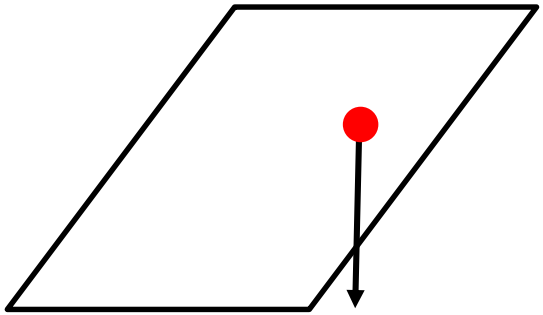
- 凸多角形とその内部の１点が与えられる
- 凸多角形の外周と点との最短距離を求めよ
- 制約
  - 3 ≦ (凸多角形の頂点数) ≦ 10
  - -100≦(座標)≦ 100

B問題　解法

AtCoder

- 各辺と高橋くんとの距離の最小値を求めればよい

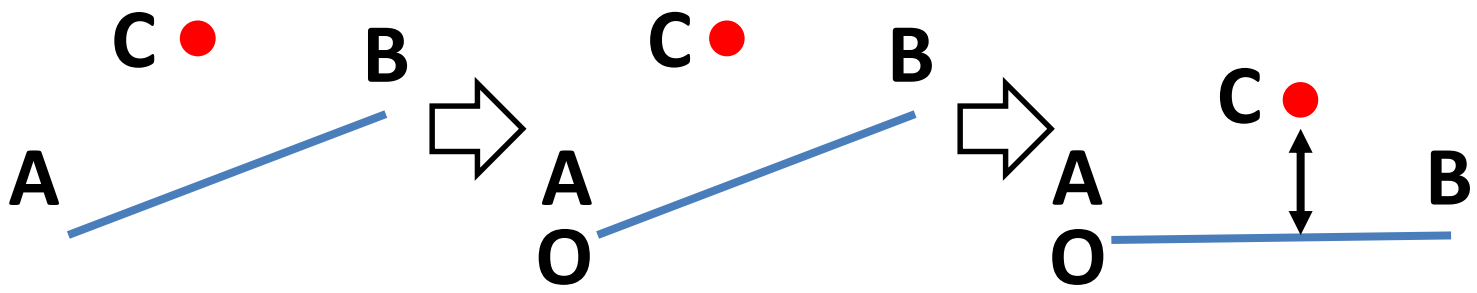
- 線分と点の距離の求め方
  - 線分上に垂線の足があるなら垂線の足との距離
  - そうでなければ端点との距離
  - 今回は垂線の足だけ考えればよい



B問題　解法

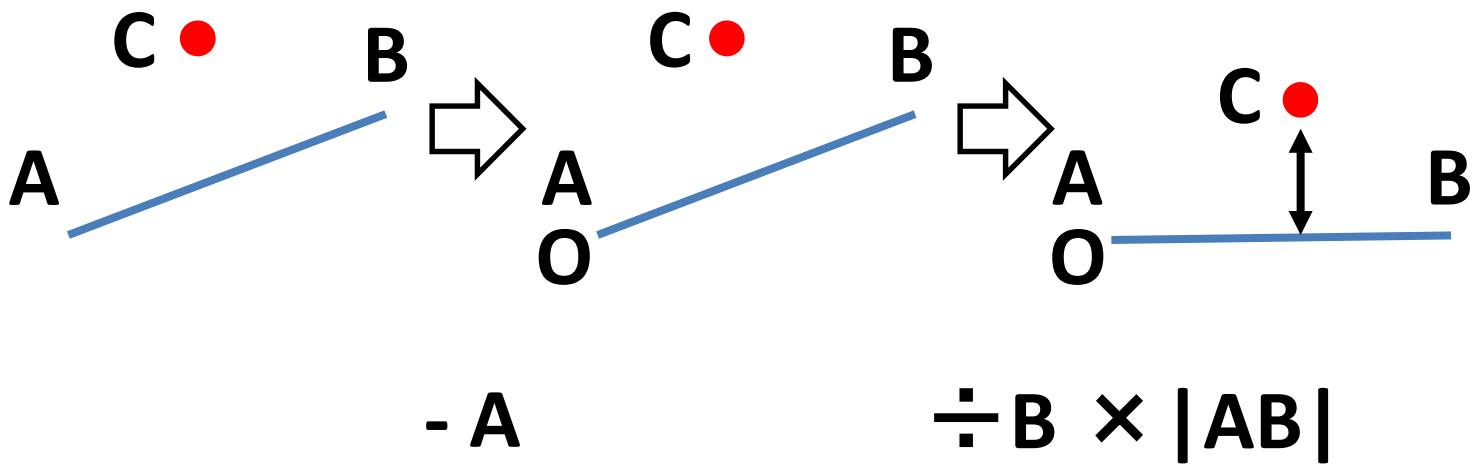
AtCoder

- 直線と点の距離の求め方
  - 直線上の1点が原点になるように平行移動
  - もう1点がX軸の正の方向になるように回転
  - 移動後の点のY座標の絶対値が答え



B問題　解法

- 複素平面を用いた方法
  - 平行移動　：　引き算
  - 回転移動　：　割り算して絶対値を掛ける



C問題　おやつ

1. 問題概要
2. 解法

©AtCoder Inc. All rights reserved.

C問題　問題概要

AtCoder

- 合計額の上限P,おやつの数N,値段a_i,満足度b_iの01ナップサック問題
- ただし、選んだおやつのうち、どの1つのおやつがなかったとしてもP以下になるならOK
- 満足度の和の最大値を求めよ
- 制約
  - $1 \leqq P \leqq 5{,}000$, $1 \leqq N \leqq 5{,}000$
  - $1 \leqq a_i \leqq 100$, $1 \leqq b_i \leqq 100$
- 部分点
  - $1 \leqq P \leqq 100$, $1 \leqq N \leqq 100$

C問題　解法

AtCoder

- 部分点解法
  - 選ぶおやつのなかで値段が一番安いものを決める
  - その値段以上のおやつでナップサック問題をDPで解く
  - 一番安いおやつの選び方がN通り、DPがO(PN)
  - 全体でO(PN^2)

- 満点解法
  - 値段が高い順にソート
  - 部分点解法と同じようにDPをする
  - DPテーブルを使いまわせるので毎回のDPがO(P)となる
  - 全体でO(N log N + PN)

D問題　あまり

1. 問題概要
2. 解法

©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

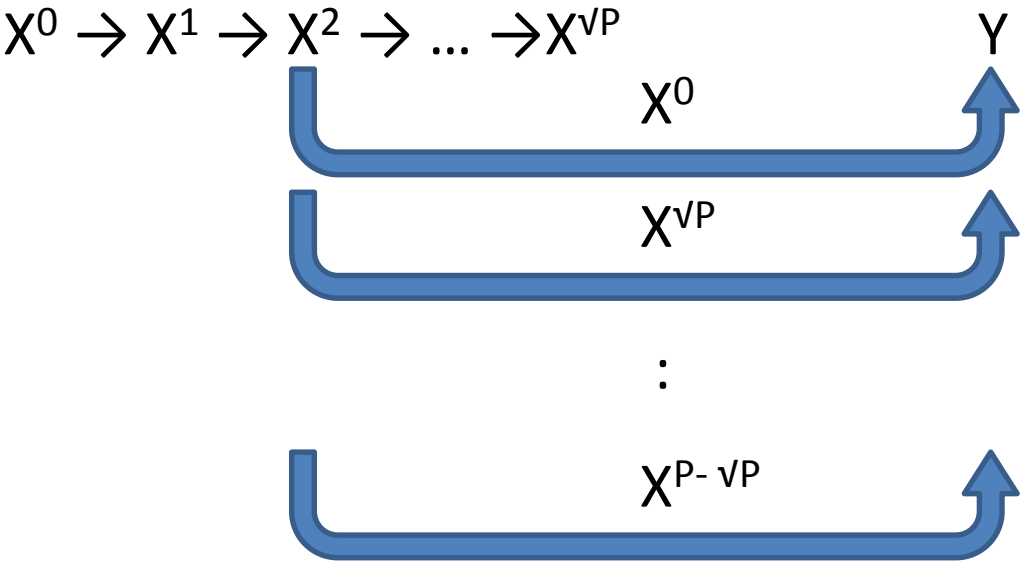
- $X^i (A \leqq i \leqq B)$ を P で割った余りの最小値を求めよ

- 制約
  - 1<=X<P<2^{31}, Pは素数
  - 0<=A <=B<2^{31}

D問題　解法

AtCoder

- 前提知識
  - $X^i = Y \pmod P$となるiを求める → 離散対数問題
- Baby-step giant-step algorithm
  - O(√P)で離散対数問題を解く



D問題　解法

AtCoder

- $X^i = 1, X^i = 2, X^i = 3, \ldots$ を順番に解けばいい？
  →答えが大きいとTLE

- 答えが大きくなるのは試すi (= B-A+1)が少ない場合
  ランダムなので悪質なケースは来ない
  B-A+1が小さいとき、答えは平均的にはP/(B-A+1)

- B-A+1が小さいときは愚直に計算すればいい
- Kで切り替えるとして、min(P/K * √P, K)
- $K=2^{24}$くらいで間に合う

ARC #041 解説

解説スライド担当：@sugim48

問題 A – コインの反転

問題概要

- 表向きのコインが x 枚、裏向きのコインが y 枚ある。ちょうど k 枚のコインを選び、それらすべてをひっくり返す。その結果、表向きのコインは最大で何枚になるか？

- $1 \leq x, y \leq 10^6,\ 1 \leq k \leq x + y$

解法 1

- ひっくり返した結果、表向きのコインをできるだけ多くしたい。
- まずは裏向きのコインからひっくり返していき、k に満たなければ表向きのコインもひっくり返していく。

- ちょうど $k = y$ ならば、すべてのコインを表向きにできる。$k \neq y$ ならば、ズレの分だけ裏向きのコインが残る。
- 答えは $x + y - |k - y|$

解法 2

- 今回は $x, y \leq 10^6$ が小さいので、全探索ができる。
- （表向きのコインをひっくり返す枚数）+（裏向きのコインをひっくり返す枚数）$= k$ である組を全通り試し、答えの最大値を求める。

問題 B – アメーバ

問題概要

- 縦 N マス、横 M マスの盤面があり、各マスにアメーバが何匹かずついる。ただし、壁際にはいない。
- 各アメーバが 4 匹に分裂し、上下左右のマスへ 1 匹ずつ移動した。
- 今の盤面が与えられるので、はじめの盤面を 1 つ求めよ。ただし、解は少なくとも 1 つ存在する。

- $3 \leq N, M \leq 500$

考察

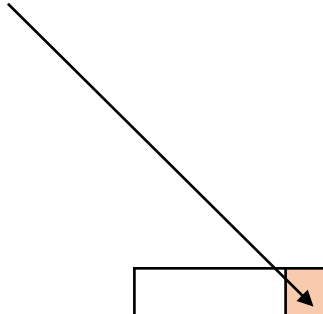
- 次の例を考える。

0	1	2	0
1	5	5	2
3	5	5	4
0	3	4	0

0	0	0	0
0	?	?	0
0	?	?	0
0	0	0	0

考察

- ここに注目すると、このアメーバは下のマスから来たと断定できる。

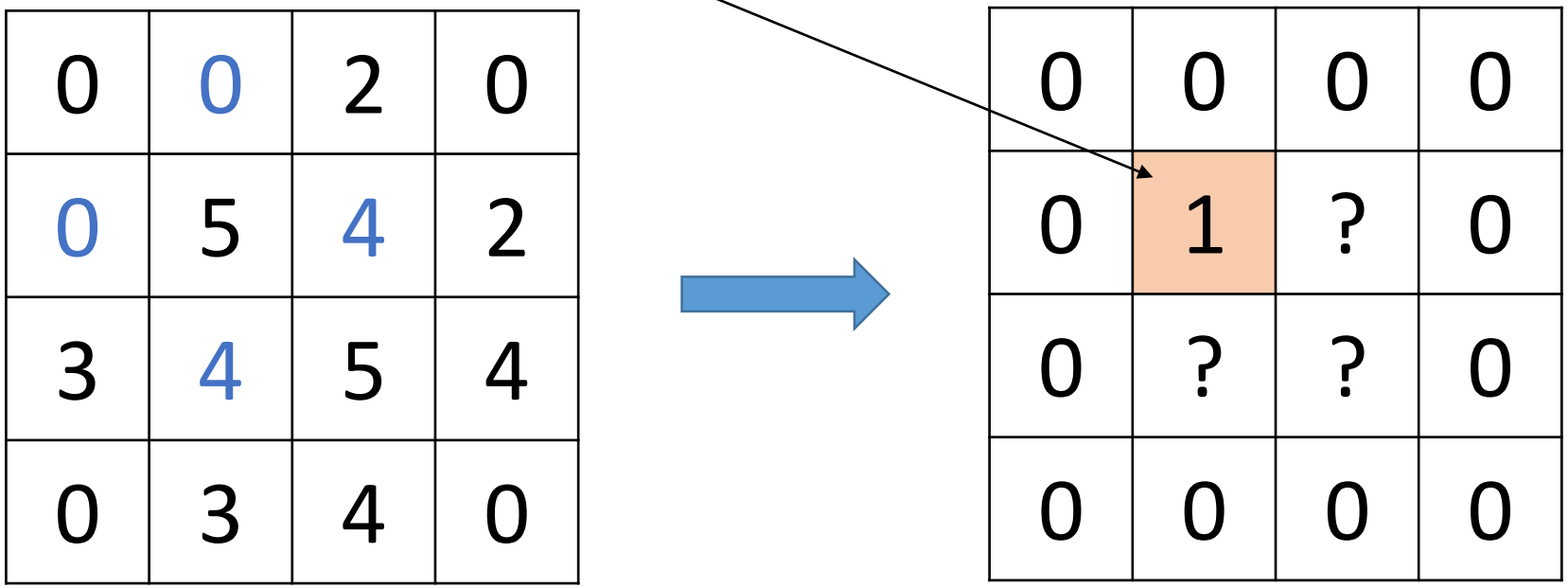


0	1	2	0
1	5	5	2
3	5	5	4
0	3	4	0

0	0	0	0
0	?	?	0
0	?	?	0
0	0	0	0

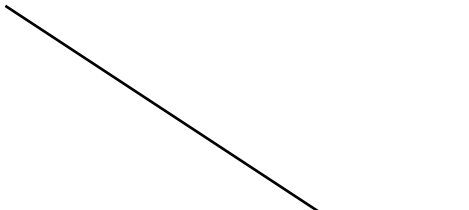
考察

- よって、ここのアメーバの数が決まる。確定したアメーバの分は左の盤面から引いておく。



考察

- ここに注目すると、このアメーバは下のマスから来たと断定できる。



0	0	2	0
0	5	4	2
3	4	5	4
0	3	4	0

0	0	0	0
0	1	?	0
0	?	?	0
0	0	0	0

考察

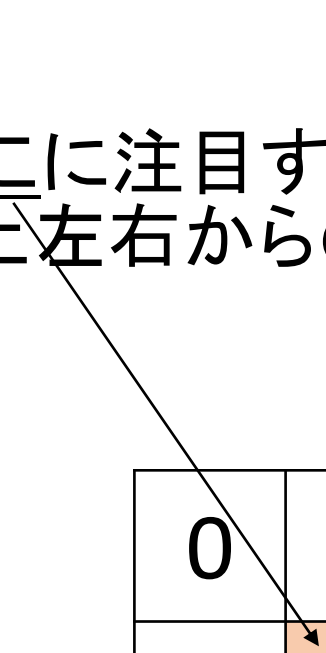
- よって、ここのアメーバの数が決まる。確定したアメーバの分は左の盤面から引いておく。

0	0	0	0
0	3	4	0
3	4	3	4
0	3	4	0

0	0	0	0
0	1	2	0
0	?	?	0
0	0	0	0

考察

- ここに注目すると、このアメーバは下のマスから来たと断定できる。
  （上左右からの影響は差し引いてあるので）



0	0	0	0
0	3	4	0
3	4	3	4
0	3	4	0

0	0	0	0
0	1	2	0
0	?	?	0
0	0	0	0

考察

- 以下同様

0	0	0	0
0	0	4	0
0	4	0	4
0	0	4	0

0	0	0	0
0	1	2	0
0	3	?	0
0	0	0	0

考察

- 以下同様

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

0	0	0	0
0	1	2	0
0	3	4	0
0	0	0	0

解法

- 上の行から順に答えを埋めていく。解の存在が保証されているので、矛盾なく答えが求まる。

- 計算量は $\mathrm{O}(NM)$

問題 C – ウサギ跳び

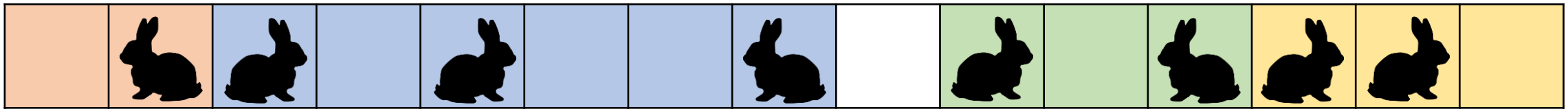
問題概要

- L 個のマスが一列に並んでいて、その上に N 匹のウサギがいる。各ウサギは左か右を向いている。
- 各ウサギは、自分の目の前が空きマスならば、ジャンプしてそのマスへ移動できる。
- ウサギがジャンプする順番を決められるとき、ジャンプの総回数の最大値を求めよ。

- $1 \leq N \leq 10^5,\ N \leq L \leq 10^9$

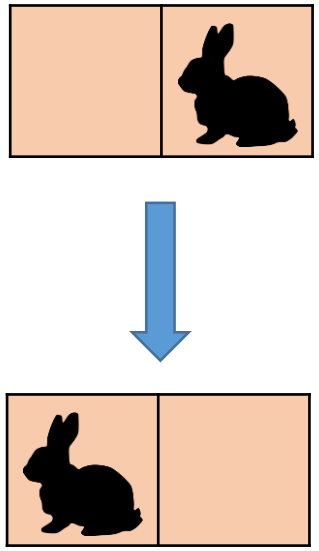
考察

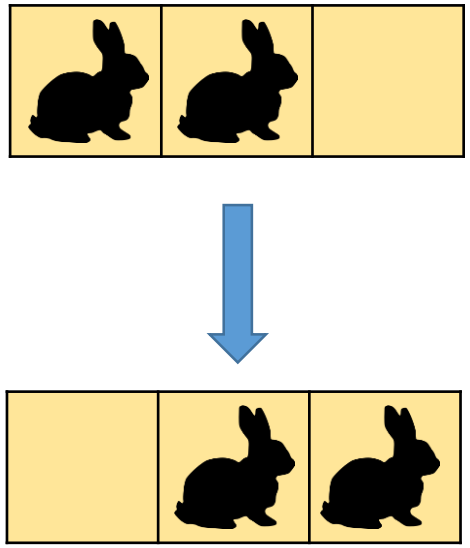
- 背中合わせのウサギたちは互いに影響しない。
- 例えば、次のような区間は別々に考えることができる。



考察

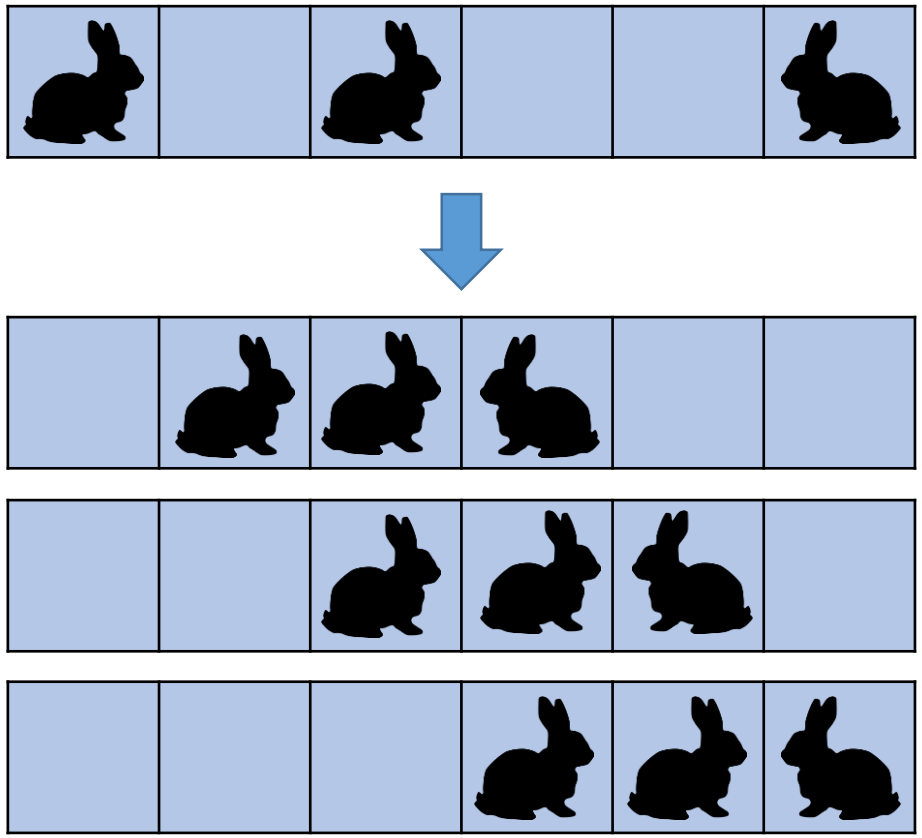
- 次のような区間は終了形が一通りに決まるので、ジャンプの総回数を計算できる。





考察

- 次のような区間は終了形が複数通り考えられる。



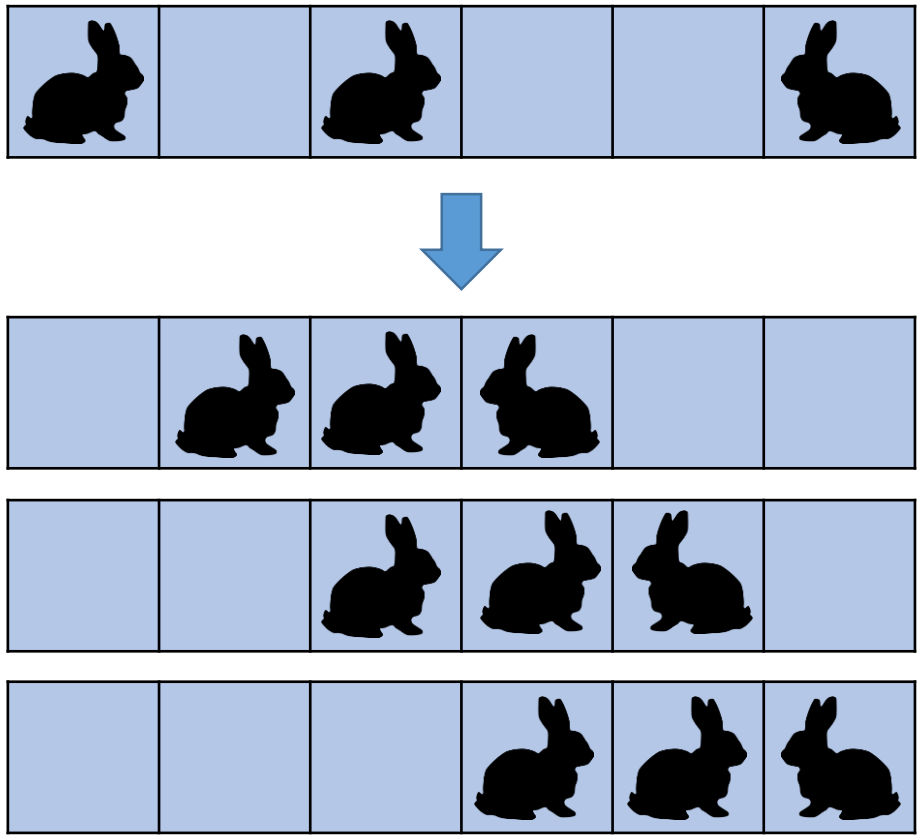
考察

- ぶつかる位置を全通り試し、ジャンプの総回数の最大値を求める？

→ $L \leq 10^9$ なので、最悪 10^9 通り試すことになり厳しい。

考察

- ジャンプの総回数を数えてみる。



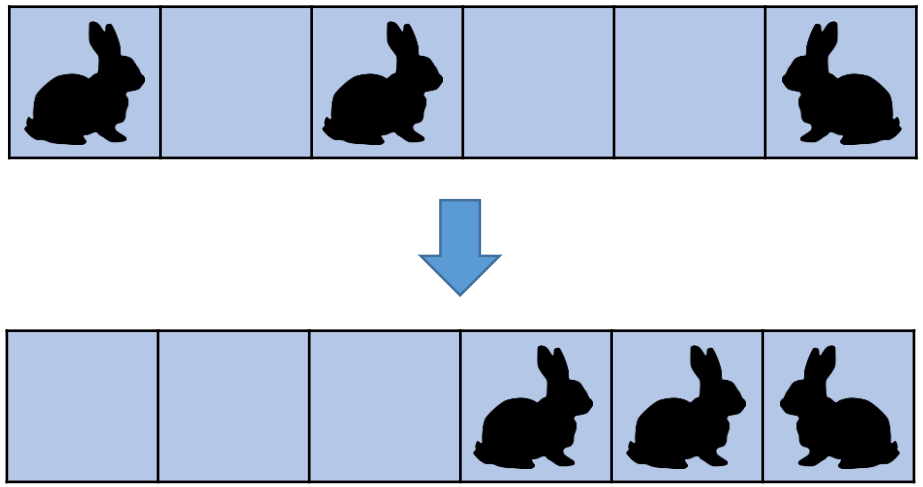
3 回

4 回

5 回

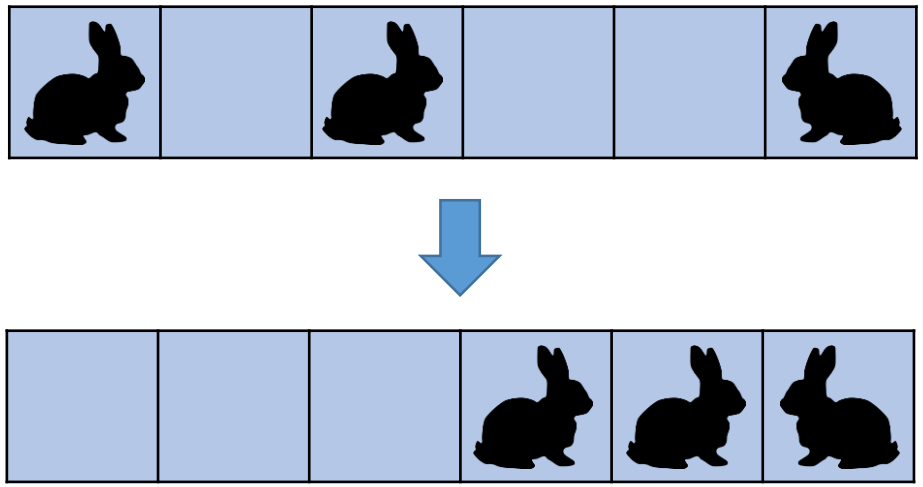
考察

- ぶつかる位置が右に 1 マス動くと、ジャンプの総回数が 1 増える。
- 左側の 2 匹のジャンプ回数が 1 ずつ増え、右側の 1 匹のジャンプ回数が 1 減るため。
- この例では、できるだけ右に寄せるのが最適。



考察

- 例から分かる通り、ウサギの少ない側に寄せるのが最適。
- 終了形が決まれば、ジャンプの総回数を計算できる。



解法

- ウサギたちを区間に分ける。
- 区間ごと別々に、ジャンプの総回数の最大値を計算する。
- その総和が答えである。ただし、オーバーフローに注意！

- 計算量は $O(N)$

問題 D – 辺彩色

問題概要

- N 頂点 M 辺の連結な無向グラフが与えられる。
- 好きな頂点から始め、好きなだけ辺を辿っていく。
- 辿った辺には 赤 → 青 → 赤 → 青 → … の順に色を塗る。ただし、色は上書きされる。
- 目標の辺の配色は可能か判定せよ。

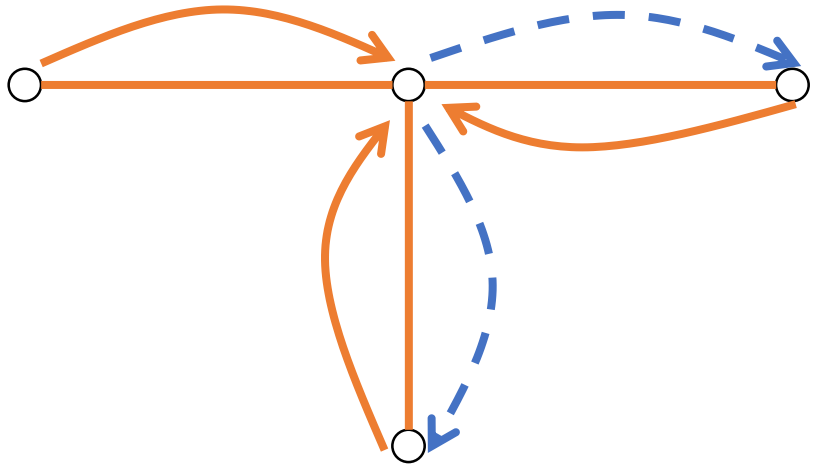
- $2 \leq N \leq 2{,}000,\ 1 \leq M \leq 2{,}000$

例 1

- 可能である。

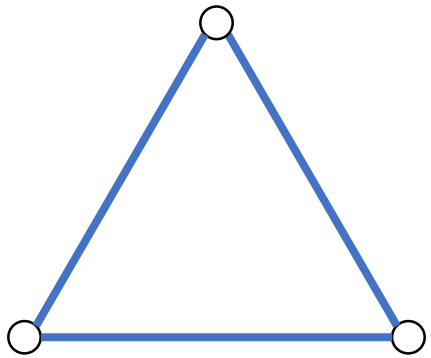
例 2

- 可能である。ただし、上書きされる色は破線で示している。



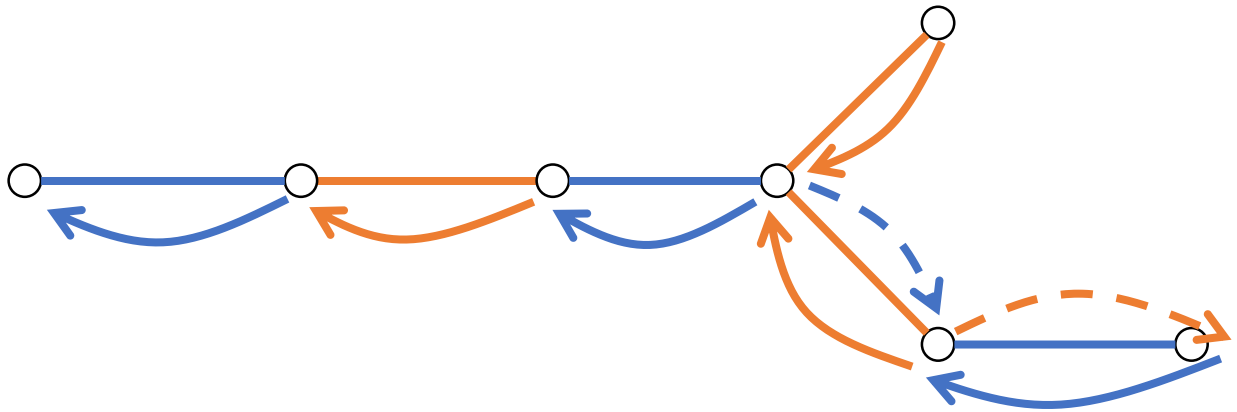
例 3

- 不可能である。



考察

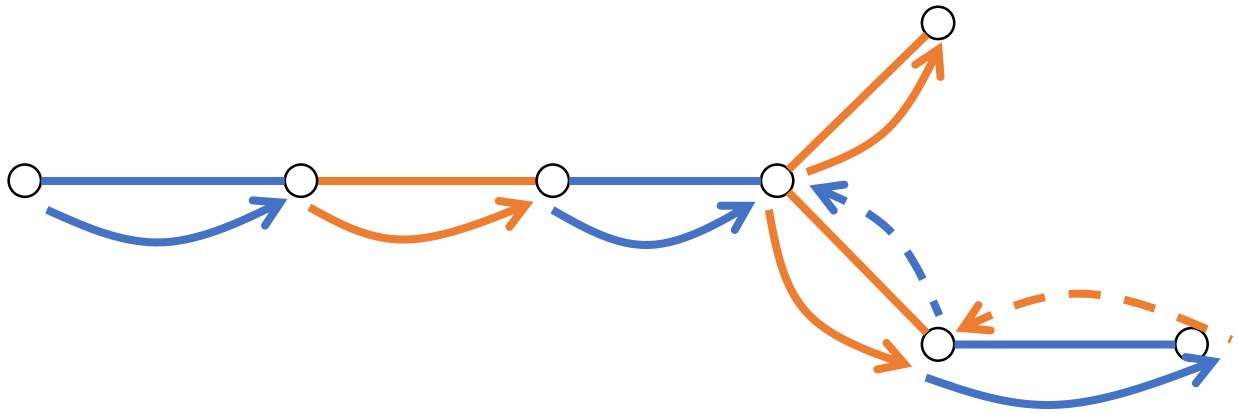
- まず、色が「上書きされる」というのがややこしい。



考察

- そこで、辺の辿り方を未来から過去へと逆向きに考えてみる。
- すると、「一度塗った色は上書きされない」というルールに変わる。

→ この方が考えやすい！



考察

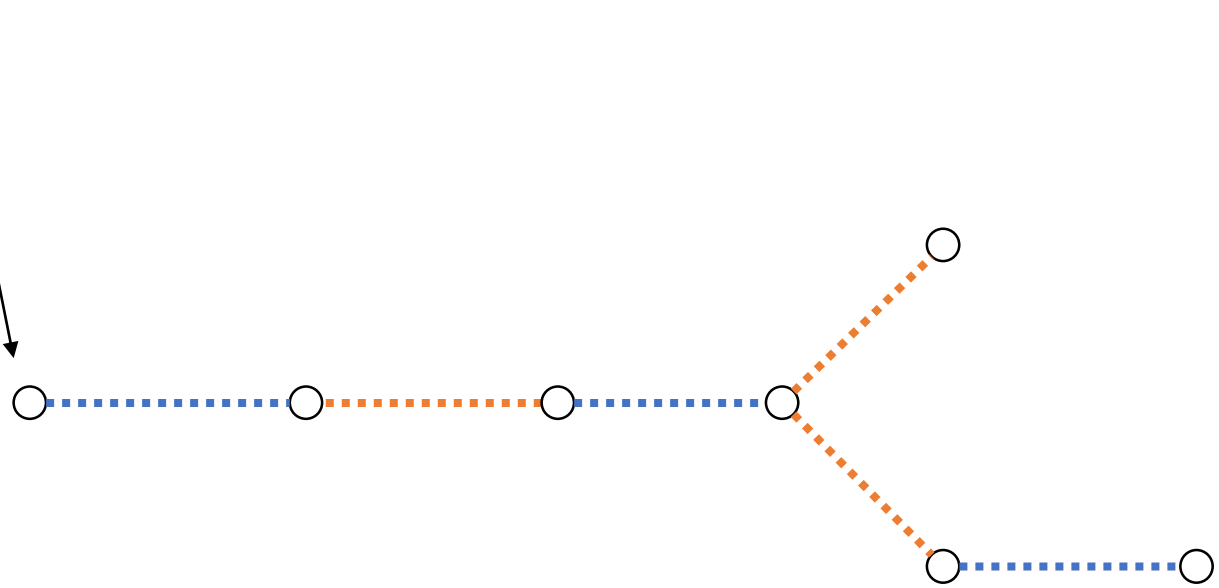
- 未来から過去へと考え直すことによって、次のように問題を言い換えられる。

  - 始点とはじめの色（赤 or 青）を好きに選び、好きなだけ辺を辿っていく。
  - 辿った辺には赤青交互に色を塗る。ただし、色は上書きされない。
  - 目標の辺の配色は可能か判定せよ。

- 見通しが良くなった！

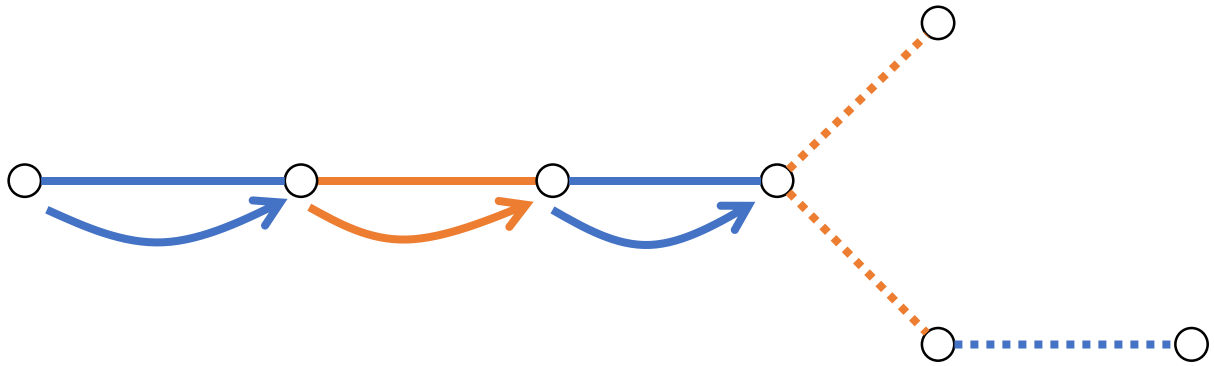
考察

- $N, M \leq 2{,}000$ と小さいので、始点とはじめの色は全探索できそう。
- 例えば、ここを始点に、青から始めてみる。



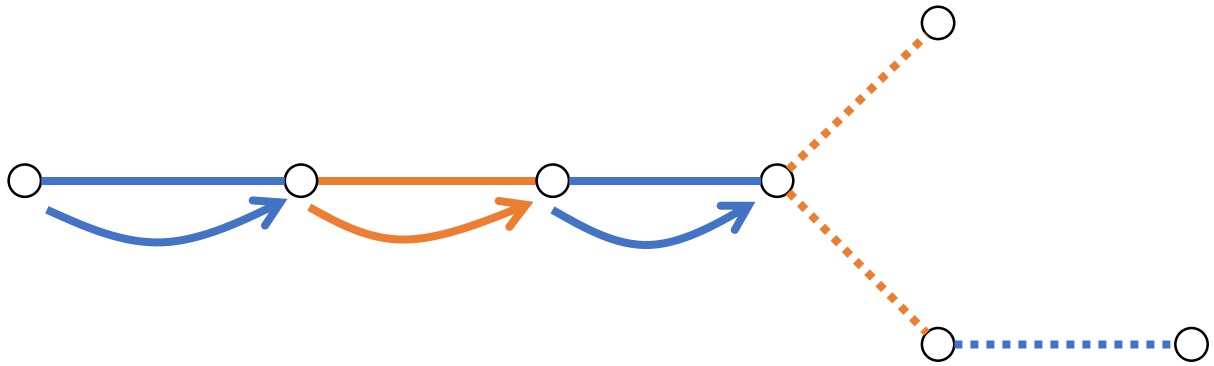
考察

- 次に塗る色と目標の色が一致していれば、その辺を塗ることができる。
- 色は上書きされないので、一度塗った辺は自由に行き来できる。



考察

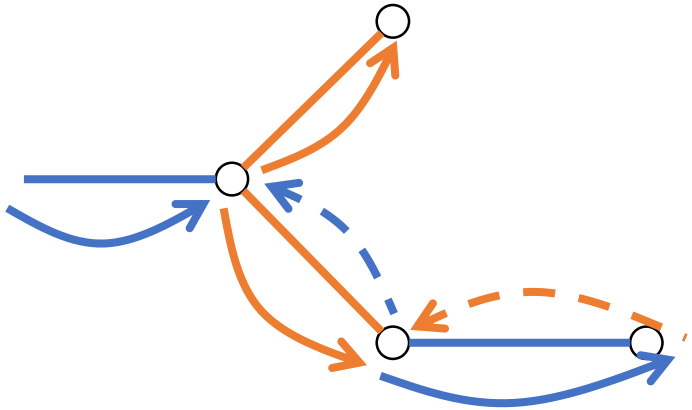
- 分かれ道はどう辿ればいいか？

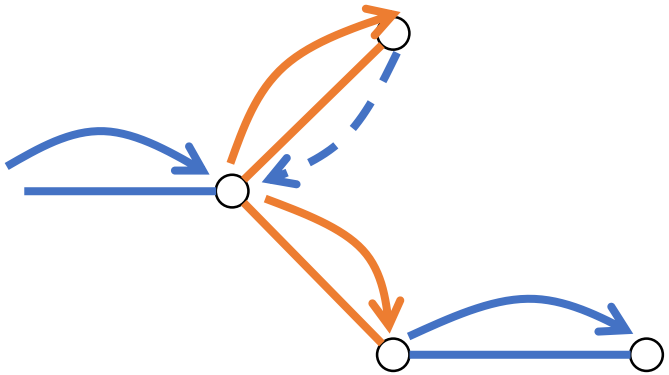


考察

- 分かれ道はどう辿ればいいか？

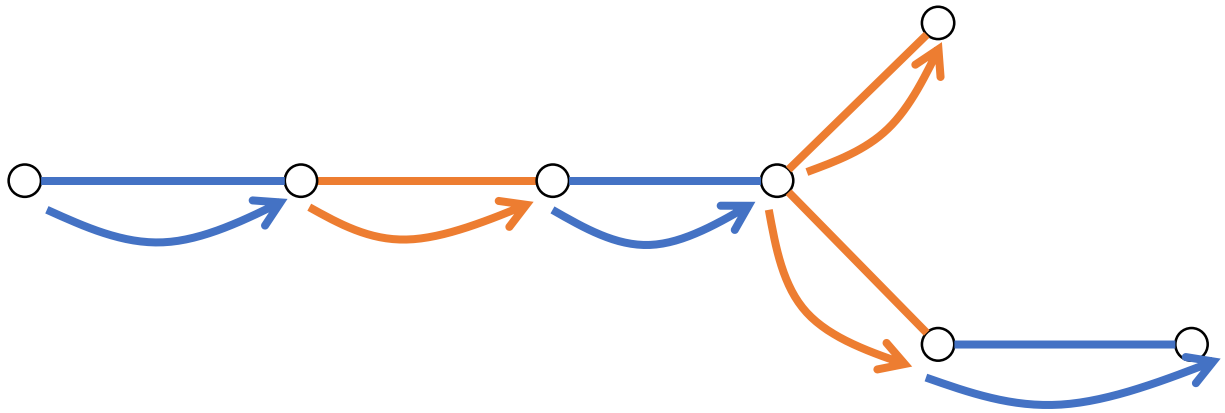
→ 実は、どれを先に選んでも同じ。





考察

- 分かれ道の選び方が自由なので、深さ優先探索や幅優先探索などで貪欲に辺を塗っていける。
- すべての辺を塗れたら、目標の辺の配色ができたことになる。



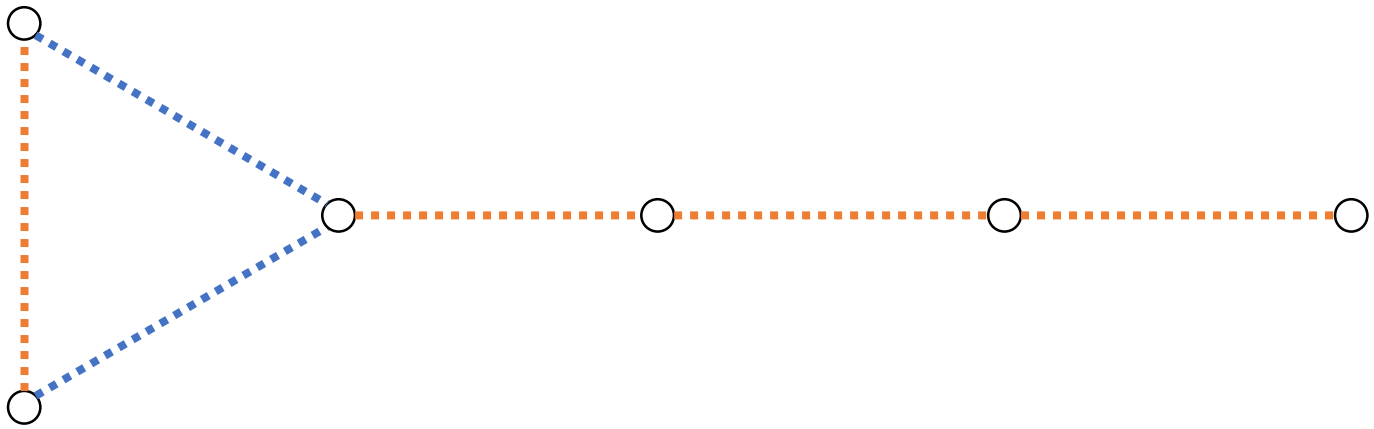
想定誤解法

- 始点とはじめの色を全探索する。
- 深さ優先探索や幅優先探索で貪欲に辺を塗っていく。
- すべての辺を塗れたら、すぐ “Yes” と判定する。
- そうでなければ “No” と判定する。

→ 重要なポイントを見落としている！

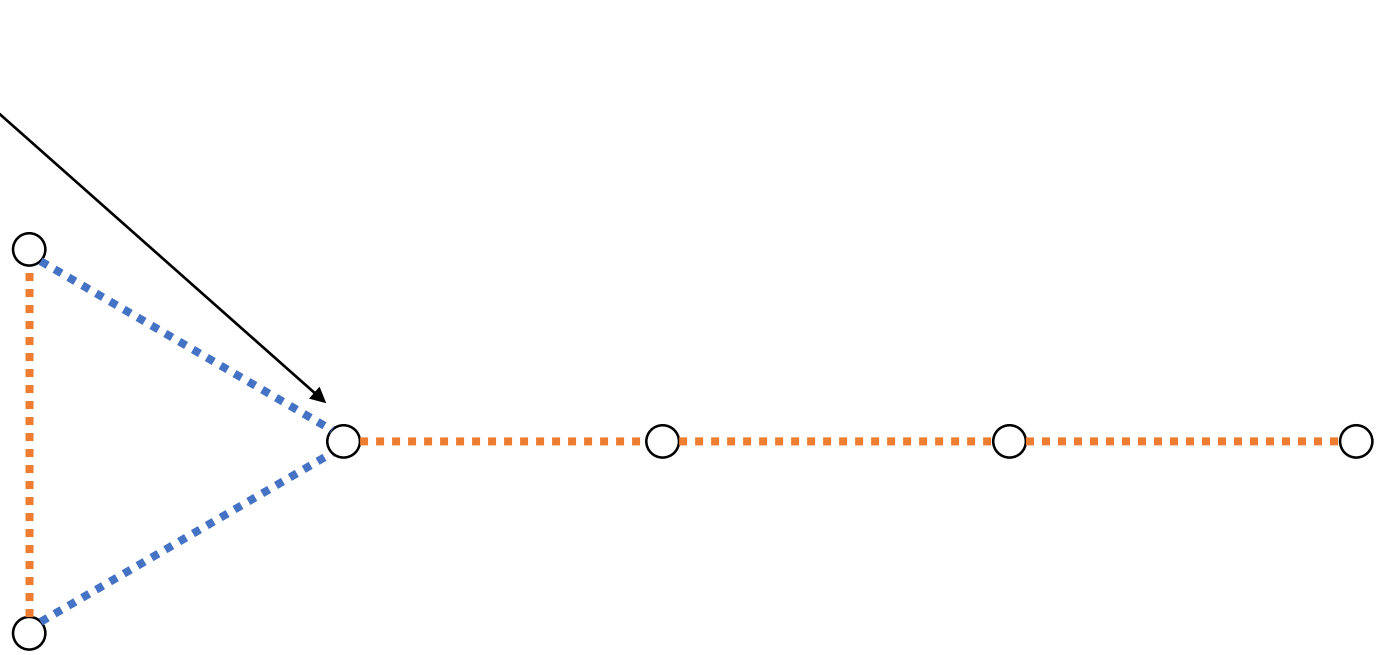
考察

- 次の例は不可能に見えるが、実は可能である。



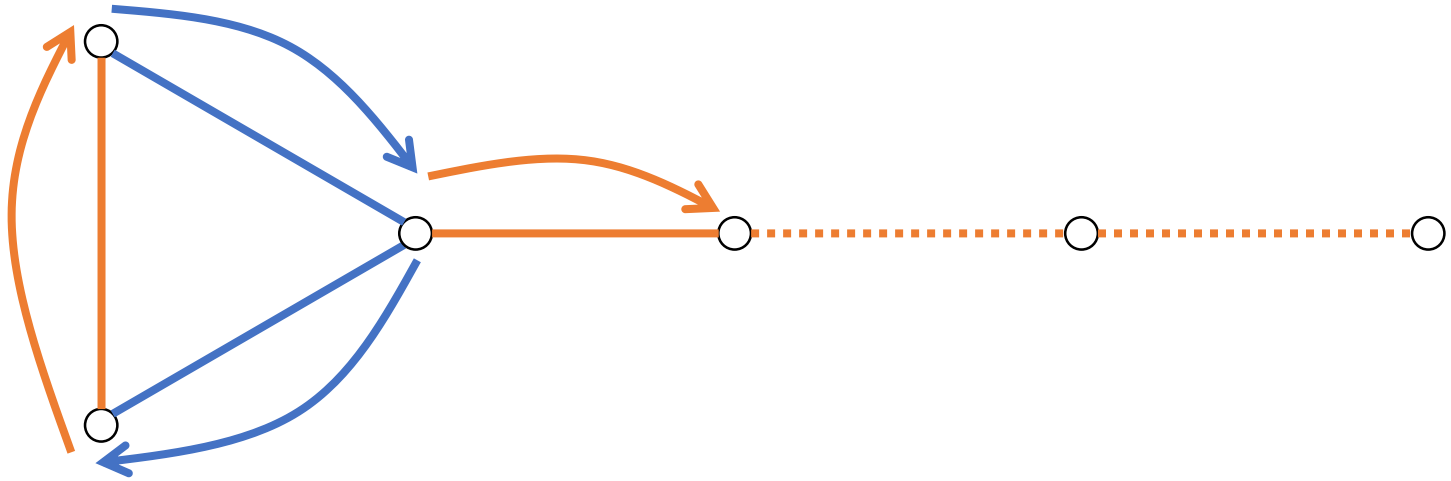
考察

- ここを始点に、青から始める。



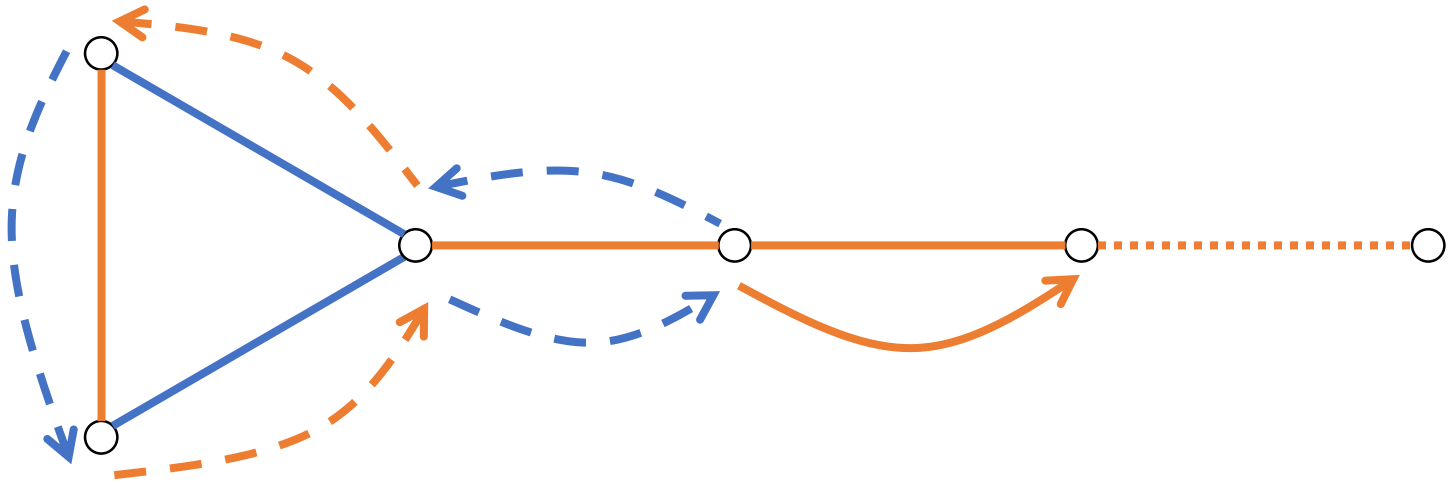
考察

- 深さ優先探索や幅優先探索で次のように塗れる。
- 三角形の閉路ができたのがポイント。



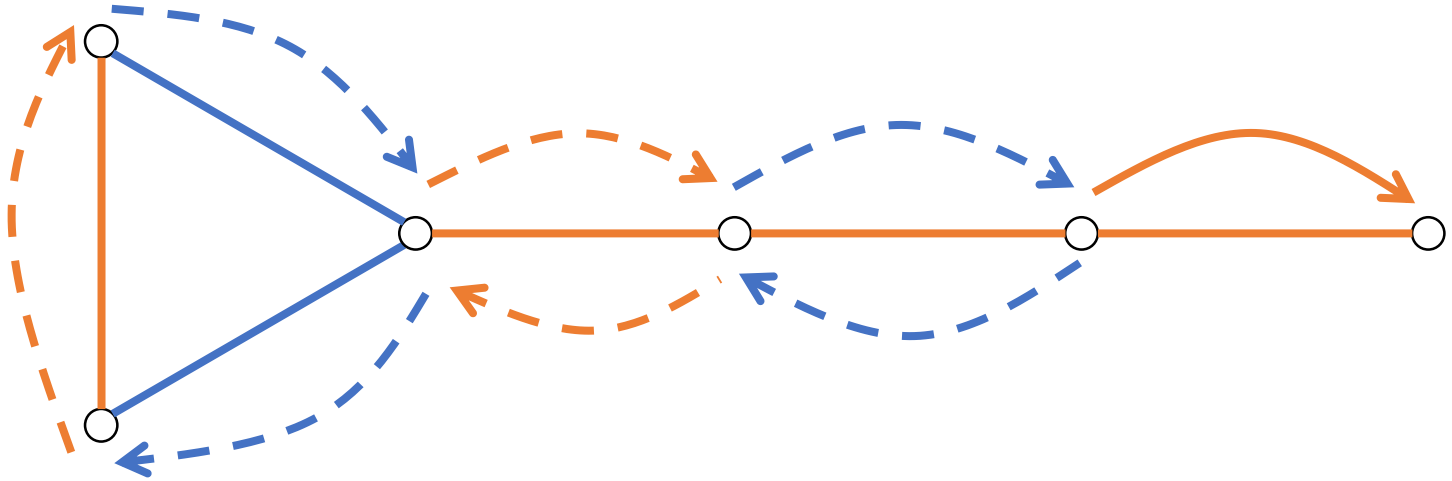
考察

- 一見詰まったように見えるが、三角形の閉路を回ることで赤 / 青を反転できる。



考察

- 再び三角形の閉路を回り、すべての辺を塗れた。



考察

- 奇数長の閉路を回るごとに赤 / 青を反転できる。
- よって、奇数長の閉路が完成すれば、あとはどんな塗り方も可能！

解法

- 始点とはじめの色を全探索する。
- 深さ優先探索や幅優先探索で貪欲に辺を塗っていく。
- すべての辺を塗れたら、すぐ “Yes” と判定する。
- 奇数長の閉路が完成したら、すぐ “Yes” と判定する。
- そうでなければ “No” と判定する。

- 計算量は $O(NM)$

AtCoder Regular Contest 040
解説

AtCoder株式会社

ARC 040 A問題 解説

「床塗り」

snuke

問題概要

N*N のマス目が赤と青のインクで塗られている

赤のマスが多いか、青のマスが多いか、同じかを判定せよ

1 ≦ N ≦ 100

解法

‘R’ の個数、’B’ の個数をそれぞれ数え、比較する

文字列の扱いに慣れていると良いでしょう

ARC 040 B問題 解説
「直線塗り」

snuke

問題概要

N 個のマスが直線状に並んでいる

高橋君は全てのマスを塗りたいと思っている

いくつかのマスはすでに塗られている

高橋君はマス 1 からスタートする

1 秒の間に、

・隣のマスに移動する

・右 R マスを塗る（マス i ~ マス i+R-1 までを塗る）

1 ≦ N ≦ 100

解法

まず、移動にかかる時間について考える

最も右にある ‘.’ を塗ることができる場所まで進めば十分で、それ以上進むメリットはない

つまり、最も右にある ‘.’ がマス i だとすると、

max(0, i-R) 回進めば十分

解法

次に、銃を撃つ回数について考える

・最も左にある ‘.’ を探す（これをマス i だとする）

・マス i ～ マス i+R-1 までを塗る

を繰り返すのが最善となる

解法

移動にかかる時間と、銃を撃つのにかかる時間を足し合わせてあげれば答えになります

ARC 040 C問題 解説

「Z塗り」

snuke

問題概要

N*N のマス目があり、いくつかのマスは既に塗られている

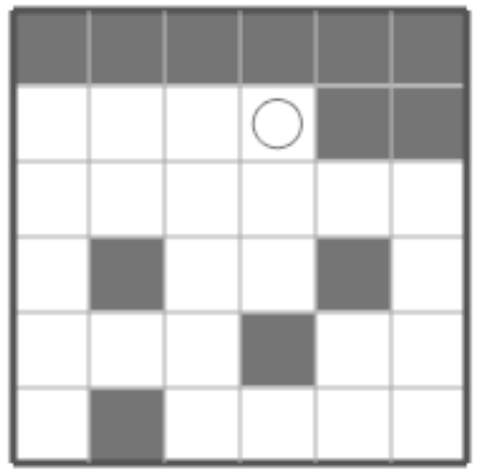
インク発射装置を使うと、Z字型の領域を塗ることができる

すべてのマスを塗るためには装置を何回使えばいいか

$1 \leqq N \leqq 100$

解法

まだ塗られていないマスのうち、最も上にあるもののうち最も右にあるものに注目する



図の○のようなマス

解法

このとき、

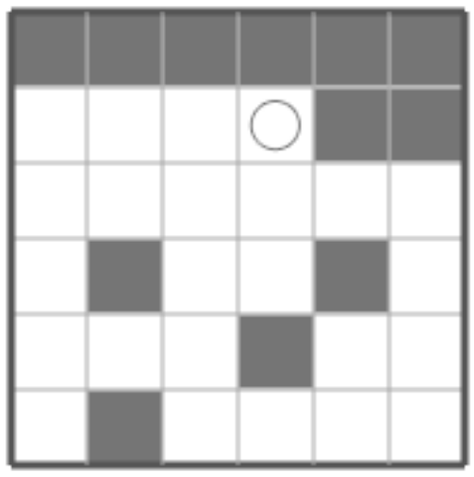
・○のマスより上の行にあるマスと、

・○のマスと同じ行で○のマスの右にあるマス

は既に塗られているはず

ここで、○のマスを塗る方法について考えると、

Z字の右上の角を○に合わせて塗る塗り方が他の塗り方よりも真に良いことが分かります



解法

解法としては、

・まだ塗られていないマスのうち右上のものを探す

・そのマスにZ字の右上の角を合わせるように塗る

を繰り返し、全部のマスが塗られるまでに繰り返した回数を答えれば良い

ARC 040 D問題 解説
「カクカク塗り」

snuke

問題概要

マス目があり、いくつかのマスには障害物がある

「今いるマスを塗って隣のマスへ移動する」を繰り返す

移動するたびに90度向きを変えなければならない

スタートのマスが決まっているとき、すべてのマスを塗ることが可能か判定せよ

$1 \leqq N \leqq 50$

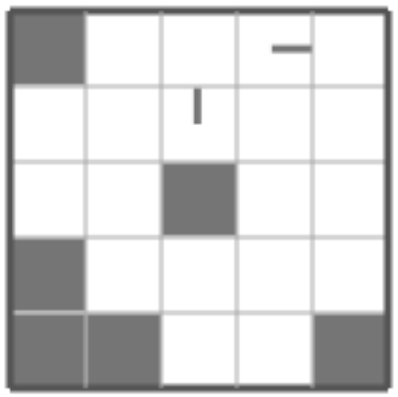
部分点（40点）

$1 \leqq N \leqq 50$

マス目が少し小さい

部分点（40点）

スタートの向き、ゴールの位置、ゴールの向きをすべて試す



例えば図のような状態を試す

このとき全てのマスが塗れるかを判定したい

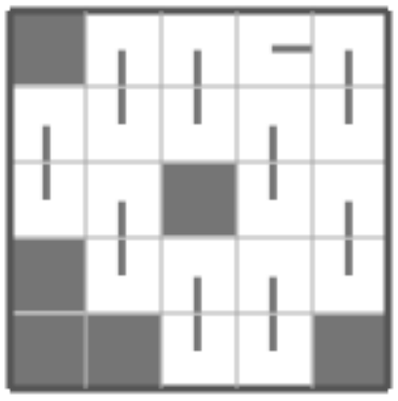
部分点（40点）

経路について考えたとき、あるマスに注目すると、そこからは上or下と左or右の2本の線が伸びるはず

つまり、上下方向のどちらかちょうど一方に線が伸びているはず

上下方向の線だけ書いてしまおう

各列に注目すると、上下方向の線の書き方は一意に定まる

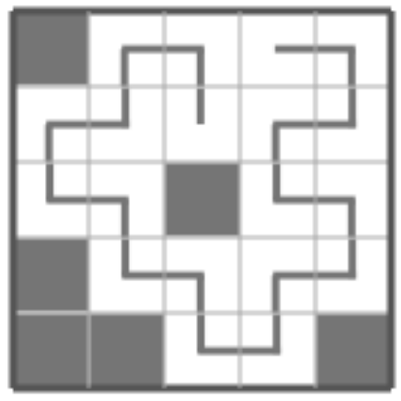


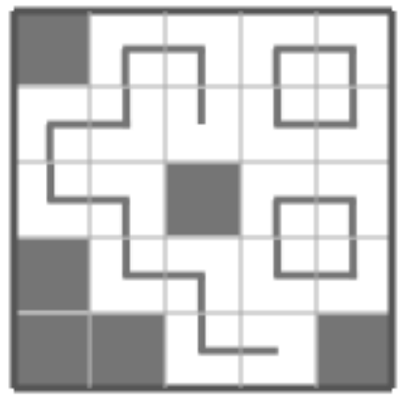
部分点（40点）

そして、左右方向も同様に書く

すると、経路ができあがる

ただし、ここで出来上がった経路が正しいものかを判定しないといけない（右の図のような例がある）





部分点（40点）

解法をまとめると、

スタートの向き、ゴールの位置、ゴールの向きをすべて試す

経路の線を書いて、それが正しいか判定する

ゴールの位置の候補がN^2個、判定がO(N^2)なので、

計算量はO(N^4)となる

満点解法

$1 \leqq N \leqq 400$

マス目が大きい

ゴール位置の候補を減らす

満点解法

スタート・ゴールの経路上の特徴を考える

スタート・ゴールでは、上下方向 or 左右方向のどちらかにしか線が出ない

つまり、上下方向 or 左右方向のどちらかの方向に対しては障害物と同じような扱いになる

満点解法

上下方向（or 左右方向）に線を書き入れるとき、それぞれの列には空きマスが**偶数**個なければいけない

奇数個だと端数が出てしまう

ゴールは「上下方向 or 左右方向のどちらかの方向に対しては障害物と同じような扱いになる」マスなので、空きマスが奇数個の列（or 行）になければならない

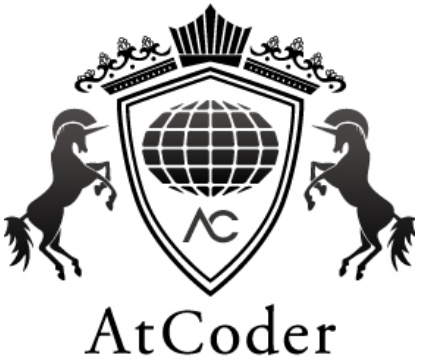
つまり、ゴールの位置としては、空きマスが奇数個のある列（or 行）のうちのどこかしか試さなくていい！

満点解法

ゴールの位置の候補がN個、判定がO(N^2)なので、

計算量はO(N^3)となる

AtCoder Regular Contest 039
解説



AtCoder株式会社 代表取締役
高橋 直大

競技プログラミングを始める前に

AtCoder

- 競技プログラミングをやったことがない人へ
  - まずはこっちのスライドを見よう！
  - http://www.slideshare.net/chokudai/abc004

AtCoder

A問題 A - B Problem

解説スライド担当: 森田 晃平

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

- 3桁の整数A, Bが与えられる
- 1桁だけ書き換えることができる
- A - Bを最大化せよ

- ただし100の位を0に書き換えてはいけない

A問題　アルゴリズム

AtCoder

- Aはできる限り大きくしたい
- Aのどこかを書き換えるなら9にするべき
- Bはできる限り小さくしたい
- Bのどこかを書き換えるなら？
  - 1, 10の位を書き換えるなら0にするべき
  - 100の位を書き換えるなら1にするべき

A問題　アルゴリズム

AtCoder

- つまり書き換え方は6パターン
  - Aの1の位を9に
  - Aの10の位を9に
  - Aの100の位を9に
  - Bの1の位を0に
  - Bの10の位を0に
  - Bの100の位を1に

A問題　アルゴリズム

AtCoder

全部試す！

A問題　アルゴリズム

AtCoder

- 文字列(10進数)⇔数字の相互変換が出来ると楽です

C++11による一例を載せておきます

```
int a = 123;
string a_str = to_string(a);
a_str[0] = '9';
int new_a = stoi(a_str);
printf("%d %d¥n", a, new_a); //123 923
```

AtCoder

B問題　高橋幼稚園

1. 問題概要
2. 考察
3. アルゴリズム

©AtCoder Inc. All rights reserved.

- N人の児童にK個のキャンディを配る
- 全体の幸福度=(各児童に与えられたキャンディの個数の積)
- 全体の幸福度が最大となるような配り方の総数を求めよ
- ただし答えは大きいので，10^9+7で余りを取ること
- 1≦N≦100, 1≦K≦500

- 最大となるような配り方はどんなものか考える
- 結論を言うと，N≦Kのとき，できるだけ均等に分けると最大
- たとえばN=4,K=10のとき順番を無視すれば，
  3個 3個 2個 2個
- というふうに配ると最大.

- したがって，
  - (N/Kを切り捨てた値)個は少なくとも全員に配れるので必ず配る
  - 残りのN%K個を各児童に高々1つ分配する
- という分配方法の総数を求める問題になる

AtCoder

- N=4,K=10のケースを図で表してみる
  - (N/Kを切り捨てた値)=2
  - N%K=2
- なので，固定キャンディを□，動かせるキャンディを■で表すと
  - □□■
  - □□■
  - □□
  - □□
- ■の分配方法の総数
  =N個の中からN%K個を選ぶ組み合わせの総数
- $={}_{N}C_{N\%K}$

AtCoder

- しかし，N<Kのときどう配っても全体の幸福度は0になってしまうため，これはコーナーケース．
- この場合に限り，均等でなくともよくどんな配り方をしてもよい
- したがって答えは，異なるN個のものから重複を許してK個取り出す組み合わせの総数に等しくなる．
- この組み合わせの総数は${}_N H_K = {}_{N+K-1} C_{K-1}$で表される．

AtCoder

- 考察をまとめると以下のようなプログラムを書けば良い
  - N≧Kのとき${}_{N}C_{N\%K}$ を10^9+7で割った余りを出力
  - N<Kのとき${}_{N+K-1}C_{K-1}$ を10^9+7で割った余りを出力
- N≧Kのケースしか考慮していない場合は80点しか取れないような部分点を設定した.

- nCrの求め方は次のスライドの24ページ目を参考にしてください: http://www.slideshare.net/chokudai/abc021

- 今回は，時間・空間計算量がO(N^2)のもので間に合います.

AtCoder

C問題　幼稚園児高橋君

1. 問題概要
2. 考察
3. アルゴリズム
4. 参考

©AtCoder Inc. All rights reserved.

AtCoder

- 無限に広がる二次元格子があり，最初原点にいる
- 上下左右のいずれかに未訪問の格子にたどり着くまで直進するという動作を繰り返す

- 各時点でどの方向を選んだかというK個の情報 $S_1, S_2, S_3, \ldots, S_K$ が与えられるので，シミュレーションして最終的に辿り着いた地点を出力せよ

- $1 \leqq K \leqq 200000$

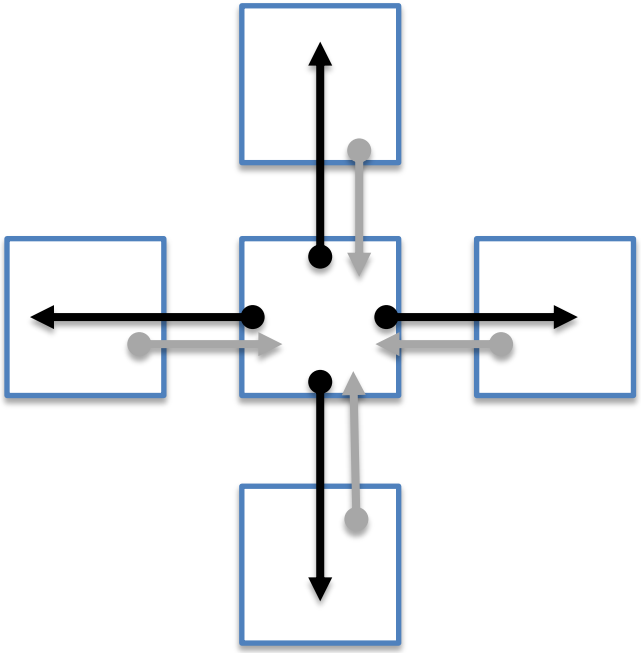
AtCoder

- 一度の直進動作には最悪O(K)かかる
(左右に動き続けるケースを想定すると良い)
- これだと最悪O(K^2)かかり，とても間に合わない
- そこで，各格子のそれぞれ上下左右で最近傍の未訪問格子の座標を記録しておき，逐次更新することを考える
- 更新方法だが，ある格子を訪問したとき，その4近傍の格子のみについて最近傍の情報を書き換えれば良い

- その情報に基づいてシミュレーションすれば即座に次の未訪問格子に辿り着ける

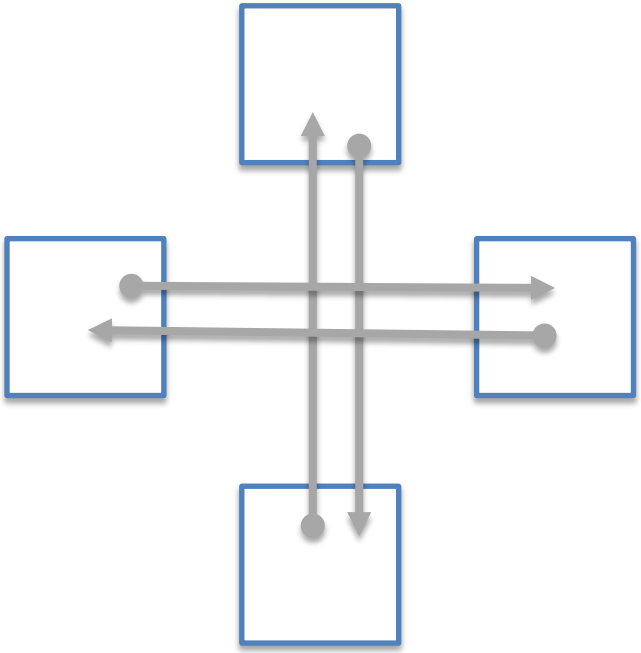
AtCoder

- イメージは以下の通り.
- このような情報を全ての格子に持たせる



AtCoder

- 格子を訪問済みにするときは，
  訪問済みにした格子の近傍情報を利用して，
  以下のように4近傍の格子の情報を張り替えればよい

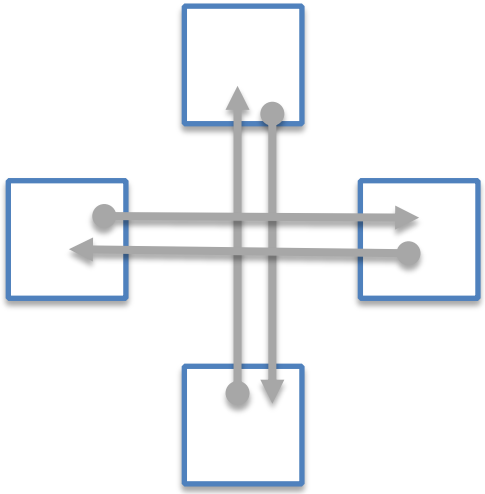


AtCoder

- 具体的には，訪問済みにする格子をxとすると
  - xの右格子の左格子=xの左格子
  - xの左格子の右格子=xの右格子
  - xの上格子の下格子=xの下格子
  - xの下格子の上格子=xの上格子

というふうに張り替え

（左格子,右格子,…という表記は全て未訪問のものを指す）



AtCoder

- 実際に格子は無限大に広がっているので，
予め全格子について初期化しておくことはできない．
- 情報を更新する必要が出てきたときに初めて近傍の情報を生成する方針を取る(遅延評価)
- 実際に訪問する格子はK個しかなく，訪問済み操作時も4近傍しか見ないので，情報が更新される格子はO(K)個
- 格子の近傍情報はハッシュや平衡二分探索木で管理する

- これで一度の直進動作はO(1)もしくはO(log K)で行えるようになった

- 近傍情報更新の遅延評価の実装方法としては，
  - 既に更新しようとしている格子の近傍情報が生成されてるなら新しく生成せず，それを更新する
  - 生成されていないならば近傍情報を生成する
- という場合分けを行うと良い．
- あとは近傍情報を使ってシミュレーションする
- 方向に対してインデックスを時計周りに定義するなどして，実装を軽くする工夫をしましょう！

- ハッシュマップまたは平衡二分探索木を使えば，全体の計算量は O(K) もしくは O(K log K) となる

AtCoder

- このような4近傍の情報を持つデータ構造はDancingLinksと呼ばれています．興味がある人は調べてみてください．

AtCoder

D問題　旅行会社高橋君

解説スライド担当: 森田 晃平

©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

- N頂点M辺の無向グラフが与えられる
- Q個クエリが飛んでくるので処理する
  - 頂点A, B, Cが与えられる
  - A -> B -> C と経由するトレイルがあるか判定

- 制約
  - 1 <= N <= 100,000
  - 1 <= M <= 200,000
  - 1 <= Q <= 100,000

D問題　アルゴリズム

AtCoder

この問題を解くには”二重辺連結成分分解”というアルゴリズムを使用する

D問題　アルゴリズム

AtCoder

二重辺連結成分分解とは？

無向グラフを二重辺連結成分に分解するアルゴリズム

二重辺連結成分とは？

橋を含まない連結な無向グラフの事を二重辺連結成分と呼ぶ

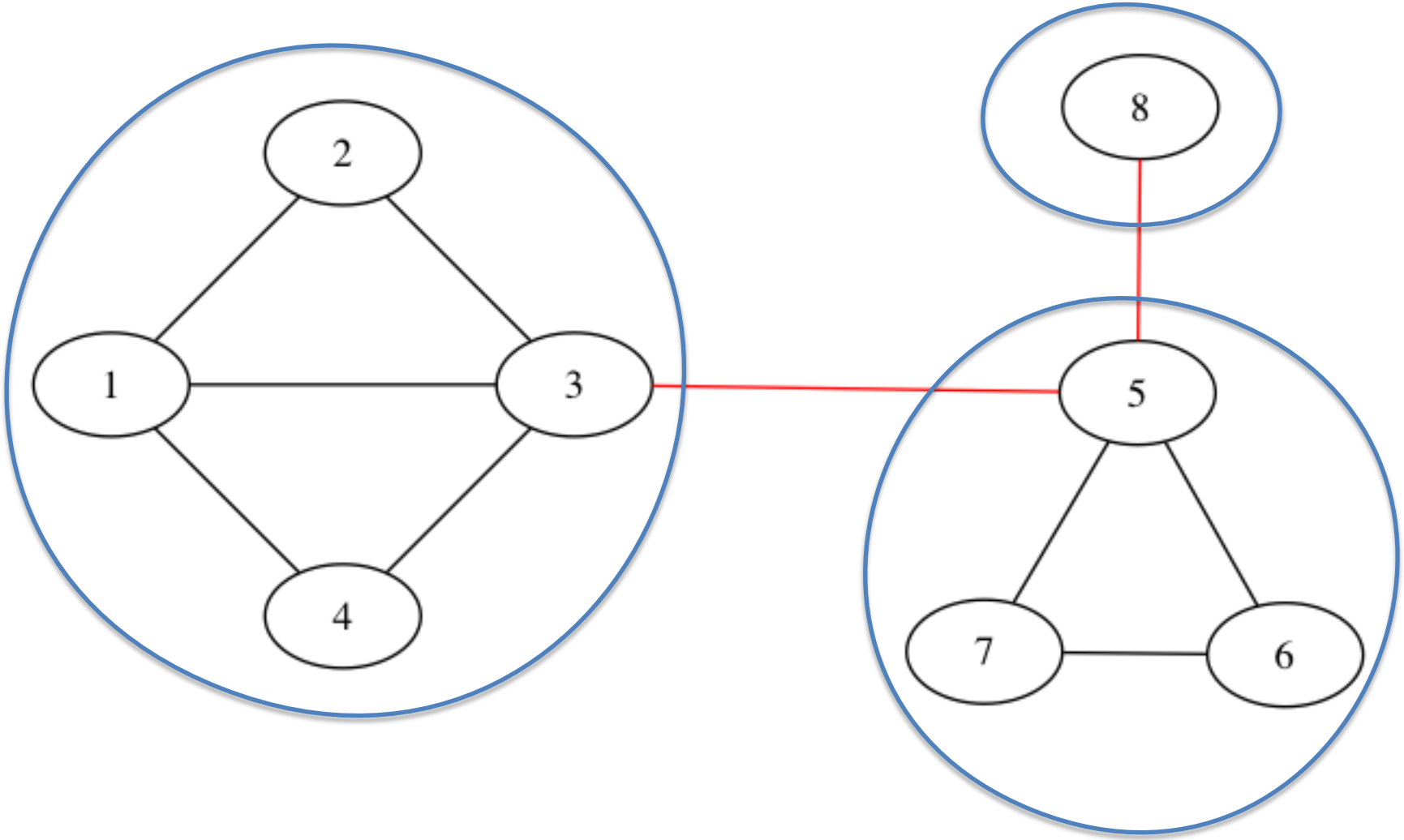
橋とは？

辺(u, v)について、その辺を取り除くと(u, v)が連結ではなくなってしまう時その辺の事を橋と呼ぶ

D問題　アルゴリズム

AtCoder

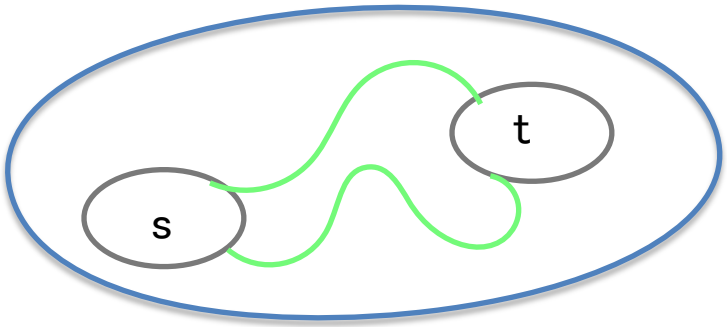
橋と二重辺連結成分の例



D問題　アルゴリズム

AtCoder

Gが二重辺連結成分の時、Gの任意の2頂点s, tについて
s, tを結ぶ、辺を共有しない2本のパス(辺素パス)が存在する



これはGの任意の2頂点s, tについて、s -> t へとフローが2以上流せるとも考えられる

ただし辺は全て容量1の無向辺として考える

D問題　アルゴリズム

AtCoder

Gは連結なので明らかにフローが1は流せる

Gに橋がなければフローが2以上流せる事を示したい

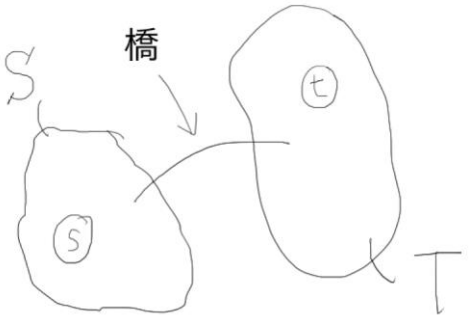
D問題　アルゴリズム

AtCoder

最大フロー最小カット定理を使う

仮に最大フローが1だとすると、S, T間に辺が1本だけ架かっているs-tカットが存在する事になる

その1本の辺を取り除くと、
SとTが非連結になってしまうため
明らかに橋



よって最大フローが1なら橋が存在する

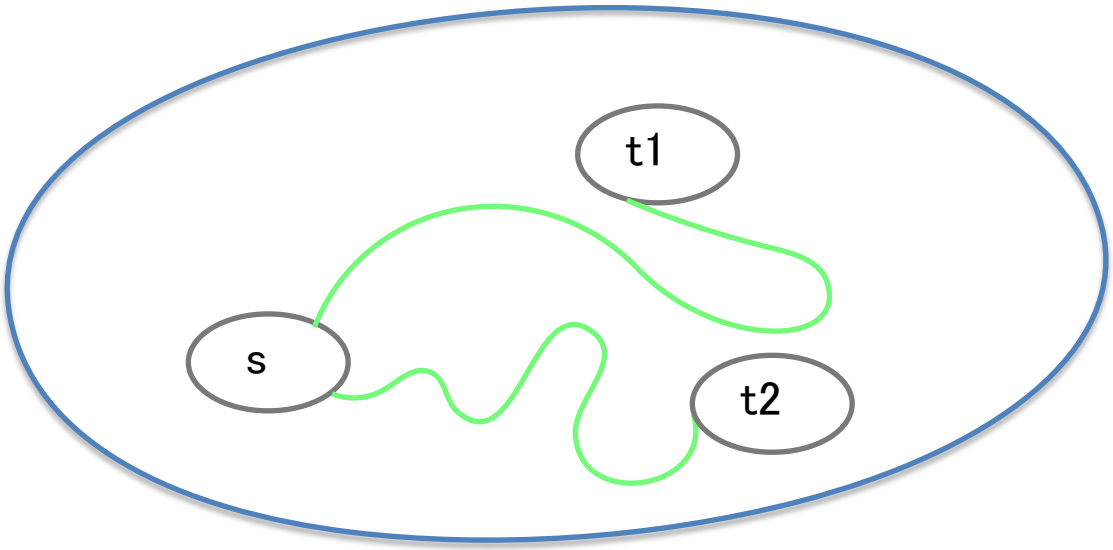
-> Gに橋がなければ必ずフローが2以上流せる
-> Gが二重辺連結成分なら必ずフローが2以上流せる

D問題　アルゴリズム

AtCoder

s, tを結ぶ、辺を共有しない2本のパス(辺素パス)が必ず存在する事がわかった

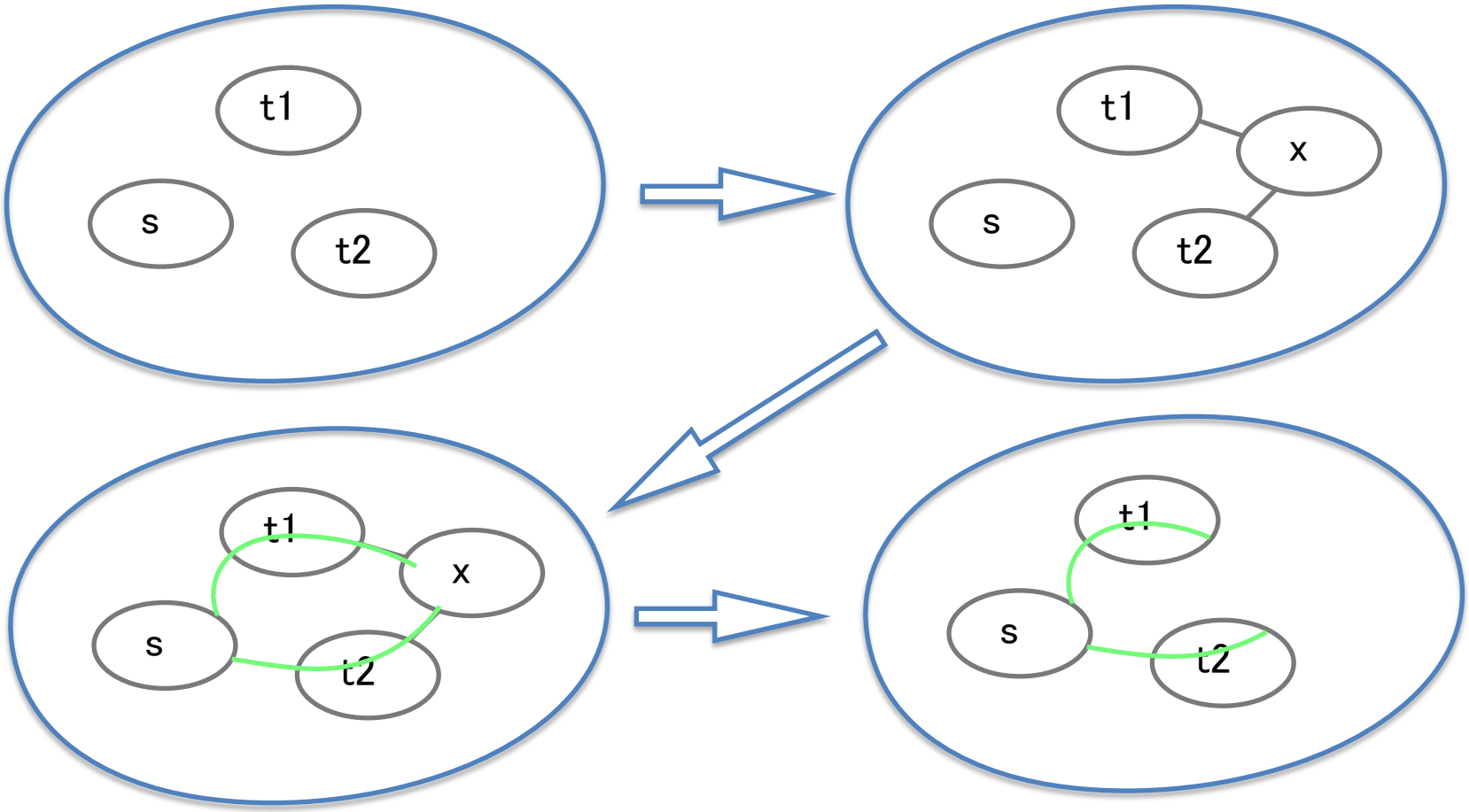
更に、任意の3頂点 s, t1, t2 について辺を共有しない s - t1 パスと s - t2パスが存在する事も言える



D問題　アルゴリズム

AtCoder

これは、新しい頂点xを用意して x-t1, x-t2という辺を貼ると、s - x 間に2本の辺素パスがある事からわかる



D問題　アルゴリズム

AtCoder

よって、A, B, C が同じ二重辺連結成分にある場合、クエリの答えは OK

じゃあ違う場合はどうする？

D問題　アルゴリズム

AtCoder

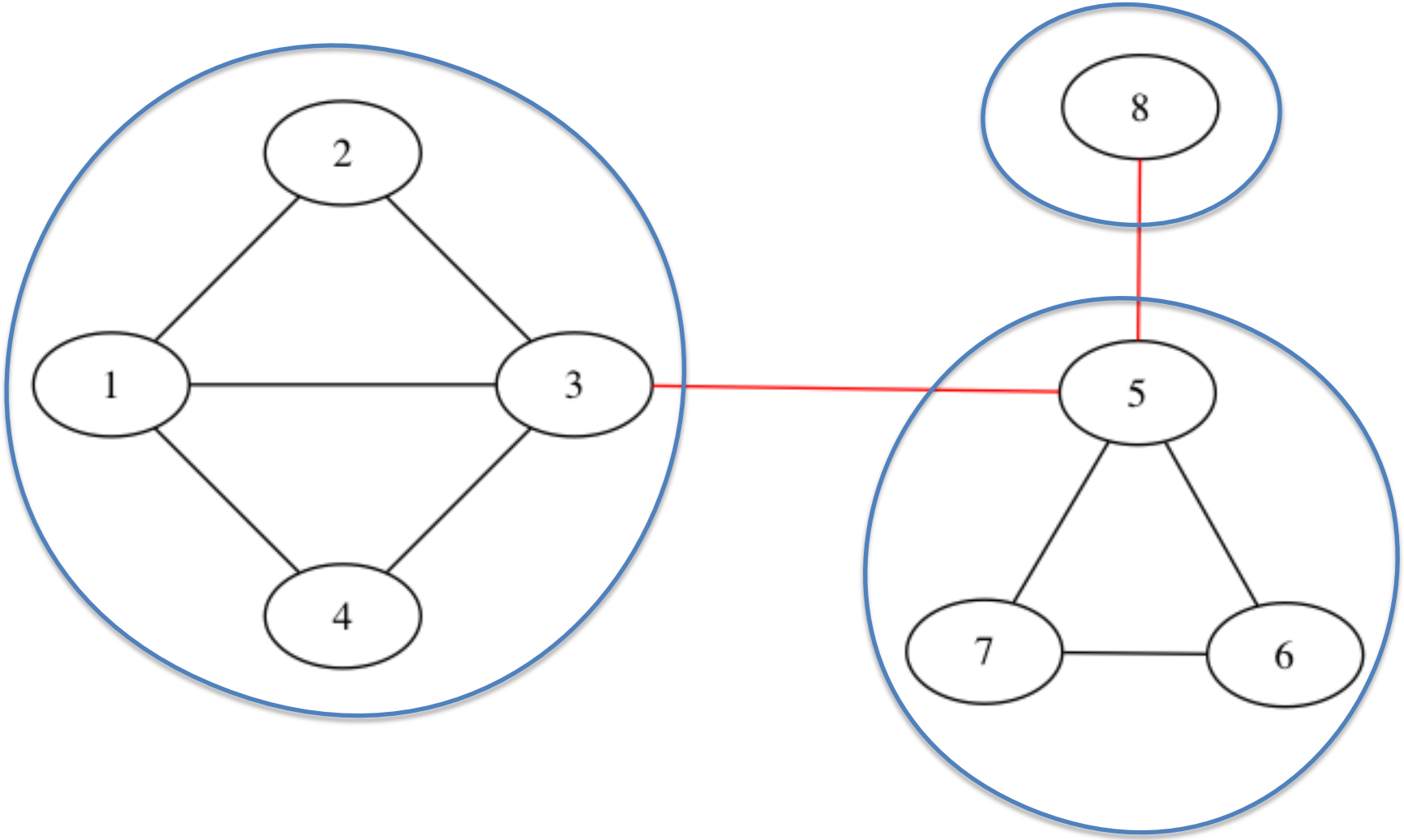
まず二重辺連結成分分解をして、二重辺連結成分を全部検出する

二重辺連結成分分解は、グラフの橋を全部検出して取り除けば残ったものが全て二重連結成分

ここで、元のグラフで二重辺連結成分を縮約して1つの頂点にしてしまうと、橋だけが辺として残り、グラフは木になる

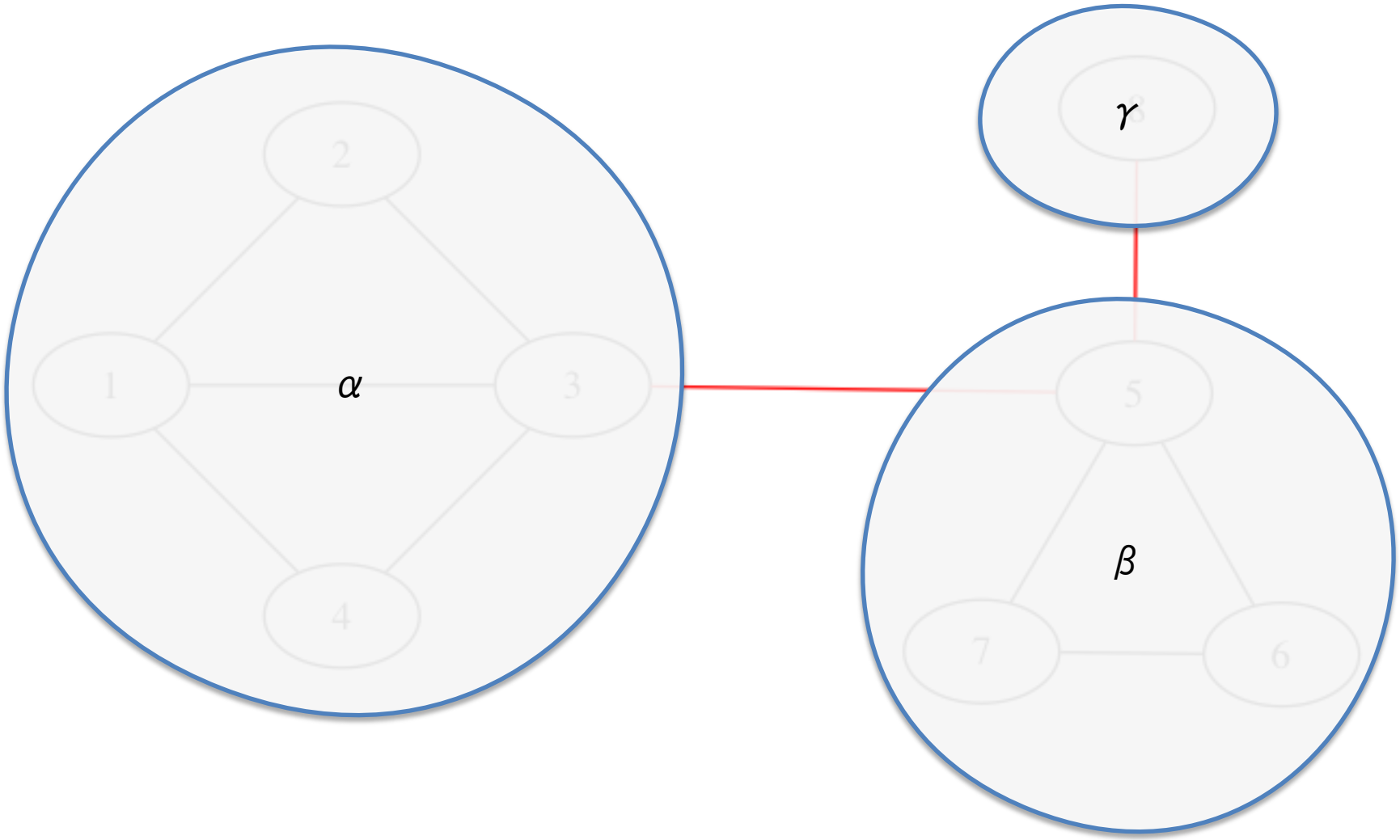
D問題　アルゴリズム

AtCoder



D問題　アルゴリズム

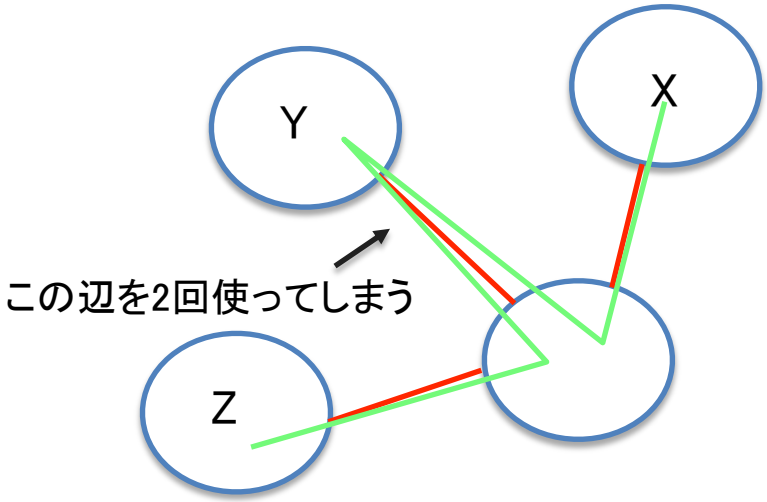
AtCoder

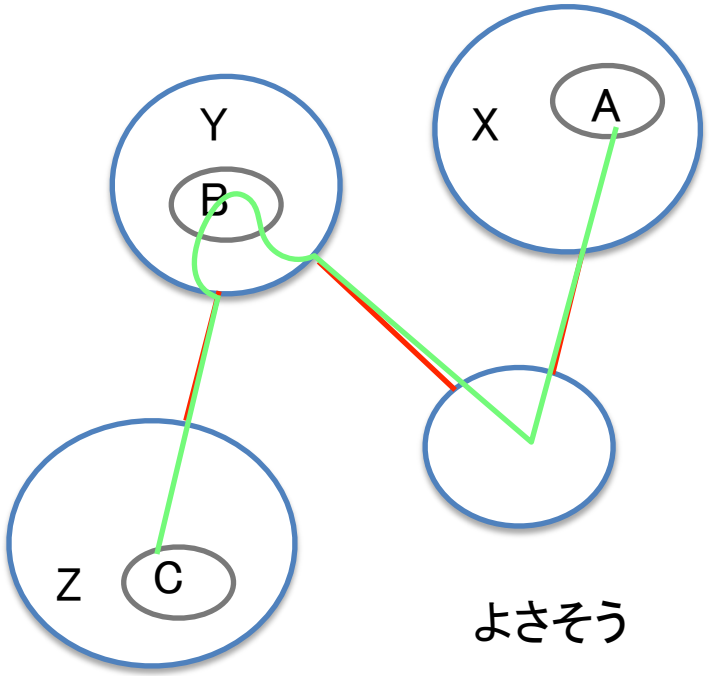


D問題　アルゴリズム

AtCoder

こうして生まれた木においてA, B, Cを含む頂点をそれぞれX, Y, Zとすると、パス X - Z 上にYが存在するかどうかがクエリの答えになる





D問題　アルゴリズム

AtCoder

パス X - Z 上にYが存在するかどうかは
dist(X, Z) == dist(X, Y) + dist(Y, Z)
で判定できる
ただしdist(a, b)は木でのパス a - b の長さ

distは適当な頂点を根にしておけば、
dist(a, b) = depth(a) + depth(b) - 2*depth(lca(a, b))
で計算できる

D問題　橋の列挙法

AtCoder

以上より二重辺連結成分分解ができればこの問題が解ける事がわかった

そして橋の列挙ができれば二重辺連結成分分解が出来る

じゃあどうやって橋を列挙するのか？

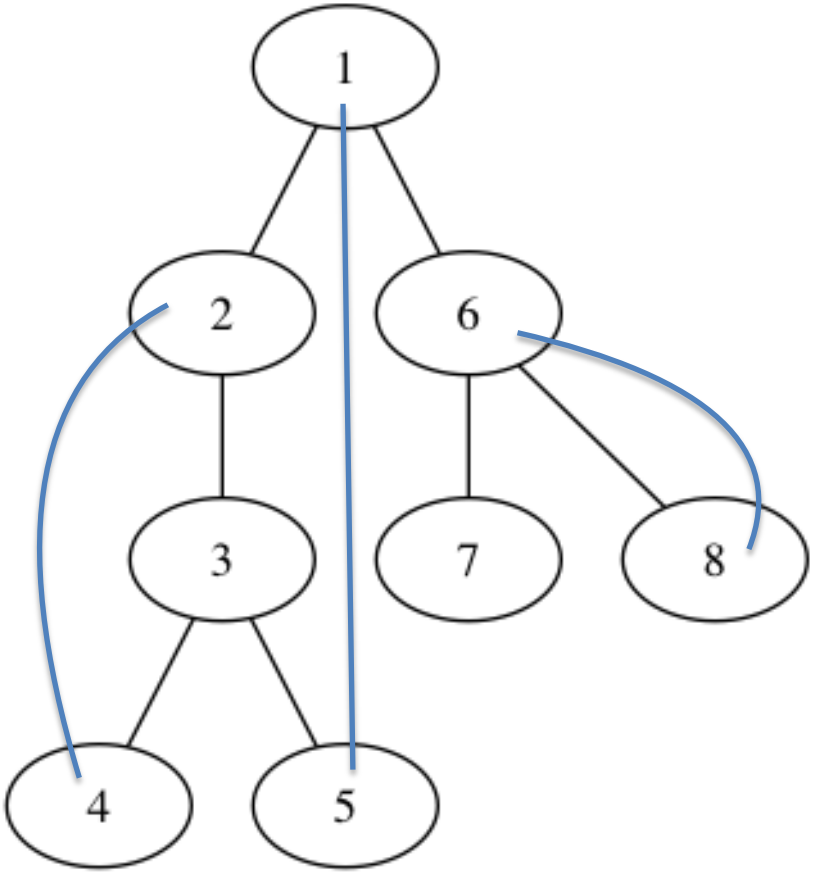
lowlinkというものを使用するアルゴリズムがあります

今回はimos法でやる方法を紹介します

D問題　橋の列挙法

AtCoder

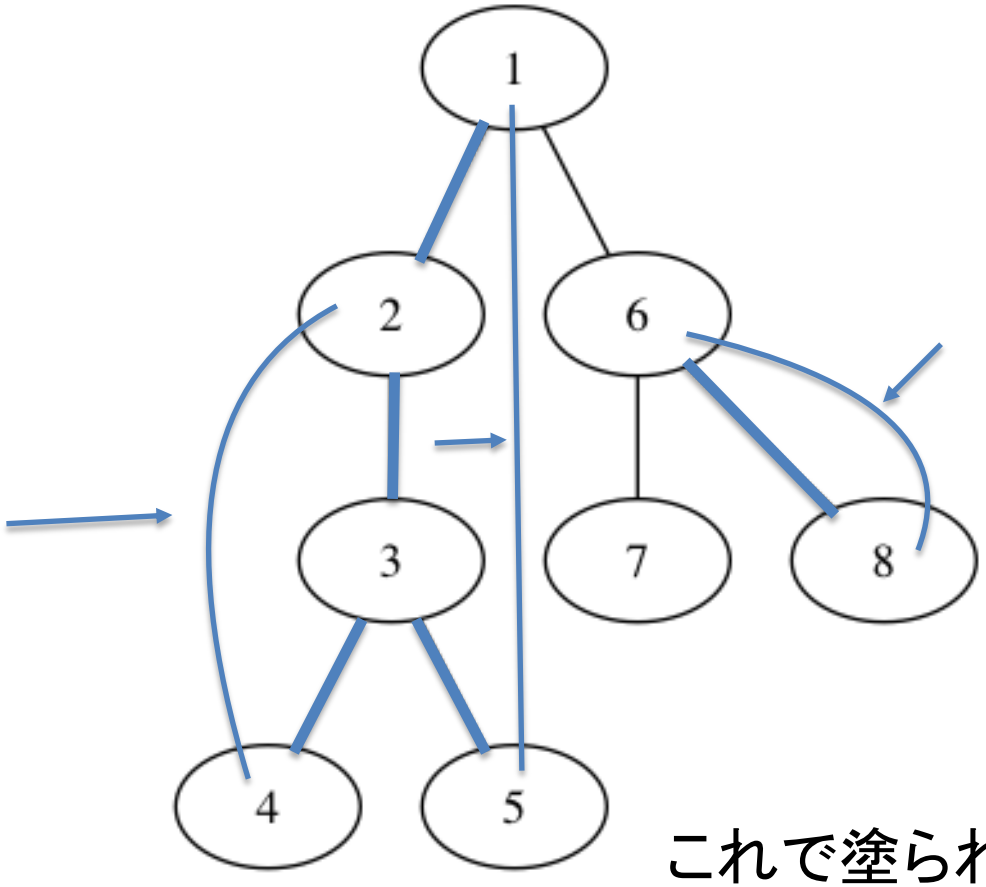
まずはdfs木を構築し、辺を木に使われる辺と後退辺に分類します



D問題　橋の列挙法

AtCoder

後退辺ごとに、対応するパス上の辺を塗る

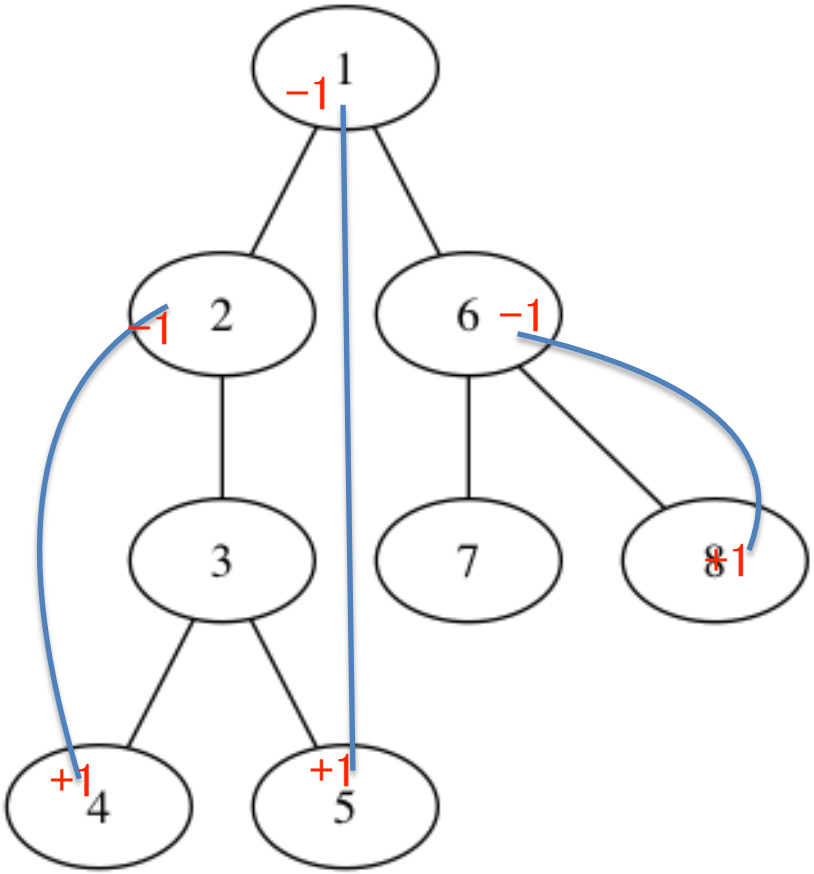


これで塗られなかった辺が橋

D問題　橋の列挙法

AtCoder

塗るのは愚直にやればO(NM)ぐらいだが, imos法でO(N+M)にできる



AtCoder

お疲れ様でした！

AtCoder Regular Contest 038
解説

AtCoder株式会社

はじめに

今回の問題は４問とも（２人）ゲーム系の問題でした

ゲーム系の問題は大体、以下いずれかで解くことが出来ます

・後ろから探索する

・Grundy数を求める

・adhoc（≒独特な）な必勝法を見つける

ARC 038 A問題 解説
「カードと兄妹」

snuke

問題概要

カードが N 枚あり、カード i には整数 A[i] が書かれている

２人で交互に１枚ずつ取っていく

取ったカードに書かれた数の和がスコアとなる

２人とも自分のスコアを最大化しようとするとき、

先手のスコアはいくらになるか？

～制約～

1 ≦ N ≦ 1000

考察

各ターンではプレイヤーはどのカードを取ればいいか？

→ 残っているカードのうち書かれている数が最大のもの！

解法

A[1~N] を降順にソートしておく

A[1]+A[3]+A[5]+…を求める

タイプ

A 問題は、分類するほど複雑なゲームではありませんが、

あえて言うならば３つ目のタイプでしょうか

- 後ろから探索する
- Grundy数を求める
- adhocな必勝法を見つける

ARC 038 B問題 解説

「マス目と駒」

snuke

問題概要

H×Wのマス目があり、いくつかのマスには障害物がある

最初、駒を左上のマスにおき、２人で交互に動かす

動かす先は、１つ下・１つ右下・１つ右のいずれかのマス

障害物のあるマスや、盤外に駒を動かすことはできない

駒を動かせなくなった方の負け

先手と後手のどちらが勝つか？

～制約～

2 ≦ H,W ≦ 100

部分点（30点）- 制約

$2 \leqq H,W \leqq 4$

盤がとても小さい

部分点（30点）- 解法

全探索をする

```
judge(i, j): # (i,j)に駒がある状態から開始したときの勝敗
  if (i,j) が盤外 or 障害物: return 勝ち
  if judge(i+1,j) == 負け: return 勝ち
  if judge(i+1,j+1) == 負け: return 勝ち
  if judge(i,j+1) == 負け: return 勝ち
  return 負け
```

judge(1,1)が答えとなる

（盤外と障害物を2行目のように処理すると実装が楽になります）

部分点（30点） - 計算量

O(分岐数^再帰の深さ) で抑えられる

再帰の深さは高々 6 （盤面が4x4以下だから）で、

各ステップでは高々 3 通りにしか分岐しないため、

この部分点では間に合う

満点 - 制約

$2 \leqq H,W \leqq 100$

盤が大きい

満点 - 解法

メモ化再帰にする

```
mem[1000][1000] を「未定」で初期化
judge(i, j): # (i,j)に駒がある状態から開始したときの勝敗
  if (i,j) が盤外 or 障害物: return 勝ち
  if mem[i][j] != 未定: return mem[i][j]
  result = 負け
  if judge(i+1,j) == 負け: result = 勝ち
  if judge(i+1,j+1) == 負け: result = 勝ち
  if judge(i,j+1) == 負け: result = 勝ち
  return mem[i][j] = result
```

満点 - 計算量

再帰呼び出しの回数を O(HW) で抑えることができる

計算量も O(HW) となり、間に合う

タイプ

B 問題は、１つ目のタイプでしょうか

・後ろから探索する

・Grundy数を求める

・adhocな必勝法を見つける

ソースコードの見た目は前から探索しているように見えますが、実際に勝敗が確定していくのは後ろの状態からなので、「後ろから探索する」ということにしています

ARC 038 C問題 解説

「茶碗と豆」

snuke

問題概要

N 個の茶碗があり、茶碗 i には整数C[i]が書かれている

茶碗 i には A[i] 個の豆が入っている

２人で交互に、茶碗 0 以外から豆を 1 つ選んで移動させていく

茶碗 i から選んだ場合は 茶碗 i-C[i] ～茶碗 i-1 のどれかに移す

豆を選べなくなった方の負け

先手と後手のどちらが勝つか？

～制約～

2 ≦ N ≦ 10^5

0 ≦ A[i] ≦ 10^9

部分点１（60点）- 制約

$2 \leqq N \leqq 100$

$0 \leqq A[i] \leqq 10$

茶碗も豆も少ない

部分点１（60点）- 解法

grundy数を求める

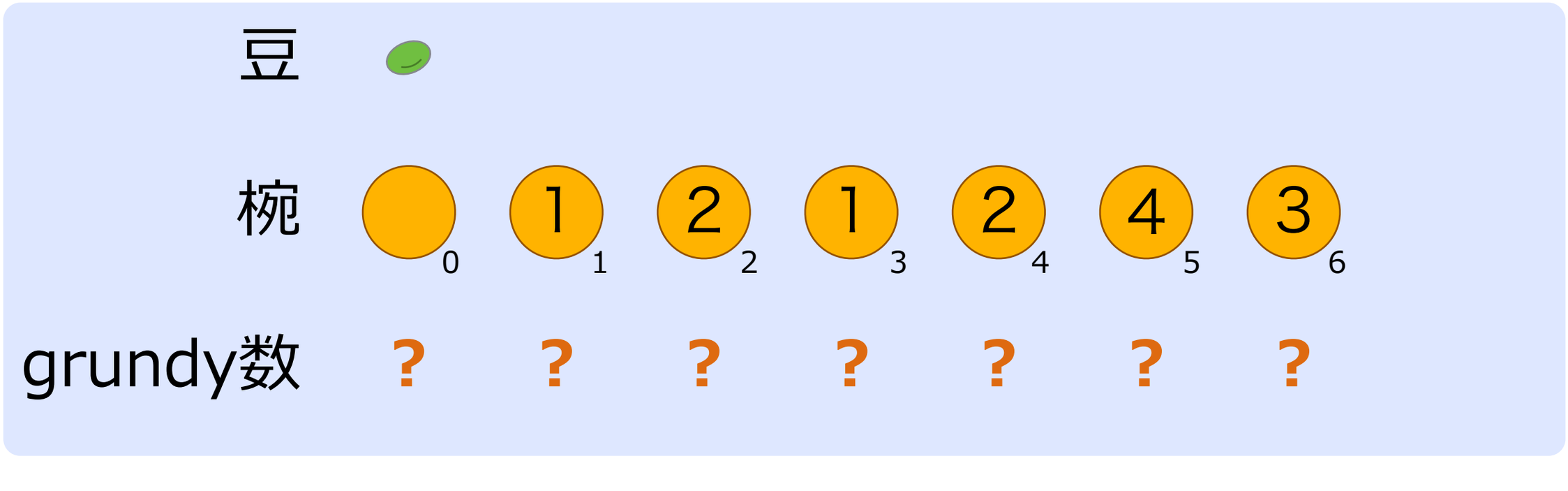
Grundy数

grundy数とは

> ある状態から 1 回の遷移で行ける状態の
> grundy数の集合に含まれない最小の非負整数

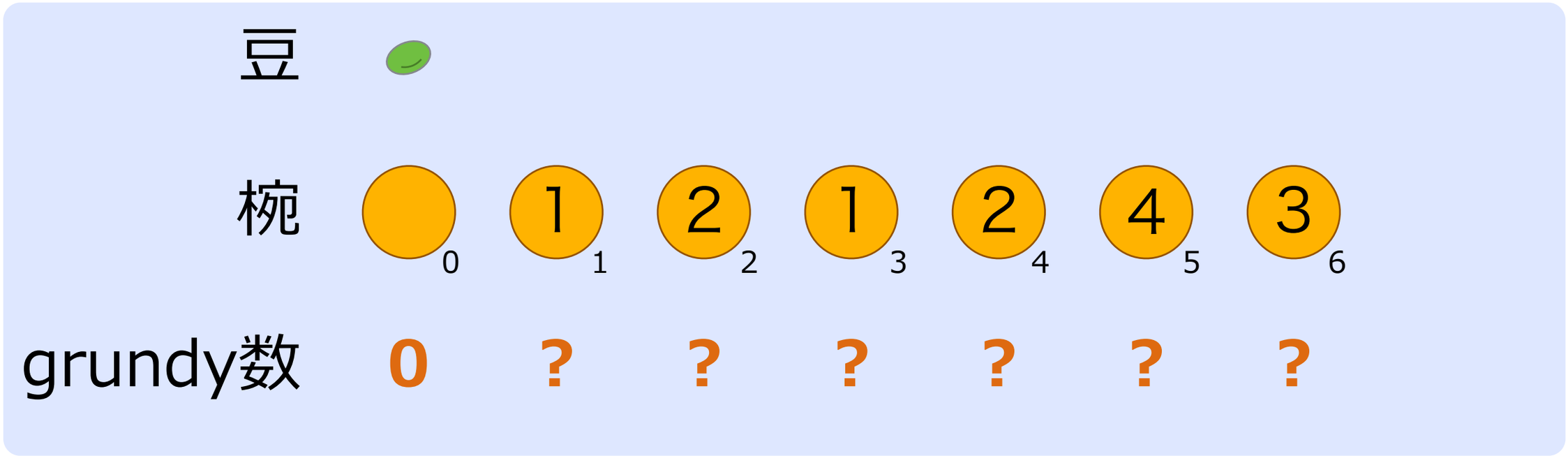
なにやら複雑そうなので、例を出して計算してみます

Grundy数



豆が１つだけのときを考えます

Grundy数



椀０に１つ豆がある状態のgrundy数を求めます

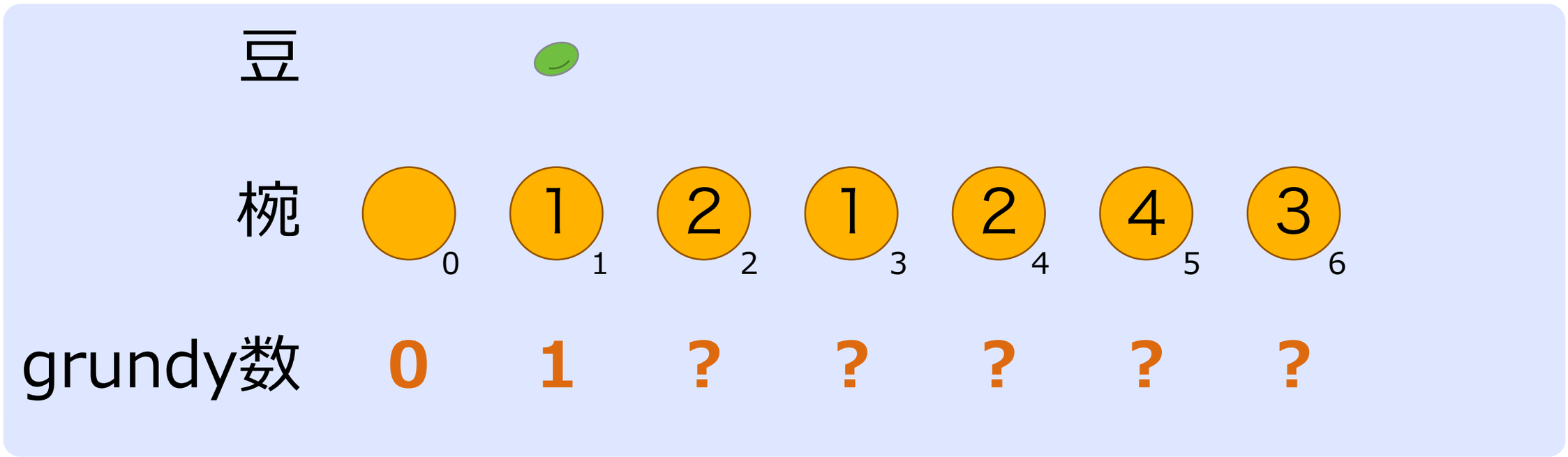
豆を移す操作がこれ以上できないため、

遷移先のgrundy数の集合は {}（空集合）となります

集合に含まれない最小の０以上の整数は０なので、

この状態のgrundy数は**0**となります

Grundy数



椀１に１つ豆がある状態のgrundy数を求めます

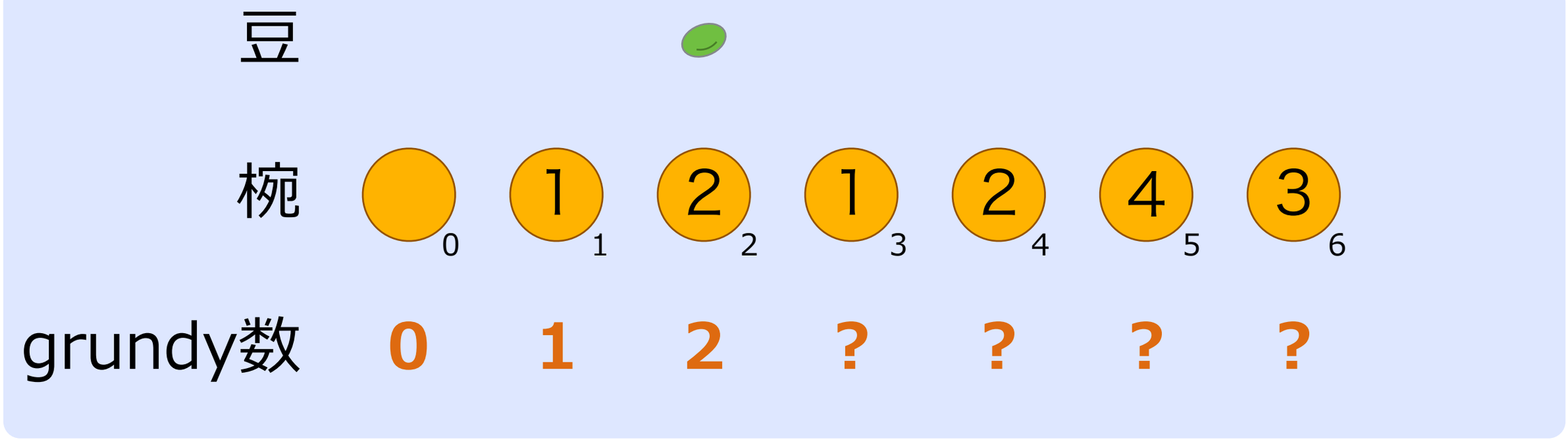
椀１からは椀０に豆を移せるため、

遷移先のgrundy数の集合は { **0** } となります

集合に含まれない最小の０以上の整数は１なので、

この状態のgrundy数は **1** となります

Grundy数



椀２に１つ豆がある状態のgrundy数を求めます

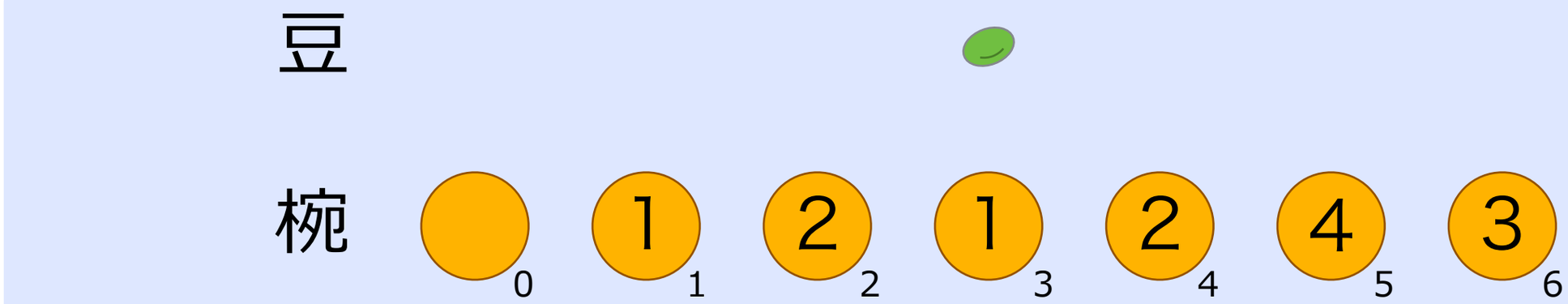
椀２からは椀０と椀１に豆を移せるため、

遷移先のgrundy数の集合は {**0**, **1**} となります

集合に含まれない最小の０以上の整数は２なので、

この状態のgrundy数は**2**となります

Grundy数



椀３に１つ豆がある状態のgrundy数を求めます

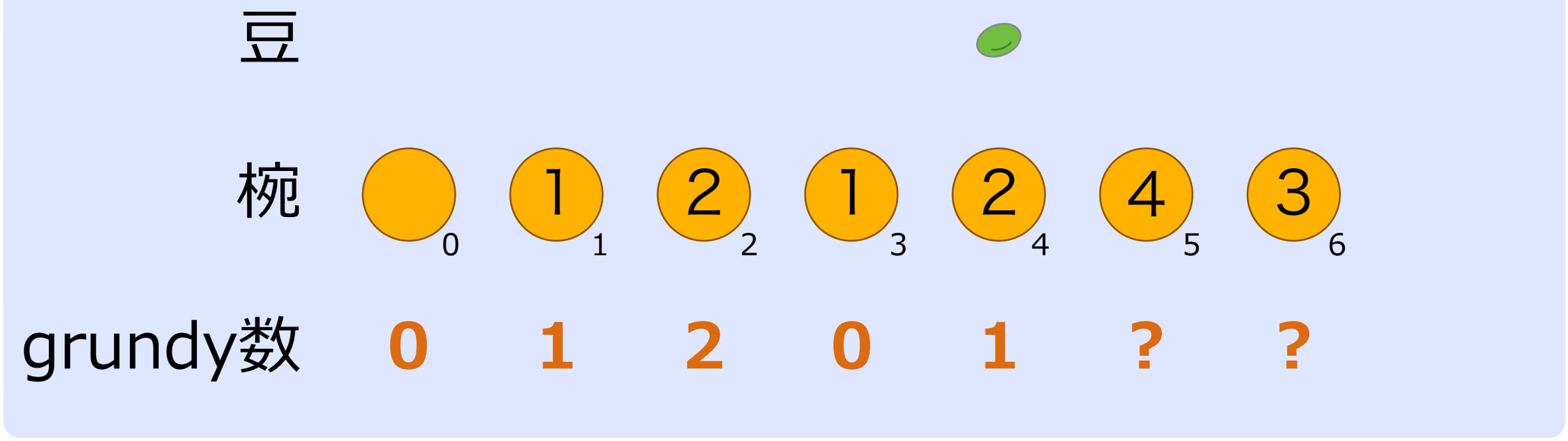
椀３からは椀２に豆を移せるため、

遷移先のgrundy数の集合は｛**2**｝となります

集合に含まれない最小の０以上の整数は０なので、

この状態のgrundy数は**0**となります

Grundy数



椀４に１つ豆がある状態のgrundy数を求めます

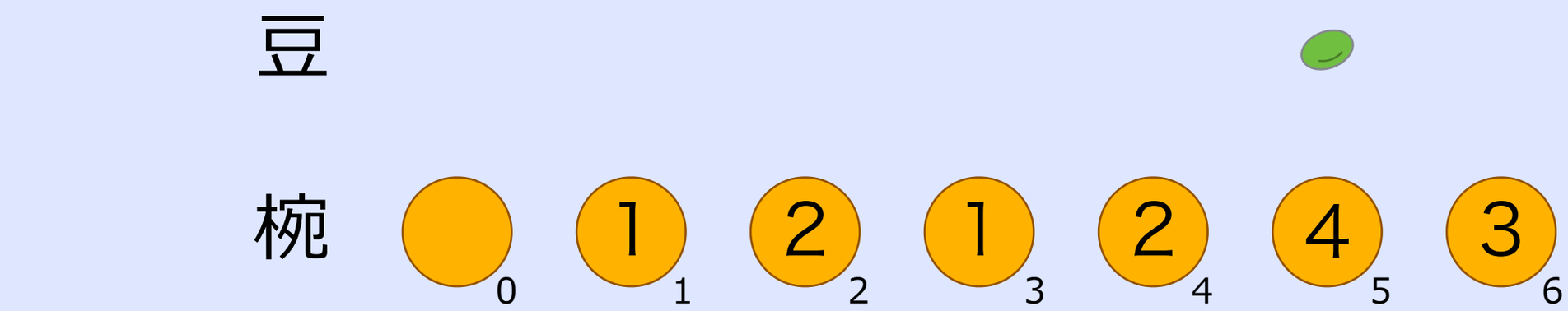
椀４からは椀２と椀３に豆を移せるため、

遷移先のgrundy数の集合は {**2**,**0**} となります

集合に含まれない最小の０以上の整数は１なので、

この状態のgrundy数は**1**となります

Grundy数



椀５に１つ豆がある状態のgrundy数を求めます

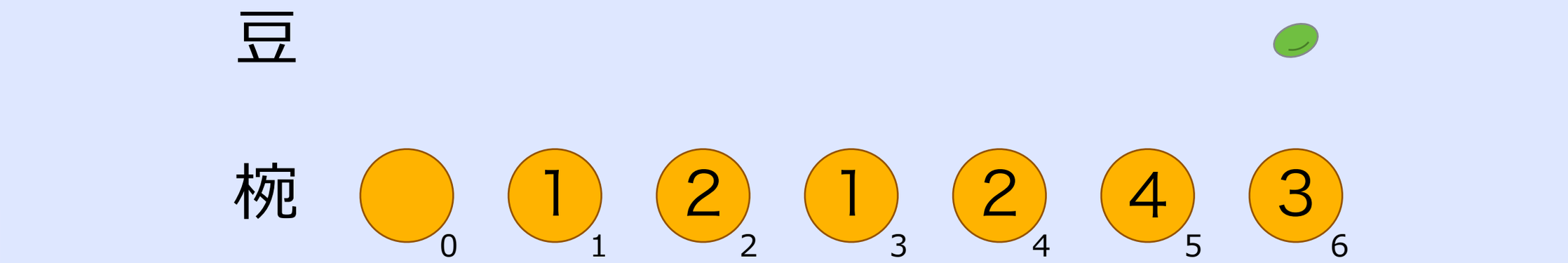
椀５からは椀１と椀２と椀３と椀４に豆を移せるため、

遷移先のgrundy数の集合は {**0**, **1**, **2**} となります

集合に含まれない最小の０以上の整数は３なので、

この状態のgrundy数は**3**となります

Grundy数



椀６に１つ豆がある状態のgrundy数を求めます

椀６からは椀３と椀４と椀５に豆を移せるため、

遷移先のgrundy数の集合は {**0**, **1**, **3**} となります

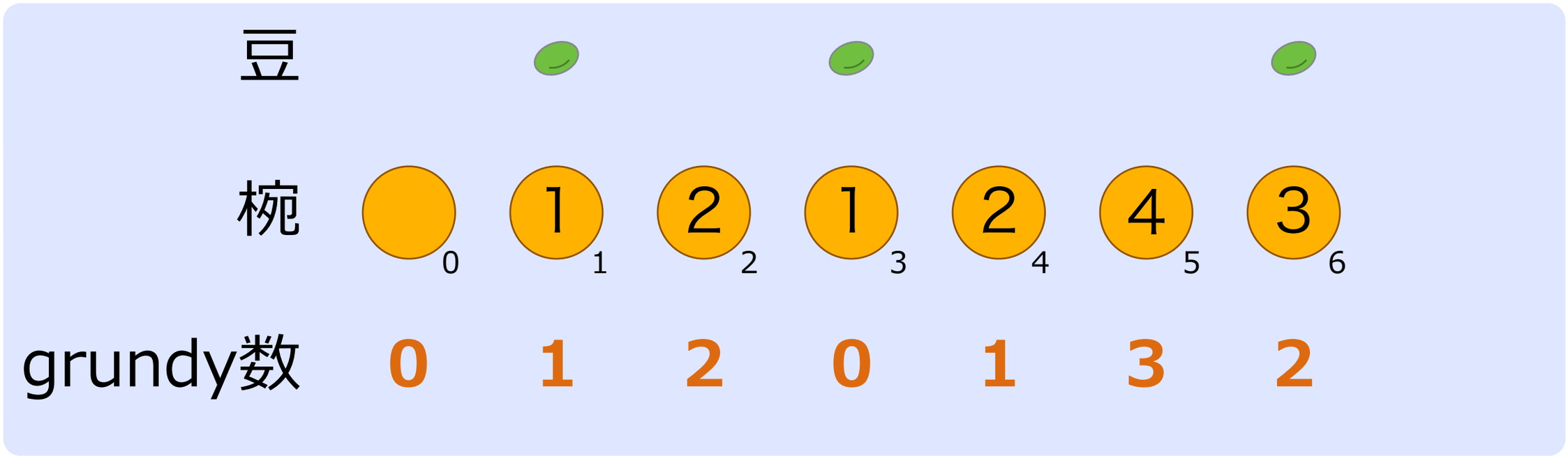
集合に含まれない最小の０以上の整数は２なので、

この状態のgrundy数は**2**となります

Grundy数

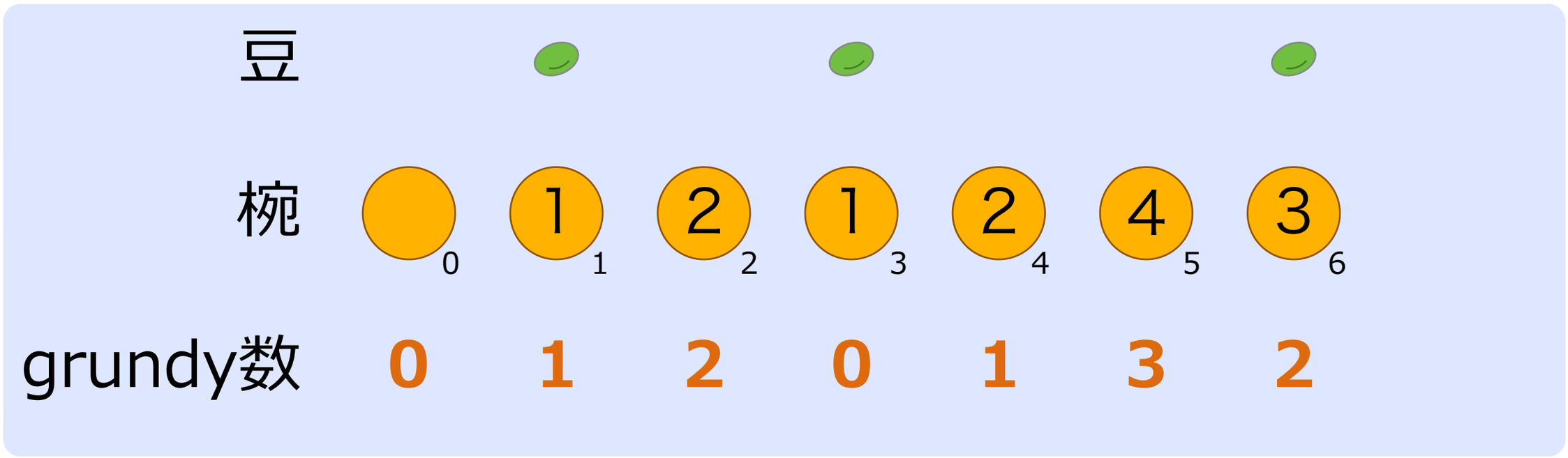
このようにしてgrundy数を求めることができます

Grundy数



豆が２つ以上ある場合はどうすればいいでしょうか

Grundy数

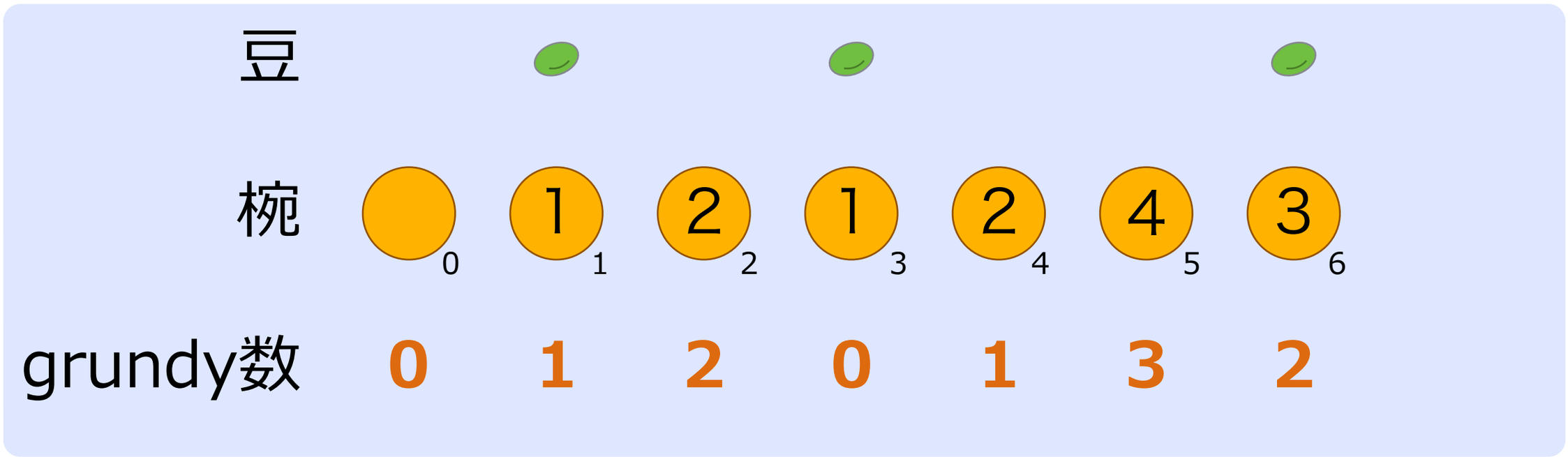


定義どおりに状態の遷移を考えていけば、

grundy数を求めることができます

しかし、実はそのようなことをしなくても求められます

Grundy数



・椀１に１つ豆がある状態のgrundy数： **1**

・椀３に１つ豆がある状態のgrundy数： **0**

・椀６に１つ豆がある状態のgrundy数： **2**

これらの **xor** の値がこの状態のgrundy数となっています

つまり、 **1** xor **0** xor **2** ＝ **3** がこの状態のgrundy数です

Grundy数

grundy数を求めると何が嬉しいのでしょうか？

- grundy数が 0 でない状態からは、
  必ずgrundy数が 0 の状態に遷移できる
- grundy数が 0 の状態からは、
  grundy数が 0 の状態に遷移できない

という性質から、あるゲームの状態のgrundy数が

- 0 でないならば先手の勝ち
- 0 ならば後手の勝ち

ということが言えるのです

部分点１（60点）- 解法

「椀 i に１つ豆がある状態のgrundy数」を

　i ＝ 0 から順に i ＝ N-1 まで求めて、

各豆についてのgrundy数のxorを求める

この値が１ならば先手の勝ちで、０ならば後手の勝ち

部分点１（60点）- 解法

「椀 i に１つ豆がある状態のgrundy数」 = g[i] とする

g[i] を求めるときには、

・g[i-C[i]]～g[i-1] に含まれない最小の０以上の整数

を求める必要があります

→

「０が含まれるか」「１が含まれるか」・・・

を順番に調べていけば O(N^2) で求めることができます

「x が含まれるか」を調べるときに適切なデータ構造（例えばsetなど）を用いることで、O(N log N)やO(N)にできます

部分点１（60点）- 計算量

g[i] を求めるのに O(N^2) かけたとすると、

すべての g[i] を求める計算量は O(N^3)

また、豆の個数は最大で max(A[i])*(N-1) 個なので、

grundy数のxorをとる計算量は O(max(A[i])*N)

となり、この部分点では間に合います

部分点２（40点）- 制約

$2 \leqq N \leqq 100$

$0 \leqq A[i] \leqq 10^9$

茶碗は少ないが、豆は多い

部分点２（40点）- 解法

部分点解法１の解法だと、

すべての g[i] を求める計算量は O(N^3)

なので問題ないですが、

grundy数のxorをとる計算量は O(max(A[i])*N)

なので間に合いません

部分点２（40点）- 解法

xorの性質として、

・同じ数の xor は 0

というものがあります

例えば、3 xor 3 = 0 で、 99 xor 99 = 0 です

部分点２（40点）- 解法

grundy数のxorをとるとき、

同じ数のxorを大量にとっています

例えば A[i] = 5 のとき、

・g[i] xor g[i] xor g[i] xor g[i] xor g[i]

という計算をしていますが、これは、

・g[i] xor g[i] xor g[i] や、

・g[i]

と同じです

このように同じ数を２つずつ相殺させることを考えます

部分点２（40点）- 解法

つまり、A[i] がどのような数であろうと、

勝敗には A[i] の偶奇しか関係ないのです

部分点２（40点）- 計算量

grundy数のxorをとる計算量は、

A[i] が奇数であるような所の g[i] のxorをとるだけなので

O(N) で良いことになり、

この部分点は間に合います

満点（おまけ）- 制約

2 ≦ N ≦ 10^5

0 ≦ A[i] ≦ 10^9

茶碗も多い

上級者向けのおまけの制約です

満点（おまけ）- 解法

g[i] をもっと高速に求めなければなりません

g[i] は、

・g[i-C[i]]～g[i-1] に含まれない最小の 0 以上の整数

でした

満点（おまけ）- 解法

「g[k]~g[i-1] に l~r がすべて含まれるような最小のk」

という情報を持ったsegtreeを更新しつつg[i]を求めていく

（各グランディー数について、一番最後に現れた位置を持つイメージ）

このsegtreeで二分探索を行うことにより、

O(log N) で g[i] を求めることができます

計算量は O(N log N) となり、間に合います

タイプ

C 問題は、２つ目のタイプでしょうか

・後ろから探索する

・Grundy数を求める

・adhocな必勝法を見つける

ARC 038 D問題 解説
「有向グラフと数」

snuke

問題概要

N 頂点 M 辺の有向グラフがある

頂点 i には整数 X[i] が書かれている

最初、頂点１に駒を置き、２人で交互に辺に沿って動かす

駒を動かす代わりに「終了宣言」をすることもできる

終了宣言がされるか、後手が10^9回駒を動かした時点でゲームが終了し、そのときに駒のある頂点に書かれている整数がゲームのスコアとなる

先手がスコアを最大化しようとし、後手がスコアを最小化しようとするとき、ゲームのスコアはいくつになるか？

部分点（30点） - 制約

$2 \leqq N \leqq 1000$

$1 \leqq M \leqq 2000$

グラフが小さい

部分点（30点）- 解法

ターン数が 10^9 でも 10^9 - 1 でも結果は変わらない

ターン数がいくら大きくても、結局堂々巡りになり、
先手がどこかで妥協しなければならない

もっというと、ターン数は 2N ターンまででも変わらない

ゲームの状態は [駒がある頂点] と [手番] で決まり、

同じ状態に 2 回訪れるということは、堂々巡りになるということなので意味がない

部分点（30点）- 解法

[ターン][駒がある頂点][手番] という状態を考えて、

それぞれの状態からの結果を求める

手番が先手の時は、遷移先の状態の結果のうち最大を選び、

手番が後手の時は、遷移先の状態の結果のうち最小を選ぶ

というメモ化再帰探索をする

（min-max法 と呼ばれる手法です）

部分点（30点） - 計算量

状態数が O(ターン数 * 頂点数)

遷移数が O(ターン数 * 辺数)

となっているため、計算量は O (N*(N+M)) となり、

この部分点では間に合う

満点 - 制約

$2 \leqq N \leqq 100,000$

$1 \leqq M \leqq 200,000$

グラフが大きい

満点 - 解法

・スコアを x 以上にすれば先手の勝ち
・スコアを x 未満にすれば後手の勝ち

というゲームの結果を計算できたとする

x が小さければ小さいほど先手が勝ちやすく、

x が大きければ大きいほど先手が勝ちにくいため、

x の値で二分探索ができる

満点 - 解法

・スコアを x 以上にすれば先手の勝ち
・スコアを x 未満にすれば後手の勝ち

というゲームは、

書かれている数が x 以上の頂点を青く塗り、

書かれている数が x 未満の頂点を白く塗ったと思えば、

・青い頂点で終了すれば先手の勝ち
・白い頂点で終了すれば後手の勝ち

というゲームになる

満点 - 解法

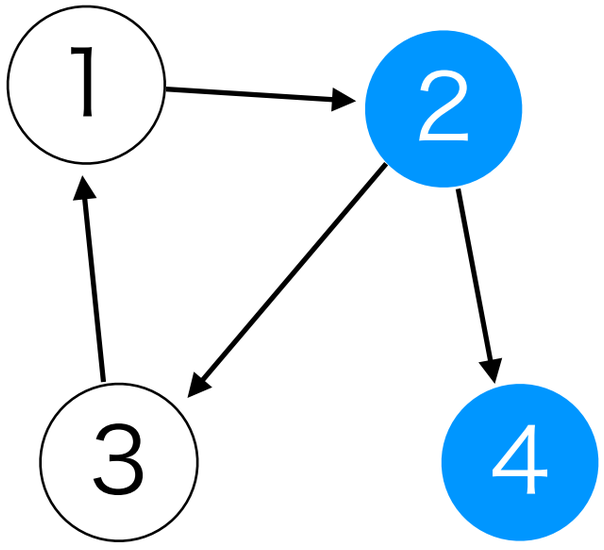
・黒い頂点で終了すれば先手の勝ち
・白い頂点で終了すれば後手の勝ち

というゲームの結果を計算できれば良くなった

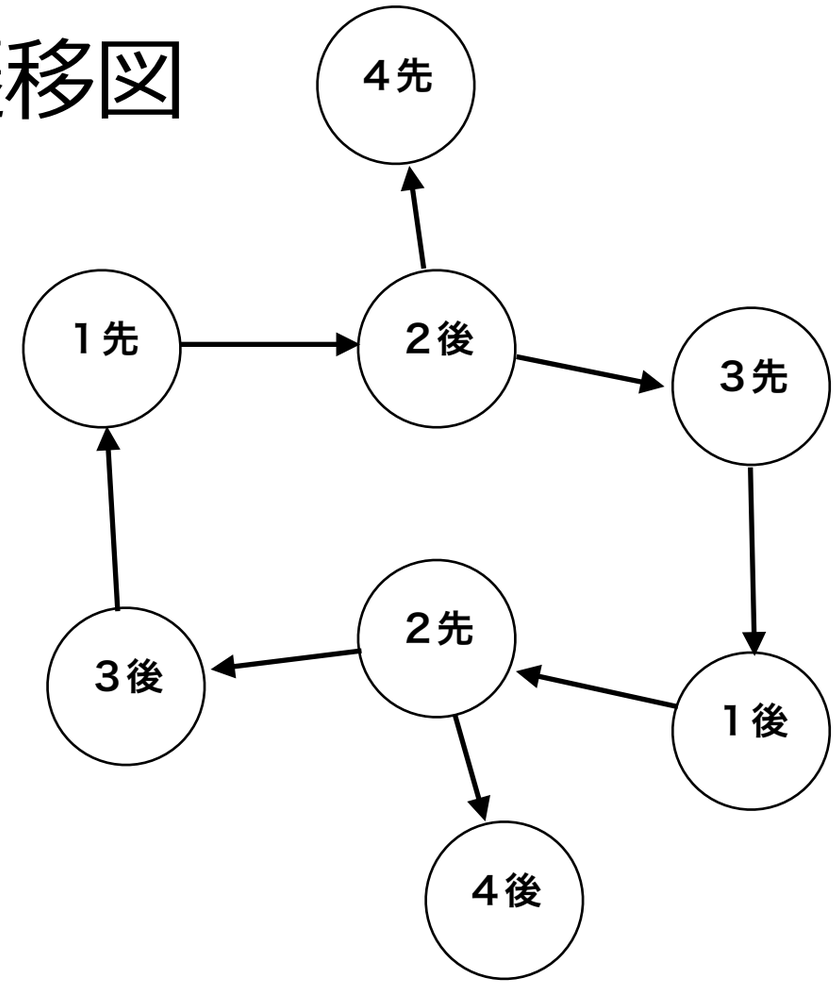
満点 - 解法

状態が [駒のある頂点][手番] の状態遷移図を考える

元のグラフ



状態遷移図



満点 - 解法

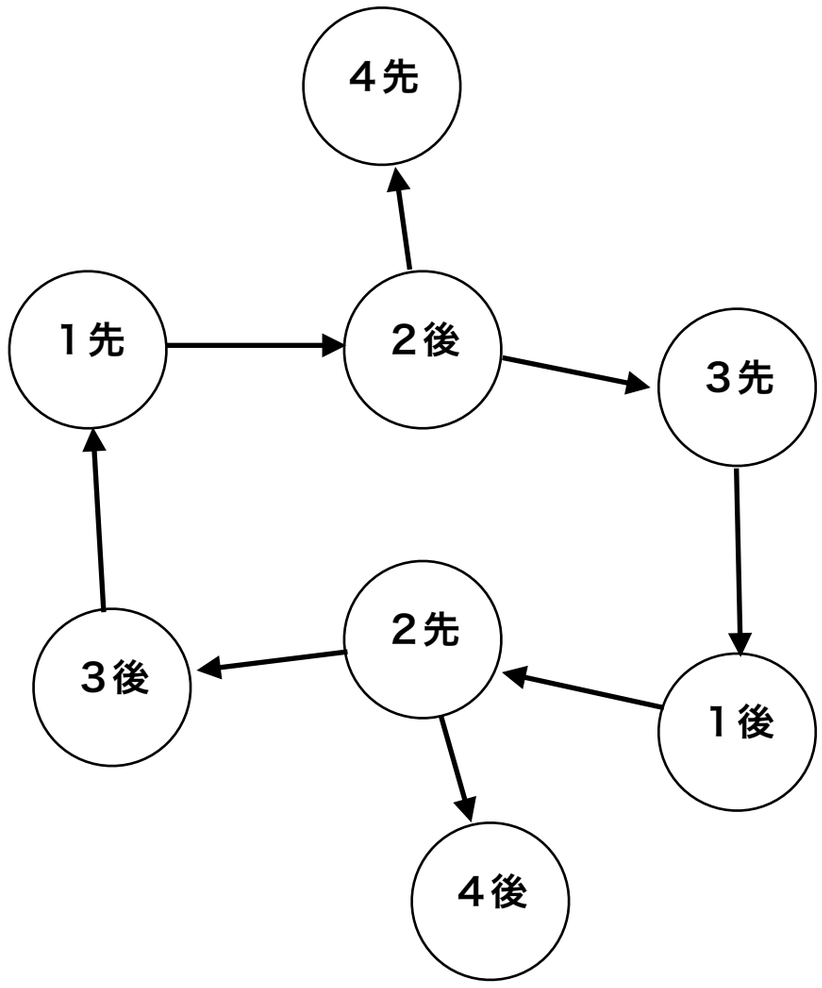
状態遷移図の頂点の色を

黒：手番プレイヤーの勝ち

灰：未定

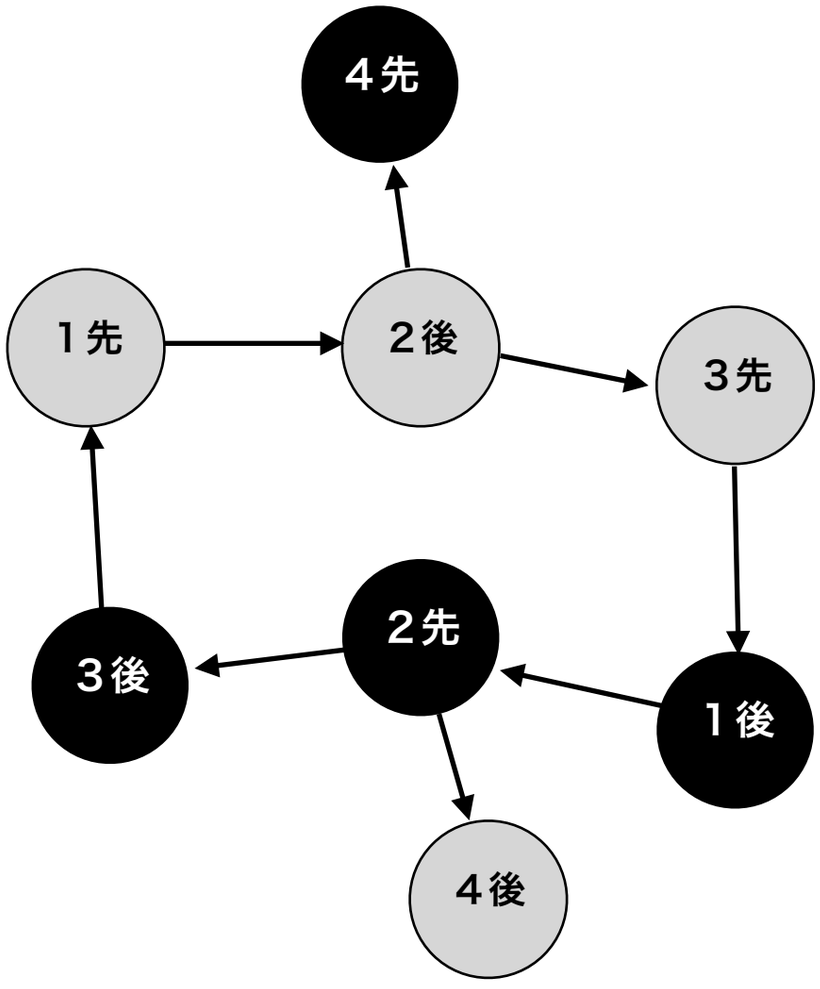
白：手番プレイヤーの負け

とします



満点 - 解法

最初はこうなっています



満点 - 解法

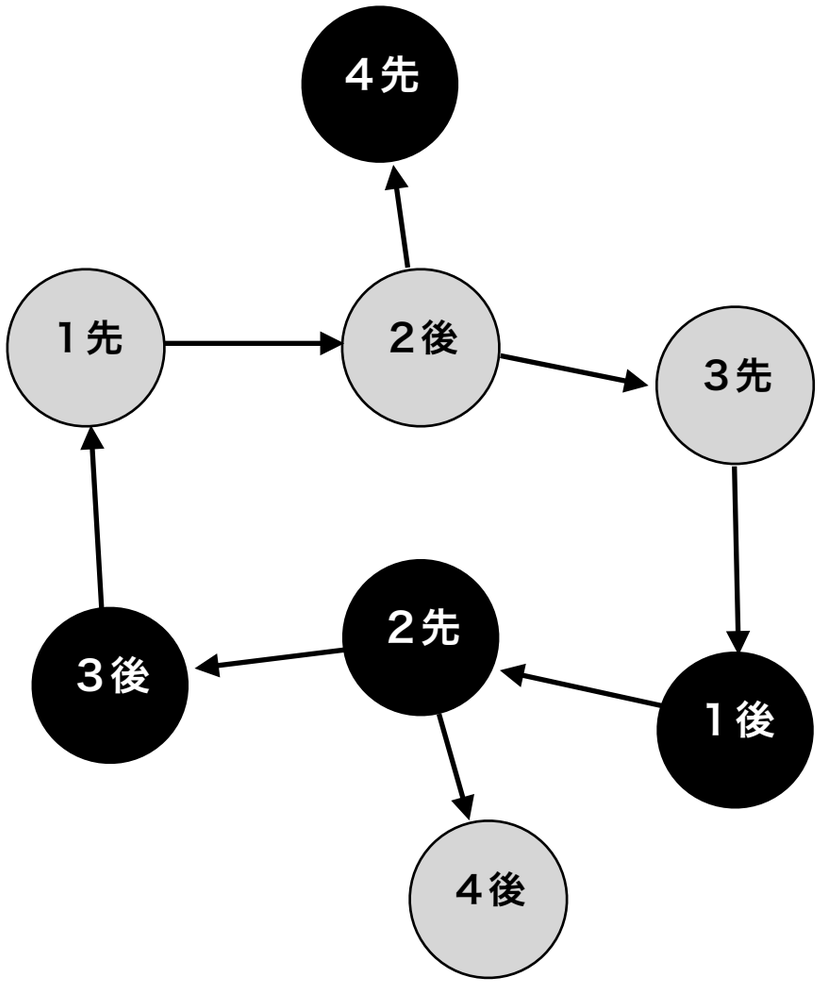
これを、

・遷移先に白があれば黒にする

・遷移先が全て黒なら白にする

というルールにそって、

色付けしていきます



満点 - 解法

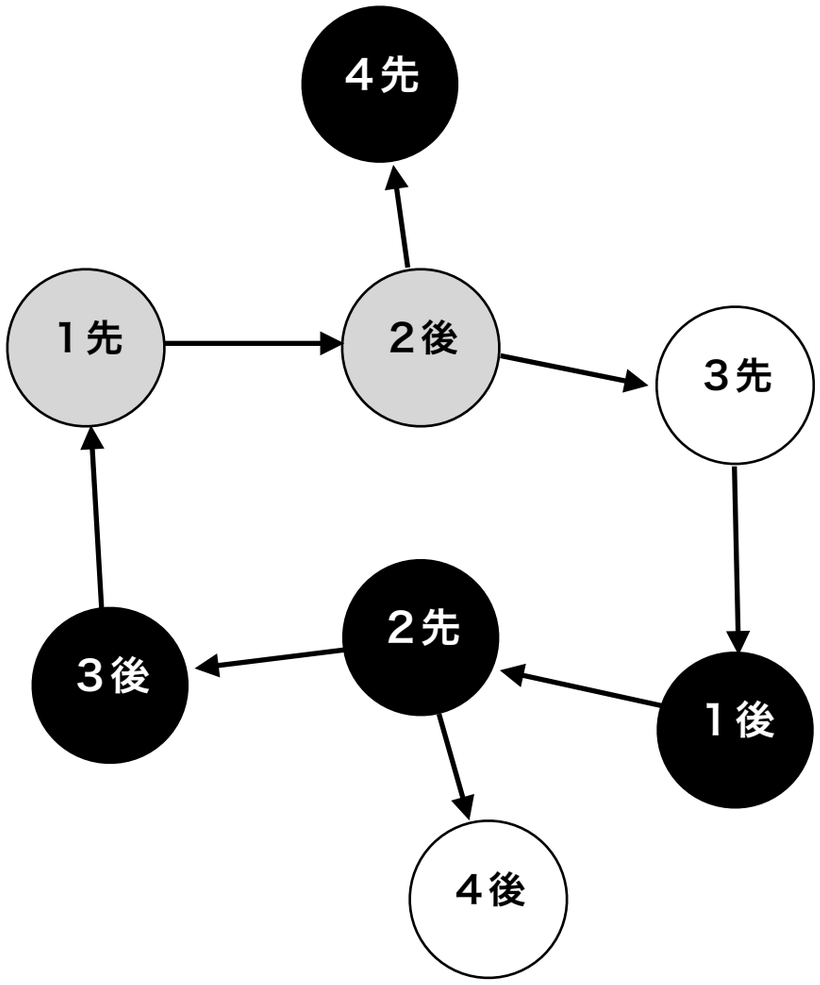
これを、

・遷移先に白があれば黒にする

・遷移先が全て黒なら白にする

というルールにそって、

色付けしていきます



満点 - 解法

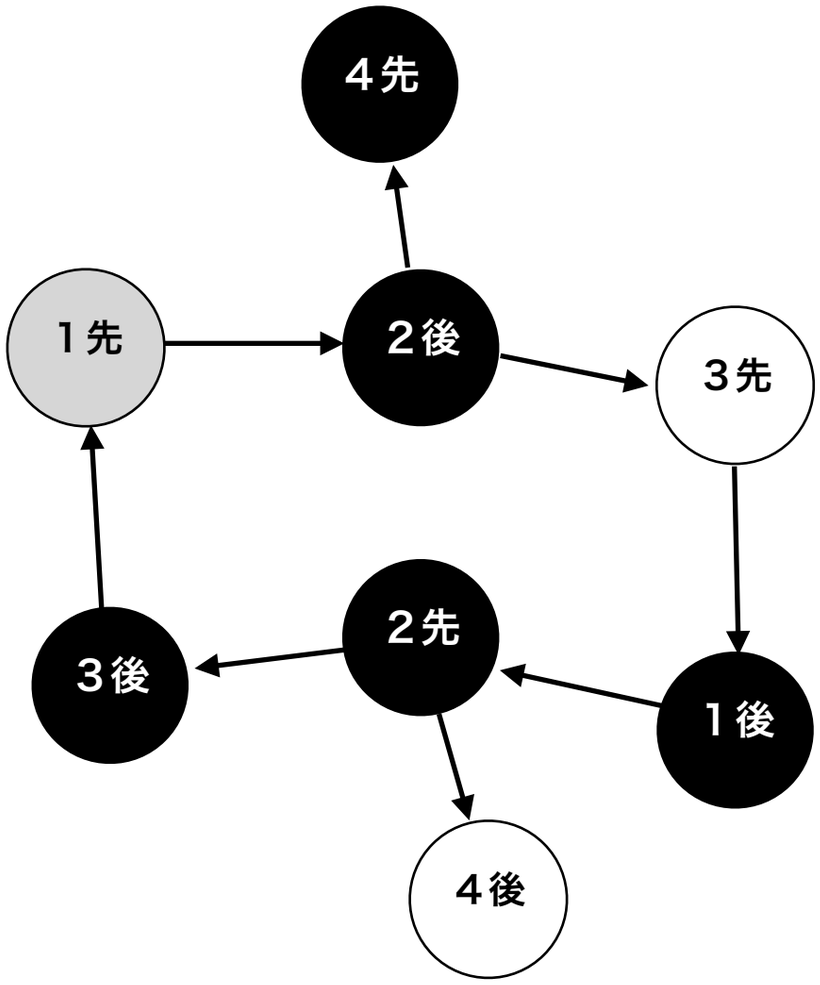
これを、

・遷移先に白があれば黒にする

・遷移先が全て黒なら白にする

というルールにそって、

色付けしていきます



満点 - 解法

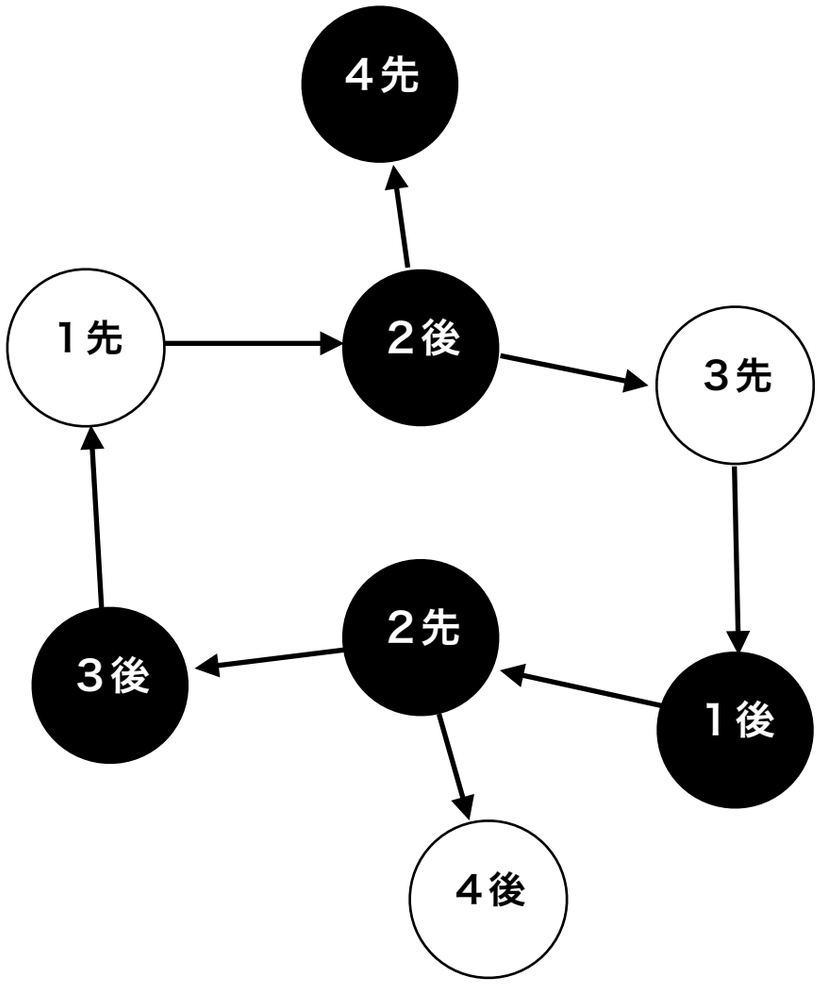
これを、

・遷移先に白があれば黒にする

・遷移先が全て黒なら白にする

というルールにそって、

色付けしていきます



満点 - 解法

このルールで塗って行ったときに、最終的に

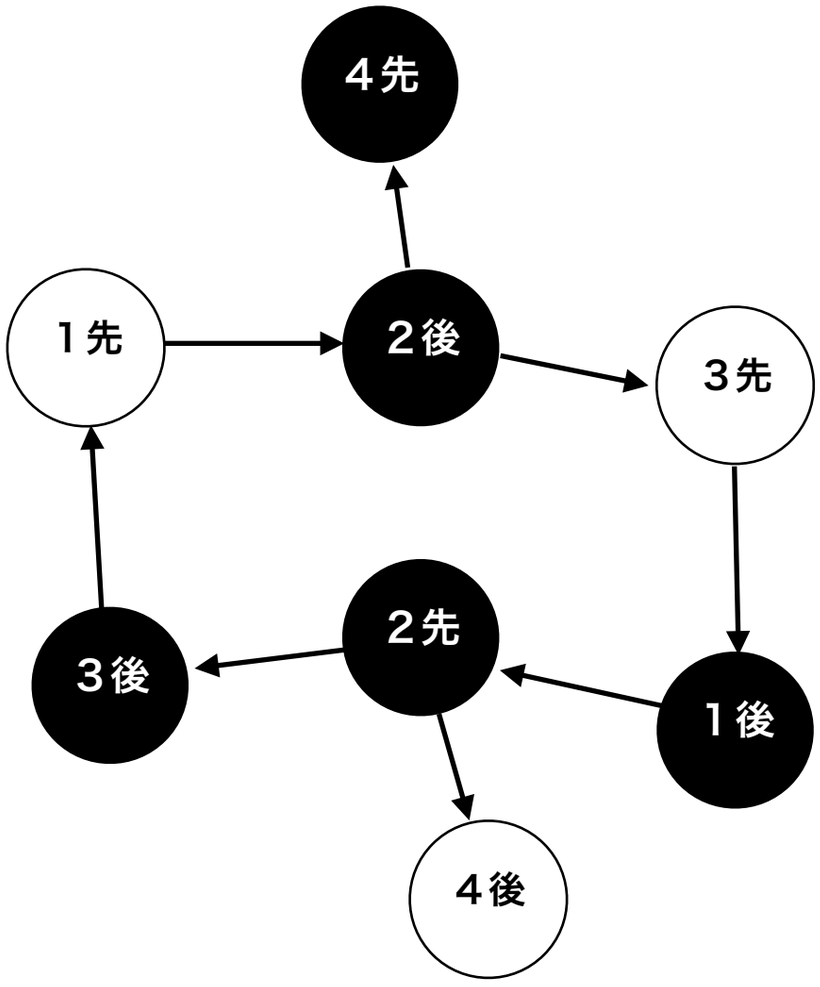
黒の頂点：勝ち

白の頂点：負け

灰の頂点：引き分け

となります

（灰の頂点の場合は堂々巡りになります）

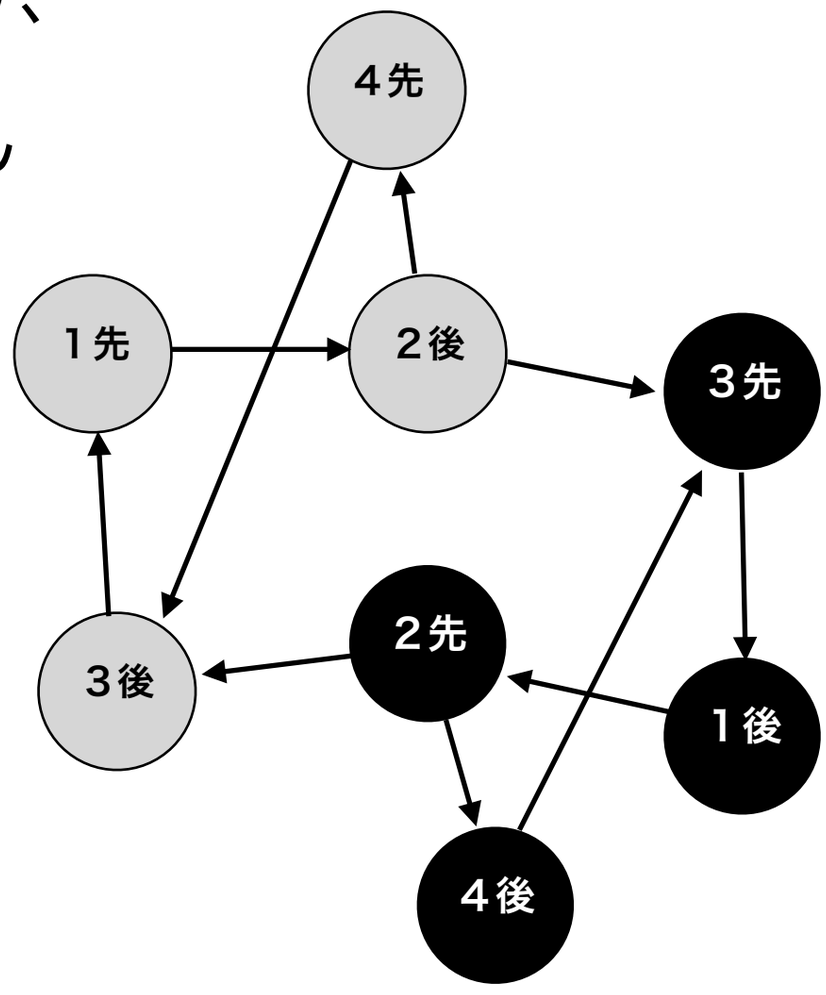


満点 - 解法

灰の頂点は例えば以下のような状況で出てきます

灰の頂点のサイクルができており、

これ以上ルールを適用できません



満点 - 解法

このような色塗りの手順は、BFSの要領で行えばよいです

「遷移先が全て黒」というのは、

「遷移先の黒の個数が出次数と等しい」と考えて実装します

このような手法で勝敗を判定するアルゴリズムを

後退解析

と呼ぶそうです

どうぶつしょうぎの解析などにも使われている手法です

満点 - 計算量

計算量は、二分探索＋後退解析で

O((N+M) log max(X))

となり間に合います

タイプ

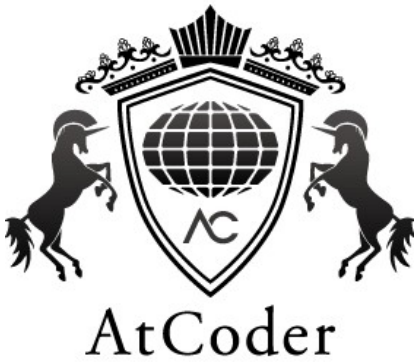
D 問題は、１つ目のタイプでしょうか

・後ろから探索する

・Grundy数を求める

・adhocな必勝法を見つける

AtCoder Regular Contest 037
解説



AtCoder株式会社 代表取締役
高橋 直大

競技プログラミングを始める前に

AtCoder

- 競技プログラミングをやったことがない人へ
  - まずはこっちのスライドを見よう!
  - http://www.slideshare.net/chokudai/abc004

A問題　全優

1.問題概要
2.アルゴリズム

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

- N科目の試験を受ける
- 勉強しなければ科目 i では m_i 点を取る

- 1分間勉強することで1つの科目の得点を
- 1点上げられる

- 全科目で80点以上を取るには
- 何分の勉強が必要？

- (科目数) $\leqq 25$, $1 \leqq m_i \leqq 100$

A問題　アルゴリズム

AtCoder

- m_i が80未満の科目は 80 - m_i 分勉強する
- 必要がある
- m_i が80以上の科目の勉強時間は 0 分でよい

- 全科目についてこれを合計したものが答え

- 各科目の勉強時間を $\max(0, 80 - m_i)$ と
- 書くこともできる

AtCoder

B問題　バウムテスト

1.問題概要
2.アルゴリズム

©AtCoder Inc. All rights reserved.

B問題　問題概要

AtCoder

- N頂点M辺の無向グラフが与えられる

- このグラフの連結成分のうち木であるもの
- （閉路がないもの）の個数を求めよ

- $N \leqq 100$, $M \leqq N(N-1)/2$

B問題　アルゴリズム

AtCoder

- 複数の異なる方針のうち1つを紹介する

- とりあえず連結成分を列挙する必要がある

- 「まだ見ていない頂点を選んでそこからDFS
- （深さ優先探索）をする」のを繰り返すとよい

- こうして連結成分を得るついでに
- 閉路がないか判定する

B問題　アルゴリズム

AtCoder

- DFSをする上で、無限ループを防ぐために
- 「すでに訪れた」頂点をマークしていき、
- 二度訪れないようにするが…

- 連結成分に閉路がなければ「すでに訪れた
- 頂点にまた訪れようとする」ということが
- 起こらないので、これで木の判別が可能

- （DFSで直前に訪れた頂点を覚えて
- おかないと、誤検知してしまうので注意）

AtCoder

C問題　壁抜け

1.問題概要
2.アルゴリズム

©AtCoder Inc. All rights reserved.

C問題　問題概要

AtCoder

- N^2 ます計算（掛け算）をする

- N^2 個の計算結果を昇順に
- ソートしたとき、小さい方から
- K番目にくる値は？

	3	5
2	6	10
3	9	15

- N ≦ 30000
- (行、列の値) ≦ 10^9

C問題　アルゴリズム (部分点)

AtCoder

- 5点: N ≦ 10
- 100マス計算
- 全部計算してソートできる

C問題　アルゴリズム (満点)

AtCoder

- 100点: N ≦ 30000

- 9億マス計算

- 全部計算するとメモリが足りない、
- もし足りてもソートは確実に間に合わない

- なんとかして K 番目の値「だけ」を求めたい

C問題　アルゴリズム (満点)

AtCoder

- そもそも「小さい方からK番目の値がXである」
- とはどういうことか？

- ずばり「X-1以下の数はK個未満しかないが、
  - X以下の数はK個以上ある」ということ

  - 1,1,2,2,2,2,2,4,4
  - 小さい方から7番目は2
  - 1以下の数は 2個 < 7個
  - 2以下の数は 7個

C問題　アルゴリズム (満点)

AtCoder

- 「X-1以下の数はK個未満しかないが、
- X以下の数はK個以上ある」

- 別の言い方では
- 「X以下の数がK個以上あるような最小のX」
- が小さい方からK番目の数

- 二分探索が使える

C問題　アルゴリズム (満点)

AtCoder

- 「Xを決めたとき、X以下の数は
- K個以上あるか？」という問題を繰り返し
- 解くことになる

- 実際に数えてK個以上か確かめればよい

- どうやって数えるか？

C問題　アルゴリズム (満点)

AtCoder

- 各行を一つずつ見ていく

- a_i * b_j ≦ K ⇔ b_j ≦ K / a_i (切り捨て)

- i行目に含まれるK以下の数の個数は、
- b_1, b_2, …, b_N のうち K / a_i 以下で
- あるようなものの個数と一致

- b をソートしておけば（再び）二分探索で
- O(log(N)) 時間で求まる

C問題　アルゴリズム (満点)

AtCoder

- 総計算量:
- O(log(MAX_A * MAX_B) * N * log(N))

- log(10^18) * 3万 * log(3万) = 2700万

- 速い言語でないと厳しいが、ゆるくすると
- C++で本当にソートして通ってしまうので
- 許してください

AtCoder

D問題　Chaotic Polygons

1.問題概要
2.アルゴリズム

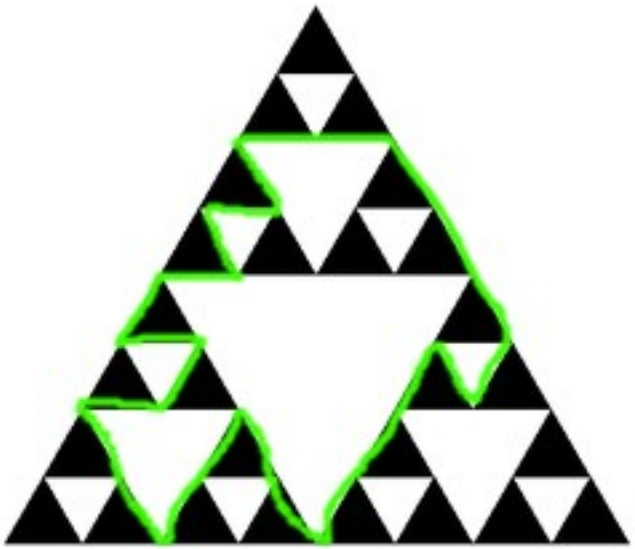
©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

- こういう→
- 多角形の個数を mod 10億7 で
- 求めよ

- L ≦ 10^5
- (右の図の正三角形の数を
- 3^L とする)



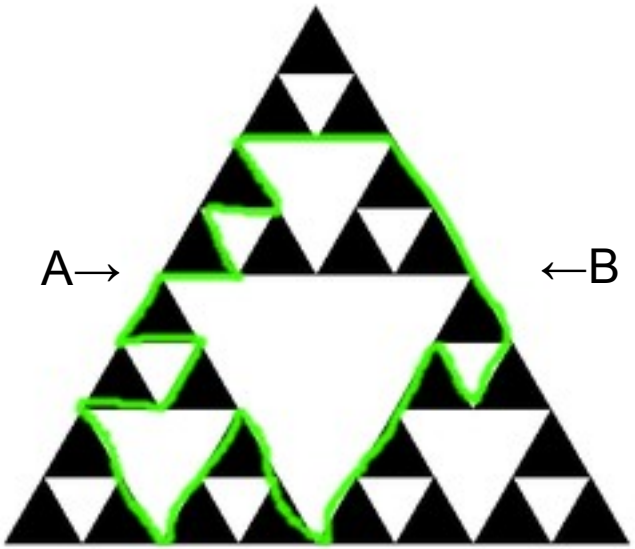
D問題　アルゴリズム

AtCoder

- 求める答え（レベルLの図形に含まれる
- 多角形の個数）を f(L) とおく

- 中央の正三角形に注目

- それを含まないような多角形の
- 個数は ３ * f(L-1)



- 含むような多角形の個数は、
- 図のA地点からB地点までの
- 「多角形の一部」の個数をg(L-1) として、g(L-1)^3

D問題　アルゴリズム

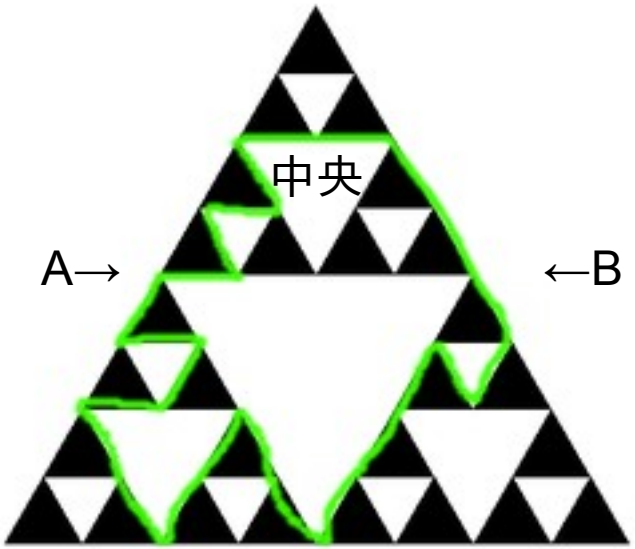
AtCoder

- g(L) を求めたい
- (地点AからBまで辿る「多角形の一部」
- （右の図はレベルL+1とする))

- やはり中央の正三角形に注目

- それを含まないような「多角形の
- 一部」の個数は先ほどと同様に
- 再帰で

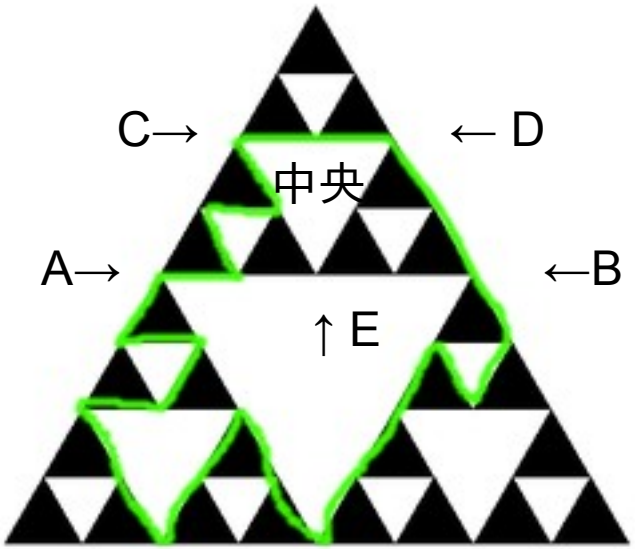
- 含む場合は…



D問題　アルゴリズム

AtCoder

- 含む場合は…
- A→C→D→B と辿る
  C→Dは g(L-1) 通り
- A→C と D→Bについては、
  単体では g(L-1) 通りだが
  両方とも E を通ってはいけない

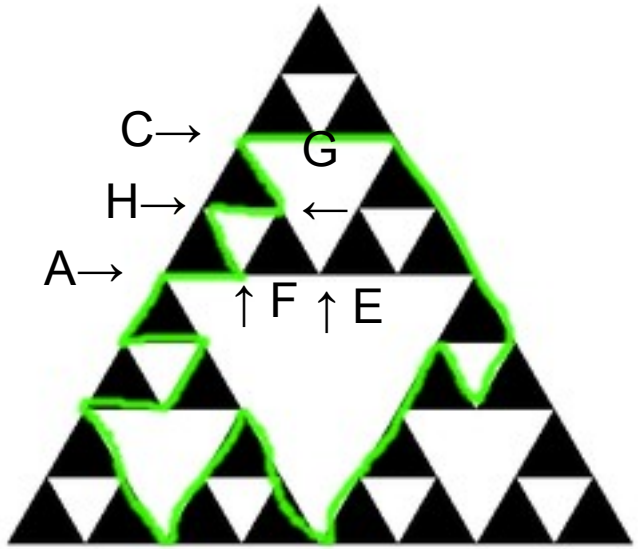


- 右の図形をレベルL+2として、
  A→E→Cと辿るような「多角形の一部」の個数を h(L)
  とおいて、これを求めて両方Eを通るケースを引く

D問題　アルゴリズム

AtCoder

右の図形をレベルL+2として、A→E→Cと辿るような「多角形の一部」の個数を h(L) とおいた



今回は中央の三角形（FGH）に対して必ずE側を通る

A→F→E→G→C という経路になるが、さきほどと同じくHを二度通ってはならない

適当に回転すれば今までと同様に h(L) が g(L-1), h(L-1) で表せる

以上で f(L),g(L),h(L) が f(L-1), g(L-1), h(L-1) によって表されたので O(L) で f(L) が求まる

D問題　アルゴリズム

AtCoder

お疲れ様でした。

AtCoder Regular Contest 036
解説

AtCoder株式会社

ARC036解説
A:ぐっすり

A-問題概要

- 高橋くんの睡眠予定がN日分与えられる。
- 連続する3日の睡眠量の合計がKを下回ると高橋くんは睡眠不足になる。
- 高橋くんが睡眠不足になるかどうか、なるなら何日目になるか求めよ。

A-数式で表す

- t[i] = i日目の高橋くんの睡眠予定時間
- ３≦ i ≦ N なる i のなかで

  t[i-2] + t[i-1] + t[i] < K

  となる最小の i を、存在するなら求めよ

A-解法

- t[i] = i日目の高橋くんの睡眠予定時間
- ３≦ i ≦ N なる i のなかで

  t[i-2] + t[i-1] + t[i] < K

  となる最小の i を、存在するなら求めよ
- N ≦ 100,000なので、iを全通り試しても十分まにあう
  - 満点解法

A-注意点

- i = 1, i = 2のときは調べないようにしなければならない
  - t[-1]に参照して実行時エラーになりうる
- 不等号に注意
  - t[i-2] + t[i-1] + t[i] ≦ K にすると失敗する
  - 問題文をよく読みましょう

B 問題

山のデータ

概要

- 長さ N の数列があります。
- $h_s \leqq h_{s+1} \leqq \cdots \leqq h_t \geqq \cdots \geqq h_u$ となる整数組 (s, t, u) のうち、$u - s + 1$ の最大値を求めてください。
- $1 \leqq N \leqq 300{,}000$
- $1 \leqq h_i \leqq 1{,}000{,}000{,}000$

部分点解法

- すべての整数組 (s, t, u) について考えます。
- 整数組 (s, t, u) が実際に山の条件を満たしているかは、O(N) 回の判定でできます。
- 考えられる整数組は O(N^3) 個あるので、全体で O(N^4) の計算量となります。

考察

- s や u に関して、外側に伸ばせるのに伸ばさないのはもったいないです。
- 先に t を固定して、s と u を外側に伸ばせるだけ伸ばすという方針で $O(N^2)$ に減らすことができます。
- $O(N^2)$ よりも高速にするために、さきほどの t に関して考えてみます。

満点解法

- 先ほどの t としては、データの両端であるか、$h_{t-1} \leqq h_t$ かつ $h_t \geqq h_{t+1}$ であるような t だけ考慮すれば良いことが分かります (そうでない場合、$t-1$ あるいは $t+1$ の方がより良い結果になることが分かります)。
- このように限定した場合、どの要素も高々定数回しかアクセスされないので、全体で O(N) となります。

C 問題

偶然ジェネレータ

概要

- 長さ N の 0,1 からなる数列があります。
- ? となっている場所をうまく 0,1 で埋めて、どのように部分列を取ってきても 0,1 の個数の差が K 以下となるようにします。
- 考えられる総数を 1,000,000,007 で割った余りを求めてください。
- $1 \leqq N \leqq 300$
- $1 \leqq K \leqq N$

部分点解法 1

- すべての 0,1 割り当てを考えます。
- すべての配置が定まれば、その配置が条件を満たしているかは $O(N^3)$ で判定できます。
- 考えられる個数は 2^N 個あるので、全体で $O(2^N N^3)$ となります。
- 1 つめのデータセットに対して正解できます。

部分点解法 2

- 動的計画法を用いて左から順に調べることを考えます。
- 考えるべき状態数としては、今まで調べた場所と、調べた場所の右端から左に伸ばしたときに (0 の個数)-(1 の個数) として何が考えられ、何が考えられないのかという情報です。
- 後者 (0,1 の差) は、(0 の個数)-(1 の個数)として、<1 が考えられる>、<2 が考えられる>、…、< K が考えられる>、<-1 が考えられる>、<-2 が考えられる>、…、< $-K$ が考えられる> という、合計で $2K$ ビットの情報を持っていれば表現できます (差 0 は、長さ 0 の列を考えれば常に存在します)。

部分点解法 2

- dp[i][j] = (場所 i まで調べていて矛盾 (差が K を上回る状態) がなく、かつ場所 i から左に伸ばしたときに考えられる差がビット列 j となるような 0,1 割り当ての総数) とします。
- 状態遷移は、今見ている場所に 0,1 のどちらが入るかを決めれば次は一意に定まるので各状態につき O(1) で判定できます。
- このような動的計画法は $O(2^K N)$ で実行できます。
- K が小さいデータセット 2 に関して正解することができます。

考察

- すべての状態を考えていても、指数オーダーの計算量となります。
- (0 の個数)-(1 の個数) の最大値および最小値が一致するもの同士をまとめて数え上げたいです。
- 実は最大値および最小値が一致するものをまとめても、特に問題はありません (なぜなら、差が K を最初に越えるのは、必ず最大値か最小値のいずれかであり、かつ状態遷移によってどの状態が考えられるかは、平行移動 (+ 差が 0) という変化しかないので、最大値と最小値だけ考慮しておけばよいことになります)。

満点解法

- dp[i][j][k] = (場所 i まで考えて矛盾 (差が K を上回る状態) がなく、i から左に伸ばしたときの (0 の個数)-(1 の個数) の最大値が j で最小値が –k であるようなものの総数) とします。
- このような動的計画法は O(N^3) で実行できます。
- このアルゴリズムなら満点を得ることができます。

ARC036解説

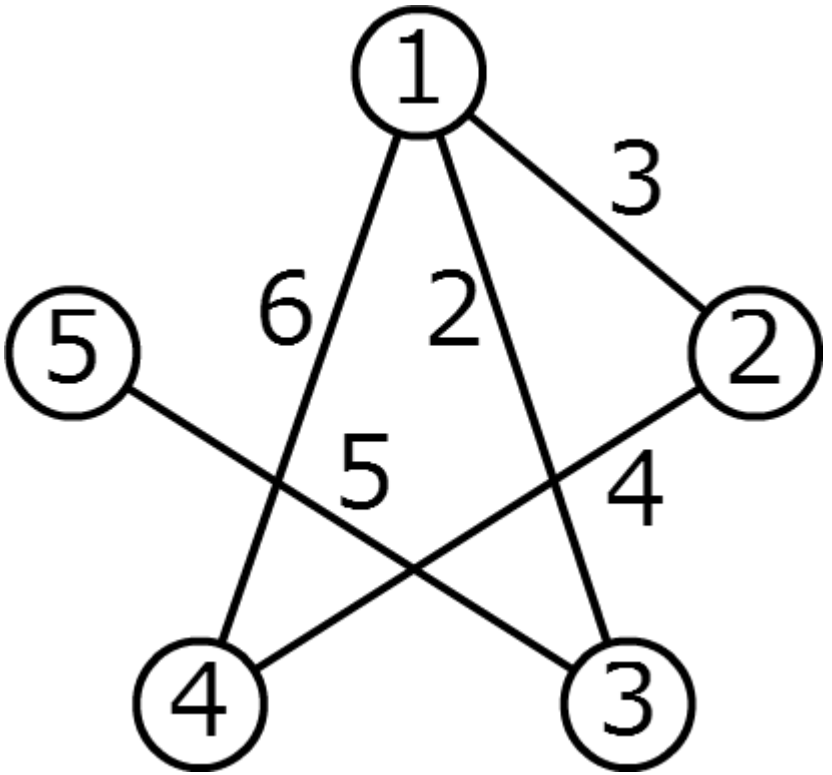
D:偶数メートル

D-問題概要

- 辺の無いグラフ（頂点数N）がある。
- 以下の2種類のクエリQ個に答えよ
  - 頂点x, yの間にdメートルの辺を引く
  - 頂点xから頂点yへのwalkのなかで偶数メートルのものがあるか判定せよ

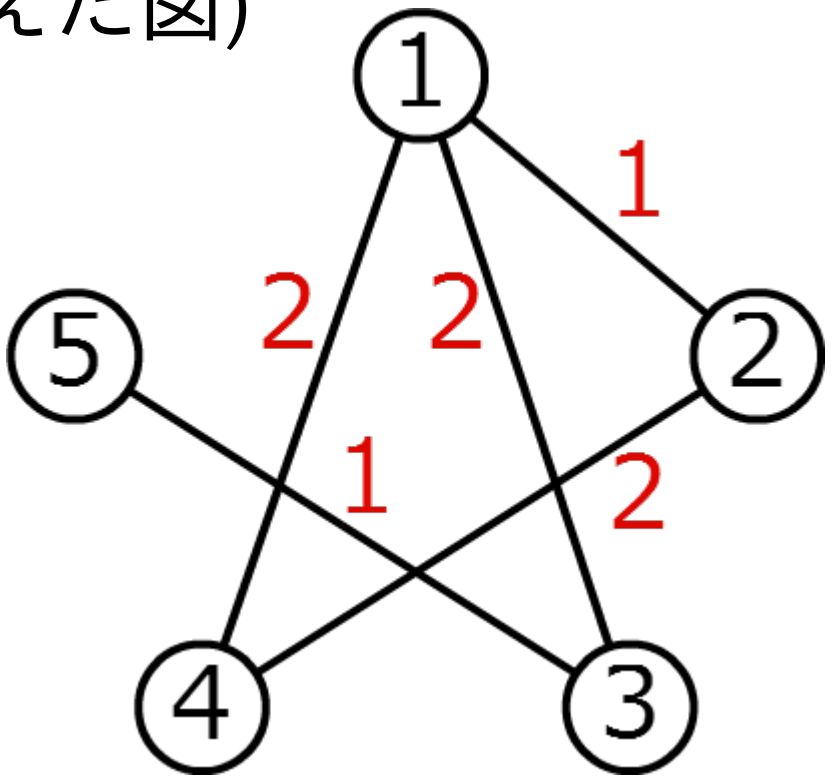
※walkというのは同じ辺を複数回通ることを許した経路のこと。

D-入力例1



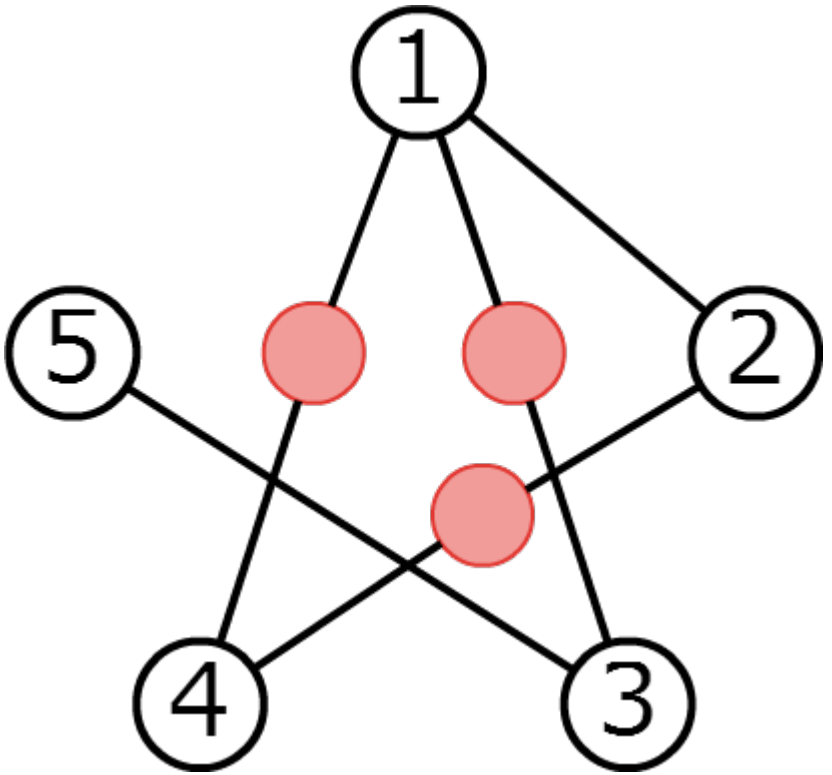
D-考察1

- 移動距離の総和が偶数かどうかしか気にしていないので、辺の長さも偶奇のみを考えれば良い(1,2にそろえた図)



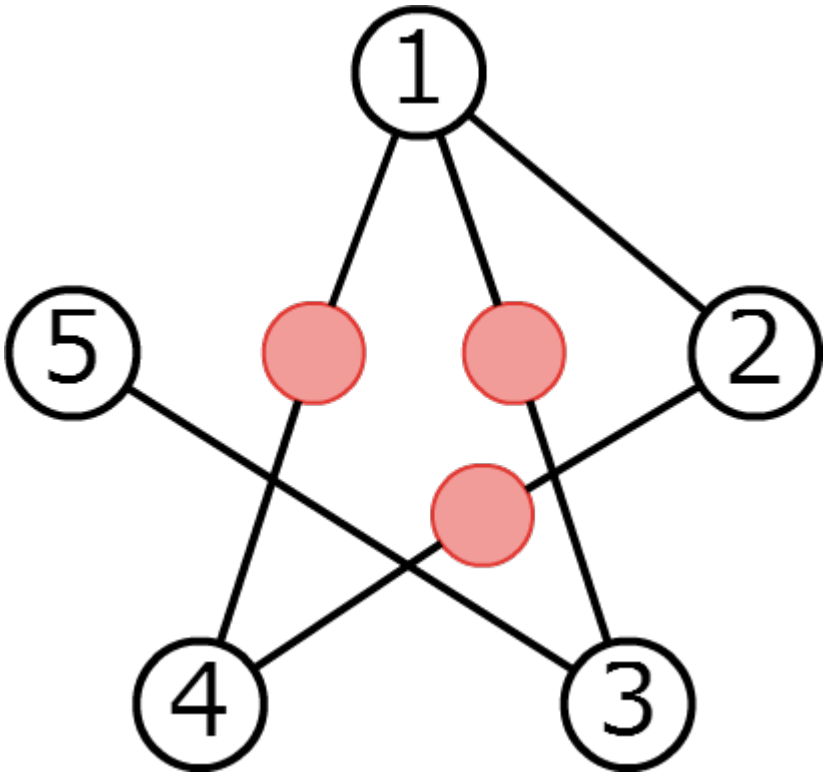
D-考察1

- 距離2の辺は新たな頂点を間に追加して2つの距離1の辺と考えて良い



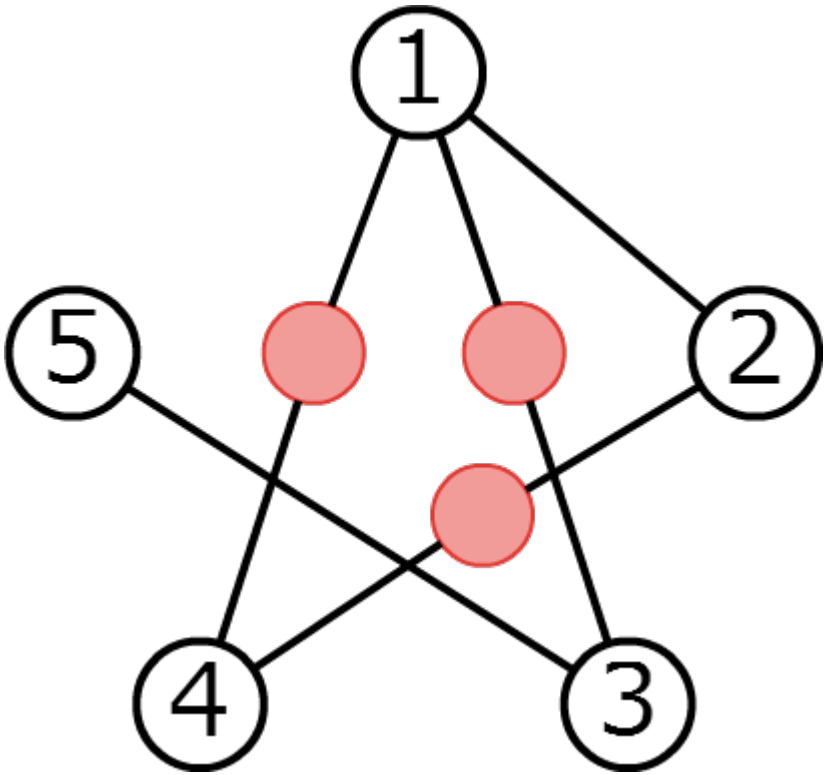
D-考察1

- 「偶数メートル」→「偶数本の辺」
- すこし扱いやすくなった



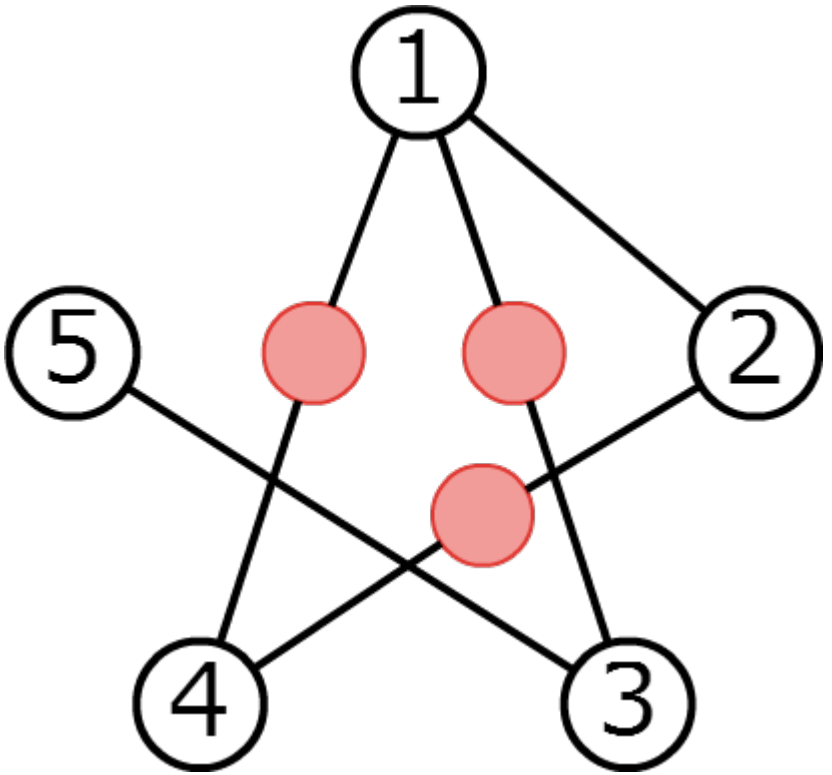
D-考察1

- クエリの数が高々2倍になるだけなので計算量にはそこまで影響しない

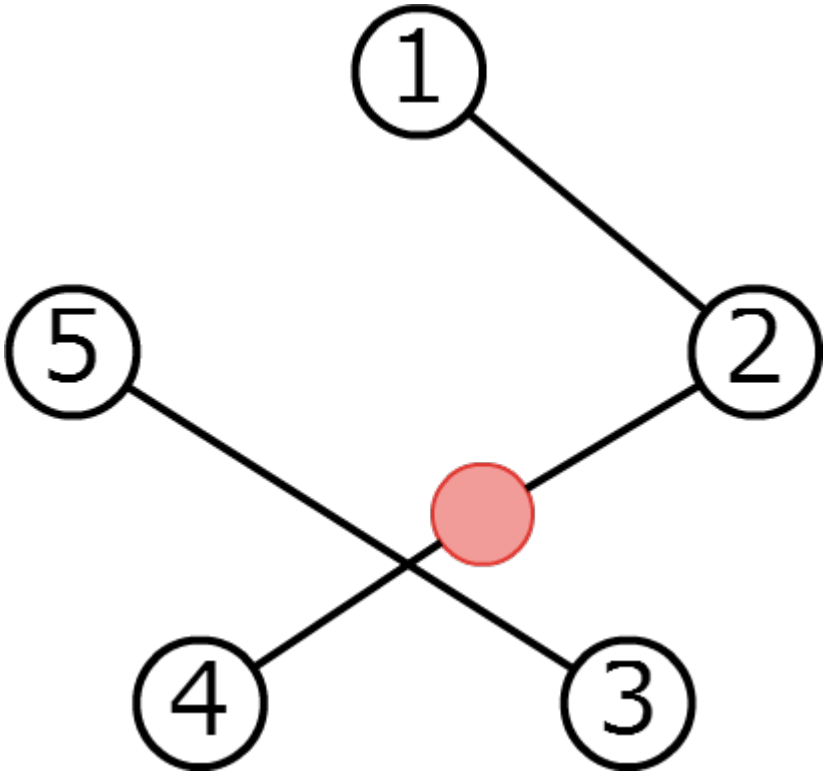


D-考察

- クエリの数が高々2倍になるだけなので計算量にはそこまで影響しない

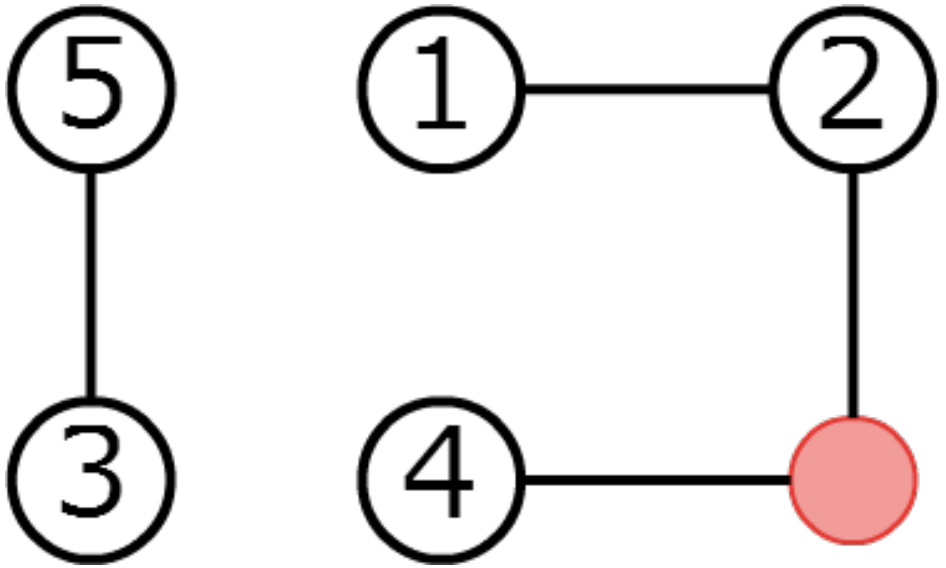


D-入力例2



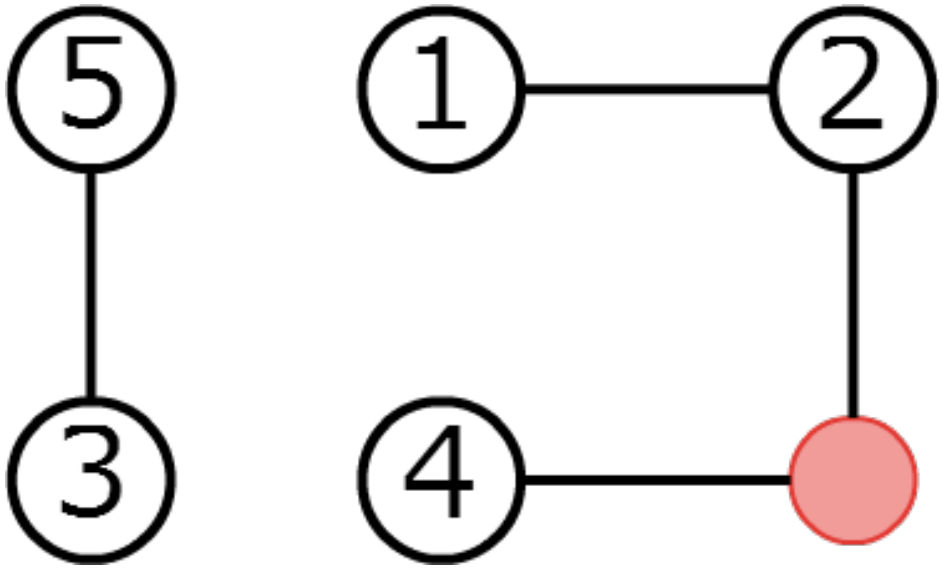
D-考察2

- 非連結な頂点は行き来できないので、連結成分ごとに分けて考えて良い



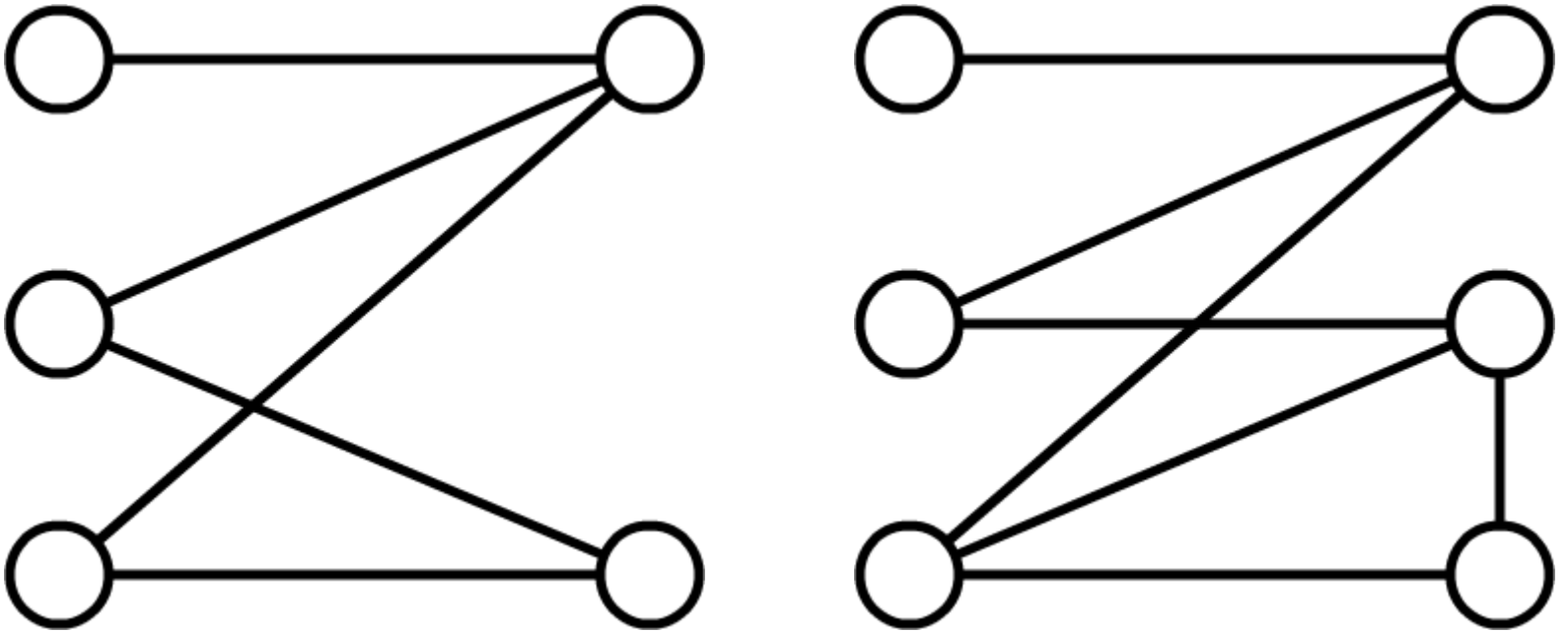
D-考察2

- 非連結な頂点は行き来できないので、連結成分ごとに分けて考えて良い



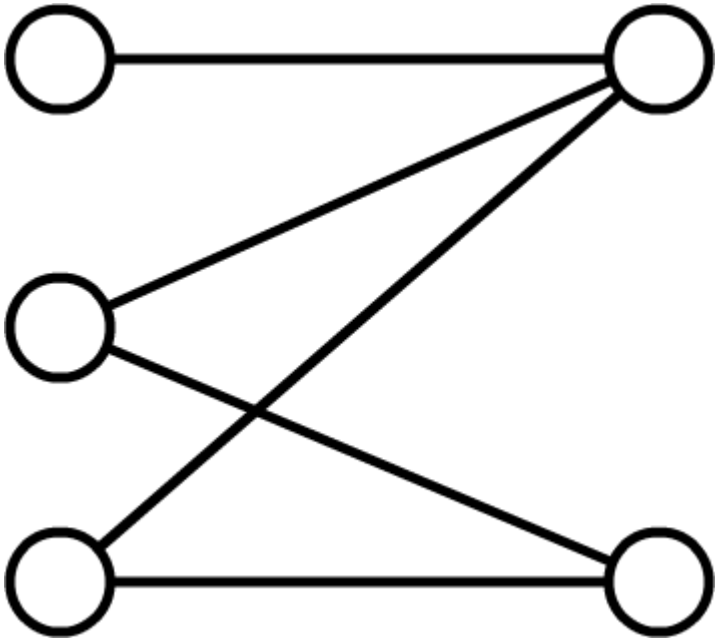
D-考察3

- 実験をすると連結成分には2種類に大別できる事がわかる



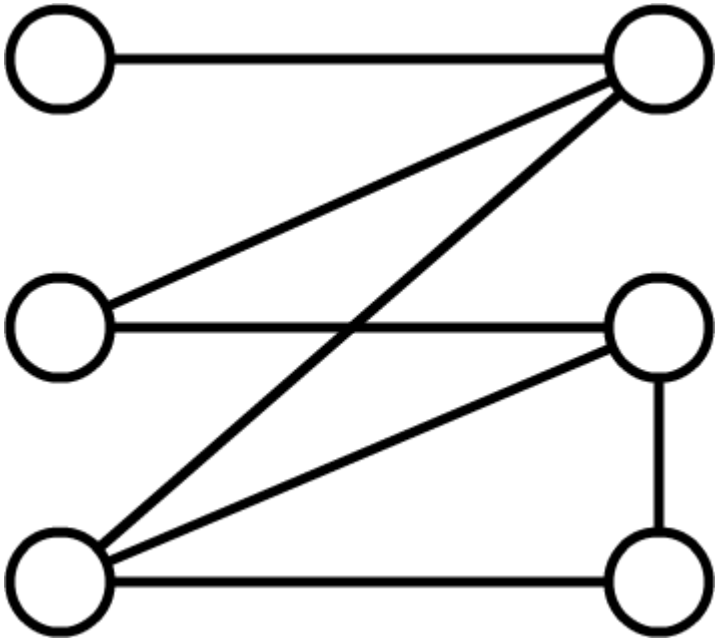
D-考察3

- 頂点対によってはちょうど偶数本の辺では行き来できない連結成分



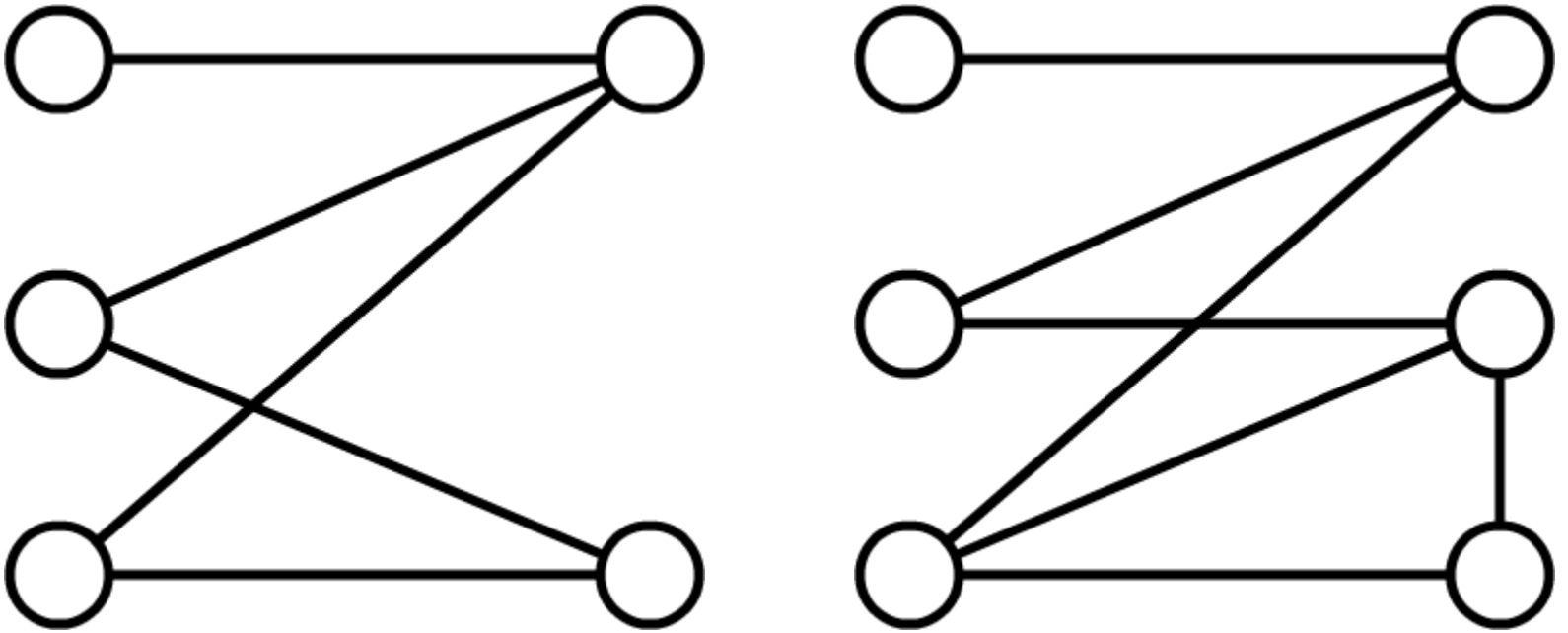
D-考察3

- どの2頂点もちょうど偶数本の辺を使って行き来できる連結成分



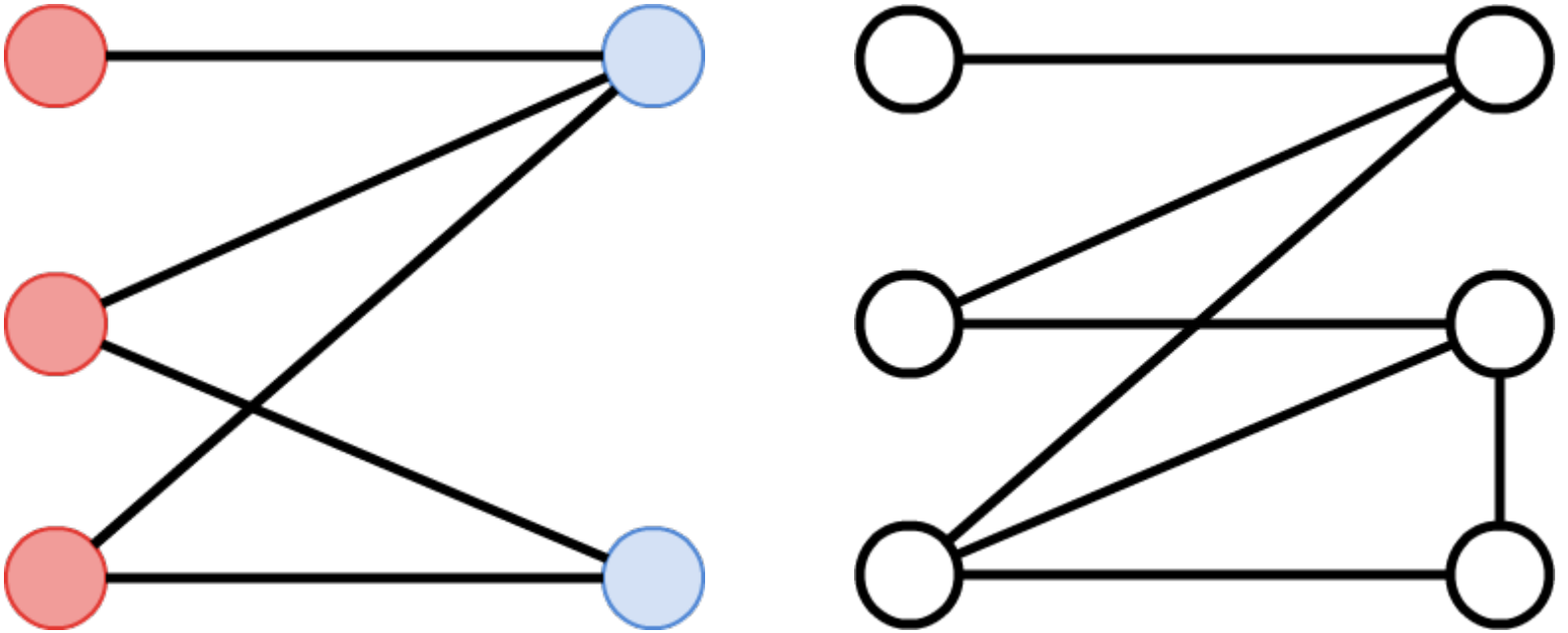
D-考察3

- 違いは？



D-考察3

- 違いは？

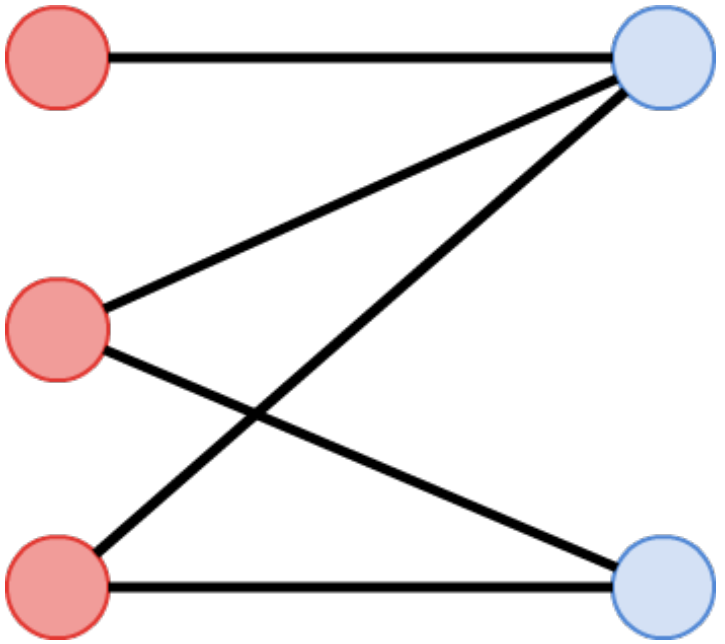


D-考察3

二部グラフ

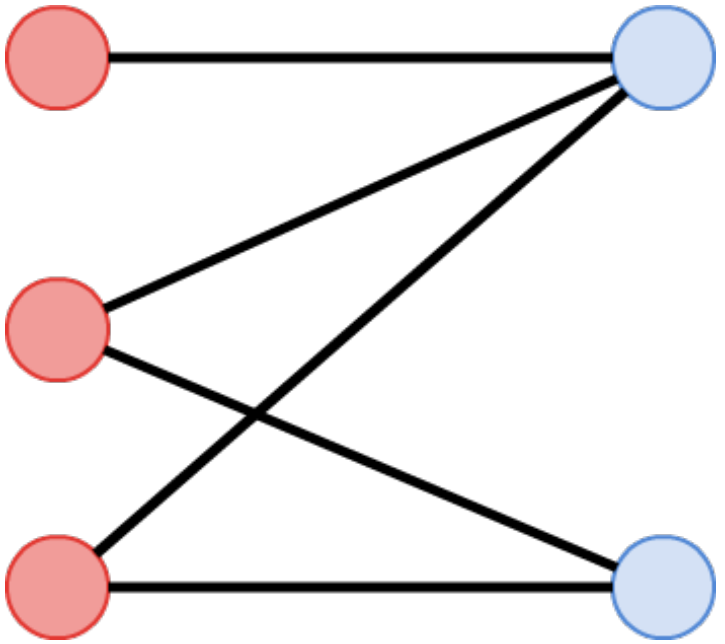
D-考察3

- 二部グラフとは
  - 頂点集合をうまく2つにわけると、すべての辺は集合間を結び、集合内を結ぶ辺がなくなるようなグラフ



D-考察3

- 二部グラフの性質
  - 片方の集合に属す頂点から、もう片方の集合に属す頂点に移動するwalkは必ず辺の数が奇数

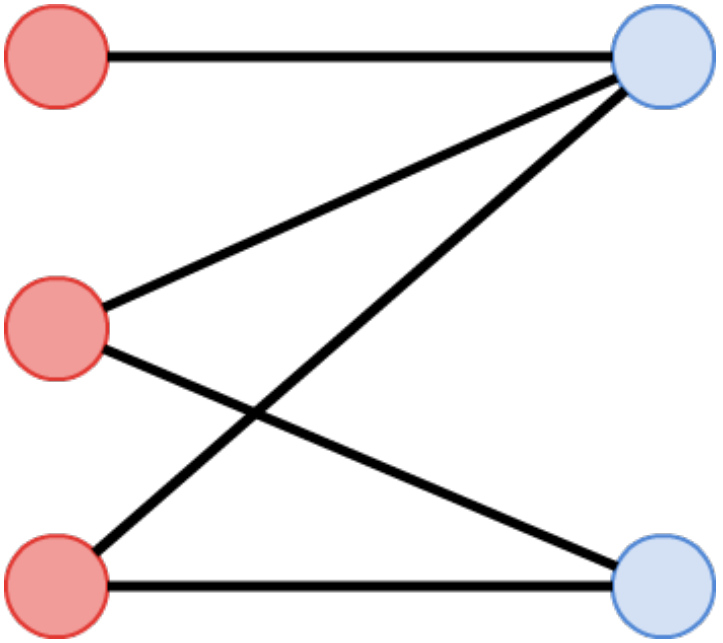


D-考察3

- 二部グラフの性質
  - 片方の集合に属す頂点から、もう片方の集合に属す頂点に移動するwalkは必ず辺の数が奇数
  - 赤から始めるとかならず

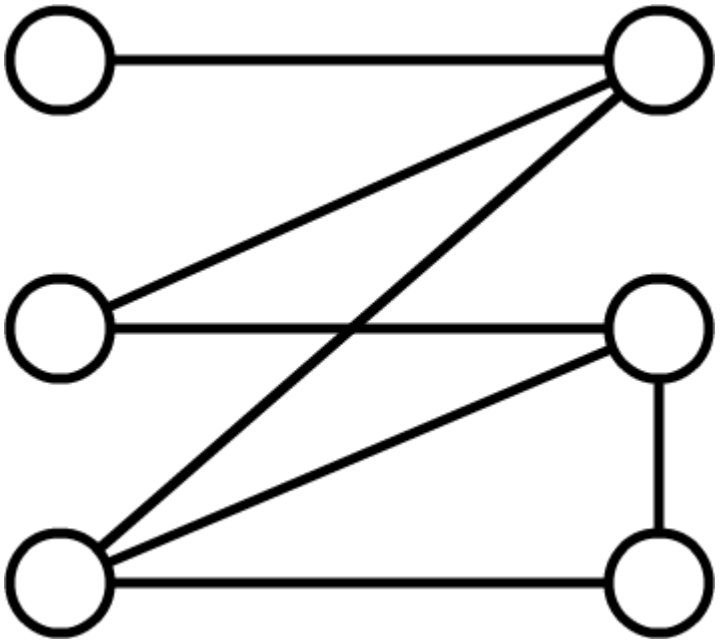
    赤、青、赤、青・・・

    というwalkになるから



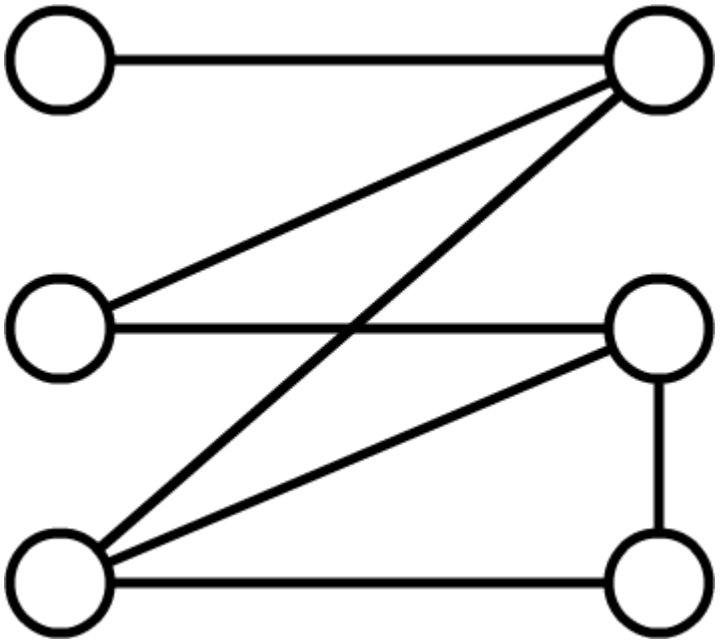
D-考察3

- 二部グラフでないグラフの性質



D-考察3

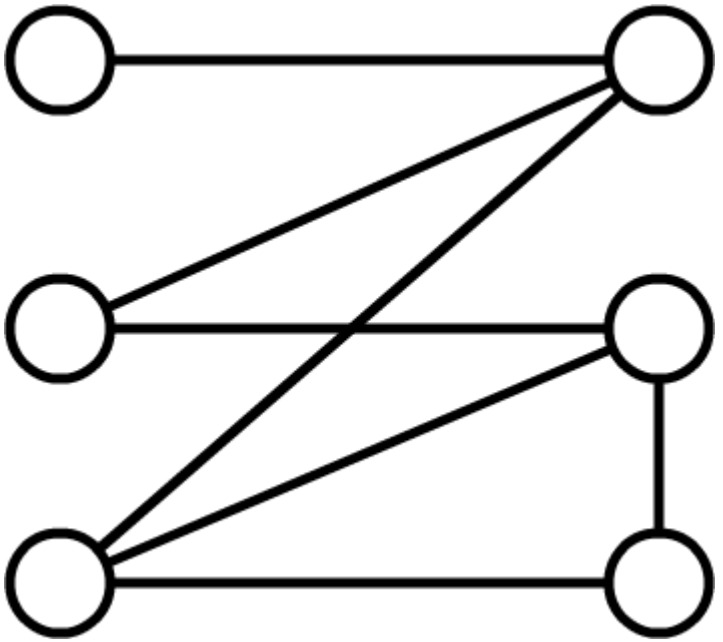
- 二部グラフでないグラフの性質
  - どの2点間も辺の数が偶数のwalkがある



D-考察3

- 二部グラフでないグラフの性質
  - どの2点間も辺の数が偶数のwalkがある

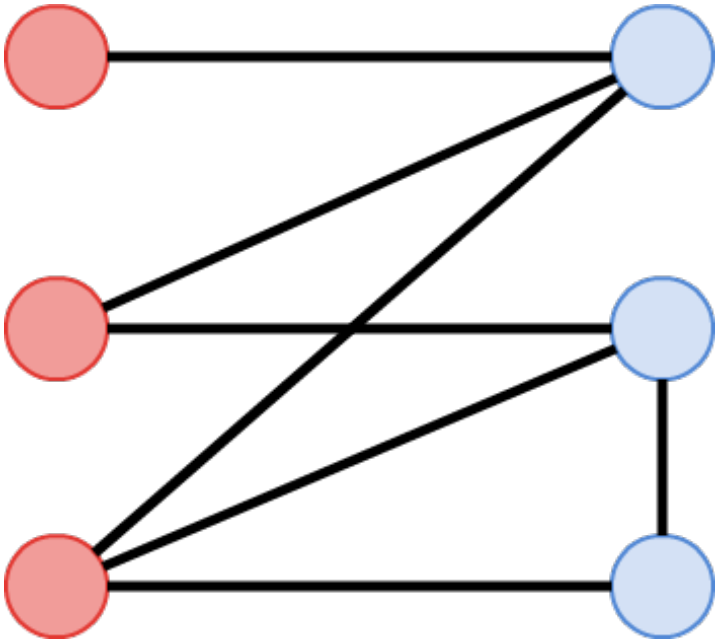
    なぜ？



D-考察3

- 二部グラフでないグラフの性質
  - どの2点間も辺の数が偶数のwalkがある

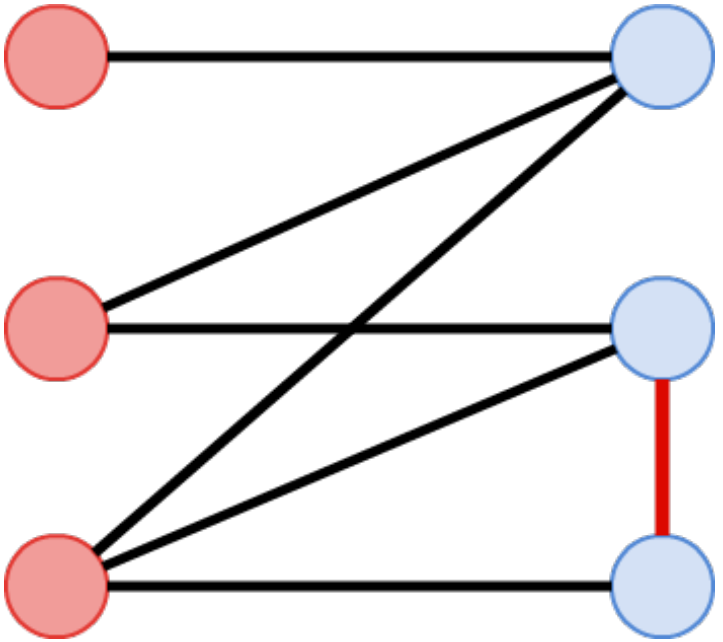
    なぜ？
  - 無理やり2つの集合に分けることを考える



D-考察3

- 二部グラフでないグラフの性質
  - どの2点間も辺の数が偶数のwalkがある

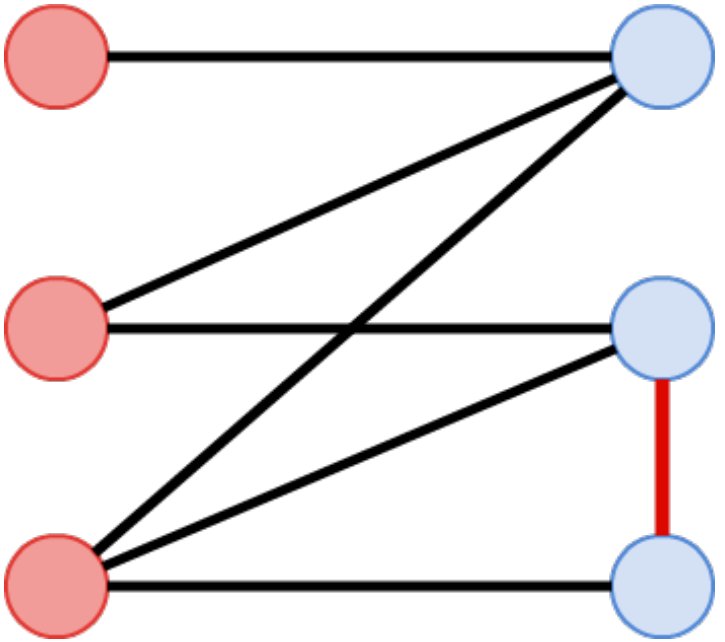
    なぜ？
  - 無理やり2つの集合に分けることを考える
  - どこかで赤 → 赤もしくは
    青 → 青の辺ができる



D-考察3

- 二部グラフでないグラフの性質
  - どの2点間も辺の数が偶数のwalkがある

    なぜ？
  - 無理やり2つの集合に分けることを考える
  - どこかで赤 → 赤もしくは
    青 → 青の辺ができる
  - その辺を経由すれば偶奇を
    自由に調整できる



D-考察まとめ

- グラフの辺の長さをすべて1にすることができる
- 連結成分ごとに別々に考えて良い
- 連結成分が二部グラフかどうかが鍵になっている

D-部分点1

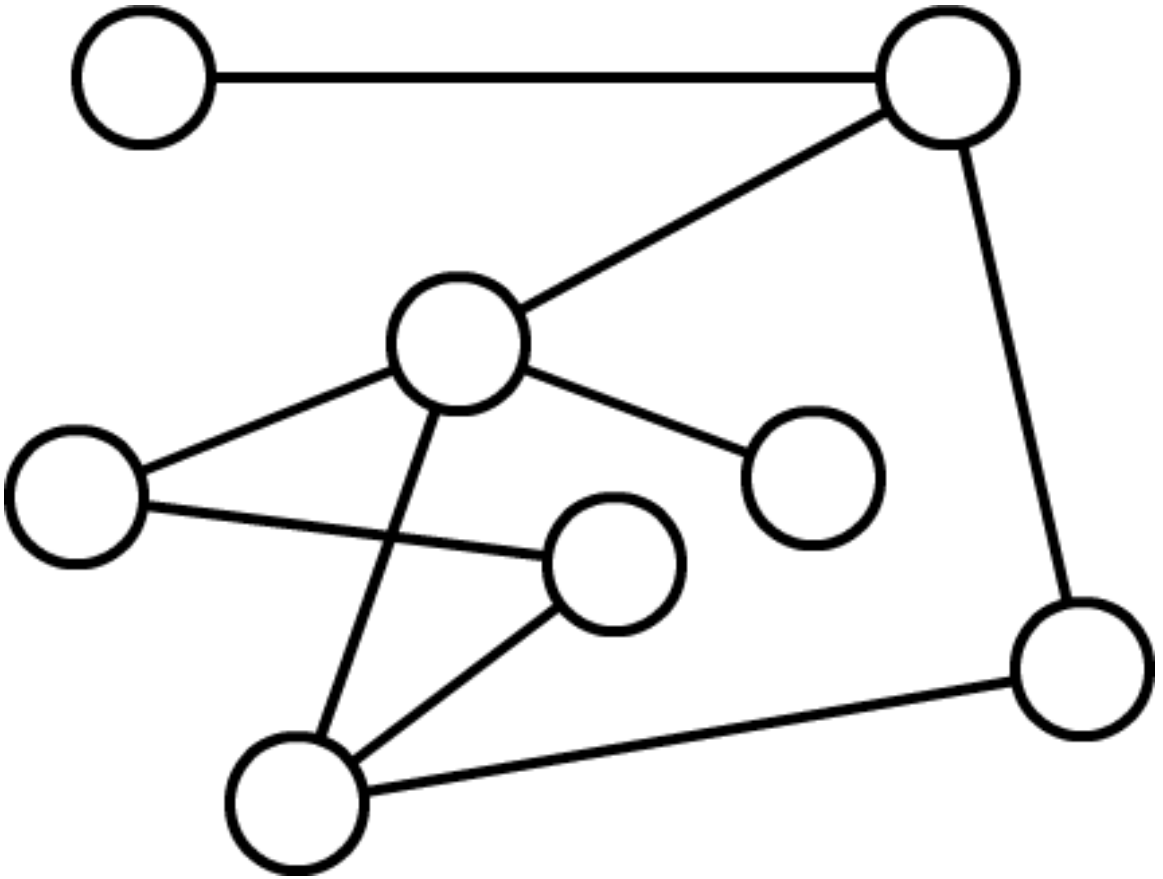
- N,Q ≦ 3,000
- クエリ毎に毎回質問された頂点が二部グラフかどうか判定しても充分間に合う

D-部分点1

- N,Q ≦ 3,000
- クエリ毎に毎回質問された頂点が二部グラフかどうか判定しても充分間に合う
- 二部グラフ判定法
  - 適当な頂点を赤く塗る
  - そこから始めて、隣接する頂点の色が異なるようにDFSしながら塗っていく
  - 隣接する頂点の色がおなじになる辺があったら二部グラフではない。なかったら二部グラフ

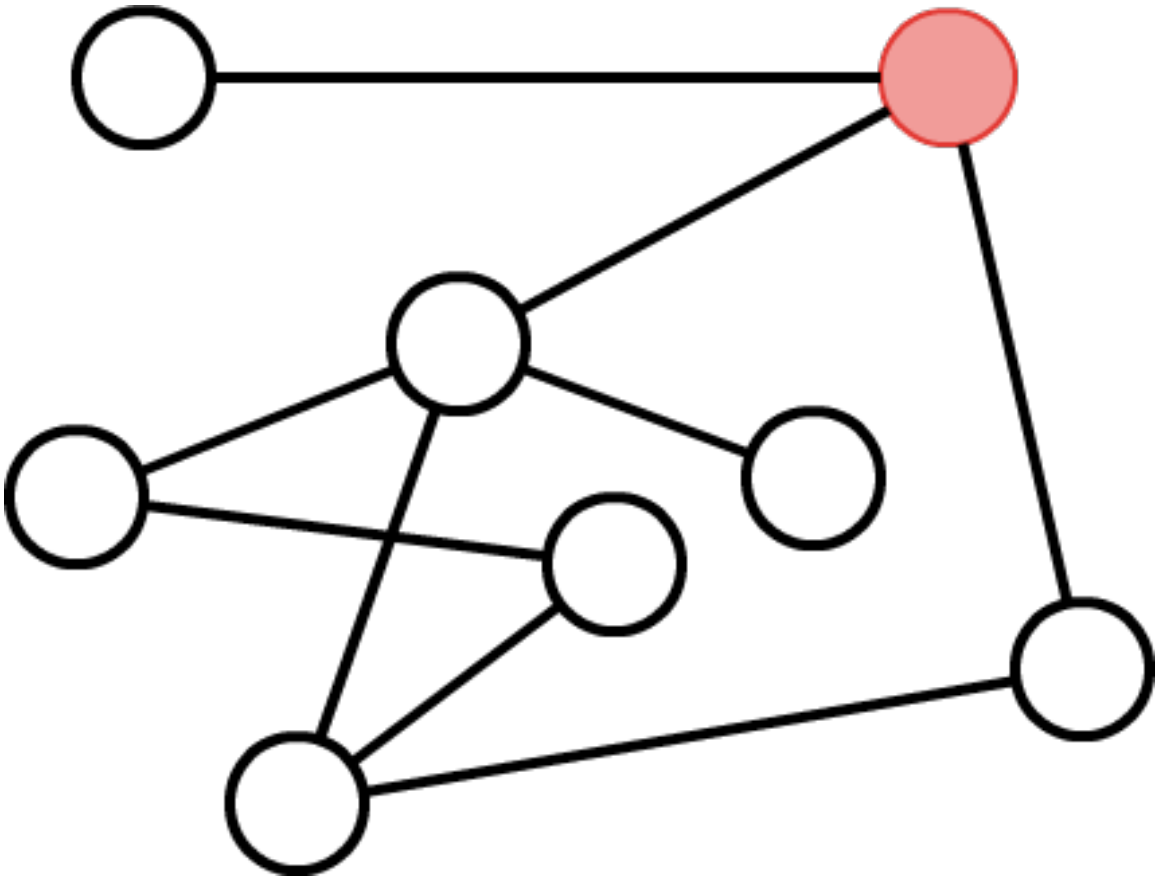
D-部分点1

- 二部グラフ判定法



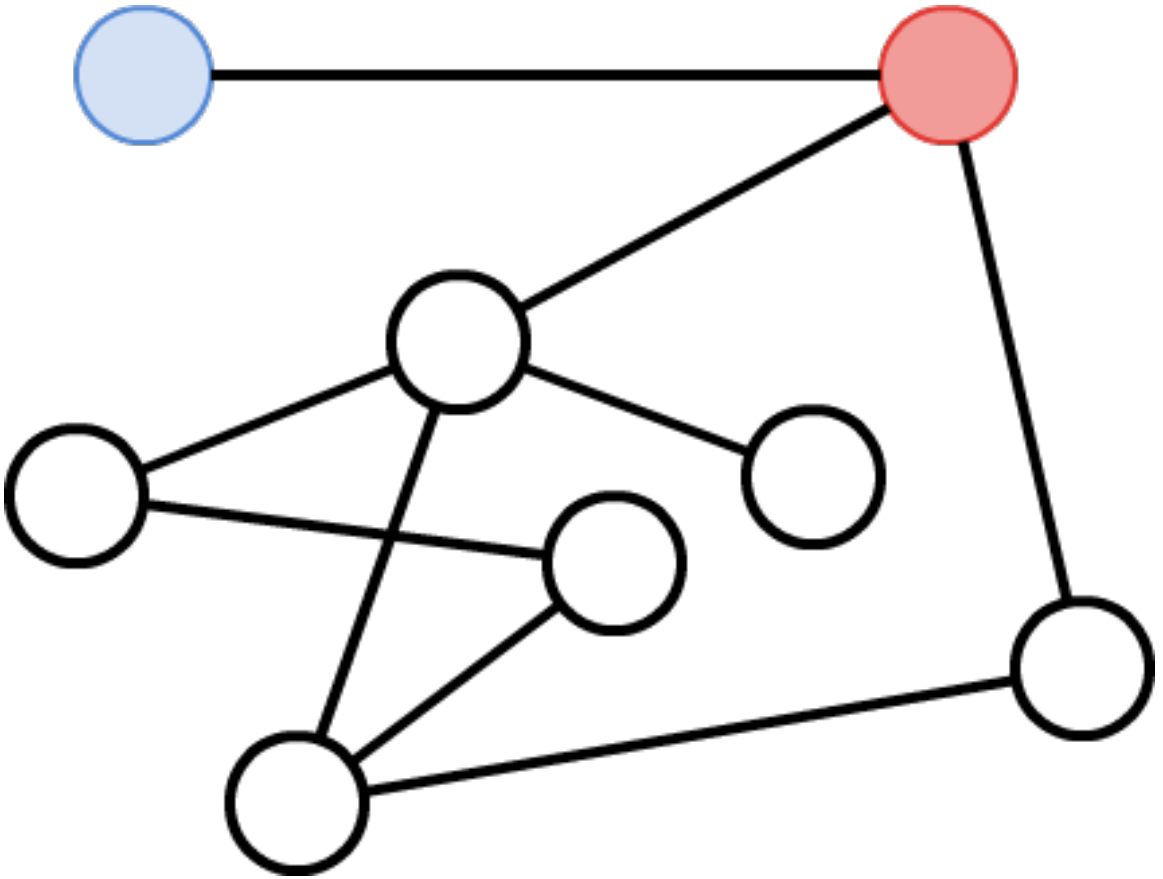
D-部分点1

- 二部グラフ判定法



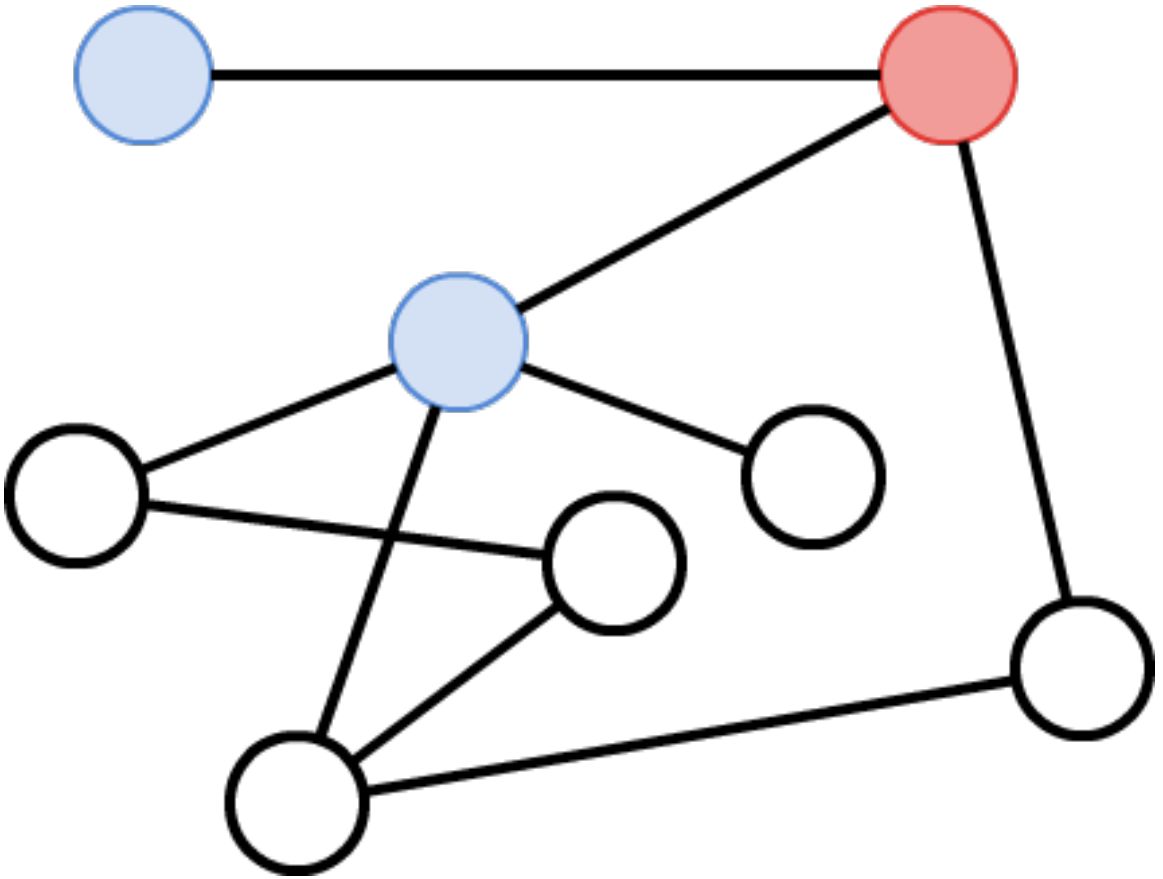
D-部分点1

- 二部グラフ判定法



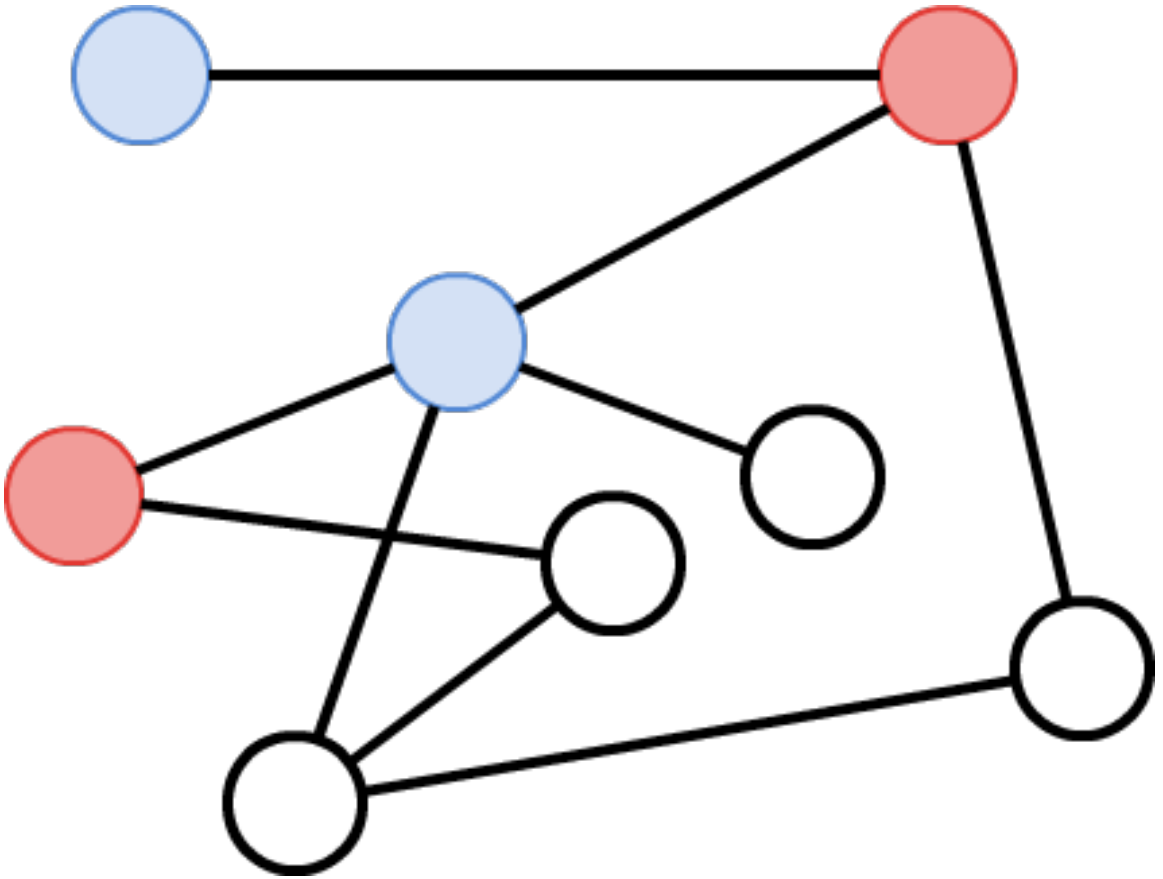
D-部分点1

- 二部グラフ判定法



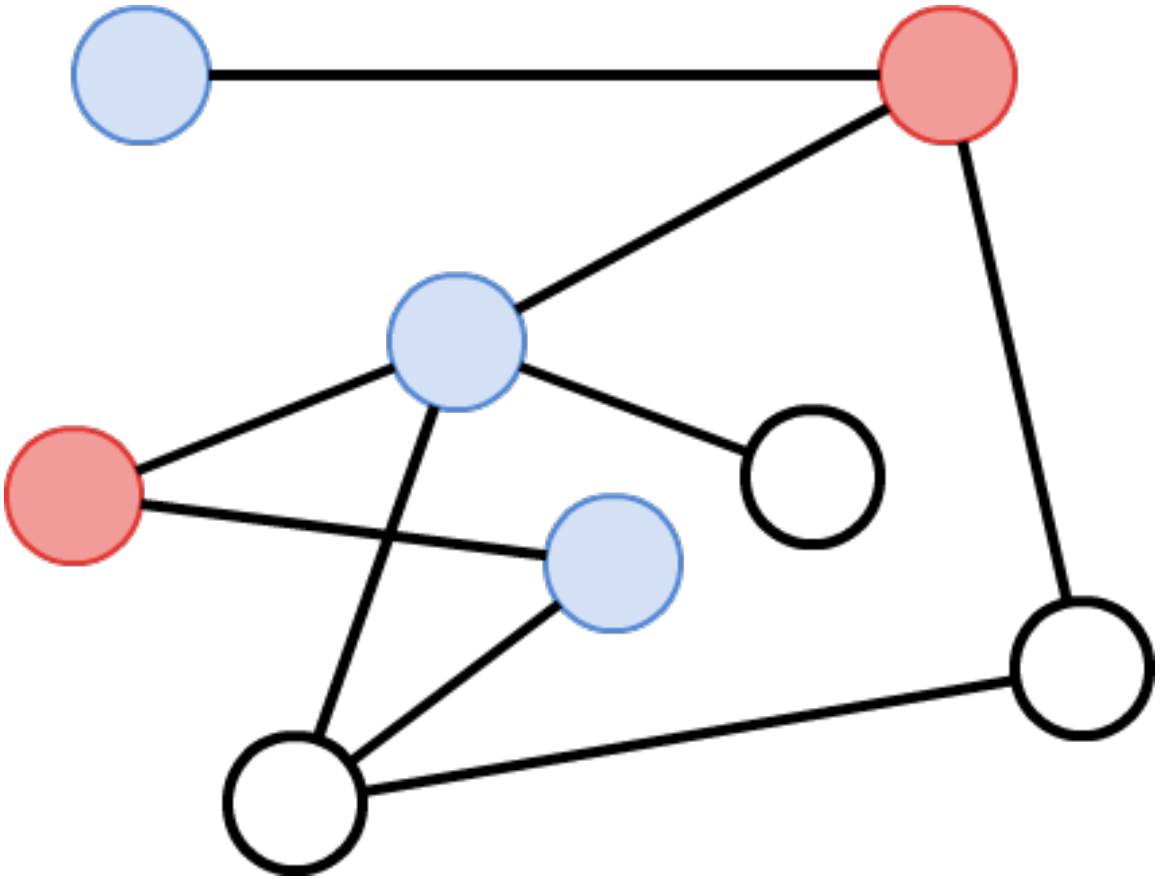
D-部分点1

- 二部グラフ判定法



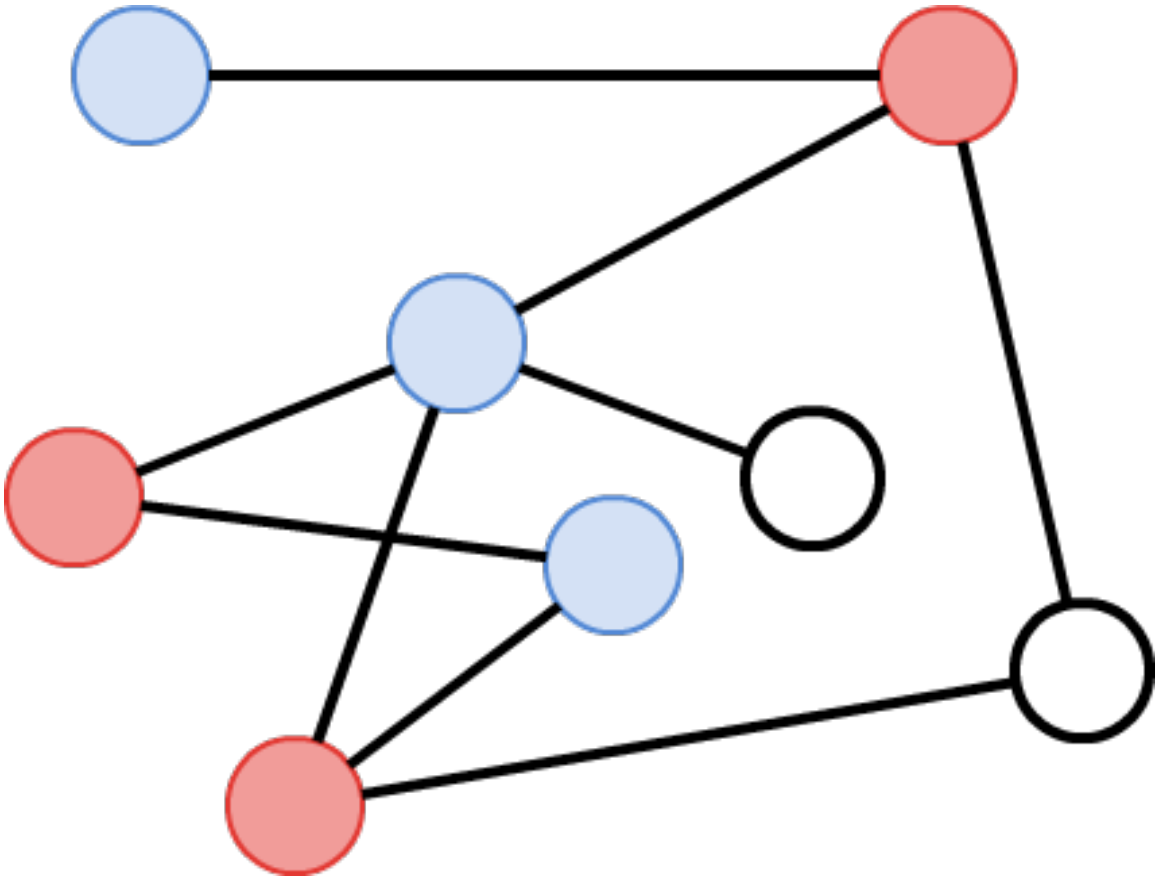
D-部分点1

- 二部グラフ判定法



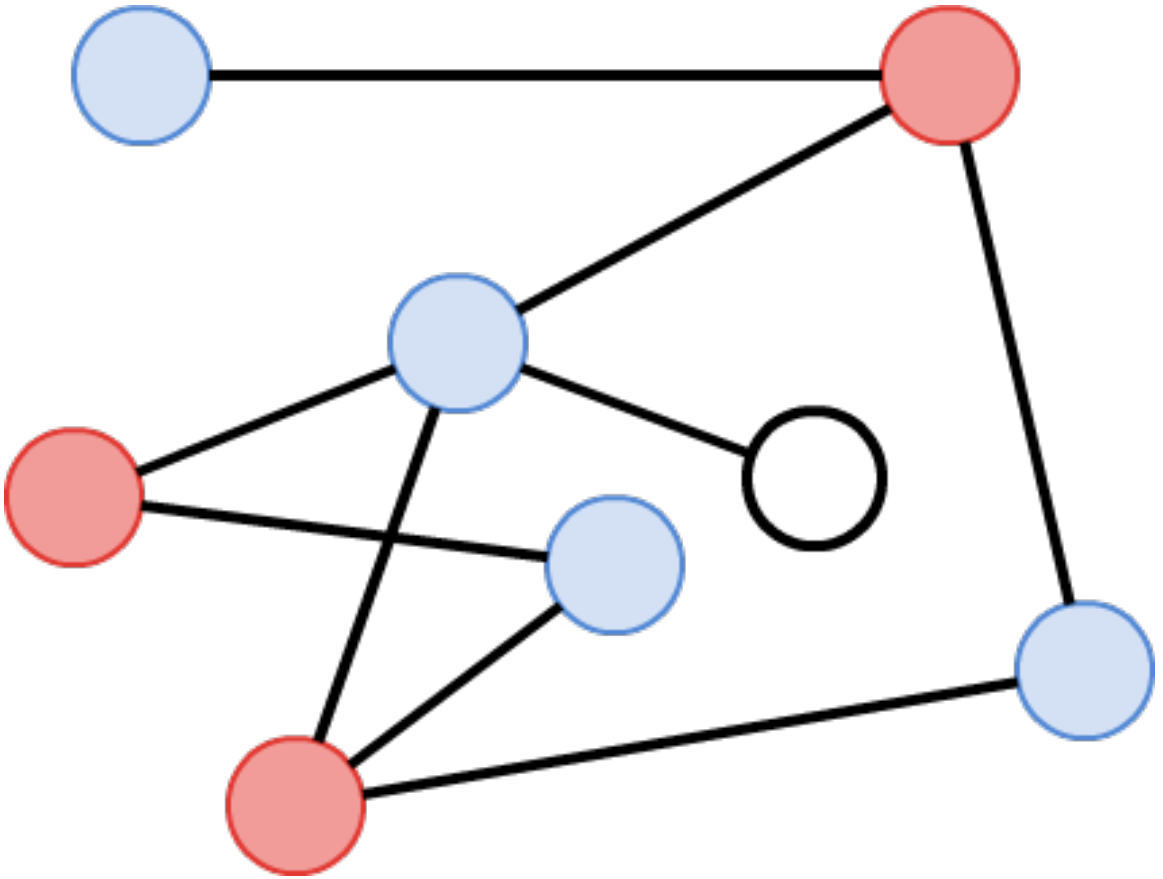
D-部分点1

- 二部グラフ判定法



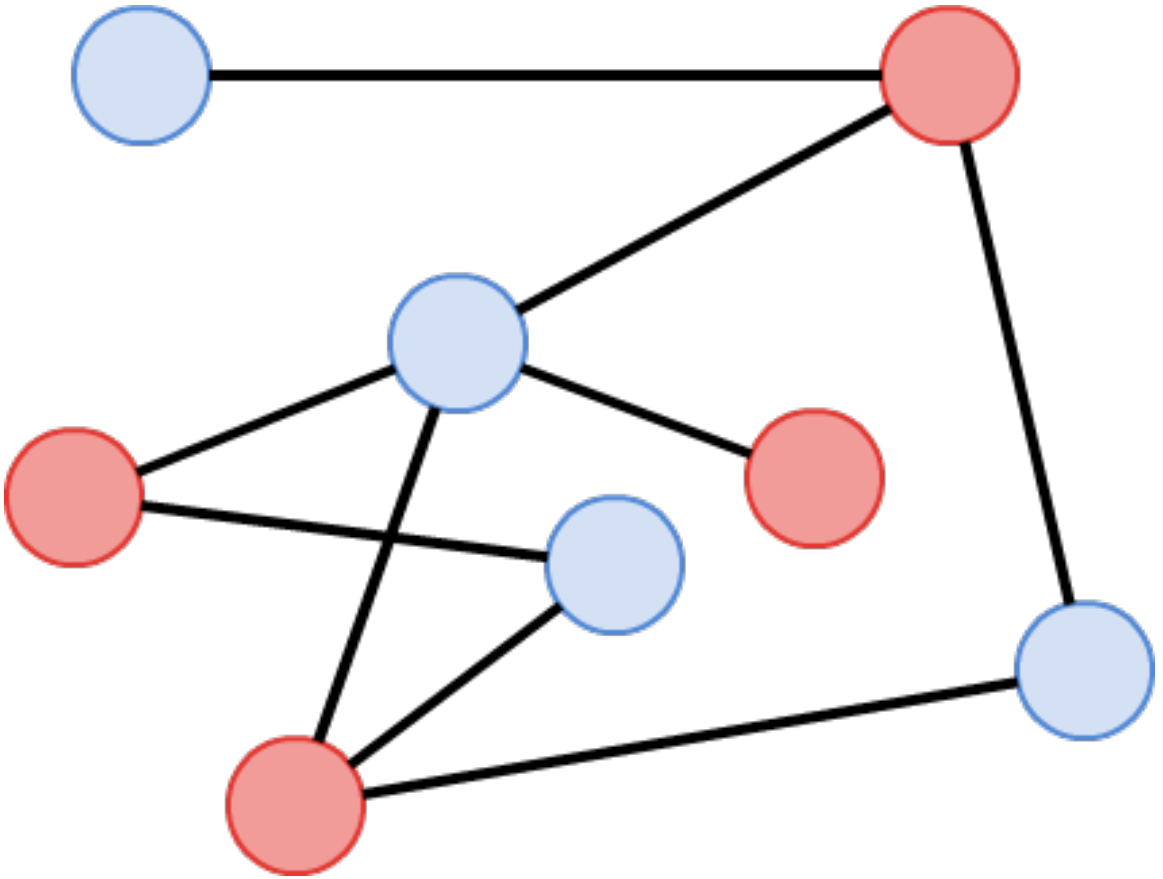
D-部分点1

- 二部グラフ判定法



D-部分点1

- 二部グラフ判定法



D-部分点1

- 二部グラフ判定法
  - 判定と同時に塗り分けもできる
- まとめ
  - 頂点X、頂点Yが連結でない　　NO
  - 連結で二部グラフでない　　YES
  - 連結で二部グラフで同じ色である　YES
  - 連結で二部グラフで違う色である　NO
- 30点獲得

D-考察4

- 辺は増える一方なので、一度でも二部グラフの異なる色となった2頂点はずっと異なる色のままである
  - 毎回DFSして塗り分けするのは無駄が多い

D-考察4

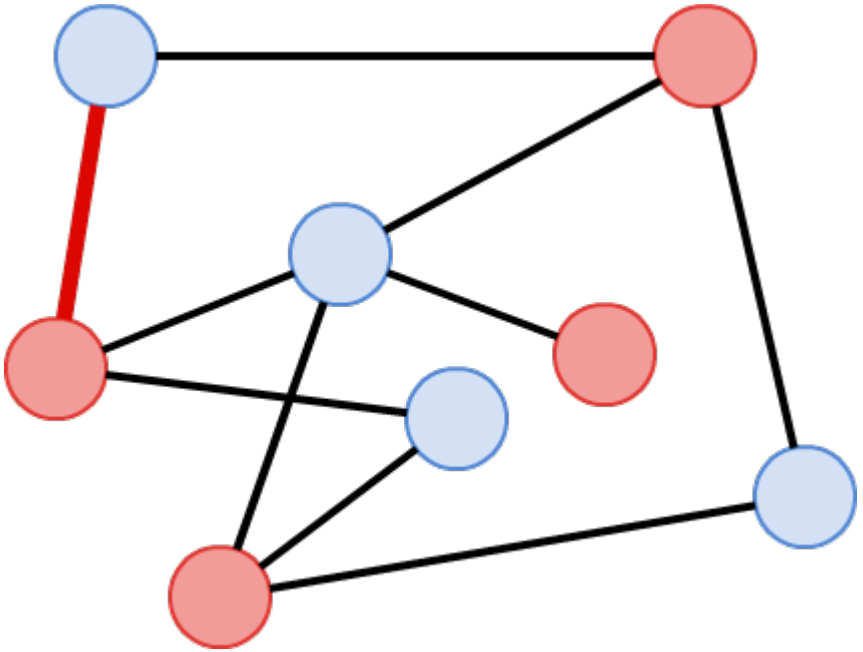
- 辺は増える一方なので、一度でも二部グラフの異なる色となった2頂点はずっと異なる色のままである
  - 毎回DFSして塗り分けするのは無駄が多い
- 辺による連結のパターンを考えてみる

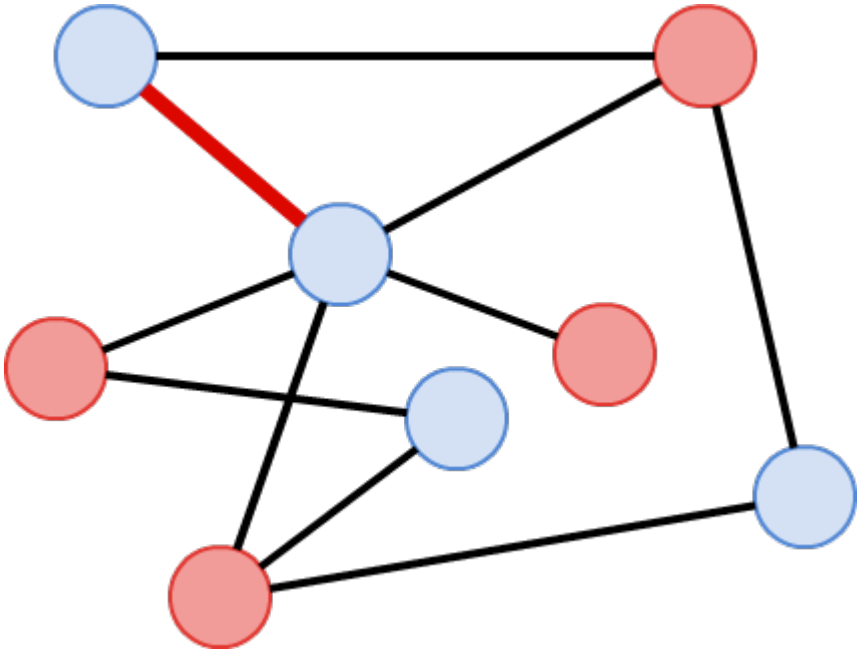
D-考察4

- パターン1
  - 連結成分の少なくとも一方が二部グラフでない辺
  - 二部グラフでないものになにを追加しても二部グラフではなくなる

D-考察4

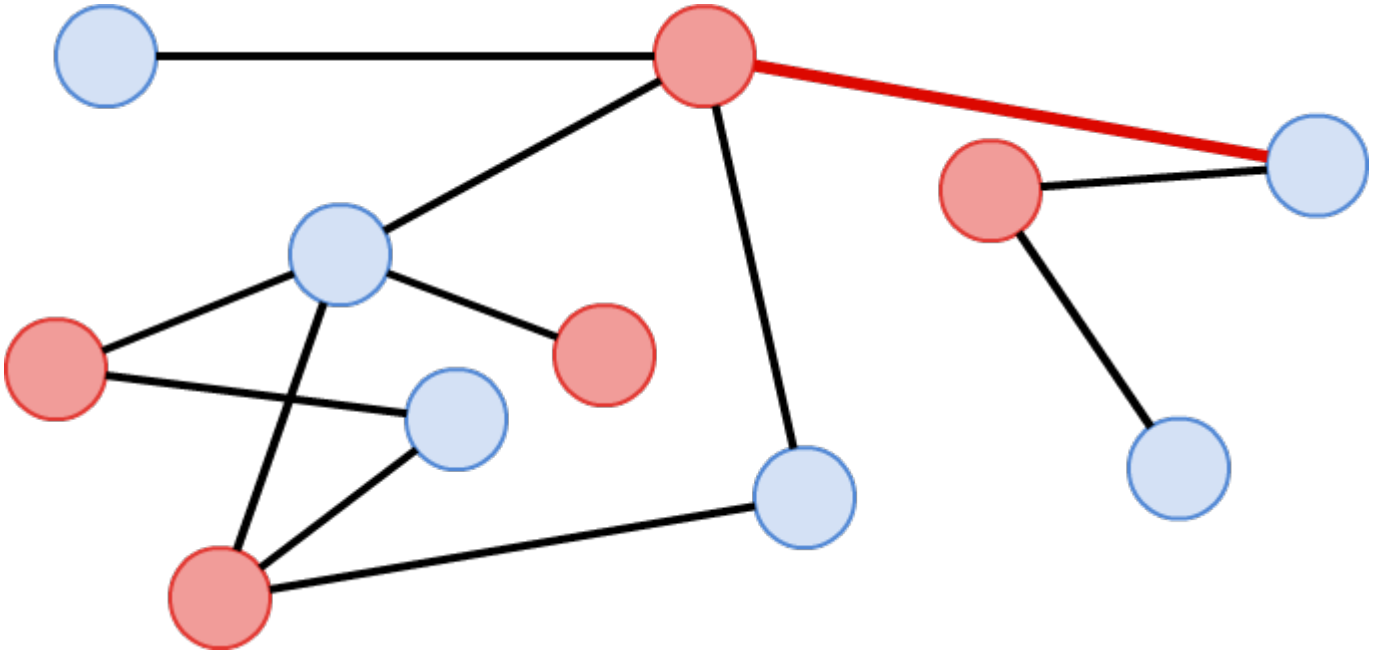
- パターン2
  - 同じ連結成分(二部グラフ)内の頂点を結ぶ辺
  - 違う色どうしを結ぶなら追加した後も二部グラフ
  - 同じ色どうしを結ぶなら二部グラフでなくなる





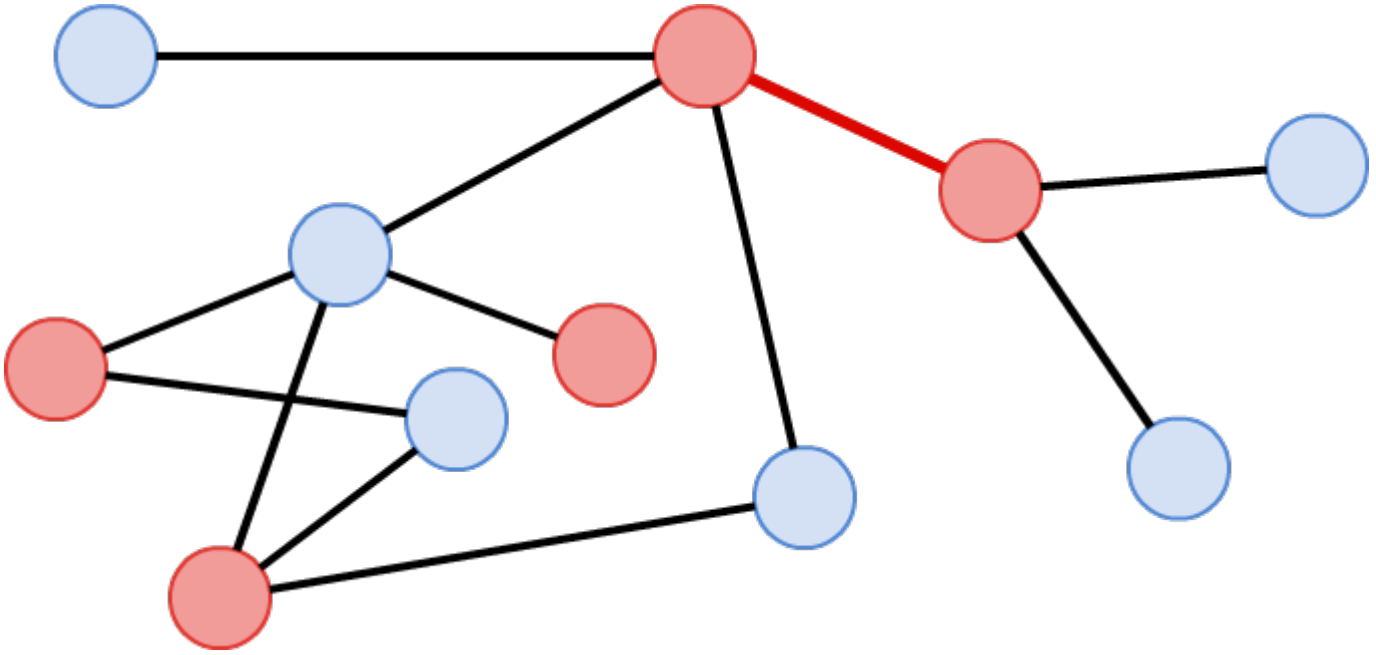
D-考察4

- パターン3
  - 異なる連結成分2つ（共に二部グラフ）を結ぶ辺
  - 追加後も必ず二部グラフ



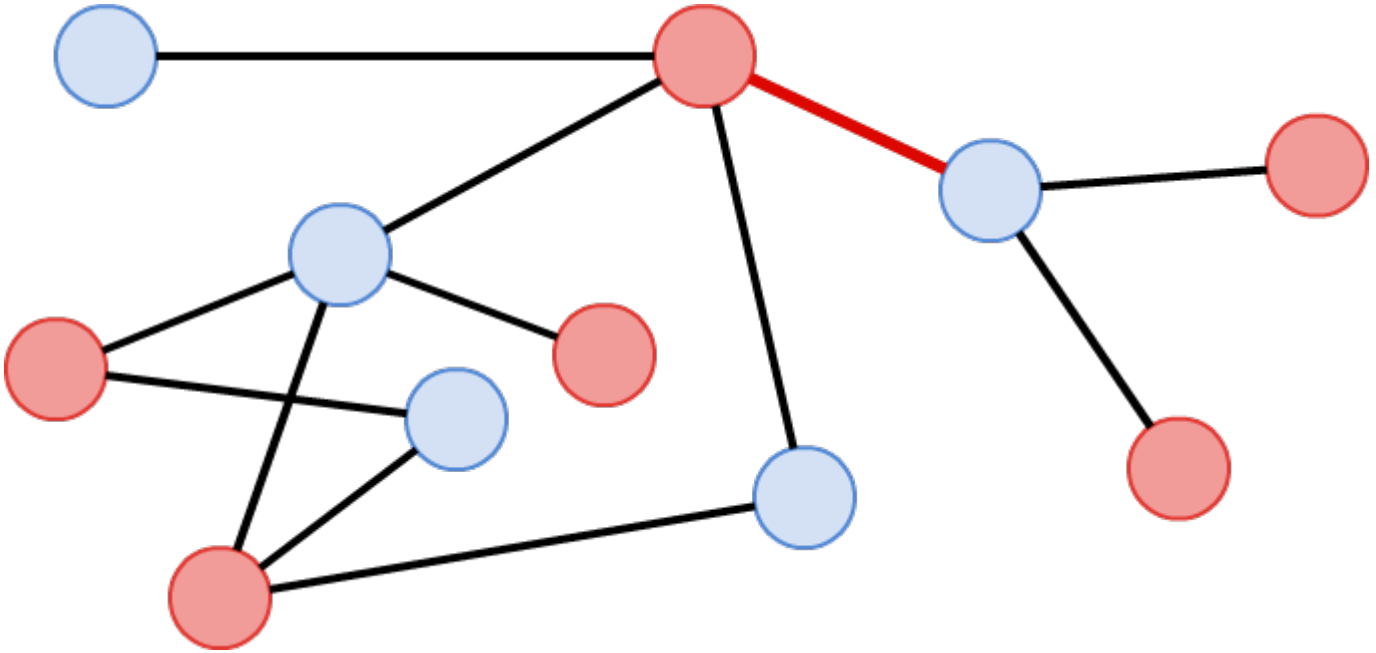
D-考察4

- パターン3
  - 異なる連結成分2つ（共に二部グラフ）を結ぶ辺
  - 追加後も必ず二部グラフ
  - ときには色を塗り替えなければならなくなる



D-考察4

- パターン3
  - 異なる連結成分2つ（共に二部グラフ）を結ぶ辺
  - 追加後も必ず二部グラフ
  - ときには色を塗り替えなければならなくなる



D-考察4

- パターン1, 2の処理は簡単そう
- パターン3をどうするか？
  - 具体的には色の塗り直しをどうするか？

D-考察4

- パターン1, 2の処理は簡単そう
- パターン3をどうするか？
  - 具体的には色の塗り直しをどうするか？

    解法1:マージテク

    解法2:縮約

D-満点解法1

- 各頂点について
  - どの連結成分に属すか？
  - 何色に塗られているか？
- 各連結成分について
  - 二部グラフか？

D-満点解法1

- 各頂点について
  - どの連結成分に属すか？

    UnionFindを使用
  - 何色に塗られているか？

    配列で管理
- 各連結成分について
  - 二部グラフか？

    配列で管理

D-満点解法1

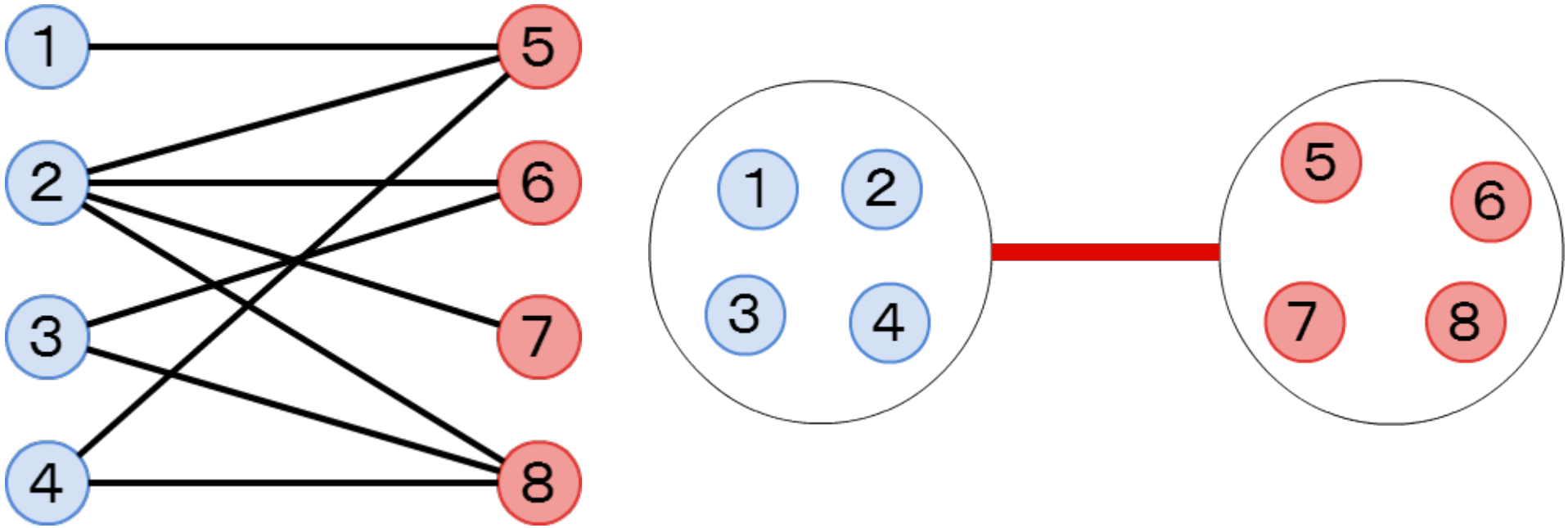
- パターン1, 2の辺については適当に処理する
  - 変更する情報は定数個なので、あまり計算量に影響しない
- パターン3の塗り直しをどうするか
  - 頂点数が少ない方を塗り直すようにすれば良い
  - いわゆる「マージテク」
  - どの頂点も塗り直されるタイミングで自分が属する連結成分の大きさが2倍以上になる
    - たかだかlogN回しか塗り直さない

D-満点解法1

- 注意
  - DFSの計算量はO(頂点数)ではなくてO(辺の数)
  - それならば頂点数ではなくて辺の数の大小でマージテクをするべきでは？
    - 結論から言うと問題ない
    - 先と同様の論法で各辺がDFSで走査される回数もO(logN)程度
  - 二部グラフかどうかわかっているので、全域木の辺のみを管理するという手もある

D-満点解法2

- 同じ連結成分(二部グラフ)の同じ色どうしを縮約する



D-満点解法2

- パターン1,2は満点解法1と同様
- パターン3が問題
  - 縮約のおかげで塗り替えが定数回で済む

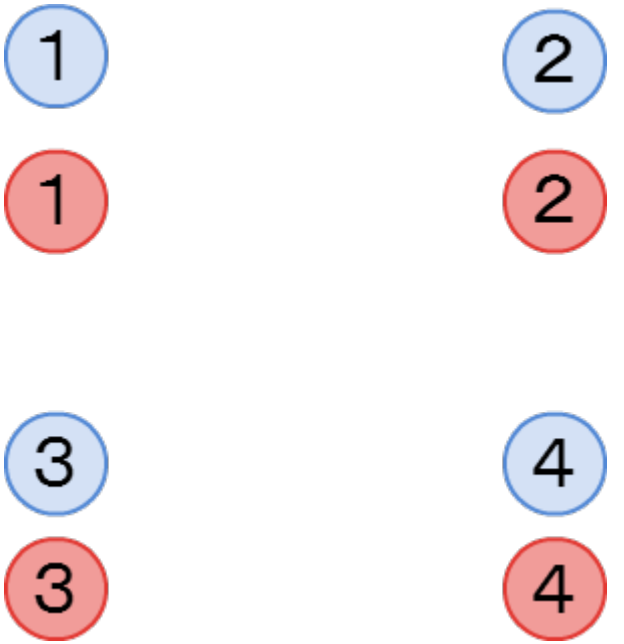
D-満点解法まとめ

- 満点解法1
  - マージテクを使っていちいち色を塗り直す
  - O(Q log N)
- 満点解法2
  - 縮約によって塗り直す色を定数個に減らす
  - O(Q α(N))　※α(N)はアッカーマン関数の逆関数

D-満点解法3

- もっとシンプルな解法がある
- 各頂点について
  - 赤く塗ったもの
  - 青く塗ったもの

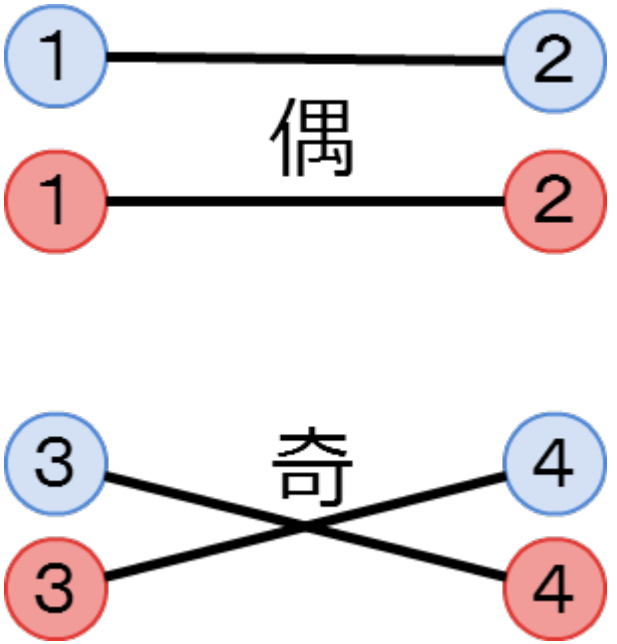
  をあらかじめ用意しておく



D-満点解法3

- もっとシンプルな解法がある
- 各頂点について
  - 赤く塗ったもの
  - 青く塗ったもの

  をあらかじめ用意しておく

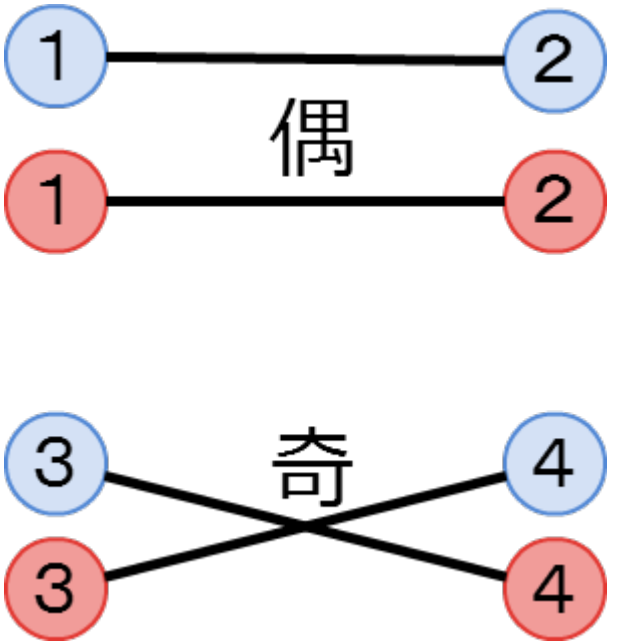


- 結ぶ辺の長さが偶数なら同じ色どうしを結ぶ
- 結ぶ辺の長さが奇数なら違う色どうしを結ぶ

D-満点解法3

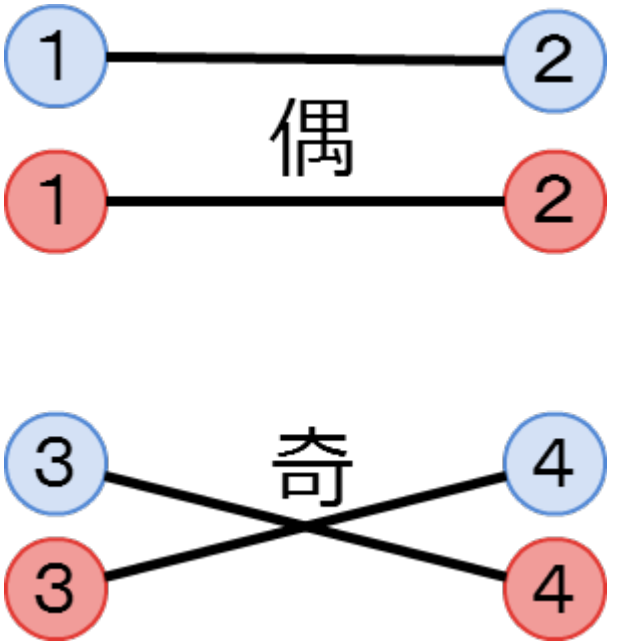
- 何をやっているのか？
- 同じ色の頂点どうしは偶数メートルでのみ行き来できる

  違う色の頂点どうしは奇数メートルでのみ行き来できる
- という状態を維持している



D-満点解法3

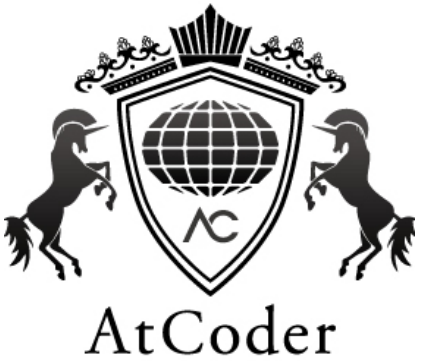
- 判定法
- 頂点X, Yが偶数メートルで移動できるかどうか
- Xの赤とYの赤が同じ連結成分に含まれる
- この二つが同値



D-満点解法3

- 実装
  - 辺を結ぶことと、同じ連結成分に属するかを判定するだけ
    - ただのUnionFind
    - 極めて軽実装
- 計算量
  - O(Qα(N))

AtCoder Regular Contest 035
解説



AtCoder株式会社 代表取締役
高橋 直大

競技プログラミングを始める前に

AtCoder

- 競技プログラミングをやったことがない人へ
  - まずはこっちのスライドを見よう！
  - http://www.slideshare.net/chokudai/abc004

A問題　高橋くんと回文

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

- 文字列 *s* が与えられる.
- *を自由な文字に置き換えて，sを回文にできるか判定
- *を置き換える文字はそれぞれ異なっていてもOK
- 制約
  - 1 ≦ |s| ≦ 1000

A問題　アルゴリズム

AtCoder

- 解法1
  - 前と後ろで対応する文字が同じになりうるか調べる
  - つまり，i 番目と |s|+1-i 番目の文字で，どちらかが * であるか，どちらも同じ文字であるか調べる(1 <= i <= |s|)
  - 1つでも同じになりえない文字の組があればNO，そうでなければYES
- 解法2
  - sとsをひっくり返した文字列tの間で，sのi番目とtのi番目が同じ文字になりうるか調べる.
  - 結局やっていることは同じ．この方が少し楽かも.

B問題　アットコーダー王国のコンテスト事情

1. 問題概要
2. 考察＆解法

©AtCoder Inc. All rights reserved.

B問題　問題概要

AtCoder

- 問題がN個ある
- それぞれの問題を解くのにかかる時間を知っている
- 解いた時間に依存するペナルティがある
  - 問題ペナルティ=コンテスト開始からその問題を正解するまでの時間
  - コンテストペナルティ=問題ペナルティの総和
- 全完するときの最小コンテストペナルティと，それを達成する解き方の数を求めよ
- 制約
  - 1 ≦ N ≦ 100000
  - 1≦(各問題を解くのにかかる時間)≦ 1000

B問題　考察＆解法

AtCoder

- コンテストペナルティ最小を達成するためには，
  かかる時間の小さい問題から解けば良い

→かかる時間順でソートして，シミュレーションする

(最大ケースで答えが32bit整数を超えるので注意)

- かかる時間が同じ問題は，解く順序を入れ替えても
  最小ペナルティが変わらない．

→かかる時間が同じ問題がx個あったら，解き方はx!通り

- 全体の組み合わせ数としては，それらの積が答え
- 計算量
  - ソートにO(n log n) ←標準ライブラリを使いましょう
  - その他 O(n)

C問題　アットコーダー王国の交通事情

1. 問題概要
2. 考察
3. 解法

©AtCoder Inc. All rights reserved.

C問題　問題概要

AtCoder

- N頂点,M本の辺から成る重み付き無向グラフがある
- このグラフにK本無向辺を新たに追加する
- 追加する度に，全点対間の距離の和を求めよ

- 制約
  - 1 ≦ N ≦ 400, 1 ≦ M ≦ 1000, 1 ≦ K ≦ 400
  - 1≦(各問題を解くのにかかる時間)≦ 1000

C問題　考察

AtCoder

- 全点間最短距離を求めるワーシャルフロイド法を用いる
→O(N^3)で全点間距離テーブルが計算可能
→辺が1つ増える毎にテーブルをO(N^3)で更新していてはTLE
- ある辺を既存の辺より小さいコストの辺に変更するときO(N^2)でテーブル更新可能
→コストがcの辺(a,b)を追加時，頂点i,j間の最短距離cost[i][j]は以下のように更新される

cost[i][j] = min(cost[i][j], cost[i][a] + cost[b][j] + c,
cost[i][b] + cost[a][j] + c)

※cost[][]は全点間最短距離を記録した二次元テーブル

C問題　解法

AtCoder

- まとめると
  - 最初にO(N^3)で全点間最短距離テーブルを構築
  - 辺の追加時に毎回O(N^2)でテーブル更新
- を行うことで満点を得れる

- 計算量: O(N^3 + KN^2)

- 余談ですが，ワーシャルフロイドのアルゴリズムはO(N^3)ですが，アルゴリズムが非常に単純なので，多少頂点数が多くても高速に動作することが多いです．

D問題　高橋くんとマラソンコース

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

- 南北・東西方向の道があり，交差点の間を北・東方向にのみ進める.
- 交差点上にN個のチェックポイントがある.
- 以下の2つのクエリーを処理
  - k_j 番目のチェックポイントを (a_j,b_j)に設定しなおす
  - チェックポイント$l1_j$からチェックポイント$r1_j$までの経路数と，チェックポイント$l2_j$からチェックポイント$r2_j$までの経路数のうち，どちらが多いか答える. ただし，多い方は少ない方の2倍以上.
- 制約
  - 2<=N<=2*10^5
  - 1<=(チェックポイントの座標)<=10^6

D問題　アルゴリズム

AtCoder

- 東西方向で東を正にX軸，南北方向に北を正にY軸を取ります．
- 隣り合うチェックポイントのx座標の差がdx，y座標の差がdyのとき，その2つのチェックポイント間の経路の数は

$$_{dx+dy}C_{dx} = (dx+dy)!/(dx!)/(dy!)$$

(dx個の→，dy個の↑の並べ替えの個数に等しいから)

- チェックポイントaからチェックポイントbまでの経路の数は，(aとa+1間の経路数)*(a+1とa+2間の経路数)*・・*(b-1とb間の経路数)
- 経路数はメチャクチャでかくなりそう．
  - (1,1) から (10^6,10^6) への経路数の時点でヤバイ
- 経路数の計算では掛け算・割り算しか出てこない．

D問題　アルゴリズム

AtCoder

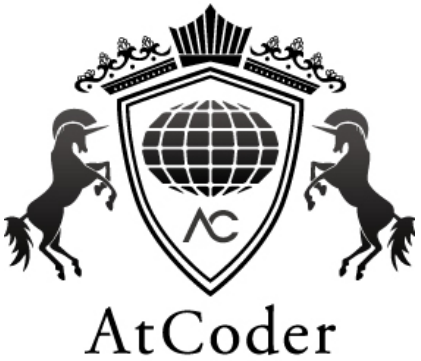
- 普通に計算するのはよくない
- logを取ってみよう！！
  - 0<a<b のとき log(a)<log(b) なので，大小関係が維持される.
  - log(a*b)=log(a)+log(b) , log(a/b)=log(a)-log(b) から，経路数は楽に計算できそう.
  - 数がそこまででかくならない
    - (1,1) から (10^6,10^6) までの経路数をwとおくと，log(w)は10^6ぐらい.
  - 大と小で2倍差があれば，logを取った値はlog(2)≒0.69以上差があるので誤差もOK

D問題　アルゴリズム

AtCoder

- 部分点解法では，番号が隣り合う2つのチェックポイントの間の経路数にlogを取ったものを毎回計算し，それらの和の大小を比較する．
  - 予め階乗にlogを取ったものを求めておく必要がある
- 満点解法では，セグメント木を使い，区間の和を求められるようにしておく．
  - チェックポイントの更新では，その前後の経路数を更新
  - 比較では，和の大小を比較

AtCoder Regular Contest 034
解説



AtCoder株式会社 代表取締役
高橋 直大

競技プログラミングを始める前に

AtCoder

- 競技プログラミングをやったことがない人へ
  - まずはこっちのスライドを見よう！
  - http://www.slideshare.net/chokudai/abc004

A問題　首席

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

- 受験生の人数Nと、それぞれの受験生の、「国語」「数学」「理科」「英語」「塗り絵」の5科目の点数が与えられる。
- 受験生の最終得点は、以下の式で表せられる
  - 国語＋数学＋理科＋英語＋（塗り絵*110/900）
- 最終得点が最も高い受験生の、最終得点を求めよ

- 制約
  - 2 ≦ N ≦ 3049

A問題　アルゴリズム

AtCoder

- 解法
  - 順番に点数を計算する
    - この時に、（塗り絵）*900/110などを整数演算ではなく、小数として計算出来るように工夫することに注意！
      - *900.0にするなど
  - 計算した中で、最も点数の高い点数を出力する
    - やり方はいろいろ
      - 配列に入れてソート
      - 最も高い点数を入れる変数を用意しておき、毎回max関数などで最大値を格納する。

B問題　方程式

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

B問題　問題概要

AtCoder

- Nの十進表記における各桁の数の和をf(n)で表す
- f(x) + x = Nとなるxを全て出力しなさい

- 制約
  - $1 \leqq N \leqq 10^{18}$

B問題　アルゴリズム

AtCoder

- 考察
  - f(x) + xについて考える
  - 全てのxについて、Nについて調べることが可能か？
    - 可能であれば、全部試せば良いので簡単！

B問題　アルゴリズム

AtCoder

- 部分点解法 (N≦1000)
  - xもf(x)も共に1以上の整数
    - つまり、x+f(x)=Nになるには、xはN以下の正整数であることが解る
  - よって、1以上N以下の整数を全て試すことで、部分点を取ることが出来る

B問題　アルゴリズム 部分点解法(N≦1000)

AtCoder

- 考察
  - xもf(x)も共に1以上の整数
    - つまり、x+f(x)=Nになるには、xはN以下の正整数であることが解る
  - よって、1以上N以下の整数を全て試すことで、部分点を取ることが出来る

B問題　アルゴリズム 満点解法

AtCoder

- 考察
  - f(x)の最大値はどれくらいか？
    - x=9999999999999999の時、f(x) = 9 * 17 = 153
    - これよりは大きくならなそう
  - つまり、調べるべき範囲は、N-153からNの間で十分
    - これは全探索可能！

B問題　アルゴリズム 満点解法

AtCoder

- 注意点
  - 数が大きいので、32bit整数型には収まらない
    - 64bit整数型などの、大きな数が格納可能な整数型を使おう！

C問題　約数かつ倍数

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

C問題　問題概要

AtCoder

- A,Bが与えられる
- A!の約数であり、B!の倍数である数の個数を出力せよ

- 制約
  - $1 \leqq B < A \leqq 10^{18}$
  - $A - 100 \leqq B$

C問題　アルゴリズム 部分点1

AtCoder

- A,B≦15のとき
  - 15! = 1307674368000
    - 全通り試すことは出来ない！
    - 何か工夫しなければいけない
  - Aの約数を全て列挙して、そのうちでBの倍数になっているかどうかを1つ1つチェックすれば間に合う？
    - 約数を、O(√A!)くらいで列挙出来れば可能！

C問題　アルゴリズム 部分点1

AtCoder

- 簡単な約数の列挙方法
  - 例えば、A=5の時、A! = 120（これを暫定的にFAとする)
    - これの約数を列挙することを考える
    - 1から√FA (11くらい)までのうち、FAの約数であるものをループで探す
      - If(FA % I == 0)みたいな感じ
    - 見つかった約数をiとすると、FA/iもFAの約数である
      - 1が約数なので、120も約数
      - 2が約数なので、60も約数
      - 3が約数なので、40も約数
      - ・・・・
    - これを、√FAまで繰り返すことによって、全ての約数を列挙できる！
    - 各約数に対して、B!の倍数になっているかどうかチェックすれば良い

C問題　アルゴリズム 部分点2

AtCoder

- 考察
  - もっと高速化するには、根本的な部分を見直す必要がある。
  - A!の倍数かつ、B!の約数とは何か？
    - (A! / B!)の約数に、B!を掛けたものである！
    - 例えば、A=5, B=3であれば、5*4の約数に、3*2*1を掛けたものである！
      - 20の約数は1,2,4,5,10,20なので、解は6,12,24,30,60,120である
    - まずはこれを利用する

C問題　アルゴリズム 部分点2

AtCoder

- さらに考察
  - A,Bが大きい時、全ての約数を列挙するのは難しい
    - 約数の個数をもっと上手に数えよう！
  - 約数を効率的に数えるには、素因数分解をすると良い！
  - 例えば、20の約数の個数を数えたい時、
    - 20 = 2 × 2 × 5と表せる。
    - この約数は、(2の0,1,2乗のいずれか) × (5の0,1乗のいずれか)であることが解る
      - 例えば、2の1乗 × 5の1乗を選ぶと、約数の1つである10が出来る
    - つまり、この組み合わせの個数を数えてあげれば良い！
      - これは、各素因数に対して、その指数が分かれば良い

C問題　アルゴリズム 部分点2

AtCoder

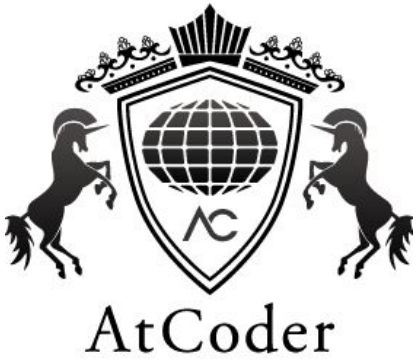
- A,B≦10^6, A-B≦100の時
  - A! / B!について、約数の個数を調べたい
  - 計算式を考えて、10^6以上の素因数は、この数には含まれない
    - よって、各素因数について、いくつ存在したかを配列で管理してあげれば良い
  - A!/B!を素因数分解するのは難しいが、B+1からAまでの整数1つ1つに対して、素因数分解してあげるのは可能
    - であれば、この1つ1つに対して素因数分解を行い、指数の計算をしてあげれば良い。1つ1つの計算量は、部分点1の約数全列挙と同じようなアルゴリズムを用いると、O(√A)
  - 指数が求まったら、あとは先ほどの公式を用いて、約数の個数を計算すれば解ける

C問題　アルゴリズム 満点解法

AtCoder

- A,B≦10^9, A-B≦100の時
  - 先ほどのように、素因数の最大値が小さくはならない
    - だが、種類数自体はそこまで多くない！
    - よって、配列で管理していた部分を、連想配列に直してあげれば良い！
  - 計算量は、素因数分解がO(√A)、その回数が100回で、1回の素因数分解で発生する素因数は、O(logA)個程度なので、十分に間に合う

AtCoder Regular Contest 034
問題D 解説 (暫定版)



AtCoder株式会社 代表取締役
高橋 直大

D問題　インフレゲーム

1. 問題概要
2. 解法

©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

- 赤い数字で整数が書かれたカードがA枚
- 青い数字で整数が書かれたカードがB枚
- 何も書かれていないカードがC枚
- 以上をシャッフルした山札を上から引いて行く
- 赤い数字aを引くと+a点
- 青い数字bを引くと得点がb倍
- 何もかかれてないカードを引くとゲームオーバー
- 最終得点の期待値は?

- 制約
  - 1 ≦ A,B,C ≦ 50
  - 1 ≦ カードの各数字 ≦ 100

D問題　部分点解法1 (5点)

AtCoder

※出題者の都合により「超」駆け足バージョンです。
申し訳ありません。by evima

- 1 ≦ A,B,C ≦ 3
- 合計9枚以下
- 山札のパターンを全て試す
- 時間計算量 O((A+B+C)! * poly(A+B))

D問題　部分点解法2 (20点)

AtCoder

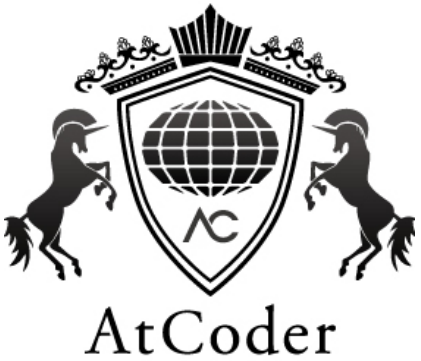
- 1 ≦ A,B,C ≦ 8
- 赤、青の合計16枚以下
- dp[今までに引いたカードの集合]
- 時間計算量 O(2^(A+B) * poly(A+B))

- $1 \leqq A, C \leqq 50, 1 \leqq B \leqq 8$
- よく考えると、赤いカードに書かれている数字の平均を R として、全ての赤いカードにRと書かれていても答えは同じ → 区別不要！
- dp[引いた赤いカードの枚数][引いた青いカードの集合]
- 時間計算量 O(2^B * poly(A+B))

AtCoder

- 1 ≦ A, B, C ≦ 50
- よく考えると青いカードも区別不要
- 青いカードをi枚引いたときのそれらの積の期待値をB[i]として、その値を計算に使う
- dp[引いた赤いカードの枚数][引いた青いカードの枚数]
- 時間計算量 O(poly(A+B))

AtCoder Regular Contest 033
解説



AtCoder株式会社

A問題　解説

隠れた言葉

問題概要

- 長さ N の文字列の部分文字列の個数を求めよ。（N <= 1000）

- 例えば N = 3 なら、文字列を“ABC”とすると、
- “A”, “B”, “C”, “AB”, “BC”, “ABC”
- の 6 個の部分文字列がある。

解法

- いろいろな考え方がありますが、いずれも、
- 1+2+3+…+N （= N*(N+1)/2）
- のような式を計算すれば良いということがわかります。
- これをループを使って計算したり、数式で一発計算をしたりしましょう。

- 考え方の例をいくつか紹介します。

考え方1

- 部分文字列の長さごとに場合分けしてみる。
- N = 3 の例だと、
- 長さ 1 の部分文字列: 3 個
- 長さ 2 の部分文字列: 2 個
- 長さ 3 の部分文字列: 1 個
- つまり長さ N の文字列の部分文字列は、
- N + (N-1) + … + 2 + 1 個

考え方２

- 取り出す区間の左端と右端を全て試してみる。
- N = 3 の例だと、
- [A]BC, [AB]C, [ABC], A[B]C, A[BC], AB[C]
- という6つの取り出し方がある。
- ２重ループで位置を全て試してみて、
- 右端が左端よりも右にあるものを数える。

考え方3

- N = 1 : 1 個
- N = 2 : 3 個
- N = 3 : 6 個
- N = 4 : 10 個
- N が 1 増えるごとに答えが N だけ増えるらしい？

→ N-1 が N に増えた時に増える部分文字列は、最後の文字を含むような部分文字列で、それは N 個あるから。

- よって、1 + 2 + … + N という式になります。

B問題　解説

メタ構文変数

問題概要

- ２つの整数の集合A,BのJaccard係数を求めよ。
- Jaccard係数とは、
- AとBの共通部分の要素数 / AとBの和集合の要素数（size(A∩B)／size(A∪B)）
- 40点：A,Bの要素数≤1000、A,Bに含まれる整数≤10^5
- 70点：A,Bの要素数≤10^5、A,Bに含まれる整数≤10^5
- 満点：A,Bの要素数≤10^5、A,Bに含まれる整数≤10^9

40点解法

- A,Bの共通部分の要素数をXとすると、A,Bの和集合の要素数は、
- Aの要素数(N_A)+Bの要素数(N_B)-X
- と計算できる。
- Xは、Aの要素とBの要素をそれぞれ舐める2重ループを回し、一致したものの個数を数えると求まります。

70点解法

- ２重ループを１重ループにしたい。
- Aの要素を舐めて、その要素がBに含まれるのかどうかを高速に判定したい。
- 要素数が10^5+αの配列を作って、Bに含まれる数の場所に1を代入しておき、それ以外の場所に0を代入しておく。
- すると、配列の値を参照するだけで、ある数がBに含まれるのかどうかを判定できる。

満点解法

- 数の上限が大きいため、配列が取れない。
- Bをソートしておき、二分探索を用いる。
- set（集合を扱うクラス）を用いる。
- などの方法で、ある数がBに含まれるのかどうかを高速に判定できる。

C問題　解説

データ構造

問題概要

- 数の集合に対するクエリを Q 個処理せよ。
- タイプ1：集合に x を追加する。
- タイプ2：集合の x 番目の数を答え、削除する。
- Q <= 200000, X <= 200000

背景

- 「えーっと、とりあえず set に突っ込んで・・・」

- Set には「X 番目の数」を
  取り出す機能はついていません。

解法

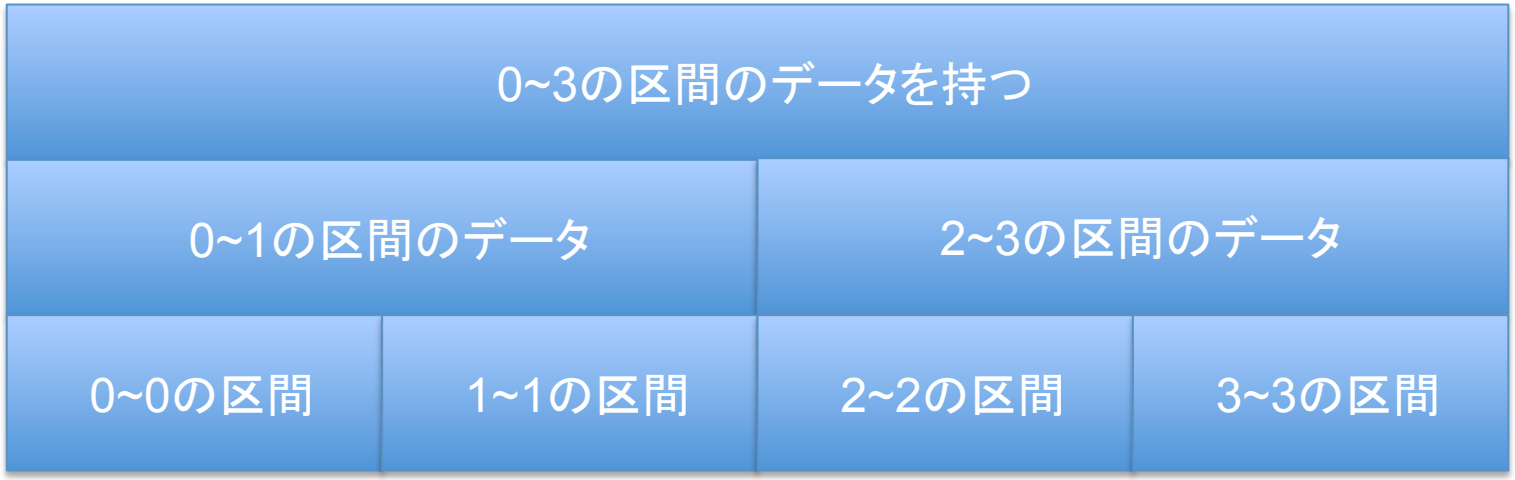
- 「X 番目の数」を言い換えてみる。
- →「それ以下の数が X 個であるような数のうち最も小さい数」
- ということは、
- 「ある数以下の数がいくつあるか」が高速に求まるデータ構造があれば二分探索で求めることができそう。

データ構造

- 「ある数以下の数がいくつあるか」を高速に求めるデータ構造？
- 区間の和を求める segment tree を使う。
- （Fenwick tree (BIT) でも良い。）
- この segment tree で対応するクエリは、
- add(i, v) : 場所 i に v を足す。
- Sum(l,r) : 場所 l から場所 r までの和を求める。
- タイプ１では、add(X,1) をする。
- タイプ２では、sum(0,i) を使って二分探索を行って答えを求め、add(答え,-1) をする。

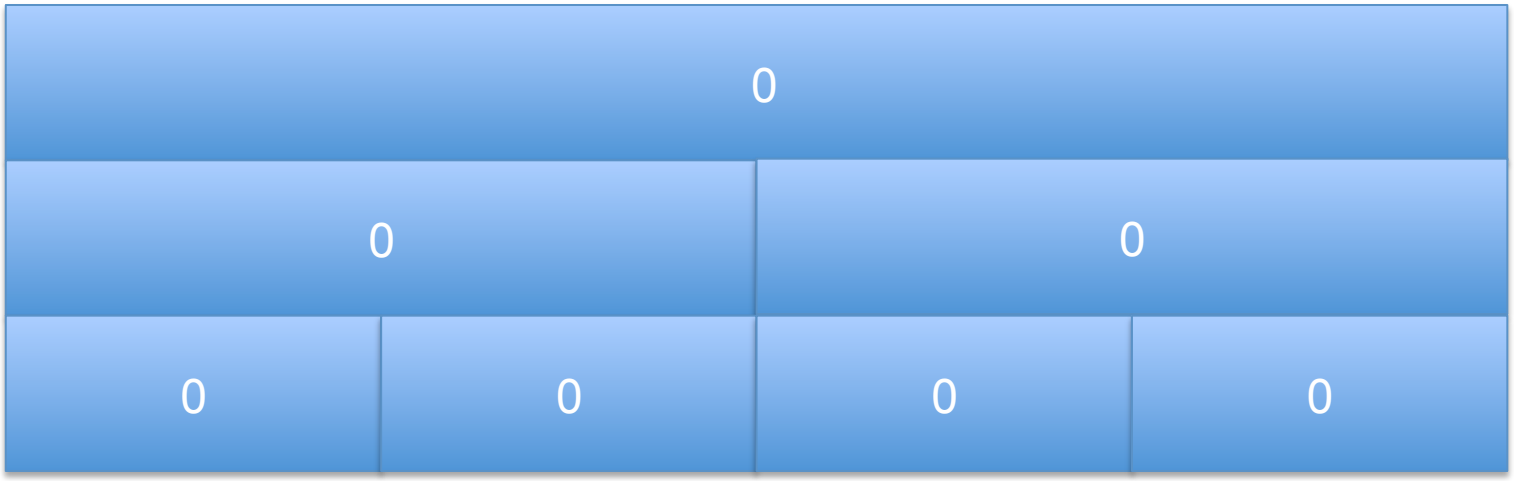
Segment tree

- このような二分木を作って区間を管理するデータ構造です。
- 今回の例だと、各ノードに「区間内の数の和」を持てば良いです。



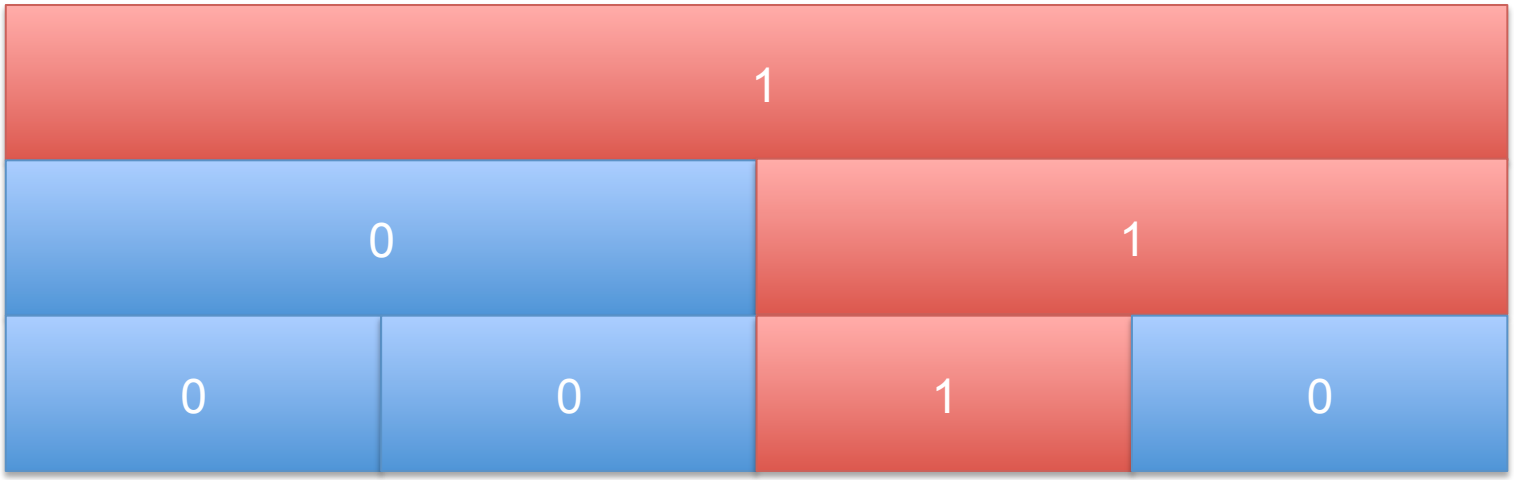
Segment tree

- 初期状態



Segment tree

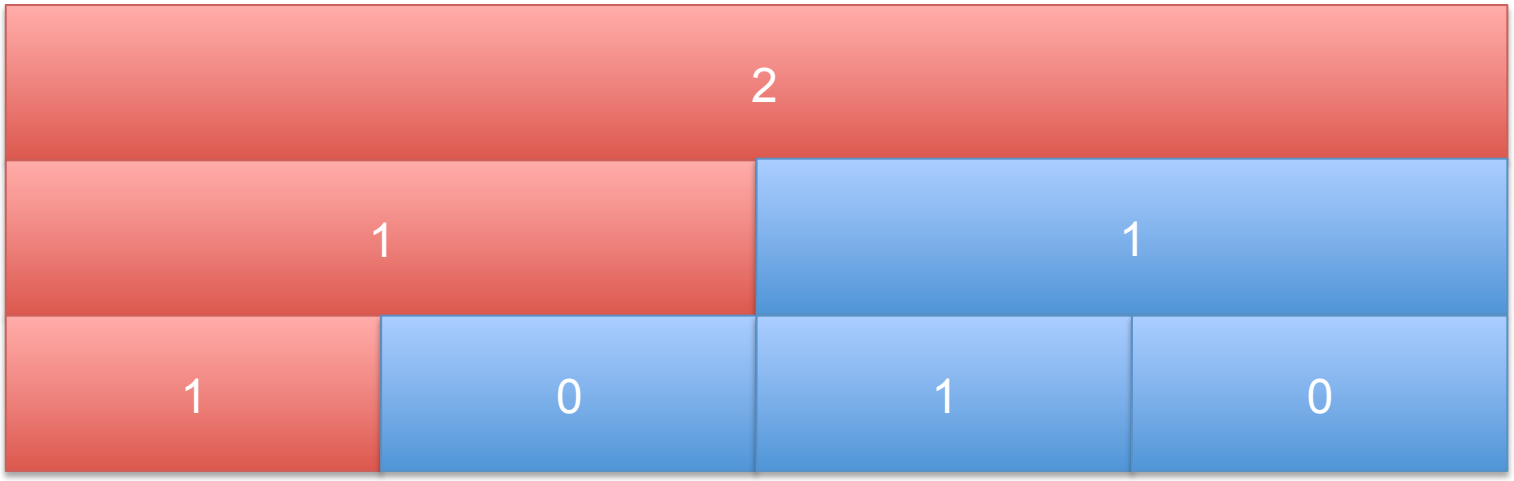
- Add(2,1)というクエリ（場所2に1を足す）が来ると・・・



- 赤い色をつけたノードに1を足します。

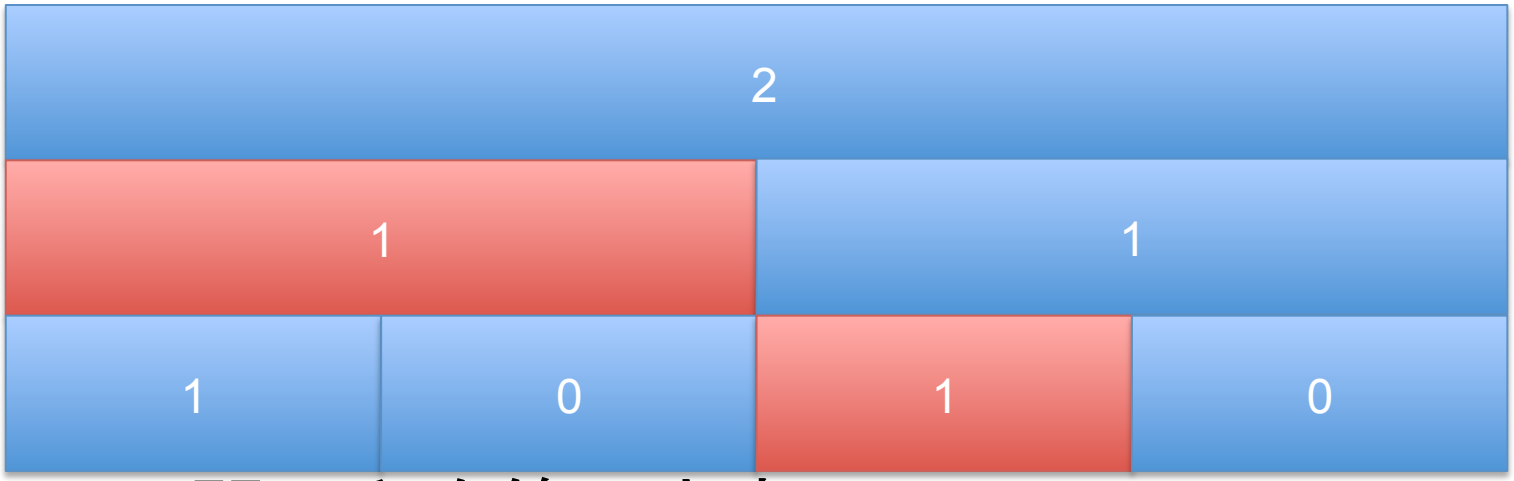
Segment tree

- 続けてAdd(0,1)というクエリ（場所0に1を足す）が来たら、



Segment tree

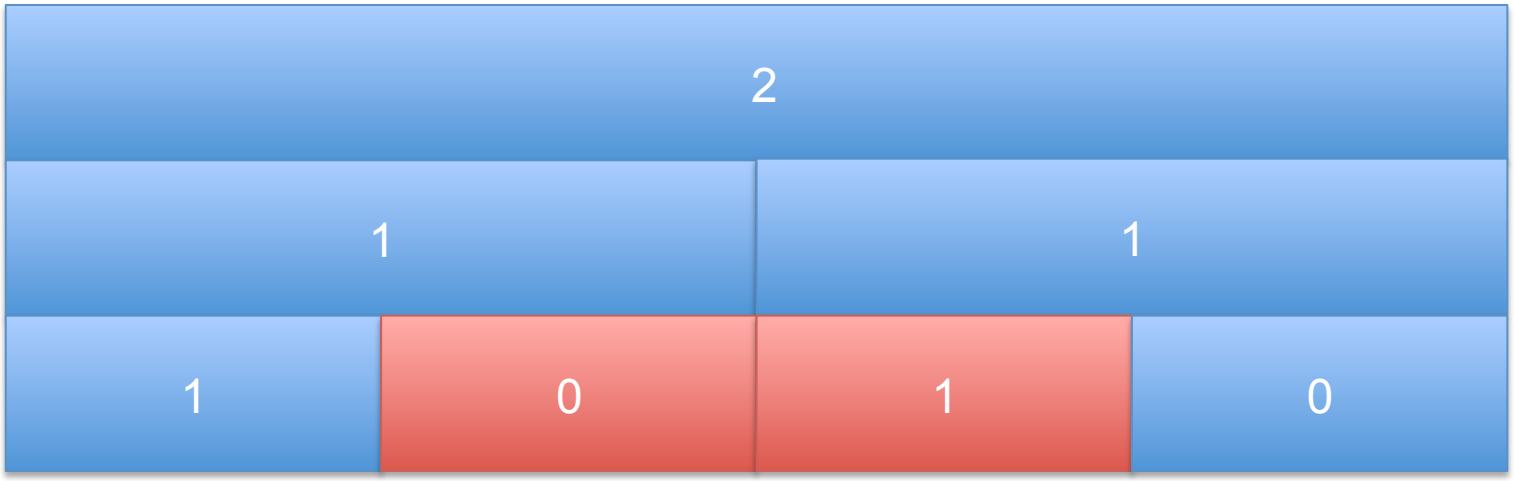
- 次に、Sum(0,2)というクエリ（0~2の区間の和を求める）が来たら・・・



- 赤い区間の和を答えます。
- 1+1 = 2 ！

Segment tree

- 続けて、Sum(1,2)というクエリ（1~2の区間の和を求める）が来たら・・・



- 0+1 = 1 ！

別解

- 平方分割
- 定数Dを決めて、1..D,D+1..2D,2D+1..3D,…ごとに分割して、N/D個の配列やリストを使って愚直に管理する。
- 例えばタイプ1のクエリは、$kD < X \leq (k+1)D$だったとすると、k番の配列にO(D)で挿入する。

別解

- タイプ２のクエリなら、「1～k-1番の配列の要素数の和がX未満」となるような最小のkを見つけます。愚直に前から見て行っても O(N/D) の計算量で見つかります。
- そして、k番目の配列のうち X-「1～k-1番の配列の要素数の和」 番目の数が答えとなるのでそれを出力し、O(D)で削除します。

別解

- タイプ１のクエリで O(D)
- タイプ２のクエリで O(max(N/D,D))
- max(N/D,D) を最小化するような D は√Nであり、Dに√N を用いると計算量は、O(Q √N) となります。
- このように、タスクを分割して計算量をO(√N)にする手法を平方分割と呼びます。汎用性の高いテクニックなので、覚えておいて損はないでしょう。

D問題　解説

見たことのない多項式

問題概要

- N 次多項式 P(x) がある。
- P(0) ～ P(N) の値が与えられる。
- P(T) の値を求めよ。(mod 1,000,000,007)
- T ≤ 10^9
- 40点 : N ≤ 100
- 80点 : N ≤ 3000
- 満点 : N ≤ 10^5

40点解法

- N=3, P(0)=1, P(1)=3, P(2)=7 の場合を考える。
- $P(x)=a_2x^2+a_1x+a_0$として、xに0,1,2を代入する。
- $0\times a_2+0\times a_1+1\times a_0 = 1$
- $1\times a_2+1\times a_1+1\times a_0 = 3$
- $4\times a_2+2\times a_1+1\times a_0 = 7$
- という連立方程式を解けば係数が計算できる。
- 連立方程式は、掃き出し法を使えば解くことができる。

80点解法

- ラグランジュ補間という手法を使う。
- $Q_i(x) = (x-0)*(x-1)*\ldots*(x-N)/(x-i)$
- という式を考える。
- P(x)はこの式を使って、
- $P(x) = C_0 \times Q_0(x) + C_1 \times Q_1(x) + \ldots + C_N \times Q_N(x)$
- という形で表すことができる。

80点解法

- $Q_i(x)$という式の特徴として、
- $Q_i(0)$～$Q_i(N)$のうち$Q_i(i)$以外の値が 0 です。
- この特徴のため、
- $P(i) = C_0 \times Q_0(i) + C_1 \times Q_1(i) + \ldots + C_N \times Q_N(i)$
- $= C_i \times Q_i(i)$
- となり、
- $C_i = Q_i(i) / P(i)$
- としてC_iを計算することができます。

80点解法

- 流れとしては、
- $C_i = Q_i(i) / P(i)$ を使ってC_0～C_Nを求める。
- $P(T) = C_0Q_0(T)+C_1Q_1(T)+…+C_NQ_N(T)$
- を計算して P(T) を求める。
- 各 Q(i) は O(N) で計算でき、O(N) 回計算するため、合計で O(N^2) となります。

満点解法

- 求めたい$Q_i(i)$の値は、
- $Q_0(0)$～$Q_N(N)$
- $Q_0(T)$～$Q_N(T)$
- これらの計算をまとめてO(N)で行いたい。

満点解法

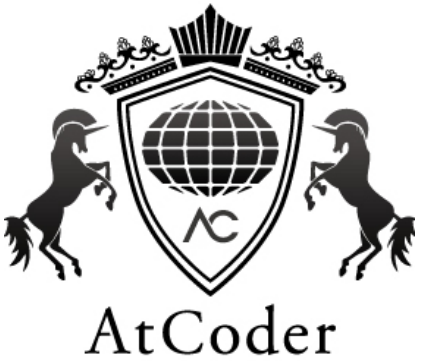
- $Q_0(0)$～$Q_N(N)$
- $Q_i(i)$と$Q_{i+1}(i+1)$が似ていることを利用する。
- $Q_i(i) = (i-0)*\ldots*(-1)*(1)*\ldots*(i-N)$
- $Q_{i+1}(i+1) = ((i+1)-0)*\ldots*(-1)*(1)*\ldots*((i+1)-N)$
- $(i-N)$の項が減り、$((i+1)-0)$の項が増える。
- その差分だけを計算して$Q_i(i)$から$Q_{i+1}(i+1)$を計算していけばよい。

満点解法

- $Q_0(T)$～$Q_N(T)$
- $Q_i(T) = (T-0)*(T-1)*\ldots*(T-N)/(T-i)$
- $N \leq T$ なので $(T-i)$ は 0 になりません。
- そのため、あらかじめ$(T-0)*(T-1)*\ldots*(T-N)$から$(T-i)$を割ればいい。

- これで、O(N log mod) 解けるようになります。（割り算にO(log mod)かかります）

AtCoder Regular Contest 032
解説



AtCoder株式会社 代表取締役
高橋 直大

競技プログラミングを始める前に

AtCoder

- 競技プログラミングをやったことがない人へ
  - まずはこっちのスライドを見よう！
  - http://www.slideshare.net/chokudai/abc004

A問題　ホリドッグ

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

- Nが与えられる
- 1+2+…+Nの素数判定せよ

- 制約
  - 1 ≦ N ≦ 999

A問題　アルゴリズム

AtCoder

- 解法1
  - S=1+2+…+Nをforループで計算する
    - N=1000のとき最大でS=1+2+…+1000 = 500500
  - それに対して素数判定をforループで行う
    - 2～S-1の範囲でSを割り切る数がないか確かめる
- 解法2
  - S=1+2+…+N=$\frac{N(N+1)}{2}$ (総和の公式)
    - 式から明らかにNが小さいとき以外は素数にならなさそう
      - N=1のときは1なので素数ではない
      - N=2のときは素数
      - N>2のとき常に合成数
    - N=2のときだけ素数，それ以外のとき非素数なので，その事実にしたがって出力すればよい

B問題　道路工事

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

B問題　問題概要

AtCoder

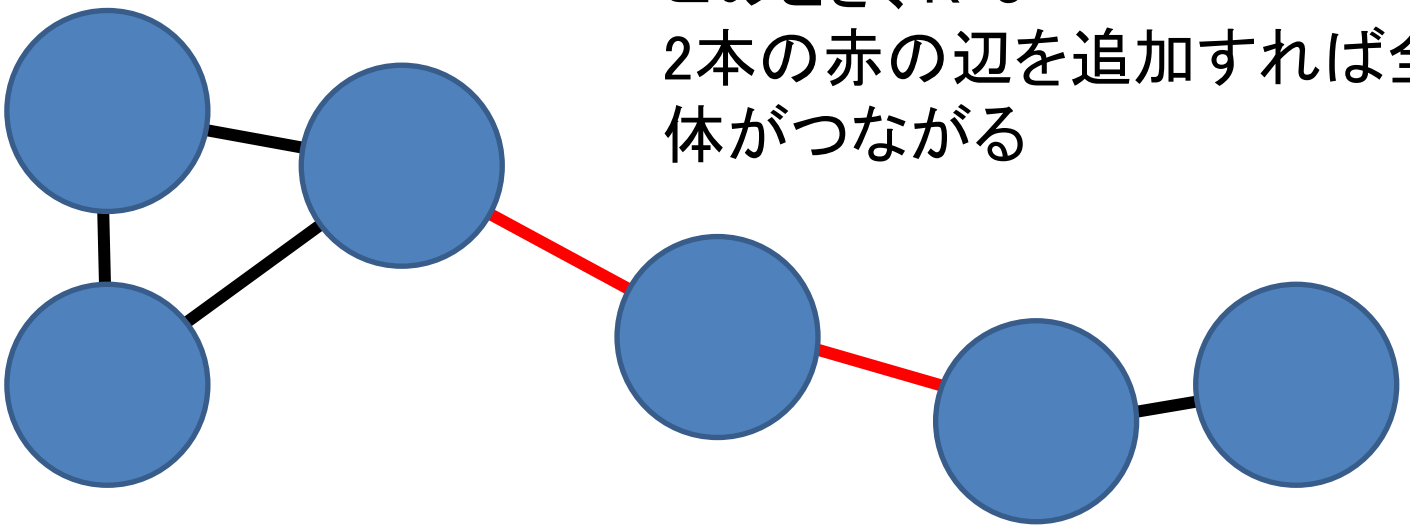
- N 頂点 M 辺の無向グラフが与えられる
- グラフに最小でいくつ辺を追加すれば、全体がつながるか調べる
- 制約
  - 0 ≦ N,M ≦ 100000

B問題　アルゴリズム

AtCoder

- 考察
  - グラフのつながっている部分がK個あるとき、最小でK-1本の辺があれば全体をつなげられる。
  - 1本つながっていない部分の間に辺を追加すると、Kは1減るから。

黒がもともとある道路
このとき、K=3
2本の赤の辺を追加すれば全体がつながる



B問題　アルゴリズム

AtCoder

- なので、グラフのつながっている部分(連結成分と呼ばれる)の数を数えればよい。
- 解法1
  - Union-Findですべての辺をつないだ後、Union-Find木でのroot の数を数える(具体的には、頂点 v のrootが v のものを数えると楽。連結成分ごとに root は共通で、自身が root となるものがちょうど 1 つあるから)
- 解法2
  - 今までででまだ訪れていない頂点を見つけたら、DFSを使いそこから行ける頂点をすべて訪れて、連結成分の数を1つ加える etc…
- どちらも(ほぼ)線形時間

C問題　仕事計画

1. 問題概要
2. アルゴリズム

©AtCoder Inc. All rights reserved.

C問題　問題概要

AtCoder

- 時刻の区間がN個与えられます。区間は順に1からNまで番号付けられています。
- i 番目の区間は a_i から b_i まで。
- 互いに被らない条件の下で、なるべく多くの区間を選びたい(端点は被ってもOK)
- 複数あるときは、区間の早い順に並べた時の辞書順最小となるものを1つ出力。
- 制約
  - 0 ≦ a_i, b_i ≦ 1000000
  - 1 ≦ N ≦ 100000

C問題　アルゴリズム

AtCoder

- これ蟻本でやったやつだ！(区間スケジューリング問題, p43)
  - 蟻本を読もう！
  - 具体的には、終了時間が早い順にソートし、貪欲に区間を選んで行けば、区間数最大の解が1つ得られる
- しかし、辞書順最小での構成を1つ出力しなければならない
- なるべく多く取れる取り方のうち、辞書順最小のものを求めるときは、最初に取れるものを最小化し続ける必要がある

C問題　アルゴリズム

AtCoder

- 時刻 i それぞれに対し、(i 以降で最大でいくつ取れるか?,最大の取り方のうち、次に取れる最小の仕事の番号はいくつか?) のペアが更新すればできそう
  - i から始めるとき、次に取る区間は辞書順最小の条件から、(最大の取り方のうち、次に取れる最小の仕事の番号) となるから
- このペアを dp[i] とおく

C問題　アルゴリズム

AtCoder

- dp[i] を求めるとき、
  - 時刻 i から始まる区間を使うとき、時刻 i から始まり、b で終わる番号idの区間を使う場合、(b以降で取れる区間の最大数+1,id) が候補
  - 時刻 i から始まる区間を使わないとき、(時刻 i+1 以降で取れる区間の最大数, そのうちの最小の仕事の番号) が候補
  - 以上のペアから、取れる区間の最大数で比較し、多いものを選ぶ。それが同じなら、次に取る仕事の番号が小さいものを選ぶ。
    - 番号が同じ区間はないので、このペアが同じになることは無い
- 以上のことから、dp[i] の更新に必要なのは i+1 以降のdpの情報
- よって、dp[i] を後ろから更新していけばよい

C問題　アルゴリズム

AtCoder

- dp[0] が求まったら、0 以降で取れる区間の最大数が1つ目の答え。
- 復元するときは、今の時刻以降で最大の取り方のうち最小の番号の区間をとり続けていく
  - dp のペアの2番目の情報にこれがそのまま入っている
  - 最初に “今の時刻”=0 とし、次に取る区間を答えに追加し、”今の時刻”を最後に取った区間の終端に更新
  - これを、取れる区間が無くなるまで繰り返す
- 合計でもO(N)!

D問題　アットコーダーモンスターズ

1. 問題概要
2. 考察
3. アルゴリズム

©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

- N匹のモンスターがいる
- 各モンスターには攻撃力と防御力というパラメータがある
- ちょうどK匹のモンスターを買いたい
- あるモンスターの対について，攻撃力の差と防御力の差の最大値を不安定度と定義
- K匹からなるチームに含まれるペアの不安定度の最大値(=チームの不安定度)を最小化し，それを達成する組み合わせ数 mod 100000007 を出力しろ.

- 制約
  - 1 ≦ K ≦ N ≦ 100,000
  - 0 ≦ 各モンスターの防御力,攻撃力 ≦ 3000

D問題　考察

AtCoder

- まず，チームの不安定度の最小値を求めたい.
- モンスターの(攻撃力,防御力)を二次元平面に点としてプロットしてみる
- あるチームの不安定度をxと決めたとき，
  - 一辺xの正方形領域に含まれる点の数がK以上であるようなところ

が存在すれば，不安定度x以下にできることが分かる.

- これで二分探索したらよさそう.
- 任意の正方形領域に含まれる点の数を高速に計算するため，予め二次元累積和を計算しておく.

D問題　考察

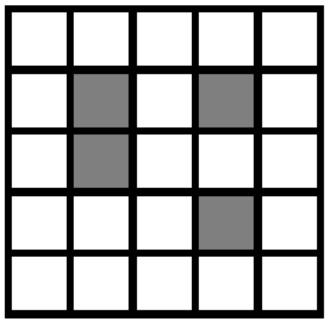
AtCoder

- 次にチームの不安定度の最小値mを達成する組み合わせ数を求めたい.
- 単純に全ての領域に対してKコを選ぶ組み合わせの数を計算するのは，重複が発生しダメ.
- ある点を角(x,y)と決めたときに，その点を角にした正方形(左上(x,y)右下(x+m,y+m))のときにしか達成できないような選び方を計算することで重複なく数えられる
- そのような選び方は2パターンある(次スライド)

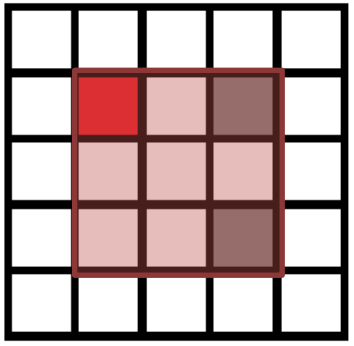
D問題　考察

AtCoder

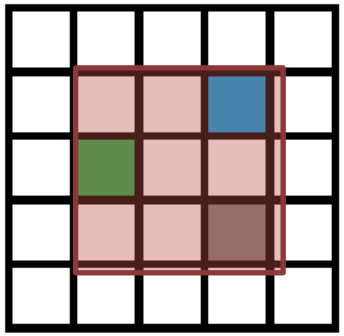
- パターン1 : 選んだ角(x,y)に1つ以上の点がある
- パターン2 : 選んだ角(x,y)に点は無いが，(x,?)と(?,y)に1つ以上ずつ点がある.



■:点がある
(実際には複数コある可能性もある)



ケース1(3通り)



ケース2(1通り)

↑4つの点がある適当な例(K=3とする)

D問題　考察

AtCoder

- 説明のために
  - 正方形領域の全ての点の数をall
  - 角(x,y)の点をcorner
  - 左端領域の点と上端領域の点の数をleft,top

  と表す

[パターン1の個数]

- nCr(all,K) - nCr(all-corner,K)

corner以外でKコ

[パターン2の個数]

- nCr(number-corner,K)
  - nCr(number-corner-top,K)
  - ncr(number-corner-left,K)
  + nCr(number-corner-(top+left),K)

cornerとtop以外でKコ

cornerとleft以外でKコ

cornerとleftとtop以外でKコ
(引きすぎた分を足す,包除原理)

D問題　アルゴリズム

AtCoder

- モンスターのパラメータの最大値をM(<=3000)とする
- Nコの点をプロットする → O(N)
- 二次元累積和を計算しておく → O(M^2)
- チームの不安定度の最小値mを二分探索で求める
  → 累積和を用いて O(M^2 log M)
- 組み合わせ数を求める
  → O(M^2)

nCrの計算は予め0≦k≦Nについてk!と(k!の逆元)をmod 10^9+7で計算しておくことでO(1)

- 全体として O(M^2 log M)

ARC #031 解説

解説スライド：水野尚人（not）

問題A

名前

問題概要

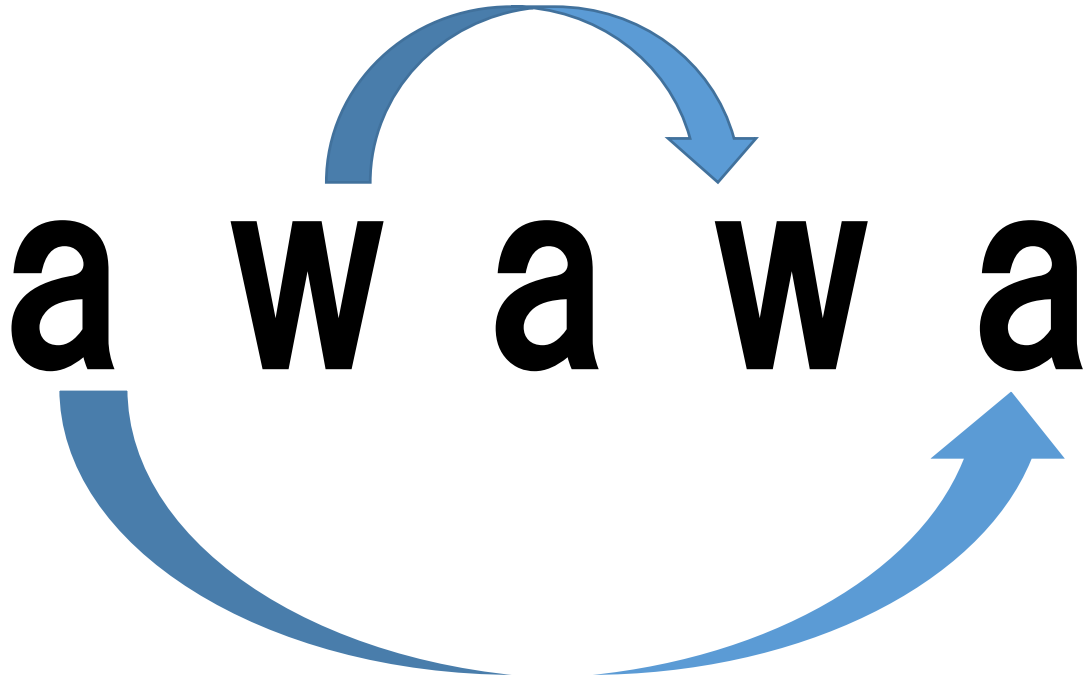
文字列が回文かどうか判定する

文字列の長さ≦１００文字

類題：ARC #019 B - こだわりの名前

解法１

前と後ろで対応する文字が同じか調べる



解法２

ひっくり返した文字列と同じになるか調べる

chokudai → iadukohc

chokudai ≠ iadukohc

問題B

埋め立て

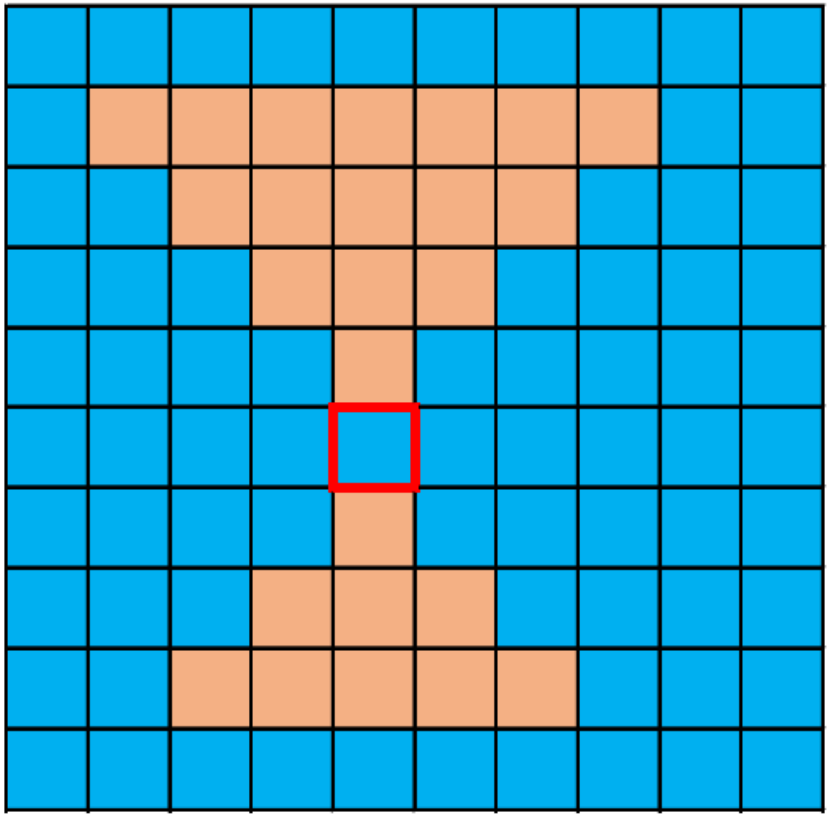
問題概要

１０×１０のグリッドが与えられる
　　各マスは陸地または海

1マスだけ海を陸地にすることで、
陸地を全部繋げられるか？

入力例１

赤枠のマスを埋め立てると
１つの島になる



解法

どこを埋め立てるかを全探索して、
１つの島になっているか調べる

１つの島になっているかは、あるマスから
すべてのマスにたどり着けるかを幅優先探索や
深さ優先探索で調べれば良い

問題C

積み木

問題概要

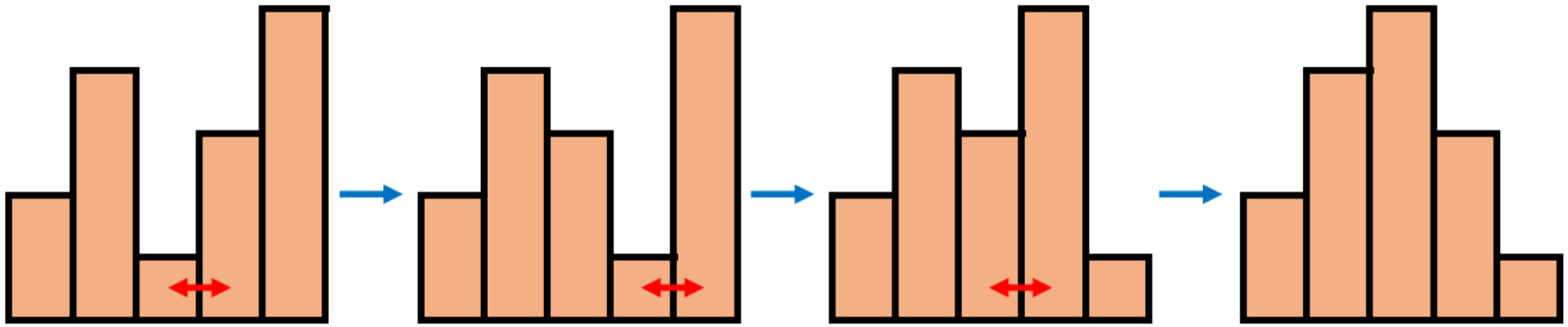
N個の積み木が１列に並べられている

隣り合う２個の積み木を並べ替えて、一番高い積み木から順に左右へ低くなるよう並び替える

最小の交換回数は？

N≦100,000　（部分点：N≦100）

入力例２



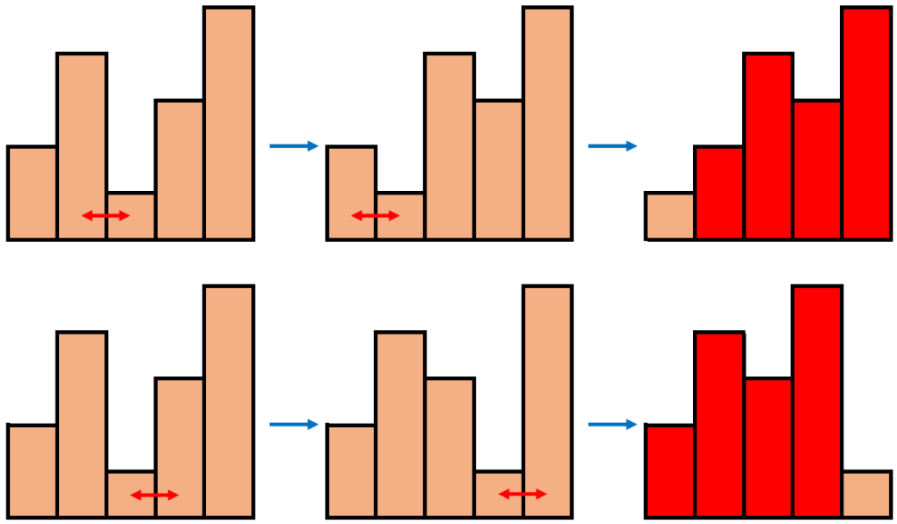
解法

まず１番小さい積み木だけ移動させてみる

　左右どちらかの端に移動させないといけない

残った部分は同じ形をしている

　近い方に移動させれば良い



解法

２番目、３番目、…と同じ操作をしていくと
最小の交換回数が求められる

愚直にやると$O(N^2)$ → ３０点

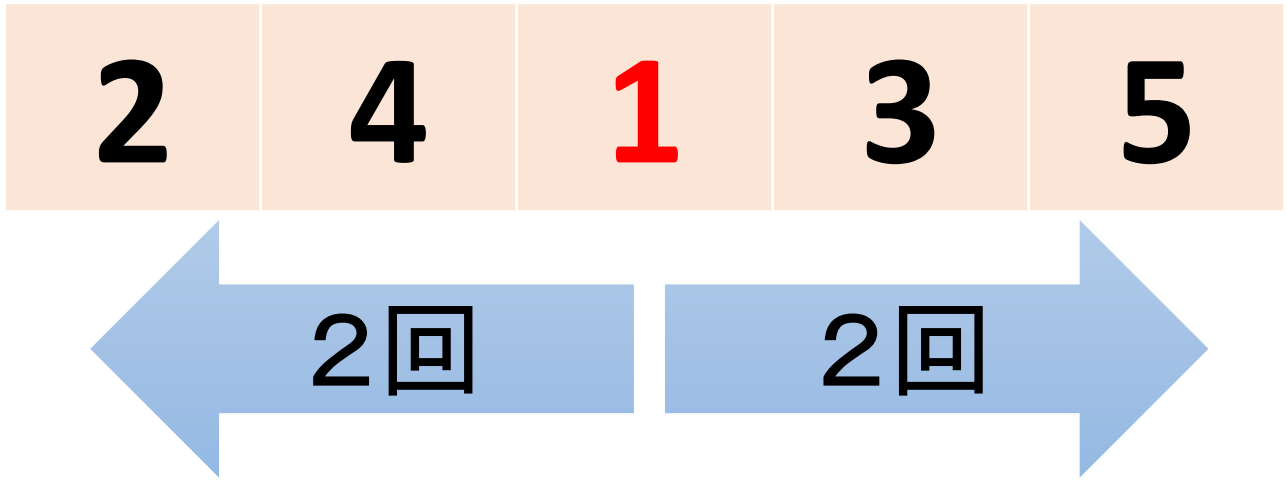
解法

各積み木を移動させる回数
＝それより大きい積み木が左側と右側に何個ずつ
　あるか調べて、その小さい方

Binary Indexed Treeなどを用いると$O(logN)$で
計算できる

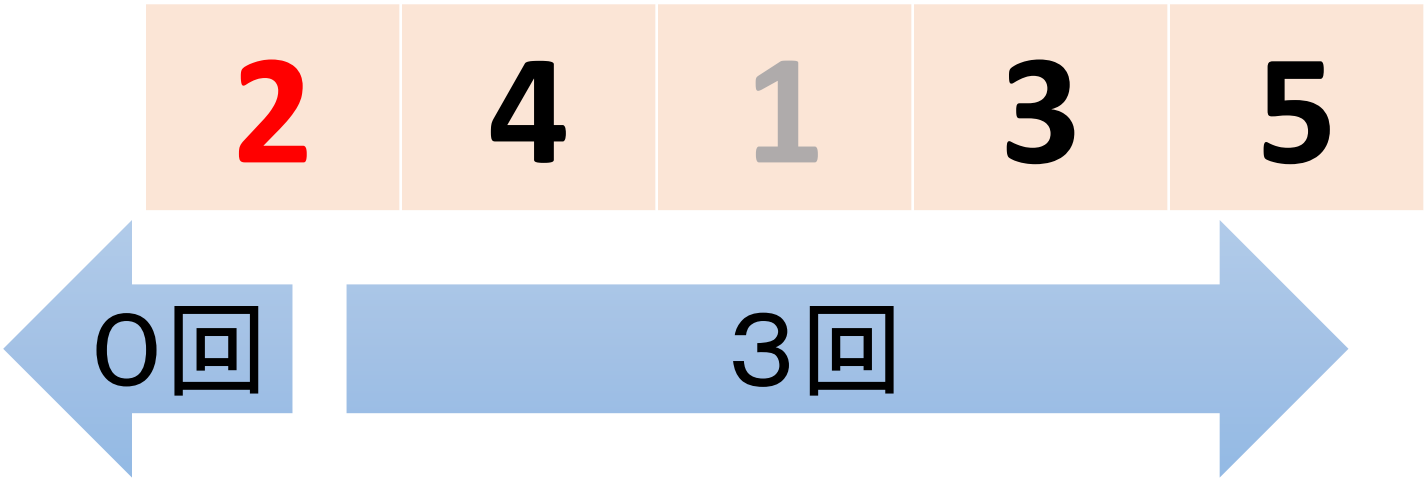
解法

１はどちらに動かしても２回交換が必要



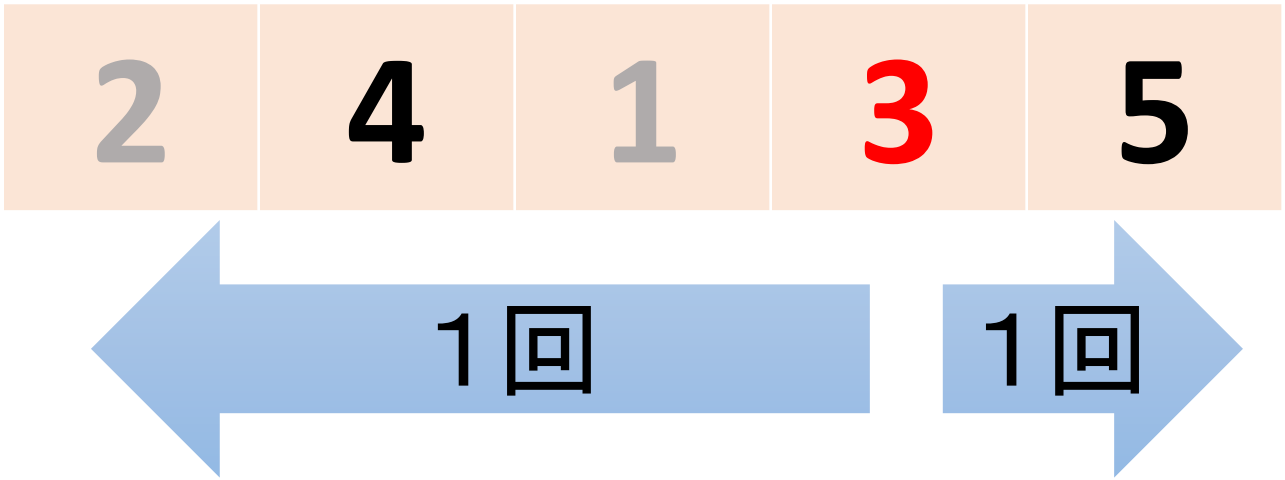
解法

２はすでに左側にあるので動かさなくて良い



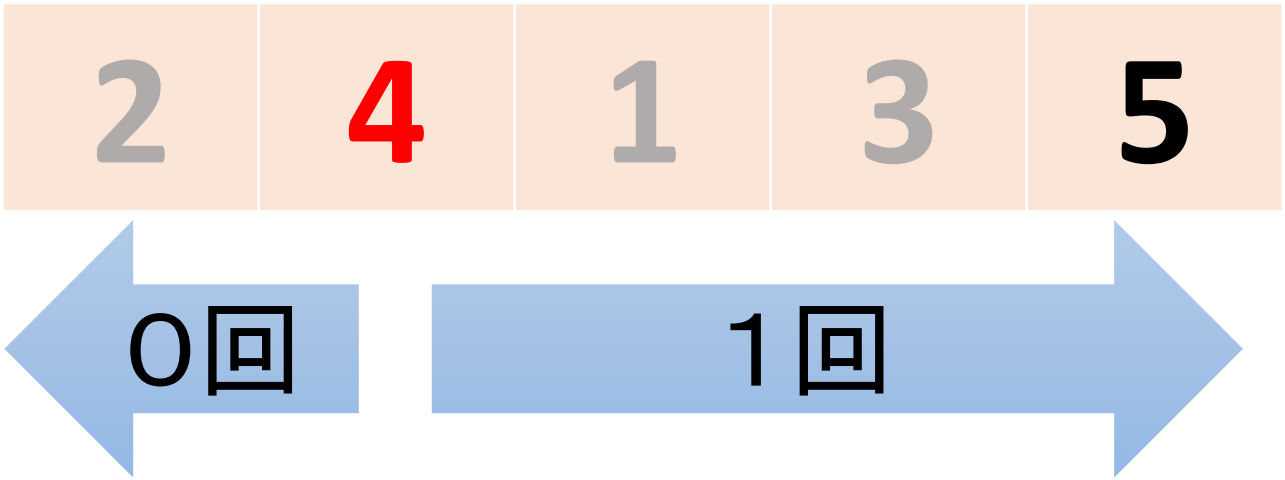
解法

３はどちらに動かしても交換回数は１回



解法

4 はそのままで良い



解法

- 最初に全要素１としてBinary Indexed Treeを構築し、小さい方から順に見ていく
- 左側と右側の和を計算する
- 各積み木を調べたら、Binary Indexed Treeの対応する要素を０に変更する
- 計算量は全体で$O(NlogN)$

問題D

買い物上手

問題概要

アイテムの組み合わせると経験値が得られる
「得られた経験値÷使ったお金」を最大化

経験値をもらえる組み合わせの数≦１００
アイテムの種類数≦１００

入力例１

アイテム1・2・3を買うと、経験値7、使ったお金7
となり答えは1

経験値：４

アイテム１
アイテム２

経験値：３

アイテム１
アイテム３

経験値：２

アイテム２
アイテム３
アイテム４

アイテム１：４
アイテム２：２
アイテム３：１
アイテム４：１０

解法

「平均」を最大化/最小化
　→答えについて二分法

類題）ARC #026 D -道を直すお仕事

得られた経験値÷使ったお金＝xと思って
得られた経験値－使ったお金×xの正負で
二分法をする

解法

経験値をもらうかどうかを$\exp(i)$とし、
アイテムを買うかどうかを$\mathrm{item}(i)$とすると

$$\sum S(i)\exp(i) - x\sum T(i)item(i)$$

を最大化すれば良い。ただし、

$$\exp(i) = 0,1、\ item(i) = 0,1$$

と、i番目の経験値にj番目のアイテムが必要なら

$$exp(i) = 1 \rightarrow item(j) = 1$$

が成り立たなければならない。

解法

$S(i) > 0$、$T(i) > 0$なので条件は以下のように書き直すことができる

$$\exp(i) \leqq 1$$
$$item(i) \geqq 0$$
$$exp(i) \leqq item(j)$$

これは線形計画法になっているのでsimplex法などで解くことができる

ただし、このままではTLEを回避するのが困難

解法

双対問題を考えると以下のように書き換えられる

条件：

$$\sum_j x_{ij} \leqq S_i \qquad \sum_i x_{ij} \leqq xT_i$$

$x_{ij} \geqq 0$ (経験値iにアイテムjが必要な場合)

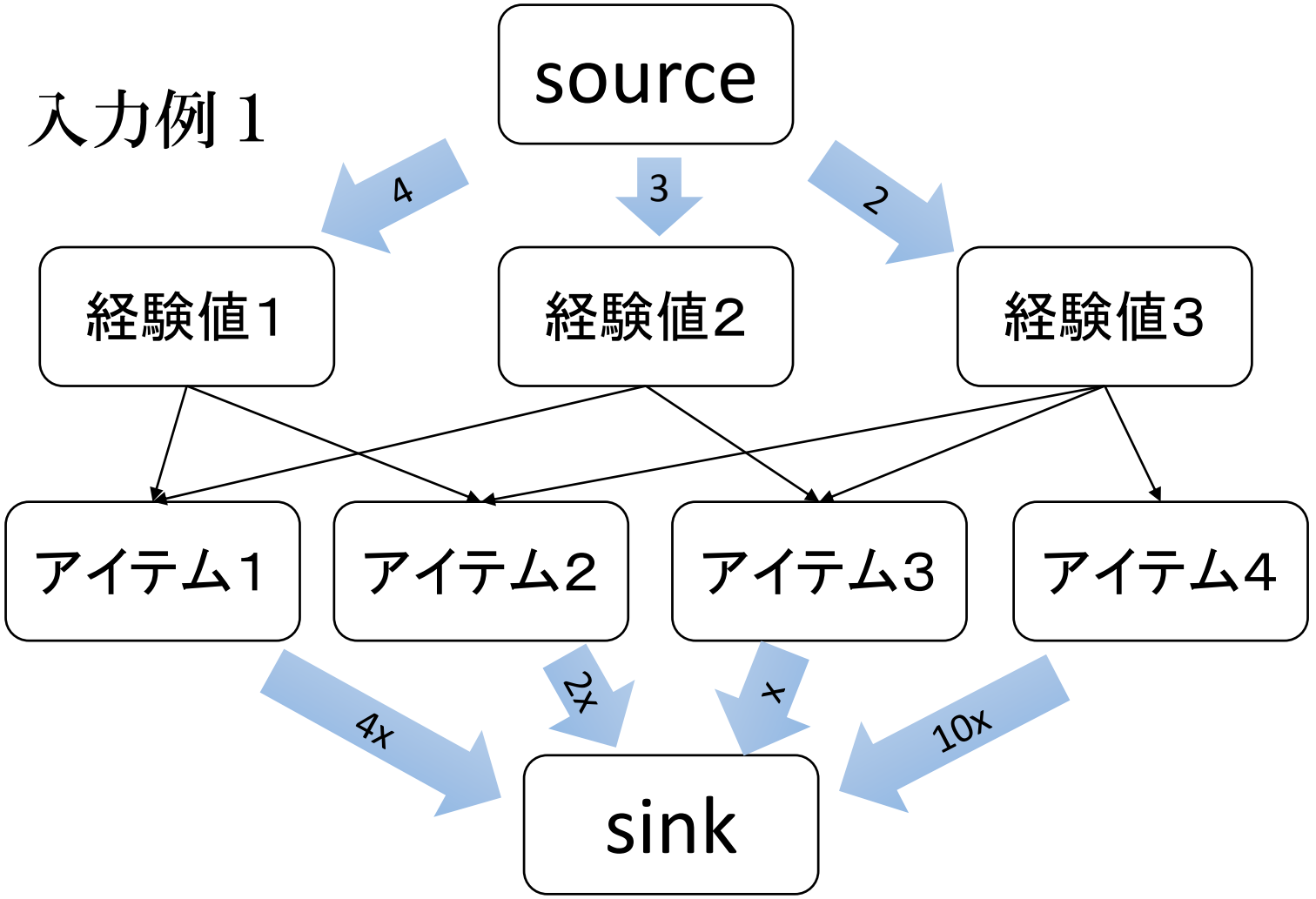
最大化：

$$\sum_{i,j} x_{ij} - \sum_i S_i$$

これは二部グラフの最大流になっている

解法

入力例１


source
4
3
2
経験値１
経験値２
経験値３
アイテム１
アイテム２
アイテム３
アイテム４
4x
2x
x
10x
sink

解法

このグラフで、sourceから出ている辺を
　流し切れるならxが小さい
　流しきれないならxが大きい

グラフをよく眺めると値段あたりの経験値を
xとして求める形になっている

参考文献：
二部グラフにおける最大重み閉包問題（古林 他）

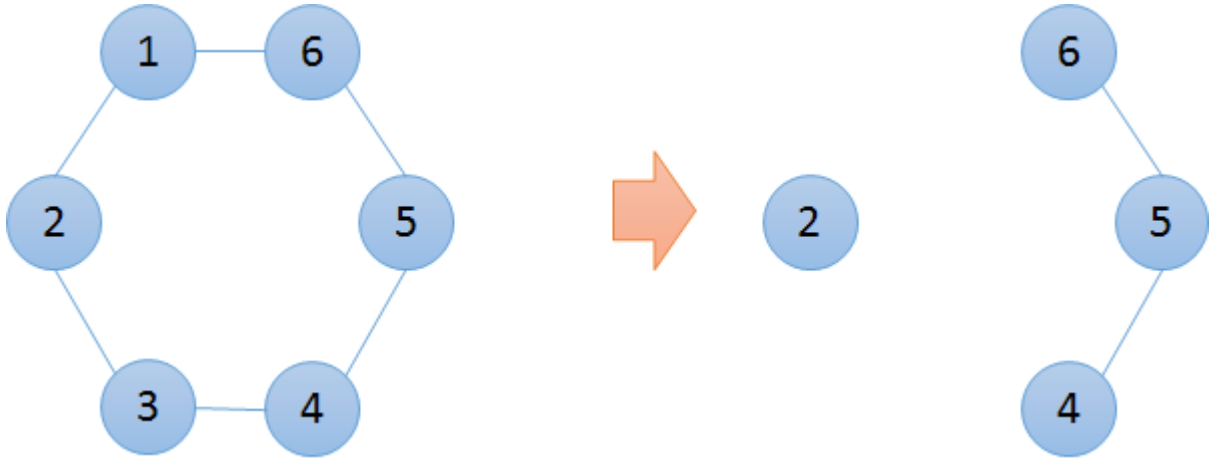
ARC030 解説

解説スライド担当: 三上 和馬
(@kyuridenamida)

問題A – 閉路グラフ

問題概要

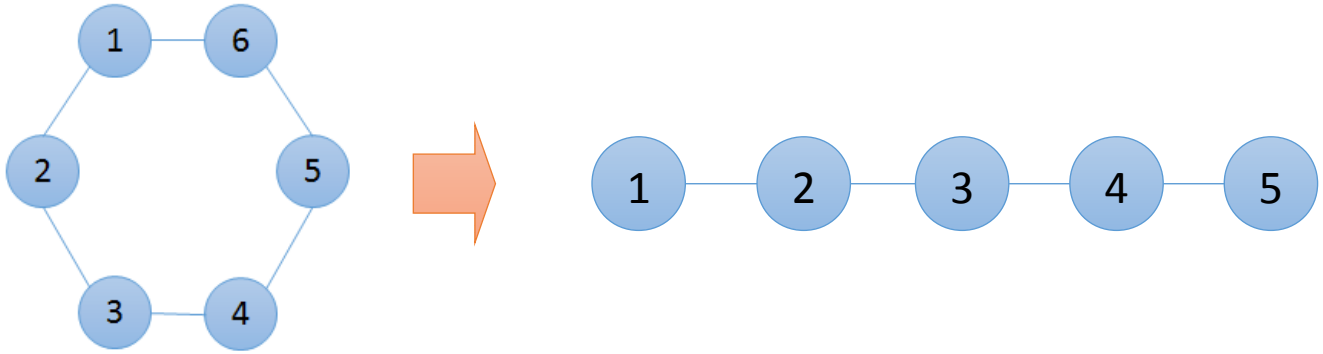
- n頂点の閉路グラフがある
- いくつかの頂点を取り除いて連結成分の数をk個にすることは可能か？　YES or NO で出力
- $3 \leqq n \leqq 100{,}000$
- $3 \leqq k \leqq 100{,}000$



n = 6 k = 2 の例　答えは「YES」

解法＆考察

- 取り出せる最大の連結成分の数を求めたい
- とりあえず頂点を1つ取り除いてみると直線グラフになる



- この状態でサイズ1の連結成分を作りまくるのが最適そう
  したがって端から偶数番目の頂点を取り除いていけばよい
- 奇数番目の頂点の数=((n-1)/2を切り上げたもの)が作れる連結成分の最大数であり，kがこれ以下であればYES，そうでなければNOを出力

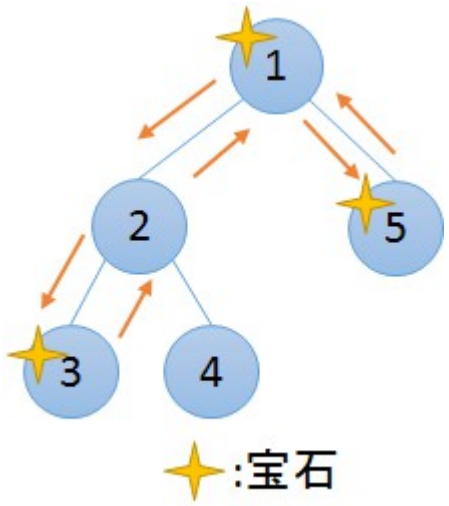
備考

- 閉路グラフはよく列を扱う問題の派生として出題されます.
- 「頂点を1つ〇〇したら列の問題に帰着できる」等の発想は，方針を考える上で手助けになるかと思われます.

問題B – ツリーグラフ

問題概要

- 頂点数nの木があり，各頂点には高々1つの宝石がある
- 辺の重みは全て1
- 頂点xから出発し，全ての宝石を回収して再びxに戻ってくるときの最短経路長を求める
- $1 \leqq n \leqq 100$
- $1 \leqq x \leqq n$



x=1のケース

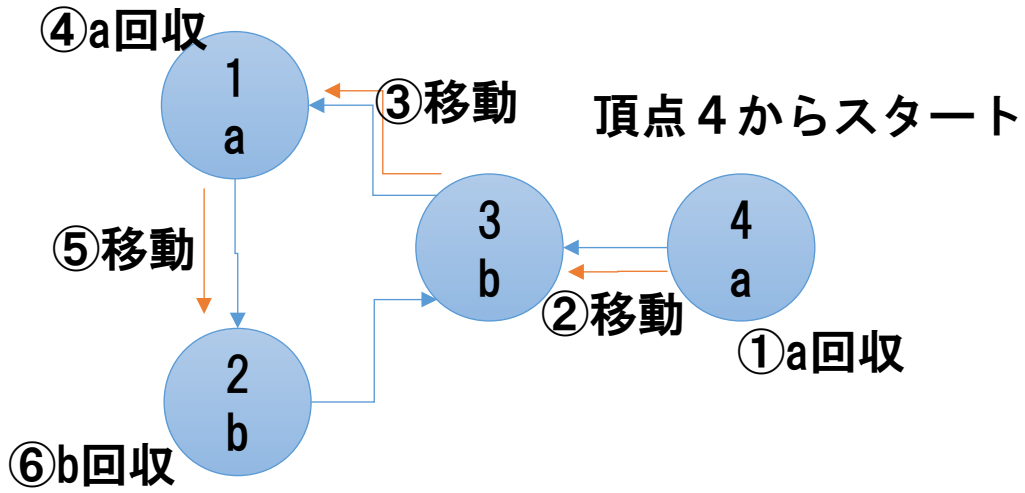
考察＆解法

- 与えられたグラフから出発地点を根とした根付き木を構築する
- ある辺をたどるべきかどうかだが，「辺を辿った先の部分木の頂点が1つでも宝石を含んでいるときに限り通り，それ以外のとき通らなくて良い」ということが分かる．部分木が宝石を含んでいるかを判定する関数を作っておく
- ある辺を使うときは行きと帰りで必ず2回通る
- なので，判定関数で，行く必要のない頂点に遷移しないような再帰関数を書き，辺をたどる毎に答えに2を加算するプログラムを書けば良い
- 任意のケースで同じ辺を3回以上辿る必要はなく，上記の戦略が最適．

問題C – 有向グラフ

問題概要

- 頂点数n，辺数mの有向グラフがあり，各頂点にはアルファベットが書かれている．
- 任意の頂点から出発し，k個のアルファベット回収し，順に並べたときに辞書順最小となるものを求める．一度回収したアルファベットは無くなる．回収方法が存在しないなら-1を出力
- $1 \leqq n \leqq 300$
- $1 \leqq m \leqq 300$
- $1 \leqq k \leqq n$



k=3のケース aabを出力

強連結成分分解

- 有向グラフにおいて，相互に行き来できる頂点同士の集合(強連結成分)を検出し，グループ分けするアルゴリズム

- 適当な未訪問の頂点を始点とし，深さ優先探索を行い，帰りがけ順で頂点を列挙することを，未訪問頂点がなくなるまで繰り返す(トポロジカルソート).
- 次に，元のグラフの逆辺グラフを考え，上記で得られた頂点の順番に，未訪問の頂点を始点とし再度深さ優先探索を繰り返し行う．このとき，この一度の深さ優先探索でたどり着ける頂点集合が1つの強連結成分となっている

考察＆解法

- どこの頂点のアルファベットを回収したかを覚えて探索は無理.
- なので，与えられたグラフに対して強連結成分分解を行う
- 同じ強連結成分に属する頂点の文字は，自由な順番で取ることができる→ある強連結成分にある文字をhoge個使うことを考えたとき，強連結成分に含まれる文字群をソートした文字列の先頭hoge個を使えば良いということが分かる
- こういった遷移を考えればDAGになり，回収した頂点を覚えなくて良いDPに帰着できる
- DPは「どの強連結成分か」「今何文字か」をキーに辞書順最小の文字列を格納するものを行う
- 強連結成分分解の計算量はO(N+M)，文字列比較に最悪O(K)かかるので，動的計画法の計算量はO((NK+M)K)

問題D – グラフではない

問題概要

- 長さNの数列Xが与えられ，その後Q個のクエリが飛んでくる．種類は
  - １ａｂｖ ― Xの区間 [a,b] に一様に値 v を加える.
  - ２ａｂｃｄ ― Xの区間 [a,b] を，クエリが呼ばれた時点での区間 [c,d] の値に書き換える(b−a=d−c).
  - ３ａｂ ― Xの区間 [a,b] の総和を出力.
- 1 ≦ N ≦ 200,000
- 1 ≦ Q ≦ 200,000

方針

今回のクエリはタイプ2が曲者なので，そのため

区間に対する操作は平衡二分木等を使うと万能なのでそれを使う．遅延評価が使えると良い．

segtreeは使えるか…？ → 今回はタイプ2があり，使いづらい．

- タイプ2のクエリ([c,d]→[a,b])がやっかい

このクエリは，区間[c,d]のコピーを区間[a,b]に上書きするクエリ

平衡二分木のある区間のコピーを作成するには，永続データ構造が必要そう．

遅延評価機能付き永続型平衡二分木をつくろう．

永続化のための平衡二分木選び

- ✕ treap (同じ乱数値を持つノードがたくさん発生するため木が偏る)
- ○ 赤黒木 (いける...けど実装重い)
- ◎ RandomizedBinarySearchTree (マージ/スプリットするときに乱数を用いるのでtreapみたいなことは起きない)

↑計算量解析は難しいらしいが実際偏らず各操作O(log n). 実装も比較的軽くおすすめ！今回はこれを実装する.

まずは普通の平衡二分木

- 今回はキーに順序関係が全くないので，「平衡二分探索木」ではなく「平衡二分木」と呼ぶが，対数計算量実現のためにやってる操作は一緒.
- 区間の問題に対処するためには，
  - ある位置の頂点を追加する/削除するというinsert/eraseベースの実装
- よりも
  - ある位置で木を2つに分割する/2つの木を順番を保って併合するsplit/mergeベースの実装

のほうが都合が良いのでそれを実装していく

各ノードが持つべき情報

- 左の子へのポインタ/右の子へのポインタ
- そのノードの持つ値
- そのノードを根とした部分木のすべてのノードの値の総和
- そのノードを根とした部分木のすべてのノードの数
- そのノードが持つ評価遅延中の値

が持つべき情報

```
struct T{
	T *l,*r;
	long long v;
	long long sum;
	long long lazy;
	int c;
};
```

作っておくと便利な関数

- 子の情報に基づきノードの情報を更新し，自身のポインタを返す関数

```
int count(T *c){
        if( !c ) return 0;
        else return c->c;
}
long long sum(T *c){
        if( !c ) return 0;
        else return c->sum;
}
```

```
T *update(T *c){
    if( c == NULL ) return c;
    if( c->lazy ){ // 評価遅延しているものを評価し子に伝搬する
        c->v += c->lazy;
        if(c->l){
            c->l->lazy += c->lazy;
            c->l->sum += count(c->l) * c->lazy;
        }
        if(c->r){
            c->r->lazy += c->lazy;
            c->r->sum += count(c->r) * c->lazy;
        }
        c->lazy = 0;
    }
    c->c = count(c->l) + count(c->r) + 1;
    c->sum = sum(c->l) + sum(c->r) + c->v;
    return c;
}
```

マージ関数の実装

- T *merge(T *a,T *b) := aの右の子にbを結合するか，bの左の子にaを結合するかを乱数で決める関数
「aの部分木サイズ>bの部分木のサイズ」のときはだいたいの確率でaの右の子にbをくっつける」などを繰り返すイメージ
- つなげたあとは，つなげたときに取り除いた子を考慮して再帰

```
T *merge(T *a,T *b){
        if(a == NULL) return b;
            if(b == NULL ) return a;
            if( rand() % ( count(a) + count(b) ) < count(a) ){
                        a = update(a); //  たどったノードはとりあえず更新しておく
                        a->r = merge(a->r,b);
                        return update(a);
            }else{
                        b = update(b); //  たどったノードはとりあえず更新しておく
                        b->l = merge(a,b->l);
                        return update(b);
            }
}
```

スプリット関数の実装

- pair<T*,T*> split(T *c,int k) := cの部分木を[0,k) [k,n)に分割したもののそれぞれの根をpairで返す

```
pair<T*,T*> split(T *c,int k){
        if(!c) return make_pair(c,c);
        c = update(c); //たどったノードはとりあえず更新しておく
        if(k <= count(c->l)){
                pair<T*,T*> s = split(c->l,k);
                c->l = s.second;
                return make_pair(s.first,update(c));
        }else{
                pair<T*,T*> s = split(c->r,k - count(c->l) - 1);
                c->r = s.first;
                return make_pair(update(c),s.second);
        }
}
```

これらの関数を永続化に対応させるには

- 基本的に全てのノードがconstなものであると考え，子を書き換えたり，遅延情報を伝搬したり，ありとあらゆる変更する際はコピーノードを作成し，それを書き換えるという発想に基づく．書き換わっていないノードは既存のものを使うようにする．
- ここで自身を返す実装が初めて生きてくる
- すると，一度の操作で増えるノードの数はたかだかO(log N)個

これらの関数をこの問題に適用していく

- 基本的に，区間に対する操作は，
  - 区間だけを切り取った木をsplitで作り，根を操作し，そのあとそれをmergeする

  で対処する.
- たとえば区間[a,b)に対するタイプ1(一様加算)クエリは今の列の[a, b)を取り出して根の遅延評価情報にvを加算し，くっつける
- タイプ2(コピー)クエリに関しては，[0,a) [b,n) そして[c,d) をコピーした木を作り，[0,a) [c,d) [b,n)の順番でマージする(実際の問題は閉区間で与えられるので注意)
- タイプ3のクエリは，同様に区間を切り出し根のsumを出力するだけ

メモリが溢れることに対する対処

- 実際は，一回クエリを処理するたびに，ノードが結構生成される
- 適当にやっているとMLEしかねないが，メモリーがやばくなったら一回列の情報を全てどこかに出力し，永続情報を破棄し再構築するという方法で簡単にごまかすことができる.

計算量

- 時間計算量
  - 一回の操作でO(log N) なので O((Q+N) log N)
  - ただし定数倍は重い
- 空間計算量
  - O((N+Q) log N)個のノードがクエリ処理の過程で生成される
  - 遅延評価するときに，たどったノードに隣接している子ノードの分もコピーするので，こちらも定数倍は重い．前スライドのような方法で対処．

ARC029 解説

解説スライド担当: 城下 慎也(@phidnight)

問題A – 高橋君とお肉

問題概要

- 2 つの肉焼き器を用いてお肉を焼く。
- N 個のお肉を肉焼き器を使って焼く。
- うまく割り当てて、すべてのお肉が焼けるまでの時間を最小化せよ。

- $1 \leqq N \leqq 4$

解法

- 全探索をすることで答えることができます。
- お肉をどちらの肉焼き器に割り当てるかという、全部で 2^N ある組み合わせを試します。
- 割り当てた後、それぞれの肉焼き器が肉を焼くのにかかる時間は、割り当てられた肉が要求する焼き時間の合計値と同じになります。
- 2 つある合計値のうち大きいほうがその割り当てにおける焼き時間となります。
- 計算量は $O(N * 2^N)$ となります。
- $N \leqq 4$ なので、満点をとることができます。

備考

- 指数全探索は競技プログラミングでは頻出です。
- 特に、部分点として出やすいので、ぜひとも習得してください！
- ちなみに、この問題は、すべてのお肉の焼き時間の合計値をMとおいたときに、$M/2$ 以上で最小の部分和を求める問題に帰着することができます。
- 上記の帰着により動的計画法で O(NM) で解くことができます。

問題B − 高橋君と禁断の書

問題概要

- ノートと箱 (ともに長方形で考える) が与えられる。
- それぞれの箱について、ノートが入るか否かを判定せよ。

- $1 \leqq N \leqq 5{,}000$
- $1 \leqq$ 辺の長さ $\leqq 100{,}000$

幾何の問題

- 幾何の問題は競技プログラミングで出てくることがあります。
- 実数計算とか特有の問題もあるので、慣れておくと良いと思います。

- 角度やベクトル計算 (外積、内積) などを知っておくと役立つことがあります。

実行できない解法

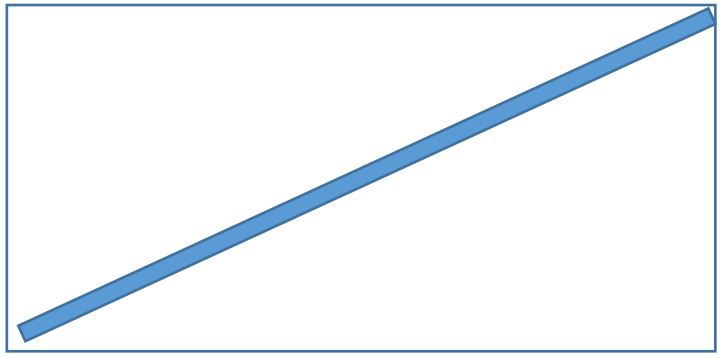
- すべての角度を試します。
- 「すべての角度」は、数え上げることができません。
- この方針だと、計算が終了しません。

実行できない解法

- すべての角度を試します。
- 「すべての角度」は、数え上げることができません。
- この方針だと、計算が終了しません。
- そのため、計算可能なように探索候補をしぼる必要があります。

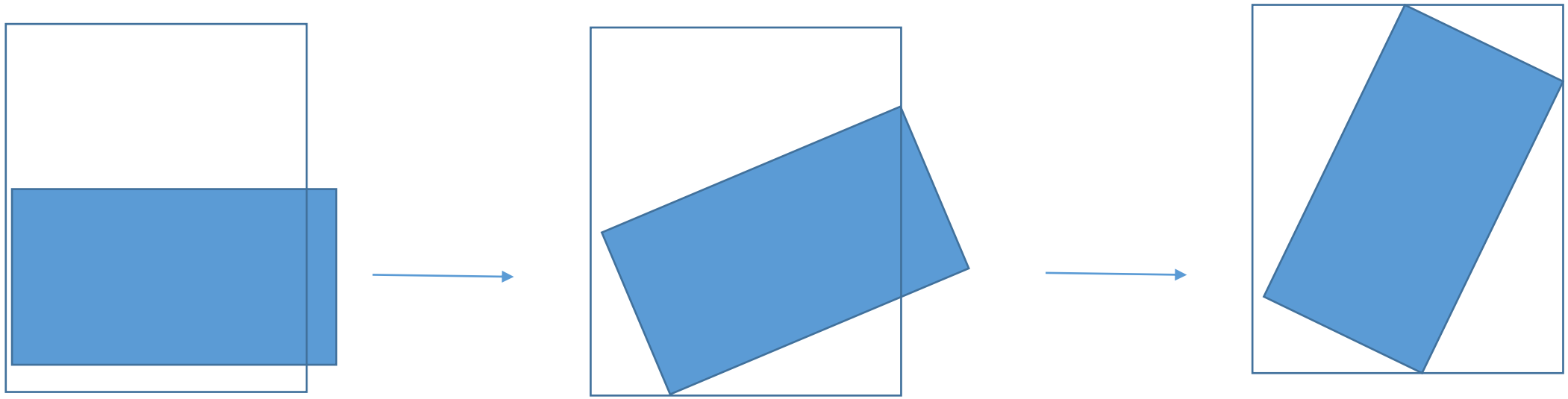
危険な解法

- ランダムに角度を決めて置く、という動作を一定回数繰り返し、何回やってもダメなら NO とし、うまく置けたら YES とします。
- 下図のような配置の場合、うまくいく確率がとても低く、NO と行ってしまう確率が高くなります。



考察

- 最初に、すべてを横向きに合わせます。
- 少しずつ回転させていくと、高さが増えていくので、高さが一致したときのみを試すと斜めのケースも対応できる。



解法

- 縦横に平行なケースを試し、斜めを、高さが一致した場合に限定することでそれぞれ O(1) で計算することができます。
- 高さが一致する瞬間は、最大値になる角度まで範囲で角度の二分探索で計算することができます。

問題C – 高橋君と国家

問題概要

- グラフが与えられます。
- 頂点に交易所、辺に舗装道を設置して、交易所へ舗装されていない道路を経由して移動できるようにします。
- 設置と舗装のコストが与えられるので、コストの合計を最小にしてください。

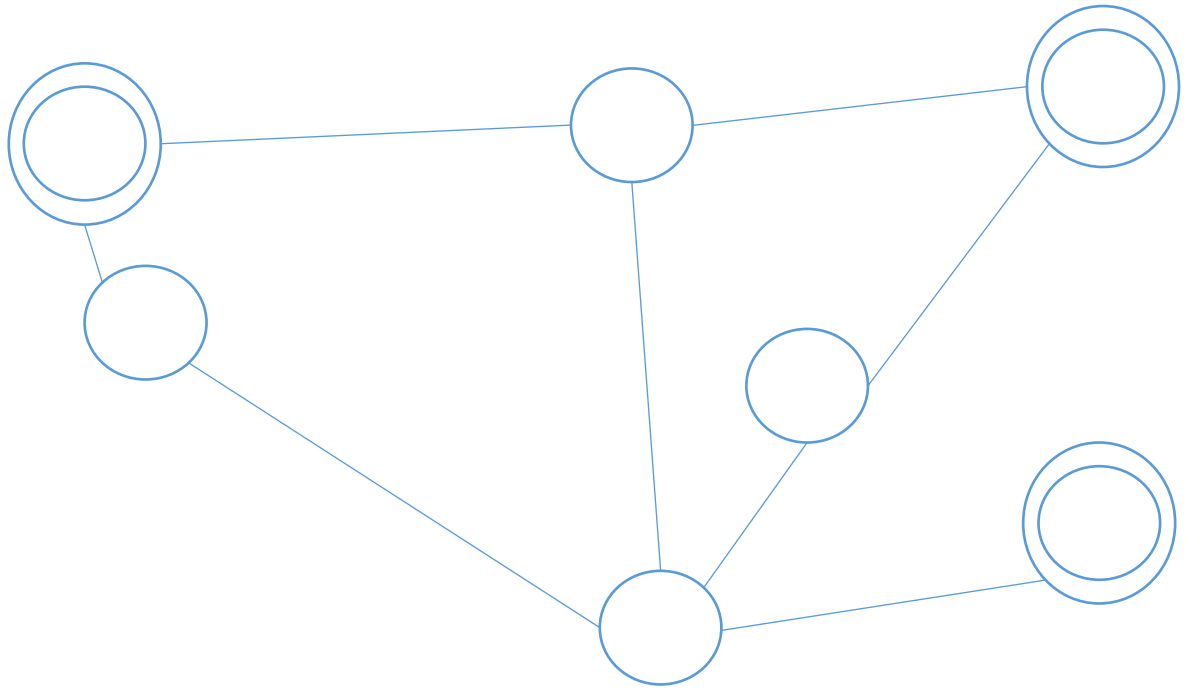
- $1 \leqq N(\text{頂点数}) \leqq 100{,}000$
- $1 \leqq M(\text{辺数}) \leqq 200{,}000$

部分点解法１(10点)

- すべての都市、道に対して、操作をするかしないかを決めて、その決め方でうまくいっているかを試します。
- うまく行っているかの判定は、各都市に対して、その都市の連結成分に交易所のある都市が入っているかを判定すれば良いです。
- 幅優先探索などを用います。
- 計算量はO($N * M * 2^{N+M}$) などになります。
- Union – find を用いると手早く連結判定はできます。

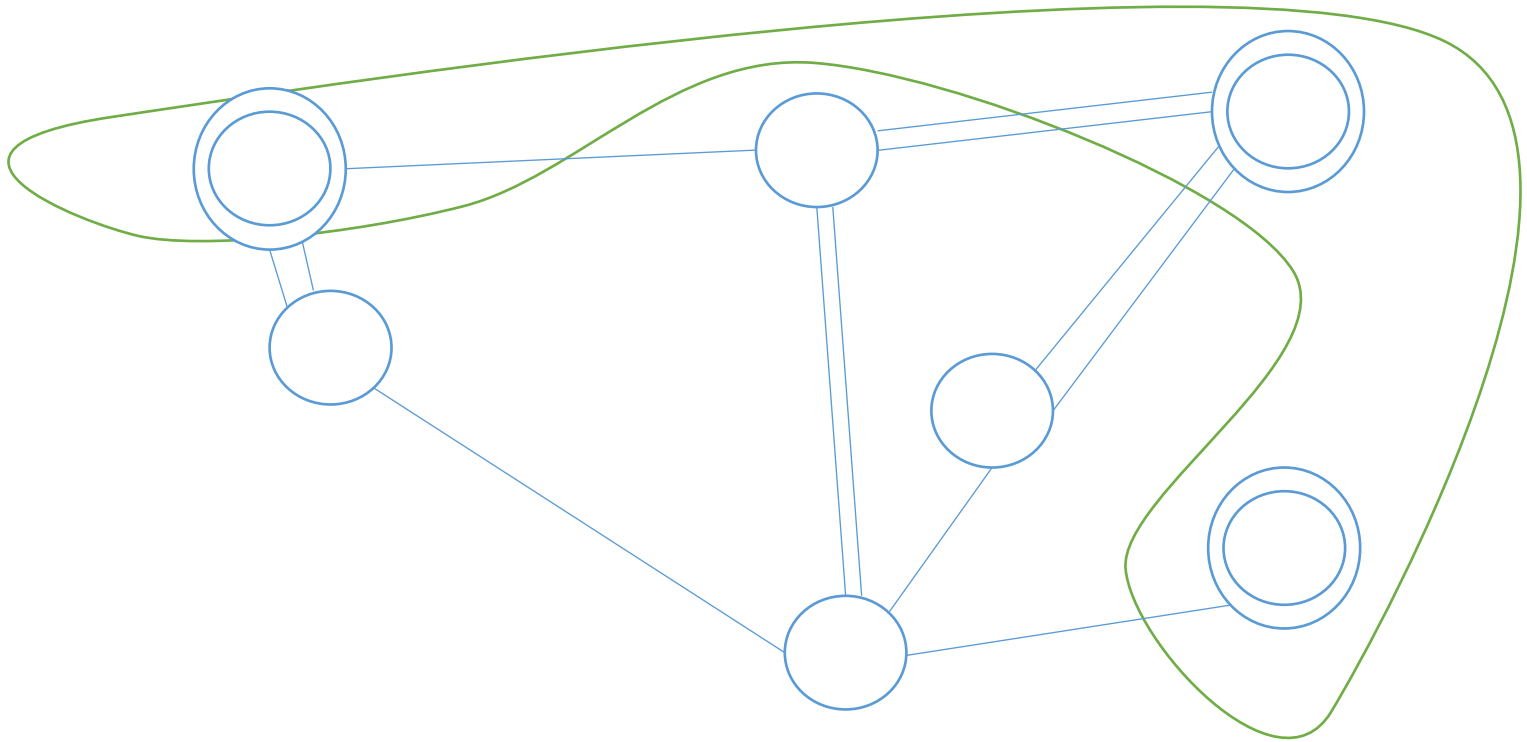
部分点解法 2 (10 + 20点)

- すべての都市に対して、操作をするかしないかを決めます。
- 辺を効率的に選択します。



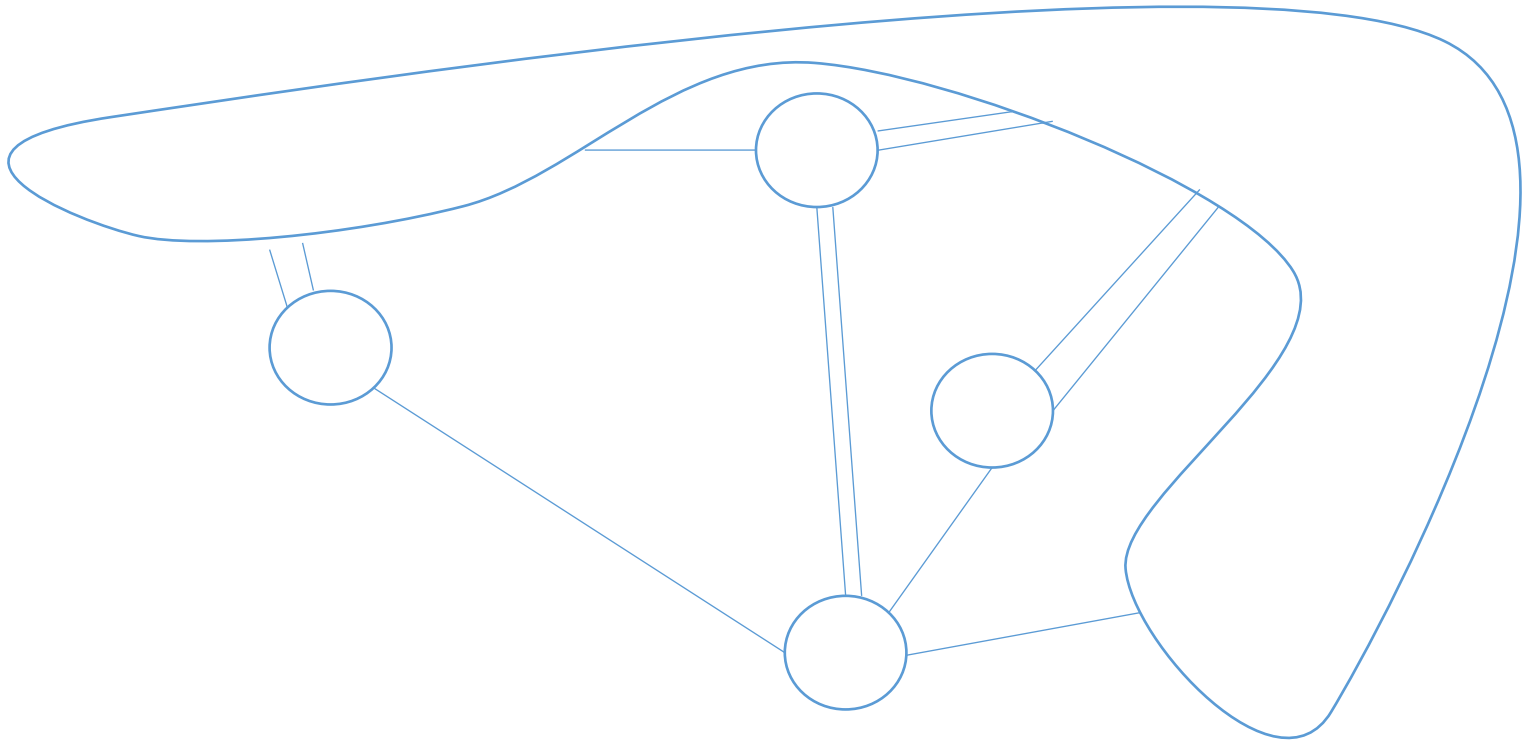
部分点解法 2 (10 + 20点)

- アイデアとして、交易所のある頂点を 1 つにまとめてみる。



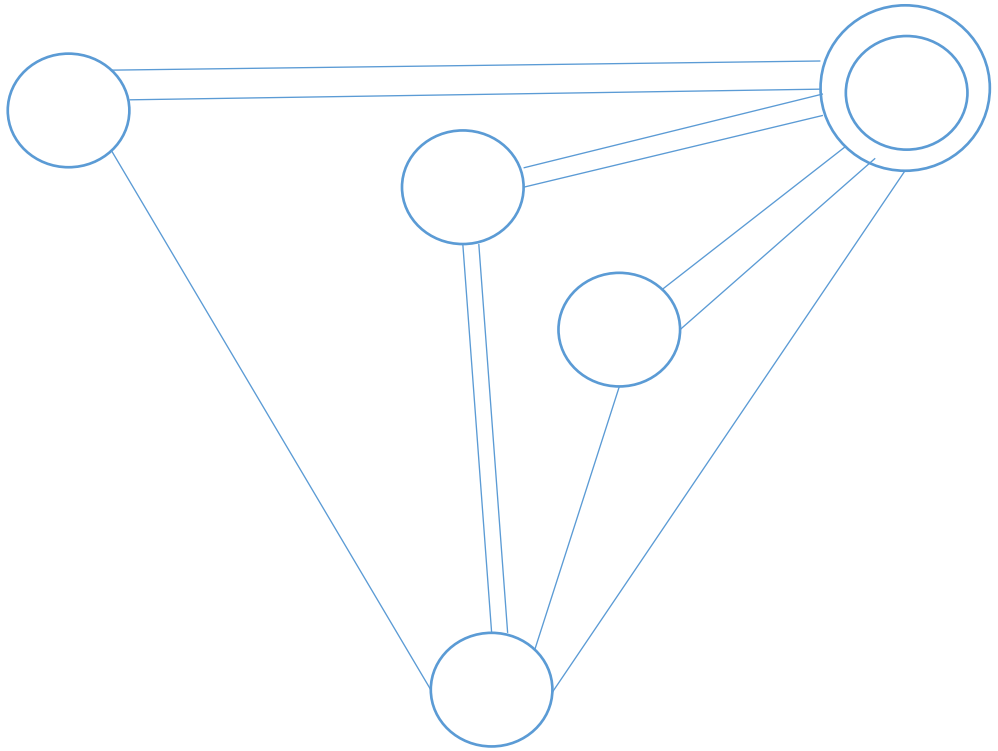
部分点解法 2 (10 + 20点)

- アイデアとして、交易所のある頂点を 1 つにまとめてみる。
- すると、全体が 1 つの連結成分になることがわかる。



部分点解法 2 (10 + 20点)

- アイデアとして、交易所のある頂点を 1 つにまとめてみる。
- すると、全体が 1 つの連結成分になることがわかる。

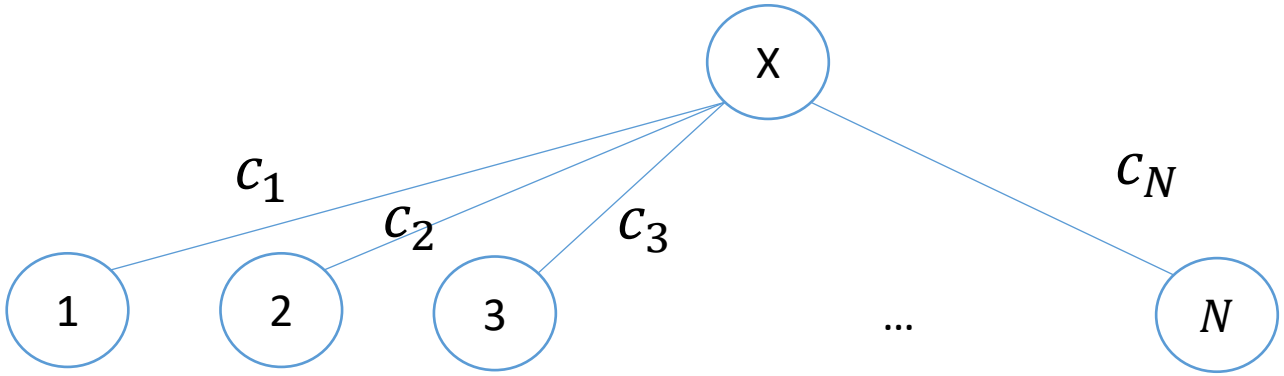


部分点解法 2 (10 + 20点)

- 「辺にコストがあって、辺の集合を選び全体を 1 つの連結成分にする選び方のうち、費用の合計値が最小となる選び方を求める問題」に帰着することができました。
- 上記の問題は、**「最小全域木問題」**として知られています。
- クラスカル法やプリム法で効率的に計算することができます。
- 交易所の設置方法は全部で 2^N 通りあり、すべてについて求めることで答えを求めることができます。
- 注意点として、交易所の設置方法によっては解が存在するとは限りません(元のグラフで、全体が連結でない場合など)。

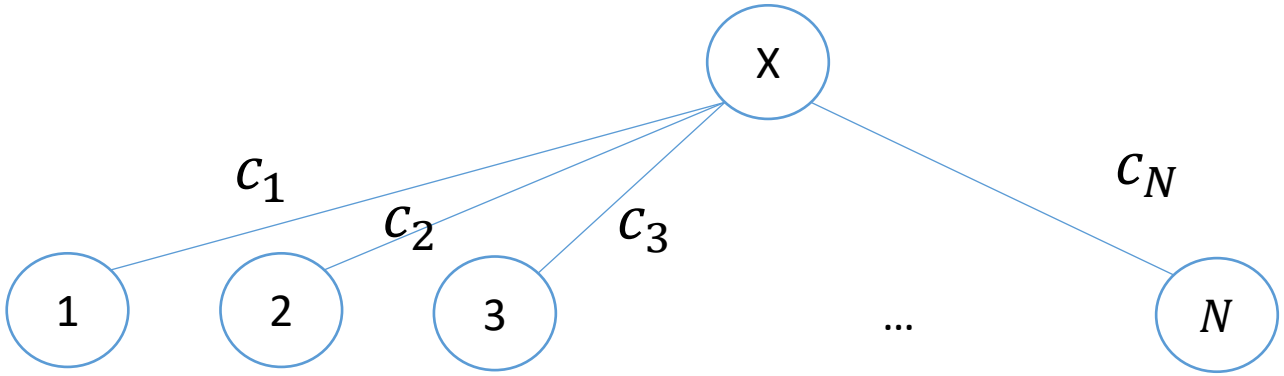
部分点解法 3 (10 + 20 + 30点)

- 最初に、交易所の頂点集合を選ぶ代わりに、仮想的な都市 X を用意します。
- 都市 X と都市 i $(1 \leqq i \leqq N)$ の間には、コストc_i(交易所の設置コストと一緒) の辺を張ります。
- 都市 1 から都市 N に関しては、交易所を置く代わりに、都市 X との辺を選ぶという操作だと考えます。



部分点解法 3 (10 + 20 + 30点)

- この処理をしたあと、良い状態であることと、都市 X を含めて全体で連結であることが同値となります。
- このグラフで最小全域木問題を解けば解が得られます。
- 有効な候補は毎回 O(N + M) 本で、最大 O(N) 回試すので、全体で O($N(N + M)$) となります。



満点解法

- 先ほどの部分点解法 3 で、プリム法やクラスカル法を用いて高速化すれば満点となります。

問題D – 高橋君と木のおもちゃ

問題概要

- 有向木(のおもちゃ)が与えられます。
- M個ある各整数を、置くか置かないかをします。
- 木の頂点に置いたら、頂点は親に古い数字を送ります。
- 最終的に残る整数の合計を最大化してください。

- $1 \leqq N(\text{頂点数}) \leqq 5{,}000$
- $1 \leqq M(\text{候補数}) \leqq 5{,}000$

部分点解法 1

- すべての置き方(置かないを含め)を試します。
- $(N+1)^M$ 通りを試すと答えが得られます。

考察

- 元からある数字について考えます。
- ある数字について、その数字が追い出されるなら、その数字を持っていた頂点の先祖となる頂点にあった数字もすべて追い出されています。
- すると、追い出される数字を持っていた頂点の集合は、頂点 1 を含む部分木となっているはずです。
- 逆に部分木(頂点数をXとする)が決まると、部分木の深いところから順に X 回おくだけで部分木のすべての整数を追い出すことができます。

考察

- この場合、残せる数字はX個だけです。
- 先ほどの方法だと、おいた数字すべてを残すことができるので、奥候補の数字から、大きいのをX個取り出せばよいです。
- すると、頂点 1 を根とする部分木の中で頂点数がXであるものを$0 \leqq X \leqq N$ ですべて求めることで計算できます。

考察+ 満点解法

- この計算は、各頂点 x に対して、その頂点 x を根する部分木で i 個選ぶ方法を dp[x][i] とする動的計画法で計算できます。
- この計算は各頂点に対して、子の頂点数が s1,s2,s3, … ,sk である場合に、O(s1s2 + (s1+s2)s3 + … + (s1+s2+…+s(k-1))sk)となります。
- これは一見 N の 2 乗に見えますが、(s1+s2+…+sk)^2 より小さいことがわかるので、頂点 1 で考えるとN の 2 乗で済みます。

A問題

小石を取るゲーム

問題概要

- 袋にN個の小石が入っている。
- Antさん、Bugくん、Antさん...の順に交互に取っていく。
- AntさんはA個ずつ、BugさんはB個ずつ小石を取る。
- どちらのターンの後に袋が空になるかを求めよ。
- 制約
- $1 \leq N, A, B \leq 1000$

解法

- シミュレーションしましょう。
- カウンタの変数を用意し、そこからAとBを交互に引いていく。
- カウンタが0以下になったときにどちらのターンだったかを出力する。

B問題

特別賞

問題概要

- コンテストが行われ、N人の人が参加した。
- i位の人が全参加者のうち何番目に若いかが入力される。
- 1位～j位の人のうちK番目に若い人の順位を、K~Nのjに対して出力せよ。
- 制約
- $1 \leq N \leq 10^5$

40点解法 (N≤1000)

- それぞれの答えを毎回求める。
- 1位からj位までの人の若さをソートして、K番目の要素を見る。

満点解法

- jを小さい方から順番に増やしていき、答えを更新していく。
- ある時点でK+1番目以降になった人は、それ以降K番目になる事はない。
- １人追加する度に、
  - K+1番目以降だったらなにもしない。
  - K番目以内だったら候補に加え、候補の中から一番老けている人を候補から削除する。
- で、候補のうち最も老けている人の順位を出力する。

満点解法

- それを実現するためにはデータ構造が必要。必要な操作は、
  - 要素を追加する
  - 要素のうち最も大きい値を取り出す
- priority queue（優先順位つきqueue）
- 各操作をO(log 要素数)で行うことが出来ます。
- 言語によって標準ライブラリがあったりするので調べてみましょう。

C問題

高橋王国の分割統治

問題概要

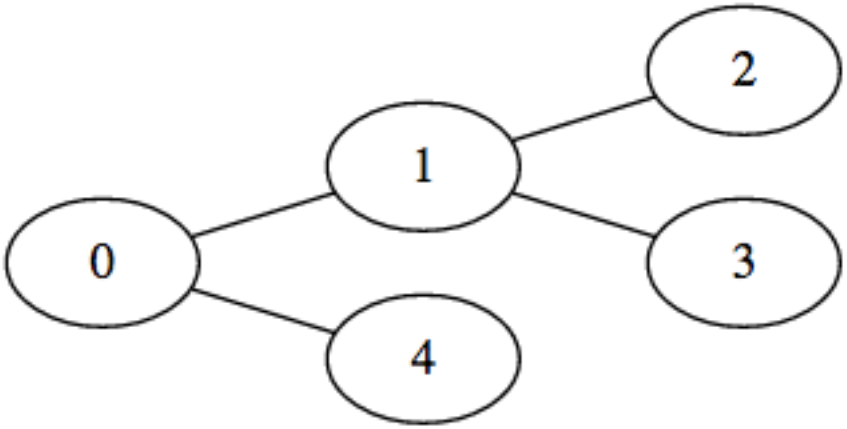
- 頂点数Nの木が与えられる。
- 頂点vが取り除かれた時にできる連結成分のうち、最も大きいものを、各vについて求めよ。
- 制約
- $1 \leq N \leq 10^5$

30点解法 (N≤1000)

- それぞれの答えを毎回求める。
- 木の問題になれていれば簡単。
- DFSなどをすればよい。

満点解法

- とりあえず根付き木として考える。
- DFSしながら木DPをすれば、子方向の部分木のサイズを求めることが出来る。
- しかし、親方向の部分木のサイズはどうしたものだろうか。
- →N-1-(子方向の部分木のサイズの和)
  で求められる！



おまけ

- 詳細については省きますが、
- 部分木のサイズの最大値を求める問題は、木の分割統治を行う時に使ったりします。

D問題

注文の多い高橋商店

問題概要

- N種類の商品がそれぞれA[i]個ある。
- 「k種類目の商品をx個選ぶとき、合計M個の商品を選ぶ方法は何通りあるか」というクエリにQ回答えよ。
- 制約
- $1 \leq N \leq 2000$
- $1 \leq M \leq 2000$
- $1 \leq Q \leq 500000$

10点解法 (N,M,Q≤100)

- 各クエリそれぞれでDPをする。
- DP[i][j]=i種類目までででj個選ぶ方法の個数
- 漸化式は
- DP[i][j] = ΣDP[i-1][j-a[i]]~DP[i-1][j]
- 累積和を取っておけばO(1)で更新できる。

30点解法 (N,M≤100)

- 商品の順番を入れ替えても答えは変わらない
- 商品i以外だけを使ったときのDPテーブルというのをあらかじめ全てのiについて求めておく。
- 前処理：O(N^2M)
- 各クエリ：O(M)
- となり、30点を得ることが出来る。

80点解法(N,M≤2000,Q≤1000)

- 各iごとに最初からDPをやり直しているのが無駄っぽい
- 0~jまでを使ったときのDPテーブルと
- j~Nまでを使ったときのDPテーブル
- を持っておく。(要するに左側と右側のDPテーブル)
- 各クエリにつき、2つのDPテーブルを参照することで、O(M)で答えを求めることが出来る。

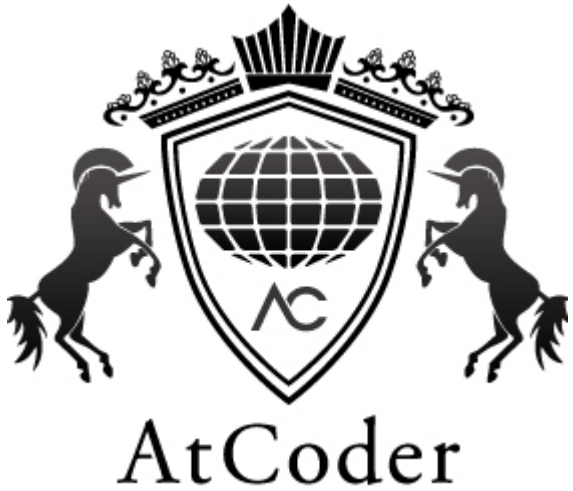
満点解法

- 各クエリにO(1)で答えなければならない。
- 商品i以外だけを使ったときのDPテーブルというのをあらかじめ全てのiについて求めておきたい。
- 考察
- 商品の順番は関係がない
- DPの漸化式はDP[i-1]における(部分)和の形式

戻すDP

- DP[i]からDP[i-1]を復元する。
- DP[i][j] = ΣDP[i-1][j-a[i]]~DP[i-1][j]
- →
- DP[i-1][j] = DP[i][j]-ΣDP[i-1][j-a[i]]~DP[i-1][j-1]
- O(M)で復元できる！
- 商品の順番は変えても答えは変わらないので、DP[N]から、「商品i以外だけを使ったときのDPテーブル」が各iについて復元できます。

AtCoder Regular Contest 027
解説



A 問題－門限

問題概要

- 現在の時間 h と分 m が与えられる。
- 後何分で 18 時 0 分かを求める。
- $0 \leqq h \leqq 17$, $0 \leqq m \leqq 59$

※標準的な入出力については、ABC004 の解説等を参照してください。

解法その 1

- 18 時まであと何時間何分かを求めて、その後分に変換する。
- 現在時刻が h 時 m 分なら、
    - $m = 0$ ⇒ あと $18 - h$ 時間
    - $m \neq 0$ ⇒ あと $17 - h$ 時間と $60 - m$ 分
- 計算結果を a 時間 b 分とすると、$\mathrm{a} \times 60 + b$ 分が答え。

- 最初に、現在時刻を 0 時ちょうどからの経過分数に変換して、1080 から引いても良い。

解法その 2

- 現在時刻から、1 分後、2 分後、…と 1 分ずつ加算して、18 時ちょうどになるまでに合計何回足したかを数える。

- どちらの方針にするかは、実装時間やバグの入りやすさなどを考慮して選択 (個人差あり)。

B 問題 −
大事な数なのでZ 回書きまLた。

問題概要

- 0 から 9 までと、大文字アルファベットのみで構成された長さ N の文字列が 2 個与えられる。
- 大文字アルファベットを 0 から 9 までの整数に変換する処理を行った結果、両者が表す整数が一致した。
- 一致した整数として全部で何通り考えられるかを求めよ。
- $1 \leqq N \leqq 18$

部分点解法

- 答えとなる数字を全探索する。
- この場合、答えとして考えられないものは、
    - 同じアルファベットに複数の数字が割り当てられている。
    - 先頭に 0 が来ている。

となる場合なので、その場合を判定して、残ったものの総数が答えとなる。

満点解法

- 部分点解法では、計算量が $O(10^N)$ となるので、Nが大きい場合には時間が掛かり過ぎる。
- 一致しているという条件をうまく言い換えたい。
- ここでは、「グラフ」の概念を導入すると理解しやすくなる。
- それぞれの文字を頂点としたグラフを考える。
- 同じ数字でなければならない文字同士を辺で結ぶ。
- 同じ数字でなければならない文字同士というのは、同じ桁 (同じ位) にある文字同士のことである。(例えば、2 つの文字列がABCDとEFEHだった場合は、AE,BF,CE,DH 間に辺を結ぶ。)

満点解法

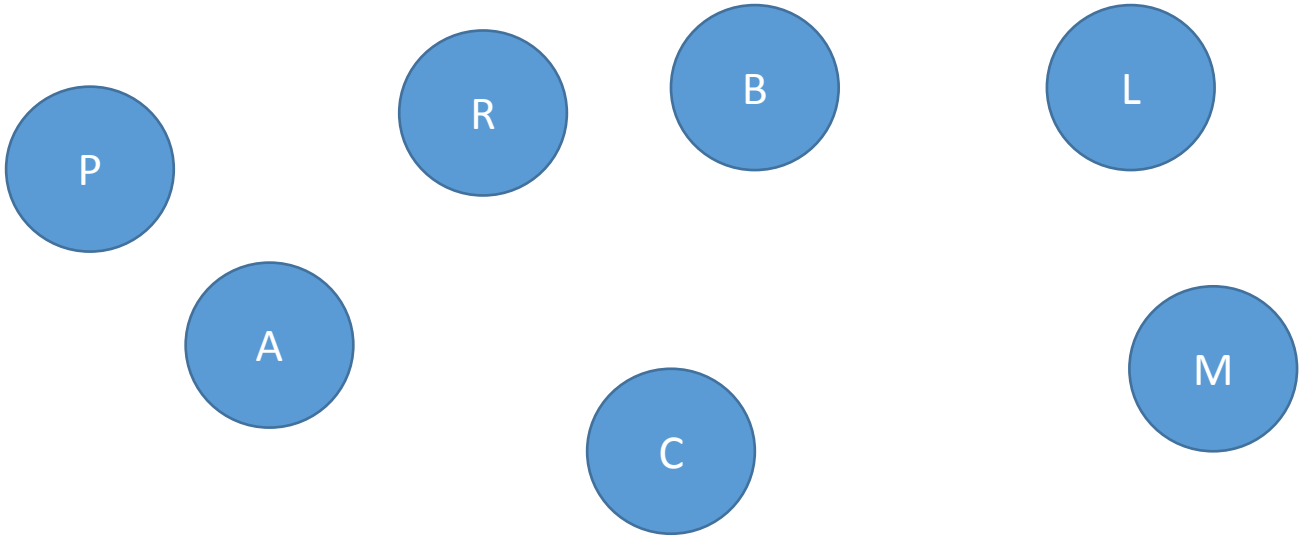
- 数字と文字が同じ桁にある場合は、その文字に、「その数字でなければならない」という条件を付加する。
- 最上位にある文字については、さらに「0以外」という条件も。
- なお、この条件は衝突しない(衝突したら答えがなくなるため)。

- その後、各連結成分(辺同士で行き来できる頂点たち)について、
    - 数字が割り当てられている⇒1 通り
    - 数字が割り当てられていない、かつ最上位⇒9 通り
    - 数字が割り当てられていない、かつ最上位でない⇒10 通り

をそれぞれ掛け合わせると答え。

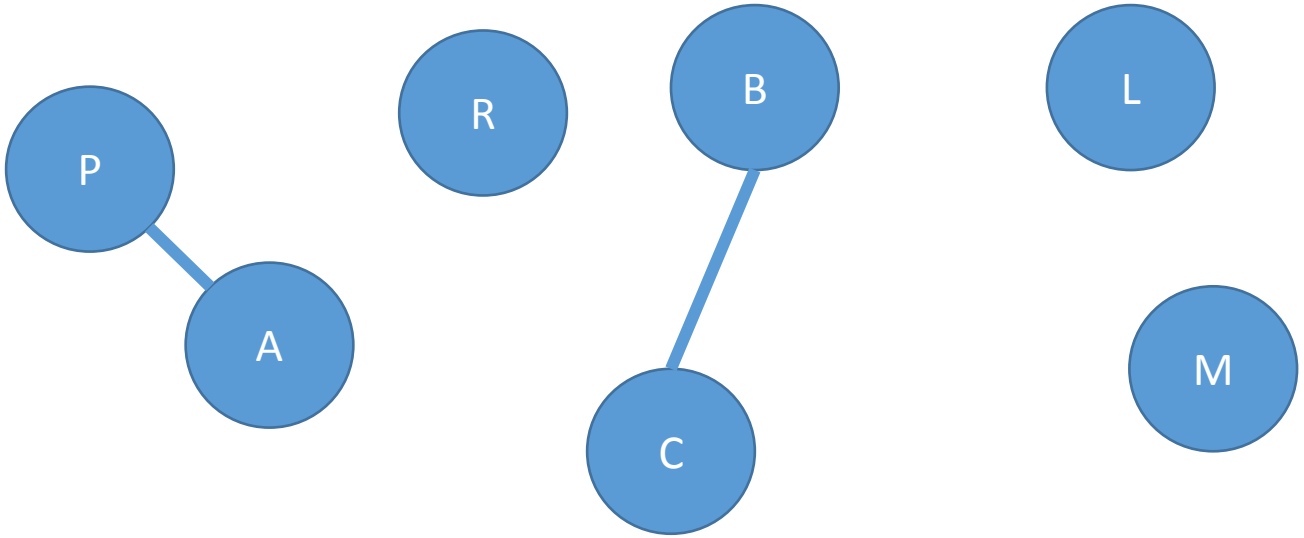
例(入力例 3 – PRBLMB と ARC027)

- 文字を頂点としたグラフを考える。



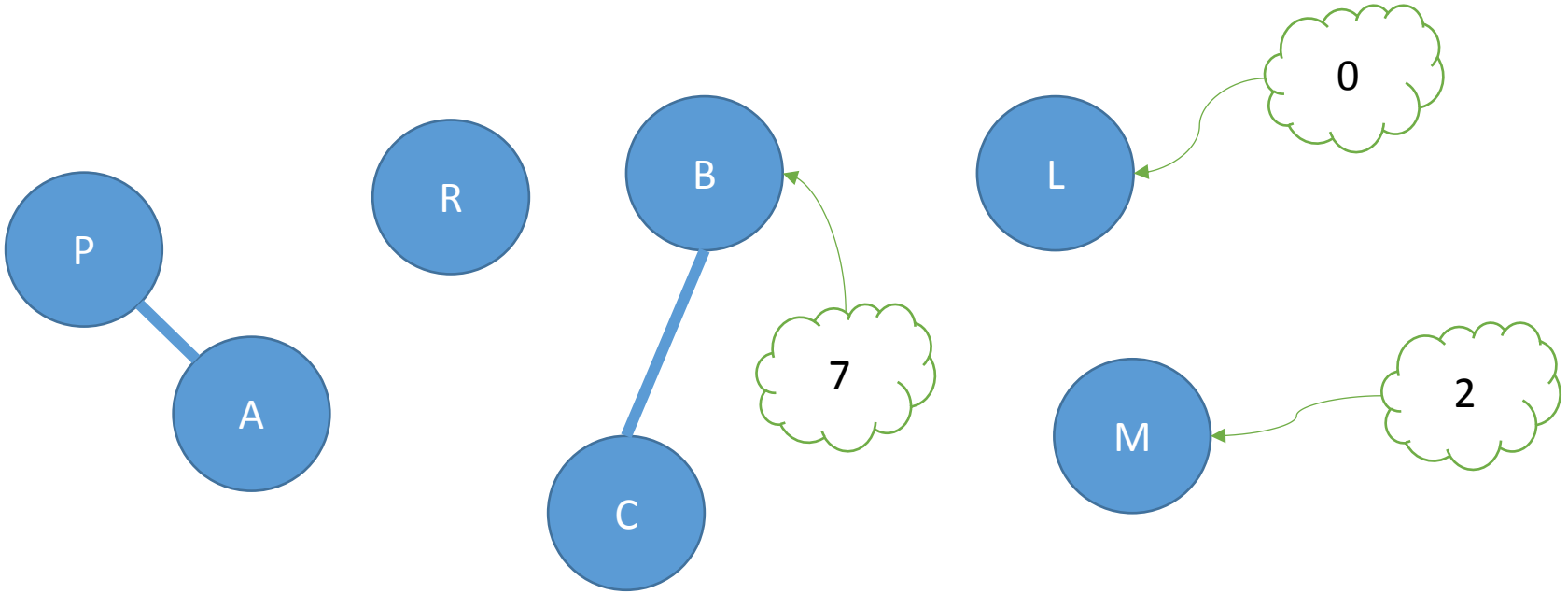
例(入力例 3 – PRBLMB と ARC027)

- 文字を頂点としたグラフを考える。
- 同じ数字になるもの同士を辺で結ぶ(PA,BC)。



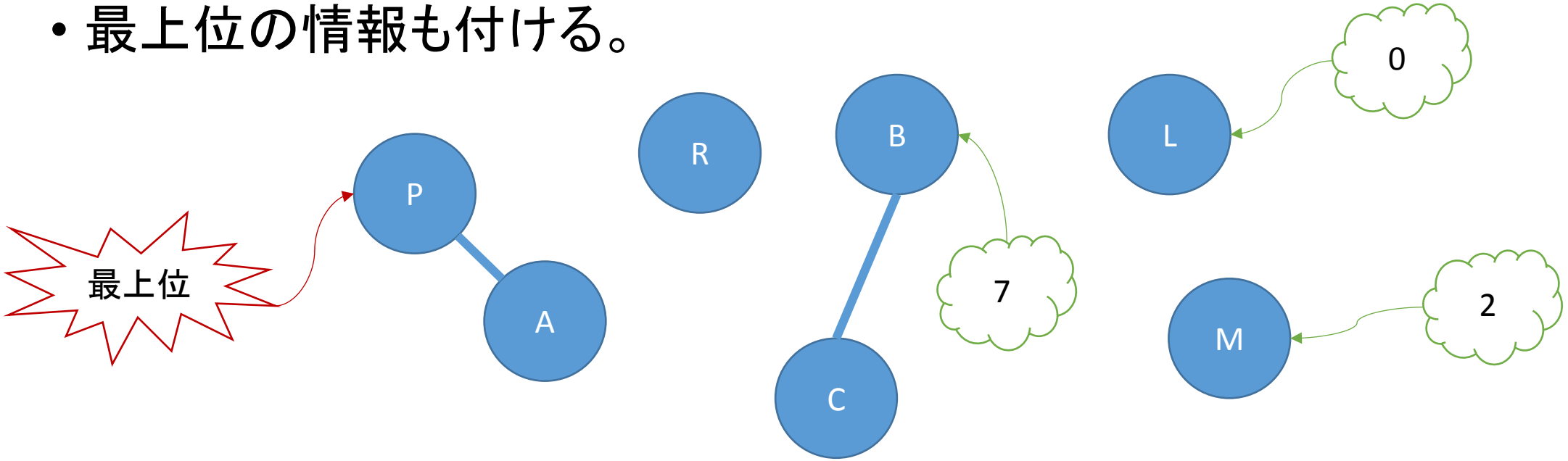
例(入力例 3 – PRBLMB と ARC027)

- 文字を頂点としたグラフを考える。
- 同じ数字になるもの同士を辺で結ぶ(PA,BC)。
- 文字と数字の割り当てを反映させる(L=0,M=2,B=7)。



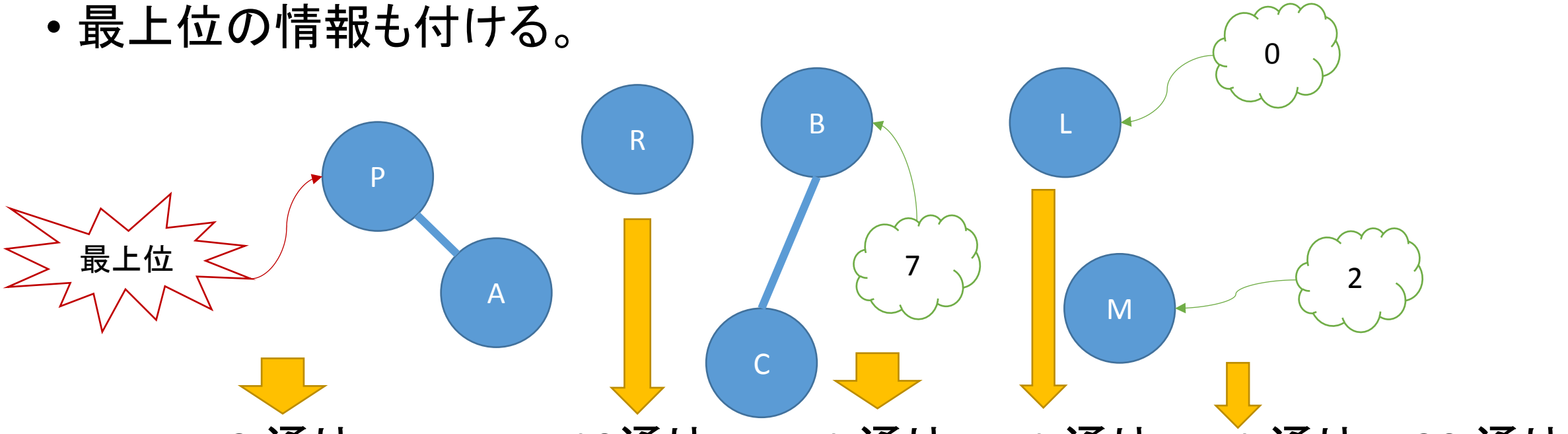
例(入力例 3 – PRBLMB と ARC027)

- 文字を頂点としたグラフを考える。
- 同じ数字になるもの同士を辺で結ぶ(PA,BC)。
- 文字と数字の割り当てを反映させる(L=0,M=2,B=7)。
- 最上位の情報も付ける。



例(入力例 3 – PRBLMB と ARC027)

- 文字を頂点としたグラフを考える。
- 同じ数字になるもの同士を辺で結ぶ(PA,BC)。
- 文字と数字の割り当てを反映させる(L=0,M=2,B=7)。
- 最上位の情報も付ける。



補足

- グラフの連結判定には、union-find を利用すると効率的 & 実装しやすい。
- サイズが小さいので探索をしても良いが、union-find を知っていると色々と便利なので、ぜひとも覚えておこう！
- 前回(ARC026)の解説にも載っているので、参考にしてください。
- http://www.slideshare.net/chokudai/arc026

C 問題 –
最高のトッピングにしような

問題概要

- スペシャルチケット X 枚と通常のチケット Y 枚を持っている。
- N 個のトッピングがあり、トッピング i は t_i 枚のチケット(うち 1 枚以上はスペシャルチケット) と交換でき、嬉しさは h_i である。
- 同じトッピングは複数回入手できない。
- 嬉しさの合計値として最大なものを求めよ。
- $1 \leqq X \leqq 300$, $0 \leqq Y \leqq 300$, $1 \leqq N \leqq 300$
- $1 \leqq t_i \leqq 600$, $1 \leqq h_i \leqq 5{,}000{,}000$

部分点解法

- この問題はナップサック問題の一種。
- 動的計画法を用いると効率的に解答することができる。
- dp[i][j][k] = (現在トッピング i まで処理しており、すでにスペシャルチケットを j 枚、通常のチケットを k 枚消費している場合での嬉しさの合計値の最大値) とすると、

  dp[i][j][k] = max{dp[i-1][j][k] ,

  dp[i-1][j-m][k- t_i +m] + h_i (1 ≦ m ≦ t_i)}

(細かい境界条件は割愛)

- O($XYN(X+Y)$) となる。

満点解法

- 先ほどの場合、交換可能なすべての組み合わせを試していたが、出来る限りスペシャルチケットの消費数を抑えられるものを選んだほうがいいのでは?
- 実際、消費枚数が最も少ない (m が最小) となるもののみを考慮しても最適解を得られる。
- 実はこの問題は、「チケットを $X+Y$ 枚持っており、トッピングは最大 X 個しか選べない場合の最大化」と同じ問題。
- dp[i][j][k] = (現在トッピング i まで処理しており、トッピングを j 個採用しており、チケットを k 枚消費している場合での嬉しさの合計値の最大値) とおく動的計画法なら O($XN(X+Y)$) となる。

D 問題 −
ぴょんぴょんトレーニング

問題概要

- N 個の石があり、石 i からは h_i 個先まで跳べる。
- 石 s_i から石 t_i までジャンプで移動する組み合わせが何個あるか、というクエリを D 個解け。
- 同じトッピングは複数回入手できない。
- 嬉しさの合計値として最大なものを求めよ。
- $1 \leqq N \leqq 300{,}000$, $1 \leqq D \leqq 5{,}000$, $1 \leqq h_i \leqq 10$
- $1 \leqq s_i < t_i \leqq N$

部分点解法

- この問題も動的計画法によって計算できる。
- dp[i]=(石 i までの行き方) とおくと、

$$\mathrm{dp[i]} = \sum_{\mathrm{j}=1}^{10} (\mathrm{dp[i-j]} * f(\mathrm{i-j, i}))$$

ここで、$f(x, y)$は$x + h_x \geqq y$なら 1 、そうでないなら 0 を取る関数とする。

- 手前 10 項 (dp[i-1],dp[i-2],…,dp[i-10])を覚えておけば良い。
- 毎回 O(N) で計算すれば O(ND) となる。

満点解法

- 毎回計算する際に、同じ部分を何回も使用していることに気がつく。
- 同じ部分を「平方分割」によって処理することを考える。
- 手前 10 項を覚えておけば次の項を計算することができることを利用する。
- func(s,t,dp[s-1],dp[s-2],…,dp[s-10]) = dp[t],dp[t-1], … , dp[t-9] のそれぞれの値を、dp[s-1], dp[s-2], … , dp[s-10] を用いて変換する関数。
- func の返り値は 10 個ある。

どゆこと?

- 例として dp[99] の値を知りたいときに、dp[20],dp[19],dp[18], … ,dp[11] の値さえわかっていれば、

  dp[99]=c(20,99,21) * dp[20] + c(19,99,21) * dp[19] + … + c(11,99,21) * dp[11]

  と表記できる。

ここで、c(x,y,z)=(x から y まで移動するものの中で、最初のジャンプで z 以上の石に移動するものの総数) となる。

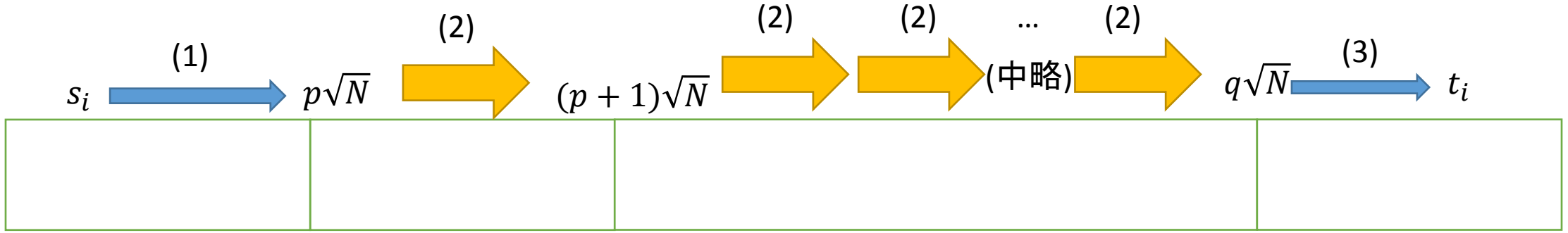
- dp[98] から dp[90] までについても同様に計算すれば func(21,99, ,dp[20],dp[19],...,dp[11]) の完成となる。

func の計算について

- c (s,t,u)は部分点解法と同じ方針で O(t-s) で計算できる。
- 「1 回目のジャンプで u 以上に移動する組み合わせ」としなければならないことに注意 (同じジャンプ重複して数えてしまう)。
- c (s,t,u) は、各 func に対して最大 100 種類計算すれば OK。
- func の計算は、1 つの func につき O($\sqrt{N}$) (100*$\sqrt{N}$) となる。

満点解法

- func($k\sqrt{N}, (k+1)\sqrt{N}$) ($0 \leqq k \leqq \sqrt{N}$) を計算しておき、各トレーニングについて、s_i から $\sqrt{N}$ の倍数になるまで部分点解法と同じDP(1)→func で $\sqrt{N}$ 個ずつ飛ばせるだけ飛ばす(2)→ t_iになるまで部分点解法と同じDP(3)
- func の計算は、1 つの func につき O($\sqrt{N}$) なので、全体で O(N) となる。
- この方針でfunc を求めた後は O($D\sqrt{N}$) となる。

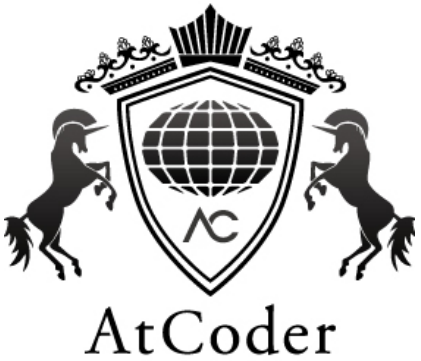


補足

- func は 10×10 行列として表現できる。
- ($n \times n$行列)* ($n \times n$行列)*…* ($n \times n$行列)* (n 要素ベクトル) の計算において、* の個数を m とする。このとき、
    - * を左から処理→行列*行列=行列を繰り返し、O($n^3 m$)
    - * を右から処理→行列*ベクトル=ベクトルを繰り返し、O($n^2 m$)

となり、この問題でも速度に 10 倍もの差が出る。このため、ベクトルを利用した計算をしたほうが速くなる。func を利用した計算の際には注意。

AtCoder Regular Contest 026
解説



AtCoder株式会社

A問題

ダイナミックなポーズ

問題概要

- 高橋君は１問の問題を解くのに
  ・普段：A分かかる
  ・ダイナミックなポーズをとりながら：B分かかる
- 体力の消耗が激しいので、ダイナミックなポーズをとりながら解ける問題は５問まで。
- N問の問題を解くのにかかる最短時間は？
- $1 \leqq N \leqq 10, 1 \leqq B < A \leqq 60$

解法

- Ｂ＜Ａなので、可能な限りダイナミックなポーズをとりながら解いたほうが良い。
- じゃあ答えは、5*B+(N-5)*A？
  （５問をダイナミックなポーズをとりながら解き、残りの問題を普段通りに解く）

  →　それは間違い

解法

- 5*B+(N-5)*A はダメ。
- Nが5より小さい時はダイナミックなポーズはN回しかしなくて良い。
- ```
  if(N < 5) answer = N*B;
  else answer = 5*B+(N-5)*A;
  ```

  というようにNが５より大きいか小さいか、
  つまり、５回の上限までダイナミックなポーズをとるかどうかで場合分けをしてあげればいい。
  ```

別のやり方

```
num_dynamic = min(N,5);
answer = num_dynamic*B + (N-num_dynamic)*A;
```

- のようにダイナミックなポーズをとる回数を計算してから答えを計算するというような方法も考えられます。

ARC026解説

B:完全数

B-問題概要

- 与えられた整数Nが完全数か、不足数か、過剰数か、判定せよ

- 部分点1　$1 \leq N \leq 10^5$
- 部分点2　$1 \leq N \leq 10^{10}$

B-基本的発想

- 約数を全て調べて、総和を求めて、元の数字との大小を調べたい
- 約数を出来るだけ速く全列挙するのが目標

- 「自分以外の約数」というのが少し厄介
- 自分を含めた約数の総和を求めて自分の2倍との大小を比べれば面倒な処理はしなくて良い

B-約数の全列挙1

- 整数Nの約数はN以下なので、N以下の全ての整数について、それがNの約数かどうかを調べれば良い
- 計算量は$O\left(N\right)$
  - 部分点1が得られる

B-約数の全列挙2

- AがNの約数ならば、N/AもNの約数
- AかN/Aの片方がわかれば、もう片方もすぐに分かる。
  - 小さい方だけを調べれば良い
- $A \leq N/A$となる条件は $A^2 \leq N$
- $\sqrt{N}$以下の約数を調べるだけで良い
- 計算量は$O(\sqrt{N})$
  - 満点が得られる

B-注意点

- Nが平方数の時$A = N / A$となるAが存在する
  - 誤ってAを2回足さないように気をつけなければならない
- 小課題2の入力が明らかにint型に収まらないので、C等の言語を使っている方は気をつけましょう

ARC024解説
C:蛍光灯

C-問題概要

- 長さがLの廊下にN個の蛍光灯がある
- i番目の蛍光灯はc_iの費用で点ける事ができる
- i番目の蛍光灯をつけると西からの距離がl_i以上r_i以下の範囲が照らされる
- 廊下全体を照らすための最小費用を求めよ

- 1 ≦ N ≦ 100,000
- 1 ≦ L ≦ 100,000
- 1 ≦ c_i ≦ 100,000

C-部分点1

- 蛍光灯をl_iが小さい順にソートする
  - 以降iにはこのソート後のインデックスを採用する
- dp[i][j] = i番目以前の蛍光灯を使って西から距離 j のところまでの全体を照らす費用の最小値
  - という値を**動的計画法**で求める
- もしjがl_iとr_iの間にあるならば

  dp[i][j] = min{dp[i-1][k] + c_i (kはl_iとr_iの間)}

  ないならば

  dp[i][j] = dp[i − 1][j]

  という漸化式が成り立つ

C-DPの更新の仕方

- l_iとr_iの間のjに対して

  dp[i][j] = min{dp[i-1][k] + c_i (kはl_iとr_iの間)}
- kをl_iからr_iまで順番に動かすときに、今まで通過したdp[i-1][k]の最小値を更新していく
  - dp[i][k] = その最小値 + c_iとなる
- dp[i]を全て求めるのにO(L)
- iはN種類なので、合計でO(NL)
  - 部分点1(N,L≦3,000)が得られる

C-DP擬似コード

```
for(int i = 0;i < n;i++){
    //dp[i-1][k]の最小値を保存する変数tmpmin
    int tmpmin = 99999999;
    for(int j = 0;j <= L;j++){
        if(j < l[i] || k > r[j]){
        dp[i][j] = dp[i - 1][j];
        }
    }
    for(int k = l[i];k <= r[i];k++){
    dp[i][k] = min(dp[i][k], tmpmin + c[i]);
        tmpmin = min(tmpmin, dp[i][k]);
    }
}
```

C-考察

- 先ほどのDPは2次元配列を使っているが、dp[i]はdp[i − 1]しか参照しない
  - 1次元配列で管理することができる
- 新しいdp配列
  - dp[i] = 西端から距離iのところまでの全体を照らすのに必要な費用の最小値

C-考察

- 実際に費用が最小になる蛍光灯の置き方を考えてみる

- 必ずr_iがとなりの蛍光灯の区間（両端含む）と重なる
  - dpではr_iの位置のみ更新すれば良い

C-擬似コード2

```
for(int i = 0;i < n;i++){
    //dp[i]の最小値を保存する変数tmpmin
    int tmpmin = 99999999;
    for(int k = l[i];k <= r[i];k++){
        tmpmin = min(tmpmin, dp[i][k]);
    }
    dp[r[i]] = min(dp[r[i]], tmpmin + c[i]);
}
```

C-高速化

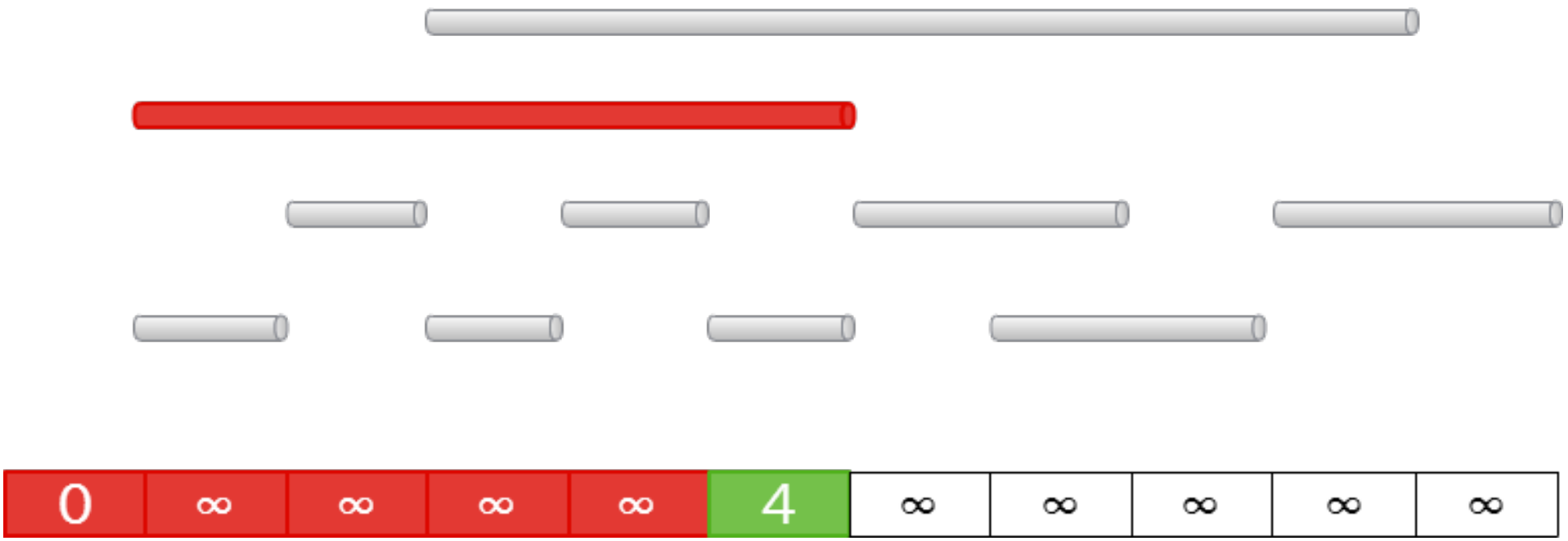
- 操作の種類は
  - dp[r[i]]の更新
  - dp[l[i] ～ r[i]]のなかの最小値を求める

- これはRMQなのでセグメントツリーがうまく利用できる

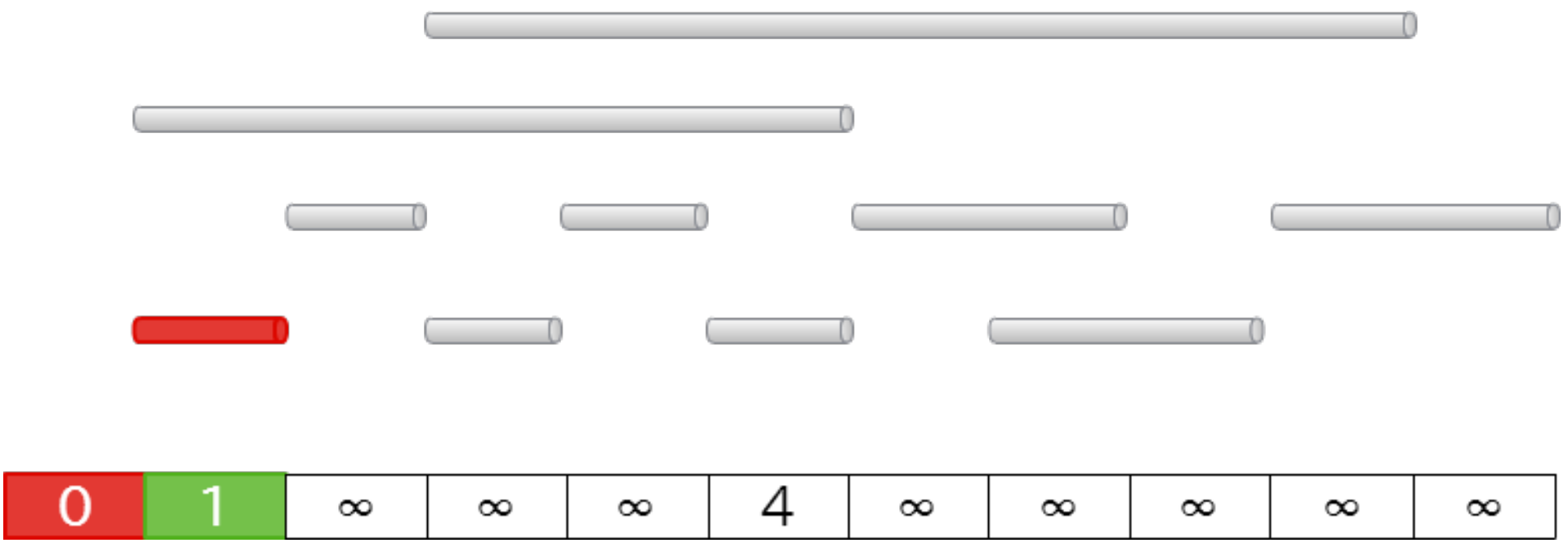
C-擬似コード3

```
for(int i = 0;i < n;i++){
    int tmp = getmin(l[i], r[i])
    //dp[l[i]～r[i]]の最小値をsegtree等で求める
    setnum(r[i], min(r[i], tmp + c[i]));
    //dp[r[i]]の値を更新
}
```

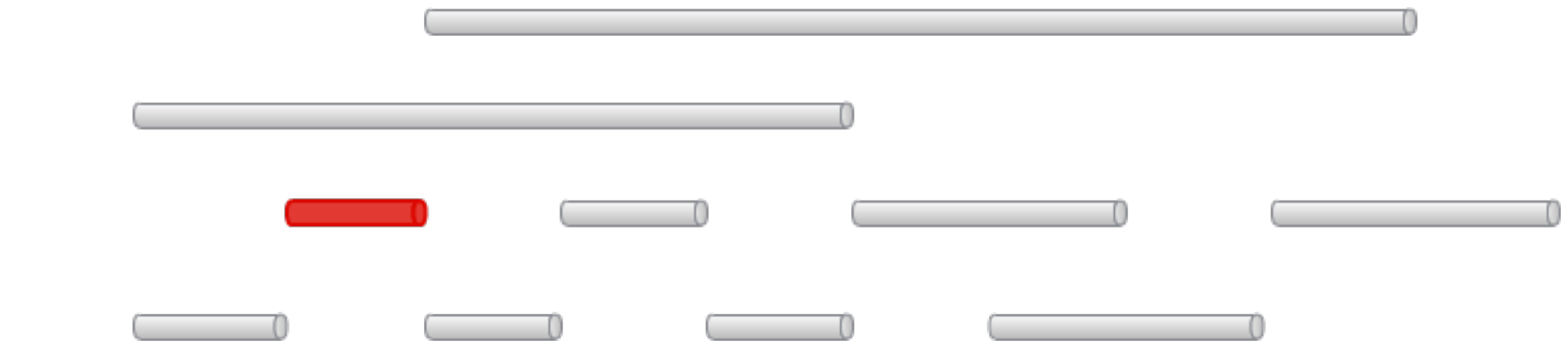
C-入出力例3図解



C-入出力例3図解

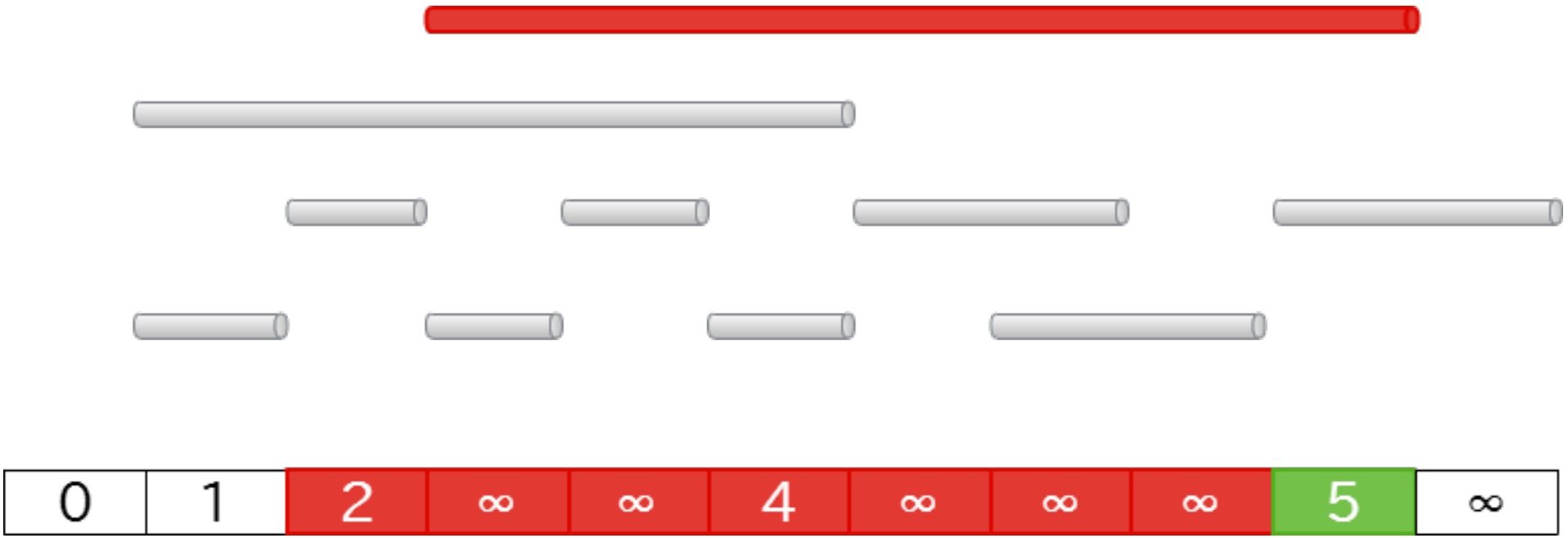


C-入出力例3図解



0	1	2	∞	∞	4	∞	∞	∞	∞	∞

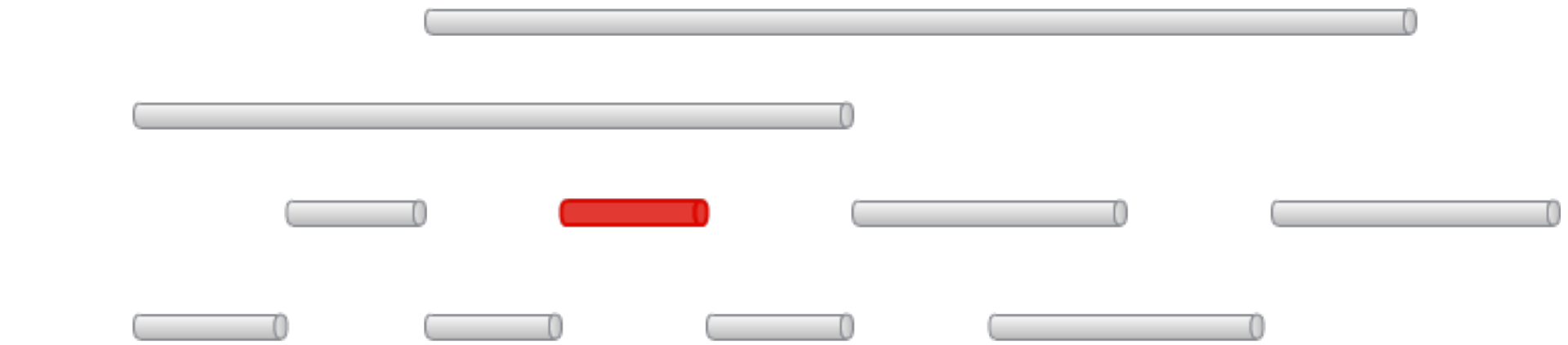
C-入出力例3図解



C-入出力例3図解

0	1	2	3	∞	4	∞	∞	∞	5	∞

C-入出力例3図解

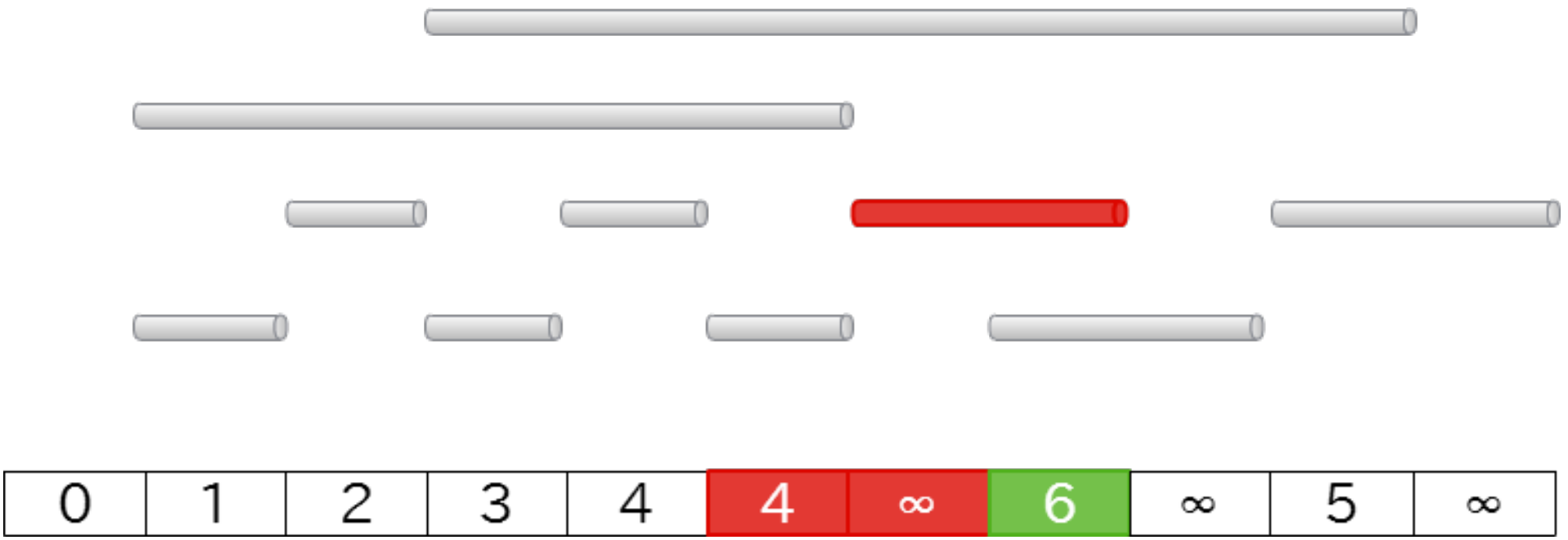


0	1	2	3	4	4	∞	∞	∞	5	∞

C-入出力例3図解

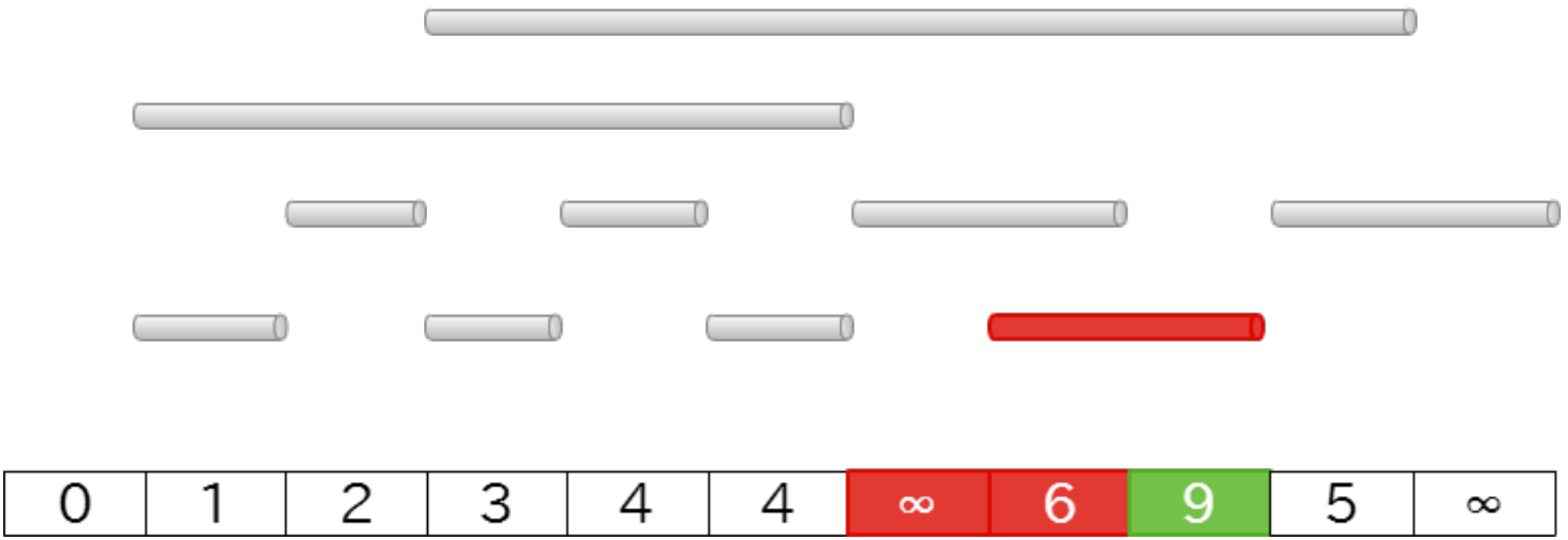
0	1	2	3	4	4	∞	∞	∞	5	∞

C-入出力例3図解

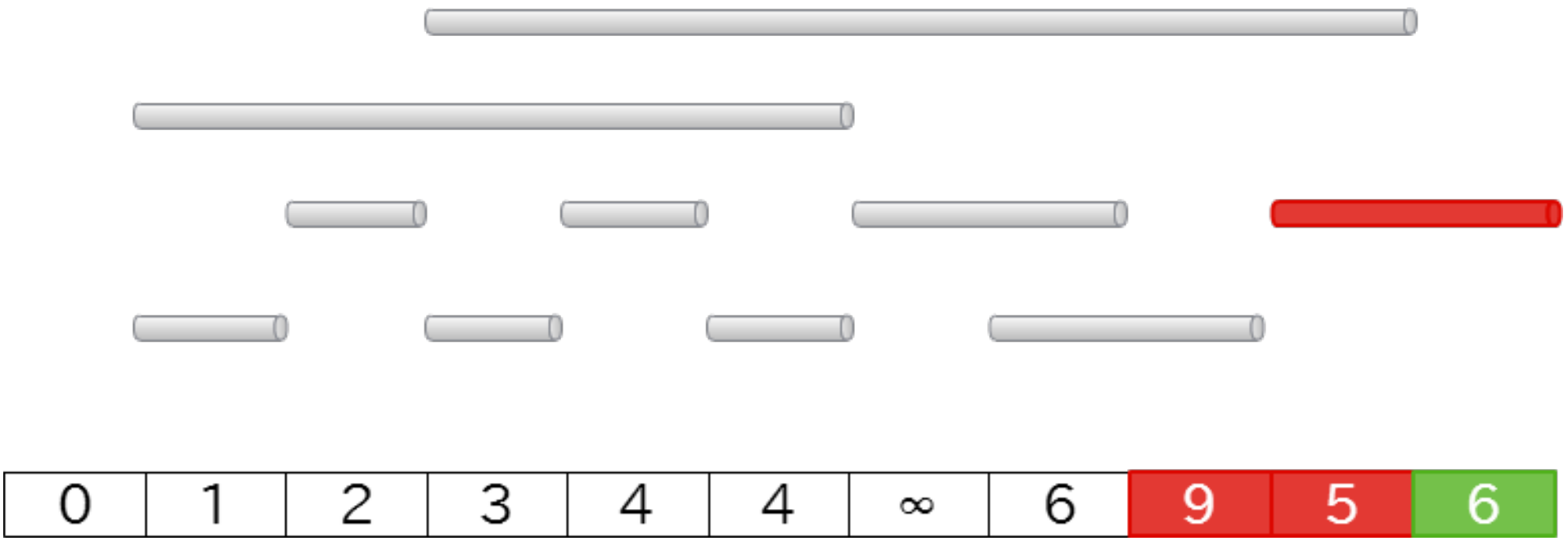


0	1	2	3	4	4	∞	6	∞	5	∞

C-入出力例3図解



C-入出力例3図解



C-高速化

- 各蛍光灯についてO(logL)でdp[r[i]]を更新する
- 計算量はO(NlogL)
  - 満点が得られる

C-参考

- セグメントツリーに似たデータ構造でBITというものがある
  - 比較的楽に実装できる
- BITは基本的にRMQはできないが、いくつかの制約のもとで処理できる
  1. 値の更新は、もとより値が小さくなるもののみ
  2. 求める範囲の左端は0でなければならない
- 今回1つめの条件は満たしている
- 2つめの条件を満たすようにすればBITを使うことができる

C-参考

- 実はmin(l[i], r[i]) は　min(l[i], L)としてもよい
  - 予めソートしてあるので、i番目の蛍光灯を見ている時、l[j] ＞ l[i]となる蛍光灯jはまだ処理されていない
  - l[i] ～ Lと重なっている蛍光灯は、l[i] ～ r[i] と重なっている蛍光灯のみであり、その他余計な蛍光灯は重なっていない
- よってL側を片端とするBITでACすることができる
- 計算量はセグメントツリーと同様

D問題

道を直すお仕事

問題概要

- N頂点M辺のグラフが与えられる
- 各辺はC,Tの二つの値を持っている
- グラフが連結になるように辺を選ぶとき、
  Cの総和/Tの総和 の最小値は？
- 1 ≦ N,M ≦ 10^4, 1 ≦ Ci,Ti ≦ 10^6

20点解法 (M ≦ 16)

- それぞれの辺を修理するかどうかを全て試す
- 各辺につき、する/しないの２通りがあるので全部で2^M通りのパターンがあり得る
- グラフが連結になるような辺の選び方のうち、Cの和/Tの和が最小となるものが答え

連結判定 方法1

- 適当な頂点を選び、その頂点を含む連結成分の大きさをDFSなどで計算し、それがNであれば連結

連結成分の大きさを求める例：

```
bool used[N]; // falseで初期化

int dfs(int v){
  if(used[v]) return 0;
  used[v] = true;
  int count = 1;
  for(Edge e : vから出ている辺){
    count += dfs(e.to);
  }
  return count;
}
```

連結判定 方法２

- 以下のアルゴリズムのように、連結成分を集合として管理する

各頂点に対し、その頂点のみを含む集合を作る

```
for(Edge e : 修理することにした辺){
  e.fromが属する集合とe.toが属する集合を合併する
}
```

全ての頂点が同じ集合に属しているかをチェックする

- これを実現するためのデータ構造として、Union Find 木を使う

Union Find 木

- 以下の２つの操作が出来るデータ構造
  - ・ある要素がどの集合に属しているかを調べる
  - ・２つの集合を合併する
- 要素を頂点とした木構造で集合を管理する
- 各頂点は親への辺を持っておく
- 根の要素をその集合の代表と考える
- 詳細は以下のスライドが分かりやすいと思います
  http://www.slideshare.net/iwiwi/ss-3578491

Union Find 木

- 実装例 (ならし計算量O(α(N)))

```
int parent[N];
void init(){
  for(int i = 0; i < N; ++i) parent[i] = -1;
}
int root(int x){
  if(parent[x] == -1) return x;
  return parent[x] = root(parent[x]);
}
void unite(int x, int y){
  x = root(x); y = root(y);
  if(parent[x] < parent[y]) swap(x,y);
  if(parent[x] == parent[y]) parent[x]--;
  parent[y] = x;
}
```

20点解法 (M ≦ 16)

- 辺の選び方を全て試すためにはビット演算を利用すると書きやすい

```
for(int i = 0; i < 1<<M; ++i){ // 選び方を全て試す
  for(int j = 0; j < M; ++j){
    if((i>>j&1) == 1) j本目の辺は今選ばれている

  }
}
```

- あわせて計算量はO(2^M * N)となり、この部分点を得ることが出来ます

満点解法 (M ≦ 10000)

- 答えで二分探索
- Σ Ci / Σ Ti ≦ x となるような最小のxが答え
- 小数の二分探索は収束判定が難しいので、
  50回だけ探索するというようにすると良い。

二分探索

- Σ Ci / Σ Ti ≦ x を式変形をすると・・・
- Σ Ci / Σ xTi ≦ 1 → Σ Ci ≦ Σ xTi → Σ Ci-xTi ≦ 0
- 辺ごとに独立して計算できるようになる！

二分探索

- Σ Ci-xTi ≦ 0 を満たすような辺の選び方があるかどうかを判定したい
- Ci-xTi ≦ 0となる辺は全て選んで良い
- グラフが連結になるように、残ったCi-xTi > 0の辺からいくつかの辺を選ぶ。
  → 最小全域木問題のアルゴリズムが使えそう

クラスカル法

- クラスカル法：最小全域木を求めるアルゴリズム

```
answer = 0
for(Edge e : 辺をコストの昇順に){
  if(e.fromとe.toが違う連結成分に属している){
    answer += e.cost
    e.fromの連結成分とe.toの連結成分を合併する
  }
}
```

連結成分の管理にはUnion Find 木を使うと良い

最小コストを求める

- 今回の問題は最小全域木問題とは少しだけ違いますが、似たようなアルゴリズムで解くことが出来ます

```
answer = 0
for(Edge e : 辺をコストの昇順に){ // コスト = c-xt
  if(e.fromとe.toが違う連結成分に属している
      または e.cost < 0){
    answer += e.cost
    e.fromの連結成分とe.toの連結成分を合併する
  }
}
```

満点解法 (M ≦ 10000)

- 二分探索で探索する回数をKとすると、
- 計算量はO(K M log M)となり、Kが100程度でも満点を得ることが出来る。

ARC #025 解説

解説:城下 慎也 (@phidnight)

A 問題 – ゴールドラッシュ

問題概要

- 7 日間かけて stove 君が鉱山採掘をする。
- それぞれの日に砂漠にある鉱山かジャングルにある鉱山から採掘を行う。
- 各日ごとにどちらの鉱山でどれだけ採掘できるのかが異なる。
- 得られる金の合計値を最大化せよ。

解法 その1

- 砂漠かジャングルをそれぞれの日に選ぶ組み合わせは全部で $2^7 = 128$ 通りなので、すべての組み合わせについて全探索して最大値を求めることで解を得ることができる。

解法 その2

- すべての組み合わせを見なくても、それぞれの日についてより多くの金を得られる鉱山を貪欲に選び続ける方針でも最適解を得ることができる。
- この解法だとO(採掘日数) の答えになるので、もっと採掘日数が多い場合でも高速に解を求めることができる。
- 今回はどちらの解法でも解けるので、実装のしやすい方とかを選ぶと良い?

B 問題 – チョコレート

問題概要

- 縦 H × 横 W のマス目が与えられる。
- ブラックチョコとホワイトチョコが市松模様を構成している。
- 各マスごとに濃度が定まっている。
- ブラックチョコのみの濃度合計とホワイトチョコのみの濃度合計が一致しているような長方形の面積最大を求めよ。
- $1 \leqq H \leqq 100$, $1 \leqq W \leqq 100$

方針

- 純粋に、すべての長方形について検討してみると、$O((HW)^3)$となり、かなり時間がかかってしまう。
- うまくまとめて計算することで、高速に計算したい。
- 白と黒を同時に考えるのをやめて、白と黒を分離してみよう！

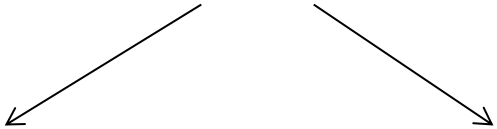
方針

3	1	4	1
5	9	2	6
5	3	5	8

- 例えば、上のようなチョコレートの配置において、

方針

3	1	4	1
5	9	2	6
5	3	5	8



3	0	4	0
0	9	0	6
5	0	5	0

0	1	0	1
5	0	2	0
0	3	0	8

- 例えば、上のようなチョコレートの配置において、次のように黒のみと白のみ(相方の色の場所は０にする)分解する。

解法

- すると、長方形が条件を満たすことと、長方形が指定する領域内の黒合計(長方形になっている)と白合計(長方形になっている)が一致することが同値であるようになった。
- 長方形領域にある数字の合計を高速に計算する方法である、累積和が使用できる！
- 累積和を用いることで、$O((HW)^2)$になる。

おまけ

- ホワイトチョコの場所を符号反転させると、条件を満たす長方形領域では濃度合計値が 0 になる。
- この方法だと累積和 1 個で済む。
- どちらかお好きな方で！

解法

- すると、長方形が条件を満たすことと、長方形が指定する領域内の黒合計(長方形になっている)と白合計(長方形になっている)が一致することが同値であるようになった。
- 長方形領域にある数字の合計を高速に計算する方法である、累積和が使用できる！
- 累積和を用いることで、$O((HW)^2)$になる。

C 問題 – ウサギとカメ

問題概要

- 頂点数 N で辺数が M のグラフが与えられる。
- (目的地, ウサギのスタート地点, カメのスタート地点) を頂点から互いに異なるように選ぶとき、カメがウサギより真に先に到着する組み合わせの総数を計算せよ。
- $3 \leqq N \leqq 2{,}500$, $2 \leqq M \leqq 3{,}000$

ウサギとカメの戦略

- ウサギもカメも、できるだけ早く目的地につけばいいので、どちらも最短経路を通ることになる。
- 目的地を固定しておけば、目的地からそれぞれの点への最短経路長を 1 回計算して知っておくことで、ウサギとカメの割り当てごとに最短経路を計算する必要がなくなる。

全探索

- 上記の方針を実装しても、そもそも 3 つの組の選び方が $O(N^3)$ あるので間に合わない。
- うまくまとめて計算したい。
- 2 つの手順を施すことで実現してみよう！

手順１

- それぞれの点について考慮するときに、調べる順番を長さの短い順に調べる方針にしても問題ない。
- あらかじめ最短経路長のリストをソートしておく。

手順 2

- ウサギにとっての最短経路長を X, カメに取っての最短経路長を Y とすると、カメが真にウサギより先に到着するためには、X/R > Y/T が成立しなければならない。
- 両辺に RT をかけることで、XT > YR が得られる。
- カメの最短経路長を固定すれば、X がいくつ以上である必要があるかがわかる。

解法

- 目的地ごとに最短経路長のリストを求めて、ソートしておく。
- カメの位置を固定すると、ウサギは特定の最短経路長以上である必要が出てくる。
- その領域は必ずリスト上で連続した領域になるので、二分探索やしゃくとり法で計算できる。

備考

- 同時についた場合に注意。
- 組に同じ地点を選べないことに注意。
- カメがウサギより速い場合があることに注意。特に同じ地点を選べないことに絡むケースがある。
- 答えは 32 bit 整数に収まるとは限りません。

D 問題 – コンセント

問題概要

- 縦 H ×横 W の細長いコンセントが与えられる。
- 1 × 1 のコンセントキャップを着脱する。
- 1 × 2 あるいは 2 × 1 のコンセントプラグを 0 個以上差し込む方法の総数を 10^9+7 で割った余りを計算せよ。
- $1 \leqq H \leqq 2\,, 1 \leqq W \leqq 10\text{^}11$

満点解法(別解)

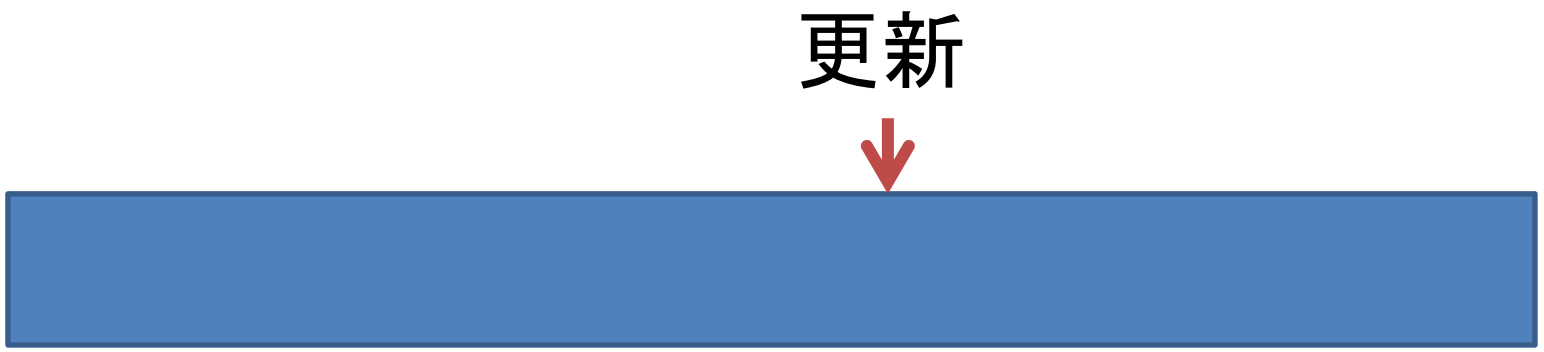
- 出現回数から算出したハッシュ関数を使用することでも高速に計算できる。
- ただし、ハッシュの衝突に注意。
- 複数キー用意するとより安全。

部分点解法 その1

- それぞれの操作後の盤面において、端から順に置く方針とする。
- X 列目を検討する際、X-1 列目の埋まり方さえ見ればそれ以前の置き方を参照する必要はない。
- Bit DP で計算することができる。
- O($WN2^{2H}$) などになる。(実装による)

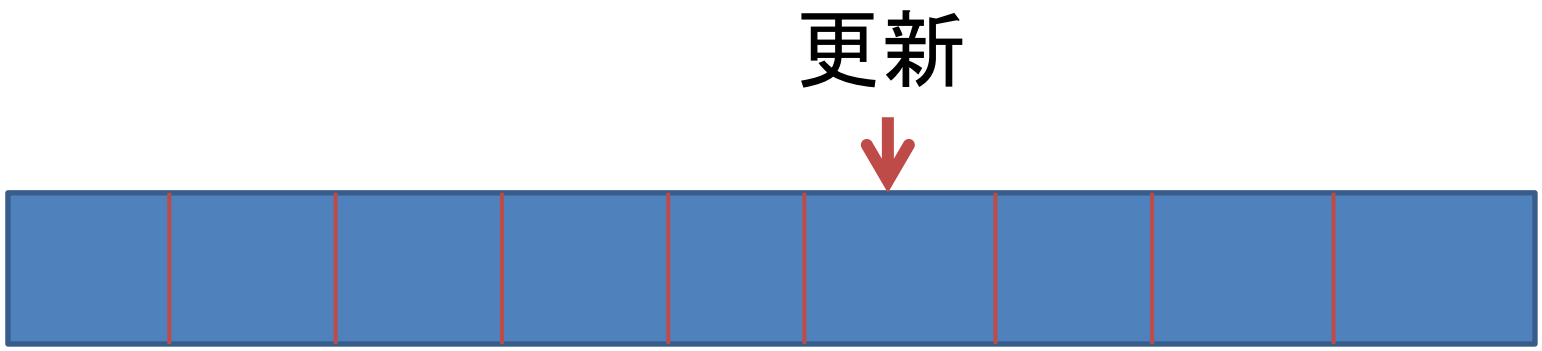
部分点解法 その2

- Bit DP で計算できるという方針は崩さずに考える。
- 毎回 1 箇所しか更新しないのにいつも幅Wを見るのは勿体ない。



部分点解法 その2

- Bit DP で計算できるという方針は崩さずに考える。
- 毎回 1 箇所しか更新しないのにいつも幅Wを見るのは勿体ない。
- 幾つかのバケットに分けて計算すると効率よい。



部分点解法 その2

- バケットが保存するべき状態:
  - 左端の使用状況
  - 右端の使用状況

  を満たした場合での組み合わせ数。
- 全体の計算においては、各バケットの使用状況の組み合わせと接合部のコンセントプラグの組み合わせと参照してマージする。
- 平方分割だと $O(\sqrt{W}N2^{4H})$ (実際は実装による)の計算量になる。

補足

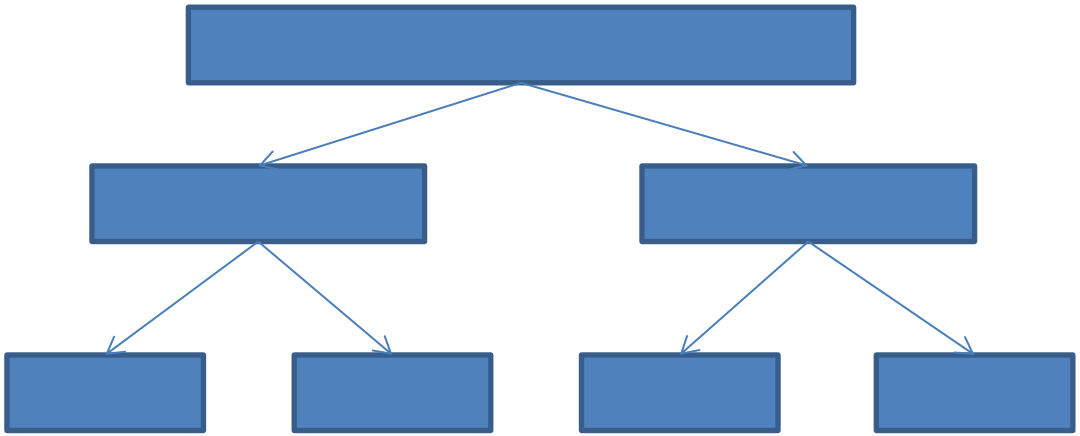
- 平方分割の方針の代わりに segment-tree の方針でも計算することができる。
- 後述の満点解法では、segment-tree を用いた場合を設定してある。

満点解法

- W がとんでもなく大きい場合、メモリとして確保することすらできなくなる。
- しかしながら、そのほとんどは何も操作を施されていない区間で構成されている。
- この性質をうまく用いることはできないだろうか?

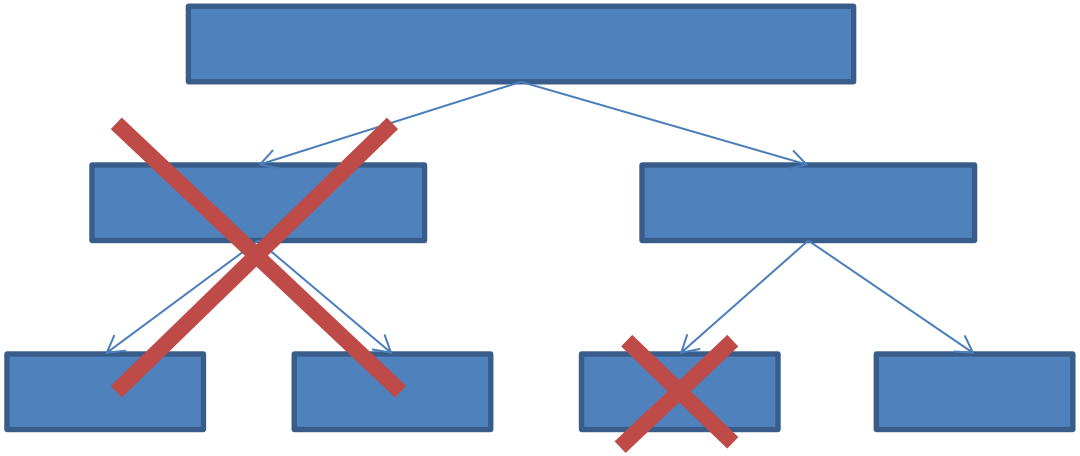
満点解法

- segment tree のノードをすべて生成する方針だときつそう。



満点解法

- segment tree のノードをすべて生成する方針だときつそう。
- キャップの生成したことのない場所を打ち切る segment tree だとサイズは OK になる。



満点解法

- segment tree のノードが存在する場所は通常通り計算すれば良い。
- そうではない場所の場合の計算結果を知りたい。
- それぞれの長さについて予め計算しておくか、2 の累乗の長さのみを前計算しておいて使いまわす方針で対策できる。
- $O(N \log W \, 2^{4H})$ 等の計算量になる。

ARC024解説

A:くつがくっつく

A-問題概要

- 左足の靴がL足、右足の靴がR足あり、それぞれのサイズが与えられる
- 両足のサイズが同じになるようペアを作るとき、最大で何組のペアを作成できるか？

- $1 \leqq L, R \leqq 100$
- $10 \leqq$ 靴のサイズ $\leqq 40$

A-問題の分割

- あるサイズの靴が、違うサイズの靴のペアを作成できるかどうかに影響しないので、同じサイズどうしに分けて独立の問題として解いて良い

- サイズSに対して、左足がL_S足、右足がR_S足あるとき、最大で何組のペアを作成できるか？
  - 明らかにL_SとR_Sの最小値を取れば良い

A-分割の仕方

- あるサイズの靴がいくつあるか数える
  - L[i]　:= サイズ i の左足の靴がいくつあるか
  - R[i]　:= サイズ i の左足の靴がいくつあるか
  - 入力のたびにインクリメントする事によって求まる

A-まとめ

- 各サイズについて左右いくつずつあるか数える
- 各サイズについていくつペアが作れるかを調べる
  - L[i]とR[i]の最小値を取ることによって求まる
- それらの総和が答えとなる

ARC024解説

B:赤と黒の木

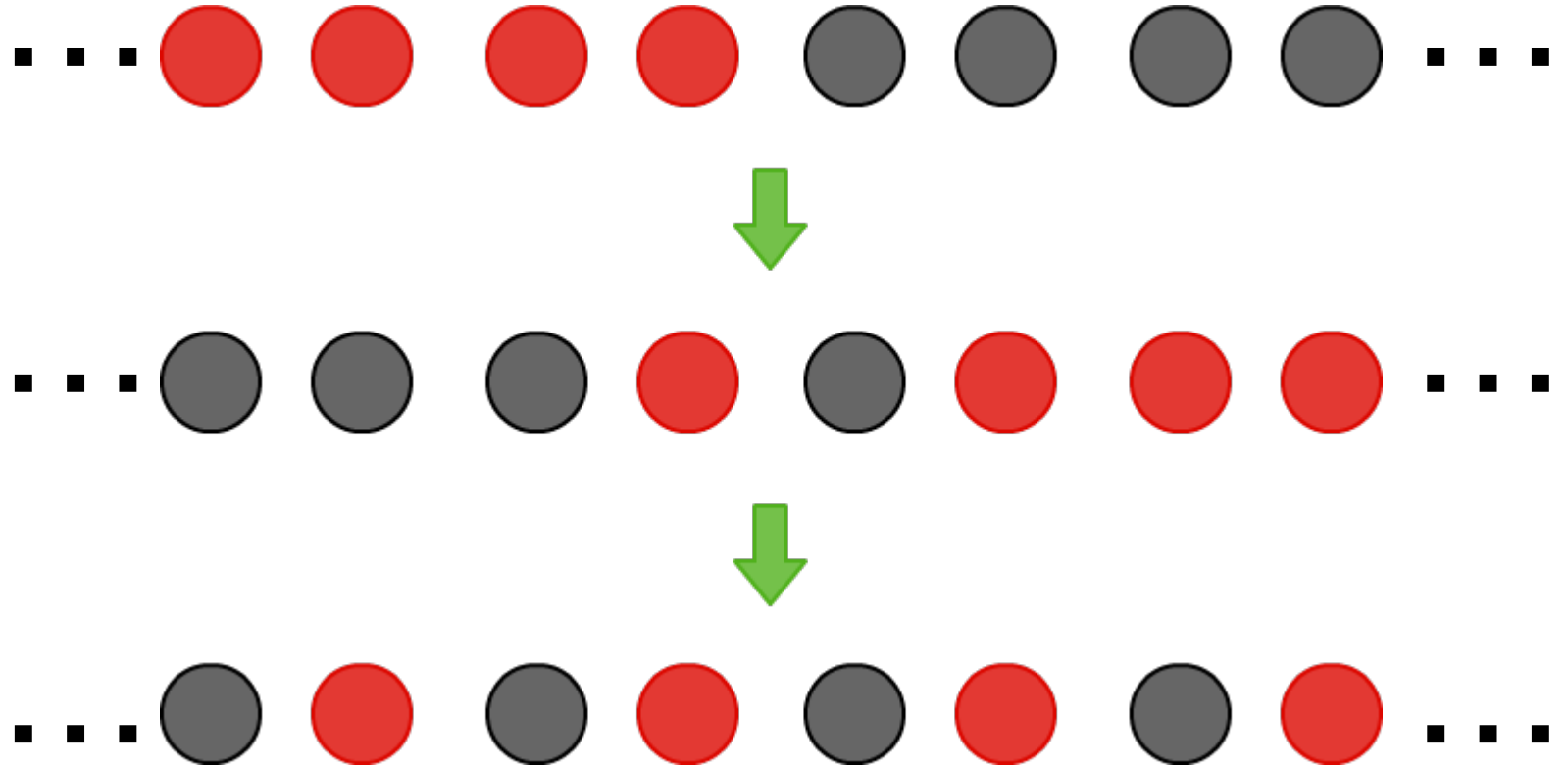
B-問題概要

- 環状に並んだ赤もしくは黒の木がN個ある
- 1日ごとに隣の状況をみて色が変化する
  - 自分と両隣の色が同じなら異なる色に変化
- 変化がなくなるまでの日数を求めよ

- $3 \leqq N \leqq 100,000$

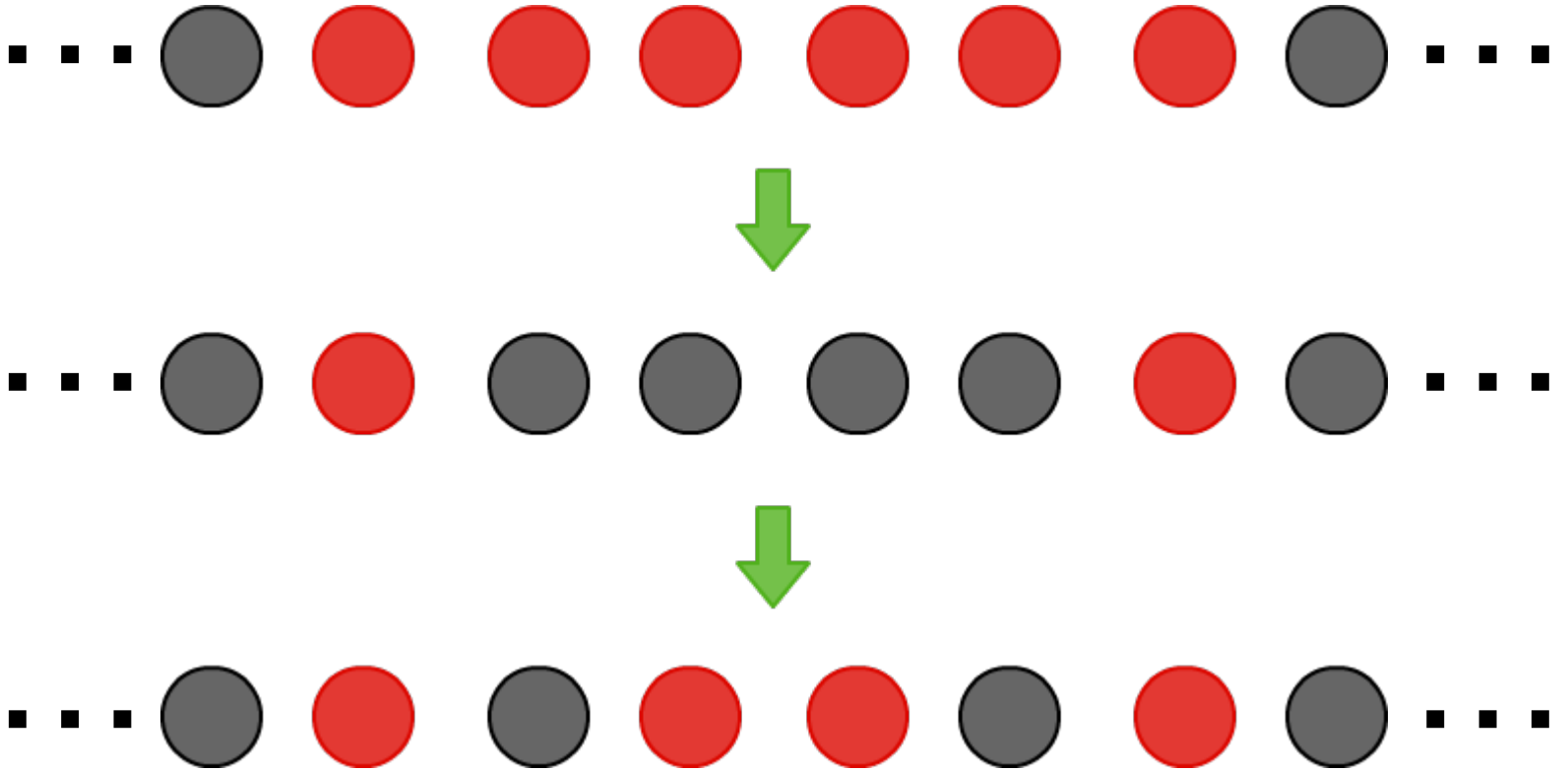
B-実験

- 見慣れないややこしい設定なので、いくつか実験して様子を観察してみる
  - 異なる色が隣り合うところ



B-実験

- 見慣れないややこしい設定なので、いくつか実験して様子を観察してみる
  - 同じ色が長く連続するところ



B-実験

- 隣に異なる色がいる木はそれ以降ずっと変わらない
  - 連続する同じ色の塊にわけて考えて良い
- X個連続して同じ色が並ぶと ［(X-1) / 2］日後に止まる
  - ただし［x］はxを超えない最大の整数を表す

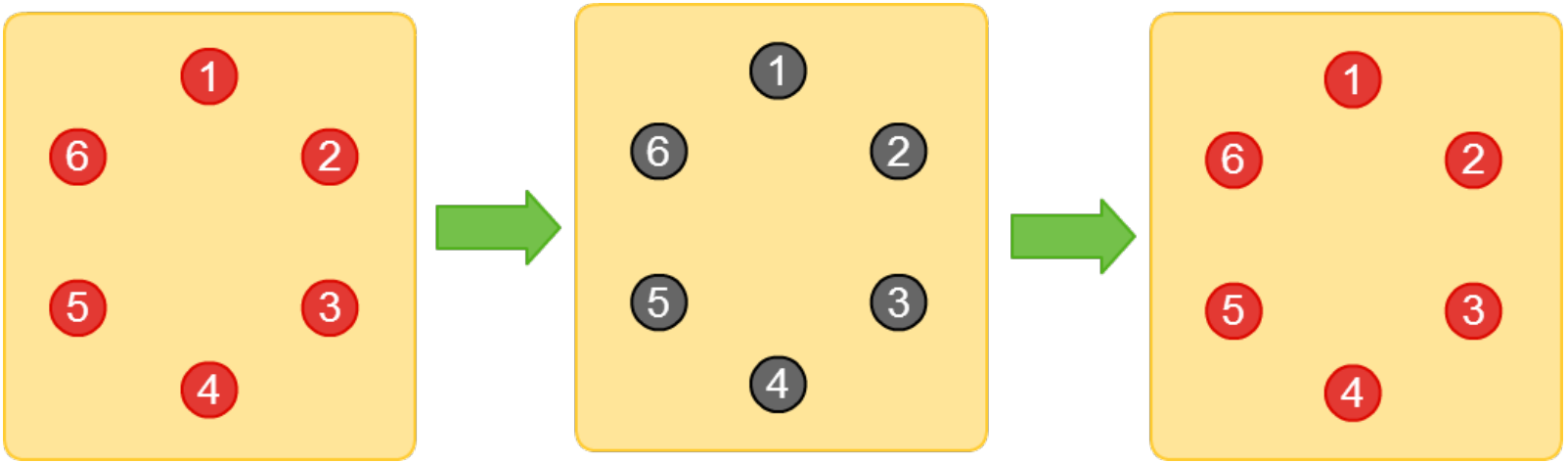
ということがわかる

B-環状

- どうやら、連続する部分を数えて適当な演算をし、最大値を取れば答えが出るらしい
- しかし、木が環状に並んでいるので少しややこしい
- 環状ではなくしたい
  - 適切な場所で切り、1列に変換する
- 適切な場所とは？
  - 異なる色が隣り合っているところ

B-コーナーケース

- 「違う色が隣り合っているところ」が無いケースがありうる
  - 全部赤、もしくは全部黒
- この場合は永久に終わらない(入出力例3)
- 他とは別に処理しなければならない



B-まとめ

- 全部同じ色かどうか調べる
  - 全部同じだったら出力は -1
- 異なる色が隣り合う場所で切り1列にする
- 同じ色が連続する塊に分け、それらの色が変わらなくなるまでの日数を求める
  - 長さがXならば［(X-1) / 2］日後
- それらの最大値が答えとなる

ARC #024 C 問題解説

解説:城下 慎也 (@phidnight)

問題概要

- 長さ N の文字列が与えられる。
- ある長さ K の(連続)部分文字列を 2 箇所で取る。
- 両者が共通部分を持たず、かつ一方が他方のアナグラムとなっているものがあるか。
- 例: abcdbcae だと、**abc**d**bca**e という 2 箇所 abc と bca は共通部分を持たず、かつ一方が他方のアナグラムとなっている。
- $1 \leqq K \leqq N \leqq 100{,}000$

部分点解法 その1

- アナグラムかどうか判定する 2 箇所の候補すべてを考えてみる。
- 開始点は左から 1 番目～ $N-K+1$ 番目までが選べるので、$O(N^2)$ 通りの組み合わせがある。
- 各組み合わせで $K!$ 通りのアナグラムの割り当てがあるので、全て試す方針で $O(N^2K!)$ 。これで 15 点が得られる。

部分点解法 その2,3

- アナグラムであることと、各文字の出現回数が一致していることが同値であることを利用する。
- 1 文字ずつ数えて出現回数を見る方針だと全体で$O(N^2K)$ で計算できる。30 点が得られる。
- 上記の計算で、文字種ごとの累積和を用いると$O(N^2)$ になり、45 点が得られる。

満点解法に向けて

- 部分点解法で行っていたことは、$N-K+1$ 通りの始点候補のうち、始点から K 文字所得した際の各文字の出現回数が一致する 2 箇所が存在するか判定することであった。
- 各文字(26通り)の出現回数は、これを大きい整数と思うことで、次の問題に帰着することができる。

帰着した問題

- 入力: 整数 N と N 個の整数。
- 出力: N 個の整数のうち 2 回以上出てくる整数が存在するか(YES or NO)。

- この問題は、すべてのペアを試すと$O(N^2)$ の計算量となるが、ソートして隣り合うペアのみを見る/二分探索木を使用することで$O(N \log N)$で計算できる。

満点解法

- 累積和などで、$N-K+1$ 通りの始点候補に対する、そこから K 文字とったときの各文字の出現回数を$O(N)$で計算しておく。
- 求めた値をキーとしてソートをするか、二分探索木を用いることで$O(N \log N)$で判定する。
- 全体で$O(N \log N)$となり、100点が得られる。

満点解法(別解)

- 出現回数から算出したハッシュ関数を使用することでも高速に計算できる。
- ただし、ハッシュの衝突に注意。
- 複数キー用意するとより安全。

ARC024解説

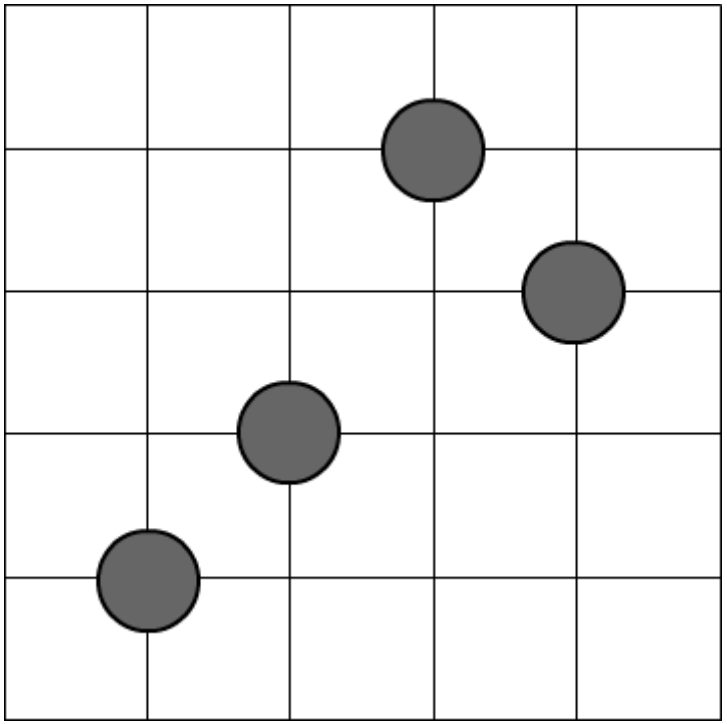
D:バス停

D-問題概要

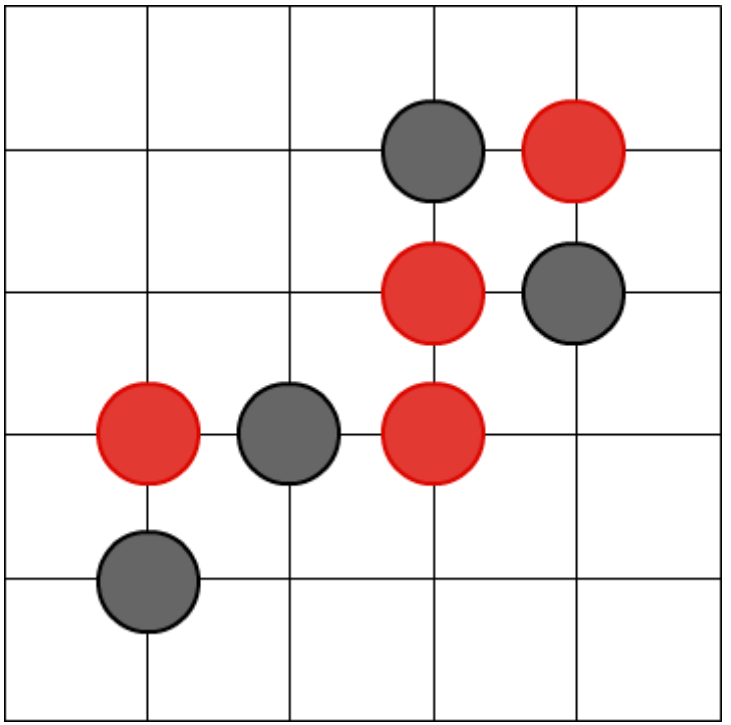
- 1,000×1,000の座標上にN個バス停が置かれる
- 1つのバスはx軸かy軸のどちらかに平行な移動しかできない
  - バス停で乗り換えればいろいろなところに行ける
- どのバス停間も乗り換えのみで総移動距離がマンハッタン距離と等しくなるような移動ができるようにする
- 合計が10,000個を超えないように新たなバス停を追加せよ
- $1 \leqq N \leqq 1{,}000$

D-入力例2図解

- 入力例2



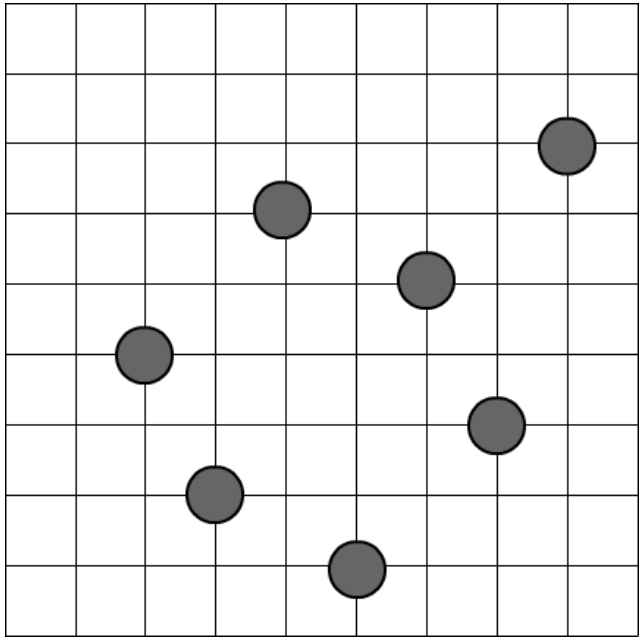
before



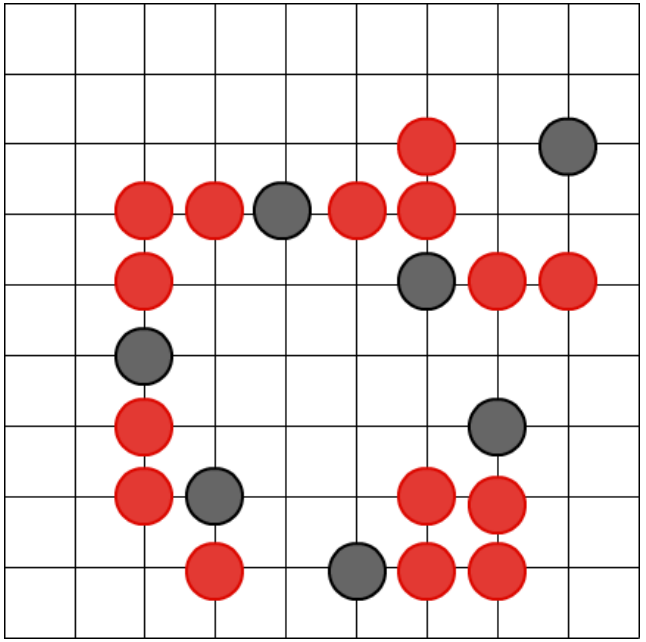
after

D-入力例3図解

- 入力例3



before



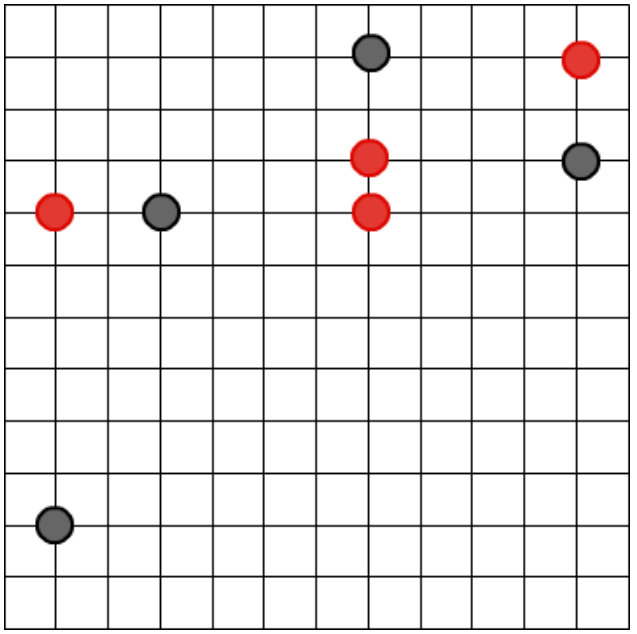
after

D-考察

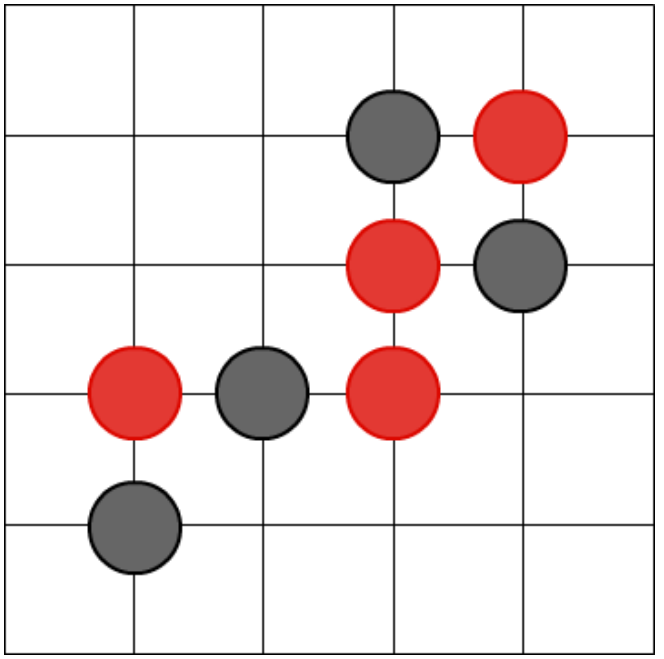
- 全ての格子点にバス停を置けば明らかに条件を満たす
  - 格子点は1,000×1,000個あり、全て埋めると制約を軽く超えてしまう
- 追加前に1つもバス停がない列や行はないものとして考えて良い
  - 座標圧縮ができる

D-座標圧縮

- 座標圧縮とは
  - 必要のない座標を無視して、位置関係を変えないように、新たに小さな座標に置き換えること



before



after

D-座標圧縮

- 圧縮の仕方
  - 圧縮前後で頂点の位置関係が変わっていなければ良い
  - 位置関係というのはxやyの大小関係
  - 出てくるxの種類について小さい順に1から番号を振っていく、それを圧縮後の座標とすればよい
    - 例: {3, 5, 10, 23} → {1, 2, 3, 4}
  - yについても同様

D-座標圧縮

- 圧縮後の性質
  - xの値とyの値が全て違ったとしても、それぞれたかだかN種類しかない
  - 圧縮後はN*Nより以下のサイズになる

D-部分点1

- 座標圧縮した後に、すべての格子点をバス停で埋める
- 合計で最大N*N個のバス停を置くことになる

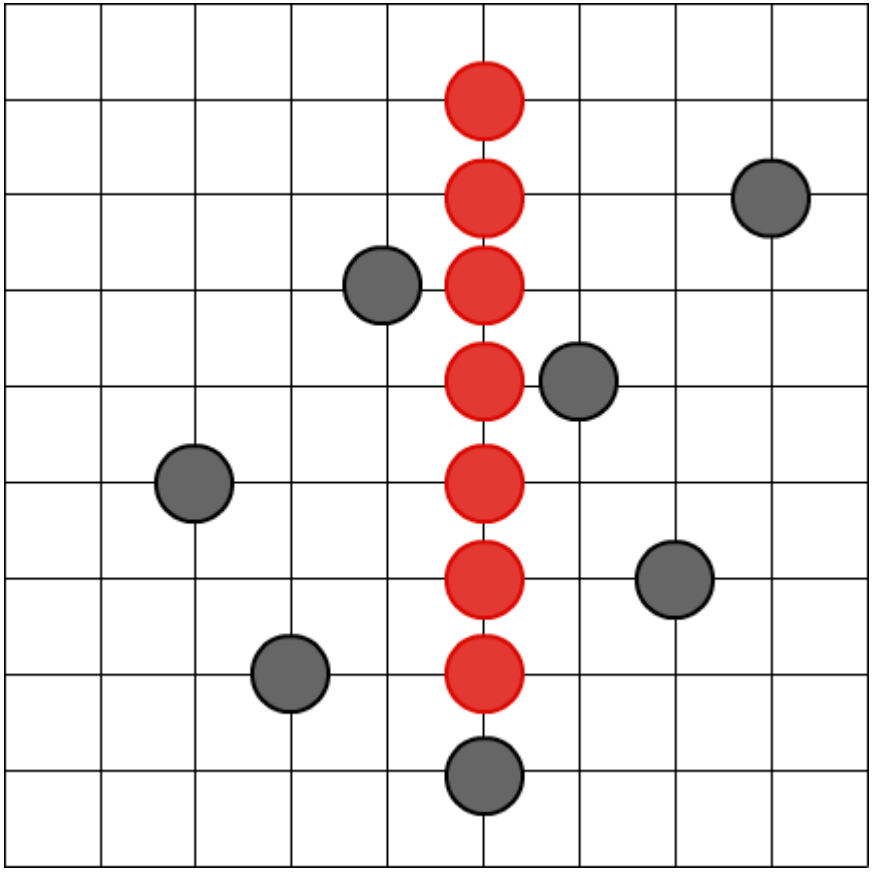
- 部分点1(N ≦ 100) 分の点数を獲得できる

D-考察2

- さすがに格子点を全部埋めるのは無駄が多すぎる
- 1列だけというのはどうだろう？
- ある1列を選んでその列すべての格子点にバス停をおいたときどうなるか考える

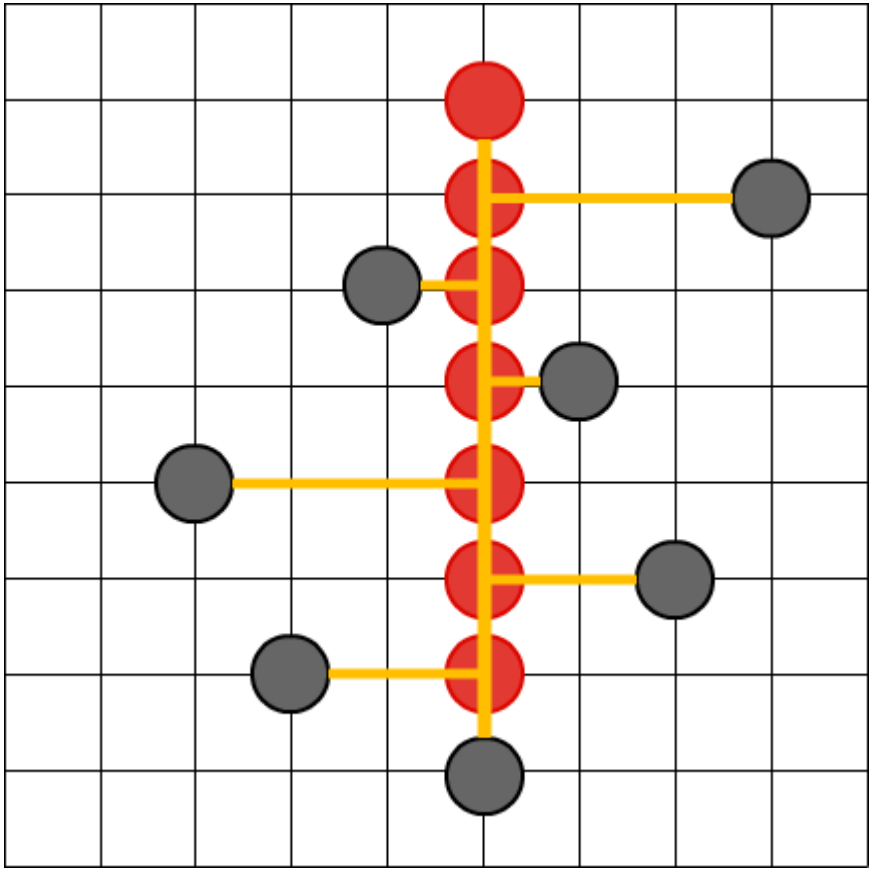
D-1列を埋める

- 入力例2の真ん中の列を埋めてみる



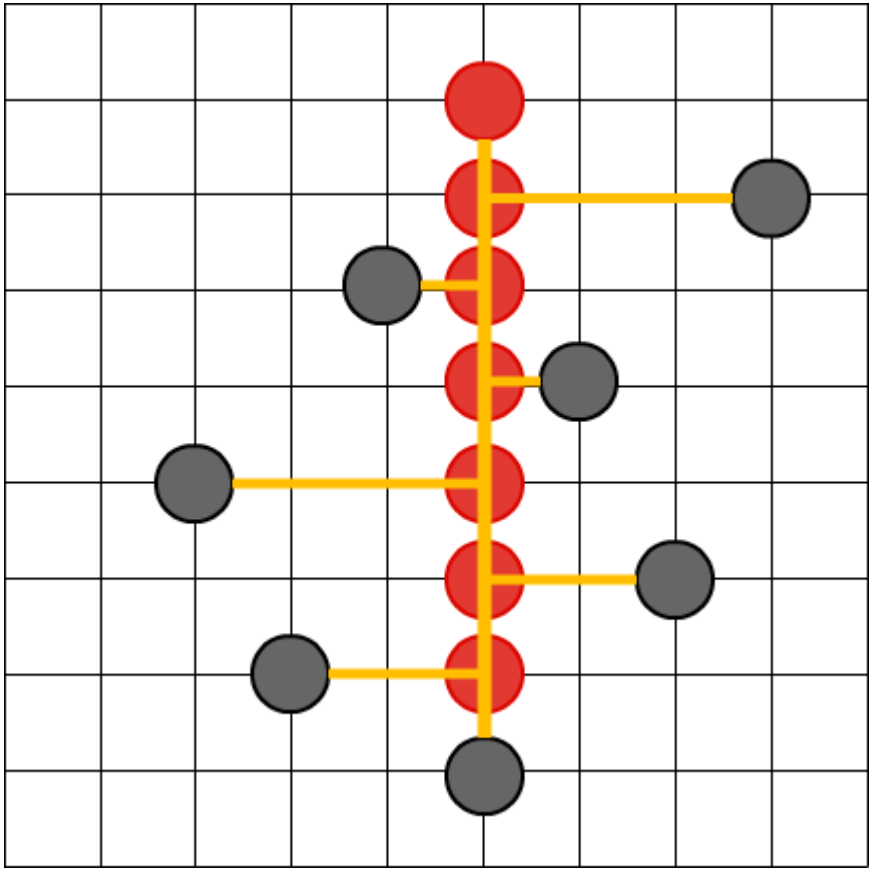
D-1列を埋める

- 入力例2の真ん中の列を埋めてみる
- 列の左右に注目



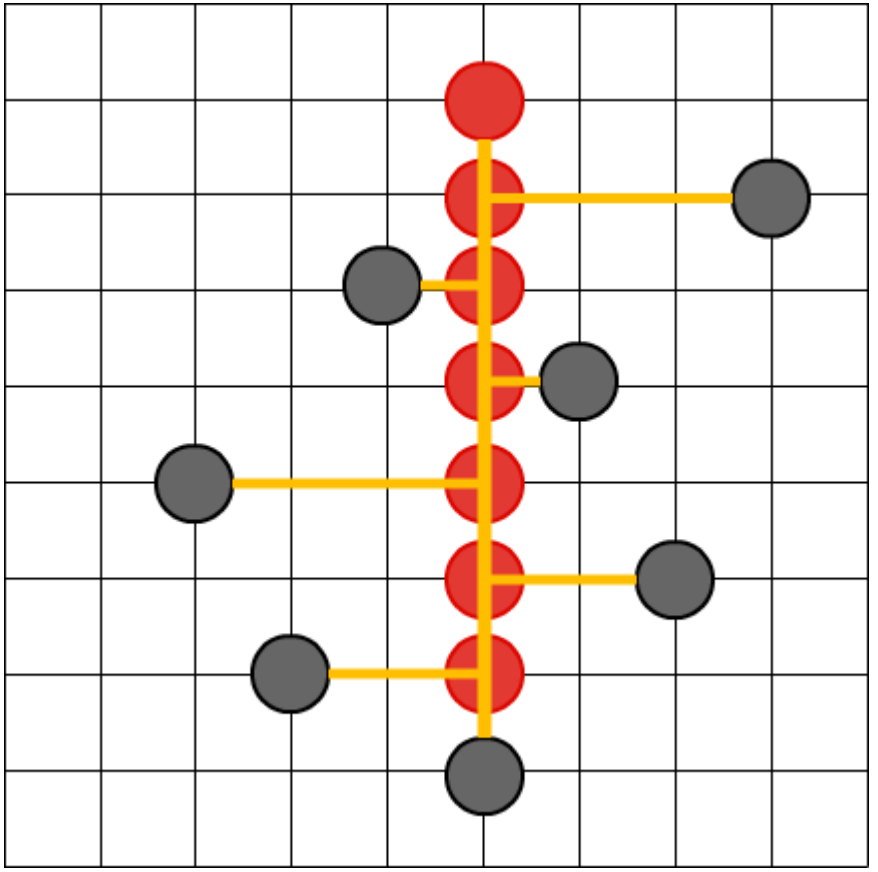
D-1列を埋める

- 入力例2の真ん中の列を埋めてみる
- 列の左右に注目
- 真ん中の列は完全に埋まっているので右図のような黄色い線が引ける



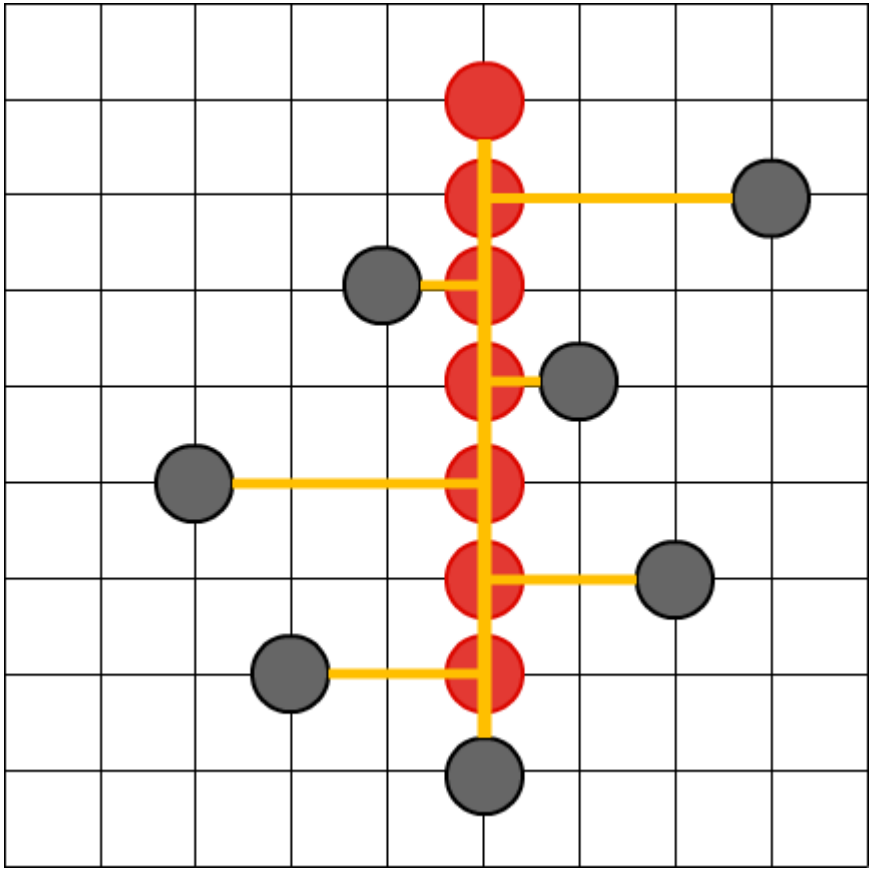
D-1列を埋める

- 入力例2の真ん中の列を埋めてみる
- 列の左右に注目
- 真ん中の列は完全に埋まっているので右図のような黄色い線が引ける
- 列をまたぐ移動はすべて条件をみたすことになる



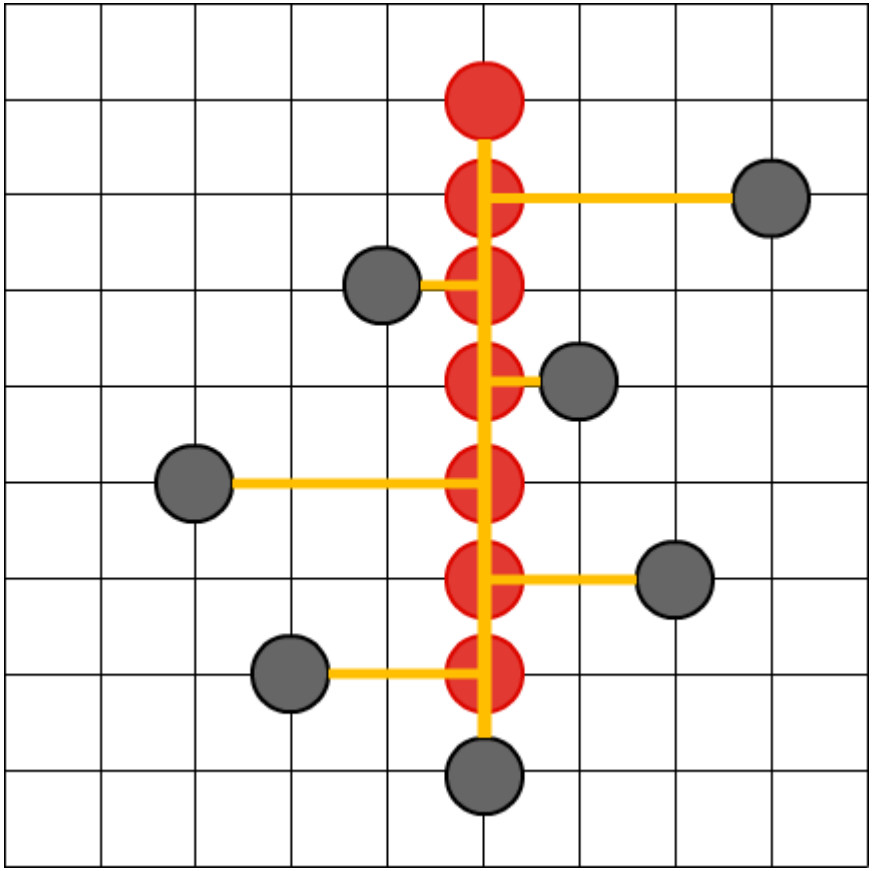
D-1列を埋める

- 入力例2の真ん中の列を埋めてみる
- 列の左右に注目
- 真ん中の列は完全に埋まっているので右図のような黄色い線が引ける
- 列をまたぐ移動はすべて条件をみたすことになる
- 左と右を別々に考えて良くなる



D-1列を埋める

- 入力例2の真ん中の列を埋めてみる
- 列の左右に注目
- 真ん中の列は完全に埋まっているので右図のような黄色い線が引ける
- 列をまたぐ移動はすべて条件をみたすことになる
- 左と右を別々に考えて良くなる
  - 良いアルゴリズムがある



D-1列を埋める

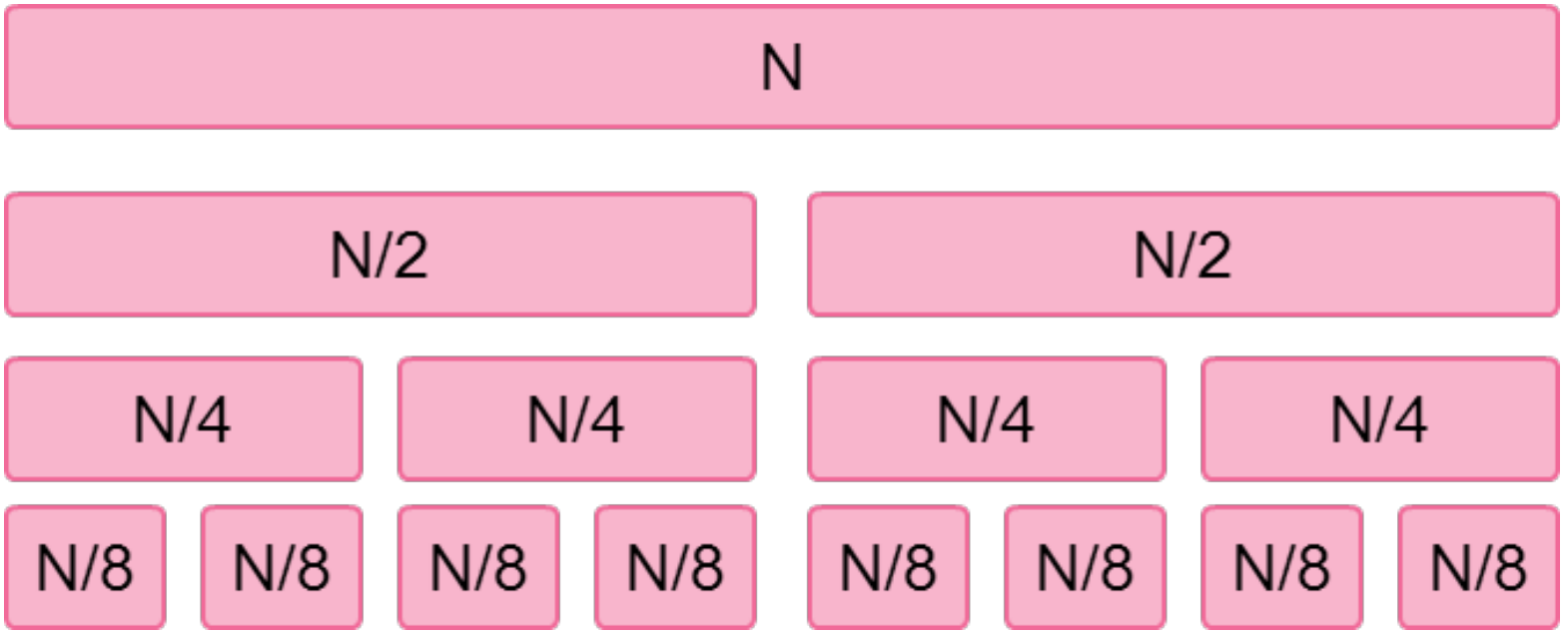
分割統治

D-分割統治

- ある問題のサイズNのバージョンを解くのに必要な計算の回数をf(N)とする
  - f(1) = O(1)が成り立つとする
- サイズNの問題はO(N)の計算でサイズがN/2の問題2つに分割することができるとする
  - f(N) = O(N) + 2 * f(N/2)　となる
- これを解くとf(N) = O(N log N)となる

D-分割統治

- イメージ図



- 横に見ると、どの段も総和がN
- サイズ1になると分割が終わるので高さはlogN

D-分割統治

- 分割統治は計算量を減らすテクニックだが、今回は置くバス停の量を減らすテクニックとして応用できる

D-想定解

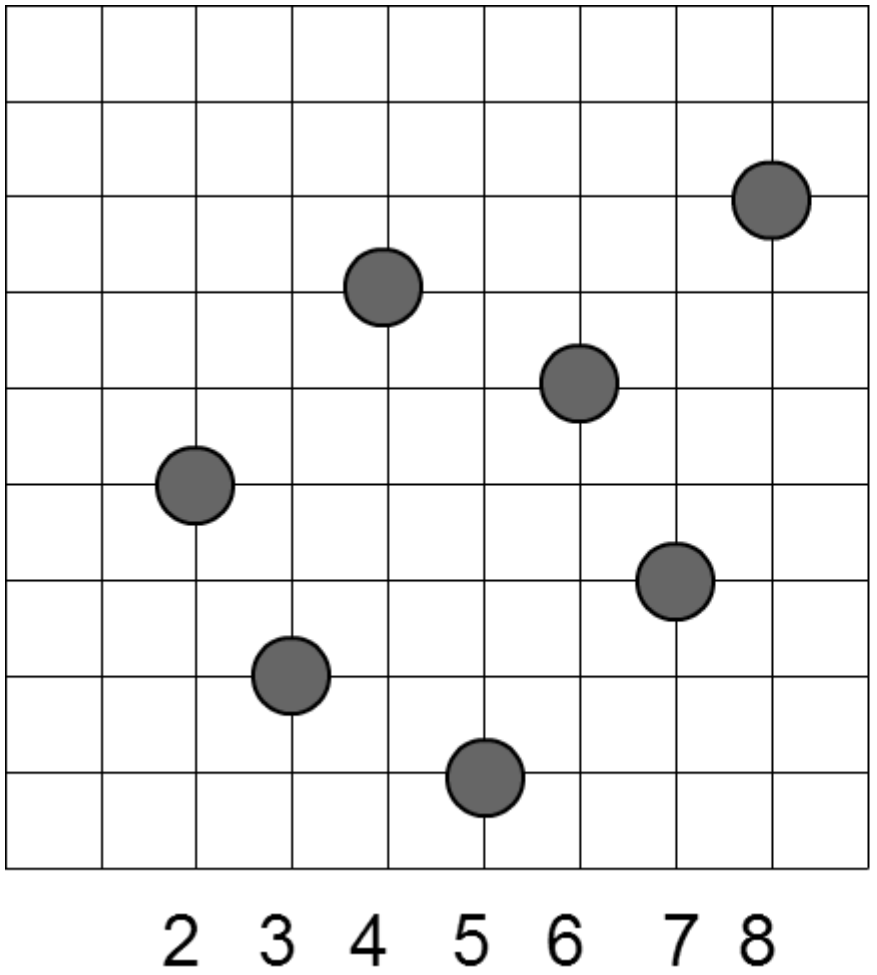
- 左右で個数が2等分されるような列を選ぶ
  - N個のバス停を設置
- N/2個のバス停の処理を2回する
- 同様に分割統治の要領で再帰していく
- サイズが半分になっても座標は大きいままなので、毎回座標圧縮することを忘れないように
- 合計でNlogN個のバス停を置くことになる

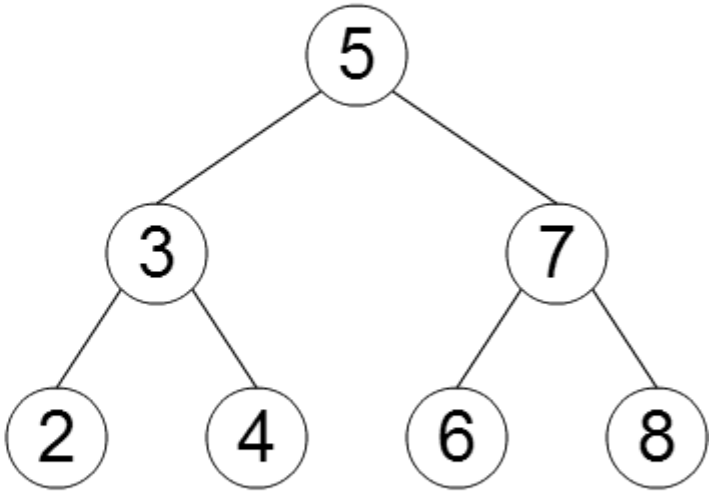
- NlogN ＜ 10,000なので満点が得られる

D-参考

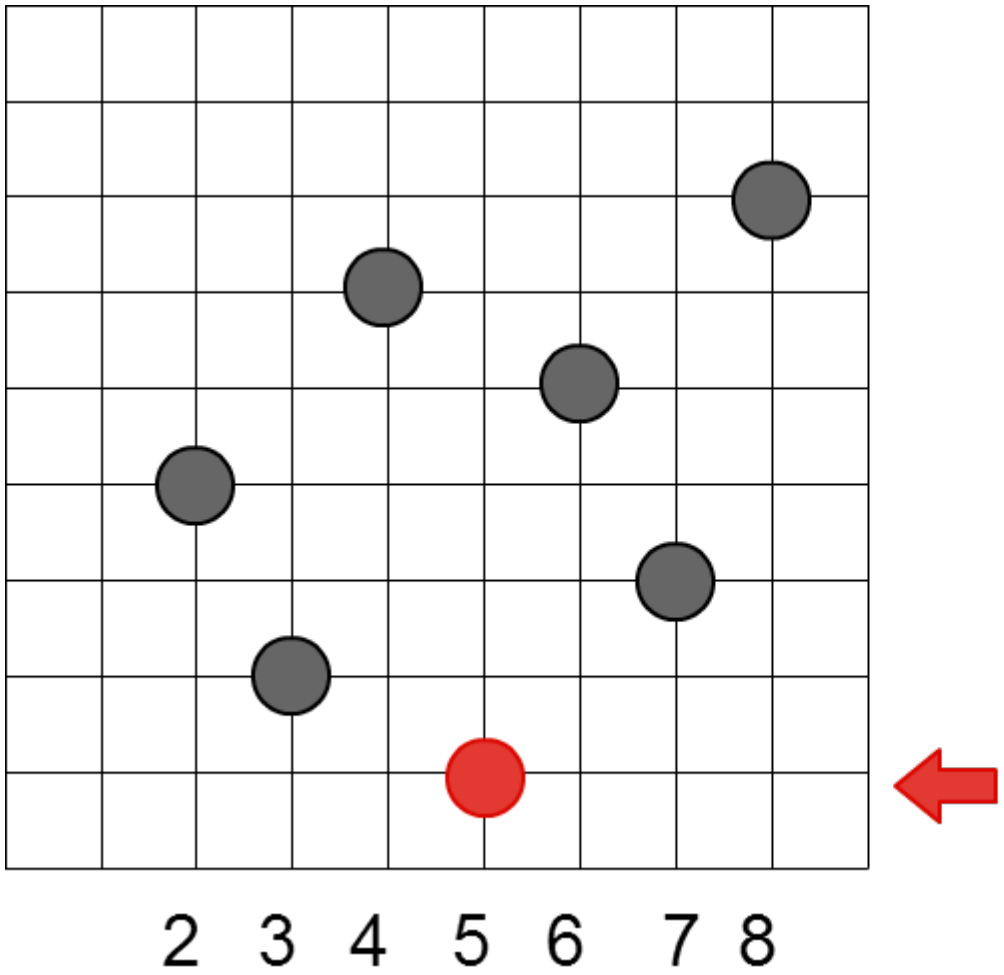
- 次のような処理をしても、条件に合うバス停の置き方ができる
- 登場するxの値を管理する二分探索木を作る
- yの値が小さい順に予め設置してあるバス停を見ていき、その二分探索木でその点のxの値を探索する
- 探索の過程で通過した頂点の値を[a1, a2...]とする
- (a1, y) (a2, y)... を追加していく

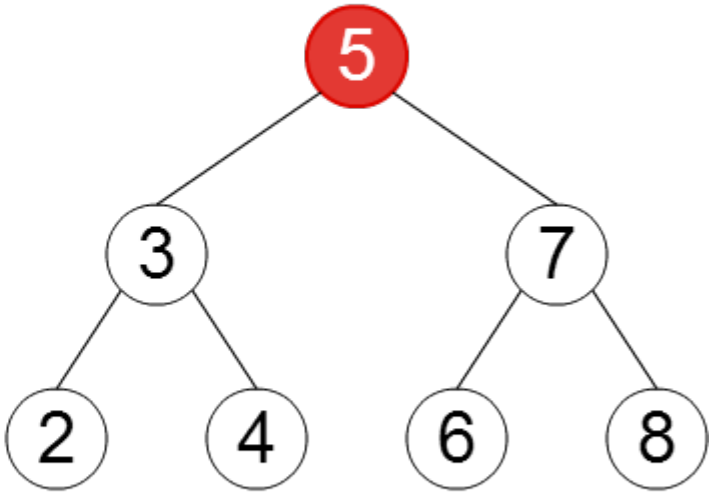
D-図解(入力例3)



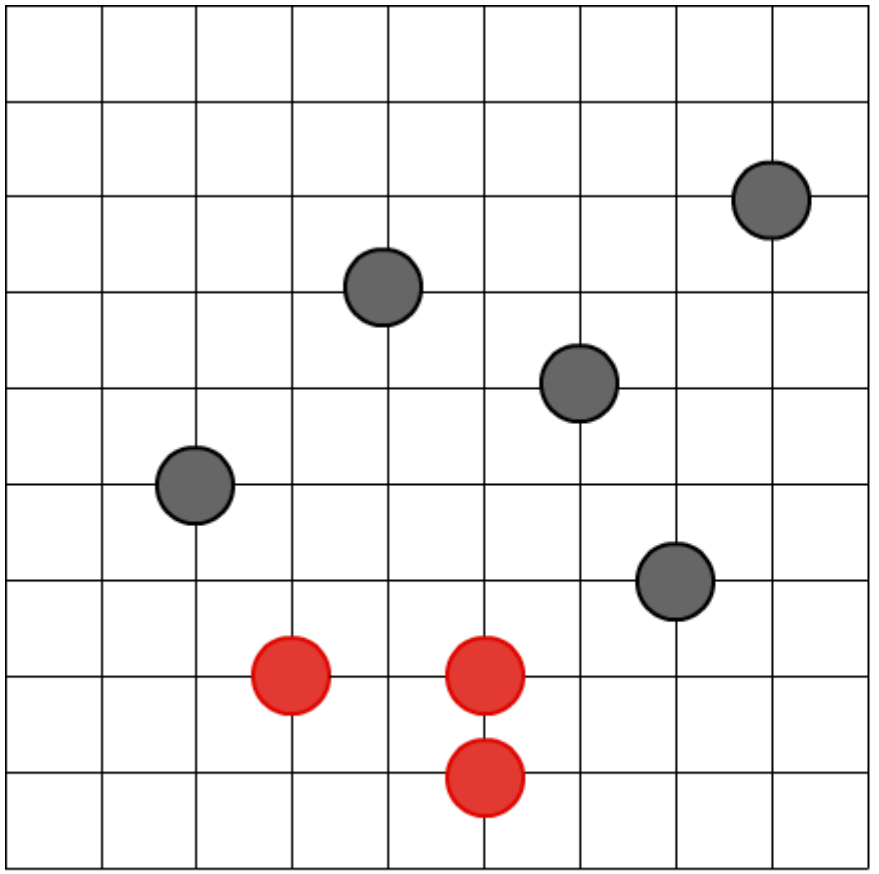


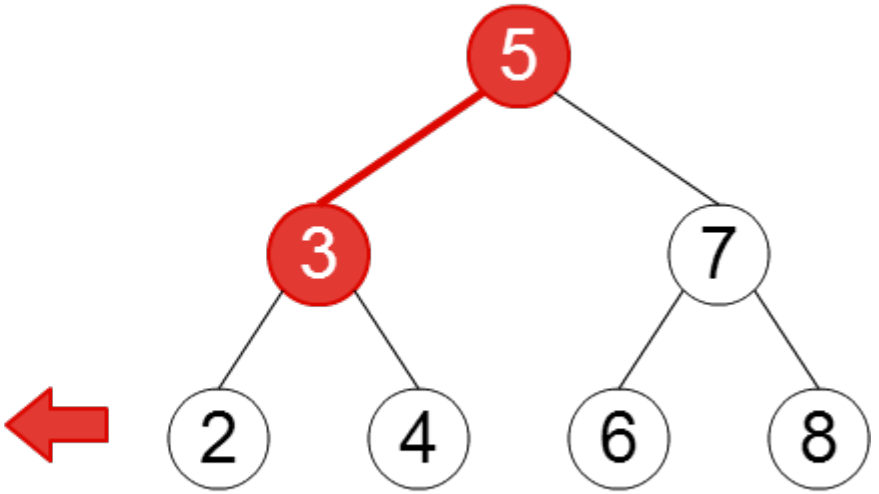
D-図解(入力例3)



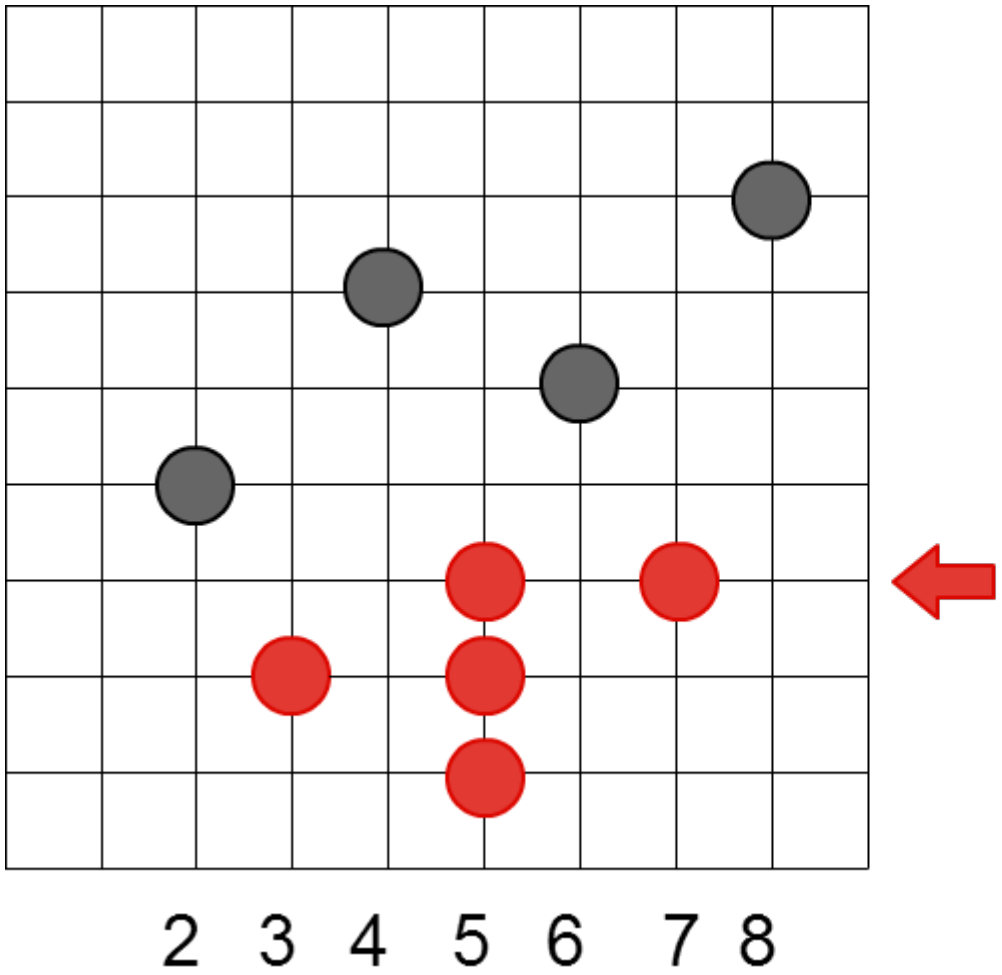


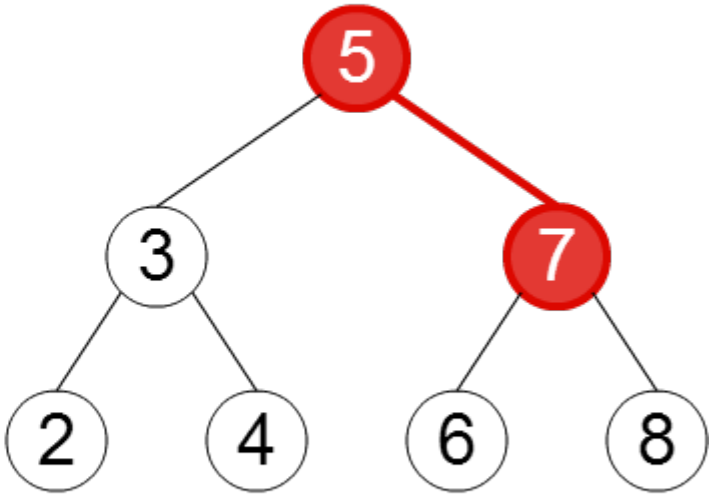
D-図解(入力例3)



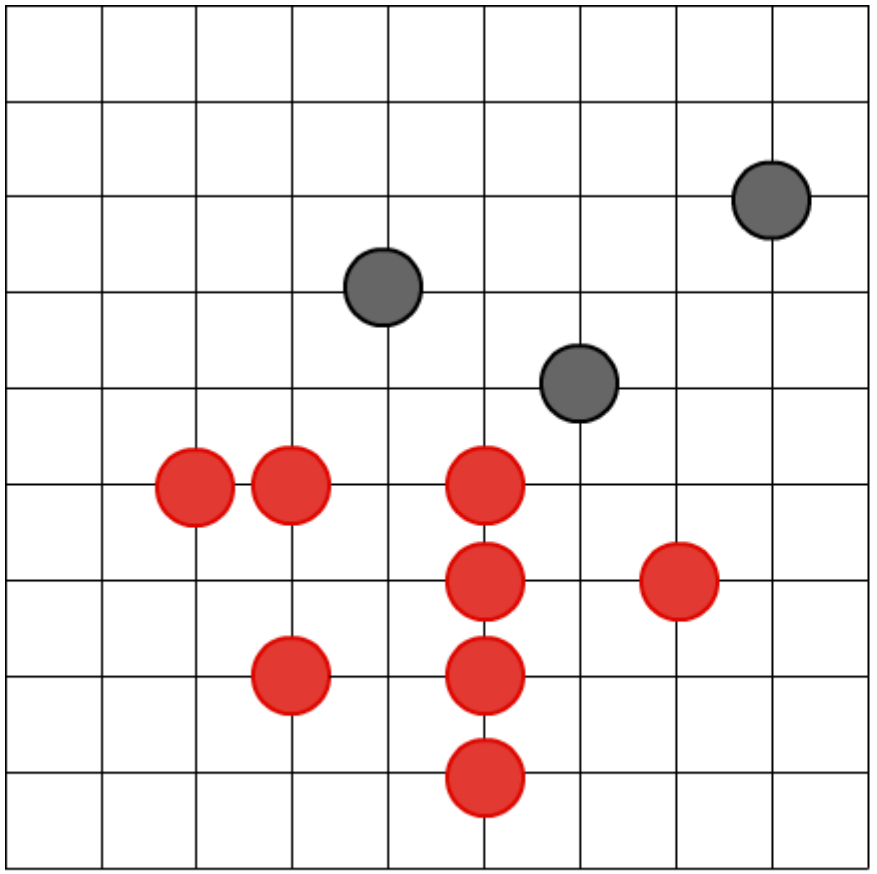


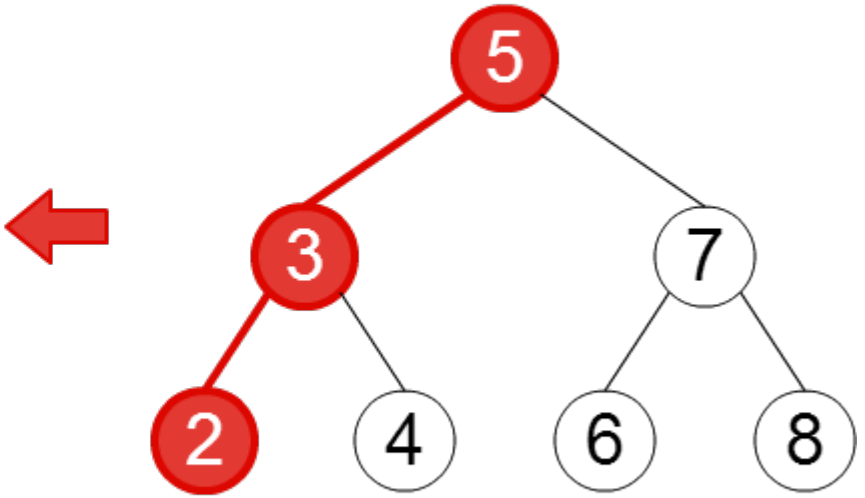
D-図解(入力例3)



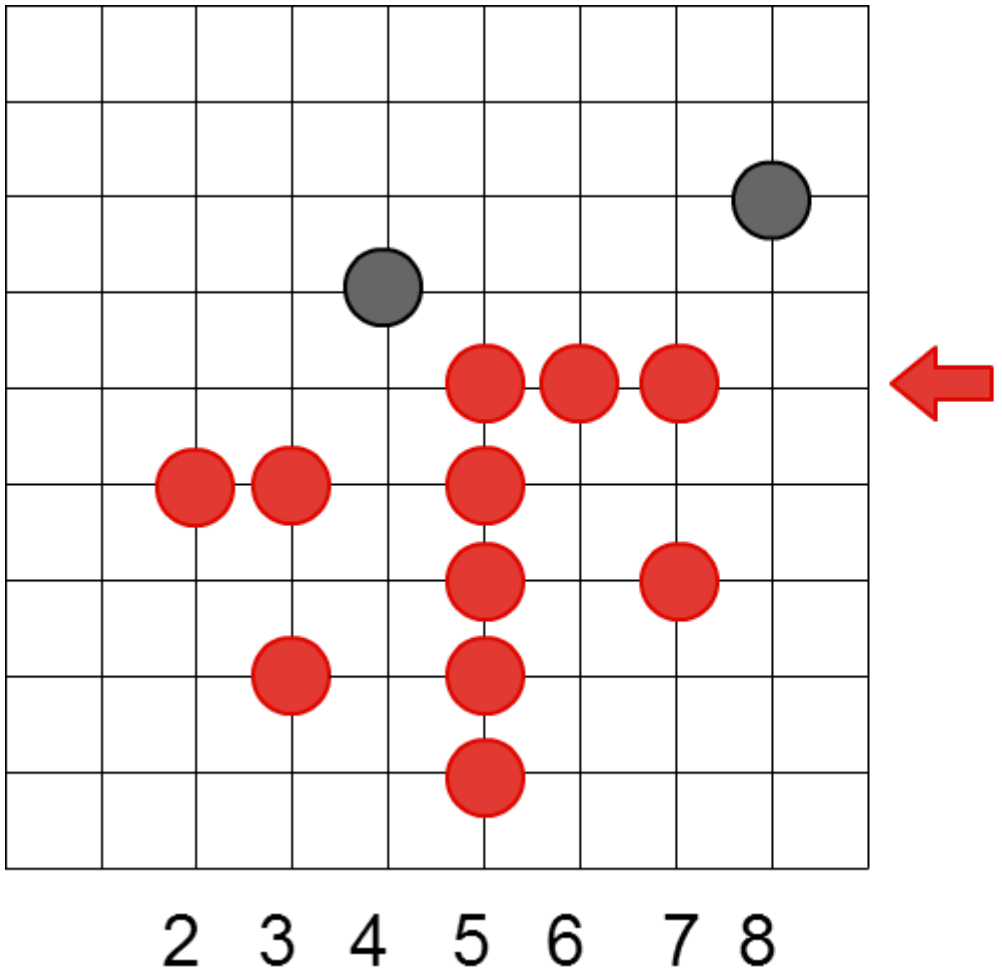


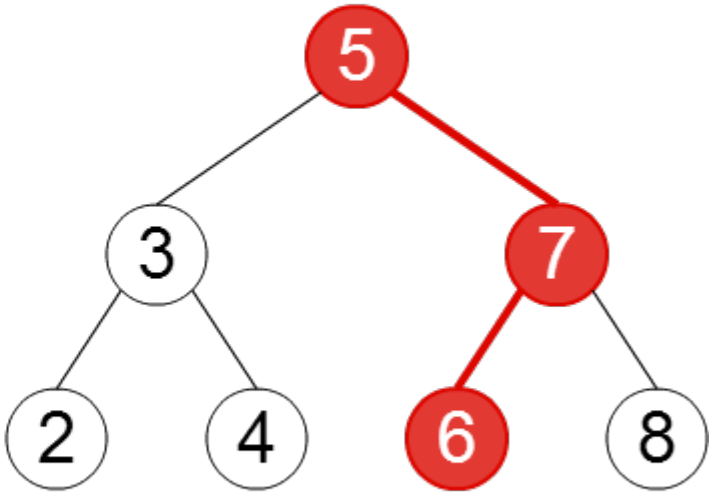
D-図解(入力例3)



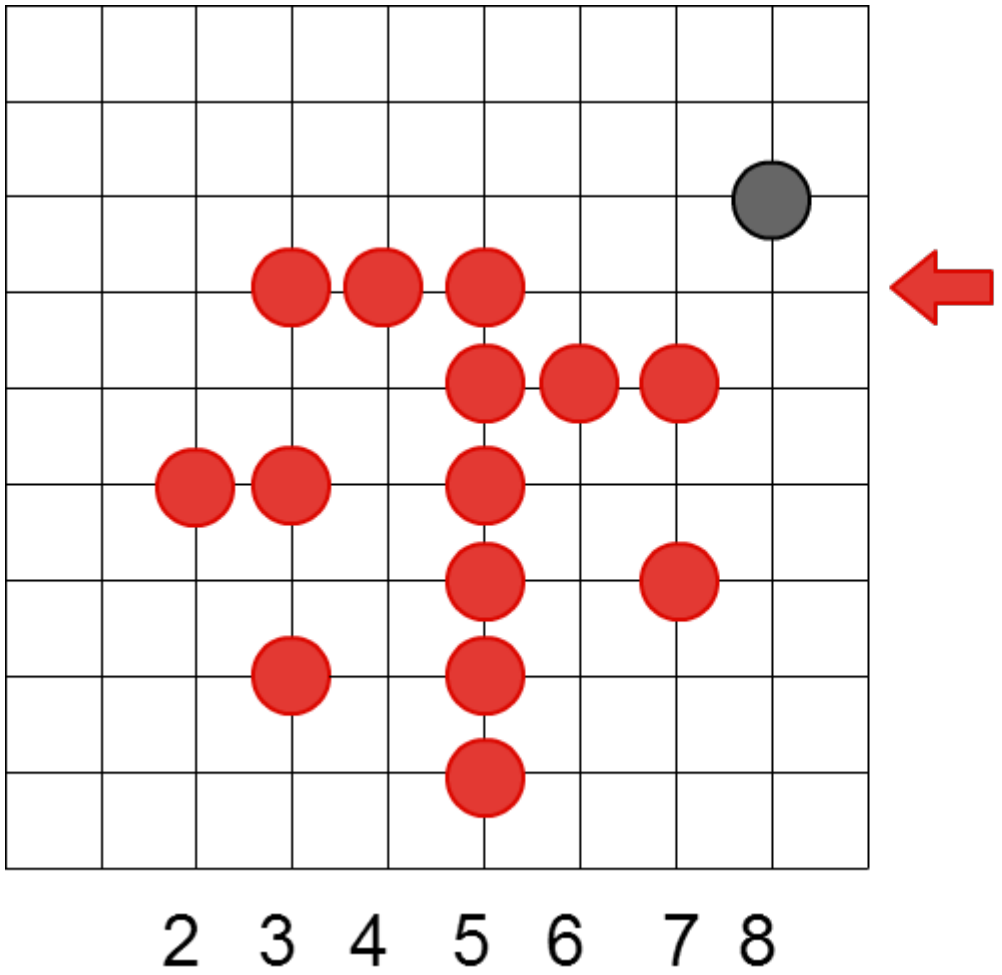


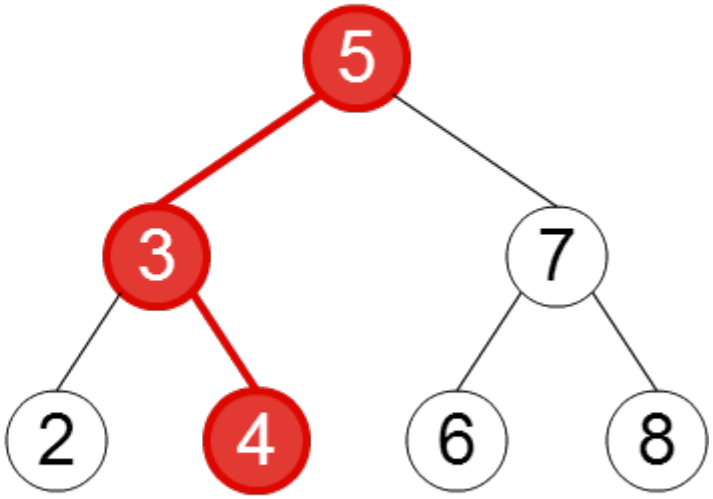
D-図解(入力例3)



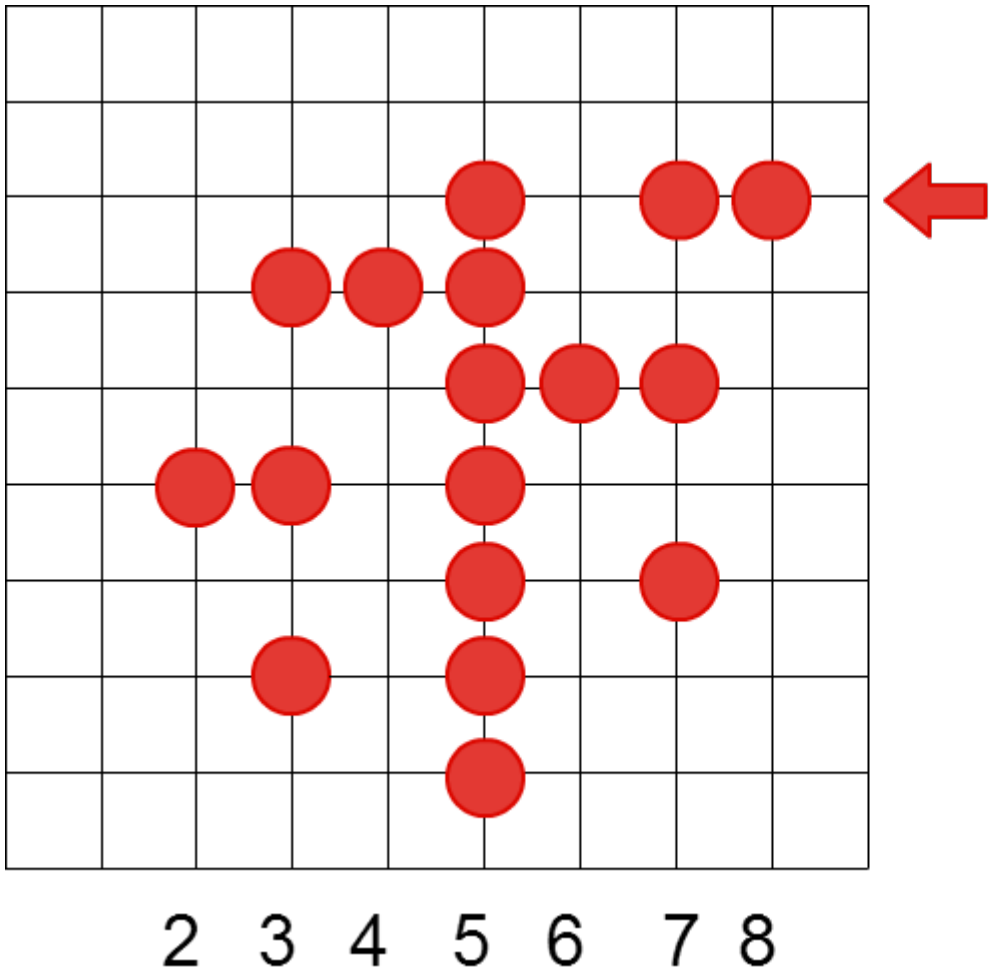


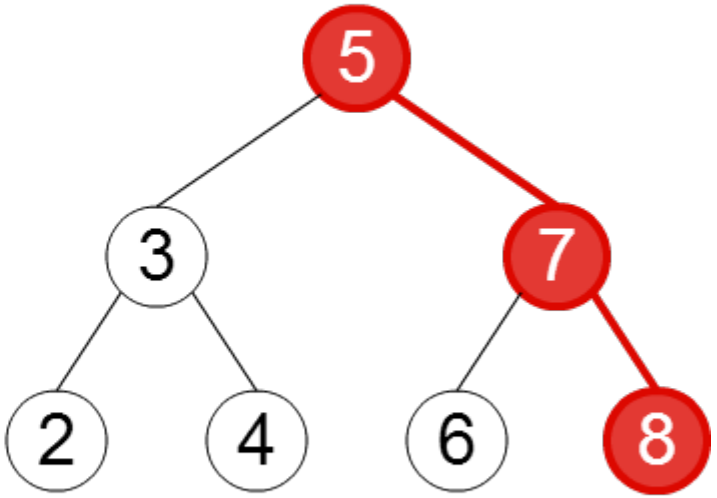
D-図解(入力例3)





D-図解(入力例3)





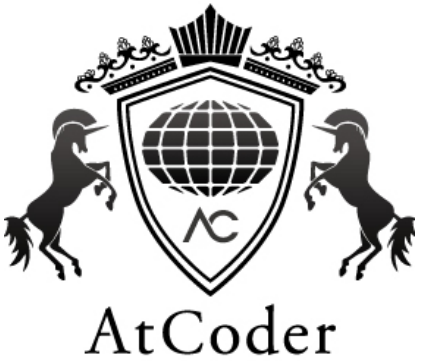
D-参考

- 先の例のように完全二分探索木を使えば探索時に触れる点の個数がたかだかlogN個なので個数制限にもひっかからない
  - AC
  - よく考えると分割統治の解と同値であることがわかる
- 実は使う二分探索木は現れるxの値だけでなく、1～1000のすべての値を持っていても大丈夫
  - 座標圧縮や再帰がなくなるので、実装が分割統治より楽になる

D-参考（発展）

- 完全二分探索木ではない二分探索木を使うことも可能
- 実は、「回転」と呼ばれる操作が許されるので平衡二分探索木でも可能
  - ただし定数倍が重く、個数制限に引っかかりWA
- なぜ可能かの証明は難しいので割愛
  - Geometry of binary search treesなどのキーワードで検索すると詳細な情報が出てくる

AtCoder Regular Contest 023
解説



AtCoder株式会社

A問題

経過日数

問題概要

- y年m月d日から2014年5月17日までの経過日数を計算せよ
- ただし以下の公式は使ってよい

西暦1年1月1日からy年m月d日までの経過日数は、1月2月の場合はそれぞれを前年の13月14月と扱った上で、以下の通り

$$365\mathrm{y} + \left\lfloor \frac{y}{4} \right\rfloor - \left\lfloor \frac{y}{100} \right\rfloor + \left\lfloor \frac{y}{400} \right\rfloor + \left\lfloor \frac{306(m+1)}{10} \right\rfloor + \mathrm{d} - 429$$

(ただし、$\lfloor x \rfloor$はxを超えない最小の整数

例えば $\lfloor 1.5 \rfloor = 1, \lfloor 2.0 \rfloor = 2$)

解法

- 適当な日付からある日付までの経過日数を計算できれば、
  それらの引き算で任意の2つの日付間の経過日数が計算できる
- したがって、それを計算する
- $\lfloor x \rfloor$はxが正なら、小数点切捨てと等価なことに注意
  (intにキャストするなり何なりで実現できる)

$$365\mathrm{y} + \left\lfloor \frac{y}{4} \right\rfloor - \left\lfloor \frac{y}{100} \right\rfloor + \left\lfloor \frac{y}{400} \right\rfloor + \left\lfloor \frac{306(m+1)}{10} \right\rfloor + \mathrm{d} - 429$$

(ただし、$\lfloor x \rfloor$はxを超えない最小の整数

例えば $\lfloor 1.5 \rfloor = 1, \lfloor 2.0 \rfloor = 2$)

B問題

謎の人物X

問題概要

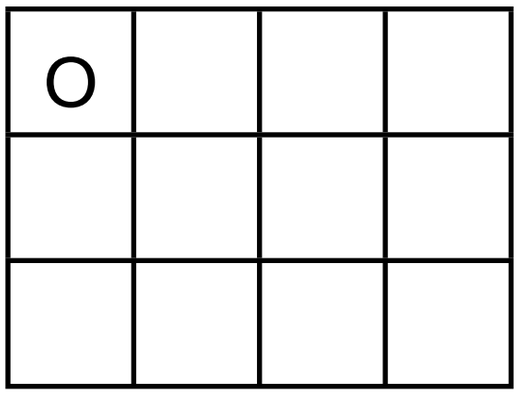
- R×Cのマス目の左上のマスから、ちょうどD回「隣のマスに移動」することで行けるマスに書いてある数字のうち、最も大きいものを求めよ。
- 部分点（60点）：R,C ≦ 100, D ≦ 200
- 満点：R,C ≦ 1000, D ≦ 2000

部分点（60点） R,C ≦ 100, D ≦ 200

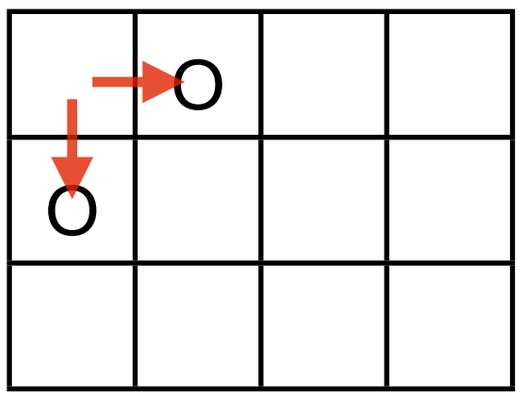
- 「i 回移動した時に行けるマス」をシミュレーションしてみる。
- 「i 回移動した時に行けるマス」は、「i-1 回移動した時に行けるマス」が隣にあるようなマス。

シミュレーション

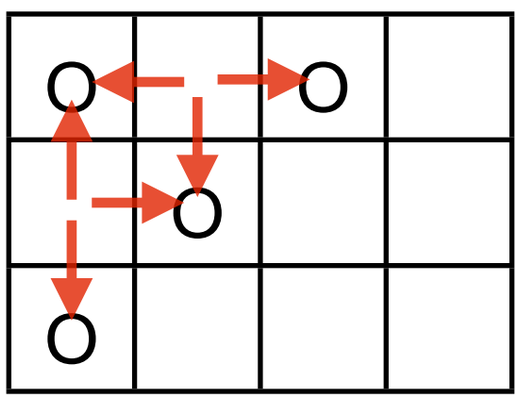
0回移動



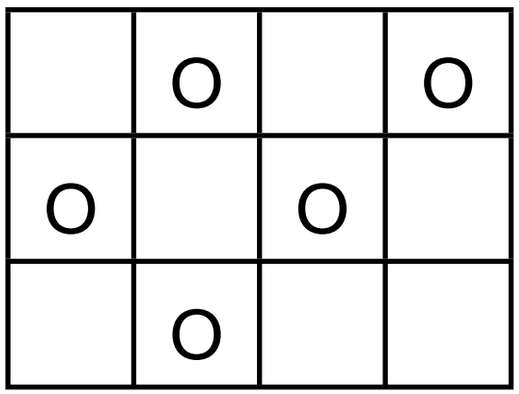
1回移動



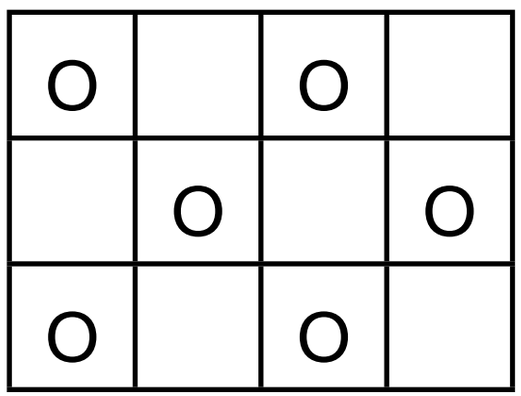
2回移動



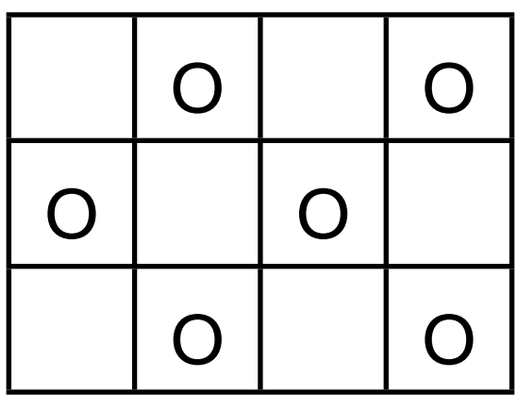
3回移動



4回移動



5回移動



部分点（60点） R,C ≦ 100, D ≦ 200

- あとは、「D回移動した時に行けるマス」のうち、書いてある数字が最大のものを求めれば良い。
- 計算量はO(マスの個数×D)くらいなので、この部分点では時間内に答えを求めることが出来ます。

満点 R,C ≦ 1000, D ≦ 2000

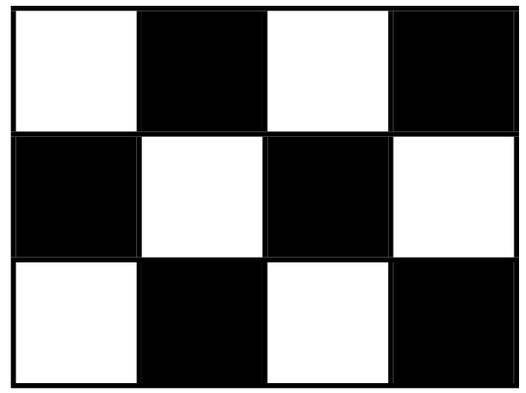
- 「D回移動した時に行けるマス」について考察してみる。
- マス(i,j)に行くことが出来る条件は、
  - ・i+j ≦ D (遠すぎると行けない)
  - ・i+jとDの偶奇が一致している
- の２つとなっています。
- ２つめの条件について少し説明します。

3回移動

	o		o
o		o	
	o		

i+jとDの偶奇について

- 図のようにマスを市松模様に塗ってみます。
- 一回の移動では、
  ・白いマスからは黒いマスにしか
  ・黒いマスからは白いマスにしか
  移動することが出来ません。



- この構造から、偶奇性が生み出されていて、先ほどのような条件が成り立ちます。

満点 R,C ≦ 1000, D ≦ 2000

- 各マスについて条件をチェックし、条件を満たすマスのうち、書かれている数字が最大であるものを求めれば良い。
- 計算量はO(マスの個数)くらいなので満点を取ることが出来ます。

C問題

タコヤ木

問題概要

- (広義)単調増加な数列の一部が与えられるので、元の数列としてありうるものは何通りか求めよ。
- 部分点（50点）：N ≦ 100, Ai ≦ 100
- 部分点（80点）：N ≦ 2000, Ai ≦ 2000
- 満点：N ≦ 2000, Ai ≦ 10^9

部分点（50点） N ≦ 100, Ai ≦ 100

- DP[i][j] = i番目の数字がjである場合の数
- というDPをします。
- 直前の数字が今の数字以下であればいいので、漸化式は以下のようになります。
- DP[i+1][j] = sum(DP[i][0]~DP[i][j])
- Aiが-1でないときは、DP[i][Ai]だけを残して、それ以外を全て0にします。

部分点（50点） N ≦ 100, Ai ≦ 100

- 状態数：O(N max(Ai))
- 遷移数：O(max(Ai))
- 計算量：O(N max(Ai)^2)
- となり、この部分点を取ることが出来ます。

部分点（80点） N ≦ 2000, Ai ≦ 2000

- ２通りの方針があります。
- まずは先ほどのDPを高速化する方針から。。

DPの高速化

- 先ほどのDPの遷移の式は、
- DP[i+1][j] = sum(DP[i][0]~DP[i][j])
- でした。
- 区間の和を高速に求められれば良さそうです。

部分点（80点） N ≦ 2000, Ai ≦ 2000

- 累積和を使いましょう。
- sum[i] = 0~iまでの和
- というものを全てのiに関してあらかじめ求めておくことで、l~rの区間の和を「sum[r]-sum[l-1]」というようにしてO(1)で求めることが出来ます。
- 今回の問題では左端が常に0なので、sum[i]がそのまま求めたいものになっています。

部分点（80点） N ≦ 2000, Ai ≦ 2000

- 状態数：O(N max(Ai))
- 遷移数：O(1)
- 計算量：O(N max(Ai))
- となり、この部分点を取ることが出来ます。

部分点（80点） N ≦ 2000, Ai ≦ 2000

- その２の解法です。
- 少し数学的な考察をします。
- -1が連続している部分ごとに分けると、それぞれを独立に計算しても良くなります。
- 例えば「2 -1 -1 9 -1 9」なら「2 -1 -1 9」と「9 -1 9」に分けて考えます。

部分点（80点） N ≦ 2000, Ai ≦ 2000

- X,(-1がY個並ぶ),Z
- となっている部分を計算する方法を考えます。
- 各項間での増加量に注目します。
- 項の間はY+1箇所あり、それらに合計Z-Xの非負の整数を割り振っていると言い換えることが出来ます。
- もう少し簡潔に言い換えると・・・

部分点（80点） N ≦ 2000, Ai ≦ 2000

- 長さLの非負整数からなる数列であって、
  合計がSとなるものの個数
- を求めることが出来れば良いです。
- 「1 -1 3」を例にして言い換えると、
- 「長さ2で合計が2の数列」の個数を求める問題
- （答えは「0 2」「1 1」「2 0」の３通り）

部分点（80点） N ≦ 2000, Ai ≦ 2000

- 長さLの非負整数からなる数列であって、
  合計がSとなるものの個数
- →数学とかでよくある、重複組み合わせの問題
- 「S個の’o’とL-1個の’|’を並べる方法の個数」と同じ
- ${}_{L}H_{S} = {}_{S+L-1}C_{S}$

部分点（80点） N ≦ 2000, Ai ≦ 2000

- ここまでくれば、あとは組み合わせを計算できれば良い。
- パスカルの三角形を作ろう。
- i段目のj番目の数字が ${}_iC_j$ になっています。

1
11
121
1331
14641
パスカルの三角形

部分点（80点） N ≦ 2000, Ai ≦ 2000

- まとめると、
- X -1 -1 ... -1 Y という部分に分割する
- それぞれの部分に対し、重複組み合わせを求める
- 重複組み合わせをパスカルの三角形を用いて求める
- パスカルの三角形を構成するところに最も時間がかかり、O(max(Ai)^2)くらいなので、この部分点を得ることが出来ます。

満点 N ≦ 2000, Ai ≦ 10^9

- 組み合わせを高速に求めたい。
- 組み合わせは、以下のように求められます。
- nCr = (n-r+1~nの積)/(1~rの積)
- これをどのように計算すれば良いでしょうか。

満点 N ≦ 2000, Ai ≦ 10^9

- nCr = (n-r+1~nの積)/(1~rの積)
- (n-r+1~nの積)と(1~rの積)は素直にかけ算をすれば良いだけです。
- しかし、割り算はそうは行きません。
- X/Y (mod p) (pは素数) の求め方が問題です。

X/Y (mod p) (pは素数) の求め方

- mod pのpが素数の場合は、**逆元**が存在します。
- 「Yの逆元」とは「Yをかけると1になる数」のことで、「1/Y」や「Y^-1」などと表します。
- フェルマーの定理「X^(p-1) ≡ 1 (mod p)」を用いると、Yの逆元は「Y^p-2」として計算することが出来ます。（(Y^(p-2)) * Y ≡ 1 (mod p)なので）
- つまり、X/Y ≡ X*(Y^(p-2)) と計算できます。

X/Y (mod p) (pは素数) の求め方

- あとは、Y^(p-2)を高速に計算できれば解決です。
- 冪乗法を用います。(繰り返し二乗法とも)
- A^B = A^(B/2)*A^(B%2) という漸化式を使ってべき乗を計算する手法です。
- 肩に乗っている数字が半分ずつに減っていくので、計算量はO(log p)となります。
- 再帰関数として実装すると良いでしょう。

満点 N ≦ 2000, Ai ≦ 10^9

- 80点の２つめの部分点解法を、mod pでの割り算をO(log p)くらいで計算するテクニックを用いて高速化することによって、満点を得られるようになりました！

D問題
GCD区間

問題概要

- 数列$A=\{a_1,a_2,\cdots,a_n\}$がある　($n \leqq 10$万)
- クエリが与えられる。クエリの内容は、
  「xが与えられるから、$GCD(A_s,A_{s+1},\cdots,A_t)=x$となる区間[s,t]の数を求める」
- クエリはたくさん与えられる(≦10万回)

考察

- 区間を決めることを考える
  tを固定してsをt→1と動かすとき、[s,t]のGCDは単調減少
- 単調減少の仕方は、因数が1つずつ除かれる感じ
- 10^9以下で、因数が一番多いのは2^29で29個
- tを固定してsをt→1と動かす方法において、現れるGCDの数はlog(At)以下

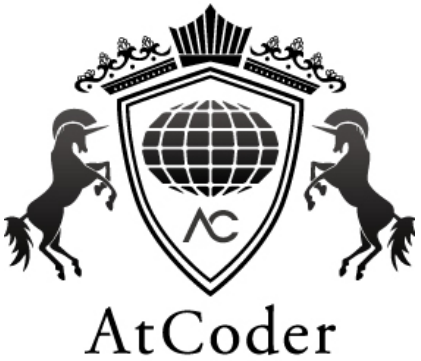
解法

- クエリを考えず、全てのGCDとそれを含む区間数を予め計算する
- 左から順番にtを決めていく。今まで現れたGCDの値とその出現回数を保持した配列をMとすると、考察からMの大きさは常に30個(GCD=1を含める)以下
- Mを利用して[?,t]をまとめて計算して答え用の配列に格納しとく
- 後処理として、Mの各要素に対してtとのGCDを取り、配列M’を計算し、M=M’として処理を続けていく
- Mと答えの保持に平衡二分木の連想配列を用いると、
  O(N log max(Ai) log N + |X| log N)

他の解法

- 区間のGCDを予め高速に計算できるようにしておく（*）
- 今度はsを固定してtを伸ばすことを考える
- GCDは単調減少かつ、変化はO(logAs)回
- 変化点見つけるのまで二分探索するとうれしくない
  - １個の変化点を見つけるためにO(log N)回反復
  - O(N log max(Ai) log N×{(*)の方法の計算量})　←とても重い
- (*)の部分にセグメントツリーを用いるとおそらくTLE
- 二分探索ではなく、平方分割を用いて、「進めてみて変化がないなら√Nずつtを進めていく」方法も間に合うはず
- その他、スパーステーブルを用いて平方分割と同様できるだけ大きな2^k(1,2,4,…)ずつ進めていく方法もある

AtCoder Regular Contest 022
解説



AtCoder株式会社

A問題
スーパーICT高校生

A-問題概要

- 文字列Sが与えられる
- Sからいくつか文字を省いて”ICT”を作れるか判定せよ
- 大文字小文字は区別しない

$$1 \leqq |S| \leqq 100$$

A-想定解

- 残す'I'はSの中で一番最初の'I'でよい。
- 残す'C'はその'I'以降で一番最初の'C'でよい。
- 残す'T'はその'C'以降で一番最初の'T'でよい。
- 順番に貪欲的に文字を取っていく。
- O(|S|)
  - 想定解

A-想定解2

- 今回は探索する'ICT'が3文字なので、さまざまなその場しのぎのアルゴリズムが考えられる。
- 3文字を全通り試す
  - O(|S|^3)
- 'C'を決めてその左右に'I'や'T'があるか判定
  - O(|S|^2)
- 最初の'I'と最後の'T'の間に'C'があるかどうか判定
  - O(|S|)

A-大文字小文字

- この問題では大文字小文字を区別しない
- 知るかぎり、どのプログラミング言語でも大文字と小文字は別物と判断されるので、何とかしなければならない
  - 自前の比較関数を作る
  - あらかじめ全て大文字か小文字に揃える

B問題

細長いお菓子

問題概要

- お菓子がうんぬんと言っていますが要するに、
- 長さNの数列Aが与えられるので、同じ数字を２つ以上含まない最長の区間の長さを求めよ。
- 部分点（50点）： N ≦ 100、Ai ≦ 100
- 部分点（99点）： N ≦ 1000、Ai ≦ 1000
- 満点：N ≦ 10^5、Ai ≦ 10^5

部分点（50点）N ≦ 100

- 区間の左端と右端を全て試し、その区間の中に同じ数字が２つ以上含まるかどうかを調べる。

→「同じ数字が２つ以上含まれるかどうか」を高速に判定したい。

判定のやり方

- 方法１：頻度を計算しておく
- 「それぞれの数字がいくつ含まれるか」を、配列を使って求めておき、２つ以上含まれる数字がないかを調べる。

頻度の計算の例

```
for (int i = L; i <= R; i++) freq[A[i]]++;
```

判定のやり方

- 方法２：集合を扱うデータ構造を使う
- 例えばsetやDictionaryなど。
  - 集合に数字をどんどん追加していき、すでに追加されている数字があれば、ダメ。

部分点（99点） N ≦ 1000

- 区間の**左端だけ**を全部試し、それぞれに対してどれだけ区間をのばすことが出来るかを調べる。
- ひとつずつ数字を見ていき、その数字がすでに区間の中に入っていたらその時点でストップ、とする。
- 判定の方法は50点の部分点の方法と同様の方法を使えば良い。（方法１でも方法２でも良い）

満点 N ≦ 10^5

- 尺取法を使う。

尺取法の例

```
l = 0;
for (r = 0; r < N; r++) {
  while ([l,r]に同じ数字が２つ以上含まれている) l++;
  answer = max(answer, r - l + 1);
}
```

- 判定の部分に”方法２”を使う場合は、データ構造が削除クエリに対応している必要があります。

C問題

ロミオとジュリエット

問題概要

- 要するに、
- 頂点数Nの木が与えられるので、距離が最長となる頂点のペアを１つ見つけよ。
- 部分点(40点)：N ≦ 1000
- 満点：N ≦ 10^5

部分点（40点） N ≦ 1000

- 片方の頂点を全て試し、その頂点から最も遠い頂点を探す。
- 最も遠い頂点を探すためにはBFSを使えば良い。木なのでDFSを使ってもOK。
- 木の上ではDFSをすることが多いので、DFSで書けるようになっておくのがオススメです。

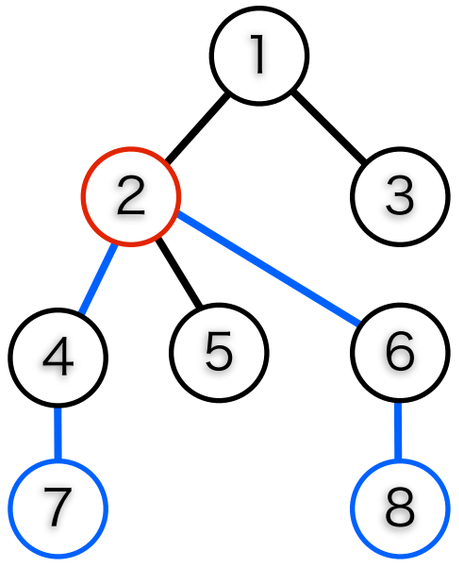
満点 N ≦ 10^5

- ２頂点間の最大の距離＝**直径**と言います。
- すなわちこの問題は、木の直径を求める問題です。
- 木の直径を求めるアルゴリズムを２通りほど紹介します。

木の直径の求め方その1

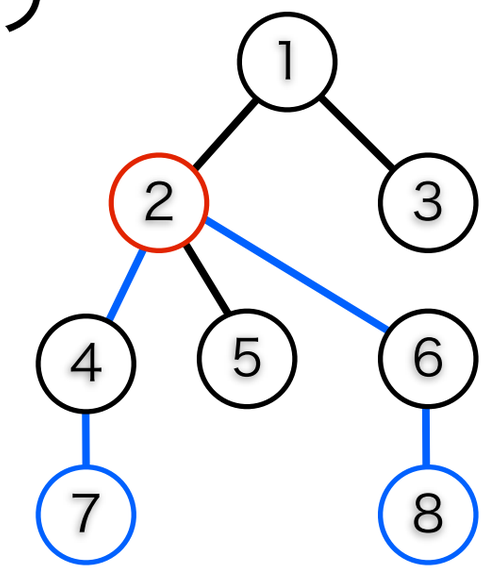
- 簡単な木DPをする。
- とりあえず根を適当に決めて根付き木にします。
- 任意の２頂点間の移動は「まず根の方向にいくつか辿り、根から離れる方向にいくつか辿る」となっています。
- LCA(Lowest Common Ancestor)に注目します。

  （移動で通る頂点のうち最も根に近い頂点がLCAです。）



木の直径の求め方その1

- ある頂点(Vとする)をLCAとする頂点のペアのうち、最も距離が離れているものは、
- 「あるVの子の子孫のうち最もVから遠い頂点」（Vの子の数だけある）と「V」のうち、Vから遠い順に２つ選んだ時の頂点のペアになります。
- これをボトムアップに（葉から順に）計算して行けば良い。

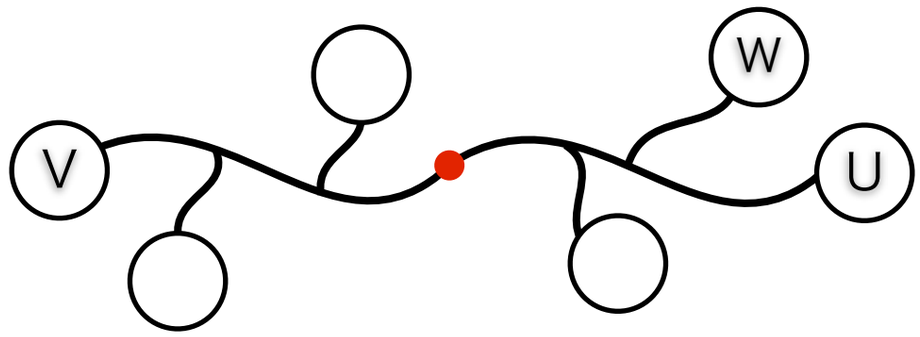


木の直径の求め方その２

- こちらは、方法としてはとても単純です。
- まず、頂点を適当に選んでWとします。
- Wから最も遠い頂点を探してVとします。
- Vから最も遠い頂点を探してUとします。
- すると、VとUが答えになっています。

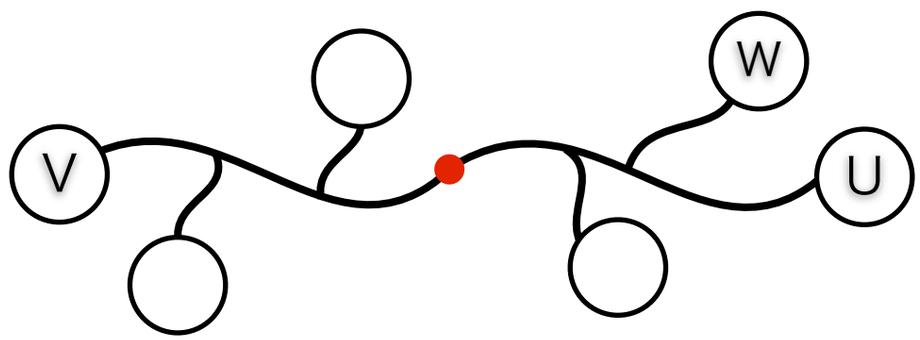
木の直径の求め方その２(仕組みについて)

- 木には中心というものがあります。
- 木の中心とは、直径の中心のことです。
  → 中心は辺上の点になることもあります。
- 全ての直径は木の中心を通ります。
  → 中心から各頂点への距離は直径/2以下で
  ・・・と考えたりすると証明できます。



木の直径の求め方その２(仕組みについて)

- Vは中心をまたいでWと反対側にある最も遠い頂点であるため、直径の端点となる。
- Uは中心をまたいでVと反対側にある最も遠い頂点であるため、直径のもう片方の端点となる。
- というからくりになっています。



まとめ

- １つ目の方法は汎用性が高いテクニックなので、書けるようになっておいた方が良いでしょう。
- ２つ目の方法は、直径を求める際には楽な方法なので、覚えておいて損は無いでしょう。

D問題
スプリンクラー

D-問題概要

- 直交座標上にN個の格子点(x, y)が与えられる
- 各格子点に対して、それを中心として原点を通る円を考える
- いずれかの円に含まれる格子点の個数を求めよ
- N ≦ 10^5
- -10^5 ≦ x, y ≦ 10^5

D-部分点1

- N ≦ 100
- -100 ≦ x, y ≦ 100
- 頂点と円の組み合わせを全て調べても間に合う
- O(N |x| |y|)
  - 10点が得られる

D-部分点2

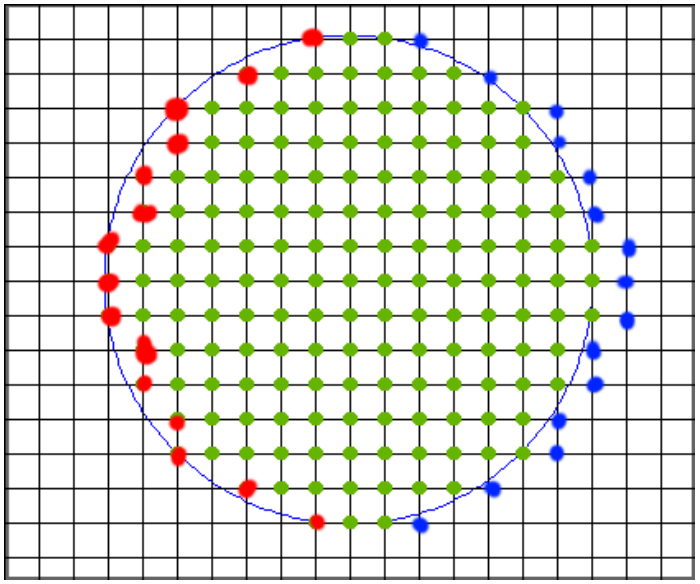
- N ≦ 1,000
- -1,000 ≦ x, y ≦ 1,000
- 全頂点の走査なら可能そう
- いろいろな図形を重ねた時に、何枚重なっているか数える都合のいいアルゴリズムがあった気がする・・・

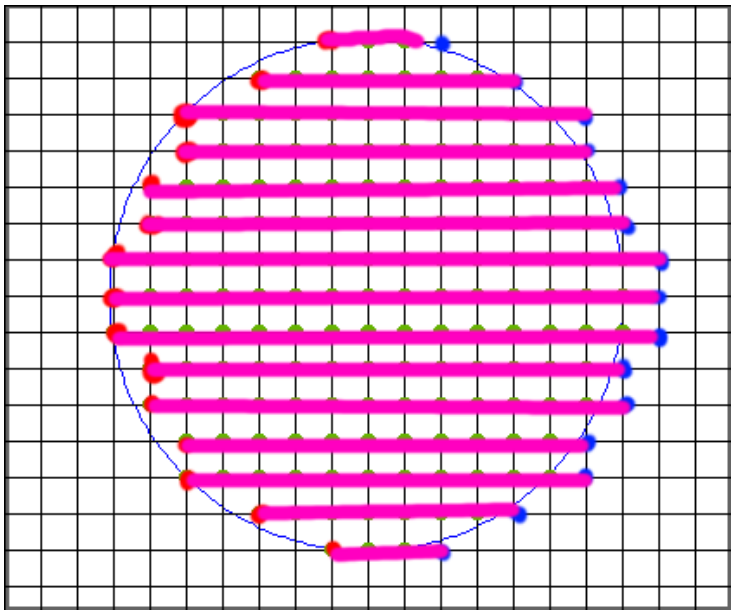
D-部分点2

累積和

D-部分点2

- 長方形ではないのでいつもの二次元累積和は使えない
  - 一次元に分けたらできる
- 円周の左側を+1 右側を-1する





D-部分点2

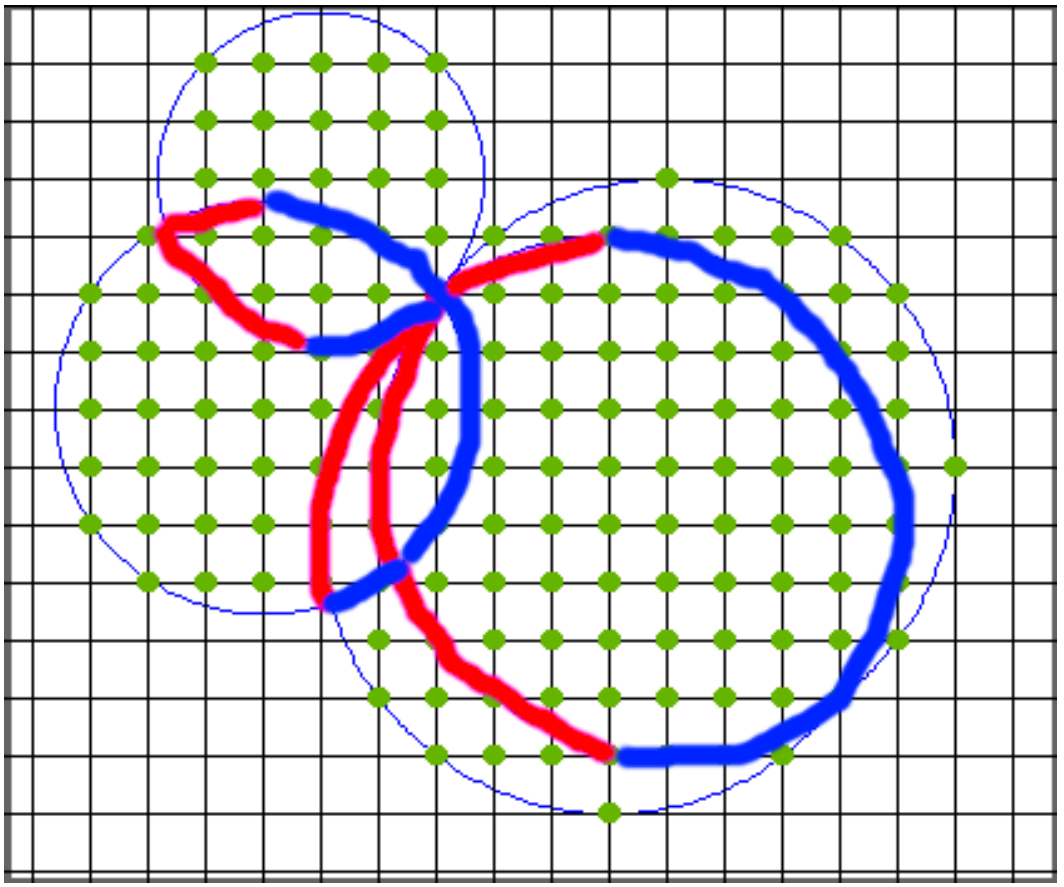
- +1や-1する点は円1つあたり、たかだかO(半径)個くらい
  - 今回なら 1000 * √2
- 総計算量はO(N max(|x|, |y|))
  - 計30点が得られる

D-考察

- 累積和をすると、そこにいくつの円が重なっているかという余計な情報も求めることになる
  - 無駄
- 1個以上重なっているかどうかさえわかればいい
  - 無駄な+1, -1をしないように削る

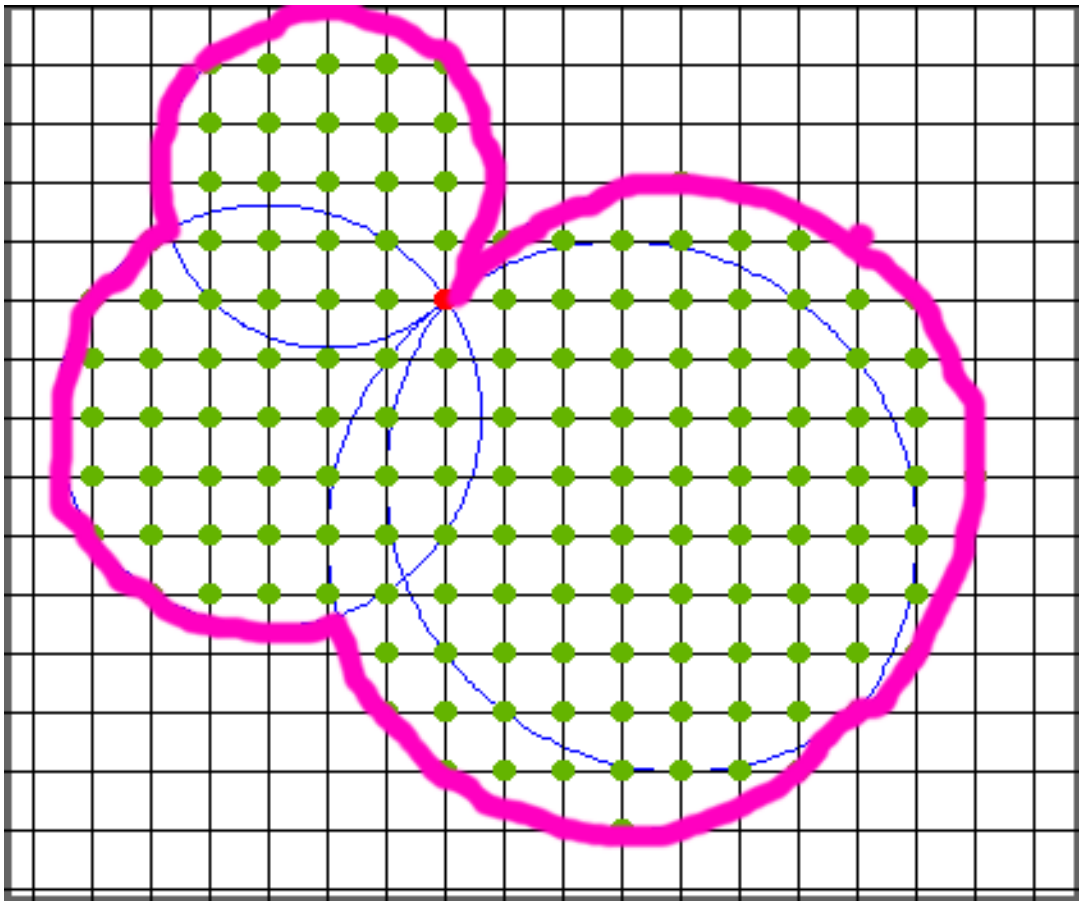
D-考察

- 無駄な+1, -1の例



D-考察

- どうやら完全に外側に接している部分付近だけの+1, -1で充分
- 外周さえ求まればなんとかなりそう
- どうすれば求まるのか・・・



D-部分点2

凸包

D-考察

- 点A,点Bと三角形OAB内の点C
- Cを中心とする円はAもしくはBを中心とする円に完全に覆われる
- Cは考えなくて良い

D-考察 部分点3

- どの三角形の内側にも入らない点だけを考えれば良い
  - つまり凸包上にある頂点だけを考える
- 頂点は全て格子点にあるので凸包上にある点はたかだか√|x|個くらい
- 凸包で頂点を減らしてimos法をすれば良い
  - O(x log x + x√|x|) 計60点が得られる

D-さらに処理を少なく

- 凸包上の点を中心とする円でも、その円周が全て外側に触れている(外周)わけではない
  - 外周のところだけで計算してよい
- 円どうしの交点、つまり円弧の両端が求まれば良い
- 交差する円は、凸包の多角形における隣り合う頂点に対応する円だけ
  - 幾何の計算をいくらかすれば交点はもとまる

D-外周だけを考える

- 各円弧の両端が求まれば、その間に含まれる範囲だけ考えれば良い。
  - ある点がある円弧に含まれるかどうかは凸包と同様に外積を使えば良い
- 外周だけ考えるのは何とかなりそうだけれどもはたしてそれで満点の制約において間に合うのか？

D-計算量

- 累積和において別々に考えるときに、y=k(kは整数)となる直線で輪切りにして考えるとする。
- そのとき、各直線上には左端と右端がいくつか乗る。それを計算すれば各直線上の条件に合う花の数が求まる。

D-計算量

- 右端や左端は外周がy=k(kは整数)と交わる点の数だけある。
- 外周を一周りするときにy=k(kは整数)と何回交わるかを求めれば、計算量が求まる。

D-計算量

- 各円弧の両端と原点を結んだ三角形の原点の部分の内角は、その円弧の円周角に当たる。
  - 全外周の円周角の総和はたかだか360°
  - よって中心角の総和はたかだか720°
- 円の半径は|x|*√2以下なので、外周の上限は

  |x|*4√2 * (円周率)
- 今回の問題ではおおよそ2,000,000くらい

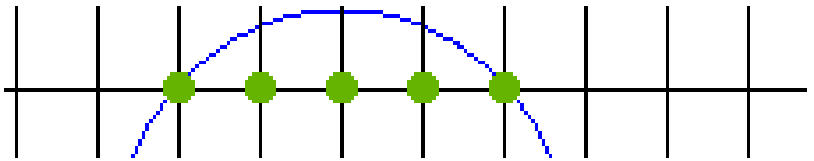
D-計算量

- 外周をたどって一回りする時、いろいろなy=kと交わる。
- いまy=aと交わったとする。その次に交わるのはy=a+1, y=a-1, y=aのいずれかである

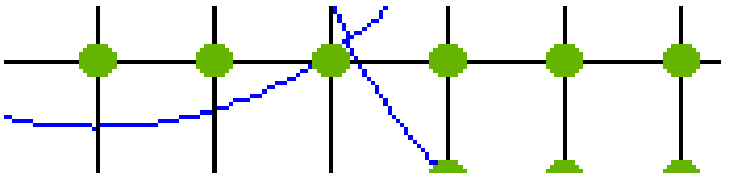
D-計算量

- 次にy=a+1,y=a-1と交わることは、外周を1以上進んでいるので2,000,000回しか起こらない
- 次にy=aと交わることは、下図のような場合のみ。共にO(N)回

接線がx軸と平行になる付近



違う円弧と変わる付近



D-計算量

- よって、外周とy=k(kは整数)との交点はたかだか2,000,000個くらいしかないので全列挙しても十分間に合う
  - 想定解

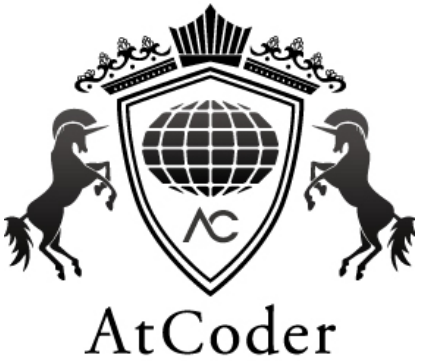
D-想定解まとめ

- スプリンクラーと世界一の花で凸包を取る。
- 凸包上の点とそれに対応する円を順番に見ていき、隣の円との交点を求め、外周となる範囲を得る
- 外周上を順番に走査してy=k(kは整数)との交点を全て列挙する
- 左端と右端が全て求まるので適当に各yについて引き算をして、総和を取ると答えになる

D-計算量まとめ

- 凸包でO(N log N)
  - バケツソートによる基数ソートをすればO(N)
  - 凸包上の点の数をMとする M = O(√|x|)
- 円どうしの交点を求めるのにO(M)
- y=k(kは整数)との交点を全列挙するのにO(|x| *4√2 * 円周率)
- おおよそ10^6オーダー

AtCoder Regular Contest 021
解説



AtCoder株式会社

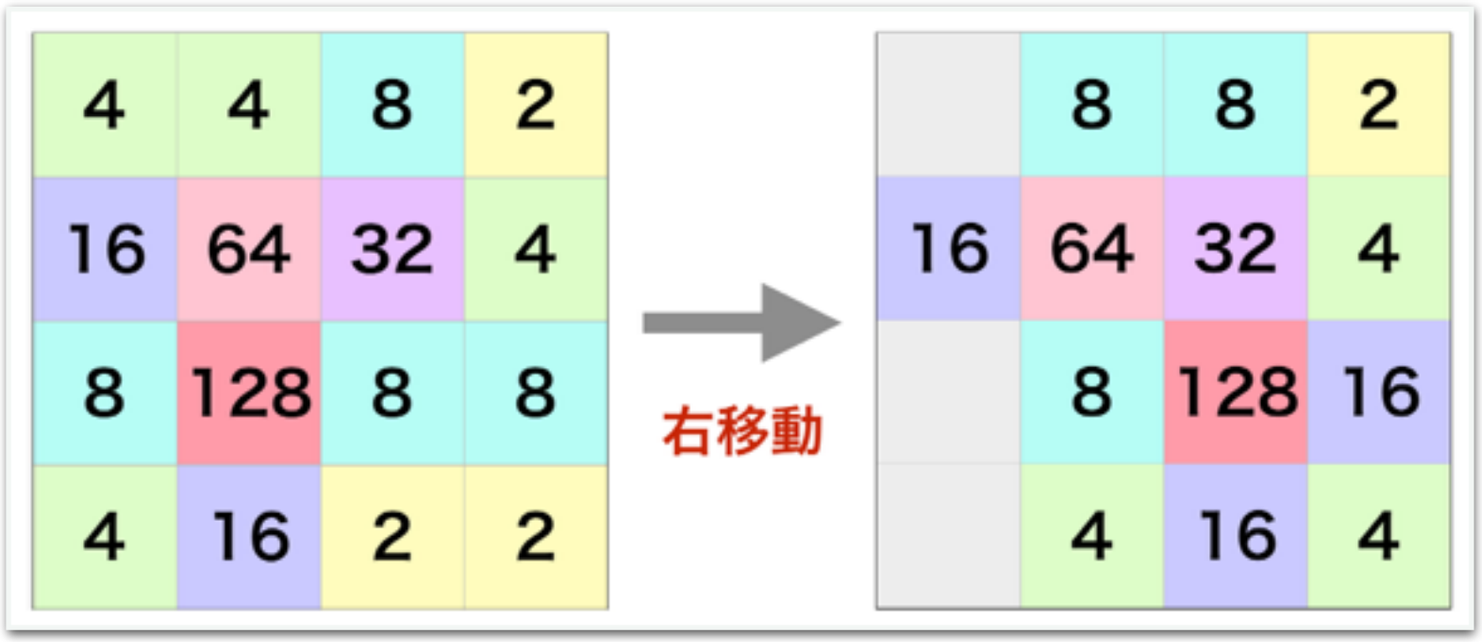
A問題

DEAD END

A - 問題概要

問題

タイルを滑らせることで同じ数をぶつけて
大きい数にしていくゲームの盤面が与えられる。
ゲームオーバーの状態かどうかを判定せよ。



A - 重要なポイント

与えられる盤面では**空いているマスはない**！

$A_{r,c}$ は 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048 のいずれかである。

↓

同じ数をぶつけるような操作が必要

しかも、**隣り合う数**をぶつけるしかない

A - 解き方

整理すると……

上下左右に隣り合う同じ数が ある　→　**CONTINUE**

ない　→　**GAMEOVER**

実装

・**2 重ループ**を回す（添字に注意）

・if 文を書きまくって全部調べる（列挙漏れに注意）

(きちんとループで書けるようになっておいた方がいい)

ARC #021 B 問題解説

解説:城下 慎也 (@phidnight)

問題概要

- N 個の非負整数 $B_1, B_2, \ldots, B_N$ が与えられる。
- $B_i = A_i \hat{} A_{i+1}$(ただし $A_{N+1} = A_1$) である。

(以降、$\hat{}$ を排他的論理和(xor) の記号として表す。)

- $B_1, B_2, \ldots, B_N$ として考えられるものが存在するなら辞書順最小を求めよ。ないならないことを指摘せよ。
- $2 \leqq N \leqq 100{,}000$

排他的論理和の性質(例)

- A, B, C を非負整数とすると
- 結合則$(A \hat{} B) \hat{} C = A \hat{} (B \hat{} C)$ が成立する。
- 交換則$A \hat{} B = B \hat{} A$ が成立する。
- $A \hat{} 0 = A$ (零元)
- $A \hat{} A = 0$ (それ自身と xor すると零元になる。)
- これらの性質を利用してこの問題を考える。

考察

- A_1を固定して考えてみる。
- B_1 =A_1^ A_2よりA_2=A_1^ B_1

(∵ A_2=0^A_2=(A_1^A_1)^A_2=A_1^(A_1^A_2)=A_1^ B_1)

- 同様に、B_2 =A_2^ A_3よりA_3=(A_1^B_1)^B_2

(∵ A_3=A_2^B_2=前述のA_2を代入)

- 一般に A_i=A_1^B_1^B_2^…^B_{i-1} ($i \geqq 2$)が成立することを示す。

考察

- A_i=A_1^B_1^B_2^…^B_{i-1} ($i \geqq 2$)が成立する。

(∵A_i=(A_1^A_1)^(A_2^A_2)^…^(A_{i-1}^A_{i-1})^A_i

=A_1^(A_1^A_2)^ (A_2^A_3) ^…^(A_{i-1}^A_i)

=A_1^B_1^B_2^…^B_{i-1})

- 以上のことより、A_1が定まれば、A_2からA_Nまでのすべての値がB_1からB_{N-1}までの値を用いて、一意に定まることがわかる。

解の存在判定について

- 使用されなかったB_Nについては、先ほどの計算で出てきた値と$B_N = A_N \hat{} A_1$が矛盾する場合がある。
- $A_N \hat{} A_1 = B_1 \hat{} B_2 \hat{} \ldots \hat{} B_{N-1}$となるので$B_N$は$A_1$に依存せずこの値に一致するはずである。
- 矛盾したならば解が存在しない。
- 矛盾しないなら、A_1を任意に指定した後、解が一意に定まる。

辞書順最小

- 解が存在した場合に、辞書順最小なものを求める必要がある。
- 先頭の値 (A_1) ができるだけ小さいほうが良い。
- 今回の場合では、A_1=0のケースがただ 1 通りだけ存在するので、その1 通りが辞書順最小の値となる。(A_2以降を比較する必要はない。)
- これを計算して答えが得られる。

まとめ

- B_N=B_1 ^ B_2 ^…^ B_{N-1}が成立しないなら解なし、成立するならA_1= 0と置いて順に解を計算する。
- 排他的論理和の性質を利用した問題は様々なものがあるので、排他的論理和の問題には慣れておこう!

ARC #021 C 問題解説

解説:城下 慎也 (@phidnight)

問題概要

- 高橋君は建物を K 回増築したい。
- 高橋君は N 軒の建物を所有している。
- 増築には、増築する建物について、現在の価格だけ費用がかかり、増築で一定値値上がりする。
- 合計費用として考えられるものの最小値はいくらか。
- $1 \leqq K \leqq 100{,}000{,}000$
- $1 \leqq N \leqq 100{,}000$

部分点解法 その1

- 毎回どの建物を増築するかをN 通りから選ぶ解法だとO(N^K) 通りになってしまう。
- 考えてみると、異なる建物同士の増築の順番を考慮しなくてもよいことが分かる。
- つまり、「建物1 を増築しまくる」→「建物2 を増築しまくる」→… という順番を付けてもOK。
- すると、dp[i][j]=(i 番目の建物までの増築が完了したときに、今まで j 回増築している場合の最小費用) とするDP を組めばO(K^2N) で計算することができる。これで 30 点が得られる。

部分点解法 その2,3

- 実は、毎回「現在の費用が最小なものを増築し続ける」という貪欲法で正解を得ることができる。
- 以降の数枚でそのことを示す。
- この貪欲法を毎回すべて見る方針で実装すれば O(KN) で解ける。計40点が得られる。
- priority_queue を使用すればO($K \log N$) となり、通算55点が得られる。

貪欲法が適用可能なことの証明

- 建物の増築回数と価格を照らしあわせた表は例えば次のようになる。

ID	A	D	0 回増築	1 回増築	2 回増築	3 回増築
1	50	40	50	90	130	170
2	40	80	40	120	200	240
3	70	50	70	120	170	220
4	60	60	60	120	180	240

貪欲法が適用可能なことの証明

- 建物の増築回数と価格を照らしあわせた表は例えば次のようになる。

ID	A	D	0 回増築	1 回増築	2 回増築	3 回増築
1	50	40	50	90	130	170
2	40	80	40	120	200	240
3	70	50	70	120	170	220
4	60	60	60	120	180	240

- ここから K 個良いものを (途中が飛んでも良い) 選ぶアルゴリズムは貪欲法で実装できる。

貪欲法が適用可能なことの証明

- 先ほどの貪欲アルゴリズムは左詰め。というのも、同じ ID 内で、X 回増築の部分を採用しないで X+1 回増築の部分を採用しているなら、代わりに X 回増築の部分を採用すればより費用が下がる。
- 左詰めに採用されているのであれば、これは今回の問題で実行可能。
- この値より下回る解は存在しないので、これが最適解となる。

満点解法に向けて

- この方針では $1 \leqq K \leqq 100{,}000{,}000$ という制約に対して充分高速でない。
- 今までの手順をどうにかして高速化したい。
- 更に考察してみよう!

アプローチを変えてみる

- 貪欲法で最後に選んだ増築の費用をVとしてみる。
- 費用がUの増築に対して、
- $V > U$なら、その増築は必ず行われる。
- $V < U$なら、その増築は決して行われない。
- $V = U$なら、合計がK回となる分だけ増築が行われる。

満点解法

- 先ほどのVを考えてみると、適当にV'を定めたとき、V'未満、V'より大きい、V'に等しい費用の増築がそれぞれ何個あるかを計算すれば、VとV'の大小関係が分かる。
- よって、Vを二分探索することで計算することができる。
- Vが分かれば、V未満の増築(M個とする)をすべて採用し、Vと等しい増築を$K - M$個採用すれば最適解を計算できる。
- O($N \log L$) (LはVの上限) で計算できる。
- 100点が得られる。

D問題

だいたい最小全域木

D - 問題概要

問題

200 次元空間に 5 000 個の点がある。

2 点を結ぶコストを (1 - コサイン類似度) とするとき

できるだけコストの小さい全域木を求めよ。

最適なものの 1.01 倍のコストに収まれば OK。

D - 方針

普通にやると……

問題自体は最小全域木だが、
全点間のコストを計算するのは時間がかかりすぎる。

最小じゃなくてもいい！

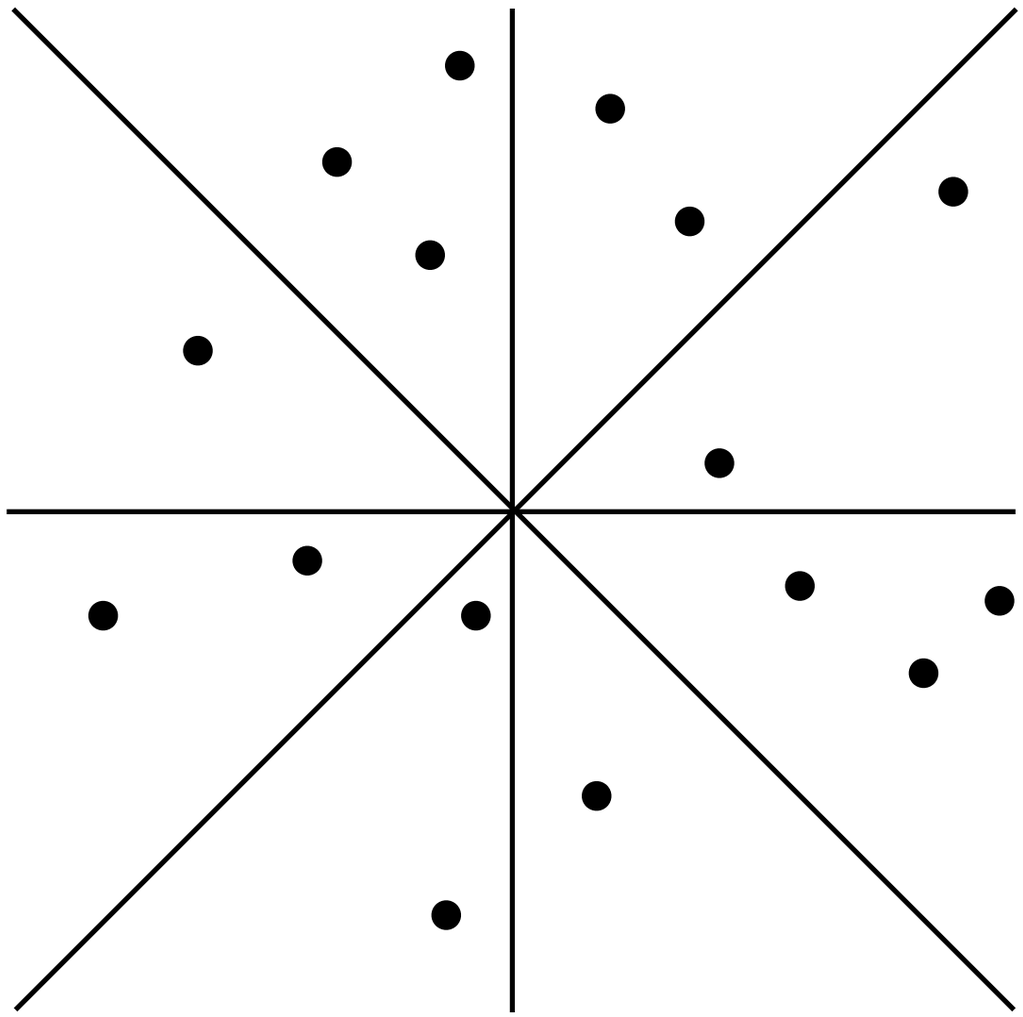
「最適なものの 1.01 倍のコストに収まれば OK。」

だった！

→ 必要そうな 2 点間だけ計算するようにしたい

D - 2次元の場合

この問題が 2 次元空間の場合は……



←こんな感じで
グループ分けして

同じか、近いグループの
点たちだけ見る

D - 200 次元でも同じことを

・ランダムなベクトルを K 個用意する
・各点とランダムベクトルの**内積が正か負か**で
K ビットのハッシュのようなものが計算できる
・このハッシュで近いものだけを考えることにする

ハッシュ同士は**ハミング距離**(異なるビットの個数)
で距離を測れば爆速！！！！！

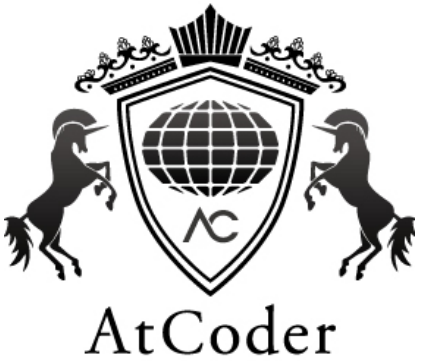
D - K の選び方

今回の場合は点がランダムに分布しているので

- **点の i 番目の座標の値が正か負か**

で 200 ビットのハッシュを作ると、
均等にグループ分けできて良いです。

AtCoder Regular Contest 020
解説



AtCoder株式会社

A問題

石を滑らせるゲーム

A - 問題概要

問題

整数が２つ与えられる。
どちらがより0に近いかを判定せよ。

制限

-1000 ≦ (与えられる整数) ≦ 1000

A - 解き方

1. ２つの整数を入力する。

2. それぞれの整数について絶対値を求める。

3. それらを比較して適切に条件分岐をする。

4. 結果に対応する文字列を出力する。

Ａ - 絶対値の求め方

(方法１) 言語が用意している関数を使う。

→　使っている言語について調べてみよう。

(方法２) 条件分岐を使って自分で書く。

例：

```
if (A < 0) A = -A;
```

Problem B

縞模様

B問題概要

問題

- 数列が与えられる。任意の2種類の数字を使ってそれが交互に繰り返されるように数列を書き換えたい(与えられる数字は10種類)
- 最小で何箇所書き換えなければならないか

B問題解法

解法

- 1番目と2番目に使う数字を決め打ちすれば目標の数列は一意に定まる

→そうしてできた数列と与えられる数列の不一致数を数える

- たとえば数列={2,3,2,1}のとき、

1番目の数字=「2」と2番目の数字=「1」とすると、

目標の数列={2,1,2,1} → この場合の不一致数は1

- それらを全て計算した上で最小値を出力

Problem C

A mod B Problem

C問題概要

問題

- 部分的に周期性のある整数A (一兆桁以上) ←周期の数は1万以下
- 整数B(ほとんどの場合 10^9+7)

が与えられるので、AをBで割った余りを出力せよ

Aの与えられ方の例:

123が2回 4が2回 56が1回繰り返された整数A→1231234456

C問題 部分点解法(20点)

- L1+L2+….+LN ≦ 10^5
- Aiの桁数は10以下
- 展開してもAの長さは10^6以下

したがって、実際に与えられる数を展開し、うまくmodを取ればよい。

(次スライド)

- 計算量は、桁数をNとしてO(N)

うまいmodの取り方

　ある整数を読み込むときに、上の桁から今の整数を10倍して1桁読み込むことを繰り返すと、逐次modが取れてオーバーフローしない。

動作例: 4321 % 11を求めるとき

4321 余り: 4

4321 余り: (4*10+3)%11 = 10

4321 余り: (10*10+2)%11 = 3

4321 余り: (3*10+1)%11 = 9

よって、4321を11で割った余りは9

C問題 ほぼ満点解法(99点)

- 各周期毎に高速に余りを計算する
- 「10^(周期の長さ)倍して周期の余りを足す」を繰り返せばよい
- 繰り返された数は等比数列の和で表されることに着目する↓

$121212(n\text{回}) \ldots = 12 + 12 \times 100 + 12 \times 100^2 + \ldots + 12 \times 100^{n-1}$

一般に公比r(r≠1,r≠0)のとき、

$$r^0 + r^1 + r^2 + \cdots + \mathrm{r}^{\mathrm{n}} = \frac{\mathrm{r}^{\mathrm{n}+1} - 1}{\mathrm{r} - 1}$$

したがって

$$12(1 + 100 + \cdots + 100^{\mathrm{n}-1}) = 12 \times \frac{100^{\mathrm{n}} - 1}{100 - 1}$$

の余りを計算すればよい

C問題 ほぼ満点解法(99点)

- 1/xをする操作が問題(ふつうに割ってmodを取るのはNG)

→1/xと等価な整数(掛けてmodを取ったら1になる数=逆元)を求め、それを掛け算すればよい

- フェルマーの小定理に基づくと、
  pが素数であり、xとpが互いに素であれば、x^(p-2)はxの逆元
- 10^9+7は素数
- 今回、xは9,99,999,….,10^10-1の値しかとらない
  →xとp=10^9+7は常に互いに素
- 逆元は二分累乗法によって、log(p)で計算可能

C問題 満点解法(100点)

- Bの値によっては逆元が求まらないことがある
- ダブリング
- 例えば123123123…を求めるとき

123(1回)

123123(2回)

123123123123(4回)

123123123123123123123123(8回)

…

のように123が2^k回繰り返されたパーツがあれば繋ぎ合せるだけでよい

(modは逐次とっておく)

C問題 満点解法(100点)

- 123が「2^n回繰り返されたもの」から「2^n+1回繰り返されたもの」を作る
- 例えば「123123」から「123123123123」をつくる

→「123123」を1000^2+1倍すればよい

- これはn=1の例だが、他のnでも同様
- これらのパーツが作れたら、パーツを適当な順番で繋ぎ合せる
- 例えば「123123123123123(5回)」なら、
  $5=(101)_2$なので「123」と「123123123123」を繋ぎ合せる
- もともとこれが想定解法でしたが緩和しました

D問題

お菓子の国の旅行

D - 問題概要

問題

N個の町が一直線上に並んでおり、町iと町jの距離(i<j)はa[i]+a[i+1]+...+a[j-1]である。それぞれの町には砂糖屋が１つずつある。ちょうどK個の異なる砂糖屋に一度ずつ訪れる方法のうち移動距離の和がMの倍数であるものは何通りか。

制限

N ≦ 100, M ≦ 30, K ≦ 10

部分点(30点)：N ≦ 12

D - 部分点解法

巡回セールスマン問題にとても似ている。

- **DP[今までに訪れた町の集合][今いる町][移動距離の和 mod M]**

というDPテーブルを更新して行けば良い。

「今までに訪れた町の集合」は二進数で表現すると処理しやすい。

状態数：2^N * N * M

遷移数：O(N)

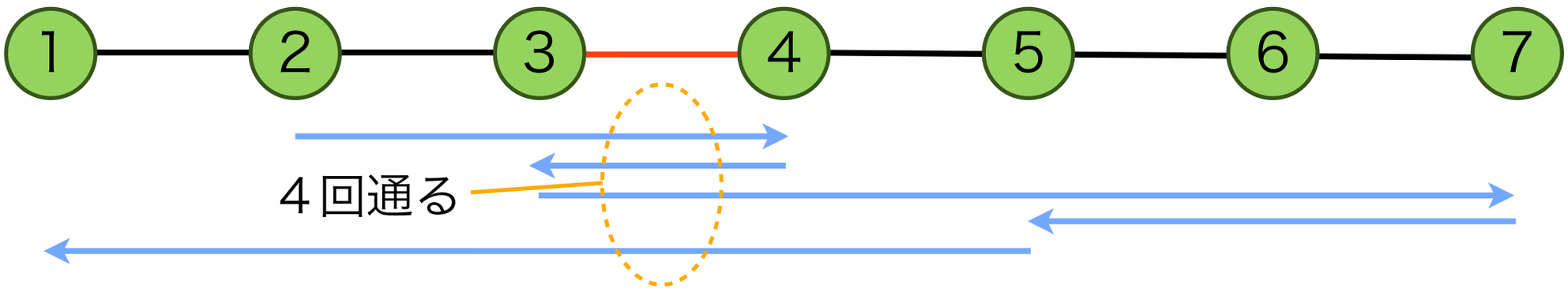
計算量：O(2^N * N^2 * M)

D - 満点解法

各道ごとに独立に計算したい。

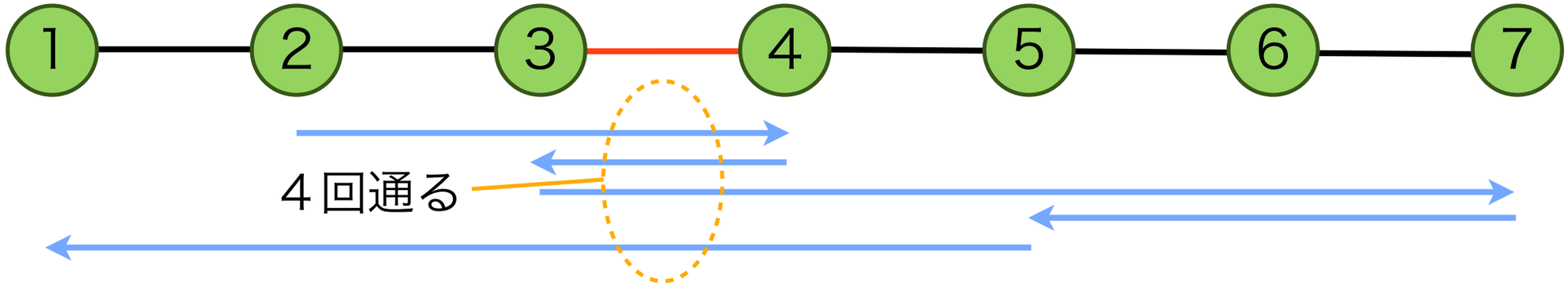
→ 各道に関して、通る回数はどうなっているだろうか？

２４３７５１の順で辿ったとき



D - 考察

２４３７５１の順で辿ったとき



町3と町4の間にある道を通るのは、
「2 → 4」「4 → 3」「3 → 7」「5 → 1」の移動をするとき。
つまり「３以下から４以上」または「４以上から３以下」に移動するとき。

ここで、「３以下」を1、「４以上」を0と表してみると、

１０１００１

となる。
この二進数において「隣り合うbitが異なる場所の個数」が「道を通る回数」となっており、この情報さえあれば事足りることが分かる。

D - DP

DP[今注目している道][さっきのような二進数][移動距離の和 mod M]
というDPを考える。

「さっきのような二進数」は
・iビット目が1のとき、i番目に訪れる町が注目している道の左にある
・iビット目が0のとき、i番目に訪れる町が注目している道の右にある
ということを表している。

DPの遷移で行うことは、
・今見ている道の左の端点の町にi番目に訪れることにする(i = 1~k)
・なにもしない
の２通りである。

D - DP

状態数：N * 2^K * M

遷移数：O(K)

計算量：O(N * 2^K * M * K)

D - 補足

実は状態数はもう少し減らせる。
もう少し考察をすると、計算に関わってくる情報は「0が繋がっている部分の長さのリスト」だけであり、それらの順番は関係がないことが分かる。

例えば、「10110011001」と「10011101001」はいずれも、
「0と00と00が1によって区切られている」という同じ状態として見なせる。

状態数はKの分割数のようなオーダーになるため、Kが20近くても解けるようになる。

（より細かく言うと「接頭辞/接尾辞に何個の0が連続しているか」という情報も持つ必要がある。）

AtCoder Regular Contest 019
解説

A問題

お買い物クライシス

A - 問題概要

問題

数字の一部がアルファベットに置き換わった
文字列 S が与えられる。
元の数を復元せよ。

アルファベット	数字
O	0
D	0
I	1
Z	2
S	5
B	8

制限

1 ≦ (S の文字数) ≦ 8

A - 解き方

1. 文字列 S を入力する

2. S を 1 文字目から順に見ていく

3. 各文字について、
    - 数字ならばそのまま出力し
    - アルファベットならば対応する数字を出力する

A - チェックポイント

1. 文字列 S を入力する

文字列の入出力や扱い方については
使っている言語ごとに調べよう

2. S を 1 文字目から順に見ていく

3. 各文字について、
   - 数字ならばそのまま出力し
   - アルファベットならば対応する数字を出力する

if 文などでそのつど調べてもよし
あらかじめ文字変換のハッシュテーブルを用意するもよし

B問題

こだわりの名前

B - 問題概要

問題

英大文字からなる名前 A が与えられる。

これを 1 文字だけ別のアルファベットに変換して

作れる**回文でない**文字列は何個あるか？

制限

$1 \leqq (A \text{ の文字数}) \leqq 3 \times 10^5$

B - 考えよう

まずはじめに

書かれていることをそのまま計算することはできる

→1文字の変更を全部試して、回文かどうか判定する

計算量は？

・変更候補が $25 \times N \leqq 7{,}500{,}000$ 通り

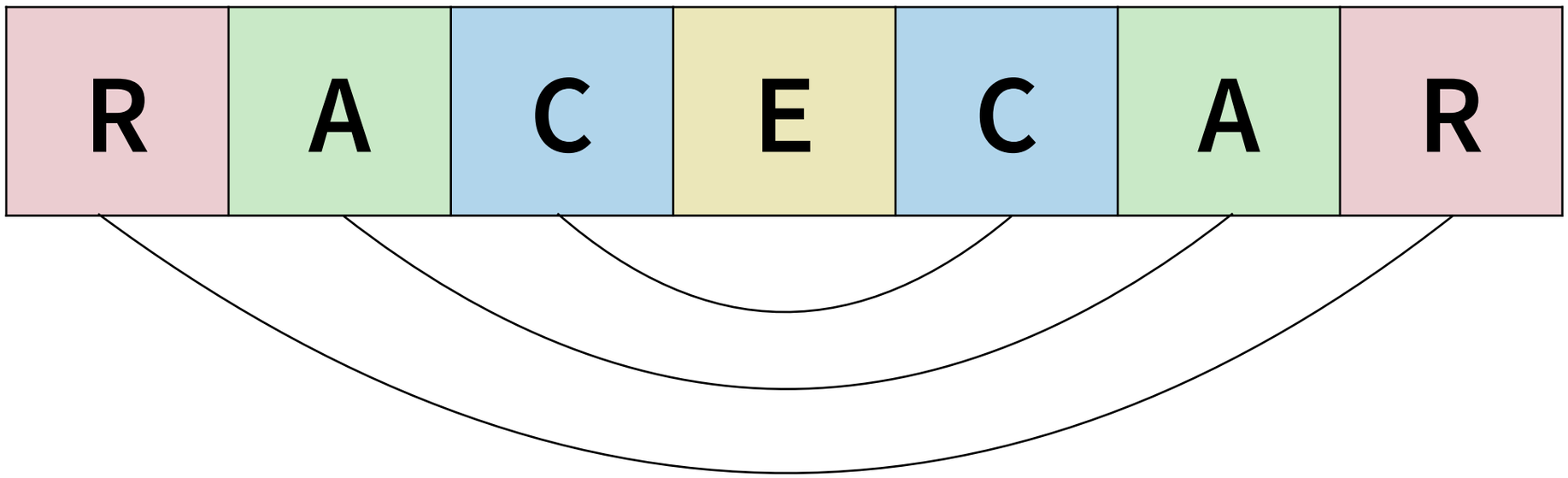
・回文かを試すのに $O(N)$ 時間

→ $25 \times N \times N \leqq 2{,}250{,}000{,}000{,}000 \fallingdotseq \mathbf{2 \times 10^{12}}$

はとうてい 2 秒では無理……

B - そもそも回文とは？

「前から読んでも後ろから読んでも同じ」とは
結局どういうことなのか？



「線で結ばれた文字(以降ではペアと呼ぶ)が同じ」
なら回文！

B - 解き方(1/2)

文字数が奇数のとき

真ん中の文字は何でも良い

・元の文字列が回文なら、真ん中の文字を変えても回文

・元の文字列が回文でないなら、真ん中の文字を変えても回文でない

→真ん中の文字は後で考える(文字数が偶数の場合に帰着)

文字数が偶数のとき

すべてのペアの文字が一致していれば回文

→**文字が一致しているペアの数に注目**しよう

B - 解き方(2/2)

重要な考察

1 文字の変更では「文字が一致するペアの数」は

1 増えるか、**1 減る**か、**そのまま**か

のいずれかである。

(1 文字しか変えてないのだから、当たり前)

これにより、1 文字変更した後の回文の判定が

O(1) の計算量でできるようになる！

→これなら時間制限にも間に合う

C 問題 最後の森

解説担当:城下 慎也 (@phidnight)

はじめに

- この問題において入力に不備が見つかったため、ノーコンテストとなってしまい、誠に申し訳ありませんでした。
- このようなことが再び起こらないように今後も気をつけていきたいと思います。

概要

- R*C のマス目が与えられる。
- 村→ほこら→城 の経路で長さ最小なものを考えたい。
- 途中にいる敵は K 体まで倒すことができる。
- 一度倒した敵は復活しない。

倒す敵が決まると?

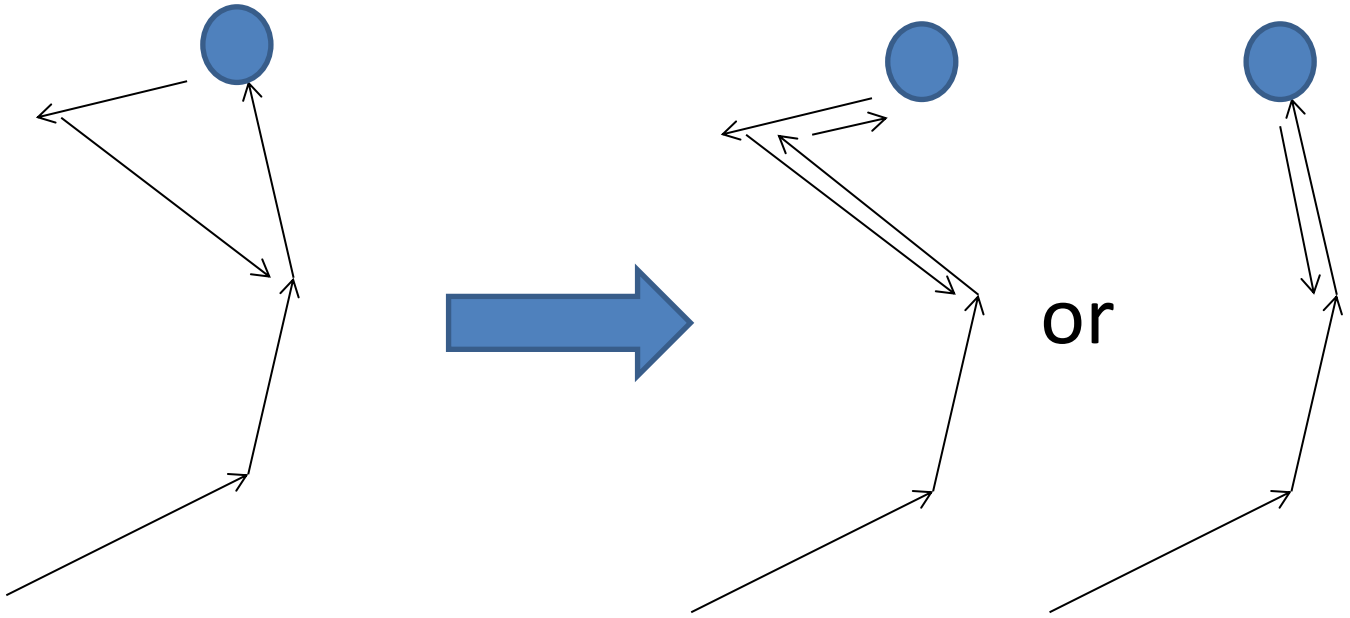
- 最短経路問題を 2 回解くことで解が得られるようになる。
- ただ、敵の総数を E としたとき、$_E C_K$だけ組み合わせを試さなければならないので指数オーダー。
- DP を使おうにも、倒した敵が復活しないところを情報としてうまく保存するのが難しい。
- まずは多項式解法を求めよう！

指数を回避するために

- 倒した敵を情報として持たないようにしたい。
- すべての経路ではなく、最適性を維持したまま、うまく状態を持てるような経路だけを考えたい。
- 必要な経路を厳選しよう！

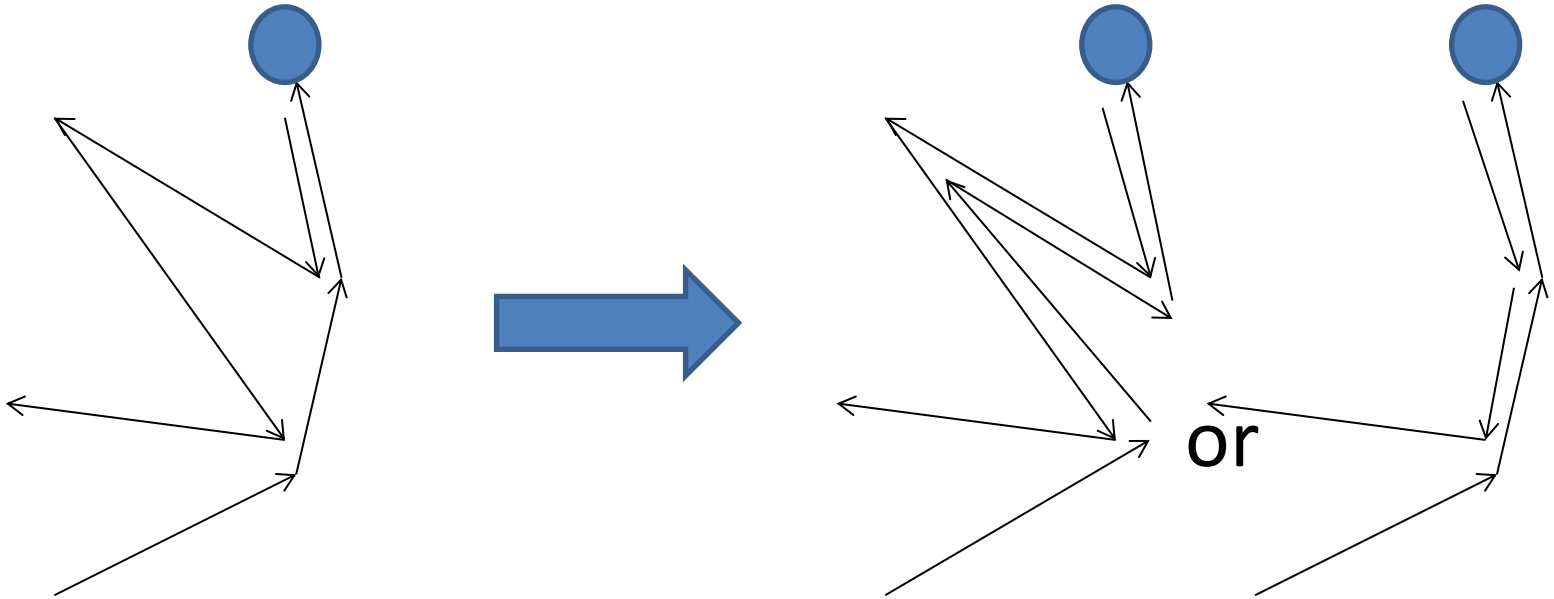
考察1

- 中継地点に行った後に、来た道を直接戻らずに途中で合流する経路は考えなくても良い。
- 次のどちらかが、敵の数、長さを増やさない、別の経路であるといえる。



考察2

- 来た道を戻る操作を止めた後は、同じ道を通らないとしてもよい。
- 次のどちらかが、敵の数、長さを増やさない経路といえる。



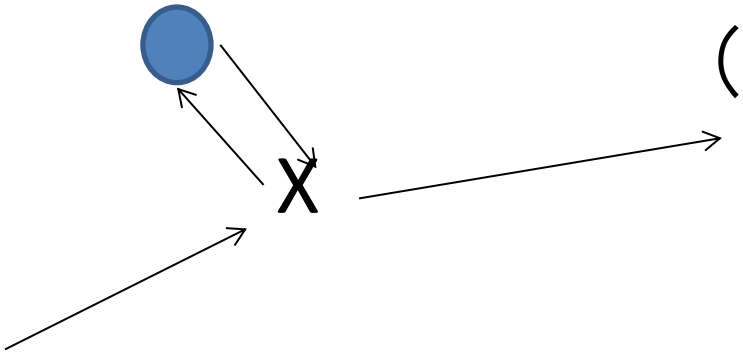
考察まとめ

- 以上より、経路として、

  村→三叉路点X→ほこら→三叉路点X →城

  だけを考えれえば良い。
- 村→三叉路、三叉路↔ほこら、三叉路→城とわけて、それぞれを足し合わせれば良い。

（この方針で多項式解法）



多項式解法その1

- X の場所を固定する。
- dp[i][j]=(X からマス i に、合計戦闘回数が j 回以下で移動するのに必要な最小移動回数)
  というdpを計算する。
- BFSを用いて、それぞれのマスに対し O(RCK) で計算することができる。
- a+b+c≦K (a,b,c≧0)に対して、
  Xに敵がいない:dp[村][a]+2*dp[ほこら][b]+dp[城][c]
  Xに敵がいる:dp[村][a]+2*dp[ほこら][b-1]+dp[城][c-1] (b,c≧1)

で解が計算できる。

多項式解法その1

- dpの計算に O(RCK) かかり、
- すべての a,b,c の組を試すのに O(K^3)かかるので、各 X に対し全体で O(RCK+K^3)
- 全体でO($R^2C^2K + RCK^3$)
- この解法だとTLEしてしまう。

多項式解法その2

- 発想を逆にして、
- dp1[i][j]=(村 からマス i に、合計戦闘回数が j 回以下で移動するのに必要な最小移動回数)
- dp2[i][j]=(ほこらからマス i に、合計戦闘回数が j 回以下で移動するのに必要な最小移動回数)
- dp3[i][j]=(城からマス i に、合計戦闘回数が j 回以下で移動するのに必要な最小移動回数)
- とすれば、各Xおよびa+b+c≦K (a,b,c≧0)に対して、
- 敵がいない場合:dp1[X][a]+2*dp2[X][b]+dp3[X][c]
  敵がいる場合:dp1[X][a]+2*dp2[X][b-1]
  +dp3[X][c-1](b,c≧1)
  で最適解が計算できる。

多項式解法その2

- すべての a,b,c に対して動かすのではなく、
- v[x]:(x=a+bとなる場合のdp1+2*dp2の最小値)
- w[y]=(y=x+cとなる場合のv+dp3の最小値)

と 2 段階に分けて計算すれば、O(K^2)で最適解を計算できる。

- 全体で O(RCK^2) となって満点が得られる。

D 問題 ほんとうのたたかい

解説担当:城下 慎也 (@phidnight)

概要

- N*N のフィールドの上にいくつか O を置く。
- 4 つの隅が軸に平行な長方形とならないように配置する。
- できるだけ多く O を配置したい。
- 2≦N≦150

部分点1

- 5 点解法 (“. ”を”X”に置き換えています)

OOO…O

OXX…X

OXX…X

: :

OXX…X

部分点2

- 10 点解法 (“. ”を”X”に置き換えています)

XOO…O

OOX…X

OXO…X

: :

OXX…O

部分点3

- それ以上を得たい場合
- ランダムに置くと強い
- うまく貪欲にすると強い
- 焼きなますと強い
- etc…

- 満点を得たい場合は?

考察

- そもそも理論的にはどれくらい置けるだろう?
- 長方形が成立する条件を考えると、
- 長方形が成立する⇔ (i,j), (i,l), (k,j), (k,l) のいずれにも O があるような i<k , j<l が存在する
- 各行 p ごとに、(p,q) と (p,r) (q<r) のいずれにも O があるなら、(q,r) というペアを保存する、という処理を行うと、上記の条件が成立する場合に、i 行目と k 行目でペア (j,l) が保存されることになる。

考察

- このように考えると、実は
  長方形が成立する⇔同じペアが複数出てくる
- という式が成立することになる。
- 裏を返すと、同じペアは高々 1 回しか出ないことになる。
- ペアの総数は N(N-1)/2 個であり、ある行に m 個あればペアの数が m(m-1)/2 だけ減るので、
- N(N-1)/2 $\leqq \sum_{i=1}^{N} m_i(m_i - 1)/2$
  が成立する。ただしm_iは i 行目の O の総数。

考察

- 右辺の式はm_iのばらつきが少ないほど$\sum_{i=1}^{N} m_i$が一定の場合に小さくなる。
- 置ける o の最大数はおよそ$N\sqrt{N}$ 個となる。
- このこと (できるだけまばらに置く) を踏まえると良い解が得られる。

N=p*p (p:素数) のとき

- このときに p*p*p 個の O を置く配置を発見する。

※素数のほうが作りやすいので、素数にしています。

- 例: 第 p*(i-1)+j (1≦i,j≦p) 行目の、p*(k-1)+1 から p*k 列(1≦k≦p)までの範囲には、

  i*k+j を p で割った余りを q としたとき、

  (p*(i-1)+j, p*(k-1)+1+q) にのみ O を置く。

p=3 の場合の例

OXXOXXOXX
XOXXOXXOX
XXOXXOXXO
OXXXOXXXO
XOXXXOOXX
XXOOXXXOX
OXXXXOXOX
XOXOXXXXO
XXOXOXOXX

このようになります。

100点解法例

- さっきの p を p=11 とすれば S=1331 が得られる。
- しかしながら 150 に対しては未使用な部分が存在する。
- そこで p=13 として、左上の 150*150 の領域を取り出すことにすれば良い。
- 結果、S=1700超となり、100点が得られる。

AtCoder Regular Contest #018

解説資料

2014年　3月 1日

©AtCoder Inc. All rights reserved.

AtCoder

A問題 BMI (解説: kyuridenamida)

©AtCoder Inc. All rights reserved.

A問題　問題概要

AtCoder

1. 身長$[cm]$とBMI $[kg/m^2]$が与えられる。

2. 以下の関係式が成り立つので体重[kg]を逆算せよ。

$$BMI[kg/m^2\ ] = \frac{(\text{体重}[kg])}{(\text{身長}[m])^2}$$

©AtCoder Inc. All rights reserved.

A問題　解法

AtCoder

1. 与えられる式を体重[kg]について解く。↓

$$体重[kg] = (BMI[kg/m^2]) \times (身長[m])^2$$

2. これを出力する。ただし、与えられる身長の単位はcmなので、100で割りメートルに直さなければならないことに気をつける。

また、出力精度に気をつける。

©AtCoder Inc. All rights reserved.

A問題　補足(出力精度について)

AtCoder

- 今回の問題は、必要精度(絶対誤差0.01が以下)がそれほどでないため、多くの言語で、特に意識しなくても正答が得れるかもしれない。
- 言語によっては、勝手に丸められて出力されることがあるため、実数を出力する問題では出力桁数に関する書式指定を明示的に行い、必要精度以上の出力をすることを強く推奨する。

(C言語の例) printf(“%.10f”,1.23); //出力は”1.2300000000”

©AtCoder Inc. All rights reserved.

AtCoder

B問題 格子点と整数 (解説: catupper)

©AtCoder Inc. All rights reserved.

B問題　問題概要

AtCoder

1. N個の格子点が与えられる
2. それらのうち3つを使って三角形を作る
3. 面積が正の整数であるものの数を求めよ

$$1 \leqq N \leqq 100$$

$$1 \leqq x, y \leqq 1{,}000{,}000{,}000$$

©AtCoder Inc. All rights reserved.

B問題　想定解

AtCoder

- $(0,0), (x_1,y_1), (x_2,y_2)$からなる三角形の面積Sは以下の式で求めることができる

$$S = | x_1 y_2 - y_1 x_2 | / 2$$

- 原点を含まなくてもいずれか一点が原点になるよう三角形を平行移動すればよい
- 三角形の頂点から面積を求める方法は頻出
  - ABC002 C「直訴」等でも出ています。

©AtCoder Inc. All rights reserved.

B問題　想定解

AtCoder

- Nが小さいので全ての三角形の面積を調べても良さそう

  → 全探索

- 同じ三角形を重複してカウントしないように気をつけましょう
- 計算量: $O(N^3)$

©AtCoder Inc. All rights reserved.

B問題　注意点

AtCoder

$$1 \leqq x, y \leqq 1{,}000{,}000{,}000$$

- 面積はこの2乗くらいの大きさになりうるのでC言語などでは

  int型だとオーバーフローしてしまう

  - long long int型にしないとWrong Answer?
  - 案外int型でも大丈夫。だけど絶対大丈夫ではない
  - double(倍精度浮動小数点数)型←かなりまずい

©AtCoder Inc. All rights reserved.

B問題　注意点

AtCoder

- 面積: $S = | x_1 y_2 - y_1 x_2 | / 2$
- Sが整数になるかどうかの判定
  - $x_1 y_2 - y_1 x_2$ が偶数かどうか
  - intならオーバーフローしても変わらない
- Sが0かどうかの判定
  - intなら $x_1 y_2 - y_1 x_2$ が2^{32}(int型の扱える範囲の大きさ)

    の倍数の時のみ判定に失敗する

©AtCoder Inc. All rights reserved.

B問題　doubleがまずい理由(1)

AtCoder

- 多くの言語の浮動小数点数型はIEEE 754 に基づいている。

  例えばC言語のdoubleは、符号部1bit・指数部11bit・仮数部52bitで構成されている。指数部が全て0で無い限り、暗黙の整数ビットとして1bit追加され、53bitが仮数部の幅となる。
- その性質上、2^53を超える整数値は、2の倍数刻みでしか表現できなくなり、2^54を超える整数値は4の倍数刻みでしか…以下同様。

©AtCoder Inc. All rights reserved.

B問題　doubleがまずい理由(2)

AtCoder

- 以下のC言語コードを実行してみると、

  実際にa==bがTrueになっていることが確認できる。

```
#include <stdio.h>
int main(){
	double a = 9007199254740992.0; // 2^53
	double b = 9007199254740993.0; // 2^53+1
	if( a == b ){
		puts("a==b");
	}
}
```

©AtCoder Inc. All rights reserved.

B問題　doubleがまずい理由(3)

AtCoder

- doubleは64bitだが、64bit整数を表現できるわけではない！
- 今回、10^9≒2^30だから、座標の掛け算で、

  2^30*2^30 = 2^60 >> 2^53となってしまうことがあり、

  そうなった場合、偶奇が正しく判定できなくなる。

  個intの場合よりWAが増えてしまう…

©AtCoder Inc. All rights reserved.

B問題　ちなみに

AtCoder

- Nがもう少し大きくても解けます
- 面積が整数かどうかは頂点のxの偶奇,yの偶奇にしかよらないので各格子点を4通りにグループ分けして、適当に掛け算をする
- 面積が0のものを除かなければならないが、これは一直線上に並ぶ3点を数えればできる

©AtCoder Inc. All rights reserved.

B問題　ちなみに

AtCoder

- 全点対の直線を挙げて、同じ直線が何個ずつでてくるか調べれば良い。直線を適当なパラメータで表して、ソートするなどすればO(N^2 log N)で処理することができる。
- このときは直線を管理するときにオーバーフローに気をつけなければならない

©AtCoder Inc. All rights reserved.

AtCoder

C問題 席替え (解説: catupper)

©AtCoder Inc. All rights reserved.

C問題　問題概要

AtCoder

1. サイズがN×Mの２次元配列Xが与えられる
2. a＜cならX[a][b]≦ X[c][d]となるように並び替える
3. 総移動マンハッタン距離の最小値を求めよ

©AtCoder Inc. All rights reserved.

C問題　入力の乱数

AtCoder

- $x_{0,}a,p$が与えられる
- $x_{i+1} = (x_i + a)\ mod\ p$で生成した値が各席の成績
  - ➢ これを並び替える
- a mod p = 0のとき → x_0以外は全て同じ値になる
- a mod p ≠0のとき → 成績は全て異なる値を取る

©AtCoder Inc. All rights reserved.

C問題　x_0以外同じ場合

AtCoder

- x_0が動く必要かあるかどうかで場合分け
  - 動かなくていいなら誰も動かなくて良い
    - 総移動距離は0
  - 動く必要があるなら一番後ろの誰かと交換
    - 総移動距離は2×(N−1)

©AtCoder Inc. All rights reserved.

C問題　値がすべて異なるとき

AtCoder

- 各生徒の移動先の行が一意に定まる
  - x番目に成績が悪い人は第$\left\lfloor \frac{x}{M} \right\rfloor$行に移動する
  - 各行について別々に考えて良い
- 行成分の移動距離は確定するので列成分だけを考えれば良い

©AtCoder Inc. All rights reserved.

C問題　値がすべて異なるとき

AtCoder

- 以下の例を考える

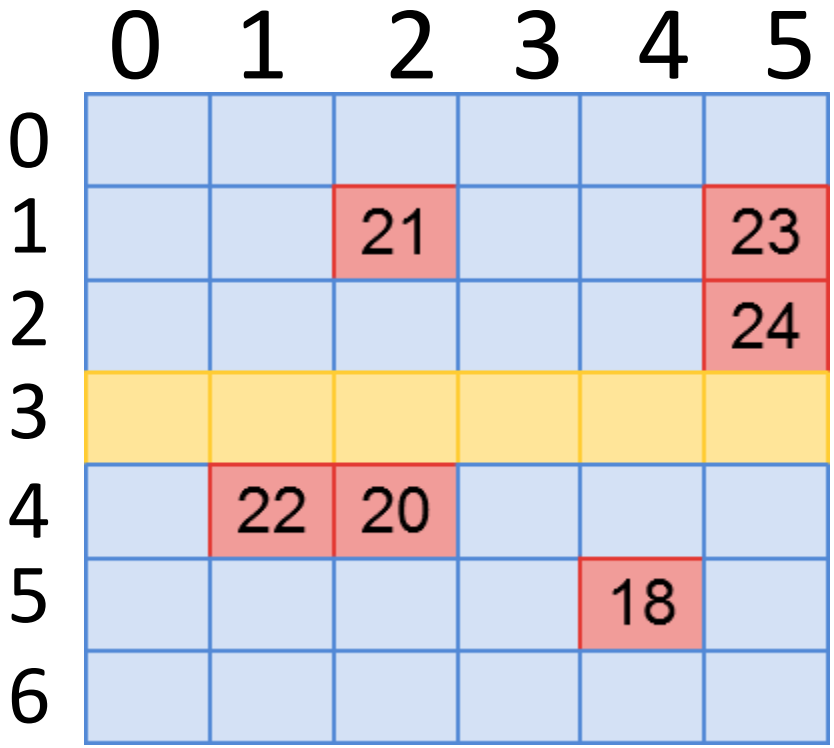
	0	1	2	3	4	5
0	6	39	8	30	33	41
1	25	3	21	15	31	23
2	17	36	34	35	28	24
3	27	29	38	14	13	26
4	40	22	20	5	16	11
5	19	4	0	7	18	37
6	10	1	12	2	32	9

©AtCoder Inc. All rights reserved.

C問題　値がすべて異なるとき

AtCoder

- 3行目に行くべき生徒だけ考える

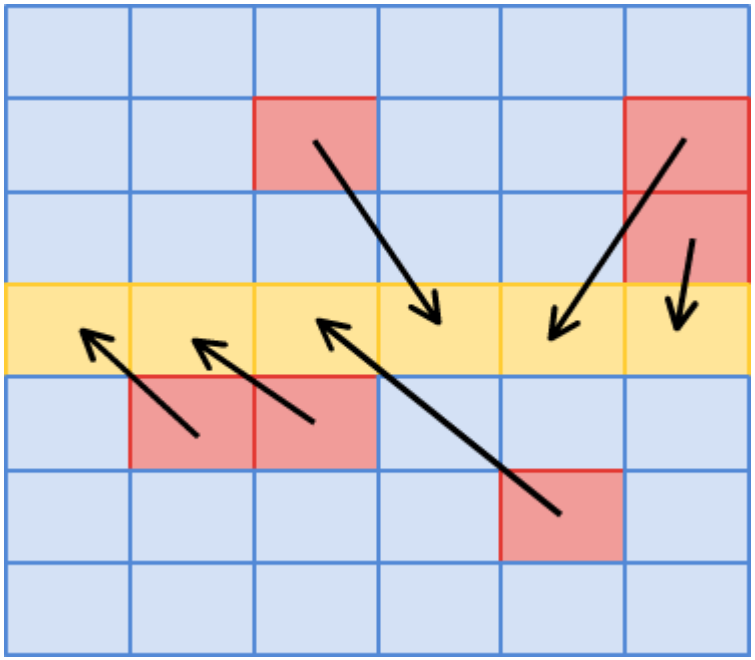


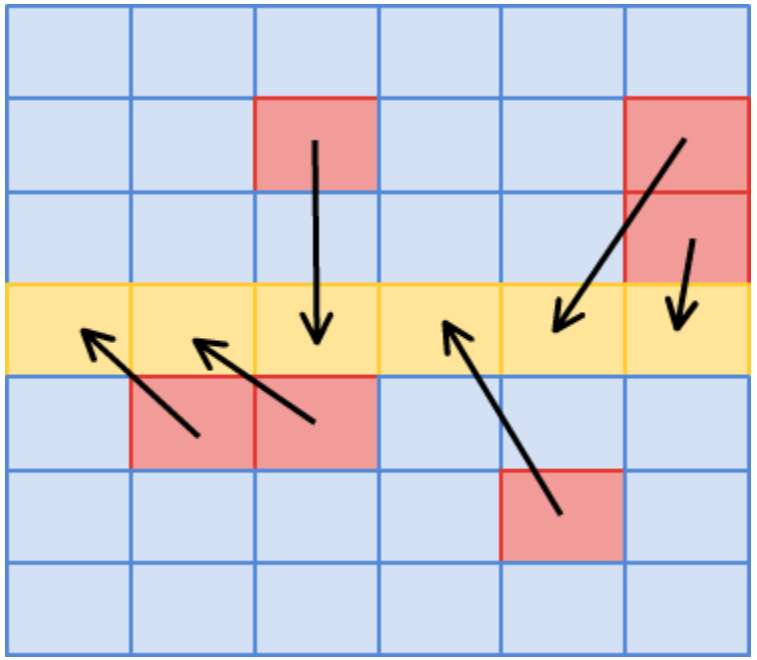
©AtCoder Inc. All rights reserved.

C問題　値がすべて異なるとき

AtCoder

- 動き方はいろいろ考えられる



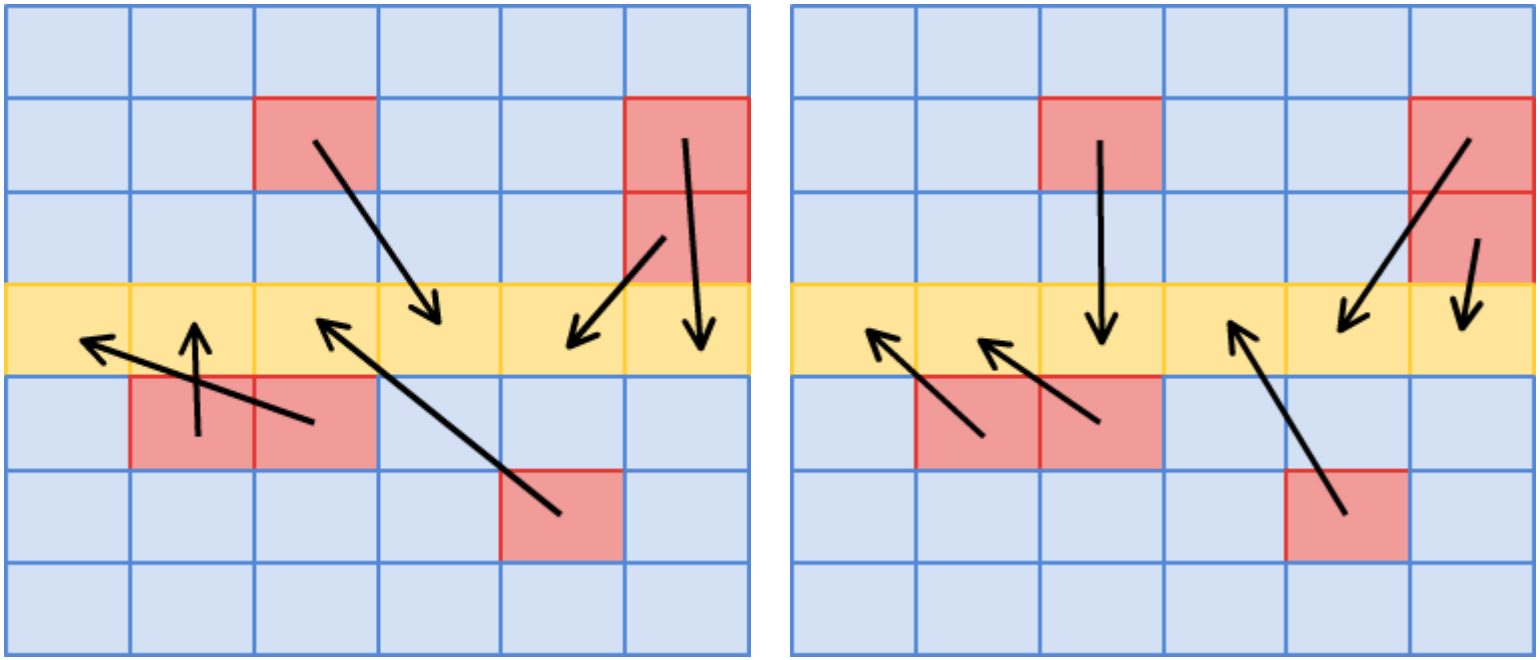


©AtCoder Inc. All rights reserved.

C問題　考察

AtCoder

- 左図のように交差の多い移動は、改善の余地がありそう



©AtCoder Inc. All rights reserved.

C問題　考察

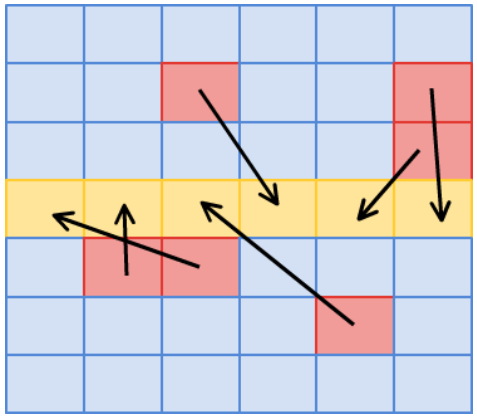
AtCoder

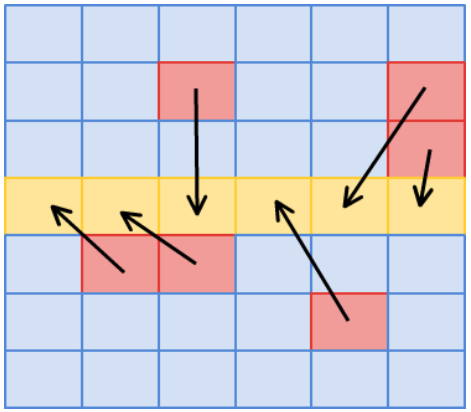
- 移動前と移動後で列の大小が入れ替わっているならば、

  それらの目的地を逆にしても損はしない

- 左図と右図では1,2列目を入れえて改善されている

- 5,6列目も入れ替えているが損も得もしていない





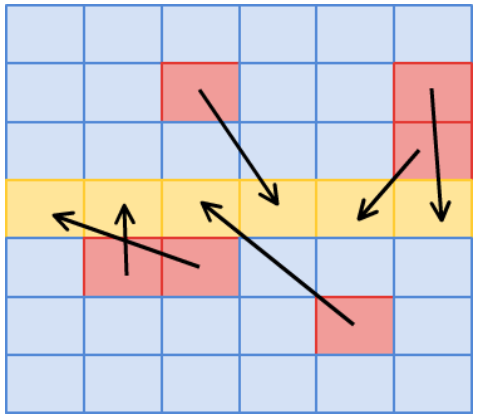
©AtCoder Inc. All rights reserved.

C問題　考察

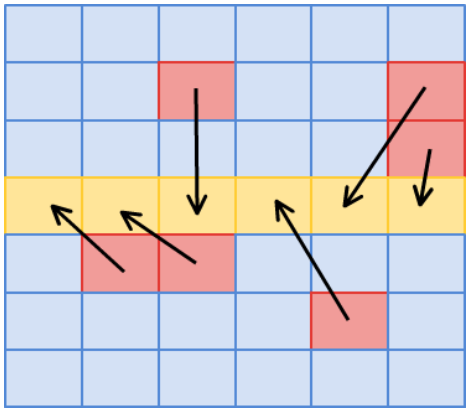
AtCoder

- 移動前の列の大小関係を保ったまま移動したら最適解の一つになる
  - 前スライドより、入れ替えをしても得をしないから

最適でない



最適



©AtCoder Inc. All rights reserved.

C問題　解法

AtCoder

- 移動後の行ごとに問題を分ける
  - あらかじめ全体でソートして自分が何行目に行くか調べておく　→　O(NMlog(NM))
- 列の大小関係を保つように移動先を決定する
  - 今度は列についてソートする　→　O(MlogM)をN行分
- 各席について移動前と移動後のマンハッタン距離を計算して総和を求める　→　O(NM)

©AtCoder Inc. All rights reserved.

C問題　解法

AtCoder

- 合計でO(NMlog(NM))
  - 満点解法

©AtCoder Inc. All rights reserved.

AtCoder

D問題 僕は友達が少ない（解説: kyuridenamida）

©AtCoder Inc. All rights reserved.

D問題　問題概要

AtCoder

1. N(≦10^4)個の頂点とM (≦10^5)本の正の重み付きの辺から成る連結な無向グラフが与えられる。ただし、同じコストの辺は100本以下。
2. 最小全域木のコストと個数を求めたい。

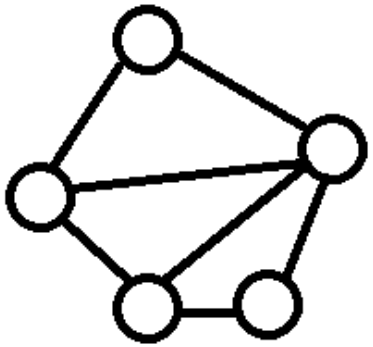
©AtCoder Inc. All rights reserved.

D問題　最小全域木とは

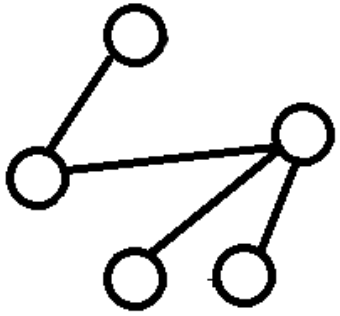
AtCoder

1. グラフが含む辺から構成される、全ての頂点を繋ぐ木

   (閉路のない連結なグラフ)のことを全域木と呼びます。



グラフ



全域木の例

2. 最小全域木とは、辺の重みの合計が最小であるよう

   な全域木のことを言います。

©AtCoder Inc. All rights reserved.

D問題　最小全域木を求めるアルゴリズム

AtCoder

代表的な2つのアルゴリズムの名前を紹介しておきます。これらについてはネットで検索するとすぐに資料が見つかると思います。

難しいアルゴリズムではないので、是非調べてみてください。

1. クラスカル法
2. プリム法

©AtCoder Inc. All rights reserved.

D問題　問題の考察

AtCoder

1. 高橋君(頂点1)が全ての新入生と友達になる(連結になる)を

   最小費用で達成する⇔最小全域木

   (費用が全て正なので必ず全域木になる)

2. 自分もしくは自分の友達から伸びてる辺のみしか使えない

   →適切にやれば任意の全域木が作れる。無視していい制約。

3. 同じコストの辺が100本以下

   →満点解法において重要

©AtCoder Inc. All rights reserved.

D問題　部分点(10点)解法

AtCoder

1. 辺のコストが全て異なる

   → 最小全域木が一意に定まる(クラスカル法が貪欲的解法だったことを考えると明らか)

2. 最小全域木のコストは普通に求め、選び方の数としては、”1”を出力すれば良い。

©AtCoder Inc. All rights reserved.

D問題　満点(100点)解法への予備知識(1)

AtCoder

1. グラフの(ラベル付き)全域木の個数は、

   ラプラシアン行列と呼ばれる以下のような行列の、任意の小行列の行列式の値に等しいことが知られている (行列木定理)。

   [ラプラシアン行列L]

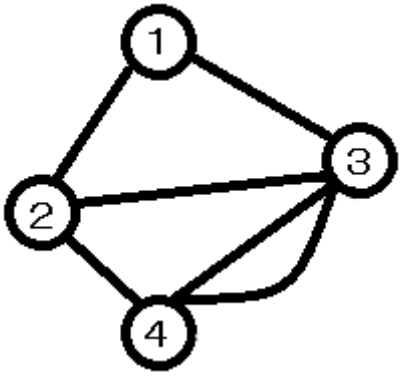
   1. 対角成分L_{ii}は、グラフの頂点iに繋がってる辺の数
   2. 非対角成分L_{ij}は、(-1)×(頂点iとjを直接繋ぐ辺の数)

©AtCoder Inc. All rights reserved.

D問題　満点(100点)解法への予備知識(2)

AtCoder

2. 具体例として、以下のようなグラフについて考える。



$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 4 & -2 \\ 0 & -1 & -2 & 3 \end{pmatrix}$$

3. 適当な小行列の行列式を計算(今回は1行1列のものを使う)

$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 4 & -2 \\ 0 & -1 & -2 & 3 \end{pmatrix}$$

$$\begin{vmatrix} 3 & -1 & -1 \\ -1 & 4 & -2 \\ -1 & -2 & 3 \end{vmatrix} = 13$$

∴全域木の数は13通り

©AtCoder Inc. All rights reserved.

D問題　満点(100点)への考察

AtCoder

1. クラスカル法は、小さいものから貪欲に、ある辺の追加によってグラフに閉路が生じなければその辺を採用していくアルゴリズムであった。どの最小全域木についても、あるコストを持つ辺の本数は変化せず、あるコストを持つ辺のみに着目したとき、それらによる連結関係は変化しない。
2. この考察を踏まえた上で最小全域木の個数を求める方法を次スライドに記載する。

©AtCoder Inc. All rights reserved.

D問題　満点(100点)　想定解

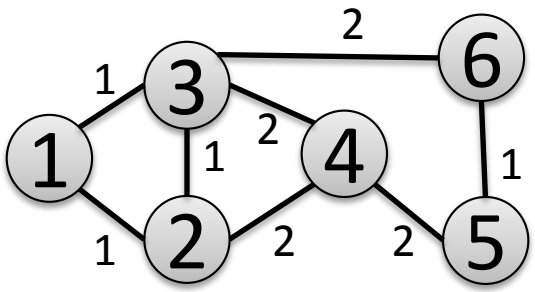
AtCoder

1. 辺をコストでグループ分けし、小さいコストの辺のグループから順番に以下の処理を繰り返し行う。最初(全体の答え)=1
    1. 同じコストの辺だけを考えたグラフを考え、そのグラフの全域森の数を数え、それを(全体の答え)に掛け合わせる。
    2. そのコストの辺が作る連結成分を、それぞれ1つの頂点に圧縮する(この処理により生じる自己辺は削除する)。
2. 以上を連結成分が1つになるまで繰り返し、答えを出力する。

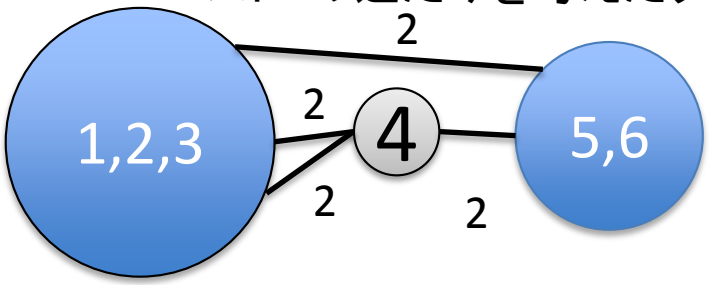
©AtCoder Inc. All rights reserved.

D問題　動作例(入出力例3)

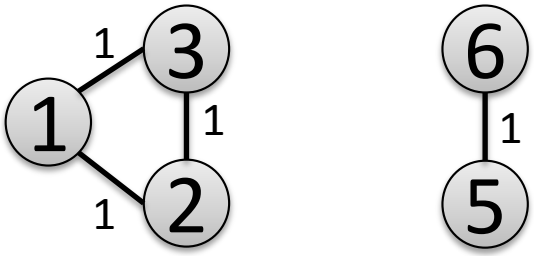
AtCoder



↓　圧縮後のグラフで
コスト2の辺だけを考えたグラフ



全域森(木)の数は5個



↑　コスト1の辺だけ考えたグラフ
全域森の数は3個

∴最小全域木の数は3*5=15通り

©AtCoder Inc. All rights reserved.

D問題　行列式の計算に掛かる計算量

AtCoder

1. 一般にN×N行列の行列式は、行列を三角化したあと、対角成分の積を計算すれば求まる。これはO(N^3)で計算できる。
   - 三角化は加減乗除が出来れば可能である。
     - Pが素数なら、Pを法として加減乗除が可能なので、逆元を求める操作を加味しても、O(N^2*(logP+N)) で三角化が可能である。 (ほぼO(N^3))

©AtCoder Inc. All rights reserved.

D問題　全体の計算量(概算)

AtCoder

1. 辺のソート O(M log M)
2. 同じ辺の数がlim(=100)本以下

   → 連結成分の頂点数はO(lim)→ 行列式の計算O(lim^3)
3. それがO(N/lim)回繰り返される。

   → O(N/lim) * O(lim^3) = O(N * lim^2)

©AtCoder Inc. All rights reserved.

AtCoder Regular Contest 017

解説

A問題

素数、コンテスト、素数

A - 問題概要

問題

整数 N が与えられる。

N が素数かどうかを判定せよ。

制限

$17 \leqq N \leqq 1{,}000{,}000$

A - 正解に必要な要素

アルゴリズム

・素数の定義

・素数の判定法

1 とその数自身でしか割り切れない
2 以上の正の整数

※問題文に書かれています

N を割り切るような数を探してみて、
それが 1 と N だけなら素数！

実装

・標準入出力

・ループ

→実装で分からないことがある場合は
「標準入出力」などで調べてみよう

A - 解き方

1. N を標準入力から入力する。

2. 2 から N-1 までのすべての整数について、
   N を割り切ることができるかを調べる。
   →多くの言語では % 演算子で調べることができる

3. いちども割り切れなければ N は素数 → YES
   そうでなければ N は素数ではない → NO
   をそれぞれ標準出力に出力する。

B問題

解像度が低い。

B - 問題概要

問題

業績を表す N 要素の整数列 A が与えられる。
そのうちの連続した K 個で、常に上昇しているものは何個あるか？

制限

$1 \leqq K \leqq N \leqq 300{,}000$
$1 \leqq A_i \leqq 300{,}000\ (1 \leqq i \leqq N)$

B - 注意点(1/2)

$K = 1$ のとき、「常に上昇している」とは？

→問題文の記述からは、あまり明らかではない……

直感的には

たとえば (100) は上昇しているようには見えないが……

ありうる解釈(今回正しいのはこっち)

常に上昇とは “$1 \leqq i <$ (Xの長さ) について $X_i < X_{i+1}$”

と考えれば、(100) も「常に上昇している」ととれる

→そもそも i がとれないから OK

B - 注意点(2/2)

プログラミングコンテストの問題文では、こういった「数学的な」解釈をすることが多い
（特に今回のように $K = 1$ などの端っこのケース）

少しでも「あれ？」と題意に疑問を持ったら……
質問しましょう。積極的に質問すると良いです。

(求めている返答が返ってくればよいですし、仮に「問題文をよく読んでください」なんて返されてしまったとしても損することはありません。ジャッジ側の仕事が増えるだけですが、そんなことを参加者は気にしなくてもよいので、疑問があればためらわずにガンガン質問を送りましょう。)

B - 解き方(1/3)

素直にループを書いて数えると？

　→もちろん答えは出るけど、時間は大丈夫？

今回は N が $300{,}000$ まであって、間に合わない！

・具体的には約 $N(N\text{-}K)$ 個ぐらいの要素を見る

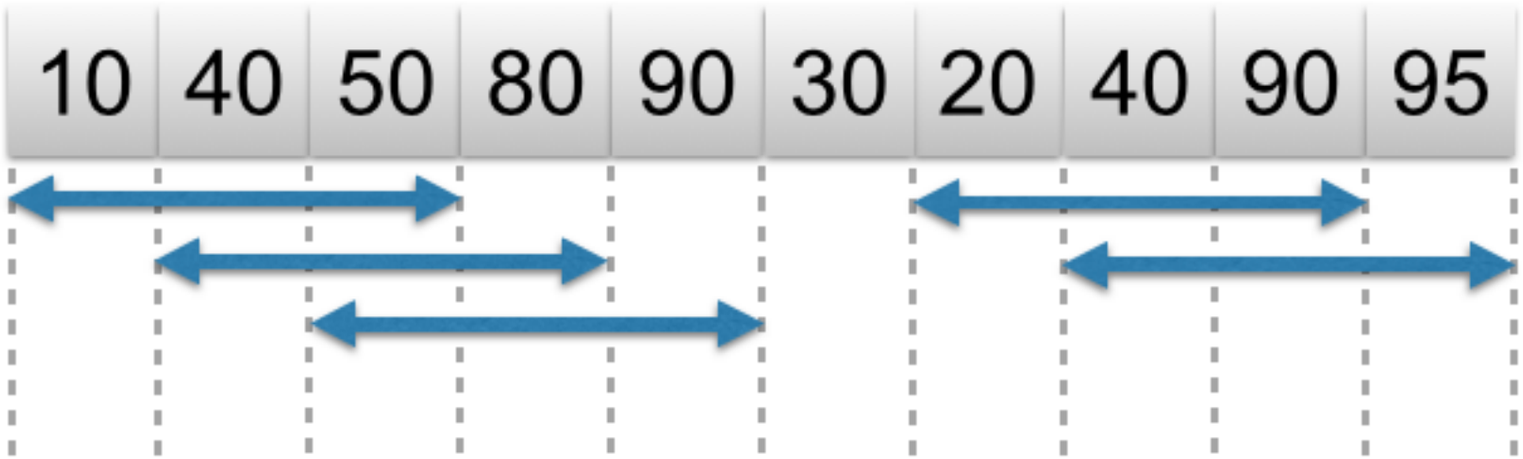
・最大で 200 億を超える個数になる！

・とてもじゃないけど 2 秒では難しい

B - 解き方(2/3)

入出力例についていた図を観察してみよう

$(N = 10, K = 3)$



最初の 5 要素が常に上昇していて、$K = 3$ なので

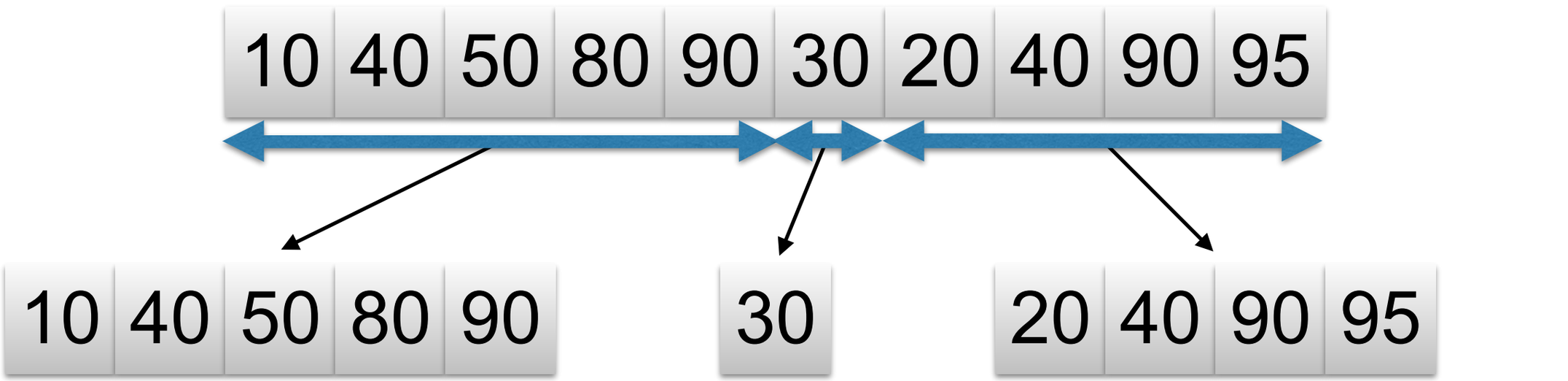
その部分が 3 回分カウントされている！

後半についても、4 要素の上昇部分が 2 回分カウントされている

B - 解き方(3/3)

元の A を、常に上昇している列たちに分解する！

次に、分解したそれぞれの部分について
その部分が K 以上の長さであれば答えにカウント！



C問題

無駄なものが嫌いな人

C - 問題概要

問題

N 個の荷物と、それぞれの大きさ w が与えられる。
大きさ X のナップサックに、ぴったり荷物を詰める方法は何通りあるか？

制限

$1 \leqq N \leqq 32$
$1 \leqq w_i \leqq 50{,}000{,}000\ (1 \leqq i \leqq N)$
$1 \leqq X \leqq 1{,}000{,}000{,}000$

C - ナップサック？

ナップサック問題(有名な動的計画法の問題、詳細は調べてね)
にそっくり！同じ方法で解けるかな？

今回の制限だと、**同じ方法ではダメ**です。

・ナップサックの大きさぐらいメモリが必要
・今回はナップサックがとても大きい

C - 制限に注目

今回の制限では

・ナップサックや荷物がとても大きい

が、そのかわりに

・**荷物の個数(N)がとても小さい**（重要）

とはいえ、全探索は間に合わない
(2^{32} 通りの詰め方はとても時間内に探索しきれない)

C - 解き方(1/3)

N が**半分**(16ぐらい)なら全探索できるのに……

つまり、
荷物を前半グループと後半グループの**半々に分けて**、
それぞれのグループ内の組み合わせで作れる大きさなら全探索ができる！

その後で、ふたつのグループの結果を
うまくまとめられないだろうか？

C - 解き方(2/3)

前半の荷物を組み合わせて、大きさ p にする方法
が A 通りあるとすると……

後半の荷物を組み合わせて、大きさ X-p にする方法
が B 通りあったとしたら

→ちょうど大きさを X にする方法が
　$A \times B$ **通り**見つかったことになる

C - 解き方(3/3)

前半でつくる大きさ p **を全部試す**として

このとき対応する

「後半で大きさ X-p を作る方法」が何通りあるか

がすぐに分かればよい。

→ 後半の**全探索の結果を作れる大きさの順にソート**しておけば**二分探索**で大きさ X-p を作る方法が何通りあるか高速に求められる！

これは**「半分全列挙」**等の名前で言及されることの多いアルゴリズム

D 問題 解説

城下 慎也 (phidnight)

問題概要

- N 個の数列が与えられる。
- 次のクエリを M 個処理する。
  - 連続する区間の数字の GCD を出す。
  - 連続する区間の数字を増減させる

※ここでは、GCD は最大公約数のこと

- 30 点分の部分点では増減クエリの幅が1

部分点解法

- 普通に 1 個ずつ区間を取って GCD を計算していると間に合わない。
- 実は、いくつかの数の GCD はどのような順番で部分の GCD を計算しても必ず同じ最終結果が返ってくる。
- あらかじめ広い区間の GCD を知っておけば効率的に計算できる。
- segment-tree を用いると効果的。

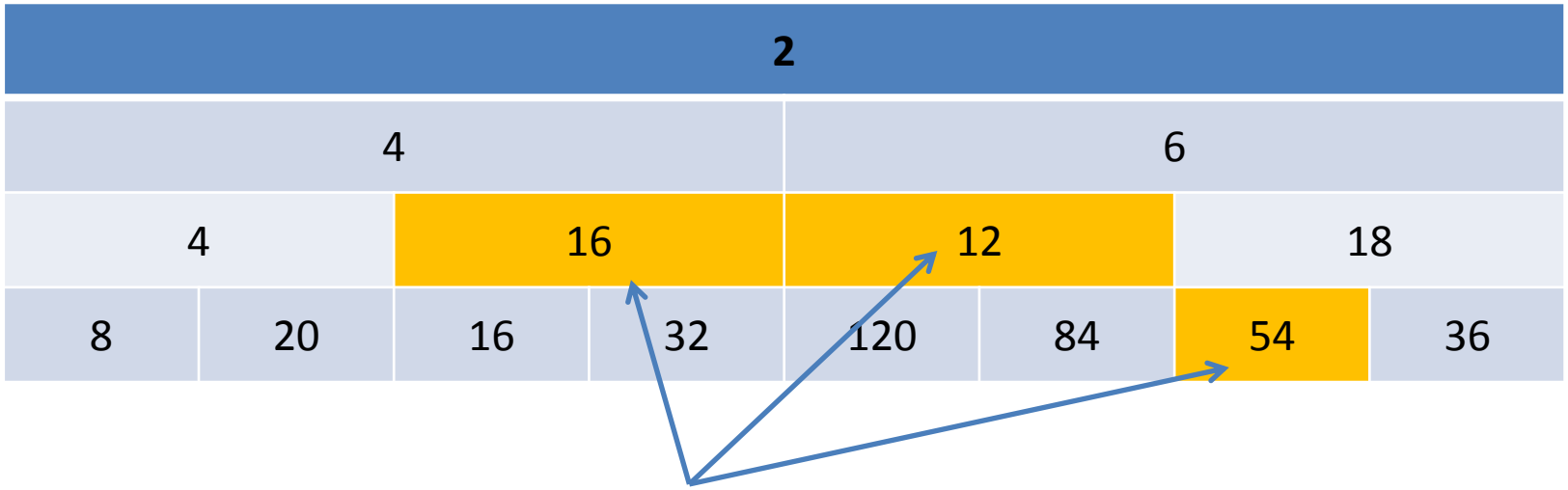
部分点解法(例)

- 例えば下のように区間を決めた場合について考えてみる。
- 左から 3 つ目を起点として右に進んだ 5 つの数 16,32,120,84,54 の GCD を計算してみる。

2							
4				6			
4		16		12		18	
8	20	16	32	120	84	54	36

部分点解法(例)

- 矢印で示した 3 つの数 16,12,54 の GCD は元の 5 つの数の GCD と一致する。
- GCD(16,32,120,84,54)=GCD(16,12,54)=2



部分点解法

- segment-tree に、その区間に含まれる要素の GCD を保存しておけば、連続する区間の GCD を O(log N) で計算することができる。
- 増減クエリは幅 1 なので、この更新によって GCD が更新される場所も O(log N)

→全体で O(M log N) で計算でき、30 点が得られる。

満点解法に向けて

- segment-tree が管理している区間全体に数字が増減された場合、増減の前後で GCD が変わることがある。
- 例: 区間が [36,43,1,71] を管理している時に、区間全体が 7 増えると GCD(42,49,7,77)=7 となって 1 ではなくなっている。
- 増減クエリに対して不変な要素がないか考える。

満点解法に向けて

- 全体に数が加算されても、それぞれの数の差は一定である。(36 と 43 の両方に 6 を足しても差は 7 のままである)
- よって区間内の隣り合う 2 項の差は不変。
- 実は、区間に含まれる要素のうち、隣り合うものの差すべてと代表元 1 個の GCD は区間の GCD と等しくなる。

$$\mathrm{GCD}(A_l, A_{l+1}, \ldots, A_r)$$
$$= \mathrm{GCD}(A_l, A_{l+1} - A_l, \ldots, A_r - A_{r-1})$$

ということ。

両者が一致することの証明

g=GCD($A_l, A_{l+1}, \ldots, A_r$)

h=GCD($A_l, A_{l+1} - A_l, \ldots, A_r - A_{r-1}$) とする。

- $l \leq i \leq r$ となるどの i についても $A_i = kg$と表すことができる。(k は正整数)
- A_i が g の倍数なのでそれらの差も g の倍数である。よって h は g の倍数である。
- $g \leq h$ であることはこのことより分かる。

両者が一致することの証明

- h の性質より $l \leq i \leq r-1$ となるどの i についても、$A_{i+1} - A_i = B_i h$ となる正の整数 B_i が存在する。
- また、$A_l = Ch$ (C は正の整数) も成立する。
- このとき、$l \leq j \leq r$ となるどの j についても、

  $A_j = A_l + \sum_{k=l}^{j-1}(A_{k+1} - A_k)$

  $= h(C + \sum_{k=l}^{j-1} B_k)$ となるから、g は h の倍数である。
- よって $g \geq h$ も成立し、$g = h$ が成立する。

満点解法

- よって、segment-tree に、管理する区間に含まれる、隣り合う差の GCD と左端の要素を保存しておき、質問クエリに対して、それぞれの区間ごとに区間内の隣り合う差の GCD をまとめ、それらの隙間の部分と左端の要素をまとめれば質問に答えることができる。これは O(log N) である。
- 増減クエリに対して、隣り合う差の GCD は、区間に一部またがる場合のみ更新される。
- この方法だと O(log N) で更新できる。

満点解法

- 左端の要素 A_l に関する情報について、増減クエリそのまま個々の要素に実行すると時間がかかる。
- 区間和として、質問クエリに対し遅延評価をするとこの問題を解決できる。
- 全体で O(M log N) となり満点が得られる。

補足

- 今回の問題に、「初期状態およびクエリ実施後にすべての数が 1 以上 10^9 以下となる」とありますが、遅延評価用にとっている区間の中身が 32 bit 整数に収まるとは限りません。
- 初期状態で $A_1 = A_2 = 1$ として
- 9×10^8 1 2→ -9×10^8 1 1→ -9×10^8 2 2 というのが 3 回以上繰り返された場合など。
- 2 の補数表記の性質上、実は考慮しなくてもうまくいく場合があるみたいです。

AtCoder Regular Contest 016 解説

AtCoder株式会社 代表取締役
高橋直大

A問題　問題概要

- クイズゲームの選択肢の種類と、正解番号が与えられる
- 不正解のものを1つ消したいので、1つ選んで出力しなさい。

A問題 解説

- 与えられた整数を2つ、標準入力から読み取る
  - わからない場合は標準入力について勉強！
- その整数2つから、不正解の番号を1つピックアップする
  - やり方は色々！
- 出てきた答えを出力する
  - わからない場合は標準出力について勉強！

A問題　解説

- 数字の選び方いろいろ
  - 1つ上の数字を選択する
    - 1なら2,2なら3のような感じ
    - 最大の数の時は1を出力する
  - 答えが1の時だけ2,それ以外は1を出力する
  - Forループなどで、全てのパターンを確認する
  - 正解でない数字になるまでランダムで選び続ける

B問題　問題概要

- 音楽ゲームの譜面が与えられる
- ボタンを押す回数を求めなさい

B問題　解き方

- １行ずつ文字列を読み込む
  - 配列に入れておくと良い
- １行目は、oまたはxの数をカウントする。
- ２行目以降は、
  - xがあった場合、1つカウントする。
  - oがあった場合、1つ上の行を確認し、oでなければ1つカウントする
- カウントした答えを出力する

B問題　ちょっと特殊な解き方

- 縦横を入れ替えて、文字列”oo”を”o”に置換できなくなるまで置換した後、oとxを数える。
  - 普通に解いた方が多分楽です。

C問題 問題概要

- くじが複数与えられる。
- 事前に出現するカードとその確率、及び値段が与えられている。
  - 外れは存在しない。
- 全てのカードを集めるのに必要な金額の期待値を出力せよ。
- カードNは10種類以下。くじMは4種類以下

C問題 部分点

- 部分点A N=1かつM=1
  - カード1枚、くじも1つ
- 部分点B N=1
  - カード1枚、くじは複数
- 部分点C C_i = 1
  - カードもくじは複数だが、1つのくじから出るカードは1種類だけ。
- 部分点D N<=2
  - カードが2種類まで
- 部分点E M=1
  - くじが1種類だけ

C問題 解説 部分点A

- くじを1回引けば必ず目的のカードが引ける。
- くじの金額を出力すれば良いだけ。

C問題 解説 部分点B

- くじは複数与えられるが、カードは1枚しか存在しない。
- 一番安いくじを引けば良い。

C問題 解説 部分点C

- 1つのくじから1種類のカードしか出現しない。
- それぞれのカードについて、一番安いくじを引けば良い。

C問題 解説 部分点D

- 2種類のカードが与えられる。
- カードAを引くために必要な金額の期待値と、カードBを引くために必要な金額の期待値をあらかじめ求めておく。
  - c円のくじで、カードAが確率pで現れる時、引く回数の期待値は1/p個。
    - よって、C/p円が期待値となる。
    - これの最小値を求めれば良い。
- それぞれのくじについて、カードAを引いた場合と、カードBを引いた場合のそれぞれについて、期待値を求めてあげれば良い。

C問題 解説 部分点E

- いわゆる「コンプガチャ問題」
- 部分点Dと同じように、「この先からいくらかかるか」をメモしてあげれば良い。
- 持っているカードの状態ごとに、それぞれbitDPで期待値を求めてあげれば良い。

C問題 解説 部分点E

- 具体例
  - 全てのカードが揃っている場合
    - 期待値は0
  - カードが4枚あり、カードAだけ存在しない場合
    - 期待値は、「カードAを引くまでに必要な回数の期待値」＊「カードの枚数」
  - カードが4枚あり、カードA,Bが存在しない場合
    - カードAが引ける確率をa,カードbを引ける確率をbとする。
    - 期待値は、(金額/(1-a-b)) + [カードAだけ存在しない場合の期待値]*a/(a+b)+[カードBだけ存在しない場合の期待値]*b/(a+b)
- このような感じでbitDPをしてあげれば良い。

bitDPって？

- 持っているカードの種類を、2進数で表す。
- 3枚のカードがあるなら、0～7で表せる。
  - A,B,Cのカードがあるなら、7 (２進数で111)
  - A,Bのカードがあるなら、3(２進数で011)
  - B,Cのカードがあるなら、6(２進数で110)
- これを利用して、状態ごとの期待値を簡単に配列に収めることが出来る。

C問題 満点解法

- 部分点Eに対し、各カードの状態に対して、くじを全て試し、最小値を求めてあげれば良い。
  - 直前のあたりはずれに関係なく、揃えているカードの状態にのみしか、引くべきくじに影響を与えないことに注意

D問題 問題概要

- DAGが与えられる
- 体力が設定されており、各頂点対して、減少する体力が設定されている。
- 頂点1からスタートし、体力1以上で頂点Nに移動するまでの期待値を求める。
- プレイヤーは、以下の2つの行動をとることが出来る。
  - 今まで受けたダメージ分だけ時間を消費し、頂点1に戻る。
  - 時間1を消費し、次の頂点に移動する。なお、この際に選択される頂点は、接続されている頂点の中から等確率でランダムに選ばれる。
- 体力0になる可能性が少しでもあるような選択肢は選ぶことが出来ない。

DAGって何？

- Directed acyclic graphの略
- 閉路のない有向グラフのこと。

D問題 問題解説

- 今いる場所、今まで受けたダメージの2つでDPを行う。
- 始点に戻る、という選択肢があるため、それだけを除くとDAGとなる。
  - よって、「始点からの期待値」を固定できれば、DAG上のDPとなる。
  - 始点からの期待値を二分探索してしまえば良い。

AtCoder Regular Contest 015 解説

AtCoder 株式会社 代表取締役
高橋直大

A問題 問題概要

- 問題概要
  - 摂氏での温度nが与えられるので、華氏に変換せよ。
  - 計算式は、$(9 \div 5 \times n)+32$である

A問題 解法

- 整数を読み込み、前述の計算式で計算する
- 結果は小数になり得ることに注意

- 解けない人は、「標準入力」「標準出力」あたりを勉強してみよう！

B問題 問題概要

- ある期間の、最低気温および最高気温が与えられる
- 猛暑日・真夏日・夏日・熱帯夜・冬日・真冬日が、それぞれ何回あったかを出力せよ

B問題 解法

- 順番に最低気温・最高気温を読み込み、それぞれ各条件を満たしているか調べていく

- 解けない場合は、「繰り返し(for)」「場合分け(if)」などを勉強してみよう！

C問題　問題概要

- いくつかの単位と、それに対する換算表を与えられる
  - 例: 1m = 100cm, 1km = 1000m
- 最大の単位を、最小の単位で表しなさい
  - 例: 1km = 100000cm
  - これは、10^9以下の整数のみで表現可能
- 最小・最大は換算表から確定出来る

C問題 解説

- 1つの単位で全部の単位を表してあげれば良い
  - 幅優先探索・深さ優先探索・ワーシャルフロイド等

C問題 落とし穴

- 全ての単位が最小単位によって整数で表現可能とは限らない！
  - 例:B=5A B=3C D=30C
  - このとき、C=1.6666….A
- 分数で表すのにも、膨大な桁数が必要
  - N=200なので、結構大変
  - 誤差が多少出ても問題ないので、小数で計算した方が安全

D問題 概要

- クッキーを毎秒1個作る
- 毎秒確率pで出現する金色クッキーをクリックすると、t_i秒の間、生産量がx_i倍になる。i番目の効果が発生する確率はp_i
- T秒間クッキーを作った時のクッキーの数の期待値を出力せよ

D問題 解説

- 出来次第再アップロードします！

AtCoder Regular Contest 002 解説

AtCoder株式会社　代表取締役
高橋 直大

A問題 問題概要

- 西暦の年数が与えられる
- その年がうるう年かどうかを判定しなさい。

A問題 解説

- 規則3,2,1,4の順で実装を行う。
- If文を並べればOK！

B問題 問題概要

- Y/M/Dの形式で日付が与えられる。
- y/m/dが整数となるような次の日付を出力せよ。

B問題 解説

- １日ずつ進めていく！
  - 日付ライブラリとかある言語を使おう！
  - 存在しない場合は、A問題のうるう年判定を合わせて、１２月までの日数を予め配列に入れておくなどして対応しよう！

C問題 問題概要

- ABXYの4種類で書かれた文字列が存在する。
- L,Rの2文字に、2文字分の意味を持たせることで、この文字列を圧縮したい
- 圧縮後の最小文字数を答えなさい

C問題 間違った解き方

- 全通り置換するだけ
- 例えば、ABABBABAのようなケースがダメになってしまう。
  - L = AB、R= BAでLLRRが正解
  - Lを出来るだけ置換すると、LLBLAになってしまい、Rに置換することが出来ない。

- テストが弱い影響で、これで通っちゃう場合があるみたいです。申し訳ありません。

C問題 解説

- L,Rを全通り試す。
- それぞれに対し、何文字目までいくつの文字で表せたかをDPで計算する。
  - ただの置換ではだめ！

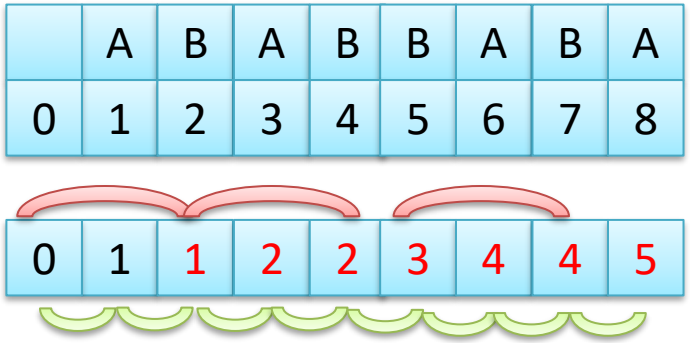
C問題 解説

- S = ABABBABA, X = AB, Y = BAのとき
  - まずは普通にABで表した時の数字を入れる

	A	B	A	B	B	A	B	A
0	1	2	3	4	5	6	7	8

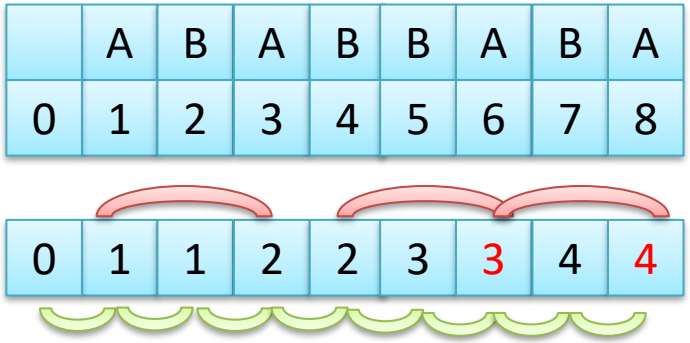
C問題 解説

- S = ABABBABA, X = AB, Y = BAのとき
  - Xを適用。ABに対して+1の更新を行う。



C問題 解説

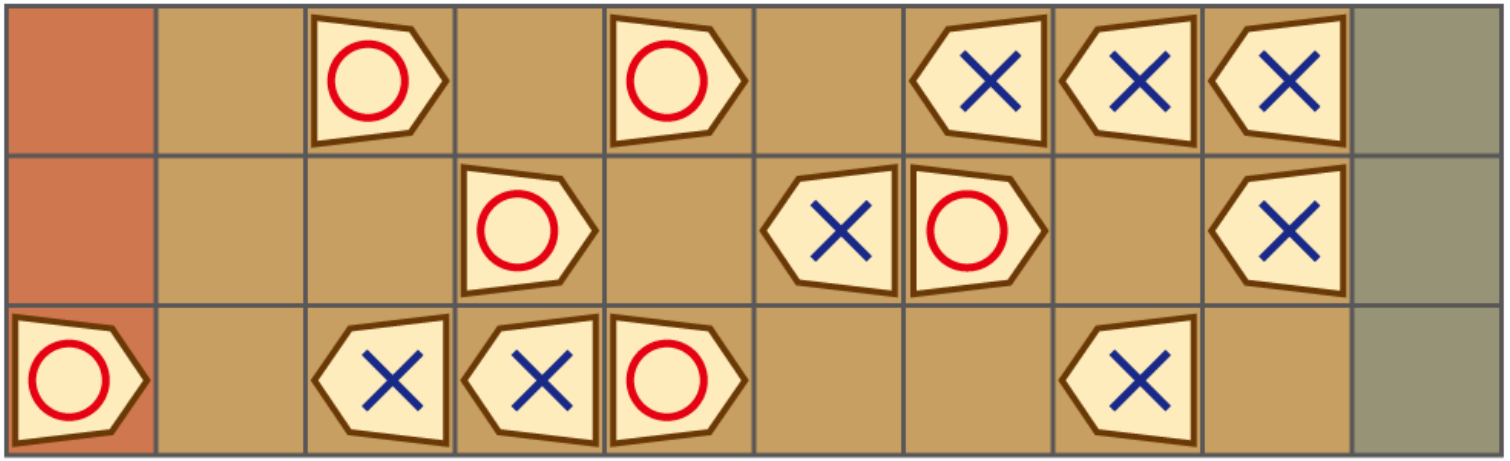
- S = ABABBABA, X = AB, Y = BAのとき
  - Yを適用。XYに対して+1の更新を行う。



- こうしたDPを全通り行う

D問題 問題概要

- 以下のように、将棋の歩の動きをする駒が大量にあるボードゲームがある
  - 持ち駒は存在しない。初期配置で取れる駒は存在しない。どちらが勝つかを求める
  - 相手の陣地（端）に到達したら勝ち



D問題 解説

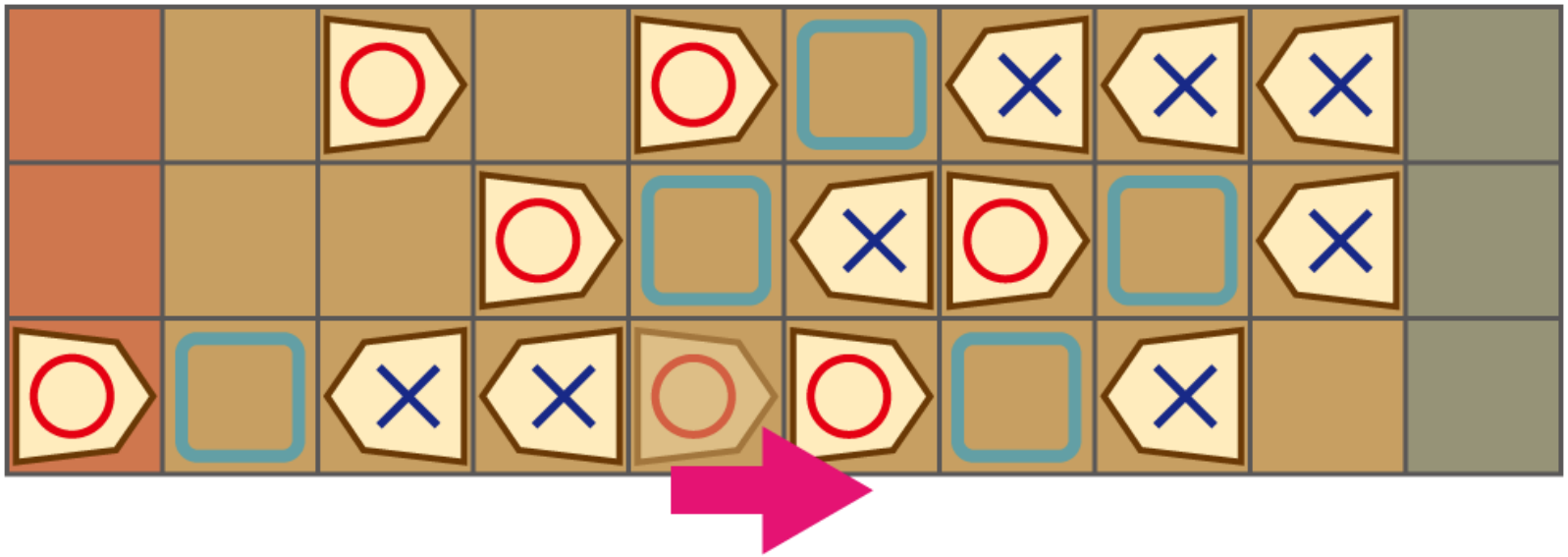
- 問題サイズは、2000*2000
  - 探索等の処理は無理
- 何か規則性を見つけよう！

D問題 解説

- 基本的な処理
  - 自分の目の前に敵がいないような駒が存在する場合
    - 片方にいる場合は、そちらの勝ち
    - 両方にいる場合は、先に辿り着ける方が勝ち
  - そうでない場合
    - 自分から取られに行くような操作は不利
      - 取られた瞬間取り返す、みたいなのは別だが、考える必要はない。理由は後述。
    - 取られないような操作だけを考えれば良い

D問題 問題解説

- 先手が動かした後、後手はどの駒を動かしても取られる状態になってしまう。
- このような状態の時、取られた直後にまた同じ状態になる。つまり負けが確定してしまう。



D問題 問題解説

- つまり、先に取られる様な動きをしてしまった方が負け
  - 相手に取られない動きを何手打てるかが重要
- この手数だけに注目し、相手との手数の差を最大化するような手だけを打てば良い。
  - これは、向かい合っている駒の数だけで判定が可能なので、ソートして貪欲。

AtCoder Regular Contest 001 解説

AtCoder株式会社 代表取締役
高橋直大

A問題 問題概要

- 1,2,3,4の４種類の文字で書かれた文字列が与えられる。
- 一番多い数字の個数と、一番少ない数字の個数を出力しなさい。

A問題 解説

- プログラムの一例は以下の通り
  - 標準入力から文字数を読み込む
  - 標準入力から文字列を読み込む
  - 要素数4の配列を用意する
  - 全ての文字に対しループで判定を行う
    - 文字に対応した配列をインクリメントする
  - 配列の中で、最小値と最大値を出力する
    - 適切な関数が存在しない場合はループで取り出す

B問題 問題概要

- エアコンの設定温度をA度からB度に変更したい
- １回ボタンを押すことで変更可能な温度は、１度、５度、１０度の３種類。上にも下にも変更できる
- ボタンを押す必要のある最小回数を出力

B問題 解説

- 解き方は複数存在する
  - 幅優先探索を用いる
  - 差が10度以下になるまで、10度の変更をする。10度以下は埋め込み
  - 全部の温度に対して最短距離をワーシャルフロイドなどで計算してしまう
- どれを書いても良い

C問題 問題概要

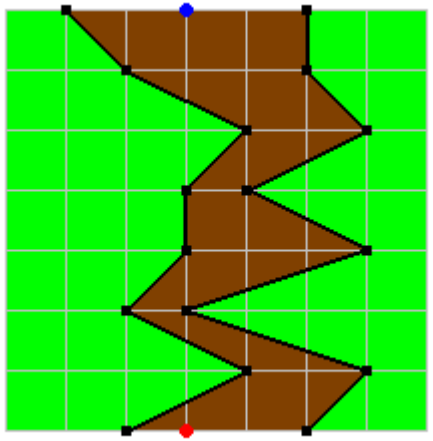
- 8-queen問題の、3つのクイーンを置いた状態が与えられる
- 残りの5つを配置せよ。もし不可能な場合はNo Answerと出力せよ。

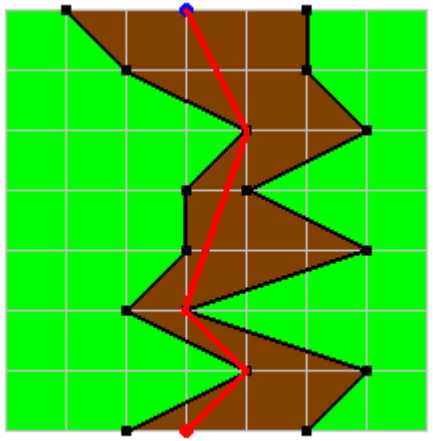
C問題 解説

- 全通り試すのが簡単に出来るため、深さ優先探索で良い。
  - 同じ列や行にすでに2つ置かれていたら失敗
  - 置かれていないのであれば、残った5行に残った5列を割り当てる。この割り当て方は5!通り
    - それぞれのパターンに対して、8-queenの条件を満たしているか確認する

D問題 問題解説

- 右図のような道が与えられる
- 道の上しか通ることが出来ない
- 最短経路を求めなさい
  - 幅は1,000,000以下
  - 高さは200,000以下





D問題 解説

- 最短経路ならダイクストラ法？
  - 頂点Vに対して、O(V^2)かO(ElogV)程度かかってしまう。
  - V=400,000, E=V^2なので、どちらも間に合わない
    - 実際は交差しないかどうかを判定しないといけないのでさらに計算量がかかる。愚直実装でO(V^3)
- グラフの特徴を利用して、なんか工夫しないとだめ！

D問題 解説

- 考察してみよう！
  - 頂点と赤・青の点以外は考える必要がない
  - 後ろに戻ることは絶対にない
    - ってことは動的計画法でいけそう？

D問題 解説

- 各頂点に対して、距離をDPで求める
  - 高さ0は距離1
  - 高さjはdp[j] = min(dp[j], dp[i] + dist(point[i], point[j]))
    - 実際は左右の頂点が存在するので、両側について考えなければならない
    - もちろんdist関数は単純な距離だけではなく、交差判定を行わなければならない。
  - 更新回数はV^2 / 2程度？

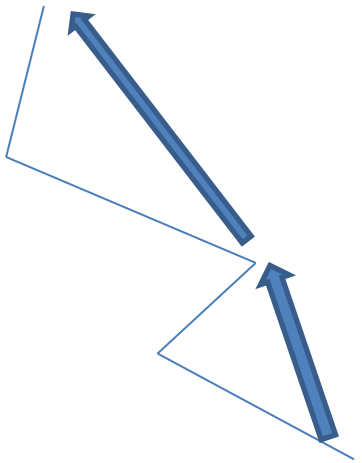
D問題 解説

- 範囲を絞って枝刈り？
  - これ以上先に絶対行けない場合は更新を止める
    - ステップを更新するごとに、行ける角度などが狭まって行く
    - 交差判定もこれだけで十分
  - 余計な頂点は無視
    - 明らかに窪んでいる点や、直線になっている点など
- 最悪ケースはそれでもO(V^2)
  - これでは通らない

» データセットが弱かったようで、かなり高速化すると通ってしまうケースもあるようです。ごめんなさい＞＜

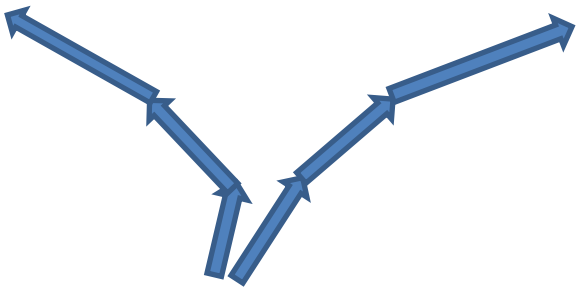
D問題 解説

- 基本的なアイデア
  - 凹んでる場所に行く必要は絶対にない



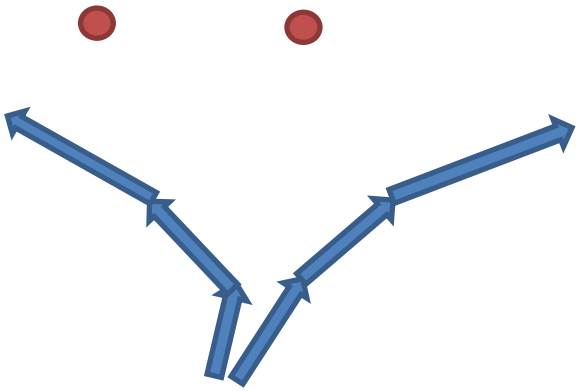
D問題 解説

- 基本的なアイデア
  - 凹んでる場所に行く必要は絶対にない
  - であれば、持つべき情報は、以下のような形のデータ
    - 現在位置から、凹まずに行ける経路を左右独立に持つ



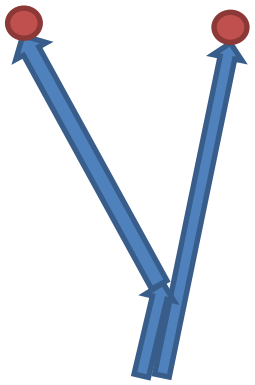
D問題 解説

- 更新方法
  - 以下のような頂点が与えられた場合



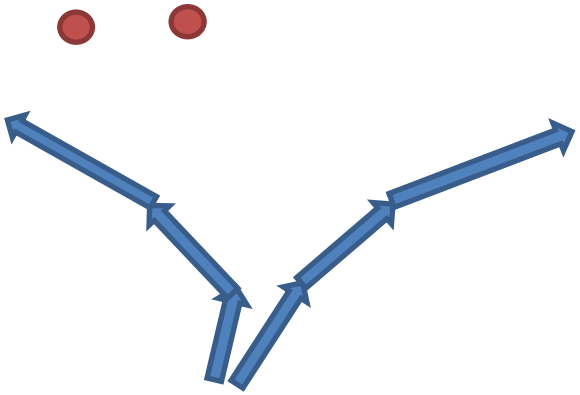
D問題 解説

- 更新方法
  - 以下のような頂点が与えられた場合
  - 角度が浅いものを消してしまい、更新していく



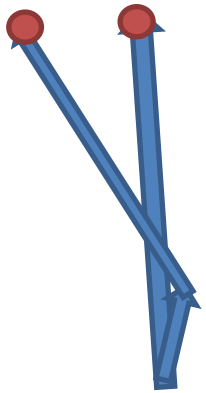
D問題 解説

- 更新方法2
  - 以下のような頂点が与えられた場合



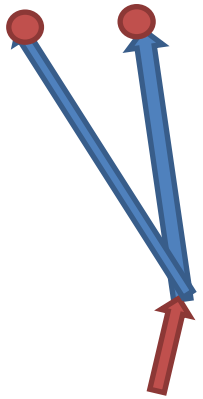
D問題 解説

- 更新方法
  - 以下のような頂点が与えられた場合
  - 交差してしまう？



D問題 解説

- 更新方法
  - 以下のような頂点が与えられた場合
  - 交差した時は、交差してしまった方を確定させ、現在位置を変更する



D問題 解説

- 先ほどの様な更新方法をした場合
- どちらが内側にあるかを調査するごとに、考えている頂点が1つ減る
  - よって、この回数はO(V)で良い。
- これで十分な証明は各自考えてください！